KingbaseES 数据库参考手册

金仓数据库管理系统 KingbaseES

文档版本：V9（V009R001C010）

发布日期：2025 年 12 月 3 日

中电科金仓（北京）科技股份有限公司

目　录

第 1 章　系统视图 1
1.1 KingbaseES 系统视图 2
1.1.1 插件相关视图 2
1.1.1.1 JOB 视图 2
1.1.1.1.1 kdb_action 2
1.1.1.1.2 kdb_job 3
1.1.1.1.3 kdb_schedule 3
1.1.1.2 闪回相关视图 4
1.1.1.2.1 sys_recyclebin 4
1.1.1.3 对象保护相关视图 5
1.1.1.3.1 sys_protect 5
1.1.1.4 资源组视图 5
1.1.1.4.1 sys_resauthid 5
1.1.1.4.2 sys_resgroup 6
1.1.1.4.3 sys_resgroupcapability 6
1.1.1.4.4 sys_resource_groups 6
1.1.1.5 历史密码管理视图 7
1.1.1.5.1 sys_pwdht 7
1.1.1.5.2 sys_pwdht_shadow 7
1.1.1.6 角色状态视图 8
1.1.1.6.1 sys_role_disable 8
1.1.1.7 来源限制视图 8
1.1.1.7.1 src_restrict.show_rules 8
1.1.1.8 安全策略视图 9
1.1.1.8.1 anon.all_policy 9
1.1.1.9 加密视图 10
1.1.2 元信息视图 10
1.1.2.1 recyclebin 14
1.1.2.2 sys_aggregate 15
1.1.2.3 sys_am 16
1.1.2.4 sys_amop 17

1.1.2.5 sys_amproc . . . 18
1.1.2.6 sys_anon_policy . . . 19
1.1.2.7 sys_attrdef . . . 19
1.1.2.8 sys_attribute . . . 20
1.1.2.9 sys_available_extension_versions . . . 22
1.1.2.10 sys_available_extensions . . . 23
1.1.2.11 sys_cast . . . 23
1.1.2.12 sys_class . . . 24
1.1.2.13 sys_collation . . . 28
1.1.2.14 sys_config . . . 29
1.1.2.15 sys_constraint . . . 29
1.1.2.16 sys_context . . . 31
1.1.2.17 sys_conversion . . . 32
1.1.2.18 sys_cursors . . . 33
1.1.2.19 sys_database . . . 33
1.1.2.20 sys_database_link . . . 35
1.1.2.21 sys_db_role_setting . . . 36
1.1.2.22 sys_default_acl . . . 36
1.1.2.23 sys_depend . . . 37
1.1.2.24 sys_depends . . . 39
1.1.2.25 sys_description . . . 40
1.1.2.26 sys_directory . . . 40
1.1.2.27 sys_enum . . . 41
1.1.2.28 sys_event_trigger . . . 41
1.1.2.29 sys_extension . . . 42
1.1.2.30 sys_file_settings . . . 43
1.1.2.31 sys_foreign_data_wrapper . . . 44
1.1.2.32 sys_foreign_server . . . 44
1.1.2.33 sys_foreign_table . . . 45
1.1.2.34 sys_grant_privileges . . . 45
1.1.2.35 sys_index . . . 46
1.1.2.36 sys_indexes . . . 48
1.1.2.37 sys_inherits . . . 49
1.1.2.38 sys_init_privs . . . 49
1.1.2.39 sys_language . . . 50
1.1.2.40 sys_largeobject . . . 51
1.1.2.41 sys_largeobject_metadata . . . 52
1.1.2.42 sys_locks . . . 52
1.1.2.43 sys_matviews . . . 55
1.1.2.44 sys_mview . . . 55
1.1.2.45 sys_namespace . . . 57
1.1.2.46 sys_object_status . . . 57

1.1.2.47 sys_opclass 58
1.1.2.48 sys_operator 58
1.1.2.49 sys_opfamily 59
1.1.2.50 sys_package 60
1.1.2.51 sys_partitioned_table 61
1.1.2.52 sys_pkgitem 62
1.1.2.53 sys_pltemplate 63
1.1.2.54 sys_policies 64
1.1.2.55 sys_policy 65
1.1.2.56 sys_prepared_statements 66
1.1.2.57 sys_prepared_xacts 67
1.1.2.58 sys_privilege 68
1.1.2.59 sys_proc 68
1.1.2.60 sys_publication 72
1.1.2.61 sys_publication_rel 73
1.1.2.62 sys_publication_tables 73
1.1.2.63 sys_query_mapping 73
1.1.2.64 sys_range 74
1.1.2.65 sys_replication_origin 75
1.1.2.66 sys_replication_origin_status 75
1.1.2.67 sys_replication_slots 76
1.1.2.68 sys_rewrite 77
1.1.2.69 sys_rules 78
1.1.2.70 sys_seclabel 78
1.1.2.71 sys_seclabels 79
1.1.2.72 sys_sequence 80
1.1.2.73 sys_sequences 80
1.1.2.74 sys_session 81
1.1.2.75 sys_settings 82
1.1.2.76 sys_shdepend 84
1.1.2.77 sys_shdescription 85
1.1.2.78 sys_shseclabel 86
1.1.2.79 sys_subpartition_table 86
1.1.2.80 sys_subscription 87
1.1.2.81 sys_subscription_rel 88
1.1.2.82 sys_synonym 88
1.1.2.83 sys_sysprivilege 89
1.1.2.84 sys_tables 89
1.1.2.85 sys_tablespace 90
1.1.2.86 sys_timezone_abbrevs 91
1.1.2.87 sys_timezone_names 91
1.1.2.88 sys_transform 92

1.1.2.89 sys_trigger . . . 92
1.1.2.90 sys_triggers . . . 94
1.1.2.91 sys_ts_config . . . 95
1.1.2.92 sys_ts_config_map . . . 95
1.1.2.93 sys_ts_dict . . . 96
1.1.2.94 sys_ts_parser . . . 96
1.1.2.95 sys_ts_template . . . 97
1.1.2.96 sys_type . . . 98
1.1.2.97 sys_user_mapping . . . 103
1.1.2.98 sys_user_mappings . . . 103
1.1.2.99 sys_views . . . 104
1.1.2.100 sysmac.sysmac_column_label . . . 105
1.1.2.101 sysmac.sysmac_obj . . . 105
1.1.2.102 user_any_privs . . . 106
1.1.3 安全视图 . . . 106
1.1.3.1 审计视图 . . . 106
1.1.3.1.1 sysaudit.all_audit_rules . . . 107
1.1.3.1.2 sysaudit.all_ids_rules . . . 107
1.1.3.1.3 sys_sysaudit_ids_setting . . . 108
1.1.3.1.4 sys_sysaudit_setting . . . 109
1.1.3.1.5 sysaudit_record_sao . . . 110
1.1.3.1.6 sysaudit_record_sso . . . 111
1.1.3.2 角色管理视图 . . . 111
1.1.3.2.1 sys_audit_blocklog . . . 112
1.1.3.2.2 sys_audit_userlog . . . 112
1.1.3.2.3 sys_auth_members . . . 113
1.1.3.2.4 sys_authid . . . 113
1.1.3.2.5 sys_group . . . 115
1.1.3.2.6 sys_role . . . 115
1.1.3.2.7 sys_roles . . . 115
1.1.3.2.8 sys_shadow . . . 116
1.1.3.2.9 sys_user . . . 117
1.1.3.2.10 sys_users . . . 118
1.1.3.2.11 sys_usergroup . . . 118
1.1.3.3 sys_hba_file_rules . . . 119
1.1.4 统计视图 . . . 120
1.1.4.1 sys_stat_activity . . . 123
1.1.4.2 sys_stat_all_indexes . . . 125
1.1.4.3 sys_stat_all_tables . . . 126
1.1.4.4 sys_stat_archiver . . . 127
1.1.4.5 sys_stat_bgwriter . . . 127
1.1.4.6 sys_stat_cached_plans . . . 128

1.1.4.7 sys_stat_database . . . 129
1.1.4.8 sys_stat_database_conflicts . . . 131
1.1.4.9 sys_stat_dbtime . . . 131
1.1.4.10 sys_stat_dmlcount . . . 133
1.1.4.11 sys_stat_gssapi . . . 133
1.1.4.12 sys_stat_instevent . . . 134
1.1.4.13 sys_stat_instio . . . 134
1.1.4.14 sys_stat_instlock . . . 135
1.1.4.15 sys_stat_metric . . . 136
1.1.4.16 sys_stat_metric_group . . . 136
1.1.4.17 sys_stat_metric_history . . . 137
1.1.4.18 sys_stat_metric_name . . . 137
1.1.4.19 sys_stat_msgaccum . . . 138
1.1.4.20 sys_stat_pre_archivewal . . . 138
1.1.4.21 sys_stat_progress_checkpoint . . . 139
1.1.4.22 sys_stat_progress_cluster . . . 140
1.1.4.23 sys_stat_progress_create_index . . . 141
1.1.4.24 sys_stat_progress_vacuum . . . 142
1.1.4.25 sys_stat_replication . . . 143
1.1.4.26 sys_stat_shmem . . . 145
1.1.4.27 sys_stat_sql . . . 145
1.1.4.28 sys_stat_sqlcount . . . 147
1.1.4.29 sys_stat_sqlio . . . 148
1.1.4.30 sys_stat_sqltime . . . 149
1.1.4.31 sys_stat_sqlwait . . . 149
1.1.4.32 sys_stat_ssl . . . 150
1.1.4.33 sys_stat_subscription . . . 150
1.1.4.34 sys_stat_sys_indexes . . . 151
1.1.4.35 sys_stat_sys_tables . . . 151
1.1.4.36 sys_stat_sysmetric . . . 151
1.1.4.37 sys_stat_sysmetric_history . . . 152
1.1.4.38 sys_stat_sysmetric_summary . . . 153
1.1.4.39 sys_stat_transaction . . . 154
1.1.4.40 sys_stat_user_functions . . . 154
1.1.4.41 sys_stat_user_indexes . . . 155
1.1.4.42 sys_stat_user_tables . . . 155
1.1.4.43 sys_stat_wait . . . 155
1.1.4.44 sys_stat_waitaccum . . . 155
1.1.4.45 sys_stat_wal_buffer . . . 156
1.1.4.46 sys_stat_wal_receiver . . . 157
1.1.4.47 sys_stat_xact_all_tables . . . 158
1.1.4.48 sys_stat_xact_sys_tables . . . 159

1.1.4.49 sys_stat_xact_user_functions . . . 159
1.1.4.50 sys_stat_xact_user_tables . . . 159
1.1.4.51 sys_statio_all_indexes . . . 159
1.1.4.52 sys_statio_all_sequences . . . 160
1.1.4.53 sys_statio_all_tables . . . 160
1.1.4.54 sys_statio_sys_indexes . . . 161
1.1.4.55 sys_statio_sys_sequences . . . 161
1.1.4.56 sys_statio_sys_tables . . . 161
1.1.4.57 sys_statio_user_indexes . . . 161
1.1.4.58 sys_statio_user_sequences . . . 162
1.1.4.59 sys_statio_user_tables . . . 162
1.1.4.60 sys_statistic . . . 162
1.1.4.61 sys_statistic_ext . . . 164
1.1.4.62 sys_statistic_ext_data . . . 164
1.1.4.63 sys_stats . . . 166
1.1.4.64 sys_stats_ext . . . 168
1.2 MySQL 兼容视图 . . . 169
1.2.1 信息模式视图 . . . 170
1.2.1.1 ADMINISTRABLE_ROLE_AUTHORIZATIONS . . . 170
1.2.1.2 APPLICABLE_ROLES . . . 170
1.2.1.3 CHARACTER_SETS . . . 171
1.2.1.4 CHECK_CONSTRAINTS . . . 172
1.2.1.5 COLLATION_CHARACTER_SET_APPLICABILITY . . . 172
1.2.1.6 COLLATIONS . . . 172
1.2.1.7 COLUMN_PRIVILEGES . . . 173
1.2.1.8 COLUMNS . . . 174
1.2.1.9 ENABLED_ROLES . . . 175
1.2.1.10 KEY_COLUMN_USAGE . . . 175
1.2.1.11 KEYWORDS . . . 176
1.2.1.12 PARAMETERS . . . 177
1.2.1.13 PARTITIONS . . . 178
1.2.1.14 PROCESSLIST . . . 179
1.2.1.15 REFERENTIAL_CONSTRAINTS . . . 180
1.2.1.16 ROLE_COLUMN_GRANTS . . . 181
1.2.1.17 ROLE_ROUTINE_GRANTS . . . 181
1.2.1.18 ROLE_TABLE_GRANTS . . . 182
1.2.1.19 ROUTINES . . . 183
1.2.1.20 SCHEMATA . . . 185
1.2.1.21 TABLE_CONSTRAINTS . . . 186
1.2.1.22 TABLE_PRIVILEGES . . . 186
1.2.1.23 TABLES . . . 187
1.2.1.24 TRIGGERS . . . 189

1.2.1.25 VIEW_ROUTINE_USAGE . . . 190
1.2.1.26 VIEW_TABLE_USAGE . . . 191
1.2.1.27 VIEWS . . . 191
1.3 Oracle 兼容视图 . . . 192
1.3.1 动态性能视图 . . . 192
1.3.1.1 V$CONTEXT . . . 192
1.3.1.2 V$DATABASE . . . 193
1.3.1.3 V$INSTANCE . . . 194
1.3.1.4 V$LOCK . . . 195
1.3.1.5 V$LOCKED_OBJECT . . . 196
1.3.1.6 V$METRIC . . . 197
1.3.1.7 V$METRIC_HISTORY . . . 197
1.3.1.8 V$METRICGROUP . . . 198
1.3.1.9 V$METRICNAME . . . 198
1.3.1.10 V$PARAMETER . . . 199
1.3.1.11 V$SESSION . . . 200
1.3.1.12 V$SYSMETRIC . . . 204
1.3.1.13 V$SYSMETRIC_HISTORY . . . 204
1.3.1.14 V$SYSMETRIC_SUMMARY . . . 204
1.3.1.15 V$SYSSTAT . . . 205
1.3.2 静态数据字典视图 . . . 206
1.3.2.1 ALL_ALL_TABLES . . . 206
1.3.2.2 ALL_ARGUMENTS . . . 210
1.3.2.3 ALL_COL_COMMENTS . . . 212
1.3.2.4 ALL_COL_PRIVS . . . 213
1.3.2.5 ALL_CONS_COLUMNS . . . 213
1.3.2.6 ALL_CONSTRAINTS . . . 214
1.3.2.7 ALL_DB_LINKS . . . 215
1.3.2.8 ALL_DIRECTORIES . . . 216
1.3.2.9 ALL_IND_COLUMNS . . . 216
1.3.2.10 ALL_INDEXES . . . 217
1.3.2.11 ALL_OBJECTS . . . 221
1.3.2.12 ALL_PART_TABLES . . . 222
1.3.2.13 ALL_SCHEDULER_EXCEPTION . . . 223
1.3.2.14 ALL_SCHEDULER_JOB_LOG . . . 224
1.3.2.15 ALL_SCHEDULER_JOB_RUN_DETAILS . . . 224
1.3.2.16 ALL_SCHEDULER_JOBS . . . 224
1.3.2.17 ALL_SCHEDULER_PROGRAMS . . . 224
1.3.2.18 ALL_SCHEDULER_SCHEDULES . . . 224
1.3.2.19 ALL_SEQUENCES . . . 224
1.3.2.20 ALL_SOURCE . . . 225
1.3.2.21 ALL_SYNONYMS . . . 226

1.3.2.22 ALL_TAB_COLS . . . 227
1.3.2.23 ALL_TAB_COLUMNS . . . 227
1.3.2.24 ALL_TAB_COMMENTS . . . 229
1.3.2.25 ALL_TAB_PARTITIONS . . . 229
1.3.2.26 ALL_TAB_PRIVS . . . 231
1.3.2.27 ALL_TABLES . . . 232
1.3.2.28 ALL_TRIGGER_COLS . . . 235
1.3.2.29 ALL_TRIGGERS . . . 235
1.3.2.30 ALL_TYPES . . . 236
1.3.2.31 ALL_USERS . . . 237
1.3.2.32 ALL_VIEWS . . . 238
1.3.2.33 DB_FILES . . . 239
1.3.2.34 DBA_ALL_TABLES . . . 240
1.3.2.35 DBA_ARGUMENTS . . . 240
1.3.2.36 DBA_COL_COMMENTS . . . 240
1.3.2.37 DBA_COL_PRIVS . . . 240
1.3.2.38 DBA_CONS_COLUMNS . . . 240
1.3.2.39 DBA_CONSTRAINTS . . . 240
1.3.2.40 DBA_DB_LINKS . . . 240
1.3.2.41 DBA_DIRECTORIES . . . 241
1.3.2.42 DBA_FREE_SPACE . . . 241
1.3.2.43 DBA_IND_COLUMNS . . . 242
1.3.2.44 DBA_INDEXES . . . 242
1.3.2.45 DBA_OBJECTS . . . 242
1.3.2.46 DBA_PART_TABLES . . . 243
1.3.2.47 DBA_ROLE_PRIVS . . . 243
1.3.2.48 DBA_ROLES . . . 244
1.3.2.49 DBA_SCHEDULER_EXCEPTION . . . 244
1.3.2.50 DBA_SCHEDULER_JOB_LOG . . . 245
1.3.2.51 DBA_SCHEDULER_JOB_RUN_DETAILS . . . 247
1.3.2.52 DBA_SCHEDULER_JOBS . . . 249
1.3.2.53 DBA_SCHEDULER_PROGRAMS . . . 254
1.3.2.54 DBA_SCHEDULER_SCHEDULES . . . 256
1.3.2.55 DBA_SEQUENCES . . . 257
1.3.2.56 DBA_SOURCE . . . 257
1.3.2.57 DBA_SYNONYMS . . . 257
1.3.2.58 DBA_TAB_COLS . . . 257
1.3.2.59 DBA_TAB_COLUMNS . . . 259
1.3.2.60 DBA_TAB_COMMENTS . . . 261
1.3.2.61 DBA_TAB_PARTITIONS . . . 262
1.3.2.62 DBA_TAB_PRIVS . . . 262
1.3.2.63 DBA_TABLES . . . 262

1.3.2.64 DBA_TABLESPACE 265
1.3.2.65 DBA_TABLESPACES 265
1.3.2.66 DBA_TRIGGER_COLS 266
1.3.2.67 DBA_TRIGGERS 266
1.3.2.68 DBA_TYPES 266
1.3.2.69 DBA_USERS 266
1.3.2.70 DBA_VIEWS 267
1.3.2.71 RECYCLEBIN 267
1.3.2.72 USER_ALL_TABLES 267
1.3.2.73 USER_ARGUMENTS 268
1.3.2.74 USER_COL_COMMENTS 268
1.3.2.75 USER_COL_PRIVS 268
1.3.2.76 USER_CONS_COLUMNS 268
1.3.2.77 USER_CONSTRAINTS 268
1.3.2.78 USER_DB_LINKS 268
1.3.2.79 USER_DIRECTORIES 268
1.3.2.80 USER_FREE_SPACE 268
1.3.2.81 USER_IND_COLUMNS 268
1.3.2.82 USER_INDEXES 269
1.3.2.83 USER_OBJECTS 269
1.3.2.84 USER_PART_TABLES 270
1.3.2.85 USER_ROLE_PRIVS 270
1.3.2.86 USER_SCHEDULER_EXCEPTION 270
1.3.2.87 USER_SCHEDULER_JOB_LOG 270
1.3.2.88 USER_SCHEDULER_JOB_RUN_DETAILS 270
1.3.2.89 USER_SCHEDULER_JOBS 271
1.3.2.90 USER_SCHEDULER_PROGRAMS 271
1.3.2.91 USER_SCHEDULER_SCHEDULES 271
1.3.2.92 USER_SEQUENCES 271
1.3.2.93 USER_SOURCE 271
1.3.2.94 USER_SYNONYMS 271
1.3.2.95 USER_TAB_COLS 271
1.3.2.96 USER_TAB_COLUMNS 272
1.3.2.97 USER_TAB_COMMENTS 272
1.3.2.98 USER_TAB_PARTITIONS 272
1.3.2.99 USER_TAB_PRIVS 272
1.3.2.100 USER_TABLE_COLS 272
1.3.2.101 USER_TABLES 274
1.3.2.102 USER_TABLESPACE 274
1.3.2.103 USER_TABLESPACES 274
1.3.2.104 USER_TRIGGER_COLS 275
1.3.2.105 USER_TRIGGERS 275

1.3.2.106 USER_TYPES . . . 275
1.3.2.107 USER_USERS . . . 275
1.3.2.108 USER_VIEWS . . . 276
1.4 PostgreSQL 兼容视图 . . . 276
1.4.1 元信息视图 . . . 276
1.4.1.1 pg_aggregate . . . 276
1.4.1.2 pg_am . . . 278
1.4.1.3 pg_amop . . . 278
1.4.1.4 pg_amproc . . . 279
1.4.1.5 pg_attrdef . . . 280
1.4.1.6 pg_attribute . . . 280
1.4.1.7 pg_auth_members . . . 283
1.4.1.8 pg_authid . . . 283
1.4.1.9 pg_available_extension_versions . . . 284
1.4.1.10 pg_available_extensions . . . 285
1.4.1.11 pg_cast . . . 286
1.4.1.12 pg_class . . . 287
1.4.1.13 pg_collation . . . 290
1.4.1.14 pg_config . . . 291
1.4.1.15 pg_constraint . . . 291
1.4.1.16 pg_conversion . . . 293
1.4.1.17 pg_cursors . . . 294
1.4.1.18 pg_database . . . 294
1.4.1.19 pg_db_role_setting . . . 296
1.4.1.20 pg_default_acl . . . 296
1.4.1.21 一个 pg_default_acl 项展示了要分配给属于一个指定用户的对象的初始权限。当前有两类项：defaclnamespace . . . 297
1.4.1.22 pg_depend . . . 297
1.4.1.23 pg_description . . . 298
1.4.1.24 pg_enum . . . 298
1.4.1.25 pg_event_trigger . . . 299
1.4.1.26 pg_extension . . . 299
1.4.1.27 pg_file_settings . . . 300
1.4.1.28 pg_foreign_data_wrapper . . . 301
1.4.1.29 pg_foreign_server . . . 302
1.4.1.30 pg_foreign_table . . . 302
1.4.1.31 pg_group . . . 303
1.4.1.32 pg_hba_file_rules . . . 303
1.4.1.33 pg_index . . . 304
1.4.1.34 pg_indexes . . . 306
1.4.1.35 pg_inherits . . . 306
1.4.1.36 pg_init_privs . . . 306

1.4.1.37 pg_language 307
1.4.1.38 pg_largeobject 308
1.4.1.39 pg_largeobject_metadata 309
1.4.1.40 pg_locks 309
1.4.1.41 pg_locks 视图显示来自于普通锁管理器和谓词锁管理器的数据，它们是独立的系统。此外，普通锁管理器把它的锁分为普通锁和快速路径锁。这些数据并不被保证是完全一致的。当视图被查询时，快速路径锁上的数据（fastpath 311
1.4.1.42 pg_matviews 312
1.4.1.43 pg_namespace 312
1.4.1.44 pg_opclass 313
1.4.1.45 pg_operator 313
1.4.1.46 pg_opfamily 314
1.4.1.47 pg_partitioned_table 315
1.4.1.48 pg_pltemplate 316
1.4.1.49 pg_policies 317
1.4.1.50 pg_policy 317
1.4.1.51 pg_prepared_statements 318
1.4.1.52 pg_prepared_xacts 319
1.4.1.53 pg_proc 319
1.4.1.54 pg_publication 323
1.4.1.55 pg_publication_rel 323
1.4.1.56 pg_publication_tables 324
1.4.1.57 pg_range 324
1.4.1.58 pg_replication_origin 325
1.4.1.59 pg_replication_origin_status 325
1.4.1.60 pg_replication_slots 325
1.4.1.61 pg_rewrite 326
1.4.1.62 pg_roles 327
1.4.1.63 pg_rules 328
1.4.1.64 pg_seclabel 329
1.4.1.65 pg_seclabels 329
1.4.1.66 pg_sequence 330
1.4.1.67 pg_sequences 330
1.4.1.68 pg_settings 331
1.4.1.69 pg_shadow 333
1.4.1.70 pg_shdepend 334
1.4.1.71 pg_shdescription 335
1.4.1.72 pg_shseclabel 336
1.4.1.73 pg_statistic 336
1.4.1.74 pg_statistic_ext 338
1.4.1.75 pg_statistic_ext_data 338
1.4.1.76 pg_stats 339

1.4.1.77 pg_stats_ext . . . 341
1.4.1.78 pg_subscription . . . 342
1.4.1.79 pg_subscription_rel . . . 343
1.4.1.80 pg_tables . . . 343
1.4.1.81 pg_tablespace . . . 344
1.4.1.82 pg_timezone_abbrevs . . . 344
1.4.1.83 pg_timezone_names . . . 345
1.4.1.84 pg_transform . . . 345
1.4.1.85 pg_trigger . . . 346
1.4.1.86 pg_ts_config . . . 347
1.4.1.87 pg_ts_config_map . . . 348
1.4.1.88 pg_ts_dict . . . 348
1.4.1.89 pg_ts_parser . . . 349
1.4.1.90 pg_ts_template . . . 350
1.4.1.91 pg_type . . . 350
1.4.1.92 pg_user . . . 355
1.4.1.93 pg_user_mapping . . . 356
1.4.1.94 pg_user_mappings . . . 356
1.4.1.95 pg_views . . . 357
1.4.2 信息模式视图 . . . 358
1.4.2.1 administrable_role_authorizations . . . 358
1.4.2.2 applicable_roles . . . 358
1.4.2.3 attributes . . . 359
1.4.2.4 character_sets . . . 362
1.4.2.5 check_constraint_routine_usage . . . 363
1.4.2.6 check_constraints . . . 363
1.4.2.7 collation_character_set_applicability . . . 364
1.4.2.8 collations . . . 364
1.4.2.9 column_column_usage . . . 365
1.4.2.10 column_domain_usage . . . 365
1.4.2.11 column_options . . . 366
1.4.2.12 column_privileges . . . 366
1.4.2.13 column_udt_usage . . . 367
1.4.2.14 columns . . . 368
1.4.2.15 constraint_column_usage . . . 372
1.4.2.16 constraint_table_usage . . . 372
1.4.2.17 data_type_privileges . . . 373
1.4.2.18 domain_constraints . . . 374
1.4.2.19 domain_udt_usage . . . 374
1.4.2.20 domains . . . 375
1.4.2.21 element_types . . . 377
1.4.2.22 enabled_roles . . . 380

1.4.2.23 foreign_data_wrapper_options . . . 380
1.4.2.24 foreign_data_wrappers . . . 381
1.4.2.25 foreign_server_options . . . 381
1.4.2.26 foreign_servers . . . 382
1.4.2.27 foreign_table_options . . . 382
1.4.2.28 foreign_tables . . . 383
1.4.2.29 模式 . . . 383
1.4.2.30 information_schema_catalog_name . . . 384
1.4.2.31 数据类型 . . . 384
1.4.2.32 key_column_usage . . . 384
1.4.2.33 parameters . . . 385
1.4.2.34 triggers . . . 387
1.4.2.35 referential_constraints . . . 389
1.4.2.36 role_column_grants . . . 390
1.4.2.37 role_package_grants . . . 391
1.4.2.38 role_routine_grants . . . 392
1.4.2.39 role_table_grants . . . 392
1.4.2.40 role_udt_grants . . . 393
1.4.2.41 role_usage_grants . . . 394
1.4.2.42 routine_privileges . . . 395
1.4.2.43 routines . . . 395
1.4.2.44 schemata . . . 402
1.4.2.45 sequences . . . 403
1.4.2.46 sql_features . . . 404
1.4.2.47 sql_implementation_info . . . 405
1.4.2.48 sql_languages . . . 405
1.4.2.49 sql_packages . . . 406
1.4.2.50 sql_parts . . . 407
1.4.2.51 sql_sizing . . . 407
1.4.2.52 sql_sizing_profiles . . . 408
1.4.2.53 table_constraints . . . 408
1.4.2.54 table_privileges . . . 409
1.4.2.55 tables . . . 410
1.4.2.56 transforms . . . 411
1.4.2.57 triggered_update_columns . . . 412
1.4.2.58 udt_privileges . . . 413
1.4.2.59 usage_privileges . . . 413
1.4.2.60 user_defined_types . . . 414
1.4.2.61 user_mapping_options . . . 416
1.4.2.62 user_mappings . . . 417
1.4.2.63 view_column_usage . . . 417
1.4.2.64 view_routine_usage . . . 418

1.4.2.65 view_table_usage . . . 419
1.4.2.66 views . . . 419
1.5 SQLServer 兼容视图 . . . 420
1.5.1 元信息视图 . . . 421
1.5.1.1 sys.all_columns . . . 421
1.5.1.2 sys.all_objects . . . 423
1.5.1.3 sys.change_tracking_tables . . . 425
1.5.1.4 sys.check_constraints . . . 426
1.5.1.5 sys.columns . . . 427
1.5.1.6 sys.configurations . . . 429
1.5.1.7 sys.data_spaces . . . 430
1.5.1.8 sys.database_files . . . 431
1.5.1.9 sys.database_permissions . . . 432
1.5.1.10 sys.database_principals . . . 434
1.5.1.11 sys.database_role_members . . . 435
1.5.1.12 sys.databases . . . 436
1.5.1.13 sys.extended_properties . . . 441
1.5.1.14 sys.filegroups . . . 442
1.5.1.15 sys.foreign_key_columns . . . 443
1.5.1.16 sys.foreign_keys . . . 444
1.5.1.17 sys.identity_columns . . . 445
1.5.1.18 sys.index_columns . . . 447
1.5.1.19 sys.indexes . . . 448
1.5.1.20 sys.key_constraints . . . 450
1.5.1.21 sys.linked_logins . . . 451
1.5.1.22 sys.master_files . . . 451
1.5.1.23 sys.objects . . . 453
1.5.1.24 sys.partitions . . . 455
1.5.1.25 sys.procedures . . . 456
1.5.1.26 sys.remote_logins . . . 457
1.5.1.27 sys.schemas . . . 457
1.5.1.28 sys.server_principals . . . 458
1.5.1.29 sys.servers . . . 459
1.5.1.30 sys.sql_modules . . . 460
1.5.1.31 sys.sysaltfiles . . . 461
1.5.1.32 sys.syscharsets . . . 462
1.5.1.33 sys.syscolumns . . . 463
1.5.1.34 sys.syscomments . . . 465
1.5.1.35 sys.sysconfigures . . . 465
1.5.1.36 sys.sysconstraints . . . 466
1.5.1.37 sys.syscurconfigs . . . 466
1.5.1.38 sys.sysdatabases . . . 467

1.5.1.39 sys.sysfilegroups 468
1.5.1.40 sys.sysfiles 468
1.5.1.41 sys.sysforeignkeys 469
1.5.1.42 sys.sysindexes 470
1.5.1.43 sys.sysindexkeys 472
1.5.1.44 sys.syslanguages 472
1.5.1.45 sys.syslogins 473
1.5.1.46 sys.sysmembers 474
1.5.1.47 sys.sysobjects 475
1.5.1.48 sys.sysoledbusers 477
1.5.1.49 sys.syspermissions 477
1.5.1.50 sys.sysprocesses 478
1.5.1.51 sys.sysreferences 480
1.5.1.52 sys.sysremotelogins 482
1.5.1.53 sys.systypes 483
1.5.1.54 sys.sysusers 484
1.5.1.55 sys.table_types 485
1.5.1.56 sys.tables 485
1.5.1.57 sys.types 487
1.5.1.58 sys.views 488
1.5.2 信息模式视图 489
1.5.2.1 check_constraints 489
1.5.2.2 column_domain_usage 490
1.5.2.3 column_privileges 490
1.5.2.4 columns 491
1.5.2.5 constraint_column_usage 493
1.5.2.6 constraint_table_usage 494
1.5.2.7 domain_constraints 495
1.5.2.8 domains 495
1.5.2.9 key_column_usage 497
1.5.2.10 parameters 498
1.5.2.11 referential_constraints 501
1.5.2.12 routines 502
1.5.2.13 schemata 506
1.5.2.14 table_constraints 507
1.5.2.15 table_privileges 508
1.5.2.16 tables 509
1.5.2.17 view_column_usage 509
1.5.2.18 view_table_usage 510
1.5.2.19 views 510

第 2 章　等待事件 512

2.1 Activity . . . 512
2.2 BufferPin . . . 513
2.3 Client . . . 514
2.4 Extension . . . 514
2.5 IO . . . 515
2.6 IPC . . . 518
2.7 LightLock . . . 520
2.8 Lock . . . 523
2.9 Timeout . . . 524

第 3 章 插件参考 525
3.1 扩展插件概述 . . . 525
3.2 amcheck . . . 529
3.2.1 插件 amcheck 简介 . . . 529
3.2.2 插件 amcheck 加载方式 . . . 529
3.2.3 插件 amcheck 的参数配置 . . . 530
3.2.4 插件 amcheck 的使用方法 . . . 530
3.2.5 插件 amcheck 卸载方法 . . . 531
3.2.6 插件 amcheck 升级方法 . . . 531
3.3 auto_bmr . . . 531
3.3.1 插件 auto_bmr 简介 . . . 531
3.3.2 插件 auto_bmr 加载方式 . . . 531
3.3.3 插件 auto_bmr 的参数配置 . . . 531
3.3.4 插件 auto_bmr 的使用方法 . . . 532
3.3.5 插件 auto_bmr 卸载方法 . . . 533
3.3.6 插件 auto_bmr 升级方法 . . . 533
3.4 auto_explain . . . 533
3.4.1 插件 auto_explain 简介 . . . 533
3.4.2 插件 auto_explain 加载方式 . . . 533
3.4.3 插件 auto_explain 的参数配置 . . . 534
3.4.4 插件 auto_explain 的使用方法 . . . 535
3.4.5 插件 auto_explain 卸载方法 . . . 535
3.4.6 插件 auto_explain 升级方法 . . . 535
3.5 backtrace . . . 535
3.5.1 插件 backtrace 简介 . . . 535
3.5.2 插件 backtrace 加载方式 . . . 535
3.5.3 插件 backtrace 的参数配置 . . . 536
3.5.4 插件 backtrace 的使用方法 . . . 536
3.5.5 插件 backtrace 卸载方法 . . . 536
3.5.6 插件 backtrace 升级方法 . . . 536
3.6 backup_pri . . . 536
3.6.1 插件 backup_pri 简介 . . . 536

3.6.2 插件 backup_pri 加载方式 536
3.6.3 插件 backup_pri 的参数配置 537
3.6.4 插件 backup_pri 使用方法 537
3.6.4.1 备份恢复权限的授予与回收 537
3.6.4.2 备份恢复权限流程 538
3.6.4.3 相关系统视图 538
3.6.5 插件 backup_pri 卸载方法 540
3.6.6 插件 backup_pri 升级方法 540
3.7 bloom 541
3.7.1 插件 bloom 简介 541
3.7.2 插件 bloom 加载方式 541
3.7.3 插件 bloom 的参数配置 541
3.7.4 插件 bloom 的使用方法 541
3.7.5 插件 bloom 卸载方法 542
3.7.6 插件 bloom 升级方法 542
3.8 btree_gin 542
3.8.1 插件 btree_gin 简介 542
3.8.2 插件 btree_gin 加载方式 543
3.8.3 插件 btree_gin 的参数配置 543
3.8.4 插件 btree_gin 的使用方法 543
3.8.5 插件 btree_gin 卸载方法 543
3.8.6 插件 btree_gin 升级方法 543
3.9 btree_gist 544
3.9.1 插件 btree_gist 简介 544
3.9.2 插件 btree_gist 加载方式 544
3.9.3 插件 btree_gist 的参数配置 544
3.9.4 插件 btree_gist 的使用方法 545
3.9.5 插件 btree_gist 卸载方法 545
3.9.6 插件 btree_gist 升级方法 545
3.10 cstore_fdw 546
3.10.1 插件 cstore_fdw 简介 546
3.10.2 插件 cstore_fdw 加载方式 546
3.10.3 插件 cstore_fdw 的参数配置 546
3.10.4 插件 cstore_fdw 的使用方法 546
3.10.4.1 示例 547
3.10.4.2 升级 549
3.10.5 插件 cstore_fdw 卸载方法 550
3.10.6 插件 cstore_fdw 升级方法 550
3.11 cube 550
3.11.1 插件 cube 简介 550
3.11.2 插件 cube 加载方式 550
3.11.3 插件 cube 的参数配置 550

3.11.4 插件 cube 的使用方法 551
3.11.4.1 外部表示 551
3.11.4.2 精度 551
3.11.4.3 操作符 551
3.11.4.4 函数 553
3.11.4.5 默认值 555
3.11.5 插件 cube 的卸载方法 556
3.11.6 插件 cube 升级方法 556
3.12 dblink 556
3.12.1 插件 dblink 简介 556
3.12.2 插件 dblink 加载方式 557
3.12.3 插件 dblink 的参数配置 557
3.12.4 插件 dblink 使用方法 557
3.12.4.1 dblink_connect 557
3.12.4.2 dblink_connect_u 559
3.12.4.3 dblink_disconnect 560
3.12.4.4 dblink 560
3.12.4.5 dblink_exec 563
3.12.4.6 dblink_open 565
3.12.4.7 dblink_fetch 566
3.12.4.8 dblink_close 568
3.12.4.9 dblink_get_connections 569
3.12.4.10 dblink_error_message 569
3.12.4.11 dblink_send_query 570
3.12.4.12 dblink_is_busy 570
3.12.4.13 dblink_get_notify 571
3.12.4.14 dblink_get_result 572
3.12.4.15 dblink_cancel_query 574
3.12.4.16 dblink_get_pkey 575
3.12.4.17 dblink_build_sql_insert 576
3.12.4.18 dblink_build_sql_delete 577
3.12.4.19 dblink_build_sql_update 578
3.12.5 插件 dblink 卸载方法 579
3.12.6 插件 dblink 升级方法 579
3.13 dbms_ddl 579
3.13.1 插件 dbms_ddl 简介 579
3.13.2 插件 dbms_ddl 加载方式 580
3.13.3 插件 dbms_ddl 的参数配置 580
3.13.4 插件 dbms_ddl 使用方法 580
3.13.4.1 DBMS_DDL.WRAP 580
3.13.4.2 DBMS_DDL.CREATE_WRAPPED 581
3.13.5 插件 dbms_ddl 卸载方法 582

3.13.6 插件 dbms_ddl 升级方法 582
3.14 dbms_lob 583
3.14.1 插件 dbms_lob 简介 583
3.14.2 插件 dbms_lob 的加载方式 583
3.14.3 插件 dbms_lob 的参数配置 583
3.14.4 插件 dbms_lob 卸载方法 583
3.14.5 插件 dbms_lob 升级方法 583
3.15 dbms_metadata 584
3.15.1 插件 dbms_metadata 简介 584
3.15.2 插件 dbms_metadata 加载方式 584
3.15.3 插件 dbms_metadata 的参数配置 584
3.15.4 插件 dbms_metadata 使用方法 584
3.15.4.1 DBMS_METADATA.GET_DDL 584
3.15.5 插件 dbms_metadata 卸载方法 587
3.15.6 插件 dbms_metadata 升级方法 587
3.16 dbms_mview 587
3.16.1 插件 dbms_mview 简介 587
3.16.2 插件 dbms_mview 加载方式 587
3.16.3 插件 dbms_mview 使用方法 588
3.16.3.1 REFRESH 存储过程 588
3.16.4 插件 dbms_mview 卸载方法 589
3.16.5 插件 dbms_mview 升级方法 589
3.17 dbms_obfuscation_toolkit 589
3.17.1 插件 dbms_obfuscation_toolkit 简介 589
3.17.2 插件 dbms_obfuscation_toolkit 加载方式 589
3.17.3 插件 dbms_obfuscation_toolkit 的参数配置 590
3.17.4 插件 dbms_obfuscation_toolkit 使用方法 590
3.17.4.1 限制 590
3.17.4.2 启用 590
3.17.4.3 禁用 590
3.17.4.4 DBMS_OBFUSCATION_TOOLKIT 系统包子函数接口 590
3.17.4.4.1 DES3ENCRYPT 590
3.17.4.4.2 DES3DECRYPT 591
3.17.4.4.3 DESENCRYPT 592
3.17.4.4.4 DESDECRYPT 593
3.17.4.4.5 MD5 593
3.17.5 插件 dbms_obfuscation_toolkit 卸载方法 594
3.17.6 插件 dbms_obfuscation_toolkit 升级方法 594
3.18 dbms_output 594
3.18.1 插件 dbms_output 简介 594
3.18.2 插件 dbms_output 加载方式 594
3.18.3 插件 dbms_output 的参数配置 595

3.18.4 插件 dbms_output 使用方法 595
3.18.4.1 限制 595
3.18.4.2 启用 595
3.18.4.3 禁用 596
3.18.4.4 DBMS_OUTPUT 系统包支持向缓冲区输入信息 596
3.18.4.4.1 向缓冲区中追加内容 596
3.18.4.4.2 向缓冲区追加单行信息 597
3.18.4.4.3 向缓冲区追加新行 598
3.18.4.5 DBMS_OUTPUT 系统包支持从缓冲区中获取信息 599
3.18.4.5.1 获取首行信息 599
3.18.4.5.2 获取多行信息 599
3.18.4.6 DBMS_OUTPUT 系统包支持客户端显示缓冲区信息 601
3.18.4.6.1 客户端命令启动输出功能 601
3.18.4.6.2 客户端命令禁用输出功能 602
3.18.5 插件 dbms_output 卸载方法 603
3.18.6 插件 dbms_output 升级方法 603
3.19 dbms_random 603
3.19.1 插件 dbms_random 简介 603
3.19.2 插件 dbms_random 加载方式 603
3.19.3 插件 dbms_random 的参数配置 603
3.19.4 插件 dbms_random 的使用方法 604
3.19.4.1 INITIALIZE Procedure 604
3.19.4.2 NORMAL Function 605
3.19.4.3 RANDOM Function 605
3.19.4.4 SEED Procedure 606
3.19.4.5 STRING Function 606
3.19.4.6 TERMINATE Procedure 607
3.19.4.7 VALUE Function 608
3.19.5 插件 dbms_random 卸载方法 609
3.19.6 插件 dbms_random 升级方法 609
3.20 dbms_session 609
3.20.1 插件 dbms_session 简介 609
3.20.2 插件 dbms_session 加载方式 609
3.20.3 插件 dbms_session 的参数配置 609
3.20.4 插件 dbms_session 的使用方法 610
3.20.4.1 DBMS_SESSION 系统包子程序 610
3.20.4.2 CLEAR_ALL_CONTEXT 过程 610
3.20.4.3 CLEAR_CONTEXT 过程 611
3.20.4.4 LIST_CONTEXT 过程 611
3.20.4.5 SET_CONTEXT 过程 612
3.20.5 插件 dbms_session 卸载方法 616
3.20.6 插件 dbms_session 升级方法 616

3.21 dbms_sql 616
3.21.1 插件 dbms_sql 简介 616
3.21.2 插件 dbms_sql 加载方式 616
3.21.3 插件 dbms_sql 的参数配置 616
3.21.4 插件 dbms_sql 使用方法 617
3.21.4.1 DBMS_SQL 包预定义类型 617
3.21.4.1.1 DBMS_SQL 常量 617
3.21.4.1.2 DBMS_SQL 集合类型 617
3.21.4.2 DBMS_SQL 包子程序 618
3.21.4.2.1 打开新游标 618
3.21.4.2.2 解析语句 619
3.21.4.2.3 绑定变量 621
3.21.4.2.3.1 绑定到游标的基本数据类型 621
3.21.4.2.3.2 绑定到游标的 CHAR 类型 624
3.21.4.2.3.3 绑定到游标的关联数组类型 627
3.21.4.2.3.4 绑定到游标的包中类型 629
3.21.4.2.4 定义类型接收列值 630
3.21.4.2.4.1 定义基本数据类型接收单行列值 630
3.21.4.2.4.2 定义 CHAR 类型接收单行列值 632
3.21.4.2.4.3 定义关联数组类型接收单行列值 634
3.21.4.2.5 执行游标 635
3.21.4.2.6 检索游标 636
3.21.4.2.7 执行游标，并检索游标 637
3.21.4.2.8 返回游标中指定类型列值 638
3.21.4.2.8.1 返回游标基础数据类型的列值 638
3.21.4.2.8.2 返回游标关联数组类型的列值 640
3.21.4.2.8.3 返回游标 CHAR 类型的列值 642
3.21.4.2.9 返回游标中给定变量的值 642
3.21.4.2.9.1 返回游标基础数据类型的变量值 642
3.21.4.2.9.2 返回游标关联数组类型的变量值 644
3.21.4.2.9.3 返回游标 CHAR 类型的变量值 646
3.21.4.2.9.4 返回游标包中类型的变量值 646
3.21.4.2.10 获取查询项的描述信息 646
3.21.4.2.10.1 获取查询项的描述信息，为 DESC_TAB 类型 646
3.21.4.2.11 关闭游标 648
3.21.4.2.12 判断游标是否打开 648
3.21.4.2.13 返回累计检索出来的行的数量 650
3.21.4.2.14 游标转换 651
3.21.4.2.14.1 将游标变量转换为 DBMS_SQL 包内部游标 651
3.21.4.2.14.2 将 DBMS_SQL 包内的游标转换为本地动态 SQL 游标 652
3.21.4.2.15 返回语句结果集 654
3.21.4.2.16 接收 RETURN_RESULT 过程的一个返回结果，并返回给客户端应用程序 . 655

3.21.4.2.17 返回 SQL 语句错误发生处的字节偏移量 658
3.21.4.2.18 返回该语句的 SQL 函数代码 660
3.21.4.2.19 其他示例 663
3.21.5 插件 dbms_sql 卸载方法 665
3.21.6 插件 dbms_sql 升级方法 665
3.22 dbms_utility 665
3.22.1 插件 dbms_utility 简介 665
3.22.2 插件 dbms_utility 加载方式 665
3.22.3 插件 dbms_utility 的参数配置 665
3.22.4 插件 dbms_utility 使用方法 665
3.22.4.1 FORMAT_CALL_STACK 函数 666
3.22.4.2 FORMAT_ERROR_BACKTRACE 函数 668
3.22.4.3 FORMAT_ERROR_STACK 函数 669
3.22.4.4 GET_TIME 函数 670
3.22.4.5 GET_HASH_VALUE 函数 670
3.22.5 插件 dbms_utility 卸载方法 672
3.22.6 插件 dbms_utility 升级方法 672
3.23 dbms_xmlquery 672
3.23.1 插件 dbms_xmlquery 简介 672
3.23.2 插件 dbms_xmlquery 加载方式 672
3.23.3 插件 dbms_xmlquery 的参数配置 672
3.23.4 插件 dbms_xmlquery 使用方法 672
3.23.4.1 常量 673
3.23.4.2 函数 675
3.23.4.2.1 getnumrowsprocessed 675
3.23.4.2.2 getXML 676
3.23.4.2.3 newContext 676
3.23.4.3 存储过程 677
3.23.4.3.1 closeContext 677
3.23.4.3.2 setCollIdAttrName 679
3.23.4.3.3 setDataHeader 679
3.23.4.3.4 setMaxRows 681
3.23.4.3.5 setRowIdAttrName 683
3.23.4.3.6 setCollIdAttrName 684
3.23.4.3.7 setRowIdAttrValue 684
3.23.4.3.8 setRowsetTag 685
3.23.4.3.9 setRowTag 686
3.23.4.3.10 setSkipRows 687
3.23.4.3.11 setStylesheetHeader 689
3.23.4.3.12 setTagCase 690
3.23.4.3.13 useTypeForCollElemTag 694
3.23.4.3.14 setEncodingTag 695

3.23.4.3.15 setSQLToXMLNameEscaping . . . 696
3.23.5 插件 dbms_xmlquery 卸载方法 . . . 697
3.23.6 插件 dbms_xmlquery 升级方法 . . . 697
3.24 dict_int . . . 697
3.24.1 插件 dict_int 简介 . . . 697
3.24.2 插件 dict_int 加载方式 . . . 697
3.24.3 插件 dict_int 的参数配置 . . . 698
3.24.4 插件 dict_int 的使用方法 . . . 698
3.24.5 插件 dict_int 卸载方法 . . . 698
3.24.6 插件 dict_int 升级方法 . . . 698
3.25 dict_xsyn . . . 699
3.25.1 插件 dict_xsyn 简介 . . . 699
3.25.2 插件 dict_xsyn 加载方式 . . . 699
3.25.3 插件 dict_xsyn 的参数配置 . . . 699
3.25.4 插件 dict_xsyn 的使用方法 . . . 700
3.25.5 插件 dict_xsyn 卸载方法 . . . 701
3.25.6 插件 dict_xsyn 升级方法 . . . 701
3.26 file_fdw . . . 701
3.26.1 插件 file_fdw 简介 . . . 701
3.26.2 插件 file_fdw 加载方式 . . . 701
3.26.3 插件 file_fdw 的参数配置 . . . 701
3.26.4 插件 file_fdw 的使用方法 . . . 703
3.26.5 插件 file_fdw 卸载方式 . . . 704
3.26.6 插件 file_fdw 升级方式 . . . 704
3.27 fix_security_hole . . . 704
3.27.1 插件 fix_security_hole 简介 . . . 704
3.27.2 插件 fix_security_hole 加载方式 . . . 704
3.27.3 插件 fix_security_hole 的参数配置 . . . 704
3.27.4 插件 fix_security_hole 使用方法 . . . 705
3.27.5 插件 fix_security_hole 卸载方法 . . . 705
3.27.6 插件 fix_security_hole 升级方法 . . . 705
3.28 ftutilx . . . 706
3.28.1 插件 ftutilx 简介 . . . 706
3.28.2 插件 ftutilx 加载方式 . . . 706
3.28.3 插件 ftutilx 的参数配置 . . . 706
3.28.4 插件 ftutilx 的使用方法 . . . 706
3.28.4.1 使用 ftutilx 全文检索的联合使用方案 . . . 707
3.28.4.2 注意事项 . . . 708
3.28.5 插件 ftutilx 卸载方法 . . . 708
3.28.6 插件 ftutilx 升级方法 . . . 708
3.29 fuzzystrmatch . . . 708
3.29.1 插件 fuzzystrmatch 简介 . . . 708

3.29.2 插件 fuzzystrmatch 加载方式 709
3.29.3 插件 fuzzystrmatch 参数配置 709
3.29.4 插件 fuzzystrmatch 使用方法 709
3.29.4.1 levenshtein 709
3.29.4.2 levenshtein_with_costs 710
3.29.4.3 levenshtein_less_equal 711
3.29.4.4 levenshtein_less_equal_with_costs 711
3.29.4.5 metaphone 711
3.29.4.6 soundex, text_soundex 712
3.29.4.7 difference 713
3.29.4.8 dmetaphone 713
3.29.5 插件 fuzzystrmatch 卸载方法 714
3.29.6 插件 fuzzystrmatch 升级方法 714
3.30 hstore 714
3.30.1 插件 hstore 简介 714
3.30.2 插件 hstore 加载方式 714
3.30.3 插件 hstore 参数配置 715
3.30.4 插件 hstore 使用方法 715
3.30.4.1 hstore 类型 715
3.30.4.2 hstore_to_jsonb 720
3.30.4.3 hstore_to_json 720
3.30.5 插件 hstore 卸载方法 721
3.30.6 插件 hstore 升级方法 721
3.31 http 721
3.31.1 插件 http 简介 721
3.31.2 插件 http 加载方式 721
3.31.3 插件 http 的参数配置 721
3.31.3.1 http.keepalive 变量 722
3.31.3.2 http.timeout_msec 变量 722
3.31.4 插件 http 使用方法 722
3.31.4.1 数据类型 722
3.31.4.1.1 UTL_HTTP.req 722
3.31.4.1.2 UTL_HTTP.resp 723
3.31.4.1.3 http_method 723
3.31.4.1.4 http_header 724
3.31.4.1.5 http_response 724
3.31.4.1.6 http_request 725
3.31.4.2 UTL_HTTP.BEGIN_REQUEST 726
3.31.4.3 UTL_HTTP.SET_HEADER 726
3.31.4.4 UTL_HTTP.GET_RESPONSE 727
3.31.4.5 UTL_HTTP.READ_LINE 728
3.31.4.6 UTL_HTTP.READ_TEXT 728

3.31.4.7 UTL_HTTP.END_RESPONSE . . . 729
3.31.4.8 UTL_HTTP.END_REQUEST . . . 729
3.31.4.9 http_header 函数 . . . 730
3.31.4.10 http 函数 . . . 731
3.31.4.11 http_get 函数 . . . 731
3.31.4.12 http_post 函数 . . . 732
3.31.4.13 http_put 函数 . . . 733
3.31.4.14 http_patch 函数 . . . 733
3.31.4.15 http_delete 函数 . . . 734
3.31.4.16 http_head 函数 . . . 735
3.31.4.17 http_set_curlopt 函数 . . . 736
3.31.4.18 http_reset_curlopt 函数 . . . 737
3.31.4.19 http_list_curlopt 函数 . . . 737
3.31.4.20 urlencode 函数 . . . 738
3.31.5 插件 http 卸载方法 . . . 739
3.31.6 插件 http 升级方法 . . . 739
3.32 identity_pwdexp . . . 739
3.32.1 插件 identity_pwdexp 简介 . . . 739
3.32.2 插件 identity_pwdexp 加载方法 . . . 739
3.32.3 插件 identity_pwdexp 的参数配置 . . . 739
3.32.3.1 示例 . . . 740
3.32.4 插件 identity_pwdexp 使用方法 . . . 741
3.32.5 插件 identity_pwdexp 的卸载插件 . . . 741
3.32.6 插件 identity_pwdexp 升级方法 . . . 742
3.33 intagg . . . 742
3.33.1 插件 intagg 简介 . . . 742
3.33.2 插件 intagg 加载方式: . . . 742
3.33.3 插件 intagg 参数配置 . . . 742
3.33.4 插件 intagg 使用方法 . . . 742
3.33.4.1 函数 . . . 742
3.33.5 插件 intagg 卸载方法 . . . 744
3.33.6 插件 intagg 升级方法 . . . 744
3.34 intarray . . . 744
3.34.1 插件 intarray 简介 . . . 744
3.34.2 插件 intarray 加载方式 . . . 744
3.34.3 插件 intarray 参数配置 . . . 745
3.34.4 插件 intarray 使用方法 . . . 745
3.34.4.1 icount . . . 745
3.34.4.2 sort . . . 745
3.34.4.3 sort_asc . . . 746
3.34.4.4 sort_desc . . . 746
3.34.4.5 uniq . . . 747

3.34.4.6 idx . . . 747
3.34.4.7 subarray . . . 747
3.34.4.8 intset . . . 748
3.34.4.9 intarray 模块中支持的操作符 . . . 748
3.34.5 插件 intarray 卸载方法 . . . 748
3.34.6 插件 intarray 升级方法 . . . 748
3.35 isn . . . 749
3.35.1 插件 isn 简介 . . . 749
3.35.2 插件 isn 加载方式 . . . 749
3.35.3 插件 isn 的参数配置 . . . 749
3.35.4 插件 isn 的使用方法 . . . 749
3.35.4.1 数据类型 . . . 749
3.35.4.2 造型 . . . 750
3.35.4.3 函数和操作符 . . . 750
3.35.4.4 示例 . . . 751
3.35.5 插件 isn 卸载方法 . . . 752
3.35.6 插件 isn 升级方法 . . . 752
3.36 kbcrypto . . . 752
3.36.1 插件 kbcrypto 简介 . . . 752
3.36.2 插件 kbcrypto 加载方式 . . . 752
3.36.3 插件 kbcrypto 的参数配置 . . . 753
3.36.4 插件 kbcrypto 的使用方法 . . . 753
3.36.4.1 decrypt 函数 . . . 753
3.36.4.2 digest 函数 . . . 755
3.36.4.3 encrypt 函数 . . . 757
3.36.4.4 encrypt_iv 函数 . . . 759
3.36.4.5 decrypt_iv 函数 . . . 760
3.36.4.6 rc4 函数 . . . 760
3.36.4.7 sm3 函数 . . . 761
3.36.4.7.1 sm3 函数 . . . 761
3.36.4.8 sm4 函数 . . . 762
3.36.4.8.1 sm4 函数 . . . 762
3.36.4.8.2 sm4_ex 函数 . . . 762
3.36.4.9 sm2 函数 . . . 763
3.36.4.9.1 sm2_genkeypair 函数 . . . 763
3.36.4.9.2 sm2_encrypt 函数 . . . 764
3.36.4.9.3 sm2_decrypt 函数 . . . 764
3.36.4.9.4 sm2_sign 函数 . . . 765
3.36.4.9.5 sm2_verify 函数 . . . 765
3.36.5 插件 kbcrypto 卸载方法 . . . 766
3.36.6 插件 kbcrypto 升级方法 . . . 766
3.37 kbrowlocks . . . 766

3.37.1 插件 kbrowlocks 简介 766
3.37.2 插件 kbrowlocks 加载方式 767
3.37.3 插件 kbrowlocks 的参数配置 767
3.37.4 插件 kbrowlocks 的使用方法 767
3.37.5 插件 kbrowlocks 卸载方法 768
3.37.6 插件 kbrowlocks 升级方法 768
3.38 kbstattuple 768
3.38.1 插件 kbstattuple 简介 768
3.38.2 插件 kbstattuple 加载方式 769
3.38.3 插件 kbstattuple 的参数配置 769
3.38.4 插件 kbstattuple 的使用方法 769
3.38.5 插件 kbstattuple 卸载方法 771
3.38.6 插件 kbstattuple 升级方法 771
3.39 kdb_database_link 772
3.39.1 插件 Database Link 简介 772
3.39.1.1 Database Link 功能 773
3.39.2 插件 kdb_database_link 的加载方式 773
3.39.3 插件 kdb_database_link 的参数配置 773
3.39.4 插件 kdb_database_link 的使用方法 774
3.39.4.1 创建 Database Link 对象 774
3.39.4.2 修改 Database Link 对象 775
3.39.4.3 删除 Database Link 对象 776
3.39.4.4 Database Link 的权限控制 776
3.39.4.5 DBLink 相关的视图 776
3.39.4.6 DATABASE LINK DML 777
3.39.4.7 DATABASE LINK MERGE 779
3.39.4.8 DATABASE LINK 调用函数/存储过程 781
3.39.5 插件 kdb_database_link 的卸载方法 782
3.39.6 插件 kdb_database_link 的升级方法 783
3.40 kdb_ddl 783
3.40.1 插件 kdb_ddl 简介 783
3.40.2 插件 kdb_ddl 加载方式 783
3.40.3 插件 kdb_ddl 的参数配置 783
3.40.4 插件 kdb_ddl 使用方法 784
3.40.4.1 获取完整的数据库对象 DDL 脚本 784
3.40.4.1.1 通过对象名获取完整的 DDL 脚本 784
3.40.4.1.2 根据 OID 获取完整的 DDL 脚本 786
3.40.4.2 获取数据库对象创建语句 787
3.40.4.2.1 通过对象名获取创建语句 787
3.40.4.2.1.1 包含权限处理的对象创建语句 787
3.40.4.2.1.2 不包含权限处理的对象创建语句 788
3.40.4.2.2 通过 OID 获取创建语句 789

3.40.4.2.2.1 包含权限处理的对象创建语句 789
3.40.4.2.2.2 不包含权限处理的对象创建语句 791
3.40.4.3 获取数据库对象删除语句 792
3.40.4.3.1 通过对象名获取删除语句 792
3.40.4.3.2 通过 OID 获取删除语句 793
3.40.4.4 获取数据库对象授权语句 793
3.40.4.4.1 通过对象名获取授权语句 793
3.40.4.4.2 通过 OID 获取授权语句 794
3.40.5 插件 kdb_ddl 卸载方法 795
3.40.6 插件 kdb_ddl 升级方法 795
3.41 kdb_exists_expand 796
3.41.1 插件 kdb_exists_expand 简介 796
3.41.2 插件 kdb_exists_expand 加载方式 796
3.41.3 插件 kdb_exists_expand 的参数配置 796
3.41.4 插件 kdb_exists_expand 的使用方法 796
3.41.5 插件 kdb_exists_expand 卸载方法 797
3.41.6 插件 kdb_exists_expand 升级方法 797
3.42 kdb_flashback 798
3.42.1 插件 kdb_flashback 简介 798
3.42.2 插件 kdb_flashback 加载方式 798
3.42.3 插件 kdb_flashback 的参数配置 798
3.42.4 插件 kdb_flashback 的使用方法 798
3.42.4.1 闪回查询 799
3.42.4.2 闪回版本查询 800
3.42.4.3 闪回表到指定时间点 801
3.42.4.4 闪回查询和闪回表技术的使用须知 802
3.42.4.5 闪回回收站 802
3.42.5 插件 kdb_flashback 卸载方法 804
3.42.6 插件 kdb_flashback 升级方法 804
3.43 kdb_license 804
3.43.1 插件 kdb_license 简介 804
3.43.2 插件 kdb_license 加载方式 804
3.43.3 插件 kdb_license 的参数配置 804
3.43.4 插件 kdb_license 的使用方法 804
3.43.5 插件 kdb_license 卸载方法 804
3.43.6 插件 kdb_license 升级方法 805
3.44 kdb_mysql_functions 805
3.44.1 插件 kdb_mysql_functions 简介 805
3.44.2 插件 kdb_mysql_functions 加载方式 805
3.44.3 插件 kdb_mysql_functions 配置参数 805
3.44.4 插件 kdb_mysql_functions 使用方法 805
3.44.5 插件 kdb_mysql_functions 卸载方法 807

3.44.6 插件 kdb_mysql_functions 升级方法 . . . 807
3.45 kdb_mysql_rum . . . 808
3.45.1 插件 kdb_mysql_rum 简介 . . . 808
3.45.2 插件 kdb_mysql_rum 加载方式 . . . 808
3.45.3 插件 kdb_mysql_rum 的参数配置 . . . 808
3.45.4 插件 kdb_mysql_rum 的使用方法 . . . 808
3.45.5 插件 kdb_mysql_rum 卸载方法 . . . 808
3.45.6 插件 kdb_mysql_rum 升级方法 . . . 808
3.46 kdb_oracle_datatype . . . 809
3.46.1 插件 kdb_oracle_datatype 简介 . . . 809
3.46.2 插件 kdb_oracle_datatype 加载方式 . . . 809
3.46.3 插件 kdb_oracle_datatype 的参数配置 . . . 809
3.46.4 插件 kdb_oracle_datatype 的使用方法 . . . 809
3.46.4.1 支持 blob、clob、nclob 类型导入导出和置空 . . . 809
3.46.4.2 bpcharbyte 类型 . . . 810
3.46.4.3 bpcharbyte 类型操作符 . . . 810
3.46.4.4 bpcharbyte 类型转换 . . . 810
3.46.5 插件 kdb_oracle_datatype 卸载方法 . . . 811
3.46.6 插件 kdb_oracle_datatype 升级方法 . . . 811
3.47 kdb_oracle_datatype_nls . . . 811
3.47.1 插件 kdb_oracle_datatype_nls 简介 . . . 811
3.47.2 插件 kdb_oracle_datatype_nls 加载方式 . . . 811
3.47.3 插件 kdb_oracle_datatype_nls 的参数配置 . . . 812
3.47.4 插件 kdb_oracle_datatype_nls 的使用方法 . . . 812
3.47.4.1 操作符的支持 . . . 812
3.47.4.2 索引的支持 . . . 812
3.47.5 插件 kdb_oracle_datatype_nls 卸载方法 . . . 812
3.47.6 插件 kdb_oracle_datatype_nls 升级方法 . . . 813
3.48 kdb_orafce . . . 813
3.48.1 插件 kdb_orafce 简介 . . . 813
3.48.2 插件 kdb_orafce 加载方式 . . . 813
3.48.3 插件 kdb_orafce 的参数配置 . . . 813
3.48.4 插件 kdb_orafce 的使用方法 . . . 813
3.48.4.1 sys_catalog.trunc . . . 813
3.48.4.2 sys_catalog.next_day . . . 815
3.48.4.3 sys_catalog.last_day . . . 815
3.48.4.4 sys_catalog.months_between . . . 816
3.48.4.5 sys_catalog.add_months . . . 816
3.48.4.6 sys_catalog.set_nls_sort . . . 817
3.48.4.7 sys_catalog.to_char . . . 819
3.48.4.8 sys_catalog.to_number . . . 819
3.48.4.9 sys_catalog.to_date . . . 820

3.48.4.10 sys_catalog.to_multi_byte . . . 820
3.48.4.11 sys_catalog.to_single_byte . . . 821
3.48.4.12 sinh . . . 822
3.48.4.13 cosh . . . 822
3.48.4.14 tanh . . . 823
3.48.4.15 dump . . . 823
3.48.4.16 plvstr.rvrs . . . 824
3.48.4.17 oracle.substr . . . 825
3.48.4.18 oracle.add_days_to_timestamp . . . 826
3.48.4.19 oracle.subtract . . . 827
3.48.4.20 oracle.add_months . . . 827
3.48.4.21 oracle.last_day . . . 828
3.48.4.22 oracle.months_between . . . 829
3.48.4.23 oracle.next_day . . . 830
3.48.4.24 oracle.to_date . . . 831
3.48.4.25 oracle.to_char . . . 832
3.48.4.26 oracle.sysdate . . . 832
3.48.4.27 oracle.sessiontimezone . . . 833
3.48.4.28 oracle.dbtimezone . . . 833
3.48.5 插件 kdb_orafce 卸载方法 . . . 834
3.48.6 插件 kdb_orafce 升级方法 . . . 834
3.49 kdb_partman . . . 834
3.49.1 插件 kdb_partman 简介 . . . 834
3.49.1.1 功能和发展 . . . 834
3.49.1.2 插入数据机制解释 . . . 834
3.49.1.3 插入数据的策略 . . . 834
3.49.1.4 partman 的设计约束和特性 . . . 835
3.49.1.4.1 时区 . . . 835
3.49.1.4.2 子分区 . . . 835
3.49.1.4.3 Retention . . . 835
3.49.1.4.4 约束排除 . . . 836
3.49.1.4.5 自定义时间间隔注意事项 . . . 836
3.49.1.4.6 命名长度限制 . . . 836
3.49.1.4.7 唯一约束和更新 . . . 837
3.49.1.4.8 日志记录/监控 . . . 837
3.49.1.4.9 后台进程 . . . 837
3.49.2 插件 kdb_partman 加载方式 . . . 838
3.49.3 插件 kdb_partman 的参数配置 . . . 838
3.49.4 插件 kdb_partman 的使用方法 . . . 838
3.49.4.1 函数 . . . 838
3.49.4.1.1 create 函数 . . . 838
3.49.4.1.2 维护函数 . . . 839

3.49.4.1.3 销毁函数 841
3.49.4.1.4 脚本 842
3.49.4.1.5 编译 842
3.49.4.1.6 示例 842
3.49.4.1.6.1 创建本地分区 842
3.49.4.1.6.2 10 个 id 值一个分区 845
3.49.4.1.6.3 分区一个已经存在的表 848
3.49.4.1.6.4 离线分区 848
3.49.4.1.6.5 在线分区 850
3.49.4.1.6.6 撤销本地分区 856
3.49.4.1.6.7 设置触发分区 857
3.49.4.1.6.8 每天一个分区 857
3.49.4.1.6.9 10 个数一个分区 859
3.49.4.1.6.10 10 个数一个分区创建的空表使用 upsert 丢弃行冲突 862
3.49.4.1.6.11 用 upsert 更新行冲突 864
3.49.4.1.6.12 创建多级子分区 866
3.49.4.1.6.13 时间多级子分区 866
3.49.4.1.6.14 从触发分区迁移到本地 877
3.49.4.1.6.15 迁移已有数据表到 partman 882
3.49.5 插件 kdb_partman 卸载方法 884
3.49.6 插件 kdb_partman 升级方法 884
3.50 kdb_protect 885
3.50.1 插件 kdb_protect 简介 885
3.50.2 插件 kdb_protect 加载方式 885
3.50.3 插件 kdb_protect 的参数配置 885
3.50.4 插件 kdb_protect 的使用方法 885
3.50.4.1 kdb_protect.protect_add 885
3.50.4.2 kdb_protect.protect_add_oid 886
3.50.4.3 kdb_protect.protect_remove 887
3.50.4.4 kdb_protect.protect_remove_oid 887
3.50.4.5 kdb_protect.protect_remove_all 888
3.50.4.6 kdb_protect.protect_show() 888
3.50.4.7 kdb_protect.protect_show_detail() 889
3.50.4.8 kdb_protect.protect_show_one() 889
3.50.4.9 kdb_protect.protect_show_one_detail(name text, type char) 890
3.50.5 插件 kdb_protect 卸载方法 890
3.50.6 插件 kdb_protect 升级方法 891
3.51 kdb_raw 891
3.51.1 插件 kdb_raw 简介 891
3.51.2 插件 kdb_raw 加载方式 891
3.51.3 插件 kdb_raw 的参数配置 891
3.51.4 插件 kdb_raw 卸载方法 891

3.51.5 插件 kdb_raw 升级方法 891
3.52 kdb_resource_group 892
3.52.1 插件 kdb_resource_group 简介 892
3.52.2 插件 kdb_resource_group 加载方式 892
3.52.3 插件 kdb_resource_group 的参数配置 892
3.52.4 插件 kdb_resource_group 的使用方法 893
3.52.4.1 创建资源组 894
3.52.4.2 修改资源组 895
3.52.4.3 删除资源组 896
3.52.4.4 查看资源组 896
3.52.4.5 用户和数据库关联资源组 898
3.52.4.5.1 创建用户指定资源组 898
3.52.4.5.2 创建库指定资源组 898
3.52.4.5.3 修改用户资源组 899
3.52.4.5.4 修改数据库资源组 899
3.52.4.6 资源组关联用户 899
3.52.4.6.1 查看资源组与用户关联关系 900
3.52.5 插件 kdb_resource_group 卸载方法 900
3.52.6 插件 kdb_resource_group 升级方法 900
3.53 kdb_schedule 900
3.53.1 概述 900
3.53.1.1 权限 900
3.53.1.2 日历表示法 901
3.53.2 kdb_schedule 的使用 903
3.53.2.1 配置及加载 903
3.53.2.2 使用 904
3.53.2.3 升级 904
3.53.2.4 卸载 905
3.53.3 kdb_schedule 的组成 905
3.53.3.1 DBMS_JOB 模式 906
3.53.3.1.1 BROKEN 存储过程 906
3.53.3.1.2 CHANGE 存储过程 907
3.53.3.1.3 INTERVAL 存储过程 907
3.53.3.1.4 NEXT_DATE 存储过程 908
3.53.3.1.5 REMOVE 存储过程 908
3.53.3.1.6 RUN 存储过程 909
3.53.3.1.7 SUBMIT 存储过程 909
3.53.3.1.8 WHAT 存储过程 910
3.53.3.1.9 INSTANCE 存储过程 910
3.53.3.2 DBMS_SCHEDULER 模式 911
3.53.3.2.1 CREATE_PROGRAM 存储过程 911
3.53.3.2.2 DROP_PROGRAM 存储过程 912

3.53.3.2.3 CREATE_SCHEDULE 存储过程 . . . 913
3.53.3.2.4 DROP_SCHEDULE 存储过程 . . . 913
3.53.3.2.5 CREATE_JOB 存储过程 . . . 914
3.53.3.2.6 DROP_JOB 存储过程 . . . 915
3.53.3.2.7 RUN_JOB 存储过程 . . . 915
3.53.3.2.8 DISABLE 存储过程 . . . 916
3.53.3.2.9 ENABLE 存储过程 . . . 916
3.53.3.3 KDB_SCHEDULE 模式 . . . 916
3.54 kdb_tinyint . . . 918
3.54.1 插件 kdb_tinyint 简介 . . . 918
3.54.2 插件 kdb_tinyint 加载方式 . . . 918
3.54.3 插件 kdb_tinyint 的参数配置 . . . 918
3.54.4 插件 kdb_tinyint 的使用方法 . . . 918
3.54.4.1 conversion functions support . . . 918
3.54.4.2 operator functions support . . . 919
3.54.4.3 index suppor . . . 920
3.54.4.4 mathematics functions support . . . 920
3.54.5 插件 kdb_tinyint 卸载方法 . . . 921
3.54.6 插件 kdb_tinyint 升级方法 . . . 921
3.55 kdb_utils_function . . . 921
3.55.1 插件 kdb_utils_function 简介 . . . 921
3.55.2 插件 kdb_utils_function 加载方式 . . . 921
3.55.3 插件 kdb_utils_function 配置参数 . . . 921
3.55.4 插件 kdb_utils_function 使用方法 . . . 921
3.55.4.1 函数列表 . . . 921
3.55.5 插件 kdb_utils_function 卸载方法 . . . 924
3.55.6 插件 kdb_utils_function 升级方法 . . . 924
3.56 kdb_xmltype . . . 924
3.56.1 插件 kdb_xmltype 简介 . . . 924
3.56.2 插件 kdb_xmltype 加载方式 . . . 924
3.56.3 插件 kdb_xmltype 的参数配置 . . . 925
3.56.4 插件 kdb_xmltype 的使用方法 . . . 925
3.56.4.1 XMLTYPE 类型 . . . 925
3.56.4.1.1 输入输出 . . . 925
3.56.4.1.2 操作符 . . . 925
3.56.4.1.3 类型转换 . . . 925
3.56.4.1.4 索引 . . . 926
3.56.4.1.5 xml 函数 . . . 926
3.56.5 XMLTYPE 成员函数 . . . 926
3.56.5.1 createXML 函数 . . . 926
3.56.5.2 existsnode 函数 . . . 926
3.56.5.3 extract 函数 . . . 927

3.56.5.4 getclobval 函数 928
3.56.5.5 getnumberval 函数 928
3.56.5.6 getstringval 函数 929
3.56.5.7 XMLSEQUENCE 函数 930
3.56.5.8 VALUE 函数 930
3.56.6 插件 kdb_xmltype 卸载方法 931
3.56.7 插件 kdb_xmltype 升级方法 931
3.57 kingbase_fdw 931
3.57.1 插件 kingbase_fdw 简介 931
3.57.2 插件 kingbase_fdw 的加载方式 932
3.57.3 插件 kingbase_fdw 的参数配置 932
3.57.3.1 连接选项 932
3.57.3.2 对象名称选项 933
3.57.3.3 代价估计选项 933
3.57.3.4 远程执行选项 934
3.57.3.5 可更新性选项 934
3.57.3.6 导入选项 934
3.57.4 插件 kingbase_fdw 的使用 935
3.57.5 插件 kingbase_fdw 的卸载 936
3.57.6 插件 kingbase_fdw 的升级 936
3.58 ktrack 936
3.58.1 插件 ktrack 简介 936
3.58.2 插件 ktrack 加载方式 936
3.58.3 插件 ktrack 的参数配置 937
3.58.4 插件 ktrack 的使用方法 937
3.58.5 插件 ktrack 卸载方法 938
3.58.6 插件 ktrack 升级方法 938
3.59 lo 938
3.59.1 插件 lo 简介 938
3.59.2 插件 lo 加载方式 938
3.59.3 插件 lo 的参数配置 939
3.59.4 插件 lo 的使用方法 939
3.59.5 插件 lo 卸载方法 939
3.59.6 插件 lo 升级方法 939
3.60 ltree 939
3.60.1 插件 ltree 简介: 939
3.60.2 插件 ltree 加载方式 940
3.60.3 插件 ltree 的参数配置 940
3.60.4 插件 ltree 的使用方法 940
3.60.4.1 ltree 类型 940
3.60.4.2 lquery 类型 941
3.60.4.3 ltree2text(ltree) 944

3.60.4.4 text2ltree(text) . . . 944
3.60.4.5 subpath(ltree,int offset) . . . 946
3.60.4.6 index(ltree a,ltree b) . . . 946
3.60.4.7 lca(ltree,ltree,⋯) . . . 949
3.60.4.8 nlevel(ltree) . . . 950
3.60.5 插件 ltree 卸载方法 . . . 951
3.60.6 插件 ltree 升级方法 . . . 951
3.61 memstat . . . 951
3.61.1 插件 memstat 简介 . . . 951
3.61.2 插件 memstat 加载方式 . . . 951
3.61.3 插件 memstat 的参数配置 . . . 952
3.61.4 插件 memstat 使用方法 . . . 952
3.61.4.1 local_memory_stats 函数 . . . 952
3.61.4.2 instance_memory_stats 函数 . . . 953
3.61.4.3 memory_stats 视图 . . . 953
3.61.5 插件 memstat 卸载方法 . . . 954
3.61.6 插件 memstat 升级方法 . . . 954
3.62 mysql_fdw . . . 954
3.62.1 插件 mysql_fdw 简介 . . . 954
3.62.2 插件 mysql_fdw 加载方式 . . . 955
3.62.3 插件 mysql_fdw 的参数配置 . . . 955
3.62.4 插件 mysql_fdw 的使用方法 . . . 955
3.62.4.1 连接 . . . 955
3.62.4.2 列 . . . 956
3.62.4.3 数据类型 . . . 956
3.62.4.4 CREATE SERVER . . . 956
3.62.4.5 CREATE USER MAPPING . . . 957
3.62.4.6 CREATE FOREIGN TABLE . . . 957
3.62.4.7 WHERE 条件和 ORDER BY 子句 . . . 958
3.62.4.8 外部表之间的 JOIN . . . 958
3.62.4.9 聚合函数的下推 . . . 958
3.62.4.10 修改外部数据 . . . 958
3.62.4.11 EXPLAIN . . . 959
3.62.4.12 支持 IMPORT FOREIGN SCHEMA . . . 959
3.62.5 插件 mysql_fdw 的卸载方法 . . . 959
3.62.6 插件 mysql_fdw 升级方法 . . . 960
3.63 mysql_json . . . 960
3.63.1 插件 mysql_json 简介 . . . 960
3.63.2 插件 mysql_json 加载方式 . . . 960
3.63.3 插件 mysql_json 的参数配置 . . . 960
3.63.4 插件 mysql_json 的使用方法 . . . 960
3.63.4.1 JSON 操作符 . . . 960

3.63.4.1.1 -> . . . 960
3.63.4.1.2 -» . . . 961
3.63.4.2 JSON 操作函数 . . . 962
3.63.4.2.1 json_array_append 函数 . . . 964
3.63.4.2.2 json_array_insert 函数 . . . 965
3.63.4.2.3 json_contains 函数 . . . 966
3.63.4.2.4 json_contains_path 函数 . . . 966
3.63.4.2.5 json_depth 函数 . . . 967
3.63.4.2.6 json_extract 函数 . . . 968
3.63.4.2.7 json_insert 函数 . . . 969
3.63.4.2.8 json_keys 函数 . . . 970
3.63.4.2.9 json_length 函数 . . . 971
3.63.4.2.10 json_object 函数 . . . 971
3.63.4.2.11 json_pretty 函数 . . . 972
3.63.4.2.12 json_merge 函数 . . . 973
3.63.4.2.13 json_merge_preserve 函数 . . . 974
3.63.4.2.14 json_quote 函数 . . . 974
3.63.4.2.15 json_remove 函数 . . . 975
3.63.4.2.16 json_replace 函数 . . . 976
3.63.4.2.17 json_search 函数 . . . 977
3.63.4.2.18 json_set 函数 . . . 977
3.63.4.2.19 json_type 函数 . . . 978
3.63.4.2.20 json_unquote 函数 . . . 979
3.63.4.2.21 json_valid 函数 . . . 980
3.63.5 插件 mysql_json 的卸载方法 . . . 981
3.63.6 插件 mysql_json 升级方法 . . . 981
3.64 oracle_fdw . . . 981
3.64.1 插件 oracle_fdw 简介 . . . 981
3.64.2 插件 oracle_fdw 的加载方式 . . . 981
3.64.3 插件 oracle_fdw 的参数配置 . . . 982
3.64.3.1 Foreign data wrapper 参数 . . . 982
3.64.3.2 Foreign server 参数 . . . 982
3.64.3.3 User mapping 参数 . . . 982
3.64.3.4 Foreign table 参数 . . . 983
3.64.3.5 列参数 . . . 984
3.64.4 插件 oracle_fdw 的使用方法 . . . 984
3.64.4.1 Oracle 权限 . . . 984
3.64.4.2 连接 . . . 984
3.64.4.3 列 . . . 984
3.64.4.4 数据类型 . . . 984
3.64.4.5 WHERE 条件和 ORDER BY 子句 . . . 986
3.64.4.6 外部表之间的 JOIN . . . 986

3.64.4.7 修改外部数据 987
3.64.4.8 EXPLAIN 987
3.64.4.9 ANALYZE 988
3.64.4.10 PostGIS 支持 988
3.64.4.11 支持 IMPORT FOREIGN SCHEMA 988
3.64.4.12 示例 989
3.64.5 插件 oracle_fdw 的卸载方法 993
3.64.6 插件 oracle_fdw 的升级方法 994
3.65 owa_util 994
3.65.1 插件 owa_util 简介 994
3.65.2 插件 owa_util 加载方式 994
3.65.3 插件 owa_util 的参数配置 994
3.65.4 插件 owa_util 使用方法 994
3.65.4.1 返回过程调用者的信息 994
3.65.4.2 示例 995
3.65.5 插件 owa_util 卸载方法 996
3.65.6 插件 owa_util 升级方法 996
3.66 pageinspect 996
3.66.1 插件 pageinspect 简介 996
3.66.2 插件 pageinspect 加载方式 996
3.66.3 插件 pageinspect 的参数配置 997
3.66.4 插件 pageinspect 的使用方法 997
3.66.4.1 通用函数 997
3.66.4.2 堆表函数 998
3.66.4.3 B 树索引函数 1001
3.66.4.4 位图索引函数 1002
3.66.5 插件 pageinspect 卸载方法 1004
3.66.6 插件 pageinspect 升级方法 1004
3.67 passwordcheck 1004
3.67.1 插件 passwordcheck 简介 1004
3.67.2 插件 passwordcheck 加载方式 1004
3.67.3 插件 passwordcheck 的参数配置 1005
3.67.4 插件 passwordcheck 的使用方法 1005
3.67.5 插件 passwordcheck 卸载方法 1009
3.67.6 插件 passwordcheck 升级方法 1009
3.68 passwordhistory 1009
3.68.1 插件 passwordhistory 简介 1009
3.68.2 插件 passwordhistory 加载方式 1010
3.68.3 插件 passwordhistory 的参数配置 1010
3.68.4 插件 passwordhistory 的使用方法 1011
3.68.5 插件 passwordhistory 卸载方法 1011
3.68.6 插件 passwordhistory 升级方法 1012

3.69 pg_pathman . . . 1012
3.69.1 插件 pg_pathman 简介 . . . 1012
3.69.1.1 背景信息 . . . 1012
3.69.1.2 pg_pathman 特性 . . . 1012
3.69.2 pg_pathman 插件的使用方法 . . . 1013
3.69.2.1 创建 pg_pathman 插件扩展 . . . 1013
3.69.2.2 查看已安装的扩展 . . . 1013
3.69.2.3 插件升级 . . . 1013
3.69.2.4 相关视图和表 . . . 1013
3.69.2.5 分区管理 . . . 1014
3.69.2.6 高级分区管理 . . . 1019
3.70 pldbgapi . . . 1021
3.70.1 插件 pldbgapi 简介 . . . 1021
3.70.2 插件 pldbgapi 加载方式 . . . 1021
3.70.3 插件 pldbgapi 的参数配置 . . . 1022
3.70.4 插件 pldbgapi 使用方法 . . . 1022
3.70.4.1 pldebugger 调试流程 . . . 1022
3.70.4.2 接口函数 . . . 1022
3.70.4.2.1 plpgsql_oid_debug 函数 . . . 1023
3.70.4.2.2 pldbg_attach_to_port 函数 . . . 1024
3.70.4.2.3 pldbg_create_listener 函数 . . . 1024
3.70.4.2.4 pldbg_set_global_breakpoint 函数 . . . 1025
3.70.4.2.5 pldbg_wait_for_target 函数 . . . 1025
3.70.4.2.6 pldbg_set_breakpoint 函数 . . . 1026
3.70.4.2.7 pldbg_get_breakpoints 函数 . . . 1026
3.70.4.2.8 pldbg_drop_breakpoint 函数 . . . 1027
3.70.4.2.9 pldbg_wait_for_breakpoint 函数 . . . 1027
3.70.4.2.10 pldbg_step_into 函数 . . . 1028
3.70.4.2.11 pldbg_step_over 函数 . . . 1028
3.70.4.2.12 pldbg_continue 函数 . . . 1029
3.70.4.2.13 pldbg_abort_target 函数 . . . 1030
3.70.4.2.14 pldbg_get_variables 函数 . . . 1030
3.70.4.2.15 pldbg_deposit_value 函数 . . . 1031
3.70.4.2.16 pldbg_get_stack 函数 . . . 1031
3.70.4.2.17 pldbg_select_frame 函数 . . . 1032
3.70.4.2.18 pldbg_get_source 函数 . . . 1033
3.70.4.2.19 plsql_extend_get_proc_ddl 函数 . . . 1033
3.70.4.2.20 pldbg_get_proxy_info 函数 . . . 1034
3.70.4.2.21 局部断点调试示例 . . . 1034
3.70.4.2.22 全局断点调试示例 . . . 1038
3.70.4.2.23 修改调试时数据及删除断点调试示例 . . . 1041
3.70.5 插件 pldbgapi 卸载方法 . . . 1047

3.70.6 插件 pldbgapi 升级方法 1047
3.71 plsql_pldbgapi 1047
3.71.1 插件 plsql_pldbgapi 简介 1047
3.71.2 插件 plsql_pldbgapi 加载方式 1047
3.71.3 插件 plsql_pldbgapi 的参数配置 1048
3.71.4 插件 plsql_pldbgapi 使用方法 1048
3.71.4.1 plsql_debugger 调试流程 1048
3.71.4.2 接口函数 1048
3.71.4.2.1 plsql_oid_debug 函数 1050
3.71.4.2.2 plsql_oid_debug_number 函数 1051
3.71.4.2.3 plsql_pldbg_attach_to_port 函数 1052
3.71.4.2.4 plsql_pldbg_create_listener 函数 1052
3.71.4.2.5 plsql_pldbg_set_global_breakpoint 函数 1052
3.71.4.2.6 plsql_pldbg_wait_for_target 函数 1054
3.71.4.2.7 plsql_pldbg_set_breakpoint 函数 1054
3.71.4.2.8 plsql_pldbg_get_breakpoints 函数 1055
3.71.4.2.9 plsql_pldbg_drop_breakpoint 函数 1055
3.71.4.2.10 plsql_pldbg_wait_for_breakpoint 函数 1056
3.71.4.2.11 plsql_pldbg_step_into 函数 1057
3.71.4.2.12 plsql_pldbg_step_over 函数 1057
3.71.4.2.13 plsql_pldbg_continue 函数 1058
3.71.4.2.14 plsql_pldbg_step_out 函数 1058
3.71.4.2.15 plsql_pldbg_step_end 函数 1059
3.71.4.2.16 plsql_pldbg_run_to_next_exception 函数 1059
3.71.4.2.17 plsql_pldbg_run_to_cursor_line 函数 1060
3.71.4.2.18 plsql_pldbg_alter_breakpoint_hit_number 函数 1061
3.71.4.2.19 plsql_pldbg_alter_breakpoint_msg 函数 1062
3.71.4.2.20 plsql_pldbg_alter_breakpoint_status 函数 1063
3.71.4.2.21 plsql_pldbg_alter_all_breakpoint_status 函数 1064
3.71.4.2.22 plsql_pldbg_set_breakpoint_src 函数 1065
3.71.4.2.23 plsql_pldbg_get_breakpoints_src 函数 1066
3.71.4.2.24 plsql_pldbg_set_exception_breakpoint_src 函数 1067
3.71.4.2.25 plsql_pldbg_get_exception_breakpoints_src 函数 1069
3.71.4.2.26 plsql_pldbg_drop_exception_breakpoint 函数 1070
3.71.4.2.27 plsql_pldbg_drop_all_breakpoint 函数 1071
3.71.4.2.28 plsql_pldbg_abort_target 函数 1071
3.71.4.2.29 plsql_pldbg_get_variables 函数 1072
3.71.4.2.30 plsql_pldbg_deposit_value 函数 1073
3.71.4.2.31 plsql_pldbg_get_stack 函数 1073
3.71.4.2.32 plsql_pldbg_select_frame 函数 1074
3.71.4.2.33 plsql_pldbg_get_source 函数 1074
3.71.4.2.34 plsql_extend_get_proc_ddl 函数 1075

3.71.4.2.35 plsql_pldbg_get_proxy_info 函数 1076
3.71.4.2.36 plsql_pldbg_set_connection_timeout 函数 1076
3.71.4.2.37 plsql_pldbg_set_operation_timeout 函数 1077
3.71.4.2.38 局部断点调试示例 1077
3.71.4.2.39 全局断点调试示例 1081
3.71.4.2.40 修改调试时数据及删除断点调试示例 1084
3.71.5 插件 plsql_pldbgapi 卸载方法 1089
3.71.6 插件 plsql_pldbgapi 升级方法 1090
3.72 plsql_plprofiler 1090
3.72.1 插件 plsql_plprofiler 简介 1090
3.72.2 插件 plsql_plprofiler 加载方式 1090
3.72.3 插件 plsql_plprofiler 的参数配置 1091
3.72.4 插件 plsql_plprofiler 使用方法 1091
3.72.4.1 plsql_plprofiler 使用说明 1091
3.72.4.2 接口函数 1093
3.72.4.2.1 pl_profiler_set_enabled_local 函数 1094
3.72.4.2.2 pl_profiler_set_enabled_pid 函数 1094
3.72.4.2.3 pl_profiler_get_enabled_local 函数 1095
3.72.4.2.4 pl_profiler_get_enabled_pid 函数 1095
3.72.4.2.5 pl_profiler_set_enabled_global 函数 1095
3.72.4.2.6 pl_profiler_get_enabled_global 函数 1096
3.72.4.2.7 pl_profiler_collect_data 函数 1096
3.72.4.2.8 pl_profiler_set_collect_interval 函数 1096
3.72.4.2.9 pl_profiler_get_collect_interval 函数 1097
3.72.4.2.10 pl_profiler_reset_local 函数 1097
3.72.4.2.11 pl_profiler_reset_shared 函数 1097
3.72.4.2.12 pl_profiler_linestats_local 函数 1098
3.72.4.2.13 pl_profiler_linestats_shared 函数 1098
3.72.4.2.14 pl_profiler_func_oids_local 函数 1099
3.72.4.2.15 pl_profiler_func_oids_shared 函数 1099
3.72.4.2.16 pl_profiler_callgraph_local 函数 1100
3.72.4.2.17 pl_profiler_callgraph_shared 函数 1101
3.72.4.2.18 pl_profiler_get_stack 函数 1101
3.72.4.2.19 pl_profiler_funcs_source 函数 1102
3.72.4.2.20 pl_profiler_version 函数 1103
3.72.4.2.21 pl_profiler_versionstr 函数 1103
3.72.4.2.22 pl_profiler_lines_overflow 函数 1103
3.72.4.3 典型用例 1104
3.72.5 插件 plsql_plprofiler 卸载方法 1109
3.72.6 插件 plsql_plprofiler 升级方法 1109
3.73 restricted_dba 1109
3.73.1 插件 restricted_dba 简介 1109

3.73.2 插件 restricted_dba 加载方法 1109
3.73.3 插件 restricted_dba 的参数配置 1110
3.73.3.1 示例 1110
3.73.4 插件 restricted_dba 使用方法 1111
3.73.5 插件 restricted_dba 卸载方法 1111
3.73.6 插件 restricted_dba 升级方法 1111
3.74 roaringbitmap 1111
3.74.1 插件 roaringbitmap 加载方式 1111
3.74.2 插件 roaringbitmap 的参数配置 1111
3.74.3 插件 roaringbitmap 外部接口 1111
3.74.3.1 数据类型 1111
3.74.3.2 函数 1112
3.74.3.3 类型转换 1114
3.74.3.4 操作符 1114
3.74.3.5 索引 1115
3.74.4 插件 roaringbitmap 卸载方法 1115
3.75 roledisable 1115
3.75.1 插件 roledisable 简介 1115
3.75.2 插件 roledisable 加载方式 1116
3.75.2.1 查看角色状态 1116
3.75.2.2 字段说明 1116
3.75.2.3 启用角色 1116
3.75.2.4 禁用角色 1116
3.75.2.5 被禁用角色的表现 1117
3.75.3 插件 roledisable 的参数配置 1117
3.75.4 插件 roledisable 的使用方法 1117
3.75.5 插件 roledisable 卸载方法 1117
3.75.6 插件 roledisable 升级方法 1118
3.76 rum 1118
3.76.1 插件 rum 简介 1118
3.76.2 插件 rum 加载方式 1118
3.76.3 插件 rum 的参数配置 1118
3.76.4 插件 rum 的使用方法 1118
3.76.5 插件 rum 卸载方法 1118
3.76.6 插件 rum 升级方法 1119
3.77 security_utils 1119
3.77.1 插件 security_utils 简介 1119
3.77.2 插件 security_utils 加载方式 1119
3.77.3 插件 security_utils 的参数配置 1119
3.77.4 插件 security_utils 使用方法 1120
3.77.4.1 口令脱敏 1120
3.77.4.2 弱口令扫描 1121

3.77.5 相关函数 1123
3.77.5.1 1.check_guc_in_cluster() 1123
3.77.5.2 2.set_guc_in_cluster() 1124
3.77.5.2.1 定时任务 1124
3.77.6 插件 security_utils 卸载方法 1124
3.77.7 插件 security_utils 升级方法 1124
3.78 seg 1125
3.78.1 插件 seg 简介 1125
3.78.2 插件 seg 加载方式 1125
3.78.3 插件 seg 的参数配置 1125
3.78.4 插件 seg 的使用方法 1125
3.78.5 插件 seg 卸载方法 1126
3.78.6 插件 seg 升级方法 1126
3.79 send_mails 1126
3.79.1 插件 send_mails 简介 1126
3.79.2 插件 send_mails 加载方式 1127
3.79.3 插件 send_mails 的参数配置 1127
3.79.4 插件 send_mails 使用方法 1127
3.79.5 插件 send_mails 卸载方法 1128
3.79.6 插件 send_mails 升级方法 1129
3.80 sepapower 1129
3.80.1 插件 sepapower 简介 1129
3.80.2 插件 sepapower 加载方式 1129
3.80.3 插件 sepapower 的参数配置 1129
3.80.4 插件 sepapower 使用方法 1130
3.80.5 插件 sepapower 卸载方法 1130
3.80.6 插件 sepapower 升级方法 1131
3.81 src_restrict 1131
3.81.1 插件 src_restrict 简介 1131
3.81.2 插件 src_restrict 加载方式 1131
3.81.3 插件 src_restrict 的参数配置 1131
3.81.4 插件 src_restrict 使用方法 1132
3.81.4.1 系统函数 src_restrict.add_rules 1132
3.81.4.2 系统函数 src_restrict.remove_rules 1133
3.81.4.3 系统视图 src_restrict.show_rules 1134
3.81.4.4 示例 1134
3.81.5 插件 src_restrict 卸载方法 1135
3.81.6 插件 src_restrict 升级方法 1135
3.82 sslinfo 1135
3.82.1 插件 sslinfo 简介 1135
3.82.2 插件 sslinfo 加载方式 1135
3.82.3 插件 sslinfo 的参数配置 1135

3.82.4 插件 sslinfo 的使用方法 1136
3.82.5 插件 sslinfo 卸载方法 1136
3.82.6 插件 sslinfo 升级方法 1136
3.83 sso_update_user 1136
3.83.1 插件 sso_update_user 简介 1136
3.83.2 插件 sso_update_user 加载方式 1136
3.83.3 插件 sso_update_user 的参数配置 1137
3.83.4 插件 sso_update_user 使用方法 1137
3.83.5 插件 sso_update_user 卸载方式 1137
3.83.6 插件 sso_update_user 升级方法 1137
3.84 sys_anon 1138
3.84.1 插件 sys_anon 简介 1138
3.84.2 插件 sys_anon 加载方式 1138
3.84.3 插件 sys_anon 的参数配置 1138
3.84.4 插件 sys_anon 使用方法 1139
3.84.5 插件 sys_anon 卸载方法 1139
3.84.6 插件 sys_anon 升级方法 1139
3.85 sys_audlog 1139
3.85.1 插件 sys_audlog 简介 1139
3.85.2 插件 sys_audlog 加载方式 1139
3.85.3 插件 sys_audlog 的参数配置 1140
3.85.4 插件 sys_audlog 的使用方法 1140
3.85.5 插件 sys_audlog 卸载方法 1140
3.85.6 插件 sys_audlog 升级方法 1140
3.86 sys_buffercache 1140
3.86.1 插件 sys_buffercache 简介 1140
3.86.2 插件 sys_buffercache 加载方式 1141
3.86.3 插件 sys_buffercache 的参数配置 1141
3.86.4 插件 sys_buffercache 的使用方法 1141
3.86.5 插件 sys_buffercache 卸载方法 1142
3.86.6 插件 sys_buffercache 升级方法 1142
3.87 sys_bulkload 1142
3.87.1 插件 sys_bulkload 简介 1142
3.87.2 插件 sys_bulkload 加载方式 1142
3.87.3 插件 sys_bulkload 的参数配置 1143
3.87.4 插件 sys_bulkload 的使用方法 1143
3.87.5 插件 sys_bulkload 卸载方法 1143
3.87.6 插件 sys_bulkload 升级方法 1143
3.88 sys_freespacemap 1143
3.88.1 插件 sys_freespacemap 简介 1143
3.88.2 插件 sys_freespacemap 加载方式 1143
3.88.3 插件 sys_freespacemap 的参数配置 1143

3.88.4 插件 sys_freespacemap 的使用方法 . . . 1144
3.88.5 插件 sys_freespacemap 卸载方法 . . . 1144
3.88.6 插件 sys_freespacemap 升级方法 . . . 1144
3.89 sys_jieba . . . 1144
3.89.1 插件 sys_jieba 简介 . . . 1144
3.89.2 插件 sys_jieba 加载方式 . . . 1145
3.89.3 插件 sys_jieba 的参数配置 . . . 1145
3.89.4 插件 sys_jieba 的使用方法 . . . 1145
3.89.5 插件 sys_jieba 的卸载方法 . . . 1145
3.89.6 插件 sys_jieba 升级方法 . . . 1146
3.90 sys_kwr . . . 1146
3.90.1 插件 sys_kwr 简介 . . . 1146
3.90.2 插件 sys_kwr 加载方式 . . . 1146
3.90.3 插件 sys_kwr 的参数配置 . . . 1146
3.90.4 插件 sys_kwr 的使用方法 . . . 1148
3.90.4.1 KWR 功能 . . . 1148
3.90.4.1.1 KWR 功能概述 . . . 1148
3.90.4.1.2 KWR 推荐参数配置 . . . 1149
3.90.4.1.3 KWR 快照 . . . 1149
3.90.4.1.3.1 自动快照 . . . 1149
3.90.4.1.3.2 手动快照 . . . 1150
3.90.4.1.3.3 快照管理 . . . 1150
3.90.4.1.4 KWR 报告 . . . 1151
3.90.4.1.4.1 生成报告 . . . 1151
3.90.4.1.4.2 常见问题 . . . 1152
3.90.4.2 KWR DIFF 功能 . . . 1153
3.90.4.2.1 KWR DIFF 功能概述 . . . 1153
3.90.4.2.2 KWR DIFF 推荐参数配置 . . . 1154
3.90.4.2.3 KWR DIFF 报告 . . . 1154
3.90.4.3 KSH 功能 . . . 1155
3.90.4.3.1 KSH 功能概述 . . . 1155
3.90.4.3.2 KSH 推荐参数配置 . . . 1155
3.90.4.3.3 KSH 采集数据说明 . . . 1155
3.90.4.3.4 KSH 内部视图和关系表 . . . 1156
3.90.4.3.5 KSH 报告 . . . 1157
3.90.4.4 KDDM 功能 . . . 1158
3.90.4.4.1 KDDM 功能概述 . . . 1158
3.90.4.4.2 KDDM 推荐参数配置 . . . 1158
3.90.4.4.3 KDDM 报告 . . . 1158
3.90.4.4.4 GUC 参数建议 . . . 1159
3.90.5 插件 sys_kwr 卸载方法 . . . 1160
3.90.6 插件 sys_kwr 升级方法 . . . 1160

3.91 sys_prewarm . . . 1161
3.91.1 插件 sys_prewarm 简介 . . . 1161
3.91.2 插件 sys_prewarm 加载方式 . . . 1161
3.91.3 插件 sys_prewarm 的参数配置 . . . 1161
3.91.4 插件 sys_prewarm 的使用方法 . . . 1162
3.91.5 插件 sys_prewarm 卸载方法 . . . 1162
3.91.6 插件 sys_prewarm 升级方法 . . . 1163
3.92 sys_qualstats . . . 1163
3.92.1 插件 sys_qualstats 简介 . . . 1163
3.92.2 插件 sys_qualstats 加载方式 . . . 1163
3.92.3 插件 sys_qualstats 的参数配置 . . . 1163
3.92.4 插件 sys_qualstats 的使用方法 . . . 1164
3.92.4.1 sys_qualstats . . . 1164
3.92.4.2 sys_qualstats_reset() . . . 1165
3.92.4.3 sys_qualstats_example_query . . . 1165
3.92.4.4 sys_qualstats_names . . . 1166
3.92.4.5 sys_qualstats_example_queries . . . 1166
3.92.4.6 sys_qualstats_pretty . . . 1167
3.92.4.7 sys_qualstats_all . . . 1167
3.92.4.8 sys_qualstats_indexes . . . 1167
3.92.4.9 sys_qualstats_by_query . . . 1168
3.92.4.10 使用索引建议 . . . 1168
3.92.4.10.1 参数配置 . . . 1168
3.92.4.10.2 创建插件 . . . 1168
3.92.4.10.3 示例 . . . 1169
3.92.4.10.3.1 准备测试数据 . . . 1169
3.92.4.10.3.2 查看执行计划 . . . 1169
3.92.4.10.3.3 查看索引建议 . . . 1169
3.92.4.10.4 根据索引建议创建索引 . . . 1170
3.92.4.10.5 查看创建完索引后的执行计划 . . . 1170
3.92.5 插件 sys_qualstats 卸载方法 . . . 1170
3.92.6 插件 sys_qualstats 升级方法 . . . 1170
3.93 sys_recovery . . . 1171
3.93.1 插件 sys_recovery 简介 . . . 1171
3.93.2 插件 sys_recovery 加载方式 . . . 1171
3.93.3 插件 sys_recovery 的参数说明配置 . . . 1171
3.93.4 插件 sys_recovery 的使用方法 . . . 1172
3.93.5 插件 sys_recovery 卸载方法 . . . 1172
3.93.6 插件 sys_recovery 升级方法 . . . 1173
3.93.7 其它说明 . . . 1173
3.94 sys_rman . . . 1173
3.94.1 插件 sys_rman 简介 . . . 1173

3.94.2 插件 sys_rman 加载方式 1173
3.94.3 插件 sys_rman 的参数配置 1173
3.94.4 插件 sys_rman 的使用方法 1173
3.94.5 插件 sys_rman 卸载方法 1174
3.94.6 插件 sys_rman 升级方法 1174
3.95 sys_spacequota 1174
3.95.1 插件 sys_spacequota 简介 1174
3.95.2 插件 sys_spacequota 加载方式 1174
3.95.3 插件 sys_spacequota 的参数配置 1174
3.95.4 插件 sys_spacequota 的使用方法 1175
3.95.4.1 开关及查询: 1175
3.95.4.2 remove_space_quota 1175
3.95.4.3 space_quota_check 1175
3.95.4.4 set_space_quota(oid, quota) 1176
3.95.5 插件 sys_spacequota 卸载方法 1176
3.95.6 插件 sys_spacequota 升级方法 1177
3.96 sys_sqltune 1177
3.96.1 插件 sys_sqltune 简介 1177
3.96.2 插件 sys_sqltune 加载方式 1177
3.96.3 插件 sys_sqltune 的参数配置 1177
3.96.4 插件 sys_sqltune 的使用方法 1179
3.96.4.1 概述 1179
3.96.4.2 SQL 监控概述 1179
3.96.4.3 SQL 监控功能 1179
3.96.4.4 SQL 监控函数接口 1179
3.96.4.5 生成监控报告 1180
3.96.4.6 SQL 监控视图 1181
3.96.5 插件 sys_sqltune 卸载方法 1185
3.96.6 插件 sys_sqltune 升级方法 1185
3.97 sys_squeeze 1186
3.97.1 插件 sys_squeeze 简介 1186
3.97.2 插件 sys_squeeze 加载方式 1186
3.97.3 插件 sys_squeeze 的参数配置 1186
3.97.4 插件 sys_squeeze 的使用方法 1187
3.97.5 插件 sys_squeeze 卸载方法 1187
3.97.6 插件 sys_squeeze 升级方法 1188
3.98 sys_stat_statements 1188
3.98.1 插件 sys_stat_statements 简介 1188
3.98.2 插件 sys_stat_statements 加载方式 1188
3.98.3 插件 sys_stat_statements 的参数配置 1188
3.98.4 插件 sys_stat_statements 的使用方法 1189
3.98.4.1 sys_stat_statements 的推荐参数配置 1189

3.98.4.2 sys_stat_statements 的视图 1189
3.98.4.3 sys_stat_statements 的函数 1194
3.98.5 插件 sys_stat_statements 卸载方法 1194
3.98.6 插件 sys_stat_statements 升级方法 1194
3.99 sys_trgm 1195
3.99.1 插件 sys_trgm 简介 1195
3.99.2 插件 sys_trgm 加载方式 1195
3.99.3 插件 sys_trgm 的参数配置 1195
3.99.4 插件 sys_trgm 的使用方法 1195
3.99.4.1 函数 1195
3.99.4.2 操作符 1197
3.99.4.3 GUC 参数 1198
3.99.4.4 索引支持 1198
3.99.4.5 文本搜索集成 1201
3.99.5 插件 sys_trgm 卸载方法 1202
3.99.6 插件 sys_trgm 升级方法 1202
3.100 sys_visibility 1202
3.100.1 插件 sys_visibility 简介 1202
3.100.2 插件 sys_visibility 加载方式 1202
3.100.3 插件 sys_visibility 的参数配置 1202
3.100.4 插件 sys_visibility 的使用方法 1202
3.100.5 插件 sys_visibility 卸载方法 1203
3.100.6 插件 sys_visibility 升级方法 1203
3.101 sysaudit 1204
3.101.1 插件 sysaudit 简介 1204
3.101.2 插件 sysaudit 加载方式 1204
3.101.3 插件 sysaudit 的参数配置 1204
3.101.4 插件 sysaudit 使用方法 1204
3.101.5 插件 sysaudit 卸载方法 1204
3.101.6 插件 sysaudit 升级方法 1204
3.102 sysencrypt 1205
3.102.1 插件 sysencrypt 简介 1205
3.102.2 插件 sysencrypt 加载方式 1205
3.102.3 插件 sysencrypt 参数配置 1205
3.102.4 插件 sysencrypt 使用方法 1206
3.102.5 插件 sysencrypt 卸载方法 1206
3.102.6 插件 sysencrypt 升级方法 1206
3.103 sysmac 1206
3.103.1 插件 sysmac 简介 1206
3.103.2 插件 sysmac 加载方式 1206
3.103.3 插件 sysmac 的参数配置 1207
3.103.4 插件 sysmac 使用方法 1207

3.103.5 插件 sysmac 卸载方法 1207
3.103.6 插件 sysmac 升级方法 1207
3.104 sysprivilege 1208
3.104.1 插件 sysprivilege 简介 1208
3.104.2 插件 sysprivilege 加载方式 1208
3.104.3 插件 sysprivilege 的参数配置 1208
3.104.4 插件 sysprivilege 使用方法 1208
3.104.4.1 相关表和视图 1209
3.104.4.2 示例 1210
3.104.5 插件 sysprivilege 卸载方法 1210
3.104.6 插件 sysprivilege 升级方法 1211
3.105 sysreuse_residual_data 1211
3.105.1 插件 sysreuse_residual_data 简介 1211
3.105.2 插件 sysreuse_residual_data 加载方式 1211
3.105.3 插件 sysreuse_residual_data 的参数配置 1211
3.105.4 插件 sysreuse_residual_data 的使用方法 1212
3.105.5 插件 sysreuse_residual_data 卸载方法 1212
3.105.6 插件 sysreuse_residual_data 升级方法 1212
3.106 tablefunc 1212
3.106.1 插件 tablefunc 简介 1212
3.106.2 插件 tablefunc 加载方式 1213
3.106.3 插件 tablefunc 的参数配置 1213
3.106.4 插件 tablefunc 的使用方法 1213
3.106.5 插件 tablefunc 卸载方法 1214
3.106.6 插件 tablefunc 升级方法 1214
3.107 tcn 1214
3.107.1 插件 tcn 简介 1214
3.107.2 插件 tcn 加载方式 1214
3.107.3 插件 tcn 的参数配置 1214
3.107.4 插件 tcn 的使用方法 1215
3.107.5 插件 tcn 卸载方法 1215
3.107.6 插件 tcn 升级方法 1216
3.108 tds_fdw 1216
3.108.1 插件 tds_fdw 简介 1216
3.108.2 插件 tds_fdw 加载方式 1216
3.108.3 插件 tds_fdw 的参数配置 1216
3.108.3.1 外部服务器参数 1216
3.108.3.2 外部表参数 1218
3.108.3.3 外部数据包装器参数 1219
3.108.3.4 导入外部模式参数 1219
3.108.3.5 变量 1220
3.108.4 插件 tds_fdw 使用方法 1220

3.108.4.1 创建外部服务器 1220
3.108.4.2 创建外部表 1220
3.108.4.3 创建外部数据包装器 1221
3.108.4.4 导入外部模式 1221
3.108.4.5 变量 1222
3.108.4.6 EXPLAIN 1222
3.108.4.7 关于字符集和编译的说明 1222
3.108.4.8 加密连接 MSSQL 1222
3.108.5 插件 tds_fdw 卸载方法 1222
3.108.6 插件 tds_fdw 升级方法 1222
3.109 tsm_system_rows 1223
3.109.1 插件 tsm_system_rows 简介 1223
3.109.2 插件 tsm_system_rows 加载方式 1223
3.109.3 插件 tsm_system_rows 的参数配置 1223
3.109.4 插件 tsm_system_rows 的使用方法 1223
3.109.5 插件 tsm_system_rows 卸载方法 1223
3.109.6 插件 tsm_system_rows 升级方法 1224
3.110 tsm_system_time 1224
3.110.1 插件 tsm_system_time 简介 1224
3.110.2 插件 tsm_system_time 加载方式 1224
3.110.3 插件 tsm_system_time 的参数配置 1224
3.110.4 插件 tsm_system_time 的使用方法 1224
3.110.5 插件 tsm_system_time 卸载方法 1225
3.110.6 插件 tsm_system_time 升级方法 1225
3.111 unaccent 1225
3.111.1 插件 unaccent 简介 1225
3.111.2 插件 unaccent 加载方式 1225
3.111.3 插件 unaccent 的参数配置 1225
3.111.4 插件 unaccent 的使用方法 1226
3.111.5 插件 unaccent 卸载方法 1227
3.111.6 插件 unaccent 升级方法 1227
3.112 utl_file 1228
3.112.1 插件 utl_file 简介 1228
3.112.2 插件 utl_file 加载方式 1228
3.112.3 插件 utl_file 的参数配置 1228
3.112.4 插件 utl_file 的使用方法 1228
3.112.4.1 utl_file.fopen 1228
3.112.4.2 utl_file.put_line 1229
3.112.4.3 utl_file.put 1231
3.112.4.4 utl_file.fclose 1231
3.112.4.5 utl_file.fclose_all 1232
3.112.4.6 utl_file.get_line 1233

3.112.4.7 utl_file.new_line . . . 1234
3.112.4.8 utl_file.fflush . . . 1235
3.112.5 插件 utl_file 卸载方法 . . . 1236
3.112.6 插件 utl_file 升级方法 . . . 1236
3.113 uuid-ossp . . . 1236
3.113.1 插件 uuid-ossp 简介 . . . 1236
3.113.2 插件 uuid-ossp 加载方式 . . . 1237
3.113.3 插件 uuid-ossp 的参数配置 . . . 1237
3.113.4 插件 uuid-ossp 的使用方法 . . . 1237
3.113.5 插件 uuid-ossp 卸载方法 . . . 1237
3.113.6 插件 uuid-ossp 升级方法 . . . 1237
3.114 walminer . . . 1238
3.114.1 插件 walminer 简介 . . . 1238
3.114.1.1 什么是 WalMiner . . . 1238
3.114.1.2 WalMiner 的适用场景 . . . 1238
3.114.1.3 WalMiner 的限制与约束 . . . 1238
3.114.2 WalMiner 的运行环境准备 . . . 1238
3.114.2.1 开启 full_page_writes . . . 1238
3.114.2.2 WAL 日志连续性检查 . . . 1239
3.114.2.3 安装 . . . 1239
3.114.2.4 数据字典检查 . . . 1239
3.114.3 WalMiner 接口使用介绍 . . . 1239
3.114.3.1 添加日志 . . . 1239
3.114.3.2 展示日志列表 . . . 1240
3.114.3.3 删除日志 . . . 1240
3.114.3.4 导入 history 文件 . . . 1240
3.114.3.5 导出数据字典 . . . 1242
3.114.3.6 导入数据字典 . . . 1242
3.114.3.7 数据解析 . . . 1242
3.114.3.7.1 普通解析 . . . 1242
3.114.3.7.2 精准解析 . . . 1243
3.114.3.7.3 单表解析 . . . 1243
3.114.3.8 查看解析结果 . . . 1244
3.114.3.9 结束解析 . . . 1244
3.114.4 WalMiner 使用示例 . . . 1245
3.114.4.1 解析本库 WAL 日志 . . . 1245
3.114.4.1.1 创建扩展 . . . 1245
3.114.4.1.2 执行普通解析 . . . 1245
3.114.4.1.3 时间范围解析 . . . 1245
3.114.4.1.4 LSN 范围解析 . . . 1245
3.114.4.1.5 单表解析 . . . 1246
3.114.4.1.6 查看分析结果 . . . 1246

3.114.4.1.7 结束分析 1247
3.114.4.2 解析其他数据库的 WAL 日志 1247
3.114.4.2.1 从其他数据库导出数据字典 1247
3.114.4.2.2 导入其他数据库的数据字典 1247
3.114.4.2.3 添加日志文件 1248
3.114.4.2.4 日志解析 1248
3.114.4.2.5 查看分析结果 1248
3.114.4.2.6 结束分析 1249
3.114.5 WalMiner 常见问题处理 1249
3.114.5.1 执行 walminer 解析报错 oid 不存在 1249
3.115 xml2 1250
3.115.1 插件 xml2 简介 1250
3.115.2 插件 xml2 加载方式 1250
3.115.3 插件 xml2 的参数配置 1250
3.115.4 插件 xml2 的使用方法 1250
3.115.4.1 xpath_table 1251
3.115.4.1.1 多值结果 1253
3.115.4.2 XSLT 函数 1254
3.115.5 插件 xml2 卸载方法 1254
3.115.6 插件 xml2 升级方法 1254
3.116 zhparser 1255
3.116.1 插件 zhparser 简介 1255
3.116.2 插件 zhparser 加载方式 1255
3.116.3 插件 zhparser 的参数配置 1255
3.116.4 插件 zhparser 的使用方法 1256
3.116.5 插件 zhparser 的卸载方法 1258
3.116.6 插件 zhparser 升级方法 1258

版权声明 1259

服务周期承诺 1260

第1章　系统视图

KingbaseES 中的系统视图由 KingbaseES 系统视图和兼容视图组成：

- KingbaseES 系统视图
- PostgreSQL 兼容视图
- Oracle 兼容视图
- MySQL 兼容视图
- SQLServer 兼容视图

其中，xx 兼容视图仅在该兼容形态下可见，如 Oracle 兼容视图 all_tab_cols 仅能在 Oracle 兼容形态下的数据库中查询到，别的兼容形态数据库不存在该视图。

下表为各兼容形态下包含的系统视图。

兼容形态	视图分类	描述
PG 兼容形态	KingbaseES 系统视图	记录对象元信息及部分性能统计信息
	PostgreSQL 兼容视图	记录对象元信息，包含元信息视图和信息模式视图
Oracle 兼容形态	KingbaseES 系统视图	记录对象元信息及部分性能统计信息
	Oracle 兼容视图	记录对象元信息，部分兼容 Oracle 静态数据字典视图和动态性能视图
MySQL 兼容形态	KingbaseES 系统视图	记录对象元信息及部分性能统计信息
	MySQL 兼容视图	部分兼容 MySQL 信息模式视图
SQLServer 兼容形态	KingbaseES 系统视图	记录对象元信息及部分性能统计信息
	SQLServer 兼容视图	记录对象元信息，部分兼容 SQLServer 元信息视图和信息模式视图

1.1　KingbaseES 系统视图

KingbaseES 提供了大量的系统视图供用户使用，不同种类的视图可以满足用户不同场景的查询需求。

1.1.1　插件相关视图

本小节介绍与插件相关的系统视图内容。

1.1.1.1　JOB 视图

1.1.1.1.1　kdb_action

视图 `kdb_action` 记录任务的动作信息，查询使用 `kdb_schedule.kdb_action`。

表 1.1.1: kdb_action 列

名称	类型	描述
acid	integer	action 的 ID，引用 job.jobid
acname	text	action 的名字
acdesc	text	action 的描述
acenabled	boolean	action 状态 false - 不执行 true - 执行
ackind	char	action 的类型 s - sql b - batch
accode	text	Job 的 action
acconnstr	text	连接串
acdbname	name	目标数据库
acnextrun	timestamp with time zone	下次运行的时间

1.1.1.1.2　kdb_job

视图 `kdb_job` 存储用户创建的 job 信息, 该表作为系统共享表, 存储了所有 dba 用户创建的 job 信息, 所有用户都可以读取该表, 但只有 dba 用户就进行写操作。

关于 job 的创建、更新、删除见使用自动作业功能 。

表 1.1.2: kdb_job 列

名称	类型	描述
jobid	integer	行标识符
jobjclid	integer	kdb_jobclass OID
jobsjid	integer	kdb_schedule_job OID
jobjaid	integer	kdb_job_action OID
jobname	text	job 名称
jobdesc	text	job 描述信息
jobagentid	integer	后端进程 ID
jobenabled	boolean	job 开启标志
jobcreated	timestamp with time zone	job 创建时间
jobchanged	timestamp with time zone	job 修改时间
jobnextrun	timestamp with time zone	job 下一次执行时间
joblastrun	timestamp with time zone	job 上一次执行时间
jobrepeattimes	integer	job 失败最大重试次数
jobdbname	name	job 所属的 database

1.1.1.1.3　kdb_schedule

视图 `kdb_schedule` 记录任务的时间计划信息，查询使用 `kdb_schedule.kdb_schedule`。

表 1.1.3: kdb_schedule 列

名称	类型	描述
scid	integer	Schedule 的 ID
scname	text	Schedule 的名字
scdesc	text	Schedule 的描述
scenabled	boolean	Schedule 的状态 false - 不执行 true - 执行
scstart	timestamp without time zone	Schedule 的开始时间
scend	timestamp without time zone	Schedule 的结束时间
screpeat_interval	text	计划的时间

1.1.1.2　闪回相关视图

1.1.1.2.1　sys_recyclebin

`sys_recyclebin` 视图在开启回收站功能，存储用户删除的表信息，用于删除表闪回使用。

关于闪回功能见插件kdb_flashback 。

表 1.1.4: sys_recyclebin

名称	类型	描述
classid	oid	对象所在系统表的 OID
reloid	oid	对象 OID
nspname	name	schema OID
object_name	name	在回收站中对象名
original_name	name	对象删除之前的名称
type	char	类型
droptime	timestamp with time zone	删除时间

1.1.1.3　对象保护相关视图

1.1.1.3.1　sys_protect

`sys_protect` 视图存储被保护的对象信息，更多保护对象功能信息参考插件kdb_protect 。

表 1.1.5: sys_protect 列

名称	类型	描述
objid	oid	被保护的对象 oid
nspid	oid	被保护的对象命名空间的 oid
dbid	oid	被保护的对象所在数据库的 oid
objkind	char	被保护的对象类型: 'd' – database(数据库) 's' – schema(模式) 't' – table(表/分区表) 'v' – view/materialized view(视图/物化视图) 'f' – function/procedure(函数/存储过程)

1.1.1.4　资源组视图

1.1.1.4.1　sys_resauthid

`sys_resauthid` 视图可查询数据库用户和资源组的关联关系。

表 1.1.6: sys_resauthid 列

名称	类型	描述
roloid	oid	用户 oid
rolname	name	用户名
resgoid	oid	关联的资源组 oid

1.1.1.4.2　sys_resgroup

`sys_resgroup` 视图存储资源组信息。

表 1.1.7: sys_resgroup 列

名称	类型	描述
oid	oid	资源组 oid
rsgname	name	资源组名称
parent	text	父资源组 oid（未启用）
rstype	text	资源组类型（未启用）

1.1.1.4.3　sys_resgroupcapability

`sys_resgroupcapability` 视图存储资源组关联的相关值限制信息。

表 1.1.8: sys_resgroup 列

名称	类型	描述
oid	oid	资源组 oid
reslimittype	text	资源限制类型： 3 代表 cpu_cfs_quota_us ，含义是绝对 CPU 使用率 4 代表 cpu_shares ，含义是相对 CPU 使用率
value	text[]	资源限制类型对应的限制值

1.1.1.4.4　sys_resource_groups

视图 `sys_resource_groups` 提供了资源组的信息。更多资源限额管理功能内容参考插件kdb_resource_group 。

表 1.1.9: sys_resource_groups 视图

名称	类型	描述
resgroup_id	oid	资源组 oid
resgroup_name	name	资源组名称
resgroup_createtime	timestamp with time zone	资源组创建时间
resgroup_users	oid[]	加入资源组的用户 oid
resgroup_databases	oid[]	加入资源组的数据库 oid
resgroup_options	text[]	资源组的资源限额设置，子项有：cpu_list、mem_percent、mem_limit、io_limit、cpu_cfs_quota_us、cpu_shares

1.1.1.5　历史密码管理视图

1.1.1.5.1　sys_pwdht

视图 `sys_pwdht` 提供口令历史管理的用户信息。更多口令历史管理功能参见插件passwordhistory 。

表 1.1.10: sys_pwdht 列

名称	类型	描述
userid	oid	用户 oid
timestamp	timestamp with time zone	用户密码创建时间
password	text	用户密码

1.1.1.5.2　sys_pwdht_shadow

`sys_pwdht_shadow` 视图存储口令历史管理的用户信息，更多口令历史管理功能参见插件passwordhistory 。

表 1.1.11: sys_pwdht_shadow 列

名称	类型	描述
userid	oid	用户 oid
timestamp	timestamp with time zone	用户密码创建时间
password	text	用户密码

1.1.1.6　角色状态视图

1.1.1.6.1　sys_role_disable

表 `sys_role_disable` 可查询角色 role 的禁用信息。角色的禁用更多信息参考插件roledisable 。

表 1.1.12: sys_role_disable 列

名称	类型	描述
oid	oid	角色 oid
disabletime	timestamp with time zone	角色被禁用的时间

1.1.1.7　来源限制视图

1.1.1.7.1　src_restrict.show_rules

视图 `show_rules` 提供查看当前的用户来源限制规则信息，包括登录数据库的用户及 IP 的黑白名单。在加载来源限制功能的插件src_restrict 后，查询使用 `src_restrict.show_rules` 。

表 1.1.13: show_rules 列

名称	类型	描述
filter_type	text	规则的类型，allow 代表白名单，reject 代表黑名单
rule_user	text[]	规则的用户名

-续表-

名称	类型	描述
rule_ip	text	规则的 IP
netmask	text	规则 IP 对应的子网掩码
login_limited_info	text[]	登录限制信息（包括 IP 段，mac 地址，日期时间段）

1.1.1.8　安全策略视图

1.1.1.8.1　anon.all_policy

视图 `all_policy` 记录数据安全策略设置信息。加载数据安全插件sys_anon 后，视图存在 anon 模式下，查询使用 `anon.all_policy` 。

表 1.1.14: all_policy 列

名称	类型	描述
policy_name	text	数据安全策略名
table_name	text	安全对象名
schema_name	text	安全对象模式名
column_name	text	安全列名
masking_user	text	被安全用户名
masking_func	text	安全函数
func_parameters	text	安全函数参数列表

1.1.1.9　加密视图

1.1.2　元信息视图

元信息视图供用户查询对象元信息及数据库状态等内容。其中，大部分系统视图都是单个数据库相关的，仅少数系统视图在同一个集簇的所有数据库间中共享。

表 1.1.15: 元信息视图

视图名称	用途
recyclebin	被删除对象信息
sys_aggregate	聚集函数
sys_am	关系访问方法
sys_amop	访问方法操作符
sys_amproc	访问方法支持函数
sys_anon_policy	安全策略
sys_attrdef	列默认值元信息
sys_attribute	表列属性元信息
sys_available_extension_versions	所有版本的扩展
sys_available_extensions	可用的扩展
sys_cast	转换（数据类型转换）
sys_class	表、索引、序列、视图等关系类对象元信息
sys_collation	排序规则（locale 信息）
sys_config	编译时配置参数
sys_constraint	检查约束、唯一约束、主键约束、外键约束元信息
sys_context	上下文信息
sys_conversion	编码转换信息
sys_cursors	打开的游标元信息
sys_database_link	dblink 外部链接
sys_database	本数据库集簇中的数据库

-续表-

视图名称	用途
sys_db_role_setting	角色和数据库的设置
sys_default_acl	对象类型的默认权限
sys_depend	数据库对象间的依赖
sys_depends	对象的依赖关系
sys_description	数据库对象上的描述或注释
sys_directory	目录对象元信息
sys_enum	枚举类型元信息
sys_event_trigger	事件触发器
sys_extension	已安装扩展元信息
sys_file_settings	配置文件内容
sys_foreign_data_wrapper	外部数据包装器元信息
sys_foreign_server	外部服务器元信息
sys_foreign_table	外部表元信息
sys_grant_privileges	授予权限的信息
sys_index	索引信息
sys_indexes	索引
sys_inherits	表继承层次
sys_init_privs	对象初始特权
sys_language	编写函数的语言
sys_largeobject_metadata	大对象的元数据
sys_largeobject	大对象的数据页
sys_locks	当前保持或者等待的锁
sys_matviews	物化视图
sys_mview	存储物化视图和物化视图日志的相关信息

-续表-

视图名称	用途
sys_namespace	模式
sys_object_status	对象状态
sys_opclass	访问方法操作符类
sys_operator	操作符
sys_opfamily	访问方法操作符族
sys_package	包对象的信息、包中子对象的信息和依赖关系
sys_partitioned_table	分区表中分区元信息
sys_pkgitem	包中对象的合法性检查
sys_pltemplate	过程语言的模板数据
sys_policies	策略
sys_policy	行安全策略
sys_prepared_statements	预备好的语句
sys_prepared_xacts	预备好的事务
sys_privilege	用户的 sysbackup 权限
sys_proc	函数和过程
sys_protect	被保护的对象信息
sys_publication_rel	发布映射的关系
sys_publication_tables	publication 和它们相关的表
sys_publication	用于逻辑复制的发布
sys_query_mapping	Query Mapping 规则的 SQL 匹配/映射信息
sys_range	范围类型的信息
sys_recyclebin	回收站信息
sys_replication_origin_status	有关复制源的信息，包括复制进度
sys_replication_origin	已注册的复制源

-续表-

视图名称	用途
sys_replication_slots	复制槽信息
sys_rewrite	查询重写规则
sys_rules	规则
sys_seclabel	数据库对象上的安全标签
sys_seclabels	安全标签
sys_sequence	有关序列的信息
sys_sequences	序列
sys_session	系统会话信息
sys_settings	参数设置
sys_shdepend	共享对象上的依赖
sys_shdescription	共享对象上的注释
sys_shseclabel	共享数据库对象上的安全标签
sys_subpartition_table	子分区表
sys_subscription_rel	订阅的关系状态
sys_subscription	逻辑复制订阅
sys_synonym	同义词信息
sys_sysprivilege	系统 ANY 权限的用户信息
sys_tables	表元信息
sys_tablespace	本数据库集簇内的表空间
sys_timezone_abbrevs	时区简写
sys_timezone_names	时区名字
sys_transform	转换（将数据类型转换为过程语言需要的形式）
sys_trigger	触发器元信息
sys_triggers	触发器元信息

-续表-

视图名称	用途
sys_ts_config_map	文本搜索配置的记号映射
sys_ts_config	文本搜索配置
sys_ts_dict	文本搜索字典
sys_ts_parser	文本搜索分析器
sys_ts_template	文本搜索模板
sys_type	数据类型
sys_user_mapping	存储从本地用户到远程的映射
sys_user_mappings	用户映射
sys_views	视图元信息
sysmac.sysmac_column_label	列级强制访问控制的标记信息
sysmac.sysmac_obj	对象级强制访问控制的标记信息
user_any_privs	用户 ANY 权限

1.1.2.1　recyclebin

视图 `recyclebin` 提供被删除对象的信息。

表 1.1.16: recyclebin 列

名称	类型	描述
oid	oid	被删除对象 OID
original_name	name	对象删除之前的名称
droptime	timestamp wiht time zone	对象删除时间
type	text	对象类型

1.1.2.2　sys_aggregate

`sys_aggregate` 视图存储关于聚集函数的信息。聚集函数是对一个数值集合（典型的是每个匹配查询条件的行中的同一个列的值）进行操作的函数，它返回从这些值中计算出的一个数值。

典型的聚集函数是 `sum`、`count` 和 `max`。`sys_aggregate` 里的每行值都是 `sys_proc` 视图中一行的扩展。`sys_proc` 视图元组记载该聚集的名字、输入和输出数据类型，以及其他一些和普通函数类似的信息。

表 1.1.17: sys_aggregate 列

名称	类型	描述
aggfnoid	regproc	聚集函数在 sys_proc 中的 OID
aggkind	char	聚集类型： n - “普通”聚集 o - “有序集”聚集 h - “假想集”聚集
aggnumdirectargs	smallint	一个有序集或者假想集聚集的直接（非聚集）参数的数量，一个可变数组算作一个参数 如果等于 pronargs，该聚集必定是可变的并且该可变数组描述聚集参数以及最终直接参数 对于普通聚集总是为零
aggtransfn	regproc	转移函数
aggfinalfn	regproc	最终函数（如果没有就为零）
aggcombinefn	regproc	结合函数（如果没有就为零）
aggserialfn	regproc	序列化函数（如果没有就为零）
aggdeserialfn	regproc	反序列化函数（如果没有就为零）
aggmtransfn	regproc	用于移动聚集模式的向前转移函数（如果没有就为零）
aggminvtransfn	regproc	用于移动聚集模式的反向转移函数（如果没有就为零）
aggmfinalfn	regproc	用于移动聚集模式的最终函数（如果没有就为零）
aggfinalextra	boolean	为真则向 aggfinalfn 传递额外的哑参数
aggmfinalextra	boolean	为真则向 aggmfinalfn 传递额外的哑参数

-续表-

名称	类型	描述
aggfinalmodify	char	aggfinalfn 是否修改传递状态值： r - 只读 s - 不能在 aggfinalfn 之后应用 aggtransfn w - 可修改
aggmfinalmodify	char	和 aggfinalmodify 类似，但是用于 aggmfinalfn
aggsortop	oid	相关联的排序操作符（如果没有则为 0）
aggtranstype	oid	聚集函数的内部转移（状态）数据的数据类型
aggtransspace	interger	转移状态数据的近似平均尺寸（字节），或者为零表示使用一个默认估算值
aggmtranstype	oid	聚集函数用于移动聚集欧氏的内部转移（状态）数据的数据类型（如果没有则为零）
aggmtransspace	interger	转移状态数据的近似平均尺寸（字节），或者为零表示使用一个默认估算值
agginitval	text	转移状态的初始值 若这个列为空，则转移状态值从空值开始
aggminitval	text	用于移动聚集模式的转移状态的初值 若这个列为空，则转移状态值从空值开始

1.1.2.3　sys_am

`sys_am` 视图存储关于关系访问方法的信息。系统支持的每种访问方法在这个视图中都对应一行。当前，只有表和索引拥有访问方法。

表 1.1.18: sys_am 列

名称	类型	描述
oid	oid	行标识符
amname	name	访问方法的名字
amhandler	regproc	负责提供有关该访问方法信息的处理器函数的 OID

-续表-

名称	类型	描述
amtype	char	t = table (包含物化视图) i = index

备注:

在 KingbaseES V8R2 之前，`sys_am` 包含很多额外的列以表示索引访问方法的性质。那些数据现在只有在 C 代码级别才是直接可见的。不过，系统中增加了 `sys_index_column_has_property()` 和一些相关函数来允许 SQL 查询检查索引访问方法的性质。

1.1.2.4　sys_amop

视图 `sys_amop` 存储关于与访问方法操作符族相关的操作符信息。对于一个操作符族中的每一个成员即操作符都在这个视图中对应一行。一个成员可以是一个搜索操作符或者一个排序操作符。一个操作符可以出现在多个族中，但在同一个组中既不能出现在多个搜索位置也不能出现在多个排序位置（虽然不太可能出现，但是允许一个操作符同时用于搜索和排序目的）。

表 1.1.19: sys_amop 列

名称	类型	描述
oid	oid	行标识符
amopfamily	oid	这个项所在的操作符族
amoplefttype	oid	操作符的左操作数数据类型
amoprighttype	oid	操作符的右操作数数据类型
amopstrategy	smallint	操作符策略号
amoppurpose	char	操作符目的 s - 表示搜索 o - 排序
amopopr	oid	操作符的 OID
amopmethod	oid	使用此操作符族的索引访问方法

-续表-

名称	类型	描述
amopsortfamily	oid	若为排序操作符，使用该操作符族的列值根据该 B 树操作符族排序 若为搜索操作符，则为 0

一个“搜索”操作符项意味着该操作符族的一个索引可以被搜索来查找所有满足如下条件的行：WHERE *indexed_column operator constant*。显然，这样的一个操作符必须返回 boolean，且它的左操作数输入类型必须匹配索引列的数据类型。

一个“排序”操作符项意味着该操作符族的一个索引可以被扫描来返回以如下顺序排列的行：ORDER BY *indexed_column operator constant*。这样一个操作符能够返回任何可排序数据类型，尽管它的左操作数输入类型必须匹配索引列的数据类型。ORDER BY 的准确语义由 amopsortfamily 列指定，它必须引用一个适合于操作符结果类型的 B 树操作符族。

备注:

目前，一个排序操作符的排序顺序被假设为其引用的操作符族的默认值，即 ASC NULLS LAST。未来可能会通过增加额外的列来显式地指定排序选项。

一个项的 amopmethod 必须和它所包含的操作符族的 opfmethod 相匹配。此外，amoplefttype 和 amoprighttype 也必须匹配被引用的 sys_operator 项的 oprleft 和 oprright 值。

1.1.2.5 sys_amproc

sys_amproc 视图存储关于访问方法操作符族相关的支持函数。属于一个操作符族的每一个支持函数在这个视图中都对应一行。

表 1.1.20: sys_amproc 列

名称	类型	描述
oid	oid	行标识符
amprocfamily	oid	使用这个项的操作符族
amproclefttype	oid	相关操作符的左操作数数据类型
amprocrighttype	oid	相关操作符的右操作数数据类型
amprocnum	smallint	支持过程编号

-续表-

名称	类型	描述
amproc	regproc	过程的 OID

`amproclefttype` 和 `amprocrighttype` 列的通常解释是它们标识了一个特定支持过程所支持的操作符的左右输入类型。对于某些访问方法它们和支持过程本身的输入数据类型相匹配，而对其他的则不会匹配。对于一个索引有一个“默认”支持过程的概念，这些支持过程的 `amproclefttype` 和 `amprocrighttype` 都等于索引操作符类的 `opcintype`。

1.1.2.6　sys_anon_policy

`sys_anon_policy` 视图存储安全策略相关信息。

表 1.1.21: sys_anon_policy 列

名称	类型	描述
oid	oid	安全策略 oid
policy_name	name	安全策略名
relid	oid	安全策略应用的表 oid
column_num	smallint	安全策略应用的列所在的表的列号
user_id	oid	安全用户 oid
function_name	smallint	安全函数名
para_list_1	integer	安全函数参数 1（仅用于自定义安全）
para_list_2	integer	安全函数参数 2（仅用于自定义安全）

1.1.2.7　sys_attrdef

`sys_attrdef` 视图存储列的默认值，列的主要信息存储在sys_attribute 视图中。只有那些显式设置默认值的列才会在这个视图中有一个项。

表 1.1.22: sys_attrdef 列

名称	类型	描述
oid	oid	行标识符
adrelid	oid	该列所属的表
adnum	smallint	列号
adbin	pg_node_tree	默认值表达式

1.1.2.8　sys_attribute

`sys_attribute` 视图存储有关表列的信息。数据库中的每一个表的每一个列都恰好在该视图中对应一行。

表 1.1.23: sys_attribute 列

名称	类型	描述
attrelid	oid	列所属的表
attname	name	列名
atttypid	oid	列的数据类型
attstattarget	integer	attstattarget 控制由 ANALYZE 对此列收集的统计信息的细节层次 0 值表示不会收集任何统计信息 一个负值则说明直接使用系统默认的目标 正值的确切含义取决于数据类型 对于标量数据类型，attstattarget 既是要收集的“最常见值”的目标号，也是要创建的柱状图容器的目标号
attlen	smallint	本列类型的 sys_type.typlen 一个拷贝
attnum	smallint	列的编号，一般列从 1 开始向上编号 系统列（如 ctid ）则拥有（任意）负值编号
attndims	integer	如果该列是一个数组类型，这里就是其维度数；否则为 0 （在目前一个数组的维度数并不被强制，因此任何非零值都能有效地表明“这是一个数组”。）

-续表-

名称	类型	描述
attcacheoff	integer	在存储中总是为-1，但是当被载入到一个内存中的行描述符后，这里可能会被更新为属性在行内的偏移
atttypmod	integer	atttypmod 记录了在表创建时提供的类型相关数据（例如一个 varchar 列的最大长度） 它会被传递给类型相关的输入函数和长度强制函数。对于那些不需要 atttypmod 的类型，这个值通常总是为-1
attbyval	boolean	该列类型的 sys_type.typbyval 的一个拷贝
attstorage	char	通常是该列类型的 sys_type.typstorage 的一个拷贝 对于可 TOAST 的数据类型，这可以在列创建后被修改以控制存储策略
attalign	char	该列类型的 sys_type.typalign 的一个拷贝
attnotnull	boolean	这表示一个非空约束
atthasdef	boolean	该列有一个默认表达式或生成的表达式，在此情况下在 sys_attrdef 视图中会有一个对应项来真正定义该表达式。 （检查 attgenerated 以确定是默认，还是生成的表达式。）
atthasmissing	boolean	该列在行中完全缺失时会用到这个列的值，如果在行创建之后增加一个有非易失 DEFAULT 值的列，就会发生这种情况 实际使用的值被存放在 attmissingval 列中
attidentity	char	如果是一个零字节（”），则不是一个标识列 否则，a = 总是生成 d = 默认生成
attgenerated	char	如果是零字节 (”)，则不生成列。 否则，s = 已存储。(其他值可能会在未来添加。)
attisdropped	boolean	该列被删除且不再有效 一个删除的列仍然物理存在于表中，但是会被分析器忽略并因此无法通过 SQL 访问

-续表-

名称	类型	描述
attislocal	boolean	该列是由关系本地定义的 注意一个列可以同时是本地定义和继承的
attinhcount	integer	该列的直接祖先的编号 一个具有非零编号祖先的列不能被删除或者重命名
attcollation	oid	该列被定义的排序规则，如果该列不是一个可排序数据类型则为 0
attacl	aclitem[]	列级访问权限
attoptions	text[]	属性级选项，以“keyword=value”形式的字符串
attfdwoptions	text[]	属性级的外部数据包装器选项，以“keyword=value”形式的字符串
attmissingval	anyarray	这个列中是一个含有一个元素的数组，其中的值被用于该列在行中完全缺失时 如果在行创建之后增加一个有非易失 DEFAULT 值的列，就会发生这种情况 只有当 atthasmissing 为真时才使用这个值 如果没有值则该列为空

在一个被删除的列的 `sys_attribute` 的项中，`atttypid` 被重置为 0，但 `attlen` 以及其他从 `sys_type` 拷贝的列值仍然有效。这种安排用于处理一种情况，即被删除列的数据类型后来被删除，并且因此不再有相应的 `sys_type` 行。`attlen` 和其他域可以被用来解释表的一行的内容。

1.1.2.9　sys_available_extension_versions

`sys_available_extension_versions` 视图列出了可用于安装的指定扩展版本。参见sys_extension 视图，它显示当前已安装的扩展。

表 1.1.24: sys_available_extension_versions 列

名称	类型	描述
name	name	扩展名
version	text	版本名

-续表-

名称	类型	描述
installed	boolean	如果此版本的扩展当前已安装则为真
superuser	boolean	如果只有超级用户被允许安装此扩展则为真
relocatable	boolean	如果扩展能被重定位到另一个模式则为真
schema	name	此扩展必须被安装到的模式名，如果此扩展是部分或者全部可以重定位的，此列为 NULL
requires	name[]	先决条件扩展的名字，如果没有则为 NULL
comment	text	来自于扩展的控制文件的注释字符串

1.1.2.10　sys_available_extensions

`sys_available_extensions` 视图列出了可用于安装的扩展。参见sys_extension 视图，它显示当前已安装的扩展。

表 1.1.25: sys_available_extensions 列

名称	类型	描述
name	name	扩展名
default_version	text	默认版本的名字，如果没有指定则为 NULL
installed_version	text	当前已安装的扩展版本，如果没有安装则为 NULL
comment	text	来自于扩展的控制文件的注释字符串

1.1.2.11　sys_cast

`sys_cast` 视图存储数据类型转换路径，包括内建的和用户定义的类型转换信息。

表 1.1.26: sys_cast 列

名称	类型	描述
oid	oid	行标识符
castsource	oid	源数据类型的 OID
casttarget	oid	目标数据类型的 OID
castfunc	oid	执行该转换的函数的 OID。如果该转换方法不需要一个函数则存储 0
castcontext	char	指示该转换能被调用的环境。 e 表示仅能作为一个显式转换（使用 CAST 或:: 语法） a 表示在赋值给目标列时隐式调用，和显式调用一样 i 表示在表达式中隐式调用，和其他转换一样
castmethod	char	指示转换如何被执行。f 表明使用 castfunc 中指定的函数 i 表明使用输入/输出函数 b 表明该类型是二进制可转换的，因此不需要转换
castflags	int	标识类型类型转换的一些特定属性

1.1.2.12　sys_class

`sys_class` 视图记录表和几乎所有具有列或者像表对象的元信息。这包括索引（索引特有属性信息请参见sys_index）、序列（序列特有属性信息请参见sys_sequence）、视图、物化视图、组合类型和 TOAST 表，参见 `relkind`（这些类型的对象下文统称为“关系”），并非所有列对于所有关系类型都有意义。

表 1.1.27: sys_class 列

名称	类型	描述
oid	oid	行标识符
relname	name	表、索引、视图等的名字
relnamespace	oid	包含该关系的名字空间的 OID

-续表-

名称	类型	描述
reltype	oid	可能存在的表行类型所对应数据类型的 OID（对索引为 0，索引没有 sys_type 项）
reloftype	oid	对于有类型的表，为底层组合类型的 OID，对于其他所有关系为 0
relowner	oid	关系的拥有者
relam	oid	如果这是一个表或索引，表或索引使用的访问方法（heap、B 树、哈希等）
relfilenode	oid	该关系的磁盘文件的名字，0 表示这是一个“映射”关系，其磁盘文件名取决于低层状态
reltablespace	oid	该关系所存储的表空间。如果为 0，使用数据库的默认表空间。（如果关系无磁盘文件时无意义）
relpages	integer	该表磁盘表示的尺寸，以页面计（页面尺寸为 BLCKSZ） 这只是一个由规划器使用的估计值，由 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATE INDEX）所更新
reltuples	real	表中的存活行数。这只是一个由规划器使用的估计值。 它被 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATE INDEX）所更新
relallvisible	integer	在表的可见性映射表中被标记为全可见的页数。这只是一个由规划器使用的估计值 它被 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATE INDEX）所更新
reltoastrelid	oid	与该表相关联的 TOAST 表的 OID，如果没有则为 0 TOAST 表将大属性“行外”存储在一个二级表中
relhasindex	boolean	如果这是一个表并且其上建有（或最近建有）索引则为真
relisshared	boolean	如果该表在集簇中的所有数据库间共享则为真

-续表-

名称	类型	描述
relpersistence	char	p = 永久表 u = 无日志表 t = 临时表
relkind	char	r = 普通表 i = 索引 S = 序列 t = TOAST 表 v = 视图 m = 物化视图 c = 组合类型 f = 外部表 p = 分区表 I = 分区索引
relnatts	smallint	关系中用户列的数目（系统列不计算在内）。 在 sys_attribute 中必须有这么多对应的项。另请参阅 sys_attribute.attnum
relchecks	smallint	表上 CHECK 约束的数目，参见 sys_constraint 视图
relhasrules	boolean	如果表有（或曾有）规则则为真，参见 sys_rewrite 视图
relhastriggers	boolean	如果表有（或曾有）触发器则为真，参见 sys_trigger 视图
relhassubclass	boolean	如果表或索引有（或曾有）任何继承子女则为真
relrowsecurity	boolean	如果表上启用了行级安全性则为真，参见 sys_policy 视图
relforcerowsecurity	boolean	如果行级安全性（启用时）也适用于表拥有者则为真，参见 sys_policy 视图
relispopulated	boolean	如果表已被填充则为真（对于所有关系该列都为真，但对于某些物化视图却不是）

-续表-

名称	类型	描述
relreplident	char	用来为行形成“replicaidentity”的列： d = 默认 (主键，如果存在) n = 无 f = 所有列 i = 索引的 indisreplident 被设置或者为默认
relispartition	boolean	如果表或索引是一个分区，则为真
relrewrite	oid	对于在要求表重写的 DDL 操作期间被写入的新关系，这个域包含原始关系的 OID，否则为 0 那种状态仅在内部可见，对于一个用户可见的关系这个域应该从不包含不是 0 的值
relfrozenxid	xid	在此之前的所有事务 ID 在表中已经被替换为一个永久的（“冻结的”）事务 ID 这用于跟踪表是否需要被清理，以便阻止事务 ID 回卷或者允许 sys_xact 被收缩 如果该关系不是一个表则为 0 （InvalidTransactionId）
relminmxid	xid	在此之前的多事务 ID 在表中已经被替换为一个事务 ID。 这被用于跟踪表是否需要被清理，以阻止多事务 ID 回卷或者允许 sys_multixact 被收缩。 如果关系不是一个表则为 0 （InvalidMultiXactId ）
relacl	aclitem[]	访问权限
reloptions	text[]	访问方法相关的选项，以 “keyword=value”字符串形式
relpartbound	pg_node_tree	如果表示一个分区（见 relispartition ），分区边界的内部表达

`sys_class` 中的一些逻辑标志并非实时更新：在正确状态时它们被保证为真，但是当条件不再为真时它们并不会被立刻重置为假。例如，`relhasindex` 由 `CREATE INDEX` 设置，但它从不会被 `DROP INDEX` 清除。作为替代，`VACUUM` 会在找到无索引表后清除其 `relhasindex`。这种安排避免了竞争条件并且提高了并发性。

1.1.2.13　sys_collation

`sys_collation` 视图描述了可用的排序规则，其本质是从一个 SQL 名字到操作系统 locale 分类的映射。

表 1.1.28: sys_collation 列

名称	类型	描述
oid	oid	行标识符
collname	name	排序规则名字（在每一个命名空间和编码中唯一）
collnamespace	oid	包含该排序规则的命名空间的 OID
collowner	oid	排序规则的拥有者
collprovider	char	排序规则的提供者： d = 数据库默认 c = libc i = icu
collisdeterministic	boolean	I 是否为确定的排序规则
collencoding	integer	该排序规则可应用的编码，-1 表示它可用于任何编码
collcollate	text	该排序规则对象的 LC_COLLATE
collctype	text	该排序规则对象的 LC_CTYPE
colliculocale	text	该排序规则对象的详细排序规则信息
collversion	text	排序规则的提供者相关的版本 这是在排序规则创建时记录下来的，并且在使用排序规则时会被检查以检测可能导致数据损坏的排序规则定义的改变

注意在这个视图中的唯一键是（`collname`、`collencoding`、`collencoding`），不仅仅是（`collname`，`collencoding`）。

在 `template0` 数据库中，创建与数据库编码不匹配的编码是有用的，因为它们可以匹配后面从 `template0` 克隆的数据库的编码。这在目前必须手动完成。

1.1.2.14　sys_config

视图 `sys_config` 描述了当前安装的 KingbaseES 版本中的编译时配置参数。它存在的本意是用于那些要和 KingbaseES 交互的软件包，让它们能找到所需要的头文件和库。

默认情况下，`sys_config` 视图只能由超级用户读取。

表 1.1.29: sys_config 列

名称	类型	描述
name	text	参数名
setting	text	参数值

1.1.2.15　sys_constraint

`sys_constraint` 视图存储表上的检查、主键、唯一、外键和排他约束（列约束也不会被特殊对待。每一个列约束都等同于某种表约束。）。非空约束不在这里，而是在 `sys_attribute` 视图中表示。

用户定义的约束触发器（使用 `CREATE CONSTRAINT TRIGGER` 创建）也会在这个表中产生一项。

域上的检查约束也存储在这里。

表 1.1.30: sys_constraint 列

名称	类型	描述
oid	oid	行标识符
conname	name	约束名字（不需要唯一）
connamespace	oid	包含此约束的名字空间的 OID
contype	char	约束类型 c = 检查约束 f = 外键约束 p = 主键约束 u = 唯一约束 t = 约束触发器 x = 排他约束
condeferrable	boolean	该约束是否能被延迟

-续表-

名称	类型	描述
condeferred	boolean	该约束是否默认被延迟
convalidated	boolean	此约束是否被验证过？ 当前对于外键和检查约束只能是假
conrelid	oid	该约束所在的表，如果不是表约束则为 0
contypid	oid	该约束所在的域，如果不是域约束则为 0
conindid	oid	如果该约束是唯一、主键、外键或排他约束，此列表示支持此约束的索引，否则为 0
conparentid	oid	如果这是一个分区中的约束，则是父分区表中对应的约束；否则为 0
confrelid	oid	如果此约束是一个外键约束，此列为被引用的表，否则为 0
confupdtype	char	外键更新动作代码： a = 无动作 r = 限制 c = 级联 n = 置空 d = 置为默认值
confdeltype	char	外键删除动作代码： a = 无动作 r = 限制 c = 级联 n = 置空 d = 置为默认值
confmatchtype	char	外键匹配类型： f = 完全 p = 部分 s = 简单
conislocal	boolean	此约束是定义在关系本地。注意一个约束可以同时是本地定义和继承
coninhcount	integer	此约束的直接继承祖先数目。一个此列非零的约束不能被删除或重命名
connoinherit	boolean	为真表示此约束被定义在关系本地。它是一个不可继承约束

-续表-

名称	类型	描述
conkey	smallint[]	如果是一个表约束（包括外键但不包括约束触发器），此列是被约束列的列表
confkey	smallint[]	如果是一个外键，此列是被引用列的列表
conpfeqop	oid[]	如果是一个外键，此列是用于 PK = FK 比较的等值操作符的列表
conppeqop	oid[]	如果是一个外键，此列是用于 PK = PK 比较的等值操作符的列表
conffeqop	oid[]	如果是一个外键，此列是用于 FK = FK 比较的等值操作符的列表
conexclop	oid[]	如果是一个排他约束，此列是没列排他操作符的列表
conbin	pg_node_tree	如果是一个检查约束，此列是表达式的一个内部表示 建议使用 sys_get_constraintdef() 提取检查约束的定义。
constatus	boolean	约束状态

在一个排他约束的情况中，`conkey` 只对约束元素是单一列引用时有用。对于其他情况，`conkey` 为 0 且必须查阅相关索引来发现被约束的表达式（`conkey` 因此和 `sys_index.indkey` 具有相同的内容）。

备注:

1. 当被引用对象改变时，`consrc` 不能被更新。例如，它不跟踪列的重命名。最好使用 `sys_get_constraintdef()` 来抽取一个检查约束的定义，而不是依赖这个域。
2. `sys_class.relchecks` 需要和每个关系在此视图中的检查约束数量保持一致。

1.1.2.16　sys_context

`sys_context` 视图存储关于 context 的信息。

表 1.1.31: sys_context 列

名称	类型	描述
oid	oid	行标识符
conname	name	context 的名字
connamespace	oid	context 所属命名空间的 id
conower	oid	context 的拥有者
pkgname	name	context 关联包的名字
pkgnamespace	oid	关联包所属命名空间的 id
conacl	aclitem[]	访问权限

1.1.2.17　sys_conversion

`sys_conversion` 视图描述编码转换函数。

表 1.1.32: sys_conversion 列

名称	类型	描述
oid	oid	行标识符
conname	name	转换的名字（在一个名字空间内唯一）
connamespace	oid	包含此转换的名字空间的 OID
conowner	oid	转换的拥有者
conforencoding	integer	源编码 ID
contoencoding	integer	目标编码 ID
conproc	regproc	转换函数
condefault	boolean	如果这是默认转换则为真

1.1.2.18　sys_cursors

sys_cursors 视图列出了当前可用的游标。游标可以以几种方式定义：

- 通过 SQL 中的 DECLARE 语句
- 通过前端/后端协议中的绑定消息
- 通过服务器编程接口（SPI）

sys_cursors 视图显示由任何这些方式创建的游标。视图只存在于定义它们的事务期间，除非声明了 WITH HOLD。因此非保持游标只在它们的创建事务结束前存在于这个视图中。

备注:

视图用于在内部实现 KingbaseES 的某些部件，例如过程语言。因此，sys_cursors 视图可能包括那些不是由用户显式创建的游标。

表 1.1.33: sys_cursors 列

名称	类型	描述
name	text	游标名称
statement	text	提交用于定义此游标的查询语句
is_holdable	boolean	如果游标是可保持的（即可以在其定义事务提交后被访问）则为 true ，否则为 false
is_binary	boolean	如果游标被声明为 BINARY 则为 true ，否则为 false
is_scrollable	boolean	如果游标是可滚动的（即，允许以一种非顺序的方式检索行）则为 true ，否则为 false
creation_time	timestamp with time zone	游标被声明的时间

1.1.2.19　sys_database

sys_database 视图存储有关可用数据库的信息。数据库通过 CREATE DATABASE 命令创建。

和大部分系统视图不同，sys_database 视图是在集簇的所有数据库之间共享的：在一个集簇中所有数据库共用一个 sys_database 视图，而不是在单个数据库下仅显示当前数据库的相关信息。

表 1.1.34: sys_database 列

名称	类型	描述
oid	oid	行标识符
datname	name	数据库名字
datdba	oid	数据库的拥有者
encoding	integer	此数据库的字符编码的编号（sys_encoding_to_char() 可将此编号转换成编码的名字）
datcollate	name	此数据库的 LC_COLLATE
datctype	name	此数据库的 LC_CTYPE
datistemplate	boolean	如果为真，则此数据库可被任何具有 CREATEDB 特权的用户克隆 如果为假，则只有超级用户或者该数据库的属主能够克隆它
datallowconn	boolean	如果为假则没有人能连接到这个数据库。这可以用来保护 template0 数据库不被修改
datconnlimit	integer	设置能够连接到这个数据库的最大并发连接数。-1 表示没有限制
datlastsysoid	oid	数据库中最后一个系统 OID，对 sys_dump 特别有用
datfrozenxid	xid	在此之前的所有事务 ID 在数据库中已经被替换为一个永久的（“冻结的”）事务 ID 这用于跟踪数据库是否需要被清理，以便组织事务 ID 回环或者允许 sys_xact 被收缩。它是此数据库中所有表的 sys_class 中 relfrozenxid 值的最小值
datminmxid	xid	此数据库中所有表的 sys_class relminmxid 值的最小值
dattablespace	oid	此数据库的默认表空间 在此数据库中，所有 sys_class reltablespace 为 0 的表都将被存储在这个表空间中，尤其是非共享系统目录都会在其中
datinitlsn	bigint	记录数据库被创建时候的 lsn 号
datcreatedts	bigint	数据库被创建的时刻

-续表-

名称	类型	描述
datlocprovider	char	数据库排序规则使用的库
daticulocale	text	数据库排序规则信息
datcollversion	text	数据库排序规则版本信息
datacl	aclitem[]	访问权限
dataflashhbacklogs	text[]	数据库闪回归档的记录

参考

- CREATE DATABASE(Oracle)
- CREATE DATABASE(SQLServer)
- CREATE DATABASE(MySQL)

1.1.2.20　sys_database_link

`sys_database_link` 视图存储有关 dblink 外部链接的信息。

表 1.1.35: sys_database_link 列

名称	类型	描述
oid	oid	行标识符
lnkname	name	dblink 名称
lnknamespace	oid	dblink 所属名字空间的 OID
lnkuser	oid	dblink 用户名行标识符
lnkowner	oid	dblink 所属用户名 ID
lnkserver	oid	dblink 服务行标识符
dbtype	name	dblink 源数据库类型
lnkoptions	text[]	dblink 连接信息
lnkcreated	timestamp with time zone	dblink 创建时间

1.1.2.21　sys_db_role_setting

`sys_db_role_setting` 视图为每一个角色和数据库组合记录被设置到运行时配置变量的默认值。

和大部分系统视图不同，`sys_db_role_setting` 是在集簇的所有数据库之间共享的：在一个集簇中所有数据库共用一个 `sys_db_role_setting` 视图，而不是在单个数据库下仅显示当前数据库的相关信息。

表 1.1.36: sys_db_role_setting 列

名称	类型	描述
setdatabase	oid	此设置可用的数据库 OID，如果不与具体数据库相关则为 0
setrole	oid	此设置可用的角色 OID，如果不与具体角色相关则为 0
setconfig	text[]	运行时配置变量的默认值

1.1.2.22　sys_default_acl

`sys_default_acl` 视图存储要被分配给新创建对象的初始权限。

表 1.1.37: sys_default_acl 列

名称	类型	描述
oid	oid	行标识符
defaclrole	oid	与此项相关的角色的 OID
defaclnamespace	oid	与此项相关的名字空间的 OID，如果没有则为 0
defaclobjtype	char	此项适合的对象类型： r = 关系（表、视图） S = 序列 f = 函数 T = 类型 n = 模式

-续表-

名称	类型	描述
defaclacl	aclitem[]	此类对象在创建时应用有的访问权限

一个 sys_default_acl 项展示了要分配给属于一个指定用户的对象的初始权限。当前有两类项：defaclnamespace=0 的“全局”项和引用一个特殊模式的“每方案”项。如果一个全局项存在，则它重载该对象类型的普通 hard-wired 默认权限。一个每模式项如果存在，表示权限将被加入到全局或 hard-wired 默认权限中。

注意当在另一个表中的一个 ACL 项为空时，它用来表示其对象的 hard-wired 默认权限，而不是当时可能在 sys_default_acl 中的任何权限。只有在对象创建期间才会查阅 sys_default_acl。

1.1.2.23　sys_depend

sys_depend 视图记录数据库对象之间的依赖关系。这些信息允许 DROP 命令查找必须被 DROP CASCADE 删除的其他对象，或者在 DROP RESTRICT 情况下阻止删除。

另请参阅sys_shdepend ，它对在一个数据库集簇中共享的对象之间的依赖提供了相似的功能。

表 1.1.38: sys_depend 列

名称	类型	描述
classid	oid	依赖对象所在的系统视图 OID
objid	oid	指定依赖对象的 OID
objsubid	integer	对于一个表列，这里是列号（objid 和 classid 指表本身） 对于所有其他对象类型，此列为 0
refclassid	oid	被引用对象所在的系统视图的 OID
refobjid	oid	指定被引用对象的 OID
refobjsubid	integer	对于一个表列，这里是列号（refobjid 和 refclassid 指表本身） 对于所有其他对象类型，此列为 0
deptype	char	定义此依赖关系语义的一个代码

在所有情况下，一个 sys_depend 项表明被引用对象不能在没有删除其依赖对象的情况下被删除。但是，其中也

有多种依赖类型，由 deptype 标识：

DEPENDENCY_NORMAL (n)
在独立创建的对象之间的一个普通关系。依赖对象可以在不影响被依赖对象的情况下被删除。被引用对象只能通过指定 CASCADE 被删除，在这种情况下依赖对象也会被删除。例子：一个表列对于其数据类型有一个普通依赖。

DEPENDENCY_AUTO (a)
依赖对象可以被独立于被依赖对象删除，且应该在被引用对象被删除时自动被删除（不管在 RESTRICT 或 CASCADE 模式）。例子：一个表上的一个命名约束应该被设置为自动依赖于表，这样在表被删除后它也会消失。

DEPENDENCY_INTERNAL (i)
依赖对象作为被引用对象创建过程的一部分被创建，并且只是其内部实现的一部分。依赖对象的直接 DROP 将被完全禁止 (我们将告诉用户对引用的对象发出 DROP)。不管是否指定了级联，对引用对象的删除都会自动删除依赖对象。如果依赖对象由于对被删除的其他对象的依赖而必须删除，则其删除将转换为引用对象的删除，以便依赖对象的正常和自动依赖关系的行为与引用对象的依赖关系非常相似。示例: 视图的 ON SELECT 规则在内部依赖于视图，从而防止在视图保留时删除它。规则的依赖项 (例如它引用的表) 的行为就好像它们是视图的依赖项一样。

DEPENDENCY_PARTITION_PRI (P) DEPENDENCY_PARTITION_SEC (S)
依赖对象是作为被引用对象创建的一部分创建的，它实际上只是其内部实现的一部分; 但是，与内部对象不同，有多个这样的引用对象。除非被引用的对象中至少有一个被删除，否则不能删除从属对象; 如果有，则无论是否指定了级联，都应该删除依赖对象。与内部对象不同的是，依赖对象所依赖的其他对象的删除不会导致任何分区引用对象的自动删除。因此，如果删除没有通过其他路径级联到这些对象中的至少一个，那么它将被拒绝。(在大多数情况下，依赖对象与至少一个分区引用的对象共享其所有非分区依赖项，因此此限制不会导致阻塞任何级联删除。) 主分区依赖项和辅助分区依赖项的行为相同，但主分区依赖项更适合在错误消息中使用; 因此，与分区相关的对象应该有一个主分区依赖项和一个或多个辅助分区依赖项。注意，分区依赖项是附加的，而不是代替的，对象通常具有的任何依赖项。这简化了 ATTACH/DETACH PARTITION 分区操作: 只需要添加或删除分区依赖项。例如: 子分区索引的分区依赖于它所在的分区表和父分区索引，因此，如果其中一个被删除，它就会消失，但如果不删除，它就不会消失。对父索引的依赖是主索引，因此，如果用户试图删除子分区索引，错误消息将建议删除父索引 (而不是表)。

DEPENDENCY_EXTENSION (e)
从属对象是引用对象的扩展的成员 (参见sys_extension)。依赖对象只能通过引用对象的 DROP EXTENSION 来删除。从功能上讲，这个依赖项类型与内部依赖项的作用相同，但是为了明晰和简化 sys_dump，它是独立的。

DEPENDENCY_AUTO_EXTENSION (x)
从属对象不是作为引用对象的扩展的成员 (因此 sys_dump 不应该忽略它)，但是没有扩展它就不能工作，如果扩展是，则应该自动删除它。依赖对象也可以自动删除。在功能上，这个依赖项类型与自动依赖项的作用相同，但是为了明晰和简化 sys_dump，它是独立的。

DEPENDENCY_PIN (p)
没有依赖对象，这种类型的项是一个信号，用于说明系统本身依赖于被引用对象，并且该对象永远不能被删除。这种类型的项只能被 initdb 创建。而此种项的依赖对象的列都为 0。

DEPENDENCY_REFERENCE (r)
被引用对象在依赖对象的定义中被使用时，会认为二者的依赖关系为引用依赖。引用依赖仅用于影响主从对象在数据库中的状态，并不会影响主从对象的删除或重建。仅在开启 PL/SQL 对象支持状态功能时会生成引用依赖关系，详情参见《KingbaseES 数据库管理手册》的 ora_func_style 参数章节。

在未来可能会需要其他依赖类型。

注意，两个对象很可能由多个 sys_depend 条目链接。例如，子分区索引对其关联的分区表具有分区类型依赖项，对其索引的表的每一列具有自动依赖项。这种情况表达了多种依赖语义的结合。如果依赖对象的任何依赖项满足自动删除的条件，则可以不使用级联删除依赖对象。相反，必须满足关于哪些对象必须一起删除的所有依赖项限制。

1.1.2.24　sys_depends

视图 sys_depends 提供对象的依赖关系信息。

表 1.1.39: sys_depends 列

名称	类型	描述
oid	oid	依赖者对象的 oid
name	name	依赖者对象名称
type	text	依赖者对象类型：r = 普通表 i = 索引 S = 序列 t = 触发器 v = 视图 m = 物化视图 c = 组合类型 f = 外部表 p = 分区表 I = 分区索引
refrelid	oid	被依赖者对象的 oid
refrelname	name	被依赖者对象名称
refreltype	char	被依赖者对象类型：r = 普通表 i = 索引 S = 序列 t = 触发器 v = 视图 m = 物化视图 c = 组合类型 f = 外部表 p = 分区表 I = 分区索引

1.1.2.25　sys_description

`sys_description` 视图存储对每一个数据库对象可选的描述（注释）。描述可以通过 `COMMENT` 操作，并可使用 ksql 的`\d` 命令查看。在 `sys_description` 的初始内容中提供了很多内建系统对象的描述。

参见sys_shdescription ，它对在一个数据库集簇中共享的对象的描述提供了相似的功能。

表 1.1.40: sys_description 列

名称	类型	描述
objoid	oid	描述所属对象的 OID
classoid	oid	对象元信息所在的系统表的 OID
objsubid	integer	若为表上的列的注释信息，此列为列号 若为其他对象类型，此列为 0
description	text	作为该对象描述的任意文本
name	text	记录模式对象的属性名称

参考

- COMMENT(Oracle)
- COMMENT(SQLServer)

1.1.2.26　sys_directory

`sys_directory` 存储目录 DIRECTORY 的信息。

表 1.1.41: sys_directory 列

名称	类型	描述
oid	oid	目录 oid
dirname	name	目录名称
dirowner	oid	目录的属主 oid
originconid	oid	默认置 0
ospath	text	目录的路径

-续表-

名称	类型	描述
diracl	aclitem[]	默认置 null

1.1.2.27　sys_enum

`sys_enum` 视图包含每一个枚举类型的项，其中包括了值和标签。一个给定枚举值的内部表示实际上是它在 `sys_enum` 中的相关行的 OID。

表 1.1.42: sys_enum 列

名称	类型	描述
oid	oid	行标识符
enumtypid	oid	包含此枚举值的 sys_type type 项的 OID
enumsortorder	real	此枚举值在其枚举类型中的排序位置
enumlabel	name	此枚举值的文本标签

`sys_enum` 行的 OID 值遵循一种特殊的规则：即 OID 的数值被保证按照其枚举类型的排序顺序进行排序。即如果两个偶数 OID 属于同一枚举类型，较小的 OID 必然具有较小的 `enumsortorder` 值。

当一个枚举类型被创建后，其成员会被分配排序位置 $1 \cdots n$。但后面增加的成员可能会被分配负值或者分数值的 `enumsortorder`。对于这些值的唯一要求是它们必须被正确地排序且和保持唯一。

1.1.2.28　sys_event_trigger

`sys_event_trigger` 视图存储事件触发器。更多信息参见 触发器。

表 1.1.43: sys_event_trigger 列

名称	类型	描述
oid	oid	行标识符
evtname	name	触发器名（必须唯一）

-续表-

名称	类型	描述
evtevent	name	此触发器触发的事件的标识符
evtowner	oid	事件触发器的拥有者
evtfoid	oid	将被调用的函数
evtenabled	char	控制事件触发器触发的 session_replication_role 模式 O = 触发器在“origin”和“local”模式触发 D = 触发器被禁用 R = 触发器在“replica”模式触发 A = 触发器总是触发
evttags	text[]	此触发器将触发的命令标签。如果为空，此触发器的触发不受命令标签的限制

1.1.2.29　sys_extension

`sys_extension` 视图存储有关已安装扩展的信息。有关扩展的细节请参见扩展插件概述 。

表 1.1.44: sys_extension 列

名称	类型	描述
oid	oid	行标识符
extname	name	扩展的名字
extowner	oid	扩展的拥有者
extnamespace	oid	包含此扩展的导出对象的模式
extrelocatable	boolean	如果扩展可被重定位到另一个模式则为真
extversion	text	扩展的版本名字
extconfig	oid[]	扩展的配置表的 regclass 项的 OID 数组，如果没有配置表则为 NULL
extcondition	text[]	扩展的配置表的 WHERE 子句过滤条件的数组，如果没有则为 NULL

注意和大部分具有一个“namespace”列的模式不同，`extnamespace` 不是用来表示扩展属于该模式。扩展的名字从不用模式进行限定。`extnamespace` 表明该模式包含了该扩展的大部分或全部对象。如果 `extrelocatable` 为真，则该模式事实上必须包含属于此扩展的全部模式限定的对象。

1.1.2.30　sys_file_settings

视图 `sys_file_settings` 提供了服务器配置文件内容的概要。这个视图中的每一行表示配置文件中出现的一个“name =value”项，还带有注解指示该值是否被成功地应用。在配置文件有问题时，有可能出现额外的行，它们没有相关的“name =value”项，一个例子是配置文件中有语法错误。

这个视图有助于检查在配置文件中打算做的修改是否能工作，或者用来诊断之前的失败。注意这个视图报告的是配置文件的当前内容，而不是服务器最后应用的值（这些值通常查看sys_settings 视图就够了）。

默认情况下，`sys_file_settings` 视图只有超级用户可读。

表 1.1.45: sys_file_settings 列

名称	类型	描述
sourcefile	text	配置文件的完整路径名
sourceline	integer	该项在配置文件中出现的行号
seqno	integer	项被处理的顺序（1..*n*）
name	text	配置参数名
setting	text	被赋予给参数的值
applied	boolean	为真表示值已被成功应用
error	text	如果非空，表示一个错误消息，它说明为什么这个项不能被应用

如果配置文件包含语法错误或者非法参数名，服务器将不会尝试从其中应用任何设置，并且因此所有的 `applied` 列值都为假。在这种情况下，将会有一个或者多个行的 `error` 列值为非空，它们说明了为什么出问题。否则，将尽可能应用每个设置。如果一个设置不能被应用（例如非法值或者该设置不能在服务器开始后改变），会有一个合适的消息存储在它的 `error` 列值中。一个项的 `applied` 列值为假的另一种情况是它被后面一个具有相同参数名的项所覆盖，这种情况不会被认为是一种错误，因此在 `error` 列值中不会有错误消息。

关于更改运行时参数的各种方法请见 设置参数。

1.1.2.31　sys_foreign_data_wrapper

`sys_foreign_data_wrapper` 视图存储外部数据包装器定义。外部数据包装器是一种访问位于外部服务器上数据的机制。

表 1.1.46: sys_foreign_data_wrapper 列

名称	类型	描述
oid	oid	行标识符
fdwname	name	外部数据包装器的名字
fdwowner	oid	外部数据包装器的拥有者
fdwhandler	oid	指一个负责为外部数据包装器提供执行例程的处理函数 如果没有提供处理函数则为 0
fdwvalidator	oid	指一个负责检查传给外部数据包装器的选项的有效性的验证函数，包括外部服务器选项以及使用外部数据包装器的用户映射 如果没有提供验证函数则为 0
fdwacl	aclitem[]	访问权限
fdwoptions	text[]	外部数据包装器特定选项，以“keyword=value”字符串形式

1.1.2.32　sys_foreign_server

`sys_foreign_server` 视图存储外部服务器定义。外部服务器定义了外部数据的来源，例如一个远程服务器。外部服务器通过外部数据包装器来访问。

表 1.1.47: sys_foreign_server 列

名称	类型	描述
oid	oid	行标识符
srvname	name	外部服务器的名字
srvowner	oid	外部服务器的拥有者

-续表-

名称	类型	描述
srvfdw	oid	此外部服务器的外部数据包装器的 OID
srvtype	text	服务器的类型（可选）
srvversion	text	服务器的版本（可选）
srvacl	aclitem[]	访问权限
srvoptions	text[]	外部服务器特定选项，以“keyword=value”字符串形式

1.1.2.33　sys_foreign_table

`sys_foreign_table` 视图包含关于外部表的辅助信息。一个外部表和普通表一样，主要由一个 `sys_class` 项表示。它的 `sys_foreign_table` 项包含外部表所特有的信息。

表 1.1.48: sys_foreign_table 列

名称	类型	描述
ftrelid	oid	外部表的 sys_class 项的 OID
ftserver	oid	外部表所在的外部服务器的 OID
ftoptions	text[]	外部表选项，以“keyword = value”字符串形式

1.1.2.34　sys_grant_privileges

视图 `sys_grant_privileges` 提供用户或者角色所授予和被授予的权限信息。

表 1.1.49: sys_grant_privileges 列

名称	数据类型	描述
grantor	name	授予该权限的用户名或角色名
grantortype	text	授予者类型 (用户或者角色)
grantee	name	被授予该权限的用户名或角色名
granteetype	text	被授予者类型 (用户或者角色)
object_name	name	权限相关的对象名称
object_type	text	权限相关的对象类型
object_schema	name	权限相关的对象所属的模式
privilege_type	text	权限类型
is_grantable	text	该特权是否为可授予的

1.1.2.35　sys_index

`sys_index` 包含关于索引的部分信息。其他信息大部分在 `sys_class` 中。

表 1.1.50: sys_index 列

名称	类型	描述
indexrelid	oid	此索引的 sys_class 项的 OID
indrelid	oid	此索引的基表的 sys_class 项的 OID
indnatts	smallint	索引中的总列数（与 sys_class.relnatts 重复），这个数目包括键和被包括的属性
indnkeyatts	smallint	索引中键列的编号，不计入任何的内含列，它们只是被存储但不参与索引的语义
indisunique	boolean	表示是否为唯一索引
indisprimary	boolean	表示索引是否表示表的主键（如果此列为真，indisunique 也总是为真）

-续表-

名称	类型	描述
indisexclusion	boolean	表示索引是否支持一个排他约束
indimmediate	boolean	表示唯一性检查是否在插入时立即被执行（如果 indisunique 为假，此列无关）
indisclustered	boolean	如果为真，表示表最后以此索引进行了聚簇
indisvalid	boolean	如果为真，此索引当前可以用于查询。 为假表示此索引可能不完整：它肯定还在被 INSERTUPDATE 操作所修改，但它不能安全地被用于查询。 如果索引是唯一索引，唯一性属性也不能被保证
indcheckxmin	boolean	如果为真，直到此 sys_index 行的 xmin 低于查询的 TransactionXmin 视界之前，查询都不能使用此索引 因为表可能包含具有它们可见的不相容行的损坏 HOT 链
indisready	boolean	如果为真，表示此索引当前可以用于插入 为假表示索引必须被 INSERT UPDATE 操作忽略
indislive	boolean	如果为假，索引正处于被删除过程中，并且必须被所有处理忽略（包括 HOT 安全的决策）
indisreplident	boolean	如果为真，这个索引被选择为使用 ALTER TABLE ... REPL ICA IDENTITY USING INDEX ... 的“replicaidentity”

-续表-

名称	类型	描述
indkey	int2vector	这是一个 indnatts 值的数组，它表示了此索引索引的表列 例如一个 1 3 值可能表示表的第一和第三列组成了索引项 键列出现在非键（内含）列前面 数组中的一个 0 表示对应的索引属性是一个在表列上的表达式，而不是一个简单的列引用
indcollation	oid[]	对于索引键（indnkeyatts 值）中的每一列，这包含要用于该索引的排序规则的 OID，如果该列不是一种可排序数据类型则为零
indclass	oid[]	对于索引键中的每一列（indnkeyatts 值），这里包含了要使用的操作符类的 OID
indoption	int2vector	这是一个 indnkeyatts 值的数组，用于存储每列的标志位。位的意义由索引的访问方法定义
indexprs	pg_node_tree	非简单列引用索引属性的表达式树（以 nodeToString() 形式）。对于 indkey 中每一个为 0 的项，这个列表中都有一个元素 如果所有的索引属性都是简单引用，此列为空
indpred	pg_node_tree	部分索引谓词的表达式树（以 nodeToString() 形式）。 如果不是部分索引，此列为空

1.1.2.36　sys_indexes

视图 `sys_indexes` 提供对于数据库中每一个索引信息的访问。

表 1.1.51: sys_indexes 列

名称	类型	描述
schemaname	name	包含表和索引的模式名
tablename	name	此索引的基表的名字
indexname	name	索引名
tablespace	name	包含索引的表空间名（如果是数据库的默认值则为空）
indexdef	text	索引定义（CREATE INDEX 命令的重构）

1.1.2.37　sys_inherits

`sys_inherits` 视图记录有关表继承层次的信息。数据库中每一个直接父子关系在这里都有一项（非直接继承可以通过顺着项构成的链来决定）。

表 1.1.52: sys_inherits 列

名称	类型	描述
inhrelid	oid	子表的 OID
inhparent	oid	父表的 OID
inhseqno	sinteger	如果一个子表有多于一个直接父表（多继承），这个数字说明了继承列被排列的顺序。计数从 1 开始
inhinterval	integer	是否为 interval 分区。0：表示 range 分区；3：表示 interval 分区
inhsubname	name	分区名称

1.1.2.38　sys_init_privs

`sys_init_privs` 视图记录系统中对象的初始特权。数据库中每一个具有非默认（非-NULL）初始特权集合的对象都有一个条目在其中。

对象可以在系统初始化（initdb）时获得其初始特权，也可以在 `CREATE EXTENSION` 期间创建该对象并且在扩展脚本中用 `GRANT` 来设置对象的初始特权。注意系统将自动处理扩展脚本执行期间对特权的记录，扩展的作者们只需要在他们的脚本中使用 `GRANT` 以及 `REVOKE` 语句以便特权被记录下来。`privtype` 列表示初始特权是被 initdb 设置还是在一次 `CREATE EXTENSION` 命令期间被设置。

具有被 initdb 设置的初始特权的对象对应行的 `privtype` 是`'i'`，而具有被 `CREATE EXTENSION` 设置的初始特权的对象的对应行中 `privtype` 为`'e'`。

表 1.1.53: sys_init_privs 列

名称	类型	描述
objoid	oid	指定对象的 OID
classoid	oid	对象所在的系统视图的 OID
objsubid	integer	对于一个表列，这里是列编号（objoid 和 classoid 指向表本身）。对于所有其他对象类型，这列为零。
privtype	char	定义这个对象初始特权类型的代码，见文字说明
initprivs	aclitem[]	初始的访问特权，详见数据定义

1.1.2.39　sys_language

`sys_language` 视图记录了可用于编写函数或存储过程的语言。

表 1.1.54: sys_language 列

名称	类型	描述
oid	oid	行标识符
lanname	name	语言的名字
lanowner	oid	语言的拥有者
lanispl	boolean	内部语言为假（如 SQL），用户定义语言为真。当前，sys_dump 仍然使用这个列来决定要转储哪些语言，但在未来这可能会被一种不同的机制所取代
lanpltrusted	boolean	为真表示这是一种可信的语言，即它被相信不会向普通 SQL 执行环境之外的任何东西授予权限 只有超级用户可以在非可信语言中创建函数

-续表-

名称	类型	描述
lanplcallfoid	oid	对于非内部语言，此列引用语言处理器，它是一个特殊函数负责执行所有用这种语言编写的函数
laninline	oid	此列引用一个负责执行“内联”匿名代码块的函数 如果不支持内联块则为 0
lanvalidator	oid	此列引用一个负责在函数创建时对其进行语法和可用性检查的语言验证函数 如果没有提供验证器则为 0
lanacl	aclitem[]	访问权限

1.1.2.40　sys_largeobject

`sys_largeobject` 视图保存构成“大对象”的数据。一个大对象在被创建时会被分配一个 OID。每个大对象被分解成段或“页”，以便小到可以被方便地作为行存储在 `sys_largeobject` 中。每页中的数据量被定义为 `LOBLKSIZE`（目前是 `BLCKSZ/4` 或是 2 kB）。

在 KingbaseES V8R2 之前，大对象没有相关的权限结构。作为结果，`sys_largeobject` 是公共可读的并且可以用来获得系统中所有大对象的 OID（和内容）。但现在不是这样了，可使用sys_largeobject_metadata 来获得大对象 OID 的列表。

表 1.1.55: sys_largeobject 列

名称	类型	描述
loid	oid	包含此页的大对象的标识符
pageno	integer	此页在它所属大对象中的页号（从 0 开始计）
data	bytea	实际存储在大对象中的数据。它从不会超过 LOBLKSIZE 字节，也可能更少

`sys_largeobject` 的每一行保存一个大对象的一个页的数据，从对象内部的字节偏移量（`pageno * LOBLKSIZE`）开始。现在的实现允许稀疏存储：页面可能丢失，并且可能比 `LOBLKSIZE` 字节短（即便不是最后一页）。一个大对象中丢失的区域会被读出为 0。

1.1.2.41　sys_largeobject_metadata

`sys_largeobject_metadata` 视图保持着与大对象有关的元数据。真正的大对象数据被存储在sys_largeobject中。

表 1.1.56: sys_largeobject_metadata 列

名称	类型	描述
oid	oid	行标识符
lomowner	oid	大对象的拥有者
lomacl	aclitem[]	访问权限

1.1.2.42　sys_locks

视图 `sys_locks` 提供了数据库服务器上活动进程中保持的锁的信息。

`sys_locks` 中对每一个活动可锁对象、请求锁模式和相关进程的组合都有一行。因此，如果多个进程持有或者正在等待一个可锁对象上的锁，同一个可锁对象可能出现很多次。但是，一个当前没有被锁的对象根本不会出现。

有多种不同类型的可锁对象：整个关系（如表）、关系的单个页、关系的单个元组、事务 ID（包括虚拟和永久 ID）和普通数据库对象（由类 OID 和对象 OID 标识，和 `sys_description` 或 `sys_depend` 中的相同方式）。扩展一个关系的权力也被表示为一个独立的可锁对象。“咨询”锁可以具有用户定义的意义。

表 1.1.57: sys_locks 列

名称	类型	描述
locktype	text	可锁对象的类型： relation extend page tuple transactionid virtualxid object userlock advisory

-续表-

名称	类型	描述
database	oid	锁目标存在的数据库的 OID，如果目标是一个共享对象则为 0，如果目标是一个事务 ID 则为空
relation	oid	作为锁目标的关系的 OID，如果目标不是一个关系或者只是关系的一部分则此列为空
page	integer	作为锁目标的页在关系中的页号，如果目标不是一个关系页或元组则此列为空
tuple	smallint	作为锁目标的元组在页中的元组号，如果目标不是一个元组则此列为空
virtualxid	text	作为锁目标的事务虚拟 ID，如果目标不是一个虚拟事务 ID 则此列为空
transactionid	xid	作为锁目标的事务 ID，如果目标不是一个事务 ID 则此列为空 ID
classid	oid	包含锁目标的系统表的 OID，如果目标不是一个普通数据库对象则此列为空
objid	oid	锁目标在它的系统表中的 OID，如果目标不是一个普通数据库对象则为空
objsubid	smallint	锁的目标列号（classid 和 objid 指表本身） 如果目标是某种其他普通数据库对象则此列为 0 如果目标不是一个普通数据库对象则此列为空
virtualtransaction	text	保持这个锁或者正在等待这个锁的事务的虚拟 ID
pid	integer	保持这个锁或者正在等待这个锁的服务器进程的 PID，如果此锁被一个预备事务所持有则此列为空
mode	text	此进程已持有或者希望持有的锁模式
granted	boolean	如果锁已授予则为真，如果锁被等待则为假
fastpath	boolean	如果锁通过快速路径获得则为真，通过主锁表获得则为假

一个行的 granted 为真表示一个被指定进程持有的锁。为假表示该进程当前正在等待获取这个锁，这意味着至少一个其他进程正持有或等待同一个可锁对象上的一个冲突锁。该等待进程将一直休眠直到其他锁被释放（或者一个死锁状态被检测到）。单个进程在同一时间只能等待最多一个锁。

贯穿一个事务的运行，一个服务器进程在其生存周期内都持有一个在其虚拟事务 ID 上的排他锁。如果一个永久 ID 被分配给事务（通常发生在事务改变数据库状态时），它也会持有一个在其永久事务 ID 上的排他锁直到它结束。当一个事务发现它需要等待另一个事务，它也会尝试获取其他事务 ID 上的共享锁（不管是虚拟还是永久 ID，视情况而定）。这只有当其他进程终止并释放其锁后才会成功。

尽管元组是一种可锁对象，关于行级锁的信息被存储在磁盘而不是内存中，因此行级锁通常不在这个视图中出现。如果一个进程正在等待一个行级锁，它通常在这个视图中出现，并且表示形式为正在等待已持有该行级锁的永久事务 ID 上的锁。

咨询锁可以在由一个单一 bigint 值或两个整形值构成的键上获取。一个 bigint 键被显示为其高位部分在 classid 列中，低位部分在 objid 列中，并且 objsubid 等于 1。原来的 bigint 值可以使用表达式 (classid::bigint << 32) | objid::bigint 重组。整形键被显示为第一个键在 classid 列中，第二个键在 objid 列中，并且 objsubid 等于 2。键的实际意义由用户决定。咨询锁是每一个数据库的本地锁，所以 database 列对于一个咨询锁没有意义。

sys_locks 提供了一个对于整个数据集簇中所有锁的全局视图，而不仅仅是与当前数据库相关的锁。尽管它的 relation 列可以被连接到 sys_class.oid 来标识被锁关系，但这种方法只有在关系属于当前数据库（database 列是当前数据库 OID 或者 0 的锁对应的关系）的情况下才会得到正确的结果。

pid 列可以被连接到 sys_stat_activity 视图的 pid 列来得到持有或等待持有每一个锁的会话的信息。

例如

```
SELECT * FROM sys_locks pl LEFT JOIN sys_stat_activity psa
ON pl.pid = psa.pid;
```

另外，如果正在使用预备事务，virtualtransaction 列可以被连接到 [sys_prepared_xacts](/docs/reference/03_database/01-系统视图/kes_views/metadata_views/sys_prepared_xacts.md 视图的 transaction 列来得到持有该锁的预备事务的信息（一个预备事务不可能正在等待一个锁，但它在运行中会一直持有已获得的锁）。

例如：

```
SELECT * FROM sys_locks pl LEFT JOIN sys_prepared_xacts ppx
ON pl.virtualtransaction = '-1/' || ppx.transaction;
```

虽然通过自连接 sys_locks 可以获得哪些进程阻塞了其他哪些进程的信息，但是很难得到其中的细节。这样一个查询隐藏了关于哪些锁模式与其他哪些锁模式冲突的知识。更糟糕的是，sys_locks 视图无法给出所等待队列中进程的等待顺序，也无法显示哪些进程是代表其他客户端会话运行的并行工作者。更好的方法是使用 sys_blocking_pids() 函数来标识一个等待进程是被哪些进程阻塞的。

sys_locks 视图显示来自于普通锁管理器和谓词锁管理器的数据，它们是独立的系统。此外，普通锁管理器把它的锁分为普通锁和 * 快速路径 * 锁。这些数据并不被保证是完全一致的。当视图被查询时，快速路径锁上的数据（fastpath = true）会被一次性从每一个后端收集起来，且并不冻结整个锁管理器的状态。因此有可能某些锁在

上述信息被收集的过程中被获得或者释放。注意，不管怎样这些锁是已知不会和任何当前正在发生的锁冲突。在所有后端已经查询了快速路径锁后，普通锁管理器的剩余部分被作为一个单元锁住，并且所有剩余锁的一个一致快照被作为一个原子动作收集。在解锁普通锁管理器后，谓词锁管理器也被类似地锁住并且所有谓词锁被作为一个原子动作收集。因此，在快速路径锁这种特殊情况下，每一个锁管理器会传递一个一致的结果组。但由于我们并不会同时锁上两个锁管理器，在我们询问完普通锁管理器后或者询问谓词锁管理器之前，锁可以被获得或者释放。

如果对此视图频繁地访问，对普通或者谓词锁管理器加锁可能会对数据库性能产生一定影响。虽然这些锁只会在最少的时间内被保持（足以从锁管理器获得数据），但这无法完全消除可能产生的性能影响。

1.1.2.43　sys_matviews

视图 `sys_matviews` 提供了关于数据库中每一个物化视图的信息。

表 1.1.58: sys_matviews 列

名称	类型	描述
schemaname	name	包含物化视图的模式的名字
matviewname	name	物化视图的名字
matviewowner	name	物化视图拥有者的名字
tablespace	name	包含物化视图的表空间名（如使用数据库默认表空间则为空）
hasindexes	boolean	如果物化视图有（或者最近有过）任何索引，则此列为真
ispopulated	boolean	如果物化视图当前已被填充，则此列为真
definition	text	物化视图的定义（一个重构的 SELECT 查询）

1.1.2.44　sys_mview

`sys_mview` 视图存储物化视图和物化视图日志的相关信息。

表 1.1.59: sys_mview 列

名称	类型	描述
oid	oid	物化视图 oid

-续表-

名称	类型	描述
logbaseid	oid	物化视图日志基表 oid
logid	oid	物化视图日志 oid
logrefreshrule	char	物化视图日志指定的刷新规则： 'i' -with ctid; 'p' -with primary key; 'r' -with rowid
logidxid	oid	物化视图日志的索引 oid
mvbaseid	oid	物化视图的基表 oid
mvidxid	oid	物化视图的索引 oid
mvrefreshmode	char	物化视图指定的刷新模式： 'd' -on demand; 't' -on commit
mvrefreshtype	char	物化视图指定的刷新类型： 'f' -fast; 'c' -complete; '?' -force
mvrefreshrule	char	物化视图指定的刷新规则： 'i' -with ctid; 'p' -with primary key; 'r' -with rowid
mvrefreshlsn	pg_lsn	物化视图刷新 lsn
mvjobid	integer	自动刷新 jobid
mvstarttime	bigint	物化视图自动刷新起始时间，保留未使用
flag	integer	物化视图标志
baseindkey	int2vector	with primary key 时，物化视图日志的基表主键
mvindkey	int2vector	物化视图的主键
mvrepeatinterval	pg_node_tree	自动刷新间隔的 nodeToString 表示
options	text	物化视图选项，保留未使用

1.1.2.45　sys_namespace

`sys_namespace` 视图存储名字空间。名字空间是 SQL 模式之下的结构：每个名字空间拥有一个独立的表、类型等的集合，且其中没有名字冲突。

表 1.1.60: sys_namespace 列

名称	类型	描述
oid	oid	行标识符
nspname	name	名字空间的名字
nspowner	oid	名字空间的拥有者
nspflags	integer	SCHEMA 的特定标志
nspacl	aclitem[]	访问权限

1.1.2.46　sys_object_status

`sys_object_status` 视图存储数据库对象的状态信息。在系统视图 sys_object_status 中，使用 classid 和 objid 两组 OID 来确定一个数据库的唯一对象，再加以 objsubid 用于区分该对象是否为对象的子对象，当主对象为包或者对象类型时，该对象的 objsubid 为 0 则表示该对象为对象声明，当 objsubid 为 1 时，则表示该对象为对象的 body，status 列则用于记录该对象的具体状态。

表 1.1.61: sys_object_status 列

名称	类型	含义
classid	oid	相关对象所属表的 OID
objid	oid	相关对象的 OID
objsubid	integer	标记对象的子对象，比如表的列，包体或者对象类型体
status	integer	相关对象的状态，包括有效、无效或者 ddl _error

1.1.2.47　sys_opclass

`sys_opclass` 视图定义索引访问方法的操作符类。每一个操作符类定义了一种特定数据类型和一种特定索引访问方法的索引列的语义。一个操作符类实际上指定了一个特定的操作符族可以用于一个特定可索引列数据类型。该族中可用于索引列的操作符能够接受该列的数据类型作为它们的左操作数。

表 1.1.62: sys_opclass 列

名称	类型	描述
oid	oid	行标识符
opcmethod	oid	操作符类所属的索引访问方法
opcname	name	操作符类的名称
opcnamespace	oid	操作符类所属的名字空间
opcowner	oid	操作符类的拥有者
opcfamily	oid	包含此操作符类的操作符族
opcintype	oid	操作符类索引的数据类型
opcdefault	boolean	如果此操作符类为 opcintype 的默认值则为真
opckeytype	oid	存储在索引中的数据的类型，如果值为 0 表示与 opcintype 相同

一个操作符类的 `opcmethod` 必须匹配包含它的操作符族的 `opfmethod`。而且，对于任何给定的 `opcmethod` 和 `opcintype` 组合，只有不超过一个 `sys_opclass` 行的 `opcdefault` 值为真。

1.1.2.48　sys_operator

`sys_operator` 存储关于操作符的信息。

表 1.1.63: sys_operator 列

名称	类型	描述
oid	oid	行标识符
oprname	name	操作符的名称
oprnamespace	oid	操作符所属的名字空间的 OID

-续表-

名称	类型	描述
oprowner	oid	操作符的拥有者
oprkind	char	b = 中缀（“并”） l = 前缀（“左”） r = 后缀（“右”）
oprcanmerge	boolean	该操作符是否支持归并连接
oprcanhash	boolean	该操作符是否支持哈希连接
oprleft	oid	左操作数类型
oprright	oid	右操作数类型
oprresult	oid	结果类型
oprcom	oid	该操作符的交换子（如果存在）
oprnegate	oid	该操作符的否定（如果存在）
oprflags	int	记录该操作符的一些常用属性
oprcode	regproc	实现该操作符的函数
oprrest	regproc	该操作符的限制选择度估算函数
oprjoin	regproc	该操作符的连接选择度估算函数

未用的列包含零值。例如，一个前缀操作符的 `oprleft` 为 0。

1.1.2.49　sys_opfamily

`sys_opfamily` 视图定义了操作符族。每一个操作符族是操作符和相关支持例程的集合，支持例程用于实现一个特定索引访问方法的语义。此外，按照访问方法指定的某种方式，一个族内的操作符都是“兼容的”。操作符族概念允许在索引中使用跨数据类型操作符，并可以使用访问方法语义的知识推导出。

表 1.1.64: sys_opfamily 列

名称	类型	描述
oid	oid	行标识符

-续表-

名称	类型	描述
opfmethod	oid	操作符族适用的索引访问方法
opfname	name	操作符族的名字
opfnamespace	oid	操作符族所属的名字空间
opfowner	oid	操作符族的拥有者

定义操作符族的主要信息不在它的 `sys_opfamily` 行，而是在相关的sys_amop 、sys_amproc 和sys_opclass 行中。

1.1.2.50　sys_package

`sys_package` 记录包对象的元信息、包中子对象的信息和依赖关系。

表 1.1.65: sys_package

名称	类型	描述
oid	oid	对象所在系统表的 OID
pkgname	name	包名
pkgnamespace	oid	模式
pkgowner	oid	属主
pkgsecdef	boolean	以创建者/访问者的身份执行存储过程
pkgspecdata	text	包规范数据拼接的字符串
pkgspecvars	oid[]	包规范中变量的 OID 数组
pkgspectypes	oid[]	包规范中类型的 OID 数组
pkgspecfuncs	oid[]	解析后的包规范功能
pkgspecsrc	text	包规范原字符串
pkgbodydata	text	包体数据拼接的字符串
pkgbodyvars	oid[]	包体中变量的 OID 数组

-续表-

名称	类型	描述
pkgbodytypes	oid[]	包体中类型的 OID 数组
pkgbodyfuncs	oid[]	解析后的包函数
pkgbodysrc	text	包体原字符串
pkginitsrc	text	初始化代码
pkgacl	aclitem[]	访问控制列表

1.1.2.51　sys_partitioned_table

`sys_partitioned_table` 视图存放有关表如何被分区的信息。子分区信息参见表sys_subpartition_table 。

表 1.1.66: sys_partitioned_table 列

名称	类型	描述
partrelid	oid	这个分区表的 sys_class 项的 OID
partstrat	char	分区策略: h = 哈希分区表 l = 列表分区表， r = 范围分区表
partnatts	smallint	分区键中的列数
partdefid	oid	这个分区表的默认分区的 sys_class 项的 OID，如果这个分区表没有默认分区则为零
partdiffnum	integer	列顺序与父表不一致的子表的数量
partattrs	int2vector	这是一个长度为 partnatts 值的数组，它指示哪些表列是分区键的组成部分 例如，值 1 3 表示第一个和第三个表列组成了分区键。 这个数组中的零表示对应的分区键列是一个表达式而不是简单的列引用
partclass	oid[]	对于分区键中的每一个列，这个域包含要使用的操作符类的 OID

-续表-

名称	类型	描述
partcollation	oid[]	对于分区键中的每一个列，这个域包含要用于分区的排序规则的 OID，如果该列不是一种可排序数据类型则为零
partexprs	sys_node_tree	非简单列引用的分区键列的表达式树（以 nodeToString() 的表达方式） 这是一个列表，partattrs 中每一个零项都有一个元素。如果所有分区键列都是简单列引用，则这个域为空
parttablespace	oid[]	分区所在表空间的 OID
intervalexpr	pg_node_tree	interval 分区表的间隔值

1.1.2.52　sys_pkgitem

`sys_pkgitem` 视图主要用于引用包变量和类型编译时，对相关对象的合法性检查。

表 1.1.67: sys_pkgitem

名称	类型	描述
TABLEOID	OID	包含该行的表的对象标识符（OID）
CMAX	CID	删除事务的标识符（事务 ID）
XMAX	XID	记录删除或更新该行版本的标识符（事务 ID）
CMIN	CID	插入事务中的命令标识符（从零开始计数）
XMIN	XID	该行版本插入事务的标识符（事务 ID）
CTID	TID	该行版本在表中的物理位置
oid	oid	对象所在系统表的 OID
pvname	name	包中对象的名称
pkgoid	oid	包的 OID

-续表-

名称	类型	描述
pvtypoid	oid	包中对象的类型
pvtypmod	integer	包中对象的类型模式
pvtypcollation	oid	包中对象的类型排序规则
pvtype	char	包中对象的类别（变量、RECORD、游标、类型）
pvconstant	boolean	如果是变量，那么指出该对象是否为常量（其它对象此值都是 FALSE）
pvnotnull	boolean	该对象是否为 notnull 对象
pvparent	oid	若该对象为包内对象的子对象，pvparent 为其父对象在当前表中的 oid
ptidxtypoid	oid	关联数组类型的索引类型 oid
ptidxtypmod	integer	关联数组类型的索引类型的模式
ptidxtypcollation	oid	关联数组类型的索引类型的排序规则
pvnargs	integer	包中对象的参数个数， 0 表示没有参数
pvncols	integer	包中游标的信息 -1 表示没有返回值 -2 表示返回 RECORD 类型 -3 表示这个游标没有定义语句
pvsrc	text	包中对象的定义语句
pvargnames	text[]	包中游标的参数名数组
pvargdefaults	text[]	包中游标的参数默认值的数组

1.1.2.53　sys_pltemplate

`sys_pltemplate` 视图存储了过程语言的“模板”信息。一个语言的模板允许我们在一个特定数据库中以一个简单的 `CREATE LANGUAGE` 命令创建语言，而不需要指定实现细节。

和大部分系统视图不同，`sys_pltemplate` 是在集簇的所有数据库之间共享的：在一个集簇中只有一份

sys_pltemplate 拷贝，而不是每个数据库一份。这使得在每个需要的数据库中都可以访问该信息。

表 1.1.68: sys_pltemplate 列

名称	类型	描述
tmplname	name	该模板适用的语言名字
tmpltrusted	boolean	如果语言被认为是可信的则为真
tmpldbacreate	boolean	如果语言可以被一个数据库拥有者创建则为真
tmplhandler	text	调用处理函数的名字
tmplinline	text	匿名阻塞处理函数的名字，如果没有则为空
tmplvalidator	text	验证函数的名字，如果没有则为空
tmpllibrary	text	实现语言的共享库的路径
tmplacl	aclitem[]	模板的访问权限（并未真正使用）

目前任何命令都不能操纵过程语言模板。要改变内建信息，超级用户必须使用普通的 INSERT、DELETE 或 UPDATE 命令修改该表。

备注:

在 KingbaseES 的某个未来版本的发布中，sys_pltemplate 很有可能会被移除，而这些关于过程语言的知识可能会保持在它们相应的扩展安装脚本中。

1.1.2.54　sys_policies

视图 sys_policies 记录有关数据库中行级安全性策略的信息。

表 1.1.69: sys_policies 列

名称	类型	描述
schemaname	name	包含策略所在表的模式的名称
tablename	name	策略所在表的名称

-续表-

名称	类型	描述
policyname	name	策略名称
permissive	text	策略是宽容性的还是限制性的
roles	name[]	这个策略适用的角色
cmd	text	这个策略适用的命令类型
qual	text	作为这个策略适用的查询的安全屏障条件增加的表达式
with_check	text	作为尝试向该表增加行的查询的 WITH CHECK 条件增加的表达式

1.1.2.55　sys_policy

`sys_policy` 视图存储着表的行级安全性策略。一个策略包括它适用于的命令种类（可能适用于所有命令）、它适用于的角色、被作为安全屏障条件增加到包括该表的查询的表达式以及被作为 `WITH CHECK` 选项增加到尝试向表增加新记录的查询的表达式。

表 1.1.70: sys_policy 列

名称	类型	描述
oid	oid	行标识符
polname	name	策略的名称
polrelid	oid	策略适用的表
polcmd	char	策略适用的命令类型： r 表示 SELECT a 表示 INSERT w 表示 UPDATE d 表示 DELETE * 表示所有命令类型
polpermissive	boolean	策略是宽容性的还是限制性的
polroles	oid[]	策略适用的角色

-续表-

名称	类型	描述
polqual	pg_node_tree	被作为安全屏障条件增加到使用该表的查询的表达式树
polwithcheck	pg_node_tree	被作为 WITH CHECK 条件增加到尝试向表增加行的查询的表达式树

备注:

存储在 `sys_policy` 中的策略只有在它们所适用的表的 `sys_class.relrowsecurity` 被设置时才其作用。

1.1.2.56 sys_prepared_statements

`sys_prepared_statements` 视图显示在当前会话中可用的所有预备语句。关于预备语句详见 PREPARE(SQLServer) 。

`sys_prepared_statements` 为每一个预备语句包含一行。当一个新的预备语句被创建时在此视图中会增加一行，反之当一个预备语句被释放时在此视图中会删除一行（例如，通过 `DEALLOCATE` 命令）。

表 1.1.71: sys_prepared_statements 列

名称	类型	描述
name	text	预备语句的标识符
statement	text	客户端提交用于创建此预备语句的查询语句。 对于通过 SQL 创建的预备语句，这里是由客户端提交的 PREPARE 语句 对于通过前端后端协议创建的预备语句这里是预备语句本身的文本
prepare_time	timestamp with time zone	预备语句被创建的时间
parameter_types	regtype[]	预备语句期望的参数类型，以一个 regtype 数组的形式。 这个数组中一个元素所对应的 OID 可通过将 regtype 值转换为 oid 获得

-续表-

名称	类型	描述
from_sql	boolean	如果预备语句通过 SQL 命令 PREPARE 创建，则为 true； 如果预备语句通过前端后端协议创建，则为 false

`sys_prepared_statements` 视图为只读。

参考

- DEALLOCATE(SQLServer)
- DEALLOCATE(Oracle)

1.1.2.57　sys_prepared_xacts

视图 `sys_prepared_xacts` 显示关于两阶段提交（详见 `PREPARE TRANSACTION` 当前准备好事务的信息）。

`sys_prepared_xacts` 为每一个预备事务包含一行。当事务被提交或回滚时，相应的项将被移除。

表 1.1.72: sys_prepared_xacts 列

名称	类型	描述
transaction	xid	预备事务的数字事务标识符
gid	text	分配给事务的全局标识符
prepared	timestamp with time zone	此事务为提交准备好的时间
owner	name	执行此事务的用户名
database	name	执行此事务所在数据库的名字

当 `sys_prepared_xacts` 视图被访问时，内部事务管理器数据结构被暂时地锁住，并为视图的显示产生一个副本。这确保了视图中是一组一致的结果，并且不会阻塞普通操作。不管怎样，当此视图被频繁访问时，会对数据库性能有所影响。

参考

- PREPARE TRANSACTION(Oracle)
- PREPARE TRANSACTION(SQLServer)

1.1.2.58　sys_privilege

`sys_privilege` 视图查询用户是否拥有 SYSBACKUP 权限。更多信息请参考备份恢复权限 。

表 1.1.73: sys_privilege 列

名称	类型	描述
userid	oid	用户 OID
sysbackup	boolean	是否拥有 sysbackup 权限
readonly	boolean	是否只读
objtype	char	目标类型

1.1.2.59　sys_proc

`sys_proc` 视图存放有关函数、过程、聚集函数以及窗口函数（共称为例程）的信息。

表 1.1.74: sys_proc 列

名称	类型	描述
oid	oid	行标识符
proname	name	函数的名字
pronamespace	oid	函数所属的名字空间的 OID
proowner	oid	函数的拥有者
proexecuteasprincipalid	oid	指定用户 OID，记录 SQLServer 语法 execute as 语法指定的信息
prolang	oid	实现语言或该函数的调用接口
procost	real	估计的执行代价（以 cpu_operator _cost 为单位），如果 proretset 为真，这是每行返回的代价
prorows	real	估计的结果行数量（如果 proretset 为假，该值为 0）
provariadic	oid	可变数组参数的元素的数据类型，如果函数没有可变参数则为 0

-续表-

名称	类型	描述
prosupport	regproc	对于该函数可选的计划器支持函数
prokind	char	f 表示普通函数，p 表示过程， a 表示聚集函数，w 表示窗口函数， P 表示 PIPELINED 函数
prosecdef	boolean	函数是一个安全性定义者（即，一个“setuid”函数）
proleakproof	boolean	该函数没有副作用。 除了通过返回值，没有关于参数的信息被传播。 任何会抛出基于其参数值的错误信息的函数都不是泄露验证的
proisstrict	boolean	当任意调用函数为空时，函数是否会返回空值。在那种情况下函数实际上根本不会被调用。 非“strict”函数必须准备好处理空值输入
proretset	boolean	函数是否返回一个集合（即，指定数据类型的多个值）
provolatile	char	provolatile 说明函数是仅仅只依赖于它的输入参数，还是会被外部因素影响 值 i 表示“不变的”函数，它对于相同的输入总是输出相同的结果。 值 s 表示“稳定的”函数，它的结果（对于固定输入）在一次扫描内不会变化 值 v 表示“不稳定的”函数，它的结果在任何时候都可能变化（使用 v 页表示函数具有副作用，所以对它们的调用无法得到优化）

-续表-

名称	类型	描述
proparallel	char	proparallel 说明该函数在并行模式下是否能安全地运行 对于能在并行模式下不受限制安全运行的函数，这列是 s 对于可以在并行模式下运行但是只限于由并行分组的领导者执行的函数，这列是 r 对于在并行模式中不安全的函数，这列是 u，这种函数的存在会强制一个顺序执行计划
pronargs	integer	输入参数的个数
pronargdefaults	integer	具有默认值的参数个数
prorettype	oid	返回值的数据类型
proargtypes	oid[]	一个函数参数的数据类型的数组 这只包括输入参数（含 INOUT 和 VARIADIC 参数），因此也表现了函数的调用特征
proallargtypes	oid[]	一个函数参数的数据类型的数组 这包括所有参数（含 OUT 和 INOUT 参数）。 但是，如果所有参数都是 IN 参数，这个域将为空。 注意下标是从 1 开始，然而由于历史原因 proargtypes 的下标是从 0 开始

-续表-

名称	类型	描述
proargmodes	char[]	一个函数参数的模式的数组 编码为： i 表示 IN 参数 o 表示 OUT 参数 b 表示 INOUT 参数 v 表示 VARIADIC 参数， t 表示 table 参数。 如果所有的参数都是 IN 参数，这个域为空 注意这里的下标对应着 proallargtypes 而不是 proargtypes 中的位置
proargnames	text[]	一个函数参数的名字的数组 没有名字的参数在数组中设置为空字符串 如果没有一个参数有名字，这个域为空 注意这里的下标对应着 proallargtypes 而不是 proargtypes 中的位置
proargdefaults	pg_node_tree	默认值的表达式树（按照 nodeToString() 的表现方式） 这是一个 pronargdefaults 元素的列表，对应于最后 N 个 input 参数（即最后 N 个 proargtypes 位置） 如果没有一个参数具有默认值，这个域为空
protrftypes	oid[]	要在其上应用转换的数据类型的 OID
prosrc	text	这个域告诉函数处理者如何调用该函数 它可能是针对解释型语言的真实源码、一个符号链接、一个文件名或任何其他东西，这取决于实现语言/调用规范
probin	text	关于如何调用函数的附加信息。其解释是与语言相关的

-续表-

名称	类型	描述
proconfig	text[]	函数对于运行时配置变量的本地设置值
proacl	aclitem[]	访问权限

对于编译好的函数，包括内建的和动态载入的，`prosrc` 包含了函数的 C 语言名字（链接符号）。对于所有其他已知的语言类型，`prosrc` 包含函数的源码文本。除了对于动态载入的 C 函数之外，`probin` 是不被使用的。对于动态载入的 C 函数，它给定了包含该函数的共享库文件的名称。

参考

- CREATE FUNCTION(Oracle)
- CREATE FUNCTION(MySQL)
- CREATE FUNCTION(SQLServer)

1.1.2.60　sys_publication

`sys_publication` 视图包含数据库中创建的所有 publication。

表 1.1.75: sys_publication 列

名称	类型	描述
oid	oid	行标识符
pubname	name	publication 的名称
pubowner	oid	publication 的拥有者
puballtables	boolean	如果为真，这个 publication 自动包括数据库中的所有表，包括未来将会创建的任何表
pubinsert	boolean	如果为真，为 publication 中的表复制 INSERT 操作
pubupdate	boolean	如果为真，为 publication 中的表复制 UPDATE 操作
pubdelete	boolean	如果为真，为 publication 中的表复制 DELETE 操作
pubtruncate	boolean	如果为真，为 publication 中的表复制 TRUNCATE 操作

1.1.2.61　sys_publication_rel

`sys_publication_rel` 视图包含数据库中关系和 publication 之间的映射。这是一种多对多映射。这些信息对用户更加友好的视图请参考sys_publication_tables 。

表 1.1.76: sys_publication_rel 列

名称	类型	描述
oid	name	行标识符
prpubid	oid	对 publication 的引用
prrelid	oid	对关系的引用

1.1.2.62　sys_publication_tables

视图 `sys_publication_tables` 提供 publication 与其所包含的表之间的映射信息。和底层的视图 `sys_publication_rel` 不同，这个视图展开了定义为 `FOR ALL TABLES` 的 publication，这样对这类 publication 来说，每一个合格的表都有一行。

表 1.1.77: sys_publication_tables 列

名称	类型	描述
pubname	name	publication 名称
schemaname	name	包含表的方案名称
tablename	name	表名

1.1.2.63　sys_query_mapping

`sys_query_mapping` 视图存储用户创建的 Query Mapping 规则。

表 1.1.78: sys_query_mapping 列

名称	类型	描述
oid	oid	行标识符
qmname	name	规则名称
qmenable	boolean	该规则是否被启用
qmfromsql	text	用于匹配的 SQL 字符串
qmreplacedsql	text	用于替换的 SQL 字符串
qmcompiledfromsql	text	匹配 SQL 对应的查询树
qmcompiledreplacesql	text	替换 SQL 对应的查询树
qmlevel	smallint	规则的级别: 0 = TEXT 1 = SEMANTICS

1.1.2.64　sys_range

`sys_range` 视图存储关于范围类型的信息。它是类型在sys_type 中项的补充。

表 1.1.79: sys_range 列

名称	类型	描述
rngtypid	oid	范围类型的 OID
rngsubtype	oid	该范围类型的元素类型（子类型）的 OID
rngcollation	oid	用于范围比较的排序规则的 OID，如果没有则为 0
rngsubopc	oid	用于范围比较的子类型的操作符类的 OID
rngcanonical	regproc	将一个范围值转换为规范形式的函数的 OID，如果没有则为 0
rngsubdiff	regproc	以双精度返回两个元素值不同的函数的 OID，如果没有则为 0

`rngsubopc`（加上 `rngcollation`，如果元素类型是可排序的）决定了被该范围类型所使用的排序顺序。`rngcanonical` 用于离散类型的元素类型。`rngsubdiff` 是可选的，但是提供它可以提高范围类型上的 GiST 索引性能。

1.1.2.65　sys_replication_origin

`sys_replication_origin` 视图包含所有已创建的复制源。和大部分系统视图不同，`sys_replication_origin` 在一个集簇的所有数据库之间共享：每个集簇只有一份 `sys_replication_origin` 拷贝，而不是每个数据库一份。

表 1.1.80: sys_replication_origin 列

名称	类型	描述
roident	oid	一个集簇范围内唯一的复制源标识符。应该绝不会脱离系统
roname	text	外部的由用户定义的复制源名称

1.1.2.66　sys_replication_origin_status

`sys_replication_origin_status` 视图包含有关一个特定源已经重放了多少的信息。

表 1.1.81: sys_replication_origin_status 列

名称	类型	描述
local_id	oid	内部的节点标识符
external_id	text	外部的节点标识符
remote_lsn	pg_lsn	源节点的 LSN，到这个位置的数据都已经被复制
local_lsn	pg_lsn	这个节点的 LSN，remote_lsn 已经被复制到这里。 使用异步提交时，在将数据持久化到磁盘前用它来刷入提交记录

1.1.2.67　sys_replication_slots

`sys_replication_slots` 视图提供了当前存在于数据库集簇上的所有复制槽的列表，其中也包括复制槽的当前状态。

表 1.1.82: sys_replication_slots 列

名称	类型	描述
slot_name	name	一个唯一的、集簇范围内的复制槽标识符
plugin	name	包含这个逻辑槽正在使用的输出插件的共享对象基础名称，这个列对于物理槽为空值
slot_type	text	槽类型 - physical （物理）或者 logical （逻辑）
datoid	oid	与这个槽相关的数据库的 OID，或者为空值。 只有逻辑槽具有相关的数据库
database	name	与这个槽相关的数据库的名称，或者为空值。 只有逻辑槽具有相关的数据库
temporary	boolean	如果这是一个临时复制槽则为真。 临时槽不会被保存在磁盘上并且会在出错或会话结束时自动被删除掉
active	boolean	如果这个槽当前正在被使用则为真
active_pid	integer	如果槽当前正在被使用，则记录使用这个槽的会话的进程 ID 如果槽没有被使用则为 NULL
xmin	xid	这个槽要需要数据库保留的最旧事务。 VACUUM 不能移除被其后续事务删除的元组
catalog_xmin	xid	这个槽要需要数据库保留的影响系统表的最旧事务 VACUUM 不能移除被其后续事务删除的表元组

-续表-

名称	类型	描述
restart_lsn	pg_lsn	可能仍被这个槽的消费者要求的最旧 WAL 地址（LSN），并且因此不会在检查点期间自动被移除 如果这个槽的 LSN 从未被保留过，则为 NULL
confirmed_flush_lsn	pg_lsn	代表逻辑槽的消费者已经确认接收数据到什么位置的地址（LSN ） 比这个地址更旧的数据已经不再可用。 对于物理槽这里是 NULL

1.1.2.68　sys_rewrite

`sys_rewrite` 视图存储对于表和视图的重写规则。

表 1.1.83: sys_rewrite 列

名称	类型	描述
oid	oid	行标识符
rulename	name	规则名称
ev_class	oid	使用该规则的表
ev_type	char	使用该规则的事件类型： 1 = SELECT 2 = UPDATE 3 = INSERT 4 = DELETE
ev_enabled	char	控制在哪种 session_replication_role 模式中触发该规则 O = 规则在“origin”和“local”模式触发， D = 规则被禁用， R = 规则在“replica”模式触发， A = 规则总是被触发
is_instead	boolean	为真表示是一个 INSTEAD 规则

-续表-

名称	类型	描述
ev_qual	pg_node_tree	规则条件的表达式树（按照 nodeToString() 的表现形式）
ev_action	pg_node_tree	规则动作的查询树（按照 nodeToString() 的表现形式）

备注:

如果一个表在这个视图中有任何规则，`sys_class.relhasrules` 必须为真。

1.1.2.69　sys_rules

视图 `sys_rules` 提供对查询重写规则的信息访问。

表 1.1.84: sys_rules 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	规则适用的表名
rulename	name	规则名
definition	text	规则定义（创建命令的重构）

`sys_rules` 视图排除了视图和物化视图的 `ON SELECT` 规则，它们可以在 `sys_views` 和 `sys_matviews` 中找到。

1.1.2.70　sys_seclabel

`sys_seclabel` 视图存储数据库对象上的安全标签。安全标签可以通过 `SECURITY LABEL` 命令操纵。简单的查看安全标签方法请见sys_seclabels 。

同时请见sys_shseclabel ，它对集簇共享的数据库对象的安全标签执行相似的功能。

表 1.1.85: sys_seclabel 列

名称	类型	描述
objoid	oid	该安全标签依附的对象的 oid
classoid	oid	该对象所出现的系统表的 oid
objsubid	integer	对于一个在表列上的安全标签，这将是列号（objoid 和 classoid 指表本身） 对于所有其他对象类型，本列为 0
provider	text	与该标签相关的标签提供者
label	text	应用于该对象的安全标签

参考

- SECURITY LABEL(Oracle)
- SECURITY LABEL(SQLServer)

1.1.2.71　sys_seclabels

视图 `sys_seclabels` 提供对安全标签的信息访问。它是sys_seclabel 视图的一个便于查询的版本。

表 1.1.86: sys_seclabels 列

名称	类型	描述
objoid	oid	安全标签所属对象的 OID
classoid	oid	对象出现的系统表的 OID
objsubid	integer	对于一个表列上的安全标签，这里是列号（objoid 和 classoid 指表本身） 对于所有其他对象类型，此列为 0
objtype	text	此标签应用的对象类型，以文本方式
objnamespace	oid	如果适用，为此对象的名字空间的 OID；否则为空
objname	text	此标签应用的对象名，以文本形式
provider	text	与此标签相关的标签提供者

-续表-

名称	类型	描述
label	text	应用于此对象的安全标签

1.1.2.72　sys_sequence

`sys_sequence` 视图包含有关序列的信息。一些序列的信息（例如名称和方案）放在 `sys_class` 中。

表 1.1.87: sys_sequence 列

名称	类型	描述
seqrelid	oid	这个序列的 sys_class 项的 OID
seqtypid	oid	序列的数据类型
seqstart	int16	序列的起始值
seqincrement	int16	序列的增量值
seqmax	int16	序列的最大值
seqmin	int16	序列的最小值
seqcache	bigint	序列的缓冲尺寸
seqcycle	boolean	序列是否循环

1.1.2.73　sys_sequences

视图 `sys_sequences` 记录对数据库中每个序列的信息的访问信息。

表 1.1.88: sys_sequences 列

名称	类型	描述
schemaname	name	包含序列的方案名

-续表-

名称	类型	描述
sequencename	name	序列的名称
sequenceowner	name	序列的拥有者的名称
data_type	regtype	序列的数据类型
start_value	int16	序列的起始值
min_value	int16	序列的最小值
max_value	int16	序列的最大值
increment_by	int16	序列的增量值
cycle	boolean	序列是否循环
cache_size	bigint	序列的缓冲尺寸
last_value	numeric	最后一个被写入到磁盘的序列值。 如果使用了缓冲，这个值可能比从序列中取出的最后一个值大。 如果还没有从该序列读取过，则为空。 此外，如果当前用户没有该序列上的 USAGE 或 SELECT 特权，则这个值为空

1.1.2.74　sys_session

视图 `sys_session` 记录当前数据库系统的会话信息。

表 1.1.89: sys_session 列

名称	类型	描述
sess_id	integer	会话 id
curr_sch	character varying(32 byte)	会话所属模式
usename	name	会话所属用户名
client_ip	inet	会话 ip 地址

-续表-

名称	类型	描述
create_start	timestamp with time zone	会话创建时间
current_query	text	会话 sql
status	text	会话状态

1.1.2.75　sys_settings

视图 `sys_settings` 记录对服务器上运行时参数的访问信息。它本质上是 `SHOW` 和 `SET` 命令的可替换接口。它还提供了 `SHOW` 不能提供的关于每一个参数的一些限制，例如最大值和最小值。

表 1.1.90: sys_settings 列

名称	类型	描述
name	text	运行时配置参数名
setting	text	参数的当前值
unit	text	参数的隐式单元
category	text	参数的逻辑组
short_desc	text	参数的简短描述
extra_desc	text	附加的参数的详细描述
context	text	要求设置此参数值的上下文
vartype	text	参数类型（boolean 、enum 、integer 、real 或 string ）
source	text	当前参数值的来源
min_val	text	参数的最小允许值（对非数字值为空）
max_val	text	参数的最大允许值（对非数字值为空）
enumvals	text[]	一个枚举参数的允许值（对非数字值为空）
boot_val	text	如果参数没有被别的其他设置，此列为在服务器启动时设定的参数值
reset_val	text	在当前会话中，RESET 将会设置的参数值

-续表-

名称	类型	描述
sourcefile	text	当前值被设置的配置文件（空值表示从非配置文件的其他来源设置，由不是超级用户也不是 sys_read_all_settings 成员的用户检查时也为空值） 在配置文件中使用 include 指令时有用
sourceline	integer	当前值被设置的配置文件中的行号（空值表示从非配置文件的其他来源设置，由不是超级用户也不是 sys_read_all _settings 成员的用户检查时也为空值）
pending_restart	boolean	如果配置文件中修改了该值但需要重启，则为 true ，否则为 false

对于 `context` 有多种可能的取值。为了降低改变设置的难度，它们是：

`internal`
这些设置不能被直接修改，它们反映了内部决定的值。某些可能在使用不同配置选项重建系统时或者改变 `initdb` 的选项时可以调整。

`kingbase`
这些设置只能在服务器启动时应用，因此任何修改都需要重启服务器。这些设置的值通常都存储在 `kingbase.conf` 文件中，或者在启动服务器时通过命令行传递。当然，具有更低 `context` 类型的设置也可以在服务器启动时间被设置。

`sighup`
对于这些设置的修改可以在 `kingbase.conf` 中完成并且不需要重启服务器。发送一个 SIGHUP 信号给 kingbase 会导致它重新读取 `kingbase.conf` 并应用修改。Kingbase 将会把 SIGHUP 信号传递给它的孩子进程，这样它们也会获得新的值。

`superuser-backend`
对于这些设置的更改可以在 `kingbase.conf` 中进行而无需重启服务器。也可以在连接请求包（例如通过 libkci 的 `KINGBASE_OPTIONS` 环境变量）中为一个特定的会话设定它们，但是只有在连接用户是超级用户时才能这样做。如果，在会话启动后这些设置就不会改变。如果在 `kingbase.conf` 改变了它们，向 kingbase 发送一个 SIGHUP 信号让 kingbase 重新读取 `kingbase.conf`。新的值将只会影响后续启动的会话。

`backend`
对于这些设置的修改可以在 `kingbase.conf` 中完成并且不需要重启服务器。它们也可以在一个连接请求包（例如，通过 libkci 的 `KINGBASE_OPTIONS` 环境变量）中为一个特定会话设置，任何用户都可以为这个会话做这种修改。然而，这些设置在会话启动后永不变化。如果在 `kingbase.conf` 中修改它们，可以向 kingbase 发送一个 SIGHUP 信号让它重读 `kingbase.conf`。新值只会影响后续启动的会话。

`superuser`
这些设置可以从 `kingbase.conf` 设置，或者在会话中用 `SET` 命令设置。仅当没有通过 `SET` 设置会话本地值时，

kingbase.conf 中的改变才会影响现有的会话。

user
这些设置可以从 kingbase.conf 设置，或者在会话中用 SET 命令设置。任何用户都被允许修改它们的会话本地值。仅当没有通过 SET 设置会话本地值时，kingbase.conf 中的改变才会影响现有的会话。

sys_settings 视图不能被插入或者从中删除，但是它可以被更新。在 sys_settings 的一行上的一个 UPDATE 等价于在该参数上执行一个 SET 命令。修改将只会影响当前会话使用的值。如果一个 UPDATE 在一个后来中断的事务中被发出，UPDATE 命令的效果也会随着事务的回滚而消失。一旦所在的事务被提交，效果将一直保持到会话结束，除非有其他 UPDATE 或 SET 重新修改它。

1.1.2.76　sys_shdepend

sys_shdepend 视图记录数据库对象和共享对象之间的依赖关系，例如角色。这些信息使得 KingbaseES 可以确保对象在被删除时没有被其他对象引用。

另请参阅sys_depend ，它对单个数据库中对象之间的依赖提供了相似的功能。

与大部分其他系统视图不同，sys_shdepend 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 sys_shdepend 的拷贝，而不是每个数据库一份。

表 1.1.91: sys_shdepend 列

名称	类型	描述
dbid	oid	依赖者对象所在的数据库 OID，如果是一个共享对象则值为 0
classid	oid	依赖者对象所在的系统表的 OID
objid	oid	依赖者对象的 OID
objsubid	integer	对于一个表列，这将是列号（objid 和 classid 指向表本身）。 对于所有其他对象类型，该列值为 0。
refclassid	oid	被引用对象所在的系统目录的 OID（必须是一个共享的表）
refobjid	oid	被引用对象的 OID
deptype	char	定义该依赖关系的特定语义的代码，见表后的说明

在所有情况下，一个 sys_shdepend 项表明被引用对象不能在没有删除其依赖对象的情况下被删除。但是，其中也有多种依赖类型，由 deptype 标识：

SHARED_DEPENDENCY_OWNER (o)

被引用对象（必须是一个角色）是依赖对象的拥有者。

SHARED_DEPENDENCY_ACL (a)
被引用对象（必须是一个角色）在依赖对象的 ACL（访问控制列表，即权限列表）中被提到。(不会为对象的拥有者创建一个 SHARED_DEPENDENCY_ACL 项，因为拥有者将会有一个 SHARED_DEPENDENCY_OWNER 项。）

SHARED_DEPENDENCY_POLICY (r)
作为一个依赖策略对象的目标被引用的对象（必须是一个角色）。

SHARED_DEPENDENCY_PIN (p)
没有依赖对象，这种类型的项是一个信号，用来指示系统本身依赖于被引用对象，并且因此该对象必须永远不能被删除。这种类型的项只能被 initdb 创建。这种项中关于依赖对象的列值都为 0。

未来可能会需要其他的依赖类型。特别要注意的是在当前定义中只支持角色作为被引用对象。

1.1.2.77　sys_shdescription

sys_shdescription 视图存储共享数据库对象的可选描述（注释）。描述可以通过 COMMENT 命令操作，并且可以使用 ksql 的\d 命令来查看。

另请参阅sys_description ，它对单个数据库中对象之间的依赖提供了相似的功能。

与大部分其他系统视图不同，sys_shdescription 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 sys_shdescription 的拷贝，而不是每个数据库一份。

表 1.1.92: sys_shdescription 列

名称	类型	描述
objoid	oid	该描述所属的对象的 OID
classoid	oid	该对象所在系统目录的 OID
name	name	扩展属性名
description	text	作为该对象描述的任意文本

参考

- COMMENT(Oracle)
- COMMENT(SQLServer)

1.1.2.78　sys_shseclabel

`sys_shseclabel` 视图存储共享数据库对象上的安全标签。安全标签可以通过 `SECURITY LABEL` 命令操纵。更简单的查看安全标签的方式请见sys_seclabels 。

另请参阅sys_seclabel ，它对单个数据库中对象的安全标签提供了相似的功能。

与大部分其他系统视图不同，[sys_shseclabel](/ docs/ reference/ 03_database/ 01-系统视图/ kes_views/ metadata_views/sys_shseclabel.md 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 `sys_shseclabel` 的拷贝，而不是每个数据库一份。

表 1.1.93: sys_shseclabel 列

名称	类型	描述
objoid	oid	该安全标签所属对象的 OID
classoid	oid	对象所属系统表的 OID
provider	text	与此标签关联的标签提供者
label	text	应用到该对象的安全标签

参考

- SECURITY LABEL(Oracle)
- SECURITY LABEL(SQLServer)

1.1.2.79　sys_subpartition_table

表 `sys_subpartition_table` 存储子分区表信息。分区表信息可参考表sys_partitioned_table 。

表 1.1.94: sys_subpartition_table 列

名称	类型	描述
partrelid	oid	分区表 oid
partstrat	char	子分区策略： h = 哈希分区， l = 列表分区， r = 范围分区
partnatts	smallint	子分区键中的列个数

-续表-

名称	类型	描述
partdefid	oid	这个分区表的默认分区的 sys_class 项的 OID，如果这个分区表没有默认分区则为 0
partattrs	int2vector	子分区键的列编号。 如果值为 0 代表分区键是一个表达式
partclass	oid[]	对于子分区键中的每一个列，这个域包含要使用的操作符类的 OID
partcollation	oid[]	对于子分区键中的每一个列，这个域包含要用于分区的排序规则的 OID，如果该列不是一种可排序数据类型则为 0
partexprs	pg_node_tree	非简单列引用的子分区键列的表达式树（以 nodeToString() 的表达方式） 这是一个列表，partattrs 中每一个零项都有一个元素 如果所有子分区键列都是简单列引用，则这个域为空
partspec	pg_node_tree	分区说明列表

1.1.2.80　sys_subscription

`sys_subscription` 视图包含所有现有的逻辑复制订阅。

和大部分系统视图不同，`sys_subscription` 在集簇的所有数据库之间共享：每个集簇只有一份 `sys_subscription` 拷贝，而不是每个数据库一份。

对列 `subconninfo` 的访问被从普通用户那里收回，因为该列可能含有明文口令。

表 1.1.95: sys_subscription 列

名称	类型	描述
oid	oid	行标识符
subdbid	oid	订阅所在的数据库的 OID
subname	name	订阅的名称
subowner	oid	订阅的拥有者
subenabled	boolean	如果为真，订阅被启用并且应该被复制

-续表-

名称	类型	描述
subconninfo	text	到上游数据库的连接字符串
subslotname	name	上游数据库中的复制槽的名称也被用于本地复制源名称
subsynccommit	text	包含订阅工作者的 synchronous_commit 设置的值。
subpublications	text[]	被订阅的 public ation 名称的数组这些引用的是发布者服务器上的 publication

1.1.2.81　sys_subscription_rel

`sys_subscription_rel` 视图包含每个订阅中每个被复制关系的状态。这是一种多对多映射。

这个视图仅包含运行 `CREATE SUBSCRIPTION` 或 `ALTER SUBSCRIPTION ... REFRESH PUBLICATION` 以后对订阅已知的表。

表 1.1.96: sys_subscription_rel 列

名称	类型	描述
srsubid	oid	对订阅的引用
srrelid	oid	对关系的引用
srsubstate	char	状态代码： i = 初始化， d = 数据正在被拷贝， s = 已同步， r = 准备好（普通复制）
srsublsn	pg_lsn	s 和 r 状态的结束 LSN

1.1.2.82　sys_synonym

`sys_synonym` 视图存储了有关同义词的信息。

名称	类型	描述
oid	oid	行标识符
synname	name	同义词名称
synnamespace	oid	同义词所在模式 oid
synowner	oid	同义词拥有者的 oid
syneditionable	boolean	同义词是否可编辑
refobjnspname	name	同义词引用对象所在模式的名称
refobjname	name	同义词引用对象的名称
dblinkschema	name	同义词关联的 dblink 所在模式的名称
dblinkname	name	同义词关联的 dblink 名称

1.1.2.83　sys_sysprivilege

`sys_sysprivilege` 视图可查询出所有拥有系统 ANY 权限的用户信息。

表 1.1.98: sys_sysprivilege 列

名称	类型	描述
grantee	oid	接受授权的用户或角色的 OID
privilege	integer	ANY 权限 ID 号
admin_option	boolean	指示授予是否与 ADMIN OPTION 一起使用

1.1.2.84　sys_tables

视图 `sys_tables` 提供了对数据库中每个表的有用信息的访问。

表 1.1.99: sys_tables 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
tableowner	name	表拥有者的名字
tablespace	name	包含表的表空间的名字（如果使用数据库的默认表空间，此列为空）
hasindexes	boolean	如果表有（或最近有过）任何索引，此列为真
hasrules	boolean	如果表有（或曾经有过）规则，此列为真
hastriggers	boolean	如果表有（或者曾经有过）触发器，此列为真
rowsecurity	boolean	如果表上启用了行安全性则为真

1.1.2.85　sys_tablespace

`sys_tablespace` 视图存储关于可用表空间的信息。表可以被放置在特定表空间中以实现磁盘布局的管理。

与大部分其他系统视图不同，`sys_tablespace` 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 `sys_tablespace` 的拷贝，而不是每个数据库一份。

表 1.1.100: sys_tablespace 列

名称	类型	描述
oid	oid	行标识符
spcname	name	表空间名
spcowner	oid	表空间的拥有者，通常是创建它的用户
spcacl	aclitem[]	访问权限
spcoptions	text[]	表空间级别的选项，形如“keyword=value”的字符串

1.1.2.86　sys_timezone_abbrevs

视图 `sys_timezone_abbrevs` 提供了对当前被时间输入例程识别的时区缩写的列表。当 timezone_abbreviations 运行时参数被修改，此视图的内容会发生变化。

表 1.1.101: sys_timezone_abbrevs 列

名称	类型	描述
abbrev	text	时区缩写
utc_offset	interval	相对于 UTC 的偏移（正值表示格林威治东部）
is_dst	boolean	如果这是一个夏令时缩写，则为真

虽然大部分时区缩写表示从 UTC 开始的固定偏移，但是有一些在历史上有值的变化。在这种情况下，这个视图表示它们现在的含义。

1.1.2.87　sys_timezone_names

视图 `sys_timezone_names` 提供了一个被 `SET TIMEZONE` 识别的时区名字的列表，以及它们的相关缩写、UTC 偏移和夏令时状态（从技术上来说，KingbaseES 不使用 UTC 是因为闰秒没有被处理）。和sys_timezone_abbrevs 中展示的缩写不同，这里很多名字隐含了一组夏令时转换日期规则。因此，相关信息在本地 DST 边界间变化。所显示的信息基于 `CURRENT_TIMESTAMP` 的当前值计算得来。

表 1.1.102: sys_timezone_names 列

名称	类型	描述
name	text	时区名
abbrev	text	时区缩写
utc_offset	interval	相对于 UTC 的偏移（正值表示格林威治东部）
is_dst	boolean	如果当前保持为夏令时则为真

1.1.2.88　sys_transform

`sys_transform` 视图存储有关转换的信息，转换是一种让数据类型适应过程语言的机制。

表 1.1.103: sys_transform 列

名称	类型	描述
oid	oid	行标识符
trftype	oid	这个转换所针对的数据类型的 OID
trflang	oid	这个转换所针对的语言的 OID
trffromsql	regproc	一个函数的 OID，该函数用来将数据类型转换为过程语言的输入（例如函数参数）。 如果不支持这种操作，这里存储零
trftosql	regproc	一个函数的 OID，该函数被用来转换过程语言的输出（例如返回值）为该数据类型。 如果不支持这种操作，这里存储零

1.1.2.89　sys_trigger

`sys_trigger` 视图存储表和视图上的触发器。详见 CREATE TRIGGER 。

表 1.1.104: sys_trigger 列

名称	类型	描述
oid	oid	行标识符
tgrelid	oid	触发器所在的表
tgname	name	触发器名（在同一个表的触发器中必须唯一）
tgfoid	oid	要被触发器调用的函数
tgtype	smallint	触发器触发条件的位掩码

-续表-

名称	类型	描述
tgenabled	char	控制触发器在 session_replication_role 模式中的触发。 O = 触发器在“origin”和“local”模式触发 D = 触发器被禁用 R = 触发器在“replica”模式触发 A = 触发器总是触发
tgisinternal	boolean	为真表示触发器是内部生成的（通常是为了强制由 tgconstraint 指定的约束）
tgconstrrelid	oid	被一个引用完整性约束引用的表
tgconstrindid	oid	支持一个唯一、主键、引用完整性约束或者排除约束的索引
tgconstraint	oid	可能存在的与触发器相关的 sys _constraint 项
tgdeferrable	boolean	如果约束触发器可推迟则为真
tginitdeferred	boolean	如果约束触发器初始可推迟则为真
tgnargs	smallint	传递给触发器函数的参数字符串个数
tgattr	int2vector	如果触发器是列限定的，这里存放列号否则这是一个空数组
tgargs	bytea	传递给触发器的参数字符串，每一个都以 NULL 结尾
tgqual	pg_node_tree	触发器 WHEN 条件的表达式树（以 nodeToString() 的表现形式），如果没有则为空
tgoldtable	name	OLD TABLE 的 REFERENCING 子句名称，如果没有则为空
tgnewtable	name	NEW TABLE 的 REFERENCING 子句名称，如果没有则为空

当前，列限定触发器只被 `UPDATE` 事件支持，因此 `tgattr` 只用于这种事件类型。`tgtype` 页可以包含用于其他事件类型的位，但其他事件类型是表范围的触发器且会忽略 `tgattr`。

备注:

当 `tgconstraint` 非零时，`tgconstrrelid`、`tgconstrindid`、`tgdeferrable` 和 `tginitdeferred` 与被引用的 `sys_constraint` 项有很大的冗余。但是，存在将一个不可延迟触发器关联到一个可延迟约束的可能性：外键约束可以有一些可延迟和一些不可延迟触发器。

备注:

如果一个关系在本目录中拥有任何触发器，其 `sys_class.relhastriggers` 必须为真。

1.1.2.90　sys_triggers

视图 `sys_triggers` 提供关于数据库触发器的信息。

表 1.1.105: sys_triggers 列

名称	类型	描述
tgrelid	oid	触发器的 OID
tgname	name	触发器名称
schemaid	oid	所在模式的 OID
schemaname	name	所在模式的名称
tgenabled	char	控制触发器在 session_replication_role 模式中的触发 O = 触发器在“origin”和“local”模式触发 D = 触发器被禁用 R = 触发器在“replica”模式触发 A = 触发器总是触发
tablename	name	触发器引用表的名称
tgtype	smallint	触发器触发条件的位掩码
tgfoid	oid	要被触发器调用的函数 OID
tgconstrrelid	oid	引用完整性约束的表 OID
tgdeferrable	boolean	触发器是否可推迟
tginitdeferred	boolean	触发器初始是否可推迟则为真

-续表-

名称	类型	描述
tridef	text	触发器 DDL 定义语句
tridefbody	text	触发体 DDL 定义语句
owner	name	触发器拥有者
isinternal	boolean	是否为内部触发器

1.1.2.91　sys_ts_config

`sys_ts_config` 视图包含表示文本搜索配置的选项。一个配置指定了一个特定的文本搜索分析器和一个用于分析器输出记号的字典列表。分析器由 `sys_ts_config` 项展现，而记号到字典的映射则由sys_ts_config_map 中的辅助项定义。

表 1.1.106: sys_ts_config 列

名称	类型	描述
oid	oid	行标识符
cfgname	name	文本搜索配置名
cfgnamespace	oid	包含该配置的名字空间的 OID
cfgowner	oid	配置的拥有者
cfgparser	oid	该配置的文本搜索分析器的 OID

1.1.2.92　sys_ts_config_map

`sys_ts_config_map` 视图包含的项展示了对于每一个文本搜索配置的每一种输出记号类型，有哪些文本搜索字典可供查询以及以何种顺序。

表 1.1.107: sys_ts_config_map 列

名称	类型	描述
mapcfg	oid	拥有该映射项的 sys_ts_config 项的 OID
maptokentype	integer	一种由配置的分析器送出的记号类型
mapseqno	integer	查询该项的顺序（mapseqno 值小的优先）
mapdict	oid	查询的文本搜索字典的 OID

1.1.2.93　sys_ts_dict

`sys_ts_dict` 视图包含定义文本搜索字典的项。一个字典依赖于一个文本搜索模板，它指定了所有需要的函数实现，字典本身则为模板支持的用户可设置参数提供值。这种分工允许无权限的用户创建字典。参数由一个文本串 `dictinitoption` 定义，其格式和意义随着模板而变化。

表 1.1.108: sys_ts_dict 列

名称	类型	描述
oid	oid	行标识符
dictname	name	文本搜索字典名
dictnamespace	oid	包含该字典的名字空间 OID
dictowner	oid	字典的拥有者
dicttemplate	oid	该字典的文本搜索模板的 OID
dictinitoption	text	模板的初始化选项串

1.1.2.94　sys_ts_parser

`sys_ts_parser` 视图包含定义文本搜索分析器的项。一个分析器负责将输入文本分割成词位并为每一个词位分配一个记号类型。由于一个分析器必须用 C 语言级别的函数实现，创建新分析器的工作只限于数据库的超级用户。

表 1.1.109: sys_ts_parser 列

名称	类型	描述
oid	oid	行标识符
prsname	name	文本搜索分析器的名字
prsnamespace	oid	包含此分析器的名字空间的 OID
prsstart	regproc	分析器启动函数的 OID
prstoken	regproc	分析器的下一记号函数的 OID
prsend	regproc	分析器的关闭函数的 OID
prsheadline	regproc	分析器的大标题函数的 OID
prslextype	regproc	分析器的词汇类型函数的 OID

1.1.2.95　sys_ts_template

`sys_ts_template` 视图包含定义文本搜索模板的项。一个模板是一类文本搜索字典的实现骨架。由于一个模板必须用 C 语言级别的函数实现，新模板的创建只限于数据库超级用户。

表 1.1.110: sys_ts_template 列

名称	类型	描述
oid	oid	行标识符
tmplname	name	文本搜索模板的名字
tmplnamespace	oid	包含此模板的名字空间的 OID
tmplinit	regproc	模板的初始化函数的 OID
tmpllexize	regproc	模板的词汇化函数的 OID

1.1.2.96　sys_type

`sys_type` 视图存储有关数据类型的信息。基类和枚举类型（标度类型）使用 `CREATE TYPE` 命令创建，而域使用 `CREATE DOMAIN` 创建。数据库中的每一个表都会有一个自动创建的组合类型，用于表示表的行结构。也可以使用 `CREATE TYPE AS` 创建组合类型。

表 1.1.111: sys_type 列

名称	类型	描述
oid	oid	行标识符
typname	name	数据类型的名字
typnamespace	oid	包含此类型的 名字空间的 OID
typowner	oid	类型的拥有者
typlen	smallint	对于一个固定尺寸的类型，typlen 是该类型内部表示的字节数 对于一个变长类型，typlen 为负值 -1 表示一个“varlena”类型（具有长度字 -2 表示一个以空值结尾的 C 字符串
typbyval	boolean	typbyval 判断内部例程传递这个类型的数值时是通过传值还是传引用 如果 typlen 不是 1、2 或 4（或者在 Datum 为 8 字节的机器上为 8），因此 typbyval 最好是假 变长类型总是传引用 注意即使长度允许传值，typbyval 也可以为假。
typtype	char	typtype 可以是： b 表示一个基类， c 表示一个组合类型（例如一个表的行类型） d 表示一个域 e 表示一个枚举类型 p 表示一个伪类型，或 r 表示一个范围类型。 另请参阅 typrelid 和 typbasetype

-续表-

名称	类型	描述
typcategory	char	typcategory 是一种任意的数据类型分类，它被分析器用来决定哪种隐式转换“更好”。
typispreferred	boolean	如果此类型在它的 typcategory 中是一个更好的转换目标，此列为真
typisdefined	boolean	如果此类型已被定义则为真，如果此类型只是一个表示还未定义类型的占位符则为假 当 typisdefined 为假，除了类型名字、名字空间和 OID 之外什么都不能被依赖
typdelim	char	在分析数组输入时，分隔两个此类型值的字符 注意该分隔符是与数组元素数据类型相关联的，而不是和数组的数据类型关联
typrelid	oid	如果这是一个复合类型（见 typtype ），那么这个列指向 sys_class 中定义对应表的项 （对于自由存在的复合类型，sys_class 项并不表示一个表，但不管怎样该类型的 sys_attribute 项需要链接到它） 对非复合类型此列为零。
typelem	oid	如果 typelem 不为 0，则它标识 sys_type 里面的另外一行 当前类型可以被加上下标得到一个值为类型 typelem 的数组来描述 一个“真的”数组类型是变长的（typlen =-1），但是一些定长的（typlen > 0）类型也拥有非零的 typelem ，name 和 point 如果一个定长类型拥有一个 typelem ，则它的内部形式必须是某个 typelem 数据类型的值，不能有其它数据 变长数组类型有一个由该数组子例程定义的头
typarray	oid	如果 typarray 不是 0，则它标识 sys_type 中的另一行，这一行是一个将此类型作为元素的“真的”数组类型

-续表-

名称	类型	描述
typinput	regproc	输入转换函数（文本格式）
typoutput	regproc	输出转换函数（文本格式）
typreceive	regproc	输入转换函数（二进制格式），如果没有则为 0
typsend	regproc	输出转换函数（二进制格式），如果没有则为 0
typmodin	regproc	类型修改器输入函数，如果类型没有提供修改器则为 0
typmodout	regproc	类型修改器输出函数，如果类型没有提供修改器则为 0
typanalyze	regproc	自定义 ANALYZE 函数，0 表示使用标准函数
typalign	char	typalign 是当存储此类型值时要求的对齐性质 它应用于磁盘存储以及该值在 KingbaseES 内部的大多数表现形式 如果数值是连续存放的，比如在磁盘上的一个完整行，在这种类型的数据前会插入填充，这样它就可以按照指定边界存储 对齐引用是该序列中第一个数据的开头 对齐引用是序列中第一个数据的开始 可能的值有： c = char 对齐，即不需要对齐 s = short 对齐（在大部分机器上为 2 字节）i = int 对齐（在大部分机器上为 4 字节）d = double 对齐（在很多机器上为 8 字节，但绝不是全部）对于系统表中使用的类型，很关键的是，sys_type 中定义的尺寸和对齐方式要和编译器在表示表行的结构中布局列的方式保持一致

-续表-

名称	类型	描述
typstorage	char	如果一个变长类型（typlen = -1）可被TOAST，typstorage 说明这种类型的列应采取的默认策略 可能的值是： p ：值必须平面存储 e ：值可以被存储在一个“二级”关系（如果有，见 sys _class.reltoastrelid ）m ：值可以被压缩线内存储 x ：值可以被压缩线内存储或存储在“二级”存储注意 m 列也可以被移动到二级存储，但只能是作为最后一种方案（e 和 x 列会先被移动）
typnotnull	boolean	typnotnull 表示类型上的一个非空约束。 只用于域。
typbasetype	oid	如果这是一个域（见 typtype ），则typbasetype 标识这个域基于的类 如果此类不是一个域则为 0
typtypmod	integer	域使用 typtypmod 来记录被应用于它们基类型的 typmod （如果基类型不使用typmod，则为-1） 如果此类型不是一个域则为-1
typndims	integer	对于一个数组上的域，typndims 是数组维度数（即 typbasetype 是一个数组类型） 除数组类型上的域之外的类型的此列为 0
typflags	integer	标识类型上的一些常用属性
typcollation	oid	typcollation 指定此类型的排序规则 如果类型不支持排序规则，此列为 0 支持排序的基类型在这里有一个非零值，通常为 DEFAULT_COLLATION_OID 如果为域指定了排序类型，则排序类型上的域可以具有与其基类型不同的排序类型OID
typdefaultbin	pg_node_tree	如果 typdefaultbin 为非空，那么它是该类型默认表达式的 nodeToString() 表现形式。 这个列只用于域

-续表-

名称	类型	描述
typdefault	text	如果某类型没有相关默认值，那么 typdefault 为空 如果 typdefaultbin 不为空，那么 typdefault 必须包含一个 typdefaultbin 所指的默认表达式的人类可读的版本 如果 typdefaultbin 为空但 typdefault 不为空，则 typdefault 是该类型默认值的外部表现形式，它可以被交给该类型的输入转换器来产生一个常量
typacl	aclitem[]	访问权限

下表列出了 `typcategory` 的系统定义值。任何未来对此列表的增加都将是大写 ASCII 字母。所有其他 ASCII 字符都保留给用户定义的类别。

表 1.1.112: typcategory 编码

编码	类别
A	数组类型
B	布尔类型
C	组合类型
D	日期/时间类型
E	枚举类型
G	几何类型
I	网络地址类型
N	数字类型
P	伪类型
R	范围类型
S	字符串类型
T	时间间隔类型

-续表-

编码	类别
U	用户定义类型
V	位串类型
X	未知类型

1.1.2.97　sys_user_mapping

`sys_user_mapping` 视图存储从本地用户到远程的映射。普通用户查询该视图存在限制，可使用视图sys_user_mappings 替代。

表 1.1.113: sys_user_mapping 列

名称	类型	描述
oid	oid	行标识符
umuser	oid	将要被映射的本地角色的 OID，如果用户映射是公共的则为 0
umserver	oid	包含此映射的外部服务器的 OID
umoptions	text[]	用户映射相关选项，以“keyword= value”字符串形式

1.1.2.98　sys_user_mappings

视图 `sys_user_mappings` 提供有关用户映射的信息。这是sys_user_mapping 的一个公共可读视图，它对无权使用的用户省去了选项域。

表 1.1.114: sys_user_mappings 列

名称	类型	描述
umid	oid	用户映射的 OID
srvid	oid	包含该映射的外部服务器的 OID

-续表-

名称	类型	描述
srvname	name	外部服务器名
umuser	oid	将被映射的本地角色的 OID，如果用户映射是公共的则为 0
usename	name	将被映射的本地用户名
umoptions	text[]	用户映射指定选项，以“keyword=value”字符串的形式

为了保护存储为用户映射选项的口令信息，`umoptions` 列将被读作空，除非满足下列情况之一：

- 当前用户就是被映射的用户，并且拥有该服务器或者持有其上的 `USAGE` 特权
- 当前用户是服务器的拥有者并且映射是用于 `PUBLIC`
- 当前用户是一个超级用户

1.1.2.99　sys_views

视图 `sys_views` 提供了数据库中每个视图的信息。

表 1.1.115: sys_views 列

名称	类型	描述
schemaname	name	包含视图的模式名
viewname	name	视图名称
viewowner	name	视图拥有者的名字
definition	text	视图定义（一个重构的 SELECT 查询）
isupdateable	integer	是否可更新

1.1.2.100　sysmac.sysmac_column_label

`sysmac.sysmac_column_label` 视图保存列上的标记。关于列级强制访问控制功能参见 强制访问控制，查询使用 `sysmac.sysmac_column_label`。

表 1.1.116: sysmac_column_label 列

名称	类型	描述
policy_id	oid	策略的 oid
reloid	oid	表 oid
col_num	integer	列的序号
label	oid	列标记的 oid

1.1.2.101　sysmac.sysmac_obj

表 `sysmac.sysmac_obj` 保存对象级客体对应的标记。关于对象级强制访问控制功能参见 强制访问控制，查询使用 `sysmac.sysmac_obj`。

表 1.1.117: sysmac_obj 列

名称	类型	描述
policy_id	oid	策略的 oid
policy_name	name	策略的名字
obj_type	name	客体对象的类型
sch_id	oid	客体对象的模式 oid
obj_id	oid	客体对象的 oid
label_id	oid	客体标记的 oid

1.1.2.102　user_any_privs

视图 `user_any_privs` 用于提供权限授予信息，显示为权限具体的名字。每个用户只可以查看自己的拥有的 ANY 权限。

表 1.1.118: user_any_privs 列

名称	类型	描述
grantee	name	接受授权的用户或角色的名称。
privilege	text	ANY 权限名称
admin_option	text	指示授予是否与 ADMIN OPTION 一起使用

1.1.3　安全视图

本小节介绍与安全相关的系统视图内容。

1.1.3.1　审计视图

表 1.1.119: 审计视图

视图名称	用途
sysaudit.all_audit_rules)	审计策略
sysaudit.all_ids_rules)	审计入侵检测
sys_sysaudit_ids_setting)	审计入侵检测
sys_sysaudit_setting)	审计策略
sysaudit_record_sao	审计记录
sysaudit_record_sso	审计记录

1.1.3.1.1　sysaudit.all_audit_rules

视图 `all_audit_rules` 记录审计策略设置信息。视图在 sysaudit 模式下，查询使用 `sysaudit.all_audit_rules`。

表 1.1.120: all_audit_rules 列

名称	类型	描述
audit_id	integer	审计策略编号
audit_target	text	审计目标（事件、语句、对象）
audit_type	text	审计类型
audit_users	text	审计用户
audit_schema	text	审计对象所在模式
audit_objnames	text	审计对象名称
audit_objoid	integer	审计对象 oid
creator_name	text	策略设置者名称

1.1.3.1.2　sysaudit.all_ids_rules

视图 `all_ids_rules` 记录审计入侵检测策略信息。视图在 sysaudit 模式下，查询使用 `sysaudit.all_ids_rules`。

表 1.1.121: all_ids_rules 列

名称	类型	描述
rulename	text	ids 规则名
actionname	text	审计的操作名
useroid	integer	用户 oid
username	text	审计用户名
schoid	integer	对象模式 oid
schname	text	审计对象模式名

-续表-

名称	类型	描述
objoid	integer	对象 oid
objname	text	审计对象名
whenever	text	检查审计的时机
ip	text	触发审计的客户端 ip
start_end_time	text	触发检查的时间区间
interval_time	integer	触发检查的最长时间间隔
times	integer	触发检查次数的阈值

1.1.3.1.3　sys_sysaudit_ids_setting

`sys_sysaudit_ids_setting` 视图存储入侵检测审计信息。

表 1.1.122: sys_sysaudit_ids_setting 列

名称	类型	描述
ruleName	name	ids 规则名
actionName	name	审计的操作名
userOid	oid	用户 oid
schOid	oid	对象模式 oid
objOid	oid	对象 oid
whenever	name	检查审计的时机
interval	integer	触发检查的最长时间间隔
times	integer	触发检查次数的阈值
userName	name	审计用户名
schName	name	审计对象模式名
objName	name	审计对象名

-续表-

名称	类型	描述
IP	text	触发审计的客户端 ip
start_end_time	text	触发检查的时间区间

1.1.3.1.4　sys_sysaudit_setting

`sys_sysaudit_setting` 视图存储审计策略信息。

表 1.1.123: sys_sysaudit_setting 列

名称	类型	描述
oid	oid	审计规则 oid
audit_target	smallint	审计目标
audit_type	integer	审计类型
audit_user	oid	审计用户
audit_dboid	oid	审计指定数据库 id
audit_schemaoid	oid	审计指定对象模式 id
audit_objoid	oid	审计指定对象 id
audit_objtype	integer	审计对象类型
audit_cid	oid	审计规则创建用户 oid
audit_dbname	name	审计指定数据库名
audit_shename	name	审计指定对象模式名
audit_objname	name	审计对象名

1.1.3.1.5　sysaudit_record_sao

数据库审计员 sao 可以在 security 数据库下查询视图 `sysaudit_record_sao`，查看超级用户（包括数据库管理员 system）和数据库安全员 sso 的审计记录信息。更多审计功能信息参见 数据库审计。

表 1.1.124: sysaudit_record_sao 列

名称	类型	描述
session_id	text	会话 id
proc_id	integer	进程号
vxid	text	虚事务 id
xid	integer	事务 id
user_id	oid	用户 id
username	text	用户名
remote_addr	text	ip 地址/端口
db_id	oid	数据库 id
db_name	text	数据库名
rule_id	bigint	审计策略编号
rule_type	text	审计类型
opr_type	text	操作类型
obj_type	text	对象类型
schm_id	oid	模式 id
schm_name	text	模式名
obj_id	oid	对象 id
obj_name	text	对象名
sqltext	text	SQL 文本
params	text	参数
errcode	text	SQLSTATE 错误码
errmsg	text	错误消息内容

-续表-

名称	类型	描述
audit_ts	timestamp with time zone	操作时间
result	text	操作成功是 success，失败是 failure
record_type	smallint	审计记录类型
aud_client	text	客户端名字
server_type	text	集群时服务器类型，M 是主机，S 是备机
authen_method	text	认证方式。非用户登录进程此字段显示 unknown

1.1.3.1.6　sysaudit_record_sso

数据库安全员 sso 可以在 security 数据库下查询视图 `sysaudit_record_sso`，查看普通用户和数据库审计员 sao 的审计记录信息。更多审计功能信息参见 数据库审计。

sysaudit_record_sso 的视图结构与sysaudit_record_sao 相同。

1.1.3.2　角色管理视图

表 1.1.125: 角色管理视图

视图名称	用途
sys_audit_blocklog	被锁定用户的登录信息
sys_audit_userlog	用户登录信息
sys_auth_members	认证标识符成员关系
sys_authid	认证标识符（角色）
sys_group	数据库用户组
sys_pwdht	口令历史管理的用户信息
sys_pwdht_shadow	口令历史管理的用户信息
sys_role	数据库角色（不包含用户和用户组）

-续表-

视图名称	用途
sys_roles	数据库角色
sys_role_disable	角色禁用信息
sys_shadow	数据库用户
sys_users	数据库用户
sys_user	数据库用户
sys_usergroup	数据库用户组

1.1.3.2.1　sys_audit_blocklog

表 `sys_audit_blocklog` 存储被锁定用户的登录信息。

表 1.1.126: sys_audit_blocklog 列

名称	类型	描述
user	name	被锁定的用户名
failedTimes	integer	登录失败次数
blockStartTime	timestamp with time zone	用户开始被锁定的时间

1.1.3.2.2　sys_audit_userlog

`sys_audit_userlog` 视图存储用户的登录信息。

表 1.1.127: sys_audit_userlog 列

名称	类型	描述
audusername	name	登陆信息记录的用户名
audhost	text	登录所在 ip 地址

-续表-

名称	类型	描述
audtimestamp	timestamp with time zone	登录时间
audtype	char	记录类型（成功/失败/锁定）
logintimes	integer	用户登录次数

1.1.3.2.3　sys_auth_members

`sys_auth_members` 视图记录角色之间的成员关系。允许任何无环的关系集合。

由于用户标识符是集簇范围的，`sys_auth_members` 在一个集簇的所有数据库之间共享：在一个集簇中只有一份 `sys_auth_members` 拷贝，而不是每个数据库一份。

表 1.1.128: sys_auth_members 列

名称	类型	描述
roleid	oid	拥有成员的角色的 ID
member	oid	roleid 的成员角色的 ID
grantor	oid	授权此成员关系的角色的 ID
admin_option	boolean	如果成员能把成员关系授予他人，则为真

1.1.3.2.4　sys_authid

`sys_authid` 视图包含关于数据库授权标识符（角色）的信息。角色把“用户”和“组”的概念包含在内。一个用户实际上就是一个 `rolcanlogin` 标志被设置的角色。任何角色（不管 `rolcanlogin` 设置与否）都能够把其他角色作为成员，参见sys_auth_members 视图。

由于这个视图包含密码，它不能是公共可读的。sys_roles 视图是基于 `sys_authid` 视图创建的一个公共可读视图，它隐藏了密码列。

由于用户标识符是集簇范围的，`sys_authid` 视图在一个集簇的所有数据库之间共享：在一个集簇中所有数据库共用一个 `sys_authid` 视图，而不是在单个数据库下仅显示当前数据库的相关信息。

表 1.1.129: sys_authid 列

名称	类型	描述
oid	oid	行标识符
rolname	name	角色名
rolsuper	boolean	角色是否拥有超级用户权限
rolinherit	boolean	如果本角色是另一个角色的成员，本角色是否自动另一个角色的权限
rolcreaterole	boolean	角色是否能创建更多角色
rolcreatedb	boolean	角色是否能创建数据库
rolcanlogin	boolean	角色是否能登录。即该角色是否能够作为初始会话授权标识符
rolreplication	boolean	角色是一个复制角色。复制角色可以启动复制连接并且创建和删除复制槽
rolbypassrls	boolean	角色是否可以绕过所有的行级安全性策略
rolconnlimit	interger	对于可以登录的角色，本列设置该角色可以同时发起最大连接数 -1 表示无限制
rolconntime	bigint	用户最近一次连接到数据库的时间
rolconninterval	text	角色/可登陆时间段
rolpassword	text	密码（可能被加密过），如果没有密码则为空。格式取决于使用的加密方法的形式
rolvaliduntil	timestamp with time zone	密码过期时间（只用于密码，如果永不过期则为空

对于一个 MD5 加密的密码，`rolpassword` 列将由字符串 `md5` 后面跟上一个 32 字符的十六进制 MD5 哈希值构成。MD5 哈希值将是该用户的密码串接上它们的用户名。例如，如果用户 `joe` 的密码是 `xyzzy`，则 KingbaseES 将存储 `xyzzyjoe` 的 md5 哈希。

如果密码采用 SCRAM-SHA-256 加密，它的格式是：

```
SCRAM-SHA-256$<iteration count>:<salt>$<StoredKey>:<ServerKey>
```

不遵守上述格式的密码被假定为未加密。

1.1.3.2.5　sys_group

视图 `sys_group` 为向后兼容而存在：它模拟了存在于 KingbaseES V7 之前版本中的一个系统表，它显示所有角色的名称和未被标记 `rolcanlogin` 的成员。

表 1.1.130: sys_group 列

名称	类型	描述
groname	name	组名
grosysid	oid	组 ID
grolist	oid[]	包含此组中角色 ID 的一个数组

1.1.3.2.6　sys_role

视图 `sys_role` 提供了关于数据库角色（不包含用户和用户组）的信息。

表 1.1.131: sys_role 视图

名称	类型	描述
role_name	name	角色名
role_oid	oid	角色 OID
role_list	oid[]	角色授予权限的目标列表

1.1.3.2.7　sys_roles

视图 `sys_roles` 提供了关于数据库角色（包含用户和用户组）的信息。这是sys_authid 的一个公共可读视图，它隐去了密码行。

此视图显示了低层表的 OID 列，因为需要它来和其他视图做连接。

表 1.1.132: sys_roles 列

名称	类型	描述
rolname	name	角色名
rolsuper	boolean	角色是否具有超级用户权限
rolinherit	boolean	如果此角色是另一个角色的成员，角色是否能自动继承另一个角色的权限
rolcreaterole	boolean	角色能否创建更多角色
rolcreatedb	boolean	角色能否创建数据库
rolcanlogin	boolean	角色是否能登录，即此角色能否被作为初始会话授权标识符
rolreplication	boolean	角色是一个复制角色。复制角色可以开启复制连接并且创建和删除复制槽。
rolconnlimit	integer	对于一个可登录的角色，这里设置角色可以发起的最大并发连接数。-1 表示无限制。
rolpassword	text	不是口令（看起来是 **** ）
rolvaliduntil	timestamp with time zone	口令失效时间（只用于口令认证），如果永不失效则为空
rolbypassrls	boolean	绕过每一条行级安全性策略的角色
rolconfig	text[]	运行时配置变量的角色特定默认值
oid	oid	角色的 oid

1.1.3.2.8　sys_shadow

视图 `sys_shadow` 的存在是为了向后兼容：它模拟了在 KingbaseESV7 版本之前的一个系统表。它显示sys_authid 视图中所有被标记为 `rolcanlogin` 的角色的属性。

由于这个表包含密码，所以不能是公众可读的，这也是采用 `sys_shadow` 这个名字的原因。而sys_user 视图是 `sys_shadow` 上的一个公共可读视图，它屏蔽了密码列。

表 1.1.133: sys_shadow 列

名称	类型	描述
usename	name	用户名
usesysid	oid	用户的 ID
usecreatedb	boolean	用户能否创建数据库
usesuper	boolean	用户是否为一个超级用户
userepl	boolean	用户能否开启流复制并将系统设置或者取消备份模式
usebypassrls	boolean	用户能否绕过所有的行级安全性策略
passwd	text	密码（可能被加密），如果没有则为空。关于加密口令如何存储请参见 sys_authid
valuntil	timestamp with time zone	密码过期时间（仅用于密码认证）
useconfig	text[]	运行时配置变量的会话默认值

1.1.3.2.9　sys_user

视图 `sys_user` 提供关于数据库用户的信息。这是sys_shadow 的一个公共可读的视图，它隐藏了真实密码。

表 1.1.134: sys_user 列

名称	类型	描述
usename	name	用户名
usesysid	oid	用户的 ID
usecreatedb	boolean	用户是否能创建数据库
usesuper	boolean	用户是否为超级用户
userepl	boolean	用户能否开启流复制以及将系统转入/转出备份模式
usebypassrls	boolean	用户能否绕过所有的行级安全性策略
passwd	text	不是口令（总是显示为 ****）

-续表-

名称	类型	描述
valuntil	timestamp with time zone	口令过期时间（只用于口令认证）
useconfig	text[]	运行时配置变量的会话默认值

1.1.3.2.10　sys_users

视图 `sys_users` 提供关于数据库用户的信息。在查询系统用户信息时，超级用户可以查看所有用户信息，非超级用户只能查看自己的信息。

表 1.1.135: sys_users 列

名称	类型	描述
usename	name	用户名
usesysid	oid	用户的 ID
usecreatedb	boolean	用户是否能创建数据库
usesuper	boolean	用户是否为超级用户
userepl	boolean	用户能否开启流复制以及将系统转入/转出备份模式
usebypassrls	boolean	用户能否绕过所有的行级安全性策略
passwd	text	不是口令（总是显示为 ****）
valuntil	timestamp with time zone	口令过期时间（只用于口令认证）
useconfig	text[]	运行时配置变量的会话默认值

1.1.3.2.11　sys_usergroup

视图 `sys_usergroup` 提供关于数据库用户组的信息。

表 1.1.136: sys_usergroup 列

名称	类型	描述
group_name	name	用户组名
group_oid	oid	用户组 OID
group_members	oid[]	用户组内成员 oid 列表

1.1.3.3　sys_hba_file_rules

视图 `sys_hba_file_rules` 提供客户端认证配置文件 `sys_hba.conf` 内容的摘要。该文件中每个非空、非注释行都会在这个视图中出现一行，行中还有标记表示该规则是否被成功地应用。

这个视图可用来检查认证配置文件中按计划的更改是否起作用，或者诊断之前的失败。注意这个视图报告的是该文件的当前内容，而不是服务器最后一次载入的内容。

默认情况下，只有超级用户可以读取 `sys_hba_file_rules` 视图。

表 1.1.137: sys_hba_file_rules 列

名称	类型	描述
line_number	integer	这条规则在 sys_hba.conf 中的行号
type	text	连接类型
database	text[]	这条规则应用的数据库名列表
user_name	text[]	这条规则应用的用户及组名列表
address	text	主机名或 IP 地址，或者 all 、samehost 、samenet 之一，对于本地连接为空
netmask	text	IP 地址掩码，如果不适用则为空
auth_method	text	认证方法
options	text[]	为认证方法指定的选项（如果有）
error	text	如果非空，则是一个错误消息，它表示为什么这一行无法被处理

通常，反映一个不正确项的行只有 `line_number` 和 `error` 列中有值。

更多有关客户端认证配置的信息请参考 强身份验证.

1.1.4 统计视图

表 1.1.138: 统计视图

视图名称	用途
实时状态统计	
sys_stat_activity	服务器进程当前活动的相关信息
sys_stat_gssapi	关于 GSSAPI 验证和加密的信息
sys_stat_progress_checkpoint	检查点（CHECKPOINT）的执行进度
sys_stat_progress_cluster	每个正在运行的 CLUSTER 或 VACUUM FULL 进度
sys_stat_progress_create_index	每个后台正在运行的 CREATE INDEX 或 REINDEX 的进度
sys_stat_progress_vacuum	每个正在运行的 CLUSTER 或 VACUUM FULL 进度
sys_stat_replication	到发送进程连接的后备服务器的复制的统计信息
sys_stat_ssl	连接上使用的 SSL 的信息
sys_stat_subscription	订阅的工作者的信息
sys_stat_wal_buffer	WAL BUFFER 实时的运作情况
sys_stat_wal_receiver	WAL 接收器所连接服务器的有关该接收器的统计信息
数据库实例统计	
sys_stat_archiver	有关 WAL 归档进程活动的统计信息
sys_stat_bgwriter	有关后台写进程的活动的统计信息
sys_stat_cached_plans	数据库实例执行计划缓存情况统计
sys_stat_database	数据库范围的统计信息
sys_stat_database_conflicts	数据库范围内由于与后备服务器的恢复过程发生冲突而被取消的查询
sys_stat_dmlcount	数据库实例的 DML SQL 语句统计
sys_stat_pre_archivewal	所有预归档 WAL 日志统计信息

-续表-

视图名称	用途
sys_stat_shmem	数据库实例共享内存分配情况统计
sys_stat_sqlcount	数据库实例的 SQL 语句统计
sys_stat_transaction	基于 database 提供事务的统计信息
数据库对象统计	
sys_stat_all_indexes	访问指定索引有关的统计信息
sys_stat_all_tables	访问指定表的统计信息
sys_stat_sys_indexes	访问系统表上的索引的统计信息
sys_stat_sys_tables	访问系统表的统计信息
sys_stat_user_functions	执行函数有关的统计信息
sys_stat_user_indexes	访问用户表上的索引的统计信息
sys_stat_user_tables	访问系统表的统计信息
sys_statio_all_indexes	与索引上的 I/O 有关的统计信息
sys_statio_all_sequences	与序列上的 I/O 有关的统计信息
sys_statio_all_tables	表上 I/O 的统计信息
sys_statio_sys_indexes	同 sys_statio_all_indexes，但只记录系统表上的索引
sys_statio_sys_sequences	同 sys_statio_sys_sequences，但只记录系统序列
sys_statio_sys_tables	和 sys_statio_all_tables 一样，但只记录系统表
sys_statio_user_indexes	同 sys_statio_all_indexes，但只记录用户表上的索引
sys_statio_user_sequences	同 sys_statio_sys_sequences，但只记录用户序列
sys_statio_user_tables	和 sys_statio_all_tables 一样，但只记录用户表
sys_stat_xact_all_tables	同 sys_stat_all_tables，但计数动作只在当前事务内发生
sys_stat_xact_sys_tables	同 sys_stat_xact_all_tables，但只记录系统表信息
sys_stat_xact_user_functions	和 sys_stat_user_functions 相似，但是只记录在当前事务期间的调用信息
sys_stat_xact_user_tables	同 sys_stat_xact_all_tables，但只记录用户表信息

-续表-

视图名称	用途
数据库时间模型统计	
sys_stat_instevent	数据库实例关键活动统计
sys_stat_instio	数据库实例 I/O 统计
sys_stat_instlock	数据库实例轻量级锁统计
sys_stat_sqlio	SQL 语句执行过程产生的 I/O 块数和时间统计
sys_stat_sqltime	SQL 报文执行次数和时间统计
sys_stat_sqlwait	SQL 语句执行过程产生的等待事件统计
sys_stat_wait	数据库时间模型，包括等待事件时间分解
sys_stat_sql	数据库时间模型，包括 SQL 语句时间分解
sys_stat_dbtime	数据库时间模型，包括基本的数据库时间分解
sys_stat_msgaccum	同 sys_stat_sqltime，忽略部分字段，该视图不再推荐使用
sys_stat_waitaccum	同 sys_stat_sqlwait，忽略部分字段，该视图不再推荐使用
实时 TPS 和 QPS 统计	
sys_stat_metric_group	实时统计项分组
sys_stat_metric	实时统计结果，目前只支持 TPS 和 QPS
sys_stat_metric_history	实时统计最近一个小时的数据
sys_stat_metric_name	实时统计项名称
sys_stat_sysmetric	同 sys_stat_metric，仅展示系统级统计
sys_stat_sysmetric_history	同 sys_stat_metric_history，仅展示系统级统计
sys_stat_sysmetric_summary	系统级实时统计汇总结果，包括最大值、最小值和平均值等
其他	
sys_statistic_ext_data	扩展的规划器统计信息 (built statistics)
sys_statistic_ext	扩展的规划器统计信息 (definition)
sys_statistic	规划器统计

-续表-

视图名称	用途
sys_stats_ext	扩展计划统计
sys_stats	规划器统计信息

1.1.4.1　sys_stat_activity

每个服务器进程对应一行，显示与后端进程当前活动相关的统计信息。

表 1.1.139: sys_stat_activity 字段

名称	类型	描述
datid	oid	后端进程所连接数据库的 OID
datname	name	后端进程所连接数据库的名称
pid	integer	后端进程的进程 ID
usesysid	oid	后端进程所登录用户的 ID
usename	name	后端进程所登录用户的名称
application_name	text	后端进程所连接应用的名称
client_addr	inet	后端进程所连接客户端的 IP 地址。若该字段为空，则表示该连接来自于服务器机器上的 Unix 套接字，或者该进程为内部进程（如自动清理进程）。
client_hostname	text	后端进程所连接客户端的主机名，该字段通过一次 client_addr 的反向 DNS 查找进行报告。 该字段仅对 IP 连接非空，并且仅在参数 log_hostname 被启用时非空。
client_port	integer	客户端与后端进程通信的 TCP 端口号，如果使用 Unix 套接字，则该字段为-1。
backend_start	timestamp with time zone	后端进程被启动的时间。对客户端的后端进程而言，该字段为客户端连接到服务器的时间。
xact_start	timestamp with time zone	后端进程当前事务的开始时间。如果没有活跃事务，则该字段为空；若当前查询是它的第一个事务，则该字段与 query_start 字段的值相同。

-续表-

名称	类型	描述
query_start	timestamp with time zone	当前查询的开始时间。若 state 字段的值不为 active，则该字段为上一个查询的开始时间。
state_change	timestamp with time zone	state 字段上一次改变的时间
wait_event_type	text	后端进程正在触发的等待事件类型，若不存在，则为 NULL。该字段可能的值有： LightLock：后端进程正在等待一个轻量级锁，每个轻量级锁保护着共享内存中的一个数据结构。Lock：后端进程正在等待一个重量级锁。重量级锁，也称锁管理器锁或简单锁，用于保护 SQL 语句可见的对象。BufferPin：后端进程正在等待访问一片数据缓冲区，而此时没有其他进程检查该缓冲区。Activity：后端进程处于空闲状态。该状态被用于在进程主循环处理中等待活动的系统进程，wait_event 将标识特定的等待点。Extension：后端进程正在一个扩展模块中等待活动。Client：后端进程正在一个套接字上等待来自用户应用的某种活动，wait_event 将标识特定的等待点。IPC：后端进程正在等待来自服务器中另一个进程的某种活动，wait_event 将标识特定的等待点。Timeout：后端进程正在等待一次超时发生，wait_event 将标识特定的等待点。IO：后端进程正在等待一次 I/O 操作完成。
wait_event	text	若后端进程处于等待状态，则为等待事件的名称，否则为 NULL。
state	text	后端进程的当前状态。该字段可能的值有： active：后端进程正在执行一个查询。idle: 后端进程正在等待一个新的客户端命令。idle in transaction：后端进程在一个事务中，但是没有正在执行查询。idle in transaction (aborted)：该状态与 idle in transaction 类似，但是该事务中的一个语句导致了错误。fastpath function call：后端进程正在执行一个 fast-path 函数。disabled：若后端进程的 track_activities 参数为 off，则报告该状态。
backend_xid	xid	后端进程正在执行事务的事务 ID
backend_xmin	xid	与后端进程相关的最小事务 ID

-续表-

名称	类型	描述
query	text	后端进程最后一次执行的 SQL 语句。 若字段 state 的值为 active，则该字段显示当前正在执行的查询；否则，该字段显示上一个被执行的查询。 默认情况下，SQL 语句文本会被截断至 1024 个字符，该值可通过参数 track_activity_query_size 更改。
backend_type	text	后端进程的类型。该字段可能的值有： autovacuum launcherautovacuum workerlogical replication launcherlogical replication workerparallel workerbackground writerclient backendcheckpointerstartupwalreceiverwalsenderwalwriter 除此以外，由扩展插件注册的后端进程有额外的类型。

1.1.4.2　sys_stat_all_indexes

当前数据库中的每个索引一行，显示：表 OID、索引 OID、模式名、表名、索引名、使用了该索引的索引扫描总数、索引扫描返回的索引记录数、使用该索引的简单索引扫描抓取的活表 (livetable) 中数据行数。当前数据库中的每个索引对应一行，记录与访问指定索引有关的统计信息。

表 1.1.140: sys_stat_all_indexes 列

名称	类型	描述
relid	oid	这个索引的基表的 OID
indexrelid	oid	这个索引的 OID
schemaname	name	这个索引所在的模式的名称
relname	name	这个索引的基表的名称
indexrelname	name	这个索引的名称
idx_scan	bigint	在这个索引上发起的索引扫描次数
idx_tup_read	bigint	在这个索引上由扫描返回的索引项数量
idx_tup_fetch	bigint	被使用这个索引的简单索引扫描取得的活着的表行数量

1.1.4.3　sys_stat_all_tables

当前数据库中每个表对应一行，记录有关访问指定表的统计信息。

表 1.1.141: sys_stat_all_tables 列

名称	类型	描述
relid	oid	一个表的 OID
schemaname	name	这个表所在的模式的名称
relname	name	这个表的名称
seq_scan	bigint	在这个表上发起的顺序扫描的次数
seq_tup_read	bigint	被顺序扫描取得的活着的行的数量
idx_scan	bigint	在这个表上发起的索引扫描的次数
idx_tup_fetch	bigint	被索引扫描取得的活着的行的数量
n_tup_ins	bigint	被插入的行数
n_tup_upd	bigint	被更新的行数（包括 HOT 更新的行）
n_tup_del	bigint	被删除的行数
n_tup_hot_upd	bigint	被更新的 HOT 行数（即不要求独立索引更新的行更新）
n_live_tup	bigint	活着的行的估计数量
n_dead_tup	bigint	死亡行的估计数量
n_mod_since_analyze	bigint	从这个表最后一次被分析后备修改的行的估计数量
last_vacuum	timestamp with time zone	上次这个表被手动清理的时间（不统计 VACUUM FULL）
last_autovacuum	timestamp with time zone	上次这个表被自动清理守护进程清理的时间
last_analyze	timestamp with time zone	上次这个表被手动分析的时间
last_autoanalyze	timestamp with time zone	上次这个表被自动清理守护进程分析的时间
vacuum_count	bigint	这个表已被手工清理的次数（不统计 VACUUM FULL）

-续表-

名称	类型	描述
autovacuum_count	bigint	这个表已被自动清理守护进程清理的次数
analyze_count	bigint	这个表已被手工分析的次数
autoanalyze_count	bigint	这个表已被自动清理守护进程分析的次数

1.1.4.4　sys_stat_archiver

只有一行，记录有关 WAL 归档进程活动的统计信息。

表 1.1.142: sys_stat_archiver 列

名称	类型	描述
archived_count	bigint	已被成功归档的 WAL 文件数量
last_archived_wal	text	最后一个被成功归档的 WAL 文件名称
last_archived_time	timestamp with time zone	最后一次成功归档操作的时间
failed_count	bigint	失败的归档 WAL 文件尝试的数量
last_failed_wal	text	最后一次失败的归档操作的 WAL 文件名称
last_failed_time	timestamp with time zone	最后一次失败的归档操作的时间
stats_reset	timestamp with time zone	这些统计信息最后一次被重置的时间

1.1.4.5　sys_stat_bgwriter

只有一行，记录有关后台写进程的活动的统计信息。

表 1.1.143: sys_stat_bgwriter 列

名称	类型	描述
checkpoints_timed	bigint	已经被执行的计划中检查点的数量
checkpoints_req	bigint	已经被执行的请求检查点的数量
checkpoint_write_time	double precision	在文件被写入磁盘的检查点处理部分花费的总时间，以毫秒计
checkpoint_sync_time	double precision	在文件被同步到磁盘中的检查点处理部分花费的总时间，以毫秒计
buffers_checkpoint	bigint	在检查点期间被写的缓冲区数目
buffers_clean	bigint	被后台写进程写的缓冲区数目
maxwritten_clean	bigint	后台写进程由于已经写了太多缓冲区而停止清洁扫描的次数
buffers_backend	bigint	被一个后端直接写的缓冲区数量
buffers_backend_fsync	bigint	一个后端不得不自己执行 fsync 调用的次数（通常即使后端自己进行写操作，后台写进程也会处理这些）
buffers_alloc	bigint	被分配的缓冲区数量
stats_reset	timestamp with time zone	这些统计信息上次被重置的时间

1.1.4.6　sys_stat_cached_plans

数据库实例执行计划缓存情况统计。

表 1.1.144: sys_stat_cached_plans 字段

名称	类型	描述
query	text	SQL 语句的文本
command	text	SQL 语句的类型
num_params	bigint	SQL 语句的绑定变量个数
cursor_options	bigint	SQL 语句的游标选项标记位，可以在游标级别指定缓存模式

-续表-

名称	类型	描述
soft_parse	bigint	该游标软解析次数
hard_parse	bigint	该游标硬解析次数
ref_count	bigint	该游标被引用次数
is_oneshot	boolean	是否为 one-shot 执行计划
is_saved	boolean	缓存计划是否已经保存
is_valid	boolean	是否可用，如关系表重建后游标自动失效
generic_cost	double precision	通用执行计划的成本，-1 表示未知
total_custom_cost	double precision	custom 执行计划的总成本
num_custom_plans	bigint	custom 执行计划总数

1.1.4.7　sys_stat_database

每个数据库对应一行，记录数据库范围的统计信息。

表 1.1.145: sys_stat_database 列

名称	类型	描述
datid	oid	一个数据库的 OID，或属于共享关系的对象为 0
datname	name	此数据库的名称，或共享对象的 NULL
numbackends	integer	当前连接到这个数据库的后端数量，或者 NULL 为共享对象。这是此视图中唯一返回反映当前状态的值的列；所有其他列都返回上次重置后的累积值
xact_commit	bigint	在这个数据库中已经被提交的事务的数量
xact_rollback	bigint	在这个数据库中已经被回滚的事务的数量
blks_read	bigint	在这个数据库中被读取的磁盘块的数量

-续表-

名称	类型	描述
blks_hit	bigint	磁盘块被发现已经在缓冲区中的次数，这样不需要一次读取（这只包括 PostgreSQL 缓冲区中的命中，而不包括在操作系统文件系统缓冲区中的命中）
tup_returned	bigint	在这个数据库中被查询返回的行数
tup_fetched	bigint	在这个数据库中被查询取出的行数
tup_inserted	bigint	在这个数据库中被查询插入的行数
tup_updated	bigint	在这个数据库中被查询更新的行数
tup_deleted	bigint	在这个数据库中被查询删除的行数
conflicts	bigint	由于与恢复冲突而在这个数据库中被取消的查询的数目（冲突只发生在后备服务器上，详见 sys_stat_database_conflicts）
temp_files	bigint	在这个数据库中被查询创建的临时文件的数量 所有临时文件都被统计，不管为什么创建这些临时文件（如排序或哈希），并且不管 log_temp_files 设置
temp_bytes	bigint	在这个数据库中被查询写到临时文件中的数据总量 所有临时文件都被统计，不管为什么创建这些临时文件（如排序或哈希），并且不管 log_temp_files 设置
deadlocks	bigint	在这个数据库中被检测到的死锁数
checksum_failures	bigint	在此数据库（或共享对象上）检测到的数据页校验和失败的次数，如果没有启用数据校验和，则为 NULL
checksum_last_failure	timestamp with time zone	在此数据库（或共享对象上）检测到最后一个数据页校验和失败的时间，如果没有启用数据校验和，则为 NULL
blk_read_time	double precision	在这个数据库中后端花费在读取数据文件块的时间，以毫秒计
blk_write_time	double precision	在这个数据库中后端花费在写数据文件块的时间，以毫秒计
stats_reset	timestamp with time zone	这些统计信息上次被重置的时间

1.1.4.8　sys_stat_database_conflicts

每个数据库对应一行，显示数据库范围的统计信息，这些信息的内容是关于由于与后备服务器的恢复过程发生冲突而被取消的查询。

表 1.1.146: sys_stat_database_conflicts 列

名称	类型	描述
datid	oid	一个数据库的 OID
datname	name	这个数据库的名称
confl_tablespace	bigint	这个数据库中由于表空间被删掉而取消的查询数量
confl_lock	bigint	这个数据库中由于锁超时而取消的查询数量
confl_snapshot	bigint	这个数据库中由于旧快照而取消的查询数量
confl_bufferpin	bigint	这个数据库中由于被占用的缓冲区而取消的查询数量
confl_deadlock	bigint	这个数据库中由于死锁而取消的查询数量

1.1.4.9　sys_stat_dbtime

将 DB Time 分解为 Foreground Wait Time 和 DB CPU，并进一步将 DB CPU 分解为解析、计划、重写、执行等关键活动时间和报文时间。

使用约束

sys_stat_dbtime 视图随底层统计视图的数据实时刷新，因此必须开启相关 GUC 参数，才能正常使用该功能。

- 底层统计视图：sys_stat_sqltime，sys_stat_sqlwait和sys_stat_instevent。
- GUC 参数：在 kingbase.conf 中添加以下设置。

```
track_sql = on
track_instance = on
```

表 1.1.147: sys_stat_dbtime 字段

名称	类型	描述
metric	text	统计项

-续表-

名称	类型	描述
calls	bigint	执行次数
total_time	bigint	总时间，单位为微秒
avg_time	numeric	平均时间，单位为微秒
dbtime_pct	numeric	占 DB Time 的百分比

sys_stat_dbtime 视图提供的统计项有：

- DB Time 总体时间概述：
  - DB Time：客户端会话执行 SQL 语句的总时间
  - Foreground Wait Time：客户端会话非空闲等待时间
  - DB CPU：DB Time - Foreground Wait Time
- 报文时间：
  - Simple Message：简单查询报文时间
  - Parse Message：Parse 报文时间
  - Bind Message：Bind 报文时间
  - Execute Message：Execute 报文时间
  - Sync Message：Sync 报文时间
  - Flush Message：Flush 报文时间
  - Describe Message：Describe 报文时间
  - Fastpath Message：Fastpath 报文时间
  - Wait Message：Wait 报文时间
  - Close Message：Close 报文时间
- 关键动作时间：
  - Parse Event：语法解析时间
  - Analyze Event：语义分析时间
  - Rewrite Event：重写时间
  - Plan Event：计划时间
  - Execute Event：执行时间

- PLSQL Compile：PL/SQL 编译时间
- PLSQL Execute：PL/SQL 执行时间
- PLPGSQL Compile：PLPGSQL 编译时间
- PLPGSQL Execute：PLPGSQL 执行时间
- Net Read：客户端会话网络读时间
- Net Write：客户端会话网络写时间

1.1.4.10　sys_stat_dmlcount

统计数据库中 DML 语句的执行次数和时间。

表 1.1.148: sys_stat_dmlcount 字段

名称	类型	描述
datid	oid	数据库的 OID
sql_type	text	SQL 语句类型
background	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	执行次数
times	bigint	执行时间，单位为微秒

1.1.4.11　sys_stat_gssapi

每个连接（常规和复制）有一行，显示关于 GSSAPI 验证和加密的信息。

表 1.1.149: sys_stat_gssapi 列

名称	类型	描述
pid	integer	后台的进程 ID
gss_authenticated	boolean	如果此连接使用了 GSSAPI 认证，则为 true
principal	text	如果没有使用 GSSAPI 来验证此连接，则为 NULL 如果 principal 长于 NAMEDATALEN（标准构建中为 64 个字符），则该字段被截断

-续表-

名称	类型	描述
encrypted	boolean	如果此连接上使用了 GSSAPI 加密，则为 True

1.1.4.12　sys_stat_instevent

统计数据库实例与关键活动相关的信息。

表 1.1.150: sys_stat_instevent 字段

名称	类型	描述
datid	oid	数据库的 OID
event_type	text	关键活动类型
event_name	text	关键活动名称
background	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	执行次数
times	bigint	执行时间，单位为微秒

1.1.4.13　sys_stat_instio

统计数据库实例与 I/O 相关的信息。

表 1.1.151: sys_stat_instio 字段

名称	类型	描述
backend_type	text	后端进程类型
datid	oid	后端进程所连接数据库的 OID
reltablespace	oid	关系表表空间的 OID
relid	oid	关系表的 OID

-续表-

名称	类型	描述
io_type	text	I/O 类型
file_type	text	文件类型
wait_event	text	等待事件名称
background	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	I/O 次数
times	bigint	I/O 时间，单位为微秒
bytes	bigint	I/O 字节数

1.1.4.14　sys_stat_instlock

统计数据库实例与锁相关的信息。

表 1.1.152: sys_stat_instlock 字段

名称	类型	描述
datid	oid	数据库的 OID
lock_name	text	锁的名称
lock_mode	text	锁的模式
acquire_type	text	锁的获取类型
background	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	执行次数
nowait_gets	bigint	未等待获得锁的次数
nowait_miss	bigint	未等待获得锁失败的次数
wait_gets	bigint	等待获得锁的次数
wait_miss	bigint	等待获得锁失败的次数
wait_times	bigint	wait_gets 和 wait_miss 两种情况的和

1.1.4.15　sys_stat_metric

统计数据库中度量的实时测量值。

表 1.1.153: sys_stat_metric 字段

名称	类型	描述
begin_time	timestamp with time zone	采样周期开始时间
end_time	timestamp with time zone	采样周期结束时间
intsize_csec	integer	采样周期时间，单位为秒
group_id	integer	度量分组的内部 ID
metric_id	integer	度量 ID
metric_name	text	度量名称
metric_unit	text	度量单元
metric_value	numeric	度量值
rel_value	bigint	辅助的度量值：采样期间的相对值
abs_value	bigint	辅助的度量值：结束时刻的绝对值

1.1.4.16　sys_stat_metric_group

统计数据库中度量分组的说明。

表 1.1.154: sys_stat_metric_group 字段

名称	类型	描述
group_id	integer	度量分组的内部 ID
group_name	text	度量分组的外部名称
intsize_csec	integer	采样周期时间，单位为秒
max_interval	integer	最大采样数

1.1.4.17　sys_stat_metric_history

记录数据库度量值的历史信息，仅保留最近一个小时的数据。

表 1.1.155: sys_stat_metric_history 字段

名称	类型	描述
begin_time	timestamp with time zone	采样周期开始时间
end_time	timestamp with time zone	采样周期结束时间
intsize_csec	integer	采样周期时间，单位为秒
group_id	integer	度量分组的内部 ID
metric_id	integer	度量 ID
metric_name	text	度量名称
metric_unit	text	度量单元
metric_value	numeric	度量值
rel_value	bigint	辅助的度量值：采样期间的相对值
abs_value	bigint	辅助的度量值：结束时刻的绝对值

1.1.4.18　sys_stat_metric_name

统计数据库中度量值的基本信息。

表 1.1.156: sys_stat_metric_name 字段

名称	类型	描述
group_id	integer	度量分组的内部 ID
group_name	text	度量分组的外部名称
metric_id	integer	度量 ID
metric_name	text	度量名称

-续表-

名称	类型	描述
metric_unit	text	度量单元

1.1.4.19　sys_stat_msgaccum

与sys_stat_sqltime视图的视图结构类似，但只显示sys_stat_sqltime视图的 message、calls 和 times 字段。

表 1.1.157: sys_stat_msgaccum 字段

名称	类型	描述
message	text	报文类型
calls	bigint	次数
times	bigint	时间

1.1.4.20　sys_stat_pre_archivewal

查询所有预归档 WAL 日志统计信息。有效查询归档进度，帮助用户进一步了解归档运行情况。可帮助用户提前一步预估预归档路径所在的磁盘开销，防止预归档文件过多导致磁盘撑爆的场景。

表 1.1.158: sys_stat_pre_archivewal 列

名称	类型	描述
path	text	数据库存放预归档 WAL 日志路径，固定显示为 sys_wal
name	text	预归档 WAL 文件名，即正在执行归档过程但是还未完成归档的 WAL 文件
size	bigint	预归档 WAL 文件大小
timeline	bigint	预归档 WAL 文件时间线
system_id	bigint	预归档 WAL 文件系统标识符

-续表-

名称	类型	描述
start_lsn	pg_lsn	预归档 WAL 文件起始 LSN 位置
end_lsn	pg_lsn	预归档 WAL 文件末尾 LSN 位置
modification	timestamp with time zone	预归档 WAL 文件最近一次被修改时间

1.1.4.21　sys_stat_progress_checkpoint

用于查询检查点（CHECKPOINT）的执行进度。

表 1.1.159: sys_stat_progress_checkpoint 列

名称	类型	描述
pid	integer	checkpointer 进程的 pid 号。
phase	text	检查点当前所处的阶段，取值如下： prepareflush dirty clog pagesflush dirty commit_ts pagesflush dirty subtrans pagesflush dirty multi transaction pagestruncate slruflush relation mapflush replication slotsremove serialized snapshotslogical rewrite mappingswrite out all dirty buffersprocess queued fsync requestswrite replorigin_checkpoint filehandle Two-phase commitremove lingering filesrecycle or remove log filesprealloc log filestruncate subtransaction segmentsskip
flags	text	检查点的触发类型 {restartpointcheckpoint} starting:{shutdownend-of-recoveryimmediateforcewaitwaltime flush-all}
buffers_scan	bigint	扫描出的脏块数量。取值范围：0 -NBuffers，其中，NBuffers 最大不会超过 2147483647
buffers_processed	bigint	处理过的脏块数量。取值范围：0 -NBuffers，其中，NBuffers 最大不会超过 2147483647
buffers_written	bigint	写回磁盘的脏块数量。取值范围：0 -NBuffers，其中，NBuffers 最大不会超过 2147483647。 由于其他进程也可能会写回脏块，所以 buffer_written 的数量可能会比 buffer_processed 的数量小

-续表-

名称	类型	描述
written_progress	text	处理进度百分比，展示当前处理脏块的进度。其计算公式为：(buffers_processed/buffers_scan)*100.0
written_rate	text	检查点写脏块的速率。系统会根据速率大小会自动调整显示单位，支持的单位有：MB/s、KB/s、B/s
start_time	text	开始执行检查点的时间

1.1.4.22　sys_stat_progress_cluster

记录每个运行着 CLUSTER 或 VACUUM FULL 的进度。

表 1.1.160: sys_stat_progress_cluster 列

名称	类型	描述
pid	integer	后台的进程 ID
datid	oid	该后端连接的数据库的 OID
datname	name	与此后端连接的数据库的名称
relid	oid	被聚类的表的 OID
command	text	正在运行的命令。CLUSTER 或 VACUUM FULL
phase	text	当前处理阶段
cluster_index_relid	oid	如果正在使用索引对表进行扫描，这就是正在使用的索引的 OID；否则为 0
heap_tuples_scanned	bigint	扫描的堆元组数 这个计数器只有在阶段为 seq scanning heap,index scanningheap or writing new heap 时才会前进
heap_tuples_written	bigint	写入的堆元组的数量 这个计数器只有在阶段为 seq scanning heap, index scanningheap or writing new heap 时才会前进
heap_blks_total	bigint	表中的堆块总数 这个数字是在 seq 扫描堆的开始时报告的

-续表-

名称	类型	描述
heap_blks_scanned	bigint	扫描的堆块数量 这个计数器只有在阶段为 seq 扫描堆时才会前进
index_rebuild_count	bigint	重建的索引数 该计数器仅在重建索引阶段时才会前进

1.1.4.23　sys_stat_progress_create_index

记录每个后台运行 CREATE INDEX 或 REINDEX 的进度。

表 1.1.161: sys_stat_progress_create_index 列

名称	类型	描述
pid	integer	后端进程 ID
datid	oid	该后端连接的数据库的 OID
datname	name	与此后端连接的数据库的名称
relid	oid	创建索引所依据的表的 OID
index_relid	oid	被创建或重新索引的索引的 OID。在非当前的 CREATEINDEX 中，这个值为 0
command	text	正在运行的命令： CREATE INDEX，CREATE INDEX CONCURRENTLY REINDEX，REINDEX CONCURRENTLY
phase	text	索引创建的当前处理阶段
lockers_total	bigint	在适用的情况下，需要等待的锁总数
lockers_done	bigint	已经等待的锁数量
current_locker_pid	bigint	目前正在等待的锁进程 ID
blocks_total	bigint	本阶段要处理的区块总数
blocks_done	bigint	当前阶段已经处理的区块数量
tuples_total	bigint	当前阶段要处理的元组总数

-续表-

名称	类型	描述
tuples_done	bigint	在当前阶段已经处理的元组数量
partitions_total	bigint	在分区表上创建索引时，该列被设置为要在其上创建索引的分区总数
partitions_done	bigint	当在分区表上创建索引时，该列被设置为在其上完成索引的分区数

1.1.4.24　sys_stat_progress_vacuum

记录每个运行着 VACUUM 的后端（包括 autovacuum 工作者进程）的进度。

表 1.1.162: sys_stat_progress_vacuum 列

名称	类型	描述
pid	integer	后端的进程 ID
datid	oid	这个后端连接的数据库的 OID
datname	name	这个后端连接的数据库的名称
relid	oid	被 vacuum 的表的 OID
phase	text	vacuum 的当前处理阶段
heap_blks_total	bigint	该表中堆块的总数。 这个数字在扫描开始时报告，之后增加的块将不会（并且不需要）被这个 VACUUM 访问
heap_blks_scanned	bigint	被扫描的堆块数量 由于可见性映射被用来优化扫描，一些块将被跳过而不做检查，被跳过的块会被包括在这个总数中，因此当清理完成时这个数字最终将会等于 heap_blks_total 仅当处于扫描堆阶段时这个计数器才会前进
heap_blks_vacuumed	bigint	被清理的堆块数量。除非表没有索引，这个计数器仅在处于清理堆阶段时才会前进。 不包含死亡元组的块会被跳过，因此这个计数器可能有时会向前跳跃一个比较大的增量

-续表-

名称	类型	描述
index_vacuum_count	bigint	已完成的索引清理周期数
max_dead_tuples	bigint	在需要执行一个索引清理周期之前我们可以存储的死亡元组数，取决于 maintenance_work_mem
num_dead_tuples	bigint	从上一个索引清理周期以来收集的死亡元组数

1.1.4.25　sys_stat_replication

每一个 WAL 发送进程一行，显示有关到该发送进程连接的后备服务器的复制的统计信息。

表 1.1.163: sys_stat_replication 列

名称	类型	描述
pid	integer	一个 WAL 发送进程的进程 ID
usesysid	oid	登录到这个 WAL 发送进程的用户的 OID
usename	name	登录到这个 WAL 发送进程的用户的名称
application_name	text	连接到这个 WAL 发送进程的应用的名称
client_addr	inet	连接到这个 WAL 发送进程的客户端的 IP 地址 如果这个域为空，它表示该客户端通过服务器机器上的一个 Unix 套接字连接
client_hostname	text	连接上的客户端的主机名，由一次对 client_addr 的逆向 DNS 查找报告这个域将只对 IP 连接非空，并且只有在 log_hostname 被启用时非空
client_port	integer	客户端用来与这个 WAL 发送进程通讯的 TCP 端口号，如果使用 Unix 套接字则为-1
backend_start	timestamp with time zone	这个进程开始的时间，即客户端是何时连接到这个 WAL 发送进程的
backend_xmin	xid	由 hot_standby_feedback 报告的这个后备机的 xmin 水平线

-续表-

名称	类型	描述
state	text	当前的 WAL 发送进程状态。 可能的值是： startup：这个 WAL 发送器正在启动 catchup：这个 WAL 发送器连接的后备机正在追赶主服务器 streaming：这个 WAL 发送器在它连接的后备服务器追上主服务器之后用流传送更改 backup：这个 WAL 发送器正在发送一个备份 stopping：这个 WAL 发送器正在停止
sent_lsn	pg_lsn	在这个连接上发送的最后一个预写式日志的位置
write_lsn	pg_lsn	被这个后备服务器写入到磁盘的最后一个预写式日志的位置
flush_lsn	pg_lsn	被这个后备服务器刷入到磁盘的最后一个预写式日志的位置
replay_lsn	pg_lsn	被重放到这个后备服务器上的数据库中的最后一个预写式日志的位置
write_lag	interval	在本地刷写近期的 WAL 与接收到后备服务器已经写入它（但还没有刷写或者应用）的通知之间流逝的时间 如果这台服务器被配置为一个同步后备，这可以用来计量在提交时 synchronous_commit 的级别 remote_write 所导致的延迟
flush_lag	interval	在本地刷写近期的 WAL 与接收到后备服务器已经写入并且刷写它（但还没有应用）的通知之间流逝的时间 如果这台服务器被配置为一个同步后备，这可以用来计量在提交时 synchronous_commit 的级别 on 所导致的延迟
replay_lag	interval	在本地刷写近期的 WAL 与接收到后备服务器已经写入它、刷写它并且应用它的通知之间流逝的时间 如果这台服务器被配置为一个同步后备，这可以用来计量在提交时 synchronous_commit 的级别 remote_apply 所导致的延迟
sync_priority	integer	在基于优先的同步复制中，这台后备服务器被选为同步后备的优先级 在基于规定数量的同步复制中，这个值没有效果
sync_state	text	这一台后备服务器的同步状态。 可能的值是： async：这台后备服务器是异步的 potential：这台后备服务器现在是异步的，但可能在当前的同步后备失效时变成同步的 sync：这台后备服务器是同步的 quorum：这台后备服务器被当做规定数量后备服务器的候选
reply_time	timestamp with time zone	从备用服务器收到的最后一条回复信息的发送时间

1.1.4.26　sys_stat_shmem

统计数据库中的共享内存信息。

表 1.1.164: sys_stat_shmem 字段

名称	类型	描述
name	text	共享内存的名称
size	bigint	共享内存大小，单位为字节

1.1.4.27　sys_stat_sql

按数据库时间模型展示 TOP SQL。

使用约束

sys_stat_sql 视图随底层统计视图的数据实时刷新，按 DB Time 排序，因此必须开启相关 GUC 参数，才能正常使用该功能。

- 底层统计视图：sys_stat_sqltime，sys_stat_sqlwait，sys_stat_sqlio和sys_stat_statements。
- GUC 参数：在 kingbase.conf 中添加以下设置。

```
track_sql = on
sys_stat_statements.track = 'top'
```

表 1.1.165: sys_stat_sql 字段

名称	类型	描述
datid	oid	数据库的 OID
datname	text	数据库的名称
userid	oid	用户的 ID
username	text	用户的名称
queryid	bigint	SQL 语句的 ID
query	text	SQL 语句的文本
calls	bigint	执行次数

-续表-

名称	类型	描述
rows	bigint	执行该 SQL 语句返回的元组数
db_time	bigint	DB Time，单位为微秒
db_cpu	bigint	DB CPU，单位为微秒
db_wait	bigint	FG Wait 时间，单位为微秒
total_db_time_pct	numeric	该 SQL 语句的 DB Time 占总 DB Time 的百分比
cpu_time_pct	numeric	该 SQL 语句 DB CPU 占 DB Time 的百分比
wait_time_pct	numeric	等待时间占该 SQL 语句 DB Time 的百分比
wait_event_1	text	等待时间最长的等待事件
wait_calls_1	bigint	等待时间最长等待事件的等待次数
wait_time_1	bigint	等待时间最长等待事件的等待时间，单位为微秒
wait_time_pct_1	numeric	等待时间最长等待事件的等待时间占该 SQL 语句 DB Time 的百分比
wait_event_2	text	等待时间次长的等待事件
wait_calls_2	bigint	等待时间次长等待事件的等待次数
wait_time_2	bigint	等待时间次长等待事件的等待时间，单位为微秒
wait_time_pct_2	numeric	等待时间次长等待事件的等待时间占该 SQL 语句 DB Time 的百分比
parse_calls	bigint	解析次数
parse_time	bigint	解析阶段花费的时间，单位为微秒
parse_time_pct	numeric	解析阶段花费的时间占该 SQL 语句 DB Time 的百分比
plan_calls	bigint	计划次数
plan_time	bigint	计划阶段花费的时间，单位为微秒
plan_time_pct	numeric	计划阶段花费的时间占该 SQL 语句 DB Time 的百分比
exec_calls	bigint	执行次数
exec_time	bigint	执行阶段花费的时间，单位为微秒

-续表-

名称	类型	描述
exec_time_pct	numeric	执行阶段花费的时间占该 SQL 语句 DB Time 的百分比
wal_size	bigint	写 WAL 日志文件的大小，单位为字节
shared_blks_read_size	bigint	读共享块的大小，单位为字节
shared_blks_write_size	bigint	写共享块的大小，单位为字节
local_blks_read_size	bigint	读本地块的大小，单位为字节
local_blks_write_size	bigint	写本地块的大小，单位为字节
temp_blks_read_size	bigint	读临时块的大小，单位为字节
temp_blks_write_size	bigint	写临时块的大小，单位为字节
shared_blks_hit	numeric	共享块缓存命中率
local_blks_hit	numeric	本地块缓存命中率
blks_read_time	bigint	读共享块、本地块和临时块花费的时间，单位为微秒
blks_write_time	bigint	写共享块、本地块和临时块花费的时间，单位为微秒

1.1.4.28　sys_stat_sqlcount

统计数据库中不同类型 SQL 语句的执行次数和时间。

表 1.1.166: sys_stat_sqlcount 字段

名称	类型	描述
datid	oid	数据库的 OID
sql_type	text	SQL 语句类型
background	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	执行次数
times	bigint	执行时间，单位为微秒

1.1.4.29　sys_stat_sqlio

统计数据库中 SQL 语句与 I/O 相关的信息。

表 1.1.167: sys_stat_sqlio 字段

名称	类型	描述
userid	oid	用户的 ID
datid	oid	数据库的 OID
queryid	bigint	SQL 语句的 ID
bgio	boolean	是否为后台工作进程，如果是，则为 TRUE
wal_records	bigint	语句生成的 WAL 记录总数
wal_fpi	bigint	语句生成的 WAL 全页图像总数
wal_bytes	bigint	语句生成的 WAL 记录的总大小，单位为字节
shared_blks_hit	bigint	语句的共享块缓存命中总数
shared_blks_read	bigint	语句读取的共享块总数
shared_blks_dirtied	bigint	被语句弄脏的共享块总数
shared_blks_written	bigint	语句写入的共享块总数
local_blks_hit	bigint	语句的本地块缓存命中总数
local_blks_read	bigint	语句读取的本地块总数
local_blks_dirtied	bigint	被语句弄脏的本地块总数
local_blks_written	bigint	语句写入的本地块总数
temp_blks_read	bigint	语句读取的临时块总数
temp_blks_written	bigint	语句写入的临时块总数
blk_read_time	bigint	语句读取块花费的总时间，单位为微秒
blk_write_time	bigint	语句写入块花费的总时间，单位为微秒

1.1.4.30　sys_stat_sqltime

统计数据库中 SQL 语句与报文相关的信息。

表 1.1.168: sys_stat_sqltime 字段

名称	类型	描述
userid	oid	用户的 ID
datid	oid	数据库的 OID
queryid	bigint	SQL 语句的 ID
message	text	报文类型
bgmsg	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	报文的发生次数
times	bigint	处理报文花费的总时间，单位为微秒

1.1.4.31　sys_stat_sqlwait

统计数据库中 SQL 语句与等待事件相关的信息。

表 1.1.169: sys_stat_sqlwait 字段

名称	类型	描述
userid	oid	用户的 ID
datid	oid	数据库的 OID
queryid	bigint	SQL 语句的 ID
wait_event_type	text	等待事件的类型
wait_event	text	等待事件的名称
bgwait	boolean	是否为后台工作进程，如果是，则为 TRUE
calls	bigint	等待事件的发生次数
times	bigint	等待事件消耗的总时间，单位为微秒

1.1.4.32　sys_stat_ssl

每个连接（常规的或者复制）一行，显示在这个连接上使用的 SSL 的信息。

表 1.1.170: sys_stat_ssl 列

名称	类型	描述
pid	integer	一个后端或者 WAL 发送进程的进程 ID
ssl	boolean	如果在这个连接上使用了 SSL 则为真
version	text	在用的 SSL 版本，如果这个连接上没有使用 SSL 则为 NULL
cipher	text	在用的 SSL 密码的名称，如果这个连接上没有使用 SSL 则为 NULL
bits	integer	使用的加密算法中的位数，如果这个连接上没有使用 SSL 则为 NULL
compression	boolean	如果使用了 SSL 压缩则为真，否则为假， 如果这个连接上没有使用 SSL 则为 NULL
client_dn	text	来自所使用的客户端证书的识别名（DN）域 如果没有提供客户端证书或者这个连接上没有使用 SSL 则为 NULL 如果 DN 域长度超过 NAMEDATALEN（标准编译中是 64 个字符），则它会被截断
client_serial	numeric	客户端证书的序列号，如果没有提供客户端证书，或者在此连接上不使用 SSL，则为 NULL。 证书序列号和证书发行者的组合可以唯一地识别一个证书（除非发行者错误地重复使用序列号）
issuer_dn	text	客户端证书的发行者的 DN，如果没有提供客户端证书，或者如果这个连接上没有使用 SSL，则为 NULL。 这个字段被截断，就像 client_dn 一样

1.1.4.33　sys_stat_subscription

每个订阅至少一行，显示有关该订阅的工作者的信息。

表 1.1.171: sys_stat_subscription 列

名称	类型	描述
subid	oid	订阅的 OID
subname	name	订阅的名称
pid	integer	订阅工作者进程的进程 ID
relid	oid	工作者正在同步的关系的 OID，对于主应用工作者为空
received_lsn	pg_lsn	接收到的最后一个预写式日志位置，这个字段的初始值是 0
last_msg_send_time	timestamp with time zone	从源头 WAL 发送器接收到的最后一个消息的发送时间
last_msg_receipt_time	timestamp with time zone	从源头 WAL 发送器接收到的最后一个消息的接收时间
latest_end_lsn	pg_lsn	最后一个报告给源头 WAL 发送器的预写式日志位置
latest_end_time	timestamp with time zone	报告给源头 WAL 发送器的最后一个预写式日志位置的时间

1.1.4.34　sys_stat_sys_indexes

和sys_stat_all_indexes 一样，但只显示系统表上的索引。

1.1.4.35　sys_stat_sys_tables

和sys_stat_all_tables 一样，但只显示系统表。

1.1.4.36　sys_stat_sysmetric

统计数据库度量值中的实时 TPS、QPS 统计。

表 1.1.172: sys_stat_sysmetric 字段

名称	类型	描述
begin_time	timestamp with time zone	采样周期开始时间
end_time	timestamp with time zone	采样周期结束时间
intsize_csec	integer	采样周期时间，单位为秒
group_id	integer	度量分组的内部 ID
metric_id	integer	度量 ID
metric_name	text	度量名称
metric_unit	text	度量单元
metric_value	numeric	度量值
rel_value	bigint	辅助的度量值：采样期间的相对值
abs_value	bigint	辅助的度量值：结束时刻的绝对值

1.1.4.37　sys_stat_sysmetric_history

记录数据库度量值中实时 TPS、QPS 统计的历史信息，仅保留最近一个小时的数据。

表 1.1.173: sys_stat_sysmetric_history 字段

名称	类型	描述
begin_time	timestamp with time zone	采样周期开始时间
end_time	timestamp with time zone	采样周期结束时间
intsize_csec	integer	采样周期时间，单位为秒
group_id	integer	度量分组的内部 ID
metric_id	integer	度量 ID
metric_name	text	度量名称
metric_unit	text	度量单元

-续表-

名称	类型	描述
metric_value	numeric	度量值
rel_value	bigint	辅助的度量值：采样期间的相对值
abs_value	bigint	辅助的度量值：结束时刻的绝对值

1.1.4.38　sys_stat_sysmetric_summary

数据库度量值的历史记录汇总。

表 1.1.174: sys_stat_sysmetric_summary 字段

名称	类型	描述
begin_time	timestamp with time zone	采样周期开始时间
end_time	timestamp with time zone	采样周期结束时间
intsize_csec	integer	采样周期时间，单位为秒
group_id	integer	度量分组的内部 ID
metric_id	integer	度量 ID
metric_name	text	度量名称
num_interval	bigint	采样数量
maxval	numeric	采样的最大值
minval	numeric	采样的最小值
average	numeric	采样的平均值
standard_devation	numeric	采样的标准方差
metric_unit	text	度量单元

1.1.4.39　sys_stat_transaction

每个数据库一行，显示前后台事务的统计信息。

表 1.1.175: sys_stat_transaction 列

名称	类型	描述
datid	oid	一个数据库的 OID，或属于共享关系的对象为 0
datname	name	数据库的名称，或共享对象的 NULL
xact_commit	bigint	在这个数据库中已经被提交的事务的数量
xact_rollback	bigint	在这个数据库中已经被回滚的事务的数量
fg_xact_commit	bigint	在这个数据库中已经被提交的前台进程事务的数量
fg_xact_rollback	bigint	在这个数据库中已经被回滚的前台进程事务的数量
bg_xact_commit	bigint	在这个数据库中已经被提交的后台进程事务的数量
bg_xact_rollback	bigint	在这个数据库中已经被回滚的后台进程事务的数量
stats_reset	timestamp with time zone	这些统计信息上次被重置的时间

1.1.4.40　sys_stat_user_functions

每一个被跟踪的函数对应视图一行，记录与执行该函数有关的统计信息。

表 1.1.176: sys_stat_user_functions 列

名称	类型	描述
funcid	oid	一个函数的 OID
schemaname	name	这个函数所在的模式的名称
funcname	name	这个函数的名称
calls	bigint	这个函数已经被调用的次数
total_time	double precision	在这个函数以及它所调用的其他函数中花费的总时间，以毫秒计
self_time	double precision	在这个函数本身花费的总时间，不包括被它调用的其他函数，以毫秒计

1.1.4.41　sys_stat_user_indexes

和sys_stat_all_indexes 一样，但只显示用户表上的索引。

1.1.4.42　sys_stat_user_tables

和sys_stat_all_tables 一样，但只显示用户表。

1.1.4.43　sys_stat_wait

该视图用于分析客户端会话进程的等待事件，帮助数据库管理员找到等待时间最多的关键等待事件。

使用约束

sys_stat_wait 视图随底层统计视图的数据实时刷新，因此必须开启相关 GUC 参数，才能正常使用该功能。

- 底层统计视图：sys_stat_sqltime和sys_stat_sqlwait。
- GUC 参数：在 kingbase.conf 中添加以下设置。

```
track_sql = on
```

表 1.1.177: sys_stat_wait 字段

名称	类型	描述
wait_event	text	等待事件的名称
calls	bigint	等待事件的发生次数
total_time	bigint	等待事件消耗的总时间，单位为微秒
avg_time	numeric	等待事件消耗的平均时间，单位为微秒
dbtime_pct	numeric	占 DB Time 的百分比
event_type	text	等待事件的类型

1.1.4.44　sys_stat_waitaccum

与sys_stat_sqlwait视图的视图结构类似，但不显示sys_stat_sqlwait视图的 userid 和 bgwait 字段。

表 1.1.178: sys_stat_waitaccum 字段

名称	类型	描述
datid	oid	数据库的 OID
queryid	bigint	SQL 语句的 ID
wait_event_type	text	等待事件的类型
wait_event	text	等待事件的名称
calls	bigint	等待事件的发生次数
times	bigint	等待事件消耗的总时间，单位为微秒

1.1.4.45　sys_stat_wal_buffer

sys_stat_wal_buffer 视图统计了 WAL BUFFER 实时的运作情况。

表 1.1.179: sys_stat_wal_buffer 列

名称	类型	描述
name	text	WAL 日志缓冲区名称，固定显示为“WAL BUFFER”
bytes	bigint	WALBUFFER 缓存空间总长度，此字段值与 Kingbase.conf 配置文件中 wal_buffers 参数关联，单位为字节
utilization_rate	text	WAL BUFFER 缓冲区使用率，此字段值表示当前 WALBUFFER 中待落盘的 WAL 日志总长度与整个 WAL BUFFER 缓冲区总长度之间的比值。如果此字段值长时间接近 100%，那么需要从两方面考虑来改善现状： （1）检查 WAL 日志文件所在磁盘 IO 状况。如果发现磁盘 IO 是性能瓶颈，那么就应该考虑使用磁盘 IO 性能更好的磁盘来存储 WAL 日志文件 （2）如果 WAL 日志文件所在磁盘 IO 不是性能瓶颈，那么可尝试适当增加 WALBUFFER 的尺寸来提升 WAL 日志的写性能
copied_to	text	后台进程最近写入 WAL BUFFER 的 WAL 日志在 WAL BUFFER 中的偏移。 注意的是，WAL BUFFER 是环形的数据结构，此字段值会在区间 [1,bytes] 之间循环取值

-续表-

名称	类型	描述
copied_to_lsn	pg_lsn	后台进程最近写入 WALBUFFER 的 WAL 日志的结束 LSN。此字段值在 KES 数据库运行过程中递增
coping_data_len	bigint	查询时刻后台进程正在写入 WAL BUFFER 的 WAL 日志总长度
written_to	text	最近一个从 WAL BUFFER 写入磁盘的 WAL 日志在 WAL BUFFER 中的偏移。 需要注意的是，WAL BUFFER 是环形的数据结构，此字段值会在区间 [1,bytes] 之间循环取值
written_to_lsn	pg_lsn	最近一个从 WAL BUFFER 写入磁盘的 WAL 日志的 LSN。此字段值在 KES 数据库运行过程中递增
writing_data_len	bigint	查询时刻正在从 WAL BUFFER 向磁盘刷写的 WAL 日志总长度
write_rate	text	两次相邻查询的时间间隔内，从 WALBUFFER 向磁盘的平均 WAL 日志刷写速率。 如果需要更精确的 writerate 值，那么可以将当前视图的查询时间间隔调小

1.1.4.46　sys_stat_wal_receiver

只有一行，显示来自 WAL 接收器所连接服务器的有关该接收器的统计信息。

表 1.1.180: sys_stat_wal_receiver 列

名称	类型	描述
pid	integer	WAL 接收器进程的进程 ID
status	text	WAL 接收器进程的活动状态
receive_start_lsn	pg_lsn	WAL 接收器启动时使用的第一个预写式日志位置
receive_start_tli	integer	WAL 接收器启动时使用的第一个时间线编号
received_lsn	pg_lsn	已经接收到并且已经被杀入磁盘的最后一个预写式日志的位置，这个域的初始值是 WAL 接收器启动时使用的第一个日志位置
received_tli	integer	已经接收到并且已经被杀入磁盘的最后一个预写式日志的时间线编号，这个域的初始值是 WAL 接收器启动时使用的第一个日志所在的时间线编号

-续表-

名称	类型	描述
last_msg_send_time	timestamp with time zone	从源头 WAL 发送器接收到的最后一个消息的发送时间
last_msg_receipt_time	timestamp with time zone	从源头 WAL 发送器接收到的最后一个消息的接收时间
latest_end_lsn	pg_lsn	报告给源头 WAL 发送器的最后一个预写式日志位置
latest_end_time	timestamp with time zone	报告给源头 WAL 发送器最后一个事务日志位置的时间
slot_name	text	这个 WAL 接收器使用的复制槽的名称
sender_host	text	这个 WAL 接收器连接到的 PostgreSQL 实例的主机。 这可以是一个主机名、一个 IP 地址，如果连接是通过 Unix 套接字则是一个目录路径（为目录的情况可以被辨别出来，因为路径将总是一个绝对路径并且以/开头）
sender_port	integer	这个 WAL 接收器连接到的 PostgreSQL 实例的端口号
conninfo	text	这个 WAL 接收器使用的连接串，安全相关的列会被隐去

1.1.4.47　sys_stat_xact_all_tables

和sys_stat_all_tables 相似，但计数动作只在当前事务内发生（还没有被包括在 sys_stat_all_tables 和相关视图中）。用于生存和死亡行数量的列以及清理和分析动作在此视图中不出现。

表 1.1.181: sys_stat_xact_all_tables 列

名称	类型	描述
relid	oid	一个表的 OID
schemaname	name	这个表所在的模式的名称
relname	name	这个表的名称
seq_scan	bigint	在这个表上发起的顺序扫描的次数

-续表-

名称	类型	描述
seq_tup_read	bigint	被顺序扫描取得的活着的行的数量
idx_scan	bigint	在这个表上发起的索引扫描的次数
idx_tup_fetch	bigint	被索引扫描取得的活着的行的数量
n_tup_ins	bigint	被插入的行数
n_tup_upd	bigint	被更新的行数（包括 HOT 更新的行）
n_tup_del	bigint	被删除的行数
n_tup_hot_upd	bigint	被更新的 HOT 行数（即不要求独立索引更新的行更新）

1.1.4.48　sys_stat_xact_sys_tables

和sys_stat_xact_all_tables 一样，但只显示系统表。

1.1.4.49　sys_stat_xact_user_functions

和sys_stat_user_functions 相似，但是只统计在当前事务期间的调用（还没有被包括在 sys_stat_user_functions 中）。

1.1.4.50　sys_stat_xact_user_tables

和sys_stat_xact_all_tables 一样，但只显示用户表。

1.1.4.51　sys_statio_all_indexes

当前数据库中的每个索引对应一行，该行显示指定索引上有关 I/O 的统计信息。

表 1.1.182: sys_statio_all_indexes 列

名称	类型	描述
relid	oid	这个索引的基表的 OID

-续表-

名称	类型	描述
indexrelid	oid	这个索引的 OID
schemaname	name	这个索引所在的模式的名称
relname	name	这个索引的基表的名称
indexrelname	name	这个索引的名称
idx_blks_read	bigint	从这个索引读取的磁盘块数
idx_blks_hit	bigint	在这个索引中的缓冲区命中数量

1.1.4.52　sys_statio_all_sequences

当前数据库中每个序列对象一行，显示：序列 OID、模式名、序列名、序列的磁盘读取总数、序列的缓冲区命中总数。当前数据库中的每个序列一行，显示与指定序列上的 I/O 有关的统计信息。

表 1.1.183: sys_statio_all_sequences 列

名称	类型	描述
relid	oid	一个序列的 OID
schemaname	name	这个序列所在的模式的名称
relname	name	这个序列的名称
blks_read	bigint	从这个序列中读取的磁盘块数
blks_hit	bigint	在这个序列中的缓冲区命中数量

1.1.4.53　sys_statio_all_tables

当前数据库中每个表对应一行 (包括 TOAST 表)，显示指定表上 I/O 的统计信息：表 OID、模式名、表名、从该表中读取的磁盘块总数、缓冲区命中次数、该表上所有索引的磁盘块读取总数、该表上所有索引的缓冲区命中总数、在该表的辅助 TOAST 表 (如果存在) 上的磁盘块读取总数、在该表的辅助 TOAST 表 (如果存在) 上的缓冲区命中总数、TOAST 表的索引的磁盘块读取总数、TOAST 表的索引的缓冲区命中总数。

表 1.1.184: sys_statio_all_tables 列

名称	类型	描述
relid	oid	一个表的 OID
schemaname	name	这个表所在的模式的名称
relname	name	这个表的名称
heap_blks_read	bigint	从这个表读取的磁盘块数量
heap_blks_hit	bigint	在这个表中的缓冲区命中数量
idx_blks_read	bigint	从这个表上所有索引中读取的磁盘块数
idx_blks_hit	bigint	在这个表上的所有索引中的缓冲区命中数量
toast_blks_read	bigint	从这个表的 TOAST 表（如果有）读取的磁盘块数
toast_blks_hit	bigint	在这个表的 TOAST 表（如果有）中的缓冲区命中数量
tidx_blks_read	bigint	从这个表的 TOAST 表索引（如果有）中读取的磁盘块数
tidx_blks_hit	bigint	在这个表的 TOAST 表索引（如果有）中的缓冲区命中数量

1.1.4.54　sys_statio_sys_indexes

视图结构和sys_statio_all_indexes一样，但只显示系统表上的索引。

1.1.4.55　sys_statio_sys_sequences

和sys_statio_all_sequences 一样，但只显示系统序列（目前没有定义系统序列，因此这个视图总是为空）。

1.1.4.56　sys_statio_sys_tables

和sys_statio_all_tables 一样，但只显示系统表。

1.1.4.57　sys_statio_user_indexes

视图结构和sys_statio_all_indexes 一样，但只显示用户表上的索引。

1.1.4.58 sys_statio_user_sequences

和sys_statio_all_sequences 一样，但只显示用户序列。

1.1.4.59 sys_statio_user_tables

和sys_statio_all_tables 一样，但只显示用户表。

1.1.4.60 sys_statistic

`sys_statistic` 视图存储有关数据库内容的统计数据。其中的项由 `ANALYZE` 创建，查询计划器会使用这些数据来进行查询计划。注意所有的统计数据天然就是近似的，即使它刚刚被更新。

通常对于数据表中一个已经被 ANALYZE 过的列，在本视图中会存在一个 `stainherit = false` 的项。如果该列所在的表具有后代（即有其他表继承该表），对于该列还会创建第二个 `stainherit = true` 的项。`stainherit = true` 的项表示列在整个继承树上的统计数据，即通过 SELECT `column` FROM table* 看到的数据的统计，而 `stainherit = false` 的项表示对 SELECT `column` FROM ONLY `table` 的结果的统计。

`sys_statistic` 也存储关于索引表达式值的统计数据，就好像它们是真正的数据列，但在这种情况中 `starelid` 指索引。对一个普通非表达式索引列不会创建项，因为它将是底层表列的项的冗余。当前，索引表达式的项都具有 `stainherit = false`。

因为不同类型的统计信息适用于不同类型的数据，`sys_statistic` 被设计成不太在意自己存储的是什么类型的统计。只有极为常用的统计信息（比如 NULL 的含量）才在 `sys_statistic` 里给予专用的字段。其它所有东西都存储在“槽位”中，而槽位是一组相关的列，它们的内容用槽位中的一个列里的代码表示。更详细的信息请参阅 `src/include/catalog/sys_statistic.h`。

`sys_statistic` 不应该是公共可读的，因为即使是一个表内容的统计性信息也可能被认为是敏感的（例子：一个薪水列的最大和最小值可能是非常有趣的）。sys_stats 是 `sys_statistic` 上的一个公共可读的视图，它只会显示出当前用户可读的表的信息。

`sys_statistic` 不应该是公众可读的，因为即使是关于表内容的统计信息也可能被认为是敏感的。（示例：工资列的最小值和最大值可能非常有趣。）sys_stats 是在 `sys_statistic` 之上的一个公共可读的视图，它只公开当前用户可读的表信息。

相似地，`sys_statistic_ext_data` 不应该是公众可读的，因为内容可能被认为是敏感的。（示例：列中最常见的值组合可能非常有趣。）sys_stats_ext 是在 `sys_statistic_ext_data` 之上的一个公共可读的视图（连接 `sys_statistic_ext` 之后），它只公开关于当前用户可读的表和列的信息。

列名称中的 [i] 代表该前缀列名在该视图中有多个，如 stakind[i] 代 sys_statistic 视图中包含 stakind1，stakind2，stakind3，stakind4，stakind5 列，这五个列有相同的 stakind 前缀名。

表 1.1.185: sys_statistic 列

名称	类型	描述
starelid	oid	对应表的 oid
staattnum	smallint	对应列的编号
stainherit	boolean	是否为继承表
stanullfrac	real	NULL 值比率，即该列中 NULL 值所占的比率
stawidth	integer	该列的平均宽度，以字节计算
stadistinct	real	表示该列消除重复后的数据的个数或者所占比率， 它有三种类型的取值： 1）= 0：表示未知或者未计算 2）> 0：表示去重后的个数 3）< 0：它的绝对值表示去重之后的个数占总个数的比率
stakind[i]	smallint	统计信息的类型值
staop[i]	oid	统计信息中涉及的操作符的 oid
stacoll[i]	oid	对于可排序的列的统计信息的排序编码的 oid
stanumbers[i]	real[]	存的是 MCV 最频值的频率值数组，或者相关系数值
stavaluesN	anyarray	存的是 MCV 最频值的字面值数组，跟 MCV 频率值一一对应，或者存的是直方图的桶的值的数组

参考

- ANALYZE(Oracle)
- ANALYZE(SQLServer)

1.1.4.61　sys_statistic_ext

`sys_statistic_ext` 记录扩展的规划器统计信息的定义。这个视图中的每一行对应于一个用 CREATE STATISTICS 创建的统计信息对象。

表 1.1.186: sys_statistic_ext 列

名称	类型	描述
oid	oid	统计对象的 oid
stxrelid	oid	统计对象对应的表的 oid
stxname	name	统计对象的名称
stxnamespace	oid	统计信息对象所在的名字空间的 oid
stxowner	oid	创建该统计对象的用户的 oid
stxkeys	int2vector	该统计信息所涉及的列的编号数组
stxkind	char[]	该统计信息对应的多列统计类型的 char 数组，共有三种，每一个字符对应一种类型： - d 代表 STATS_EXT_NDISTINCT，和单列中的 stadistinct 类似 stadistinct 中记录的是单列中去掉 NULL 值和消除重复值之后的数据量或者数据比例 - f 代表 STATS_EXT_DEPENDENCIES，计算各个列之间的函数依赖度再通过函数依赖度计算各个列之间的依赖关系，从而得到统计信息 m 代表 STATS_EXT_MCV，多列的 MCV 最频值

创建统计对象时填充 `stxkind` 字段，指示需要的统计类型。统计信息 (通过 `ANALYZE` 计算) 存储在sys_statistic_ext_data 视图中。

1.1.4.62　sys_statistic_ext_data

`sys_statistic_ext_data` 视图记录在 `sys_statistic_ext` 中定义的扩展计划统计数据。这个视图中的每一行都对应一个统计对象，该对象是用 CREATE STATISTICS 创建的。

sys_statistic_ext 视图记录元信息，而 sys_statistic_ext_data 视图记录的是具体的多列统计信息数据。

表 1.1.187: sys_statistic_ext_data 列

名称	类型	描述
stxoid	oid	对应 sys_statistic_ext 视图的 oid， 即多列统计对象的 oid
stxdndistinct	pg_ndistinct	STATS_EXT_NDISTINCT 类型的统计信息数据， 它使用了一个类似数组的表达形式，其每一个元素的格式为： {" 列号 1，列号 2": 去掉重复值之后的数量（包含 NULL）}
stxddependencies	pg_dependencies	STATS_EXT_DEPENDENCIES 类型表示的是多列之间的函数依赖度，所谓函数依赖（functional dependency），是对一个表上的两个列（属性）进行描述的一种方法： 如果对于 X 属性中的每一个值在 Y 属性上有且只有一个和它一一对应的值，那么 Y 对 X 满足函数依赖关系，记做 X->Y；主键的特性满足：" 一一对应" 和" 有且只有。实际上，很多属性列之间并不严格满足函数依赖关系，因此对于多列统计信息，KES 采取的方式是建立一种"SOFT" 的函数依赖关系，记录的是一个“函数依赖度”，即两个属性之间满足函数依赖的值占总体数量的比例。使用了一个类似数组的表达形式，其每一个元素的格式为：{" 列号 1 => 列号 2"：函数依赖比例}

-续表-

名称	类型	描述
stxdmcv	pg_mcv_list	STATS_EXT_MCV 类型表示多列的 MCV 最频值，它保存的是序列化后的多列 MCV 数据，序列化前对应的数据结构为 MCVList，多列的多个 MCV 字面值 (value) 和出现比例 (frequency) 保存在 MCVList->items[] 数组元素里，而该数组的元素类型为 MCVItemt

1.1.4.63　sys_stats

`sys_stats` 视图提供对存储在sys_statistic 视图中信息的访问。此视图能访问 `sys_statistic` 行是有限制的，可访问行所对应的表必须是用户有读权限的。因此让所有用户都可以读此视图是安全的。

`sys_stats` 也被设计为能以更适合阅读的格式显示底层视图的信息—但代价是只要为 `sys_statistic` 定义了新的槽类型，就必须要扩展此视图的模式。

表 1.1.188: sys_stats 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
attname	name	被此行描述的列名
inherited	boolean	如果为真，表示此行包括继承子列，不仅仅是指定表中的值
null_frac	real	列项中为空的比例
avg_width	integer	列项的平均字节宽度

-续表-

名称	类型	描述
n_distinct	real	如果大于零，表示列中可区分值的估计个数。 如果小于零，是可区分值个数除以行数的负值（当 ANALYZE 认为可区分值的数量会随着表增长而增加时采用负值的形式，而如果认为列具有固定数量的可选值时采用正值的形式） 例如，-1 表示一个唯一列，即其中可区分值的个数等于行数
most_common_vals	anyarray	列中最常用值的一个列表（如果没有任何一个值看起来比其他值更常用，此列为空）
most_common_freqs	real[]	最常用值的频率列表，即每一个常用值的出现次数除以总行数（如果 most_common_vals 为空，则此列为空）
histogram_bounds	anyarray	将列值划分成大小接近的组的值列表如果存在 most_common_vals，其中的值会被直方图计算所忽略（如果列类型没有一个 < 操作符或者 most_common_vals 等于整个值集合，则此列为空）
correlation	real	物理行顺序和列值逻辑顺序之间的统计关联 其范围从-1 到 +1。 当值接近-1 或 +1 时，在列上的一个索引扫描被认为比值接近 0 时的代价更低，因为这种情况减少了对磁盘的随机访问（如果列数据类型不具有一个 < 操作符，则此列为空）
most_common_elems	anyarray	在列值中，最经常出现的非空元素列表（对标度类型为空）

-续表-

名称	类型	描述
most_common_elem_freqs	real[]	最常用元素值的频度列表，即含有至少一个给定值实例的行的分数 在每个元素的频度之后有二至三个附加值，它们是每个元素频度的最小和最大值，以及可选的空元素的频度（如果 most_common_elems 为空，则此列为空）
elem_count_histogram	real[]	在列值中可区分非空元素值计数的一个直方图，后面跟随可区分非空元素的平均数（对于标度类型为空）

在数组域中项的最大数目可以使用 `ALTER TABLE SET STATISTICS` 命令控制，或者设置 default_statistics_target 运行时参数从全局上进行控制。

1.1.4.64 sys_stats_ext

`sys_stats_ext` 视图提供对存储在sys_statistic_ext 和sys_statistic_ext_data 视图的信息的访问。该视图只允许访问 `sys_statistic_ext` 和 `sys_statistic_ext_data` 中用户有权读取的表，因此允许公共访问读取这个视图是安全的。

`sys_stats_ext` 也被设计成以比底层视图更可读的格式来表示信息 - 代价是每当向 `sys_statistic_ext` 添加新类型的扩展统计信息时，必须对其模式进行扩展。

表 1.1.189: sys_stats_ext 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
statistics_schemaname	name	包含扩展统计的模式名
statistics_name	name	扩展统计名称
statistics_owner	name	扩展统计的所有者
attnames	name[]	定义扩展统计信息的列的名称
kinds	char[]	为记录启用的扩展统计信息类型

-续表-

名称	类型	描述
n_distinct	pg_ndistinct	对于列值的组合，有 n 个不同的计数 如果远大于零，不同值的估计数目在组合中 如果小于 0，则用不同值的负数除以根据行数。 （使用否定形式，当 ANALYZE 认为不同值的数量可能会随着表的增长而增加; 当列似乎有固定数目的可能值时，使用正数形式） 例如，-1 表示列的唯一组合，其中不同组合的数量与行数相同
dependencies	pg_dependencies	函数相关统计数据
most_common_vals	text[]	列中最常见的值组合的列表。 （如果没有比其他任何组合更常见的组合，则为 Null）
most_common_val_nulls	boolean[]	空标志最常见值组合的列表。 （当 most_common_vals 时为 Null）
most_common _freqs	double precision[]	频率最常见组合的列表，即，每个的出现次数除以总行数（当 most _common_vals 时为 Null）
most_common_base_freqs	double precision[]	基频率最常见组合的列表，即 per-value 频率的乘积（当 most _common_vals 时为 Null）

可以使用 `ALTER TABLE SET STATISTICS` 指令，或者通过全局地设置 default_statistics_target 运行时参数，来按列控制数组字段中的最大条目数。

1.2　MySQL 兼容视图

当前 KingbaseES 中仅支持部分 MySQL 信息模式视图，即 information_schema.xxx 视图。

1.2.1　信息模式视图

信息模式视图记录数据库元数据（数据库或表的名称、列的数据类型或访问权限）信息，位于 information_schema 下，需要通过"information_schema. 视图名" 访问。

1.2.1.1　ADMINISTRABLE_ROLE_AUTHORIZATIONS

该视图记录适用于当前用户或角色可用的且可以将其权限授予其他用户或角色的角色信息。

表 1.2.1: ADMINISTRABLE_ROLE_AUTHORIZATIONS 列

名称	数据类型	描述
USER	VARCHAR(97)	当前用户账户的用户名
HOST	VARCHAR(256)	当前用户账户的主机名，格式为 IP：PORT
GRANTEE	VARCHAR(97)	被授予这个角色的成员关系的角色名（可以是当前用户，或者在嵌套角色成员关系情况下的一个不同角色）
GRANTEE_HOST	VARCHAR(256)	角色被授予的账户的主机名，格式为 IP：PORT
ROLE_NAME	VARCHAR(255)	角色名
ROLE_HOST	VARCHAR(256)	授予角色的主机名，格式为 IP：PORT
IS_GRANTABLE	VARCHAR(3)	总是 YES
IS_DEFAULT	VARCHAR(3)	该角色是否为默认角色
IS_MANDATORY	VARCHAR(3)	该角色是否为强制性角色

1.2.1.2　APPLICABLE_ROLES

该视图记录适用于当前用户的角色的信息。

表 1.2.2: APPLICABLE_ROLES 列

名称	数据类型	描述
USER	VARCHAR(97)	当前用户账户的用户名
HOST	VARCHAR(256)	当前用户账户的主机名，格式为 IP：PORT
GRANTEE	VARCHAR(97)	被授予这个角色的成员关系的角色名（可以是当前用户，或者在嵌套角色成员关系情况下的一个不同角色）
GRANTEE_HOST	VARCHAR(256)	角色被授予的账户的主机名，格式为 IP：PORT
ROLE_NAME	VARCHAR(255)	角色名
ROLE_HOST	VARCHAR(256)	授予角色的主机名，格式为 IP：PORT
IS_GRANTABLE	VARCHAR(3)	总是 YES
IS_DEFAULT	VARCHAR(3)	该角色是否为默认角色
IS_MANDATORY	VARCHAR(3)	该角色是否为强制性角色

1.2.1.3　CHARACTER_SETS

该视图用于列出 MySQL 数据库支持的字符集编码及字符集编码的相关信息。

表 1.2.3: CHARACTER_SETS 列

名称	类型	描述
CHARACTER_SET_NAME	VARCHAR(64)	字符集名称
DEFAULT_COLLATE_NAME	VARCHAR(64)	字符集的默认排序规则
DESCRIPTION	VARCHAR(2048)	对字符集的描述
MAXLEN	UINT4	存储一个字符所需的最大字节数

1.2.1.4　CHECK_CONSTRAINTS

CREATE TABLE 允许表和列 check 约束，CHECK_CONSTRAINTS 视图记录这些约束的相关信息。

表 1.2.4: CHECK_CONSTRAINTS 列

名称	数据类型	描述
CONSTRAINT_CATALOG	VARCHAR(64)	约束所属的 CATALOG 的名称，这里只改变数据类型
CONSTRAINT_SCHEMA	VARCHAR(64)	包含该约束的模式名
CONSTRAINT_NAME	VARCHAR(64)	约束名
CHECK_CLAUSE	LONGTEXT	该检查约束的检查表达式

1.2.1.5　COLLATION_CHARACTER_SET_APPLICABILITY

该视图记录了字符集及其适用的排序规则的对应关系。

表 1.2.5: COLLATION_CHARACTER_SET_APPLICABILITY 列

名称	数据类型	描述
COLLATION_NAME	VARCHAR(64)	排序规则名称
CHARACTER_SET_NAME	VARCHAR(64)	与排序规则关联的字符集的名称

1.2.1.6　COLLATIONS

该视图记录有关每个字符集的排序规则的信息。

表 1.2.6: COLLATIONS 列

名称	数据类型	描述
COLLATION_NAME	VARCHAR(64)	排序规则名称

-续表-

名称	数据类型	描述
CHARACTER_SET_NAME	VARCHAR(64)	与排序规则关联的字符集的名称，当为空时表此排序规则适用于任何字符集
ID	BIGINT UNSIGNED	排序规则 OID
IS_DEFAULT	VARCHAR(3)	排序规则是否为其字符集的默认排序规则，固定为 YES
IS_COMPILED	VARCHAR(3)	字符集是否编译到服务器中，固定为 YES
SORTLEN	INT UNSIGNED	这与对以字符集表示的字符串进行排序所需的内存量有关，固定为 NULL
PAD_ATTRIBUTE	ENUM('PAD SPACE', 'NO PAD')	排序规则 PAD 属性，固定为 NO PAD

1.2.1.7　COLUMN_PRIVILEGES

该临时表记录列级别权限的信息。

表 1.2.7: COLUMN_PRIVILEGES 列

名称	数据类型	描述
GRANTEE	VARCHAR(292)	被授予权限的用户名称
TABLE_CATALOG	VARCHAR(512)	包含该列的表所在的数据库名
TABLE_SCHEMA	VARCHAR(64)	包含该列的表所在的模式名
TABLE_NAME	VARCHAR(64)	包含该列的表名
COLUMN_NAME	VARCHAR(64)	列名称
PRIVILEGE_TYPE	VARCHAR(64)	授予的权限类型
IS_GRANTABLE	VARCHAR(3)	该权限是否可授予其他用户

1.2.1.8　COLUMNS

该视图包含数据库中有关所有表列（或视图列）的信息。

表 1.2.8: COLUMNS 列

名称	类型	描述
TABLE_CATALOG	VARCHAR(64)	包含该列的表所属目录的名称
TABLE_SCHEMA	VARCHAR(64)	包含该列的表所属模式的名称
TABLE_NAME	VARCHAR(64)	包含该列的表的名称
COLUMN_NAME	VARCHAR(64)	列的名称
ORDINAL_POSITION	UINT4	列在所属表中的位置
COLUMN_DEFAULT	TEXT	列的默认值
IS_NULLABLE	VARCHAR(3)	列是否允许为空
DATA_TYPE	LONGTEXT	列的数据类型。ATA_TYPE 只是类型名称，没有其他信息
CHARACTER_MAXIMUM_LENGTH	BIGINT	仅适用于字符数据类型，表示列允许的最大长度
CHARACTER_OCTET_LENGTH	BIGINT	对于字符数据类型，列允许的最大字节数
NUMERIC_PRECISION	UINT8	仅适用于数字数据类型，表示列允许的最大精度（总位数）
NUMERIC_SCALE	UINT8	仅适用于数字数据类型，表示列允许的最大小数位数
DATETIME_PRECISION	UINT4	对于日期时间数据类型，这个列表示秒的小数位数
CHARACTER_SET_NAME	VARCHAR(64)	字符数据类型的列所使用的字符集名称
COLLATION_NAME	VARCHAR(64)	字符数据类型的列所使用的排序规则名称
COLUMN_TYPE	MEDIUMTEXT	列的完整数据类型。DATA_TYPE 只包含数据类型，没有其他信息，COLUMN_TYPE 包含数据类型名称和可能的其他信息，例如精度或长度
COLUMN_KEY	COLUMN_KEY_ENUM	该列是否被索引
EXTRA	VARCHAR(256)	该列是否具有其它额外信息

-续表-

名称	类型	描述
PRIVILEGES	VARCHAR(154)	当前用户对列的权限信息
COLUMN_COMMENT	TEXT	列定义中包含的注释信息，在创建表时通过 COMMENT 关键字指定的
GENERATION_EXPRESSION	LONGTEXT	对于生成列，这个列包含生成列的表达式。对于非生成列，该列为空
SRS_ID	UINT4	该列存储了 SRID 值，该值指示了存储在该列中的空间数据的空间参系统

1.2.1.9　ENABLED_ROLES

该视图记录当前会话中可用角色的信息。

表 1.2.9: ENABLED_ROLES 列

名称	数据类型	描述
ROLE_NAME	VARCHAR(255)	角色的用户名部分
ROLE_HOST	VARCHAR(255)	角色的主机名部分
IS_DEFAULT	VARCHAR(3)	该角色是否为默认角色
IS_MANDATORY	VARCHAR(3)	该角色是否为强制角色

1.2.1.10　KEY_COLUMN_USAGE

该视图仅记录约束包含的键列信息（键列为表达式除外）。

表 1.2.10: KEY_COLUMN_USAGE 列

名称	类型	描述
CONSTRAINT_CATALOG	VARCHAR(64)	约束所属的目录名称

-续表-

名称	类型	描述
CONSTRAINT_SCHEMA	VARCHAR(64)	约束所属的模式名称
CONSTRAINT_NAME	VARCHAR(64)	约束名称
TABLE_CATALOG	VARCHAR(64)	表所属的目录名称
TABLE_SCHEMA	VARCHAR(64)	表所属的模式名称
TABLE_NAME	VARCHAR(64)	具有约束的表的名称
COLUMN_NAME	VARCHAR(64)	具有约束的列的名称
ORDINAL_POSITION	UINT4	列在约束中的位置
POSITION_IN_UNIQUE_CONSTRAINT	UINT4	对于唯一约束和主键约束，该值为 NULL 对于外键约束，此列是正在被引用的表的建中的顺序为止
REFERENCED_TABLE_SCHEMA	VARCHAR(64)	外键约束中被引用的表所在的模式
REFERENCED_TABLE_NAME	VARCHAR(64)	外键约束中被引用的表名
REFERENCED_COLUMN_NAME	VARCHAR(64)	外键约束中被引用的列名

1.2.1.11　KEYWORDS

记录 KingbaseES 中的关键字信息，若该关键字为保留字，RESERVED 列为 1，否则该列值为 0。

表 1.2.11: KEYWORDS 列

名称	类型	描述
WORD	varchar(128)	mysql 兼容模式下 KingbaseES 中的关键字
RESERVED	int	保留关键字为 1，非保留关键字为 0

1.2.1.12　PARAMETERS

该视图记录有关存储例程（存储过程和函数）的参数以及函数的返回值的信息。

表 1.2.12: PARAMETERS 列

名称	类型	描述
SPECIFIC_CATALOG	VARCHAR(64)	包含参数的例程所属的目录的名称
SPECIFIC_SCHEMA	VARCHAR(64)	包含参数的例程所属的目录的名称
SPECIFIC_NAME	VARCHAR(64)	包含参数的例程的名字
ORDINAL_POSITION	UINT8	例程的参数位置，从 1 开始递增，对于存储函数的返回值，该参数值为 0，并且 PARAMETER_MODE 和 PARAMETER_NAME 都为空
PARAMETER_MODE	VARCHAR(5)	参数的模式。IN、OUT 或 INOUT 之一
PARAMETER_NAME	VARCHAR(64)	参数的名称
DATA_TYPE	LONGTEXT	参数的数据类型，只是类型名称，不包含其他信息
CHARACTER_MAXIMUM_LENGTH	BIGINT	字符型参数的最大长度
CHARACTER_OCTET_LENGTH	BIGINT	字符型参数的最大字节数
NUMERIC_PRECISION	UINT4	数值型参数的精度
NUMERIC_SCALE	BIGINT	数值型参数的最小位数
DATETIME_PRECISION	BIGINT	对于日期时间的参数，表示秒的小数位数
CHARACTER_SET_NAME	VARCHAR(64)	字符型参数的字符集名称
COLLATION_NAME	VARCHAR(64)	字符型参数的排序规则名称
DTD_IDENTIFIER	MEDIUMTEXT	参数的数据类型。该值包含类型名称以及可能的其他信息，比如精度或长度
ROUTINE_TYPE	ROUTINE_TYPE_ENUM	参数属于存储过程还是存储函数 PROCEDURE：表示存储过程 FUNCTION：表示存储函数

1.2.1.13　PARTITIONS

PARTITIONS 视图记录有关表分区的信息。

表 1.2.13: PARTITIONS 列

名称	类型	描述
TABLE_CATALOG	varchar(64)	包含该表的数据库名（总是当前数据库）
TABLE_SCHEMA	varchar(64)	模式名
TABLE_NAME	varchar(64)	表的名称
PARTITION_NAME	varchar(64)	分区的名称
SUBPARTITION_NAME	varchar(64)	子分区的名称
PARTITION_ORDINAL_POSITION	int unsigned	分区表的索引，获取当前分区为父分区中的第几个位置
SUBPARTITION_ORDINAL_POSITION	int unsigned	子分区索引，获取子分区为父分区中的第几个位置
PARTITION_METHOD	varchar(13)	分区的策略。（KES 分区类型有 hash 为哈希分区表，list 为列表分区表，range 为范围分区表）
SUBPARTITION_METHOD	varchar(13)	子分区的策略。（KES 分区类型有 hash 为哈希分区表，list 为列表分区表，range 为范围分区表）
PARTITION_EXPRESSION	varchar(2048)	分区函数表达式信息
SUBPARTITION_EXPRESSION	varchar(2048)	子分区表达式的信息
PARTITION_DESCRIPTION	text	解析当前分区表的内部表达
TABLE_ROWS	bigint unsigned	分区表中的表行数
AVG_ROW_LENGTH	bigint unsigned	分区存储所有行的平均长度
DATA_LENGTH	bigint unsigned	分区存储所有行的总长度
MAX_DATA_LENGTH	bigint unsigned	分区可以存储的最大字节数
INDEX_LENGTH	bigint unsigned	索引文件的长度

-续表-

名称	类型	描述
DATA_FREE	bigint unsigned	分区表中还未使用的字节数
CREATE_TIME	timestamp	分区建立的时间
UPDATE_TIME	datetime	分区最后修改的时间
CHECK_TIME	datetime	上一次检查此分区的时间
CHECKSUM	bigint	校验和值
PARTITION_COMMENT	text	注释的信息
NODEGROUP	varchar(256)	分区所属的节点组
TABLESPACE_NAME	varchar(268)	表空间名称

1.2.1.14　PROCESSLIST

该视图记录当前由服务器内执行的线程集执行的操作。

表 1.2.14: PROCESSLIST 列

名称	数据类型	描述
ID	BIGINT UNSIGNED	进程连接标识符，进程 PID
USER	VARCHAR(32)	发出语句的 MYSQL 用户
HOST	VARCHAR(261)	发出语句的客户端的主机名
DB	VARCHAR(64)	进程的默认数据库，如果尚未选择任何数据库则为 NULL
COMMAND	VARCHAR(16)	代表客户端执行的命令类型，如果会话处于空闲状态，则为 SLEEP
TIME	INT	进程查询的经过时间
STATE	VARCHAR(64)	指示进程正在执行的动作、事件或状态
INFO	VARCHAR(65535)	进程正在执行的查询语句，如果则为 NULL

1.2.1.15　REFERENTIAL_CONSTRAINTS

该视图记录当前数据库中外键约束的的信息。

表 1.2.15: REFERENTIAL_CONSTRAINTS 列

名称	数据类型	描述
CONSTRAINT_CATALOG	VARCHAR(64)	包含该约束的数据库名（总是当前数据库）
CONSTRAINT_SCHEMA	VARCHAR(64)	约束所属的模式的名称
CONSTRAINT_NAME	VARCHAR(64)	约束名
UNIQUE_CONSTRAINT _CATALOG	VARCHAR(64)	总是当前数据库
UNIQUE_CONSTRAINT_SCHEMA	VARCHAR(64)	包含外键约束引用的唯一约束的模式（数据库）名称
UNIQUE_CONSTRAINT_NAME	VARCHAR(64)	外键约束引用的唯一约束的名称
MATCH_OPTION	enum('NONE', 'PARTIAL', 'FULL')	约束的 MATCH 属性的值目前唯一有效的值为"NONE" 固定置为 NONE
UPDATE_RULE	enum('NO ACTION', 'RESTRICT', 'CASCADE', 'SET NULL',	约束的 ON UPDATE 属性的值可能的取值包括 CASCADE、SET NULL、SETDEFAULT、RESTRICT、NO ACTION
DELETE_RULE	enum('NO ACTION', 'RESTRICT', 'CASCADE', 'SET NULL',	约束的 ON DELETE 属性的值。可能的取值包括 CASCADE、SET NULL、SETDEFAULT、RESTRICT、NO ACTION
TABLE_NAME	VARCHAR(64)	包含外键约束的表的名称
REFERENCED_TABLE_NAME	VARCHAR(64)	外键约束引用的表的名称

1.2.1.16　ROLE_COLUMN_GRANTS

该视图记录有关当前启用的角色可用或由当前启用的角色授予的角色的列权限的信息。

表 1.2.16: ROLE_COLUMN_GRANTS 列

名称	数据类型	描述
GRANTOR	VARCHAR(97)	授予权限角色的帐户的用户名部分，
GRANTOR_HOST	CHAR(1)	授予权限角色的帐户的主机名部分，固定为%
GRANTEE	CHAR(32)	被授予权限角色的帐户的用户名部分
GRANTEE_HOST	CHAR(1)	被授予权限角色的帐户的主机名部分，固定为%
TABLE_CATALOG	VARCHAR(64)	总是当前数据库
TABLE_SCHEMA	CHAR(64)	此表所属的模式名称
TABLE_NAME	CHAR(64)	表的名称
COLUMN_NAME	CHAR(64)	列的名称
PRIVILEGE_TYPE	SET('SELECT', 'INSERT','UPDATE', 'REFERENCES')	授予的特权
IS_GRANTABLE	VARCHAR(3)	YES 或 NO，取决于该角色是否可授予其他帐户

1.2.1.17　ROLE_ROUTINE_GRANTS

该视图记录有关当前启用的角色可用或由当前启用的角色授予的角色的例程权限的信息。

表 1.2.17: ROLE_ROUTINE_GRANTS 列

名称	数据类型	描述
GRANTOR	VARCHAR(97)	授予权限角色的帐户的用户名部分，

-续表-

名称	数据类型	描述
GRANTOR_HOST	CHAR(1)	授予权限角色的帐户的主机名部分，固定 NULL
GRANTEE	CHAR(32)	被授予权限角色的帐户的用户名部分
GRANTEE_HOST	CHAR(1)	被授予权限角色的帐户的主机名部分，固定 NULL
SPECIFIC_CATALOG	VARCHAR(64)	总是当前数据库
SPECIFIC_SCHEMA	CHAR(64)	包含此函数的模式名称
SPECIFIC_NAME	CHAR(64)	函数的“specific name”（具体的名称）
ROUTINE_CATALOG	VARCHAR(64)	总是当前数据库
ROUTINE_SCHEMA	CHAR(64)	此函数所属的模式名称
ROUTINE_NAME	CHAR(64)	函数的名称（可能重复，因为有重载）
PRIVILEGE_TYPE	SET('EXECUTE', 'ALTER ROUTINE', 'GRANT')	授予的特权, 总是 EXECUTE
IS_GRANTABLE	VARCHAR(3)	YES 或 NO，取决于该角色是否可授予其他帐户

1.2.1.18　ROLE_TABLE_GRANTS

该视图记录有关当前启用的角色可用或由当前启用的角色授予的角色的表权限的信息。

表 1.2.18: ROLE_TABLE_GRANTS 列

名称	数据类型	描述
GRANTOR	VARCHAR(97)	授予权限角色的帐户的用户名部分，
GRANTOR_HOST	CHAR(1)	授予权限角色的帐户的主机名部分，固定 NULL
GRANTEE	CHAR(32)	被授予权限角色的帐户的用户名部分

-续表-

名称	数据类型	描述
GRANTEE_HOST	CHAR(1)	被授予权限角色的帐户的主机名部分，固定 NULL
TABLE_CATALOG	VARCHAR(64)	包含该表的数据库名（总是当前数据库）
TABLE_SCHEMA	CHAR(64)	此表所属的模式名称
TABLE_NAME	CHAR(64)	表名称
PRIVILEGE_TYPE	SET('SELECT', 'INSERT','UPDATE', 'DELETE', 'TRUNCATE', 'REFERENCES','TRIGGER')	授予的特权
IS_GRANTABLE	VARCHAR(3)	YES 或 NO，取决于该角色是否可授予其他帐户

1.2.1.19　ROUTINES

该视图记录有关存储例程（存储过程和函数）的信息。

表 1.2.19: ROUTINES 列

名称	类型	描述
SPECIFIC_NAME	VARCHAR(64)	例程的名称
ROUTINE_CATALOG	VARCHAR(64)	例程所属目录的名称
ROUTINE_SCHEMA	VARCHAR(64)	例程所属的模式名称
ROUTINE_NAME	VARCHAR(64)	例程的名称
ROUTINE_TYPE	ROUTINE_TYPE_ENUM	例程的类型 PROCEDURE：表示存储过程 'FUNCTION：表示存储函数

-续表-

名称	类型	描述
DATA_TYPE	LONGTEXT	该值只是类型名称，没有其他信息 存储函数：返回值的数据类型 存储过程：该值为空
CHARACTER_MAXIMUM_LENGTH	BIGINT	存储函数的字符串返回值的最大长度
CHARACTER_OCTET_LENGTH	BIGINT	存储函数的字符串返回值的最大字节数
NUMERIC_PRECISION	UINT4	存储函数的数值型返回值的精度
NUMERIC_SCALE	UINT4	存储函数的数值型返回值的最小位数
DATETIME_PRECISION	BIGINT	存储函数的时间返回值的秒最小位数
CHARACTER_SET_NAME	VARCHAR(64)	存储函数的字符串返回值的字符集名称
COLLATION_NAME	VARCHAR(64)	存储函数的字符串返回值的排序规则名称
DTD_IDENTIFIER	LONGTEXT	类型名称，还可能包括类型的其他信息 存储函数：返回值的数据类型 存储过程：该值为空
ROUTINE_BODY	VARCHAR(3)	用于例程定义的语言该值始终为 SQL
ROUTINE_DEFINITION	LONGTEXT	例程执行的 SQL 语句文本
EXTERNAL_NAME	BINARY(1)	该值始终为 NULL
EXTERNAL_LANGUAGE	VARCHAR(64)	存储例程的语言
PARAMETER_STYLE	VARCHAR(3)	该值始终为 SQL
IS_DETERMINISTIC	VARCHAR(3)	例程是否使用 DETERMINISTIC 特征定义
SQL_DATA_ACCESS	SQL_DATA_ACCESS_ENUM	例程的数据访问特征
SQL_PATH	BINARY(1)	该值始终为 NULL

-续表-

名称	类型	描述
SECURITY_TYPE	SECURITY_TYPE_ENUM	例程 SQL SECURITY 特性
CREATED	TIMESTAMP	创建例程的日期和时间
LAST_ALTERED	TIMPSTAMP	上次修改例程的日期和时间
SQL_MODE	SQL_MODE_SET	创建或更改例程时生效的 SQL 模式以及例程执行时的 SQL 模式
ROUTINE_COMMENT	TEXT	注释问题。如果没有注释，则为 NULL
DEFINER	VARCHAR(288)	创建例程用户的信息
CHARACTER_SET_CLIENT	VARCHAR(64)	创建例程时，CHARACTER_SET_CLIENT 系统变量的会话值
COLLATION_CONNECTION	VARCHAR(64)	创建例程时 COLLATION_CONNECTION 系统变量的会话值
DATABASE_COLLATION	VARCHAR(64)	与例程关联的数据库的排序规则

1.2.1.20　SCHEMATA

SCHEMATA 视图记录数据库的相关信息。

表 1.2.20: SCHEMATA 列

名称	数据类型	描述
CATALOG_NAMe	VARCHAR(64)	包含该表的数据库名（总是当前数据库）
SCHEMA_NAME	VARCHAR(64)	模式的名称
DEFAULT_CHARACTER_SET_NAME	VARCHAR(64)	模式的默认字符集，默认为数据库的字符集
DEFAULT_COLLATION_NAME	VARCHAR(64)	模式的默认排序规则（COLLATION），默认为数据库的字符集

-续表-

名称	数据类型	描述
SQL_PATH	BINARY(1)	该值始终为 NULL
DEFAULT_ENCRYPTION	ENUM('NO','YES')	模式的默认加密设置固定置为 NO

1.2.1.21　TABLE_CONSTRAINTS

该视图记录表上约束信息。

表 1.2.21: TABLE_CONSTRAINTS 列

名称	数据类型	描述
CONSTRAINT_CATALOG	VARCHAR(64)	该约束所属的目录名称
CONSTRAINT_SCHEMA	VARCHAR(64)	该约束所属的模式名称
CONSTRAINT_NAME	VARCHAR(64)	约束名称
TABLE_SCHEMA	VARCHAR(64)	表所属的模式名称
TABLE_NAME	VARCHAR(64)	表名称
CONSTRAINT_TYPE	VARCHAR(11)	约束类型
ENFORCED	VARCHAR(3)	该约束是否被强制执行，除了 check 约束可以选择是否强制执行，其余约束都是强制执行

1.2.1.22　TABLE_PRIVILEGES

该视图记录表级别权限的信息。

表 1.2.22: TABLE_PRIVILEGES 列

名称	数据类型	描述
GRANTEE	VARCHAR(292)	被授予该特权的角色名
TABLE_CATALOG	VARCHAR(512)	包含该列的表所在的数据库名
TABLE_SCHEMA	VARCHAR(64)	包含该列的表所在的模式名
TABLE_NAME	VARCHAR(64)	包含该列的表名
PRIVILEGE_TYPE	VARCHAR(64)	授予的权限类型
IS_GRANTABLE	VARCHAR(3)	该权限是否可授予其他用户

1.2.1.23　TABLES

该视图记录数据库中表的相关信息。

表 1.2.23: TABLES 列

名称	数据类型	描述
TABLE_CATALOG	VARCHAR(64)	包含该表的数据库名（总是当前数据库）
TABLE_SCHEMA	VARCHAR(64)	表所属的模式的名称
TABLE_NAME	VARCHAR(64)	表名称
TABLE_TYPE	ENUM('BASE TABLE','VIEW', 'SYSTEM VIEW')	BASE TABLE 表示表，VIEW 表示视图，SYSTEM VIEW 表示系统视图
ENGINE	VARCHAR(64)	表的存储引擎。固定置为 NULL
VERSION	INT	此列未使用本专栏现在报告硬编码值 10
ROW_FORMAT	ENUM('FIXED', 'DYNAMIC', 'COMPRESSED')	行存储格式，

-续表-

名称	数据类型	描述
TABLE_ROWS	BIGINT UNSIGNED	行数。此值是近似值，可能与实际值相差如下高达 40% 至 50%。在这种情况下，请使用 SELECT COUNT（*）来获得准确的计数。对于 INFORMATION_SCHEMA 表，TABLE_ROWS 为 NULL
AVG_ROW_LENGTH	BIGINT UNSIGNED	平均行长度
DATA_LENGTH	BIGINT UNSIGNED	数据文件的长度
MAX_DATA_LENGTH	BIGINT UNSIGNED	数据文件的最大长度，此列未使用，固定置为 0
INDEX_LENGTH	BIGINT UNSIGNED	索引文件的长度
DATA_FREE	BIGINT UNSIGNED	已分配但未使用的字节数
AUTO_INCREMENT	BIGINT UNSIGNED	下一个 AUTO_INCREMENT 值
CREATE_TIME	TIMESTAMP	创建表时间。固定置为 NULL
UPDATE_TIME	DATETIME	上次更新表的时间。固定置为 NULL
CHECK_TIME	VARCHAR(64)	上次检查表格的时间。固定置为 NULL
TABLE_COLLATION	VARCHAR(64)	表默认排序规则，默认为数据库的排序规则
CHECKSUM	BIGINT	实时校验和值（如果有）固定置为 NULL
CREATE_OPTIONS	VARCHAR(256)	创建表的选项
TABLE_COMMENT	VARCHAR(256)	创建表时使用的注释

1.2.1.24　TRIGGERS

该视图记录有关触发器的信息。

表 1.2.24: TRIGGERS 列

名称	数据类型	描述
TRIGGER_CATALOG	VARCHAR(64)	触发器所属的目录
TRIGGER_SCHEMA	VARCHAR(64)	触发器所属的模式
TRIGGER_NAME	VARCHAR(64)	触发器的名称
EVENT_MANIPULATION	EVENT_MANIPULATION_ENUM	触发器的事件类型
EVENT_OBJECT_CATALOG	VARCHAR(64)	触发器关联的表所属的目录
EVENT_OBJECT_SCHEMA	VARCHAR(64)	触发器关联的表所属的模式
EVENT_OBJECT_TABLE	VARCHAR(64)	触发器关联的表名称
ACTION_ORDER	UINT4	触发器动作在同一表中与响应时间类型和触发动作时机相同的触发器列表中的位置
ACTION_CONDITION	BINARY(1)	触发器的 WHEN 条件，总为 NULL
ACTION_STATEMENT	LONGTEXT	触发器被触发时执行的语句
ACTION_ORIENTATION	VARCHAR(3)	标识触发器是对每个被处理的行触发一次还是为每个语句触发一次。该值总为 ROW
ACTION_TIMING	ACTION_TIMING_ENUM	触发器执行时机
ACTION_REFERENCE_OLD_TABLE	BINARY(1)	“旧”传递表的名称，该值总为 NULL
ACTION_REFERENCE_NEW_TABLE	BINARY(1)	“新”传递表的名称，该值总为 NULL
ACTION_REFERENCE_OLD_ROW	VARCHAR(3)	UPDATE 触发器中表示更新前的行，该值总为 OLD
ACTION_REFERENCE_NEW_ROW	VARCHAR(3)	UPDATE 触发器中表示更新后的行，该值总为 NEW
CREATED	TIMESTAMP(2)	触发器的创建时间，精确到百分之一秒级

-续表-

名称	数据类型	描述
SQL_MODE	SQL_MODE_SET	触发器创建以及执行时采用的 SQL 模
DEFINER	VARCHAR(288)	创建触发器的用户
CHARACTER_SET_CLIENT	VARCHAR(64)	触发器创建时的 character_set _cl 系统变量的会话值
COLLATION_CONNECTION	VARCHAR(64)	触发器创建时的 collation _connec 系统变量的会话值
DATABASE_COLLATION	VARCHAR(64)	触发器关联的数据库排序规则信息

1.2.1.25　VIEW_ROUTINE_USAGE

VIEW_ROUTINE_USAGE 视图记录对视图定义中使用的存储函数信息的访问，该视图没有列出有关定义中使用的内置（本机）函数或可加载函数的信息。只能查看具有某些权限的视图信息，以及具有某种权限的函数的信息。

表 1.2.25: VIEW_ROUTINE_USAGE 列

名称	数据类型	描述
TABLE_CATALOG	VARCHAR(64)	视图所属的 catalog 的名称，这里只改变数据类型
TABLE_SCHEMA	VARCHAR(64)	包含该视图的模式名
TABLE_NAME	VARCHAR(64)	该视图的名称
SPECIFIC_CATALOG	VARCHAR(64)	函数所属的 catalog 的名称，这里只改变数据类型
SPECIFIC_SCHEMA	VARCHAR(64)	包含该函数的模式名
SPECIFIC_NAME	VARCHAR(64)	该函数的“专用名”

1.2.1.26　VIEW_TABLE_USAGE

该视图记录对视图定义中使用的表和视图信息的访问。只能查看具有某些权限的视图信息，以及具有某种权限的表的信息。

表 1.2.26: VIEW_TABLE_USAGE 列

名称	数据类型	描述
VIEW_CATALOG	VARCHAR(64)	视图所属的 catalog 的名称，这里只改变数据类型
VIEW_SCHEMA	VARCHAR(64)	包含该视图的模式名
VIEW_NAME	VARCHAR(64)	该视图的名称
TABLE_CATALOG	VARCHAR(64)	函数所属的 catalog 的名称，这里只改变数据类型
TABLE_SCHEMA	VARCHAR(64)	包含被该视图所使用的表的模式名
TABLE_NAME	VARCHAR(64)	包含被该视图所使用的表的名称

1.2.1.27　VIEWS

该视图记录关于数据库中视图（view）的相关信息。

表 1.2.27: VIEWS 列

名称	数据类型	描述
TABLE_CATALOG	VARCHAR(64)	包含该表的数据库名（总是当前数据库）
TABLE_SCHEMA	VARCHAR(64)	视图所属的模式（数据库）的名称
TABLE_NAME	VARCHAR(64)	视图的名称
VIEW_DEFINITION	LONGTEXT	用于定义视图的 SELECT 语句
CHECK_OPTION	ENUM('NONE', 'LOCAL', 'CASCADED')	该值是 NONE、CASCADE 或 LOCAL 中的一个

-续表-

名称	数据类型	描述
IS_UPDATABLE	ENUM('NO','YES')	如果对视图允许 UPDATE、DELETE（和类似的操作），则该标志被设置为 YES（真），否则设置为 NO（假）
DEFINER	VARCHAR(288)	视图的所有者
SECURITY_TYPE	VARCHAR(7)	视图的 SQL SECURITY 特性。该值是 DEFINER 或 INVOKER 中的一个。固定为 DEFINER
CHARACTER_SET_CLIENT	VARCHAR(64)	视图的字符集。默认为数据库的字符集
COLLATION_CONNECTION	VARCHAR(64)	视图的排序规则。默认为数据库的排序规则

1.3　Oracle 兼容视图

KingbaseES 为保证兼容 Oracle 用户使用习惯, 提供了静态数据字典视图和动态性能视图供用户使用。

1.3.1　动态性能视图

动态性能视图只包含 V$ 视图，该类视图提供数据库实例运行时状态的实时信息。

1.3.1.1　V$CONTEXT

记录当前会话中 context 的属性值。

表 1.3.1: V$CONTEXT 列

名称	类型	描述
NAMESPACE	TEXT	Context 名字
ATTRIBUTE	TEXT	Context 中属性名
VALUE	TEXT	Context 中属性值
CON_ID	INTEGER	保留列

1.3.1.2　V$DATABASE

记录当前数据库信息。

表 1.3.2: V$DATABASE 列

名称	类型	描述
DBID	NUMERIC(38，0)	创建数据库并将其存储在所有文件头中时计算出的数据库标识符
NAME	CHARACTER VARYING (63 BYTE)	数据库名称
CREATED	DATE	数据库的创建日期
LOG_MODE	CHARACTER VARYING (12 BYTE)	存档日志模式
CHECKPOINT_CHANGE	NUMERIC	上一个 SCN 检查点
CONTROLFILE_TIME	DATE	备份控制文件中的最后一个时间戳；如果控制文件不是备份，则为 null
OPEN_MODE	CHARACTER VARYING (10 BYTE)	开放模式信息
CONTROLFILE_CHANGE	NUMERIC	备份控制文件中的最后一个 SCN；如果控制文件不是备份，则为 null
CURRENT_SCN	NUMERIC	当前的 SCN；如果数据库当前未打开，则为 null。对于备用数据库，它是介质恢复期间已安装的物理备用数据库的检查点 SCN，并且始终小于在中跟踪的最后应用的 SCN
MAX_SIZE	NUMERIC(38,0)	数据库单个文件最大大小

-续表-

名称	类型	描述
TOTAL_SIZE	NUMERIC(38,0)	数据库总大小
STATUS$	CHARACTER VARYING (10 BYTE)	当前的 SCN；如果数据库当前未打开，则为 null。对于备用数据库，它是介质恢复期间已安装的物理备用数据库的检查点 SCN，并且始终小于在中跟踪的最后应用的 SCN

1.3.1.3　V$INSTANCE

记录当前实例信息。

表 1.3.3: V$INSTANCE 列

名称	类型	描述
INSTANCE_NAME	CHARACTER VARYING(16 BYTE)	实例名称
HOST_NAME	CHARACTER VARYING(64 BYTE)	主机名
VERSION	CHARACTER VARYING(17 BYTE)	数据库版本
STARTUP_TIME	DATE	实例启动时间
STATUS	CHARACTER VARYING(12 BYTE)	实例状态
ARCHIVER	CHARACTER VARYING(7 BYTE)	实例的静默状态

-续表-

名称	类型	描述
LOGIN	CHARACTER VARYING(10 BYTE)	指示实例是处于非受限模式下，允许所有用户登录（ALLOWED，还是处于受限模式下，仅允许数据库管理员登录（RESTRICTED）
SHUTDOWN_PENDING	CHARACTER VARYING(3 BYTE)	备份控制文件中的最后一个 SCN；如果控制文件不是备份，则为 null
DATABASE_STATUS	CHARACTER VARYING(17 BYTE)	数据库状态
INSTANCE_ROLE	CHARACTER VARYING(18 BYTE)	指示实例是活动实例（PRIMARY_INSTANCE）还是非活动辅助实例（SECONDARY _INSTANCE），或者 UNKNOWN 实例是否已启动但尚未挂载
ACTIVE_STATE	CHARACTER VARYING(9 BYTE)	实例的静默状态
BLOCKED	CHARACTER VARYING(3 BYTE)	表示是否阻止了所有服务

1.3.1.4　V$LOCK

列出了数据库当前持有的锁以及对锁的未完成请求。

表 1.3.4: V$LOCK 列

名称	类型	描述
ADDR	CHARACTER VARYING(16 BYTE)	锁定状态对象的地址
KADDR	CHARACTER VARYING(16 BYTE)	锁地址

-续表-

名称	类型	描述
SID	NUMERIC	用户或系统锁定的类型
TYPE	character varying(2 BYTE)	存档日志模式
LMODE	NUMERIC	锁模式
REQUEST	NUMERIC	进程请求锁定的锁定模式
CTIME	NUMERIC	自当前模式授予以来的时间
BLOCK	NUMERIC	指示有问题的锁是否正在阻止其他进程

1.3.1.5　V$LOCKED_OBJECT

记录当前被锁定的对象信息。

表 1.3.5: V$LOCKED_OBJECT 列

名称	类型	描述
XIDUSN	NUMERIC	撤销段号
XIDSLOT	NUMERIC	插槽号
XIDSQN	NUMERIC	序列号
OBJECT_ID	NUMERIC	对象 ID 被锁定
SESSION_ID	NUMERIC	会话 ID
ORACLE_USERNAME	CHARACTER VARYING(63 BYTE)	Oracle 用户名
OS_USER_NAME	CHARACTER VARYING(63 BYTE)	操作系统用户名
PROCESS	CHARACTER VARYING(63 BYTE)	操作系统进程 ID
LOCKED_MODE	NUMERIC	锁定模式

1.3.1.6　V$METRIC

记录当前库下 AWR 自动采集的最近度量值的全部集合。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

表 1.3.6: V$METRIC 列

名称	类型	描述
BEGIN_TIME	TIMESTAMP WITH TIME ZONE	采样周期开始时间
END_TIME	TIMESTAMP WITH TIME ZONE	采样周期结束时间
INTSIZE_CSEC	INTEGER	采样周期时间
GROUP_ID	INTEGER	度量分组的内部 ID
ENTITY_ID	INTEGER	基于分组 ID 的扩展 ID
ENTITY_SEQUENCE	INTEGER	基于扩展 ID 的序列 ID
METRIC_ID	INTEGER	度量 ID
METRIC_NAME	TEXT	度量名称
VALUE	NUMERIC	度量值
METRIC_UNIT	TEXT	度量单元

1.3.1.7　V$METRIC_HISTORY

记录当前库下v$metric的历史明细，保留每小时的度量值。表结构同v$metric。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

1.3.1.8　V$METRICGROUP

记录当前库下度量分组说明，主要是分组 ID、名称、采样间隔、最大采样数。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

表 1.3.7: V$METRICGROUP 列

名称	类型	描述
GROUP_ID	INTEGER	度量分组的内部 ID
NAME	TEXT	度量分组的外部名称
INTSIZE_CSEC	INTEGER	采样周期时间
MAX_INTERVAL	INTEGER	最大采样数

1.3.1.9　V$METRICNAME

记录当前库下度量名称说明，包括度量 ID、度量名称、度量单元。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

表 1.3.8: V$METRICNAME 列

名称	类型	描述
GROUP_ID	INTEGER	度量分组的内部 ID
GROUP_NAME	TEXT	度量分组的外部名称
METRIC_ID	INTEGER	度量 ID

-续表-

名称	类型	描述
METRIC_NAME	TEXT	度量名称
METRIC_UNIT	TEXT	度量单元

1.3.1.10　V$PARAMETER

列出影响当前会话的初始化参数信息。

表 1.3.9: V$PARAMETER 列

名称	类型	描述
NUM	NUMERIC	参数编号
NAME	CHARACTER VARYING(80 BYTE)	参数名称
TYPE	NUMERIC	参数类型
VALUE	CHARACTER VARYING(512 BYTE)	会话的参数值（如果在会话中修改）；否则，实例范围的参数值
DISPLAY_VALUE	CHARACTER VARYING(512 BYTE)	用户友好格式的参数值
ISSES_MODIFIABLE	CHARACTER VARYING(5 BYTE)	指示是否可以通过 ALTER SESSION 更改参数
ISpg_MODIFIALBE	CHARACTER VARYING(9 BYTE)	指示是否可以在 PG 内修改参数
ISMODIFIED	CHARACTER VARYING(10 BYTE)	指示在实例启动后是否已修改参数
DESCRIPTION	CHARACTER VARYING(255 BYTE)	参数说明

1.3.1.11　V$SESSION

记录当前会话信息。

表 1.3.10: V$SESSION 列

名称	类型	描述
SADDR	CHARACTER VARYING(20 BYTE)	会话地址
SID	NUMERIC	会话 ID
SERIAL#	NUMERIC	保留列
AUDSID	NUMERIC	保留列
PADDR	CHARACTER VARYING(20 BYTE)	保留列
USER#	NUMERIC(38,0)	用户标识
USERNAME	CHARACTER VARYING(63 BYTE)	用户名
COMMAND	NUMERIC	保留列
OWNERID	NUMERIC	保留列
TADDR	CHARACTER VARYING(20 BYTE)	保留列
LOCKWAIT	CHARACTER VARYING(8 BYTE)	会话正在等待的锁的地址
STATUS	CHARACTER VARYING(8 BYTE)	会话状态
SERVER	CHARACTER VARYING(9 BYTE)	服务器类型
SCHEMA#	NUMERIC(38,0)	模式用户标识符
SCHEMANAME	CHARACTER VARYING(63 BYTE)	保留列
OSUSER	CHARACTER VARYING(63 BYTE)	保留列

-续表-

名称	类型	描述
PROCESS	CHARACTER VARYING(12 BYTE)	保留列
MACHINE	CHARACTER VARYING(64 BYTE)	保留列
TERMINAL	CHARACTER VARYING(63 BYTE)	保留列
PROGRAM	CHARACTER VARYING(48 BYTE)	保留列
TYPE	CHARACTER VARYING(10 BYTE)	保留列
SQL_ADDRESS	CHARACTER VARYING(20 BYTE)	保留列
SQL_HASH_VALUE	NUMERIC	保留列
SQL_ID	CHARACTER VARYING(13 BYTE)	保留列
SQL_CHILD_NUMBER	NUMERIC	当前正在执行的 SQL 语句的子代号
PREV_SQL_ADDR	CHARACTER VARYING(20 BYTE)	保留列
PREV_HASH_VALUE	NUMERIC	保留列
PREV_SQL_ID	CHARACTER VARYING(13 BYTE)	保留列
PREV_CHILD_NUMBER	NUMERIC	保留列
MODULE	CHARACTER VARYING(48 BYTE)	保留列
MODULE_HASH	NUMERIC	保留列
ACTION	CHARACTER VARYING(32 BYTE)	保留列
ACTION_HASH	NUMERIC	保留列
CLIENT_INFO	CHARACTER VARYING(64 BYTE)	保留列

-续表-

名称	类型	描述
FIXED_TABLE_SEQUENCE	NUMERIC	保留列
ROW_WAIT_OBJ#	NUMERIC	保留列
ROW_WAIT_FILE#	NUMERIC	保留列
ROW_WAIT_BLOCK#	NUMERIC	保留列
ROW_WAIT_ROW#	NUMERIC	保留列
LOGON_TIME	DATE	登录时间
LAST_CALL_ET	NUMERIC	保留列
PDML_ENABLED	CHARACTER VARYING(3 BYTE)	保留列
FAILOVER_TYPE	CHARACTER VARYING(13 BYTE)	指示是否为会话启用透明应用程序故障转移，以及在何种程度上启用了透明应用程序故障转移
FAILOVER_METHOD	CHARACTER VARYING(10 BYTE)	指示会话的透明应用程序故障转移方法
FAILED_OVER	CHARACTER VARYING(3 BYTE)	表示在故障切换模式会话是否正在运行和故障转移发生与否
RESOURCE_CONSUMER _GROUP	CHARACTER VARYING(32 BYTE)	保留列
PDML_STATUS	CHARACTER VARYING(8 BYTE)	如果为 ENABLED ，则会话处于 PARALLEL DML 启用模式
PDDL_STATUS	CHARACTER VARYING(8 BYTE)	如果为 ENABLED ，则会话处于 PARALLEL DDL 启用模式
PQ_STATUS	CHARACTER VARYING(8 BYTE)	会话的状态
CURRENT_QUEUE _DURATION	NUMERIC	保留列
CLIENT_IDENTIFIER	CHARACTER VARYING(64 BYTE)	保留列
BLOCKING_SESSION_STATUS	CHARACTER VARYING(11 BYTE)	标志是否是锁定的会话
BLOCKING_INSTANCE	NUMERIC	保留列

-续表-

名称	类型	描述
BLOCKING_SESSION	NUMERIC	保留列
SEQ#	NUMERIC	保留列
EVENT#	NUMERIC	保留列
EVENT	CHARACTER VARYING(64 BYTE)	保留列
P1TEXT	CHARACTER VARYING(64 BYTE)	保留列
P1	NUMERIC	保留列
P1RAW	CHARACTER VARYING(20 BYTE)	保留列
P2TEXT	CHARACTER VARYING(64 BYTE)	保留列
P2	NUMERIC	保留列
P2RAW	CHARACTER VARYING(20 BYTE)	保留列
P3TEXT	CHARACTER VARYING(64 BYTE)	保留列
P3	NUMERIC	保留列
P3RAW	CHARACTER VARYING(20 BYTE)	保留列
WAIT_CLASS_ID	NUMERIC	等待事件的类别的标识符
WAIT_CLASS#	NUMERIC	保留列
WAIT_CLASS	CHARACTER VARYING(64 BYTE)	保留列
WAIT_TIME	NUMERIC	保留列
SECONDS_IN_WAIT	NUMERIC	保留列
STATE	CHARACTER VARYING(19 BYTE)	等待状态
SERVICE_NAME	NUMERIC	保留列

-续表-

名称	类型	描述
SQL_TRACE	CHARACTER VARYING(8 BYTE)	指示是启用还是禁用 SQL 跟踪
SQL_TRACE_WAITS	CHARACTER VARYING(5 BYTE)	指示是否启用了等待跟踪
SQL_TRACE_BINDS	CHARACTER VARYING(5 BYTE)	指示绑定跟踪是否被启用

1.3.1.12　V$SYSMETRIC

记录当前库下v$metric的子集，只包含 Name 为 System Metric Long/Short Duration 的统计项。表结构同v$metric。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

1.3.1.13　V$SYSMETRIC_HISTORY

记录当前库下v$metric_history的子集。表结构同v$metric。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

1.3.1.14　V$SYSMETRIC_SUMMARY

记录当前库下v$metric_history的最大值、最小值、平均值、标准方差等。

备注:

KingbaseES 使用 GUC 参数 track_real_stats 控制实时统计的开启（on）/关闭（off），默认为关闭。

表 1.3.11: V$SYSMETRIC_SUMMARY 列

名称	类型	描述
BEGIN_TIME	TIMESTAMP WITH TIME ZONE	采样周期开始时间
END_TIME	TIMESTAMP WITH TIME ZONE	采样周期结束时间
INTSIZE_CSEC	DOUBLE PRECISION	采样周期时间
GROUP_ID	INTEGER	度量分组的内部 ID
METRIC_ID	INTEGER	度量 ID
METRIC_NAME	TEXT	度量名称
NUM_INTERVAL	BIGINT	采样数量
MAXVAL	NUMERIC	采样的最大值
MINVAL	NUMERIC	采样的最小值
AVERAGE	NUMERIC	采样的平均值
STANDARD_DEVIATION	NUMERIC	采样的标准方差
METRIC_UNIT	TEXT	度量单元

1.3.1.15　V$SYSSTAT

记录当前系统统计信息。

表 1.3.12: V$SYSSTAT 列

名称	类型	描述
STATISTIC#	NUMERIC	统计号
NAME	CHARACTER VARYING(63 BYTE)	统计名称
VALUE	NUMERIC	统计值

1.3.2　静态数据字典视图

静态数据字典视图通常包含三类：

- DBA_ 包括所有用户的表、索引、视图、存储过程等，某些 DBA_ 视图具有包含对管理员有用的信息的附加列。
- ALL_ 包括用户拥有的对象，但是只显示当前用户有权限访问的对象。
- USER_ 包括当前用户的表、索引、视图、存储过程等，且带有前缀 USER_ 的视图通常不包括列 OWNER。

1.3.2.1　ALL_ALL_TABLES

记录当前用户可以访问的对象表和关系表。

表 1.3.13: ALL_ALL_TABLES 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	表的属主
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表名
TABLESPACE_NAME	CHARACTER VARYING(30 BYTE)	表空间名
CLUSTER_NAME	CHARACTER VARYING(128 BYTE)	cluster 名字
IOT_NAME	CHARACTER VARYING(128 BYTE)	索引结构表的名字
STATUS	CHARACTER VARYING(8 BYTE)	表的状态
PCT_FREE	NUMERIC	块中空闲空间的最小百分率
PCT_USED	NUMERIC	块中已使用空间的最小百分率
INI_TRANS	NUMERIC	事务数的初始化大小
MAX_TRANS	NUMERIC	最大事务数
INITIAL_EXTENT	NUMERIC	初始化的大小
NEXT_EXTENT	NUMERIC	扩展的大小
MIN_EXTENTS	NUMERIC	段中允许扩展的最小值

-续表-

名称	类型	描述
MAX_EXTENTS	NUMERIC	段中允许扩展的最大值
PCT_INCREASE	NUMERIC	扩展时增加的百分率
FREELISTS	NUMERIC	空闲进程列表的序号
FREELIST_GROUPS	NUMERIC	空闲列表组的序号
LOGGING	CHARACTER VARYING(3 BYTE)	表上的修改是否记录
BACKED_UP	CHARACTER VARYING(1 BYTE)	该表从上次修改到现在是否做了备份
NUM_ROWS	NUMERIC	表的行数
BLOCKS	NUMERIC	已使用的数据块数
EMPTY_BLOCKS	NUMERIC	空闲的数据块数
AVG_SPACE	NUMERIC	平均的空闲空间数
CHAIN_CNT	NUMERIC	表中跨数据块的行数
AVG_ROW_LEN	NUMERIC	平均一行的数据长度
AVG_SPACE_FREELIST _BLOCKS	NUMERIC	一个空闲列表中的所有数据块的平均空闲值
NUM_FREELIST_BLOCKS	NUMERIC	空闲列表中的数据块数
DEGREE	CHARACTER VARYING(10 BYTE)	扫描表时一个实例中的线程数
INSTANCES	CHARACTER VARYING(10 BYTE)	扫描表时的实例数
CACHE	CHARACTER VARYING(5 BYTE)	该表是否在缓存 CACHE 中
TABLE_LOCK	CHARACTER VARYING(8 BYTE)	该表是否被锁住
SAMPLE_SIZE	NUMERIC	分析表时的采样率
LAST_ANALYZED	DATE	最近一次 ANALYZE 的时间
PARTITIONED	CHARACTER VARYING(3 BYTE)	该表是否是分区表

-续表-

名称	类型	描述
IOT_TYPE	CHARACTER VARYING(12 BYTE)	索引组织表的类型
OBJECT_ID_TYPE	CHARACTER VARYING(16 BYTE)	对象类型
TABLE_TYPE_OWNER	CHARACTER VARYING(128 BYTE)	表类型属主
TABLE_TYPE	CHARACTER VARYING(128 BYTE)	表类型
TEMPORARY	CHARACTER VARYING(1 BYTE)	该数据是否为临时表
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NESTED	CHARACTER VARYING(3 BYTE)	该表是否是嵌套的
BUFFER_POOL	CHARACTER VARYING(7 BYTE)	该对象的缺省的缓冲区
FLASH_CACHE	CHARACTER VARYING(7 BYTE)	表数据块使用的数据库缓存提示
CELL_FLASH_CACHE	CHARACTER VARYING(7 BYTE)	表数据块使用的数据库闪存单元缓存提示
ROW_MOVEMENT	CHARACTER VARYING(8 BYTE)	分区表的行是否允许移动
GLOBAL_STATS	CHARACTER VARYING(3 BYTE)	收集统计信息或者增量维护则为‘YES’，否则为‘NO’
USER_STATS	CHARACTER VARYING(3 BYTE)	用户的状态
DURATION	CHARACTER VARYING(15 BYTE)	临时表的存在时间
SKIP_CORRUPT	CHARACTER VARYING(8 BYTE)	在索引扫描时是否忽略块标记中断
MONITORING	CHARACTER VARYING(3 BYTE)	该表是否设置了监控属性

-续表-

名称	类型	描述
CLUSTER_OWNER	CHARACTER VARYING(128 BYTE)	cluster 的属主
DEPENDENCIES	CHARACTER VARYING(8 BYTE)	行级依赖跟踪是否可用
COMPRESSION	CHARACTER VARYING(8 BYTE)	该表是否压缩
COMPRESS_FOR	CHARACTER VARYING(30 BYTE)	哪种压缩操作
DROPPED	CHARACTER VARYING(3 BYTE)	该表是否被删除
SEGMENT_CREATED	CHARACTER VARYING(3 BYTE)	段是否已经创建
INMEMORY	CHARACTER VARYING(8 BYTE)	表是否使用内存列存储
INMEMORY_PRIORITY	CHARACTER VARYING(8 BYTE)	表在内存列存储中的填充优先级
INMEMORY_DISTRIBUTE	CHARACTER VARYING(15 BYTE)	表在内存列存储分布方式
INMEMORY_COMPRESSION	CHARACTER VARYING(17 BYTE)	内存存储压缩等级
INMEMORY_DUPLICATE	CHARACTER VARYING(13 BYTE)	内存列存储重复设置方式
EXTERNAL	CHARACTER VARYING(3 BYTE)	是否为外部表
HYBRID	CHARACTER VARYING(3 BYTE)	是否为混合分区
CELLMEMORY	CHARACTER VARYING(24 BYTE)	闪存单元列压缩取值
INMEMORY_SERVICE	CHARACTER VARYING(12 BYTE)	内存列存储在不同实例中存放方式
INMEMORY_SERVICE_NAME	CHARACTER VARYING(1000 BYTE)	内存列存储名称

-续表-

名称	类型	描述
MEMOPTIMIZE_READ	CHARACTER VARYING(8 BYTE)	是否使用基于快速键访问
MEMOPTIMIZE_WRITE	CHARACTER VARYING(8 BYTE)	仅内部使用
HAS_SENSITIVE_COLUMN	CHARACTER VARYING(3 BYTE)	是否有敏感列
LOGICAL_REPLICATION	CHARACTER VARYING(8 BYTE)	是否使用逻辑复制

1.3.2.2　ALL_ARGUMENTS

ALL_ARGUMENTS 视图列出了当前用户可以访问的过程和函数的参数。

表 1.3.14: ALL_ARGUMENTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 CHAR)	对象所有者
PACKAGE_NAME	CHARACTER VARYING(128 CHAR)	包名称
OBJECT_NAME	CHARACTER VARYING(128 CHAR)	对象名称
OBJECT_ID	OID	对象编号
POSITION	INTEGER	参数位置
ARGUMENT_NAME	CHARACTER VARYING(128 CHAR)	参数名称

-续表-

名称	类型	描述
IN_OUT	CHARACTER VARYING(9 CHAR)	参数 IN OUT 类型
TYPE_NAME	CHARACTER VARYING(128 CHAR)	参数类型名称
OVERLOAD	CHARACTER VARYING(40 CHAR)	保留列
SUBPROGRAM_ID	INTEGER	保留列
SEQUENCE	INTEGER	参数序列
DATA_LEVEL	INTEGER	保留列
DATA_TYPE	CHARACTER VARYING(30 CHAR)	数据类型
DEFAULTED	CHARACTER VARYING(2 CHAR)	（包除外）是否含默认值
DEFAULT_VALUE	TEXT	保留列
DEFAULT_LENGTH	INTEGER	保留列
DATA_LENGTH	INTEGER	数据长度
DATA_PRECISION	INTEGER	十进制或者二进制表示的长度
DATA_SCALE	INTEGER	保留列
RADIX	INTEGER	数字的参数基数
CHARACTER_SET_NAME	CHARACTER VARYING(44 CHAR)	保留列
TYPE_OWNER	CHARACTER VARYING(128 CHAR)	类型所有者

-续表-

名称	类型	描述
TYPE_SUBNAME	CHARACTER VARYING(128 CHAR)	保留列
TYPE_LINK	CHARACTER VARYING(128 CHAR)	保留列
TYPE_OBJECT_TYPE	CHARACTER VARYING(7 CHAR)	保留列
PLS_TYPE	CHARACTER VARYING(128 CHAR)	保留列
CHAR_LENGTH	INTEGER	保留列
CHAR_USED	CHARACTER VARYING(2 CHAR)	BYTE 参数或 CHAR 类型参数
ORIGIN_CON_ID	INTEGER	数据来源的容器的 ID

1.3.2.3　ALL_COL_COMMENTS

记录所有对象上的注释信息。

表 1.3.15: ALL_COL_COMMENTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	对象的所有者
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表的名称
COLUMN_NAME	CHARACTER VARYING(128 BYTE)	列名
COMMENTS	CHARACTER VARYING(4000 BYTE)	列注释
ORIGIN_CON_ID	NUMERIC	数据来源容器的 ID

1.3.2.4　ALL_COL_PRIVS

当前用户下可以查看的所有列级权限，既包括user_col_privs 中的内容，同时也包含 GRANTEE 是当前角色的列级权限。

表 1.3.16: ALL_COL_PRIVS 列

名称	类型	描述
GRANTOR	CHARACTER VARYING(128 BYTE)	授权者
GRANTEE	CHARACTER VARYING(128 BYTE)	被授权者
TABLE_SCHEMA	CHARACTER VARYING(128 BYTE)	模式名
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表名或者视图名
COLUMN_NAME	CHARACTER VARYING(128 BYTE)	列名
PRIVILEGE	CHARACTER VARYING(40 BYTE)	权限
GRANTABLE	CHARACTER VARYING(3 BYTE)	是否具有可将该权限授予其他对象的权限
COMMON	CHARACTER VARYING(3 BYTE)	授权方式
INHERITED	CHARACTER VARYING(3 BYTE)	该权限授予是否从其他容器继承

1.3.2.5　ALL_CONS_COLUMNS

记录当前用户可访问的所有约束的列。

表 1.3.17: ALL_CONS_COLUMNS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	约束定义的所有者
CONSTRAINT_NAME	CHARACTER VARYING(63 BYTE)	约束定义的名称
TABLE_NAME	CHARACTER VARYING(63 BYTE)	约束定义的表名
COLUMN_NAME	CHARACTER VARYING(4000 CHAR)	约束定义中指定的列名称或对象类型列的属性
POSITION	NUMERIC	列或属性在对象定义中的原始位置

1.3.2.6　ALL_CONSTRAINTS

记录当前用户可以访问的表上的约束定义。

表 1.3.18: ALL_CONSTRAINTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	约束的所有者
CONSTRAINT_NAME	CHARACTER VARYING(63 BYTE)	约束的名字
CONSTRAINT_TYPE	CHARACTER VARYING(1 BYTE)	约束的类型
TABLE_NAME	CHARACTER VARYING(63 BYTE)	与约束相关的表
SEARCH_CONDITION	TEXT	检查约束的条件
R_OWNER	CHARACTER VARYING(63 BYTE)	一个参照完整性约束所引用表的所有者
R_CONSTRAINT_NAME	CHARACTER VARYING(63 BYTE)	所引用表的约束定义名称

-续表-

名称	类型	描述
DELETE_RULE	CHARACTER VARYING(9 BYTE)	参照完整性约束的删除规则
STATUS	CHARACTER VARYING(8 BYTE)	约束的状态
DEFERRABLE	CCHARACTER VARYING(14 CHAR)	定义约束是否为可延迟操作
DEFERRED	CHARACTER VARYING(9 CHAR)	指定约束是否已经被设置为延迟操作
VALIDATED	CHARACTER VARYING(13 BYTE)	是否所有数据都遵循约束
GENERATED	CHARACTER VARYING(14 BYTE)	是否约束的名字由用户或者系统生成
BAD	CHARACTER VARYING(3 CHAR)	标志是否为两位模糊的年份格式
RELY	CHARACTER VARYING(4 CHAR)	是否是一个可用的约束
LAST_CHANGE	TIMESTAMP(0) WITHOUT TIME ZONE	当前约束的最后启用或禁用的时间
INDEX_OWNER	CHARACTER VARYING(63 BYTE)	索引所有者的用户名
INDEX_NAME	CHARACTER VARYING(63 BYTE)	索引的名称
INVALID	CHARACTER VARYING(7 BYTE)	是否约束被置为无效
VIEW_RELATED	CHARACTER VARYING(14 BYTE)	视图上的约束是否含有依赖关系

1.3.2.7　ALL_DB_LINKS

记录数据库 dblink 的信息, 视图结构与dba_db_links 相同。

表 1.3.19: ALL_DB_LINKS 列

名称	类型	描述
OWNER	VARCHAR	DBLINK 的属主
DB_LINK	NAME	DBLINK 名称
USERNAME	VARCHAR	DBLINK 连接用户名
HOST	VARCHAR	DBLINK 连接 IP 地址
CREATED	TIMESTAMP WITH TIME ZONE	DBLINK 创建时间

1.3.2.8　ALL_DIRECTORIES

记录数据库目录的信息, 视图结构与dba_directories 相同。

表 1.3.20: ALL_DIRECTORIES 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	目录的属主
DIRECTORY_NAME	CHARACTER VARYING(63 BYTE)	目录名称
DIRECTORY_PATH	CHARACTER VARYING(1023 BYTE)	目录路径
ORIGIN_CON_ID	CHARACTER VARYING(63 BYTE)	默认置 0

1.3.2.9　ALL_IND_COLUMNS

记录当前用户所能获取到的表上建有索引的列信息。

表 1.3.21: ALL_IND_COLUMNS 列

名称	类型	描述
INDEX_OWNER	CHARACTER VARYING(128 BYTE)	索引的所有者
INDEX_NAME	CHARACTER VARYING(128 BYTE)	索引的名字
TABLE_OWNER	CHARACTER VARYING(128 BYTE)	该索引所属对象的所有者
TABLE_NAME	CHARACTER VARYING(128 BYTE)	该索引所属对象的名字
COLUMN_NAME	CHARACTER VARYING(4000 CHAR)	含有索引的列的名字
COLUMN_POSITION	NUMERIC	列在索引中的位置，比如在 (col2, col1) 上建有索引，那么 col2 的这一列的值是 1，col1 为 2
COLUMN_LENGTH	NUMERIC	本列显示了索引所在的列的数据类型的长度，以 byte 为单位（如果是以 char 为单位的字符串，那么是 NULL；numeric/number 类型的话，该值为 0，此时建议与 ALL_TAB_COLUMNS 组合使用）
CHAR_LENGTH	NUMERIC	显示了所有以 char 为单位的字符串的长度，其他类型为 NULL
DESCEND	CHARACTER VARYING(4 BYTE)	索引是升序（ASC）还是降序（DESC）的

1.3.2.10　ALL_INDEXES

记录当前用户所能获取到的表上的索引信息。

表 1.3.22: ALL_INDEXES 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	索引的所有者
INDEX_NAME	CHARACTER VARYING(63 BYTE)	索引的名字

-续表-

名称	类型	描述
INDEX_TYPE	CHARACTER VARYING(27 BYTE)	索引的类型
TABLE_OWNER	CHARACTER VARYING(63 BYTE)	该索引所属对象的所有者
TABLE_NAME	CHARACTER VARYING(63 BYTE)	该索引所属对象的名字
TABLE_TYPE	TEXT	该索引所属对象的类型
UNIQUENESS	CHARACTER VARYING(9 CHAR)	该索引是唯一索引或者非唯一索引
COMPRESSION	CHARACTER VARYING(8 CHAR)	是否启用压缩索引
PREFIX_LENGTH	NUMERIC	压缩键的前缀的长度
TABLESPACE_NAME	CHARACTER VARYING(63 BYTE)	索引所在表空间的名字
INI_TRANS	CHARACTER VARYING(7 CHAR)	事务的初始值
MAX_TRANS	CHARACTER VARYING(7 CHAR)	事务的最大值
INITIAL_EXTENT	CHARACTER VARYING(7 CHAR)	块的初始块的大小
NEXT_EXTENT	CHARACTER VARYING(7 CHAR)	递增块大小
MIN_EXTENTS	CHARACTER VARYING(7 CHAR)	在块中允许的最小扩展的数量
MAX_EXTENTS	CHARACTER VARYING(7 CHAR)	在块中允许最大扩展的数量
PCT_INCREASE	CHARACTER VARYING(7 CHAR)	增量的百分比
PCT_THRESHOLD	CHARACTER VARYING(7 CHAR)	阈值的百分比
INCLUDE_COLUMN	CHARACTER VARYING(7 CHAR)	包含的列

-续表-

名称	类型	描述
FREELISTS	CHARACTER VARYING(7 CHAR)	分配空间的 freelists 的进程号
FREELIST_GROUPS	CHARACTER VARYING(7 CHAR)	分配空间的 freelists 的组号
PCT_FREE	CHARACTER VARYING(7 CHAR)	在一个块中，释放空间的最低百分比
LOGGING	CHARACTER VARYING(7 CHAR)	日志信息
BLEVEL	CHARACTER VARYING(7 CHAR)	深度的索引从根到叶子遍历
LEAF_BLOCKS	CHARACTER VARYING(7 CHAR)	索引中叶子的块数
DISTINCT_KEYS	CHARACTER VARYING(7 CHAR)	DISTINCT 键值
AVG_LEAF_BLOCKS_PER_KEY	CHARACTER VARYING(7 CHAR)	叶子节点块的平均数
AVG_DATA_BLOCKS_PER_KEY	CHARACTER VARYING(7 CHAR)	数据块的平均数
CLUSTERING_FACTOR	CHARACTER VARYING(7 CHAR)	基本表上索引的行数
STATUS	CHARACTER VARYING(8 CHAR)	索引的状态
NUM_ROWS	CHARACTER VARYING(7 CHAR)	索引包含数据行数
SAMPLE_SIZE	CHARACTER VARYING(7 CHAR)	索引分析所使用的样本大小
LAST_ANALYZED	CHARACTER VARYING(20 CHAR)	在这个索引上最接近分析的数据号
DEGREE	CHARACTER VARYING(40 CHAR)	扫描索引时，每个实例的线程数
INSTANCES	CHARACTER VARYING(40 CHAR)	扫描索引时，实例数

-续表-

名称	类型	描述
PARTITIONED	CHARACTER VARYING(3 CHAR)	是否是分区索引
TEMPORARY	CHARACTER VARYING(1 CHAR)	索引所在的表是否是临时表
GENERATED	CHARACTER VARYING(1 CHAR)	索引的名称是否是系统生成的
SECONDARY	CHARACTER VARYING(1 CHAR)	索引是否是次要对象
BUFFER_POOL	CHARACTER VARYING(7 CHAR)	被用于索引块的默认缓冲池的名称
USER_STATS	CHARACTER VARYING(3 CHAR)	统计数据是否由用户输入
DURATION	CHARACTER VARYING(15 CHAR)	表示一个临时表的持续时间
PCT_DIRECT_ACCESS	CHARACTER VARYING(7 CHAR)	对于在一个索引组织表的辅助索引，在估计有效的行的百分比
ITYP_OWNER	CHARACTER VARYING(63 CHAR)	一个索引的区域，类型的所有者
ITYP_NAME	CHARACTER VARYING(63 CHAR)	一个索引的区域，类型的名称
PARAMETERS	CHARACTER VARYING(1000 CHAR)	一个索引的区域，参数字符串
GLOBAL_STATS	CHARACTER VARYING(3 CHAR)	是否通过分析收集的统计数据作为一个整体索引
DOMIDX_STATUS	CHARACTER VARYING(12 CHAR)	范围索引的状态
DOMIDX_OPSTATUS	CHARACTER VARYING(6 CHAR)	范围索引的操作状态
FUNCIDX_STATUS	CHARACTER VARYING(8 CHAR)	函数索引的状态
JOIN_INDEX	TEXT	索引是否是联合索引
IOT_REDUNDANT_PKEY_ELIM	TEXT	索引是否有多余的主键从索引组织表的辅助索引中被消除

-续表-

名称	类型	描述
DROPPED	TEXT	索引是否已经被删除并且已经在回收站

1.3.2.11　ALL_OBJECTS

记录当前用户下可以查看的所有对象。

表 1.3.23: ALL_OBJECTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	对象的属主
OBJECT_NAME	CHARACTER VARYING(128 BYTE)	对象名
SUBOBJECT_NAME	CHARACTER VARYING(128 BYTE)	子对象名，目前值为 NULL
OBJECT_ID	NUMERIC	对象的 OID
DATA_OBJECT_ID	NUMERIC	保留字段
OBJECT_TYPE	CHARACTER VARYING(23 BYTE)	对象的类型
CREATED	DATE	对象创建的时间，目前值为 NULL
LAST_DDL_TIME	DATE	对象最近一次的修改时间。目前值为 NULL
TIMESTAMP	CHARACTER VARYING(19 BYTE)	该对象的说明定义的时间
STATUS	CHARACTER VARYING(7 BYTE)	对象的状态
TEMPORARY	CHARACTER VARYING(1 BYTE)	该对象是否是临时的
GENERATED	CHARACTER VARYING(1 BYTE)	对象名是否是系统生成的

-续表-

名称	类型	描述
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NAMESPACE	NUMERIC	对象所属模式

1.3.2.12　ALL_PART_TABLES

记录当前用户所拥有及能访问到的所有分区表，其字段与dba_part_tables 相同。

表 1.3.24: ALL_PART_TABLES 列

列名	数据类型	功能描述
OWNER	CHARACTER VARYING(128 BYTE)	创建分区表的用户
TABLE_NAME	CHARACTER VARYING(128 BYTE)	显示数据库中分区表的名称（只支持声明式分区表，不支持继承式分区表）
PARTITIONING_TYPE	CHARACTER VARYING(9 BYTE)	分区表的分区策略，包含： RANGE：范围分区 HASH：哈希分区 LIST：列表分区
SUBPARTITIONING_TYPE	CHARACTER VARYING(9 BYTE)	子分区分区策略，支持： RANGE：范围分区 HASH：哈希分区 LIST：列表分区 NONE：分区表没有声明子分区（当使用 pg 语法创建分区时，此字段显示为 NONE）
PARTITION_COUNT	NUMERIC	显示表分区的分区数量，对于 interval 分区表, 此列数值始终为 1048575
DEF_SUBPARTITION_COUNT	NUMERIC	使用声明式分区时在语句中显式指定的子分区数量，（例如哈希分区和模板分区）
PARTITIONING_KEY_COUNT	NUMERIC	分区键所包含的列的数量
SUBPARTITIONING_KEY_COUNT	NUMERIC	子分区键所包含的列的数量
STATUS	CHARACTER VARYING(8 BYTE)	固定置 valid

-续表-

列名	数据类型	功能描述
DEF_TABLESPACE_NAME	CHARACTER VARYING(30 BYTE)	固定置 sys_default
DEF_PCT_FREE	NUMERIC	固定置 NULL
DEF_PCT_USED	NUMERIC	固定置 NULL
DEF_INI_TRANS	NUMERIC	固定置 NULL
DEF_MAX_TRANS	NUMERIC	固定置 NULL
DEF_INITIAL_EXTENT	CHARACTER VARYING(40 BYTE)	固定置 DEFAULT
DEF_NEXT_EXTENT	CHARACTER VARYING(40 BYTE)	固定置 DEFAULT
DEF_MIN_EXTENTS	CHARACTER VARYING(40 BYTE)	固定置 DEFAULT
DEF_MAX_EXTENTS	CHARACTER VARYING(40 BYTE)	固定置 DEFAULT
DEF_PCT_INCREASE	CHARACTER VARYING(40 BYTE)	固定置 DEFAULT
DEF_FREELISTS	NUMERIC	固定置 NULL
DEF_FREELIST_GROUPS	NUMERIC	固定置 NULL
DEF_LOGGING	CHARACTER VARYING(7 BYTE)	固定置 YES
DEF_COMPRESSION	CHARACTER VARYING(8 BYTE)	固定置 DISABLED
DEF_BUFFER_POOL	CHARACTER VARYING(7 BYTE)	固定置 DEFAULT
INTERVAL	CHARACTER VARYING(1000 BYTE)	记录 INTERVAL 分区表的间隔表达式

1.3.2.13　ALL_SCHEDULER_EXCEPTION

该视图记录用户拥有的处于异常状态的 schedule 作业信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_EXCEPTION”。

视图结构与dba_scheduler_exception 相同。

1.3.2.14　ALL_SCHEDULER_JOB_LOG

该视图记录用户可访问的 schedule 中作业的日志信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_JOB_LOG”。

视图结构与dba_scheduler_job_log 相同。

1.3.2.15　ALL_SCHEDULER_JOB_RUN_DETAILS

该视图记录用户可访问的 schedule 中作业的详细日志信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_JOB_RUN_DETAILS”。

视图结构与dba_scheduler_job_run_details 相同。

1.3.2.16　ALL_SCHEDULER_JOBS

该视图记录用户可访问的 schedule 的作业的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_JOBS”。

视图结构与dba_scheduler_jobs 相同。

1.3.2.17　ALL_SCHEDULER_PROGRAMS

该视图记录用户可访问的 scheduler 程序的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_PROGRAMS”。

视图结构与dba_scheduler_programs 相同。

1.3.2.18　ALL_SCHEDULER_SCHEDULES

该视图记录用户可访问的 scheduler 计划的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.ALL_SCHEDULER_SCHEDULES”。

视图结构与dba_scheduler_schedules 相同。

1.3.2.19　ALL_SEQUENCES

记录当前用户可访问的所有序列。

表 1.3.25: ALL_SEQUENCES 列

名称	类型	描述
SEQUENCE_OWNER	CHARACTER VARYING(63 BYTE)	序列所有者的名称
SEQUENCE_NAME	CHARACTER VARYING(63 BYTE)	序列名称
MIN_VALUE	NUMERIC(38, 0)	序列的最小值
MAX_VALUE	NUMERIC(38, 0)	序列的最大值
INCREMENT_BY	NUMERIC(38, 0)	序列的步长
CYCLE_FLAG	CHARACTER VARYING(1 BYTE)	序列是否在达到最大最小值时是否循环
ORDER_FLAG	CHARACTER VARYING(1 BYTE)	是否按顺序生成序列号
CACHE_SIZE	NUMERIC(38, 0)	要缓存的序列号数
LAST_NUMBER	NUMERIC(38, 0)	最后写入磁盘的序列号。如果序列使用缓存，则写入磁盘的数字是序列缓存中的最后一个数字。该数字可能大于所使使用的最后一个序列号
START_VALUE	BIGINT	序列起始值

1.3.2.20　ALL_SOURCE

记录当前用户可访问的存储对象的文本源。

表 1.3.26: ALL_SOURCE 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	对象所有者
NAME	CHARACTER VARYING(128 BYTE)	对象名称

-续表-

名称	类型	描述
TYPE	CHARACTER VARYING(12 BYTE)	对象类型：FUNCTION，PROCEDURE，AGGREGATE，WINDOW，PACKAGE，TRIGGER，PACKAGE BODY，TYPE，TYPE BODY
LINE	NUMERIC	在源文本中的行号
TEXT	CHARACTER VARYING(4000 CHAR)	对象定义
ORIGIN_CON_ID	CHARACTER VARYING(256 BYTE)	数据来源容器的 ID，默认值为 1

1.3.2.21　ALL_SYNONYMS

记录当前用户所能查看的同义词信息。包括：

1. 当前用户的私有同义词。

2. 所有的 public 同义词。

3. 本地同义词（DB_LINK 列为空）所指向，或者嵌套指向的对象，当前用户有权限查看。

对于指向远程数据库对象的同义词，及一些指向非法对象的同义词，它们指向对象的属主 (TABLE_OWNER) 为空。

表 1.3.27: ALL_SYNONYMS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 CHAR)	同义词的所有者
SYNONYM_NAME	CHARACTER VARYING(63 BYTE)	同义词名称
TABLE_OWNER	CHARACTER VARYING(63 BYTE)	同义词指向对象的所有者
TABLE_NAME	CHARACTER VARYING(63 CHAR)	同义词指向对象的名字

-续表-

名称	类型	描述
DB_LINK	CHARACTER VARYING(63 CHAR)	同义词所使用的数据库链接（如果存在）
ORIGIN_CON_ID	NUMERIC	所在容器 ID

1.3.2.22　ALL_TAB_COLS

记录当前用户可以访问的表信息，视图的列。视图结构与dba_tab_cols 相同。

1.3.2.23　ALL_TAB_COLUMNS

记录当前用户可以访问的表，视图的非隐藏列。视图结构与dba_tab_columns 基本一致。

表 1.3.28: DBA_TAB_COLUMNS 列

名称	类型	描述
OWNER	TEXT	表、视图的所有者
TABLE_NAME	CHARACTER VARYING(63 BYTE)	表、视图的名字
COLUMN_NAME	CHARACTER VARYING(63 BYTE)	列名字
DATA_TYPE	CHARACTER VARYING(106 BYTE)	列的数据类型
DATA_TYPE_MOD	CHARACTER VARYING(3 BYTE)	列的修改的数据类型
DATA_TYPE_OWNER	CHARACTER VARYING(63 BYTE)	列的类型的所有者
DATA_LENGTH	NUMERIC	列的长度
DATA_PRECISION	NUMERIC	数据精度。针对 numeric 和 float 类型
DATA_SCALE	NUMERIC	数字的小数点右侧位数。针对 numeric 类型

-续表-

名称	类型	描述
NULLABLE	CHARACTER VARYING(1 CHAR)	描述列是否可以为 NULL，'Y'/'N'
COLUMN_ID	NUMERIC	列的编号
DEFAULT_LENGTH	NUMERIC	列的默认长度没有默认值时为 NULL
DATA_DEFAULT	TEXT	列的默认值没有默认值时为 NULL
NUM_DISTINCT	NUMERIC	列上的 distinct 的值的数量
LOW_VALUE	NUMERIC	列的最小值
HIGH_VALUE	NUMERIC	列的最大值
DENSITY	NUMERIC	列的密度
NUM_NULLS	NUMERIC	在列上空值的数量
NUM_BUCKETS	NUMERIC	列上的大量数据的直方图
LAST_ANALYZED	TIMESTAMP(0) WITHOUT TIME ZONE	是否约束被置为无效
SAMPLE_SIZE	NUMERIC	声明列的字符类型长度
CHARACTER_SET_NAME	CHARACTER VARYING(44 CHAR)	字符集名称
CHAR_COL_DECL_LENGTH	NUMERIC	声明列的字符类型长度
GLOBAL_STATS	CHARACTER VARYING(3 CHAR)	是否将分区表作为整体来收集列的信息
USER_STATS	CHARACTER VARYING(3 CHAR)	是否由用户输入
AVG_COL_LEN	NUMERIC	平均长度（字节）
CHAR_LENGTH	NUMERIC	列中的字符的长度。单位为字符（CHAR）。非 VARCHAR,BPCHAR，CHAR 类型为 NULL
CHAR_USED	CHARACTER VARYING(1 CHAR)	使用的类型。'B' 表示字串以 byte 为单位，'C' 表示以 CHAR 为单位。例如 VARCHAR(40 CHAR)

-续表-

名称	类型	描述
V80_FMT_IMAGE	CHARACTER VARYING(3 CHAR)	列中的数据是否是 8.0 图像格式
DATA_UPGRADED	CHARACTER VARYING(3 CHAR)	列的数据是否已经升级到最新版本的格式类型
HISTOGRAM	CHARACTER VARYING(15 CHAR)	类型使用频率的柱状图

1.3.2.24　ALL_TAB_COMMENTS

记录当前用户可以访问的表和视图上的注释。

表 1.3.29: ALL_TAB_COMMENTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	对象所有者
TABLE_NAME	CHARACTER VARYING(128 BYTE)	对象名称
TABLE_TYPE	CHARACTER VARYING(13 BYTE)	对象类型
COMMENTS	CHARACTER VARYING(4000 BYTE)	对象的注释

1.3.2.25　ALL_TAB_PARTITIONS

显示当前用户所拥有的所有分区，字段与dba_tab_partitions 相同。

表 1.3.30: ALL_TAB_PARTITIONS 列

列名	数据类型	功能描述
TABLE_OWNER	CHARACTER VARYING(128 BYTE)	表用户
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表名
COMPOSITE	CHARACTER VARYING(3 BYTE)	分区表是否声明子分区 YES: 声明子分区 NO: 未声明子分区（当使用 pg 语法创建多级子分区时，此字段返回 NO。）
PARTITION_NAME	CHARACTER VARYING(128 BYTE)	分区名。与 oracle 不同的是，oracle 只显示分区名，KES 中会以“表名_ 分区名”的格式进行显示。
SUBPARTITION_COUNT	NUMERIC	如果声明了子分区，返回子分区数量，否则返回 0。当使用 pg 语法创建多级子分区时，此字段返回 0。
HIGH_VALUE	TEXT	分区边界的表达式（适用于 RANGE 和 LIST 分区），哈希分区此列为 NULL。与 oracle 不同的是，KES 在此列显示时会在表达式外添加小括号“()”，oracle 在表达式外不显示小括号。
HIGH_VALUE_LENGTH	NUMERIC	分区边界表达式的长度。（由于 KES 添加了括号，会比 ORACLE 对应的值大 2）
PARTITION_POSITION	NUMERIC	分区在表中的索引，根据 HIGH_VALUE 的字符顺序，从 1 开始计数。此处与 Oracle 不同，Oracle 中按照表达式声明的顺序排序。
TABLESPACE_NAME	CHARACTER VARYING(30 BYTE)	分区所在表空间，未指定时默认为 sys_default。
PCT_FREE	NUMERIC	固定置 NULL
PCT_USED	NUMERIC	固定置 NULL
INI_TRANS	NUMERIC	固定置 NULL
MAX_TRANS	NUMERIC	固定置 NULL
INITIAL_EXTENT	NUMERIC	固定置 NULL
NEXT_EXTENT	NUMERIC	固定置 NULL
MIN_EXTENT	NUMERIC	固定置 NULL
MAX_EXTENT	NUMERIC	固定置 NULL

-续表-

列名	数据类型	功能描述
PCT_INCREASE	NUMERIC	固定置 NULL
FREELISTS	NUMERIC	固定置 NULL
FREELIST_GROUPS	NUMERIC	固定置 NULL
LOGGING	CHARACTER VARYING(7 BYTE)	固定置 YES
COMPRESSION	CHARACTER VARYING(8 BYTE)	固定置 DISABLED
NUM_ROWS	NUMERIC(38, 0)	同 sys_class.reltuples（此列不实时更新）
BLOCKS	NUMERIC(38, 0)	同 sys_class.relpages（此列不实时更新）
EMPTY_BLOCKS	NUMERIC	固定置 NULL
AVG_SPACE	NUMERIC	固定置 NULL
CHAIN_CNT	NUMERIC	固定置 NULL
AVG_ROW_LEN	NUMERIC(38, 0)	分区中每一行的平均存储长度
SAMPLE_SIZE	NUMERIC	固定置 NULL
INTERVAL	CHARACTER VARYING(3 BYTE)	标识分区表是否为 INTERVAL 分区

1.3.2.26　ALL_TAB_PRIVS

记录当前用户对所有对象授予信息。

表 1.3.31: ALL_TAB_PRIVS 列

名称	类型	描述
GRANTEE	CHARACTER VARYING(63 BYTE)	被授予权限的用户
OWNER	CHARACTER VARYING(63 BYTE)	权限的拥有者
TABLE_NAME	CHARACTER VARYING(63 BYTE)	表名
GRANTOR	CHARACTER VARYING(63 BYTE)	被授予权限的用户

-续表-

名称	类型	描述
PRIVILEGE	CHARACTER VARYING(40 CHAR)	权限类型
GRANTABLE	CHARACTER VARYING(3 BYTE)	是否可以转授权限

1.3.2.27　ALL_TABLES

记录当前用户下可以查看的所有表信息。

表 1.3.32: ALL_TABLES 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	表的属主
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表名
TABLESPACE_NAME	CHARACTER VARYING(30 BYTE)	表空间名
CLUSTER_NAME	CHARACTER VARYING(128 BYTE)	CLUSTER 名字
IOT_NAME	CHARACTER VARYING(128 BYTE)	索引结构表的名字
STATUS	CHARACTER VARYING(8 BYTE)	表的状态
PCT_FREE	NUMERIC	块中空闲空间的最小百分率
PCT_USED	NUMERIC	块中已使用空间的最小百分率
INI_TRANS	NUMERIC	事务数的初始化大小
MAX_TRANS	NUMERIC	最大事务数
INITIAL_EXTENT	NUMERIC	初始化的大小
NEXT_EXTENT	NUMERIC	扩展的大小
MIN_EXTENTS	NUMERIC	段中允许扩展的最小值

-续表-

名称	类型	描述
MAX_EXTENTS	NUMERIC	段中允许扩展的最大值
PCT_INCREASE	NUMERIC	扩展时增加的百分率
FREELISTS	NUMERIC	空闲的进程列表序号
FREELIST_GROUPS	NUMERIC	空闲列表组的序号
LOGGING	CHARACTER VARYING(3 BYTE)	LOGGING 属性
BACKED_UP	CHARACTER VARYING(1 BYTE)	该表从上次修改到现在是否做了备份
NUM_ROWS	NUMERIC	表的行数
BLOCKS	NUMERIC	已使用的数据块数
EMPTY_BLOCKS	NUMERIC	空闲的数据块数
AVG_SPACE	NUMERIC	平均的空闲空间数
CHAIN_CNT	NUMERIC	表中跨数据块的行数
AVG_ROW_LEN	NUMERIC	平均一行的数据长度
AVG_SPACE_FREELIST_BLOCKS	NUMERIC	一个空闲列表中的所有数据块的平均空闲值
NUM_FREELIST_BLOCKS	NUMERIC	空闲列表中的数据块数
DEGREE	CHARACTER VARYING(10 BYTE)	扫描表时一个实例中的线程数
INSTANCES	CHARACTER VARYING(10 BYTE)	扫描表时的实例数
CACHE	CHARACTER VARYING(5 BYTE)	该表是否在缓存 CACHE 中
TABLE_LOCK	CHARACTER VARYING(8 BYTE)	该表是否被锁住
SAMPLE_SIZE	NUMERIC	分析表时的采样率
LAST_ANALYZED	DATE	最近一次 ANALYZE 的时间
PARTITIONED	CHARACTER VARYING(3 BYTE)	该表是否是分区表

-续表-

名称	类型	描述
IOT_TYPE	CHARACTER VARYING(12 BYTE)	索引组织表的类型
TEMPORARY	CHARACTER VARYING(1 BYTE)	该数据是否只有当前会话可见
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NESTED	CHARACTER VARYING(3 BYTE)	该表是否是嵌套的
BUFFER_POOL	CHARACTER VARYING(7 BYTE)	该对象的缺省的缓冲区
ROW_MOVEMENT	CHARACTER VARYING(8 BYTE)	分区表的行是否允许移动
GLOBAL_STATS	CHARACTER VARYING(3 BYTE)	分区表的统计信息
USER_STATS	CHARACTER VARYING(3 BYTE)	用户的状态
DURATION	CHARACTER VARYING(15 BYTE)	临时表的生存时间
SKIP_CORRUPT	CHARACTER VARYING(8 BYTE)	在索引扫描时是否忽略块标记中断
MONITORING	CHARACTER VARYING(3 BYTE)	该表是否设置了监控属性
CLUSTER_OWNER	CHARACTER VARYING(128 BYTE)	CLUSETR 的属主
DEPENDENCIES	CHARACTER VARYING(8 BYTE)	行级依赖跟踪是否可用
COMPRESSION	CHARACTER VARYING(8 BYTE)	该表是否压缩
DROPPED	CHARACTER VARYING(3 BYTE)	该表是否被删除

1.3.2.28　ALL_TRIGGER_COLS

记录当前用户可访问的所有触发器的列。

表 1.3.33: ALL_TRIGGER_COLS 列

名称	类型	描述
TRIGGER_OWNER	CHARACTER VARYING(63 BYTE)	触发器所有者的名称
TRIGGER_NAME	CHARACTER VARYING(63 BYTE)	触发器名称
TABLE_OWNER	CHARACTER VARYING(63 BYTE)	定义触发器的表的所有者
TABLE_NAME	CHARACTER VARYING(63 BYTE)	定义触发器的表
COLUMN_NAME	CHARACTER VARYING(4000 CHAR)	触发器中使用的列的名称

1.3.2.29　ALL_TRIGGERS

记录当前用户可访问的所有触发器。

表 1.3.34: ALL_TRIGGERS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	触发器所有者的名称
TRIGGER_NAME	CHARACTER VARYING(63 BYTE)	触发器名称
TRIGGER_TYPE	CHARACTER VARYING(16 BYTE)	触发器的类型
TRIGGERING_EVENT	CHARACTER VARYING(216 BYTE)	触发触发器的 DML，DDL 或数据库事件
TABLE_OWNER	CHARACTER VARYING(63 BYTE)	定义触发器的表的所有者

-续表-

名称	类型	描述
TABLE_NAME	CHARACTER VARYING(63 BYTE)	如果触发器的基础对象类型为 SCHEMA 或 DATABASE，则此列为 NULL; 如果触发器的基础对象类型为 TABLE 或 VIEW，则此列指示定义触发器的表/视图名称
BASE_OBJECT_TYPE	CHARACTER VARYING(16 BYTE)	基本对象类型
COLUMN_NAME	CHARACTER VARYING(4000 CHAR)	表列的名称
REFERENCING_NAMES	CHARACTER VARYING(422 BYTE)	引用的名称
DESCRIPTION	CHARACTER VARYING(4000 BYTE)	描述
STATUS	CHARACTER VARYING(8 BYTE)	指示触发器是启用还是禁用
TRIGGER_BODY	TEXT	触发器触发时执行的语句

1.3.2.30　ALL_TYPES

记录当前用户可以访问的对象表和关系表。

表 1.3.35: ALL_TYPES 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	类型的属主
TYPE_NAME	CHARACTER VARYING(128 BYTE)	类型的名字
TYPE_OID	OID	类型的 oid
TYPECODE	CHARACTER VARYING(128 BYTE)	类型的编码
ATTRIBUTES	NUMERIC	类型的属性个数
METHODS	NUMERIC	类型的方法个数

-续表-

名称	类型	描述
PREDEFINED	CHARACTER VARYING(3 BYTE)	类型是否为预定义
INCOMPLETE	CHARACTER VARYING(3 BYTE)	类型是否为不完整类型
FINAL	CHARACTER VARYING(3 BYTE)	类型是否为最终类型
INSTANTIABLE	CHARACTER VARYING(3 BYTE)	类型是否可实例化
PERSISTABLE	CHARACTER VARYING(3 BYTE)	类型是否可持久化
SUPERTYPE_OWNER	CHARACTER VARYING(128 BYTE)	父类型的属主
SUPERTYPE_NAME	CHARACTER VARYING(128 BYTE)	父类型的名称
LOCAL_ATTRIBUTES	NUMERIC	子类型不包含继承的属性个数
LOCAL_METHODS	NUMERIC	子类型不包含继承的方法个数
TYPEID	BYTEA	类型的 ID 值

1.3.2.31　ALL_USERS

记录当前用户可见的数据库的所有用户。

表 1.3.36: ALL_USERS 列

名称	类型	描述
USERNAME	CHARACTER VARYING(128 BYTE)	用户名
USER_ID	NUMERIC	用户的 ID 号
CREATED	DATE	用户创建日期

1.3.2.32　ALL_VIEWS

记录当前用户所能查看的所有的视图信息。

表 1.3.37: ALL_VIEWS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	视图的所有者
VIEW_NAME	CHARACTER VARYING(128 BYTE)	视图的名字
TEXT_LENGTH	NUMBER	生成视图的语句的长度
TEXT	TEXT	生成视图的语句
TYPE_TEXT_LENGTH	NUMBER	由该视图所形成的数据类型的名字长度
TYPE_TEXT	CHARACTER VARYING(4000 CHAR)	由该视图形成的数据类型的名字。该类型的每一个对象，其结构相当于视图中的每一个元组的结构
OID_TEXT_LENGTH	NUMBER	带有 OID 的指向临时对象的语句长度，为 NULL
OID_TEXT	CHARACTER VARYING(4000 CHAR)	带有 OID 的指向临时对象的语句，为 NULL
VIEW_TYPE_OWNER	CHARACTER VARYING(128 BYTE)	由视图所形成的数据类型的所有者
VIEW_TYPE	CHARACTER VARYING(128 BYTE)	由视图所形成的数据类型，如果它是一个类型视图的话。解释详见 TYPE_TEXT
SUPERVIEW_NAME	CHARACTER VARYING(128 BYTE)	父视图，为 NULL
EDITIONING_VIEW	CHARACTER VARYING(1 BYTE)	留待后用，目前为'N'
READ_ONLY	CHARACTER VARYING(1 BYTE)	该视图是否是只读的（'Y'/'N'，'N' 表示视图可更新）
CONTAINER_DATA	CHARACTER VARYING(1 BYTE)	该视图是否包含容器的特定数据。可能的值包括： Y：如果视图是使用 CONTAINER_DATA 子句创建的 N：其他情况

-续表-

名称	类型	描述
BEQUEATH	CHARACTER VARYING(12 BYTE)	CURRENT_USER：当视图是一个 BEQUEATH CURRENT_USER 视图时 DEFINER：当视图是一个 BEQUEATH DEFINER 视图时
ORIGIN_CON_ID	NUMERIC	数据来源的容器 ID。
DEFAULT_COLLATION	CHARACTER VARYING(100 BYTE)	视图的默认排序规则
CONTAINERS_DEFAULT	CHARACTER VARYING(3 BYTE)	视图是否默认启用 CONTAINERS() 功能
CONTAINERS_MAP	CHARACTER VARYING(3 BYTE)	视图是否启用与 container_map 数据库属性一起使用
EXTENDED_DATA_LINK	CHARACTER VARYING(3 BYTE)	视图是否启用从根获取扩展数据链接
EXTENDED_DATA _LINK_MAP	CHARACTER VARYING(3 BYTE)	仅供内部使用
HAS_SENSITIVE _COLUMN	CHARACTER VARYING(3 BYTE)	视图是否包含一个或多个敏感列
ADMIT_NULL	CHARACTER VARYING(3 BYTE)	视图是否允许空 CON_ID 数据
PDB_LOCAL_ONLY	CHARACTER VARYING(3 BYTE)	仅供内部使用

1.3.2.33　DB_FILES

数据库中所有表空间、表空间使用的存储容量及表空间所在存储设备的容量信息。

表 1.3.38: DB_FILES 列

名称	类型	描述
NAME	NAME	表空间名称或总用量 TOTAL
USED	BIGINT	表空间已使用容量，单位 MB。TOTAL 时为数据目录所在存储设备已使用容量，单位 MB
CAPACITY	BIGINT	表空间所在存储设备容量，单位 MB。TOTAL 时为数据目录所在存储设备容量，单位 MB

-续表-

名称	类型	描述
RATIO	NUMERIC	使用率，scale 为 15

1.3.2.34　DBA_ALL_TABLES

记录当前数据库所有的对象表和关系表。视图结构与all_all_tables 相同。

1.3.2.35　DBA_ARGUMENTS

记录数据库中可用的过程和函数的参数。视图结构与all_arguments 结构相同。

1.3.2.36　DBA_COL_COMMENTS

记录数据库中所有表和视图的列上的注释。视图结构与all_col_comments 相同。

1.3.2.37　DBA_COL_PRIVS

记录所有列级权限。与all_col_privs 视图结构相同.

1.3.2.38　DBA_CONS_COLUMNS

记录约束中指定的数据库中的所有列。视图结构与all_cons_columns 相同。

1.3.2.39　DBA_CONSTRAINTS

记录当前用户所拥有的表的所有约束定义。视图结构与all_constraints 相同。

1.3.2.40　DBA_DB_LINKS

记录数据库 dblink 的信息，视图结构与all_db_links 相同。

表 1.3.39: DBA_DB_LINKS 列

名称	类型	描述
OWNER	VARCHAR	DBLINK 的属主
DB_LINK	NAME	DBLINK 名称
USERNAME	VARCHAR	DBLINK 连接用户名
HOST	VARCHAR	DBLINK 连接 IP 地址
CREATED	TIMESTAMP WITH TIME ZONE	DBLINK 创建时间

1.3.2.41　DBA_DIRECTORIES

记录数据库目录的信息，视图结构与all_directories 相同。

表 1.3.40: DBA_DIRECTORIES 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	目录的属主
DIRECTORY_NAME	CHARACTER VARYING(63 BYTE)	目录名称
DIRECTORY_PATH	CHARACTER VARYING(1023 BYTE)	目录路径
ORIGIN_CON_ID	CHARACTER VARYING(63 BYTE)	默认置 0

1.3.2.42　DBA_FREE_SPACE

记录数据库中每个表空间的可用空间。

表 1.3.41: DBA_FREE_SPACE 列

名称	类型	描述
TABLESPACE_NAME	CHARACTER VARYING(63 BYTE)	包含空闲空间的表空间的名称
FILE_ID	NUMERIC(38, 0)	包含空闲空间的文件的文件标识号
BLOCK_ID	NUMERIC(38, 0)	空闲空间的起始块号
BYTES	NUMERIC(38, 0)	空闲空间的大小（以字节为单位)
BLOCKS	NUMERIC(38, 0)	空闲空间的大小（以块为单位）
RELATIVE_FNO	NUMERIC(38, 0)	包含空闲空间的文件的相对文件号

1.3.2.43　DBA_IND_COLUMNS

记录数据库中所有建有索引的列的信息。视图结构与all_ind_columns 相同。

1.3.2.44　DBA_INDEXES

记录数据库中所有索引的信息。视图结构与all_indexes 相同

1.3.2.45　DBA_OBJECTS

记录当前用户下可以查看的所有对象。

表 1.3.42: DBA_OBJECTS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	对象的属主
OBJECT_NAME	CHARACTER VARYING(128 BYTE)	对象名
SUBOBJECT_NAME	CHARACTER VARYING(128 BYTE)	子对象名，目前值为 NULL
OBJECT_ID	NUMERIC	对象的 OID

-续表-

名称	类型	描述
DATA_OBJECT_ID	NUMERIC	保留字段
OBJECT_TYPE	CHARACTER VARYING(23 BYTE)	对象的类型
CREATED	DATE	对象创建的时间，目前值为 NULL
LAST_DDL_TIME	DATE	对象最近一次的修改时间。目前值为 NULL
TIMESTAMP	CHARACTER VARYING(19 BYTE)	该对象的说明定义的时间
STATUS	CHARACTER VARYING(7 BYTE)	对象的状态
TEMPORARY	CHARACTER VARYING(1 BYTE)	该对象是否是临时的
GENERATED	CHARACTER VARYING(1 BYTE)	对象名是否是系统生成的
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NAMESPACE	NUMERIC	对象所属模式

1.3.2.46　DBA_PART_TABLES

记录显示数据库内所有分区表，其字段与all_part_tables 相同。

1.3.2.47　DBA_ROLE_PRIVS

记录用户和数据库中的角色的授权关系信息。

表 1.3.43: DBA_ROLE_PRIVS 列

名称	类型	描述
GRANTEE	CHARACTER VARYING(128 BYTE)	接受授权的用户或角色的名称

-续表-

名称	类型	描述
GRANTED_ROLE	CHARACTER VARYING(128 BYTE)	授予角色名称
ADMIN_OPTION	CHARACTER VARYING(3 BYTE)	指示授予是否与 ADMIN OPTION 一起使用
DEFAULT_ROLE	CHARACTER VARYING(3 BYTE)	指示是否将角色指定为 DEFAULT ROLE 用户

1.3.2.48　DBA_ROLES

列出数据库中存在的所有角色。

表 1.3.44: DBA_ROLES 列

名称	类型	描述
ROLE	CHARACTER VARYING(63 BYTE)	角色名称
PASSWORD_REQUIRED	CHARACTER VARYING(8 BYTE)	指示角色是否需要启用密码

1.3.2.49　DBA_SCHEDULER_EXCEPTION

该视图记录数据库中所有处于异常状态的 schedule 作业信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_EXCEPTION”。

表 1.3.45: DBA_SCHEDULER_EXCEPTION 列

名称	数据类型	描述
OWNER	CHARACTER VARYING(63 CHAR)	任务的所有者
PROGRAM_NAME	TEXT	action 的名字

-续表-

名称	数据类型	描述
JOB_NAME	TEXT	Job 的名字
SCHEDULE_NAME	TEXT	Schedule 的名字
PROGRAM_ACTION	TEXT	action 要执行的命令
PROGRAM_TYPE	CHARACTER VARYING(63 CHAR)	Action 要执行的命令的类型 支持两种类型： SQL BATCH
EXCEPTION_TIME	TIMESTAMP WITHOUT TIME ZONE	异常发生的时间

1.3.2.50　DBA_SCHEDULER_JOB_LOG

该视图记录数据库中所有处于异常状态的 schedule 作业信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_JOB_LOG”。

表 1.3.46: DBA_SCHEDULER_JOB_LOG 列

名称	数据类型	描述
LOG_ID	NUMERIC	log 的 ID
LOG_DATE	TIMESTAMP WITHOUT TIME ZONE	记录 log 的时间
OWNER	CHARACTER VARYING(63 CHAR)	任务的所有者
JOB_NAME	TEXT	job 的名字
JOB_SUBNAME	TEXT	子任务名字
JOB_CLASS	CHARACTER VARYING(63 CHAR)	Job 的类型

-续表-

名称	数据类型	描述
OPERATION	TEXT	操作内容
STATUS	TEXT	Job 的状态，共有五种： running success failed internal failure aborted
USER_NAME	CHARACTER VARYING(63 CHAR)	用户名
CLIENT_ID	CHARACTER VARYING(7 CHAR)	恒为 NULL
GLOBAL_UID	CHARACTER VARYING(7 CHAR)	恒为 NULL
CREDENTIAL_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
CREDENTIAL_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
DESTINATION_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
DESTINATION	TEXT	数据库连接串
ADDITIONAL_INFO	CHARACTER VARYING(7 CHAR)	恒为 NULL

1.3.2.51　DBA_SCHEDULER_JOB_RUN_DETAILS

该视图记录数据库中所有 schedule 中作业的详细日志信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_JOB_RUN_DETAILS”。

表 1.3.47: DBA_SCHEDULER_JOB_RUN_DETAILS 列

名称	数据类型	描述
LOG_ID	NUMERIC	log 的 ID
LOG_DATE	TIMESTAMP WITHOUT TIME ZONE	记录 log 的时间
OWNER	CHARACTER VARYING(63 CHAR)	任务所有者
JOB_NAME	TEXT	job 的名字
JOB_SUBNAME	TEXT	子任务名称
STATUS	TEXT	Job 的状态，共有五种： running success failed internal failure aborted
ERROR	NUMERIC	恒为 NULL
REQ_START_DATE	TIMESTAMP WITHOUT TIME ZONE	恒为 NULL
ACTUAL_START_DATE	TIMESTAMP WITHOUT TIME ZONE	该任务开始执行的时间
RUN_DURATION	INTERVAL	任务持续的时间
INSTANCE_ID	NUMERIC	恒为 NULL
SESSION_ID	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	数据类型	描述
SLAVE_PID	CHARACTER VARYING(7 CHAR)	恒为 NULL
CPU_USED	CHARACTER VARYING(7 CHAR)	恒为 NULL
CREDENTIAL_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
CREDENTIAL_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
DESTINATION_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
DESTINATION	TEXT	数据库连接串
ADDITIONAL_INFO	CHARACTER VARYING(7 CHAR)	恒为 NULL
ERRORS	CHARACTER VARYING(7 CHAR)	恒为 NULL
OUTPUT	TEXT	任务执行情况
BINARY_ERRORS	CHARACTER VARYING(7 CHAR)	恒为 NULL
BINARY_OUTPUT	CHARACTER VARYING(7 CHAR)	恒为 NULL

1.3.2.52　DBA_SCHEDULER_JOBS

该视图记录数据库中所有 schedule 的作业的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_JOBS”。

表 1.3.48: DBA_SCHEDULER_JOBS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 CHAR)	Job 的拥有者
JOB_NAME	CHARACTER VARYING(63 CHAR)	Job 名字
JOB_SUBNAME	CHARACTER VARYING(63 CHAR)	恒为 NULL
JOB_STYLE	CHARACTER VARYING(7 CHAR)	恒为 NULL
JOB_CREATOR	CHARACTER VARYING(63 CHAR)	Job 的创建者
CLIENT_ID	CHARACTER VARYING(7 CHAR)	恒为 NULL
GLOBAL_UID	CHARACTER VARYING(7 CHAR)	恒为 NULL
PROGRAM_OWNER	CHARACTER VARYING(63 CHAR)	Job 关联的 program 的属主
PROGRAM_NAME	CHARACTER VARYING(63 CHAR)	Job 关联的 program 的名字

-续表-

名称	类型	描述
JOB_TYPE	CHARACTER VARYING(63 CHAR)	Job 的类型 PLSQL_BLOCK STORED_PROCEDURE EXECUTABLE EXTERNAL_SCRIPT SQL_SCRIPT BACKUP_SCRIPT HAS_CONSTRAINTS
JOB_ACTION	TEXT	Job 执行的动作
NUMBER_OF_ARGUMENTS	NUMERIC	恒为 0
SCHEDULE_OWNER	CHARACTER VARYING(63 CHAR)	Job 关联的 schedule 的属主
SCHEDULE_NAME	CHARACTER VARYING(63 CHAR)	Job 关联的 schedule 的名字
SCHEDULE_TYPE	CHARACTER VARYING(7 CHAR)	恒为 NULL
START_DATE	TIMESTAMP WITH TIME ZONE	Job 的开始时间
REPEAT_INTERVAL	TEXT	Job 的运行周期
EVENT_QUEUE_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_QUEUE_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_QUEUE_AGENT	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	类型	描述
EVENT_CONDITION	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_RULE	CHARACTER VARYING(7 CHAR)	恒为 NULL
FILE_WATCHER_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
FILE_WATCHER_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
END_DATE	TIMESTAMP WITH TIME ZONE	Job 的结束时间
JOB_CLASS	CHARACTER VARYING(63 CHAR)	Job 的类型
ENABLED	CHARACTER VARYING(7 CHAR)	Job 是否启用
AUTO_DROP	CHARACTER VARYING(7 CHAR)	恒为 NULL
RESTART_ON_RECOVERY	CHARACTER VARYING(7 CHAR)	恒为 NULL
RESTART_ON_FAILURE	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	类型	描述
STATE	TEXT	Job 的状态： RUNNING COMPLETED BROKEN FAILED SUCCEEDED STOPPED
JOB_PRIORITY	NUMERIC	恒为 0
RUN_COUNT	NUMERIC	Job 的执行次数
UPTIME_RUN_COUNT	NUMERIC	恒为 NULL
MAX_RUNS	NUMERIC	恒为 NULL
FAILURE_COUNT	NUMERIC	Job 的失败次数
UPTIME_FAILURE_COUNT	NUMERIC	恒为 NULL
MAX_FAILURES	NUMERIC	Job 的最大可失败次数
RETRY_COUNT	NUMERIC	Job 失败后重试次数
LAST_START_DATE	TIMESTAMP WITH TIME ZONE	Job 上一次执行的时间
LAST_RUN_DURATION	INTERVAL	Job 上一次执行持续时间
NEXT_RUN_DATE	TIMESTAMP WITH TIME ZONE	Job 下一次执行时间
SCHEDULE_LIMIT	INTERVAL	恒为 NULL
MAX_RUN_DURATION	INTERVAL	恒为 NULL
LOGGING_LEVEL	CHARACTER VARYING(7 CHAR)	恒为 NULL
STORE_OUTPUT	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	类型	描述
STOP_ON_WINDOW_CLOSE	CHARACTER VARYING(7 CHAR)	恒为 NULL
INSTANCE_STICKINESS	CHARACTER VARYING(7 CHAR)	恒为 NULL
RAISE_EVENTS	CHARACTER VARYING(7 CHAR)	恒为 NULL
SYSTEM	CHARACTER VARYING(7 CHAR)	恒为 NULL
JOB_WEIGHT	NUMERIC	恒为 NULL
NLS_ENV	CHARACTER VARYING(7 CHAR)	恒为 NULL
SOURCE	CHARACTER VARYING(7 CHAR)	恒为 NULL
NUMBER_OF_DESTINATIONS	NUMERIC	恒为 1
DESTINATION	TEXT	目标数据库
CREDENTIAL_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
CREDENTIAL_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
INSTANCE_ID	NUMERIC	恒为 NULL
DEFERRED_DROP	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	类型	描述
ALLOW_RUNS_IN_RESTRICTED_MODE	CHARACTER VARYING(7 CHAR)	恒为 NULL
COMMENTS	TEXT	注释
FLAGS	NUMERIC	标志符
RESTARTABLE	CHARACTER VARYING(7 CHAR)	恒为 NULL
HAS_CONSTRAINTS	CHARACTER VARYING(7 CHAR)	恒为 NULL
CONNECT_CREDENTIAL_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
CONNECT_CREDENTIAL_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
FAIL_ON_SCRIPT_ERROR	CHARACTER VARYING(7 CHAR)	恒为 NULL

1.3.2.53　DBA_SCHEDULER_PROGRAMS

该视图记录数据库中所有 Scheduler 程序的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_PROGRAMS”。

表 1.3.49: DBA_SCHEDULER_PROGRAMS 列

名称	数据类型	描述
OWNER	CHARACTER VARYING(63 CHAR)	program 的所有者

-续表-

名称	数据类型	描述
PROGRAM_NAME	CHARACTER VARYING(63 CHAR)	Program 的名字
PROGRAM_TYPE	CHARACTER VARYING(63 CHAR)	Program 的类型
PROGRAM_ACTION	TEXT	Program 要执行的命令
NUMBER_OF_ARGUMENTS	NUMERIC	恒为 0
ENABLED	BOOLEAN	Program 是否被启用
DETACHED	CHARACTER VARYING(7 CHAR)	恒为 NULL
SCHEDULE_LIMIT	INTERVAL	恒为 NULL
PRIORITY	NUMERIC	恒为 0
WEIGHT	NUMERIC	恒为 0
MAX_RUNS	NUMERIC	失败后最大可重复次数
MAX_FAILURES	NUMERIC	最大可失败次数
MAX_RUN_DURATION	INTERVAL	最长持续时间
HAS_CONSTRAINTS	CHARACTER VARYING(7 CHAR)	恒为 NULL
NLS_ENV	CHARACTER VARYING(7 CHAR)	恒为 NULL
COMMENTS	TEXT	注释

1.3.2.54　DBA_SCHEDULER_SCHEDULES

该视图记录数据库中所有 Scheduler 程序的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.DBA_SCHEDULER_SCHEDULES”。

表 1.3.50: DBA_SCHEDULER_SCHEDULES 列

名称	数据类型	描述
OWNER	CHARACTER VARYING(63 CHAR)	Schedule 的所有者
SCHEDULE_NAME	TEXT	Schedule 的名字
SCHEDULE_TYPE	CHARACTER VARYING(10 CHAR)	恒为 CALENDAR
START_DATE	TIMESTAMP WITHOUT TIME ZONE	job 开始时间
REPEAT_INTERVAL	TEXT	Job 执行周期
EVENT_QUEUE_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_QUEUE_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_QUEUE_AGENT	CHARACTER VARYING(7 CHAR)	恒为 NULL
EVENT_CONDITION	CHARACTER VARYING(7 CHAR)	恒为 NULL
FILE_WATCHER_OWNER	CHARACTER VARYING(7 CHAR)	恒为 NULL
FILE_WATCHER_NAME	CHARACTER VARYING(7 CHAR)	恒为 NULL

-续表-

名称	数据类型	描述
END_DATE	TIMESTAMP WITHOUT TIME ZONE	结束日期
COMMENTS	TEXT	注释

1.3.2.55　DBA_SEQUENCES

记录数据库中所有的序列信息。视图结构比all_sequences 视图少一个 START_VALUE 列，其余列名及列数据类型均相同。

1.3.2.56　DBA_SOURCE

记录数据库中所有存储对象的文本源。视图结构与all_source 相同。

1.3.2.57　DBA_SYNONYMS

记录数据库中所有同义词的信息。视图结构与all_synonyms 相同

1.3.2.58　DBA_TAB_COLS

记录当前所在数据库中，所有的表、视图的列信息，包括隐藏列和虚拟列。

表 1.3.51: DBA_TAB_COLS 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	表、视图的所有者
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表、视图的名字
COLUMN_NAME	CHARACTER VARYING(128 BYTE)	列名字

-续表-

名称	类型	描述
DATA_TYPE	CHARACTER VARYING(128 BYTE)	列的数据类型
DATA_TYPE_MOD	CHARACTER VARYING(3 CHAR)	列的修改的数据类型
DATA_TYPE_OWNER	CHARACTER VARYING(128 BYTE)	列的类型的所有者
DATA_LENGTH	NUMERIC	列的长度
DATA_PRECISION	NUMERIC	数据精度
DATA_SCALE	NUMERIC	数字的小数点右侧位数
NULLABLE	CHARACTER VARYING(1 BYTE)	描述列是否可以为 NULL
COLUMN_ID	NUMERIC	列的编号
DEFAULT_LENGTH	NUMERIC	列的默认长度
DATA_DEFAULT	TEXT	列的默认值
NUM_DISTINCT	NUMERIC	列上的 distinct 的值的数量
LOW_VALUE	NUMERIC	列的最小值
HIGH_VALUE	NUMERIC	列的最大值
DENSITY	NUMERIC	列的密度
NUM_NULLS	NUMERIC	在列上空值的数量
NUM_BUCKETS	NUMERIC	列上的大量数据的直方图
LAST_ANALYZED	DATE	是否约束被置为无效
SAMPLE_SIZE	NUMERIC	索引分析所使用的遍历大小
CHARACTER_SET_NAME	CHARACTER VARYING(44 BYTE)	字符集名称
CHAR_COL_DECL_LENGTH	NUMERIC	声明列的字符类型长度
GLOBAL_STATS	CHARACTER VARYING(3 BYTE)	是否将分区表作为整体来收集列的信息

-续表-

名称	类型	描述
USER_STATS	CHARACTER VARYING(3 BYTE)	是否由用户输入
AVG_COL_LEN	NUMERIC	平均长度（字节）
CHAR_LENGTH	NUMERIC	列中的字符的长度
CHAR_USED	CHARACTER VARYING(1 BYTE)	使用的类型
V80_FMT_IMAGE	CHARACTER VARYING(3 BYTE)	列中的数据是否是 8.0 图像格式
DATA_UPGRADED	CHARACTER VARYING(3 BYTE)	列的数据是否已经升级到最新版本的格式类型
HIDDEN_COLUMN	CHARACTER VARYING(3 BYTE)	该列是否是一个隐藏的列
VIRTUAL_COLUMN	CHARACTER VARYING(3 BYTE)	列是否是一个虚拟列
SEGMENT_COLUMN_ID	NUMERIC	在段中，列的编号
INTERNAL_COLUMN_ID	NUMERIC	内部的编号
HISTOGRAM	CHARACTER VARYING(15 BYTE)	类型使用频率的柱状图
QUALIFIED_COL_NAME	CHARACTER VARYING(4000 BYTE)	符合要求的列名

1.3.2.59　DBA_TAB_COLUMNS

记录当前所在数据库中，所有的表、视图的非隐藏列信息。

该视图的所有列来自于 DBA_TAB_COLS 的同名列，但是该视图显示的是非隐藏列信息。

表 1.3.52: DBA_TAB_COLUMNS 列

名称	类型	描述
OWNER	CHARACTER VARYING(63 BYTE)	表、视图的所有者
TABLE_NAME	CHARACTER VARYING(63 BYTE)	表、视图的名字
COLUMN_NAME	CHARACTER VARYING(63 BYTE)	列名字
DATA_TYPE	CHARACTER VARYING(106 BYTE)	列的数据类型
DATA_TYPE_MOD	CHARACTER VARYING(3 BYTE)	列的修改的数据类型
DATA_TYPE_OWNER	CHARACTER VARYING(63 BYTE)	列的类型的所有者
DATA_LENGTH	NUMERIC	列的长度
DATA_PRECISION	NUMERIC	数据精度。针对 numeric 和 float 类型
DATA_SCALE	NUMERIC	数字的小数点右侧位数。针对 numeric 类型
NULLABLE	CHARACTER VARYING(1 CHAR)	描述列是否可以为 NULL，'Y'/'N'
COLUMN_ID	NUMERIC	列的编号
DEFAULT_LENGTH	NUMERIC	列的默认长度没有默认值时为 NULL
DATA_DEFAULT	TEXT	列的默认值没有默认值时为 NULL
NUM_DISTINCT	NUMERIC	列上的 distinct 的值的数量
LOW_VALUE	NUMERIC	列的最小值
HIGH_VALUE	NUMERIC	列的最大值
DENSITY	NUMERIC	列的密度
NUM_NULLS	NUMERIC	在列上空值的数量
NUM_BUCKETS	NUMERIC	列上的大量数据的直方图

-续表-

名称	类型	描述
LAST_ANALYZED	TIMESTAMP(0) WITHOUT TIME ZONE	是否约束被置为无效
SAMPLE_SIZE	NUMERIC	声明列的字符类型长度
CHARACTER_SET_NAME	CHARACTER VARYING(44 CHAR)	字符集名称
CHAR_COL_DECL_LENGTH	NUMERIC	声明列的字符类型长度
GLOBAL_STATS	CHARACTER VARYING(3 CHAR)	是否将分区表作为整体来收集列的信息
USER_STATS	CHARACTER VARYING(3 CHAR)	是否由用户输入
AVG_COL_LEN	NUMERIC	平均长度（字节）
CHAR_LENGTH	NUMERIC	列中的字符的长度。单位为字符（CHAR）。非 VARCHAR,BPCHAR，CHAR 类型为 NULL
CHAR_USED	CHARACTER VARYING(1 CHAR)	使用的类型。'B' 表示字串以 byte 为单位，'C' 表示以 CHAR 为单位。例如 VARCHAR(40 CHAR)
V80_FMT_IMAGE	CHARACTER VARYING(3 CHAR)	列中的数据是否是 8.0 图像格式
DATA_UPGRADED	CHARACTER VARYING(3 CHAR)	列的数据是否已经升级到最新版本的格式类型
HISTOGRAM	CHARACTER VARYING(15 CHAR)	类型使用频率的柱状图

1.3.2.60　DBA_TAB_COMMENTS

记录数据库中所有表和视图的注释。视图结构与all_tab_comments 相同。

1.3.2.61　DBA_TAB_PARTITIONS

显示 DBA 所拥有的所有分区。视图结构与all_tab_partitions 相同。

1.3.2.62　DBA_TAB_PRIVS

记录数据库中的所有对象授予。视图结构与all_tab_privs 相同。

1.3.2.63　DBA_TABLES

记录当前数据库中的所有表的元信息。

表 1.3.53: DBA_TABLES 列

名称	类型	描述
OWNER	CHARACTER VARYING(128 BYTE)	表的属主
TABLE_NAME	CHARACTER VARYING(128 BYTE)	表名
TABLESPACE_NAME	CHARACTER VARYING(30 BYTE)	表空间名
CLUSTER_NAME	CHARACTER VARYING(128 BYTE)	CLUSTER 名字
IOT_NAME	CHARACTER VARYING(128 BYTE)	索引结构表的名字
STATUS	CHARACTER VARYING(8 BYTE)	表的状态
PCT_FREE	NUMERIC	块中空闲空间的最小百分率
PCT_USED	NUMERIC	块中已使用空间的最小百分率
INI_TRANS	NUMERIC	事务数的初始化大小
MAX_TRANS	NUMERIC	最大事务数
INITIAL_EXTENT	NUMERIC	初始化的大小
NEXT_EXTENT	NUMERIC	扩展的大小
MIN_EXTENTS	NUMERIC	段中允许扩展的最小值
MAX_EXTENTS	NUMERIC	段中允许扩展的最大值

-续表-

名称	类型	描述
PCT_INCREASE	NUMERIC	扩展时增加的百分率
FREELISTS	NUMERIC	空闲的进程列表序号
FREELIST_GROUPS	NUMERIC	空闲列表组的序号
LOGGING	CHARACTER VARYING(3 BYTE)	LOGGING 属性
BACKED_UP	CHARACTER VARYING(1 BYTE)	该表从上次修改到现在是否做了备份
NUM_ROWS	NUMERIC	表的行数
BLOCKS	NUMERIC	已使用的数据块数
EMPTY_BLOCKS	NUMERIC	空闲的数据块数
AVG_SPACE	NUMERIC	平均的空闲空间数
CHAIN_CNT	NUMERIC	表中跨数据块的行数
AVG_ROW_LEN	NUMERIC	平均一行的数据长度
AVG_SPACE_FREELIST_BLOCKS	NUMERIC	一个空闲列表中的所有数据块的平均空闲值
NUM_FREELIST_BLOCKS	NUMERIC	空闲列表中的数据块数
DEGREE	CHARACTER VARYING(10 BYTE)	扫描表时一个实例中的线程数
INSTANCES	CHARACTER VARYING(10 BYTE)	扫描表时的实例数
CACHE	CHARACTER VARYING(5 BYTE)	该表是否在缓存 CACHE 中
TABLE_LOCK	CHARACTER VARYING(8 BYTE)	该表是否被锁住
SAMPLE_SIZE	NUMERIC	分析表时的采样率
LAST_ANALYZED	DATE	最近一次 ANALYZE 的时间
PARTITIONED	CHARACTER VARYING(3 BYTE)	该表是否是分区表

-续表-

名称	类型	描述
IOT_TYPE	CHARACTER VARYING(12 BYTE)	索引组织表的类型
TEMPORARY	CHARACTER VARYING(1 BYTE)	该数据是否只有当前会话可见
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NESTED	CHARACTER VARYING(3 BYTE)	该表是否是嵌套的
BUFFER_POOL	CHARACTER VARYING(7 BYTE)	该对象的缺省的缓冲区
ROW_MOVEMENT	CHARACTER VARYING(8 BYTE)	分区表的行是否允许移动
GLOBAL_STATS	CHARACTER VARYING(3 BYTE)	分区表的统计信息
USER_STATS	CHARACTER VARYING(3 BYTE)	用户的状态
DURATION	CHARACTER VARYING(15 BYTE)	临时表的生存时间
SKIP_CORRUPT	CHARACTER VARYING(8 BYTE)	在索引扫描时是否忽略块标记中断
MONITORING	CHARACTER VARYING(3 BYTE)	该表是否设置了监控属性
CLUSTER_OWNER	CHARACTER VARYING(128 BYTE)	CLUSETR 的属主
DEPENDENCIES	CHARACTER VARYING(8 BYTE)	行级依赖跟踪是否可用
COMPRESSION	CHARACTER VARYING(8 BYTE)	该表是否压缩
DROPPED	CHARACTER VARYING(3 BYTE)	该表是否被删除

1.3.2.64　DBA_TABLESPACE

记录数据库中的表空间。视图结构与dba_tablespaces 相同。

1.3.2.65　DBA_TABLESPACES

记录数据库中的所有表空间。

表 1.3.54: DBA_TABLESPACES 列

名称	类型	描述
TABLESPACE_NAME	CHARACTER VARYING(63 BYTE)	表空间名称
INITIAL_EXTENT	NUMERIC	默认初始范围大小
NEXT_EXTENT	NUMERIC	默认增量扩展区大小
MIN_EXTENTS	NUMERIC	默认最小范围
MAX_EXTENTS	NUMERIC	默认最大范围数
PCT_INCREASE	NUMERIC	范围大小的默认增加百分比
MIN_EXTLEN	NUMERIC	该表空间的最小扩展区大小
STATUS	CHARACTER VARYING(9 BYTE)	表空间状态：ONLINE、OFFLINE、READ ONLY
CONTENTS	CHARACTER VARYING(9 BYTE)	表空间内容
LOGGING	CHARACTER VARYING(9 BYTE)	默认日志记录属性
EXTENT_MANAGEMENT	CHARACTER VARYING(10 BYTE)	指示表空间中的扩展区是字典管理的（DICTIONARY）还是本地管理的（LOCAL）
ALLOCATION_TYPE	CHARACTER VARYING(9 BYTE)	对表空间有效的扩展区分配类型
PLUGGED_IN	CHARACTER VARYING(3 BYTE)	指示表是否插好（YES）或（NO）

1.3.2.66　DBA_TRIGGER_COLS

记录数据库中所有触发器的列信息。视图结构与all_trigger_cols 相同。

1.3.2.67　DBA_TRIGGERS

记录数据库中所有的触发器。视图结构与all_triggers 相同。

1.3.2.68　DBA_TYPES

记录当前数据库所有的对象类型。视图结构与all_types 相同。

1.3.2.69　DBA_USERS

记录数据库中所有用户的信息。

表 1.3.55: DBA_USERS 列

名称	类型	描述
USERNAME	CHARACTER VARYING(128 BYTE)	用户名
USER_ID	NUMERIC	用户 ID
PASSWORD	CHARACTER VARYING(4000 BYTE)	用户的密码，此处为 NULL
ACCOUNT_STATUS	CHARACTER VARYING(32 BYTE)	用户状态
LOCK_DATE	DATE	该帐号被锁定的时间
EXPIRY_DATE	DATE	该帐号密码到期日期
DEFAULT_TABLESPACE	CHARACTER VARYING(63 BYTE)	缺省表空间，保留字段
TEMPORARY_TABLESPACE	CHARACTER VARYING(63 BYTE)	临时表空间。保留字段
CREATED	DATE	创建该用户的时间
PROFILE	CHARACTER VARYING(128 BYTE)	保留字段

-续表-

名称	类型	描述
INITIAL_RSRC_CONSUMER_GROUP	CHARACTER VARYING(128 BYTE)	保留字段
EXTERNAL_NAME	CHARACTER VARYING(4000 BYTE)	用户的外部名
PASSWORD_VERSIONS	CHARACTER VARYING(17 BYTE)	密码类别信息
EDITIONS_ENABLED	CHARACTER VARYING(1 BYTE)	是否可编辑

1.3.2.70　DBA_VIEWS

记录当前用户的所有视图的信息。视图结构与all_views 视图相同。

1.3.2.71　RECYCLEBIN

记录当前库下处于回收站中的对象信息和被删除的时间。

表 1.3.56: RECYCLEBIN 列

名称	类型	描述
OID	OID	对象编号
ORIGINAL_NAME	NAME	对象名称
DROPTIME	TIMESTAMP WITH TIME ZONE	对象被删除的时间
TYPE	TEXT	对象的类型

1.3.2.72　USER_ALL_TABLES

记录当前用户拥有的对象表和关系表。视图结构比all_all_tables少一个 OWNER 列，其余列名及列数据类型均相同。

1.3.2.73　USER_ARGUMENTS

记录当前用户拥有的过程和函数的参数。视图结构与all_arguments 视图相同。

1.3.2.74　USER_COL_COMMENTS

记录当前用户拥有的表和视图列上注释。视图结构与all_col_comments 视图相同。

1.3.2.75　USER_COL_PRIVS

记录当前用户下的列级权限。与all_col_privs 视图结构相同。

1.3.2.76　USER_CONS_COLUMNS

记录当前用户拥有并在约束中指定的列。视图结构与all_cons_columns 相同。

1.3.2.77　USER_CONSTRAINTS

记录当前用户所拥有的表的所有约束定义。视图结构与all_constraints 相同。

1.3.2.78　USER_DB_LINKS

记录当前用户所拥有的 dblink 信息。视图结构与all_db_links 相同。

1.3.2.79　USER_DIRECTORIES

记录当前用户可访问的目录信息。视图结构与all_directories 相同。

1.3.2.80　USER_FREE_SPACE

记录当前用户可访问的表空间中的自由扩展区。其列与dba_free_space 中的列相同。

1.3.2.81　USER_IND_COLUMNS

记录数据库中所有建有索引的列的信息。视图结构比all_ind_columns 少一个 INDEX_OWNER 列，其余列名及列数据类型均相同。

1.3.2.82　USER_INDEXES

记录当前用户拥有的所有索引信息。视图结构比all_indexes 少一个 OWNER 列，其余列名及列数据类型均相同。

1.3.2.83　USER_OBJECTS

记录当前用户的可访问的所有对象的信息。

表 1.3.57: USER_OBJECTS 列

名称	类型	描述
OBJECT_NAME	CHARACTER VARYING(63 BYTE)	对象名
SUBOBJECT_NAME	CHARACTER VARYING(63 BYTE)	子对象名，目前值为 NULL
OBJECT_ID	NUMERIC(38,0)	对象的 OID
DATA_OBJECT_ID	NUMERIC(38,0)	保留字段
OBJECT_TYPE	CHARACTER VARYING(19 BYTE)	对象的类型
CREATED	DATE	对象创建的时间，目前值为 NULL
LAST_DDL_TIME	DATE	对象最近一次的修改时间。目前值为 NULL
TIMESTAMP WITHOUT TIME ZONE	CHARACTER VARYING(20 BYTE)	该对象的说明定义的时间
STATUS	CHARACTER VARYING(7 BYTE)	对象的状态
TEMPORARY	CHARACTER VARYING(1 BYTE)	该对象是否是临时的
GENERATED	CHARACTER VARYING(1 BYTE)	对象名是否是系统生成的
SECONDARY	CHARACTER VARYING(1 BYTE)	该对象是否是其他方法生成的二级对象
NAMESPACE	NUMERIC(38,0)	对象的模式名

1.3.2.84　USER_PART_TABLES

显示当前用户所拥有的所有分区表，除了不显示 owner 字段，其他字段与all_part_tables 相同。

1.3.2.85　USER_ROLE_PRIVS

记录授予给当前用户的角的信息。

表 1.3.58: USER_ROLE_PRIVS 列

名称	类型	描述
USERNAME	CHARACTER VARYING(128 BYTE)	接受授权的用户或角色的名称
GRANTED_ROLE	CHARACTER VARYING(128 BYTE)	授予角色名称
ADMIN_OPTION	CHARACTER VARYING(3 BYTE)	指示授予是否与 ADMINOPTION 一起使用
DEFAULT_ROLE	CHARACTER VARYING(3 BYTE)	指示是否将角色指定为 DEFAULT ROLE 用户

1.3.2.86　USER_SCHEDULER_EXCEPTION

该视图记录用户拥有的处于异常状态的 schedule 作业信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_EXCEPTION”。

视图结构与dba_scheduler_exception 相同。

1.3.2.87　USER_SCHEDULER_JOB_LOG

该视图记录用户拥有的 schedule 中作业的日志信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_JOB_LOG”。

视图结构与dba_scheduler_job_log 相同。

1.3.2.88　USER_SCHEDULER_JOB_RUN_DETAILS

该视图记录用户拥有的 schedule 中作业的详细日志信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_JOB_RUN_DETAILS”。

视图结构与dba_scheduler_job_run_details 相同。

1.3.2.89　USER_SCHEDULER_JOBS

该视图记录用户的 schedule 的作业的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_JOBS”。

视图结构与dba_scheduler_jobs 相同。

1.3.2.90　USER_SCHEDULER_PROGRAMS

该视图记录用户拥有的 Scheduler 程序的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_PROGRAMS”。

视图结构与dba_scheduler_programs 相同。

1.3.2.91　USER_SCHEDULER_SCHEDULES

该视图记录用户拥有的 Scheduler 计划的信息，存放在 KDB_SCHEDULE 模式下，查询时请使用”KDB_SCHEDULE.USER_SCHEDULER_SCHEDULES”。

视图结构与dba_scheduler_schedules 相同。

1.3.2.92　USER_SEQUENCES

记录当前用户的所有序列的信息。视图结构比all_sequences 视图少一个 START_VALUE 列，其余列名及列数据类型均相同。

1.3.2.93　USER_SOURCE

记录当前用户的所有程序源的信息。视图结构相较all_source 视图少一个 OWNER 列。

1.3.2.94　USER_SYNONYMS

记录数据库中用户所有同义词的信息。视图结构比all_synonyms 少一个 OWNER 列，其余列名及列数据类型均相同。

1.3.2.95　USER_TAB_COLS

记录当前用户可以访问的表，视图的列。视图结构与dba_tab_cols 的区别只是没有 OWNER 列。

1.3.2.96　USER_TAB_COLUMNS

记录数据库中所有表列的信息。视图结构与dba_tab_columns 的区别只是没有 OWNER 列。

1.3.2.97　USER_TAB_COMMENTS

记录当前用户拥有的表和视图上的注释。视图结构与all_tab_comments 的区别只是没有 OWNER 列。

1.3.2.98　USER_TAB_PARTITIONS

显示当前用户所拥有及能访问到的所有分区，除了无 table_owner 字段，其他字段与all_tab_partitions 相同。

1.3.2.99　USER_TAB_PRIVS

记录当前用户的对象权限，该用户可以是对象的所有者、授予者或者被授予者。视图结构与dba_tab_privs 相同。

1.3.2.100　USER_TABLE_COLS

记录当前用户可访问的所有的表、视图的列信息，包括隐藏列和虚拟列。

表 1.3.59: USER_TABLE_COLS 列

名称	类型	描述
TABLE_NAME	CHARACTER VARYING(63 BYTE)	表、视图的名字。
COLUMN_NAME	CHARACTER VARYING(63 BYTE)	列名字。
DATA_TYPE	CHARACTER VARYING(106 BYTE)	列的数据类型。
DATA_TYPE_MOD	CHARACTER VARYING(3 CHAR)	列的修改的数据类型。
DATA_TYPE_OWNER	CHARACTER VARYING(63 BYTE)	列类型的所有者。
DATA_LENGTH	NUMERIC	列的长度。
DATA_PRECISION	NUMERIC	列数据精度。

-续表-

名称	类型	描述
DATA_SCALE	NUMERIC	数字的小数点右侧位数。针对 numeric 类型
NULLABLE	CHARACTER VARYING(1 CHAR)	描述列是否可以为 NULL，'Y'/'N'
COLUMN_ID	NUMERIC	列的编号。
DEFAULT_LENGTH	NUMERIC	列的默认长度。没有默认值时为 NULL
DATA_DEFAULT	TEXT	列的默认值。没有默认值时为 NULL
NUM_DISTINCT	NUMERIC	列上的 distinct 的值的数量
LOW_VALUE	NUMERIC	列的最小值
HIGH_VALUE	NUMERIC	列的最大值
DENSITY	NUMERIC	列的密度
NUM_NULLS	NUMERIC	在列上空值的数量
NUM_BUCKETS	NUMERIC	列上的大量数据的直方图
LAST_ANALYZED	TIMESTAMP(0) WITHOUT TIME ZONE	是否约束被置为无效
SAMPLE_SIZE	NUMERIC	声明列的字符类型长度
CHARACTER_SET_NAME	CHARACTER VARYING(44 CHAR)	字符集名称
CHAR_COL_DECL_LENGTH	NUMERIC	声明列的字符类型长度
GLOBAL_STATS	CHARACTER VARYING(3 CHAR)	是否将分区表作为整体来收集列的信息
USER_STATS	CHARACTER VARYING(3 CHAR)	是否由用户输入
AVG_COL_LEN	NUMERIC	平均长度（字节）
CHAR_LENGTH	NUMERIC	列中的字符的长度。单位为字符（CHAR）。非 VARCHAR，BPCHAR，CHAR 类型为 NULL

-续表-

名称	类型	描述
CHAR_USED	CHARACTER VARYING(1 CHAR)	使用的类型。'B' 表示字串以 byte 为单位，'C' 表示以 CHAR 为单位。例如 VARCHAR(40 CHAR)
V80_FMT_IMAGE	CHARACTER VARYING(3 CHAR)	列中的数据是否是 8.0 图像格式
DATA_UPGRADED	CHARACTER VARYING(3 CHAR)	列的数据是否已经升级到最新版本的格式类型
HIDDEN_COLUMN	CHARACTER VARYING(3 CHAR)	该列是否是一个隐藏的列
VIRTUAL_COLUMN	CHARACTER VARYING(3 CHAR)	列是否是一个虚拟列
SEGMENT_COLUMN_ID	NUMERIC	在划分中，列的编号
INTERNAL_COLUMN_ID	NUMERIC	内部的列编号
HISTOGRAM	CHARACTER VARYING(15 CHAR)	类型使用频率的柱状图
QUALIFIED_COL_NAME	CHARACTER VARYING(4000 CHAR)	符合要求的列名

1.3.2.101　USER_TABLES

记录用户的所有表的信息。与all_tables 的视图结构的区别只是没有 OWNER 列。

1.3.2.102　USER_TABLESPACE

记录当前用户可访问的表空间。其列（除外 PLUGGED_IN）与dba_tablespace 相同。

1.3.2.103　USER_TABLESPACES

记录当前用户可访问的表空间。其列（除外 PLUGGED_IN）与dba_tablespaces 相同。

1.3.2.104　USER_TRIGGER_COLS

记录当前用户用的所有触发器的列信息。视图结构与all_trigger_cols 相同。

1.3.2.105　USER_TRIGGERS

记录当前用户的所有触发器的信息。视图结构与all_triggers 的区别只是没有 OWNER 列。

1.3.2.106　USER_TYPES

记录当前用户拥有的对象类型。视图结构比all_types 少一个 OWNER 列，其余列名及列数据类型均相同。

1.3.2.107　USER_USERS

记录当前用户可见的数据库的所有用户信息。

表 1.3.60: USER_USERS 列

名称	类型	描述
USERNAME	CHARACTER VARYING(63 BYTE)	用户名
USER_ID	NUMERIC(38,0)	用户 ID
ACCOUNT_STATUS	CHARACTER VARYING(32 BYTE)	账户状态
LOCK_DATE	DATE	被锁定时间
EXPIRY_DATE	DATE	用户到期时间
DEFAULT_TABLESPACE	CHARACTER VARYING(63 BYTE)	默认表空间
TEMPORARY _TABLESPACE	CHARACTER VARYING(63 BYTE)	临时表空间
CREATED	DATE	被创建的时间
INITIAL_RSRC _CONSUMER_GROUP	CHARACTER VARYING(63 BYTE)	默认置 NULL
EXTERNAL_NAME	CHARACTER VARYING(63 BYTE)	默认置 NULL

1.3.2.108　USER_VIEWS

记录当前用户所有的视图的信息。视图结构比all_views 少一个 OWNER 列，其余列名及列数据类型均相同。

1.4　PostgreSQL 兼容视图

KingbaseES 为保证兼容 PostgreSQL 用户使用习惯, 提供了两部分 PostgreSQL 兼容视图供用户使用：元信息视图（兼容 PostgreSQL 系统表）和信息模式视图。

元信息视图都放在 pg_catalog 模式下，用于记录对象元信息；信息模式视图都放在 information_schema 模式下，用于提供对外的元信息，需要注意的是信息模式视图访问时需要加上模式限定-“information_schema.”。

1.4.1　元信息视图

PostgreSQL 元信息视图用于记录对象元信息数据，便于用户查询对象操作结果。

1.4.1.1　pg_aggregate

pg_aggregate 存储关于聚集函数的信息。聚集函数是对一个数值集合（典型的是每个匹配查询条件的行中的同一个列的值）进行操作的函数，它返回从这些值中计算出的一个数值。典型的聚集函数是 sum、count 和 max。pg_aggregate 里的每个项都是一个 pg_proc 项的扩展。pg_proc 项记载该聚集的名字、输入和输出数据类型，以及其他一些和普通函数类似的信息。

表 1.4.1: pg_aggregate 列

名称	类型	描述
aggfnoid	regproc	聚集函数在 pg_proc 中的 OID
aggkind	char	聚集类型： n 表示“普通”聚集 o 表示“有序集”聚集 或者 h 表示“假想集”聚集
aggnumdirectargs	int2	一个有序集或者假想集聚集的直接（非聚集）参数的数量，一个可变数组算作一个参数。如果等于 pronargs，该聚集必定是可变的并且该可变数组描述聚集参数以及最终直接参数。对于普通聚集总是为零
aggtransfn	regproc	转移函数

-续表-

名称	类型	描述
aggfinalfn	regproc	最终函数（如果没有就为零）
aggcombinefn	regproc	结合函数（如果没有就为零）
aggserialfn	regproc	序列化函数（如果没有就为零）
aggdeserialfn	regproc	反序列化函数（如果没有就为零）
aggmtransfn	regproc	用于移动聚集模式的向前转移函数（如果没有就为零）
aggminvtransfn	regproc	用于移动聚集模式的反向转移函数（如果没有就为零）
aggmfinalfn	regproc	用于移动聚集模式的最终函数（如果没有就为零）
aggfinalextra	bool	为真则向 aggfinalfn 传递额外的哑参数
aggmfinalextra	bool	为真则向 aggmfinalfn 传递额外的哑参数
aggfinalmodify	char	aggfinalfn 是否修改传递状态值：如果是只读则为 r，如果不能在 aggfinalfn 之后应用 aggtransfn 则为 s，如果它修改该值则为 w
aggmfinalmodify	char	和 aggfinalmodify 类似，但是用于 aggmfinalfn
aggsortop	oid	相关联的排序操作符（如果没有则为 0）
aggtranstype	oid	聚集函数的内部转移（状态）数据的数据类型
aggtransspace	int4	转移状态数据的近似平均尺寸（字节），或者为零表示使用一个默认估算值
aggmtranstype	oid	聚集函数用于移动聚集欧氏的内部转移（状态）数据的数据类型（如果没有则为零）
aggmtransspace	int4	转移状态数据的近似平均尺寸（字节），或者为零表示使用一个默认估算值
agginitval	text	转移状态的初始值。这是一个文本域，它包含初始值的外部字符串表现形式。如果这个域为空，则转移状态值从空值开始。
aggminitval	text	用于移动聚集模式的转移状态初值。这是一个文本域，它包含了以其文本字符串形式表达的初值。如果这个域为空，则转移状态值从空值开始

1.4.1.2　pg_am

pg_am 视图存储关于关系访问方法的信息。系统支持的每种访问方法在这个视图中都有一行。目前只有表和索引拥有访问方法。

表 1.4.2: pg_am 列

名称	类型	描述
oid	oid	行标识符
amname	name	访问方法的名字
amhandler	regproc	负责提供有关该访问方法信息的处理器函数的 OID
amtype	char	t = 表 (包括物化视图), i = 索引

1.4.1.3　pg_amop

视图 pg_amop 存储关于与访问方法操作符族相关的操作符信息。对于一个操作符族中的每一个成员即操作符都在这个视图中有一行。一个成员可以是一个搜索操作符或者一个排序操作符。一个操作符可以出现在多个族中，但在同一个组中既不能出现在多个搜索位置也不能出现在多个排序位置（虽然不太可能出现，但是允许一个操作符同时用于搜索和排序目的）。

表 1.4.3: pg_amop 列

名称	类型	描述
oid	oid	行标识符
amopfamily	oid	这个项所在的操作符族
amoplefttype	oid	操作符的左操作数数据类型
amoprighttype	oid	操作符的右操作数数据类型
amopstrategy	int2	操作符策略号
amoppurpose	char	操作符目的，s 表示搜索，o 表示排序
amopopr	oid	操作符的 OID

-续表-

名称	类型	描述
amopmethod	oid	使用此操作符族的索引访问方法
amopsortfamily	oid	如果是一个排序操作符，该项会根据这个 B 树操作符族排序，如果是一个搜索操作符则为 0

一个“搜索”操作符项意味着该操作符族的一个索引可以被搜索来查找所有满足如下条件的行：WHERE indexed_column operator constant。显然，这样的一个操作符必须返回 boolean，且它的左手输入类型必须匹配索引列的数据类型。

一个“排序”操作符项意味着该操作符族的一个索引可以被扫描来返回以如下顺序排列的行：ORDER BY indexed_column operator constant。这样一个操作符能够返回任何可排序数据类型，尽管它的左手输入类型必须匹配索引列的数据类型。ORDER BY 的准确语义由 amopsortfamily 列指定，它必须引用一个适合于操作符结果类型的 B 树操作符族。

一个项的 amopmethod 必须和它所包含的操作符族的 opfmethod 相匹配。此外，amoplefttype 和 amoprighttype 也必须匹配被引用的 pg_operator 项的 oprleft 和 oprright 列。

1.4.1.4　pg_amproc

视图 pg_amproc 存储关于访问方法操作符族相关的支持函数。属于一个操作符族的每一个支持函数在这个视图中都有一行。

表 1.4.4: pg_amproc 列

名称	类型	描述
oid	oid	行标识符
amprocfamily	oid	使用这个项的操作符族
amproclefttype	oid	相关操作符的左操作数数据类型
amprocrighttype	oid	相关操作符的右操作数数据类型
amprocnum	int2	支持过程编号
amproc	regproc	过程的 OID

amproclefttype 和 amprocrighttype 列的通常解释是它们标识了一个特定支持过程所支持的操作符的左右输入类

型。对于某些访问方法它们和支持过程本身的输入数据类型相匹配，而对其他的则不会匹配。对于一个索引有一个“默认”支持过程的概念，这些支持过程的 amprocleftype 和 amprocrighttype 都等于索引操作符类的 opcintype。

1.4.1.5　pg_attrdef

pg_attrdef 存储列的默认值。列的主要信息存储在 pg_attribute。只有那些显式指定了一个默认值的列才会在这个视图中有一个项。

表 1.4.5: pg_attrdef 列

名称	类型	描述
oid	oid	行标识符
adrelid	oid	该列所属的表
adnum	int2	列号
adbin	pg_node_tree	列的默认值，以 nodeToString() 表示。用 pg_get_expr(adbin,adrelid) 将其转换为 SQL 表达式

1.4.1.6　pg_attribute

视图 pg_attribute 存储有关表列的信息。数据库中的每一个表的每一个列都恰好在 pg_attribute 中有一行。（这其中也会有索引的属性项，并且事实上所有具有 pg_class 项的对象在这里都有属性项）entries.)

术语属性等同于列，这里使用它只是出于历史原因。

表 1.4.6: pg_attribute 列

名称	类型	描述
attrelid	oid	列所属的表
attname	name	列名
atttypid	oid	列的数据类型

-续表-

名称	类型	描述
attstattarget	int4	attstattarget 控制由 ANALYZE 对此列收集的统计信息的细节层次 0 值表示不会收集任何统计信息。 一个负值则说明直接使用系统默认的目标 正值的确切含义取决于数据类型 对于标量数据类型，attstattarget 既是要收集的“最常见值”的目标号，也是要创建的柱状图容器的目标号
attlen	int2	本列类型的 pg_type.typlen 一个拷贝
attnum	int2	列的编号。一般列从 1 开始向上编号。系统列（如 ctid）则拥有（任意）负值编号
attndims	int4	如果该列是一个数组类型，这里就是其维度数；否则为 0。（在目前一个数组的维度数并不被强制，因此任何非零值都能有效地表明“这是一个数组”。）
attcacheoff	int4	在存储中总是为-1，但是当被载入到一个内存中的行描述符后，这里可能会被更新为属性在行内的偏移
atttypmod	int4	atttypmod 记录了在表创建时提供的类型相关数据（例如一个 varchar 列的最大长度）。它会被传递给类型相关的输入函数和长度强制函数。对于那些不需要 atttypmod 的类型，这个值通常总是为-1
attbyval	bool	该列类型的 pg_type.typbyval 的一个拷贝
attstorage	char	通常是该列类型的 pg_type.typstorage 的一个拷贝。对于可 TOAST 的数据类型，这可以在列创建后被修改以控制存储策略
attalign	char	该列类型的 pg_type.typalign 的一个拷贝
attnotnull	bool	这表示一个非空约束
atthasdef	bool	该列有一个默认表达式或生成的表达式，在此情况下在 pg_attrdef 视图中会有一个对应项来真正定义该表达式。（检查 attgenerated 以确定是默认还是生成的表达式。）

-续表-

名称	类型	描述
atthasmissing	bool	该列在行中完全缺失时会用到这个列的值，如果在行创建之后增加一个有非易失 DEFAULT 值的列，就会发生这种情况。实际使用的值被存放在 attmissingval 列中
attidentity	char	如果是一个零字节（”），则不是一个标识列。否则，a = 总是生成，d = 默认生成
attgenerated	char	如果为零字节 (”)，则不是生成的列。否则，s= stored
attisdropped	bool	该列被删除且不再有效。一个删除的列仍然物理存在于表中，但是会被分析器忽略并因此无法通过 SQL 访问
attislocal	bool	该列是由关系本地定义的。注意一个列可以同时是本地定义和继承的
attinhcount	int4	该列的直接祖先的编号。一个具有非零编号祖先的列不能被删除或者重命名
attcollation	oid	该列被定义的排序规则，如果该列不是一个可排序数据类型则为 0
attacl	aclitem[]	列级访问权限
attoptions	text[]	属性级选项，以“keyword=value”形式的字符串
attfdwoptions	text[]	属性级的外部数据包装器选项，以“keyword=value”形式的字符串
attmissingval	anyarray	这个列中是一个含有一个元素的数组，其中的值被用于该列在行中完全缺失时，如果在行创建之后增加一个有非易失 DEFAULT 值的列，就会发生这种情况 只有当 atthasmissing 为真时才使用这个值。如果没有值则该列为空

在一个被删除的列的 pg_attribute 的项中，atttypid 被重置为 0，但 attlen 以及其他从 pg_type 拷贝的域仍然有效。这种安排用于处理一种情况，即被删除列的数据类型后来被删除，并且因此不再有相应的 pg_type 行。attlen 和其他域可以被用来解释表的一行的内容。

1.4.1.7　pg_auth_members

视图 pg_auth_members 展示了角色之间的成员关系。允许任何无环的关系集合。

由于用户标识符是集簇范围的，pg_auth_members 在一个集簇的所有数据库之间共享：在一个集簇中只有一份 pg_auth_members 拷贝，而不是每个数据库一份。

表 1.4.7: pg_auth_members 列

名称	类型	描述
roleid	oid	拥有成员的角色的 ID
member	oid	roleid 的成员角色的 ID
grantor	oid	授权此成员关系的角色的 ID
admin_option	bool	如果 member 能把 roleid 的成员关系授予他人，则为真

1.4.1.8　pg_authid

视图 pg_authid 包含关于数据库授权标识符（角色）的信息。角色把“用户”和“组”的概念包含在内。一个用户实际上就是一个 rolcanlogin 标志被设置的角色。任何角色（不管 rolcanlogin 设置与否）都能够把其他角色作为成员，参见 pg_auth_members。

由于这个视图包含密码，它不能是公共可读的。pg_roles 是在 pg_authid 上的一个公共可读视图，它隐去了密码列。

由于用户标识符是集簇范围的，pg_authid 在一个集簇的所有数据库之间共享：在一个集簇中只有一份 pg_authid 拷贝，而不是每个数据库一份。

表 1.4.8: pg_authid 列

名称	类型	描述
oid	oid	行标识符
rolname	name	角色名
rolsuper	bool	角色是否拥有超级用户权限
rolinherit	bool	如果本角色是另一个角色的成员，本角色是否自动另一个角色的权限

-续表-

名称	类型	描述
rolcreaterole	bool	角色是否能创建更多角色
rolcreatedb	bool	角色是否能创建数据库
rolcanlogin	bool	角色是否能登录。即该角色是否能够作为初始会话授权标识符
rolreplication	bool	角色是一个复制角色。复制角色可以启动复制连接并且创建和删除复制槽
rolbypassrls	bool	角色是否可以绕过所有的行级安全性策略
rolconnlimit	int4	对于可以登录的角色，本列设置该角色可以同时发起最大连接数。-1 表示无限制
rolpassword	text	密码（可能被加密过），如果没有密码则为空。格式取决于使用的加密方法的形式
rolvaliduntil	timestamptz	密码过期时间（只用于密码鉴定），如果永不过期则为空

对于一个 MD5 加密的密码，rolpassword 列将由字符串 md5 后面跟上一个 32 字符的十六进制 MD5 哈希值构成。MD5 哈希值将是该用户的密码串接上它们的用户名。例如，如果用户 joe 的密码是 xyzzy，则将存储 xyzzyjoe 的 md5 哈希。

如果密码采用 SCRAM-SHA-256 加密，它的格式是：

`SCRAM-SHA-256$<iteration count>:<salt>$<StoredKey>:<ServerKey>` 其中 salt、StoredKey 和 ServerKey 是 Base64 编码格式。这种格式与 RFC 5803 说明的格式相同。

不遵守上述格式的密码被假定为未加密。

1.4.1.9　pg_available_extension_versions

pg_available_extension_versions 视图列出了可用于安装的指定扩展版本。参见 pg_extension 视图，它显示当前已安装的扩展。

表 1.4.9: pg_available_extension_versions 列

名称	类型	描述
name	name	扩展名
version	text	版本名
installed	bool	如果此版本的扩展当前已安装则为真
superuser	bool	如果只有超级用户被允许安装此扩展则为真
relocatable	bool	如果扩展能被重定位到另一个模式则为真
schema	name	此扩展必须被安装到的模式名，如果此扩展是部分或者全部可以重定位的，此列为 NULL
requires	name[]	先决条件扩展的名字，如果没有则为 NULL
comment	text	来自于扩展的控制文件的注释字符串

pg_available_extension_versions 视图是只读的。

1.4.1.10　pg_available_extensions

pg_available_extensions 视图列出了可用于安装的扩展。参见 pg_extension 视图，它显示当前已安装的扩展。

表 1.4.10: pg_available_extensions 列

名称	类型	描述
name	name	扩展名
default_version	text	默认版本的名字，如果没有指定则为 NULL
installed_version	text	当前已安装的扩展版本，如果没有安装则为 NULL
comment	text	来自于扩展的控制文件的注释字符串

pg_available_extensions 视图是只读的。

1.4.1.11　pg_cast

视图 pg_cast 存储数据类型转换路径，包括内建的和用户定义的类型。

需要注意的是，pg_cast 并不表示系统知道如何执行的所有类型转换，它只包括哪些不能从某些普通规则推导出的转换。例如，一个域及其基类型之间的转换并未显式地在 pg_cast 中展示。另一个重要的例外是“自动 I/O 转换造型”，它们通过数据类型自己的 I/O 函数来转换成（或者转换自）text 或其他字符串类型，这些转换也没有显式地在 pg_cast 中表示。

表 1.4.11: pg_cast 列

名称	类型	描述
oid	oid	行标识符
castsource	oid	源数据类型的 OID
casttarget	oid	目标数据类型的 OID
castfunc	oid	执行该转换的函数的 OID。如果该转换方法不需要一个函数则存储 0
castcontext	char	指示该转换能被调用的环境 e 表示仅能作为一个显式转换（使用 CAST 或:: 语法） a 表示在赋值给目标列时隐式调用，和显式调用一样 i 表示在表达式中隐式调用，和其他转换一样
castmethod	char	指示转换如何被执行 f 表明使用 castfunc 中指定的函数 i 表明使用输入/输出函数 b 表明该类型是二进制可转换的，因此不需要转换

在 pg_cast 里列出的类型转换函数必须总是以转换的源类型作为它的第一个参数类型，并且返回转换的目标类型作为它的结果类型。一个类型转换函数最多有三个参数。如果出现了第二个参数，必须是 int4 类型如果没有，就是 -1。如果出现了第三个参数，那么必须是 boolean 类型；如果该类型转换是一种明确的转换，那么它接受 true，否则接受 false。

在 pg_cast 里创建一条源类型和目标类型相同的记录是合理的，只要相关联的函数接受多过一个参数。这样的记录代表“长度转换函数”，它们把该类型的值转换为对特定的类型合法的值。

如果一个 pg_cast 的项有着不同的原类型和目标类型，并且有一个接收多于一个参数的函数，那么它会在一个步骤中完成从一种类型到另外一种类型的转换并应用一个长度转换。如果没有这样的项，使用一个类型修改器的转换涉

及两个步骤，一个是在数据类型之间转换，另外一个是应用修改器。

1.4.1.12　pg_class

视图 pg_class 记录表和几乎所有具有列或者像表的对象的信息。这包括索引（但还要参见 pg_index）、序列（但还要参见 pg_sequence）、视图、物化视图、组合类型和 TOAST 表，参见 relkind。下面，当我们提及所有这些类型的对象时我们使用“关系”。并非所有列对于所有关系类型都有意义。

表 1.4.12: pg_class 列

名称	类型	描述
oid	oid	行标识符
relname	name	表、索引、视图等的名字
relnamespace	oid	包含该关系的名字空间的 OID
reltype	oid	可能存在的表行类型所对应数据类型的 OID（对索引为 0，索引没有 pg_type 项）
reloftype	oid	对于有类型的表，为底层组合类型的 OID，对于其他所有关系为 0
relowner	oid	关系的拥有者
relam	oid	如果这是一个表或者索引，表示索引使用的访问方法（堆、B 树、哈希等）
relfilenode	oid	该关系的磁盘文件的名字，0 表示这是一个“映射”关系，其磁盘文件名取决于低层状态
reltablespace	oid	该关系所存储的表空间。如果为 0，使用数据库的默认表空间。（如果关系无磁盘文件时无意义）
relpages	int4	该表磁盘表示的尺寸，以页面计（页面尺寸为 BLCKSZ）。这只是一个由规划器使用的估计值。它被 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATEINDEX）所更新
reltuples	float4	表中的存活行数。这只是一个由规划器使用的估计值。它被 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATEINDEX）所更新
relallvisible	int4	在表的可见性映射表中被标记为全可见的页数。这只是一个由规划器使用的估计值。它被 VACUUM、ANALYZE 以及一些 DDL 命令（如 CREATEINDEX）所更新

-续表-

名称	类型	描述
reltoastrelid	oid	与该表相关联的 TOAST 表的 OID，如果没有则为 0。TOAST 表将大属性“线外”存储在一个二级表中
relhasindex	bool	如果这是一个表并且其上建有（或最近建有）索引则为真
relisshared	bool	如果该表在集簇中的所有数据库间共享则为真。只有某些系统视图（如 pg_database）是共享的
relpersistence	char	p = 永久表 u = 无日志表 t = 临时表
relkind	char	r = 普通表 i = 索引 S = 序列 t = TOAST 表 v = 视图 m = 物化视图 c = 组合类型 f = 外部表 p = 分区表 I = 分区索引
relnatts	int2	关系中用户列的数目（系统列不计算在内）。在 pg_attribute 中必须有这么多对应的项。另请参阅 pg_attribute.attnum
relchecks	int2	表上 CHECK 约束的数目，参见 pg_constraint 视图
relhasrules	bool	如果表有（或曾有）规则则为真，参见 pg_rewrite 视图
relhastriggers	bool	如果表有（或曾有）触发器则为真，参见 pg_trigger 视图
relhassubclass	bool	如果表或者索引有（或曾有）任何继承子女则为真
relrowsecurity	bool	如果表上启用了行级安全性则为真，参见 pg_policy 视图
relforcerowsecurity	bool	如果行级安全性（启用时）也适用于表拥有者则为真，参见 pg_policy 视图

-续表-

名称	类型	描述
relispopulated	bool	如果表已被填充则为真（对于所有关系该列都为真，但对于某些物化视图却不是）
relreplident	char	用来为行形成“replica identity”的列： d = 默认 (主键，如果存在) n = 无 f = 所有列 i = 索引的 indisreplident 被设置或者为默认
relispartition	bool	如果表或索引是一个分区，则为真
relrewrite	oid	对于在要求表重写的 DDL 操作期间被写入的新关系，这个域包含原始关系的 OID，否则为 0。那种状态仅在内部可见，对于一个用户可见的关系这个域应该从不包含不是 0 的值
relfrozenxid	xid	在此之前的所有事务 ID 在表中已经被替换为一个永久的（“冻结的”）事务 ID。这用于跟踪表是否需要被清理，以便阻止事务 ID 回卷或者允许 pg_xact 被收缩。如果该关系不是一个表则为 0（InvalidTransactionId）
relminmxid	xid	在此之前的多事务 ID 在表中已经被替换为一个事务 ID。这被用于跟踪表是否需要被清理，以阻止多事务 ID 回卷或者允许 pg_multixact 被收缩。如果关系不是一个表则为 0（InvalidMultiXactId）
relacl	aclitem[]	访问权限
reloptions	text[]	访问方法相关的选项，以“keyword=value”字符串形式
relpartbound	pg_node_tree	如果表示一个分区（见 relispartition），分区边界的内部表达

pg_class 中的一些逻辑标志被以一种懒惰的方式维护：在正确状态时它们被保证为真，但是当条件不再为真时它们并不会被立刻重置为假。例如，relhasindex 由 CREATE INDEX 设置，但它从不会被 DROP INDEX 清除。作为替代，VACUUM 会在找到无索引表后清除其 relhasindex。这种安排避免了竞争条件并且提高了并发性。

1.4.1.13　pg_collation

视图 pg_collation 描述了可用的排序规则，其本质是从一个 SQL 名字到操作系统 locale 分类的映射。

表 1.4.13: pg_collation 列

名称	类型	描述
oid	oid	行标识符
collname	name	排序规则名字（在每一个名字空间和编码中唯一）
collnamespace	oid	包含该排序规则的名字空间的 OID
collowner	oid	排序规则的拥有者
collprovider	char	排序规则的提供者： d = 数据库默认 c = libc i = icu
collisdeterministic	bool	排序规则是确定性的吗？
collencoding	int4	该排序规则可应用的编码，-1 表示它可用于任何编码
collcollate	name	该排序规则对象的 LC_COLLATE
collctype	name	该排序规则对象的 LC_CTYPE
collversion	text	排序规则的提供者相关的版本。这是在排序规则创建时记录下来的，并且在使用排序规则时会被检查以检测可能导致数据损坏的排序规则定义的改变。

备注:

注意在这个视图中的唯一键是（collname、collencoding、collnamespace），不仅仅是（collname，collnamespace）。所有 collencoding 不等于当前数据库编码或-1 的编码规则通常都会被忽略，且禁止创建和 collencoding=-1 的项重名的项。因此使用一个受限的 SQL 名字（schema.name）来标识一个排序规则是足够的，即使这根据视图定义是不唯一的。

在 template0 数据库中，创建与数据库编码不匹配的编码是有用的，因为它们可以匹配后面从 template0 克隆的数据库的编码。这在目前必须手动完成。

1.4.1.14　pg_config

视图 pg_config 描述了当前安装的 PostgreSQL 版本中的编译时配置参数。它存在的本意是用于那些要和 PostgreSQL 交互的软件包，让它们能找到所需要的头文件和库。它提供了和 pg_config PostgreSQL 客户端应用相同的基本信息。

默认情况下，pg_config 视图只能由超级用户读取。

表 1.4.14: pg_config 列

名称	类型	描述
name	text	参数名
setting	text	参数值

1.4.1.15　pg_constraint

视图 pg_constraint 存储表上的检查、主键、唯一、外键和排他约束（列约束也不会被特殊对待。每一个列约束都等同于某种表约束），非空约束不在这里，而是在 pg_attribute 视图中表示。

用户定义的约束触发器（使用 CREATE CONSTRAINT TRIGGER 创建）也会在这个表中产生一项。

域上的检查约束也存储在这里。

表 1.4.15: pg_constraint 列

名称	类型	描述
oid	oid	行标识符
conname	name	约束名字（不需要唯一！）
connamespace	oid	包含此约束的名字空间的 OID
contype	char	c = 检查约束 f = 外键约束 p = 主键约束 u = 唯一约束 t = 约束触发器 x = 排他约束
condeferrable	bool	该约束是否能被延迟

-续表-

名称	类型	描述
condeferred	bool	该约束是否默认被延迟
convalidated	bool	此约束是否被验证过？当前对于外键和检查约束只能是假
conrelid	oid	该约束所在的表，如果不是表约束则为 0
contypid	oid	该约束所在的域，如果不是域约束则为 0
conindid	oid	如果该约束是唯一、主键、外键或排他约束，此列表示支持此约束的索引，否则为 0
conparentid	oid	如果这是一个分区中的约束，则是父分区表中对应的约束；否则为 0
confrelid	oid	如果此约束是一个外键约束，此列为被引用的表，否则为 0
confupdtype	char	外键更新动作代码： a = 无动作 r = 限制 c = 级联 n = 置空 d = 置为默认值
confdeltype	char	外键删除动作代码： a = 无动作 r = 限制 c = 级联 n = 置空 d = 置为默认值
confmatchtype	char	外键匹配类型： f = 完全 p = 部分 s = 简单
conislocal	bool	此约束是定义在关系本地。注意一个约束可以同时是本地定义和继承
coninhcount	int4	此约束的直接继承祖先数目。一个此列非零的约束不能被删除或重命名
connoinherit	bool	为真表示此约束被定义在关系本地。它是一个不可继承约束

-续表-

名称	类型	描述
conkey	int2[]	如果是一个表约束（包括外键但不包括约束触发器），此列是被约束列的列表
confkey	int2[]	如果是一个外键，此列是被引用列的列表
conpfeqop	oid[]	如果是一个外键，此列是用于 PK = FK 比较的等值操作符的列表
conppeqop	oid[]	如果是一个外键，此列是用于 PK = PK 比较的等值操作符的列表
conffeqop	oid[]	如果是一个外键，此列是用于 FK = FK 比较的等值操作符的列表
conexclop	oid[]	如果是一个排他约束，此列是没列排他操作符的列表
conbin	pg_node_tree	如果是一个检查约束，此列是表达式的一个内部表示。建议使用 pg_get_constraintdef() 提取检查约束的定义

在一个排他约束的情况中，conkey 只对约束元素是单一列引用时有用。对于其他情况，conkey 为 0 且必须查阅相关索引来发现被约束的表达式（conkey 因此和 pg_index.indkey 具有相同的内容）。

1.4.1.16　pg_conversion

pg_conversion 描述编码转换函数。

表 1.4.16: pg_conversion 列

名称	类型	描述
oid	oid	行标识符
connamespace	oid	包含此转换的名字空间的 OID
conowner	oid	转换的拥有者
conforencoding	int4	源编码 ID
contoencoding	int4	目标编码 ID
conproc	regproc	转换函数

-续表-

名称	类型	描述
condefault	bool	如果这是默认转换则为真

1.4.1.17　pg_cursors

pg_cursors 视图列出了当前可用的游标。游标可以以几种方式定义：

- 通过 SQL 中的 DECLARE 语句
- 通过前端/后端协议中的绑定消息
- 通过服务器编程接口（SPI）

pg_cursors 视图显示由任何这些方式创建的游标。视图只存在于定义它们的事务期间，除非声明了 WITH HOLD。因此非保持游标只在它们的创建事务结束前存在于这个视图中。

表 1.4.17: pg_cursors 列

名称	类型	描述
name	text	游标名
statement	text	提交用于定义此游标的查询语句
is_holdable	boolean	如果游标是可保持的（即，它可以在其定义事务提交后被访问）则为 true，否则为否
is_binary	boolean	如果游标被声明为 BINARY 则为 true，否则为 false
is_scrollable	boolean	如果游标是可滚动的（即，允许以一种非顺序的方式检索行）则为 true，否则为 false
creation_time	timestamptz	游标被声明的时间

pg_cursors 视图是只读的。

1.4.1.18　pg_database

视图 pg_database 存储有关可用数据库的信息。数据库通过 CREATE DATABASE 命令创建。

和大部分系统视图不同，pg_database 是在集簇的所有数据库之间共享的：在一个集簇中只有一份 pg_database 拷贝，而不是每个数据库一份。

表 1.4.18: pg_database 列

名称	类型	描述
oid	oid	行标识符
datname	name	数据库名字
datdba	oid	数据库的拥有者，通常是创建它的用户
encoding	int4	此数据库的字符编码的编号（pg_encoding_to_char() 可将此编号转换成编码的名字）
datcollate	name	此数据库的 LC_COLLATE
datctype	name	此数据库的 LC_CTYPE
datistemplate	bool	如果为真，则此数据库可被任何具有 CREATEDB 特权的用户克隆；如果为假，则只有超级用户或者该数据库的属主能够克隆它
datallowconn	bool	如果为假则没有人能连接到这个数据库。这可以用来保护 template0 数据库不被修改
datconnlimit	int4	设置能够连接到这个数据库的最大并发连接数。-1 表示没有限制
datlastsysoid	oid	数据库中最后一个系统 OID，对 pg_dump 特别有用
datfrozenxid	xid	在此之前的所有事务 ID 在数据库中已经被替换为一个永久的（“冻结的”）事务 ID。这用于跟踪数据库是否需要被清理，以便组织事务 ID 回环或者允许 pg_xact 被收缩。它是此数据库中所有表的 pg_class.relfrozenxid 值的最小值
datminmxid	xid	在此之前的所有多事务 ID 在数据库中已经被替换为一个事务 ID。这用于跟踪数据库是否需要被清理，以便组织事务 ID 回环或者允许 pg_multixact 被收缩。它是此数据库中所有表的 pg_class.relminmxid 值的最小值
dattablespace	oid	此数据库的默认表空间。在此数据库中，所有 pg_class.reltablespace 为 0 的表都将被存储在这个表空间中，尤其是非共享系统视图都会在其中
datacl	aclitem[]	访问权限

1.4.1.19　pg_db_role_setting

视图 pg_db_role_setting 为每一个角色和数据库组合记录被设置到运行时配置变量的默认值。

和大部分系统视图不同，pg_db_role_setting 是在集簇的所有数据库之间共享的：在一个集簇中只有一份 pg_db_role_setting 拷贝，而不是每个数据库一份。

表 1.4.19: pg_db_role_setting 列

名称	类型	描述
setdatabase	oid	此设置可用的数据库 OID，如果不与具体数据库相关则为 0
setrole	oid	此设置可用的角色 OID，如果不与具体角色相关则为 0
setconfig	text[]	运行时配置变量的默认值

1.4.1.20　pg_default_acl

视图 pg_default_acl 存储要被分配给新创建对象的初始权限。

表 1.4.20: pg_default_acl 列

名称	类型	描述
oid	oid	行标识符
defaclrole	oid	与此项相关的角色的 OID
defaclnamespace	oid	与此项相关的名字空间的 OID，如果没有则为 0
defaclobjtype	char	此项适合的对象类型： r = 关系（表、视图） S = 序列 f = 函数 T = 类型 n = 方案
defaclacl	aclitem[]	此类对象在创建时应用有的访问权限

1.4.1.21　一个 pg_default_acl 项展示了要分配给属于一个指定用户的对象的初始权限。当前有两类项：defaclnamespace

0 的“全局”项和引用一个特殊模式的“每方案”项。如果一个全局项存在，则它重载该对象类型的普通 hard-wired 默认权限。一个每模式项如果存在，表示权限将被加入到全局或 hard-wired 默认权限中。

注意当在另一个表中的一个 ACL 项为空时，它用来表示其对象的 hard-wired 默认权限，而不是当时可能在 pg_default_acl 中的任何权限。只有在对象创建期间才会查阅 pg_default_acl。

1.4.1.22　pg_depend

视图 pg_depend 记录数据库对象之间的依赖关系。这些信息允许 DROP 命令查找必须被 DROP CASCADE 删除的其他对象，或者在 DROP RESTRICT 情况下阻止删除。

表 1.4.21: pg_depend 列

名称	类型	描述
classid	oid	依赖对象所在的系统视图 OID
objid	oid	指定依赖对象的 OID
objsubid	int4	对于一个表列，这里是列号（objid 和 classid 指表本身）。对于所有其他对象类型，此列为 0
refclassid	oid	被引用对象所在的系统视图的 OID
refobjid	oid	指定被引用对象的 OID
refobjsubid	int4	对于一个表列，这里是列号（refobjid 和 refclassid 指表本身）。对于所有其他对象类型，此列为 0
deptype	char	定义此依赖关系语义的一个代码，见文本

要注意的是，两个对象很有可能由不止一个 pg_depend 条目来链接。例如子分区索引有一个依赖于其关联的分区表的分区类型的依赖关系和依赖于该表索引的每一列的自动依赖关系。此类情形表示多重依赖关系语义的并集，依赖对象的删除可以没有 CASCADE，如果其任一依赖关系满足自动删除的条件。相反地，关于哪些对象必须一起删除的所有依赖关系的限制必须满足。

1.4.1.23　pg_description

视图 pg_description 存储对每一个数据库对象可选的描述（注释）。描述可以通过 COMMENT 操作，并可使用 psql 的 d 命令查看。在 pg_description 的初始内容中提供了很多内建系统对象的描述。

表 1.4.22: pg_description 列

名称	类型	描述
objoid	oid	描述所属对象的 OID
classoid	oid	对象所述的系统视图的 OID
objsubid	int4	对于一个表列上的一个注释，这里是列号（objoid 和 classoid 指表本身）。对所有其他对象类型，此列为 0
description	text	作为该对象描述的任意文本

1.4.1.24　pg_enum

pg_enum 视图包含每一个枚举类型的项，其中包括了值和标签。一个给定枚举值的内部表示实际上是它在 pg_enum 中的相关行的 OID。

表 1.4.23: pg_enum 列

名称	类型	描述
oid	oid	行标识符
enumtypid	oid	包含此枚举值的 pg_type 项的 OID
enumsortorder	float4	此枚举值在其枚举类型中的排序位置
enumlabel	name	此枚举值的文本标签

pg_enum 行的 OID 值遵循一种特殊的规则：即 OID 的数值被保证按照其枚举类型的排序顺序进行排序。即如果两个偶数 OID 属于同一枚举类型，较小的 OID 必然具有较小的 enumsortorder 值。

当一个枚举类型被创建后，其成员会被分配排序位置 1…n。但后面增加的成员可能会被分配负值或者分数值的 enumsortorder。对于这些值的唯一要求是它们必须被正确地排序且和保持唯一。

1.4.1.25　pg_event_trigger

视图 pg_event_trigger 存储事件触发器。

表 1.4.24: pg_event_trigger 列

名称	类型	描述
oid	oid	触发器 oid
evtname	name	触发器名（必须唯一）
evtevent	name	此触发器触发的事件的标识符
evtowner	oid	事件触发器的拥有者
evtfoid	oid	将被调用的函数
evtenabled	char	控制事件触发器触发的 session_replication_role 模式。 O = 触发器在“origin”和“local”模式触发 D = 触发器被禁用 R = 触发器在“replica”模式触发 A = 触发器总是触发
evttags	text[]	此触发器将触发的命令标签。如果为空，此触发器的触发不受命令标签的限制

1.4.1.26　pg_extension

视图 pg_extension 存储有关已安装扩展的信息。

表 1.4.25: pg_extension 列

名称	类型	描述
oid	oid	行标识符
extname	name	扩展的名字
extowner	oid	扩展的拥有者
extnamespace	oid	包含此扩展的导出对象的模式

-续表-

名称	类型	描述
extrelocatable	bool	如果扩展可被重定位到另一个模式则为真
extversion	text	扩展的版本名字
extconfig	oid[]	扩展的配置表的 regclass 项的 OID 数组，如果没有配置表则为 NULL
extcondition	text[]	扩展的配置表的 WHERE 子句过滤条件的数组，如果没有则为 NULL

注意和大部分具有一个“namespace”列的模式不同，extnamespace 不是用来表示扩展属于该模式。扩展的名字从不用模式进行限定。extnamespace 表明该模式包含了该扩展的大部分或全部对象。如果 extrelocatable 为真，则该模式事实上必须包含属于此扩展的全部模式限定的对象。

1.4.1.27　pg_file_settings

视图 pg_file_settings 提供了服务器配置文件内容的概要。这个视图中的每一行表示配置文件中出现的一个“name = value”项，还带有注解指示该值是否被成功地应用。在配置文件有问题时，有可能出现额外的行，它们没有相关的“name = value”项，一个例子是配置文件中有语法错误。

这个视图有助于检查在配置文件中打算做的修改是否能工作，或者用来诊断之前的失败。注意这个视图报告的是配置文件的当前内容，而不是服务器最后应用的值（这些值通常查看 pg_settings 视图就够了）。

默认情况下，pg_file_settings 视图只有超级用户可读。

表 1.4.26: pg_file_settings 列

名称	类型	描述
sourcefile	text	配置文件的完整路径名
sourceline	int4	该项在配置文件中出现的行号
seqno	int4	项被处理的顺序（1..n）
name	text	配置参数名
setting	text	被赋予给参数的值
applied	boolean	为真表示值已被成功应用

-续表-

名称	类型	描述
error	text	如果非空，表示一个错误消息，它说明为什么这个项不能被应用

如果配置文件包含语法错误或者非法参数名，服务器将不会尝试从其中应用任何设置，并且因此所有的 applied 列都为假。在这种情况下，将会有一个或者多个行的 error 列为非空，它们说明了为什么出问题。否则，将尽可能应用每个设置。如果一个设置不能被应用（例如非法值或者该设置不能在服务器开始后改变），会有一个合适的消息存储在它的 error 列中。一个项的 applied 列为假的另一种情况是它被后面一个具有相同参数名的项所覆盖，这种情况不会被认为是一种错误，因此在 error 列列中不会有错误消息。

1.4.1.28　pg_foreign_data_wrapper

视图 pg_foreign_data_wrapper 存储外部数据包装器定义。外部数据包装器是一种访问位于外部服务器上数据的机制。

表 1.4.27: pg_foreign_data_wrapper 列

名称	类型	描述
oid	oid	行标识符
fdwname	name	外部数据包装器的名字
fdwowner	oid	外部数据包装器的拥有者
fdwhandler	oid	指一个负责为外部数据包装器提供执行例程的处理函数。如果没有提供处理函数则为 0
fdwvalidator	oid	指一个负责检查传给外部数据包装器的选项的有效性的验证函数，包括外部服务器选项以及使用外部数据包装器的用户映射。如果没有提供验证函数则为 0
fdwacl	aclitem[]	访问权限
fdwoptions	text[]	外部数据包装器特定选项，以“keyword=value”字符串形式

1.4.1.29　pg_foreign_server

视图 pg_foreign_server 存储外部服务器定义。外部服务器定义了外部数据的来源，例如一个远程服务器。外部服务器通过外部数据包装器来访问。

表 1.4.28: pg_foreign_server 列

名称	类型	描述
oid	oid	行标识符
srvname	name	外部服务器的名字
srvowner	oid	外部服务器的拥有者
srvfdw	oid	此外部服务器的外部数据包装器的 OID
srvtype	text	服务器的类型（可选）
srvversion	text	服务器的版本（可选）
srvacl	aclitem[]	访问权限
srvoptions	text[]	外部服务器特定选项，以“keyword=value”字符串形式

1.4.1.30　pg_foreign_table

视图 pg_foreign_table 包含关于外部表的辅助信息。一个外部表和普通表一样，主要由一个 pg_class 项表示。它的 pg_foreign_table 项包含外部表所特有的信息。

表 1.4.29: pg_foreign_table 列

名称	类型	描述
ftrelid	oid	外部表的 pg_class 项的 OID
ftserver	oid	外部表所在的外部服务器的 OID
ftoptions	text[]	外部表选项，以“keyword=value”字符串形式

1.4.1.31　pg_group

视图 pg_group 为向后兼容而存在：它模拟了存在于 PostgreSQL 8.1 之前版本中的一个系统表。它显式所有角色的名称和未被标记 rolcanlogin 的成员，它是被用做组的角色集合的近似。

表 1.4.30: pg_group 列

名称	类型	描述
groname	name	组名
grosysid	oid	组 ID
grolist	oid[]	包含此组中角色 ID 的一个数组

1.4.1.32　pg_hba_file_rules

视图 pg_hba_file_rules 提供客户端认证配置文件 pg_hba.conf 内容的摘要。该文件中每个非空、非注释行都会在这个视图中出现一行，行中还有标记表示该规则是否被成功地应用。

这个视图可用来检查认证配置文件中按计划的更改是否起作用，或者诊断之前的失败。注意这个视图报告的是该文件的当前内容，而不是服务器最后一次载入的内容。

默认情况下，只有超级用户可以读取 pg_hba_file_rules 视图。

表 1.4.31: pg_hba_file_rules 列

名称	类型	描述
line_number	int4	这条规则在 pg_hba.conf 中的行号
type	text	连接类型
database	text[]	这条规则应用的数据库名列表
user_name	text[]	这条规则应用的用户及组名列表
address	text	主机名或 IP 地址，或者 all、samehost、samenet 之一，对于本地连接为空
netmask	text	IP 地址掩码，如果不适用则为空
auth_method	text	认证方法
options	text[]	为认证方法指定的选项（如果有）

-续表-

名称	类型	描述
error	text	如果非空，则是一个错误消息，它表示为什么这一行无法被处理

通常，反映一个不正确项的行只有 line_number 和 error 域中有值。

1.4.1.33　pg_index

视图 pg_index 包含关于索引的部分信息。其他信息大部分在 pg_class 中。

表 1.4.32: pg_index 列

名称	类型	描述
indexrelid	oid	此索引的 pg_class 项的 OID
indrelid	oid	此索引的基表的 pg_class 项的 OID
indnatts	int2	索引中的总列数（与 pg_class.relnatts 重复），这个数目包括键和被包括的属性
indnkeyatts	int2	索引中键列的编号，不计入任何的内含列，它们只是被存储但不参与索引的语义
indisunique	bool	表示是否为唯一索引
indisprimary	bool	表示索引是否表示表的主键（如果此列为真，indisunique 也总是为真）
indisexclusion	bool	表示索引是否支持一个排他约束
indimmediate	bool	表示唯一性检查是否在插入时立即被执行（如果 indisunique 为假，此列无关）
indisclustered	bool	如果为真，表示表最后以此索引进行了聚簇
indisvalid	bool	如果为真，此索引当前可以用于查询。为假表示此索引可能不完整：它肯定还在被 INSERT/UPDATE 操作所修改，但它不能安全地被用于查询。如果索引是唯一索引，唯一性属性也不能被保证

-续表-

名称	类型	描述
indcheckxmin	bool	如果为真，直到此 pg_index 行的 xmin 低于查询的 TransactionXmin 视界之前，查询都不能使用此索引，因为表可能包含具有它们可见的不相容行的损坏 HOT 链
indisready	bool	如果为真，表示此索引当前可以用于插入。为假表示索引必须被 INSERT/UPDATE 操作忽略
indislive	bool	如果为假，索引正处于被删除过程中，并且必须被所有处理忽略（包括 HOT 安全的决策）
indisreplident	bool	如果为真，这个索引被选择为使用 ALTER TABLE ... REPLICA IDENTITY USINGINDEX ... 的“replica identity”
indkey	int2[]	这是一个 indnatts 值的数组，它表示了此索引索引的表列。例如一个 13 值可能表示表的第一和第三列组成了索引项。键列出现在非键（内含）列前面。数组中的一个 0 表示对应的索引属性是一个在表列上的表达式，而不是一个简单的列引用
indcollation	oid[]	对于索引键（indnkeyatts 值）中的每一列，这包含要用于该索引的排序规则的 OID，如果该列不是一种可排序数据类型则为零
indclass	oid[]	对于索引键中的每一列（indnkeyatts 值），这里包含了要使用的操作符类的 OID。详见 pg_opclass
indoption	int2[]	这是一个 indnkeyatts 值的数组，用于存储每列的标志位。位的意义由索引的访问方法定义
indexprs	pg_node_tree	非简单列引用索引属性的表达式树（以 nodeToString() 形式）。对于 indkey 中每一个为 0 的项，这个列表中都有一个元素。如果所有的索引属性都是简单引用，此列为空
indpred	pg_node_tree	部分索引谓词的表达式树（以 nodeToString() 形式）。如果不是部分索引，此列为空

1.4.1.34　pg_indexes

视图 pg_indexes 提供对于数据库中每一个索引信息的访问。

表 1.4.33: pg_indexes 列

名称	类型	描述
schemaname	name	包含表和索引的模式名
tablename	name	此索引的基表的名字
indexname	name	索引名
tablespace	name	包含索引的表空间名（如果是数据库的默认值则为空）
indexdef	text	索引定义（CREATE INDEX 命令的重构）

1.4.1.35　pg_inherits

视图 pg_inherits 记录有关表继承层次的信息。数据库中每一个直接父子关系在这里都有一项（非直接继承可以通过顺着项构成的链来决定）。

表 1.4.34: pg_inherits 列

名称	类型	描述
inhrelid	oid	孩子表的 OID
inhparent	oid	父表的 OID
inhseqno	int4	如果一个孩子表有多于一个直接父表（多继承），这个数字说明了继承列被排列的顺序。计数从 1 开始

1.4.1.36　pg_init_privs

视图 pg_init_privs 记录系统中对象的初始特权。数据库中每一个具有非默认（非-NULL）初始特权集合的对象都有一个条目在其中。

对象可以在系统初始化（initdb）时获得其初始特权，也可以在 CREATE EXTENSION 期间创建该对象并且在扩展脚本中用 GRANT 来设置对象的初始特权。注意系统将自动处理扩展脚本执行期间对特权的记录，扩展的作者们只需要在他们的脚本中使用 GRANT 以及 REVOKE 语句以便特权被记录下来。privtype 列表示初始特权是被 initdb 设置还是在一次 CREATE EXTENSION 命令期间被设置。

具有被 initdb 设置的初始特权的对象的条目中 privtype 是’i’，而具有被 CREATE EXTENSION 设置的初始特权的对象的条目中 privtype 为’e’。

表 1.4.35: pg_init_privs 列

名称	类型	描述
objoid	oid	指定对象的 OID
classoid	oid	对象所在的系统视图的 OID
objsubid	int4	对于一个表列，这里是列编号（objoid 和 classoid 指向表本身）。对于所有其他对象类型，这列为零
privtype	char	定义这个对象初始特权类型的代码，见文字说明
initprivs	aclitem[]	初始的访问权限

1.4.1.37　pg_language

视图 pg_language 注册了可用于编写函数或存储过程的语言。

表 1.4.36: pg_language 列

名称	类型	描述
oid	oid	行标识符
lanname	name	语言的名字
lanowner	oid	语言的拥有者
lanispl	bool	内部语言为假（如 SQL），用户定义语言为真。当前，pg_dump 仍然使用这个列来决定要转储哪些语言，但在未来这可能会被一种不同的机制所取代

-续表-

名称	类型	描述
lanpltrusted	bool	为真表示这是一种可信的语言，即它被相信不会向普通 SQL 执行环境之外的任何东西授予权限。只有超级用户可以在非可信语言中创建函数
lanplcallfoid	oid	对于非内部语言，此列引用语言处理器，它是一个特殊函数负责执行所有用这种语言编写的函数
laninline	oid	此列引用一个负责执行“内联”匿名代码块的函数（DO 块）。如果不支持内联块则为 0
lanvalidator	oid	此列引用一个负责在函数创建时对其进行语法和可用性检查的语言验证函数。如果没有提供验证器则为 0
lanacl	aclitem[]	访问权限

1.4.1.38　pg_largeobject

视图 pg_largeobject 保存构成“大对象”的数据。

一个大对象在被创建时会被分配一个 OID。每个大对象被分解成段或“页”，以便小到可以被方便地作为行存储在 pg_largeobject 中。每页中的数据量被定义为 LOBLKSIZE（目前是 BLCKSZ/4 或是 2 kB）。

在 PostgreSQL 9.0 之前，大对象没有相关的权限结构。作为结果，pg_largeobject 是公共可读的并且可以用来获得系统中所有大对象的 OID（和内容）。但现在不是这样了，可使用 pg_largeobject_metadata 来获得大对象 OID 的列表。

表 1.4.37: pg_largeobject 列

名称	类型	描述
loid	oid	包含此页的大对象的标识符
pageno	int4	此页在它所属大对象中的页号（从 0 开始计）
data	bytea	实际存储在大对象中的数据

pg_largeobject 的每一行保存一个大对象的一个页的数据，从对象内部的字节偏移量（pageno * LOBLKSIZE）开始。

现在的实现允许稀疏存储：页面可能丢失，并且可能比 LOBLKSIZE 字节短（即便不是最后一页）。一个大对象中丢失的区域会被读出为 0。

1.4.1.39　pg_largeobject_metadata

视图 pg_largeobject_metadata 记录与大对象有关的元数据。真正的大对象数据被存储在 pg_largeobject 中。

表 1.4.38: pg_largeobject_metadata 列

名称	类型	描述
oid	oid	行标识符
lomowner	oid	大对象的拥有者
lomacl	aclitem[]	访问权限

1.4.1.40　pg_locks

视图 pg_locks 提供了数据库服务器上活动进程中保持的锁的信息。

pg_locks 中对每一个活动可锁对象、请求锁模式和相关进程的组合都有一行。因此，如果多个进程持有或者正在等待一个可锁对象上的锁，同一个可锁对象可能出现很多次。但是，一个当前没有被锁的对象根本不会出现。

有多种不同类型的可锁对象：整个关系（如表）、关系的单个页、关系的单个元组、事务 ID（包括虚拟和永久 ID）和普通数据库对象（由类 OID 和对象 OID 标识，和 pg_description 或 pg_depend 中的相同方式）。扩展一个关系的权力也被表示为一个独立的可锁对象。“咨询”锁可以具有用户定义的意义。

表 1.4.39: pg_locks 列

名称	类型	描述
locktype	text	可锁对象的类型： relation，extend，page tuple，transactionid，virtualxid object，userlock 或 advisory
database	oid	锁目标存在的数据库的 oid，如果目标是一个共享对象则为 0，如果目标是一个事务 id 则为空
relation	oid	作为锁目标的关系的 oid，如果目标不是一个关系或者只是关系的一部分则此列为空

-续表-

名称	类型	描述
page	int4	作为锁目标的页在关系中的页号，如果目标不是一个关系页或元组则此列为空
tuple	int2	作为锁目标的元组在页中的元组号，如果目标不是一个元组则此列为空
virtualxid	text	作为锁目标的事务虚拟 id，如果目标不是一个虚拟事务 id 则此列为空
transactionid	xid	作为锁目标的事务 id，如果目标不是一个事务 id 则此列为空 id
classid	oid	包含锁目标的系统表的 oid，如果目标不是一个普通数据库对象则此列为空
objid	oid	锁目标在它的系统表中的 oid，如果目标不是一个普通数据库对象则为空
objsubid	int2	锁的目标列号（classid 和 objid 指表本身），如果目标是某种其他普通数据库对象则此列为 0，如果目标不是一个普通数据库对象则此列为空
virtualtransaction	text	保持这个锁或者正在等待这个锁的事务的虚拟 id
pid	int4	保持这个锁或者正在等待这个锁的服务器进程的 pid，如果此锁被一个预备事务所持有则此列为空
mode	text	此进程已持有或者希望持有的锁模式
granted	boolean	如果锁已授予则为真，如果锁被等待则为假
fastpath	boolean	如果锁通过快速路径获得则为真，通过主锁表获得则为假

一个行的 granted 为真表示一个被指定进程持有的锁。为假表示该进程当前正在等待获取这个锁，这意味着至少一个其他进程正持有或等待同一个可锁对象上的一个冲突锁。该等待进程将一直休眠直到其他锁被释放（或者一个死锁状态被检测到）。单个进程在同一时间只能等待最多一个锁。

贯穿一个事务的运行，一个服务器进程在其生存周期内都持有一个在其虚拟事务 ID 上的排他锁。如果一个永久 ID 被分配给事务（通常发生在事务改变数据库状态时），它也会持有一个在其永久事务 ID 上的排他锁直到它结束。当一个事务发现它需要等待另一个事务，它也会尝试获取其他事务 ID 上的共享锁（不管是虚拟还是永久 ID，视情况而定）。这只有当其他进程终止并释放其锁后才会成功。

尽管元组是一种可锁对象，关于行级锁的信息被存储在磁盘而不是内存中，因此行级锁通常不在这个视图中出

现。如果一个进程正在等待一个行级锁，它通常在这个视图中出现，并且表示形式为正在等待已持有该行级锁的永久事务 ID 上的锁。

咨询锁可以在由一个单一 int8 值或两个整形值构成的键上获取。一个 int8 键被显示为其高位部分在 classid 列中，低位部分在 objid 列中，并且 objsubid 等于 1。原来的 int8 值可以使用表达式 (classid::int8 « 32) | objid::int8 重组。整形键被显示为第一个键在 classid 列中，第二个键在 objid 列中，并且 objsubid 等于 2。键的实际意义由用户决定。咨询锁是每一个数据库的本地锁，所以 database 列对于一个咨询锁没有意义。

pg_locks 提供了一个对于整个数据集簇中所有锁的全局视图，而不仅仅是与当前数据库相关的锁。尽管它的 relation 列可以被连接到 pg_class.oid 来标识被锁关系，但这种方法只有在关系属于当前数据库（database 列是当前数据库 OID 或者 0 的锁对应的关系）的情况下才会得到正确的结果。

pid 列可以被连接到 pg_stat_activity 视图的 pid 列来得到持有或等待持有每一个锁的会话的信息。例如

```
SELECT * FROM pg_locks pl LEFT JOIN pg_stat_activity psa
ON pl.pid = psa.pid;
```

另外，如果正在使用预备事务，virtualtransaction 列可以被连接到 pg_prepared_xacts 视图的 transaction 列来得到持有该锁的预备事务的信息（一个预备事务不可能正在等待一个锁，但它在运行中会一直持有已获得的锁）。例如：

```
SELECT * FROM pg_locks pl LEFT JOIN pg_prepared_xacts ppx
ON pl.virtualtransaction = '-1/' || ppx.transaction;
```

虽然通过自连接 pg_locks 可以获得哪些进程阻塞了其他哪些进程的信息，但是很难得到其中的细节。这样一个查询隐藏了关于哪些锁模式与其他哪些锁模式冲突的知识。更糟糕的是，pg_locks 视图无法给出所等待队列中进程的等待顺序，也无法显示哪些进程是代表其他客户端会话运行的并行工作者。更好的方法是使用 pg_blocking_pids() 函数来标识一个等待进程是被哪些进程阻塞的。

1.4.1.41 pg_locks 视图显示来自于普通锁管理器和谓词锁管理器的数据，它们是独立的系统。此外，普通锁管理器把它的锁分为普通锁和快速路径锁。这些数据并不被保证是完全一致的。当视图被查询时，快速路径锁上的数据（fastpath

true）会被一次性从每一个后端收集起来，且并不冻结整个锁管理器的状态。因此有可能某些锁在上述信息被收集的过程中被获得或者释放。注意，不管怎样这些锁是已知不会和任何当前正在发生的锁冲突。在所有后端已经查询了快速路径锁后，普通锁管理器的剩余部分被作为一个单元锁住，并且所有剩余锁的一个一致快照被作为一个原子动作收集。在解锁普通锁管理器后，谓词锁管理器也被类似地锁住并且所有谓词锁被作为一个原子动作收集。因此，在快速路径锁这种特殊情况下，每一个锁管理器会传递一个一致的结果组。但由于我们并不会同时锁上两个锁管理器，在我们询问完普通锁管理器后或者询问谓词锁管理器之前，锁可以被获得或者释放。

如果对此视图频繁地访问，对普通或者谓词锁管理器加锁可能会对数据库性能产生一定影响。虽然这些锁只会在最少的时间内被保持（足以从锁管理器获得数据），但这无法完全消除可能产生的性能影响。

1.4.1.42　pg_matviews

视图 pg_matviews 提供了关于数据库中每一个物化视图的信息。

表 1.4.40: pg_matviews 列

名称	类型	描述
schemaname	name	包含物化视图的模式的名字
matviewname	name	物化视图的名字
matviewowner	name	物化视图拥有者的名字
tablespace	name	包含物化视图的表空间名（如使用数据库默认表空间则为空）
hasindexes	boolean	如果物化视图有（或者最近有过）任何索引，则此列为真
ispopulated	boolean	如果物化视图当前已被填充，则此列为真
definition	text	物化视图的定义（一个重构的 SELECT 查询）

1.4.1.43　pg_namespace

视图 pg_namespace 存储名称空间的元信息。名字空间是 SQL 模式之下的结构：每个名字空间拥有一个独立的表、类型等的集合，且其中没有名字冲突。

表 1.4.41: pg_namespace 列

名称	类型	描述
oid	oid	行标识符
nspname	name	名字空间的名字
nspowner	oid	名字空间的拥有者
nspacl	aclitem[]	访问权限

1.4.1.44　pg_opclass

视图 pg_opclass 定义索引访问方法的操作符类。每一个操作符类定义了一种特定数据类型和一种特定索引访问方法的索引列的语义。一个操作符类实际上指定了一个特定的操作符族可以用于一个特定可索引列数据类型。该族中可用于索引列的操作符能够接受该列的数据类型作为它们的左输入。

表 1.4.42: pg_opclass 列

名称	类型	描述
oid	oid	行标识符
opcmethod	oid	操作符类所属的索引访问方法
opcname	name	操作符类的名称
opcnamespace	oid	操作符类所属的名字空间
opcowner	oid	操作符类的拥有者
opcfamily	oid	包含此操作符类的操作符族
opcintype	oid	操作符类索引的数据类型
opcdefault	bool	如果此操作符类为 opcintype 的默认值则为真
opckeytype	oid	存储在索引中的数据的类型，如果值为 0 表示与 opcintype 相同

一个操作符类的 opcmethod 必须匹配包含它的操作符族的 opfmethod。而且，对于任何给定的 opcmethod 和 opcintype 组合，只有不超过一个 pg_opclass 行的 opcdefault 值为真。

1.4.1.45　pg_operator

视图 pg_operator 存储关于操作符的信息。

表 1.4.43: pg_operator 列

名称	类型	描述
oid	oid	行标识符
oprname	name	操作符的名称
oprnamespace	oid	操作符所属的名字空间的 OID

-续表-

名称	类型	描述
oprowner	oid	操作符的拥有者
oprkind	char	b = 中缀（“并”） l = 前缀（“左”） r = 后缀（“右”）
oprcanmerge	bool	该操作符是否支持归并连接
oprcanhash	bool	该操作符是否支持哈希连接
oprleft	oid	左操作数类型
oprright	oid	右操作数类型
oprresult	oid	结果类型
oprcom	oid	该操作符的交换子（如果存在）
oprnegate	oid	该操作符的否定（如果存在）
oprcode	regproc	实现该操作符的函数
oprrest	regproc	该操作符的限制选择度估算函数
oprjoin	regproc	该操作符的连接选择度估算函数

未用的列包含零值。例如，一个前缀操作符的 oprleft 为 0。

1.4.1.46　pg_opfamily

视图 pg_opfamily 定义了操作符族。每一个操作符族是操作符和相关支持例程的集合，支持例程用于实现一个特定索引访问方法的语义。此外，按照访问方法指定的某种方式，一个族内的操作符都是“兼容的”。操作符族概念允许在索引中使用跨数据类型操作符，并可以使用访问方法语义的知识推导出。

表 1.4.44: pg_opfamily 列

名称	类型	描述
oid	oid	行标识符
opfmethod	oid	操作符族适用的索引访问方法

-续表-

名称	类型	描述
opfname	name	操作符族的名字
opfnamespace	oid	操作符族所属的名字空间
opfowner	oid	操作符族的拥有者

定义操作符族的主要信息不在它的 pg_opfamily 行，而是在相关的 pg_amop、pg_amproc 和 pg_opclass 行中。

1.4.1.47　pg_partitioned_table

视图 pg_partitioned_table 存放有关表如何被分区的信息。

表 1.4.45: pg_partitioned_table 列

名称	类型	描述
partrelid	oid	这个分区表的 pg_class 项的 OID
partstrat	char	分区策略 h = 哈希分区表 l = 列表分区表 r = 范围分区表
partnatts	int2	分区键中的列数
partdefid	oid	这个分区表的默认分区的 pg_class 项的 OID，如果这个分区表没有默认分区则为零
partattrs	int2[]	这是一个长度为 partnatts 值的数组，它指示哪些表列是分区键的组成部分。例如，值 13 表示第一个和第三个表列组成了分区键。这个数组中的零表示对应的分区键列是一个表达式而不是简单的列引用
partclass	oid[]	对于分区键中的每一个列，这个域包含要使用的操作符类的 OID。详见 pg_opclass
partcollation	oid[]	对于分区键中的每一个列，这个域包含要用于分区的排序规则的 OID，如果该列不是一种可排序数据类型则为零

-续表-

名称	类型	描述
partexprs	pg_node_tree	非简单列引用的分区键列的表达式树（以 nodeToString() 的表达方式）。这是一个列表，partattrs 中每一个零项都有一个元素。如果所有分区键列都是简单列引用，则这个列为空

1.4.1.48　pg_pltemplate

视图 pg_pltemplate 存储了过程语言的“模板”信息。一个语言的模板允许我们在一个特定数据库中以一个简单的 CREATE LANGUAGE 命令创建语言，而不需要指定实现细节。

和大部分系统视图不同，pg_pltemplate 是在集簇的所有数据库之间共享的：在一个集簇中只有一份 pg_pltemplate 拷贝，而不是每个数据库一份。这使得在每个需要的数据库中都可以访问该信息。

表 1.4.46: pg_pltemplate 列

名称	类型	描述
tmplname	name	该模板适用的语言名字
tmpltrusted	boolean	如果语言被认为是可信的则为真
tmpldbacreate	boolean	如果语言可以被一个数据库拥有者创建则为真
tmplhandler	text	调用处理函数的名字
tmplinline	text	匿名阻塞处理函数的名字，如果没有则为空
tmplvalidator	text	验证函数的名字，如果没有则为空
tmpllibrary	text	实现语言的共享库的路径
tmplacl	aclitem[]	模板的访问权限（并未真正使用）

目前任何命令都不能操纵过程语言模板。要改变内建信息，超级用户必须使用普通的 INSERT、DELETE 或 UPDATE 命令修改该表。

1.4.1.49　pg_policies

视图 pg_policies 提供了有关数据库中行级安全性策略的信息。

表 1.4.47: pg_policies 列

名称	类型	描述
schemaname	name	包含策略所在表的模式的名称
tablename	name	策略所在表的名称
policyname	name	策略名称
permissive	text	策略是宽容性的还是限制性的
roles	name[]	这个策略适用的角色
cmd	text	这个策略适用的命令类型
qual	text	作为这个策略适用的查询的安全屏障条件增加的表达式
with_check	text	作为尝试向该表增加行的查询的 WITH CHECK 条件增加的表达式

1.4.1.50　pg_policy

视图 pg_policy 存储着表的行级安全性策略。一个策略包括它适用于的命令种类（可能适用于所有命令）、它适用于的角色、被作为安全屏障条件增加到包括该表的查询的表达式以及被作为 WITH CHECK 选项增加到尝试向表增加新纪录的查询的表达式。

表 1.4.48: pg_policy 列

名称	类型	描述
polname	name	策略的名称
polrelid	oid	策略适用的表

-续表-

名称	类型	描述
polcmd	char	策略适用的命令类型： r 表示 SELECT a 表示 INSERT w 表示 UPDATE d 表示 DELETE * 表示所有命令类型
polpermissive	boolean	策略是宽容性的还是限制性的？
polroles	oid[]	策略适用的角色
polqual	pg_node_tree	被作为安全屏障条件增加到使用该表的查询的表达式树
polwithcheck	pg_node_tree	被作为 WITH CHECK 条件增加到尝试向表增加行的查询的表达式树

1.4.1.51　pg_prepared_statements

pg_prepared_statements 视图显示在当前会话中可用的所有预备语句。

pg_prepared_statements 为每一个预备语句包含一行。当一个新的预备语句被创建时在此视图中会增加一行，反之当一个预备语句被释放时在此视图中会删除一行（例如，通过 DEALLOCATE 命令）。

表 1.4.49: pg_prepared_statements 列

名称	类型	描述
name	text	预备语句的标识符
statement	text	客户端提交用于创建此预备语句的查询语句。对于通过 SQL 创建的预备语句，这里是由客户端提交的 PREPARE 语句。对于通过前端/后端协议创建的预备语句，这里是预备语句本身的文本
prepare_time	timestamptz	预备语句被创建的时间
parameter_types	regtype[]	预备语句期望的参数类型，以一个 regtype 数组的形式。这个数组中一个元素所对应的 OID 可通过将 regtype 值转换为 oid 获得

-续表-

名称	类型	描述
from_sql	boolean	如果预备语句通过 SQL 命令 PREPARE 创建，则为 true；如果预备语句通过前端/后端协议创建，则为 false

pg_prepared_statements 视图为只读。

1.4.1.52　pg_prepared_xacts

视图 pg_prepared_xacts 显示关于两阶段提交的当前准备好事务的信息。

pg_prepared_xacts 为每一个预备事务包含一行。当事务被提交或回滚时，相应的项将被移除。

表 1.4.50: pg_prepared_xacts 列

名称	类型	描述
transaction	xid	预备事务的数字事务标识符
gid	text	分配给事务的全局标识符
prepared	timestamp with time zone	此事务为提交准备好的时间
owner	name	执行此事务的用户名
database	name	执行此事务所在数据库的名字

当 pg_prepared_xacts 视图被访问时，内部事务管理器数据结构被暂时地锁住，并为视图的显示产生一个副本。这确保了视图中是一组一致的结果，并且不会阻塞普通操作。不管怎样，当此视图被频繁访问时，会对数据库性能有所影响。

1.4.1.53　pg_proc

视图 pg_proc 存放有关函数、过程、聚集函数以及窗口函数（共称为例程）的信息。

如果 proisagg 为真，则该项是一个聚集函数，在 pg_aggregate 中应该有一个相匹配的行。

表 1.4.51: pg_proc 列

名称	类型	描述
oid	oid	行标识符
proname	name	函数的名字
pronamespace	oid	函数所属的名字空间的 OID
proowner	oid	函数的拥有者
prolang	oid	实现语言或该函数的调用接口
procost	float4	估计的执行代价（以 cpu_operator_cost 为单位），如果 proretset 为真，这是每行返回的代价
prorows	float4	估计的结果行数量（如果 proretset 为假，该值为 0）
provariadic	oid	可变数组参数的元素的数据类型，如果函数没有可变参数则为 0
prosupport	regproc	对于该函数可选的计划器支持函数
prokind	char	f 表示普通函数 p 表示过程 a 表示聚集函数 w 表示窗口函数
prosecdef	bool	函数是一个安全性定义者（即，一个“setuid”函数）
proleakproof	bool	该函数没有副作用。除了通过返回值，没有关于参数的信息被传播。任何会抛出基于其参数值的错误信息的函数都不是泄露验证的
proisstrict	bool	当任意调用函数为空时，函数是否会返回空值。在那种情况下函数实际上根本不会被调用。非“strict”函数必须准备好处理空值输入
proretset	bool	函数是否返回一个集合（即，指定数据类型的多个值）

-续表-

名称	类型	描述
provolatile	char	provolatile 说明函数是仅仅只依赖于它的输入参数，还是会被外部因素影响。 值 i 表示“不变的”函数，它对于相同的输入总是输出相同的结果。 值 s 表示“稳定的”函数，它的结果（对于固定输入）在一次扫描内不会变化。 值 v 表示“不稳定的”函数，它的结果在任何时候都可能变化（使用 v 页表示函数具有副作用，所以对它们的调用无法得到优化）
proparallel	char	proparallel 说明该函数在并行模式下是否能安全地运行。 对于能在并行模式下不受限制安全运行的函数，这列是 s。 对于可以在并行模式下运行但是只限于由并行分组的领导者执行的函数，这列是 r。 对于在并行模式中不安全的函数，这列是 u，这种函数的存在会强制一个顺序执行计划。
pronargs	int2	输入参数的个数
pronargdefaults	int2	具有默认值的参数个数
prorettype	oid	返回值的数据类型
proargtypes	oid[]	一个函数参数的数据类型的数组。这只包括输入参数（含 INOUT 和 VARIADIC 参数），因此也表现了函数的调用特征
proallargtypes	oid[]	一个函数参数的数据类型的数组。这包括所有参数（含 OUT 和 INOUT 参数）。但是，如果所有参数都是 IN 参数，这个列将为空。注意下标是从 1 开始，然而由于历史原因 proargtypes 的下标是从 0 开始

-续表-

名称	类型	描述
proargmodes	char[]	一个函数参数的模式的数组。编码为： i 表示 IN 参数 o 表示 OUT 参数 b 表示 INOUT 参数 v 表示 VARIADIC 参数 t 表示 TABLE 参数。 如果所有的参数都是 IN 参数，这个列为空。 注意这里的下标对应着 proallargtypes 而不是 proargtypes 中的位置
proargnames	text[]	一个函数参数的名字的数组。没有名字的参数在数组中设置为空字符串。如果没有一个参数有名字，这个列为空。注意这里的下标对应着 proallargtypes 而不是 proargtypes 中的位置
proargdefaults	pg_node_tree	默认值的表达式树（按照 nodeToString() 的表现方式）。这是一个 pronargdefaults 元素的列表，对应于最后 N 个 input 参数（即最后 N 个 proargtypes 位置）。如果没有一个参数具有默认值，这个列为空
protrftypes	oid[]	要在其上应用转换的数据类型的 OID
prosrc	text	这个列告诉函数处理者如何调用该函数。它可能是针对解释型语言的真实源码、一个符号链接、一个文件名或任何其他东西，这取决于实现语言/调用规范
probin	text	关于如何调用函数的附加信息。其解释是与语言相关的
proconfig	text[]	函数对于运行时配置变量的本地设置值
proacl	aclitem[]	访问权限

对于编译好的函数，包括内建的和动态载入的，prosrc 包含了函数的 C 语言名字（链接符号）。对于所有其他已知的语言类型，prosrc 包含函数的源码文本。除了对于动态载入的 C 函数之外，probin 是不被使用的。对于动态载入的 C 函数，它给定了包含该函数的共享库文件的名称。

1.4.1.54　pg_publication

视图 pg_publication 包含数据库中创建的所有 publication。

表 1.4.52: pg_publication 列

名称	类型	描述
oid	oid	行标识符
pubname	name	publication 的名称
pubowner	oid	publication 的拥有者
puballtables	bool	如果为真，这个 publication 自动包括数据库中的所有表，包括未来将会创建的任何表
pubinsert	bool	如果为真，为 publication 中的表复制 INSERT 操作
pubupdate	bool	如果为真，为 publication 中的表复制 UPDATE 操作
pubdelete	bool	如果为真，为 publication 中的表复制 DELETE 操作
pubtruncate	bool	如果为真，为 publication 中的表复制 TRUNCATE 操作

1.4.1.55　pg_publication_rel

视图 pg_publication_rel 包含数据库中关系和 publication 之间的映射。

表 1.4.53: pg_publication_rel 列

名称	类型	描述
prpubid	oid	对 publication 的引用
prrelid	oid	对关系的引用

1.4.1.56　pg_publication_tables

视图 pg_publication_tables 提供 publication 与其所包含的表之间的映射信息。和底层的视图 pg_publication_rel 不同，这个视图展开了定义为 FOR ALL TABLES 的 publication，这样对这类 publication 来说，每一个合格的表都有一行。

表 1.4.54: pg_publication_tables 列

名称	类型	描述
pubname	name	publication 名称
schemaname	name	包含表的方案名称
tablename	name	表名

1.4.1.57　pg_range

视图 pg_range 存储关于范围类型的信息。它是类型在 pg_type 中项的补充。

表 1.4.55: pg_range 列

名称	类型	描述
rngtypid	oid	范围类型的 OID
rngsubtype	oid	该范围类型的元素类型（子类型）的 OID
rngcollation	oid	用于范围比较的排序规则的 OID，如果没有则为 0
rngsubopc	oid	用于范围比较的子类型的操作符类的 OID
rngcanonical	regproc	将一个范围值转换为规范形式的函数的 OID，如果没有则为 0
rngsubdiff	regproc	以双精度返回两个元素值不同的函数的 OID，如果没有则为 0

rngsubopc （加上 rngcollation，如果元素类型是可排序的）决定了被该范围类型所使用的排序顺序。rngcanonical 用于离散类型的元素类型。rngsubdiff 是可选的，但是提供它可以提高范围类型上的 GiST 索引性能。

1.4.1.58　pg_replication_origin

pg_replication_origin 视图包含所有已创建的复制源。

和大部分系统视图不同，pg_replication_origin 在一个集簇的所有数据库之间共享：每个集簇只有一份 pg_replication_origin 拷贝，而不是每个数据库一份。

表 1.4.56: pg_replication_origin 列

名称	类型	描述
roident	oid	一个集簇范围内唯一的复制源标识符
roname	text	外部的由用户定义的复制源名称

1.4.1.59　pg_replication_origin_status

pg_replication_origin_status 视图包含复制源的状态。

表 1.4.57: pg_replication_origin_status 列

名称	类型	描述
local_id	oid	内部的节点标识符
external_id	text	外部的节点标识符
remote_lsn	pg_lsn	源节点的 LSN，到这个位置的数据都已经被复制
local_lsn	pg_lsn	这个节点的 LSN，remote_lsn 已经被复制到这里。使用异步提交时，在将数据持久化到磁盘前用它来写入提交记录

1.4.1.60　pg_replication_slots

pg_replication_slots 视图提供了当前存在于数据库集簇上的所有复制槽的列表，其中也包括复制槽的当前状态。

表 1.4.58: pg_replication_slots 列

名称	类型	描述
slot_name	name	一个唯一的、集簇范围内的复制槽标识符
plugin	name	包含这个逻辑槽正在使用的输出插件的共享对象基础名称，这个列对于物理槽为空值
slot_type	text	槽类型 - physical（物理）或者 logical（逻辑）
datoid	oid	与这个槽相关的数据库的 OID，或者为空值。只有逻辑槽具有相关的数据库
database	name	与这个槽相关的数据库的名称，或者为空值。只有逻辑槽具有相关的数据库
temporary	boolean	如果这是一个临时复制槽则为真。临时槽不会被保存在磁盘上并且会在出错或会话结束时自动被删除掉
active	boolean	如果这个槽当前正在被使用则为真
active_pid	int4	如果槽当前正在被使用，则记录使用这个槽的会话的进程 ID。如果槽没有被使用则为 NULL
xmin	xid	这个槽要需要数据库保留的最旧事务。VACUUM 不能移除被其后续事务删除的元组
catalog_xmin	xid	这个槽要需要数据库保留的影响系统表的最旧事务。VACUUM 不能移除被其后续事务删除的表元组
restart_lsn	pg_lsn	可能仍被这个槽的消费者要求的最旧 WAL 地址（LSN），并且因此不会在检查点期间自动被移除。如果这个槽的 LSN 从未被保留过，则为 NULL
confirmed_flush_lsn	pg_lsn	代表逻辑槽的消费者已经确认接收数据到什么位置的地址（LSN）。比这个地址更旧的数据已经不再可用。对于物理槽这里是 NULL

1.4.1.61　pg_rewrite

视图 pg_rewrite 存储对于表和视图的重写规则。

表 1.4.59: pg_rewrite 列

名称	类型	描述
oid	oid	行标识符
rulename	name	规则名称
ev_class	oid	使用该规则的表
ev_type	char	使用该规则的事件类型： 1 = SELECT 2 = UPDATE 3 = INSERT 4 = DELETE
ev_enabled	char	控制在哪种 session_replication_role 模式中触发该规则。 O = 规则在“origin”和“local”模式触发 D = 规则被禁用 R = 规则在“replica”模式触发 A = 规则总是被触发
is_instead	bool	为真表示是一个 INSTEAD 规则
ev_qual	pg_node_tree	规则条件的表达式树（按照 nodeToString() 的表现形式）
ev_action	pg_node_tree	规则动作的查询树（按照 nodeToString() 的表现形式）

1.4.1.62　pg_roles

视图 pg_roles 提供了关于数据库角色的信息。这是 pg_authid 的一个公共可读视图，它隐去了密码域。

表 1.4.60: pg_roles 列

名称	类型	描述
rolname	name	角色名
rolsuper	bool	角色是否具有超级用户权限

-续表-

名称	类型	描述
rolinherit	bool	如果此角色是另一个角色的成员，角色是否能自动继承另一个角色的权限
rolcreaterole	bool	角色能否创建更多角色
rolcreatedb	bool	角色能否创建数据库
rolcanlogin	bool	角色是否能登录？即此角色能否被作为初始会话授权标识符
rolreplication	bool	角色是一个复制角色。复制角色可以开启复制连接并且创建和删除复制槽
rolconnlimit	int4	对于一个可登录的角色，这里设置角色可以发起的最大并发连接数。-1 表示无限制
rolpassword	text	不是密码（看起来是 ****）
rolvaliduntil	timestamptz	密码失效时间（只用于密码认证），如果永不失效则为空
rolbypassrls	bool	绕过每一条行级安全性策略的角色
rolconfig	text[]	运行时配置变量的角色特定默认值
oid	oid	角色的 ID

1.4.1.63　pg_rules

视图 pg_rules 提供对查询重写规则的信息访问。

表 1.4.61: pg_rules 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	规则适用的表名
rulename	name	规则名
definition	text	规则定义（创建命令的重构）

pg_rules 视图排除了视图和物化视图的 ON SELECT 规则，它们可以在 pg_views 和 pg_matviews 中找到。

1.4.1.64　pg_seclabel

视图 pg_seclabel 存储数据库对象上的安全标签。安全标签可以通过 SECURITY LABEL 命令操纵。

同时请见 pg_shseclabel，它对集簇共享的数据库对象的安全标签执行相似的功能。

表 1.4.62: pg_seclabel 列

名称	类型	描述
objoid	oid	该安全标签依附的对象的 OID
classoid	oid	该对象所出现的系统视图的 OID
objsubid	int4	对于一个在表列上的安全标签，这将是列号（objoid 和 classoid 指表本身）。对于所有其他对象类型，本列为 0
provider	text	与该标签相关的标签提供者
label	text	应用于该对象的安全标签

1.4.1.65　pg_seclabels

视图 pg_seclabels 提供对安全标签的信息访问。它是 pg_seclabel 视图的一个便于查询的版本。

表 1.4.63: pg_seclabels 列

名称	类型	描述
objoid	oid	安全标签所属对象的 OID
classoid	oid	对象出现的系统视图的 OID
objsubid	int4	对于一个表列上的安全标签，这里是列号（objoid 和 classoid 指表本身）。对于所有其他对象类型，此列为 0
objtype	text	此标签应用的对象类型，以文本方式

-续表-

名称	类型	描述
objnamespace	oid	如果适用，为此对象的名字空间的 OID；否则为空
objname	text	此标签应用的对象名，以文本形式
provider	text	与此标签相关的标签提供者
label	text	应用于此对象的安全标签

1.4.1.66　pg_sequence

视图 pg_sequence 包含有关序列的信息。一些序列的信息（例如名称和方案）放在 pg_class 中。

表 1.4.64: pg_sequence 列

名称	类型	描述
seqrelid	oid	这个序列的 pg_class 项的 OID
seqtypid	oid	序列的数据类型
seqstart	int8	序列的起始值
seqincrement	int8	序列的增量值
seqmax	int8	序列的最大值
seqmin	int8	序列的最小值
seqcache	int8	序列的缓冲尺寸
seqcycle	bool	序列是否循环

1.4.1.67　pg_sequences

视图 pg_sequences 提供对数据库中每个序列的信息的访问。

表 1.4.65: pg_sequences 列

名称	类型	描述
schemaname	name	包含序列的方案名
sequencename	name	序列的名称
sequenceowner	name	序列的拥有者的名称
data_type	regtype	序列的数据类型
start_value	int8	序列的起始值
min_value	int8	序列的最小值
max_value	int8	序列的最大值
increment_by	int8	序列的增量值
cycle	boolean	序列是否循环
cache_size	int8	序列的缓冲尺寸
last_value	numeric	最后一个被写入到磁盘的序列值。如果使用了缓冲，这个值可能比从序列中取出的最后一个值大。如果还没有从该序列读取过，则为空。此外，如果当前用户没有该序列上的 USAGE 或 SELECT 特权，则这个值为空

1.4.1.68　pg_settings

视图 pg_settings 提供了对服务器上运行时参数的访问。它本质上是 SHOW 和 SET 命令的可替换接口。它还提供了 SHOW 不能提供的关于每一个参数的一些现实，例如最大值和最小值。

表 1.4.66: pg_settings 列

名称	类型	描述
name	text	运行时配置参数名
setting	text	参数的当前值
unit	text	参数的隐式单元

-续表-

名称	类型	描述
category	text	参数的逻辑组
short_desc	text	参数的简短描述
extra_desc	text	附加的参数的详细描述
context	text	要求设置此参数值的上下文
vartype	text	参数类型（bool、enum、int4、real 或 string）
source	text	当前参数值的来源
min_val	text	参数的最小允许值（对非数字值为空）
max_val	text	参数的最大允许值（对非数字值为空）
enumvals	text[]	一个枚举参数的允许值（对非数字值为空）
boot_val	text	如果参数没有被别的其他设置，此列为在服务器启动时设定的参数值
reset_val	text	在当前会话中，RESET 将会设置的参数值
sourcefile	text	当前值被设置的配置文件（空值表示从非配置文件的其他来源设置，由不是超级用户也不是 pg_read_all_settings 成员的用户检查时也为空值），在配置文件中使用 include 指令时有用
sourceline	int4	当前值被设置的配置文件中的行号（空值表示从非配置文件的其他来源设置，由不是超级用户也不是 pg_read_all_settings 成员的用户检查时也为空值）
pending_restart	boolean	如果配置文件中修改了该值但需要重启，则为 true，否则为 false

对于 context 有多种可能的取值。为了降低改变设置的难度，它们是：

- internal
  这些设置不能被直接修改，它们反映了内部决定的值。某些可能在使用不同配置选项重建系统时或者改变 initdb 的选项时可以调整。

- postmaster
  这些设置只能在服务器启动时应用，因此任何修改都需要重启服务器。这些设置的值通常都存储在 kingbase.conf 文件中，或者在启动服务器时通过命令行传递。当然，具有更低 context 类型的设置也可以在服务器

启动时间被设置。

- sighup
  对于这些设置的修改可以在 kingbase.conf 中完成并且不需要重启服务器。发送一个 SIGHUP 信号给 postmaster 会导致它重新读取 kingbase.conf 并应用修改。Postmaster 将会把 SIGHUP 信号传递给它的孩子进程，这样它们也会获得新的值。

- superuser-backend
  对于这些设置的更改可以在 kingbase.conf 中进行而无需重启服务器。也可以在连接请求包（例如通过 libpq 的 PGOPTIONS 环境变量）中为一个特定的会话设定它们，但是只有在连接用户是超级用户时才能这样做。如果，在会话启动后这些设置就不会改变。如果你在 kingbase.conf 改变了它们，向 postmaster 发送一个 SIGHUP 信号让 postmaster 重新读取 kingbase.conf。新的值将只会影响后续启动的会话。

- backend
  对于这些设置的修改可以在 kingbase.conf 中完成并且不需要重启服务器。它们也可以在一个连接请求包（例如，通过 libpq 的 PGOPTIONS 环境变量）中为一个特定会话设置，任何用户都可以为这个会话做这种修改。然而，这些设置在会话启动后永不变化。如果你在 kingbase.conf 中修改它们，可以向 postmaster 发送一个 SIGHUP 信号让它重读 kingbase.conf。新值只会影响后续启动的会话。

- superuser
  这些设置可以从 kingbase.conf 设置，或者在会话中用 SET 命令设置。仅当没有通过 SET 设置会话本地值时，kingbase.conf 中的改变才会影响现有的会话。

- user
  这些设置可以从 kingbase.conf 设置，或者在会话中用 SET 命令设置。任何用户都被允许修改它们的会话本地值。仅当没有通过 SET 设置会话本地值时，kingbase.conf 中的改变才会影响现有的会话。

pg_settings 视图不能被插入或者从中删除，但是它可以被更新。在 pg_settings 的一行上的一个 UPDATE 等价于在该参数上执行一个 SET 命令。修改将只会影响当前会话使用的值。如果一个 UPDATE 在一个后来中断的事务中被发出，UPDATE 命令的效果也会随着事务的回滚而消失。一旦所在的事务被提交，效果将一直保持到会话结束，除非有其他 UPDATE 或 SET 重新修改它。

1.4.1.69　pg_shadow

视图 pg_shadow 的存在是为了向后兼容：它模拟了在 PostgreSQL 版本 8.1 之前的一个系统目录。它显示 pg_authid 中所有被标记为 rolcanlogin 的角色的属性。

由于这个表包含密码，所以不能是公众可读的，这也是采用 pg_shadow 这个名字的原因。而 pg_user 是 pg_shadow 上的一个公共可读视图，它屏蔽了密码列。

表 1.4.67: pg_shadow 列

名称	类型	描述
usename	name	用户名

-续表-

名称	类型	描述
usesysid	oid	用户的 ID
usecreatedb	bool	用户能否创建数据库
usesuper	bool	用户是否为一个超级用户
userepl	bool	用户能否开启流复制并将系统设置或者取消备份模式
usebypassrls	bool	用户能否绕过所有的行级安全性策略
passwd	text	密码（可能被加密），如果没有则为空。关于加密密码如何存储请参见 pg_authid
valuntil	timestamptz	密码过期时间（仅用于密码认证）
useconfig	text[]	运行时配置变量的会话默认值

1.4.1.70　pg_shdepend

视图 pg_shdepend 记录数据库对象和共享对象之间的依赖关系，例如角色。这些信息使得 PostgreSQL 可以确保对象在被删除时没有被其他对象引用。

与大部分其他系统视图不同，pg_shdepend 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 pg_shdepend 的拷贝，而不是每个数据库一份。

表 1.4.68: pg_shdepend 列

名称	类型	描述
dbid	oid	依赖者对象所在的数据库 OID，如果是一个共享对象则值为 0
classid	oid	依赖者对象所在的系统视图的 OID
objid	oid	依赖者对象的 OID
objsubid	int4	对于一个表列，这将是列号（objid 和 classid 指向表本身）。对于所有其他对象类型，该列值为 0
refclassid	oid	被引用对象所在的系统视图的 OID（必须是一个共享的视图）

-续表-

名称	类型	描述
refobjid	oid	被引用对象的 OID
deptype	char	定义该依赖关系的特定语义的代码，见表后的说明

在所有情况下，一个 pg_shdepend 项表明被引用对象不能在没有删除其依赖对象的情况下被删除。但是，其中也有多种依赖类型，由 deptype 标识：

- SHARED_DEPENDENCY_OWNER (o)
  被引用对象（必须是一个角色）是依赖对象的拥有者。
- SHARED_DEPENDENCY_ACL (a)
  被引用对象（必须是一个角色）在依赖对象的 ACL（访问控制列表，即权限列表）中被提到。(不会为对象的拥有者创建一个 SHARED_DEPENDENCY_ACL 项，因为拥有者将会有一个 SHARED_DEPENDENCY_OWNER 项。）
- SHARED_DEPENDENCY_POLICY ®
  作为一个依赖策略对象的目标被引用的对象（必须是一个角色）。
- SHARED_DEPENDENCY_PIN (p)
  没有依赖对象，这种类型的项是一个信号，用来指示系统本身依赖于被引用对象，并且因此该对象必须永远不能被删除。这种类型的项只能被 initdb 创建。这种项中关于依赖对象的列值都为 0。

未来可能会需要其他的依赖类型。特别要注意的是在当前定义中只支持角色作为被引用对象。

1.4.1.71　pg_shdescription

视图 pg_shdescription 存储共享数据库对象的可选描述（注释）。描述可以通过 COMMENT 命令操作，并且可以使用 psql 的 d 命令来查看。

与大部分其他系统视图不同，pg_shdescription 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 pg_shdescription 的拷贝，而不是每个数据库一份。

表 1.4.69: pg_shdescription 列

名称	类型	描述
objoid	oid	该描述所属的对象的 OID
classoid	oid	该对象所在系统视图的 OID
description	text	作为该对象描述的任意文本

1.4.1.72　pg_shseclabel

视图 pg_shseclabel 存储共享数据库对象上的安全标签。安全标签可以通过 CREATE SECURITY LABEL 命令创建。

与大部分其他系统视图不同，pg_shseclabel 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 pg_shseclabel 的拷贝，而不是每个数据库一份。

表 1.4.70: pg_shseclabel 列

名称	类型	描述
objoid	oid	该安全标签所属对象的 OID
classoid	oid	对象所属系统视图的 OID
provider	text	与此标签关联的标签提供者
label	text	应用到该对象的安全标签

1.4.1.73　pg_statistic

视图 pg_statistic 存储有关数据库内容的统计数据。其中的项由 ANALYZE 创建，查询计划器会使用这些数据来进行查询计划。注意所有的统计数据天然就是近似的，即使它刚刚被更新。

通常对于数据表中一个已经被 ANALYZE 过的列，在本视图中会存在一个 stainherit = false 的项。如果该列所在的表具有后代（即有其他表继承该表），对于该列还会创建第二个 stainherit = true 的项。stainherit = true 的项表示列在整个继承树上的统计数据，即通过 SELECT column FROM table* 看到的数据的统计，而 stainherit = false 的项表示对 SELECT column FROM ONLY table 的结果的统计。

pg_statistic 也存储关于索引表达式值的统计数据，就好像它们是真正的数据列，但在这种情况中 starelid 指索引。对一个普通非表达式索引列不会创建项，因为它将是底层表列的项的冗余。当前，索引表达式的项都具有 stainherit = false。

因为不同类型的统计信息适用于不同类型的数据，pg_statistic 被设计成不太在意自己存储的是什么类型的统计。只有极为常用的统计信息（比如 NULL 的含量）才在 pg_statistic 里给予专用的字段。其它所有东西都存储在“槽位”中，而槽位是一组相关的列，它们的内容用槽位中的一个列里的代码表示。更详细的信息请参阅 src/include/catalog/pg_statistic.h。

pg_statistic 不应该是公共可读的，因为即使是一个表内容的统计性信息也可能被认为是敏感的（例子：一个薪水列的最大和最小值可能是非常有趣的）。pg_stats 是 pg_statistic 上的一个公共可读的视图，它只会显示出当前用

户可读的表的信息。

列名称中的 [i] 代表该前缀列名在该视图中有多个，如 stakind[i] 代 sys_statistic 视图中包含 stakind1，stakind2，stakind3，stakind4，stakind5 列，这五个列有相同的 stakind 前缀名。

表 1.4.71: pg_statistic 列

名称	类型	描述
starelid	oid	被描述列所属的表或索引
staattnum	int2	被描述列的编号
stainherit	bool	如果为真，统计包含了继承后代的列而不仅仅是指定关系的列
stanullfrac	float4	列的项为空的比例
stawidth	int4	非空项的平均存储宽度，以字节计
stadistinct	float4	列中非空唯一值的数目。一个大于零的值是唯一值的真正数目。一个小于零的值是表中行数的乘数的负值； 例如，对于一个 80% 的值为非空且每个非空值平均出现两次的列，可以表示为 stadistinct = -0.4 一个 0 值表示唯一值的数目未知
stakindN	int2	一个代码，它表示存储在该 pg_statistic 行中第 N 个“槽位”的统计类型
staopN	oid	一个用于生成这些存储在第 N 个“槽位”的统计信息的操作符。比如，一个柱面图槽位会用 < 操作符，该操作符定义了该数据的排序顺序
stacollN	oid	排序规则用于导出存储在第 N 个“槽”中的统计信息。例如，可排序列的直方图槽将显示定义数据排序顺序的排序规则。对于不可整理数据，为零
stanumbersN	float4[]	第 N 个“槽位”的类型的数值类型统计，如果该槽位不涉及数值类型则为 NULL
stavaluesN	anyarray	第 N 个“槽位”的类型的列值，如果该槽位类型不存储任何数据值则为 NULL 每个数组的元素值实际上都是指定列的数据类型或者是一个相关类型（如数组元素类型），因此，除了把这些列的类型定义成 anyarray 之外别无他法

1.4.1.74　pg_statistic_ext

视图 pg_statistic_ext 包含了扩展的规划器统计信息的定义。这个视图中的每一行对应于一个用 CREATE STATISTICS 创建的统计信息对象。

表 1.4.72: pg_statistic_ext 列

名称	类型	描述
stxrelid	oid	包含这个对象所描述的列的表
stxname	name	统计信息对象的名称
stxnamespace	oid	包含这个统计信息对象的名字空间的 OID
stxowner	oid	统计信息对象的拥有者
stxkeys	int2[]	一个属性编号的数组，表示哪些表列被这个统计信息对象覆盖。例如值 13 表示第一个和第三个表列被覆盖
stxkind	char[]	包含被启用统计类型代码的数组，可用的值有： d 表示 n-distinct 统计信息 f 表示函数依赖统计信息 m 表示最常见值（MCV）列表的统计信息

pg_statistic_ext 条目在 CREATE STATISTICS 期间完全填充，但是随后不计算实际的统计值。后来 ANALYZE 命令计算所需的值，并在 pg_statistic_ext_data 视图中填充条目。

1.4.1.75　pg_statistic_ext_data

视图 pg_statistic_ext_data 保存在 pg_statistic_ext 中定义的扩展规划器统计信息的数据。该视图的每一行对应用 CREATE STATISTICS 创建的一个统计信息对象。

就像 pg_statistic，pg_statistic_ext_data 不应该被公众读取到，因为其内容可能被视为敏感内容。（例如：列中值的最常见组合可能非常令人关注。）pg_stats_ext 是 pg_statistic_ext_data 上的公共可读视图（与 pg_statistic_ext 连接后），它仅暴露有关当前用户可读的表和列的信息。

表 1.4.73: pg_statistic_ext_data 列

名称	类型	描述
stxoid	oid	包含此数据定义的扩展统计信息对象
stxdndistinct	pg_ndistinct	N-distinct 计数，序列化为 pg_ndistinct 类型
stxddependencies	pg_dependencies	函数依赖统计信息，序列化为 pg_dependencies 类型
stxdmcv	pg_mcv_list	MCV（最频值）列表统计信息，序列化为 pg_mcv_list 类型

1.4.1.76　pg_stats

视图 pg_stats 提供对存储在 pg_statistic 视图中信息的访问。此视图能访问 pg_statistic 行是有限制的，可访问行所对应的表必须是用户有读权限的。因此让所有用户都可以读此视图是安全的。

pg_stats 也被设计为能以更适合阅读的格式显示底层视图的信息—但代价是只要为 pg_statistic 定义了新的槽类型，就必须要扩展此视图的模式。

表 1.4.74: pg_stats 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
attname	name	被此行描述的列名
inherited	bool	如果为真，表示此行包括继承子列，不仅仅是指定表中的值
null_frac	real	列项中为空的比例
avg_width	int4	列项的平均字节宽度

-续表-

名称	类型	描述
n_distinct	real	如果大于零，表示列中可区分值的估计个数。 如果小于零，是可区分值个数除以行数的负值（当 ANALYZE 认为可区分值的数量会随着表增长而增加时采用负值的形式，而如果认为列具有固定数量的可选值时采用正值的形式） 例如，-1 表示一个唯一列，即其中可区分值的个数等于行数
most_common_vals	anyarray	列中最常用值的一个列表（如果没有任何一个值看起来比其他值更常用，此列为空）
most_common_freqs	real[]	最常用值的频率列表，即每一个常用值的出现次数除以总行数（如果 most_common_vals 为空，则此列为空）
histogram_bounds	anyarray	将列值划分成大小接近的组的值列表。如果存在 most_common_vals，其中的值会被直方图计算所忽略（如果列类型没有一个 < 操作符或者 most_common_vals 等于整个值集合，则此列为空）
correlation	real	物理行顺序和列值逻辑顺序之间的统计关联，其范围从-1 到 +1 当值接近-1 或 +1 时，在列上的一个索引扫描被认为比值接近 0 时的代价更低，因为这种情况减少了对磁盘的随机访问（如果列数据类型不具有一个 < 操作符，则此列为空）
most_common_elems	anyarray	在列值中，最经常出现的非空元素列表（对标度类型为空）
most_common_elem_freqs	real[]	最常用元素值的频度列表，即含有至少一个给定值实例的行的分数。在每个元素的频度之后有二至三个附加值，它们是每个元素频度的最小和最大值，以及可选的空元素的频度（如果 most_common_elems 为空，则此列为空）
elem_count_histogram	real[]	在列值中可区分非空元素值计数的一个直方图，后面跟随可区分非空元素的平均数（对于标度类型为空）

在数组域中项的最大数目可以使用 ALTER TABLE SET STATISTICS 命令控制，或者设置 de-

fault_statistics_target 运行时参数从全局上进行控制

1.4.1.77　pg_stats_ext

视图 pg_stats_ext 提供了访问存储在 pg_statistic_ext 和 pg_statistic_ext_data 视图中的信息的手段。该视图仅允许访问 pg_statistic_ext 和 pg_statistic_ext_data 的行，这些行对应于用户有权读取的表，因此允许公众读取该视图是安全的。

pg_stats_ext 也设计成以比底层视图更可读的格式来展示信息—代价是每当有扩展的统计信息的新类型加到 pg_statistic_ext 中时其模式必须扩展。

表 1.4.75: pg_stats_ext 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
statistics_schemaname	name	包含扩展的统计信息的模式名
statistics_name	name	扩展的统计信息的名称
statistics_owner	name	扩展的统计信息对的拥有者
attnames	name[]	扩展的统计信息定义所在的列名称
kinds	text[]	为此记录启用的扩展统计信息类型
n_distinct	pg_ndistinct	列值组合的 N-不同计数 如果大于零，则为组合中不同值的估计数量 如果小于零，则为不同值数量的负数除以行数（当 ANALYZE 认为随着表的增长不同值的数量可能会增加时，使用负数形式；当该列似乎具有固定数量的可能值时，则使用正数形式。） 例如，-1 表示列的唯一组合，其中不同组合的数量与行数相同
dependencies	pg_dependencies	功能的依赖关系统计信息
most_common_vals	anyarray	列中值的最常见组合的列表（如果没有组合看上去比其它的更常见，则为空。）
most_common_val_nulls	anyarray	值最常见组合的 NULL 标志的列表（当 most _common_vals 是空值时，为空。）

-续表-

名称	类型	描述
most_common_freqs	real[]	最常见组合的频率的列表，即每个出现的数量除以行的总数（当 most_common_vals 是空值时，为空）
most_common_base_freqs	real[]	最常见组合的基本频率的列表，即每个值频率的乘积。（当 most_common_vals 是空值时，为空。）

数组字段条目的最大数量可以基于一列列地使用 ALTER TABLE SET STATISTICS 命令来控制，或者通过设置全局的 default_statistics_target 运行时参数来控制。

1.4.1.78　pg_subscription

视图 pg_subscription 包含所有现有的逻辑复制订阅。

和大部分系统视图不同，pg_subscription 在集簇的所有数据库之间共享：每个集簇只有一份 pg_subscription 拷贝，而不是每个数据库一份。

对列 subconninfo 的访问被从普通用户那里收回，因为该列可能含有明文口令。

表 1.4.76: pg_subscription 列

名称	类型	描述
oid	oid	行标识符
subdbid	oid	订阅所在的数据库的 OID
subname	name	订阅的名称
subowner	oid	订阅的拥有者
subenabled	bool	如果为真，订阅被启用并且应该被复制
subconninfo	text	到上游数据库的连接字符串
subslotname	name	上游数据库中的复制槽的名称。也被用于本地复制源名称
subsynccommit	text	包含订阅工作者的 synchronous_commit 设置的值

-续表-

名称	类型	描述
subpublications	text[]	被订阅的 publication 名称的数组。这些引用的是发布者服务器上的 publication

1.4.1.79　pg_subscription_rel

视图 pg_subscription_rel 包含每个订阅中每个被复制关系的状态。这是一种多对多映射。

这个视图仅包含运行 CREATE SUBSCRIPTION 或 ALTER SUBSCRIPTION …REFRESH PUBLICATION 以后对订阅已知的表。

表 1.4.77: pg_subscription_rel 列

名称	类型	描述
srsubid	oid	对订阅的引用
srrelid	oid	对关系的引用
srsubstate	char	状态代码：i = 初始化 d = 数据正在被拷贝 s = 已同步 r = 准备好（普通复制）
srsublsn	pg_lsn	s 和 r 状态的结束 LSN

1.4.1.80　pg_tables

视图 pg_tables 提供对数据库中每个表的信息的访问。

表 1.4.78: pg_tables 列

名称	类型	描述
schemaname	name	包含表的模式名
tablename	name	表名
tableowner	name	表拥有者的名字

-续表-

名称	类型	描述
tablespace	name	包含表的表空间的名字（如果使用数据库的默认表空间，此列为空）
hasindexes	boolean	如果表有（或最近有过）任何索引，此列为真
hasrules	boolean	如果表有（或曾经有过）规则，此列为真
hastriggers	boolean	如果表有（或者曾经有过）触发器，此列为真
rowsecurity	boolean	如果表上启用了行安全性则为真

1.4.1.81　pg_tablespace

视图 pg_tablespace 存储关于可用表空间的信息。表可以被放置在特定表空间中以实现磁盘布局的管理。

与大部分其他系统视图不同，pg_tablespace 在整个集簇的所有数据库之间共享：在每一个集簇中只有一个 pg_tablespace 的拷贝，而不是每个数据库一份。

表 1.4.79: pg_tablespace 列

名称	类型	描述
oid	oid	行标识符
spcname	name	表空间名
spcowner	oid	表空间的拥有者，通常是创建它的用户
spcacl	aclitem[]	访问权限
spcoptions	text[]	表空间级别的选项，形如“keyword=value”的字符串

1.4.1.82　pg_timezone_abbrevs

视图 pg_timezone_abbrevs 提供了对当前被时间输入例程识别的时区缩写的列表。当 timezone_abbreviations 运行时参数被修改，此视图的内容会发生变化。

表 1.4.80: pg_timezone_abbrevs 列

名称	类型	描述
abbrev	text	时区缩写
utc_offset	interval	相对于 UTC 的偏移（正值表示格林威治东部）
is_dst	boolean	如果这是一个夏令时缩写，则为真

虽然大部分时区缩写表示从 UTC 开始的固定偏移，但是有一些在历史上有值的变化。在这种情况下，这个视图表示它们现在的含义。

1.4.1.83　pg_timezone_names

视图 pg_timezone_names 提供了一个被 SET TIMEZONE 识别的时区名字的列表，以及它们的相关缩写、UTC 偏移和夏令时状态（从技术上来说，KingbaseES 不使用 UTC 是因为闰秒没有被处理）。和 pg_timezone_abbrevs 中展示的缩写不同，这里很多名字隐含了一组夏令时转换日期规则。因此，相关信息在本地 DST 边界间变化。所显示的信息基于 CURRENT_TIMESTAMP 的当前值计算得来。

表 1.4.81: pg_timezone_names 列

名称	类型	描述
name	text	时区名
abbrev	text	时区缩写
utc_offset	interval	相对于 UTC 的偏移（正值表示格林威治东部）
is_dst	boolean	如果当前保持为夏令时则为真

1.4.1.84　pg_transform

视图 pg_transform 存储有关转换的信息，转换是一种让数据类型适应过程语言的机制。详见 CREATE TRANSFORM。

表 1.4.82: pg_transform 列

名称	类型	描述
oid	oid	这个转换的 OID
trftype	oid	这个转换所针对的数据类型的 OID
trflang	oid	这个转换所针对的语言的 OID
trffromsql	regproc	一个函数的 OID，该函数用来将数据类型转换为过程语言的输入（例如函数参数）。如果不支持这种操作，这里存储零
trftosql	regproc	一个函数的 OID，该函数被用来转换过程语言的输出（例如返回值）为该数据类型。如果不支持这种操作，这里存储零

1.4.1.85　pg_trigger

视图 pg_trigger 存储表和视图上的触发器。详见 CREATE TRIGGER。

表 1.4.83: pg_trigger 列

名称	类型	描述
oid	oid	行标识符
tgrelid	oid	触发器所在的表
tgname	name	触发器名（在同一个表的触发器中必须唯一）
tgfoid	oid	要被触发器调用的函数
tgtype	int2	触发器触发条件的位掩码
tgenabled	char	控制触发器在 session_replication_role 模式中的触发 O = 触发器在“origin”和“local”模式触发 D = 触发器被禁用 R = 触发器在“replica”模式触发 A = 触发器总是触发

-续表-

名称	类型	描述
tgisinternal	bool	为真表示触发器是内部生成的（通常是为了强制由 tgconstraint 指定的约束）
tgconstrrelid	oid	被一个引用完整性约束引用的表
tgconstrindid	oid	支持一个唯一、主键、引用完整性约束或者排除约束的索引
tgconstraint	oid	可能存在的与触发器相关的 pg_constraint 项
tgdeferrable	bool	如果约束触发器可推迟则为真
tginitdeferred	bool	如果约束触发器初始可推迟则为真
tgnargs	int2	传递给触发器函数的参数字符串个数
tgattr	int2[]	如果触发器是列限定的，这里存放列号；否则这是一个空数组
tgargs	bytea	传递给触发器的参数字符串，每一个都以 NULL 结尾
tgqual	pg_node_tree	触发器 WHEN 条件的表达式树（以 nodeToString() 的表现形式），如果没有则为空
tgoldtable	name	OLD TABLE 的 REFERENCING 子句名称，如果没有则为空
tgnewtable	name	NEW TABLE 的 REFERENCING 子句名称，如果没有则为空

当前，列限定触发器只被 UPDATE 事件支持，因此 tgattr 只用于这种事件类型。tgtype 页可以包含用于其他事件类型的位，但其他事件类型是表范围的触发器且会忽略 tgattr。

1.4.1.86　pg_ts_config

pg_ts_config 视图包含表示文本搜索配置的选项。一个配置指定了一个特定的文本搜索分析器和一个用于分析器输出记号的字典列表。分析器由 pg_ts_config 项展现，而记号到字典的映射则由 pg_ts_config_map 中的辅助项定义。pg_ts_config 视图包含表示文本搜索配置的选项。一个配置指定了一个特定的文本搜索分析器和一个用于分析器输出记号的字典列表。分析器由 pg_ts_config 项展现，而记号到字典的映射则由 pg_ts_config_map 中的辅助项定义。

表 1.4.84: pg_ts_config 列

名称	类型	描述
oid	oid	行标识符
cfgname	name	文本搜索配置名
cfgnamespace	oid	包含该配置的名字空间的 OID
cfgowner	oid	配置的拥有者
cfgparser	oid	该配置的文本搜索分析器的 OID

1.4.1.87　pg_ts_config_map

pg_ts_config_map 视图包含的项展示了对于每一个文本搜索配置的每一种输出记号类型，有哪些文本搜索字典可供查询以及以何种顺序。

表 1.4.85: pg_ts_config_map 列

名称	类型	描述
mapcfg	oid	拥有该映射项的 pg_ts_config 项的 OID
maptokentype	int4	一种由配置的分析器送出的记号类型
mapseqno	int4	查询该项的顺序（mapseqno 值小的优先）
mapdict	oid	查询的文本搜索字典的 OID

1.4.1.88　pg_ts_dict

pg_ts_dict 视图包含定义文本搜索字典的项。一个字典依赖于一个文本搜索模板，它指定了所有需要的函数实现，字典本身则为模板支持的用户可设置参数提供值。这种分工允许无权限的用户创建字典。参数由一个文本串 dictinitoption 定义，其格式和意义随着模板而变化。

表 1.4.86: pg_ts_dict 列

名称	类型	描述
oid	oid	行标识符
dictname	name	文本搜索字典名
dictnamespace	oid	包含该字典的名字空间 OID
dictowner	oid	字典的拥有者
dicttemplate	oid	该字典的文本搜索模板的 OID
dictinitoption	text	模板的初始化选项串

1.4.1.89　pg_ts_parser

pg_ts_parser 视图包含定义文本搜索分析器的项。一个分析器负责将输入文本分割成词位并为每一个词位分配一个记号类型。由于一个分析器必须用 C 语言级别的函数实现，创建新分析器的工作只限于数据库的超级用户。

表 1.4.87: pg_ts_parser 列

名称	类型	描述
oid	oid	行标识符
prsname	name	文本搜索分析器的名字
prsnamespace	oid	包含此分析器的名字空间的 OID
prsstart	regproc	分析器启动函数的 OID
prstoken	regproc	分析器的下一记号函数的 OID
prsend	regproc	分析器的关闭函数的 OID
prsheadline	regproc	分析器的大标题函数的 OID
prslextype	regproc	分析器的词汇类型函数的 OID

1.4.1.90　pg_ts_template

pg_ts_template 视图包含定义文本搜索模板的项。一个模板是一类文本搜索字典的实现骨架。由于一个模板必须用 C 语言级别的函数实现，新模板的创建只限于数据库超级用户。

表 1.4.88: pg_ts_template 列

名称	类型	描述
oid	oid	行标识符
tmplname	name	文本搜索模板的名字
tmplnamespace	oid	包含此模板的名字空间的 OID
tmplinit	regproc	模板的初始化函数的 OID
tmpllexize	regproc	模板的词汇化函数的 OID

1.4.1.91　pg_type

视图 pg_type 存储有关数据类型的信息。基类和枚举类型（标度类型）使用 CREATE TYPE 创建，而域使用 CREATE DOMAIN 创建。数据库中的每一个表都会有一个自动创建的组合类型，用于表示表的行结构。也可以使用 CREATE TYPE AS 创建组合类型。

表 1.4.89: pg_type 列

名称	类型	描述
oid	oid	行标识符
typname	name	数据类型的名字
typnamespace	oid	包含此类型的名字空间的 OID
typowner	oid	类型的拥有者
typlen	int2	对于一个固定尺寸的类型，typlen 是该类型内部表示的字节数 对于一个变长类型，typlen 为负值 -1 表示一个“varlena”类型（具有长度字） -2 表示一个以空值结尾的 C 字符串

-续表-

名称	类型	描述
typbyval	bool	typbyval 判断内部例程传递这个类型的数值时是通过传值还是传引用 如果 typlen 不是 1、2 或 4（或者在 Datum 为 8 字节的机器上为 8），因此 typbyval 最好是假 变长类型总是传引用 注意即使长度允许传值，typbyval 也可以为假
typtype	char	typtype 可以是： b 表示一个基类 c 表示一个组合类型（例如一个表的行类型） d 表示一个域 e 表示一个枚举类型 p 表示一个伪类型 r 表示一个范围类型 另请参阅 typrelid 和 typbasetype
typcategory	char	typcategory 是一种任意的数据类型分类，它被分析器用来决定哪种隐式转换“更好”
typispreferred	bool	如果此类型在它的 typcategory 中是一个更好的转换目标，此列为真
typisdefined	bool	如果此类型已被定义则为真，如果此类型只是一个表示还未定义类型的占位符则为假。当 typisdefined 为假，除了类型名字、名字空间和 OID 之外什么都不能被依赖
typdelim	char	在分析数组输入时，分隔两个此类型值的字符。注意该分隔符是与数组元素数据类型相关联的，而不是和数组的数据类型关联
typrelid	oid	如果这是一个复合类型（见 typtype），那么这个列指向 pg_class 中定义对应表的项（对于自由存在的复合类型，pg_class 项并不表示一个表，但不管怎样该类型的 pg_attribute 项需要链接到它）对非复合类型此列为零

-续表-

名称	类型	描述
typelem	oid	如果 typelem 不为 0，则它标识 pg_type 里面的另外一行 当前类型可以被加上下标得到一个值为类型 typelem 的数组来描述 一个“真的”数组类型是变长的（typlen = -1），但是一些定长的（typlen> 0）类型也拥有非零的 typelem，比如 name 和 point 如果一个定长类型拥有一个 typelem，则它的内部形式必须是某个 typelem 数据类型的值，不能有其它数据 变长数组类型有一个由该数组子例程定义的头
typarray	oid	如果 typarray 不是 0，则它标识 pg_type 中的另一行，这一行是一个将此类型作为元素的“真的”数组类型
typinput	regproc	输入转换函数（文本格式）
typoutput	regproc	输出转换函数（文本格式）
typreceive	regproc	输入转换函数（二进制格式），如果没有则为 0
typsend	regproc	输出转换函数（二进制格式），如果没有则为 0
typmodin	regproc	类型修改器输入函数，如果类型没有提供修改器则为 0
typmodout	regproc	类型修改器输出函数，如果类型没有提供修改器则为 0
typanalyze	regproc	自定义 ANALYZE 函数，0 表示使用标准函数

-续表-

名称	类型	描述
typalign	char	typalign 是当存储此类型值时要求的对齐性质。它应用于磁盘存储以及该值在 PostgreSQL 内部的大多数表现形式。 如果数值是连续存放的，比如在磁盘上的一个完整行，在这种类型的数据前会插入填充，这样它就可以按照指定边界存储。 对齐引用是该序列中第一个数据的开头, 对齐引用是序列中第一个数据的开始。可能的值有： c = char 对齐，即不需要对齐 s = short 对齐（在大部分机器上为 2 字节） i = int 对齐（在大部分机器上为 4 字节） d = double 对齐（在很多机器上为 8 字节，但绝不是全部） 注意：对于系统表中使用的类型，很关键的是，pg_type 中定义的尺寸和对齐方式要和编译器在表示表行的结构中布局列的方式保持一致
typstorage	char	如果一个变长类型（typlen =-1）可被 TOAST，typstorage 说明这种类型的列应采取的默认策略。可能的值是： p：值必须平面存储 e：值可以被存储在一个“二级”关系（如果有，见 pg_class.reltoastrelid） m：值可以被压缩线内存储 x：值可以被压缩线内存储或存储在“二级”存储 注意 m 列也可以被移动到二级存储，但只能是作为最后一种方案（e 和 x 列会先被移动）
typnotnull	bool	typnotnull 表示类型上的一个非空约束。只用于域
typbasetype	oid	如果这是一个域（见 typtype），则 typbasetype 标识这个域基于的类。如果此类不是一个域则为 0
typtypmod	int4	域使用 typtypmod 来记录被应用于它们基类型的 typmod（如果基类型不使用 typmod，则为-1）。如果此类型不是一个域则为-1

-续表-

名称	类型	描述
typndims	int4	对于一个数组上的域，typndims 是数组维度数（即，typbasetype 是一个数组类型）。除数组类型上的域之外的类型的此列为 0
typcollation	oid	typcollation 指定此类型的排序规则 如果类型不支持排序规则，此列为 0 一个支持排序规则的基类型此处会有一个非零值，典型值为 DEFAULT_COLLATION_OID 如果为该域指定了一个排序规则 OID 的话，可排序类型上的域可以有一个不同于其基类型的排序规则 OID
typdefaultbin	pg_node_tree	如果 typdefaultbin 为非空，那么它是该类型默认表达式的 nodeToString() 表现形式。这个列只用于域
typdefault	text	如果某类型没有相关默认值，那么 typdefault 为空。如果 typdefaultbin 不为空，那么 typdefault 必须包含一个 typdefaultbin 所指的默认表达式的人类可读的版本 如果 typdefaultbin 为空但 typdefault 不为空，则 typdefault 是该类型默认值的外部表现形式，它可以被交给该类型的输入转换器来产生一个常量
typacl	aclitem[]	访问权限

下表列出了 typcategory 的系统定义值。任何未来对此列表的增加都将是大写 ASCII 字母。所有其他 ASCII 字符都保留给用户定义的类别。

表 1.4.90: typcategory 编码

编码	类别
A	数组类型
B	布尔类型
C	组合类型
D	日期/时间类型

-续表-

编码	类别
E	枚举类型
G	几何类型
I	网络地址类型
N	数字类型
P	伪类型
R	范围类型
S	字符串类型
T	时间间隔类型
U	用户定义类型
V	位串类型
X	未知类型

1.4.1.92　pg_user

视图 pg_user 提供关于数据库用户的信息。这是 pg_shadow 的一个公共可读的视图，它消除了密码域。

表 1.4.91: pg_user 列

名称	类型	描述
usename	name	用户名
usesysid	oid	用户的 ID
usecreatedb	bool	用户是否能创建数据库
usesuper	bool	用户是否为超级用户
userepl	bool	用户能否开启流复制以及将系统转入/转出备份模式
usebypassrls	bool	用户能否绕过所有的行级安全性策略

-续表-

名称	类型	描述
passwd	text	不是密码（总是显示为 ****）
valuntil	timestamptz	密码过期时间（只用于密码认证）
useconfig	text[]	运行时配置变量的会话默认值

1.4.1.93　pg_user_mapping

视图 pg_user_mapping 存储从本地用户到远程的映射。对这个视图的访问对普通用户有限制，可使用视图 pg_user_mappings 替代。

表 1.4.92: pg_user_mapping 列

名称	类型	描述
oid	oid	行标识符
umuser	oid	将要被映射的本地角色的 OID，如果用户映射是公共的则为 0
umserver	oid	包含此映射的外部服务器的 OID
umoptions	text[]	用户映射相关选项，以“keyword=value”字符串形式

1.4.1.94　pg_user_mappings

视图 pg_user_mappings 提供有关用户映射的信息。这是 pg_user_mapping 的一个公共可读视图，它对无权使用的用户省去了选项列。

表 1.4.93: pg_user_mappings 列

名称	类型	描述
umid	oid	用户映射的 OID

-续表-

名称	类型	描述
srvid	oid	包含该映射的外部服务器的 OID
srvname	name	外部服务器名
umuser	oid	将被映射的本地角色的 OID，如果用户映射是公共的则为 0
usename	name	将被映射的本地用户名
umoptions	text[]	用户映射指定选项，以“keyword=value”字符串的形式

为了保护存储为用户映射选项的口令信息，umoptions 列将被读作空，除非满足下列情况之一：

- 当前用户就是被映射的用户，并且拥有该服务器或者持有其上的 USAGE 特权
- 当前用户是服务器的拥有者并且映射是用于 PUBLIC
- 当前用户是一个超级用户

1.4.1.95　pg_views

视图 pg_views 提供了数据库中每个视图的信息。

表 1.4.94: pg_views 列

名称	类型	描述
schemaname	name	包含视图的模式名
viewname	name	视图名称
viewowner	name	视图拥有者的名字
definition	text	视图定义（一个重构的 SELECT 查询）

1.4.2 信息模式视图

信息模式由一组视图构成，它们包含定义在当前数据库中对象的信息。信息模式以 SQL 标准定义，能够被移植并且保持稳定.

信息模式视图位于 information_schema 下，需要通过 information_schema. 视图名访问。

1.4.2.1 administrable_role_authorizations

视图 `administrable_role_authorizations` 标识当前用户对其有管理选项的所有角色。

表 1.4.95: administrable_role_authorizations 列

名称	数据类型	描述
grantee	sql_identifier	被授予这个角色的成员关系的角色名（可以是当前用户，或者在嵌套角色成员关系情况下的一个不同角色）
role_name	sql_identifier	角色名
is_grantable	yes_or_no	总是 YES

1.4.2.2 applicable_roles

视图 `applicable_roles` 当前用户可以使用其特权的所有角色。这意味着有某种角色授权链从当前用户到讨论中的角色。当前用户本身也是一个可应用的角色。可应用的角色的集合通常被用于权限检查。

表 1.4.96: applicable_roles 列

名称	数据类型	描述
grantee	sql_identifier	被授予这个角色的成员关系的角色名（可以是当前用户，或者在嵌套角色成员关系情况下的一个不同角色）
role_name	sql_identifier	一个角色的名字
is_grantable	yes_or_no	YES 表示被授予者在该角色上有管理选项，NO 表示没有管理选项

1.4.2.3　attributes

视图 `attributes` 包含数据库中定义的组合数据类型的属性的有关信息（注意该视图并不给出有关表列的信息，表列有时候在 KingbaseES 上下文环境中也被称为属性）。只有当前用户能够访问（由于是拥有者获得的权限或是在类型上有某些特权）的那些属性会被显示。

表 1.4.97: attributes 列

名称	数据类型	描述
udt_catalog	sql_identifier	包含该数据类型的数据库名（总是当前数据库）
udt_schema	sql_identifier	包含该数据类型的模式名
udt_name	sql_identifier	数据类型名
attribute_name	sql_identifier	属性名
ordinal_position	cardinal_number	属性在该数据类型内部的顺序位置（从 1 开始计算）
attribute_default	character_data	该属性的默认表达式
is_nullable	yes_or_no	如果该属性是可能为空的，值为 YES，否则为 NO
data_type	character_data	如果该属性是一个内建类型，此列值为该属性的数据类型；如果该属性是某种数组，此列值为 ARRAY（在这种情况下，见视图 element_types）；其他情况，此列值为 USER-DEFINED（在这种情况下，该类型在 attribute_udt _name 和相关列中标识）
character_maximum_length	cardinal_number	如果 data_type 标识一个字符或位串类型，这里是声明的最大长度；如果没有声明最大长度，则对于所有其他数据类型为空
character_octet_length	cardinal_number	如果 data_type 标识一个字符类型，这里是一个数据的最大可能长度（以字节计）；对其他所有数据类型为空。最大字节长度取决于声明的字符最大长度（见上文）和服务器编码
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	包含此属性排序规则的数据库名（总是当前数据库），如果默认或者该属性的数据类型不可排序则为空
collation_schema	sql_identifier	包含此属性排序规则的模式名，如果默认或者该属性的数据类型不可排序则为空
collation_name	sql_identifier	该属性排序规则的名称，如果默认或者该属性的数据类型不可排序则为空
numeric_precision	cardinal_number	如果 data_type 标识一种数字类型，这列包含这个属性类型的（声明的或隐式的）精度。精度指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空
numeric_precision_radix	cardinal_number	如果 data_type 标识一种数字类型，这一列指示 numeric_precision 和 numeric_scale 列中的值是基于什么来表示。该值为 2 或 10。对于所有其他数据类型，这一列为空
numeric_scale	cardinal_number	如果 data_type 标识一种准确数字类型，这列包含这个属性类型的（声明的或隐式的）比例。比例指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空

-续表-

名称	数据类型	描述
datetime_precision	cardinal_number	如果 data_type 标识一种日期、时间、时间戳或时间间隔类型，这一列包含这个属性类型的（声明的或隐式的）分数秒的精度，也就是秒值的小数点后的十进制位数。对于所有其他数据类型，这一列为空
interval_type	character_data	如果 data_type 标识一种时间间隔类型，这一列包含时间间隔为这个属性包括哪些域的声明，例如 YEAR TO MONTH、DAY TO SECOND 等等。如果没有指定域限制（也就是该时间间隔接受所有域），并且对于所有其他数据类型，这个域为空
interval_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性（关于时间间隔类型属性的分数秒精度可见 datetime_precision）
attribute_udt_catalog	sql_identifier	属性数据类型被定义的数据库名（总是当前数据库）
attribute_udt_schema	sql_identifier	属性数据类型被定义的模式名
attribute_udt_name	sql_identifier	属性数据类型的名称
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大势
dtd_identifier	sql_identifier	该列的数据类型描述符的一个标识符，在从属于该表的数据类型标识符之中唯一。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）

-续表-

名称	数据类型	描述
is_derived_reference_attribute	yes_or_no	应用于一个 KingbaseES 中不可用的特性

关于某些列的详情，参见columns 之下的一个相似结构的视图。

1.4.2.4　character_sets

视图 `character_sets` 标识当前数据库中可用的字符集。因为 KingbaseES 不支持在同一个数据库中有多个字符集，这个视图只显示一个字符集，它就是数据库编码。

表 1.4.98: character_sets 列

名称	数据类型	描述
character_set_catalog	sql_identifier	当前字符集并未被实现为模式对象，因此这一列为空
character_set_schema	sql_identifier	当前字符集并未被实现为模式对象，因此这一列为空
character_set_name	sql_identifier	该字符集的名字，当前实现为显示该数据库编码的名字
character_repertoire	sql_identifier	字元集，如果编码为 UTF8 则显示 UCS，否则只显示编码名称
form_of_use	sql_identifier	字符编码形式，与数据库编码相同
default_collate_catalog	sql_identifier	包含该默认排序规则的数据库名（如果任意排序规则被标识，总是当前数据库）
default_collate_schema	sql_identifier	包含该默认排序规则的模式名
default_collate_name	sql_identifier	默认排序规则的名字 该默认排序规则被标识为匹配当前数据库的 COLLATE 和 CTYPE 设置的排序规则。如果没有那种排序规则，那么这一列和相关模式以及目录列为空

1.4.2.5　check_constraint_routine_usage

视图 `check_constraint_routine_usage` 标识使用检查约束的例程（函数和过程）。仅显示当前用户所拥有的例程信息。

表 1.4.99: check_constraint_routine_usage 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“指定名称”

1.4.2.6　check_constraints

视图 `check_constraints` 包含所有检查约束，不管是定义在一个表上的还是定义在一个域上的，它们被一个当前启用的角色所拥有（表或域的拥有者就是约束的拥有者）。

表 1.4.100: check_constraints 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
check_clause	character_data	该检查约束的检查表达式

1.4.2.7　collation_character_set_applicability

视图 `collation_character_set_applicability` 标识可用的排序规则适用的字符集。在 KingbaseES 中，每个数据库中只有一种字符集，因此这个视图没有提供很有用的信息。

表 1.4.101: collation_character_set_applicability 列

名称	数据类型	描述
collation_catalog	sql_identifier	包含该排序规则的数据库名（总是当前数据库）
collation_schema	sql_identifier	包含该排序规则的模式名
collation_name	sql_identifier	默认排序规则的名称
character_set_catalog	sql_identifier	当前字符集还未被实现为模式对象，所以这一列为空
character_set_schema	sql_identifier	当前字符集还未被实现为模式对象，所以这一列为空
character_set_name	sql_identifier	字符集名称

1.4.2.8　collations

视图 `collations` 包含在当前数据库中可用的排序规则。

表 1.4.102: collations 列

名称	数据类型	描述
collation_catalog	sql_identifier	包含该排序规则的数据库名（总是当前数据库）
collation_schema	sql_identifier	包含该排序规则的模式名
collation_name	sql_identifier	默认排序规则的名称
pad_attribute	character_data	总是 NOPAD（另一种选择 PAD SPACE 没有被 KingbaseES 支持）

1.4.2.9　column_column_usage

视图 `column_column_usage` 标识依赖于同一表中的另一个基本列生成的所有生成列的信息。只包含当前启用的角色所拥有的表。

表 1.4.103: constraint_column_usage 列

名称	数据类型	描述
table_catalog	sql_identifier	包含表的数据库的名称 (总是当前数据库)
table_schema	sql_identifier	包含表的模式名称
table_name	sql_identifier	表名
column_name	sql_identifier	生成列所依赖的基本列名
dependent_column	sql_identifier	生成的列名

1.4.2.10　column_domain_usage

视图 `column_domain_usage` 标识所有使用定义在当前数据库中并且被一个当前启用的角色拥有的域的列（表列或视图列）。

表 1.4.104: column_domain_usage 列

名称	数据类型	描述
domain_catalog	sql_identifier	包含该域的数据库名（总是当前数据库）
domain_schema	sql_identifier	包含该域的模式名
domain_name	sql_identifier	域名称
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
column_name	sql_identifier	列名称

1.4.2.11　column_options

视图 `column_options` 包含为当前数据库中外部表列定义的所有选项。只有当前用户能够访问（作为拥有者或具有某些特权）的那些外部表列才被显示。

表 1.4.105: column_options 列

名称	数据类型	描述
table_catalog	sql_identifier	包含该外部表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该外部表的模式名
table_name	sql_identifier	外部表名
column_name	sql_identifier	列名称
option_name	sql_identifier	一个选项名
option_value	character_data	该选项的值

1.4.2.12　column_privileges

视图 `column_privileges` 标识所有授予给一个当前启用的角色或者被一个当前启用的角色授予的列的特权。对每一个列、授予者、被授予者的组合只有一行。

如果一个特权被授予在一整个表上，它在这个视图中被显示为在每一列上授予，但是只有可用于列粒度的特权类型才会这样：`SELECT`、`INSERT`、`UPDATE`、`REFERENCES`。

表 1.4.106: column_privileges 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
table_catalog	sql_identifier	包含该列的表所在的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该列的表所在的模式名

-续表-

名称	数据类型	描述
table_name	sql_identifier	包含该列的表名
column_name	sql_identifier	列名称
privilege_type	character_data	特权类型：SELECT、INSERT、UPDATE 或 REFERENCES
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.13　column_udt_usage

视图 `column_udt_usage` 标识所有使用当前用户拥有的数据类型的列。注意在 KingbaseES 中，内建数据类型的行为和用户定义的类型相似，因此它们也被包括在这里。详见columns 。

表 1.4.107: column_udt_usage 列

名称	数据类型	描述
udt_catalog	sql_identifier	该列数据类型（如果适用，底层的域类型）被定义的数据库名（总是当前数据库）
udt_schema	sql_identifier	该列数据类型（如果适用，底层的域类型）被定义的模式名
udt_name	sql_identifier	该列数据类型（如果适用，底层的域类型）的名称
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
column_name	sql_identifier	列名称

1.4.2.14　columns

视图 `columns` 包含数据库中有关所有表列（或视图列）的信息。系统列（`oid` 等）不被包括在内。只有那些当前用户能够访问（作为拥有者或具有某些特权）的列才被显示。

表 1.4.108: columns 列

名称	数据类型	描述
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
column_name	sql_identifier	列名称
ordinal_position	cardinal_number	该列在表内的顺序位置（从 1 开始计）
column_default	character_data	该列的默认表达式
is_nullable	yes_or_no	如果该列可以为空，则为 YES，否则为 NO。一个非空约束是让一列成为不能为空的方法，但还有其他方法
data_type	character_data	如果该列的数据类型是一种内建类型，则为该列的数据类型；如果是某种数组（此种情况见视图 element_types），则为 ARRAY；否则为 USER-DEFINED（此种情况下该类型被标识在 udt_name 和相关列中）。如果该列基于一个域，这一列引用该域底层的类型（该列被标识在 domain_name 和相关列中）
character_maximum_length	cardinal_number	如果 data_type 标识一种字符或位串类型，这里是声明的最大长度；如果没有声明最大长度或者所有其他数据类型，这里为空
character_octet_length	cardinal_number	如果 data_type 标识一个字符类型，这里是一个数据的最大可能长度（以字节计）；对其他所有数据类型为空。最大字节长度取决于声明的字符最大长度（见上文）和服务器编码

-续表-

名称	数据类型	描述
numeric_precision	cardinal_number	如果 data_type 标识一种数字类型，这列包含这个属性类型的（声明的或隐式的）精度。精度指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空
numeric_precision_radix	cardinal_number	如果 data_type 标识一种数字类型，这一列指示 numeric_precision 和 numeric_scale 列中的值是基于什么来表示。该值为 2 或 10。对于所有其他数据类型，这一列为空
numeric_scale	cardinal_number	如果 data_type 标识一种准确数字类型，这列包含这个属性类型的（声明的或隐式的）比例。比例指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空
datetime_precision	cardinal_number	如果‘data_type‘标识一种日期、时间、时间戳或时间间隔类型，这一列包含这个属性类型的（声明的或隐式的）分数秒的精度，也就是秒值的小数点后的十进制位数。对于所有其他数据类型，这一列为空
interval_type	character_data	如果 data_type 标识一种时间间隔类型，这一列包含时间间隔为这个属性包括哪些域的声明，例如 YEAR TO MONTH、DAY TO SECOND 等等。如果没有指定域限制（也就是该时间间隔接受所有域），并且对于所有其他数据类型，这个域为空
interval_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性（关于时间间隔类型属性的分数秒精度可见 datetime_precision）

-续表-

名称	数据类型	描述
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	包含此属性排序规则的数据库名（总是当前数据库），如果默认或者该属性的数据类型不可排序则为空
collation_schema	sql_identifier	包含此属性排序规则的模式名，如果默认或者该属性的数据类型不可排序则为空
collation_name	sql_identifier	该属性排序规则的名称，如果默认或者该属性的数据类型不可排序则为空
domain_catalog	sql_identifier	如果该列有一个域类型，这里是该域所在的数据库名（总是当前数据库），否则为空
domain_schema	sql_identifier	如果该列有一个域类型，这里是该域所在的模式名，否则为空
domain_name	sql_identifier	如果该列有一个域类型，这里是该域的名称，否则为空
udt_catalog	sql_identifier	该列数据类型（如果适用，底层的域类型）被定义的数据库名（总是当前数据库）
udt_schema	sql_identifier	该列数据类型（如果适用，底层的域类型）被定义的模式名
udt_name	sql_identifier	该列数据类型（如果适用，底层的域类型）的名称
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大势
dtd_identifier	sql_identifier	该列的数据类型描述符的一个标识符，在从属于该表的数据类型标识符之中唯一。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）
is_self_referencing	yes_or_no	应用于一个 KingbaseES 中不可用的特性
is_identity	yes_or_no	如果该列是一个标识列，则为 YES，否则为 NO
identity_generation	character_data	如果该列是一个标识列，则为 ALWAYS 或者 BY DEFAULT，它反映该列的定义
identity_start	character_data	如果该列是一个标识列，则是内部序列的起始值，否则为空
identity_increment	character_data	如果该列是一个标识列，则是内部序列的增量，否则为空
identity_maximum	character_data	如果该列是一个标识列，则是内部序列的最大值，否则为空
identity_minimum	character_data	如果该列是一个标识列，则是内部序列的最小值，否则为空
identity_cycle	yes_or_no	如果该列是一个标识列，则 YES 和 NO 分别表示内部序列可循环和不可循环，否则为空
is_generated	character_data	应用于一个 KingbaseES 中不可用的特性
generation_expression	character_data	应用于一个 KingbaseES 中不可用的特性
is_updatable	yes_or_no	如果该列是可更新的，则为 YES，否则为 NO（基表中的列总是可更新的，视图中的列则不一定）

因为在 SQL 中有多种方式定义数据类型，而 KingbaseES 还包含额外的方式来定义数据类型，它们在信息模式中的表示可能有点困难。列 `data_type` 应该标识列的底层内建类型。在 KingbaseES 中，这表示定义在系统目录模式 `sys_catalog` 中的类型。如果应用能够特别地（例如以不同方式格式化数字类型或使用精度列中的数据）处理众所周知的内建类型，这列可能会有用。列 `udt_name`、`udt_schema` 和 `udt_catalog` 总是标识列的底层数据类型，即使该列是基于一个域的（因为 KingbaseES 对待内建类型和用户定义类型的方式是一样的，内建类型也出现在这里。这是 SQL 标准的一种扩展）。如果一个应用想要根据该类型以不同的方式处理数据，就应该使用这些列，因为在那种情况下即使该列真地基于一个域也没有关系。如果该列是基于一个域，该域的标识被存储在列 `domain_name`、`domain_schema` 和 `domain_catalog`。如果你想要把列和它们相关的数据类型配对并且把域视作单独的类型，你可以写 `coalesce(domain_name, udt_name)` 等等。

1.4.2.15　constraint_column_usage

视图 `constraint_column_usage` 标识在当前数据库中被某个约束使用的所有列。只有包含在被一个当前启用的角色拥有的表中的那些列才被显示。对于一个检查约束，这个视图标识被用在该检查约束中的列。对于一个外键约束，这个视图标识外键引用的列。对于一个唯一或主键约束，这个视图标识被约束的列。

表 1.4.109: constraint_column_usage 列

名称	数据类型	描述
table_catalog	sql_identifier	包含被某个约束使用的列的表所在的数据库名（总是当前数据库）
table_schema	sql_identifier	包含被某个约束使用的列的表所在的模式名
table_name	sql_identifier	包含被某个约束使用的列的表名
column_name	sql_identifier	包含被某个约束使用的列名
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名

1.4.2.16　constraint_table_usage

视图 `constraint_table_usage` 标识在当前数据库中被某个约束使用的所有表（这与视图 `table_constraints` 不同，它标识哪些表约束定义在哪些表上）。对于一个外键约束，这个视图标识该外键引用的表。对于一个唯一或主键约束，这个视图仅标识该约束属于的表。检查约束和非空约束不被包括在这个视图中。

表 1.4.110: constraint_table_usage 列

名称	数据类型	描述
table_catalog	sql_identifier	包含被某个约束使用的表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含被某个约束使用的表的模式名
table_name	sql_identifier	包含被某个约束使用的表名
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名

1.4.2.17　data_type_privileges

视图 `data_type_privileges` 标识当前用户能够访问（作为被描述对象的拥有者或者具有其上的某种特权）的所有数据类型。只要一个数据类型被用在一个表列、一个域或一个函数（作为参数或返回类型）就会生成一个数据类型描述符并且在那个实例中存储一些有关该数据类型如何被使用的信息（例如，声明的最大长度，如果适用）。每一个数据类型描述符被赋予一个任意的标识符，它在被赋予给一个对象（表、域、函数）的数据类型描述符中唯一。这个视图对于应用可能没什么用，但是它被用于定义信息模式中的一些其他视图。

表 1.4.111: data_type_privileges 列

名称	数据类型	描述
object_catalog	sql_identifier	包含该被描述对象的数据库名（总是当前数据库）
object_schema	sql_identifier	包含该被描述对象的模式名
object_name	sql_identifier	该描述对象的名字
object_type	character_data	被描述对象的类型：TABLE（从属于表的一列的数据类型描述符）、DOMAIN （从属于域的数据类型描述符）、ROUTINE（从属于函数的一个参数或返回数据类型的数据类型描述符）
dtd_identifier	sql_identifier	数据类型描述符的标识符，它在同一对象的数据类型描述符之间唯一

1.4.2.18　domain_constraints

视图 `domain_constraints` 包含所有属于当前数据库中定义的域的约束。只有当前用户能访问的那些域才被显示（作为拥有者或具有某些特权）。

表 1.4.112: domain_constraints 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
domain_catalog	sql_identifier	包含该域的数据库名（总是当前数据库）
domain_schema	sql_identifier	包含该域的模式名
domain_name	sql_identifier	该域的名称
is_deferrable	yes_or_no	如果该约束是可延迟的，则为 YES，否则为 NO
initially_deferred	yes_or_no	如果该约束是可延迟的且初始就被延迟，则为 YES，否则为 NO

1.4.2.19　domain_udt_usage

视图 `domain_udt_usage` 标识所有基于被一个当前启用的角色拥有的数据类型的域。注意在 KingbaseES 中，内建数据类型的行为相似于用户定义的类型，因此它们也被包括在这里。

表 1.4.113: domain_udt_usage 列

名称	数据类型	描述
udt_catalog	sql_identifier	该域数据类型被定义的数据库名（总是当前数据库）
udt_schema	sql_identifier	该域数据类型被定义的模式名

-续表-

名称	数据类型	描述
udt_name	sql_identifier	该域数据类型的名称
domain_catalog	sql_identifier	包含该域的数据库名（总是当前数据库）
domain_schema	sql_identifier	包含该域的模式名
domain_name	sql_identifier	该域的名称

1.4.2.20　domains

视图 `domains` 包含定义在当前数据库中的所有域。只有当前用户能够访问（作为拥有者或具有某些特权）的域才被显示。

表 1.4.114: domains 列

名称	数据类型	描述
domain_catalog	sql_identifier	包含该域的数据库名（总是当前数据库）
domain_schema	sql_identifier	包含该域的模式名
domain_name	sql_identifier	该域的名称
data_type	character_data	该域的数据类型如果是一种内建类型，这里是该域的数据类型；如果是某种数组（此种情况见视图 element_types），则为 ARRAY；否则为 USER-DEFINED （此种情况中，该类型被标识在 udt_name 和相关列中）
character_maximum_length	cardinal_number	如果该域有一个字符或位串类型，这里是声明的最大长度；如果没有声明最大长度，则对于所有其他数据类型为空
character_octet_length	cardinal_number	如果该域有一个字符类型，这里是一个数据的最大可能长度（以字节计）；对其他所有数据类型为空。最大字节长度取决于声明的字符最大长度（见上文）和服务器编码
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	包含此域排序规则的数据库名（总是当前数据库），如果默认或者该域的数据类型不可排序则为空
collation_schema	sql_identifier	包含此域排序规则的模式名，如果默认或者该域的数据类型不可排序则为空
collation_name	sql_identifier	该域排序规则的名称，如果默认或者该域的数据类型不可排序则为空
numeric_precision	cardinal_number	如果该域有一种数字类型，这列包含这个域类型的（声明的或隐式的）精度。精度指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空
numeric_precision_radix	cardinal_number	如果该域有一种数字类型，这一列指示 numeric_precision 和 numeric_scale 列中的值是基于什么来表示。该值为 2 或 10。对于所有其他数据类型，这一列为空
numeric_scale	cardinal_number	如果该域有一种准确数字类型，这列包含这个域类型的（声明的或隐式的）比例。比例指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）。对于所有其他数据类型，这一列为空
datetime_precision	cardinal_number	如果 data_type 标识一种日期、时间、时间戳或时间间隔类型，这一列包含这个域类型的（声明的或隐式的）分数秒的精度，也就是秒值的小数点后的十进制位数。对于所有其他数据类型，这一列为空

-续表-

名称	数据类型	描述
interval_type	character_data	如果 data_type 标识一种时间间隔类型，这一列包含时间间隔为这个域包括哪些域的声明，例如 YEAR TO MONTH 、DAY TO SECOND 等等。如果没有指定域限制（也就是该时间间隔接受所有域），并且对于所有其他数据类型，这个域为空
interval_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性（关于时间间隔类型域的分数秒精度可见 datetime_precision）
domain_default	character_data	该域的默认表达式
udt_catalog	sql_identifier	该域数据类型被定义的数据库名（总是当前数据库）
udt_schema	sql_identifier	该域数据类型被定义的模式名
udt_name	sql_identifier	该域数据类型的名称
scope_catalog	sql_identifier	应用于一个 King baseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 King baseES 中不可用的特性
scope_name	sql_identifier	应用于一个 King baseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大势
dtd_identifier	sql_identifier	该域的数据类型描述符的一个标识符，在从属于该域的数据类型标识符之中唯一（这不重要，因为一个域只包含一个数据类型描述符）。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）

1.4.2.21　element_types

视图 `element_types` 包含数组元素的数据类型描述符。当一个表列、组合类型属性、域、函数参数或函数返回值被定义为一种数组类型，相应的信息模式视图只在列 `data_type` 中包含 `ARRAY`。要获得该数组元素类型的信息，

你可以连接该相应的视图和这个视图。例如，要显示一个表的列及其数据类型和数组元素类型，你可以：

```
SELECT c.column_name, c.data_type, e.data_type AS element_type
FROM information_schema.columns c LEFT JOIN information_schema.element_types e
ON ((c.table_catalog, c.table_schema, c.table_name, 'TABLE', c.dtd_identifier)
= (e.object_catalog, e.object_schema, e.object_name, e.object_type,
e.collection_type_identifier))
WHERE c.table_schema = '...' AND c.table_name = '...'
ORDER BY c.ordinal_position;
```

这个视图只包括当前用户能够访问（作为拥有者或具有某些特权）的对象。

表 1.4.115: element_types 列

名称	数据类型	描述
object_catalog	sql_identifier	包含使用被描述的数组的对象的数据库名（总是当前数据库）
object_schema	sql_identifier	包含使用被描述的数组的对象的模式名
object_name	sql_identifier	使用被描述的模式的对象名
object_type	character_data	使用被描述的数组的对象的类型：TABLE（被一个表列使用的数组）、USER-DEFINED TYPE（被组合类型的一个属性使用的数组）、DOMAIN （被域使用的数组）、ROUTINE（被函数的一个参数或返回数据类型使用的数组）
collection_type_identifier	sql_identifier	被描述的数组的数据类型描述符的标识符。使用这个去与其他信息模式视图的 dtd_identifier 列连接
data_type	character_data	如果数组元素的数据类型是内建类型，这里是数组元素的数据类型，否则为 USER-DEFINED（在那种情况下，该类型被标识在 udt_name 和相关列中）
character_maximum_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
character_octet_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	包含元素类型排序规则的数据库名（总是当前数据库），如果默认或该元素的数据类型是不可排序的则为空
collation_schema	sql_identifier	包含元素类型排序规则的模式名，如果默认或该元素的数据类型是不可排序的则为空
collation_name	sql_identifier	元素类型的排序规则名，如果默认或该元素的数据类型是不可排序的则为空
numeric_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
numeric_precision_radix	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
numeric_scale	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
datetime_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
interval_type	character_data	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
interval_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的数组元素数据类型
domain_default	character_data	还未被实现
udt_catalog	sql_identifier	元素的数据类型所在的数据库名（总是当前数据库）
udt_schema	sql_identifier	元素的数据类型所在的模式名
udt_name	sql_identifier	模式的数据类型名
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中拥有无限制的基数

-续表-

名称	数据类型	描述
dtd_identifier	sql_identifier	该元素的数据类型描述符的标识符。当前无用

1.4.2.22　enabled_roles

视图 `enabled_roles` 标识当前“已被启用的角色”。已被启用的角色被递归地定义为：当前用户以及被授予给具有自动继承的已被启用角色的所有角色。换句话说，就是当前用户是其直接或间接成员的所有角色。

为了权限检查，“可应用角色”的集合被应用，它会比已被启用角色的集合包含的角色范围更宽。因此通常使用视图 `applicable_roles` 要更好，`applicable_roles` 视图的详情请见applicable_roles 。

表 1.4.116: enabled_roles 列

名称	数据类型	描述
role_name	sql_identifier	角色名称

1.4.2.23　foreign_data_wrapper_options

视图 `foreign_data_wrapper_options` 包含当前数据库中外部数据包装器定义的所有选项。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部数据包装器被显示。

表 1.4.117: foreign_data_wrapper_options 列

名称	数据类型	描述
foreign_data_wrapper_catalog	sql_identifier	该外部数据包装器所在的数据库名（总是当前数据库）
foreign_data_wrapper_name	sql_identifier	该外部数据包装器的名字
option_name	sql_identifier	一个选项名
option_value	character_data	该选项的值

1.4.2.24　foreign_data_wrappers

视图 `foreign_data_wrappers` 包含定义在当前数据库中的所有外部数据包装器。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部数据包装器才会被显示。

表 1.4.118: foreign_data_wrappers 列

名称	数据类型	描述
foreign_data_wrapper_catalog	sql_identifier	包含该外部数据包装器的数据库名（总是当前数据库）
foreign_data_wrapper_name	sql_identifier	外部数据包装器的名字
authorization_identifier	sql_identifier	外部服务器拥有者的名字
library_name	character_data	实现这个外部数据包装器的库文件名
foreign_data_wrapper_language	character_data	用于实现这个外部数据包装器的语言

1.4.2.25　foreign_server_options

视图 `foreign_server_options` 包含当前数据库中外部服务器定义的所有选项。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部服务器才会被显示。

表 1.4.119: foreign_server_options 列

名称	数据类型	描述
foreign_server_catalog	sql_identifier	该外部服务器所在的数据库名（总是当前数据库）
foreign_server_name	sql_identifier	该外部服务器的名字
option_name	sql_identifier	一个选项名
option_value	character_data	该选项的值

1.4.2.26　foreign_servers

视图 `foreign_servers` 包含当前数据库中定义的所有外部服务器。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部服务器才会被显示。

表 1.4.120: foreign_servers 列

名称	数据类型	描述
foreign_server_catalog	sql_identifier	该外部服务器所在的数据库名（总是当前数据库）
foreign_server_name	sql_identifier	该外部服务器的名字
foreign_data_wrapper_catalog	sql_identifier	包含被该外部服务器使用的外部数据包装器的数据库名（总是当前数据库）
foreign_data_wrapper_name	sql_identifier	被该外部服务器所使用的外部数据包装器的名字
foreign_server_type	character_data	外部服务器类型信息（如果在创建时指定过）
foreign_server_version	character_data	外部服务器版本信息（如果在创建时指定过）
authorization_identifier	sql_identifier	该外部服务器的拥有者名字

1.4.2.27　foreign_table_options

视图 `foreign_table_options` 包含当前数据库中外部表定义的所有选项。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部表才会被显示。

表 1.4.121: foreign_table_options 列

名称	数据类型	描述
foreign_table_catalog	sql_identifier	包含该外部表的数据库名（总是当前数据库）
foreign_table_schema	sql_identifier	包含该外部表的模式名
foreign_table_name	sql_identifier	外部表的名字
option_name	sql_identifier	一个选项名

-续表-

名称	数据类型	描述
option_value	character_data	该选项的值

1.4.2.28　foreign_tables

视图 `foreign_tables` 包含定义在当前数据库中的所有外部表。只有那些当前用户能够访问（作为拥有者或具有某些特权）的外部表才会被显示。

表 1.4.122: foreign_tables 列

名称	数据类型	描述
foreign_table_catalog	sql_identifier	该外部表所在的数据库名（总是当前数据库）
foreign_table_schema	sql_identifier	包含该外部表的模式名
foreign_table_name	sql_identifier	该外部表的名字
foreign_server_catalog	sql_identifier	该外部服务器所在的数据库名（总是当前数据库）
foreign_server_name	sql_identifier	该外部服务器的名字

1.4.2.29　模式

信息模式本身是一个名为 `information_schema` 的模式。这个模式自动存在于所有数据库中。这个模式的拥有者是集簇中的初始数据库用户，并且该用户默认拥有这个模式上的所有特权，包括删除它的能力（但是这样节省的空间是很小的）。

默认情况下，信息模式不在模式搜索路径中，因此你需要使用限定名访问其中的所有对象。因为信息模式中的某些对象的名称是可能出现在用户应用中的一般名称，若将该信息模式放在 search_path 中，使用用户定义对象时请加上模式限定。

1.4.2.30　information_schema_catalog_name

`information_schema_catalog_name` 是一个表，它总是包含一行和一列，其中包含了当前数据库（SQL 术语中的当前目录）的名字。

表 1.4.123: information_schema_catalog_name 列

名称	数据类型	描述
catalog_name	sql_identifier	包含这个信息模式的数据库名

1.4.2.31　数据类型

信息模式视图的列使用定义在信息模式中的特殊数据类型。它们被定义为普通内建类型之上的简单类型，一般不建议在信息模式之外使用这些类型。

这些类型是：

`cardinal_number`
一种非负整数。

`character_data`
一种字符串（没有指定最大长度）。

`sql_identifier`
一种字符串。这种类型被用于 SQL 标识符，类型 `character_data` 被用于任何其他类型的文本数据。

`time_stamp`
在类型 `timestamp with time zone` 之上的一个域。

`yes_or_no`
一种字符串域，它包含 `YES` 或 `NO`。这被用来在信息模式中表示布尔（真/假）（信息模式是在类型 `boolean` 被加到 SQL 标准之前被发明的，因此这个惯例是用来使信息模式向后兼容）。

信息模式中的每一列都是这五种类型之一。

1.4.2.32　key_column_usage

视图 `key_column_usage` 标识当前数据库中所有被某种唯一、主键或外键约束限制的列。检查约束不被包括在这个视图中。只有那些当前用户能够访问的列才会被显示（作为拥有者或具有某些特权）。

表 1.4.124: key_column_usage 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
table_catalog	sql_identifier	包含被这个约束限制的列的表所在的数据库名（总是当前数据库）
table_schema	sql_identifier	包含被这个约束限制的列的表所在的模式名
table_name	sql_identifier	包含被这个约束限制的列的表的名字
column_name	sql_identifier	被这个约束限制的列名
ordinal_position	cardinal_number	该列在约束键中的顺序位置（从 1 开始计数）
position_in_unique_constraint	cardinal_number	对于一个外键约束，被引用行在其唯一约束中的顺序位置（从 1 开始计数）；对于其他约束为空

1.4.2.33　parameters

视图 `parameters` 包含当前数据库中所有函数的参数的有关信息。只有那些当前用户能够访问（作为拥有者或具有某些特权）的函数才会被显示。

表 1.4.125: parameters 列

名称	数据类型	描述
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”。详见 routines

-续表-

名称	数据类型	描述
ordinal_position	cardinal_number	该参数在函数参数列表中的顺序位置（从 1 开始计数）
parameter_mode	character_data	IN 表示输入参数，OUT 表示输出参数，INOUT 表示输入输出参数
is_result	yes_or_no	应用于一个 KingbaseES 中不可用的特性
as_locator	yes_or_no	应用于一个 KingbaseES 中不可用的特性
parameter_name	sql_identifier	参数名，如果参数没有名称则为空
data_type	character_data	该参数的数据类型如果是一种内建类型，这里是该参数的数据类型；如果是某种数组（此种情况见视图 element_types），则为 ARRAY；否则为 USER-DEFINED（此种情况中，该类型被标识在 udt _name 和相关列中）
character_maximum_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_octet_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_schema	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_name	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
numeric_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
numeric_precision_radix	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型

-续表-

名称	数据类型	描述
numeric_scale	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
datetime_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_type	character_data	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
udt_catalog	sql_identifier	该参数的数据类型所在的数据库名（总是当前数据库）
udt_schema	sql_identifier	该参数的数据类型所在的模式名
udt_name	sql_identifier	该参数的数据类型的名字
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大值
dtd_identifier	sql_identifier	该参数的数据类型描述符的一个标识符，在从属于该函数的数据类型标识符之中唯一（这不重要，因为一个域只包含一个数据类型描述符）。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）
parameter_default	character_data	该参数的默认表达式，如果没有或者该函数不被一个当前启用的角色拥有则为空值

1.4.2.34　triggers

视图 `triggers` 包含所有定义在当前数据库中表和视图上的触发器，并且只显示当前用户拥有的触发器或者是当前用户在其上具有某种除 `SELECT` 之外特权的触发器。

表 1.4.126: triggers 列

名称	数据类型	描述
trigger_catalog	sql_identifier	包含该触发器的数据库名（总是当前数据库）
trigger_schema	sql_identifier	包含该触发器的模式名
trigger_name	sql_identifier	该触发器的名称
event_manipulation	character_data	触发该触发器的事件（INSERT、UPDATE 或 DELETE）
event_object_catalog	sql_identifier	包含触发器所在的表的数据库名（总是当前数据库）
event_object_schema	sql_identifier	包含该触发器所在的表的模式名
event_object_table	sql_identifier	该触发器所在的表的名称
action_order	cardinal_number	同一个表上具有相同 event_manipulation、action_timing 和 action_orientation 的触发器之间的触发顺序。在 KingbaseES 中，触发器按照名称顺序被触发，因此这一列会反映这种规则
action_condition	character_data	触发器的 WHEN 条件，如果没有则为空（如果该表不被一个当前已启用角色拥有也是为空）
action_statement	character_data	该触发器执行的语句（当前总是 EXECUTE FUNCTION function(...)）
action_orientation	character_data	标识触发器是对每个被处理的行触发一次还是为每个语句触发一次（ROW 或 STATEMENT）
action_timing	character_data	触发器在什么时候触发（BEFORE、AFTER 或 INSTEAD OF）
action_reference_old_table	sql_identifier	“旧”传递表的名称，如果没有则为空

-续表-

名称	数据类型	描述
action_reference_new_table	sql_identifier	“新”传递表的名称，如果没有则为空
action_reference_old_row	sql_identifier	应用于一个 KingbaseES 中不可用的特性
action_reference_new_row	sql_identifier	应用于一个 KingbaseES 中不可用的特性
created	time_stamp	应用于一个 KingbaseES 中不可用的特性

KingbaseES 中的触发器有两点与 SQL 标准不兼容，这会影响在该信息模式中的表示。第一，在 KingbaseES 中触发器的名字是局限于每个表的，而不是独立于模式对象。因此可能在一个模式中会有重复的触发器名称，只要它们属于不同的表（`trigger_catalog` 和 `trigger_schema` 才真正标识了触发器被定义在哪个表上）。第二，在 KingbaseES 中触发器可以被定义为在多个事件上触发（例如 `ON INSERT OR UPDATE`），而在 SQL 标准中只允许一个。如果一个触发器被定义为在多个事件上触发，它在信息模式中被表示为多行，每一行对应于一类事件。作为这两个问题的结果，视图 `triggers` 的主键实际上是 (`trigger_catalog`, `trigger_schema`, `event_object_table`, `trigger_name`, `event_manipulation`)，而不是 (`trigger_catalog`, `trigger_schema`, `trigger_name`)（这是 SQL 标准指定的）。尽管如此，如果你以符合 SQL 标准（在模式中触发器名称唯一并且每个触发器只能有一种事件类型）的方式定义你的触发器，这将不会影响你。

备注:

在 KingbaseES V8R2 之前，这个视图的列 `action_timing`、`action_reference_old_table`、`action_reference_new_table`、`action_reference_old_row` 和 `action_reference_new_row` 分别被命名为 `condition_timing`、`condition_reference_old_table`、`condition_reference_new_table`、`condition_reference_old_row` 和 `condition_reference_new_row`。那也是它们在 SQL:1999 标准中的命名。新的命名遵循 SQL:2003 及其后的版本。

1.4.2.35　referential_constraints

视图 `referential_constraints` 包含当前数据库中的所有引用（外键）约束。只有那些当前用户具有其引用表上写权限（作为拥有者或具有某些除 `SELECT` 之外的特权）的约束才会被显示。

表 1.4.127: referential_constraints 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
unique_constraint_catalog	sql_identifier	包含该外键约束所引用的唯一或主键约束的数据库名（总是当前数据库）
unique_constraint_schema	sql_identifier	包含该外键约束所引用的唯一或主键约束的模式名
unique_constraint_name	sql_identifier	包含该外键约束所引用的唯一或主键约束的名字
match_option	character_data	外键约束的匹配选项：FULL、PARTIAL 或 NONE
update_rule	character_data	外键约束的更新规则：CASCADE、SET NULL、SET DEFAULT、RESTRICT 或 NO ACTION
delete_rule	character_data	外键约束的删除规则：CASCADE、SET NULL、SET DEFAULT、RESTRICT 或 NO ACTION

1.4.2.36　role_column_grants

视图 `role_column_grants` 标识所有在列上授予的特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。更多信息可以在 `column_privileges` 中找到。这个视图和 `column_privileges` 之间的唯一实质性区别是：这个视图忽略那些以授予给 `PUBLIC` 的方式使当前用户获得其访问权限的列。

表 1.4.128: role_column_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
table_catalog	sql_identifier	包含该列的表所在的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该列的表所在的模式名
table_name	sql_identifier	包含该列的表名
column_name	sql_identifier	列名称
privilege_type	character_data	特权类型：SELECT、INSERT、UPDATE 或 REFERENCES
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.37　role_package_grants

视图 `role_package_grants` 可查看启用角色中具有查询包的执行权限。

表 1.4.129: role_package_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予者
grantee	sql_identifier	被授予者
owner	varcharbyte	包的 owner
package_catalog	sql_identifier	包所在的数据库
package_schema	sql_identifier	包所在的模式
package_name	sql_identifier	包名
privilege_type	character_data	包的执行权限
is_grantable	yes_or_no	是否可授予包权限

1.4.2.38　role_routine_grants

视图 `role_routine_grants` 标识所有在函数上授予的特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。更多信息可以在 `routine_privileges` 中找到。这个视图和 `routine_privileges` 之间的唯一实质性区别是：这个视图忽略那些以授予给 `PUBLIC` 的方式使当前用户获得其访问权限的函数。

表 1.4.130: role_routine_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”
routine_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
routine_schema	sql_identifier	包含该函数的模式名
routine_name	sql_identifier	该函数的名字（在重载的情况下可能会重复）
privilege_type	character_data	总是为 EXECUTE（函数唯一的特权类型）
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.39　role_table_grants

视图 `role_table_grants` 标识所有在表或视图上授予的特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。更多信息可以在 `table_privileges` 中找到。这个视图和 `table_privileges` 之间的唯一实质性区别是：这个视图忽略那些以授予给 `PUBLIC` 的方式使当前用户获得其访问权限的表。

表 1.4.131: role_table_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
privilege_type	character_data	该特权的类型：SELECT、INSERT、UPDATE、DELETE、TRUNCATE、REFERENCES 或 TRIGGER
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO
with_hierarchy	yes_or_no	在 SQL 标准中，WITH HIERARCHY OPTION 是一个独立的（子）特权，它允许在表继承层级上的特定操作。在 KingbaseES 中，这被包括在 SELECT 特权中，因此这一列在特权为 SELECT 时显示 YES，其他时候显示 NO

1.4.2.40　role_udt_grants

视图 `role_udt_grants` 标识所有在用户定义类型上授予的 `USAGE` 特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。更多信息可以在 `udt_privileges` 中找到。这个视图和 `udt_privileges` 之间的唯一实质性区别是：这个视图忽略那些以授予给 `PUBLIC` 的方式使当前用户获得其访问权限的对象。因为数据类型在 KingbaseES 中并没有真正的特权，而是只有一个给 `PUBLIC` 的隐式授予，这个视图为空。

表 1.4.132: role_udt_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名

-续表-

名称	数据类型	描述
udt_catalog	sql_identifier	包含该类型的数据库名（总是当前数据库）
udt_schema	sql_identifier	包含该类型的模式名
udt_name	sql_identifier	该类型的名字
privilege_type	character_data	总是 TYPE USAGE
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.41　role_usage_grants

视图 `role_usage_grants` 标识所有在多种对象上授予的 `USAGE` 特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。更多信息可以在 `usage_privileges` 中找到。这个视图和 `usage_privileges` 之间的唯一实质性区别是：这个视图忽略那些以授予给 `PUBLIC` 的方式使当前用户获得其访问权限的对象。

表 1.4.133: role_usage_grants 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
object_catalog	sql_identifier	包含该对象的数据库名（总是当前数据库）
object_schema	sql_identifier	如果适用，则为包含该对象的模式名，否则为一个空字符串
object_name	sql_identifier	对象的名字
object_type	character_data	COLLATION 或 DOMAIN 或 FOREIGN DATA WRAPPER 或 FOREIGN SERVER 或 SEQUENCE
privilege_type	character_data	总是 USAGE
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.42　routine_privileges

视图 `routine_privileges` 标识所有在函数上授予的特权，其授予者或被授予者是一个当前已被启用的角色。对于每一种函数、授予者和被授予者的组合，这里都有一行。

表 1.4.134: routine_privileges 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”
routine_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
routine_schema	sql_identifier	包含该函数的模式名
routine_name	sql_identifier	该函数的名字（在重载的情况下可能重复）
privilege_type	character_data	总是 EXECUTE（函数唯一的特权类型）
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.43　routines

视图 `routines` 包含当前数据库中所有的函数和过程。只有那些当前用户能够访问（作为拥有者或具有某些特权）的函数和过程才会被显示。

表 1.4.135: routines 列

名称	数据类型	描述
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）

-续表-

名称	数据类型	描述
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”。这是一个在模式中唯一标识该函数的名称，即使该函数真正的名称已经被重载。专用名的格式尚未被定义，它应当仅被用来与指定例程名称的其他实例进行比较
routine_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
routine_schema	sql_identifier	包含该函数的模式名
routine_name	sql_identifier	该函数的名字（在重载的情况下可能重复）
routine_type	character_data	FUNCTION 表示是一个函数，PROCEDURE 表示是一个过程
module_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
module_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
module_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
data_type	character_data	该函数的返回数据类型如果是一种内建类型，这里是该数据类型；如果是某种数组（此种情况见视图 element_types），则为 ARRAY；否则为 USER-DEFINED（此种情况中，该类型被标识在 type_udt_name 和相关列中）。如果是过程则此项为空
character_maximum_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_octet_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_schema	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_name	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
numeric_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型

-续表-

名称	数据类型	描述
numeric_precision_radix	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
numeric_scale	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
datetime_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_type	character_data	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
type_udt_catalog	sql_identifier	该函数的返回数据类型所在的数据库名（总是当前数据库）。如果是过程则此项为空
type_udt_schema	sql_identifier	该函数的返回数据类型所在的模式名。如果是过程则此项为空
type_udt_name	sql_identifier	该函数的返回数据类型的名字。如果是过程则此项为空
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大势

-续表-

名称	数据类型	描述
dtd_identifier	sql_identifier	该函数返回数据类型的数据类型描述符的一个标识符，在从属于该函数的数据类型标识符之中唯一。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）
routine_body	character_data	如果该函数是一个 SQL 函数，则为 SQL，否则为 EXTERNAL
routine_definition	character_data	该函数的源文本（如果该函数不属于一个当前已被启用的角色，则为空）。（根据 SQL 标准，只有 routine_body 为 SQL 时这一列才适用。但是在 KingbaseES 中，它将会包含该函数被创建时所指定的任何源文本。）
external_name	character_data	如果这个函数是一个 C 函数，则为该函数的外部名称（链接符号），否则为空（这会产生和显示在 routine_definition 中相同的值）
external_language	character_data	该函数所用的语言
parameter_style	character_data	总是 GENERAL（SQL 标准定义了其他参数风格，但在 KingbaseES 中不可用）
is_deterministic	yes_or_no	如果该函数被声明为不变（在 SQL 标准中被称为确定性的），则为 YES，否则为 NO（你不能通过该信息模式查询在 KingbaseES 中可用的其他易变级别）

-续表-

名称	数据类型	描述
sql_data_access	character_data	总是 MODIFIES，表示该函数可能修改 SQL 数据。这种信息对 KingbaseES 没有用处
is_null_call	yes_or_no	如果该函数在任一参数为空时自动返回空值，则为 YES，否则为 NO。如果是过程则此项为空
sql_path	character_data	应用于一个 KingbaseES 中不可用的特性
schema_level_routine	yes_or_no	总是 YES（反例是一个用户定义类型的方法，这是在 KingbaseES 不可用的一种特性）
max_dynamic_result_sets	cardinal_number	应用于一个 KingbaseES 中不可用的特性
is_user_defined_cast	yes_or_no	应用于一个 KingbaseES 中不可用的特性
is_implicitly_invocable	yes_or_no	应用于一个 KingbaseES 中不可用的特性
security_type	character_data	如果该函数以当前用户的特权运行，则为 INVOKER；如果该函数以定义它的用户的特权运行，则为 DEFINER
to_sql_specific_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
to_sql_specific_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
to_sql_specific_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
as_locator	yes_or_no	应用于一个 KingbaseES 中不可用的特性
created	time_stamp	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
last_altered	time_stamp	应用于一个 KingbaseES 中不可用的特性
new_savepoint_level	yes_or_no	应用于一个 KingbaseES 中不可用的特性
is_udt_dependent	yes_or_no	当前总是 NO。另一个选项 YES 应用于一个 KingbaseES 中不可用的特性
result_cast_from_data_type	character_data	应用于一个 KingbaseES 中不可用的特性
result_cast_as_locator	yes_or_no	应用于一个 KingbaseES 中不可用的特性
result_cast_char_max_length	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_char_octet_length	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_char_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_char_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_char_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_collation_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_collation_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_collation_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_numeric_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_numeric_precision_radix	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_numeric_scale	cardinal_number	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
result_cast_datetime_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_interval_type	character_data	应用于一个 KingbaseES 中不可用的特性
result_cast_interval_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_type_udt_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_type_udt_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_type_udt_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
result_cast_maximum_cardinality	cardinal_number	应用于一个 KingbaseES 中不可用的特性
result_cast_dtd_identifier	sql_identifier	应用于一个 KingbaseES 中不可用的特性

1.4.2.44　schemata

视图 `schemata` 包含当前数据库中被当前用户（作为属主或具有某些特权）可访问的所有模式。

表 1.4.136: schemata 列

名称	数据类型	描述
catalog_name	sql_identifier	该模式所在的数据库名（总是当前数据库）

-续表-

名称	数据类型	描述
schema_name	sql_identifier	该模式的名称
schema_owner	sql_identifier	该模式拥有者的名称
default_character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
default_character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
default_character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
sql_path	character_data	应用于一个 KingbaseES 中不可用的特性

1.4.2.45　sequences

视图 `sequences` 包含所有定义在当前数据库中的序列。只有那些当前用户能够访问（作为拥有者或具有某些特权）的序列才会被显示。

表 1.4.137: sequences 列

名称	数据类型	描述
sequence_catalog	sql_identifier	包含该序列的数据库名（总是当前数据库）
sequence_schema	sql_identifier	包含该序列的模式名
sequence_name	sql_identifier	该序列的名字
data_type	character_data	该序列的数据类型 |
numeric_precision	cardinal_number	这列包含这个序列数据类型（见上文）的（声明的或隐式的）精度。精度指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）|

-续表-

名称	数据类型	描述
numeric_precision_radix	cardinal_number	这一列指示 numeric_precision 和 numeric_scale 列中的值是基于什么来表示。该值为 2 或 10 |
numeric_scale	cardinal_number	这列包含这个序列数据类型（见上文）的（声明的或隐式的）比例。比例指示了有效位数。它可以按照列 numeric_precision_radix 中指定的被表示为十进制（基于 10）或二进制（基于 2）|
start_value	character_data	该序列的开始值
minimum_value	character_data	该序列的最小值
maximum_value	character_data	该序列的最大值
increment	character_data	该序列的增量
cycle_option	yes_or_no	如果该序列会循环，则为 YES，否则为 NO

注意依照 SQL 标准，开始值、最小值、最大值和增量值被作为字符串返回。

1.4.2.46　sql_features

表 `sql_features` 包含的信息指示了哪些 SQL 标准中定义的正式特性被 KingbaseES 所支持。

表 1.4.138: sql_features 列

名称	数据类型	描述
feature_id	character_data	该特性的标识符字符串
feature_name	character_data	该特性的描述性名称
sub_feature_id	character_data	该子特性的标识符字符串，或者如果不是一个子特性则为一个长度为零的字符串
sub_feature_name	character_data	该子特性的描述性名称，或者如果不是一个子特性则为一个长度为零的字符串

-续表-

名称	数据类型	描述
is_supported	yes_or_no	如果该特性被当前版本的 KingbaseES 完全支持，则为 YES，否则为 NO
is_verified_by	character_data	总是为空，因为 KingbaseES 开发组没有对特性的一致性执行正式的测试
comments	character_data	可能会是关于该特性被支持状态的一段注释

1.4.2.47　sql_implementation_info

表 `sql_implementation_info` 包含的信息指示剩下的由 SQL 标准实现定义的多个接口。这类信息主要用来在 ODBC 接口的情境中使用；其它接口的用户可能将发现这类信息用处不大。

表 1.4.139: sql_implementation_info 列

名称	数据类型	描述
implementation_info_id	character_data	该实现信息项的标识符字符串
implementation_info_name	character_data	该实现信息项的描述性名称
integer_value	cardinal_number	该实现信息项的值，如果该值被包含在 character_value 列中则为空
character_value	character_data	该实现信息项的值，如果该值被包含在 integer_value 列中则为空
comments	character_data	可能是从属于该实现信息项的一段注释

1.4.2.48　sql_languages

表 `sql_languages` 为每一种被 KingbaseES 支持的 SQL 语言绑定包含一行。KingbaseES 支持在 C 中的直接 SQL 和嵌入式 SQL，这是你从这张表中知道的所有东西。

这个表在 SQL:2008 中已被从 SQL 标准中移除，因此这里没有项引用 SQL:2003 之后的标准。

表 1.4.140: sql_languages 列

名称	数据类型	描述
sql_language_source	character_data	该语言定义的源名称，总是 ISO 9075，即 SQL 标准
sql_language_year	character_data	sql_language_source 中引用的标准被通过的年份。
sql_language_conformance	character_data	该语言绑定的标准一致性级别。对于 ISO 9075:2003 总是 CORE
sql_language_integrity	character_data	总是为空（这个值与一个早期版本的 SQL 标准相关）
sql_language_implementation	character_data	总是为空
sql_language_binding_style	character_data	语言绑定风格，为 DIRECT 或 EMBEDDED
sql_language_programming _language	character_data	如果绑定风格为 EMBEDDED，则为编程语言，否则为空。KingbaseES 仅支持 C 语言

1.4.2.49　sql_packages

表 `sql_packages` 包含的信息指示哪些定义在 SQL 标准中的特性包被 KingbaseES 支持。

表 1.4.141: sql_packages 列

名称	数据类型	描述
feature_id	character_data	该包的标识符字符串
feature_name	character_data	该包的描述性名称
is_supported	yes_or_no	如果该包被当前版本的 KingbaseES 支持，则为 YES，否则为 NO
is_verified_by	character_data	总是为空，因为 KingbaseES 开发组没有对特性的一致性执行正式的测试
comments	character_data	可能会是关于该包被支持状态的一段注释

1.4.2.50　sql_parts

表 `sql_parts` 包含的信息指示哪些定义在 SQL 标准中的部分被 KingbaseES 支持。

表 1.4.142: sql_parts 列

名称	数据类型	描述
feature_id	character_data	包含该部分编号的一个标识符字符串
feature_name	character_data	该部分的描述性名称
is_supported	yes_or_no	如果当前版本的 KingbaseES 完全支持该部分，则为 YES，否则为 NO
is_verified_by	character_data	总是为空，因为 KingbaseES 开发组没有对特性的一致性执行正式的测试
comments	character_data	可能会是关于该部分被支持状态的一段注释

1.4.2.51　sql_sizing

表 `sql_sizing` 包含有关 KingbaseES 中多种尺寸限制和最大值的信息。这类信息主要用来在 ODBC 接口的情境中使用；其它接口的用户可能将发现这类信息用处不大。

表 1.4.143: sql_sizing 列

名称	数据类型	描述
sizing_id	cardinal_number	该尺寸项的标识符
sizing_name	character_data	该尺寸项的描述性名称
supported_value	cardinal_number	尺寸项的值，如果尺寸是不受限制或不能确定的则为 0，如果尺寸项适用的特性不受支持则为空
comments	character_data	可能是从属于尺寸项的一段注释

1.4.2.52　sql_sizing_profiles

表 `sql_sizing_profiles` 包含有关 SQL 标准的多种 profile 所需的 `sql_sizing` 值的信息。KingbaseES 不追踪任何 SQL profile，因此这个表为空。

表 1.4.144: sql_sizing_profiles 列

名称	数据类型	描述
sizing_id	cardinal_number	该尺寸项的标识符
sizing_name	character_data	该尺寸项的描述性名称
profile_id	character_data	一个 profile 的标识符字符串
required_value	cardinal_number	该 SQL profile 对尺寸项所要求的值，如果该 profile 对尺寸项没有限制则为 0，如果该 profile 不要求该尺寸项所适用的任何特性则为空
comments	character_data	可能是从属于该 profile 中尺寸项的一段注释

1.4.2.53　table_constraints

视图 `table_constraints` 包含属于特定表的所有约束，这些表要满足的条件是：当前用户拥有表或者是当前用户在表上具有某种除 `SELECT` 之外的特权。

表 1.4.145: table_constraints 列

名称	数据类型	描述
constraint_catalog	sql_identifier	包含该约束的数据库名（总是当前数据库）
constraint_schema	sql_identifier	包含该约束的模式名
constraint_name	sql_identifier	约束名
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称

-续表-

名称	数据类型	描述
constraint_type	character_data	该约束的类型：CHECK、FOREIGNKEY、PRIMARYKEY 或 UNIQUE
is_deferrable	yes_or_no	如果该约束是可延迟的，则为 YES，否则为 NO
initially_deferred	yes_or_no	如果该约束是可延迟的并且是初始已被延迟，则为 YES，否则为 NO
enforced	yes_or_no	适用于一种 KingbaseES 中不可用的特性（当前总是 YES）

1.4.2.54　table_privileges

视图 `table_privileges` 标识在表或视图上所有被授予的特权，这些特权必须是被一个当前已被启用角色授出或者被授予给一个当前已被启用角色。对每一个表、授予者和被授予者的组合都有一行。

表 1.4.146: table_privileges 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
privilege_type	character_data	特权类型：SELECT、INSERT、UPDATE、DELETE、TRUNCATE、REFERENCES 或 TRIGGER
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

-续表-

名称	数据类型	描述
with_hierarchy	yes_or_no	在 SQL 标准中，WITH HIERARCHY OPTION 是一个独立的（子）特权，它允许在表继承层级上的特定操作。在 KingbaseES 中，这被包括在 SELECT 特权中，因此这一列在特权为 SELECT 时显示 YES，其他时候显示 NO

1.4.2.55　tables

视图 `tables` 包含定义在当前数据库中的所有表和视图。只有那些当前用户能够访问（作为拥有者或具有某些特权）的表和视图才会被显示。

表 1.4.147: tables 列

名称	数据类型	描述
table_catalog	sql_identifier	包含该表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该表的模式名
table_name	sql_identifier	表名称
table_type	character_data	该表的类型：BASE TABLE 表示一个持久的基本表（常见表类型），VIEW 表示一个视图，FOREIGN 表示一个外部表，LOCAL TEMPORARY 表示一个临时表
self_referencing_column_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
reference_generation	character_data	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
user_defined_type_catalog	sql_identifier	如果该表是一个有类型的表，则是包含其底层数据类型的数据库名（总是当前数据库），否则为空
user_defined_type_schema	sql_identifier	如果该表是一个有类型的表，则是包含其底层数据类型的模式名，否则为空
user_defined_type_name	sql_identifier	如果该表是一个有类型的表，则是其底层数据类型的名称，否则为空
is_insertable_into	yes_or_no	如果该表能够被插入，则为 YES，否则为 NO（基本表总是能被插入，而视图则不一定）
is_typed	yes_or_no	如果该表是一个有类型的表，则为 YES，否则为 NO
commit_action	character_data	还未被实现

1.4.2.56　transforms

视图 `transforms` 包含定义在当前数据库中的转换的信息。更准确来说，包含在转换中的每一个函数（“FROM SQL”或者“TOSQL”函数）在其中都有一行。

表 1.4.148: transforms 列

名称	数据类型	描述
udt_catalog	sql_identifier	包含该转换所适用类型的数据库的名称（总是当前数据库）
udt_schema	sql_identifier	包含该转换所适用类型的模式的名称
udt_name	sql_identifier	该转换所适用类型的名称
specific_catalog	sql_identifier	包含该函数的数据库的名称（总是当前数据库）

-续表-

名称	数据类型	描述
specific_schema	sql_identifier	包含该函数的模式的名称
specific_name	sql_identifier	该函数的“专用名”。更多信息请见 routines
group_name	sql_identifier	SQL 标准允许在“组”中定义转换，并且在运行时选择一个组。KingbaseES 不支持这种做法，转换是与一种语言相关的。作为一种折衷，这个域包含该转换所适用的语言
transform_type	character_data	FROM SQL 或者 TO SQL

1.4.2.57　triggered_update_columns

对于当前数据库中指定一个列列表（如 `UPDATE OF column1, column2`）的触发器，视图 `triggered_update_columns` 标识这些列。没有指定一个列列表的触发器不被包括在这个视图中。只有那些当前用户拥有或具有某种除 `SELECT` 之外特权的列才会被显示。

表 1.4.149: triggered_update_columns 列

名称	数据类型	描述
trigger_catalog	sql_identifier	包含该触发器的数据库名（总是当前数据库）
trigger_schema	sql_identifier	包含该触发器的模式名
trigger_name	sql_identifier	该触发器的名称
event_object_catalog	sql_identifier	包含触发器所在的表的数据库名（总是当前数据库）
event_object_schema	sql_identifier	包含触发器所在的表的模式名
event_object_table	sql_identifier	触发器所在的表的名称
event_object_column	sql_identifier	触发器所在的列的名称

1.4.2.58　udt_privileges

视图 `udt_privileges` 标识所有在用户定义类型上授予的 `USAGE` 特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。对每一个类型、授予者和被授予者的组合都有一行。这个视图只显示组合类型（原因见下面的user_defined_types ）。

域特权见usage_privileges 。

表 1.4.150: udt_privileges 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
udt_catalog	sql_identifier	包含该类型的数据库名（总是当前数据库）
udt_schema	sql_identifier	包含该类型的模式名
udt_name	sql_identifier	该类型的名字
privilege_type	character_data	总是 TYPE USAGE
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.59　usage_privileges

视图 `usage_privileges` 标识所有在多种对象上授予的 `USAGE` 特权，这些特权的授予者或者被授予者是一个当前已被启用的角色。在 KingbaseES 中，这当前适用于排序规则、域、外部数据包装器、外部服务器和序列。对每一个对象、授予者和被授予者都有一行。

由于在 KingbaseES 中排序规则并没有真正的特权，这个视图对所有排序规则显示由拥有者授予给 `PUBLIC` 的隐式非可授予的 `USAGE` 特权。但是对其他对象类型则显示真实的特权。

在 KingbaseES 中，序列也支持除 `USAGE` 之外的 `SELECT` 和 `UPDATE` 特权。这些是非标准的并且因此在该信息模式中不可见。

表 1.4.151: usage_privileges 列

名称	数据类型	描述
grantor	sql_identifier	授予该特权的角色名
grantee	sql_identifier	被授予该特权的角色名
object_catalog	sql_identifier	包含该对象的数据库名（总是当前数据库）
object_schema	sql_identifier	如果适用，则是包含该对象的模式名，否则为一个空字符串
object_name	sql_identifier	该对象的名称
object_type	character_data	COLLATION 或 DOMAIN 或 FOREIGN DATA WRAPPER 或 FOREIGN SERVER 或 SEQUENCE
privilege_type	character_data	总是 USAGE
is_grantable	yes_or_no	如果该特权是可授予的，则为 YES，否则为 NO

1.4.2.60　user_defined_types

视图 `user_defined_types` 目前包含定义在当前数据库中的所有组合类型。只有那些当前用户能够访问（作为拥有者或具有某些特权）的类型才会被显示。

表 1.4.152: user_defined_types 列

名称	数据类型	描述
user_defined_type_catalog	sql_identifier	包含该类型的数据库名（总是当前数据库）
user_defined_type_schema	sql_identifier	包含该类型的模式名
user_defined_type_name	sql_identifier	该类型的名字
user_defined_type_category	character_data	当前总是 STRUCTURED
is_instantiable	yes_or_no	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
is_final	yes_or_no	应用于一个 KingbaseES 中不可用的特性
ordering_form	character_data	应用于一个 KingbaseES 中不可用的特性
ordering_category	character_data	应用于一个 KingbaseES 中不可用的特性
ordering_routine_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
ordering_routine_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
ordering_routine_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
reference_type	character_data	应用于一个 KingbaseES 中不可用的特性
data_type	character_data	应用于一个 KingbaseES 中不可用的特性
character_maximum_length	cardinal_number	应用于一个 KingbaseES 中不可用的特性
character_octet_length	cardinal_number	应用于一个 KingbaseES 中不可用的特性
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
collation_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
numeric_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
numeric_precision_radix	cardinal_number	应用于一个 KingbaseES 中不可用的特性
numeric_scale	cardinal_number	应用于一个 KingbaseES 中不可用的特性
datetime_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性
interval_type	character_data	应用于一个 KingbaseES 中不可用的特性
interval_precision	cardinal_number	应用于一个 KingbaseES 中不可用的特性
source_dtd_identifier	sql_identifier	应用于一个 KingbaseES 中不可用的特性
ref_dtd_identifier	sql_identifier	应用于一个 KingbaseES 中不可用的特性

1.4.2.61　user_mapping_options

视图 `user_mapping_options` 包含在当前数据库中为用户映射定义的所有选项。只有那些当前用户能够访问其相应外部服务器（作为拥有者或具有某些特权）的用户映射才会被显示。

表 1.4.153: user_mapping_options 列

名称	数据类型	描述
authorization_identifier	sql_identifier	被映射的用户名，如果映射是公共的则为 PUBLIC
foreign_server_catalog	sql_identifier	这个映射所使用的外部服务器所在的数据库名（总是当前数据库）
foreign_server_name	sql_identifier	这个映射所使用的外部服务器的名称
option_name	sql_identifier	一个选项名

-续表-

名称	数据类型	描述
option_value	character_data	选项的值。除非当前用户是被映射的用户或者映射是 PUBLIC 的并且当前用户是服务器拥有者或者超级用户，这一列将显示为空。这样做的目的是保护作为用户映射选项存储的口令信息

1.4.2.62　user_mappings

视图 `user_mappings` 包含定义在当前数据库中的所有用户映射。只有当前用户能够访问其对应外部服务器（作为拥有者或具有某些特权）的用户映射才会被显示。

表 1.4.154: user_mappings 列

名称	数据类型	描述
authorization_identifier	sql_identifier	被映射的用户名，如果映射是公共的则为 PUBLIC
foreign_server_catalog	sql_identifier	这个映射所使用的外部服务器所在的数据库名（总是当前数据库）
foreign_server_name	sql_identifier	这个映射所使用的外部服务器的名称

1.4.2.63　view_column_usage

视图 `view_column_usage` 标识被使用在一个视图的查询表达式（定义该视图的 `SELECT` 语句）中的所有列。只有当包含一列的表被一个当前用户拥有时，该列才会被包括在这个视图中。

备注:

系统表列不被包括。在某个时候这应该会被修复。

表 1.4.155: view_column_usage 列

名称	数据类型	描述
view_catalog	sql_identifier	包含该视图的数据库名（总是当前数据库）
view_schema	sql_identifier	包含该视图的模式名
view_name	sql_identifier	该视图的名称
table_catalog	sql_identifier	被视图所使用的列所属表的数据库名（总是当前数据库）
table_schema	sql_identifier	被视图所使用的列所属表的模式名
table_name	sql_identifier	被视图所使用的列所属表的名称
column_name	sql_identifier	被该视图所使用的列名

1.4.2.64　view_routine_usage

视图 `view_routine_usage` 标识被使用在一个视图的查询表达式（定义该视图的 `SELECT` 语句）中的所有例程（函数和过程）。只有被一个当前已被启用角色拥有的例程才会被包括在这个视图中。

表 1.4.156: view_routine_usage 列

名称	数据类型	描述
table_catalog	sql_identifier	包含该视图的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该视图的模式名
table_name	sql_identifier	该视图的名称
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”

1.4.2.65　view_table_usage

视图 `view_table_usage` 标识被使用在一个视图的查询表达式（定义该视图的 `SELECT` 语句）中的所有表。只有被一个当前已被启用角色拥有的表才会被包括在这个视图中。

备注:

1. 查询结果不包括系统表；
2. 查询的视图对象不包括物化视图。

表 1.4.157: view_table_usage 列

名称	数据类型	描述
view_catalog	sql_identifier	包含该视图的数据库名（总是当前数据库）
view_schema	sql_identifier	包含该视图的模式名
view_name	sql_identifier	该视图的名称
table_catalog	sql_identifier	包含被该视图所使用的表的数据库名（总是当前数据库）
table_schema	sql_identifier	包含被该视图所使用的表的模式名
table_name	sql_identifier	包含被该视图所使用的表的名称

1.4.2.66　views

视图 `views` 包含定义在当前数据库中的所有视图。只有当前用户能够访问（作为拥有者或具有某些特权）的视图才会被显示。

表 1.4.158: views 列

名称	数据类型	描述
table_catalog	sql_identifier	包含该视图的数据库名（总是当前数据库）
table_schema	sql_identifier	包含该视图的模式名
table_name	sql_identifier	该视图的名称

-续表-

名称	数据类型	描述
view_definition	character_data	定义视图的查询表达式（如果该视图不被一个当前已被启用角色拥有则为空）
check_option	character_data	如果该视图有一个 CHECKOPTION 上的定义，则为 CASCADED 或者 LOCAL，否则为 NONE
is_updatable	yes_or_no	如果该视图是可更新的（允许 UPDATE 和 DELETE），则为 YES，否则为 NO
is_insertable_into	yes_or_no	如果该视图是可插入的（允许 INSERT），则为 YES，否则为 NO
is_trigger_updatable	yes_or_no	如果该视图上有一个 INSTEAD OFUPDATE 触发器，则为 YES，否则为 NO
is_trigger_deletable	yes_or_no	如果该视图上有一个 INSTEAD OFDELETE 触发器，则为 YES，否则为 NO
is_trigger_insertable_into	yes_or_no	如果该视图上有一个 INSTEAD OFINSERT 触发器，则为 YES，否则为 NO

1.5　SQLServer 兼容视图

KingbaseES 为保证兼容 SQLServer 用户使用习惯, 提供了两部分 SQLServer 兼容视图供用户使用：元信息视图（包含 SQLServer 中的部分 System catalog views 和部分 System Compatibility Views ）和信息模式视图。

元信息视图都放在 sys 模式下，用于记录对象元信息和部分设置信息；信息模式视图都放在 information_schema 模式下，用于提供对外的元信息，需要注意的是信息模式视图访问时需要加上模式限定-“information_schema.”。

1.5.1　元信息视图

1.5.1.1　sys.all_columns

sys.all_columns 视图记录 toast 表（对应内部表）、系统表、用户表（包含普通表、外表、分区表）、视图、表类型创建的表对应的列信息，每个列在该视图中均仅对应一行。该系统视图只能查询到用户拥有权限的列的信息。

表 1.5.1: sys.all_columns 列

名称	数据类型	描述
object_id	oid	此列所属表或视图的 OID
name	name	列的名称
column_id	int	列号
system_type_id	oid	列的数据类型的 OID
user_type_id	oid	用户定义的列类型的 OID
max_length	smallint	列的最大长度（字节） 若列类型为数值类型，则为最大字节长度 若列类型为字符类型，则为最大长度
precision	tinyint	列数据类型的精度 若列为数值类型，则为该列的精度；否则为 0
scale	tinyint	列数据类型的标度 若列为数值类型，则为列的小数位数；否则为 0
collation_name	sysname	列的排序规则 若列为字符类型，则为该列排序规则的名称；否则为 NULL
is_nullable	bit	列是否为空

-续表-

名称	数据类型	描述
is_ansi_padded	bit	列是否为字符、二进制或变量类型，且列使用 ANSI_PADDINGON 行为 1= 如果列为字符、二进制或变量类型，则该列使用 ANSI_PADDINGON 行为 0= 列不是字符、二进制或变量类型，KES 定长的字符类型为 ANSI _PADDINGON 行为
is_rowguidcol	bit	列是否声明为 ROWGUIDCOL
is_identity	bit	列是否为 identity 列
is_computed	bit	列是否为计算列
is_filestream	bit	列是否为 FILESTREAM 列 固定置为 0
is_replicated	bit	列是否已复制
is_non_sql_subscribed	bit	列是否为非 SQLServer 订阅服务器 固定置为 0
is_merge_published	bit	列是否已合并发布 固定置为 0
is_dts_replicated	bit	是否使用 SSIS 复制列 固定置为 0
is_xml_document	bit	列是否为 xml 文档 1= 列的数据类型或基类型是 xml 类型
xml_collection_id	int	XML 架构命名空间的集合的 ID 固定置为 0
default_object_id	int	列级 DEFAULT 约束的 OID 0= 无默认值
rule_object_id	int	列级 CHECK 约束的 OID 0= 无 CHECK 约束
is_sparse	bit	列是否使用了稀疏存储 1= 列为稀疏列 固定置为 0

-续表-

名称	数据类型	描述
is_column_set	bit	列是否为列集 1= 列为列集 固定置为 0
generated_always_type	tinyint	目前暂不支持该字段，当前字段固定置为 0
generated_always_type_desc	nvarchar(60)	固定置为 NOT_APPLICABLE
encryption_type	int	加密类型 固定置为 NULL
encryption_type_desc	nvarchar(64)	加密类型说明 固定置为 NULL
encryption_algorithm_name	sysname	加密算法的名称 固定置为 NULL
column_encryption_key_id	int	列加密秘钥的 ID 固定置为 NULL
column_encryption_key_database_name	sysname	列加密密钥所在数据库的名称 固定置为 NULL
is_hidden	bit	是否为隐含列 固定置为 0
is_masked	bit	列是否由动态数据掩码进行掩码处理 固定置为 0

1.5.1.2　sys.all_objects

sys.all_objects 包含了当前数据库中所有用户定义的对象及系统元数据对象的列表，这个视图是 sys.objects 和其他特定于服务器范围对象的 UNION（包含用户定义对象和系统对象）。该系统视图只能查询到用户对其拥有权限的对象的信息。

表 1.5.2: sys.all_objects 列

名称	数据类型	描述
name	name	对象名称

-续表-

名称	数据类型	描述
object_id	oid	对象 oid(不保证唯一)，需和 type 列组合使用才能唯一标识对象
principal_id	oid	对象所有者 id 对于模式对象，当对象所有者 id 和模式所有者 id 相同时，该列为 NULL 如果对象类型为下列类型之一，则为 NULL： C = CHECK 约束 D = DEFAULT（约束或独立） F = FOREIGN KEY 约束 PK = PRIMARY KEY 约束 TR = SQL 触发器 UQ = UNIQUE 约束 其余则为对象的 relowner id
schema_id	oid	对象的模式的 oid
parent_object_id	oid	此对象所属对象的 oid 0 = 不是子对象 若为约束、触发器该字段为依赖表的 oid
type	char(2)	对象类型： AF = 聚合函数 C = CHECK 约束 F = FOREIGN KEY 约束 UQ = UNIQUE 约束 PK = PRIMARY KEY 约束 P = SQL 存储过程 FN = SQL 标量函数 IT = 内部表 S = 系统基表 U = 表（用户定义类型） ET = 外部表 V = 视图 SN = 同义词 SO = 序列对象 TR = SQL DML 触发器 MV = 物化视图

-续表-

名称	数据类型	描述
type_desc	nvarchar(60)	对应 type 列的详细描述如下： AGGREGATE_FUNCTION CHECK_CONSTRAINT FOREIGN_KEY_CONSTRAINT UNIQUE_CONSTRAINT PRIMARY_KEY_CONSTRAINT SQL_STORED_PROCEDURE SQL_SCALAR_FUNCTION INTERNAL_TABLE SYSTEM_TABLE USER_TABLE EXTERNAL_TABLE VIEW SYNONYM SEQUENCE_OBJECT SQL_TRIGGER MATERIALIZED_VIEW
create_date	datetime	对象的创建日期
modify_date	datetime	上次使用 ALTER 语句修改对象的日期
is_ms_shipped	bit	对象是否由系统内部创建
is_published	bit	对象是否为发布对象
is_schema_published	bit	仅发布对象的模式 默认为 0

1.5.1.3　sys.change_tracking_tables

记录表数据捕获功能相关信息。

表 1.5.3: sys.change_tracking_tables 列

名称	数据类型	描述
object_id	oid	具有更改日志的表的 ID

-续表-

名称	数据类型	描述
is_track_columns_updated_on	bit	该表的更改跟踪的当前状态 0 = OFF 1 = ON
begin_version	bigint	开始对该表进行更改跟踪时数据库的版本
cleanup_version	bigint	版本号，在该版本以前的版本中，清除操作可能会删除更改跟踪信息。
min_valid_version	bigint	该表可用的更改跟踪信息的最低有效版本

1.5.1.4　sys.check_constraints

该系统视图只能查询到用户对其拥有权限的对象上的 check 约束信息。每个 check 约束在该系统视图中对应一行，sys.objects.type = ‘C’ 。

表 1.5.4: sys.check_constraints 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同该视图列数据类型	sys.objects.type = ’C’ 的部分数据
is_disabled	bit	是否禁用 CHECK 约束
is_not_for_replication	bit	目前暂不支持该字段，当前字段默认值为 0
is_not_trusted	bit	系统是否不针对所有行验证 CHECK 约束
parent_column_id	int	标识 check 约束类别 0 - 表级 CHECK 约束 非零值 - 针对具有指定 ID 值的列定义的列级 CHECK 约束
definition	nvarchar(max)	定义该 CHECK 约束的 SQL 表达式
uses_database_collation	bit	check 约束是否使用数据库默认排序规则 1 = 约束定义依赖数据库的默认排序顺序进行正确计算；否则为 0

-续表-

名称	数据类型	描述
is_system_named	bit	命名是否为系统生成 1 = 名称由系统生成 0 = 名称由用户提供

1.5.1.5　sys.columns

sys.columns 视图记录 toast 表（对应内部表）、系统表、用户表（包含普通表、外表、分区表）、用户创建视图、表类型创建的表对应的列信息，每个列在该视图中均仅对应一行。该系统视图仅支持查询用户拥有权限的列的信息。

表 1.5.5: sys.columns 列

名称	数据类型	描述
object_id	oid	此列所属表或视图的 OID
name	name	列的名称
column_id	int	列号
system_type_id	oid	列的数据类型的 OID
user_type_id	oid	用户定义的列类型的 OID
max_length	smallint	列的最大长度（字节） 若列数据类型为数值类型，则为最大字节长度 若列数据类型为字符类型，则为最大长度
precision	tinyint	列数据类型的精度 若列数据类型为数值类型，则为该列的精度；否则为 0
scale	tinyint	列数据类型的标度 若列数据类型为数值类型，则为列的小数位数；否则为 0
collation_name	sysname	列的排序规则 若列数据类型为字符类型，则为该列排序规则的名称；否则为 NULL

-续表-

名称	数据类型	描述
is_nullable	bit	列是否为空
is_ansi_padded	bit	列是否为字符、二进制或变量类型，且列使用 ANSI_PADDINGON 行为 1= 如果列为字符、二进制或变量类型，则该列使用 ANSI_PADDINGON 行为 0= 列不是字符、二进制或变量类型，KES 定长的字符类型为 ANSI _PADDINGON 行为
is_rowguidcol	bit	列是否声明为 ROWGUIDCOL
is_identity	bit	列是否为 identity 列
is_computed	bit	列是否为计算列
is_filestream	bit	列是否为 FILESTREAM 列 固定置为 0
is_replicated	bit	列是否已复制
is_non_sql_subscribed	bit	列是否为非 SQLServer 订阅服务器 固定置为 0
is_merge_published	bit	列是否已合并发布 固定置为 0
is_dts_replicated	bit	是否使用 SSIS 复制列 固定置为 0
is_xml_document	bit	列是否为 xml 文档 1= 列的数据类型或基类型是 xml 类型
xml_collection_id	int	XML 架构命名空间的集合的 ID 固定置为 0
default_object_id	int	列级 DEFAULT 约束的 OID 0= 无默认值
rule_object_id	int	列级 CHECK 约束的 OID 0= 无 CHECK 约束
is_sparse	bit	列是否使用了稀疏存储 1= 列为稀疏列 固定置为 0

-续表-

名称	数据类型	描述
is_column_set	bit	列是否为列集 1= 列为列集 固定置为 0
generated_always_type	tinyint	目前暂不支持该字段，当前字段固定置为 0
generated_always_type_desc	nvarchar(60)	固定置为 NOT_APPLICABLE
encryption_type	int	加密类型 固定置为 NULL
encryption_type_desc	nvarchar(64)	加密类型说明 固定置为 NULL
encryption_algorithm_name	sysname	加密算法的名称 固定置为 NULL
column_encryption_key_id	int	列加密秘钥的 ID 固定置为 NULL
column_encryption_key_database_name	sysname	列加密密钥所在数据库的名称 固定置为 NULL
is_hidden	bit	是否为隐含列 固定置为 0
is_masked	bit	列是否由动态数据掩码进行掩码处理 固定置为 0

1.5.1.6　sys.configurations

该视图记录服务器运行期间，每个配置参数的信息。

表 1.5.6: sys.configurations 列

名称	数据类型	描述
configuration_id	int	配置值的唯一 ID
name	nvarchar(35)	配置选项的名称
value	sql_variant	该选项的配置值

-续表-

名称	数据类型	描述
minimum	sql_variant	配置选项的最小值
maximum	sql_variant	配置选项的最大值
value_in_use	sql_variant	该选项当前使用的运行值
description	nvarchar(255)	配置选项的描述
is_dynamic	bit	修改配置项后是否需要重启 0 = 修改该配置选项后需要重启 1 = 修改该配置选项后需要重启
is_advanced	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.7　sys.data_spaces

该视图记录表空间的信息，每个表空间在该视图中对应一行，且该系统视图只能查询到用户对其拥有权限的表空间的信息。

表 1.5.7: sys.data_spaces 列

名称	数据类型	描述
name	name	表空间的名称
data_space_id	oid	表空间的 oid
type	char(2)	FG = 表空间
type_desc	nvarchar(60)	表空间类型的说明： FG(type) - ROWS_FILEGROUP
is_default	bit	当前表空间是否为默认表空间 1 = 是默认表空间 0 = 不是默认表空间
is_system	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.8　sys.database_files

当前数据库的文件目录信息在该视图中对应一行，内容随着数据库的切换而改变。

表 1.5.8: sys.database_files 列

名称	数据类型	描述
file_id	int	数据库文件目录的标识号
file_guid	uniqueidentifier	数据库文件目录的唯一标识符
type	tinyint	文件类型 0 = 行
type_desc	nvarchar(60)	文件类型的说明 ROWS 对应 type 为 0
data_space_id	oid	表空间的 oid
name	sysname	表空间的名称拼上数据库的名称作为唯一标识
physical_name	nvarchar(260)	数据库文件目录的名称，包括完整路径
state	tinyint	文件状态 0 = ONLINE
state_desc	nvarchar(60)	文件状态的说明 ONLINE 对应 state 为 0
size	int	数据库文件目录的大小（以 8KB 为单位的页数）
max_size	int	最大文件大小（以 8 KB 为单位的页） -1 = 文件将一直增长到磁盘充满为止
growth	int	默认按 8KB 个页面增长
is_media_read_only	bit	默认为 0，文件位于可读/写介质上
is_read_only	bit	当前数据库文件目录是否只读，只读取值为 1，可读可写取值为 0
is_sparse	bit	目前暂不支持该字段，当前字段默认值为 0
is_percent_growth	bit	默认为 0，以页数为单位表示绝对增长大小

-续表-

名称	数据类型	描述
is_name_reserved	bit	默认为 0，文件名不可重复使用
create_lsn	numeric(25,0)	数据库创建时的 lsn
drop_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
read_only_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
read_write_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_time	datetime	目前暂不支持该字段，当前字段默认值为 NULL
redo_start_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
redo_start_fork_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值其 NULL
redo_target_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
redo_target_fork_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值为 NULL
backup_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL

1.5.1.9　sys.database_permissions

该视图记录当前数据库中当前用户拥有的对象（数据库、对象、列、模式和类型）权限信息。

表 1.5.9: sys.database_permissions 列

名称	数据类型	描述
class	tinyint	标识权限所属对象类别 0 = 数据库 1 = 对象和列（包括：用户表、视图、物化视图、函数、存储过程、聚合函数、SQL 触发器、序列对象） 3 = 模式 6 = 类型
class_desc	nvarchar(60)	对象类别描述 DATABASE OBJECT_OR_COLUMN SCHEMA TYPE
major_id	oid	存在权限的对象的 ID 0 = 数据库本身 其他则为对象的 oid
minor_id	int	通常为 0，只有为列权限时，表示列 id。
grantee_principal_id	oid	被授权用户 id
grantor_principal_id	oid	授权者用户 id
type	char (4)	权限类型： SL IN UP DL TRU RF TRI CR CO TEMP US EX
permission_name	nvarchar(128)	权限类别，支持的权限类别如下：

-续表-

名称	数据类型	描述
		SELECT INSERT UPDATE DELETE TRUNCATE REFERENCES TRIGGER CREATE CONNECT TEMPORARY USAGE EXECUTE
state	char(1)	权限状态 G= 授予 R= 撤销 W = 带授权选项的授权
state_desc	nvarchar(60)	权限状态描述，支持值如下： GRANT REVOKE GRANT_WITH_GRANT_OPTION

1.5.1.10　sys.database_principals

每个数据库主体对应一行视图数据，对于 kes 目前只有用户和角色。

表 1.5.10: sys.database_principals 列

名称	数据类型	描述
name	name	数据库中的角色名称
principal_id	oid	角色的行标识符
type	char(1)	角色的类型 R = 数据库角色 S = SQL 用户

-续表-

名称	数据类型	描述
type_desc	nvarchar(60)	角色的类型的说明
default_schema_name	names	未指定命名空间时默认使用的名称 NULL - 非 S、U 或 A 类型的主体 同名模式 - 当存在用户同名模式且当前用户有使用权限，否则为 dbo
create_date	datetime	角色的创建时间
modify_date	datetime	上次修改角色的时间
owning_principal_id	oid	拥有此成员的角色的 ID
sid	varbinary(85)	角色的 SID（安全标识符）
is_fixed_role	bit	固定置为 0
authentication_type	int	指示身份验证类型
authentication_type_desc	nvarchar(60)	身份验证类型说明
default_language_name	nvarchar(128)	指示此角色的默认语言
default_language_lcid	int	指示此角色的默认语言的 lcid(引用 sys.syslanguage 视图中的 lcid)
allow_encrypted_value_modifications	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.11　sys.database_role_members

每组成员关系对应一行视图数据。

表 1.5.11: sys.database_role_members 列

名称	数据类型	描述
role_principal_id	oid	拥有成员的角色的 OID
member_principal_id	oid	成员的数据库主体 OID

1.5.1.12　sys.databases

每个数据库对应一行视图数据。

表 1.5.12: sys.databases 列

名称	数据类型	描述
name	name	数据库的名称
database_id	oid	数据库的 ID
source_database_id	int	数据库依赖的源数据库的 ID 恒为 NULL
owner_sid	varbinary(85)	数据库拥有者的安全标识符
create_date	datetime	数据库的创建时间
compatibility_level	tinyint	兼容版本，恒为 130
collation_name	sysname	数据库的排序规则名称
user_access	tinyint	用户访问设置 0 - 允许多个用户连接该数据库 1 - 仅允许单个用户连接该数据库 3 - 不允许用户连接到数据库上
user_access_desc	nvarchar(60)	用户访问设置的说明，对应用户访问设置的值: 0 = 已指定 MULTI_USER ，允许多个用户连接该数据库 1 = 已指定 SINGLE_USER ，仅允许单个用户连接该数据库 3 = NO_ACCESS，不允许用户连接到数据库上
is_read_only	bit	是否为只读数据库 1 = 数据库为 READ_ONLY 0 = 数据库为 READ_WRITE
is_auto_close_on	bit	数据库是否自动关闭 恒为 0
is_auto_shrink_on	bit	数据库是否自动收缩 恒为 0

-续表-

名称	数据类型	描述
state	tinyint	数据库的状态 固定置为 0 = ONLINE
state_desc	nvarchar(60)	数据库状态的说明 固定置为 ONLINE
is_in_standby	bit	数据库是否处于备用状态
is_cleanly_shutdown	bit	数据库是否清理并关闭 1 = 数据库完全关闭, 在启动时不需要恢复 0 = 数据库并未完全关闭, 在启动时需要恢复
is_supplemental_logging_enabled	bit	是否启用补充日志记录 恒为 0
snapshot_isolation_state	tinyint	允许的快照隔离事务状态 1 = 允许使用快照隔离
snapshot_isolation_state_desc	nvarchar(60)	允许的快照隔离事务状态的说明，始终为 ON 状态
is_read_committed_snapshot_on	bit	read-committed 隔离级别下的读操作是否没有获取锁 恒为 1
recovery_model	tinyint	数据库恢复的模式 3-简单模式, 数据库无法还原到特定备份中的特定时间点
recovery_model_desc	nvarchar(60)	选定的恢复模式的说明 SIMPLE - 对应恢复模式取值为 3
page_verify_option	tinyint	PAGE_VERIFY 属性的值，恒为 2
page_verify_option_desc	nvarchar(60)	PAGE_VERIFY 选项设置的说明 CHECKSUM-对应 PAGE_VERIFY 属性值的 2
is_auto_create_stats_on	bit	是否自动创建统计信息 恒为 0
is_auto_create_stats_incremental_on	bit	是否自动创建统计信息的增量信息 恒为 0
is_auto_update_stats_on	bit	是否自动更新统计信息 恒为 0

-续表-

名称	数据类型	描述
is_auto_update_stats_async_on	bit	是否自动异步更新统计信息 恒为 0
is_ansi_null_default_on	bit	新建的列不指定默认值和空属性，是否该列是否默认允许 NULL 值 恒为 1
is_ansi_nulls_on	bit	列与 NULL 进行运算是否返回 NULL（不包含 IS 和 IS NOT NULL 操作） 恒为 1
is_ansi_padding_on	bit	是否以空格或零填充到列指定的长度 恒为 1
is_ansi_warnings_on	bit	函数中出现 null 值，是否产生警告消息 恒为 0
is_arithabort_on	bit	发生错误时是否异常结束 恒为 0.
is_concat_null_yields_null_on	bit	NULL 与字符串拼接时是否为 NULL 恒为 0
is_numeric_roundabort_on	bit	出现精度损失时是否生成错误 恒为 0
is_quoted_identifier_on	bit	是否可以用双引号 (” ”) 分隔标识符 恒为 1
is_recursive_triggers_on	bit	是否允许递归激发 AFTER 触发器 恒为 0.
is_cursor_close_on_commit_on	bit	提交或回滚时是否关闭所有打开的游标 恒为 0
is_local_cursor_default	bit	控制游标作用域是使用 LOCAL 还是 GLOBAL 恒为 0
is_fulltext_enabled	bit	是否为数据库启用全文索引 恒为 1
is_trustworthy_on	bit	数据库是否已被标记为可信 恒为 0.

-续表-

名称	数据类型	描述
is_db_chaining_on	bit	跨数据库所有权链接 恒为 OFF
is_parameterization_forced	bit	是否强制参数化 1 = 参数化为 FORCED 0 = 参数化为 SIMPLE
is_master_key_encrypted_by_server	bit	数据库是否具有加密的主密钥 恒为 0
is_query_store_on	bit	查询是否存储 恒为 0
is_published	bit	是否为发布数据库 恒为 0
is_subscribed	bit	是否为订阅数据库 恒为 0
is_merge_published	bit	数据库是否是合并复制拓扑中的发布数据库 恒为 0
is_distributor	bit	数据库是否为复制拓扑的分发数据库 恒为 0
is_sync_with_backup	bit	数据库是否标记为与备份进行复制同步 恒为 0
service_broker_guid	uniqueidentifier	该数据库的服务代理标识符
is_broker_enabled	bit	无实际意义，恒为 0
log_reuse_wait	tinyint	事务日志空间重复使用前等待最后一个检查点的内容 恒为 0，不等待任何内容
log_reuse_wait_desc	nvarchar(60)	日志空间的重复使用正在等待最后一个检查点的描述 恒为 NOTHING
is_date_correlation_on	bit	是否存在日期相关优化 恒为 0
is_cdc_enabled	bit	是否对数据库启用数据变更捕获 恒为 0

-续表-

名称	数据类型	描述
is_encrypted	bit	数据库是否加密 恒为 0
is_honor_broker_priority_on	bit	数据库是否遵守会话优先级 恒为 0
replica_id	uniqueidentifier	数据库参与的可用性组（如果有）的本地 AlwaysOn 可用性组可用性副本的唯一标识符 恒为 NULL-数据库不是可用性组中的可用性副本的一部分
group_database_id	uniqueidentifier	数据库在其参与的 AlwaysOn 可用性组（如果有）中的唯一标识符 恒为 NULL-数据库不是任何可用性组中的可用性副本的一部分
resource_pool_id	int	映射到此数据库的资源池的 ID 恒为 NULL
default_language_lcid	smallint	contained 数据库的默认语言的本地 ID (lcid) 对于非 contained 数据库，此值为 NULL
default_language_name	nvarchar(128)	指示 contained 数据库的默认语言 对于非 contained 数据库，此值为 NULL
default_fulltext_language_lcid	int	指示 contained 数据库的默认全文语言的本地 ID(lcid) 对于非 contained 数据库，此值为 NULL
default_fulltext_language_name	nvarchar(128)	指示 contained 数据库的默认全文语言 对于非 contained 数据库，此值为 NULL
is_nested_triggers_on	bit	contained 数据库中是否允许使用嵌套触发器 对于非 contained 数据库，此值为 NULL

-续表-

名称	数据类型	描述
is_transform_noise_words_on	bit	是否应在 contained 数据库中转换干扰词 对于非 contained 数据库，此值为 NULL
two_digit_year_cutoff	smallint	指示 1753 到 9999 之间的数字值，以表示将两位数的年份解释为四位数的年份的截止年份 对于非 contained 数据库，此值为 NULL
containment	tinyint	数据库的包含状态 对于 ontained 数据库，恒为 0
containment_desc	nvarchar(60)	数据库的包含状态描述 NONE ，对应 containment 的 0 值
target_recovery_time_in_seconds	int	恢复数据库的估计时间（秒） 恒为 60
delayed_durability	int	延迟持续性是否生效 恒为 0 - 失效
delayed_durability_desc	nvarchar(60)	延迟持续性生效的描述 恒为 DISABLED，对应 delayed _durability 中的 0
is_memory_optimized_elevate_to_snapshot_on	bit	无意义，恒为 0
is_federation_member	bit	该数据库是否是联合数据库中的成员 恒为 0
is_remote_data_archive_enabled	bit	数据库时否是 stretch database 恒为 0
is_mixed_page_allocation_on	bit	数据库中的表和索引是否可以从混合分片分配初始页 恒为 0

1.5.1.13　sys.extended_properties

该系统视图只能查询到用户对其拥有权限的对象的信息。当前数据库中的每个扩展属性（允许用户向数据库对象添加自定义的元数据的特性，包括对象的描述、创建日期等）在视图中对应一行。

表 1.5.13: sys.extended_properties 列

名称	数据类型	描述
class	tinyint	拥有扩展属性的对象的类别，包含以下几类： 0 = 数据库 1 = 对象或列 (支持对象包括：用户表、视图、物化视图、聚合函数、存储过程、函数、约束、序列） 3 = 模式 4= 角色 (role) 6 = 类型 7 = 索引 20 = 表空间
class_desc	nvarchar(60)	拥有扩展属性的对象的类别描述： DATABASE OBJECT_OR_COLUMN SCHEMA DATABASE_PRINCIPAL TYPE INDEX DATASPACE
major_id	oid	拥有扩展属性的对象的 oid
minor_id	int	拥有扩展属性的对象的子 id 当 class = 1 时，若为列的扩展属性，则为列 id，否则为 0 当 class = 7 时，为 index_id
name	oid	扩展属性名
value	sql_variant	扩展属性的值

1.5.1.14　sys.filegroups

每个表空间的信息在视图中对应一行。

表 1.5.14: sys.filegroups 列

名称	数据类型	描述
name	name	表空间的名称
data_space_id	oid	表空间的 oid
type	char(2)	FG = 表空间
type_desc	nvarchar(60)	表空间类型的说明 ROWS_FILEGROUP - 对应 type 值 FG
is_default	bit	当前表空间是否是默认表空间 1 = 这是默认表空间 0 = 这不是默认表空间
is_system	bit	目前暂不支持该字段，当前字段默认值为 0
filegroup_guid	uniqueidentifier	表空间的唯一标识符 NULL = sys_default 表空间
log_filegroup_id	int	目前暂不支持该字段，当前字段默认值为 0
is_read_only	bit	表空间是否只读 1 = 表空间为只读表空间 0 = 表空间为可读/写的表空间
is_autogrow_all_files	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.15　sys.foreign_key_columns

包含有关数据库内的表定义中的 FOREIGN KEY 约束的约束键的信息。

表 1.5.15: sys.foreign_key_columns 列

名称	数据类型	描述
constraint_object_id	oid	约束的 id
constraint_column_id	int	组成 FOREIGN KEY（1..n，其中 n 为列数）的列或列集的列号
parent_object_id	oid	具有该约束的表的 id

-续表-

名称	数据类型	描述
parent_column_id	int	该表中作为外键列的列号
referenced_object_id	oid	约束引用的表的 id
referenced_column_id	int	被引用列（候选键列）的列号

1.5.1.16　sys.foreign_keys

该系统视图只能查询到用户对其拥有权限的对象的信息。对于每个 FOREIGN KEY 约束对象都包含一行，对应 sys.object.type = F。

表 1.5.16: sys.foreign_keys 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同 SQL Server	同 SQL Server 的定义，对应 sys.objects.type = ’F’ 的数据
referenced_object_id	oid	所引用对象的 oid
key_index_id	int	所引用对象内键索引的 ID(对应 sys.index 的 index_id 列)
is_disabled	bit	是否禁用 FOREIGN KEY 约束
is_not_for_replication	bit	目前暂不支持该字段，当前字段默认值为 0
is_not_trusted	bit	系统是否不针对所有行验证 FOREIGN KEY 约束
delete_referential_action	tinyint	执行删除时为此 FOREIGN KEY 声明的引用操作 0 = 不执行任何操作 1 = 级联 2 = 设置 NULL 3 = 设置默认值

-续表-

名称	数据类型	描述
delete_referential_action_desc	nvarchar(60)	执行删除时为此 FOREIGN KEY 声明的引用操作的说明: NO_ACTION CASCADE SET_NULL SET_DEFAULT
update_referential_action	tinyint	执行更新时为此 FOREIGN KEY 声明的引用操作 0 = 不执行任何操作 1 = 级联 2 = 设置 NULL 3 = 设置默认值
update_referential_action_desc	nvarchar(60)	执行更新时为此 FOREIGN KEY 声明的引用操作的说明： NO_ACTION CASCADE SET_NULL SET_DEFAULT
is_system_named	bit	命名是否为系统生成 1 = 名称由系统生成 0 = 名称由用户提供

1.5.1.17　sys.identity_columns

sys.identity_columns 视图记录数据库中表的 IDENTITY 属性的起始值、增长值、最新 IDENTITY 值（last_value）及列上其他相关信息。

表 1.5.17: sys.identity_columns 列

名称	数据类型	描述
object_id	oid	列对应的表 ID
name	name	列名
column_id	int	列号

-续表-

名称	数据类型	描述
system_type_id	oid	列的基类型
user_type_id	oid	列的类型 ID
max_length	smallint	列的数据类型对应的字节数
precision	tinyint	精度
scale	tinyint	标度，IDENTITY 列恒为 0
collation_name	nvarchar(128)	列上的排序规则，恒为 NULL
is_nullable	bit	列是否为空，恒为 0
is_ansi_padded	bit	列是否自动补充空格，恒为 0
is_rowguidcol	bit	是否是全局唯一标识列，IDENTITY 列不能是全局唯一标识符列，恒为 0
is_identity	bit	是否是 IDENTITY 列，恒为 1
is_filestream	bit	目前暂不支持该字段，当前字段默认值为 0
is_replicated	bit	是否被复制
is_non_sql_subscribed	bit	无实际意义，恒为 0
is_merge_published	bit	无实际意义，恒为 0
is_dts_replicated	bit	无实际意义，恒为 0
is_xml_document	bit	目前暂不支持该字段，当前字段默认值为 0
xml_collection_id	int	目前暂不支持该字段，当前字段默认值为 0
default_object_id	int	列上是否有 DEFAULT，IDENTITY 列与默认值不能同时指定, 恒为 0
rule_object_id	int	目前暂不支持该字段，当前字段默认值为 0
seed_value	sql_variant	查询 IDENTITY 列创建时的起始值
increment_value	sql_variant	查询 IDENTITY 列创建时的增长值
last_value	sql_variant	查询 IDENTITY 列最新使用的 IDENTITY 值
is_not_for_replication	bit	复制是否跳过该属性, 恒为 0

-续表-

名称	数据类型	描述
is_computed	bit	目前暂不支持该字段，当前字段默认值为 0
is_sparse	bit	目前暂不支持该字段，当前字段默认值为 0
is_column_set	bit	目前暂不支持该字段，当前字段默认值为 0
generated_always_type	tinyint	目前暂不支持该字段，当前字段默认值为 0
generated_always_type_desc	nvarchar(60)	上一列的描述，0 对应'NOT_APPLICABLE'
encryption_type	int	加密类型，IDENTITY 列不能加密，恒为 NULL
encryption_type_desc	nvarchar(64)	加密类型描述，IDENTITY 列不能加密，恒为 NULL
encryption_algorithm_name	sysname	加密算法，IDENTITY 列不能加密，恒为 NULL
column_encryption_key_id	int	目前暂不支持该字段，当前字段默认值为 NULL
column_encryption_key_database_name	sysname	目前暂不支持该字段，当前字段默认值为 NULL
is_hidden	bit	版本控制表上的属性，恒为 0
is_masked	bit	列上是否有动态掩码函数，恒为 0

1.5.1.18　sys.index_columns

该视图记录索引键列信息。

表 1.5.18: sys.index_columns 列

名称	数据类型	描述
object_id	oid	定义索引的对象的 Oid
index_id	int	定义了列的索引的 ID
index_column_id	int	索引列的 ID
column_id	int	索引键列在表中的列号

-续表-

名称	数据类型	描述
key_ordinal	tinyint	键列集内的序数（从 1 开始）
partition_ordinal	tinyint	分区列集内的序数（从 1 开始） 0= 非分区列
is_descending_key	bit	索引键列排序顺序是否为降序 1= 索引键列具有降序排序方向 0= 索引键列具有升序排序方向，或列是列存储或哈希索引的一部分
is_included_column	bit	键列是否为非索引键 1= 列是使用 CREATE INDEX INCLUDE 子句添加到索引中的非键列，或者列是列存储索引的一部分 0= 列不是包含的列
column_store_order_ordinal	tinyint	目前暂不支持该字段，当前字段默认值为 0

1.5.1.19　sys.indexes

每个表格对象（例如，表、视图或表值函数）的索引或堆都在视图中返回一行。

表 1.5.19: sys.indexes 列

名称	数据类型	描述
object_id	oid	该索引所属对象的 ID 或堆表的 ID
name	name	索引的名称
index_id	int	索引的 ID 0 = 堆 1 = 聚集索引 > 1 = 非聚集索引

-续表-

名称	数据类型	描述
type	tinyint	索引的类型，当前支持的索引类型如下： 0 = heap 2 = btree 3 = gist/spgist 7 = hash 8 = gin 9 = brin 10 = bitmap 对于分区表父表返回 NULL
type_desc	nvarchar(60)	索引类型的说明，对应类型列的值： heap = 0 btree = 2 gist/spgist = 3 hash = 7 gin = 8 brin = 9 bitmap = 10 对于分区表父表返回 NULL
is_unique	bit	索引是否是唯一索引 1 = 索引是唯一索引 0 = 索引不是唯一索引
data_space_id	oid	此索引的数据空间的 ID
ignore_dup_key	bit	是否忽略重复键
is_primary_key	bit	索引是否是主键的组成部分 1 = 索引是 PRIMARY KEY 约束的一部分，否则为 0 聚集列存储索引始终为 0
is_unique_constraint	bit	索引是否是唯一约束的组成部分 1 = 索引是 UNIQUE 约束的一部分，否则为 0
fill_factor	tinyint	创建或重新生成索引时使用的 FILLFACTOR
is_padded	bit	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
is_disabled	bit	目前暂不支持该字段，当前字段默认值为 0
is_hypothetical	bit	目前暂不支持该字段，当前字段默认值为 0
is_ignored_in_optimization	bit	目前暂不支持该字段，当前字段默认值为 0
allow_row_locks	bit	索引是否允许行级锁 1 = 索引允许行锁 0 = 索引不允许行锁
allow_page_locks	bit	索引是否允许页级锁 1 = 索引允许页锁 0 = 索引不允许页锁
has_filter	bit	索引是否包含筛选条件 1 = 索引具有一个筛选器，且仅包含符合筛选器定义的行 0 = 索引不具有筛选器
filter_definition	nvarchar(max)	包含在筛选索引中的行子集的表达式对于堆、未筛选索引或对表的权限不足，为 NULL
compression_delay	int	目前暂不支持该字段，当前字段默认值为 NULL
suppress_dup_key_messages	bit	目前暂不支持该字段，当前字段默认值为 0
auto_created	bit	索引是否为数据库自动创建 1 = 索引是通过自动优化创建的 0 = 索引由用户创建
optimize_for_sequential_key	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.20　sys.key_constraints

该系统视图只能查询到用户对其拥有权限的对象的主键和唯一约束信息。每个主键或唯一约束的对象都在视图中对应一行。包括 sys.objects.type 等于 PK 和 UQ。

表 1.5.20: sys.key_constraints 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同 SQL Server 数据类型	同 SQL Server 的定义，对应 sys.objects.type = 'PK' 和 sys.objects.type ='UQ' 的数据
unique_index_id	int	唯一索引的 ID(对应 sys.indexes 的 index_id 列)
is_system_named	bit	命名是否为系统生成 1 = 名称由系统生成 0 = 名称由用户提供

1.5.1.21　sys.linked_logins

每个连接服务器的信息在视图中对应一行。

表 1.5.21: sys.linked_logins 列

名称	数据类型	描述
server_id	oid	连接服务器的 oid
local_principal_id	oid	将被映射的本地角色的 OID
uses_self_credential	bit	取值为 0，表示会话使用提供的用户名和密码登录
remote_name	name	远程登录的用户名
modify_date	datetime	上次更改外部服务器的日期和时间

1.5.1.22　sys.master_files

每个数据库的文件目录信息在该视图中对应一行。

表 1.5.22: sys.master_files 列

名称	数据类型	描述
database_id	oid	数据库的 oid
file_id	int	数据库文件目录的标识号
file_guid	uniqueidentifier	数据库文件目录的唯一标识符
type	tinyint	文件类型 0 = 行
type_desc	nvarchar(60)	文件类型的说明 ROWS 对应 type 值 0
data_space_id	oid	表空间的 oid
name	sysname	文件目录名
physical_name	nvarchar(260)	数据库文件目录的名称，包括完整路径
state	tinyint	目前暂不支持该字段，当前字段默认值为 0
state_desc	nvarchar(60)	文件状态的说明，目前暂不支持该字段，当前字段默认值为'ONLINE'
size	int	数据库的大小（以 8KB 为单位的页数）
max_size	int	最大文件大小（以 8 KB 为单位的页数） -1 = 文件将一直增长到磁盘充满为止
growth	int	默认按 8KB 个页面增长
is_media_read_only	bit	目前暂不支持该字段，当前字段默认值为 0
is_read_only	bit	数据库文件目录是否只读 1 - 只读 0 - 可读可写
is_sparse	bit	目前暂不支持该字段，当前字段默认值为 0
is_percent_growth	bit	目前暂不支持该字段，当前字段默认值为 0
is_name_reserved	bit	目前暂不支持该字段，当前字段默认值为 0
create_lsn	numeric(25,0)	数据库创建时的 lsn
drop_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
read_only_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL

-续表-

名称	数据类型	描述
read_write_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值为 NULL
differential_base_time	datetime	目前暂不支持该字段，当前字段默认值为 NULL
redo_start_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
redo_start_fork_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值为 NULL
redo_target_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
redo_target_fork_guid	uniqueidentifier	目前暂不支持该字段，当前字段默认值为 NULL
backup_lsn	numeric(25,0)	目前暂不支持该字段，当前字段默认值为 NULL
credential_id	int	目前暂不支持该字段，当前字段默认值为 NULL

1.5.1.23　sys.objects

该系统视图只能查询到用户对其拥有权限的对象的信息。包含了当前数据库中所有用户定义的对象信息，包括表、视图、存储过程、函数、触发器等。每一个记录代表一个对象，并提供该对象的详细属性。

表 1.5.23: sys.objects 列

名称	数据类型	描述
name	name	对象名称
object_id	oid	对象 oid(不保证唯一)，需和 type 列组合使用才能唯一标识对象

-续表-

名称	数据类型	描述
principal_id	oid	对象所有者 id 对于模式对象，当对象所有者 id 和模式所有者 id 相同时，该列为 NULL 如果对象类型为下列类型之一，则为 NULL： C = CHECK 约束 D = DEFAULT（约束或独立） F = FOREIGN KEY 约束 PK = PRIMARY KEY 约束 TR = SQL 触发器 UQ = UNIQUE 约束 其余则为对象的 relowner id
schema_id	oid	对象的模式的 oid
parent_object_id	oid	此对象所属对象的 oid，0 = 不是子对象
type	char(2)	对象类型，目前支持以下类型： AF = 聚合函数 C = CHECK 约束 F = FOREIGN KEY 约束 UQ = UNIQUE 约束 PK = PRIMARY KEY 约束 P = SQL 存储过程 FN = SQL 标量函数 IT = 内部表 S = 系统基表 U = 表（用户定义类型） ET = 外部表 V = 视图 SN = 同义词 SO = 序列对象 TR = SQL DML 触发器 MV = 物化视图

-续表-

名称	数据类型	描述
type_desc	nvarchar(60)	对应 type 列的详细描述 AGGREGATE_FUNCTION CHECK_CONSTRAINT FOREIGN_KEY_CONSTRAINT UNIQUE_CONSTRAINT PRIMARY_KEY_CONSTRAINT SQL_STORED_PROCEDURE SQL_SCALAR_FUNCTION INTERNAL_TABLE SYSTEM_TABLE USER_TABLE EXTERNAL_TABLE VIEW SYNONYM SEQUENCE_OBJECT SQL_TRIGGER MATERIALIZED_VIEW
create_date	datetime	对象的创建日期
modify_date	datetime	上次使用 ALTER 语句修改对象的日期
is_ms_shipped	bit	对象是否由系统内部创建
is_published	bit	对象是否为发布对象
is_schema_published	bit	仅发布对象的模式，默认为 0

1.5.1.24　sys.partitions

sys.partitions 视图记录分区表、分区、分区索引、分区索引、全局非分区索引、普通表、外表、toast 表、普通索引信息。若表或索引无分区信息，则仅显示一行当前对象的信息。

表 1.5.24: sys.partitions 列

名称	数据类型	描述
partition_id	bigint	分区 id
object_id	oid	分区的对象 id
index_id	int	指示此分区所属的对象内的索引的 ID 0 = 堆 2 = 非聚集索引
partition_number	int	所属索引或堆中的从 1 开始的分区号 对于未分区的表和索引，此列的值为 1
hobt_id	bigint	指示包含此分区行的数据堆或 B 树 (HoBT) 的 ID，与分区 id 相同取值
rows	bigint	指示此分区中的大约行数
filestream_filegroup_id	smallint	目前暂不支持该字段，当前字段默认值为 0
data_compression	tinyint	指示每个分区的压缩状态，恒为 0
data_compression_desc	nvarchar(60)	指示每个分区的压缩状态，恒为'NONE'

1.5.1.25　sys.procedures

每个存储过程对应一行视图数据。

表 1.5.25: sys.procedures 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同 sys.objects	对应 sys.objects.type = 'P' 数据
is_auto_executed	bit	目前暂不支持该字段，当前字段默认值为 0
is_execution_replicated	bit	目前暂不支持该字段，当前字段默认值为 0
is_repl_serializable_only	bit	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
skips_repl_constraints	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.26　sys.remote_logins

记录有关用户映射的信息。

表 1.5.26: sys.remote_logins 列

名称	数据类型	描述
server_id	oid	包含该映射的连接服务器的 OID
remote_name	name	远程登陆的用户名
local_principal_id	oid	将被映射的本地角色的 OID，如果用户映射是公共的则为 0
modify_date	datetime	上次更改远程登录映射到本地登录的日期和时间

1.5.1.27　sys.schemas

记录当前数据库中的模式信息。

表 1.5.27: sys.schemas 列

名称	数据类型	描述
name	name	模式的名称。在该数据库中是唯一的
schema_id	oid	模式的 id。在该数据库中是唯一的
principal_id	oid	此架模式所属的数据库的 id

1.5.1.28　sys.server_principals

每个用户和角色的信息在视图中对应一行。

表 1.5.28: sys.server_principals 列

名称	数据类型	描述
name	name	角色名或用户名
principal_id	oid	角色或用户的 oid
sid	varbinary(85)	角色的 SID（安全标识符）
type	char(1)	主体类型 S = SQL 登录名 R = 服务器角色
type_desc	nvarchar(60)	主体类型的说明 SQL_LOGIN 对应对应 S SERVER_ROLE 对应服务器角色 R
is_disabled	int	角色是否可以登录 0 - 可以登录 1 - 不可以登录
create_date	datetime	创建角色的日期和时间
modify_date	datetime	上次修改角色的日期和时间
default_database_name	name	该角色的默认数据库
default_language_name	sysname	该角色的默认语言
credential_id	int	目前暂不支持该字段，当前字段默认值为 NULL
owning_principal_id	int	目前暂不支持该字段，当前字段默认值为 NULL
is_fixed_role	bit	主体是否是固定服务器角色 0 - 不是固定服务器角色 1 - 是固定服务器角色

1.5.1.29　sys.servers

每个外部服务器的信息都在视图中对应一行。

表 1.5.29: sys.servers 列

名称	数据类型	描述
server_id	oid	外部服务器的 oid
name	name	外部服务器或连接服务器的名称
product	sysname	连接服务器所连接的远程数据源的产品名称 “KingbaseES”表示外部服务器所连接的远程数据源的产品名称是 KingbaseES
provider	sysname	用于连接到外部服务器的 OLEDB 访问接口名称
data_source	nvarchar(4000)	OLEDB 数据源连接属性 若该服务器为外部服务器，值为外部服务器的名称
location	nvarchar(4000)	OLEDB 位置连接属性
provider_string	nvarchar(4000)	OLEDB 访问接口字符串连接属性
catalog	sysname	目前暂不支持该字段，当前字段默认值为 NULL
connect_timeout	int	目前暂不支持该字段，当前字段默认值为 0
query_timeout	int	目前暂不支持该字段，当前字段默认值为 0
is_linked	bit	服务器是否为连接服务器
is_remote_login_enabled	bit	目前暂不支持该字段，当前字段默认值为 0
is_rpc_out_enabled	bit	目前暂不支持该字段，当前字段默认值为 0
is_data_access_enabled	bit	目前暂不支持该字段，当前字段默认值为 1

-续表-

名称	数据类型	描述
is_collation_compatible	bit	目前暂不支持该字段，当前字段默认值为 NULL
uses_remote_collation	bit	目前暂不支持该字段，当前字段默认值为 1
collation_name	sysname	目前暂不支持该字段，当前字段默认值为 NULL
lazy_schema_validation	bit	目前暂不支持该字段，当前字段默认值为 0
is_system	bit	目前暂不支持该字段，当前字段默认值为 0
is_publisher	bit	目前暂不支持该字段，当前字段默认值为 0
is_subscriber	bit	目前暂不支持该字段，当前字段默认值为 0
is_distributor	bit	目前暂不支持该字段，当前字段默认值为 0
is_nonsql_subscriber	bit	目前暂不支持该字段，当前字段默认值为 0
is_remote_proc_transaction_promotion_enabled	bit	目前暂不支持该字段，当前字段默认值为 1
modify_date	datetime	目前暂不支持该字段，当前字段默认值为 NULL
is_rda_server	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.30　sys.sql_modules

SQL 语言定义模块的每个对象（包括本机编译的标量用户定义函数）都在视图中对应一行。视图涉及的对象有：存储过程、函数、视图、DML 触发器。

表 1.5.30: sys.sql_modules 列

名称	数据类型	描述
object_id	oid	包含对象的对象 ID。在数据库中是唯一的
definition	nvarchar(max)	定义此模块的 SQL 文本
uses_ansi_nulls	bit	目前暂不支持该字段，当前字段默认值为 1
uses_quoted_identifier	bit	目前暂不支持该字段，当前字段默认值为 1
is_schema_bound	bit	目前暂不支持该字段，当前字段默认值为 0
uses_database_collation	bit	目前暂不支持该字段，当前字段默认值为 0
is_recompiled	bit	目前暂不支持该字段，当前字段默认值为 0
null_on_null_input	bit	模块被声明为针对任何 NULL 输入生成 NULL 输出
execute_as_principal_id	int	目前暂不支持该字段，当前字段默认值为为 NULL
uses_native_compilation	bit	目前暂不支持该字段，当前字段默认值为 0
inline_type	bit	目前暂不支持该字段，当前字段默认值为 0
is_inlineable	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.31　sys.sysaltfiles

每个数据库的文件目录信息在该视图中对应一行。

表 1.5.31: sys.sysaltfiles 列

名称	数据类型	描述
fileid	smallint	数据库文件目录的标识号
groupid	smallint	表空间的 oid
size	int	数据库文件目录的大小（以 8 KB 页为单位）
maxsize	int	最大文件大小（以 8 KB 为单位的页） -1 = 文件将一直增长到磁盘充满为止

-续表-

名称	数据类型	描述
growth	int	默认按 8KB 个页面增长
status	int	默认该文件的状态取值为 2
perf	int	保留，值为 0
dbid	smallint	数据库的 oid
name	sysname	表空间的名称拼上数据库的名称作为唯一标识
filename	nvarchar(260)	数据库文件目录的名称，包括完整路径

1.5.1.32　sys.syscharsets

该系统视图只能查询到用户对其拥有权限的对象的信息。该视图记录数据库中的字符集及排序顺序信息，每个字符集或排序顺序对应视图中的一行数据。

表 1.5.32: sys.syscharsets 列

名称	数据类型	描述
type	smallint	此行表示的实体的类型：字符集 =1001
id	tinyint	字符集编码 ID
csid	tinyint	目前暂不支持该字段，当前字段默认值为固定为 0
status	smallint	目前暂不支持该字段，当前字段默认值为 0
name	sysname	字符集的唯一名称
description	nvarchar(255)	字符集的具体说明 (可选)
binarydefinition	varbinary(6000)	保留列，默认为 NULL
definition	image	目前暂不支持该字段，当前字段默认值为 NULL

1.5.1.33　sys.syscolumns

sys.syscolumns 视图记录聚集函数、用户自定义 SQL 标量函数、表类型创建的表、系统表、toast 表、SQL 存储过程、用户表（包含普通表、外表、分区表）、SQL 表值函数、SQL 内联表值函数、用户自定义视图的参数及列信息。

表 1.5.33: sys.syscolumns 列

名称	数据类型	描述
name	name	表和视图的列名
id	oid	所属表或视图的 OID
xtype	oid	列的数据类型的 OID 若为域类型，则为基类型的 OID
typestat	tinyint	记录类型的状态 0 = 允许为 NULL，非字符类型或不允许填充空格 1 = 不允许为 NULL，非字符类型或不允许填充空格 2 = 不允许为 NULL，为字符类型且长度不够时允许填充到列长 3 = 允许为 NULL，为字符类型且长度不够时允许填充到列长
xusertype	oid	列的数据类型的 OID，或为用户自定义类型的 OID
length	smallint	列的最大长度或字节数 若为可变长度类型，则为指定的最大长度
xprec	tinyint	数值类型的精度
xscale	tinyint	数值类型的标度
colid	smallint	列号
xoffset	smallint	目前暂不支持该字段，当前字段默认值为
bitpos	tinyint	目前暂不支持该字段，当前字段默认值为 0
reserved	tinyint	目前暂不支持该字段，当前字段默认值为 0
colstat	smallint	列的状态 1 = is_identity= 该列为标识列 4 = is_computed= 该列为计算列
cdefault	int	此列的默认值的 OID，恒为 0
domain	int	此列的 CHECK 约束的 OID
number	smallint	过程分组时的子过程号

-续表-

名称	数据类型	描述
colorder	smallint	列号
autoval	varbinary(8000)	目前暂不支持该字段，当前字段默认值为 NULL
offset	smallint	目前暂不支持该字段，当前字段默认值为 0
collationid	int	列的排序规则的 OID 对于非字符列，此值为 NULL
language	int	全文索引指定的 lcid 列值
status	tinyint	列的属性的位图 0x08= 列允许空值 0x10= 列为 varchar 或 varbinary 列时，ANSI 填充有效——为 varchar 保留尾随空白，为 varbinary 列保留尾随零 0x80= 列为标识列
type	tinyint	列的数据类型的 OID，或为基类型的 OID
usertype	smallint	列的数据类型的 OID，或为用户自定义类型的 OID
printfmt	varchar(255)	目前暂不支持该字段，当前字段默认值为 NULL
prec	smallint	此列的精度
scale	int	此列的标度
iscomputed	int	指示列是否为计算列的标志 0= 非计算列 1= 计算列
isoutparam	int	指示过程参数是否为输出参数
isnullable	int	指示列是否允许空值 1 = True 0 = False
collation	sysname	列的排序规则的名称
tdscollation	binary(5)	目前暂不支持该字段，当前字段默认值为 NULL

1.5.1.34　sys.syscomments

包含数据库中的每个视图、触发器、CHECK 约束和存储过程的条目。文本列包含原始 SQL 定义语句。

表 1.5.34: sys.syscomments 列

名称	数据类型	描述
id	oid	该文本适用的对象 id
number	smallint	如果进行分组，则为过程分组内的号码 0 = 项不是过程
colid	smallint	超过 4,000 个字符的对象定义的行序列号
status	smallint	保留列，恒为 0
ctext	varbinary(8000)	SQL 定义语句的原始字节
texttype	smallint	目前暂不支持该字段，当前字段默认值为 2
language	smallint	保留列，恒为 0
encrypted	bit	目前暂不支持该字段，当前字段默认值为 0
compressed	bit	目前暂不支持该字段，当前字段默认值为 0
text	nvarchar(4000)	SQL 定义语句的实际文本

1.5.1.35　sys.sysconfigures

服务器运行期间，每个配置参数的信息对应视图的一行。

表 1.5.35: sys.sysconfigures 列

名称	数据类型	描述
value	sql_variant	用户可修改的变量值
config	int	配置变量号
comment	nvarchar(255)	对配置选项的描述
status	smallint	若修改该配置选项后需要重启，值为 0，否则为 1

1.5.1.36　sys.sysconstraints

包含约束映射，映射到数据库中拥有该约束的对象。

表 1.5.36: sys.sysconstraints 列

名称	数据类型	描述
constid	oid	约束的 id
id	oid	拥有该约束的表的 id
colid	smallint	列号 若约束列中包含多列，则为 0，反之则为被定义约束的列号
spare1	tinyint	保留，暂时固定为 0
status	int	约束类型，可能的值为： 1 = 主键约束 2 = 唯一约束 3 = 外键约束 4 = check 约束 16 = 列级约束 32 = 表级约束
actions	int	保留，暂时固定为 4096
error	int	保留，暂时固定为 0

1.5.1.37　sys.syscurconfigs

服务器运行期间，每个配置参数的信息对应视图的一行，该视图包含四列，用于说明配置结构。

表 1.5.37: sys.syscurconfigs 列

名称	数据类型	描述
value	text	用户可修改的变量值
config	smallint	配置变量号

-续表-

名称	数据类型	描述
comment	nvarchar(255)	对配置选项的描述
status	smallint	若修改该配置选项后需要重启，值为 0，否则为 1

1.5.1.38　sys.sysdatabases

记录 KES 中的数据库（database）对象的信息，详细视图结构请见下表。

表 1.5.38: sys.sysdatabases 列

名称	数据类型	描述
name	name	数据库名称
dbid	smallint	数据库的行标识符
sid	varbinary(85)	数据库所属的角色的 SID（安全标识符）
mode	smallint	目前暂不支持该字段，当前字段默认值为 0
status	int	状态位，仅支持以下状态位： 1024 = read only 4096 = single user 65536 = CHECKSUM 1073741824 = cleanly shutdown
status2	int	数据库状态位 16384 = ANSI null default 8388608 = quoted identifier 67108864 = ANSI nulls 536870912 = full text enabled
crdate	datetime	数据库创建的日期
reserved	datetime	保留列，固定置为 1900-01-01 00:00:00.000
category	int	与复制数据库相关的位值，恒为 0
cmptlevel	tinyint	数据库的兼容性级别，恒为 130

-续表-

名称	数据类型	描述
filename	nvarchar(260)	数据库主文件的操作系统路径和名称
version	smallint	数据库的内部版本号

1.5.1.39　sys.sysfilegroups

每个表空间的信息在视图中对应一行。

表 1.5.39: sys.sysfilegroups 列

名称	数据类型	描述
groupid	smallint	表空间的 oid
allocpolicy	smallint	保留，值为 0
status	int	表空间状态 若该表空间不是默认表空间且可读写，status 的值为 0 若该表空间不是默认表空间且仅可读，status 的值为 8 若该表空间是默认表空间且仅可读，status 的值为 24 若该表空间是默认表空间且可读写，status 的值为 16
groupname	name	表空间的名称

1.5.1.40　sys.sysfiles

当前数据库的文件目录信息在该视图中对应一行，内容随着数据库的切换而改变。

表 1.5.40: sys.sysfiles 列

名称	数据类型	描述
fileid	smallint	数据库文件目录的标识号
groupid	smallint	表空间的 oid

-续表-

名称	数据类型	描述
size	int	数据库文件目录的大小（以 8 KB 页为单位）
maxsize	int	最大文件大小（以 8 KB 为单位的页） -1 = 文件将一直增长到磁盘充满为止
growth	int	默认按 8KB 个页面增长
status	int	目前暂不支持该字段，当前字段默认值为 2
perf	int	保留列，值恒为 0
name	sysname	表空间的名称拼上数据库的名称作为唯一标识
filename	nvarchar(260)	数据库文件目录的名称，包括完整路径

1.5.1.41　sys.sysforeignkeys

包含有关数据库内的表定义中的 FOREIGN KEY 约束的信息，该系统视图只能查询到用户对其拥有权限的对象的信息。

表 1.5.41: sys.sysforeignkeys 列

名称	数据类型	描述
constid	oid	FOREIGN KEY 约束的 id
fkeyid	oid	具有 FOREIGN KEY 约束的表的 id
rkeyid	oid	在 FOREIGN KEY 约束中引用的表的 id
fkey	smallint	引用列的 ID
rkey	smallint	被引用列的 ID
keyno	smallint	该列在引用列列表中的位置

1.5.1.42　sys.sysindexes

当前数据库中的每个索引和堆表各对应一行。

表 1.5.42: sys.sysindexes 列

名称	数据类型	描述
id	oid	索引所属表的 ID 以及堆表的 ID
status	int	保留列，恒为 0
first	binary(6)	指向第一页或根页的指针 indid = 0 时未使用 NULL = indid> 1 时索引分区
indid	smallint	索引 ID 对于表来说，该列为 0 对于表的索引来说，该列 >1
root	binary(6)	对于 indid>= 1，root 是指向根页的指针 indid = 0 时未使用 NULL = indid> 1 时索引分区
minlen	smallint	行的最小大小
keycnt	smallint	键数
groupid	oid	在其上创建对象的文件组 ID NULL = indid> 1 时索引分区 NULL = 当 indid 为 0 或 1 时，表已分区
dpages	int	对于 indid = 0 或 indid = 1，dpages 是所用数据页的计数 对于 indid> 1，dpages 是使用的索引页计数 0 = 索引在 indid> 1 时分区 0 = 当 indid 为 0 或 1 时，表已分区 如果发生行溢出，则不会得出准确的结果
reserved	int	对于 indid = 0 或 indid = 1，reserved 是为所有索引和表数据分配的页计数 对于 indid> 1，reserved 是为索引分配的页计数 0 = 索引在 indid> 1 时分区 0 = 当 indid 为 0 或 1 时，表已分区 如果发生行溢出，则不会得出准确的结果

-续表-

名称	数据类型	描述
used	int	对于 indid = 0 或 indid = 1，used 是用于所有索引和表数据的总页数 对于 indid> 1，使用的是用于索引的页计数 0 = 索引在 indid> 1 时分区 0 = 当 indid 为 0 或 1 时，表已分区 如果发生行溢出，则不会得出准确的结果
rowcnt	bigint	表的行数，对于该表上的索引重复该值 0 = 索引在 indid> 1 时分区 0 = 当 indid 为 0 或 1 时，表已分区
rowmodctr	int	对自上次更新表的统计信息后插入、删除或更新行的总数进行计数 0 = 索引在 indid> 1 时分区 0 = 当 indid 为 0 或 1 时，表已分区 其他情况为空
reserved3	tinyint	保留列，恒为 0
reserved4	tinyint	保留列，恒为 0
xmaxlen	smallint	行的最大大小
maxirow	smallint	目前暂不支持该字段，当前字段默认值为 NULL
OrigFillFactor	tinyint	创建索引时使用的初始填充因子值
StatVersion	tinyint	保留列，恒为 0
reserved2	int	保留列，恒为 0
FirstIAM	binary(6)	保留列，恒为 NULL
impid	smallint	保留列，恒为 0
lockflags	smallint	目前暂不支持该字段，当前字段默认值为 0
pgmodctr	int	保留列，恒为 0
keys	varbinary(816)	组成索引键的列 ID 列表，恒返回 NULL
name	name	索引或统计信息的名称。当 indid = 0 时返回 NULL
statblob	bytea	统计信息二进制大型对象 (BLOB)，恒返回 NULL
maxlen	int	保留列，恒为 8000
rows	int	表的行数，对于该表上的索引重复该值

1.5.1.43　sys.sysindexkeys

包含有关数据库的索引中的键或列的信息，该视图依赖于 sys.index_columns 视图。

表 1.5.43: sys.sysindexkeys 列

名称	数据类型	描述
id	oid	表的 OID
indid	smallint	索引的 ID
colid	smallint	列的 ID
keyno	smallint	该列在索引中的位置

1.5.1.44　sys.syslanguages

该系统视图只能查询到用户对其拥有权限的对象的信息，该视图为数据库实例中支持的不同自然语言。

表 1.5.44: sys.syslanguages 列

名称	数据类型	描述
langid	smallint	唯一语言 ID
dateformat	nchar(3)	日期顺序
datefirst	tinyint	每周的第一天
upgrade	int	固定为 0
name	sysname	官方语言名称
alias	sysname	代替语言名称
months	nvarchar(372)	以逗号分隔的月份名全称列表，按一月到十二月的顺序排列，每个名称最多可有 20 个字符
shortmonths	nvarchar(132)	以逗号分隔的月份名简称列表，按一月到十二月的顺序排列，每个名称最多可有 9 个字符

-续表-

名称	数据类型	描述
days	nvarchar(217)	以逗号分隔的星期名称，按星期一到星期日的顺序排列，每个名称最多可有 30 个字符
lcid	int	目前暂不支持该字段，当前字段默认值为 2052
msglangid	smallint	数据库引擎消息组 ID

1.5.1.45　sys.syslogins

每个用户的信息在视图中对应一行。

表 1.5.45: sys.syslogins 列

名称	数据类型	描述
sid	varbinary(85)	用户的安全标识符
status	smallint	保留列，恒为 9
createdate	datetime	目前暂不支持该字段，当前字段默认值为 NULL
updatedate	datetime	目前暂不支持该字段，当前字段默认值为 NULL
accdate	datetime	保留列，恒为 NULL
totcpu	int	保留列，恒为 0
totio	int	保留列，恒为 0
spacelimit	int	保留列，恒为 0
timelimit	int	保留列，恒为 0
resultlimit	int	保留列，恒为 0
name	name	用户名
dbname	name	目前暂不支持该字段，当前字段默认值为 NULL
password	nvarchar(128)	登录时对应的登录密码
language	sysname	默认的用户语言

-续表-

名称	数据类型	描述
denylogin	int	目前暂不支持该字段，当前字段默认值为 0
hasaccess	int	目前暂不支持该字段，当前字段默认值为 0
isntname	int	目前暂不支持该字段，当前字段默认值为 0
isntgroup	int	目前暂不支持该字段，当前字段默认值为 0
isntuser	int	目前暂不支持该字段，当前字段默认值为 0
sysadmin	int	目前暂不支持该字段，当前字段默认值为 0
securityadmin	int	目前暂不支持该字段，当前字段默认值为 0
serveradmin	int	目前暂不支持该字段，当前字段默认值为 0
setupadmin	int	目前暂不支持该字段，当前字段默认值为 0
processadmin	int	目前暂不支持该字段，当前字段默认值为 0
diskadmin	int	目前暂不支持该字段，当前字段默认值为 0
dbcreator	int	目前暂不支持该字段，当前字段默认值为 0
bulkadmin	int	目前暂不支持该字段，当前字段默认值为 0
loginname	nvarchar(128)	用户名

1.5.1.46　sys.sysmembers

每组成员关系对应一行视图数据，数据同 sys.database_role_members。

表 1.5.46: sys.sysmembers 列

名称	数据类型	描述
memberuid	smallint	成员角色的 ID
groupuid	smallint	拥有成员的角色的 ID

1.5.1.47　sys.sysobjects

在数据库中创建的每个对象（例如约束、默认值、日志、规则以及存储过程）都对应一行。

表 1.5.47: sys.sysobjects 列

名称	数据类型	描述
name	name	对象名
id	oid	对象 oid(不保证唯一)，需和 type 列组合使用才能唯一标识对象
xtype	char(2)	对象类型 AF = 聚合函数 C = CHECK 约束 D = 默认值或 DEFAULT 约束 F = FOREIGN KEY 约束 FN = 标量函数 IT = 内部表 P = 存储过程 PK = PRIMARY KEY 约束（type 为 K） S = 系统表 SN = 同义词 SO = 序列 TR = SQL DML 触发器 TT = 表类型 U = 用户表 UQ = UNIQUE 约束（type 为 K） V = 视图 ET = 外部表 MV = 物化视图
uid	smallint	对象模式 oid
info	smallint	目前暂不支持该字段，当前字段默认值为 0
status	int	目前暂不支持该字段，当前字段默认值为 0
base_schema_ver	int	目前暂不支持该字段，当前字段默认值为 0
replinfo	int	目前暂不支持该字段，当前字段默认值为 0
parent_obj	oid	父对象的对象标识号
crdate	datetime	对象创建日期
ftcatid	smallint	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
schema_ver	int	目前暂不支持该字段，当前字段默认值为 0
stats_schema_ver	int	目前暂不支持该字段，当前字段默认值为 0
type	char(2)	对象类型 AF = 聚合函数 C = CHECK 约束 D = 默认值或 DEFAULT 约束 F = FOREIGN KEY 约束 FN = 标量函数 IT = 内部表 P = 存储过程 K = PRIMARY KEY 约束或 UNIQUE 约束 S = 系统表 SO = 序列 SN = 同义词 TR = SQL DML 触发器 TT = 表类型 U = 用户表或外部表 V = 视图
userstat	smallint	type 列取值对应 sysstat 值映射如下： 'S'、'U'、'ET':1 其他对象:0
sysstat	smallint	type 列取值对应 sysstat 值映射如下： 'S': 1 'V' :2 'U' : 3 'P' : 4 'D' :6 'TR' : 8 'PK' : 9 'UQ' : 9 'C' : 10 'F' : 11 其余对象：0
indexdel	smallint	目前暂不支持该字段，当前字段默认值为 0
refdate	datetime	对象的创建时间

-续表-

名称	数据类型	描述
version	int	目前暂不支持该字段，当前字段默认值为 0
deltrig	int	若当前记录为触发器，则该字段值为所属对象 oid 若当前记录为表，则该字段值为表上触发 Delete 行为触发器的 oid
instrig	int	若当前记录为表，则该字段值为表上触发 Insert 行为触发器的 oid
updtrig	int	若当前记录为表，则该字段值为表上触发 Update 行为触发器的 oid
seltrig	int	目前暂不支持该字段，当前字段默认值为 0
category	int	用于发布、约束和标识
cache	smallint	目前暂不支持该字段，当前字段默认值为 0

1.5.1.48　sys.sysoledbusers

每个连接服务器的用户和密码都对应视图中的一行记录。

表 1.5.48: sys.sysoledbusers 列

名称	数据类型	描述
rmtsrvid	smallint	连接服务器的 oid
rmtloginame	nvarchar (128)	远程登录的用户名
rmtpassword	nvarchar (128)	返回 NULL
loginsid	varbinary(85)	角色的 SID（安全标识符）
status	smallint	取值为 0，表示使用提供的用户名和密码进行登录
changedate	datetime	上次更改该连接服务器的日期和时间

1.5.1.49　sys.syspermissions

该视图记录数据库中的用户和角色授予和拒绝权限的信息。

表 1.5.49: sys.syspermissions 列

名称	数据类型	描述
id	oid	对象 id，对于 database 该字段为 0
grantee	smallint	被授予权限用户的主体 id
grantor	smallint	授予权限用户的主体 id
actadd	smallint	固定置为 NULL
actmod	smallint	固定置为 NULL
seladd	varbinary(4000)	固定置为 NULL
selmod	varbinary(4000)	固定置为 NULL
updadd	varbinary(4000)	固定置为 NULL
updmod	varbinary(4000)	固定置为 NULL
refadd	varbinary(4000)	固定置为 NULL
refmod	varbinary(4000)	固定置为 NULL

1.5.1.50　sys.sysprocesses

包含有关在 kingbaseES 实例上运行的进程的信息。这些进程可以是客户端进程或系统进程。只有当前用户为超级用户时，才可以查询所有的进程信息，否则只能查看当前会话的。

表 1.5.50: sys.sysprocesses 列

名称	数据类型	描述
spid	smallint	kingbaseES 会话 ID
kpid	smallint	Windows 线程 ID。在 Linux 下，该值始终为 0
blocked	smallint	正在阻塞请求的会话的 ID 如果此列为 NULL，则表示请求未被阻塞，或锁定会话的会话信息不可用（或无法进行标识）
waittype	binary(2)	目前暂不支持该字段，当前字段默认值为 NULL

-续表-

名称	数据类型	描述
waittime	bigint	当前等待时间（毫秒） 0 = 进程不等待
lastwaittype	nchar(32)	指示上次或当前等待类型的名称
waitresource	nchar(256)	目前暂不支持该字段，当前字段默认值为 NULL
dbid	oid	当前正由进程使用的数据库 id
uid	oid	执行命令的用户 ID
cpu	int	进程的累计 CPU 时间
physical_io	bigint	目前暂不支持该字段，当前字段默认值为 NULL
memusage	int	目前暂不支持该字段，当前字段默认值为 NULL
login_time	datetime	客户端进程登录到服务器的时间
last_batch	datetime	目前暂不支持该字段，当前字段默认值为 NULL
ecid	smallint	目前暂不支持该字段，当前字段默认值为 NULL
open_tran	smallint	进程的打开事务数
status	nchar(30)	进程状态。可能的值为：active：后端正在执行一个查询 idle：后端正在等待一个新的客户端命令 idle intransaction：后端在一个事务中，但是当前没有正在执行一个查询 idle in transaction (aborted)：这个状态与 idle intransaction 相似，不过在该事务中的一个语句导致了一个错误 fastpath function call：后端正在执行一个 fast-path 函数 disabled：如果在这个后端中 track_activities 被禁用，则报告这个状态
sid	binary(86)	用户的全局唯一标识符
hostname	nchar(128)	工作站的名称
program_name	nchar(128)	应用程序的名称
hostprocess	nchar(10)	工作站进程 ID 号
cmd	nchar (52)	当前正在执行的命令
nt_domain	nchar(128)	目前暂不支持该字段，当前字段默认值为 NULL

-续表-

名称	数据类型	描述
nt_username	nchar(128)	目前暂不支持该字段，当前字段默认值为 NULL
net_address	nchar(12)	目前暂不支持该字段，当前字段默认值为 NULL
net_library	nchar(12)	目前暂不支持该字段，当前字段默认值为 NULL
loginame	nchar(128)	登录名
context_info	binary(128)	目前暂不支持该字段，当前字段默认值为 0x000
sql_handle	binary(20)	目前暂不支持该字段，当前字段默认值为 NULL
stmt_start	int	目前暂不支持该字段，当前字段默认值为 NULL
stmt_end	int	目前暂不支持该字段，当前字段默认值为 NULL
request_id	int	目前暂不支持该字段，当前字段默认值为 NULL
page_resource	binary(8)	目前暂不支持该字段，当前字段默认值为 NULL

1.5.1.51　sys.sysreferences

该系统视图只能查询到用户对其拥有权限的对象的信息，记录外键约束引用的主表及主表中被引用键的信息。

表 1.5.51: sys.sysreferences 列

名称	数据类型	描述
constid	oid	FOREIGN KEY 约束的 oid
fkeyid	oid	执行引用表的 oid
rkeyid	oid	被引用表的 oid
rkeyindid	smallint	包含被引用的键列的被引用表的唯一索引的索引 ID
keycnt	smallint	键中的列数
forkeys	varbinary(32)	目前暂不支持该字段，当前字段默认值为 NULL
refkeys	varbinary(32)	目前暂不支持该字段，当前字段默认值为 NULL
fkeydbid	smallint	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
rkeydbid	smallint	目前暂不支持该字段，当前字段默认值为 0
fkey1	smallint	对应引用列的列 id
fkey2	smallint	对应引用列的列 id
fkey3	smallint	对应引用列的列 id
fkey4	smallint	对应引用列的列 id
fkey5	smallint	对应引用列的列 id
fkey6	smallint	对应引用列的列 id
fkey7	smallint	对应引用列的列 id
fkey8	smallint	对应引用列的列 id
fkey9	smallint	对应引用列的列 id
fkey10	smallint	对应引用列的列 id
fkey11	smallint	对应引用列的列 id
fkey12	smallint	对应引用列的列 id
fkey13	smallint	对应引用列的列 id
fkey14	smallint	对应引用列的列 id
fkey15	smallint	对应引用列的列 id
fkey16	smallint	对应引用列的列 id
rkey1	smallint	被引用列的 id
rkey2	smallint	被引用列的 id
rkey3	smallint	被引用列的 id
rkey4	smallint	被引用列的 id
rkey5	smallint	被引用列的 id
rkey6	smallint	被引用列的 id
rkey7	smallint	被引用列的 id

-续表-

名称	数据类型	描述
rkey8	smallint	被引用列的 id
rkey9	smallint	被引用列的 id
rkey10	smallint	被引用列的 id
rkey11	smallint	被引用列的 id
rkey12	smallint	被引用列的 id
rkey13	smallint	被引用列的 id
rkey14	smallint	被引用列的 id
rkey15	smallint	被引用列的 id
rkey16	smallint	被引用列的 id

1.5.1.52　sys.sysremotelogins

包含有关用户映射的信息。

表 1.5.52: sys.sysremotelogins 列

名称	数据类型	描述
remoteserverid	smallint	包含该映射的连接服务器的 OID
remoteusername	name	远程登陆的用户名
status	smallint	目前暂不支持该字段，当前字段默认值为 0
sid	varbinary(85)	角色的 sid（安全标识符）
changedate	datetime	添加远程登录映射到本地登录的日期和时间

1.5.1.53　sys.systypes

记录 kingbaseES 中所有可以在表、函数或 PROCEDURE 中使用的类型及 SHELL 类型的元信息，每个类型在该视图中对应一行。

表 1.5.53: sys.systypes 列

名称	数据类型	描述
name	name	数据类型名称
xtype	oid	类型的基类型
status	tinyint	类型的状态 1 = 不允许为空 0 = 允许为空
xusertype	oid	类型的实际 OID
length	smallint	数据类型的最大字节长度
xprec	tinyint	类型精度
xscale	tinyint	类型标度
tdefault	int	类型上面绑定的默认值对象 OID
domain	int	类型上面绑定的 RULE 对象 OID
uid	smallint	类型所属的命名空间 OID
reserved	smallint	无含义
collationid	int	排序规则
usertype	smallint	用户类型 OID
variable	bit	是否为可变长度类型
allownulls	bit	类型是否允许为空
type	tinyint	类型类型 OID
printfmt	varchar(255)	无意义
prec	smallint	数据类型的精度
scale	tinyint	数据类型标度
collation	sysname	类型的排序规则

1.5.1.54　sys.sysusers

每个数据库用户和角色对应一行视图数据。

表 1.5.54: sys.sysusers 列

名称	数据类型	描述
uid	smallint	用户 ID
status	smallint	用户的类型。目前暂不支持该字段，当前字段默认值为 0
name	name	用户名或组名（角色名）
sid	varbinary(85)	用户的 SID（安全标识符）
roles	varbinary(2048)	目前暂不支持该字段，当前字段默认值为 NULL
createdate	datetime	目前暂不支持该字段，当前字段默认值为 NULL
updatedate	datetime	目前暂不支持该字段，当前字段默认值为 NULL
altuid	smallint	目前暂不支持该字段，当前字段默认值为 NULL
password	varbinary(256)	目前暂不支持该字段，当前字段默认值为 NULL
gid	smallint	此用户所属的组 ID
environ	varchar(255)	目前暂不支持该字段，当前字段默认值为 NULL
hasdbaccess	int	账户是否具有数据库访问权 0 = 没有 1 = 有
islogin	int	帐户是否能登录 0 = 账户不能登录 1 = 帐户是具有登录帐户的用户
isntname	int	目前暂不支持该字段，当前字段默认值为 0
isntgroup	int	目前暂不支持该字段，当前字段默认值为 0
isntuser	int	目前暂不支持该字段，当前字段默认值为 0
issqluser	int	帐户是否为 SQL 用户 0 = 帐户不是 SQL 用户 1 = 帐户是 SQL 用户

-续表-

名称	数据类型	描述
isaliased	int	目前暂不支持该字段，当前字段默认值为 0
issqlrole	int	帐户是否为 SQL 角色 0 = 帐户不是 SQL 角色 1 = 帐户是 SQL 角色
isapprole	int	目前暂不支持该字段，当前字段默认值为 0

1.5.1.55　sys.table_types

记录用户定义表类型的元信息，每个表类型在该视图中对应一行，且该系统视图只能查询到用户对其拥有权限的对象的信息。

表 1.5.55: sys.table_types 列

名称	数据类型	描述
从 sys.types 继承的列	同 sys.types	对应 sys.types，对应的 is_table_type 列为 1 的数据
type_table_object_id	oid	对象标识号
is_memory_optimized	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.1.56　sys.tables

该系统视图只能查询到用户对其拥有权限的对象的信息。为 SQL Server 中的每个用户表返回一行。用于查询 sys.objects.type = U 的表对象。

表 1.5.56: sys.tables 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同 sys.objects	对应 sys.objects.type = 'U' 数据

-续表-

名称	数据类型	描述
lob_data_space_id	oid	存放二进制大型对象 (LOB) 数据的数据空间
filestream_data_space_id	int	目前暂不支持该字段，当前字段默认值为 NULL
max_column_id_used	int	表曾使用的最大列 ID
lock_on_bulk_load	bit	目前暂不支持该字段，当前字段默认值为 0
uses_ansi_nulls	bit	目前暂不支持该字段，当前字段默认值为 1
is_replicated	bit	目前暂不支持该字段，当前字段默认值为 0
has_replication_filter	bit	目前暂不支持该字段，当前字段默认值为 0
is_merge_published	bit	目前暂不支持该字段，当前字段默认值为 0
is_sync_tran_subscribed	bit	目前暂不支持该字段，当前字段默认值为 0
has_unchecked_assembly_data	bit	目前暂不支持该字段，当前字段默认值为 0
text_in_row_limit	int	Text in row 允许的最大字节数 0 = 未设置 text in row 选项
large_value_types_out_of_row	bit	是否在行外存储大数据 1 = 在行外存储大值类型
is_tracked_by_cdc	bit	目前暂不支持该字段，当前字段默认值为 0
lock_escalation	tinyint	目前暂不支持该字段，当前字段默认值为
lock_escalation_desc	nvarchar(60)	目前暂不支持该字段，当前字段默认值为'TABLE'
is_filetable	bit	目前暂不支持该字段，当前字段默认值为 0
is_memory_optimized	bit	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
durability	tinyint	目前暂不支持该字段，当前字段默认值为 0
durability_desc	nvarchar(60)	目前暂不支持该字段，当前字段默认值为 SCHEMA_AND_DATA
temporal_type	tinyint	目前暂不支持该字段，当前字段默认值为 0
temporal_type_desc	nvarchar(60)	目前暂不支持该字段，当前字段默认值为'NON_TEMPORAL_TABLE'
history_table_id	int	目前暂不支持该字段，当前字段默认值为 NULL
is_remote_data_archive_enabled	bit	目前暂不支持该字段，当前字段默认值为 0
is_external	bit	表是否为外表

1.5.1.57　sys.types

sys.types 视图记录 kingbaseES 中所有可以在表、函数或 PROCEDURE 中使用的类型及 SHELL 类型的元信息，每个类型在该视图中对应一行。

表 1.5.57: sys.types 列

名称	数据类型	描述
name	nvarchar(128)	类型的名称
system_type_id	oid	类型对应基类型的 OID
user_type_id	oid	类型的 OID
schema_id	oid	类型所属命名空间的 OID
principal_id	oid	类型拥有者的 OID
max_length	smallint	类型的最大字节长度
precision	tinyint	类型的最大精度

-续表-

名称	数据类型	描述
scale	tinyint	类型的最大标度
collation_name	sysname	类型排序规则的名称
is_nullable	bit	类型是否为 NULL
is_user_defined	bit	类型是否是用户定义的类型
is_assembly_type	bit	是否是程序集类型
default_object_id	int	类型上默认值对象的 OID
rule_object_id	int	类型上 RULE 对象的 OID
is_table_type	bit	类型是否为表类型

1.5.1.58　sys.views

该系统视图只能查询到用户对其拥有权限的对象的信息。每个用户定义视图对象都包含一行。对应 sys.objects.type =’ V’

表 1.5.58: sys.views 列

名称	数据类型	描述
< 从 sys.objects 继承的列 >	同 sys.objects	同 SQL Server 的定义，对应 sys.objects.type = ’V’ 的数据
is_replicated	bit	目前暂不支持该字段，当前字段默认值为 0
has_replication_filter	bit	目前暂不支持该字段，当前字段默认值为 0
has_opaque_metadata	bit	目前暂不支持该字段，当前字段默认值为 0
has_unchecked_assembly_data	bit	目前暂不支持该字段，当前字段默认值为 0

-续表-

名称	数据类型	描述
with_check_option	bit	视图定义时是否指定 WITH CHECK OPTION 1 = 在视图定义中指定了 WITH CHECK OPTION 0 = 未指定 WITH CHECK OPTION
is_date_correlation_view	bit	目前暂不支持该字段，当前字段默认值为 0
is_tracked_by_cdc	bit	目前暂不支持该字段，当前字段默认值为 0

1.5.2　信息模式视图

1.5.2.1　check_constraints

视图 `check_constraints` 记录当前数据库下当前用户拥有的表或 domain 上的检查约束。

表 1.5.59: check_constraints 列

名称	类型	描述
CONSTRAINT_CATALOG	information_schema.sql_identifier	检查约束所在数据库
CONSTRAINT_SCHEMA	information_schema.sql_identifier	检查约束所属模式
CONSTRAINT_NAME	information_schema.sql_identifier	检查约束名称
CHECK_CLAUSE	information_schema.character_data	检查约束定义

1.5.2.2　column_domain_usage

视图 `column_domain_usage` 记录当前数据库下所有当前用户拥有的使用用户自定义数据类型的列信息。

表 1.5.60: column_domain_usage 列

名称	类型	描述
DOMAIN_CATALOG	information _schema.sql_identifier	自定义类型所属数据库
DOMAIN_SCHEMA	information _schema.sql_identifier	自定义类型所属模式
DOMAIN_NAME	information _schema.sql_identifier	自定义类型名称
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	列所属表名
COLUMN_NAME	information _schema.sql_identifier	列名

1.5.2.3　column_privileges

视图 `column_privileges` 记录当前数据库下当前用户拥有权限的列信息。

表 1.5.61: column_privileges 列

名称	类型	描述
GRANTOR	information _schema.sql_identifier	授权者
GRANTEE	information _schema.sql_identifier	被授权者
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库

-续表-

名称	类型	描述
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名
COLUMN_NAME	information _schema.sql_identifier	列名
PRIVILEGE_TYPE	information _schema.character _data	权限类型
IS_GRANTABLE	information _schema.yes_or_no	被授权者是否拥有将权限授予其他用户的能力

1.5.2.4　columns

视图 `columns` 包含当前数据库下中有关所有表列（或视图列）的信息。仅显示当前用户能够访问（作为拥有者或具有某些特权）的列，系统列被排除在外。

表 1.5.62: columns 列

名称	类型	描述
TABLE_CATALOG	information _schema.sql _identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql _identifier	表所属模式
TABLE_NAME	information _schema.sql _identifier	表名
COLUMN_NAME	information _schema.sql _identifier	列名

-续表-

名称	类型	描述
ORDINAL_POSITION	information_schema.cardinal_number	列位置
COLUMN_DEFAULT	information_schema.character_data	列默认值
IS_NULLABLE	information_schema.yes_or_no	列是否为空
DATA_TYPE	information_schema.character_data	列数据类型
CHARACTER_MAXIMUM_LENGTH	information_schema.cardinal_number	字符类型的最大字符长度
CHARACTER_OCTET_LENGTH	information_schema.cardinal_number	字符类型的最大字节长度
NUMERIC_PRECISION	information_schema.cardinal_number	数值类型精度
NUMERIC_PRECISION_RADIX	information_schema.cardinal_number	数值类型精度的基数
NUMERIC_SCALE	information_schema.cardinal_number	数值类型标度
DATETIME_PRECISION	information_schema.cardinal_number	日期类型精度
CHARACTER_SET_CATALOG	information_schema.sql_identifier	字符集所属数据库
CHARACTER_SET_SCHEMA	information_schema.sql_identifier	字符集所属模式

-续表-

名称	类型	描述
CHARACTER_SET_NAME	information _schema.sql _identifier	字符集名称
COLLATION_CATALOG	information _schema.sql _identifier	排序规则所属数据库
COLLATION_SCHEMA	information _schema.sql _identifier	排序规则所属模式
COLLATION_NAME	information _schema.sql _identifier	排序规则名称
DOMAIN_CATALOG	information _schema.sql _identifier	用户自定义类型所属数据库
DOMAIN_SCHEMA	information _schema.sql _identifier	用户自定义类型所属模式
DOMAIN_NAME	information _schema.sql _identifier	用户自定义数据类型名称

1.5.2.5　constraint_column_usage

视图 `constraint_column_usage` 标识在当前数据库中被某个约束使用的所有列，仅显示当前用户拥有的表上的约束信息。

表 1.5.63: constraint_column_usage 列

名称	类型	描述
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式

-续表-

名称	类型	描述
TABLE_NAME	information _schema.sql_identifier	表名
COLUMN_NAME	information _schema.sql_identifier	列名
CONSTRAINT_CATALOG	information _schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information _schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	information _schema.sql_identifier	约束名

1.5.2.6　constraint_table_usage

视图 `constraint_table_usage` 记录表上使用的约束信息，仅显示当前数据库下当前用户拥有的表上的约束信息。

表 1.5.64: constraint_table_usage 列

名称	类型	描述
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名
CONSTRAINT_CATALOG	information _schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information _schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	information _schema.sql_identifier	约束名

1.5.2.7　domain_constraints

视图 `domain_constraints` 记录当前数据库下当前用户可访问的用户自定义类型上的约束信息。

表 1.5.65: domain_constraints 列

名称	类型	描述
CONSTRAINT_CATALOG	information _schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information _schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	sysname	约束名称
DOMAIN_CATALOG	information _schema.sql_identifier	自定义数据类型所属数据库
DOMAIN_SCHEMA	information _schema.sql_identifier	自定义数据类型所属模式
DOMAIN_NAME	information _schema.sql_identifier	自定义数据类型名称
IS_DEFERRABLE	information _schema.yes_or_no	约束检查是否可延迟
INITIALLY_DEFERRED	information _schema.yes_or_no	约束检查在事务开始时被设置为延迟模式

1.5.2.8　domains

视图 `domains` 包含定义在当前数据库中的所有自定义数据类型。只有当前用户能够访问（作为拥有者或具有某些特权）的自定义数据类型才被显示。

表 1.5.66: domains 列

名称	类型	描述
DOMAIN_CATALOG	information _schema.sql _identifier	自定义数据类型所属数据库

-续表-

名称	类型	描述
DOMAIN_SCHEMA	information_schema.sql_identifier	自定义数据类型所属模式
DOMAIN_NAME	information_schema.sql_identifier	自定义数据类型名称
DATA_TYPE	information_schema.character_data	自定义数据类型的基类型
CHARACTER_MAXIMUM_LENGTH	information_schema.cardinal_number	字符类型的最大字符长度
CHARACTER_OCTET_LENGTH	information_schema.cardinal_number	字符类型的最大字节长度
CHARACTER_SET_CATALOG	information_schema.sql_identifier	字符集所属数据库
CHARACTER_SET_SCHEMA	information_schema.sql_identifier	字符集所属模式
CHARACTER_SET_NAME	information_schema.sql_identifier	字符集名称
COLLATION_CATALOG	information_schema.sql_identifier	排序规则所属数据库
COLLATION_SCHEMA	information_schema.sql_identifier	排序规则所属模式
COLLATION_NAME	information_schema.sql_identifier	排序规则名称
NUMERIC_PRECISION	information_schema.cardinal_number	数值类型精度

-续表-

名称	类型	描述
NUMERIC_PRECISION_RADIX	information _schema.cardinal _number	数值类型精度的基数
NUMERIC_SCALE	information _schema.cardinal _number	数值类型标度
DATETIME_PRECISION	information _schema.cardinal _number	日期类型精度
DOMAIN_DEFAULT	information _schema.character _data	自定义数据类型的默认值

1.5.2.9　key_column_usage

视图 `key_column_usage` 标识当前数据库中所有被某种唯一、主键或外键约束限制的列，仅显示当前用户能够访问的列（作为拥有者或具有某些特权）。

表 1.5.67: key_column_usage 列

名称	类型	描述
CONSTRAINT_CATALOG	information _schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information _schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	information _schema.sql_identifier	约束名
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名

-续表-

名称	类型	描述
COLUMN_NAME	information_schema.sql_identifier	列名
ORDINAL_POSITION	information_schema.cardinal_number	列位置

1.5.2.10　parameters

视图 `parameters` 包含当前数据库中所有函数的参数的有关信息。只有那些当前用户能够访问（作为拥有者或具有某些特权）的函数才会被显示。

表 1.5.68: parameters 列

名称	类型	描述
SPECIFIC_CATALOG	information_schema.sql_identifier	routine 所属数据库
SPECIFIC_SCHEMA	information_schema.sql_identifier	routine 所属模式
SPECIFIC_NAME	information_schema.sql_identifier	routine 名称
ORDINAL_POSITION	information_schema.cardinal_number	参数位置，从 1 开始
PARAMETER_MODE	information_schema.character_data	IN 参数还是 OUT 参数
IS_RESULT	information_schema.yes_or_no	是否为返回参数

-续表-

名称	类型	描述
AS_LOCATOR	information_schema.yes_or_no	无实际意义，恒为'NO'
PARAMETER_NAME	information_schema.sql_identifier	参数名称
DATA_TYPE	information_schema.character_data	参数数据类型
CHARACTER_MAXIMUM_LENGTH	information_schema.cardinal_number	参数为字符类型时的最大字符长度
CHARACTER_OCTET_LENGTH	information_schema.cardinal_number	参数为字符类型时的最大字节长度
CHARACTER_SET_CATALOG	information_schema.sql_identifier	参数字符集所属数据库
CHARACTER_SET_SCHEMA	information_schema.sql_identifier	参数字符集所属模式
CHARACTER_SET_NAME	information_schema.sql_identifier	参数字符集名称
COLLATION_CATALOG	information_schema.sql_identifier	参数排序规则所属数据库
COLLATION_SCHEMA	information_schema.sql_identifier	参数排序规则所属模式
COLLATION_NAME	information_schema.sql_identifier	参数排序规则名称
NUMERIC_PRECISION	information_schema.cardinal_number	参数为数值类型精度

-续表-

名称	类型	描述
NUMERIC_PRECISION_RADIX	information_schema.cardinal_number	参数为数值类型精度的基数
NUMERIC_SCALE	information_schema.cardinal_number	参数为数值类型的标度
DATETIME_PRECISION	information_schema.cardinal_number	参数为日期类型的精度
INTERVAL_TYPE	nvarchar(30)	无实际意义，恒为 NULL
INTERVAL_PRECISION	smallint	无实际意义，恒为 NULL
USER_DEFINED_TYPE_CATALOG	information_schema.sql_identifier	无实际意义，恒为 NULL
USER_DEFINED_TYPE_SCHEMA	information_schema.sql_identifier	无实际意义，恒为 NULL
USER_DEFINED_TYPE_NAME	information_schema.sql_identifier	无实际意义，恒为 NULL
SCOPE_CATALOG	information_schema.sql_identifier	无实际意义，恒为 NULL
SCOPE_SCHEMA	information_schema.sql_identifier	无实际意义，恒为 NULL
SCOPE_NAME	information_schema.sql_identifier	无实际意义，恒为 NULL

1.5.2.11　referential_constraints

视图 `referential_constraints` 包含当前数据库中的所有引用（外键）约束。只有那些当前用户具有其引用表上写权限（作为拥有者或具有某些除 `SELECT` 之外的特权）的约束才会被显示。

表 1.5.69: referential_constraints 列

名称	类型	描述
CONSTRAINT_CATALOG	information_schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information_schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	information_schema.sql_identifier	约束名
UNIQUE_CONSTRAINT_CATALOG	information_schema.sql_identifier	唯一约束所属数据库
UNIQUE_CONSTRAINT_SCHEMA	information_schema.sql_identifier	唯一约束所属模式
UNIQUE_CONSTRAINT_NAME	information_schema.sql_identifier	唯一约束名称
MATCH_OPTION	information_schema.character_data	匹配情况
UPDATE_RULE	information_schema.character_data	更新规则
DELETE_RULE	information_schema.character_data	删除规则

1.5.2.12　routines

视图 `routines` 包含当前数据库中所有的函数和过程。只有那些当前用户能够访问（作为拥有者或具有某些特权）的函数和过程才会被显示。

表 1.5.70: routines 列

名称	数据类型	描述
specific_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
specific_schema	sql_identifier	包含该函数的模式名
specific_name	sql_identifier	该函数的“专用名”。这是一个在模式中唯一标识该函数的名称，即使该函数真正的名称已经被重载。专用名的格式尚未被定义，它应当仅被用来与指定例程名称的其他实例进行比较
routine_catalog	sql_identifier	包含该函数的数据库名（总是当前数据库）
routine_schema	sql_identifier	包含该函数的模式名
routine_name	sql_identifier	该函数的名字（在重载的情况下可能重复）
routine_type	character_data	FUNCTION 表示是一个函数，PROCEDURE 表示是一个过程
module_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
module_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
module_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
udt_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性

-续表-

名称	数据类型	描述
data_type	character_data	该函数的返回数据类型如果是一种内建类型，这里是该数据类型；如果是某种数组（此种情况见视图 element_types），则为 ARRAY；否则为 USER-DEFINED（此种情况中，该类型被标识在 type_udt_name 和相关列中）。如果是过程则此项为空
character_maximum_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_octet_length	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_catalog	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_schema	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
collation_name	sql_identifier	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
character_set_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
character_set_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
numeric_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型

-续表-

名称	数据类型	描述
numeric_precision_radix	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
numeric_scale	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
datetime_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_type	character_data	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
interval_precision	cardinal_number	总是为空，因为这种信息不适用于 KingbaseES 中的返回数据类型
type_udt_catalog	sql_identifier	该函数的返回数据类型所在的数据库名（总是当前数据库）。如果是过程则此项为空
type_udt_schema	sql_identifier	该函数的返回数据类型所在的模式名。如果是过程则此项为空
type_udt_name	sql_identifier	该函数的返回数据类型的名字。如果是过程则此项为空
scope_catalog	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_schema	sql_identifier	应用于一个 KingbaseES 中不可用的特性
scope_name	sql_identifier	应用于一个 KingbaseES 中不可用的特性
maximum_cardinality	cardinal_number	总是空，因为数组在 KingbaseES 中总是有无限制的最大势

-续表-

名称	数据类型	描述
dtd_identifier	sql_identifier	该函数返回数据类型的数据类型描述符的一个标识符，在从属于该函数的数据类型标识符之中唯一。这主要用于与这类标识符的其他实例进行连接（该标识符的指定格式没有被定义并且不保证在未来的版本中保持相同）
routine_body	character_data	如果该函数是一个 SQL 函数，则为 SQL，否则为 EXTERNAL
routine_definition	character_data	该函数的源文本（如果该函数不属于一个当前已被启用的角色，则为空）。（根据 SQL 标准，只有 routine_body 为 SQL 时这一列才适用。但是在 KingbaseES 中，它将会包含该函数被创建时所指定的任何源文本。）
external_name	character_data	如果这个函数是一个 C 函数，则为该函数的外部名称（链接符号），否则为空（这会产生和显示在 routine_definition 中相同的值）
external_language	character_data	该函数所用的语言
parameter_style	character_data	总是 GENERAL（SQL 标准定义了其他参数风格，但在 KingbaseES 中不可用）
is_deterministic	yes_or_no	如果该函数被声明为不变（在 SQL 标准中被称为确定性的），则为 YES，否则为 NO（你不能通过该信息模式查询在 KingbaseES 中可用的其他易变级别）

-续表-

名称	数据类型	描述
sql_data_access	character_data	总是 MODIFIES，表示该函数可能修改 SQL 数据。这种信息对 KingbaseES 没有用处
is_null_call	yes_or_no	如果该函数在任一参数为空时自动返回空值，则为 YES，否则为 NO。如果是过程则此项为空
sql_path	character_data	应用于一个 KingbaseES 中不可用的特性
schema_level_routine	yes_or_no	总是 YES（反例是一个用户定义类型的方法，这是在 KingbaseES 不可用的一种特性）
max_dynamic_result_sets	cardinal_number	应用于一个 KingbaseES 中不可用的特性
is_user_defined_cast	yes_or_no	应用于一个 KingbaseES 中不可用的特性
created	time_stamp	应用于一个 KingbaseES 中不可用的特性
last_altered	time_stamp	应用于一个 KingbaseES 中不可用的特性

1.5.2.13　schemata

视图 `schemata` 包含当前数据库中被当前用户（作为属主或具有某些特权）可访问的所有模式。

表 1.5.71: schemata 列

名称	类型	描述
CATALOG_NAME	information_schema.sql_identifier	模式所属数据库

-续表-

名称	类型	描述
SCHEMA_NAME	information _schema.sql _identifier	模式名称
SCHEMA_OWNER	information _schema.sql _identifier	模式拥有者
DEFAULT_CHARACTER_SET_CATALOG	information _schema.sql _identifier	默认字符集所属数据库
DEFAULT_CHARACTER_SET_SCHEMA	information _schema.sql _identifier	默认字符集所属模式
DEFAULT_CHARACTER_SET_NAME	information _schema.sql _identifier	默认字符集名称

1.5.2.14　table_constraints

视图 `table_constraints` 包含属于特定表的所有约束，这些表要满足的条件是：当前用户拥有表或者是当前用户在表上具有某种除 `SELECT` 之外的特权。

表 1.5.72: table_constraints 列

名称	类型	描述
CONSTRAINT_CATALOG	information _schema.sql_identifier	约束所属数据库
CONSTRAINT_SCHEMA	information _schema.sql_identifier	约束所属模式
CONSTRAINT_NAME	information _schema.sql_identifier	约束名称
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式

-续表-

名称	类型	描述
TABLE_NAME	information _schema.sql_identifier	约束所在的表的名称
CONSTRAINT_TYPE	information _schema.character _data	约束类型
IS_DEFERRABLE	information _schema.yes_or_no	约束检查是否可延迟
INITIALLY_DEFERRED	information _schema.yes_or_no	约束检查在事务开始时被设置为延迟模式

1.5.2.15　table_privileges

视图 `table_privileges` 记录当前数据库中在表或视图上所有被授予的特权，这些特权必须是被一个当前已被启用角色授出或者被授予给一个当前已被启用角色。

表 1.5.73: table_privileges 列

名称	类型	描述
GRANTOR	information _schema.sql_identifier	授权者
GRANTEE	information _schema.sql_identifier	被授予者
TABLE_CATALOG	information _schema.sql_identifier	表所属的数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名
PRIVILEGE_TYPE	information _schema.character _data	权限类别
IS_GRANTABLE	information _schema.yes_or_no	被授权者是否拥有赋予其他用户表上该权限的能力

1.5.2.16　tables

视图 `tables` 包含定义在当前数据库中的所有表和视图。只有那些当前用户能够访问（作为拥有者或具有某些特权）的表和视图才会被显示。

表 1.5.74: tables 列

名称	类型	描述
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名称
TABLE_TYPE	information _schema.character _data	表类型

1.5.2.17　view_column_usage

视图 `view_column_usage` 记录视图中的被查询的列信息，仅显示当前用户拥有的表上的视图信息。

表 1.5.75: view_column_usage 列

名称	类型	描述
VIEW_CATALOG	information _schema.sql_identifier	视图所属数据库
VIEW_SCHEMA	information _schema.sql_identifier	视图所属模式
VIEW_NAME	information _schema.sql_identifier	视图名
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库

-续表-

名称	类型	描述
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名称
COLUMN_NAME	information _schema.sql_identifier	列名称

1.5.2.18　view_table_usage

视图 `view_table_usage` 记录视图查询的基表信息，仅显示当前用户拥有的基表对应的视图信息。

表 1.5.76: view_table_usage 列

名称	类型	描述
VIEW_CATALOG	information _schema.sql_identifier	视图所属数据库
VIEW_SCHEMA	information _schema.sql_identifier	视图所属模式
VIEW_NAME	information _schema.sql_identifier	视图名称
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名称

1.5.2.19　views

视图 `views` 包含定义在当前数据库中的所有视图。只有当前用户能够访问（作为拥有者或具有某些特权）的视图才会被显示。

表 1.5.77: views 列

名称	类型	描述
TABLE_CATALOG	information _schema.sql_identifier	表所属数据库
TABLE_SCHEMA	information _schema.sql_identifier	表所属模式
TABLE_NAME	information _schema.sql_identifier	表名称
VIEW_DEFINITION	information _schema.character _data	视图定义
CHECK_OPTION	information _schema.character _data	WITH CHECK OPTION 选项的取值
IS_UPDATABLE	information _schema.yes_or_no	视图是否可更新

第2章　等待事件

等待事件 (Wait event) 是 KingbaseES 数据库性能分析的一项重要的参考指标。

本部分提供 KingbaseES 数据库系统等待事件的相关信息。

你可以通过下面的 SQL 语句查询/分析 KingbaseES 等待事件相关信息。

下面的例子展示了如何查看实时等待事件 sys_stat_activity 视图

```
SELECT pid, wait_event_type, wait_event FROM sys_stat_activity WHERE
wait_event is NOT NULL;

pid   | wait_event_type  | wait_event

------+------------------+---------------

2540  | Lock             | relation

6644  | LightLock        | ProcArrayLock

(2 rows)
```

每一行代表一个服务器进程, 显示进程当前所处的等待事件。

2.1　Activity

后台辅助进程活动等待：服务器进程处于闲置状态。这类等待被用于在其主处理循环中等待系统进程的活动。

表 2.1.1: Activity 类型等待事件

等待事件名称	描述
ArchiverMain	在归档进程的主循环中等待。

-续表-

等待事件名称	描述
AutoVacuumMain	在 autovacuum 启动器进程的主循环中等待。
BgWriterHibernate	在后台写入器进程中等待，休眠中。
BgWriterMain	在后台写入器进程的后台工作者的主循环中等待。
CheckpointerMain	在检查点进程的主循环中等待。
LogicalApplyMain	在逻辑应用进程的主循环中等待。
LogicalLauncherMain	在逻辑启动器进程的主循环中等待。
SysStatMain	在统计收集器进程的主循环中等待。
RecoveryWalAll	在恢复时等待来自于任意类型来源（本地、归档或流）的 WAL。
RecoveryWalStream	在恢复时等待来自于一个流的 WAL。
SysLoggerMain	在系统日志进程的主循环中等待。
WalReceiverMain	在 WAL 接收器进程的主循环中等待。
WalSenderMain	在 WAL 发送器进程的主循环中等待。
WalWriterMain	在 WAL 写入器进程的主循环中等待。
KshMain	在 KSH 进程的主循环中等待。
KwrMain	在 KWR 进程的主循环中等待。

2.2　BufferPin

服务器进程在等待访问一个数据缓冲区：此时没有其他进程在检查该缓冲区。如果另一个进程持有一个即将需要读取的数据缓存区的游标，缓冲区 pin 等待可能会被延长。

表 2.2.1: BufferPin 类型等待事件

等待事件名称	描述
BufferPin	等待在一个缓冲区上加 pin。

2.3 Client

客户端等待：服务器进程在一个套接字上等待来自用户应用的某种活动，并且该服务器预计将有预期之外的事件发生。

表 2.3.1: Client 类型等待事件

等待事件名称	描述
ClientRead	等待从客户端读取数据。
ClientWrite	等待向客户端写入数据。
LibPQWalReceiverConnect	在 WAL 接收器中等待建立与远程服务器的连接。
LibPQWalReceiverReceive	在 WAL 接收器中等待从远程服务器接收数据。
SSLOpenServer	在尝试连接期间等待 SSL。
WalReceiverWaitStart	等待 startup 进程发送流复制的初始数据。
WalSenderWaitForWAL	在 WAL 发送器进程中等待 WAL 被刷写。
WalSenderWriteData	在 WAL 发送器进程中处理来自 WAL 接收器的回复时等待任意活动。

2.4 Extension

扩展插件等待：服务器进程在一个扩展模块中等待活动。这一类等待用于跟踪自定义等待点。

表 2.4.1: Extension 类型等待事件

等待事件名称	描述
Extension	在一个扩展中等待。

2.5　IO

IO 等待：服务器进程在等待一次 IO 完成。

表 2.5.1: IO 类型等待事件

等待事件名称	描述
BufFileRead	等待从一个缓存的文件中读取。
BufFileWrite	等待向一个缓存的文件中写入。
ControlFileRead	等待从控制文件中读取。
ControlFileSync	等待控制文件到达稳定存储。
ControlFileSyncUpdate	等待对控制文件的更新到达稳定存储。
ControlFileWrite	等待一个对控制文件的写入。
ControlFileWriteUpdate	等待一个写操作更新控制文件。
CopyFileRead	在文件拷贝操作期间等待一个读操作。
CopyFileWrite	在文件拷贝操作期间等待一个写操作。
DataFileExtend	等待一个关系数据文件被扩充。
DataFileFlush	等待一个关系数据文件到达稳定存储。
DataFileImmediateSync	等待一个关系数据文件的立即同步到达稳定存储。
DataFilePrefetch	等待从一个关系数据文件中的一次异步预取。
DataFileRead	等待一次对一个关系数据文件的读操作。
DataFileSync	等待对一个关系数据文件的更改到达稳定存储。
DataFileTruncate	等待一个关系数据文件被截断。
DataFileWrite	等待一次对一个关系数据文件的写操作。
DSMFillZeroWrite	等待向一个动态共享内存备份文件中写零字节。
LockFileAddToDataDirRead	在向数据目录锁文件中增加一行时等待一个读操作。
LockFileAddToDataDirSync	在向数据目录锁文件中增加一行时等待数据到达稳定存储。
LockFileAddToDataDirWrite	在向数据目录锁文件中增加一行时等待一个写操作。
LockFileCreateRead	在创建数据目录锁文件期间等待读取。

-续表-

等待事件名称	描述
LockFileCreateSync	在创建数据目录锁文件期间等待数据到达稳定存储。
LockFileCreateWrite	在创建数据目录锁文件期间等待一个写操作。
LockFileReCheckDataDirRead	在重新检查数据目录锁文件的过程中等待一个读操作。
LogicalRewriteCheckpointSync	在一个检查点期间等待逻辑重写映射到达稳定存储。
LogicalRewriteMappingSync	在一次逻辑重写期间等待映射数据到达稳定存储。
LogicalRewriteMappingWrite	在一次逻辑重写期间等待对映射数据的写操作。
LogicalRewriteSync	等待逻辑重写映射到达稳定存储。
LogicalRewriteWrite	等待对逻辑重写映射的写操作。
RelationMapRead	等待对关系映射文件的读操作。
RelationMapSync	等待关系映射文件到达稳定存储。
RelationMapWrite	等待对关系映射文件的写操作。
ReorderBufferRead	在重排序缓冲区管理期间等待一个读操作。
ReorderBufferWrite	在重排序缓冲区管理期间等待一个写操作。
ReorderLogicalMappingRead	在重排序缓冲区管理期间等待对一个逻辑映射的读操作。
ReplicationSlotRead	等待对一个复制槽控制文件的读操作。
ReplicationSlotRestoreSync	在把一个复制槽控制文件恢复到内存的过程中等待它到达稳定存储。
ReplicationSlotSync	等待一个复制槽控制文件到达稳定存储。
ReplicationSlotWrite	等待对一个复制槽控制文件的写操作。
SLRUFlushSync	在检查点或者数据库关闭期间等待 SLRU 数据到达稳定存储。
SLRURead	等待对一个 SLRU 页面的读操作。
SLRUSync	等待 SLRU 数据在一个页面写之后到达稳定存储。
SLRUWrite	等待一个 SLRU 页面上的写操作。
SnapbuildRead	等待一个序列化历史目录快照的读操作。
SnapbuildSync	等待一个序列化历史目录快照到达稳定存储。

-续表-

等待事件名称	描述
SnapbuildWrite	等待一个序列化历史目录快照的写操作。
TimelineHistoryFileSync	等待一个通过流复制接收到的时间线历史文件到达稳定存储。
TimelineHistoryFileWrite	等待一个通过流复制接收到的时间线历史文件的读操作。
TimelineHistoryRead	等待一个时间线历史文件上的读操作。
TimelineHistorySync	等待一个新创建的时间线历史文件达到稳定存储。
TimelineHistoryWrite	等待一个新创建的时间线历史文件上的写操作。
TwophaseFileRead	等待一个两阶段状态文件的读操作。
TwophaseFileSync	等待一个两阶段状态文件到达稳定存储。
TwophaseFileWrite	等待一个两阶段状态文件的写操作。
WALBootstrapSync	在自举期间等待 WAL 到达稳定存储。
WALBootstrapWrite	在自举期间等待一个 WAL 页面的写操作。
WALCopyRead	在通过拷贝一个已有 WAL 段创建一个新的 WAL 段时等待一个读操作。
WALCopySync	等待一个通过拷贝已有 WAL 段创建的新 WAL 段到达稳定存储。
WALCopyWrite	在通过拷贝一个已有 WAL 段创建一个新的 WAL 段时等待一个写操作。
WALInitSync	等待一个新初始化的 WAL 文件到达稳定存储。
WALInitWrite	在初始化一个新的 WAL 文件期间等待一个写操作。
WALRead	等待一次对一个 WAL 文件的读操作。
WALSenderTimelineHistoryRead	在 walsender 的时间线命令期间等待对一个时间线历史文件的读操作。
WALSync	等待 WAL 文件到达持久的存储空间。
WALSyncMethodAssign	在指派 WAL 同步方法时等待数据到达稳定存储。
WALWrite	等待一次对一个 WAL 文件的写操作。

2.6 IPC

进程间通讯等待：服务器进程在等待来自服务器中另一个进程的某种活动。

表 2.6.1: IPC 类型等待事件

等待事件名称	描述
BgWorkerShutdown	等待后台工作者关闭。
BgWorkerStartup	等待后台工作者启动。
BtreePage	等待继续并行 B-树扫描所需的页号变得可用。
CheckpointDone	等待检查点完成。
CheckpointStart	等待检查点开始。
ClogGroupUpdate	等待组领袖在事务结束时更新事务状态。
ExecuteGather	在执行 Gather 节点时等待来自子进程的活动。
Hash/Batch/Allocating	等待一个选出的 Parallel Hash 参与者分配哈希表。
Hash/Batch/Electing	在选出一个 Parallel Hash 参与者来分配一个哈希表。
Hash/Batch/Loading	等待其他 Parallel Hash 参与者完成装载哈希表。
Hash/Build/Allocating	等待一个选出的 Parallel Hash 参与者分配初始哈希表。
Hash/Build/Electing	在选出一个 Parallel Hash 参与者以分配初始哈希表。
Hash/Build/HashingInner	等待其他 Parallel Hash 参与者完成对内关系的哈希操作。
Hash/Build/HashingOuter	等待其他 Parallel Hash 参与者完成对外关系的哈希操作。
Hash/GrowBatches/Allocating	等待一个选出的 Parallel Hash 参与者分配更多批次。
Hash/GrowBatches/Deciding	在选出一个 Parallel Hash 参与者决定未来的批次增长。

-续表-

等待事件名称	描述
Hash/GrowBatches/Electing	在选出一个 Parallel Hash 参与者分配更多批次。
Hash/GrowBatches/Finishing	在等待一个选出的 Parallel Hash 参与者决定未来的批次增长。
Hash/GrowBatches/Repartitioning	等待其他 Parallel Hash 参与者完成重新分区。
Hash/GrowBuckets/Allocating	等待一个选出的 Parallel Hash 参与者完成更多桶的分配。
Hash/GrowBuckets/Electing	在选出一个 Parallel Hash 参与者分配更多桶。
Hash/GrowBuckets/Reinserting	等待其他 Parallel Hash 参与者完成将元组插入到新桶的操作。
LogicalSyncData	等待逻辑复制的远程服务器发送用于初始表同步的数据。
LogicalSyncStateChange	等待逻辑复制的远程服务器更改状态。
MessageQueueInternal	等待其他进程被挂接到共享消息队列。
MessageQueuePutMessage	等待把一个协议消息写到一个共享消息队列。
MessageQueueReceive	等待从一个共享消息队列接收字节。
MessageQueueSend	等待向一个共享消息队列中发送字节。
ParallelBitmapScan	等待并行位图扫描被初始化。
ParallelCreateIndexScan	等待并行 CREATE INDEX 工作者完成堆扫描。
ParallelFinish	等待并行工作者完成计算。
ProcArrayGroupUpdate	等待组领袖在事务结束时清除事务 ID。
Promote	等待备用节点升级。
ReplicationOriginDrop	等待一个复制源头变得不活跃以便被删除。

-续表-

等待事件名称	描述
ReplicationSlotDrop	等待一个复制槽变得不活跃以便被删除。
SafeSnapshot	等待一个用于 READ ONLY DEFERRABLE 事务的快照。
SyncRep	在同步复制期间等待来自远程服务器的确认。

2.7　LightLock

后端在等待一个轻量级锁：轻量级锁用于保护共享内存中的数据结构，防止并发问题。

表 2.7.1: LightLock 类型等待事件

等待事件名称	描述
ShmemIndexLock	等待在共享内存中查找或者分配空间。
OidGenLock	等待分配或者赋予一个 OID。
XidGenLock	等待分配或者赋予一个事务 ID。
ProcArrayLock	等待在事务结尾得到一个快照或者清除事务 ID。
SInvalReadLock	等待从共享无效消息队列中检索或者移除消息。
SInvalWriteLock	等待在共享无效消息队列中增加一个消息。
WALBufMappingLock	等待在 WAL 缓冲区中替换一个页面。
WALWriteLock	等待 WAL 缓冲区被写入到磁盘。
ControlFileLock	等待读取或者更新控制文件或创建一个新的 WAL 文件。
CheckpointLock	等待执行检查点。
CLogControlLock	等待读取或者更新事务状态。
SubtransControlLock	等待读取或者更新子事务信息。
MultiXactGenLock	等待读取或者更新共享多事务状态。

-续表-

等待事件名称	描述
MultiXactOffsetControlLock	等待读取或者更新多事务偏移映射。
MultiXactMemberControlLock	等待读取或者更新多事务成员映射。
RelCacheInitLock	等待读取或者写入关系缓冲区初始化文件。
CheckpointerCommLock	等待管理 fsync 请求。
TwoPhaseStateLock	等待读取或者更新预备事务的状态。
TablespaceCreateLock	等待创建或者删除表空间。
BtreeVacuumLock	等待读取或者更新一个 B-树索引的 vacuum 相关的信息。
AddinShmemInitLock	等待管理共享内存中的空间分配。
AutovacuumLock	自动清理工作者或者启动器等待更新或者读取自动清理工作者的当前状态。
AutovacuumScheduleLock	等待确认选中进行清理的表仍需要清理。
SyncScanLock	等待为同步扫描得到一个表上扫描的开始位置。
RelationMappingLock	等待更新用来存储目录到文件节点映射的关系映射文件。
AsyncCtlLock	等待读取或者更新共享通知状态。
AsyncQueueLock	等待读取或者更新通知消息。
SerializableXactHashLock	等待检索或者存储有关可序列化事务的信息。
SerializableFinishedListLock	等待访问已结束可序列化事务的列表。
SerializablePredicateLockListLock	等待在由可序列化事务持有的所列表上执行一个操作。
OldSerXidLock	等待读取或者记录冲突的可序列化事务。
SyncRepLock	等待读取或者更新有关同步复制的信息。
BackgroundWorkerLock	等待读取或者更新后台工作者状态。
DynamicSharedMemoryControlLock	等待读取或者更新动态共享内存状态。
AutoFileLock	等待更新 kingbase.auto.conf 文件。
ReplicationSlotAllocationLock	等待分配或者释放一个复制槽。
ReplicationSlotControlLock	等待读取或者更新复制槽状态。

-续表-

等待事件名称	描述
CommitTsControlLock	等待读取或者更新事务提交时间戳。
CommitTsLock	等待读取或者更新事务时间戳的最新设置值。
ReplicationOriginLock	等待设置、删除或者使用复制源头。
MultiXactTruncationLock	等待读取或者阶段多事务信息。
OldSnapshotTimeMapLock	等待读取或者更新旧的快照控制信息。
LogicalRepWorkerLock	等待逻辑复制工作者上的动作完成。
CLogTruncationLock	等待执行 txid_status 或更新可用的最新事务 id。
clog	在等地 clog(事务状态) 缓冲区上的 I/O。
commit_timestamp	等待提交时间戳缓冲区上的 I/O。
subtrans	等待子事务缓冲区上的 I/O。
multixact_offset	等待多事务偏移缓冲区上的 I/O。
multixact_member	等待多事务成员缓冲区上的 I/O。
async	等待 async（通知）缓冲区上的 I/O。
oldserxid	等待 oldserxid 缓冲区上的 I/O。
wal_insert	等待把 WAL 插入到一个内存缓冲区。
buffer_content	等待读取或者写入内存中的一个数据页。
buffer_io	等待一个数据页面上的 I/O。
replication_origin	等待读取或者更新复制进度。
replication_slot_io	等待一个复制槽上的 I/O。
proc	等待读取或者更新 fast-path 锁信息。
buffer_mapping	等待把一个数据块与缓冲池中的一个缓冲区关联。
lock_manager	等待增加或检查用于后端的锁，或者等待加入或退出一个锁定组。
predicate_lock_manager	等待增加或者检查谓词锁信息。
serializable_xact	等待在并行查询中对一个可序列化事务执行操作。

-续表-

等待事件名称	描述
parallel_query_dsa	等待并行查询动态共享内存分配锁。
tbm	等待 TBM 共享迭代器锁。
parallel_append	在 Parallel Append 计划执行期间等待选择下一个子计划。
parallel_hash_join	在 Parallel Hash 计划执行期间等待分配或交换一块内存或者更新计数器。

2.8 Lock

后端在等待一个重量级锁：重量级锁也被称为锁管理器锁或简单锁，主要保护 SQL 可见的对象，例如表，同时也被用于确保特定内部操作的互斥，如关系扩展。

表 2.8.1: Lock 类型等待事件

等待事件名称	描述
relation	等待获得一个关系上的锁。
extend	等待扩展一个关系。
page	等待获得一个关系上的页面的锁。
tuple	等待获得一个元组上的锁。
transactionid	等待一个事务结束。
virtualxid	等待获得一个虚拟 xid 锁。
speculative token	等待获取一个 speculative insertion lock。
object	等待获得一个非关系数据库对象上的锁。
userlock	等待获得一个用户锁。
advisory	等待获得一个咨询用户锁。

2.9 Timeout

超时等待：服务器进程在等待一次超时发生。

表 2.9.1: Timeout 类型等待事件

等待事件名称	描述
BaseBackupThrottle	当有限流活动时在基础备份期间等待。
SysSleep	在调用 sys_sleep 的进程中等待。
RecoveryApplyDelay	在恢复时等待应用 WAL，因为它被延迟了。

第3章　插件参考

3.1　扩展插件概述

KingbaseES 开发了大量的扩展包。如：dbms_output，dbms_ddl，dbms_metadata 等。在日常使用过程中，如果功能在 KingbaseES 中没有默认支持，可以在操作系统中进入 $KINGBASE_HOME/ Server/share/extension 目录，搜索扩展 sql 文件来确认是否已有相应的扩展。也可以查询当前版本支持的扩展插件：

```
select * from sys_available_extensions;
```

执行 \dx 查询当前已安装的扩展插件：

```
test=# \dx
                                  已安装扩展列表
        名称          | 版本 |  架构模式  |          描述
----------------------+------+------------+-----------------------------------------------
 dbms_ddl             | 1.0  | sys        | DBMS_DDL system package
 dbms_output          | 1.0  | sys        | DBMS_OUTPUT system package
 dbms_utility         | 1.0  | sys        | dbms_utility extension package
 kdb_license          | 1.0  | sys_catalog| kdb_license extension
 kdb_oracle_datatype  | 1.0  | sys        | kdb_oracle_datatype extension
 kdb_schedule         | 3.0  | sys        | A KingbaseES job scheduler
```

表 plug-TABLE 列出了 KingbaseES 所提供的扩展插件。后文为每个扩展插件的详细文档。

插件名称	用途
amcheck	让用户能验证关系逻辑结构的一致性
auto_bmr	支持自动块修复
auto_explain	自动记录 SQL 语句的执行计划，并写入到 KingbaseES 日志文件中
backup_pri	允许物理备份 sys_basebackup 连接到目标数据库，执行物理备份操作
bloom	提供了 bloom 过滤器索引

-续表-

插件名称	用途
dblink	支持在一个数据库会话中连接到其他 Kingbase 数据库
dbms_ddl	创建一个 PL/SQL 包体、函数、过程。
dbms_lob	对 CLOB/BLOB 大对象就行的操作
dbms_metadata	创建特定类型对象的 DDL 语句
dbms_mview	提供对物化视图的一些操作
dbms_output	提供将文本行写入缓冲区、供以后提取和显示的功能
dbms_random	提供一个随机数或者字符的生成器
dbms_session	实现对 session 中内容的操作
dbms_sql	定义了一系列的过程和函数，专门用于动态 SQL 语句的操作
dbms_utility	提供一些具有通用功能的子程序
dict_int	支持了对于数字的 text 类型搜索字典
dict_xsyn	提供一个具有附加全文搜索功能的字典模板
file_fdw	用来访问存储在外部文件中的数据
fix_security_hole	禁止通过 copy 执行操作系统命令
ftutilx	流版式文件内容抽取
fuzzystrmatch	提供了两个字符串之间的编辑距离的相关函数
http	KingbaseES 的 http 客户端，允许在数据库内检索网页
identity_pwdexp	管理和检查用户口令的有效期
kbcrypto	提供如 rc4,sm3,sm4 等加密函数的访问接口
kbrowlocks	提供了一个函数来显示指定表的行级锁信息
kbstattuple	提供了多种函数来获得关系中元组的统计信息
kdb_database_link	用于访问外部数据库对象
kdb_ddl	用于生成数据库对象 DDL 语句
kdb_license	初始化内置，基本类型

-续表-

插件名称	用途
kdb_mysql_rum	MySQL 模式下的 RUM 索引类型
kdb_protect	用于保护 database、schema、table 等逻辑对象不被删除
kdb_raw	实现对 raw 数据类型的操作
kdb_resource_group	实现资源组功能
kdb_schedule	基于后台进程来实现自动作业功能
kdb_utils_function	工具函数, 包括 xml, 字符操作等
kdb_xmltype	Oracle 兼容 XMLTYPE 类型和相关的操作函数
kingbase_fdw	用来访问存储在外部 KingbaseES 服务器中的数据
ktrack	块级别增量备份引擎
ltree	包含 ltree 和 lquery 类型的实现，为树状结构组织的数据提供索引和查询
mysql_fdw	用来访问存储在外部 Mysql 数据库中的数据
oracle_fdw	用来访问存储在外部 Oracle 数据库中的数据
owa_util	支持 OWA_UTIL 系统包中的 WHO_CALLED_ME 过程
pageinspect	提供函数可以查看数据库页面的内容
passwordcheck	管理和检查用户口令的复杂度
passwordhistory	管理用户口令历史，检查用户更新的口令
pldbgapi	PG 语法调试
plsql_pldbgapi	oracle 语法调试
plsql_plprofiler	收集分析器（性能）数据以提高性能或确定 PL/SQL 应用程序的代码覆盖率
repmgr	详见《读写分离集群使用手册》repmgr
restricted_dba	实现对超级用户权限进行限制
roledisable	在本地库内使角色失效, 使失效的角色再生效
rum	RUM 索引类型
security_utils	支持安全方面的功能

-续表-

插件名称	用途
send_mails	审计入侵检测的邮件告警功能
sepapower	三权分立
src_restrict	来源限制
sso_update_user	进一步划分安全员和管理员权限，满足特殊场景三权分立需求
sys_anon	初始化内置，数据脱敏。参见数据脱敏
sys_audlog	账户异常登录锁定和账户登录信息显示
sys_buffercache	实时检查共享缓冲区工具
sys_bulkload	快速加载数据的命令行工具
sys_freespacemap	提供了查看 FSM 页面的方法
sys_jieba	用于全文搜索的中文分词插件
sys_kwr	负载信息记录和保存，并提供相关的性能报告
sys_sqltune	语句级调优及资源监控功能，并提供相应报告
sys_prewarm	将关系数据载入到操作系统缓冲区或者 KingbaseES 缓冲区，实现数据预热
sys_qualstats	保存 where 和 join 子句中的谓词统计信息，生成索引建议
sys_recovery	从死元组中读取数据
sys_rman	查询备份集和归档信息
sys_spacequota	针对表空间限额的操作
sys_squeeze	无锁清理空闲表空间的工具
sys_stat_statements	追踪所有 SQL 语句的执行统计信息
sys_trgm	提供基于 trigram 匹配的字母数字文本相似度的函数和操作符
sys_visibility	提供了查看表的 VM 页数据和页级别可见性的函数
sysaudit	初始化内置，审计。参见数据库审计
sysencrypt	实现数据加解密
sysmac	初始化内置，强制访问控制。参见强制访问控制

-续表-

插件名称	用途
sysprivilege	实现对数据库系统 ANY 权限的扩展
sysreuse_residual_data	资源申请和释放时清除介质上的残留信息以达到客体重用的要求
utl_file	补充了一些对于文件操作的函数
uuid-ossp	uuid 生成工具
walminer	提供从 WAL 日志解析 REDO/UNDO SQL 语句的功能
zhparser	用于全文搜索的中文分词插件

一个扩展包通常需要一个包含创建该扩展的对象的 SQL 命令的脚本文件以及一个指定扩展本身的一些基本属性的控制文件。如果扩展包括 C 代码，通常还有一个 C 代码编译而成的共享库文件。控制文件和脚本默认被放在数据库的安装目录下的 Server/share/extension 目录中，C 编译的共享库文件则放在 Server/lib 中。

3.2 amcheck

3.2.1 插件 amcheck 简介

amcheck 提供的函数可以让用户能验证关系逻辑结构的一致性，如果结构错误会报错。B 树的验证函数会检查逻辑结构中应该满足的多种条件。

索引扫描以及其他操作的正确性依赖这些结构的正确性。例如，一些函数会验证 B 树页面中所有索引项都按照逻辑顺序（比如 text 类型列上的 B 树索引项应该按照词典顺序）排列。

如果这个条件由于某种原因不成立，我们可以预计受影响页面上的二分搜索将无法正确的执行索引扫描，导致整个 SQL 查询得到错误的结果。

amcheck 函数默认只能由超级用户使用。

- 插件名为 amcheck
- 插件版本 V1.2

3.2.2 插件 amcheck 加载方式

```
CREATE EXTENSION amcheck;
```

备注:

创建扩展前，需要关闭禁用多态函数参数，执行’set ora_forbid_func_polymorphism to off;’。

3.2.3 插件 amcheck 的参数配置

无需配置任何参数

3.2.4 插件 amcheck 的使用方法

插件提供以下函数

```
bt_index_check(index regclass, heapallindexed boolean) returns void
```

bt_index_check 验证指定的 B 树索引的逻辑结构正确性。

示例

```
CREATE TABLE t001(id int,i int,j int,msg text);
CREATE INDEX sys_package_oid_index ON t001(id);

SELECT bt_index_check('sys_package_oid_index', false);
  bt_index_check
 ----------------


 (1 row)
```

示例中函数执行没有报错，指定索引的逻辑结构是正确的。

bt_index_check 对目标索引及其所属的堆表加 AccessShareLock。这个锁模式和简单 SELECT 语句在关系上加的锁模式相同。bt_index_check 不验证跨越父子关系的结构条件。在 heapallindexed 为 true 时会验证堆表的所有元组都有对应的索引项存在。这个行为是在完整验证和减少对业务的影响间的权衡，函数可以作为快速的轻量验证使用。

```
bt_index_parent_check(index regclass, heapallindexed boolean, rootdescend boolean) returns void
```

bt_index_parent_check 验证指定的 B 树索引的逻辑结构正确性。

当 heapallindexed 参数为 true 时，函数验证堆表的所有元组都有对应的索引项存在。和 bt_index_check 不同，函数会验证索引结构中的父子关系，会检查父节点是否有缺失的 down link。当可选参数 rootdescend 值为 true 时，函数会通过从根页面开始搜索来查找叶节点的元组。bt_index_parent_check 执行的检查是 bt_index_check 执行检查的超集。

bt_index_parent_check 对目标索引及其所属的堆表加 ShareLock，这个所模式会阻塞 INSERT、UPDATE 以及 DELETE 的并发数据修改，同时防止所属堆表被 VACUUM。

注意该函数只在其运行期间而不是整个事务期间持有锁。bt_index_parent_check 的额外验证有更大可能检测到索引结构的异常，但与 bt_index_check 不同，函数不能在开启 hot_standby 的备机上使用。

3.2.5　插件 amcheck 卸载方法

```
DROP EXTENSION amcheck;
```

3.2.6　插件 amcheck 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION amcheck UPDATE TO '1.1';
```

3.3　auto_bmr

3.3.1　插件 auto_bmr 简介

主数据库访问系统表数据、索引、持久化用户表数据、索引时，从磁盘读取数据块至共享缓冲区，如果检测到坏块，自动从备节点获取坏块的副本，并修复坏块。

- 插件名为 auto_bmr
- 插件版本 V1.1

3.3.2　插件 auto_bmr 加载方式

```
CREATE EXTENSION auto_bmr;
```

3.3.3　插件 auto_bmr 的参数配置

1、auto_bmr.enable_auto_bmr：是否开启自动块修复功能，取值范围 0/1，默认为 1，支持 reload。

2、auto_bmr.auto_bmr_req_timeout：块自动修复的超时时间，取值范围 0 至 INT_MAX，默认为 60s，0 表示不限制，支持 set。

3、auto_bmr.auto_bmr_sess_threshold：session 级最大块修复数量，取值范围 0 至 INT_MAX，默认为 100，0 表示不限制，支持 set。

4、auto_bmr.auto_bmr_sys_threshold：系统级最大块修复数量，取值范围 0 至 INT_MAX，默认为 1024，0 表示不限制，支持 reload。

3.3.4　插件 auto_bmr 的使用方法

块自动修复功能当检测到坏块和坏块修复成功时，在日志中有对应的 WARNING 提示信息。检测到坏块时提示信息如下所示：

```
2021-10-12 18:59:06.264 CST [9128] WARNING:  page is invalid: base/16089/16385, blockNum: 3
```

修复坏块成功时提示信息如下所示：

```
2021-10-12 18:59:06.292 CST [9128] WARNING:  repair invalid page: base/16089/16385, blockNum: 3
successfully.
```

如果在日志中有类似如下提示：

```
WARNING:  Exec get buffer page failed,errMsg:ERROR:  function public.get_lsn_reached_page(integer, integer,
integer, integer, integer, integer) does not exist
LINE 1: select public.get_lsn_reached_page(1663, 16832, 16881, 0, 1,...

HINT:  No function matches the given name and argument types. You might need to add explicit type casts.
```

需创建 auto_bmr 扩展插件。

```
test=# create extension auto_bmr;
CREATE EXTENSION
```

如果系统修复的块数量超过系统级最大块修复数量，可通过 select reset_auto_bmr_sys_bad_blk() 重置系统块修复数量。

```
test=# select reset_auto_bmr_sys_bad_blk();
reset_auto_bmr_sys_bad_blk
----------------------------

(1 行记录)
```

如果一个 session 修复的块数量超过 session 级最大块修复数量，可通过 select reset_auto_bmr_sess_bad_blk() 重置 session 修复的块数量。

```
test=# select reset_auto_bmr_sess_bad_blk();
reset_auto_bmr_sess_bad_blk
-----------------------------
```

```
(1 行记录)
```

3.3.5 插件 auto_bmr 卸载方法

```
DROP EXTENSION auto_bmr;
```

3.3.6 插件 auto_bmr 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION auto_bmr UPDATE TO '1.1';
```

3.4 auto_explain

3.4.1 插件 auto_explain 简介

auto_explain 是 KingbaseES 的一个扩展插件，主要用于在服务器端自动记录 SQL 语句的执行计划，写入到 KingbaseES 日志文件中。

插件在被服务程序加载后，其相关功能生效，插件在服务端程序启动时不是默认加载。

- 插件名为 auto_explain
- 插件版本 V1.0

3.4.2 插件 auto_explain 加载方式

如需 KingbaseES 数据库启动时默认加载该插件，将其添加到 kingbase.conf 文件的 shared_preload_libraries 中。

如需在数据库启动后使用该插件，在 ksql 中使用 LOAD 命令即可。

示例：

```
shared_preload_libraries = 'auto_explain'
```

3.4.3 插件 auto_explain 的参数配置

`auto_explain.log_min_duration`

SQL 语句执行时间的阈值，所有时间超过这个阈值的 SQL 语句的执行计划都会被记录到日志中，单位为毫秒，0 表示记录所有语句，-1 表示不记录

`auto_explain.log_analyze`

该参数控制是否加入 analyze 功能，即除了计划外把实际执行时间等信息也打出来，注意该参数打开后会对性能产生较大影响，默认值为 off

`auto_explain.log_settings`

该参数控制是否打印已修改的配置选项的信息，输出中仅包含影响执行计划的配置选项，默认值为 off

`auto_explain.log_verbose`

该参数控制是否在执行计划中打印更为详细的输出信息，默认值为 off

`auto_explain.log_buffers`

该参数控制是否在执行计划中打印缓冲区使用的信息，默认值为 off

`auto_explain.log_wal`

该参数控制是否打印 WAL 日志的信息，默认值为 off

`auto_explain.log_triggers`

该参数控制是否打印触发器的统计信息，注意只有当 auto_explain.log_analyze 参数开启时，该参数才有效，默认值为 off

`auto_explain.log_format`

该参数用于控制执行计划的输出格式，可以是 text、xml、json、yaml，默认值为 text

`auto_explain.log_level`

该参数用于控制 auto_explain 插件的日志级别，其值同服务器的日志级别相同，默认值为 LOG

`auto_explain.log_nested_statements`

该参数用于控制是否支持嵌套语句（在一个函数内执行的语句）的执行计划输出，当它关闭时，只记录顶层语句的计划，默认值为 off

`auto_explain.log_timing`

该参数用于控制是否打印计划中每个节点的计时信息，注意只有当 auto_explain.log_analyze 参数开启时，该参数才有效，默认值为 off

`auto_explain.sample_rate`

该参数用于控制打印会话中语句计划的比例，默认值为 1，表示输出所有语句的计划

3.4.4 插件 auto_explain 的使用方法

KingbaseES 加载 auto_explain 插件后，根据其参数配置，将 SQL 语句的执行计划输出到日志文件里。

3.4.5 插件 auto_explain 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.4.6 插件 auto_explain 升级方法

auto_explain 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

3.5 backtrace

3.5.1 插件 backtrace 简介

插件 backtrace 是 KingbaseES 的一个扩展插件。主要功能是在服务端程序崩溃时捕获宕机堆栈信息，并将 core 堆栈信息打印输出到前端或日志文件里。可用于协助分析 KingbaseES 宕机的原因。通常 release 版本的 KingbaseES 需要使用 debug 文件协助分析宕机原因。

插件 backtrace 相关功能在服务程序崩溃时自动触发，无须人为干预，插件在服务端程序启动时默认加载。

- 插件名为 backtrace
- 插件版本 V1.0

3.5.2 插件 backtrace 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'backtrace'
```

3.5.3 插件 backtrace 的参数配置

无需配置任何参数

3.5.4 插件 backtrace 的使用方法

KingbaseES 加载 backtrace 插件后，程序崩溃时会自动捕获宕机的堆栈信息，并将 core 堆栈信息打印输出到前端或日志文件里。

3.5.5 插件 backtrace 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.5.6 插件 backtrace 升级方法

backtrace 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.6 backup_pri

3.6.1 插件 backup_pri 简介

backup_pri 是 KingbaseES 的一个扩展插件，主要用于对备份/恢复操作使用权限限制，插件增加了一个新的管理特权 `SYSBACKUP`，允许物理备份 sys_basebackup 连接到目标数据库，执行物理备份操作。

- 插件名为 backup_pri
- 插件版本 V1.0

3.6.2 插件 backup_pri 加载方式

在使用 backup_pri 之前，需要将他添加到 kingbase.conf 文件的 `shared_preload_libraries` 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'backup_pri'
```

在使用 `create extension` 命令进行创建扩展：

```
create extension backup_pri;
```

3.6.3　插件 backup_pri 的参数配置

backup_pri.enable_backup_pri

启用备份恢复权限功能，取值范围为 ture 或者 false，缺省为 false。

```
show backup_pri.enable_backup_pri;
backup_pri.enable_backup_pri
--------------------------------
off
(1 row)

alter system set backup_pri.enable_backup_pri to on;
select sys_reload_conf();
show backup_pri.enable_backup_pri;
backup_pri.enable_backup_pri
--------------------------------
on
(1 row)
```

3.6.4　插件 backup_pri 使用方法

3.6.4.1　备份恢复权限的授予与回收

在数据库初始化之后，SYSTEM 用户级 SUPERUSER 用户不具有备份恢复的权限; 但具有该权限的授予与回收的权利，SYSTEM 用户级 SUPERUSER 用户也可以给自身赋权。

授予与回收的语法：

```
alter user u1 sysbackup;

alter user u1 nosysbackup;
```

同时建用户的时候可以直接授予或回收

```
create user u1 sysbackup;

create user u2 nosysbackup;
```

不带默认 nosysbackup，或者建库初始化时的用户也是默认 nosysbackup 的，想要赋予权限，可以用 alter 语句实现。

删除用户的时候语法：

```
drop user u1;
```

会自动删除这个用户的备份恢复权限，和其他系统权限一样，记录将不存在。

3.6.4.2　备份恢复权限流程

存储 sysbackup 权限的系统表 sys_privilege。物理备份 sys_basebackup 时，检查是否持有这个权限，若没有，报错退出。

具体流程如下：

1. 备份恢复权限与其他普通权限一样，都是限制一些用户进行指定的操作;
2. 备份恢复权限是用户级别的，在执行备份恢复时，步骤如下：
    - 检查当前用户是否对要备份还原的数据库有备份恢复的权限;
    - 检查备份恢复权限，才可以进行备份恢复的操作;
    - 无备份恢复权限，SYSTEM 或 SUPERUSER 用户为其授予备份恢复权限。才可以执行备份恢复的操作。在执行备份恢复时，增加一项备份恢复的权限检查;

3.6.4.3　相关系统视图

sysbackup 权限相关的三个系统视图如下：

- backup_pri.dba_sys_privs

查询出所有的用户的 sysbackup 权限和其他系统权限，若用户没有 sysbackup 权限，有可能显示的是空，或者 nosysbackup 两种情况。

backup_pri.dba_sys_privs 视图的每个字段意义如下表所示：

表 3.6.1: backup_pri.dba_sys_privs 视图

列名	描述
username	用户名
userid	用户 id
superuser	超级用户
inherit	继承
createrole	创建角色

-续表-

列名	描述
createdb	创建库
canlogin	登录
replication	流复制
bypassrls	安全测试 RLS
sysbackup	备份恢复

- backup_pri.user_sys_privs

查询出当前的用户的 sysbackup 权限和其他系统权限，若当前用户没有 sysbackup 权限，有可能显示的是空，或者 nosysbackup 两种情况。

backup_pri.user_sys_privs 视图的每个字段意义如下表所示：

表 3.6.2: backup_pri.user_sys_privs 视图

列名	描述
username	用户名
userid	用户 id
superuser	超级用户
inherit	继承
createrole	创建角色
createdb	创建库
canlogin	登录
replication	流复制
bypassrls	安全测试 RLS
sysbackup	备份恢复

- backup_pri.role_sys_privs

查询出角色的 sysbackup 权限和其他系统权限，若角色没有 sysbackup 权限，有可能显示的是空，或者 nosysbackup 两种情况。

backup_pri.role_sys_privs 视图的每个字段意义如下表所示：

表 3.6.3: backup_pri.role_sys_privs 视图

列名	描述
rolename	角色名
roleid	角色 id
superuser	超级用户
inherit	继承
createrole	创建角色
createdb	创建库
canlogin	登录
replication	流复制
bypassrls	安全测试 RLS
sysbackup	备份恢复

3.6.5　插件 backup_pri 卸载方法

通过 `drop extension` 来卸载插件：

```
drop extension backup_pri;
```

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.6.6　插件 backup_pri 升级方法

backup_pri 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.7 bloom

3.7.1 插件 bloom 简介

插件 bloom 是 KingbaseES 的一个扩展插件。bloom 提供了一种基于 bloom 过滤器的索引访问方法。

bloom 过滤器是一种节省空间的数据结构，用于测试元素是否是集合的成员。对于索引访问方法，它允许通过签名快速排除不匹配的元组。签名的大小在索引创建是确定。

签名是索引属性的一种有损表示，并且因此容易报告误报。也就是说，当元素不在集合中时，可能会报告该元素在集合中。因此，必须始终使用堆条目中实际属性值重新检查索引的搜索结果。更大的签名可以降低误报的机率。从而减少无用的堆访问次数，但也会让索引更大，从而降低扫描速度。

当一个表有许多属性并且查询测试他们的任意组合时，这种类型的索引最有效，传统的 btree 索引比 bloom 索引快，但它可能需要许多 btree 索引来支持所有可能的查询。而 bloom 索引只需要一个。请注意，bloom 索引只支持等值查询，而 btree 索引也可以支持非等和范围查询。

- 插件名为 bloom
- 插件版本 V1.0

3.7.2 插件 bloom 加载方式

KingbaseES 默认不加载 bloom 插件，需要用户使用示例中的命令，人为手工加载。

示例：

```
create extension bloom;
```

3.7.3 插件 bloom 的参数配置

bloom 索引在其 WITH 子句中接受下列参数

- length

每个签名（索引项）的长度位数，它会被圆整成为最近的 16 的倍数。默认是 80 位，最长是 4096 位。

- col1 —col32

从每一个索引列产生的位数。每个参数的名字表示它所控制的索引列的编号。默认是 2 位，最大是 4095 位。没有实际使用的索引列的参数会被忽略。

3.7.4 插件 bloom 的使用方法

示例：

这是一个创建布鲁姆索引的例子。

```
CREATE TABLE tbloom(i1 varchar, i2 varchar, i3 varchar, i4 text);
CREATE INDEX bloomidx ON tbloom USING bloom (i1,i2,i3)  WITH (length=80, col1=2, col2=2, col3=4);
```

该索引是用长度为 80 位的签名所创建，其中属性 i1 和 i2 被映射为 2 位，属性 i3 被映射为 4 位。我们可以省略 length、col1 和 col2 说明，因为它们都有默认值。这里是布鲁姆索引定义和使用的更完整的例子，其中还与等效的 btree 做了对比。布鲁姆索引比 btree 索引更小，并且效率更高。

3.7.5 插件 bloom 卸载方法

bloom 插件的卸载也需要用户使用示例中的命令，人为手工卸载。

示例：

```
drop extension bloom;
```

3.7.6 插件 bloom 升级方法

bloom 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

3.8 btree_gin

3.8.1 插件 btree_gin 简介

btree_gin 为以下数据类型提供了 B 树等价行为的 GIN 操作符类：

```
macaddr8 int2 int4 int8 float4 float8 money oid timestamp timestamptz
time timetz date interval macaddr inet cidr text varchar char bytea
bit varbit numeric enum uuid name bool bpchar
```

一般情况下，这些操作符类不会比 B 树的表现更好，而且缺少 B 树最重要的特性之一支持唯一索引。但这些操作符可以用于 GIN 测试或是作为其他 GIN 操作符类开发的参考。在特定情况下，例如查询同时涉及一列适合 GIN 索引的列和一列适合 B 树索引的列，这时创建 GIN 的两列联合索引可能比创建 GIN 索引和 B 树索引再通过位图与扫描方式查询效率更高。

插件可以被任何在当前库具有 CREATE 权限的用户创建。

- 插件名为 btree_gin
- 插件版本 V1.3

备注:

仅 pg 模式支持该插件。

3.8.2 插件 btree_gin 加载方式

KingbaseES 默认不加载 btree_gin 插件，需要用户使用示例中的命令，人为手工加载。

示例：

```
CREATE EXTENSION btree_gin;
```

3.8.3 插件 btree_gin 的参数配置

无需配置任何参数

3.8.4 插件 btree_gin 的使用方法

```
CREATE TABLE test (a int4);
-- create index
CREATE INDEX testidx ON test USING GIN (a);
-- query
SELECT * FROM test WHERE a < 10;
```

3.8.5 插件 btree_gin 卸载方法

btree_gin 插件的卸载也需要用户使用示例中的命令，人为手工卸载。

示例：

```
DROP EXTENSION btree_gin;
```

3.8.6 插件 btree_gin 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION btree_gin UPDATE TO '1.1';
```

3.9 btree_gist

3.9.1 插件 btree_gist 简介

btree_gist 为以下数据类型提供了 B 树等价行为的 GIST 操作符类

```
uuid oid int2 int4 int8 float4 float8 timestamp timestamptz
time timetz date interval cash macaddr text bpchar bytea
numeric bit vbit inet cidr macaddr8 enum
```

一般情况下，这些操作符类不会比 B 树的表现更好，而且缺少 B 树最重要的特性之一支持唯一索引。但这些操作符可以用于 GiST 测试或是作为其他 GiST 操作符类开发的参考。在特定情况下，例如查询同时涉及一列适合 GiST 索引的列和一列适合 B 树索引的列，这时创建 GiST 的两列联合索引可能比创建 GiST 索引和 B 树索引再通过位图与扫描方式查询效率更高。

插件还提供了 B 树索引不支持的“<>”（不等于）操作符，这在创建 EXCLUDE 约束时很有用。插件对于可以作为距离度量的数据类型提供了距离操作符 <->，为使用这个操作符的最近邻搜索提供了 GiST 索引支持。插件可以被任何在当前库具有 CREATE 权限的用户创建。

- 插件名为 btree_gist
- 插件版本 V1.5

备注:

仅 pg 模式支持该插件。

3.9.2 插件 btree_gist 加载方式

KingbaseES 默认不加载 btree_gist 插件，需要用户使用示例中的命令，人为手工加载。

示例：

```
CREATE EXTENSION btree_gist;
```

3.9.3 插件 btree_gist 的参数配置

无需配置任何参数

3.9.4　插件 btree_gist 的使用方法

示例如下：

```
CREATE TABLE test (a int4);
CREATE INDEX testidx ON test USING GIST (a);
SELECT * FROM test WHERE a < 10;

-- 最近邻搜索，找距离 42 最近的 10 行数据
SELECT *, a <-> 42 AS dist FROM test ORDER BY a <-> 42 LIMIT 10;

-- 使用 EXCLUDE 约束来创建规则一个动物园里的一个笼子只能装一种动物
CREATE TABLE zoo (
 cage INTEGER,
 animal TEXT,
 EXCLUDE USING GIST (cage WITH =, animal WITH <>)
);
INSERT INTO zoo VALUES(123, 'zebra');
INSERT INTO zoo VALUES(123, 'zebra');
INSERT INTO zoo VALUES(123, 'lion');

-- 预期违反约束报错
ERROR: conflicting key value violates exclusion constraint
 "zoo_cage_animal_excl"
DETAIL: Key (cage, animal)=(123, lion) conflicts with existing key (cage,
 animal)=(123, zebra).
=> INSERT INTO zoo VALUES(124, 'lion');
```

3.9.5　插件 btree_gist 卸载方法

btree_gist 插件的卸载也需要用户使用示例中的命令，人为手工卸载。

示例：

```
DROP EXTENSION btree_gist;
```

3.9.6　插件 btree_gist 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1

```
ALTER EXTENSION btree_gist UPDATE TO '1.1';
```

3.10 cstore_fdw

3.10.1 插件 cstore_fdw 简介

cstore_fdw 是一款列存扩展插件。列存储在数据批量导入的分析场景能够提供更好的性能。cstore_fdw 通过只读取磁盘上相关的列数据来提升性能。

同时，由于每列的数据来自同一个域，因此更利于数据压缩，cstore_fdw 提供 6~10 倍的数据压缩能力，从而减小了对磁盘存储的需求。

cstore_fdw 采用 Optimized Row Columnar (ORC) 格式作为其数据的物理存储格式。ORC 优化了 Facebook 的 RCFile 存储格式，并具有以下优点：

- 压缩 (Compression) - 大约减少了 2~4 倍的内存和磁盘存储空间。易于扩展以支持不同的编码。
- 列投影 (Column Projections) - 仅仅读取与该查询有关的数据列，提高了 I/O 效率。
- 跳跃索引 (Skip Indexes) - 为每个行组 (Row Groups) 存储其最大值和最小值，并利用他们来跳过不相关的数据行。

KingbaseES 中通过 cstore_fdw 插件来实现列式存储及压缩。

- 插件名为 cstore_fdw
- 插件版本 V1.7

3.10.2 插件 cstore_fdw 加载方式

启动 KingbaseES 数据库后执行 SQL 命令创建插件。

示例：

```
CREATE EXTENSION cstore_fdw;
```

3.10.3 插件 cstore_fdw 的参数配置

无需配置任何参数

3.10.4 插件 cstore_fdw 的使用方法

以下四个选项可以在创建 cstore 外部表的时候指定：

- filename (可选)

该参数存放列存表数据的绝对路径，如果没有指定该选项，那么 cstore_fdw 则采用默认的 $PGHOME/cstore_fdw 来存储列存表数据。如果为该参数指定了值，则使用该值作为前缀来存储列存表数据信息。例如，当指定的 filename 值为 /cstore_fdw/my_table，那么 cstore_fdw 将使用 /cstore_fdw/my_table 来存储列存表用户数据，同时，使用 /cstore_fdw/my_table.footer 来存储列存表的元数据信息。

- compression (可选)

该参数用于指定用户数据的压缩算法，支持 none，pglz，zstd（Zstandard 压缩，可通过 cstore_fdw.zstd_compression_level 设置压缩级别，级别范围 0~9，默认级别 1）和 dict（字典压缩）四个值，默认值为 none。

- stripe_row_count (可选)

该参数指定每个 stripe 中行记录数，默认值为 150000。该值越小，加载数据或者查询时使用的内存也就越小，相反，其性能也就越低。

- block_row_count (可选) - 该参数指定每个列数据块 (column block) 中的行记录数，默认为 10000。cstore_fdw 压缩数据、创建跳跃索引以及磁盘读取时都是以块 (block) 为最小单元。该值越大，则利用数据压缩，并可以减少磁盘读取的次数量，然而，这将影响到跳过不相关的数据块的概率。

cstore_fdw 提供了两种方式用于向其导入数据：

- 使用 COPY 命令将文件、程序或者标准输入中导入数据；
- 使用 INSERT INTO cstore_table SELECT …语法从其他表导入数据。

我们可以使用 ANALYZE 命令收集列存表的统计信息，从而帮助优化器选择最优的查询计划。

备注:

1. cstore_fdw 目前并不支持使用 UPDATE 或 DELETE 命令来对表进行更新。同样，他也不支持单条记录的插入，这是由于每次导入数据都会形成至少一个数据块，若是支持单条记录插入，那么每个 INSERT 命令插入一条记录，将导致数据块过多从而影响性能，为此，cstore_fdw 不支持单条记录的插入。
2. cstore_fdw 目前不支持创建字段约束。如果在创建 cstore 外部表时定义了约束，该创建语句不会报错，但约束条件无效。

3.10.4.1　示例

```
-- 安装 cstore_fdw
CREATE EXTENSION cstore_fdw;
CREATE SERVER cstore_server FOREIGN DATA WRAPPER cstore_fdw;

-- 加载参数
LOAD 'cstore_fdw';

-- 查看当前 zstd 压缩级别参数值
```

```
show cstore_fdw.zstd_compression_level;
cstore_fdw.zstd_compression_level
-----------------------------------
1
(1 row)

-- 设置 zstd 压缩级别
SET cstore_fdw.zstd_compression_level = 3;
show cstore_fdw.zstd_compression_level;
cstore_fdw.zstd_compression_level
-----------------------------------
3
(1 row)

-- 创建无压缩列式存储外部表并指定存储文件
CREATE FOREIGN TABLE contestant (handle TEXT, birthdate DATE, rating INT,
      percentile FLOAT, country CHAR(3), achievements TEXT[])
      SERVER cstore_server
      OPTIONS(filename '/home/test/data/contestant.cstore');

-- 创建基于 pglz 压缩的列式存储外部表
CREATE FOREIGN TABLE contestant_compressed (handle TEXT, birthdate DATE, rating INT,
      percentile FLOAT, country CHAR(3), achievements TEXT[])
      SERVER cstore_server
      OPTIONS(compression 'pglz');

-- 创建基于 zstd 压缩算法的列式存储外部表
CREATE FOREIGN TABLE contestant_zstd (handle TEXT, birthdate DATE, rating INT,
      percentile FLOAT, country CHAR(3), achievements TEXT[])
      SERVER cstore_server
      OPTIONS(compression 'zstd');

-- 创建字典压缩算法的列式存储外部表
CREATE FOREIGN TABLE contestant_dict (handle TEXT, birthdate DATE, rating INT,
      percentile FLOAT, country CHAR(3), achievements TEXT[])
      SERVER cstore_server
      OPTIONS(compression 'dict');

-- 导入数据
-- 从文件导入
COPY contestant FROM '/home/test/input/data/contestants.1.csv' WITH CSV;
-- 从程序导入
COPY contestant FROM PROGRAM 'cat /home/test/input/data/contestants.2.csv' WITH CSV;
--从标准输入导入
copy contestant (handle, birthdate, rating, percentile, country, achievements)
```

```
from STDIN WITH CSV;
c,1988-11-01,2907,99.4,XB ,"{w,y}"
d,1985-05-05,2314,98.3,XB ,{}
e,1995-05-05,2236,98.2,XC ,{a}
\.

-- 解析
ANALYZE contestant;

--卸载
--卸载 cstore_fdw 之前，需要删除所有的 cstore 列存表、server 和扩展
DROP FOREIGN TABLE contestant;
DROP FOREIGN TABLE contestant_compressed;
DROP SERVER cstore_server;
DROP EXTENSION cstore_fdw;
```

备注:

cstore_fdw 会自动的创建目录来存储列存相关的数据，我们可以执行下面的命令来删除：

```
$ rm -rf $KESDATA/cstore_fdw
```

上面给出的是 cstore_fdw 的默认路径，若在建表的时候指定了 filename，其位置可能不同。另外还需要移除 kingbase.conf 文件中 shared_preload_libraries 中的 cstore_fdw。

3.10.4.2 升级

cstore 表在使用 sys_upgrade 工具升级默认行为如下：

- 传输模式为拷贝（copy）

用户指定 filename：在指定文件目录下，拷贝一份与成原文件相同的文件，文件名为加 _upg。新库继续使用原文件。

用户不指定 filename：使用默认位置将旧库的数据目录下的 cstroe_fdw 目录下的数据文件拷贝到新库数据目录下 cstore_fdw 目录下，新库使用新文件。

- 传输模式为克隆（clone）

用户指定 filename：在指定文件目录下，克隆一份与成原文件相同的文件，文件名为加 _upg。新库继续使用原文件。

用户不指定 filename，使用默认位置将旧库的数据目录下的 cstroe_fdw 目录下的数据文件克隆到新库数据目录下 cstore_fdw 目录下，新库使用新文件。

- 传输模式为连接（link）

用户指定 filename：在指定文件目录下，连接一份与成原文件相同的文件，文件名为加 _upg。新库继续使用原文件。这里要注意，由于是连接，upg 文件也将随原文件一起变更。

用户不指定 filename：使用默认位置将旧库的数据目录下的 cstroe_fdw 目录下的数据文件连接到新库数据目录下 cstore_fdw 目录下，新库使用新文件，这里要注意，由于是连接，新旧文件相互影响。

3.10.5　插件 cstore_fdw 卸载方法

在数据库中执行 SQL 命令卸载插件。

```
DROP EXTENSION cstore_fdw;
```

3.10.6　插件 cstore_fdw 升级方法

cstore_fdw 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.11　cube

3.11.1　插件 cube 简介

这个模块实现了一种数据类型 cube 来表示多维立方体。

- 插件名为 cube
- 插件版本 V1.4

3.11.2　插件 cube 加载方式

```
create extension cube;
```

3.11.3　插件 cube 的参数配置

无需配置任何参数

3.11.4　插件 cube 的使用方法

3.11.4.1　外部表示

在 立方体外部表示展示了 cube 类型有效的外部表示。x、y 等表示浮点数。

表 3.11.1: 立方体外部表示

外部语法	含义
x	一个一维点（或者长度为零的一维区间）
(x)	同上
x1,x2,...,xn	n-维空间中的一个点，内部表示为一个零容积立方体
(x1,x2,...,xn)	同上
(x),(y)	开始于 x 并且结束于 y 的一个一维区间，反之亦然。顺序并不重要
[(x),(y)]	同上
(x1,...,xn),(y1,...,yn)	一个 n-维立方体，用它的对角顶点对表示
[(x1,...,xn),(y1,...,yn)]	同上

一个立方体的对角录入的顺序无关紧要。如果需要创建一种统一的“左下—右上”的内部表示，cube 函数会自动地交换值。当角重合时，cube 只存储一个角和一个“is point”标志，这样避免浪费空间。

输入中的空白空间会被忽略，因此 [(x),(y)] 与 [(x), (y)] 相同。

3.11.4.2　精度

值在内部被存储为 64 位浮点数。这意味着超过 16 位有效位的数字将被截断。

3.11.4.3　操作符

在 立方体操作符展示了为类型 cube 提供的操作符。

表 3.11.2: 立方体操作符

操作符	结果	描述
a = b	boolean	立方体 a 和 b 相同。
a && b	boolean	立方体 a 和 b 重叠。
a @> b	boolean	立方体 a 包含立方体 b。
a <@ b	boolean	立方体 a 被包含在立方体 b 中。
a < b	boolean	立方体 a 小于立方体 b。
a <= b	boolean	立方体 a 小于或者等于立方体 b。
a > b	boolean	立方体 a 大于立方体 b。
a >= b	boolean	立方体 a 大于或者等于立方体 b。
a <> b	boolean	立方体 a 不等于立方体 b。
a -> n	float8	得到立方体的第 n 个坐标（从 1 开始数）。
a ~> n	float8	以下列方式获取多维数据集的第 n 个坐标 n = 2 * k - 1 表示第 k 维度的下限，n = 2 * k 表示第 k 维度的上限。负的 n 表示相应正坐标的倒数值。此运算符专为 KNN-GiST 支持而设计。

标量排序操作符（<、>= 等）除了用来排序之外没有什么实际用途。这些操作符首先比较第一个坐标，如果它们相等再比较第二个坐标等等。它们主要为支持 cube 的 b-树索引操作符类而存在，这类操作符对支持 cube 列上的 UNIQUE 约束等很有用。

cube 模块也为 cube 值提供了一个 GiST 索引操作符类。cube GiST 索引可以被用于在 WHERE 子句中通过 =、&&、@> 以及 <@ 操作符来搜索值。

此外，cube GiST 索引可以被用在 ORDER BY 子句中通过度量操作符 <->、<#> 和 <=> 来查找最近邻。例如，3-D 点 (0.5, 0.5, 0.5) 的最近邻可以用下面的查询很快地找到：

```
create table test (c cube);

insert into test values(cube(array[2,2,2])),(cube(array[3,3,3]));

SELECT c FROM test ORDER BY c <-> cube(array[0.5,0.5,0.5]) LIMIT 1;
    c
-----------
```

```
(2, 2, 2)
(1 row)
```

也可以用这种方式使用 ~> 操作符来高效地检索通过选定坐标排序后的前几个值。例如，可以用下面的查询得到通过第一个坐标（左下角）升序排列后的前几个立方体：

```
SELECT c FROM test ORDER BY c ~> 1 LIMIT 5;
    c
-----------
(2, 2, 2)
(3, 3, 3)
(2 rows)
```

以及得到通过右上角第一个坐标降序排列后的二维立方体：

```
SELECT c FROM test ORDER BY c ~> 3 DESC LIMIT 5;
    c
-----------
(3, 3, 3)
(2, 2, 2)
(2 rows)
```

3.11.4.4　函数

立方体函数显示了可用的函数：

表 3.11.3: 立方体函数

函数	结果	描述	例子
cube(float8)	cube	制造一个一维立方体，坐标都是相同的。	cube(1) == '(1)'
cube(float8, float8)	cube	制造一个一维立方体。	cube(1,2) == '(1),(2)'
cube(float8[])	cube	使用数组定义的坐标制造一个零容积的立方体。	cube(ARRAY[1,2]) == '(1,2)'
cube(float8[], float8[])	cube	用由两个数组定义的右上和左下坐标制造一个立方体，两个数组必须等长。	cube(ARRAY[1,2], ARRAY[3,4]) == '(1,2),(3,4)'

-续表-

函数	结果	描述	例子
cube(cube, float8)	cube	在一个现有的立方体上增加一维来制造一个新立方体，对新坐标的各个端点都采用相同的值。这可以用于从计算得到的值逐渐地构建立方体。	cube('(1,2),(3,4)'::cube, 5) == '(1,2,5),(3,4,5)'
cube(cube, float8, float8)	cube	在一个现有的立方体上增加一维来制造一个新立方体。这可以用于从计算得到的值逐渐地构建立方体。	cube('(1,2),(3,4)'::cube, 5, 6) == '(1,2,5),(3,4,6)'
cube_dim(cube)	integer	返回该立方体的维数	cube_dim('(1,2),(3,4)') == '2'
cube_ll_coord(cube, integer)	float8	返回一个立方体的左下角的第 n 个坐标值	cube_ll_coord('(1,2),(3,4)', 2) == '2'
cube_ur_coord(cube, integer)	float8	返回一个立方体的右上角的第 n 个坐标值	cube_ur_coord('(1,2),(3,4)', 2) == '4'
cube_is_point(cube)	boolean	如果一个立方体是一个点则返回真，也就是两个定义点相同。	
cube_distance(cube, cube)	float8	返回两个立方体之间的距离。如果两个都是点，这就是普通距离函数。	
cube_subset(cube, integer[])	cube	从一个现有的立方体制造一个新立方体，使用来自于一个数组的维索引列表。它可以被用来抽取一个单一维度的端点，或者它可以被用来去除维度，或者按照需要对它们重新排序。	cube_subset(cube('(1,3,5),(6,7,8)'), ARRAY[2]) == '(3),(7)'cube_subset(cube('(1,3,5),(6,7,8)'), ARRAY[3,2,1,1]) =='(5,3,1,1),(8,7,6,6)'
cube_union(cube, cube)	cube	产生两个立方体的并	
cube_inter(cube, cube)	cube	产生两个立方体的交	

-续表-

函数	结果	描述	例子
cube_enlarge(c cube, r double, n integer)	cube	用一个指定的半径 r 在至少 n 个维度上增加立方体的尺寸。如果该半径是负值，则该立方体会收缩。这有助于围绕一个点创建一个外包盒来搜索附近点。所有已定义的维度都会按照半径 r 被改变。左下坐标按照 r 被减小并且右上坐标按照 r 被增加。如果一个左下坐标被增加得超过对应的右上坐标（这只会发生在 r< 0 时），则两个坐标会被设置为它们的均值。如果 n 大于已定义的维度数并且该立方体被增加（r >= 0），则额外的维度会被加入以让维度数达到 n，对于额外的坐标将使用 0 作为初始值。这个函数可用来创建围绕一个点的外包盒以搜索临近点。	cube_enlarge('(1,2),(3,4)', 0.5, 3) ==='(0.5,1.5,-0.5),(3.5,4.5,0.5)'

3.11.4.5　默认值

以下为并操作：

```
select cube_union('(0,5,2),(2,3,1)', '0');
cube_union
-------------------
(0, 0, 0),(2, 5, 2)
(1 row)
```

以下为交操作：

```
select cube_inter('(0,-1),(1,1)', '(-2),(2)');
cube_inter
-------------
(0, 0),(1, 0)
(1 row)
```

在所有不同维度立方体的二元操作中，我假定低纬度的那一个要做笛卡尔投影，即为字符串表示中被省略的坐标取零。上面的例子等同于：

```
select cube_union('(0,5,2),(2,3,1)','(0,0,0),(0,0,0)');
select cube_inter('(0,-1),(1,1)','(-2,0),(2,0)');
```

下列包含谓词使用点语法，不过实际上第二个参数在内部被表示为一个盒体。这种语法让我们不必定义一种单独的点类型以及用于（盒体, 点）谓词的函数。

```
select cube_contains('(0,0),(1,1)', '0.5,0.5');
cube_contains
--------------
t
(1 row)
```

备注:

用法的例子可见回归测试 sql/cube.sql。为了不容易出问题，对于立方体的维度数有 100 的限制。如果你想要更大的立方体，可以在 cubedata.h 中修改。

3.11.5　插件 cube 的卸载方法

```
drop extension cube;
```

3.11.6　插件 cube 升级方法

cube 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.12　dblink

3.12.1　插件 dblink 简介

dblink 是 KingbaseES 的一个扩展插件，支持在一个数据库会话中连接到其他 Kingbase 数据库的模块。

还可以参考kingbase_fdw，它以一种更现代和更加兼容标准的架构提供了相同的功能。

- 插件名为 dblink
- 插件版本 V1.2

3.12.2　插件 dblink 加载方式

```
CREATE EXTENSION dblink;
```

3.12.3　插件 dblink 的参数配置

无需配置任何参数。

3.12.4　插件 dblink 使用方法

3.12.4.1　dblink_connect

dblink_connect —打开一个到远程数据库的持久连接

语法

```
dblink_connect(text connstr) 返回 text
dblink_connect(text connname, text connstr) 返回 text
```

描述

dblink_connect() 建立一个到远程 KingbaseES 数据库的连接。要连接的服务器和数据库通过一个标准的 libpq 连接串来标识。可以选择将一个名字赋予给该连接。多个命名的连接可以被一次打开，但是一次只允许一个未命名连接。连接将会持续直到被关闭或者数据库会话结束。

连接串也可以是一个现存外部服务器的名字。在使用外部服务器时，推荐使用外部数据包装器 dblink_fdw。见下面的例子，以及 CREATE SERVER 和 CREATE USER MAPPING。

参数

`connname`

要用于这个连接的名字。如果被忽略，将打开一个未命名连接并且替换掉任何现有的未命名连接。

`connstr`

libpq-风格的连接信息串，例如 hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd。此外，还可以是一个外部服务器的名字。

返回值

返回状态，它总是 OK（因为任何错误会导致该函数抛出一个错误而不是返回）。

注解

如果不可信用户能够访问一个没有采用安全方案使用模式的数据库，应该在开始每个会话时从 search_path 中移除公共可写的方案。例如，可以把 options=-csearch_path= 增加到 connstr。这种考虑不是特别针对 dblink，它适用于每一种执行任意 SQL 命令的接口。

只有超级用户能够使用 dblink_connect 来创建无口令认证连接。如果非超级用户需要这种能力，使用 dblink_connect_u。

选择包含等号的连接名是不明智的，因为这会产生与在其他 dblink 函数中的连接信息串混淆的风险。

示例

```
SELECT dblink_connect('hostaddr=127.0.0.1 port=54321 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT dblink_connect('myconn', 'hostaddr=127.0.0.1 port=54321 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

 -- FOREIGN DATA WRAPPER functionality
 -- Note: local connection must require password authentication for this
    to work properly
 --       Otherwise, you will receive the following error from dblink_connect():
 --       ----------------------------------------------------------------------
 --       ERROR:  password is required
 --       DETAIL:  Non-superuser cannot connect if the server does not
          request a password.
 --       HINT:  Target server's authentication method must be changed.

 CREATE SERVER fdtest FOREIGN DATA WRAPPER dblink_fdw OPTIONS (hostaddr
 '127.0.0.1', dbname 'contrib_regression');

CREATE USER regress_dblink_user WITH PASSWORD 'secret';
CREATE USER MAPPING FOR regress_dblink_user SERVER fdtest OPTIONS (user
 'regress_dblink_user', password 'secret');
GRANT USAGE ON FOREIGN SERVER fdtest TO regress_dblink_user;
GRANT SELECT ON TABLE foo TO regress_dblink_user;

\set ORIGINAL_USER :USER
\c - regress_dblink_user
SELECT dblink_connect('myconn', 'fdtest');
 dblink_connect
----------------
 OK
(1 row)

SELECT * FROM dblink('myconn','SELECT * FROM foo') AS t(a int, b text,
```

```
 c text[]);
 a  | b |       c
----+---+---------------
  0 | a | {a0,b0,c0}
  1 | b | {a1,b1,c1}
  2 | c | {a2,b2,c2}
  3 | d | {a3,b3,c3}
  4 | e | {a4,b4,c4}
  5 | f | {a5,b5,c5}
  6 | g | {a6,b6,c6}
  7 | h | {a7,b7,c7}
  8 | i | {a8,b8,c8}
  9 | j | {a9,b9,c9}
 10 | k | {a10,b10,c10}
(11 rows)

 \c - :ORIGINAL_USER
 REVOKE USAGE ON FOREIGN SERVER fdtest FROM regress_dblink_user;
 REVOKE SELECT ON TABLE foo FROM regress_dblink_user;
 DROP USER MAPPING FOR regress_dblink_user SERVER fdtest;
 DROP USER regress_dblink_user;
 DROP SERVER fdtest;
```

3.12.4.2　dblink_connect_u

dblink_connect_u —不安全地打开一个到远程数据库的持久连接

语法

```
dblink_connect_u(text connstr) 返回 text
dblink_connect_u(text connname, text connstr) 返回 text
```

描述

dblink_connect_u() 和 dblink_connect() 一样，不过它将允许非超级用户使用任意认证方式来连接。

如果远程服务器选择了一种不涉及口令的认证方式，那么可能发生模仿以及后续的扩大权限，因为该会话看起来像由运行 KingbaseES 的用户发起的。此外，即使远程服务器不要求一个口令，也可能从服务器环境提供该口令，例如一个属于服务器用户的 ~/.kbpass 文件。这带来的不只是模仿的风险，而且还有将口令暴露给不可信的远程服务器的风险。因此，dblink_connect_u() 最初是用所有从 PUBLIC 撤销的特权安装的，这让它只能被超级用户调用。在某些情况中，为 dblink_connect_u() 授予 EXECUTE 权限给可信的指定用户是合适的，但是必须小心。我们也推荐任何属于服务器用户的 ~/.kbpass 文件不能包含任何指定了一个通配符主机名的记录。

详见 dblink_connect()。

3.12.4.3　dblink_disconnect

dblink_disconnect —关闭一个到远程数据库的持久连接

语法

```
dblink_disconnect() 返回 text
dblink_disconnect(text connname) 返回 text
```

描述

dblink_disconnect() 关闭一个之前被 dblink_connect() 打开的连接。不带参数的形式关闭一个未命名连接。

参数

`connname`

要被关闭的命名连接的名字。

返回值

它总是 OK（因为任何错误会导致该函数抛出一个错误而不是返回）。

示例

```
SELECT dblink_disconnect();
 dblink_disconnect
-------------------
 OK
(1 row)

SELECT dblink_disconnect('myconn');
 dblink_disconnect
-------------------
 OK
(1 row)
```

3.12.4.4　dblink

dblink —在一个远程数据库中执行一个查询

语法

```
dblink(text connname, text sql [, bool fail_on_error]) 返回记录集
dblink(text connstr, text sql [, bool fail_on_error]) 返回记录集
dblink(text sql [, bool fail_on_error]) 返回记录集
```

描述

dblink 在一个远程数据库中执行一个查询（通常是一个 SELECT，但是也可以是任意返回行的 SQL 语句）。

当给定两个 text 参数时，第一个被首先作为一个持久连接的名称进行查找；如果找到，该命令会在该连接上被执行。如果没有找到，第一个参数被视作一个用于 dblink_connect 的连接信息字符串，并且被指出的连接只是在这个命令的持续期间被建立。

参数

`connname`

要使用的连接名。忽略这个参数将使用未命名连接。

`connstr`

如之前为 dblink_connect 所描述的一个连接信息字符串。

`sql`

希望在远程数据库中执行的 SQL 查询，例如 select * from foo。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数不返回行。

返回值

该函数返回查询产生的行。因为 dblink 能与任何查询一起使用，它被声明为返回 record，而不是指定任意特定的列集合。这意味着你必须指定在调用的查询中所期待的列集合，否则 KingbaseES 将不知道会得到什么。这里是一个例子

```
SELECT *
    FROM dblink('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd', 'select proname,
prosrc from sys_proc')
    AS t1(proname name, prosrc text)
    WHERE proname LIKE 'bytea%';
```

FROM 子句的“alias”部分必须指定函数将返回的列名及类型（在一个别名中指定列名实际上是标准 SQL 语法，但是指定列类型是一种 KingbaseES 扩展）。这允许系统在尝试执行该函数之前就理解 * 将展开成什么，以及 WHERE 子句中的 proname 指的什么。在运行时，如果来自远程数据库的实际查询结果和 FROM 子句中显示的列数不同，将会抛出一个错误。不过，列名不需要匹配，并且 dblink 并不坚持精确地匹配类型。只要被返回的数据字符串是 FROM 子句中声明的列类型的合法输入，它就将会成功。

注解

一种将预定义查询用于 dblink 的方法是创建一个视图。这允许列类型信息被埋藏在该视图中，而不是在每一个查询中都拼写出来。例如

```
CREATE VIEW myremote_sys_proc AS
SELECT *
    FROM dblink('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd',
                'select proname, prosrc from sys_proc')
    AS t1(proname name, prosrc text);
```

```
SELECT * FROM myremote_sys_proc WHERE proname LIKE 'bytea%';
```

示例

```
SELECT * FROM dblink('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd',
                     'select proname, prosrc from sys_proc')
    AS t1(proname name, prosrc text) WHERE proname LIKE 'bytea%';
  proname   |   prosrc
------------+------------
 byteacat   | byteacat
 byteaeq    | byteaeq
 bytealt    | bytealt
 byteale    | byteale
 byteagt    | byteagt
 byteage    | byteage
 byteane    | byteane
 byteacmp   | byteacmp
 bytealike  | bytealike
 byteanlike | byteanlike
 byteain    | byteain
 byteaout   | byteaout
(12 rows)

SELECT dblink_connect('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT * FROM dblink('select proname, prosrc from sys_proc')
    AS t1(proname name, prosrc text) WHERE proname LIKE 'bytea%';
  proname   |   prosrc
------------+------------
 byteacat   | byteacat
 byteaeq    | byteaeq
 bytealt    | bytealt
 byteale    | byteale
 byteagt    | byteagt
 byteage    | byteage
 byteane    | byteane
 byteacmp   | byteacmp
 bytealike  | bytealike
 byteanlike | byteanlike
 byteain    | byteain
```

```
 byteaout   | byteaout
(12 rows)

SELECT dblink_connect('myconn', 'hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT * FROM dblink('myconn', 'select proname, prosrc from sys_proc')
    AS t1(proname name, prosrc text) WHERE proname LIKE 'bytea%';
  proname   |   prosrc
------------+------------
 bytearecv  | bytearecv
 byteasend  | byteasend
 byteale    | byteale
 byteagt    | byteagt
 byteage    | byteage
 byteane    | byteane
 byteacmp   | byteacmp
 bytealike  | bytealike
 byteanlike | byteanlike
 byteacat   | byteacat
 byteaeq    | byteaeq
 bytealt    | bytealt
 byteain    | byteain
 byteaout   | byteaout
(14 rows)
```

3.12.4.5　dblink_exec

dblink_exec —在一个远程数据库中执行一个命令

语法

```
dblink_exec(text connname, text sql [, bool fail_on_error]) returns text
dblink_exec(text connstr, text sql [, bool fail_on_error]) returns text
dblink_exec(text sql [, bool fail_on_error]) returns text
```

描述

dblink_exec 在一个远程数据库中执行一个命令（也就是，任何不返回行的 SQL 语句）。

当给定两个 text 参数时，第一个被首先作为一个持久连接的名称进行查找；如果找到，该命令会在该连接上被执行。如果没有找到，第一个参数被视作一个用于 dblink_connect 的连接信息字符串，并且被指出的连接只是在这

个命令的持续期间被建立。

参数

`connname`

要使用的连接名。忽略这个参数将使用未命名连接。

`connstr`

如之前为 dblink_connect 所描述的一个连接信息字符串。

`sql`

希望在远程数据库中执行的 SQL 命令，例如 insert into foo values(0,‘a’，‘{“a0”,“b0”,“c0”}’)。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数的返回值被设置为 ERROR。

返回值

返回状态，可能是命令的状态字符串或 ERROR。

示例

```
SELECT dblink_connect('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT dblink_exec('insert into foo values(21,''z'',''{"a0","b0","c0"}'');');
 dblink_exec
-----------------
 INSERT 943366 1
(1 row)

SELECT dblink_connect('myconn', 'hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT dblink_exec('myconn',
       'insert into foo values(21,''z'',''{"a0","b0","c0"}'');');
 dblink_exec
------------------
 INSERT 6432584 1
(1 row)
```

3.12.4.6　dblink_open

dblink_open —在一个远程数据库中打开一个游标

语法

```
dblink_open(text cursorname, text sql [, bool fail_on_error]) 返回 text
dblink_open(text connname, text cursorname, text sql [, bool fail_on_error]) 返回 text
```

描述

dblink_open() 在一个远程数据库中打开一个游标。该游标能够随后使用 dblink_fetch() 和 dblink_close() 进行操纵。

参数

`connname`

要使用的连接名。忽略这个参数将使用未命名连接。

`cursorname`

要赋予给这个游标的名称。

`sql`

希望在远程数据库中执行的 SELECT 语句，例如 select * from sys_class。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数的返回值被设置为 ERROR。

返回值

返回状态，OK 或者 ERROR。

注解

因为一个游标只能在一个事务中持续，如果远端还没有在一个事务中，dblink_open 会在远端开始一个显式事务块（BEGIN）。当匹配的 dblink_close 被执行时，这个事务将再次被关闭。注意如果你使用 dblink_exec 在 dblink_open 和 dblink_close 之间改变数据，并且接着发生了一个错误或者你在 dblink_close 之前使用了 dblink_disconnect，你的更改将被丢失，因为事务将被中止。

示例

```
SELECT dblink_connect('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
```

```
(1 row)

SELECT dblink_open('foo', 'select proname, prosrc from sys_proc');
 dblink_open
-------------
 OK
(1 row)
```

3.12.4.7　dblink_fetch

dblink_fetch —从一个远程数据库中的打开的游标返回行

语法

```
dblink_fetch(text cursorname, int howmany [, bool fail_on_error]) 返回 record 集合
dblink_fetch(text connname, text cursorname, int howmany [, bool fail_on_error]) 返回 record 集合
```

描述

dblink_fetch 从一个之前由 dblink_open 建立的游标中取得行。

参数

`connname`

要使用的连接名。忽略这个参数将使用未命名连接。

`cursorname`

要从中取数据的游标名。

`howmany`

要检索的最大行数。从当前游标位置向前的接下来 howmany 个行会被取出。一旦该游标已经到达了它的末端，将不会产生更多行。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数的返回值被设置为 ERROR。

返回值

该函数返回从游标中取出的行。要使用这个函数，你将需要指定想要的列集合，如前面 dblink 中所讨论的。

注解

当 FROM 子句中指定的返回列的数量和远程游标返回的实际列数不匹配时，将抛出一个错误。在这个事件中，远程游标仍会被前进错误没发生时应该前进的行数。对于远程 FETCH 完成之后在本地查询中发生的任何其他错误，情况也是一样。

示例

```
SELECT dblink_connect('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT dblink_open('foo', 'select proname, prosrc from sys_proc where proname
 like ''bytea%''');
 dblink_open
-------------
OK
(1 row)

SELECT * FROM dblink_fetch('foo', 5) AS (funcname name, source text);
 funcname |  source
----------+----------
 byteacat | byteacat
 byteacmp | byteacmp
 byteaeq  | byteaeq
 byteage  | byteage
 byteagt  | byteagt
(5 rows)

SELECT * FROM dblink_fetch('foo', 5) AS (funcname name, source text);
 funcname  |  source
-----------+-----------
 byteain   | byteain
 byteale   | byteale
 bytealike | bytealike
 bytealt   | bytealt
 byteane   | byteane
(5 rows)

SELECT * FROM dblink_fetch('foo', 5) AS (funcname name, source text);
  funcname  |   source
------------+------------
 byteanlike | byteanlike
 byteaout   | byteaout
(2 rows)

SELECT * FROM dblink_fetch('foo', 5) AS (funcname name, source text);
 funcname | source
----------+--------
(0 rows)
```

3.12.4.8　dblink_close

dblink_close —关闭一个远程数据库中的游标

语法

```
dblink_close(text cursorname [, bool fail_on_error]) 返回 text
dblink_close(text connname, text cursorname [, bool fail_on_error]) 返回 text
```

描述

dblink_close 关闭一个之前由 dblink_open 打开的游标。

参数

`connname`

要使用的连接名。忽略这个参数将使用未命名连接。

`cursorname`

要关闭的游标名。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数的返回值被设置为 ERROR。

返回值

返回状态，OK 或者 ERROR。

注解

如果 dblink_open 开始了一个显式事务块，并且这是这个连接中最后一个保持打开的游标，dblink_close 将发出匹配的 COMMIT。

示例

```
SELECT dblink_connect('hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

SELECT dblink_open('foo', 'select proname, prosrc from sys_proc');
 dblink_open
-------------
 OK
(1 row)

SELECT dblink_close('foo');
 dblink_close
```

```
--------------
 OK
(1 row)
```

3.12.4.9　dblink_get_connections

dblink_get_connections —返回所有打开的命名 dblink 连接的名称

语法

```
dblink_get_connections() 返回 text[]
```

描述

dblink_get_connections 返回一个数组，其中是所有打开的命名 dblink 连接的名称。

返回值

返回一个连接名称的文本数组，如果没有则为 NULL。

示例

```
SELECT dblink_get_connections();
```

3.12.4.10　dblink_error_message

dblink_error_message —得到在命名连接上的最后一个错误消息

语法

```
dblink_error_message(text connname) 返回 text
```

描述

dblink_error_message 为一个给定连接取得最近的远程错误消息。

参数

`connname`

要使用的连接名。

返回值

返回最后一个错误消息，如果在这个连接上没有错误则返回一个 OK。

注解

当异步查询由 dblink_send_query 启动时，与连接相关的错误消息可能不会得到更新，直到服务器的响应消息被用掉。这通常意味着 dblink_is_busy 或 dblink_get_result 应在 dblink_error_message 之前被调用，以使得异步查询产生的任何错误都是可见的。

示例

```
SELECT dblink_error_message('dtest1');
```

3.12.4.11　dblink_send_query

dblink_send_query —发送一个异步查询到远程数据库

语法

```
dblink_send_query(text connname, text sql) 返回 int
```

描述

dblink_send_query 发送一个要被异步执行的查询，也就是不需要立即等待结果。在该连接上不能有还在处理中的异步查询。

在成功地派送一个异步查询后，可以用 dblink_is_busy 检查完成状态，并且结果最终由 dblink_get_result 收集。也可以使用 dblink_cancel_query 尝试取消一个活动中的异步查询。

参数

`connname`

要使用的连接名。

`sql`

希望在远程数据库中执行的 SQL 语句，例如 select * from sys_class。

返回值

如果查询被成功地派送返回 1，否则返回 0。

示例

```
SELECT dblink_send_query('dtest1', 'SELECT * FROM foo WHERE f1 < 3');
```

3.12.4.12　dblink_is_busy

dblink_is_busy —检查连接是否正在忙于一个异步查询

语法

```
dblink_is_busy(text connname) 返回 int
```

描述

dblink_is_busy 测试是否一个异步查询正在进行中。

参数

`connname`

要检查的连接名。

返回值

如果连接正忙则返回 1，如果不忙则返回 0。如果这个函数返回 0，dblink_get_result 将被保证不会阻塞。

示例

```
SELECT dblink_is_busy('dtest1');
```

3.12.4.13　dblink_get_notify

dblink_get_notify —在一个连接上检索异步通知

语法

```
dblink_get_notify() 返回 (notify_name text, be_pid int, extra text) 集合
dblink_get_notify(text connname) 返回 (notify_name text, be_pid int, extra text) 集合
```

描述

dblink_get_notify 在一个未命名连接或者一个指定的命名连接上检索通知。要通过 dblink 接收通知，首先必须使用 dblink_exec 发出 LISTEN。详见 LISTEN 和 NOTIFY。

参数

`connname`

要在其上得到通知的命名连接的名称。

返回值

返回 (notify_name text, be_pid int, extra text) 集合，或者一个空集。

示例

```
SELECT dblink_exec('LISTEN virtual');
 dblink_exec
-------------
 LISTEN
(1 row)
```

```
SELECT * FROM dblink_get_notify();
 notify_name | be_pid | extra
-------------+--------+-------
(0 rows)

NOTIFY virtual;
NOTIFY

SELECT * FROM dblink_get_notify();
 notify_name | be_pid | extra
-------------+--------+-------
 virtual     |   1229 |
(1 row)
```

3.12.4.14　dblink_get_result

dblink_get_result —得到一个异步查询结果

语法

```
dblink_get_result(text connname [, bool fail_on_error]) 返回 record 集合
```

描述

dblink_get_result 收集之前 dblink_send_query 发送的一个异步查询的结果。如果该查询还没有完成，dblink_get_result 将等待直到它完成。

参数

`connname`

要使用的连接名。

`fail_on_error`

如果为真（忽略时的默认值），那么在连接的远端抛出的一个错误也会导致本地抛出一个错误。如果为假，远程错误只在本地被报告为一个 NOTICE，并且该函数不返回行。

返回值

对于一个异步查询（也就是一个返回行的 SQL 语句），该函数返回查询产生的行。要使用这个函数，你将需要指定所期待的列集合，如前面为 dblink 所讨论的那样。

对于一个异步命令（也就是一个不返回行的 SQL 语句），该函数返回一个只有单个文本列的单行，其中包含了该命令的状态字符串。仍必须在调用的 FROM 子句中指定结果将具有一个单一文本行。

注解

如果 dblink_send_query 返回 1，这个函数就必须被调用。对每一个已发送的查询都必须调用一次这个函数，且在连接再次可用之前还要多调用一次来得到一个空结果集。

当使用 dblink_send_query 和 dblink_get_result 时，在将结果集中的任何一行返回给本地查询处理器之前，dblink 将取得整个远程查询结果。如果该查询返回大量的行，这可能会导致本地会话中短暂的内存膨胀。最好将这样的一个查询用 dblink_open 打开成一个游标并且接着每次取得数量可管理的行。也可以使用简单的 dblink()，它会避免缓冲大型结果集到磁盘上导致的内存膨胀。

示例

```
contrib_regression=# SELECT dblink_connect('dtest1',
 'hostaddr=127.0.0.1 port=5432 dbname=mydb user=myname password=mypasswd');
 dblink_connect
----------------
 OK
(1 row)

contrib_regression=# SELECT * FROM
 contrib_regression-# dblink_send_query('dtest1', 'select * from foo where f1 <
 3') AS t1;
 t1
----
 1
(1 row)

contrib_regression=# SELECT * FROM dblink_get_result('dtest1') AS t1(f1 int, f2
 text, f3 text[]);
 f1 | f2 |     f3
----+----+------------
 0 | a  | {a0,b0,c0}
 1 | b  | {a1,b1,c1}
 2 | c  | {a2,b2,c2}
(3 rows)

contrib_regression=# SELECT * FROM dblink_get_result('dtest1') AS t1(f1 int, f2
 text, f3 text[]);
 f1 | f2 | f3
----+----+----
(0 rows)

contrib_regression=# SELECT * FROM
 contrib_regression-# dblink_send_query('dtest1', 'select * from foo where f1 <
 3; select * from foo where f1 > 6') AS t1;
 t1
----
 1
```

```
(1 row)

contrib_regression=# SELECT * FROM dblink_get_result('dtest1') AS t1(f1 int, f2
 text, f3 text[]);
 f1 | f2 |     f3
----+----+------------
  0 | a  | {a0,b0,c0}
  1 | b  | {a1,b1,c1}
  2 | c  | {a2,b2,c2}
(3 rows)

contrib_regression=# SELECT * FROM dblink_get_result('dtest1') AS t1(f1 int, f2
 text, f3 text[]);
 f1 | f2 |      f3
----+----+---------------
  7 | h  | {a7,b7,c7}
  8 | i  | {a8,b8,c8}
  9 | j  | {a9,b9,c9}
 10 | k  | {a10,b10,c10}
(4 rows)

contrib_regression=# SELECT * FROM dblink_get_result('dtest1') AS t1(f1 int, f2
 text, f3 text[]);
 f1 | f2 | f3
----+----+----
(0 rows)
```

3.12.4.15　dblink_cancel_query

dblink_cancel_query —在命名连接上取消任何活动查询

语法

```
dblink_cancel_query(text connname) 返回 text
```

描述

dblink_cancel_query 尝试取消命名连接上正在进行的任何查询。注意这不一定会成功（例如，远程查询可能已经结束）。一个取消请求仅仅提高了该查询将很快失败的几率。你仍必须完成通常的查询协议，例如通过调用 dblink_get_result。

参数

connname

要使用的连接名。

返回值

如果取消请求已经被发送，则返回 OK；如果失败，则返回一个错误消息的文本。

示例

```
SELECT dblink_cancel_query('dtest1');
```

3.12.4.16　dblink_get_pkey

dblink_get_pkey —返回一个关系的主键域的位置和域名称

语法

```
dblink_get_pkey(text relname) 返回 dblink_pkey_results 集合
```

描述

dblink_get_pkey 提供有关于本地数据库中一个关系的主键的信息。这有时候有助于生成要被发送到远程数据库的查询。

参数

`relname`

一个本地关系的名称，例如 foo 或者 myschema.mytab。如果该名称是大小写混合的或包含特殊字符，要包括双引号，例如"FooBar"；如果没有引号，字符串将被折叠到小写形式。

返回值

为每一个主键域返回一行，如果该关系没有主键则不返回行。结果行类型被定义为

```
CREATE TYPE dblink_pkey_results AS (position int, colname text);
```

position 列值可以从 1 到 N，它是该域在主键中的编号，而不是在表列中的编号。

示例

```
CREATE TABLE foobar (
    f1 int,
    f2 int,
    f3 int,
    PRIMARY KEY (f1, f2, f3)
);

SELECT * FROM dblink_get_pkey('foobar');
 position | colname
----------+---------
        1 | f1
        2 | f2
```

```
        3 | f3
(3 rows)
```

3.12.4.17　dblink_build_sql_insert

dblink_build_sql_insert 一使用一个本地元组构建一个 INSERT 语句，将主键域值替换为提供的值

语法

```
dblink_build_sql_insert(text relname,
                        int2vector primary_key_attnums,
                        integer num_primary_key_atts,
                        text[] src_pk_att_vals_array,
                        text[] tgt_pk_att_vals_array) 返回 text
```

描述

dblink_build_sql_insert 在选择性地将一个本地表复制到一个远程数据库时很有用。它基于主键从本地表选择一行，并且接着构建一个复制该行的 INSERT 命令，但是其中主键值被替换为最后一个参数中的值（要创建该行的一个准确拷贝，只要为最后两个参数指定相同的值）。

参数

`relname`

一个本地关系的名称，例如 foo 或者 myschema.mytab。如果该名称是大小写混合的或包含特殊字符，要包括双引号，例如"FooBar"；如果没有引号，字符串将被折叠到小写形式。

`primary_key_attnums`

主键域的属性号（从 1 开始），例如 1 2。

`num_primary_key_atts`

主键域的数量。

`src_pk_att_vals_array`

要被用来查找本地元组的主键域值。每一个域都被表示为文本形式。如果没有行具有这些主键值，则抛出一个错误。

`tgt_pk_att_vals_array`

要被替换到结果 INSERT 命令中的主键域值。每一个域被表示为文本形式。

返回值

将要求的 SQL 语句返回为文本。

注解

自 KingbaseES 9.0 开始，primary_key_attnums 中的属性号被解释为逻辑列号，对应于列在 SELECT * FROM relname 中的位置。之前的版本将属性号解释为物理列位置。如果指示出的列的左边有任意列在该表的生存期内被删除，这两种解释就有区别。

示例

```
SELECT dblink_build_sql_insert('foo', '1 2', 2, '{"1", "a"}', '{"1", "b''a"}');
             dblink_build_sql_insert
--------------------------------------------------
 INSERT INTO foo(f1,f2,f3) VALUES('1','b''a','1')
(1 row)
```

3.12.4.18　dblink_build_sql_delete

dblink_build_sql_delete —使用所提供的主键域值构建一个 DELETE 语句

语法

```
int2vector primary_key_attnums,
integer num_primary_key_atts,
text[] tgt_pk_att_vals_array) 返回 text
```

描述

dblink_build_sql_delete 在选择性地将一个本地表复制到一个远程数据库时很有用。它构建一个 SQL DELETE 命令用来删除具有给定主键值的行。

参数

`relname`

一个本地关系的名称，例如 foo 或者 myschema.mytab。如果该名称是大小写混合的或包含特殊字符，要包括双引号，例如"FooBar"；如果没有引号，字符串将被折叠到小写形式。

`primary_key_attnums`

主键域的属性号（从 1 开始），例如 1 2。

`num_primary_key_atts`

主键域的数量。

`tgt_pk_att_vals_array`

要用在结果 DELETE 命令中的主键域值。每一个域都被表示为文本形式。

返回值

将要求的 SQL 语句返回为文本。

注解

自 KingbaseES 9.0 开始，primary_key_attnums 中的属性号被解释为逻辑列号，对应于列在 SELECT * FROM relname 中的位置。之前的版本将属性号解释为物理列位置。如果指示出的列的左边有任意列在该表的生存期内被删除，这两种解释就有区别。

示例

```
SELECT dblink_build_sql_delete('"MyFoo"', '1 2', 2, '{"1", "b"}');
         dblink_build_sql_delete
--------------------------------------------
 DELETE FROM "MyFoo" WHERE f1='1' AND f2='b'
(1 row)
```

3.12.4.19　dblink_build_sql_update

dblink_build_sql_update —使用一个本地元组构建一个 UPDATE 语句，将主键域值替换为提供的值

语法

```
dblink_build_sql_update(text relname,
                        int2vector primary_key_attnums,
                        integer num_primary_key_atts,
                        text[] src_pk_att_vals_array,
                        text[] tgt_pk_att_vals_array) 返回 text
```

描述

dblink_build_sql_update 在选择性地将一个本地表复制到一个远程数据库时很有用。它从本地表基于主键选择一行，并且接着构建一个 SQL UPDATE 命令来复制该行，但是其中的主键值被替换为最后一个参数中的值（要创建该行的一个准确拷贝，只要为最后两个参数指定相同的值）。UPDATE 命令总是为该行的所有域赋值—这个函数与 dblink_build_sql_insert 之间的主要区别是它假定目标行已经存在于远程表中。

参数

`relname`

一个本地关系的名称，例如 foo 或者 myschema.mytab。如果该名称是大小写混合的或包含特殊字符，要包括双引号，例如"FooBar"；如果没有引号，字符串将被折叠到小写形式。

`primary_key_attnums`

主键域的属性号（从 1 开始），例如 1 2。

`num_primary_key_atts`

主键域的数量。

`src_pk_att_vals_array`

要被用来查找本地元组的主键域值。每一个域都被表示为文本形式。如果没有行具有这些主键值，则抛出一个错误。

tgt_pk_att_vals_array

要用在结果 UPDATE 命令中的主键域值。每一个域都被表示为文本形式。

返回值

将要求的 SQL 语句返回为文本。

注解

自 KingbaseES 9.0 开始，primary_key_attnums 中的属性号被解释为逻辑列号，对应于列在 SELECT * FROM relname 中的位置。之前的版本将属性号解释为物理列位置。如果指示出的列的左边有任意列在该表的生存期内被删除，这两种解释就有区别。

示例

```
SELECT dblink_build_sql_update('foo', '1 2', 2, '{"1", "a"}', '{"1", "b"}');
            dblink_build_sql_update
------------------------------------------------------------
 UPDATE foo SET f1='1',f2='b',f3='1' WHERE f1='1' AND f2='b'
(1 row)
```

3.12.5 插件 dblink 卸载方法

```
DROP EXTENSION dblink;
```

3.12.6 插件 dblink 升级方法

```
ALTER EXTENSION dblink UPDATE;
```

3.13 dbms_ddl

3.13.1 插件 dbms_ddl 简介

插件 dbms_ddl 是 KingbaseES 的一个扩展插件。插件 dbms_ddl 功能是提供 dbms_ddl 系统包。使用 DBMS_DDL 包可以在存储过程、函数、触发器等 PL/SQL 命名块和匿名块中执行 DDL 语句，以及执行一些 DDL 语句的加密操作。

- 插件名为 dbms_ddl
- 插件版本 V1.0

3.13.2　插件 dbms_ddl 加载方式

该插件为初始化数据库实例时默认创建。

3.13.3　插件 dbms_ddl 的参数配置

dbms_ddl 扩展插件无需配置任何参数。

3.13.4　插件 dbms_ddl 使用方法

3.13.4.1　DBMS_DDL.WRAP

功能描述

只加密不执行语句。

函数 WRAP 以 CREATE [OR REPLACE] 语句作为输入，指定创建包体、函数、过程。并返回 CREATE [OR REPLACE] 语句的加密语句。

语法格式

```
DBMS_DDL.WRAP（ddl VARCHAR2）RETURN VARCHAR2;
```

参数说明

ddl 指定用于创建一个包体、函数、过程的 CREATE [OR REPLACE] 语句。

返回值

返回 32k 字节以内已加密的字符串。

注意事项

- 任何试图调用该子程序的语句都应使用标准的包名 SYS.DBMS_DDL 以避免与本地定义的 DBMS_DDL 同义。
- 不能对 trigger 进行加密，如果需要对 trigger 进行加密请直接加密函数，然后通过触发器去调用该函数即可。
- 支持 $...*pg*..$ 的 pg 语法的 plsql 对象或 plpgsql 对象。
- wrap 函数中一次只能输入单条的 PL/SQL 或者 PL/pgSQL 语句。

异常说明

如果不是 CREATE [OR REPLACE] 指定的语句，则报错“向 DBMS_DDL.WRAP 输入的内容不是合法的 PL/SQL 单元”。

示例

为 PL/SQL 语句和 PL/pgSQL 语句加密，用例如下。

```
\set SQLTERM /
select dbms_ddl.wrap(
'create or replace function func4 return int is
begin
NULL;
return 3;
end;');
/
                            wrap
-------------------------------------------------------------
create or replace function func4 return int is WRAPPED+
htpt6DqPHq8uEHewSse+kc8myKyoI7I9f5mTI2Ob72I=          +
END;
(1 row)


\set SQLTERM /
select dbms_ddl.wrap(
'CREATE OR replace function func_test() returns int AS $$
BEGIN
return 1;
END $$language plpgsql;');
/
                                wrap
-------------------------------------------------------------------
CREATE OR replace function func_test() returns int AS $$ WRAPPED+
EmUKDO9/8ueGxdvzZRYwU3o6X3YY9jZ6FeYIauqV+oM=                    +
END;                                                            +
$$language plpgsql;
(1 row)
```

3.13.4.2　DBMS_DDL.CREATE_WRAPPED

功能描述

加密并执行语句。

过程 CREATE_WRAPPED 的功能是创建一个包体、函数、过程。它的实现流程是将输入参数的 CREATE [OR REPLACE] 语句转换为加密后的 CREATE [OR REPLACE] 语句，并执行转换后语句。实际上，该过程将加密与创建语句合为一体。

语法格式

```
DBMS_DDL.CREATE_WRAPPED (ddl VARCHAR2);
```

参数说明

ddl 指定用于创建一个 PL/SQL 包体、函数、过程的 CREATE [OR REPLACE] 语句。

注意事项

- 任何试图调用这些 PL/SQL 代码都应使用标准的包名 SYS.DBMS_DDL 以避免与本地定义的 DBMS_DDL 同义。
- CREATE_WRAPPED 过程与 WRAP 函数一次只能输入单条 PL/SQL 或 PL/pgSQL 语句。
- 不能对 trigger 进行加密，如果需要对 trigger 进行加密请先加密函数，然后通过触发器去调用该函数。
- 支持 $...*pg*..$ 的 pg 语法的 PL/SQL 对象或 PL/pgSQL 对象。

异常说明

如果不是 CREATE [OR REPLACE] 指定的 PL/SQL 语句，则报错“向 DBMS_DDL.WRAP 输入的内容不是合法的 PL/SQL 单元”。

示例

以下用例为使用 create_wrapped 子程序创建加密的 PL/SQL 对象和 PL/pgSQL 对象。

```
\set SQLTERM /
exec dbms_ddl.create_wrapped(
'create or replace function func4 return int is
begin
   NULL;
   return 3;
end;');
/

\set SQLTERM /
exec dbms_ddl.create_wrapped(
'CREATE OR replace function func_test() returns int AS $$
BEGIN
    return 1;
END $$language plpgsql;');
/
```

3.13.5　插件 dbms_ddl 卸载方法

无法卸载。

3.13.6　插件 dbms_ddl 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1

```
ALTER EXTENSION dbms_ddl UPDATE TO '1.1';
```

3.14　dbms_lob

3.14.1　插件 dbms_lob 简介

dbms_lob 是 KingbaseES 的一个扩展插件，它定义了一套对于 CLOB/BLOB 大对象的操作。支持 dbms_lob。参见《PL/SQL 系统包和类型参考手册》中的 DBMS_LOB 包

- 插件名为 dbms_lob
- 插件版本 V1.2

3.14.2　插件 dbms_lob 的加载方式

此插件依赖 kdb_raw 插件，需要在 kdb_raw 插件加载之后才可以成功创建。或者使用 cascade 关键字同时加载依赖的插件。

```
create extension dbms_lob [cascade];
```

3.14.3　插件 dbms_lob 的参数配置

无需配置任何参数

3.14.4　插件 dbms_lob 卸载方法

```
drop extension dbms_lob;
```

3.14.5　插件 dbms_lob 升级方法

如果使用旧版本插件，可以升级到最新版本。

备注:

不支持跨版本升级。建议由低到高依次升级版本。

```
alter extension dbms_lob update to '1.2';
```

3.15 dbms_metadata

3.15.1 插件 dbms_metadata 简介

DBMS_METADATA 包是 KingbaseES 数据库中提供的从数据库字典中以 XML 或者 DDL 语句形式检索元数据的一系列函数。

使用时需要 create extension dbms_metadata，不需要时 drop extension dbms_metadata 即可。

- 插件名为 dbms_metadata
- 插件版本 V1.3

3.15.2 插件 dbms_metadata 加载方式

```
CREATE EXTENSION dbms_metadata;
```

3.15.3 插件 dbms_metadata 的参数配置

无需配置任何参数。

3.15.4 插件 dbms_metadata 使用方法

3.15.4.1 DBMS_METADATA.GET_DDL

功能

KingbaseES 兼容 ORACLE 的 DBMS_METADATA.GET_DDL() 函数。该函数用于创建特定类型对象的 DDL 语句。

目前可支持获取以下类型对象的 DDL

- TABLE
- VIEW
- INDEX
- FUNCTION

- PROCEDURE
- TRIGGER
- SEQUENCE
- DBLINK
- SYNONYM
- PACKAGE
- OBJECT TYPE

语法格式

使用该函数前需要先创建 DBMS_METADATA 扩展。

```
DBMS_METADATA.GET_DDL(
        OBJTYPE VARCHAR2(63),
        OBJNAME VARCHAR2(63),
        SCHEMA VARCHAR2(63) DEFAULT NULL,
        VERSION VARCHAR2(63) DEFAULT NULL,
        MODEL VARCHAR2(63) DEFAULT NULL,
        TRANSFORM VARCHAR2(63) DEFAULT NULL);
```

参数说明

表 3.15.1: DBMS_METADATA.GET_DDL 参数

参数名称	描述
OBJTYPE	指定对象的数据类型
OBJNAME	指定对象名
SCHEMA	指定对象的模式名，默认值是 NULL
VERSION	指定元数据的版本，默认值是 NULL
MODEL	指定要使用的对象的模型，默认值是 NULL
TRANSFORM	指定输出转换的名称，默认值是 NULL

备注:

1. VERSION，MODEL，TRANSFORM 这三个参数值在目前的 KingbaseES 中暂时不做处理。
2. 参数内容大小写不敏感。

返回值类型

以 TEXT 返回指定对象的 DDL 语句。暂不支持指定输出转换类型。

对于 TABLE 类型的对象，该函数的返回值包含以下内容：

- SCHEMA 名；
- TABLE 名；
- 列名，列的数据类型，默认值，约束；
- FOREIGN KEY；（如果存在）；
- PARTITION 类型和字段（如果存在）。

权限

DBA 创建 DBMS_METADATA 扩展后，允许其他用户执行该扩展下的 GET_DDL 函数，且其他模式均可使用。

示例

```
--TABLE
CREATE TABLE test_tab1(
    a smallserial,
    b int,
    c text,
    d text default 'KB',
    e text
) PARTITION BY HASH(a,c,e);
CREATE TABLE
SELECT dbms_metadata.get_ddl('table', 'test_tab1');
             get_ddl
---------------------------------
 CREATE TABLE public.test_tab1(+
     a smallserial NOT NULL,   +
     b integer,                +
     c text,                   +
     d text DEFAULT 'KB'::text,+
     e text)                   +
 PARTITION BY HASH (a, c, e);
(1 row)

--FUNCTION
CREATE FUNCTION test_func1(text, date) RETURNS bool LANGUAGE 'sql' AS 'SELECT $1 = ''abcd'' AND $2 > ''2001-
01-01''';

SELECT dbms_metadata.get_ddl('FUNCTION', 'TEST_FUNC1');
```

```
                                get_ddl
-------------------------------------------------------------------
 CREATE OR REPLACE FUNCTION public.test_func1(text, date)       +
  RETURNS boolean                                               +
  LANGUAGE sql                                                  +
 AS $function$SELECT $1 = 'abcd' AND $2 > '2001-01-01'$function$+
 ;
(1 row)

--INDEX
CREATE TABLE test_tab5(a varchar(32), b varchar(32));
CREATE INDEX idx5 ON test_tab5((a||' '||b));
SELECT dbms_metadata.get_ddl('INDEX', 'idx5');
                                                 get_ddl
-----------------------------------------------------------------------------------------------------
 CREATE INDEX idx5 ON public.test_tab5 USING btree ((((a || ' '::varchar) || b)));
(1 row)
```

3.15.5　插件 dbms_metadata 卸载方法

```
DROP EXTENSION dbms_metadata;
```

3.15.6　插件 dbms_metadata 升级方法

```
ALTER EXTENSION dbms_metadata UPDATE;
```

3.16　dbms_mview

3.16.1　插件 dbms_mview 简介

插件 dbms_mview 是 KingbaseES 的一个扩展插件。插件 dbms_mview 功能是提供 dbms_mview 系统包。dbms_mview 包可以对物化视图进行一些功能操作。

3.16.2　插件 dbms_mview 加载方式

启动 KingbaseES 数据库后执行 SQL 命令创建 dbms_mview 插件。

示例

```
CREATE EXTENSION dbms_mview;
```

3.16.3　插件 dbms_mview 使用方法

3.16.3.1　REFRESH 存储过程

功能描述

刷新不是同一刷新组的一或多个物化视图。

语法格式

```
DBMS_MVIEW.REFRESH (
 list  IN  VARCHAR2,
 method  IN  VARCHAR2 DEFAULT NULL,,
 rollback_seg  IN  VARCHAR2  DEFAULT NULL,
 push_deferred_rpc  IN  BOOLEAN  DEFAULT TRUE,
 refresh_after_errors  IN  BOOLEAN  DEFAULT FALSE,
 purge_option  IN  INTEGER DEFAULT 1,
 parallelism  IN  INTEGER DEFAULT 0,
 heap_size  IN  BINTEGER DEFAULT 0,
 atomic_refresh  IN  BOOLEAN  DEFAULT TRUE,
 nested  IN  BOOLEAN  DEFAULT FALSE,
 out_of_place  IN  BOOLEAN  DEFAULT FALSE);
```

参数说明

`list`

VARCHAR2 类型，待刷新的物化视图名称。

`method`

VARCHAR2 类型，指定刷新物化视图的方式，默认值为 null 。

Oracle 模式下支持以下几种刷新方式：- FAST 刷新：增量刷新，参数为’f’或’F’；- COMPLETE 刷新：全量刷新，参数为’c’或’C’；- FORCE 刷新：优先执行 `FAST` 刷新，执行失败，则进行 `COMPLETE` 刷新，参数为’?’。

若未指定刷新方式，则根据创建物化视图时指定的刷新方式进行刷新。

备注:

目前实现 `list` 和 `method` 参数功能，其他参数仅语法支持。

示例

```
CREATE TABLE matview (id INT,name TEXT);
CREATE MATERIALIZED VIEW mymatview AS SELECT * FROM matview;
CALL DBMS_MVIEW.REFRESH ('mymatview', 'c');

-- Oracle 模式下指定其它刷新方式:
CALL DBMS_MVIEW.REFRESH ('mymatview', 'f');
CALL DBMS_MVIEW.REFRESH ('mymatview', '?');
CALL DBMS_MVIEW.REFRESH ('mymatview');
```

3.16.4　插件 dbms_mview 卸载方法

在数据库中执行 SQL 命令卸载插件。

```
DROP EXTENSION dbms_mview;
```

3.16.5　插件 dbms_mview 升级方法

dbms_mview 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.17　dbms_obfuscation_toolkit

3.17.1　插件 dbms_obfuscation_toolkit 简介

dbms_obfuscation_toolkit 系统包可以提供 DES 或 3DES 算法进行数据的加解密。同时，该包也提供了数据的 MD5 计算接口。

- 插件名为 dbms_obfuscation_toolkit
- 插件版本 V1.0

3.17.2　插件 dbms_obfuscation_toolkit 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'dbms_obfuscation_toolkit'
```

3.17.3　插件 dbms_obfuscation_toolkit 的参数配置

无

3.17.4　插件 dbms_obfuscation_toolkit 使用方法

3.17.4.1　限制

1. DBMS_OBFUSCATION_TOOLKIT 包函数目前支持 bytea 格式数据输入。对于带有转义字符的数据源，解析时会按转义字符进行处理，使用时需要注意数据的实际长度。
2. DES/3DES 加解密接口对于数据和密钥长度有限制。

   a）输入的数据或密钥长度为空时，报错 28231 - “Invalid input to Obfuscation toolkit”

   b）输入的数据长度必须为 8 的倍数，否则报错 28232 - “Invalid input size for Obfuscation toolkit.”

   c）密钥长度缺失或者小于 8 时，报错 28234 - “Key length too short.”

   d）3DES 加解密接口的 which 参数目前支持取值 1，否则报错 28236 - “Invalid Triple DES mode”

3.17.4.2　启用

语法格式

```
CREATE EXTENSION DBMS_OBFUSCATION_TOOLKIT;
```

功能描述

加载插件，启用 DBMS_OBFUSCATION_TOOLKIT 包。

3.17.4.3　禁用

语法格式

```
DROP EXTENSION DBMS_OBFUSCATION_TOOLKIT;
```

功能描述

卸载插件，禁用 DBMS_OBFUSCATION_TOOLKIT 包。

3.17.4.4　DBMS_OBFUSCATION_TOOLKIT 系统包子函数接口

3.17.4.4.1　DES3ENCRYPT

语法格式

```
des3encrypt(input_string IN bytea, key_string IN bytea, which IN integer) RETURNS bytea;

des3encrypt(input_string IN bytea, key_string IN bytea, which IN integer, iv_string IN bytea) RETURNS bytea;
```

功能描述

使用 3DES 算法对数据进行加密。

参数说明

`input_string`
待加密数据。

`key_string`
用于加密数据的密钥。

`which`
使用的加解密密钥方式。取值 1，表示 3DES 密钥。

`iv_string`
仅当使用 CBC 加密方式时使用，初始 IV 值。

返回值说明

返回加密后的数据。

示例

参见 DES3DECRYPT。

3.17.4.4.2　DES3DECRYPT

语法格式

```
des3decrypt(input_string IN bytea, key_string IN bytea, which IN integer) RETURNS bytea;

des3decrypt(input_string IN bytea, key_string IN bytea, which IN integer, iv_string IN bytea) RETURNS bytea;
```

功能描述

使用 3DES 算法对数据进行解密。

参数说明

`input_string`
待解密数据。

`key_string`
用于解密数据的密钥。

`which`
使用的加解密密钥方式。取值 1，表示 3DES 密钥。

iv_string
仅当使用 CBC 加密方式时使用，初始 IV 值。

返回值说明

返回解密后的数据。

示例

```
\c - system
set bytea_output to escape;
select DBMS_OBFUSCATION_TOOLKIT.des3decrypt(DBMS_OBFUSCATION_TOOLKIT.des3encrypt('0123456789abcdef',
'0123456789abcdef87654321', 1), '0123456789abcdef87654321', 1);
    des3decrypt
 ------------------
   0123456789abcdef
    (1 行记录)

select DBMS_OBFUSCATION_TOOLKIT.des3decrypt(DBMS_OBFUSCATION_TOOLKIT.des3encrypt('0123456789abcdef',
'0123456789abcdef87654321', 1, '12345678'), '0123456789abcdef87654321', 1, '12345678');
    des3decrypt
------------------
   0123456789abcdef
    (1 行记录)
```

3.17.4.4.3　DESENCRYPT

语法格式

```
desencrypt(input_string IN bytea, key_string IN bytea) RETURNS bytea;
```

功能描述

使用 DES 算法对数据进行加密。

参数说明

input_string
待加密数据。

key_string
用于加密数据的密钥。

返回值说明

返回加密后的数据。

示例

参见 DESDECRYPT。

3.17.4.4.4　DESDECRYPT

语法格式

```
desdecrypt(input_string IN bytea, key_string IN bytea) RETURNS bytea
```

功能描述

使用 DES 算法对数据进行解密。

参数说明

`input_string`
待解密数据。

`key_string`
用于解密数据的密钥。

返回值说明

返回解密后的数据。

示例

```
select DBMS_OBFUSCATION_TOOLKIT.desdecrypt(DBMS_OBFUSCATION_TOOLKIT.desencrypt('0123456789abcdef',
'012345678'), '012345678');
    desdecrypt
------------------
    0123456789abcdef
    (1 行记录)
```

3.17.4.4.5　MD5

语法格式

```
md5(input_string IN bytea) RETURNS bytea;
```

功能描述

实现数据的 MD5 哈希值计算，生成的哈希值为 128-bits 的数据摘要。

参数说明

`input_string`
待计算 hash 的数据。

返回值说明

返回生成的 hash 值。

示例

```
\c - system
set bytea_output to hex;
select DBMS_OBFUSCATION_TOOLKIT.MD5('123456#$');
    md5
------------------------------------
\xd9aa3f4d907b5d633b78e9e1d699cf96
(1 行记录)
```

3.17.5 插件 dbms_obfuscation_toolkit 卸载方法

通过 drop extension 来卸载插件：

```
drop extension dbms_obfuscation_toolkit;
```

如果设置了 shared_preload_libraries，则需要删除‘dbms_obfuscation_toolkit’：

```
shared_preload_libraries = ''
```

3.17.6 插件 dbms_obfuscation_toolkit 升级方法

dbms_obfuscation_toolkit 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.18 dbms_output

3.18.1 插件 dbms_output 简介

插件 dbms_output 是 KingbaseES 的一个扩展插件。插件 dbms_output 功能是提供 dbms_output 系统包。

DBMS_OUTPUT 系统包可以将提供的文本行写入缓冲区，供以后提取和显示的功能。主要用于调试 PL/SQL 程序，或者在 Ksql 客户端命令中显示信息和报表，譬如我们可以写一个简单的匿名 PL/SQL 程序块，而该块出于某种目的可以使用此包来显示一些信息。

- 插件名为 dbms_output
- 插件版本 V1.0

3.18.2 插件 dbms_output 加载方式

该插件为初始化数据库实例时默认创建。

3.18.3　插件 dbms_output 的参数配置

dbms_output 扩展插件无需配置任何参数。

3.18.4　插件 dbms_output 使用方法

3.18.4.1　限制

DBMS_OUTPUT 行和缓冲区大小的限制：

1. 最大行大小为 32767 字节。
2. 默认缓冲区大小为 20000 字节。最小为 2000 字节，最大不超过 1000000 字节。

3.18.4.2　启用

语法格式

```
DBMS_OUTPUT.ENABLE(buffer_size IN INTEGER DEFAULT 20000);
```

功能描述

这个过程允许调用 PUT、PUT_LINE、NEW_LINE、GET_LINE 和 GET_LINES 等过程。

参数说明

buffer_size: 缓冲区最大尺寸为 1000000 字节，最小为 2000 字节，默认为 20000 字节。将 buffer_size 设置为 null，缓冲区大小将没有限制。

使用说明

1. 如果没有激活 DBMS_OUTPUT 包，对这些过程的调用将被忽略。
2. 如果有多个调用要启用，那么 buffer_size 是最后调用者指定的值。
3. 当 set serveroutput on 开启时可以不用调用 enable 和 disable 过程。
4. 缓存区设定大小的范围为 2000 至 1000000，如果 buffer_size 的值小于 2000，则为 2000；如果 buffer_size 的值超过 1000000，则报错；如果输入缓存区的数据大小超过缓存区设定的最大值，将直接报错。

示例

输入参数不同，启用缓冲区大小也不同，示例如下：

```
call dbms_output.enable();
call dbms_output.enable(NULL);
call dbms_output.enable(3000);
```

3.18.4.3　禁用

语法格式

```
DBMS_OUTPUT.DISABLE();
```

功能描述

此过程禁用对 PUT、PUT_LINE、NEW_LINE、GET_LINE 和 GET_LINES 等过程的调用，并清空缓冲区。

使用说明

1. 如果该 DBMS_OUTPUT 包被禁用了，那么所有对其子程序 (subprogram) 的调用都将被忽略。这样用户可以设计应用程序，仅在客户端程序能够处理这些信息的时候启用这些子程序。
2. 如果 set serveroutput off 时调用了 disable 过程，下次调用其他过程前，必须先调用 enable 过程。

3.18.4.4　DBMS_OUTPUT 系统包支持向缓冲区输入信息

3.18.4.4.1　向缓冲区中追加内容

语法格式

```
DBMS_OUTPUT.PUT(item IN VARCHAR2);
```

功能描述

该过程用来追加部分内容到 Buffer 中的最后一行。

参数说明

item: 追加的字符串。

使用说明

1. 当追加内容时可以使用 put 过程输入，如需整行方式输入最好使用 put_line 过程。
2. 若单行长度超过限制那么将报错，或输入数据超过缓存区设定值将报错。
3. 注意在调用 PUT 或 PUT_LINE 的 PL/SQL 程序单元结束之前，对 PUT 或 PUT_LINE 指定的内容不会输出。
4. 当调用 PUT_LINE 过程时将自动加入换行符，若使用 PUT 来构建行，那么必须手动使用 NEW_LINE 过程来加入换行符，不然由 PUT 构建的行将不会返回任何值。GET_LINE 和 GET_LINES 过程不会返回没有以换行符终结的一行。

示例

向缓冲区追加内容，示例如下：

```
set serveroutput on
\set SQLTERM /
begin
    dbms_output.put('test1');
    dbms_output.new_line();
    dbms_output.put('test2');
end;
/

ANONYMOUS BLOCK
test1

set serveroutput on
\set SQLTERM /
begin
    dbms_output.put('test1');
    dbms_output.new_line();
    dbms_output.put('test2');
    dbms_output.new_line();
end;
/

ANONYMOUS BLOCK
test1
test2
```

3.18.4.4.2　向缓冲区追加单行信息

语法格式

```
DBMS_OUTPUT.PUT_LINE(item IN VARCHAR2);
```

功能描述

该存储过程用来向 Buffer 中新添一行信息。

参数说明

item: 追加的字符串。

使用说明

1. 输入数据超过缓存区设定值将报错。
2. GET_LINE 和 GET_LINES 不返回未使用换行符终止的行。

示例

向缓冲区追加单行信息，示例如下：

```
set serverout on
\set SQLTERM /
call dbms_output.put_line('hi' || chr(10) || 'hello'||chr(10) || '
world');
/

CALL
hi
hello

world
```

3.18.4.4.3　向缓冲区追加新行

语法格式

```
DBMS_OUTPUT.NEW_LINE();
```

功能描述

该存储过程向缓冲区添加一个换行符，产生新一行。

使用说明

每调用一次生成一次新行。

示例

向缓冲区追加新行，示例如下：

```
set serverout on
\set SQLTERM /
begin
    dbms_output.put('test1');
    dbms_output.new_line();
    dbms_output.put('test2');
    dbms_output.new_line();
    dbms_output.new_line();
    dbms_output.put_line('test3');
end;
/

ANONYMOUS BLOCK
test1
test2

test3
```

3.18.4.5　DBMS_OUTPUT 系统包支持从缓冲区中获取信息

3.18.4.5.1　获取首行信息

语法格式

```
DBMS_OUTPUT.GET_LINE(line OUT VARCHAR2, status OUT INTEGER);
```

功能描述

该过程从 buffer 中获取单行信息，先加入的信息先获取，获取后 buffer 中的此信息将被删除。

参数说明

1. line 将获取 buffer 中的一行信息，但不包括最后的换行符。
2. status 若调用成功则返回 0，调用失败则返回 1。

使用说明

调用 GET_LINE 成功获取内容后，在下一次调用 PUT，PUT_LINE 或 NEW_LINE 过程前没有被获取的缓存都将被丢弃，以避免误会。

示例

获取首行信息，示例如下：

```
set serverout on
\set SQLTERM /
declare
    line varchar2(120);
    status integer;
begin
    dbms_output.put('hello');
    dbms_output.put_line('world');
    dbms_output.get_line(line, status);
    dbms_output.put_line(line || status);
end;
/

ANONYMOUS BLOCK
helloworld0
```

3.18.4.5.2　获取多行信息

语法格式

```
PROCEDURE get_lines(lines OUT CHARARR, numlines IN OUT integer);
PROCEDURE get_lines(lines OUT DBMSOUTPUT_LINESARRAY, numlines IN OUT integer);
```

功能描述

该存储过程用以从 Buffer 中获取一个多行的数组，获取后 buffer 中的此信息将被删除。

参数说明

1. CHARARR(包内类型)TYPE CHARARR IS TABLE OF VARCHAR2(32767) INDEX BY INT;
2. DBMSOUTPUT_LINESARRAY(包外类型)TYPE DBMSOUTPUT_LINESARRAY IS VARRAY(2147483647) OF VARCHAR2(32767);
3. lines 返回缓冲信息的行数组。数组中每一行的最大长度为 32767 字节。
4. numlines 当缓存区行数多于或等于检索的行数时，返回检索行数；当缓存区行数少于检索的行数时，返回缓存区行数。

使用说明

1. 在获取信息之后，若下次调用输入过程，则没有检索到的任何行都将被丢弃，以避免与下一条消息混淆。
2. CHARARR 类型 numlines 参数当不指定参数或此参数小于等于 0，不获取内容，并返回 numlines 为 0；当此参数大于 0，如果缓存区行数多于或等于检索的行数时，返回检索行数；如果缓存区行数少于检索的行数时，返回缓存区行数。
3. DBMSOUTPUT_LINESARRAY 类型 numlines 参数当不指定参数，默认获取全部内容，并返回 numlines 为获取行数；当此参数小于 0，将报错；当参数大于等于 0 时，如果缓存区行数多于或等于检索的行数时，返回检索行数，当缓存区行数少于检索的行数时，返回缓存区行数。

示例

get_lines 有多种重载方式，示例如下：

```
set serverout on
\set SQLTERM /
DECLARE
    v_data  dbms_output.chararr;
    numlines integer := 2;
BEGIN
    dbms_output.put_line('TEST 1');
    dbms_output.put_line('TEST 2');
    dbms_output.put_line('TEST 3');
    dbms_output.get_lines(v_data, numlines);
    dbms_output.put_line(v_data(1));
    dbms_output.put_line(v_data(2));
end;
/

ANONYMOUS BLOCK
TEST 1
TEST 2
```

```
DECLARE
    v_data  dbmsoutput_linesarray;
    numlines integer := 2;
BEGIN
    dbms_output.put_line('TEST 1');
    dbms_output.put_line('TEST 2');
    dbms_output.get_lines(v_data, numlines);
    dbms_output.put_line(numlines);
    dbms_output.put_line(v_data(1));
    dbms_output.put_line(v_data(2));
end;
/

ANONYMOUS BLOCK
2
TEST 1
TEST 2
```

3.18.4.6　DBMS_OUTPUT 系统包支持客户端显示缓冲区信息

3.18.4.6.1　客户端命令启动输出功能

语法格式

```
set serverout[put] on[;]
```

功能描述

1. 启用 DBMS_OUTPUT 系统包功能。
2. 缓存大小设为不限制大小。
3. 能代替 enable 过程，即可以不需要 enable 过程。
4. 客户端输出缓存区消息。

使用说明

set serverout[put] 跟‘on’或‘on;’，当输入无关命令时会提示命令错误。

示例

通过在 ksql 客户端设置 set serveroutput on 命令，即可显示缓冲区信息，示例如下：

```
set serverout on
\set SQLTERM /
create or replace procedure protest_A_1 is
```

```
begin
  dbms_output.put_line('....test1');
  dbms_output.put_line('....test2');
end;
/
CREATE PROCEDURE

declare
  v_status integer := 0;
  v_data varchar2(100);
begin
  protest_A_1();
  dbms_output.put_line('test3');
  dbms_output.get_line(v_data, v_status);
  dbms_output.put_line('v_data: ' ||v_data || 'v_status: ' ||v_status);
  dbms_output.put_line('test4');
end;
/
ANONYMOUS BLOCK
v_data: ....test1v_status: 0
test4
```

3.18.4.6.2　客户端命令禁用输出功能

语法格式

```
set serverout[put] off[;]
```

功能描述

1. 禁用 DBMS_OUTPUT 系统包功能。
2. 清除缓存。
3. 能代替 disable 过程，即可以不需要 disable 过程。

使用说明

set serverout[put] 跟‘off’或‘off;’，当输入无关命令时会提示命令错误。

示例

通过在 ksql 客户端工具设置 set serveroutput off 命令，即可禁用 dbms_output 输出功能。示例如下：

```
set serverout off
\set SQLTERM /
call dbms_output.put_line('hi' || chr(10) || 'hello'||chr(10) || '
    world');
```

```
/
CALL
```

3.18.5　插件 dbms_output 卸载方法

无法卸载。

3.18.6　插件 dbms_output 升级方法

dbms_output 插件随着 KingbaseES 安装包一并升级。通常，用户无须单独升级该插件。

3.19　dbms_random

3.19.1　插件 dbms_random 简介

dbms_random 是 KingbaseES 的一个扩展插件，主要提供一组函数产生随机数或者随机字符。

插件需要手动创建，在服务端程序启动时没有默认加载。

- 插件名为 dbms_random
- 插件版本 V1.0

3.19.2　插件 dbms_random 加载方式

启动 KingbaseES 数据库后执行 SQL 命令创建插件。

示例

```
create extension dbms_random;
```

3.19.3　插件 dbms_random 的参数配置

无需配置任何参数

3.19.4　插件 dbms_random 的使用方法

创建插件后提供一组函数来产生随机数或随机字符。

下表列出了 dbms_random 系统包并简要的表述了它们。

表 3.19.1: dbms_random 软件包子程序和函数

子程序/函数	描述
INITIALIZE Procedure	使用一个种子值来初始化包
NORMAL Function	产生标准的正态分布函数
RANDOM Function	产生一个随机整数
SEED Procedure	重置随机数种子
STRING Function	产生一个随机字符串
TERMINATE Procedure	提供语法上的兼容不执行任何操作
VALUE Function	产生指定范围的随机数

3.19.4.1　INITIALIZE Procedure

使用一个种子值来初始化 dbms_random 包，默认情况下，dbms_random 包是根据用户、时间、会话来初始化。Initialize 这个函数已经过时，虽然目前受到支持，但考虑历史遗留原因和兼容性保留此初始化方式。

语法格式

```
dbms_random.initialize(init INTEGER);
```

参数说明

表 3.19.2: initialize 参数

参数	描述
init	随机数种子值

返回值说明

VOID

示例

```
CALL dbms_random.initialize(8);
```

3.19.4.2　NORMAL Function

产生标准的正态分布函数，此正态分布的标准差为 1，期望值为 0。

语法格式

```
dbms_random.normal();
```

参数说明

无参

返回值说明

DOUBLE

示例

```
SELECT dbms_random.normal();
```

3.19.4.3　RANDOM Function

产生一个随机整数，这个函数已经过时了，虽然目前还支持，但不应该使用它，考虑兼容性保留此函数。

语法格式

```
dbms_random.random();
```

参数说明

无参

返回值说明

INTEGER

示例

```
CALL dbms_random.random();
```

3.19.4.4　SEED Procedure

重置种子，类似于 initialize 函数，initialize 函数已经被淘汰。seed 函数同时支持数值和字符作为种子值，而 initialize 只支持数值。

语法格式

```
dbms_random.seed(i INTEGER);
dbms_random.seed(t TEXT);
```

参数说明

表 3.19.3: seed 参数

参数	描述
i	随机数种子值
t	随机数字符种子值

返回值说明

表 3.19.4: seed 返回值

函数	类型
seed(i INTEGER)	VOID
seed(t TEXT)	VOID

示例

```
CALL dbms_random.seed(8);
CALL dbms_random.seed('test');
```

3.19.4.5　STRING Function

获取一个随机字符串，第一个参数是字符串的格式，第二个参数是指字符串的长度。长度最大为 5000，参数如果超过 5000 会自动以 5000 处理。

语法格式

```
dbms_random.string(type TEXT,len INTEGER);
```

参数说明

表 3.19.5: string 参数

参数	描述
type	产生字符串的类型
len	产生字符串的长度

表 3.19.6: type 参数对应含义

值	描述
'u','U'	返回大写字母的字符串
'l','L'	返回小写字母的字符串
'a','A'	返回大小写字母混合的字符串
'x','X'	返回大写字母和数字的字符串
'p','P'	返回任何可打印字符的字符串

返回值说明

TEXT

示例

```
SELECT dbms_random.string('U',5);
SELECT dbms_random.string('P',2);
SELECT dbms_random.string('x',4);
SELECT dbms_random.string('a',2);
SELECT dbms_random.string('l',3);
```

3.19.4.6　TERMINATE Procedure

原功能为使用完 dbms_random 包的时候执行此函数，但是现在此函数不执行任何功能，为了语法兼容保留。

语法格式

```
dbms_random.terminate();
```

参数说明

无参

返回值说明

VOID

示例

```
CALL dbms_random.terminate();
```

3.19.4.7　VALUE Function

第一种无参的使用方式是获取一个大于或者等于 0 且小于 1 的随机数，精度为 16 位的小数。

第二种使用方式可以指定最小值和最大值，返回值的范围大于或者等于 low，小于 high，精度同为 16 位小数。

语法格式

```
dbms_random.value();
dbms_random.value(low INTEGER,high INTEGER);
```

参数说明

表 3.19.7: value 参数

参数	描述
low	产生的随机数最小值
high	产生的随机数最大值

返回值说明

表 3.19.8: value 返回值

函数	类型
value()	double precision
value(low INTEGER,high INTEGER)	double precision

示例

```
SELECT dbms_random.value();
SELECT dbms_random.value(10,15);
```

3.19.5　插件 dbms_random 卸载方法

在数据库中执行 SQL 命令卸载插件。

```
drop extension dbms_random;
```

3.19.6　插件 dbms_random 升级方法

dbms_random 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.20　dbms_session

3.20.1　插件 dbms_session 简介

dbms_session 系统包可以用来访问或者设置会话信息。定义了对于 session 的一套操作，使用前需创建扩展 dbms_session。

- 插件名为 dbms_session
- 插件版本 V1.0

3.20.2　插件 dbms_session 加载方式

启动 KingbaseES 数据库后执行 SQL 命令创建插件。

示例：

```
create extension dbms_session;
```

3.20.3　插件 dbms_session 的参数配置

无需配置任何参数

3.20.4　插件 dbms_session 的使用方法

在使用 dbms_session 之前，需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'dbms_session'
```

使用时需要 create extension dbms_session。

不需要使用时 drop extension dbms_session 即可。

3.20.4.1　DBMS_SESSION 系统包子程序

DBMS_SESSION 系统包包含的子程序及简介如下表所示：

表 3.20.1: DBMS_SESSION 包子程序

子程序	简介
CLEAR_ALL_CONTEXT 过程	清理当前会话的指定 namespace 的所有上下文。
CLEAR_CONTEXT 过程	清除 namespace 中的 attribute 值。
LIST_CONTEXT 过程	返回当前会话所有上下文属性和值。
SET_CONTEXT 过程	设置上下文 namespace 的属性和值。

3.20.4.2　CLEAR_ALL_CONTEXT 过程

清理当前会话的指定 namespace 的所有上下文。该过程需通过 namespace 关联的 package 调用。

语法

```
DBMS_SESSION.CLEAR_ALL_CONTEXT(namespace VARCHAR2(63));
```

参数

表 3.20.2: CLEAR_ALL_CONTEXT 参数

参数名称	描述
namespace	应用程序上下文信息的名称。最大长度 63bytes。

备注:

示例见SET_CONTEXT 过程 。

3.20.4.3　CLEAR_CONTEXT 过程

清除 namespace 中的 attribute 值。该过程需通过上下文 namespace 关联的 package 调用。

语法

```
DBMS_SESSION.CLEAR_CONTEXT(namespace VARCHAR2(63),client_identifier VARCHAR2(30),attribute VARCHAR2(128));
```

参数

表 3.20.3: CLEAR_CONTEXT 参数

参数名称	描述
namespace	要清除上下文信息的名称。最大长度 63bytes，不区分大小写。
client_identifier	用于全局访问的 client 标识，不起作用，如果指定，则忽略。
attribute	要清除的上下文的属性名，不区分大小写。若为 null，则清除 namespace 下所有 key-value。

备注:

示例见 SET_CONTEXT 过程 。

3.20.4.4　LIST_CONTEXT 过程

返回当前会话所有上下文属性和值。

语法

```
TYPE AppCtxRecTyp IS RECORD(
   namespace VARCHAR2(63),
   attribute VARCHAR2(128),
   value VARCHAR2(4000)
```

```
   );
TYPE AppCtxTabTyp IS TABLE OF AppCtxRecTyp INDEX BY BINARY_INTEGER;
DBMS_SESSION.LIST_CONTEXT(
   list OUT AppCtxTabTyp,
   size OUT NUMBER
   );
```

参数

表 3.20.4: LIST_CONTEXT 参数

参数名称	描述
list	输出参数，类型为索引表。
size	输出参数，该索引表中元素的总数。

示例

```
\set SQLTERM /
DECLARE
lcontext DBMS_SESSION.APPCTXTABTYP;
lsize NUMBER;
BEGIN
DBMS_SESSION.LIST_CONTEXT (lcontext, lsize);
FOR k IN 1 .. (lcontext.COUNT)
LOOP
DBMS_OUTPUT.PUT_LINE('Namespace: ' || lcontext(k).namespace || ' Attribute: ' || lcontext(k).
attribute || ' Value: ' || lcontext(k).value);
END LOOP;
DBMS_OUTPUT.PUT_LINE('num: '|| lsize);
END;
/
\set SQLTERM ;
```

3.20.4.5　SET_CONTEXT 过程

设置上下文 namespace 的属性和值。此过程必须由受信任包直接或间接调用。

语法

```
DBMS_SESSION.SET_CONTEXT (
   namespace VARCHAR（63）,
   attribute VARCHAR（128）,
   value VARCHAR（4000）,
   username VARCHAR（128）,
   client_id VARCHAR（64）
);
```

参数

表 3.20.5: SET_CONTEXT 参数

参数名称	描述
namespace	要设置的 context 名。
attribute	要设置的 context 属性名。
value	要设置的 context 属性值。
username	不起作用，如果指定，则忽略。
client_id	不起作用，如果指定，则忽略。

备注:

attribute 不可为空，value 可以为空。设置 set ora_input_emptystr_isnull=off;

示例

```
第一步 创建 package
CREATE EXTENSION dbms_session;

set ora_input_emptystr_isnull=off;

create schema context_schema1;

grant usage on schema context_schema1 to public;

grant all on schema context_schema1 to public;

create or replace context context1 using context_schema1.test_package;
```

```
\set SQLTERM /

CREATE OR REPLACE PACKAGE context_schema1.test_package as

procedure set_context(ts_name varchar, key varchar, value varchar);

procedure clear_context(ts_name varchar,client_identifier varchar, key varchar);

procedure clear_all_context(ts_name varchar);

end test_package;

/

CREATE or replace PACKAGE BODY context_schema1.test_package as

procedure set_context(ts_name varchar, key varchar, value varchar) as

begin

dbms_session.set_context(ts_name, key,value);

end;

procedure clear_context(ts_name varchar,client_identifier varchar, key varchar) as

begin

dbms_session.clear_context(ts_name,client_identifier, key);

end;

procedure clear_all_context(ts_name varchar) as

begin

dbms_session.clear_all_context(ts_name);

end;

end test_package;

/

\set SQLTERM ;
```

```
第二步 创建 context
create or replace context c_user01 using context_schema1.test_package;

第三步 设置 namespace 的 key-value
call context_schema1.test_package.set_context('c_user01', 'u_k2', 'u_v2');

第四步 查询
select sys_context('c_user01', 'u_k2');
sys_context
----------------------------------------------------
u_v2

第五步 修改 u_k2 的值并查询
call context_schema1.test_package.set_context('c_user01', 'u_k2', 'u_v2222');
select sys_context('c_user01', 'u_k2');
sys_context
----------------------------------------------------
u_v2222

第六步 增加新的属性值 u_k3 并查询
call context_schema1.test_package.set_context('c_user01', 'u_k3', 'u_v3');
select sys_context('c_user01', 'u_k3');
sys_context
---------------------------------------------------
u_v3

第七步 清除上下文 c_user01 属性 u_k2 的值并再次查询
call context_schema1.test_package.clear_context('c_user01', null,'u_k2');
select sys_context('c_user01', 'u_k2');
 sys_context
-------------

(1 行记录)
select sys_context('c_user01', 'u_k3');
 sys_context
-------------
u_v3
(1 行记录)

第八步 清除上下文 c_user01
call context_schema1.test_package.clear_all_context('c_user01');
select sys_context('c_user01', 'u_k3');
sys_context
-------------
```

```
(1 行记录)
```

3.20.5　插件 dbms_session 卸载方法

在数据库中执行 SQL 命令卸载插件。

```
drop extension dbms_session;
```

3.20.6　插件 dbms_session 升级方法

dbms_session 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.21　dbms_sql

3.21.1　插件 dbms_sql 简介

dbms_sql 是 KingbaseES 的 plsql 扩展插件提供的系统包。

DBMS_SQL 包内定义了一系列的过程和函数，专门用于动态 SQL 语句的操作。

- 插件名为 plsql
- 插件版本 V1.0

3.21.2　插件 dbms_sql 加载方式

plsql 插件为初始化数据库实例时默认创建，因此 dbms_sql 系统包为系统内置包。

3.21.3　插件 dbms_sql 的参数配置

无需配置任何参数。

3.21.4　插件 dbms_sql 使用方法

3.21.4.1　DBMS_SQL 包预定义类型

3.21.4.1.1　DBMS_SQL 常量

Name	Type	Value	Description
NATIVE	INTEGER	1	兼容性常量，仅语法兼容
V6	INTEGER	0	兼容性常量，仅语法兼容
V7	INTEGER	2	兼容性常量，仅语法兼容
FOREIGN_SYNTAX	INTEGER	-1	兼容性常量，仅语法兼容

3.21.4.1.2　DBMS_SQL 集合类型

```
TYPE bfile_table IS TABLE OF BFILE INDEX BY BINARY_INTEGER;

TYPE desc_tab IS TABLE OF desc_rec INDEX BY BINARY_INTEGER;

TYPE binary_double_table IS TABLE OF BINARY_DOUBLE INDEX BY BINARY_INTEGER;

TYPE binary_float_table IS TABLE OF BINARY_FLOAT INDEX BY BINARY_INTEGER;

TYPE blob_table IS TABLE OF BLOB INDEX BY BINARY_INTEGER;

TYPE clob_table IS TABLE OF CLOB INDEX BY BINARY_INTEGER;

TYPE date_table IS TABLE OF DATE INDEX BY BINARY_INTEGER;

TYPE number_table IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;

TYPE time_table IS TABLE OF TIME_UNCONSTRAINED INDEX BY BINARY_INTEGER;

TYPE time_with_time_zone_table IS TABLE OF TIME_TZ_UNCONSTRAINED INDEX BY BINARY_INTEGER;

TYPE timestamp_table IS TABLE OF TIMESTAMP_UNCONSTRAINED INDEX BY BINARY_INTEGER;

TYPE timestamp_with_ltz_table IS TABLE OF TIMESTAMP_LTZ_UNCONSTRAINED INDEX BY BINARY_INTEGER;
```

```
TYPE timestamp_with_time_zone_table IS TABLE OF TIMESTAMP_TZ_UNCONSTRAINED INDEX BY BINARY_INTEGER;

TYPE varchar2_table IS TABLE OF VARCHAR2(2000) INDEX BY BINARY_INTEGER;

TYPE varchar2a IS TABLE OF VARCHAR2(32767) INDEX BY BINARY_INTEGER;

TYPE varchar2s IS TABLE OF VARCHAR2(256) INDEX BY BINARY_INTEGER;
```

3.21.4.2　DBMS_SQL 包子程序

3.21.4.2.1　打开新游标

语法格式

```
DBMS_SQL.OPEN_CURSOR (
    treat_as_client    BOOL  DEFAULT FALSE)
RETURN INTEGER;
```

功能描述

打开一个新游标，返回游标的 ID。

参数说明

treat_as_client 如果为 TRUE，则允许过程将自己设置为客户端；如果为 FALSE，将返回结果返回给客户端应用程序。该语句的执行结果可以通过 GET_NEXT_RESULT 方法获取。

返回值

返回新游标的游标 ID 号。

使用说明

当不再需要此游标时，必须通过调用 close_cursor 过程来显式关闭它。可以使用游标来重复运行相同的 SQL 语句。当重用游标时，在解析新的 SQL 语句时重用相应的游标数据区域的内容。在重用游标之前，没有必要关闭和重新打开它。

示例

```
\set SQLTERM /
DECLARE
    c integer;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.close_cursor(c);
END;
/
```

```
------------------
ANONYMOUS BLOCK
```

3.21.4.2.2　解析语句

语法格式

```
DBMS_SQL.PARSE (
handle              INTEGER,
query               TEXT,
flag                INTEGER);

DBMS_SQL.PARSE (
handle              INTEGER,
query               CLOB,
flag                INTEGER);

DBMS_SQL.PARSE (
handle              INTEGER,
query               VARCHAR2A,
lb INTEGER,
ub INTEGER,
lfflag BOOLEAN,
flag INTEGER);

DBMS_SQL.PARSE (
handle              INTEGER,
query               VARCHAR2S,
lb INTEGER,
ub INTEGER,
lfflag BOOLEAN,
flag INTEGER);
```

功能描述

对游标中的语句进行解析。DDL 语句在解析的同时立即执行。

参数说明

参数	描述
handle	解析语句的游标 ID
query	要进行解析的 SQL 语句
flag	指定 SQL 语句的行为

使用说明

使用 DBMS_SQL 动态运行 DDL 语句可能会导致程序停止响应。例如，对包中的过程的调用会导致包被锁定，直到执行返回到用户端为止。任何导致锁冲突的操作（例如在第一个锁被释放之前动态的尝试删除包）都会停止程序的运行。

使用不同 flag 参数，则存在不同的数据库行为。

query 的类型可以是 TEXT,CLOB,VARCHAR2A,VARCHAR2S(参考 DBMS_SQL 预定义集合类型)

示例

parse 接口有多个重载方式，如可传入 text 或集合，示例如下:

```
\set SQLTERM /
DECLARE
    c integer;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'create table if not exists dbmssql_table1(i int, J TEXT)', dbms_sql.native);
    dbms_sql.close_cursor(c);
END;
/
select count(*) from sys_class where relname='dbmssql_table1';
/

--------------------
ANONYMOUS BLOCK
 count
-------
    1
(1 row)

\set SQLTERM /
DECLARE
    c integer;
    sql_string dbms_sql.varchar2s;
BEGIN
    c := dbms_sql.open_cursor();
    sql_string(1) := 'create table if not exists ';
    sql_string(2) := 'dbmssql_table2(i int, J TEXT)';
    dbms_sql.parse(c, sql_string, 1 ,2, true, dbms_sql.native);
    dbms_sql.close_cursor(c);
END;
/
select count(*) from sys_class where relname='dbmssql_table2';
/
```

```
--------------------
ANONYMOUS BLOCK
 count
-------
    1
(1 row)
```

3.21.4.2.3　绑定变量

3.21.4.2.3.1　绑定到游标的基本数据类型

语法格式

```
DBMS_SQL.BIND_VARIABLE (
    handle     INTEGER,
    pos        INTEGER,
    val        ANYELEMENT
    [,out_value_size IN INTEGER]);

DBMS_SQL.BIND_VARIABLE (
    handle     INTEGER,
    placeholder   TEXT,
    val        ANYELEMENT
    [,out_value_size IN INTEGER]);
```

功能描述

将一个值或一个集合与游标定义中的占位符绑定，可以通过占位符的位置绑定，也可以通过占位符的名称绑定。

参数说明

参数	描述
handle	游标 ID
pos	绑定变量的位置

-续表-

参数	描述
val	与游标中的变量绑定的值或本地变量。 <datatype> ADT (user-defined object types) BINARY_DOUBLE BINARY_FLOAT BFILE BLOB BOOLEAN CLOB DATE DSINTERVAL_UNCONSTRAINED NESTED table NUMBER OPAQUE types REF TIME_UNCONSTRAINED TIME_TZ_UNCONSTRAINED TIMESTAMP_LTZ_UNCONSTRAINED TIMESTAMP_TZ_UNCONSTRAINED TIMESTAMP_UNCONSTRAINED UROWID VARCHAR2 VARRAY YMINTERVAL_UNCONSTRAINED
out_value_size	如果没有给定大小，则使用当前值的长度。只有当 val 类型为 char，varchar，text 时能指定该参数。

使用说明

如果变量是 IN 或 IN/OUT 变量或 IN 集合，则给定的绑定值必须对该变量或数组类型有效。OUT 变量的值将被忽略。

SQL 语句的绑定变量或集合由它们的名称标识。当将值绑定到变量或绑定数组时，语句中标识它的字符串必须包含一个前导冒号，如下所示

```
select emp_name from emp where sal > :x;
```

对于本例，对应的 bind 调用类似于

```
bind_variable(cursor_name, ':1', 3500);
or
bind_variable(cursor_name, 1, 3500);
```

示例

通过参数位置和参数名称两种绑定方式，来对一个 DML 语句做动态 sql 操作，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1(i int, J TEXT);
/
insert into dbmssql_table1 values (1, 'bbbb');
/
DECLARE
    c integer;
    r int;
    x int := 1;
    y varchar2(100) := 'bbbb';
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'delete from dbmssql_table1 where i = :xx and j = :yy', dbms_sql.native);
    --通过参数位置绑定
    dbms_sql.bind_variable(c, 1, x);
    dbms_sql.bind_variable(c, 2, y);
    r := dbms_sql.execute(c);
    dbms_sql.close_cursor(c);
    raise notice 'd%', c;
END;
/
----------------------------
NOTICE:  d<NULL>
ANONYMOUS BLOCK

\set SQLTERM /
create table if not exists dbmssql_table1(i int, J TEXT);
/
insert into dbmssql_table1 values (1, 'bbbb');
/
DECLARE
    c integer;
    r int;
    x int := 1;
    y varchar2(100) := 'bbbb';
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'delete from dbmssql_table1 where i = :xx and j = :yy', dbms_sql.native);
```

```
    --通过参数名称绑定（可以加':'，也可以不加）
    dbms_sql.bind_variable(c, 'xx', x);
    dbms_sql.bind_variable(c, ':yy', y);
    r := dbms_sql.execute(c);
    dbms_sql.close_cursor(c);
    raise notice 'd%', c;
END;
/
----------------------------
NOTICE:  d<NULL>
ANONYMOUS BLOCK
```

3.21.4.2.3.2　绑定到游标的 CHAR 类型

语法格式

```
DBMS_SQL.BIND_VARIABLE_CHAR (
    handle     INTEGER,
    pos        INTEGER,
    val        ANYELEMENT,
    out_value_size IN INTEGER);

DBMS_SQL.BIND_VARIABLE_CHAR (
    handle     INTEGER,
    placeholder   TEXT,
    val        ANYELEMENT,
    out_value_size IN INTEGER);
```

功能描述

将一个值或一个集合与游标定义中的占位符绑定，可以通过占位符的位置绑定，也可以通过占位符的名称绑定。

参数说明

参数	描述
handle	游标 ID
pos	绑定变量的位置

-续表-

参数	描述
val	与游标中的变量绑定的值或本地变量。 <datatype> ADT (user-defined object types) BINARY_DOUBLE BINARY_FLOAT BFILE BLOB BOOLEAN CLOB DATE DSINTERVAL_UNCONSTRAINED NESTED table NUMBER OPAQUE types REF TIME_UNCONSTRAINED TIME_TZ_UNCONSTRAINED TIMESTAMP_LTZ_UNCONSTRAINED TIMESTAMP_TZ_UNCONSTRAINED TIMESTAMP_UNCONSTRAINED UROWID VARCHAR2 VARRAY YMINTERVAL_UNCONSTRAINED
out_value_size	如果没有给定大小，则使用当前值的长度。只有当 val 类型为 char，varchar，text 时能指定该参数。

使用说明

该函数只能绑定字符串类型的变量，且必须指定长度；如果指定长度大于实际长度，则取实际长度，否则取指定长度。

示例

调用 BIND_VARIABLE_CHAR 子程序，且指定字符串长度的示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1(i int, J TEXT);
```

```
/
insert into dbmssql_table1 values (1, 'bbbb');
/
DECLARE
    c integer;
    r int;
    x int := 1;
    y varchar2(100) := 'bbbbaaaa';
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'delete from dbmssql_table1 where i = :xx and j = :yy', dbms_sql.native);
    --通过参数位置绑定
    dbms_sql.bind_variable(c, 1, x);
    dbms_sql.bind_variable_char(c, 2, y, 4);
    r := dbms_sql.execute(c);
    raise notice 'r = % ', 1;
    dbms_sql.close_cursor(c);
    raise notice 'd%', c;
END;
/
----------------------------
NOTICE:  r = 1
NOTICE:  d<NULL>
ANONYMOUS BLOCK

\set SQLTERM /
create table if not exists dbmssql_table1(i int, J TEXT);
/
insert into dbmssql_table1 values (1, 'bbbb');
/
DECLARE
    c integer;
    r int;
    x int := 1;
    y varchar2(100) := 'bbbbaaaa';
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'delete from dbmssql_table1 where i = :xx and j = :yy', dbms_sql.native);
    --通过参数名称绑定（可以加':'，也可以不加）
    dbms_sql.bind_variable(c, 'xx', x);
    dbms_sql.bind_variable_char(c, ':yy', y, 4);
    r := dbms_sql.execute(c);
    dbms_sql.close_cursor(c);
    raise notice 'd%', c;
END;
```

```
/
----------------------------
NOTICE:  d<NULL>
ANONYMOUS BLOCK
```

3.21.4.2.3.3　绑定到游标的关联数组类型

语法格式

```
DBMS_SQL.BIND_ARRAY (
    handle     INTEGER,
    pos        INTEGER,
    val        ANYELEMENT);

DBMS_SQL.BIND_ARRAY (
    handle     INTEGER,
    placeholder   TEXT,
    val        ANYELEMENT);
```

功能描述

将 dbms_sql 预定义的关联数组与游标定义中的占位符绑定，可以通过占位符的位置绑定，也可以通过占位符的名称绑定。

dbms_sql 将拆分出关联数组中的元素，以所有元素的值作为参数批量执行 dml 语句。

参数说明

参数	描述
handle	游标 ID
pos	绑定变量的位置

-续表-

参数	描述
val	与游标中的变量绑定的值或本地变量。 datatype clob_table binary_float_table binary_double_table blob_table bfile_table date_table number_table varchar2_table time_table time_with_time_zone_table timestamp_table timestamp_with_ltz_table timestamp_with_time_zone_table
index1	集合元素索引值的下限
index2	集合元素索引值的上限

使用说明

为了绑定范围，表必须包含指定范围的元素——tab(index1) 和 tab(index2)，但是范围不必是密集的。Index1 必须小于等于 index2。在绑定中使用 tab(index1) 和 tab(index2) 之间的所有元素。

如果没有在 bind 调用中指定索引，并且语句中的两个不同绑定指定包含不同数量元素的表，然后实际使用的元素数量是所有表之间的最小数量。如果指定索引，也会出现这种情况——为所有表选择两个索引之间的最小范围。

不是查询中的所有绑定变量都必须是数组绑定。有些可以是常规绑定，在表达式求值中对集合的每个元素使用相同的值。

批量 select, insert, update 和 delete 可以通过将多个调用绑定到一个调用来提高应用程序的性能。此过程允许使用 DBMS_SQL 包预定义的 PL/SQL TABLE 类型。

示例

使用 bind_array 接口来做批量 insert 操作，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1(i int, J TEXT);
/
```

```
DECLARE
    c integer;
    r int;
    x dbms_sql.number_table;
    y dbms_sql.varchar2_table;
BEGIN
    c := dbms_sql.open_cursor();
    x(1) := 1;
    x(2) := 2;
    x(3) := 3;
    x(4) := 4;
    y(1) := 'aaaa';
    y(2) := 'bbbb';
    y(3) := 'cccc';
    y(4) := 'dddd';
    dbms_sql.parse(c, 'insert into dbmssql_table1 values (:a, :b);', dbms_sql.native);
    dbms_sql.bind_array(c, 'a', x);
    dbms_sql.bind_array(c, 'b', y);
    r := dbms_sql.execute(c);
    dbms_sql.close_cursor(c);
    raise notice 'd%', c;
END;
/
----------------------------
NOTICE:  d<NULL>
ANONYMOUS BLOCK

select * from dbmssql_table1;
/
----------------------------
 i |  j
---+------
 1 | aaaa
 2 | bbbb
 3 | cccc
 4 | dddd
(4 rows)
```

3.21.4.2.3.4　绑定到游标的包中类型

语法格式

```
DBMS_SQL.BIND_VARIABLE_PKG (
    handle     INTEGER,
    pos        INTEGER,
```

```
    val         ANYELEMENT);

DBMS_SQL.BIND_VARIABLE_PKG (
    handle      INTEGER,
    placeholder   TEXT,
    val         ANYELEMENT);
```

功能描述

同 BIND_VARIABLE，BIND_VARIABLE 和 BIND_VARIABLE_PKG 都可以用来绑定包中或者非包中定义的类型。

3.21.4.2.4　定义类型接收列值

3.21.4.2.4.1　定义基本数据类型接收单行列值

语法格式

```
DBMS_SQL.DEFINE_COLUMN (
    handle      IN INTEGER,
    pos         IN INTEGER,
    val         ANYELEMENT
    [,column_size    IN INTEGER]);
```

功能描述

为 select 游标定义被选择的列，定义后可通过 column_value 取出对应列的值。

参数说明

参数	描述
handle	游标 ID
pos	列在定义行中的相对位置。语句在第一列位置为 1。

-续表-

参数	描述
val	定义的列的值。此值的类型决定了所定义列的类型。 <datatype> BINARY_DOUBLE BINARY_FLOAT BFILE BLOB CLOB DATE DSINTERVAL_UNCONSTRAINED NUMBER TIME_UNCONSTRAINED TIME_TZ_UNCONSTRAINED TIMESTAMP_LTZ_UNCONSTRAINED TIMESTAMP_TZ_UNCONSTRAINED TIMESTAMP_UNCONSTRAINED UROWID YMINTERVAL_UNCONSTRAINED ADT(user-defined object types) COLLECTIONS (varrays and nested tables) REFS OPAQUE TYPES
column_size	类型为 CHAR,VARCHAR2,TEXT 的列的预期列值的最大字节大小

示例

define_column 接口在 parse 接口后调用，为 select 游标定义被选择的列，如下所示:

```
\set SQLTERM ;
drop table t2;
create table t2(id int,name int);
insert into t2 values (1,2);

\set SQLTERM /
DECLARE
        c integer;
        n int;
        result int;
        dc int;
```

```
BEGIN
        c := dbms_sql.open_cursor();
        dbms_sql.parse(c, 'select * from t2', dbms_sql.native);
        dbms_sql.define_column(c, 1, n);
        dbms_sql.define_column(c, 2, result);
        dbms_sql.close_cursor(c);
END;
/
----------------------------
ANONYMOUS BLOCK

\set SQLTERM ;
drop table t2;
create table t2(id int,name text);
insert into t2 values (1,'zs');

--define_column 接口可以定义预期列值的最大字节大小
\set SQLTERM /
DECLARE
        c integer;
        n int;
        result text;
        dc int;
BEGIN
        c := dbms_sql.open_cursor();
        dbms_sql.parse(c, 'select * from t2', dbms_sql.native);
        dbms_sql.define_column(c, 1, n);
        dbms_sql.define_column(c, 2, result, 2);
        dbms_sql.close_cursor(c);
END;
/
----------------------------
ANONYMOUS BLOCK
```

3.21.4.2.4.2　定义 CHAR 类型接收单行列值

语法格式

```
DBMS_SQL.DEFINE_COLUMN (
    handle     IN INTEGER,
    pos        IN INTEGER,
    val        ANYELEMENT,
    column_size    IN INTEGER);
```

功能描述

为 select 游标定义被选择的列，定义后可通过 column_value 取出对应列的值，其中列类型必须是 CHAR,VARCHAR,TEXT。

参数说明

参数	描述
handle	游标 ID
pos	列在定义行中的相对位置。语句在第一列位置为 1。
val	定义的列的值。此值的类型决定了所定义列的类型。 <datatype> CHAR VARCHAR TEXT
column_size	类型为 CHAR,VARCHAR2,TEXT 的列的预期列值的最大字节大小

示例

define_column_char 可以指定预期列值的最大字节大小，如下所示:

```
\set SQLTERM ;
drop table t2;
create table t2(id int,name text);
insert into t2 values (1,'zs');

\set SQLTERM /
DECLARE
        c integer;
        n int;
        result varchar2(10);
        dc int;
BEGIN
        c := dbms_sql.open_cursor();
        dbms_sql.parse(c, 'select * from t2', dbms_sql.native);
        dbms_sql.define_column(c, 1, n);
        dbms_sql.define_column_char(c, 2, result, 2);
        dbms_sql.close_cursor(c);
END;
/
----------------------------
ANONYMOUS BLOCK
```

3.21.4.2.4.3　定义关联数组类型接收单行列值

语法格式

```
DBMS_SQL.DEFINE_ARRAY (
   handle      IN INTEGER,
   pos           IN INTEGER,
   val           IN ANYELEMENT,
   cnt         IN INTEGER,
   lower_bnd   IN INTEGER);
```

功能描述

定义一个集合类型接收某一列的值。这个过程允许您从单个 SELECT 语句批量提取行。FETCH_ROWS 调用后，会将许多行存入 PL/SQL 聚合对象中。当你获取这些行时，它们将被赋值到 DBMS_SQL 缓存区中，直到运行一个 COLUMN_VALUE 调用，这时这些行将被复制到作为参数传递给 COLUMN_VALUE 调用的表中。

参数说明

参数	描述
handle	游标 ID
pos	列在定义行中的相对位置。语句在第一列位置为 1。
val	定义的列的值。此值的类型决定了所定义列的类型。 datatype clob_table binary_float_table binary_double_table blob_table bfile_table date_table number_table varchar2_table time_table time_with_time_zone_table timestamp_table timestamp_with_ltz_table timestamp_with_time_zone_table
cnt	必须提取的行数
lower_bnd	从下限索引开始，将结果复制到集合中

示例

define_array 接口定义关联数组类型接收单行列值，如下所示:

```
\set SQLTERM ;
drop table t2;
create table t2(id int,name text);
insert into t2 values (1,'zs');

\set SQLTERM /
DECLARE
    c integer;
    n dbms_sql.number_table;
    result dbms_sql.varchar2_table;
    dc int;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'select * from t2', dbms_sql.native);
    dbms_sql.define_array(c, 1, n, 1, 1);
    dbms_sql.define_array(c, 2, result, 1, 1);
    dbms_sql.close_cursor(c);
END;
/
----------------------------
ANONYMOUS BLOCK
```

3.21.4.2.5　执行游标

语法格式

```
DBMS_SQL.EXECUTE (
    handle INTEGER)
    RETURN INTEGER;
```

功能描述

执行给定的游标，返回已处理的行数（仅对 INSERT、UPDATE 和 DELETE 语句，其他类型的语句返回值是不确定的）。

参数说明

参数	描述
handle	游标 ID

返回值

返回已处理的行数。

使用说明

TO_CURSOR_NUMBER 函数返回的 DBMS_SQL 游标的执行方式与已经执行的 DBMS_SQL 游标相同。

示例：

```
\set SQLTERM /
declare
    c NUMBER;
    r NUMBER;
BEGIN
    c := DBMS_SQL.OPEN_CURSOR();
    DBMS_SQL.PARSE(c, 'create table if not exists tx (i int)', DBMS_SQL.NATIVE);
    r := DBMS_SQL.EXECUTE(c);
    DBMS_SQL.close_cursor(c);
END;
/
```

3.21.4.2.6　检索游标

语法格式

```
DBMS_SQL.FETCH_ROWS (
    handle        INTEGER)
    RETURN INTEGER;
```

功能描述

从给定游标中获取数据，并且返回实际获取的行数。只要还有行需要提取，就可以重复调用 FETCH_ROWS。这些行被检索到缓冲区中，如果需要读取数据则需要调用 COLUMN_VALUE 函数来读取。不能采用 NO_DATA_FOUND 或游标属性%NOTFOUND 判断是否检索到数据。

参数说明

参数	描述
handle	游标 ID

返回值

返回实际获取的行数。

使用说明

如果该游标不是关联 select 语句，调用该方法则报错“fetch out of sequence”。

示例

FETCH_ROWS 接口用于检索游标，如下所示:

```
set serverout on
\set SQLTERM /
drop table t1;
/
create table t1(i int);
/
insert  into t1 values(1),(2),(3);
/
declare
        c NUMBER;
        d NUMBER;
begin
        c := DBMS_SQL.OPEN_CURSOR;
        DBMS_SQL.PARSE(c, 'SELECT * FROM t1', DBMS_SQL.NATIVE);
        d := DBMS_SQL.EXECUTE_AND_FETCH(c);
        dbms_output.put_line('d:'||d);
        d := DBMS_SQL.FETCH_ROWS(c);
        dbms_output.put_line('d:'||d);
        DBMS_SQL.CLOSE_CURSOR(c);
end;
/
------------------------
ANONYMOUS BLOCK
d:1
d:1
```

3.21.4.2.7　执行游标，并检索游标

语法格式

```
DBMS_SQL.EXECUTE_AND_FETCH (
    handle          INTEGER,
    exact           BOOL DEFAULT FALSE)
    RETURN INTEGER;
```

功能描述

执行游标，检索数据，返回实际检索行的数量。

参数说明

参数	描述
handle	游标 ID
exact	设置为 TRUE，如果当查询行的数量与检索行的数量不同，则抛出异常。

返回值

返回指定的行数。

示例

参照 FETCH_ROWS 示例。

3.21.4.2.8　返回游标中指定类型列值

3.21.4.2.8.1　返回游标基础数据类型的列值

语法格式

```
DBMS_SQL.COLUMN_VALUE (
    handle          INTEGER,
    pos             INTEGER,
    val             IN OUT ANYELEMENT );
```

功能描述

用于访问给定游标，给定位置，指定为基本数据类型的列值。此过程用于获取 fetch_rows 调用后的数据。

参数说明

参数	描述
handle	游标 ID
pos	列在定义行中的相对位置。语句在第一列位置为 1。

-续表-

参数	描述
val	返回指定列处的值。 <datatype> BINARY_DOUBLE BINARY_FLOAT BFILE BLOB CLOB DATE DSINTERVAL_UNCONSTRAINED NUMBER TIME_TZ_UNCONSTRAINED TIME_UNCONSTRAINED TIMESTAMP_LTZ_UNCONSTRAINED TIMESTAMP_TZ_UNCONSTRAINED TIMESTAMP_UNCONSTRAINED UROWID VARCHAR2 YMINTERVAL_UNCONSTRAINED ADT(user-defined object types) COLLECTIONS (varrays and nested tables) REFS OPAQUE TYPES

示例

使用 dbms_sql 接口函数从 dbmssql_table1 表中获取结果集，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1 (i int, j TEXT);
/
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
/
DECLARE
    c integer;
    n int;
    i int;
    j text := 'who';
```

```
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'select i, j from dbmssql_table1', dbms_sql.native);
    dbms_sql.define_column(c, 1, i);
    dbms_sql.define_column(c, 2, j);
    n := dbms_sql.execute(c);
    LOOP
        exit when dbms_sql.fetch_rows(c) <= 0;
        dbms_sql.column_value(c, 1, i);
        dbms_sql.column_value(c, 2, j);
        raise notice 'i = %, j = %', i, j;
        raise notice 'last_row_count = %', dbms_sql.last_row_count();
    END LOOP;
    dbms_sql.close_cursor(c);
END;
/
---------------------------
NOTICE:  i = 1, j = foo
NOTICE:  last_row_count = 1
NOTICE:  i = 2, j = bar
NOTICE:  last_row_count = 2
ANONYMOUS BLOCK
```

3.21.4.2.8.2　返回游标关联数组类型的列值

语法格式

```
DBMS_SQL.COLUMN_VALUE (
    handle          INTEGER,
    pos             INTEGER,
    val             IN OUT ANYELEMENT );
```

功能描述

用于访问给定游标，给定位置，指定为关联数组类型的列值。此过程用于获取 fetch_rows 调用后的数据。

参数说明

参数	描述
handle	游标 ID
pos	列在定义行中的相对位置。语句在第一列位置为 1。

-续表-

参数	描述
val	返回指定列处的值。类型为 <table_type> clob_table binary_float_table binary_double_table blob_table bfile_table date_table number_table varchar2_table time_table time_with_time_zone_table timestamp_table timestamp_with_ltz_table timestamp_with_time_zone_table

示例

使用 dbms_sql 接口函数从 dbmssql_table1 表中获取结果集，并将结果集赋值到关联数组中，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1 (i int, j TEXT);
/
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
/
DECLARE
    c integer;
    n int;
    i dbms_sql.number_table;
    j dbms_sql.varchar2_table;
    r int;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'select i, j from dbmssql_table1', dbms_sql.native);

    dbms_sql.define_array(c, 1, i, 3, 1);
    dbms_sql.define_array(c, 2, j, 3, 1);
```

```
    n := dbms_sql.execute(c);
    n := dbms_sql.fetch_rows(c);

    dbms_sql.column_value(c, 1, i);
    dbms_sql.column_value(c, 2, j);

    for r in i.first..i.last LOOP
        raise notice 'i(%) = %, j(%) = %', r, i(r), r, j(r);
    END LOOP;
    dbms_sql.close_cursor(c);
END;
/
---------------------------
NOTICE:  i(1) = 1, j(1) = foo
NOTICE:  i(2) = 2, j(2) = bar
ANONYMOUS BLOCK
```

3.21.4.2.8.3　返回游标 CHAR 类型的列值

语法格式

```
DBMS_SQL.COLUMN_VALUE_CHAR (
    handle           INTEGER,
    pos              INTEGER,
    val              IN OUT ANYELEMENT );
```

功能描述

同 COLUMN_VALUE，COLUMN_VALUE 和 COLUMN_VALUE_CHAR 都可以用来返回 CHAR 类型或者非 CHAR 的类型的值。

3.21.4.2.9　返回游标中给定变量的值

3.21.4.2.9.1　返回游标基础数据类型的变量值

语法格式

```
DBMS_SQL.VARIABLE_VALUE (
    handle          IN   INTEGER,
    pos             IN   INTEGER,
    val             IN OUT ANYELEMENT);
```

功能描述

这个过程返回给定游标的命名变量的值，它用于返回 PL/SQL 块或带有 RETURNING 短语的 DML 语句中绑定变量的值。

参数说明

参数	描述
handle	游标 ID
pos	需要返回值的占位符的位置
val	返回指定位置的变量的值。 <datatype> BINARY_DOUBLE BINARY_FLOAT BFILE BLOB CLOB DATE DSINTERVAL_UNCONSTRAINED NUMBER TIME_TZ_UNCONSTRAINED TIME_UNCONSTRAINED TIMESTAMP_LTZ_UNCONSTRAINED TIMESTAMP_TZ_UNCONSTRAINED TIMESTAMP_UNCONSTRAINED UROWID VARCHAR2 YMINTERVAL_UNCONSTRAINED ADT(user-defined object types) COLLECTIONS (varrays and nested tables) REFS OPAQUE TYPES

示例

使用 variable_value 接口函数获取绑定变量的值，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1 (i int, j TEXT);
/
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
/
DECLARE
    c integer;
```

```
    r int;
    x text := 'bar';
    y varchar2;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'select * from dbmssql_table1 where j = :xx', dbms_sql.native);
    dbms_sql.bind_variable(c, 1, x);
    r := dbms_sql.execute(c);
    dbms_sql.variable_value(c, 1, y);
    raise notice 'y1 = %', y;
    dbms_sql.close_cursor(c);
END;
/
---------------------------
NOTICE:  y1 = bar
ANONYMOUS BLOCK
```

3.21.4.2.9.2　返回游标关联数组类型的变量值

语法格式

```
DBMS_SQL.VARIABLE_VALUE (
    handle          IN   INTEGER,
    pos             IN   INTEGER,
    val             IN OUT ANYELEMENT);
```

功能描述

这个过程返回给定游标的命名变量的值，它用于返回 PL/SQL 块或带有 RETURNING 短语的 DML 语句中绑定变量的值。

参数说明

参数	描述
handle	游标 ID
pos	需要返回值的占位符的位置

-续表-

参数	描述
val	数组选项返回指定位置的变量的值。 <table_type> clob_table binary_float_table binary_double_table blob_table bfile_table date_table number_table varchar2_table time_table time_with_time_zone_table timestamp_table timestamp_with_ltz_table timestamp_with_time_zone_table

示例

使用 variable_value 接口函数获取绑定变量的值，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1 (i int, j TEXT);
/
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
/
DECLARE
    c integer;
    r int;
    i int;
    x dbms_sql.varchar2_table;
    y dbms_sql.varchar2_table;
BEGIN
    c := dbms_sql.open_cursor();
    x(1) := 'foo';
    x(2) := 'bar';
    dbms_sql.parse(c, 'select * from dbmssql_table1 where j = :xx', dbms_sql.native);
    dbms_sql.bind_array(c, 1, x);
    r := dbms_sql.execute(c);
```

```
    dbms_sql.variable_value(c, 1, y);
    for i in y.first..y.last LOOP
        raise notice 'y(%) = %', i, y(i);
    END LOOP;
    dbms_sql.close_cursor(c);
END;
/
---------------------------
NOTICE:  y(1) = foo
NOTICE:  y(2) = bar
ANONYMOUS BLOCK
```

3.21.4.2.9.3　返回游标 CHAR 类型的变量值

语法格式

```
DBMS_SQL.VARIABLE_VALUE_CHAR (
    handle          IN   INTEGER,
    pos             IN   INTEGER,
    val             IN OUT ANYELEMENT);
```

功能描述

同 VARIABLE_VALUE，VARIABLE_VALUE 和 VARIABLE_VALUE_CHAR 都可以用来返回 CHAR 类型或者非 CHAR 的类型的值。

3.21.4.2.9.4　返回游标包中类型的变量值

语法格式

```
DBMS_SQL.VARIABLE_VALUE_PKG (
    handle          IN   INTEGER,
    pos             IN   INTEGER,
    val             IN OUT ANYELEMENT);
```

功能描述

同 VARIABLE_VALUE，VARIABLE_VALUE 和 VARIABLE_VALUE_PKG 都可以用来返回包中类型或者非包中类型的值。

3.21.4.2.10　获取查询项的描述信息

3.21.4.2.10.1　获取查询项的描述信息，为 DESC_TAB 类型

语法格式

```
DBMS_SQL.DESCRIBE_COLUMNS (
    handle         INTEGER,
    col_cnt        OUT INTEGER,
    desc_t         OUT DBMS_SQL.DESC_TAB);
```

功能描述

这个过程返回描述查询列表的所有列信息，需要经过调用 DBMS_SQL 包中 OPEN_CURSOR 过程和 PARSE 过程。

参数说明

参数	描述
handle	被描述列的游标 ID 号
col_cnt	select 语句中列表中的列数
desc_t	返回描述表，表中记录了查询列表中每个列的描述

示例

对 select 语句做 PARSE 操作后，可以使用 DESCRIBE_COLUMNS 接口函数获取相关表的表结构信息，示例如下:

```
set serverout on
\set SQLTERM ;
drop table t1;
create table t1(id int,name varchar(50));
\set SQLTERM /
CREATE OR REPLACE PROCEDURE pr1 IS
        v3 DBMS_SQL.DESC_TAB;
        v4 int;
        col_num int;
        cursor_id int;
BEGIN
        cursor_id :=DBMS_SQL.OPEN_CURSOR;
        DBMS_SQL.PARSE(cursor_id, 'SELECT * FROM t1', DBMS_SQL.NATIVE);
        DBMS_SQL.DESCRIBE_COLUMNS(cursor_id,v4,v3);
        col_num := v3.first;
        dbms_output.put_line('col_num:'||col_num);
        WHILE col_num IS NOT NULL LOOP
        dbms_output.put_line(v3(col_num).col_max_len);
        col_num := v3.next(col_num);
        dbms_output.put_line('col_num:'||col_num);
```

```
        END LOOP;
        DBMS_SQL.CLOSE_CURSOR(cursor_id);
END;
/
call pr1();
/
-------------------------------
CALL
col_num:1
4
col_num:2
50
col_num:
```

3.21.4.2.11　关闭游标

语法格式

```
DBMS_SQL.CLOSE_CURSOR (
    handle    IN OUT INTEGER);
```

功能描述

用于关闭游标 ID，并释放占用的内存空间。

参数说明

参数	模式	描述
handle	IN	游标 ID
handle	OUT	游标被设置为 NULL。在调用 CLOSE_CURSOR 后，分配给游标的内存被释放，您不能再从该游标中获取数据。

示例

参照 OPEN_CURSOR 示例。

3.21.4.2.12　判断游标是否打开

语法格式

```
DBMS_SQL.IS_OPEN (
    c              IN INTEGER)
    RETURN BOOLEAN;
```

功能描述

检查指定的游标是否已经打开。

参数说明

参数	模式	描述
c	IN	游标 ID

返回值说明

对于已打开但未关闭的任何游标编号返回 TRUE，对于 NULL 游标编号返回 FALSE，请注意，CLOSE_CURSOR 过程会将传递给它的游标变量设为 NULL。

示例

is_open 接口用于判断游标是否打开，如下所示:

```
\set SQLTERM ;
create table if not exists dbmssql_table1(i int, J TEXT);
\set SQLTERM /
DECLARE
        c integer;
        r int;
BEGIN
        c := dbms_sql.open_cursor();
        dbms_sql.parse(c, 'select i, j from dbmssql_table1', dbms_sql.native);

        if dbms_sql.is_open(c) then
                dbms_sql.close_cursor(c);
                raise notice 'A opened, now close it';
        end if;
END;
/
-----------------------------------
NOTICE:  A opened, now close it
ANONYMOUS BLOCK
```

3.21.4.2.13　返回累计检索出来的行的数量

语法格式

```
DBMS_SQL.LAST_ROW_COUNT
    RETURN INTEGER;
```

功能描述

这个函数返回所获取的行数的累计计数。

返回值说明

返回所获取的行数的累计计数。

使用说明

在 FETCH_ROWS 或 EXECUTE_AND_FETCH 调用之后调用这个函数。如果在 EXECUTE 调用之后马上调用此函数，则返回的值为 0。

示例

通过 last_row_count 接口可以获取当前处理的行数，示例如下:

```
set serverout on
\set SQLTERM ;
drop table t2;
create table t2(id int,name int);
\set SQLTERM /
DECLARE
    c integer;
    r int;
    a int := 10;
    b int := 20;
BEGIN
    c := dbms_sql.open_cursor();
    dbms_sql.parse(c, 'insert into t2 values (:1, :2)', dbms_sql.native);
    dbms_sql.bind_variable(c, '1', a);
    dbms_sql.bind_variable(c, '2', b);
    r := dbms_sql.execute(c);
    dbms_output.put_line( 'r:' || r);
    dbms_output.put_line( dbms_sql.last_row_count);
    dbms_sql.close_cursor(c);
END;
/

----------------------------
ANONYMOUS BLOCK
r:1
1
```

3.21.4.2.14　游标转换

3.21.4.2.14.1　将游标变量转换为 DBMS_SQL 包内部游标

语法格式

```
DBMS_SQL.TO_CURSOR_NUMBER(
    c INOUT refcursor)
    RETURN INTEGER
```

功能描述

该函数接受一个打开的强类型或弱类型 ref 游标，并将其转换为 DBMS_SQL 游标号。

参数说明

参数	描述
c	游标变量

返回值说明

返回从 REF 游标转换而来的 DBMS_SQL 中可管理的游标 ID。

使用说明

1) 必须打开传入的 REF 游标，否则会引发错误。
2) 在将 REF 游标转换为 DBMS_SQL 游标号之后，任何本地动态 SQL 操作不再可以访问 REF 游标。
3) 这个子程序返回的 DBMS_SQL 游标的执行方式与已经执行的 DBMS_SQL 游标相同。

示例

游标变量转换为 DBMS_SQL 包内部游标，示例如下:

```
\set SQLTERM ;
create table if not exists dbmssql_table1 (i int, j TEXT);
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
\set SQLTERM /
DECLARE
    v_cur int;
    i int;
    j text;
    v_count int;
```

```
    cur refcursor;
BEGIN
    open cur for select i, j from dbmssql_table1;
    v_cur := dbms_sql.TO_CURSOR_NUMBER(cur);

    dbms_sql.define_column(v_cur, 1, i);
    dbms_sql.define_column(v_cur, 2, j);

    loop
        v_count := dbms_sql.fetch_rows(v_cur);
        exit when v_count <= 0;
        dbms_sql.column_value_char(v_cur, 1, i);
        dbms_sql.column_value(v_cur, 2, j);
        raise notice 'i = %, j = %', i, j;
    END LOOP;

    dbms_sql.close_cursor(v_cur);
END;
/

----------------------------
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
ANONYMOUS BLOCK
```

3.21.4.2.14.2　将 DBMS_SQL 包内的游标转换为本地动态 SQL 游标

语法格式

```
DBMS_SQL.TO_REFCURSOR(
    c INTEGER)
    RETURN refcursor
```

功能描述

将使用 DBMS_SQL 包打开、解析、执行过的游标 ID 转换为本地动态 SQL 可管理的 REF 游标（弱类型游标），可供本地动态 SQL 使用。此子程序只与 select 游标一起使用。

参数说明

参数	描述
c	游标 ID

返回值说明

返回从 DBMS_SQL 游标 ID 转换而来的 PL/SQL REF 游标。

使用说明

1) 通过 cursor_name 传入的游标必须被打开、解析和执行，否则会引发错误。
2) 将 cursor_name 转换为 REF 游标后，任何 DBMS_SQL 操作都不能访问 cursor_number。
3) 在使用 DBMS_SQL 将 cursor_name 转换为 REF 游标之后，IS_OPEN 检查 cursor_name 是否打开会导致错误。

示例

DBMS_SQL 包内的游标经过 execute 操作后，可使用 to_refcursor 转换为本地动态 SQL 游标，示例如下:

```
\set SQLTERM ;
create table if not exists dbmssql_table1 (i int, j TEXT);
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
\set SQLTERM /
DECLARE
    v_cur number;
    sql_string varchar2(1024);
    i int;
    j text;
    v_count int;
    cur refcursor;
    v_name varchar2(60);
BEGIN
    v_name := 'zs';
    v_cur := dbms_sql.open_cursor();
    sql_string := 'select i, j from dbmssql_table1';
    dbms_sql.parse(v_cur, sql_string, dbms_sql.native);

    v_count := dbms_sql.execute(v_cur);

    cur := dbms_sql.to_refcursor(v_cur);
    loop
        fetch cur into i, j;
        exit when cur%notfound;
        raise notice 'i = %, j = %', i, j;
    END LOOP;
    close cur;
END;
/

----------------------------
```

```
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
ANONYMOUS BLOCK
```

3.21.4.2.15　返回语句结果集

语法格式

```
DBMS_SQL.RETURN_RESULT(
    rc IN refcursor,
    to_client IN BOOLEAN DEFAULT TRUE)

DBMS_SQL.RETURN_RESULT(
    rc IN INTEGER,
    to_client IN BOOLEAN DEFAULT TRUE)
```

参数说明

参数	描述
rc	游标变量或游标 ID
to_client	预留参数，暂无意义

功能描述

DBMS_SQL.RETURN_RESULT 过程让子程序隐式地将查询结果返回给客户机程序（它间接调用子程序）或子程序的直接调用者。直接调用者执行递归语句，在递归语句中返回该语句结果。在 DBMS_SQL.RETURN_RESULT 调用后，该语句结果只有接收者可以访问它。

使用说明

1) 目前只能返回 SQL 查询，不支持通过远程过程调用返回语句结果。
2) 一旦使用该方法后，除了返回它的客户端或直接调用者外，就不能再访问它了。
3) 当客户端执行的语句或任何中间递归语句是 SQL 查询并引发错误时，不能返回语句结果。
4) 返回的 ref 游标可以是强类型的，也可以是弱类型的。
5) 返回的查询可以部分获取。
6) 要从 PL/SQL 中的递归语句检索返回的语句结果，可以使用 DBMS_SQL 执行递归语句。

示例

通过 return_result 接口将结果集返回到客户端，示例如下:

```
\set SQLTERM ;
create table if not exists dbmssql_table1 (i int, j TEXT);
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
\set SQLTERM /
DECLARE
    cur1 refcursor;
    cur2 int;
    i_ret int;
BEGIN
    OPEN cur1 for select * from dbmssql_table1;
    dbms_sql.return_result(cur1);

    cur2 := dbms_sql.open_cursor(TRUE);
    dbms_sql.parse(cur2, 'select * from dbmssql_table1;',dbms_sql.native);
    i_ret := dbms_sql.execute(cur2);
    dbms_sql.return_result(cur2);
END;
/

----------------------------
 i |  j
---+-----
 1 | foo
 2 | bar
(2 rows)

 i |  j
---+-----
 1 | foo
 2 | bar
(2 rows)

ANONYMOUS BLOCK
```

3.21.4.2.16　接收 RETURN_RESULT 过程的一个返回结果，并返回给客户端应用程序

语法格式

```
DBMS_SQL.GET_NEXT_RESULT(
    c IN INTEGER,
    rc OUT refcursor)
```

参数	描述
c	表示一个已经打开的游标 ID，该游标直接或间接调用一个子程序，而子程序调用 RETURN _RESULT 过程隐式返回一个查询结果。
rc	表示一个 SYS_REFCURSOR 类型的游标变量或者一个已经打开的游标 ID，接收一个 RETURN _RESULT 过程返回的结果。

功能描述

GET_NEXT_RESULT 过程获取 RETURN_RESULT 过程返回的一个查询结果，并返回给接收者。GET_NEXT_RESULT 与 RETURN_RESULT 返回结果的顺序相同。

使用说明

1) 在检索语句结果的游标之后，调用者必须在不再需要游标时正确关闭该游标。
2) 所有未检索返回语句的游标将在递归语句的游标关闭后关闭。
3) 要打开游标并获得它的游标 ID，请调用 DBMS_SQL.OPEN_CURSOR 方法,DBMS_SQL.OPEN_CURSOR 有一个可选的参数 treat_as_client。当此参数为 FALSE（默认参数）时，打开此游标（用来调用子程序）的调用方不会被视为客户端接受查询结果的子程序，相反，这些查询结果在较上层返回给客户机。如果 treat_as_client 为 TRUE，调用者将被视为客户端。

示例

使用游标变量和 dbms_sql 游标获取 RETURN_RESULT 过程返回的结果集，示例如下:

```
\set SQLTERM ;
create table if not exists dbmssql_table1 (i int, j TEXT);
insert into dbmssql_table1 values (1, 'foo');
insert into dbmssql_table1 values (2, 'bar');
\set SQLTERM /
--将游标的结果集返回给 get_results 的调用者
CREATE OR REPLACE PROCEDURE get_results(p_id IN NUMBER DEFAULT NULL) AS
    cur1 refcursor;
    cur2 refcursor;
BEGIN
    IF p_id IS NOT NULL THEN
        OPEN cur1 for select * from dbmssql_table1;
        dbms_sql.return_result(cur1);
    END IF;

    OPEN cur2 for select * from dbmssql_table1;
    dbms_sql.return_result(cur2);

END;
```

```
/
--使用游标变量获取 RETURN_RESULT 过程返回的结果集
DECLARE
    c integer;
    c1 integer;
    sqlstmt varchar2(1024);
    rc refcursor;
    i_ret int;
    i int;
    j text;
BEGIN
    c := dbms_sql.open_cursor(TRUE);
    sqlstmt := 'begin get_results(1);end;';
    dbms_sql.parse(c, sqlstmt,dbms_sql.native);
    i_ret := dbms_sql.execute(c);
    loop
        BEGIN
            --使用 rc 接收 c 的结果集
            dbms_sql.get_next_result(c,rc);
        exception
            when no_data_found then
                exit;
        END;
        loop
            fetch rc into i, j;
            exit when rc%NOTFOUND;
            raise notice 'i = %, j = %', i, j;
        end loop;

    end loop;
    dbms_sql.close_cursor(c);
END;
/

----------------------------
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
ANONYMOUS BLOCK

--使用 dbms_sql 游标获取 RETURN_RESULT 过程返回的结果集
\set SQLTERM /
DECLARE
    c integer;
```

```
    c1 integer;
    sqlstmt varchar2(1024);
    rc refcursor;
    i_ret int;
    i int;
    j text;
BEGIN
    c := dbms_sql.open_cursor(TRUE);
    sqlstmt := 'begin get_results(1);end;';
    dbms_sql.parse(c, sqlstmt,dbms_sql.native);
    i_ret := dbms_sql.execute(c);

    loop
        BEGIN
            dbms_sql.get_next_result(c,c1);
        exception
            when no_data_found then
                exit;
        END;
        dbms_sql.define_column(c1, 1, i);
        dbms_sql.define_column(c1, 2, j);

        LOOP
            exit when dbms_sql.fetch_rows(c1) <= 0;
            dbms_sql.column_value(c1, 1, i);
            dbms_sql.column_value(c1, 2, j);
            raise notice 'i = %, j = %', i, j;
        END LOOP;

    end loop;

    dbms_sql.close_cursor(c1);
END;
/

----------------------------
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
NOTICE:  i = 1, j = foo
NOTICE:  i = 2, j = bar
ANONYMOUS BLOCK
```

3.21.4.2.17　返回 SQL 语句错误发生处的字节偏移量

语法格式

```
DBMS_SQL.LAST_ERROR_POSITION()
    RETURN INTEGER
```

功能描述

这个函数返回发生错误的 SQL 语句文本中的字节偏移量。SQL 语句中的第一个字符位于位置 0。

返回值说明

返回发生错误的 SQL 语句文本中的字节偏移量。

使用说明

调用该函数在 PARSE 调用之后，在调用任何其他 DBMS_SQL 过程或函数之前。

示例

在异常块中使用 LAST_ERROR_POSITION 接口获取语句错误位置，示例如下:

```
\set SQLTERM ;
create table if not exists dbmssql_table1 (i int, j TEXT);
\set SQLTERM /
DECLARE
    v_cur int;
    sql_string varchar2(1024);
    v_count int;
    error_offset number;
    b int := 1;
BEGIN
    sql_string := 'select i, j, k from dbmssql_table1';
    v_cur := dbms_sql.open_cursor();
    dbms_sql.parse(v_cur, sql_string, dbms_sql.native);

    v_count := dbms_sql.execute(v_cur);

    v_count := dbms_sql.fetch_rows(v_cur);

    exception
        when others then
            error_offset := DBMS_SQL.LAST_ERROR_POSITION();
            raise notice 'error_offset:%',error_offset;

    dbms_sql.close_cursor(v_cur);
END
/
----------------------------
NOTICE:  error_offset:13
ANONYMOUS BLOCK
```

3.21.4.2.18　返回该语句的 SQL 函数代码

语法格式

```
DBMS_SQL.LAST_SQL_FUNCTION_CODE
    RETURN INTEGER;
```

功能描述

返回该语句的 SQL 函数代码。

返回值说明

返回该语句的 SQL 函数代码。

使用说明

1) 该函数在未执行任何语句时调用返回默认值 0。
2) 在 PARSE 语句之后该函数返回语句的 SQL 函数代码。
3) 同一个 session 中，如果之前执行的 PLSQL 过程已调用一次 PARSE 语句，且当前 PLSQL 过程未调用 OPEN_CURSOR 函数，该函数返回上一次 PARSE 的值；调用 OPEN_CURSOR 后，PARSE 语句前调用，返回默认值 0。

SQL 代码	SQL 语句	SQL 代码	SQL 语句	SQL 代码	SQL 语句
01	CREATE TABLE	43	(NOT USED)	85	TRUNCATE TABLE
02	SET ROLE	44	(NOT USED)	86	(NOT USED)
03	INSERT	45	(NOT USED)	87	(NOT USED)
04	SELECT	46	(NOT USED)	88	(NOT USED)
05	UPDATE	47	(NOT USED)	89	(NOT USED)
06	DROP ROLE(USER)	48	(NOT USED)	90	SET CONSTRAINTS
07	DROP VIEW	49	(NOT USED)	91	CREATE FUNCTION
08	DROP TABLE	50	(NOT USED)	92	(NOT USED)
09	DELETE	51	(NOT USED)	93	DROP FUNCTION
10	CREATE VIEW	52	(NOT USED)	94	CREATE PACKAGE
11	(NOT USED)	53	(NOT USED)	95	(NOT USED)
12	CREATE ROLE(USER)	54	(NOT USED)	96	DROP PACKAGE
13	CREATE SEQUENCE	55	(NOT USED)	97	(NOT USED)

-续表-

SQL 代码	SQL 语句	SQL 代码	SQL 语句	SQL 代码	SQL 语句
14	ALTER SEQUENCE	56	(NOT USED)	98	(NOT USED)
15	(NOT USED)	57	(NOT USED)	99	DROP PACKAGE BODY
16	DROP SEQUENCE	58	(NOT USED)	157	(NOT USED)
17	CREATE SCHEMA	59	CREATE TRIGGER	158	DROP DIRECTORY
18	(NOT USED)	60	(NOT USED)	159	(NOT USED)
19	(NOT USED)	61	DROP TRIGGER	160	(NOT USED)
20	CREATE INDEX	62	ANALYZE TABLE	161	(NOT USED)
21	DROP INDEX	63	(NOT USED)	162	(NOT USED)
22	(NOT USED)	64	(NOT USED)	163	CREATE OPERATOR
23	(NOT USED)	65	(NOT USED)	164	(NOT USED)
24	(NOT USED)	66	(NOT USED)	165	(NOT USED)
25	(NOT USED)	67	(NOT USED)	166	(NOT USED)
26	(NOT USED)	68	DROP PROCEDURE	167	DROP OPERATOR
27	EXPLAIN	69	(NOT USED)	168	(NOT USED)
28	GRANT	70	(NOT USED)	169	(NOT USED)
29	(NOT USED)	71	(NOT USED)	170	CALL METHOD
30	CREATE SYNONYM	72	(NOT USED)	171	(NOT USED)
31	DROP SYNONYM	73	(NOT USED)	172	(NOT USED)
32	(NOT USED)	74	(NOT USED)	173	(NOT USED)
33	SET TRANSACTION	75	(NOT USED)	174	(NOT USED)
34	PL/SQL EXECUTE	76	(NOT USED)	175	(NOT USED)
35	LOCK	77	CREATE TYPE	176	(NOT USED)
36	(NOT USED)	78	DROP TYPE	177	(NOT USED)

-续表-

SQL 代码	SQL 语句	SQL 代码	SQL 语句	SQL 代码	SQL 语句
37	RENAME	79	(NOT USED)	178	(NOT USED)
38	COMMENT	80	(NOT USED)	179	(NOT USED)
39	(NOT USED)	81	(NOT USED)	180	(NOT USED)
40	(NOT USED)	82	(NOT USED)	181	(NOT USED)
41	(NOT USED)	83	DROP TYPE BODY	182	(NOT USED)
42	(NOT USED)	84	(NOT USED)	183	ALTER OPERATOR

示例

LAST_SQL_FUNCTION_CODE 接口在不同情况下调用，获取值可能不同，示例如下:

```
\set SQLTERM /
create table if not exists dbmssql_table1 (i int, j TEXT);
/
\set SQLTERM /
DECLARE
    sql_code int;
    c int;
    vc int;
BEGIN
    c := dbms_sql.open_cursor;
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    dbms_sql.parse(c, 'insert into dbmssql_table1 values (1,''xxx'')', dbms_sql.native);
    vc := dbms_sql.execute(c);
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    dbms_sql.close_cursor(c);
END;
/
\set SQLTERM ;

----------------------------
NOTICE:  0
```

```
NOTICE:  3
NOTICE:  3
ANONYMOUS BLOCK

set serverout on
\set SQLTERM /
DECLARE
    sql_code int;
    c int;
    vc int;
BEGIN
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    c := dbms_sql.open_cursor;
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    dbms_sql.parse(c, 'insert into t1 values (1,''xx'')', dbms_sql.native);
    vc := dbms_sql.execute(c);
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    raise notice '%',sql_code;
    sql_code := dbms_sql.LAST_SQL_FUNCTION_CODE();
    dbms_output.put_line(sql_code);
    raise notice '%',sql_code;
END;
/
\set SQLTERM ;
----------------------------
NOTICE:
NOTICE:  0
NOTICE:  3
NOTICE:  3
ANONYMOUS BLOCK
3
```

3.21.4.2.19　其他示例

在一个匿名块分别对 create 语句、truncate 语句，以及 select 语句做动态 sql 操作，示例如下:

```
set serverout on
\set SQLTERM /
DECLARE
    cnt integer;
    count integer := 0;
    descs DBMS_SQL.DESC_TAB;
    hander integer;
```

```
    res int;
    result_name text;
    my_id int := 10;
    my_name text := 'hello world';
    my_text text;
BEGIN
    hander := dbms_sql.open_cursor();
    dbms_sql.parse(hander,'create table if not exists test_tmp(id int,sname text)',dbms_sql.native);
    count=dbms_sql.execute(hander);
    dbms_output.put_line( 'count:'||count);

    dbms_sql.parse(hander,'truncate table test_tmp',dbms_sql.native);
    count=dbms_sql.execute(hander);
    dbms_output.put_line( 'count:'||count);

    dbms_sql.parse(hander, 'select * from test_tmp', dbms_sql.native);
    dbms_sql.define_column(hander, 1, res);
    dbms_sql.define_column(hander, 2, result_name);
    count = dbms_sql.execute_and_fetch(hander);

    if count > 0 then
        dbms_sql.column_value(hander, 1, res);
        dbms_sql.column_value(hander, 2, result_name);
        dbms_output.put_line( dbms_sql.last_row_count);
        dbms_sql.describe_columns(hander, cnt, descs);
        dbms_output.put_line( 'describe_columns: ' || dbms_sql.last_row_count);
    end if;

    WHILE dbms_sql.fetch_rows(hander) > 0
    LOOP
        dbms_sql.column_value(hander, 1, res);
        dbms_sql.column_value(hander, 2, result_name);
        dbms_output.put_line(dbms_sql.last_row_count);
    END LOOP;

    dbms_sql.close_cursor(hander);
END;
/

------------------------------
ANONYMOUS BLOCK
count:0
count:0
```

3.21.5 插件 dbms_sql 卸载方法

无法卸载。

3.21.6 插件 dbms_sql 升级方法

dbms_sql 随着 KingbaseES 安装包一并升级。通常，用户无须单独升级该插件。

3.22 dbms_utility

3.22.1 插件 dbms_utility 简介

插件 dbms_utility 是 KingbaseES 的一个扩展插件。插件 dbms_utility 功能是提供 dbms_utility 系统包。

dbms_utility 系统包提供一些具有通用功能的子程序，如查看错误堆栈、查看时间等。

- 插件名为 dbms_utility
- 插件版本 V1.0

3.22.2 插件 dbms_utility 加载方式

该插件为初始化数据库实例时默认创建。

3.22.3 插件 dbms_utility 的参数配置

dbms_utility 扩展插件无需配置任何参数。

3.22.4 插件 dbms_utility 使用方法

DBMS_UTILITY 系统包包含的子程序及简介如下表所示：

子程序	简介
FORMAT_CALL_STACK 函数	返回当前的调用堆栈。
FORMAT_ERROR_BACKTRACE 函数	返回当前的错误回溯信息。
FORMAT_ERROR_STACK 函数	返回当前的错误堆栈。
GET_TIME 函数	返回一个时间戳。

-续表-

子程序	简介
GET_HASH_VALUE 函数	返回指定字符串的散列值。

3.22.4.1　FORMAT_CALL_STACK 函数

该函数可以返回当前的调用堆栈信息，包括调用对象的地址，调用的行号和调用对象的名称，它可以用在函数、存储过程或者触发器等 PL/SQL 对象中，用于帮助调试 PL/SQL 程序。

语法

```
DBMS_UTILITY.FORMAT_CALL_STACK() RETURN TEXT;
DBMS_UTILITY.FORMAT_CALL_STACK(FORMAT TEXT) RETURN TEXT;
```

备注:

目前调用对象的地址为函数或过程的 oid 值，如果是内建函数或过程则为对应包的 oid 值。

使用说明

FORMAT: 指定调用堆栈信息的显示模式，分别有‘o’，‘p’，‘s’三种模式。默认使用模式为’p’模式，输出 8 位有效长度的十进制数输出对象地址信息。

- 其中‘o’参数将按照‘%8x%8d%s’格式打印对象地址，调用行号，对象名称，如示例 2 所示。
- 其中‘p’参数将按照‘%8d%8d%s’格式打印对象地址，调用行号，对象名称，如示例 3 所示。
- 其中‘s’参数将按照‘%d,%d,%s’格式打印对象地址，调用行号，对象名称，如示例 4 所示。

示例 1

示例 1 调用无参数的 format_call_stack 子程序，示例如下：

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION checkCallStack() RETURNS TEXT AS
  stack TEXT;
BEGIN
  SELECT dbms_utility.format_call_stack() INTO stack ;
  RETURN stack;
END;
/
\set SQLTERM ;
```

```
call checkCallStack();

                          checkcallstack
-------------------------------------------------------------------
 ----- PL/SQL Call Stack -----
   object   line    object
   handle   number  name
    14575      30   package body sys.dbms_utility.format_call_stack+
    16387       4   function public.checkcallstack
(1 row)
```

示例 2

示例 2 调用传入’o’参数的 format_call_stack 子程序，示例如下：

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION checkHexCallStack() RETURNS TEXT AS
  stack TEXT;
BEGIN
  SELECT dbms_utility.format_call_stack('o') INTO stack ;
  RETURN stack;
END;
/
\set SQLTERM ;

CALL checkHexCallStack();

                        checkhexcallstack
-------------------------------------------------------------------
 ----- PL/SQL Call Stack -----
   object   line    object
   handle   number  name
     38ef      22   package body sys.dbms_utility.format_call_stack+
     4004       4   function public.checkhexcallstack
(1 row)
```

示例 3

示例 3 调用传入’p’参数的 format_call_stack 子程序，示例如下：

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION checkIntCallStack() RETURNS TEXT AS
  stack TEXT;
BEGIN
  SELECT dbms_utility.format_call_stack('p') INTO stack ;
```

```
  RETURN stack;
END;
/
\set SQLTERM ;

CALL checkIntCallStack();

                          checkintcallstack
----------------------------------------------------------------------

    14575      22   package body sys.dbms_utility.format_call_stack+
    16389       4   function public.checkintcallstack
(1 row)
```

示例 4

示例 4 调用传入’s’参数的 format_call_stack 子程序，示例如下：

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION checkIntUnpaddedCallStack() RETURNS TEXT AS
  stack TEXT;
BEGIN
  SELECT dbms_utility.format_call_stack('s') INTO stack ;
  RETURN stack;
END;
/
\set SQLTERM ;

call checkIntUnpaddedCallStack();

        checkintunpaddedcallstack
---------------------------------------------
                                            +
14575,22,sys.dbms_utility.format_call_stack+
16390,4,public.checkintunpaddedcallstack
(1 row)
```

3.22.4.2　FORMAT_ERROR_BACKTRACE 函数

该函数用于返回当前程序发生异常时的错误回溯堆栈，若程序未发生异常，则返回 NULL。

语法

```
DBMS_UTILITY.FORMAT_ERROR_BACKTRACE() RETURN TEXT;
```

示例

format_error_backtrace 函数在异常时使用，示例如下：

```
set serverout on
\set SQLTERM /
CREATE OR REPLACE PROCEDURE p01() AS
  i INT := 0;
BEGIN
  i = i/0;
END;
/
\set SQLTERM ;

\set SQLTERM /
CREATE OR REPLACE PROCEDURE p02() AS
  detail TEXT;
  stack TEXT;
BEGIN
  p01();
EXCEPTION
  WHEN DIVISION_BY_ZERO THEN
    detail = dbms_utility.format_error_stack();
    stack = dbms_utility.format_error_backtrace();
    DBMS_OUTPUT.PUT_LINE('FORMAT_ERROR_STACK IS:');
    DBMS_OUTPUT.PUT_LINE(detail);
    DBMS_OUTPUT.PUT_LINE('FORMAT_ERROR_BACKTRACE IS:');
    DBMS_OUTPUT.PUT_LINE(stack);
END;
/
\set SQLTERM ;

CALL p02();

FORMAT_ERROR_STACK IS:
division by zero
at "public.p01", line 4
FORMAT_ERROR_BACKTRACE IS:
at "public.p01", line 4
at "public.p02", line 5
```

3.22.4.3　FORMAT_ERROR_STACK 函数

该函数用于返回当前程序发生异常时的错误堆栈，若程序未发生异常，则返回 NULL。

语法

```
DBMS_UTILITY.FORMAT_ERROR_STACK() RETURN TEXT;
```

示例

见FORMAT_ERROR_BACKTRACE 函数 。

3.22.4.4　GET_TIME 函数

该函数用于返回一个时间戳，该时间戳不是标准的系统时间戳，单位为厘秒（百分之一秒）。通常在 PL/SQL 程序开始和结束时各调用一次该函数，然后用后一个数字减去前一个数字，以确定当前程序的执行耗时。

语法

```
DBMS_UTILITY.GET_TIME() RETURN NUMBER;
```

使用说明

函数的返回值在 -2147483648 到 2147483647 之间，具体数值取决于机器和系统。调用该函数时，应用程序需要考虑到返回值的符号。例如，两次调用皆为负数，或者第一次调用为负数，第二次调用为正数等情况。

示例

get_time 函数可用于计算语句运行时间，示例如下：

```
\set SQLTERM /
CREATE OR REPLACE PROCEDURE TestGetTime() AS
  t1  PLS_INTEGER;
  t2  PLS_INTEGER;
BEGIN
  t1 := dbms_utility.get_time();
  perform sys_sleep(3);
  t2 := dbms_utility.get_time();

  RAISE NOTICE 'sleeped: % sec.', (t2 - t1) / 100;
END;
/
\set SQLTERM ;

call TestGetTime();

NOTICE:  sleeped: 3 sec.
```

3.22.4.5　GET_HASH_VALUE 函数

对于指定的字符串，返回范围在 [base, base+hase_size-1] 的散列值。

语法

```
DBMS_UTILITY.GET_HASH_VALUE(
      NAME VARCHAR2,
      BASE NUMBER,
      HASH_SIZE NUMBER
      )RETURN NUMBER;
```

参数说明

参数	描述
NAME	指定的要被 HASH 的字符串。
BASE	返回的 HASH 值的基准值，即返回值的最小值。
HASH_SIZE	指定的 HASH 表的大小。

返回值

基于输入字符串的哈希值。

异常说明

- BASE 参数为 NULL，则报错”bad argument”。
- HASH_SIZE 参数为 NULL，则报错”bad argument”。
- HASH_SIZE 为 0 时，则报错”bad argument”。

使用说明

- BASE 可为负值。
- 返回 HASH 值的最小值为 BASE；返回 HASH 值的最大值 BASH+HASH_SIZE-1。例如，若要获取哈希值介于 1000 和 3047 之间的字符串的哈希值，请使用 1000 作为基值，使用 2048 作为 HASH 表大小。
- NAME 为 NULL 或空字符串，若其他参数正常，则返回 BASE 值。
- 若 HASH_SIZE 为负值则返回的哈希值不在哈希范围内，哈希值可能为正，也可能为负。

示例

```
select DBMS_UTILITY.GET_HASH_VALUE('hello world',1,100) from dual;

get_hash_value
----------------
             24
(1 row)
```

3.22.5　插件 dbms_utility 卸载方法

无法卸载。

3.22.6　插件 dbms_utility 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1

```
ALTER EXTENSION dbms_utility UPDATE TO '1.1';
```

3.23　dbms_xmlquery

3.23.1　插件 dbms_xmlquery 简介

插件 dbms_xmlquery 是 KingbaseES 的一个扩展插件，兼容 Oracle 的 dbms_xmlquery 系统包。

dbms_xmlquery 系统包提供了一组子程序，用于将数据库表数据转换为 XML 格式文本。

- 插件名为 dbms_xmlquery
- 插件版本 V1.0

3.23.2　插件 dbms_xmlquery 加载方式

该插件通过 create extension dbms_xmlquery 手动创建。

3.23.3　插件 dbms_xmlquery 的参数配置

dbms_xmlquery 扩展插件无需配置任何参数。

3.23.4　插件 dbms_xmlquery 使用方法

dbms_xmlquery 系统包包含的子程序及简介如下表所示：

子程序	描述
CLOSECONTEXT	关闭或取消分配特定的查询上下文

-续表-

子程序	描述
GETNUMROWSPROCESSED	返回查询处理的行数
GETXML	生成 XML 文档
NEWCONTEXT	创建查询上下文并返回上下文句柄
SETCOLLIDATTRNAME	为集合元素的分隔符标记 ID 设置属性名称
SETDATAHEADER	设置 XML 数据标头
SETENCODINGTAG	设置 XML 文档中编码处理指令
SETMAXROWS	设置转换为 XML 的最大行数
SETROWIDATTRNAME	为行封闭标记的 ID 设置属性名称
SETROWIDATTRVALUE	指定标量列，其值将配分配给行封闭标记的 ID 属性
SETROWSETTAG	设置封闭 XML 数据集的标记
SETROWTAG	设置将 XML 元素括起来的标记
SETSKIPROWS	设置跳过的行数
SETSQLTOXMLNAMEESCAPING	如果映射到 XML 标识符的 SQL 对象名称不是有效的 XML 标识符，则启用或禁用 XML 标记的转义
SETSTYLESHEETHEADER	设置样式表标题
SETTAGCASE	指定生成 XML 标记的大小写样式
USENULLATTRIBUTEINDICATOR	指定是否使用 XML 属性来表示 NULL
USETYPEFORCOLLELEMTAG	指示 XSU 使用集合元素的类型名作为集合元素的标记名

3.23.4.1　常量

DBMS_XMLQUERY 系统包在调用时 (包括指定参数时) 要使用的多个常量，下表列出了常量兼容范围与兼容格式类型。

表 3.23.2: DBMS_XMLQUERY 系统包常量

常量	Oracle	KES	描述
DB_ENCODING	—	UTF-8	设置数据库字符编码
DEFAULT_ROWSETTAG	ROWSET	rowset	设置从结果集生成的 XML 的元素的标记名称（在大多数情况下，根节点的标记名称）– ROWSET
DEFAULT_ERRORTAG	ERROR	不兼容	包含引发错误的默认标记 - ERROR
DEFAULT_ROWIDATTR	NUM	num	设置数据库 XML 元素的基数属性的默认名称
DEFAULT_ROWTAG	ROW	row	设置数据库元素的默认名称标记
DEFAULT _DATE _FORMAT	MM/dd/yyyy HH:mm:ss	不兼容	默认日期掩码 - ‘MM/DD/YYYY HH:MM:SS’
ALL_ROWS	-1	2147483647	指定输出需要包含所有行
NONE	0	不兼容	指定输出不应包含任何 XML 元数据（例如不包含 DTD）
DTD	1	不兼容	指定需要生成 DTD
SCHEMA	2	不兼容	指定生成的 XML 模式
LOWER_CASE	1	1	使用小写标记名
UPPER_CASE	2	2	使用大写标记名

3.23.4.2　函数

3.23.4.2.1　getnumrowsprocessed

实现兼容 Oracle 包 dbms_xmlquery 中的 GetNumrowsProcessed 函数，返回查询处理的行数。

语法

```
FUNCTION  getNumRowsProcessed(ctxHdl IN ctxType) RETURN NUMBER;
```

参数说明

ctxHdl：被声明且打开的上下文查询句柄。

返回类型

NUMBER

示例

```
DECLARE
  qryCtx dbms_xmlquery.ctxHandle;
  result CLOB;
  colms number;
BEGIN
  qryCtx := dbms_xmlquery.newContext('select * from my_table');
  result := dbms_xmlquery.getXML(qryCtx);
  colms :=dbms_xmlquery.getnumrowsprocessed(qryCtx);
  dbms_output.put_line(colms);
  qryCtx := dbms_xmlquery.newContext('select * from my_table');
  dbms_xmlquery.setSkiprows(qryCtx,5);
  dbms_xmlquery.setmaxrows(qryCtx,5);
  colms :=dbms_xmlquery.getnumrowsprocessed(qryCtx);
  dbms_output.put_line(colms);
  result := dbms_xmlquery.getXML(qryCtx);
  colms :=dbms_xmlquery.getnumrowsprocessed(qryCtx);
  dbms_output.put_line(colms);
  dbms_xmlquery.closeContext(qryCtx);
 end;
  /
8
0
3
PL/SQL procedure successfully completed.
```

3.23.4.2.2　getXML

实现兼容 Oracle 包 dbms_xmlquery 中的 getXML 过程及函数，用于创建上下文，执行查询，返回 XML 并关闭上下文，这是一个非常有用的函数，上下文不需要显示的打开或关闭。

语法

```
FUNCTION  getXML(ctxHdl IN ctxType, metaType IN NUMBER := NONE) RETURN CLOB;
PROCEDURE getXML(ctxHdl IN ctxType, xDoc IN CLOB, metaType IN NUMBER := NONE);
FUNCTION  getXML(sqlQuery IN VARCHAR2, metaType IN NUMBER := NONE) RETURN CLOB;
FUNCTION  getXML(sqlQuery IN CLOB, metaType IN NUMBER := NONE) RETURN CLOB;
```

参数说明

ctxHdl：被声明且打开的上下文查询句柄。

返回类型

CLOB

示例

```
DECLARE
  qryCtx dbms_xmlquery.ctxHandle;
  qryCtx1 dbms_xmlquery.ctxHandle;
  result CLOB;
  result1 CLOB;
BEGIN
  qryCtx := dbms_xmlquery.newContext('select * from my_table');
  result := dbms_xmlquery.getXML(qryCtx);
  result1 := dbms_xmlquery.getXML('select * from my_table');
  qryCtx1 := dbms_xmlquery.newContext('select * from test2');
  dbms_xmlquery.getXML(qryCtx1,result);
  dbms_xmlquery.closeContext(qryCtx);
  dbms_xmlquery.closeContext(qryCtx1);
  end;
```

3.23.4.2.3　newContext

实现兼容 Oracle 包 dbms_xmlquery 中的 newContext 函数，创建并保存一个上下文，并将其上下文句柄返回。

语法

```
FUNCTION newContext(sqlQuery IN VARCHAR2) RETURN ctxType;
FUNCTION newContext(sqlQuery IN CLOB) RETURN ctxType;
```

参数说明

sqlQuery：查询结果集以及查询语句。

返回类型

ctxtype

示例

```
DECLARE
  qryCtx dbms_xmlquery.ctxHandle;
  result CLOB;
  newCtx dbms_xmlquery.ctxHandle;
  result1 CLOB;
  result2 CLOB;
BEGIN
  qryCtx := dbms_xmlquery.newContext('select * from test2');
  result := dbms_xmlquery.getXML(qryCtx);
  insert into xml_test values(result);
  newctx := dbms_xmlquery.newContext('select * from xml_test');
  result2 := dbms_xmlquery.getXML(newctx);
  dbms_xmlquery.closeContext(qryCtx);
  dbms_xmlquery.closeContext(newCtx);
 end;
```

3.23.4.3　存储过程

3.23.4.3.1　closeContext

实现兼容 Oracle 包 dbms_xmlquery 中的 CloseContext 过程，用于关闭或解除分配特定的查询上下文。

语法

```
procedure CloseContext（ctxHdl in ctxtype）;
```

参数说明

ctxHdl：被声明且打开的上下文查询句柄。

示例

```
create extension dbms_xmlquery;
create table test1 (id int, name text);
create table test2 (id int, name text);

insert into test1 values(1, 'aaa'), (2, 'bbb');
insert into test2 values(3, 'ccc'), (4, 'ddd');

create table my_table (ID number, NAME varchar2(40), DEPT varchar2(40));
insert into my_table values(1,'test_1','math'),
```

```
(2,'test_2','chinese'),
(3,'test_3','sql_oracle'),
(4,'test_4','chinese'),
(5,'test_5','math'),
(6,'test_6','sql_oracle'),
(7,'test_7','sql_oracle'),
(8,'test_8','chinese');

 create table city1(id number,name varchar2(40));
 create table city2(id number,name varchar2(40));
 insert into city1(id,name) values(1,'广州');
 insert into city1(id,name) values(2,'深圳');
 insert into city1(id,name) values(3,'珠海');
 insert into city1(id,name) values(4,'北京');
 insert into city1(id,name) values(5,'上海');
 insert into city2(id,name) values(1,'南京');
 insert into city2(id,name) values(2,'无锡');
 insert into city2(id,name) values(3,'苏州');
 insert into city2(id,name) values(4,'北京');
 insert into city2(id,name) values(5,'上海');

create table my_students (sid int primary key, sname varchar2(40),subj varchar2(255)[]);

insert into my_students(sid,sname,subj) values(1,'test_1-CN-张三', array['化学','物理','生物']);
insert into my_students(sid,sname,subj) values(2,'test_2-CN-李四', array['化学','物理','生物']);
insert into my_students(sid,sname,subj) values(3,'test_3-EN-aili', array['math','chinese','english']);
insert into my_students(sid,sname,subj) values(4,'test_4-EN-BOB', array['SQL-server','KingbaseES','Oracle
']);
insert into my_students(sid,sname,subj) values(5,'test_1-JP-あい', array['イキ','ふへ','なにぬふ']);

create table my_NULL (id int primary key, nullname varchar2(50) default null,dept varchar2(50)default null);
insert into my_NULL (id,nullname,dept) values(1,'test1','math');
insert into my_NULL (id,nullname,dept) values(2,'test2',null);
insert into my_NULL (id,nullname,dept) values(3,null,'chinese');
insert into my_NULL (id,nullname,dept) values(4,null,null);
insert into my_NULL (id,nullname,dept) values(5,'test5','nothing');

DECLARE
qryCtx1 dbms_xmlquery.ctxHandle;
BEGIN
qryCtx1 := dbms_xmlquery.newContext('select * from my_table');
dbms_xmlquery.closeContext(qryCtx1);
END;
```

3.23.4.3.2　setCollIdAttrName

实现兼容 Oracle 包 dbms_xmlquery 中的 setCollIdAttrName 过程，用于设置集合元素的分隔符标记的 ID 属性名称，设置 NULL 或者空的字符串标记，则会忽略行的 ID 属性。

语法

```
PROCEDURE setCollIdAttrName(ctxHdl IN ctxType, attrName IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- attrName：设置的属性名称。

示例

```
DECLARE
  qryCtx dbms_xmlquery.ctxHandle;
  result CLOB;
BEGIN
  qryCtx := dbms_xmlquery.newContext('select * from my_students');
  dbms_xmlquery.SETCOLLIDATTRNAME(qryCtx,'kes');
  result := dbms_xmlquery.getXML(qryCtx);
  dbms_output.put_line(result);
  dbms_xmlquery.closeContext(qryCtx);
 end;
```

3.23.4.3.3　setDataHeader

实现兼容 Oracle 包 dbms_xmlquery 中的 setDataHeader 过程，用于设置 XML 数据标头，数据标头是添加到查询生成的 XML 实体开头的部分，将这两个实体括在参数中，使用最后指定的数据标头，传递参数 NULL 会取消设置数据标头。

语法

```
PROCEDURE setDataHeader(ctxHdl IN ctxType, header IN CLOB := null, tag IN VARCHAR2 := null);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Header：设置数据头的内容。
- Tag：xml 文档标签名称，默认为：<DOCUMENT>。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
```

```
result CLOB;
header CLOB:='<?KingbaseES-xml:version=1.0?>';
tag varchar2(20):='my_table';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from my_table');
dbms_xmlquery.SETDATAHEADER(qryCtx,header,tag);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_output.put_line(qryCtx);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<my_table>
   <?KingbaseES-xml:version=1.0?>

<ROWSET>
      <ROW num="1">
         <ID>1</ID>
         <NAME>test_1</NAME>

<DEPT>math</DEPT>
      </ROW>
      <ROW num="2">
         <ID>2</ID>

<NAME>test_2</NAME>
         <DEPT>chinese</DEPT>
      </ROW>
      <ROW
num="3">
         <ID>3</ID>
         <NAME>test_3</NAME>

<DEPT>sql_oracle</DEPT>
      </ROW>
      <ROW num="4">
         <ID>4</ID>

<NAME>test_4</NAME>
         <DEPT>chinese</DEPT>
      </ROW>
      <ROW
num="5">
         <ID>5</ID>
```

```
         <NAME>test_5</NAME>

<DEPT>math</DEPT>
      </ROW>
      <ROW num="6">
         <ID>6</ID>

<NAME>test_6</NAME>
         <DEPT>sql_oracle</DEPT>
      </ROW>
      <ROW
num="7">
         <ID>7</ID>
         <NAME>test_7</NAME>

<DEPT>sql_oracle</DEPT>
      </ROW>
      <ROW num="8">
         <ID>8</ID>

<NAME>test_8</NAME>
         <DEPT>chinese</DEPT>
      </ROW>

</ROWSET>
</my_table>
```

3.23.4.3.4　setMaxRows

实现兼容 Oracle 包 dbms_xmlquery 中的 setMaxRows 过程，设置转换为 XML 的最大行数，默认情况下，不设置最大行数。

语法

```
PROCEDURE setMaxRows (ctxHdl IN ctxType, rows IN NUMBER);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Rows：给定的最大查询行数。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
```

```
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from my_table');
dbms_xmlquery.setMAXrows(qryCtx,2);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.setMAXrows(qryCtx,2);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
 /
<?xml version = '1.0'?>
<ROWSET>
   <ROW num="1">
      <ID>1</ID>

<NAME>test_1</NAME>
      <DEPT>math</DEPT>
   </ROW>
   <ROW num="2">

<ID>2</ID>
      <NAME>test_2</NAME>
      <DEPT>chinese</DEPT>

</ROW>
</ROWSET>

<?xml version = '1.0'?>
<ROWSET>
   <ROW num="3">
      <ID>3</ID>

<NAME>test_3</NAME>
      <DEPT>sql_oracle</DEPT>
   </ROW>
   <ROW num="4">

<ID>4</ID>
      <NAME>test_4</NAME>
      <DEPT>chinese</DEPT>

</ROW>
</ROWSET>
```

```
PL/SQL procedure successfully completed.
```

3.23.4.3.5　setRowIdAttrName

实现兼容 Oracle 包 dbms_xmlquery 中的 setRowIdAttrName 过程，用于设置包含标记行的属性名称，如果传值为 NULL 或空值，则会省略。

语法

```
PROCEDURE setRowIdAttrName(ctxHdl IN ctxType, attrName IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Attrname：给定的属性值名称。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
attrName varchar2(40):='wangbo';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETROWIDATTRNAME(qryCtx,attrName);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <ROW wangbo="1">
      <ID>1</ID>

<NAME>aaa</NAME>
   </ROW>
   <ROW wangbo="2">
      <ID>2</ID>

<NAME>bbb</NAME>
   </ROW>
</ROWSET>
```

3.23.4.3.6　setCollIdAttrName

实现兼容 Oracle 包 dbms_xmlquery 中的 setCollIdAttrName 过程，用于设置包含标记行的属性名称，如果传值为 NULL 或空值，则会省略属性。

语法

```
PROCEDURE setCollIdAttrName(ctxHdl IN ctxType, attrName IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- attrName：设置的属性名称。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
attrName varchar2(40):='wangbo';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETROWIDATTRNAME(qryCtx,attrName);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <ROW wangbo="1">
      <ID>1</ID>

<NAME>aaa</NAME>
   </ROW>
   <ROW wangbo="2">
      <ID>2</ID>

<NAME>bbb</NAME>
   </ROW>
</ROWSET>
```

3.23.4.3.7　setRowIdAttrValue

实现兼容 Oracle 包 dbms_xmlquery 中的 setRowIdAttrValue 过程，指定将其值分配给行封装标记的属性的索引列，传值为 NULL 或者空字符，则为属性分配基数值（0,1,2⋯）

语法

```
PROCEDURE setRowIdAttrValue(ctxHdl IN ctxType, colName IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Colname：设置查询结果中的索引列。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
attrName varchar2(40):='KingbaseES-xml';
colName varchar2(40):='NAME';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETROWIDATTRNAME(qryCtx,attrName);
dbms_xmlquery.SETROWIDATTRVALUE(qryCtx,colName);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <ROW KingbaseES-xml="aaa">
      <ID>1</ID>

<NAME>aaa</NAME>
   </ROW>
   <ROW KingbaseES-xml="bbb">
      <ID>2</ID>

<NAME>bbb</NAME>
   </ROW>
</ROWSET>
PL/SQL procedure successfully completed.
```

3.23.4.3.8　setRowsetTag

实现兼容 Oracle 包 dbms_xmlquery 中的 setRowsetTag 过程，用于设置 rowset 数据集标签的名称。设置根节点的标记值（rowset）的默认名称修改。

语法

```
PROCEDURE setRowsetTag(ctxHdl IN ctxType, tag IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Tag：给定的标签名称。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
tag varchar2(20):='wangbo';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test2');
dbms_xmlquery.SETROWSETTAG(qryCtx,tag);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
 /
<?xml version = '1.0'?>
<wangbo>
   <ROW num="1">
      <ID>3</ID>

<NAME>ccc</NAME>
   </ROW>
   <ROW num="2">
      <ID>4</ID>

<NAME>ddd</NAME>
   </ROW>
</wangbo>
PL/SQL procedure successfully completed.
```

3.23.4.3.9　setRowTag

实现兼容 Oracle 包 dbms_xmlquery 中的 setRowTag 过程，用于设置行（row）级标签名称。

语法

```
PROCEDURE setRowTag(ctxHdl IN ctxType, tag IN VARCHAR2);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Tag：给定的标签名称。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
tag varchar2(20):='wangbo';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test2');
dbms_xmlquery.SETROWTAG(qryCtx,tag);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <wangbo num="1">
      <ID>3</ID>

<NAME>ccc</NAME>
   </wangbo>
   <wangbo num="2">
      <ID>4</ID>

<NAME>ddd</NAME>
   </wangbo>
</ROWSET>
PL/SQL procedure successfully completed.
```

3.23.4.3.10　setSkipRows

实现兼容 Oracle 包 dbms_xmlquery 中的 setSkipRows 过程，设置要跳过的行数，默认情况下，不跳过任何行。

语法

```
PROCEDURE setSkipRows(ctxHdl IN ctxType, rows IN NUMBER);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Rows：需要跳过的行数。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from my_table');
dbms_xmlquery.setskiprows(qryCtx,2);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.setskiprows(qryCtx,2);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <ROW num="3">
      <ID>3</ID>

<NAME>test_3</NAME>
      <DEPT>sql_oracle</DEPT>
   </ROW>
   <ROW num="4">

<ID>4</ID>
      <NAME>test_4</NAME>
      <DEPT>chinese</DEPT>
   </ROW>

<ROW num="5">
      <ID>5</ID>
      <NAME>test_5</NAME>

<DEPT>math</DEPT>
   </ROW>
   <ROW num="6">
      <ID>6</ID>

<NAME>test_6</NAME>
      <DEPT>sql_oracle</DEPT>
   </ROW>
   <ROW num="7">

<ID>7</ID>
      <NAME>test_7</NAME>
```

```
      <DEPT>sql_oracle</DEPT>
   </ROW>

<ROW num="8">
      <ID>8</ID>
      <NAME>test_8</NAME>

<DEPT>chinese</DEPT>
   </ROW>
</ROWSET>

<?xml version = '1.0'?>
<ROWSET/>
PL/SQL procedure successfully completed.
```

3.23.4.3.11　setStylesheetHeader

实现兼容 Oracle 包 dbms_xmlquery 中的 setStylesheetHeader 过程，用于设置生成 XML 文档的样式表的标头，传递 NULL 值到 uri 会取消设置样式表标题和样式表类型

语法

```
PROCEDURE setStylesheetHeader(ctxHdl IN ctxType, uri IN VARCHAR2, type IN VARCHAR2 := 'text/xsl');
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Uri：设置 stylesheet 的 uri。
- Type：设置 stylesheet 类型。

示例

```
DECLARE
  qryCtx dbms_xmlquery.ctxHandle;
  result CLOB;
  uri varchar2(40):='KES';
  tag varchar2(20):='KingbaseES-xml';
  BEGIN
  qryCtx := dbms_xmlquery.newContext('select * from test1');
  dbms_xmlquery.SETSTYLESHEETHEADER(qryCtx,uri,tag);
  result := dbms_xmlquery.getXML(qryCtx);
  dbms_output.put_line(result);
  dbms_xmlquery.closeContext(qryCtx);
 end;
/
```

```
<?xml version = '1.0'?>
<?xml-stylesheet href="KES"
type="KingbaseES-xml"?>
<ROWSET>
  <ROW num="1">
     <ID>1</ID>

<NAME>aaa</NAME>
  </ROW>
  <ROW num="2">
     <ID>2</ID>

<NAME>bbb</NAME>
  </ROW>
  <ROW num="3">
     <ID>3</ID>

<NAME>vvv</NAME>
  </ROW>
</ROWSET>
PL/SQL procedure successfully completed.
```

3.23.4.3.12　setTagCase

实现兼容 Oracle 包 dbms_xmlquery 中的 setTagCase 过程，用于指定 XML 标记的大小写。

语法

```
PROCEDURE setTagCase(ctxHdl IN ctxType, tCase IN NUMBER);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Tcase：给定大小写参数的值，0 位默认不操作，1 为小写，2 为大写。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETTAGCASE(qryCtx,2);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_output.put_line('-------------------------------------------');
```

```
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETTAGCASE(qryCtx,1);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_xmlquery.closeContext(qryCtx);
 end;
  /
<?xml version = '1.0'?>
<ROWSET>
   <ROW NUM="1">
      <ID>1</ID>

<NAME>aaa</NAME>
   </ROW>
   <ROW NUM="2">
      <ID>2</ID>

<NAME>bbb</NAME>
   </ROW>
</ROWSET>

-------------------------------------------
<?xml version = '1.0'?>
<rowset>
   <row num="1">
      <id>1</id>

<name>aaa</name>
   </row>
   <row num="2">
      <id>2</id>

<name>bbb</name>
   </row>
</rowset>

PL/SQL procedure successfully completed.
```

useNullAttributeIndicator ~~~~~~~~~~~~~~~~~~~~~~~~~~

实现兼容 Oracle 包 dbms_xmlquery 中的 useNullAttributeIndicator 过程，此过程使用 XML 属性来指定它是 NULL 或指定是否省略 XML 文档中的某些实体。

语法

```
PROCEDURE useNullAttributeIndicator(ctxHdl IN ctxType, flag IN BOOLEAN := true);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Flag：开启 NULL 值显示开关 true 表示开启，false 表示关闭。

示例

```
DECLARE
   qryCtx dbms_xmlquery.ctxHandle;
   result CLOB;
 BEGIN
   qryCtx := dbms_xmlquery.newContext('select * from my_null');
   dbms_xmlquery.useNullAttributeIndicator(qryCtx,false);
   result := dbms_xmlquery.getXML(qryCtx);
   dbms_output.put_line(result);
   dbms_output.put_line('-------------------------------------------');
  qryCtx := dbms_xmlquery.newContext('select * from my_null');
   dbms_xmlquery.useNullAttributeIndicator(qryCtx,true);
   result := dbms_xmlquery.getXML(qryCtx);
   dbms_output.put_line(result);
   dbms_output.put_line('-------------------------------------------');
   result :=dbms_xmlquery.getxml('select * from my_null');
   dbms_output.put_line(result);
  dbms_xmlquery.closeContext(qryCtx);
end;
/
<?xml version = '1.0'?>
<ROWSET>
 <ROW num="1">
    <ID>1</ID>
    <NULLNAME>test1</NULLNAME>
    <DEPT>math</DEPT>
 </ROW>
 <ROW num="2">
    <ID>2</ID>

<NULLNAME>test2</NULLNAME>
 </ROW>
 <ROW num="3">
    <ID>3</ID>
    <DEPT>chinese</DEPT>
 </ROW>
 <ROW num="4">
    <ID>4</ID>
```

```
 </ROW>
 <ROW num="5">
    <ID>5</ID>

<NULLNAME>test5</NULLNAME>
    <DEPT>nothing</DEPT>
 </ROW>
</ROWSET>

-------------------------------------------
<?xml version = '1.0'?>
<ROWSET>
 <ROW num="1">
    <ID>1</ID>
    <NULLNAME>test1</NULLNAME>
    <DEPT>math</DEPT>
 </ROW>
 <ROW num="2">
    <ID>2</ID>

<NULLNAME>test2</NULLNAME>
    <DEPT NULL="TRUE"/>
 </ROW>
 <ROW num="3">
    <ID>3</ID>
    <NULLNAME NULL="TRUE"/>
    <DEPT>chinese</DEPT>
 </ROW>
 <ROW num="4">
    <ID>4</ID>

<NULLNAME NULL="TRUE"/>
    <DEPT NULL="TRUE"/>
 </ROW>
 <ROW num="5">
    <ID>5</ID>
    <NULLNAME>test5</NULLNAME>
    <DEPT>nothing</DEPT>
 </ROW>
</ROWSET>

-------------------------------------------
<?xml version = '1.0'?>
<ROWSET>
 <ROW num="1">
```

```
    <ID>1</ID>
    <NULLNAME>test1</NULLNAME>
    <DEPT>math</DEPT>
 </ROW>
 <ROW num="2">
    <ID>2</ID>

<NULLNAME>test2</NULLNAME>
 </ROW>
 <ROW num="3">
    <ID>3</ID>
    <DEPT>chinese</DEPT>
 </ROW>
 <ROW num="4">
    <ID>4</ID>
 </ROW>
 <ROW num="5">
    <ID>5</ID>

<NULLNAME>test5</NULLNAME>
    <DEPT>nothing</DEPT>
 </ROW>
</ROWSET>

PL/SQL procedure successfully completed.
```

3.23.4.3.13　useTypeForCollElemTag

实现兼容 Oracle 包 dbms_xmlquery 中的 useTypeForCollElemTag 过程，此过程指定是否将集合元素的类型名称用作该元素的标记名称。默认情况下，集合元素的标记名称是集合类型，false 时集合标记的名称后跟集合的标记名称 _item。

语法

```
PROCEDURE useTypeForCollElemTag(ctxhdl IN ctxhandle, flag IN boolean := true);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Flag：设置是否开启使用类型名作为集合元素的标签名称。

示例

```
DECLARE
 result CLOB;
```

```
 qryCtx dbms_xmlquery.ctxHandle;
BEGIN
 qryCtx := dbms_xmlquery.newContext('select * from my_students');
 dbms_xmlquery.useTypeForCollElemTag(qryCtx);
 result := dbms_xmlquery.getXML(qryCtx);
 dbms_output.put_line(result);
 dbms_xmlquery.closeContext(qryCtx);
end;
```

3.23.4.3.14　setEncodingTag

实现兼容 Oracle 包 dbms_xmlquery 中的 setEncodingTag 过程，设置 XML 文档的编码处理指令。

语法

```
PROCEDURE setEncodingTag(ctxHdl IN ctxType, enc IN VARCHAR2:= 'UTF-8');
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Enc：编码格式名称。

示例

```
DECLARE
qryCtx dbms_xmlquery.ctxHandle;
result CLOB;
enc varchar2(20) := 'UTF-8';
BEGIN
qryCtx := dbms_xmlquery.newContext('select * from test1');
dbms_xmlquery.SETENCODINGTAG(qryCtx,enc);
result := dbms_xmlquery.getXML(qryCtx);
dbms_output.put_line(result);
dbms_output.put_line(qryCtx);
dbms_xmlquery.closeContext(qryCtx);
 end;
 /
<?xml version = '1.0' encoding = 'UTF-8'?>
<ROWSET>
   <ROW num="1">

<ID>1</ID>
      <NAME>aaa</NAME>
   </ROW>
   <ROW num="2">
      <ID>2</ID>
```

```
<NAME>bbb</NAME>
   </ROW>
   <ROW num="3">
      <ID>3</ID>

<NAME>vvv</NAME>
   </ROW>
</ROWSET>
5
```

3.23.4.3.15　setSQLToXMLNameEscaping

实现兼容 Oracle 包 dbms_xmlquery 中的 setSQLToXMLNameEscaping 过程，如果映射到 XML 标识符的 SQL 对象名称不是有效的 XML 标识符，则关闭或打开 XML 标记转义。

语法

```
PROCEDURE setSQLToXMLNameEscaping(ctxHdl IN ctxType, flag IN BOOLEAN := true);
```

参数说明

- ctxHdl：被声明且打开的上下文查询句柄。
- Flag：是否开启非法行标签字符转义，元素内容强制开启。

示例

```
DECLARE
 qryCtx dbms_xmlquery.ctxhandle;
 result CLOB;
 BEGIN
 qryCtx := dbms_xmlquery.newContext('select 1 "<" from dual');
 dbms_xmlquery.SETSQLTOXMLNAMEESCAPING(qryCtx,false);
 result := dbms_xmlquery.getXML(qryCtx);
 dbms_output.put_line(result);
 dbms_output.put_line('--------------------------------');
 qryCtx := dbms_xmlquery.newcontext('select 1 "<" from dual');
 dbms_xmlquery.SETSQLTOXMLNAMEESCAPING(qryCtx,true);
 result := dbms_xmlquery.getXML(qryCtx);
 dbms_output.put_line(result);
 dbms_xmlquery.closeContext(qryCtx);
end;
/
<?xml version = '1.0'?>
<ERROR>oracle.xml.sql.OracleXMLSQLException: Character
'<' is not allowed in an XML tag name.</ERROR>
```

```
--------------------------------
<?xml version = '1.0'?>
<ROWSET>
 <ROW num="1">
    <_x003C_>1</_x003C_>

</ROW>
</ROWSET>
PL/SQL procedure successfully completed.
```

3.23.5　插件 dbms_xmlquery 卸载方法

通过 drop extension dbms_xmlquery 手动卸载。

3.23.6　插件 dbms_xmlquery 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1

```
ALTER EXTENSION dbms_xmlquery UPDATE TO '1.1';
```

3.24　dict_int

3.24.1　插件 dict_int 简介

dict_int 是一个附加的全文搜索词典模板的示例。这个示例词典的动机是控制有符号和无符号整数的索引，允许对此类数字进行索引，同时防止唯一词数量的过度增长严重影响搜索性能。

- 插件名为 dict_int
- 插件版本 V1.0

3.24.2　插件 dict_int 加载方式

```
CREATE EXTENSION dict_int;
```

3.24.3　插件 dict_int 的参数配置

该词典接受两个选项

`maxlen` 参数指定在一个整数词中允许的最大位数。默认值为 6。

`rejectlong` 参数指定一个超长整数是否应该被截断或忽略。如果 `rejectlong` 为 false（默认），该词典返回该整数的第一个数字。如果 `rejectlong` 为 true，该词典将一个超长整数作为一个停用词对待，因此它将不会被索引。** 注意 ** 这也意味着这样一个整数不能被搜索。

3.24.4　插件 dict_int 的使用方法

安装 dict_int 扩展会使用默认参数创建一个文本搜索模板 intdict_template 和一个基于它的词典 intdict。

您可以修改 intdict 词典参数

```
ALTER TEXT SEARCH DICTIONARY intdict (MAXLEN = 4, REJECTLONG = true);
```

或者创建一个基于该模板的新词典

```
CREATE TEXT SEARCH DICTIONARY myintdict (
  TEMPLATE = intdict_template
);
```

要测试 intdict 词典，可以尝试

```
SELECT ts_lexize('intdict', '12345678');

 ts_lexize
-----------
 {}
```

也可以将字典应用于文本搜索配置中

```
ALTER TEXT SEARCH CONFIGURATION english ALTER MAPPING FOR int, uint WITH intdict;
```

3.24.5　插件 dict_int 卸载方法

```
DROP EXTENSION dict_int;
```

3.24.6　插件 dict_int 升级方法

dict_int 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

```
ALTER EXTENSION dict_int UPDATE TO new_version;
```

3.25 dict_xsyn

3.25.1 插件 dict_xsyn 简介

dict_xsyn（扩展同义词字典）是一个附加全文搜索字典模板的例子。这种字典类型将词替换为它们的同义词分组，并且让使用其任一同义词进行搜索变得可能。

3.25.2 插件 dict_xsyn 加载方式

```
create extension dict_xsyn;
```

3.25.3 插件 dict_xsyn 的参数配置

一个 dict_xsyn 词典接受以下选项

- matchorig 控制该词典是否接受原生词。默认为 true。
- matchsynonyms 控制该词典是否接受同义词。默认为 false。
- keeporig 控制原生词是否被包括在词典的输出中。默认为 true。
- keepsynonyms 控制同义词是否被包括在词典的输出中。默认为 true。
- rules 是包含同义词列表的文件的基本名。这个文件必须被存储在 $SHAREDIR/tsearch_data/$SHAREDIR 表示 KingbaseES 安装的共享数据目录）中。它的名称必须以.rules 结束（这不包括在 rules 参数中）。

规则文件具有以下格式：

- 每一行表示一个单一词的一组同义词，该词位于在该行的行首。同义词之间用空格分隔，如下所示

```
word syn1 syn2 syn3
```

- 井号（#）是注释定界符。它可以出现在一行中的任何位置。该行的剩余部分将被跳过。

具体用例可以参考模板 `xsyn_sample.rules`，该文件位于 `$SHAREDIR/tsearch_data/`目录，其中 `$SHAREDIR` 表示 KingbaseES 安装的共享数据目录，通常是`/usr/local/share/kingbase`（如果不确定，可使用 `sys_config --sharedir` 查看）。

3.25.4　插件 dict_xsyn 的使用方法

安装 dict_xsyn 扩展会用默认参数创建一个文本搜索模板 xsyn_template 以及一个基于它的词典 xsyn。

用户可以修改参数或者基于该模板创建新的词典 `my_rules`，同时插入数据，例如：

```
word syn1 syn2 syn3
```

要测试该词典，你可以尝试：

```
ALTER TEXT SEARCH DICTIONARY xsyn (RULES='my_rules', KEEPORIG=false);

SELECT ts_lexize('xsyn', 'word');
      ts_lexize
-----------------------
 {syn1,syn2,syn3}
(1 row)

ALTER TEXT SEARCH DICTIONARY xsyn (RULES='my_rules', KEEPORIG=true);

SELECT ts_lexize('xsyn', 'word');
      ts_lexize
-----------------------
 {word,syn1,syn2,syn3}
(1 row)

ALTER TEXT SEARCH DICTIONARY xsyn (RULES='my_rules', KEEPORIG=false, MATCHSYNONYMS=true);

SELECT ts_lexize('xsyn', 'syn1');
      ts_lexize
-----------------------
 {syn1,syn2,syn3}
(1 row)

ALTER TEXT SEARCH DICTIONARY xsyn (RULES='my_rules', KEEPORIG=true, MATCHORIG=false, KEEPSYNONYMS=false);

SELECT ts_lexize('xsyn', 'syn1');
      ts_lexize
-----------------------
 {word}
(1 row)
```

在实际使用时可以将它包含在一个文本搜索配置中。如下所示：

```
ALTER TEXT SEARCH CONFIGURATION english
    ALTER MAPPING FOR word, asciiword WITH xsyn, english_stem;
```

3.25.5　插件 dict_xsyn 卸载方法

```
drop extension dict_xsyn;
```

3.25.6　插件 dict_xsyn 升级方法

dict_xsyn 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.26　file_fdw

3.26.1　插件 file_fdw 简介

file_fdw 模块提供外部数据包装器 file_fdw，它能被用来访问服务器的文件系统中的数据文件，或者在服务器上执行程序并读取它们的输出。数据文件或程序输出必须是能够被 COPY FROM 读取的格式，详见 COPY。当前只能读取数据文件。

3.26.2　插件 file_fdw 加载方式

在 ksql 中运行：

```
create extension file_fdw;
```

3.26.3　插件 file_fdw 的参数配置

用这个包装器创建的一个外部表可以有下列选项：

- filename

  指定要被读取的文件。必须是一个绝对路径名。必须指定 filename 或 program，但不能同时指定两个。

- program

  指定要执行的命令。该命令的标准输出将被读取，就像使用 COPY FROM PROGRAM 一样。必须指定 program 或 filename，但不能同时指定两个。

- format

  指定数据的格式，和 COPY 的 FORMAT 选项相同。

- header

  指定数据是否具有一个头部行，和 COPY 的 HEADER 选项相同。

- delimiter

  指定数据的定界符字符，和 COPY 的 DELIMITER 选项相同。

- quote

  指定数据的引用字符，和 COPY 的 QUOTE 选项相同。

- escape

  指定数据的转义字符，和 COPY 的 ESCAPE 选项相同。

- null

  指定数据的空字符串，和 COPY 的 NULL 选项相同。

- encoding

  指定数据的编码，和 COPY 的 ENCODING 选项相同。

注意虽然 COPY 允许诸如 HEADER 的选项不用一个相应的值指定，但是外部表选项语法要求在所有情况下都出现一个值。要激活通常写入没有值的 COPY 选项，你可以传递值 TRUE，因为所有这些选项都是布尔值。

使用这个包装器创建的表的一列可以具有下列选项：

- force_not_null

  这是一个布尔选项。如果为真，它指定该列的值不应该与空字符串匹配（也就是表级别的 null 选项）。这和把该列放在 COPY 的 FORCE_NOT_NULL 选项中具有相同的效果。

- force_null

  这是一个布尔选项。如果为真，它指定匹配空值字符串的列值会被返回为 NULL，即使该值被引号引用。如果没有这个选项，只有匹配空值字符串的未被引用的值会被返回为 NULL。这和在 COPY 的 FORCE_NULL 选项中列出该列有同样的效果。

COPY 的 FORCE_QUOTE 选项当前不被 file_fdw 支持。

这些选项只能为一个外部表及其列指定，而不能在 file_fdw 外部数据包装器的选项中指定，也不能在使用该包装器的服务器或者用户映射的选项中指定。

出于安全原因，改变表级别的选项要求超级用户特权或具有默认角色 sys_read_server_files（使用文件名）或默认角色 sys_execute_server_program （使用程序）的权限只有特定用户能够控制读取哪个文件或者运行哪个程序。原则上普通用户可以被允许改变其它选项，但是当前还不支持这样做。

当指定 program 选项时，请记住，选项字符串是通过 shell 执行的。如果想传递任何参数到来自不受信任的源的命令，必须小心去掉或转义任何对 shell 来说可能有特殊含义的字符。安全起见，最好使用固定的命令字符串，或者至少避免传递任何用户输入。

对于一个使用 file_fdw 的外部表，EXPLAIN 显示要读取的文件名或要运行的程序。对于文件来说，除非指定 COSTS OFF，否则文件尺寸（以字节计）也会被显示。

3.26.4　插件 file_fdw 的使用方法

一种 file_fdw 的用法是把可用的 PostgreSQL 活动日志变成一个表用于查询。要这样做，首先你必须正在将日志记录到一个 CSV 文件，这里我们称其为 pglog.csv。首先，将 file_fdw 安装为一个扩展：

```
CREATE EXTENSION file_fdw;
```

然后创建一个外部服务器

```
CREATE SERVER pglog FOREIGN DATA WRAPPER file_fdw;
```

现在你已经准备好创建外部数据表。使用 CREATE FOREIGN TABLE 命令，你将需要为该表定义列、CSV 文件名以及格式。

```
CREATE FOREIGN TABLE pglog (
  log_time timestamp(3) with time zone,
  user_name text,
  database_name text,
  process_id integer,
  connection_from text,
  session_id text,
  session_line_num bigint,
  command_tag text,
  session_start_time timestamp with time zone,
  virtual_transaction_id text,
  transaction_id bigint,
  error_severity text,
  sql_state_code text,
  message text,
  detail text,
  hint text,
  internal_query text,
  internal_query_pos integer,
  context text,
  query text,
  query_pos integer,
  location text,
  application_name text
) SERVER pglog
OPTIONS ( filename '/home/josh/data/log/pglog.csv', format 'csv' );
```

就是这样了—现在你能够直接查询你的日志了。当然，在生产中你会需要定义一些方法来处理日志轮转。

3.26.5　插件 file_fdw 卸载方式

在 ksql 中运行

```
drop extension file_fdw;
```

3.26.6　插件 file_fdw 升级方式

file_fdw 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.27　fix_security_hole

3.27.1　插件 fix_security_hole 简介

fix_security_hole 是 KingbaseES 的一个扩展插件，主要用于禁止数据库通过 copy 执行操作系统 shell 命令。

- 插件名为 fix_security_hole
- 插件版本 V1.0

3.27.2　插件 fix_security_hole 加载方式

在使用 fix_security_hole 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

示例：

```
shared_preload_libraries = 'fix_security_hole'
```

3.27.3　插件 fix_security_hole 的参数配置

close_copy_crogram
启动禁止 copy 执行操作系统 shell 命令，取值范围为 on 或者 off，缺省为 on。

```
show close_copy_crogram;
 close_copy_crogram
--------------------
 on
(1 row)
```

```
\c - system
alter system set close_copy_crogram = off;
select sys_reload_conf();
show close_copy_crogram;
 close_copy_crogram
--------------------
 off
(1 row)
```

3.27.4　插件 fix_security_hole 使用方法

KingbaseES 加载 fix_security_hole 插件后，超级用户将无法通过 copy 执行操作系统的 shell 命令。

示例：

```
--功能为关闭状态时
create table t1 (a text);
copy t1 from program 'ifconfig';
COPY 51

--功能为开启状态时
create table t1 (a text);
copy t1 from program 'ifconfig';
错误:  not support Copy_To/From_Porgram feature
```

3.27.5　插件 fix_security_hole 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.27.6　插件 fix_security_hole 升级方法

fix_security_hole 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.28 ftutilx

3.28.1 插件 ftutilx 简介

ftutilx 是一个 KingbaseES 的扩展，主要用于从存储流版式文件的 blob 类型字段中抽取文本内容。其中 blob 类型字段内容可以包括 pdf、doc、docx、wps、xls、xlsx、ppt 和 pptx 格式文件。ftutilx 插件不支持加密文件格式。

3.28.2 插件 ftutilx 加载方式

在使用 ftutilx 之前，需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'ftutilx' # (change requires restart)
```

使用 KSQL 创建插件：

```
CREATE EXTENSION ftutilx;
```

3.28.3 插件 ftutilx 的参数配置

`ftutilx.max_string_length`
抽取结果最大长度，默认值 128M，该参数设置后立即生效。

`ftutilx.jvm_option_string`
JVM 初始化参数，默认值"-Xmx1024m,-Xms1024m,-Xmn256m,-XX:MetaspaceSize=64m,-XX:MaxMetaspaceSize=128m,-XX:CompressedClassSpaceSize=256m"，该参数只在会话进程中首次调用 extracttext 函数创建 JVM 时生效，再次设置该参数不再有效。

在数据库默认扩展加载机制下，在一个会话中创建扩展以后，在新的会话开始后并不是立即加载扩展动态库，而是直到首次调用扩展中的接口时才会加载扩展动态库，导致在这之前在新会话中设置扩展参数无效。解决方法是修改数据库配置文件中的 shared_preload_libraries 或者 session_preload_libraries 两个参数之一，使参数值包括 ftutilx，即可在新会话开始后立即加载 ftutilx 扩展动态库，并设置扩展参数。

3.28.4 插件 ftutilx 的使用方法

ftutilx 插件提供了 extracttext 函数用于抽取存储在 blob 类型字段内的文件内容。extracttext 函数接受一个代表文件内容的 blob 类型参数，返回抽取的 text 类型文本内容。

```
CREATE TABLE tab (title text, body blob);

INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));
```

```
SELECT title, length(extracttext(body)) FROM tab;
```

3.28.4.1　使用 ftutilx 全文检索的联合使用方案

由于电子文档内容抽取速度较慢，为提高全文检索性能，可以在表中添加存储列，用于存储内容抽取结果或者词位列表。

方案一

```
CREATE EXTENSION zhparser;

CREATE TEXT SEARCH CONFIGURATION zhparsercfg (PARSER = zhparser);

ALTER TEXT SEARCH CONFIGURATION zhparsercfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;

CREATE EXTENSION ftutilx;

CREATE TABLE tab (title text, body blob);

ALTER TABLE tab ADD COLUMN content text GENERATED ALWAYS AS (extracttext(body)) STORED;

CREATE INDEX tab_idx ON tab USING GIN (to_tsvector('zhparsercfg', content));

INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));

SELECT title FROM tab WHERE to_tsvector('zhparsercfg', content) @@ to_tsquery('日志');
```

方案二

```
CREATE EXTENSION zhparser;

CREATE TEXT SEARCH CONFIGURATION zhparsercfg (PARSER = zhparser);

ALTER TEXT SEARCH CONFIGURATION zhparsercfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;

CREATE EXTENSION ftutilx;

CREATE TABLE tab (title text, body blob);

ALTER TABLE tab ADD COLUMN tab_idx_col tsvector GENERATED ALWAYS AS (to_tsvector('zhparsercfg',
extracttext(body))) STORED;

CREATE INDEX tab_idx ON tab USING GIN (tab_idx_col);
```

```
INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));

SELECT title FROM tab WHERE tab_idx_col @@ to_tsquery('日志');
```

3.28.4.2　注意事项

1）ftutilx 需要依赖于 jre-1.8.0 运行时环境，针对不同的操作系统，需要配置环境变量完成适配。

对于 Linux 平台，部署后需要设置 LD_LIBRARY_PATH 系统环境变量包含 jre-1.8.0 的 libjvm.so 路径。

对于 Windows 平台，部署后需要设置 PATH 系统环境变量包含 jre-1.8.0 的 jvm.dll 路径。

2）ftutilx.max_string_length 参数用于配置抽取结果的最大长度，但由于 tsvector 目前最大支持 (1M-1)，所以 extracttext 结合 to_tsvector 使用时，分词结果大小不能超过 (1M-1)。

3）ftutilx 需要创建 JVM，JVM 会占用较多内存。虽然调整 ftutilx.jvm_option_string 的-Xmx 可以限制 JVM 的内存占用，但过小的-Xmx 值会导致大文件解析时 JVM 发生内存不足异常。

4）基于前面的全文检索联合使用方案，在系统内存较少的环境中，需要限制并行插入数据的会话进程数量，以免系统内存被耗尽。

5）用户需要从 apache 网站自行下载 tika-app-1.26.jar 或者更高版本的包，改名为 ftutilx.jar 后放入 ftutilx.so 所在目录，方可使用 ftutilx 扩展。

3.28.5　插件 ftutilx 卸载方法

```
DROP EXTENSION ftutilx;
```

3.28.6　插件 ftutilx 升级方法

```
ALTER EXTENSION UPDATE ftutilx TO '2.0';
```

3.29　fuzzystrmatch

3.29.1　插件 fuzzystrmatch 简介

fuzzystrmatch 提供了两个字符串之间的编辑距离的相关函数，即由一个转换成另一个所需的最少编辑操作次数。

- 插件名为 fuzzystrmatch
- 插件版本 V1.1

3.29.2　插件 fuzzystrmatch 加载方式

```
create extension fuzzystrmatch;
```

3.29.3　插件 fuzzystrmatch 参数配置

备注:

在使用该插件的过程中，禁止加载 ora_forbid_func_polymorphism 参数，或将其设置为 off。

3.29.4　插件 fuzzystrmatch 使用方法

3.29.4.1　levenshtein

计算两个字符串之间编辑距离。

语法

levenshtein(t1 text, t2 text) RETURNS int;

参数

`t1`
源字符串。

`t2`
目标字符串。

示例

```
TEST=# select levenshtein('text','tst');
 levenshtein
-------------
           2
(1 行记录)

TEST=# select levenshtein('text','test');
 levenshtein
-------------
```

```
           1
(1 行记录)

TEST=# select levenshtein('text','text');
 levenshtein
-------------
           0
(1 行记录)
```

3.29.4.2　levenshtein_with_costs

计算两个字符串之间的编辑距离。

语法

levenshtein_with_costs(t1 text, t2 text, i1 int, i2 int, i3 int) RETURNS int;

参数

t1
源字符串。

t2
目标字符串。

i1
insert 的代价权重。

i2
replace 的代价权重。

i3
delete 的代价权重。

示例

```
TEST=# select levenshtein('tet','text', 1 ,2 ,3);
 levenshtein
-------------
           1
(1 行记录)

TEST=# select levenshtein('tet','text', 2 ,2 ,3);
 levenshtein
-------------
           2
(1 行记录)
```

3.29.4.3　levenshtein_less_equal

levenshtein_less_equal 是 levenshtein 函数速度更快的版本，它被用于只对小距离感兴趣的情况。如果实际距离小于 i，那么 levenshtein_less_equal 返回正确的距离。否则它返回某个大于 i 的值。如果 i 是负值，那么其行为等同于 levenshtein 函数。

语法

levenshtein_less_equal(t1 text, t2 text, i int) RETURNS int;

参数

t1
源字符串。

t2
目标字符串。

i
参与比较的值。

示例

```
SELECT levenshtein_less_equal('extensive', 'exhaustive', 2);
 levenshtein_less_equal
------------------------
                       3
(1 row)

SELECT levenshtein_less_equal('extensive', 'exhaustive', 4);
 levenshtein_less_equal
------------------------
                       4
(1 row)
```

3.29.4.4　levenshtein_less_equal_with_costs

与上述函数相似，增加如 levenshtein_with_costs 一样的代价花销。

3.29.4.5　metaphone

计算字符串的 metaphone 键。

语法

metaphone(text, int) returns text。

语法

text
参与计算的字符串。

int
参与运算的参考键值。

示例

```
SELECT metaphone('GUMBO', 4);
 metaphone
-----------
 KM
(1 row)
```

3.29.4.6　soundex, text_soundex

生成字符串的 soundex 键。

语法

soundex(text) rerturns text。

参数

text
参与计算的字符串。

示例

```
TEST=# select soundex('A500');
 soundex
---------
 A000
(1 行记录)

TEST=# select soundex('A529sd800');
 soundex
---------
 A230
(1 行记录)

TEST=# select soundex('A529sds800');
 soundex
---------
 A232
(1 行记录)
```

```
TEST=# select soundex('hello world');
 soundex
---------
 H464
(1 行记录)
```

3.29.4.7　difference

获得两个对比的字符串转换为 soundex 值后匹配的位数。

语法

difference(t1 text, t2 text) RETURNS int;

参数

t1
参与计算的字符串 1。

t2
参与计算的字符串 2。

示例

```
TEST=# select difference('ab', 'as');
 difference
------------
          3
(1 行记录)

TEST=# select difference('ab', 'ab');
 difference
------------
          4
(1 行记录)
```

3.29.4.8　dmetaphone

获取 text 的 dmetaphone。

语法

dmetaphone(text) returns text。

参数

text
参与计算的 text 字符串。

示例

```
TEST=# select dmetaphone('tetx');
 dmetaphone
------------
 TTKS
(1 行记录)
```

3.29.5 插件 fuzzystrmatch 卸载方法

```
drop extension fuzzystrmatch;
```

3.29.6 插件 fuzzystrmatch 升级方法

如果使用旧版本插件，可以升级到最新版本。

备注:

不支持跨版本升级。建议由低到高依次升级版本。

```
alter extension fuzzystrmatch update to '1.1';
```

3.30 hstore

3.30.1 插件 hstore 简介

hstore 支持 hstore 类型及其相关的函数。

- 插件名为 hstore
- 插件版本 V1.0

3.30.2 插件 hstore 加载方式

使用时需要 create extension hstore。

示例：

```
create extension hstore;
```

备注:

创建扩展前，需要关闭禁用多态函数参数，执行’set ora_forbid_func_polymorphism to off;’。

3.30.3　插件 hstore 参数配置

无需配置任何参数。

3.30.4　插件 hstore 使用方法

3.30.4.1　hstore 类型

字符串的一种，可以将字符串与’=>’进行单元分割重组形成新的字符串。

示例

```
select ''::hstore;
 hstore
--------

(1 row)

select 'a=>b'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select ' a=>b'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select 'a =>b'::hstore;
  hstore
----------
 "a"=>"b"
```

```
(1 row)

select 'a=>b '::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select 'a=> b'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select '"a"=>"b"'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select ' "a"=>"b"'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select '"a" =>"b"'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select '"a"=>"b" '::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select '"a"=> "b"'::hstore;
  hstore
----------
 "a"=>"b"
(1 row)

select 'aa=>bb'::hstore;
```

```
   hstore
------------
 "aa"=>"bb"
(1 row)

select ' aa=>bb'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select 'aa =>bb'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select 'aa=>bb '::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select 'aa=> bb'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select '"aa"=>"bb"'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select ' "aa"=>"bb"'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select '"aa" =>"bb"'::hstore;
   hstore
------------
 "aa"=>"bb"
```

```
(1 row)

select '"aa"=>"bb" '::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select '"aa"=> "bb"'::hstore;
   hstore
------------
 "aa"=>"bb"
(1 row)

select 'aa=>bb, cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>bb , cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>bb ,cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>bb, "cc"=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>bb , "cc"=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>bb ,"cc"=>dd'::hstore;
```

```
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>"bb", cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>"bb" , cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>"bb" ,cc=>dd'::hstore;
         hstore
------------------------
 "aa"=>"bb", "cc"=>"dd"
(1 row)

select 'aa=>null'::hstore;
   hstore
------------
 "aa"=>NULL
(1 row)

select 'aa=>NuLl'::hstore;
   hstore
------------
 "aa"=>NULL
(1 row)

select 'aa=>"NuLl"'::hstore;
    hstore
--------------
 "aa"=>"NuLl"
(1 row)

select e'\\=a=>q=w'::hstore;
   hstore
-------------
 "=a"=>"q=w"
```

```
(1 row)

select e'"=a"=>q\\=w'::hstore;
   hstore
-------------
 "=a"=>"q=w"
(1 row)

select e'"\\"a"=>q>w'::hstore;
    hstore
--------------
 "\"a"=>"q>w"
(1 row)
```

3.30.4.2　hstore_to_jsonb

将 hstore 类型转换为 jsonb 类型。

语法

```
hstore_to_jsonb(hstore) returns jsonb。
```

示例

```
select hstore_to_jsonb('"a key" =>1, b => t, c => null, d=> 12345, e => 012345, f=> 1.234, g=> 2.345e+4');
                                        hstore_to_jsonb
-----------------------------------------------------------------------------------------------
 {"b": "t", "c": null, "d": "12345", "e": "012345", "f": "1.234", "g": "2.345e+4", "a key": "1"}
(1 row)
```

3.30.4.3　hstore_to_json

将 hstore 转换为 json 类型。

语法

```
hstore_to_json(hstore) returns json。
```

示例

```
select hstore_to_json('"a key" =>1, b => t, c => null, d=> 12345, e => 012345, f=> 1.234, g=> 2.345e+4');
                                        hstore_to_json
-----------------------------------------------------------------------------------------------
```

```
 {"b": "t", "c": null, "d": "12345", "e": "012345", "f": "1.234", "g": "2.345e+4", "a key": "1"}
(1 row)
```

3.30.5　插件 hstore 卸载方法

不需要 hstore 插件只需要卸载插件即可。

示例：

```
drop extension hstore;
```

3.30.6　插件 hstore 升级方法

hstore 扩展插件通常随 KingbaseES 安装包一并升级。通常情况下用户无法单独升级插件。

3.31　http

3.31.1　插件 http 简介

插件 http 是 KingbaseES 的一个扩展插件。插件功能是提供数据库的 http 功能，允许在数据库内检索网页。

- 插件名为 http
- 插件版本 V1.4

备注:

http 插件不支持在 Windows 平台使用。

3.31.2　插件 http 加载方式

```
CREATE EXTENSION http;
```

3.31.3　插件 http 的参数配置

http 插件可以设置 http.keepalive 参数和 http.timeout_msec 参数。

3.31.3.1　http.keepalive 变量

功能描述

默认情况下，每个请求都使用一个全新的连接，并确保在完成请求后关闭该连接。当扩展程序在较长的时间段内运行时，设置 http.keepalive 为’on’的操作减少了消耗系统资源（套接字）的机会。

高性能应用程序可能希望启用保持活动状态和连接持久性，以减少延迟并提高吞吐量。该 GUC 变量更改了 http 扩展名的行为，以尽可能长时间地保持连接。

示例

```
set http.keepalive = 'on';
```

3.31.3.2　http.timeout_msec 变量

功能描述

默认情况下，http.timeout_msec 为 0。如果需要不同的超时时间，则可以使用该 GUC 变量以毫秒为单位进行设置。

示例

```
set http.timeout_msec = 200;
```

3.31.4　插件 http 使用方法

3.31.4.1　数据类型

3.31.4.1.1　UTL_HTTP.req

功能描述

使用此类型表示 HTTP 请求。

语法格式

```
CREATE TYPE UTL_HTTP.req AS(
    url           VARCHAR2(32767),
    method        VARCHAR2(64),
    http_version  VARCHAR2(64));
```

类型说明

参数	描述
url	http 请求的 URL
method	对 URL 所标识的资源执行的方法，目前仅支持 GET
http_version	兼容性参数，无意义

3.31.4.1.2　UTL_HTTP.resp

功能描述

使用此类型表示 HTTP 响应。

语法格式

```
CREATE TYPE UTL_HTTP.resp AS(
        status_code   INTEGER ,
        reason_phrase  VARCHAR2(256),
        http_version   VARCHAR2(64),
        content        TEXT);
```

类型说明

参数	描述
status_code	web 服务器返回的状态代码
reason_phrase	兼容性参数，无意义
http_version	兼容性参数，无意义
content	web 服务器返回的内容

3.31.4.1.3　http_method

功能描述

在 text 类型上增加检查 http 方法名的约束。

语法格式

```
CREATE DOMAIN http_method AS text
CHECK (
    VALUE ILIKE 'get' OR
    VALUE ILIKE 'post' OR
    VALUE ILIKE 'put' OR
    VALUE ILIKE 'delete' OR
    VALUE ILIKE 'patch' OR
    VALUE ILIKE 'head'
);
```

3.31.4.1.4　http_header

功能描述

表示一个 http 的头信息。

语法格式

```
CREATE TYPE http_header AS (
    field VARCHAR,
    value VARCHAR
);
```

类型说明

参数	描述
field	http 请求头的名字
value	http 请求头的值

3.31.4.1.5　http_response

功能描述

表示 http 返回的响应。

语法格式

```
CREATE TYPE http_response AS (
    status INTEGER,
    content_type VARCHAR,
    headers http_header[],
```

```
    content VARCHAR
);
```

类型说明

参数	描述
status	web 服务器返回的状态代码
content_type	web 服务器返回内容的类型
headers	web 服务器返回的 http 头信息
content	web 服务器返回的内容

3.31.4.1.6　http_request

功能描述

表示 http 请求。

语法格式

```
CREATE TYPE http_request AS (
    method http_method,
    uri VARCHAR,
    headers http_header[],
    content_type VARCHAR,
    content VARCHAR
);
```

类型说明

参数	描述
method	对 URL 所标识的资源执行的方法
uri	http 请求的 URL
headers	请求头信息
content_type	请求类型
content	请求内容

3.31.4.2　UTL_HTTP.BEGIN_REQUEST

功能描述

这个函数开始一个新的 HTTP 请求。

语法格式

```
UTL_HTTP.BEGIN_REQUEST (
    url               IN  VARCHAR2,
    method            IN  VARCHAR2 DEFAULT 'GET',
    http_version      IN  VARCHAR2 DEFAULT NULL,
    request_context   IN  VARCHAR2 DEFAULT NULL,
    https_host        IN  VARCHAR2 DEFAULT NULL)
RETURN UTL_HTTP.req
```

参数说明

参数	描述
url	http 请求的 URL
method	对 URL 所标识的资源执行的方法，目前仅支持 GET
http_version	兼容性参数，无意义
request_context	兼容性参数，无意义
https_host	兼容性参数，无意义

返回值说明

返回 HTTP 请求。

3.31.4.3　UTL_HTTP.SET_HEADER

功能描述

设置一个 HTTP 请求头。

语法格式

```
UTL_HTTP.SET_HEADER (
     r       IN OUT UTL_HTTP.req,
     name    IN VARCHAR2,
     value   IN VARCHAR2
```

参数说明

参数	描述
r	http 的请求
name	http 请求头的名字
value	http 请求头的值

3.31.4.4 UTL_HTTP.GET_RESPONSE

功能描述

此函数用于读取 http 响应，目前仅支持 get 功能。

语法格式

```
UTL_HTTP.GET_RESPONSE (
       r                       IN UTL_HTTP.req,
       return_info_response    IN BOOLEAN DEFAULT FALSE)
    RETURN UTL_HTTP.resp
```

参数说明

参数	描述
r	http 的请求
return_info_response	兼容性参数，无意义

返回值说明

返回 HTTP 响应。

3.31.4.5　UTL_HTTP.READ_LINE

功能描述

这个过程以文本形式读取 HTTP 响应体，直到到达行尾，并在调用者提供的缓冲区返回输出。如果到达 HTTP 响应正文的末尾，将引发 NO_DATA_FOUND 异常。

语法格式

```
UTL_HTTP.READ_LINE(
   r            IN OUT UTL_HTTP.resp,
   data         OUT VARCHAR2,
   remove_crlf  IN  BOOLEAN DEFAULT FALSE)
```

参数说明

参数	描述
r	http 的响应
data	http 的响应正文
remove_crlf	兼容性参数，无意义

3.31.4.6　UTL_HTTP.READ_TEXT

功能描述

这个过程以文本形式读取 HTTP 响应体，并在调用者提供的缓冲区返回输出。如果到达 HTTP 响应正文的末尾，将引发 NO_DATA_FOUND 异常。

语法格式

```
UTL_HTTP.READ_TEXT(
   r            IN OUT UTL_HTTP.resp,
   data         OUT VARCHAR2,
   len          IN INTEGER DEFAULT NULL)
```

参数说明

参数	描述
r	http 的响应
data	http 的响应正文

-续表-

参数	描述
len	要读取数据的最大字符数

备注:

如果 len 为 NULL, 这个过程将读取尽可能多的输入来填充数据中分配的字符串。

3.31.4.7　UTL_HTTP.END_RESPONSE

功能描述

此过程结束 http 响应。

语法格式

```
UTL_HTTP.END_RESPONSE ( r  IN OUT UTL_HTTP.resp)
```

参数说明

参数	描述
r	http 的响应

3.31.4.8　UTL_HTTP.END_REQUEST

功能描述

此过程结束 http 请求。

语法格式

```
UTL_HTTP.END_REQUEST ( r  IN OUT UTL_HTTP.req)
```

参数说明

参数	描述
r	http 的请求

3.31.4.9　http_header 函数

功能描述

组成一个简单数组返回。

语法格式

```
http_header(field VARCHAR, value VARCHAR) returns http_header
```

参数说明

参数名称	描述
field	指定名字
value	指定值

返回值说明

返回一个请求头。

示例

```
SELECT content::json->'headers'->>'Authorization' FROM http((
         'GET',
          'http://httpbin.org/headers',
          ARRAY[http_header('Authorization','Bearer eyJOeXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9')],
          NULL,
          NULL
       )::http_request);
                  content
-----------------------------------------------
Bearer eyJOeXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9
(1 row)
```

3.31.4.10　http 函数

功能描述

可以进行 get、post、put、delete、patch 功能。

语法格式

```
http(request http_request) returns http_response
```

参数说明

参数名称	描述
request	输入的请求

返回值说明

从网页获取的响应。

示例

```
SELECT status,
    content::json->'args' as args,
    content::json->>'data' as data,
    content::json->'url' as url,
    content::json->'method' as method
    from http(('GET', 'https://httpbin.org/anything', NULL, 'application/json', '{"search": "toto"}'));
```

3.31.4.11　http_get 函数

功能描述

请求指定的页面信息，并返回响应主体。

语法格式

```
http_get(uri VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串

返回值说明

从网页获取的响应。

示例

```
SELECT content FROM http_get('http://httpbin.org/ip');
           content
-----------------------------
 {"origin":"24.69.186.43"}                        +
(1 row)
```

3.31.4.12　http_post 函数

功能描述

该请求会向指定的资源提交数据，请求服务器处理，如表单数据提交、文件上传等。

语法格式

```
http_post(uri VARCHAR, content VARCHAR, content_type VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串
content	请求内容
content_type	请求类型

返回值说明

从网页获取的响应。

示例

```
SELECT status,
    content::json->'data' AS data,
    content::json->'args' AS args,
    content::json->'url' AS url,
    content::json->'method' AS method
    FROM http_post('https://httpbin.org/anything?foo=bar','payload','text/plain');
     status |    data     |        args        |                    url                    | method
```

```
--------+-----------+------------------+---------------------------------------+--------
    200 | "payload" | {                +| "https://httpbin.org/anything?foo=bar" | "POST"
        |           |     "foo": "bar"+|                                       |
        |           |   }              |                                       |
 (1 row)
```

3.31.4.13　http_put 函数

功能描述

put 请求会向指定资源位置上传其最新的内容，不能创建新的资源。

语法格式

```
http_put(uri VARCHAR, content VARCHAR, content_type VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串
content	请求内容
content_type	请求类型

返回值说明

从网页获取的响应。

示例

```
SELECT status, content_type, content::json->>'data' AS data
  FROM http_put('http://httpbin.org/put', 'some text', 'text/plain');
 status |   content_type   |   data
--------+------------------+-----------
    200 | application/json | some text
```

3.31.4.14　http_patch 函数

功能描述

patch 请求与 put 请求类似都是用于资源的更新。但二者有以下不同 patch 请求一般用于资源的部分更新，而 put 一般用于资源的整体更新。

语法格式

```
http_patch(uri VARCHAR, content VARCHAR, content_type VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串
content	请求内容
content_type	请求类型

返回值说明

从网页获取的响应。

示例

```
SELECT status,
    content::json->'data' AS data,
    content::json->'args' AS args,
    content::json->'url' AS url,
    content::json->'method' AS method
    FROM http_patch('https://httpbin.org/anything?foo=bar','{"this":"that"}','application/json');
     status |         data          |       args       |                  url                   | method
    --------+-----------------------+------------------+----------------------------------------+---------
        200 | "{\"this\":\"that\"}" | {               +| "https://httpbin.org/anything?foo=bar" | "PATCH"
            |                       |     "foo": "bar"+|                                        |
            |                       |   }              |                                        |
(1 row)
```

3.31.4.15　http_delete 函数

功能描述

删除资源。

语法格式

```
http_delete(uri VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串

返回值说明

从网页获取的响应。

示例

```
SELECT status,
content::json->'args' AS args,
content::json->'url' AS url,
content::json->'method' AS method
FROM http_delete('https://httpbin.org/anything?foo=bar');
 status |       args       |                  url                    |  method
--------+------------------+-----------------------------------------+----------
    200 | {               +| "https://httpbin.org/anything?foo=bar" | "DELETE"
        |     "foo": "bar"+|                                         |
        |   }              |                                         |
(1 row)
```

3.31.4.16　http_head 函数

功能描述

请求网页响应的头部信息。

语法格式

```
http_head(uri VARCHAR) returns http_response
```

参数说明

参数名称	描述
uri	标识互联网资源名称的字符串

返回值说明

从网页获取的响应。

示例

```
SELECT
    http.status,
    headers.value AS location
FROM
    http_head('http://google.com') AS http
    LEFT OUTER JOIN LATERAL (SELECT value
        FROM unnest(http.headers)
        WHERE field = 'Location') AS headers
        ON true;
 status |                         location
--------+-----------------------------------------------------------
    302 | http://www.google.ch/?gfe_rd=cr&ei=ACESWLy_KuvI8zeghL64Ag
```

3.31.4.17　http_set_curlopt 函数

功能描述

设置 curl 选项。

语法格式

```
http_set_curlopt(curlopt VARCHAR, value varchar) returns boolean
```

参数说明

参数名称	描述
curlopt	设置项
value	设置值

返回值说明

返回设置状态，为真则设置成功，为假则设置失败。

示例

```
-- Alter options and and reset them and throw errors
SELECT http_set_curlopt('CURLOPT_PROXY', '127.0.0.1');
 http_set_curlopt
------------------
 t
(1 row)
```

```
-- Error because proxy is not there
SELECT status FROM http_get('https://httpbin.org/status/555');
ERROR:  Failed to connect to 127.0.0.1 port 1080: Connection refused
-- Still an error
SELECT status FROM http_get('https://httpbin.org/status/555');
ERROR:  Failed to connect to 127.0.0.1 port 1080: Connection refused
```

3.31.4.18　http_reset_curlopt 函数

功能描述

重置 curl 选项。

语法格式

```
http_reset_curlopt() returns boolean
```

返回值说明

返回状态，为真则设置成功，为假则设置失败。

示例

```
SELECT http_reset_curlopt();
 http_reset_curlopt
--------------------
 t
(1 row)

-- Now it should work
SELECT status FROM http_get('https://httpbin.org/status/555');
 status
--------
    555
(1 row)
```

3.31.4.19　http_list_curlopt 函数

功能描述

打印已经设置的 curl 选项。

语法格式

```
http_list_curlopt() returns setof(curlopt text, value text)
```

返回值说明

返回已经设置的 curl 选项。

示例

```
SELECT http_set_curlopt('CURLOPT_PROXY', '127.0.0.1');
 http_set_curlopt
------------------
 t
(1 row)

select http_list_curlopt();
     http_list_curlopt
---------------------------
 (CURLOPT_PROXY,127.0.0.1)
```

3.31.4.20　urlencode 函数

功能描述

对包含任何“特殊”字符（实际上是 a-z 和 0-9 以外的任何其他字符）的内容进行 URL 编码。

语法格式

```
urlencode(string VARCHAR) returns text
```

参数说明

参数名称	描述
string	需要编码的字符串

返回值说明

返回已编码后的字符串。

示例

```
SELECT urlencode('my special string''s & things?');
              urlencode
-------------------------------------
 my+special+string%27s+%26+things%3F
(1 row)
```

3.31.5　插件 http 卸载方法

```
drop EXTENSION http;
```

3.31.6　插件 http 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.4 升级到 1.5：

```
ALTER EXTENSION http UPDATE TO '1.5';
```

3.32　identity_pwdexp

3.32.1　插件 identity_pwdexp 简介

identity_pwdexp 是 KingbaseES 的一个扩展插件，用于设置口令有效期。

KingbaseES 的用户管理中含有口令有效期这一属性，用户密码过期检查就是通过设置用户密码的有效期，在用户密码过期后限制用户登录数据库，并输入新密码的功能。

KingbaseES 通过插件的方式来进行用户密码过期检查。这种方式更为灵活，当数据库的实用场景需要进行用户密码过期检查时，加载插件即可。而不需要该功能时，卸载插件即可。

KingbaseES 中通过 1 个全局级参数配合插件来实现用户密码过期检查。

- 插件名为 identity_pwdexp
- 插件版本 V1.0

3.32.2　插件 identity_pwdexp 加载方法

在使用 identity_pwdexp 之前，需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'identity_pwdexp' # (change requires restart)
```

3.32.3　插件 identity_pwdexp 的参数配置

`identity_pwdexp.password_change_interval`

密码有效期，单位是天，取值范围为 [1，INT_MAX]，缺省为 30。

参数只能在 postmaster 启动或由安全管理员通过 SQL 语言 (alter 命令) 进行设置。

语句 Alter system set 参数名 = 参数值；

修改后再运行 select sys_reload_conf(); 不用重启服务器，对所有数据库及连接立即生效。

指定密码创建用户时，可通过 valid until 选项指定该用户的密码有效期，指定的密码有效期必须晚于当前时间且早于更换周期。`identity_pwdexp.password_change_interval` 指定的时间。

对已创建成功且已拥有密码的用户，也可通过 alter 语句的 valid until 选项修改其密码有效期，但仅安全管理员有这个权限，其它用户无法修改自己及他人的口令更换周期。

若在创建用户或修改用户密码时未显示的通过 valid until 选项指定该用户的密码有效期，那么系统会根据 `identity_pwdexp.password_change_interval` 参数设定的值自动为其计算密码有效期。

`identity_pwdexp.max_password_change_interval`

最大密码有效期，单位是天，取值范围为 [1，INT_MAX]，缺省为 30。

参数只能在 postmaster 启动或由安全管理员通过 SQL 语言 (alter 命令) 进行设置。

此参数用于限制密码有效期的设置范围，当设置的密码有效期大于最大密码有效期时，系统会报错提示。

3.32.3.1　示例

```
-- 创建扩展
\c test system
create extension identity_pwdexp;
CREATE EXTENSION

-- 设置密码有效期是 5 天
\c test sso
show identity_pwdexp.password_change_interval;
 identity_pwdexp.password_change_interval
-----------------------------------------
 30
(1 row)
alter system set identity_pwdexp.password_change_interval = 5;
ALTER SYSTEM
show identity_pwdexp.password_change_interval;
 identity_pwdexp.password_change_interval
-----------------------------------------
 30
(1 行记录)

select sys_reload_conf();
 sys_reload_conf
```

```
-----------------
 t
(1 行记录)

show identity_pwdexp.password_change_interval;
 identity_pwdexp.password_change_interval
------------------------------------------
 5
(1 行记录)

\c test system
call now();
             now
-------------------------------
 2020-04-30 15:34:30.408304+08
(1 行记录)

CREATE USER u_pwd_et PASSWORD '1234567890abC/.' VALID UNTIL '2020-05-01';
CREATE ROLE

SELECT USENAME, VALUNTIL FROM SYS_USER WHERE USENAME = 'u_pwd_et';
 usename  |        valuntil
----------+------------------------
 u_pwd_et | 2020-05-01 00:00:00+08
(1 行记录)

ALTER USER u_pwd_et PASSWORD '/.1234567890abC';
警告:  user "u_pwd_et" does not be locked
ALTER ROLE

SELECT USENAME, VALUNTIL FROM SYS_USER WHERE USENAME = 'u_pwd_et';
 usename  |           valuntil
----------+-------------------------------
 u_pwd_et | 2020-05-05 15:35:23.448381+08
(1 行记录)
```

3.32.4　插件 identity_pwdexp 使用方法

具体参考《KingbaseES 数据库安全指南》3.2.2.4 口令有效期设置章节。

3.32.5　插件 identity_pwdexp 的卸载插件

通过 drop extension 来卸载插件：

```
drop extension identity_pwdexp;
```

如果设置了 shared_preload_libraries，则需要删除‘identity_pwdexp’：

```
shared_preload_libraries = ''
```

3.32.6　插件 identity_pwdexp 升级方法

identity_pwdexp 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.33　intagg

3.33.1　插件 intagg 简介

intagg 模块提供了一个整数聚合器和一个枚举器，作为内置函数的兼容性封装器提供。

- 插件名为 intagg
- 插件版本 V1.0

3.33.2　插件 intagg 加载方式:

使用时需要 create extension intagg。

示例:

```
create extension intagg;
```

3.33.3　插件 intagg 参数配置

无需配置任何参数

3.33.4　插件 intagg 使用方法

3.33.4.1　函数

聚合器是一个生产正好包含输入的整数的整数数组的聚合函数 int_array_agg(integer)。这是 array_agg 的封装器，array_agg 对于任意数组类型做相同的事情。

枚举器是返回 setof integer 类型的函数 int_array_enum(integer[])。本质上是聚合器的反向操作：给出一个整数数组，将其展开为一组行。这是 unnest 的封装器，unnest 对于任意数组类型做相同的事情。

示例：

许多数据库系统有一到多个表的概念。这样的一个表通常位于两个索引表之间

例如:

```
CREATE TABLE left_table(id INT PRIMARY KEY, ...);

CREATE TABLE right_table(id INT PRIMARY KEY, ...);

CREATE TABLE one_to_many(l INT REFERENCES left_table, r INT REFERENCES right_table);
```

通常这样使用:

```
SELECT right_table.* from right_table JOIN one_to_many ON (right_table.id = one_to_many.r) WHERE one_to_
many.l = item;
```

这将返回所有在左手边的表里有记录的右手边表里的条目。这在 SQL 中是一个非常普通的构造。现在，这个方法在一个有非常大数量的记录的 one_to_many 表里是很难处理的。通常，像这样的连接将会导致索引扫描和抓取表中有左手边记录的每个右手边的记录。

如果你有一个非常动态的系统，那么没有什么你可以做的。不过，如果你有一些静态数据，你可以使用该聚合器创建一个汇总表。

```
CREATE TABLE summary AS SELECT l, int_array_aggregate(r) AS right_num

FROM one_to_many

GROUP BY l;
```

这将创建一个表，这个表有每个左边的条目和一个左边条目的数组。现在，如果没有使用该数组的方法则是相当无用的; 这就是为什么有一个数组枚举器。你可以：

```
SELECT l,int_array_enum(r) FROM summary WHERE l = item;
```

上面的查询使用 int_array_enum 产生下面相同的结果：

```
SELECT l , r FROM one_to_many WHERE l = item;
```

不同之处是针对 summary 表的查询必须只从表中获取一行，而针对 one_to_many 的直接查询必须索引扫描然后从每条记录中获取一行。

在一个系统上，一个显示了消耗 8488 的查询的 EXPLAIN 减少到消耗 329. 原始查询时一个包括 one_to_many 表的连接，替换为：

```
SELECT r, count(r) FROM

    ( SELECT l, int_array_enum(r) AS right_num

        FROM summary JOIN (SELECT l FROM left_table WHERE left_table.id = item) AS lefts

        ON (summary.l = lefts.l ) AS list

GROUP BY r

ORDER BY count DESC;
```

3.33.5　插件 intagg 卸载方法

不需要 intagg 插件只需要卸载插件即可。

示例：

```
drop extension intagg;
```

3.33.6　插件 intagg 升级方法

intagg 扩展插件通常随 KingbaseES 安装包一并升级。通常情况下用户无法单独升级插件。

3.34　intarray

3.34.1　插件 intarray 简介

intarray 模块为操作整数的 null-free 数组提供一些有用的函数和操作符。也支持使用其中的一些操作符执行索引搜索。如果提供的数组包含任何空元素，那么所有这些操作符都将抛出一个错误。这些操作符中的一些只对一维数组敏感。尽管他们将接受多个维数的输入数组，数组将按照存储的顺序当做一维数组。

- 插件名为 intarray
- 插件版本 V1.0

3.34.2　插件 intarray 加载方式

使用时需要 create extension intarray。

示例:

```
create extension intarray;
```

3.34.3　插件 intarray 参数配置

无需配置任何参数。

3.34.4　插件 intarray 使用方法

3.34.4.1　icount

计算数组中元素的个数

语法

icount(_int4) RETURNS int;

示例

```
TEST=# select icount('{1,2,3}'::int[]);
 icount
--------
      3
(1 行记录)
```

3.34.4.2　sort

对 int 数组进行排序。

语法

sort(int[]) returns int[]; sort(int[], text dir) returns int[];

参数

dir
如果 dir 是 asc，则对数组进行升序排序。如果 dir 是 desc，则对数组进行降序排序。

示例

```
TEST=# select sort('{2,1,5,7,3}');
    sort
-------------
 {1,2,3,5,7}
```

```
(1 行记录)

TEST=# select sort('{2,1,5,7,3}', 'asc');
    sort
-------------
 {1,2,3,5,7}
(1 行记录)

TEST=# select sort('{2,1,5,7,3}', 'desc');
    sort
-------------
 {7,5,3,2,1}
(1 行记录)
```

3.34.4.3　sort_asc

以升序的方式对数组进行排序。

语法

sort_asc(int[]) returns int[];

示例

```
TEST=# select sort_asc('{2,1,5,7,3}');
  sort_asc
-------------
 {1,2,3,5,7}
(1 行记录)
```

3.34.4.4　sort_desc

以降序的方式对 int 数组进行排序。

语法

sort_desc(int[]) returns int[];

示例

```
TEST=# select sort_desc('{2,1,5,7,3}');
  sort_desc
-------------
 {7,5,3,2,1}
(1 行记录)
```

3.34.4.5　uniq

消除相邻的相同的元素。

语法

uniq(int[]) returns int[];

示例

```
TEST=# select uniq('{2,1,5,7,7,7,3}');
    uniq
-------------
 {2,1,5,7,3}
(1 行记录)
```

3.34.4.6　idx

匹配 item 的第一个元素中的索引（如果没有则为 0）。

语法

idx(int[], int item) returns int;

参数

item
需要匹配的值。

示例

```
TEST=# select idx(array[1,23,5,2,5,2], 2);
 idx
-----
   4
(1 行记录)
```

3.34.4.7　subarray

在 start 位置开始的，len 个元素的数组的一部分。

语法

subarray(int[], int start) returns int[]; subarray(int[], int start, int len) returns int[];

参数

start
开始的位置。

len
指定子数组的长度。

示例

```
TEST=# select subarray('{1,2,3,2,1}'::int[], 2, 3);
 subarray
----------
 {2,3,2}
(1 行记录)
```

3.34.4.8　intset

制作单个元素的数组。

语法

intset(int) returns int[];

示例

```
TEST=# select intset(42);
 intset
--------
 {42}
(1 行记录)
```

3.34.4.9　intarray 模块中支持的操作符

&& @> @< # + - | & @@

3.34.5　插件 intarray 卸载方法

不需要 intarray 插件只需要卸载插件即可。

示例

```
drop extension intarray;
```

3.34.6　插件 intarray 升级方法

intarray 扩展插件通常随 KingbaseES 安装包一并升级。通常情况下用户无法单独升级插件。

3.35　isn

3.35.1　插件 isn 简介

插件 isn 为下列国际产品编号标准提供数据类型 EAN13、UPC、ISBN（图书）、ISMN（音乐）以及 ISSN（期刊）。

在输入时会按照一个硬编码的前缀列表对输入进行验证，这个前缀的列表也被用来在输出时连接号码。

因为新的前缀总是不时地出现，这个前缀列表可能会过时。

这个模块的一个未来版本有希望得到一个来自于一个或多个表的前缀列表，这样用户可以根据需要来方便地更新前缀列表。

不过，在当前该列表只能通过修改源代码并且重新编译来更新。

另外一种方案是，这个模块的未来版本中可能会直接移除前缀验证和连接支持。

- 插件名为 isn
- 插件版本 V1.2

3.35.2　插件 isn 加载方式

使用时需要在命令行执行 create extension isn;

3.35.3　插件 isn 的参数配置

无需配置任何参数

3.35.4　插件 isn 的使用方法

加载插件 isn 后，按照如下方式使用相应功能。

3.35.4.1　数据类型

如下表所示：

数据类型	描述
EAN13	欧洲文章号，总是以 EAN13 格式显示
ISBN13	国际标准图书号，以新的 EAN13 格式显示

-续表-

数据类型	描述
ISMN13	国际标准音乐号，以新的 EAN13 格式显示
ISSN13	国际标准期刊号，以新的 EAN13 格式显示
ISBN	国际标准图书号，以旧的短格式显示
ISMN	国际标准音乐号，以旧的短格式显示
ISSN	国际标准期刊号，以旧的短格式显示
UPC	通用产品代码

3.35.4.2　造型

isn 模块提供了下列类型之间的造型：

```
ISBN13 <=> EAN13
ISMN13 <=> EAN13
ISSN13 <=> EAN13
ISBN <=> EAN13
ISMN <=> EAN13
UPC <=> EAN13
ISBN <=> ISBN13
ISMN <=> ISMN13
ISSN <=> ISSN13
```

当从 EAN13 造型为另一种类型时，会有对该值是否在另一种类型的域中的运行时检查，如果不在则抛出一个错误。其他的造型则是简单地重新贴个标签，因而总是会成功。

3.35.4.3　函数和操作符

函数如下表所示：

函数	描述
述 is_weak(boolean) return boolean	设置弱输入模式（返回新设置）。
isn_weak() return boolean	得到弱模式的当前状态。

-续表-

函数	描述
make_valid(isn) return isn	验证一个非法号码（清除非法标志）。
is_valid(isn) return boolean	检查非法标志的存在。

弱模式被用来允许插入非法数据到一个表中。非法意味着校验位错误，而不是有丢失号码。

为什么你会想要使用弱模式？你可能有一个巨大的 ISBN 号码集合并且出于某种奇怪的原因其中具有错误的校验位。

当你使用弱模式在一个表中插入非法号码时，被插入的号码将会被加上修正过的校验位，但是它的最后将会有一个’!’。例如 0-11-000322-5！。这种非法标志符可以用 is_valid 函数检查并且可以用 make_valid 函数清除。

即使不在弱模式中，你也能通过在号码某位追加！字符来强制非法号码的插入。

另一个特殊特性是输入过程中，你可以写一个？代替校验位，然后正确的校验位将被自动插入。

3.35.4.4　示例

```
--直接使用类型
select isbn('978-0-393-04002-9');
select isbn13('0901690546');
select issn('1436-4522');

--转换类型
select upc(ean13('022-035648348-1'));
select ean13(upc('220356483481'));

--创建一个表，它有一个单一列来保存 ISBN 号码
create table test(id isbn);
INSERT INTO test VALUES('9780393040029');

--自动计算校验位
INSERT INTO test VALUES('220500896?');
INSERT INTO test VALUES('978055215372?');

select issn('3251231?');
select ismn('979047213542?');

--使用弱模式
SELECT isn_weak(true);
```

```
INSERT INTO test VALUES('978-0-11-000533-4');
insert into test values('2-205-00876-X');
select isn_weak(false);

select id from test where not is_valid(id);
update test set id = make_valid(id) where id = '2-205-00876-X!';

select * from test;

select isbn13(id) from test;
```

3.35.5　插件 isn 卸载方法

不需要使用插件时执行指令 drop extension isn 即可。

3.35.6　插件 isn 升级方法

isn 扩展插件通常随 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.36　kbcrypto

3.36.1　插件 kbcrypto 简介

kbcrypto 是 KingbaseES 的一个扩展插件，可以提供如 rc4,sm3,sm4,sm2 等加密函数的访问，可以提供明文的加密打印输出到前端。加密函数功能详情还可参见《KingbaseES 安全指南》中的 数据加密数据章节。

- 插件名为 kbcrypto
- 插件版本 V1.3

3.36.2　插件 kbcrypto 加载方式

KingbaseES 数据库默认将它添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

```
shared_preload_libraries = 'kbcrypto' # (change requires restart)
```

再使用 create extension 命令进行创建扩展：

```
create extension kbcrypto;
```

3.36.3　插件 kbcrypto 的参数配置

无需配置任何参数。

3.36.4　插件 kbcrypto 的使用方法

加载 kbcrypto 插件后，程序可以提供如 encrypt/decrypt，rc4，digest，sm3，sm4，sm2 等加密函数的访问，可以提供明文的加密打印输出到前端。

提示:

解密之后的结果是 bytea 类型，要想转换成可视字符，需要使用 convert，set bytea_output = hex 等方式进行编码，否则可能会是乱码。

3.36.4.1　decrypt 函数

功能

根据指定的密钥 key 和加解密算法类型，对数据进行解密，并返回解密结果。

语法

```
FUNCTION decrypt(cipher bytea, key bytea, type text) RETURNS bytea
```

参数说明

- cipher bytea：待解密的密文数据
- key bytea：解密密钥
- type　text：加解密算法类型，支持的加解密算法有：bf-cbc,bf-ecb,bf-cfb,des-ecb,des-cbc,des3-ecb,des3-cbc,cast5-ecb,cast5-cbc,aes-ecb,aes-cbc。另外新增了两种加解密算法：mysql.aes_encrypt/mysql.aes_decrypt，mysql.encode/mysql.decode 以支持 MySQL 中的加解密函数 aes_encrypt/aes_decrypt，encode/decode。

返回类型

bytea

示例

```
select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789','des'), '123456789','des');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789','aes'), '123456789','aes');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', '3des'), '123456789', '3des');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'blowfish'), '123456789', 'blowfish');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'bf'), '123456789', 'bf');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'mysql.aes_encrypt'), '123456789', 'mysql.aes_
encrypt');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'mysql.encode'), '123456789', 'mysql.encode');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'aes-ecb/pad:none'), '123456789', 'aes-ecb/pad:
none');
```

```
                                decrypt
------------------------------------------------------------------------
 qwertyuiopasdfghjkl\000\000\000\000\000\000\000\000\000\000\000\000\000
(1 row)

select decrypt(encrypt('qwertyuiopasdfghjkl', '123456789', 'aes-ecb/pad:pkcs'), '123456789', 'aes-ecb/pad:
pkcs');
       decrypt
---------------------
 qwertyuiopasdfghjkl
(1 row)
```

3.36.4.2　digest 函数

功能

根据指定的摘要算法类型，对数据进行摘要计算，并返回结果。

语法

```
FUNCTION digest(data bytea, type text) RETURNS bytea
FUNCTION digest(data text, type text) RETURNS bytea
```

参数说明

- data bytea/text：进行摘要计算的数据
- type text：支持 md5,sha1,sha224,sha256,sha384,sha512 算法

返回类型

bytea

示例

1、摘要数据为 bytea 类型：

```
set bytea_output to hex;
set ora_input_emptystr_isnull to false;
select digest('12345678','md5');
               digest
------------------------------------
 \x25d55ad283aa400af464c76d713c07ad
(1 row)

select digest('12345678','sha1');
                   digest
--------------------------------------------
```

```
 \x7c222fb2927d828af22f592134e8932480637c0d
(1 row)

select digest('12345678','sha224');
                           digest
------------------------------------------------------------
 \x7e6a4309ddf6e8866679f61ace4f621b0e3455ebac2e831a60f13cd1
(1 row)

select digest('12345678','sha256');
                               digest
--------------------------------------------------------------------
 \xef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f
(1 row)

select digest('12345678','sha384');
                                                digest
------------------------------------------------------------------------------------------------------
 \x8cafed2235386cc5855e75f0d34f103ccc183912e5f02446b77c66539f776e4bf2bf87339b4518a7cb1c2441c568b0f8
(1 row)

select digest('12345678','sha512');
                                                                    digest
------------------------------------------------------------------------------------------------------------------------
-------------------------
 \
xfa585d89c851dd338a70dcf535aa2a92fee7836dd6aff1226583e88e0996293f16bc009c652826e0fc5c706695a03cddce372f139eff4d13959da6
(1 row)
```

2、摘要数据为 text 类型：

```
set bytea_output = 'hex';
TEST=# select digest('12345678'::text,'sha256');
                               digest
--------------------------------------------------------------------
 \xef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f
(1 row)

TEST=# select digest('12345678'::text,'md5');
               digest
------------------------------------
 \x25d55ad283aa400af464c76d713c07ad
(1 row)

TEST=# select digest('12345678'::text,'sha224');
```

```
                           digest
------------------------------------------------------------
 \x7e6a4309ddf6e8866679f61ace4f621b0e3455ebac2e831a60f13cd1
(1 row)

TEST=# select digest('12345678'::text,'sha256');
                                digest
--------------------------------------------------------------------
 \xef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f
(1 row)

TEST=# select digest('12345678'::text,'sha384');
                                                digest
------------------------------------------------------------------------------------------------------
 \x8cafed2235386cc5855e75f0d34f103ccc183912e5f02446b77c66539f776e4bf2bf87339b4518a7cb1c2441c568b0f8
(1 row)

TEST=# select digest('12345678'::text,'sha512');
                                                                    digest
------------------------------------------------------------------------------------------------------------------------
-----------------------
 \
xfa585d89c851dd338a70dcf535aa2a92fee7836dd6aff1226583e88e0996293f16bc009c652826e0fc5c706695a03cddce372f139eff4d13959da6
(1 row)
```

3.36.4.3　encrypt 函数

功能

根据指定的密钥 key 和加解密算法类型，对数据进行加密，并返回加密结果。

语法

```
FUNCTION encrypt(plain bytea, key bytea, type text) RETURNS bytea
```

参数说明

- plain bytea：待加密的明文数据
- key bytea：加密密钥
- type　text：加解密算法类型，支持的加解密算法有：bf-cbc,bf-ecb,bf-cfb,des-ecb,des-cbc,des3-ecb,des3-cbc,cast5-ecb,cast5-cbc,aes-ecb,aes-cbc。另外新增了两种加解密算法：mysql.aes_encrypt/mysql.aes_decrypt，mysql.encode/mysql.decode 以支持 MySQL 中的加解密函数 aes_encrypt/aes_decrypt，encode/decode。

返回类型

bytea

示例

```
set bytea_output to escape;
set ora_input_emptystr_isnull to false;
select encrypt('qwertyuiopasdfghjkl', '123456789', 'des');
                                 encrypt
-----------------------------------------------------------------------
 q\320]DYGs\260\335 \224\373\346\363\006\206\033\311F\205\240\215\303\372
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'aes');
                                      encrypt
-----------------------------------------------------------------------------------------
 8]>xf\326X\\\253;*B\3128m\310\006\305\344\276,\224\322E\354\201:\346\332\222\344\271
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', '3des');
                                   encrypt
-----------------------------------------------------------------------------
 \240\255m\346\036\376\204!\332\370\337F~\377\232\213\312\3069\034PD\022\305
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'blowfish');
                                 encrypt
-----------------------------------------------------------------------
 \353P\313\015\371\202|7zH\253\364\337\220\223\002\245T\235^v\226\342\252
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'bf');
                                 encrypt
-----------------------------------------------------------------------
 \353P\313\015\371\202|7zH\253\364\337\220\223\002\245T\235^v\226\342\252
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'mysql.aes_encrypt');
                                    encrypt
-------------------------------------------------------------------------------------
 8]>xf\326X\\\253;*B\3128m\310\331\331\333$X\314w\027\275\256]b\014\276\236\233
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'mysql.encode');
                           encrypt
-----------------------------------------------------------
 \177H\201F\253\357\317_\241w\272\301k\016\007\342$\320\351
```

```
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'aes-ecb/pad:none');
                                 encrypt
-------------------------------------------------------------------------
 8]>xf\326X\\\253;*B\3128m\310a\371\221;\226cK\023]<\015\225X\226\326\210
(1 row)

select encrypt('qwertyuiopasdfghjkl', '123456789', 'aes-ecb/pad:pkcs');
                                    encrypt
-----------------------------------------------------------------------------
 8]>xf\326X\\\253;*B\3128m\310\331\331\333$X\314w\027\275\256]b\014\276\236\233
(1 row)
```

3.36.4.4　encrypt_iv 函数

功能

根据指定的密钥 key 和加解密算法类型，对数据进行加密，并返回加密结果。

语法

```
FUNCTION encrypt_iv(plain bytea, key bytea, iv bytea, type text) RETURNS bytea
```

参数说明

- plain bytea：待加密的明文数据
- key bytea：加密密钥
- iv bytea：加密向量
- type　text：加解密算法类型，支持的加解密算法有：bf-cbc,bf-ecb,bf-cfb,des-ecb,des-cbc,des3-ecb,des3-cbc,cast5-ecb,cast5-cbc,aes-ecb,aes-cbc。

返回类型

bytea

示例

```
select encrypt_iv('qwertyuiopasdfghjkl', '123456789', '123456','aes');
                              encrypt_iv
----------------------------------------------------------------------
 \x1a8062925f728febf21949c8a1e585c34b379acaafa12f3902bc03e26e8697d4
(1 row)
```

3.36.4.5　decrypt_iv 函数

功能

根据指定的密钥 key 和加解密算法类型，对数据进行解密，并返回解密结果。

语法

```
FUNCTION decrypt_iv(cipher bytea, key bytea, iv bytea, type text) RETURNS bytea
```

参数说明

- cipher bytea：待解密的密文数据
- key bytea：解密密钥
- iv bytea：解密向量
- type　text：加解密算法类型，支持的加解密算法有：bf-cbc,bf-ecb,bf-cfb,des-ecb,des-cbc,des3-ecb,des3-cbc,cast5-ecb,cast5-cbc,aes-ecb,aes-cbc。

返回类型

bytea

示例

```
set bytea_output = 'escape';
select decrypt_iv(encrypt_iv('qwertyuiopasdfghjkl', '123456789', '123456','aes'), '123456789', '123456','aes
');
     decrypt_iv
---------------------
 qwertyuiopasdfghjkl
(1 row)
```

3.36.4.6　rc4 函数

功能

根据指定的密钥 key 和 rc4 加解密算法，对数据进行加密/解密，并返回加密/解密结果。

语法

```
FUNCTION rc4(data bytea, key bytea, flag int4) RETURNS bytea
```

参数说明

- data bytea：待加密/解密的数据
- key bytea：加密解/密密钥
- flag int4：加密解密标志

其中：

- 0-传统的加密方式加密；
- 1-传统的解密方式解密；
- 2-通用的加密方式加密；
- 3-通用的解密方式解密。

```
传统的加密方式，会对密钥进行限制，最大长度为 16 字节，超过会被截取。

通用的加密方式，不会限制密钥长度。
```

返回类型

bytea

示例

```
select rc4('1234qwer', '123456789qwer', 0);
        rc4
--------------------
 \x360c182ffd952552
(1 row)

select rc4('123456789', '123456789', 2);
         rc4
-----------------------
 \x05fb691574615fe1d8
(1 row)
select rc4(rc4('1234qwer', '123456789qwer', 0), '123456789qwer', 1);
        rc4
--------------------
 \x3132333471776572
(1 row)

select rc4(rc4('123456789', '123456789', 2), '123456789', 3);
         rc4
------------------------
 \x313233343536373839
(1 row)
```

3.36.4.7　sm3 函数

3.36.4.7.1　sm3 函数

功能

实现 sm3 的加密。

语法

```
sm3(text)
```

参数说明

加密数据。

示例

```
SELECT sm3('123456789');
                                sm3
------------------------------------------------------------------
 c7ae0aec3d2f9beb84dc1885aa7a576baa7a07b38060afc64c5600f93a5456b5
```

3.36.4.8　sm4 函数

3.36.4.8.1　sm4 函数

功能

实现 sm4 的加密、解密。

语法

```
sm4(text,text,int)
```

参数说明

加/解密数据、密钥、加密/解密标识。标识为 0 代表加密，1 代表解密。

示例

```
-- 加密
SELECT sm4('123456abcdef','0123456789ABCDEF',0);

-- 解密
SELECT sm4(sm4('123456abcdef','0123456789ABCDEF',0), '0123456789ABCDEF',1);
```

3.36.4.8.2　sm4_ex 函数

功能

实现国密 sm4 的加解密函数，本函数与上个章节中 sm4 函数的区别，在于本函数可以选择填充模式。如果原文长度不是 16 字节的整数倍，则可以通过填充模式知晓具体填充了多少字节，故便于某些有特殊需求的场景使用。

语法

```
sm4_ex(text,text,int,int)
```

参数说明

加/解密数据、密钥、加密/解密标识。标识为 0 代表加密，1 代表解密。加/解密数据、密钥、加密/解密标识、填充模式。填充模式如下：

1）填充模式选 1, 数据按 16 字节倍数强制填充，缺 m 个字节则填充 m 个字节的 m 值 (m 最大值为 16)。

2）填充模式选 0，数据按 16 字节倍数非强制填充 0x0, 同 sm4。

示例

```
-- 1）填充模式选 1
-- 加密
SELECT sm4_ex('123456abcdef','0123456789ABCDEF',0,1);
-- 解密
SELECT sm4_ex(sm4_ex('123456abcdef','0123456789ABCDEF',0,1), '0123456789ABCDEF',1,1);

-- 2）填充模式选 0
-- 加密
SELECT sm4_ex('123456abcdef','0123456789ABCDEF',0,0);
-- 解密
SELECT sm4_ex(sm4_ex('123456abcdef','0123456789ABCDEF',0,0), '0123456789ABCDEF',1,0);
```

3.36.4.9　sm2 函数

3.36.4.9.1　sm2_genkeypair 函数

功能

用于生成 sm2 密钥对, 包含公钥和私钥。

语法

```
sm2_genkeypair()
```

示例

```
SELECT sm2_genkeypair();

                          sm2_genkeypair
-----------------------------------------------------------------
 -----BEGIN EC PRIVATE KEY-----                                  +
 MHcCAQEEIE9Gm3HKpfHNq+tKEbEbPNbJ2g3CRffK2DrORokpwwepoAoGCCqGSM49+
 AwEHoUQDQgAE/eyOgN64tpxIjGmcSX3WtMXEDOMRwL6EEqEctRlQMeOzGkGA/TsM+
 sE6nbdqy/AdlazI4xnfQNJ/TzYCA1P3n5w==                            +
```

```
-----END EC PRIVATE KEY-----                                    +
-----BEGIN PUBLIC KEY-----                                      +
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE/eyOgN64tpxIjGmcSX3WtMXED0MR+
wL6EEqEctRlQMeOzGkGA/TsMsE6nbdqy/AdlazI4xnfQNJ/TzYCA1P3n5w==    +
-----END PUBLIC KEY-----                                        +
```

3.36.4.9.2　sm2_encrypt 函数

功能

实现基于 sm2 的加密，加密使用公钥。

语法

```
sm2_encrypt(text, text)
```

参数说明

第一个参数是明文，第二个参数是公钥。

示例

明文设置为“hello^”。

```
SELECT sm2_encrypt('hello^', 'MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE/
eyOgN64tpxIjGmcSX3WtMXED0MRwL6EEqEctRlQMeOzGkGA/TsMsE6nbdqy/AdlazI4xnfQNJ/TzYCA1P3n5w==');

                    sm2_encrypt
-------------------------------------------------------
306f02200e467e63de5937b645efc7110f58be9bec16e769291352+
04c8fd49b44aafbe0f022100c961b9420abeb74a7a149f3b3603c2+
5bcea9482285deca98c28ae4e8c3c7471e0420f86602b170334b55+
d6e1e62e7c0299886d9b9516eaec3745d91450ed19702eec04068e+
a88764ef26                                             +
```

3.36.4.9.3　sm2_decrypt 函数

功能

实现基于 sm2 的解密，解密使用私钥。

语法

```
sm2_decrypt(text, text)
```

参数说明

第一个参数为密文，第二个参数为私钥。

示例

解密出明文结果为“hello^”。

```
SELECT sm2_decrypt(
'306f02200e467e63de5937b645efc7110f58be9bec16e76929135204c8fd49b44aafbe0f022100c961b9420abeb74a7a149f3b3603c25bcea94822
', 'MHcCAQEEIE9Gm3HKpfHNq+tKEbEbPNbJ2g3CRffK2DrORokpwwepoAoGCCqGSM49AwEHoUQDQgAE/
eyOgN64tpxIjGmcSX3WtMXEDOMRwL6EEqEctRlQMeOzGkGA/TsMsE6nbdqy/AdlazI4xnfQNJ/TzYCA1P3n5w==');

     sm2_decrypt
---------------------
 hello^
```

3.36.4.9.4　sm2_sign 函数

功能

实现基于 sm2 的签名，签名使用私钥。

语法

```
sm2_sign(text, text)
```

参数说明

第一个参数为待签名内容，第二个参数为私钥。

示例

待签名内容例如设置为“hello sign^”。

```
SELECT sm2_sign('hello sign^',
'MHcCAQEEIE9Gm3HKpfHNq+tKEbEbPNbJ2g3CRffK2DrORokpwwepoAoGCCqGSM49AwEHoUQDQgAE/
eyOgN64tpxIjGmcSX3WtMXEDOMRwL6EEqEctRlQMeOzGkGA/TsMsE6nbdqy/AdlazI4xnfQNJ/TzYCA1P3n5w==');

                    sm2_sign
-------------------------------------------------------
304502205314636312faba3b0661fbfe0f4076dd2068b8e7b0f1c6+
9fdea67b642927272e02210093dfaf70db2c8b293f235d2176747b+
64eba44f5829c20f05d9dac7d0c1f02a50                     +
```

3.36.4.9.5　sm2_verify 函数

功能

实现基于 sm2 的验签，验签使用公钥。

语法

```
sm2_verify(text, text, text)
```

参数说明

第一个参数为待签名内容，第二个参数为签名后内容，第三个参数为公钥。

返回值：函数执行后，会产生两种结果 t（true）或者 f（false），分别代表验签成功和验签失败。

示例

验签结果成功。

```
SELECT sm2_verify('hello sign^',
'304502205314636312faba3b0661fbfe0f4076dd2068b8e7b0f1c69fdea67b642927272e02210093dfaf70db2c8b293f235d2176747b64eba44f58
', 'MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE/eyOgN64tpxIjGmcSX3WtMXED0MRwL6EEqEctRlQMeOzGkGA/TsMsE6nbdqy/
AdlazI4xnfQNJ/TzYCA1P3n5w==');

    sm2_verify
------------------
t
```

3.36.5　插件 kbcrypto 卸载方法

通过 drop extension 来卸载插件：

```
drop extension kbcrypto;
```

如果设置了 shared_preload_libraries，则需要删除‘kbcrypto’：

```
shared_preload_libraries = ''
```

3.36.6　插件 kbcrypto 升级方法

kbcrypto 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.37　kbrowlocks

3.37.1　插件 kbrowlocks 简介

kbrowlocks 提供了一个函数来显示指定表的行级锁信息。默认仅限超级用户、sys_stat_scan_tables 角色的成员和在该表上拥有 SELECT 权限的用户使用。

- 插件名为 kbrowlocks
- 插件版本 V1.2

3.37.2　插件 kbrowlocks 加载方式

```
CREATE EXTENSION kbrowlocks;
```

3.37.3　插件 kbrowlocks 的参数配置

无需配置任何参数。

3.37.4　插件 kbrowlocks 的使用方法

表 3.37.1: kbrowlocks 输出列

名称	类型	描述
locked_row	tid	该行的 tid
locker	xid	持有锁的 xid 号 (可能是 multi xid 号)
multi	boolean	locker 是否是 multi xid
xids	xid[]	持锁的实际 xid
modes	text[]	持锁的模式
pids	integer[]	持锁 backend 的 pid

kbrowlocks 会为目标表加 AccessShareLock 并且逐行读取来收集锁信息，对于大表可能很慢。

备注:

1. 如果表被加了排他锁，kbrowlocks 将被阻塞。
2. kbrowlocks 没有用一个一致的快照扫描。在执行过程中，数据可能被加上新行锁，也可能有旧行锁被释放。kbrowlocks 不显示被锁定行的内容。如果想同时查看行内容，可以这样做

```
CREATE TABLE  test_rowlock(a int);
INSERT INTO test_rowlock VALUES (1);
```

```
BEGIN;
SELECT * FROM test_rowlock for share;
 a
---
 1
SELECT * FROM kbrowlocks('test_rowlock');
locked_row | locker | multi | xids  |     modes     |  pids
------------+--------+-------+-------+---------------+---------
(0,1)      |    935 | f     | {935} | {"For Share"} | {11273}
COMMIT;
```

但要注意，这样的查询将非常低效。

3.37.5　插件 kbrowlocks 卸载方法

```
DROP EXTENSION kbrowlocks;
```

3.37.6　插件 kbrowlocks 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1

```
ALTER EXTENSION kbrowlocks UPDATE TO '1.1';
```

3.38　kbstattuple

3.38.1　插件 kbstattuple 简介

kbstattuple 提供了多种函数来获得关系中元组的统计信息。默认情况下，只有角色 sys_stat_scan_tables 的成员和超级用户可以使用。

- 插件名为 kbstattuple
- 插件版本 V1.6

3.38.2　插件 kbstattuple 加载方式

```
CREATE EXTENSION kbstattuple;
```

3.38.3　插件 kbstattuple 的参数配置

无需配置任何参数。

3.38.4　插件 kbstattuple 的使用方法

kbstattuple 提供以下函数：

`kbstattuple(regclass) returns record`
返回给定关系的物理长度、“死亡”元组百分比等信息。这些信息可以帮助用户决定是否要对关系做清理。

表 3.38.1: kbstattuple 输出列

名称	类型	描述
table_len	bigint	关系大小，单位字节
tuple_count	bigint	“活着”的元组数
tuple_len	bigint	“活着”的元组总大小，单位字节
tuple_percent	float8	”活着”的元组大小占关系大小百分比
dead_tuple_count	bigint	“死亡”的元组数
dead_tuple_len	bigint	“死亡”的元组总大小，单位字节
dead_tuple_percent	float8	”死亡”的元组大小占关系大小百分比
free_space	bigint	空闲空间大小，单位字节
free_percent	float8	空闲空间大小占关系大小百分比

`kbstattuple(text) returns record`
和 kbstattuple(regclass) 相同，区别是通过字符串指定关系。

`kbstatindex(regclass) returns record`
返回 B 树索引的相关信息。

表 3.38.2: kbstatindex 输出列

名称	类型	描述
version	integer	B 树页面类型版本号
tree_level	integer	root 页面的高度
index_size	bigint	索引大小，单位字节
root_block_no	bigint	root 页面的块号
internal_pages	bigint	中间页面的数量
leaf_pages	bigint	叶子页面的数量
empty_pages	bigint	空页面的数量
deleted_pages	bigint	被删除页面的数量
avg_leaf_density	float8	叶子页面的平均填充度
leaf_fragmentation	float8	叶子页面的碎片化程度

`kbstatindex(text) returns record`
和 kbstatindex(regclass) 相同，区别是通过字符串指定关系。

`kbstatginindex(regclass) returns record`
返回 GIN 索引的相关信息。

表 3.38.3: kbstatginindex 输出列

名称	类型	描述
version	integer	GIN 索引页面类型版本号
pending_pages	integer	pending 列表中的页面数
pending_tuples	bigint	pending 列表中的元组数

`kbstathashindex(regclass) returns record`
返回 HASH 索引的相关信息。

表 3.38.4: kbstathashindex 输出列

名称	类型	描述
version	integer	HASH 索引页面类型版本号
bucket_pages	bigint	桶页面的页面数
overflow_pages	bigint	溢出页的页面数
bitmap_pages	bigint	位图页的页面数
unused_pages	bigint	未使用页面的页面数
live_items	bigint	可用元组数
dead_tuples	bigint	无效元组数
free_percent	float	空闲空间占比

`sys_relpages(regclass) returns bigint`
返回关系的页面数。

`sys_relpages(text) returns bigint`
和 sys_relpages(regclass) 相同，区别是通过字符串指定关系。

`kbstattuple_approx(regclass) returns record`
kbstattuple_approx 是 kbstattuple 的一个更快速的替代，它返回近似的结果。这个函数通过 VM 页面跳过标记了“全部可见”的页面扫描，并从 FSM 页面中获取这类页面的空闲空间大小并假设剩余的空间都有“活着的”元组使用。对于不能被跳过的页面，函数会扫描每个元组并进行统计。最后，它会基于已扫描的页面元组数量来估计“活着”的元组总数。除对应 kbstattuple 的列外，有一个 float8 类型的列 scanned_percent 显示扫描的页面占比。

3.38.5　插件 kbstattuple 卸载方法

```
DROP EXTENSION kbstattuple;
```

3.38.6　插件 kbstattuple 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1

```
ALTER EXTENSION kbstattuple UPDATE TO '1.1';
```

3.39 kdb_database_link

3.39.1 插件 Database Link 简介

Database Link 简称 DBLink，是数据库管理系统提供的用于访问外部数据库对象的机制。用户可以通过 DBLink 来访问外部数据库的表、视图对象。DBLink 实现了两个数据库之间的通信。DBLink 包含一个基于网络的数据连接以及用于登陆远程数据库的用户名、密码信息。本地数据库系统可以通过 DBLink 建立与远程数据库之间的会话，从而完成对远程数据库对象的访问。DBLink 实现了 SQL/MED 标准中定义的对外部数据源进行访问的部分功能。

- 插件名为 kdb_database_link
- 插件版本 V1.0.0

备注:

SQL/MED 是 SQL Management of External Data 的缩写，是 ISO/IEC 9075-9:2003 标准中对 SQL 语言的扩展。SQL/MED 标准规定了如何通过外部数据封装器（Foreign Data Wrapper）和数据连接（datalink）实现对外部数据的访问。这里的外部数据指的是可以被基于 SQL 的 DBMS 访问的数据。

---

dblink 对象是一个数据库内部对象，可以通过 dblink 对象连接引用其他数据库的对象。支持连接 Oracle、KingbaseES、Postgresql、SQLServer 的 dblink 对象。

相关定义如下：

- 本地数据库当前数据库客户端所连接到的数据库，可以直接访问和操作在此数据库内拥有相应权限的对象，客户端持有该连接。
- 远程数据库通过定义于本地数据库内的 dblink 访问的数据库，虽然客户端看起来可以直接访问远程数据库的对象，实际上此类连接由本地数据库持有，对象定义等系统信息在本地数据库并不存在。远程数据库也可能只是本地数据库通过设置连接到了自己的另一个服务进程或者线程。
- 远程连接通过 dblink 访问远程数据库对象时创建的数据库连接，对于目标数据库，本地数据库是它为之提供服务的客户端。本地数据库对自己创建和拥有的远程连接，需要进行合理的管理，在必要时应及时关闭，防止过多的资源开销。
- 远程表（视图、物化视图）存在于远程数据库上的表，也可能是视图或者物化视图，本地数据库对此并不关心，只关心所取得的数据结构定义和数据本身。
- 远程序列存在于远程数据库上的序列发生器，通常用于统一 id 的生成。
- 远程函数（存储过程）存在于远程数据库上的函数或者存储过程，一般远程调用函数的场景居多。
- 远程同义词存在于本地数据库指向远程对象（表、序列、函数等等）的同义词。

3.39.1.1 Database Link 功能

Dblink 功能主要是为了满足常见语法的适配，让用户应用的代码能够适用于更宽泛的产品而无需在移植时大量修改

- 支持连接管理，在适当的时候关闭连接减少远程数据库的资源开销。
- 支持远程表（视图、物化视图）的查询，并且支持下推查询条件减少数据传输的网络开销。
- 支持远程表的插入。
- 支持远程序列的访问。
- 支持远程用户自定义函数 (存储过程) 的访问。目前仅支持 KingbaseES 数据库。

3.39.2 插件 kdb_database_link 的加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

```
shared_preload_libraries = 'kdb_database_link'
```

在 ksql 中，运行

```
CREATE EXTENSION kdb_database_link;
```

备注:

如果是 windows 下运行 KingbaseES，且远程库是 Oracle 时，需要安装 oracle 的客户端库，并且把安装后的目录添加到 PATH 环境变量下。

3.39.3 插件 kdb_database_link 的参数配置

- DriverName

  连接驱动名称。

- Host

  远程数据库网络地址。

- Port

  远程数据库服务端口。

- Dbname

  远程数据库名称。

- DbType

远程数据库类型。DbType 支持 Oracle，KingbaseES，Postgres，ODBC 数据库类型，其中 ODBC 用来连接远程 SQLServer 数据库。

3.39.4　插件 kdb_database_link 的使用方法

3.39.4.1　创建 Database Link 对象

在创建 dblink 对象时指定连接信息，包括远程数据库网络地址、端口、数据库名称、用户名和密码，具体需要提供哪些信息会因为不同数据库存在差异。

语法格式

```
CREATE [ PUBLIC ] DATABASE LINK dblink
       CONNECT TO user IDENTIFIED BY password USING
       {(connect_string) | config_tag };
```

参数说明

`PUBLIC`
指定 PUBLIC 创建公有 dblink 对象，所有用户都可以访问公有 dblink 对象。公有 dblink 是创建在 PUBLIC 模式下面的。未指定 PUBLIC 时创建私有 dblink，不可在 PUBLIC 模式下创建私有 dblink。

dblink 指定要创建的数据库链接的名称，同一个模式下的 dblink 不能重名。

`user IDENTIFIED BY password`
指定远端数据库的用户名和密码。

`connect_string`
配置连接串信息，`connect_string` 用来提供用于连接到远程数据库的信息。

`config_tag`
配置文件标签名, 用于指明配置文件（sys_database_link.conf）中的一项, 从而通过配置文件获取用于建立数据库连接的远程数据库的网络地址、端口以及数据库名称。

注解

1、在配置文件中的格式如下

[名称]

DriverName= 连接驱动名称。

Host= 远程数据库网络地址。

Port= 远程数据库服务端口。

Dbname= 远程数据库名称。

DbType= 远程数据库类型。

DbType 支持 Oracle，KingbaseES，Postgres，ODBC 数据库类型。

2、使用该功能时，需将 kdb_database_link 加入 shared_preload_libraries 中。

3、查询外部数据库时，需创建对应的数据库插件，如:kingbase_fdw、oracle_fdw、postgres_fdw、odbc_fdw。

示例

创建一个到 Oracle 数据库的数据库连接，它可以被所有数据库用户使用

```
CREATE PUBLIC DATABASE LINK mylink CONNECT TO 'SYSTEM' IDENTIFIED BY 'password' USING 'ORADB';
```

创建一个到 Oracle 数据库的数据库连接，直接指定连接串信息

```
CREATE PUBLIC DATABASE LINK mylink CONNECT TO 'SYSTEM' IDENTIFIED BY 'password' USING (DriverName='Oracle
ODBC Driver', Host='192.168.0.1', Port=1521, Dbname='TEST', dbType='Oracle');
```

3.39.4.2　修改 Database Link 对象

`ALTER DATABASE LINK` 修改一个 dblink 数据库对象。

第一种形式更改 dblink 的拥有者，要修改拥有者，你必须拥有该 dblink 并且也是新拥有角色的一个直接或间接成员。

第二种形式更改 dblink 的名称，只有 dblink 拥有者或者超级用户可以重命名一个 dblink。

语法格式

```
ALTER [PUBLIC] DATABASE LINK dblink_name OWNER TO new_owner;
ALTER [PUBLIC] DATABASE LINK dblink_name RENAME TO new_name;
```

参数说明

`dblink_name`
一个现有 dblink 的名称（可以是模式限定的）。

`new_owner`
该 dblink 的新拥有者的用户名。

`dblink_name`
该 dblink 的新名称。

注解

`ALTER DATABASE LINK` 语法类似于 ALTER VIEW。

示例

把 `DATABASE LINK` kdb_dblink_regress.link_d 拥有者修改为 kdb_dblink_user:

```
ALTER DATABASE LINK kdb_dblink_regress.link_d OWNER TO kdb_dblink_user;
```

兼容性

`ALTER DATABASE LINK` 语句是一个 KingbaseES 扩展。

3.39.4.3　删除 Database Link 对象

`DROP DATABASE LINK` 移除一个 dblink。要执行这个命令必须拥有该 dblink 的权限。

语法格式

```
DROP DATABASE LINK dblink_name;
```

参数说明

`dblink_name`
删除名为 dblink_name 的 `DATABASE LINK`

注解

DBLink 可以被拥有者和 DBA 删除, 删除 DBLink 对象之后，所有的用户会话中的对象都会被清除。

如果 DBLink 正在被其他用户使用，则无法删除此 DBLink。

暂不支持 PUBLIC 关键字。

兼容性

该命令是 KingbaseES 的一个扩展，兼容 Oracle。

3.39.4.4　Database Link 的权限控制

对于私有的 DBLink，其创建者和 DBA 具有对此 DBLink 对象进行访问的权限。除 DBLink 创建者之外的其他数据库用户可以对以 PUBLIC 方式建立的 DBLink 进行访问。

本地用户对远程数据库对象的访问权限由远程数据库系统的用户认证机制来控制。通过 DBLink 连接到远程数据库的本地用户将得到远程数据库的用户拥有的查询权限。

3.39.4.5　DBLink 相关的视图

DBLink 向用户提供的视图如下：

表 3.39.1: DBLink 相关视图

名称	说明
ALL_DB_LINKS	描述当前用户可以访问的所有数据库连接
USER_DB_LINKS	描述当前用户拥有的所有数据库连接
DBA_DB_LINKS	描述 DBA 可以访问的所有数据库连接

3.39.4.6　DATABASE LINK DML

DATABASE LINK —dblink 的 DML 操作

语法格式

- INSERT

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
INSERT INTO table_name@dblink [ AS alias ] [ ( column_name [, ...] ) ]
    [ OVERRIDING { SYSTEM | USER} VALUE ]
    { VALUES ( { expression } [, ...] ) [, ...] | query }
    [ RETURNING * | output_expression [ [ AS ] output_name ] [, ...] ];
```

- UPDATE

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
UPDATE table_name@dblink [ * ] [ [ AS ] alias ]
    SET { column_name = { expression } |
        ( column_name [, ...] ) = [ ROW ] ( { expression } [, ...] ) |
        ( column_name [, ...] ) = ( sub-SELECT )
        } [, ...]
    [ FROM from_list ]
    [ WHERE condition ]
    [ RETURNING * | output_expression [ [ AS ] output_name ]  [, ...] ];
```

- DELETE

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
DELETE FROM table_name@dblink [ * ] [ [ AS ] alias ]
    [ WHERE condition ]
    [ RETURNING * | output_expression [ [ AS ] output_name ] [, ...] ];
```

描述

使用 dblink 对远程表进行插入，更新，删除。

参数

dblink dblink 的名称，使用 `CREATE DATABASE LINK` 创建的 dblink 数据库连接的名称。

`with_query`
`WITH` 子句允许你指定一个或者多个子查询，在 `INSERT UPDATE DELETE` 查询中可以用子查询的名字来引用它们详见 WITH 查询 和 SELECT 。

`table_name`
一个已有表的名称（可以被模式限定）。

`alias`
目标表名的替补名称。当提供了一个别名时，它会完全隐藏掉表的实际名称。

`column_name`
名为 `table_name` 的表中的一个列的名称，可以用表名 (可以被模式名限定) 做限定。如有必要，列名还可以用一个子域名 (子域名需要和表名、模式名一同用小括号包围起来进行限定) 或者数组下标限定。

`expression`
要赋予给相应列的表达式或者值。

`output_expression`
在每一行被插入或更新或删除后由 `INSERT` 或 `UPUDATE` 或 `DELETE` 命令计算并且返回的表达式。该表达式可以使用 *`table_name`* 指定的表中的任何列。写成 `*` 可返回被插入或更新或删除行的所有列。

`output_name`
要用于被返回列的名称。

`condition`
一个能返回 `boolean` 值的表达式。只有让这个表达式返回 `true` 的行才将被更新，删除。

使用限制

- INSERT

  远端表插入操作不支持 ON CONFLICT 子句

  不支持显示指定生成列的值

  对于 RETURNING，不支持 RETURNING ctid 等系统列

  暂不支持列的默认值，对于有默认值的远程表，在进行 insert 的时候，如果显式使用例如 insert into {tablename} default value 或者 insert into {tablename} values(default) 会报错 dblink not support insert with default value；如果 insert into 的时候，没有指定全部的列也会报上面的错误比如当表 tab 有两列 a, b 的时候，那么下面 sql 都会报错

  1. insert into <tab@dblink> values(1)
  2. insert into <tab@dblink>(a) values(1)

3. insert into <tab@dblink> select a from tab
4. insert into <tab@dblink>(a) select a from tab

- UPDATE

  当远端表为继承表时，不支持仅更新父表数据，指定 ONLY 关键字时报错

  对于 RETURNING，不支持 RETURNING ctid 等系统列

  目前 update 不支持列的默认值，在进行 update 的时候，如果显式的指定使用默认值，比如 update {tablename} set {columname}=default，会报错 dblink not support insert with default value

  目前不支持 `WHERE CURRENT OF cursor_name` 的用法

  目前 update 对于 分区表和 继承表的行为表现不确定，所以禁止使用 dblink 更新远程 分区表或者 继承表中的数据

- DELETE

  当远端表为继承表时，不支持仅删除父表数据，指定 ONLY 关键字时报错

  对于 RETURNING，不支持 RETURNING ctid 等系统列

  目前 delete 对于 分区表和 继承表的行为表现不确定，所以禁止使用 dblink 删除远程 分区表或者 继承表中的数据

ORACLE **兼容**

当 dblink 创建的数据库连接是连接到 Oracle 的时候，除了以上的使用限制外，目前 dblink 还不支持 UPDATE 和 DELETE

3.39.4.7　DATABASE LINK MERGE

DATABASE LINK —dblink 的 MERGE 操作

语法格式

```
MERGE INTO [ schema. ] { target_table@dblink } [ [ AS ] target_table_alias ]
    USING { [ schema. ] { source_table } [ [ AS ] source_table_alias ]
    ON ( condition_expression )
[ merge_update_clause ]
[ merge_insert_clause ];

merge_update_clause:
WHEN MATCHED THEN
UPDATE SET column = { expr }[, column = { expr } ]...
[ where_clause ]
[ delete_clause ]

delete_clause:
```

```
[DELETE where_clause]

merge_insert_clause:
WHEN NOT MATCHED THEN
INSERT [ ( column [, column ]...) ]
VALUES ({ expr }[, { expr } ]...)
[ where_clause ]

where_clause:
WHERE condition
```

描述

使用 dblink 对远程表进行 MERGE 操作。

参数

`target_table`
MERGE 远程目标表的名称。

dblink dblink 的名称，使用 `CREATE DATABASE LINK` 创建的 dblink 数据库连接的名称。

`source_table`
MERGE 源表的名称。源表可以是一个本地表，普通外表，也可以是一个 dblink 的远程表，即 table-name@dblinkname。

`target_table_alias`
MERGE 目标表的别名。

`source_table_alias`
MERGE 源表的别名。

`expr`
要赋予给相应列的表达式或者值。

`condition_expression`
指定目标表与源表之间进行联接的联接条件。如果该条件为真，且指定了 WHEN MATCHED THEN UPDATE 子句，则对匹配到的目标表的该元组执行更新操作；否则，如果该条件为假且指定了 WHEN NOT MATCHED THEN INSERT 子句，则对目标表执行插入操作。

`merge_update_clause`
当目标表和源表的 ON 条件为真时，执行该子句，即更新目标表数据。该更新操作会触发目标表上面的触发器。更新的列不能是 ON 条件中被引用的列，更新时可以通过 WHERE 条件指明要更新的行，条件中既可以包含源表的列，也可以包含目标表的列，当指明 WHERE 条件且条件为假时，则不更新。

`delete_clause`
DELETE 子句只删除目标表和源表的 ON 条件为真、并且是更新后的符合删除条件的记录，DELETE 子句不影响 INSERT 项插入的行。删除条件作用在更新后的记录上，既可以和源表相关，也可以和目标表相关，或者都相关。如果 ON 条件为真，但是不符合更新条件，并没有更新数据，那么 DELETE 将不会删除任何数据。

`merge_insert_clause`
当目标表和源表的 ON 条件为假时，执行该语句。可指定插入条件，插入时的 WHERE 条件只能引用源表中的列。VALUES 后面也只能引用源表中的列，不能包含目标表的列。

3.39.4.8 DATABASE LINK 调用函数/存储过程

DATABASE LINK —dblink 调用用户自定义函数/存储过程

语法格式

```
[schema_name.] func_name@dblink([args])

CALL  [schema_name.]proc_name@dblink([args])
```

描述

使用 dblink 访问远程用户自定义函数/存储过程。

参数

dblink dblink 的名称，使用 `CREATE DATABASE LINK` 创建的 dblink 数据库连接的名称。

`schema_name`
远程模式名称。

`func_name`
远程用户自定义函数/储存过程名称。

`args`
函数/储存过程的参数。

使用限制

- 函数

  当在同一用户模式下存在同名函数时，该函数不支持通过远程调用。dblink 函数的调用方式，支持 select <func@dblink>(arg) 这种方式。不支持 select * from <func@dblink>(arg) 方式，以及 call <func@dbink>(arg) 的这种方式。

  支持在以下语句类型中调用：

  1. SELECT 列表；
  2. WHERE/HAVING 条件；
  3. WITH/GROUP BY/ORDER BY；
  4. INSERT 的 VALUES 子句；
  5. UPDATE 的 SET 子句；
  6. JOIN 条件；

7. PL/SQL 中引用。

不支持以下函数类型：

- 聚集函数
- 窗口函数
- PIPELINED 函数

函数属性支持情况如下：

1. IMMTABLE：该函数不能修改数据库，并且对于给定的参数值总是会返回相同的值；
2. STABLE：该函数不能修改数据库，并且对于相同的参数值，它在一次表扫描中将返回相同的结果；
3. VOLATILE：该函数的值在一次表扫描中有可能改变，不能做优化；
4. PARALLEL：
    1. UNSAFE：该函数不能在并行模式中运行并且 SQL 语句中存在一个这样的函数会强制使用顺序执行计划；
    2. RESTRICTED：该函数能在并行模式中运行，但是其执行被限制在并行组的 leader 中；
    3. SAFE：该函数可以不受限制的在并行模式中执行；
5. STRICT：该函数中任意参数为空值时，函数返回值也为空值。

其中，函数参数只支持 IN 模式，支持远程函数参数默认值。参数个数最大支持 510 个。参数类型和返回值支持 KingbaseES 的基本内置数据类型。

- 存储过程

  支持使用 “CALL“调用远程用户自定义存储过程。在 dlbink 用户有执行权限的情况下，允许跨模式调用其他用户定义的存储过程。另外，当同一模式下存在同名存储过程时，该存储过程不能通过 dblink 远程调用。

  目前仅支持 in 参数，不支持 out 参数和 inout 参数。存储过程的参数个数不大于 98 个。

兼容性

目前仅支持 KingbaseES 远程数据库调用函数、存储过程。

3.39.5 插件 kdb_database_link 的卸载方法

在 ksql 中运行：

```
drop extension kdb_database_link;
```

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数，去掉 kdb_database_link 后重启数据库。

```
shared_preload_libraries = '';
```

3.39.6 插件 kdb_database_link 的升级方法

kdb_database_link 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.40 kdb_ddl

3.40.1 插件 kdb_ddl 简介

kdb_ddl 是 KingbaseES 的一个扩展插件，用于生成数据库对象 DDL 语句，主要包括数据库对象创建、修改、删除语句的生成。每个用户仅可以查询自己有使用和查询权限的数据库对象，不包含数据库系统的预创建对象，如系统视图，INT 类型等。

目前仅支持以下类型对象使用 kdb_ddl 接口查询生成数据库对象 DDL 语句：

- FUNCTION
- PACKAGE
- PROCEDURE
- TRIGGER
- TYPE
- VIEW

上述对象在当前数据库中的状态不论为有效还是无效，均可以通过 kdb_ddl 拓展中提供的接口获取相应的语句脚本。

备注:

仅当 KingbaseES 处于 ORACLE 模式时，支持 kdb_ddl 扩展中的接口功能。

3.40.2 插件 kdb_ddl 加载方式

kdb_ddl 不在数据库初始化时创建，需要通过命令创建扩展 kdb_ddl：

```
CREATE EXTENSION kdb_ddl;
```

3.40.3 插件 kdb_ddl 的参数配置

无需配置任何参数。

3.40.4　插件 kdb_ddl 使用方法

3.40.4.1　获取完整的数据库对象 DDL 脚本

数据库对象完整 DDL 脚本的定义包括：对象包括目标对象的删除，创建以及授权语句。在脚本语句中，依次为删除语句，创建语句和授权语句。其中删除语句可以根据用户提供的选项决定是否显示，以及是否添加 if exists 子句。其中，授权语句只有在授权动作产生后才会显示。

3.40.4.1.1　通过对象名获取完整的 DDL 脚本

函数 ddlx_script 用于获取数据库对象的完整定义的 DDL 脚本。该接口有两个重载版本，其中一个版本可以通过对象名称获取完整的 DDL 脚本。

语法

```
FUNCTION kdb_ddl.ddlx_script(ddlx_name text,
                 ddlx_schema text DEFAULT NULL,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

`ddlx_name`

对象的名称。对于函数或存储过程，可以通过传入参数类型来确保唯一性，未传入参数时若匹配到多个目标对象，输出错误提示。用户传入的对象名不存在时，输出错误提示。例如：

```
SELECT kdb_ddl.ddlx_script('func1(int)');
SELECT kdb_ddl.ddlx_script('func1()');
SELECT kdb_ddl.ddlx_script('func1');
```

`ddlx_schema`

目标对象的模式名，用于确定目标对象的在数据库中的唯一性，默认为 NULL。当该参数为 NULL 时，ddlx_script 接口会在用户指定的当前的 search_path 内利用目标对象的名称与目标对象的类型进行对象匹配，若未匹配到对象或者匹配到多个对象，则报错。

`ddlx_type`

目标对象的类型，默认为 NULL，使用时仅可以提供一个选项。当该参数为 NULL 时，ddlx_script 接口会在支持的对象范围内搜索与对象名匹配的对象并返回完整的 DDL 脚本，若有多个对象匹配，输出错误提示。当用户使用的类型不属于预定义类型内容时，输出错误提示。具体可选值详见表 ddlx_type-parameter 。

`ddlx_options`

控制 DDL 脚本内容的选项参数，默认为空，使用时可以同时传递多个选项，当多个选项之间存在冲突的选项时，输出错误提示。当用户使用的选项不属于预定义选项内容时，输出错误提示。具体可选值详见表 ddlx_options-parameter。

返回值

对象的 DDL 脚本语句。

示例

获取目标函数的 DDL 脚本。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
  RETURN 1;
END;
/
CREATE OR REPLACE FUNCTION func2(i INT DEFAULT func1(1)) RETURN INT AS
BEGIN
RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_script('func1(int)', 'public', 'FUNCTION', '{owner}');
                              ddlx_script
-----------------------------------------------------------------------------
DROP FUNCTION func1(integer);                                                 +
                                                                             +
-- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID          +
                                                                             +
-- Depend:                                                                   +
-- Not Found                                                                 +
                                                                             +
-- Depended:                                                                 +
-- Type: FUNCTION; Name: func2(integer);                                     +
                                                                             +
CREATE OR REPLACE FUNCTION public.func1(integer)                              +
RETURNS integer                                                              +
AS                                                                            +
BEGIN                                                                         +
   RETURN 1;                                                                 +
END;                                                                          +
                                                                             +
ALTER FUNCTION func1(integer) OWNER TO test;                                  +
```

3.40.4.1.2　根据 OID 获取完整的 DDL 脚本

函数 ddlx_script 用于获取数据库对象的完整定义的 DDL 脚本。该接口有两个重载版本，其中一个版本可以通过对象 OID 获取完整的 DDL 脚本。

语法

```
FUNCTION kdb_ddl.ddlx_script(ddlx_id oid,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

ddlx_id

对象的 OID，用户传入的 OID 不存在时，输出错误提示。

其它参数与通过对象名获取完整的 DDL 脚本 的 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明。

返回值说明

对象的 DDL 脚本语句。

示例

获取目标函数的 DDL 脚本。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
CREATE OR REPLACE FUNCTION func2(i INT DEFAULT func1(1)) RETURN INT AS
BEGIN
  RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_script('func1(int)'::regprocedure, 'FUNCTION', '{owner}');
                                ddlx_script
-----------------------------------------------------------------------------
 DROP FUNCTION func1(integer);                                              +
                                                                            +
 -- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID        +
                                                                            +
 -- Depend:                                                                 +
```

```
-- Not Found                                                            +
                                                                        +
-- Depended:                                                            +
-- Type: FUNCTION; Name: func2(integer);                                +
                                                                        +
CREATE OR REPLACE FUNCTION public.func1(integer)                        +
 RETURNS integer                                                        +
AS                                                                      +
BEGIN                                                                   +
    RETURN 1;                                                           +
END;                                                                    +
                                                                        +
ALTER FUNCTION func1(integer) OWNER TO test;                            +
```

3.40.4.2　获取数据库对象创建语句

3.40.4.2.1　通过对象名获取创建语句

ddlx_create 和 ddlx_createonly 用于获取数据库对象的创建语句，二者的区别为 ddlx_create 接口为完整的创建语句，包含了目标对象的权限处理语句，ddlx_createonly 接口就是纯粹的对象定义创建语句，并不包含权限处理语句。两个接口均有两个重载版本，其中一个版本可以通过对象名称获取对象的创建语句。

3.40.4.2.1.1　包含权限处理的对象创建语句

ddlx_create 函数用于获取包含权限处理的对象的创建语句。

语法

```
FUNCTION kdb_ddl.ddlx_create (ddlx_name text,
                 ddlx_schema text DEFAULT NULL,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的创建语句，包含权限处理语句。

示例

获取目标函数的创建语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_create ('func1(int)', 'public', 'FUNCTION', '{owner}');
                                ddlx_create
-----------------------------------------------------------------------------
 -- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID +
                                                                      +
 -- Depend:                                                           +
 -- Not Found                                                         +
                                                                      +
 -- Depended:                                                         +
 -- Type: FUNCTION; Name: func2(integer);                             +
                                                                      +
 CREATE OR REPLACE FUNCTION public.func1(integer)                     +
  RETURNS integer                                                     +
 AS                                                                   +
 BEGIN                                                                +
     RETURN 1;                                                        +
 END;                                                                 +
                                                                      +
 ALTER FUNCTION func1(integer) OWNER TO test;                         +
                                                                      +
```

3.40.4.2.1.2　不包含权限处理的对象创建语句

ddlx_create 函数用于获取不包含权限处理的对象的创建语句。

语法

```
FUNCTION kdb_ddl.ddlx_createonly (ddlx_name text,
                 ddlx_schema text DEFAULT NULL,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
        RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的创建语句，不包含权限处理语句。

示例

获取目标函数的创建语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_createonly ('func1(int)', 'public', 'FUNCTION', '{owner}');
                               ddlx_createonly
-----------------------------------------------------------------------------
 -- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID        +
                                                                            +
 -- Depend:                                                                 +
 -- Not Found                                                               +
                                                                            +
 -- Depended:                                                               +
 -- Type: FUNCTION; Name: func2(integer);                                   +
                                                                            +
 CREATE OR REPLACE FUNCTION public.func1(integer)                           +
  RETURNS integer                                                           +
 AS                                                                         +
 BEGIN                                                                      +
     RETURN 1;                                                              +
 END;                                                                       +
                                                                            +
 ALTER FUNCTION func1(integer) OWNER TO test;                               +
```

3.40.4.2.2　通过 OID 获取创建语句

ddlx_create 和 ddlx_createonly 用于获取数据库对象的创建语句，二者的区别为 ddlx_create 接口为完整的创建语句，包含了目标对象的权限处理语句，ddlx_createonly 接口就是纯粹的对象定义创建语句，并不包含权限处理语句。两个接口均有两个重载版本，其中一个版本可以通过对象 OID 获取对象的创建语句。

3.40.4.2.2.1　包含权限处理的对象创建语句

ddlx_create 函数用于获取包含权限处理的对象的创建语句。

语法

```
FUNCTION kdb_ddl.ddlx_create (ddlx_id oid,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的创建语句，包含权限处理语句。

示例

获取目标函数的创建语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_create ('func1(int)'::regprocedure, 'FUNCTION', '{owner}');
                                ddlx_create
-----------------------------------------------------------------------------
 -- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID +
                                                                      +
 -- Depend:                                                           +
 -- Not Found                                                         +
                                                                      +
 -- Depended:                                                         +
 -- Type: FUNCTION; Name: func2(integer);                             +
                                                                      +
 CREATE OR REPLACE FUNCTION public.func1(integer)                     +
  RETURNS integer                                                     +
 AS                                                                   +
 BEGIN                                                                +
     RETURN 1;                                                        +
 END;                                                                 +
                                                                      +
 ALTER FUNCTION func1(integer) OWNER TO test;                         +
```

3.40.4.2.2.2　不包含权限处理的对象创建语句

ddlx_create 函数用于获取不包含权限处理的对象的创建语句。

语法

```
FUNCTION kdb_ddl.ddlx_createonly (ddlx_name text,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的创建语句，不包含权限处理语句。

示例

获取目标函数的创建语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_createonly ('func1(int)'::regprocedure, 'FUNCTION', '{owner}');
                              ddlx_createonly
-----------------------------------------------------------------------------
 -- Type: FUNCTION; Name: func1(integer); Owner: test; Status: VALID        +
                                                                            +
 -- Depend:                                                                 +
 -- Not Found                                                               +
                                                                            +
 -- Depended:                                                               +
 -- Type: FUNCTION; Name: func2(integer);                                   +
                                                                            +
 CREATE OR REPLACE FUNCTION public.func1(integer)                           +
  RETURNS integer                                                           +
 AS                                                                         +
 BEGIN                                                                      +
     RETURN 1;                                                              +
 END;                                                                       +
                                                                            +
```

```
ALTER FUNCTION func1(integer) OWNER TO test;                                +
```

3.40.4.3　获取数据库对象删除语句

删除语句可以根据用户提供的选项决定是否显示，以及是否添加 if exists 子句。

3.40.4.3.1　通过对象名获取删除语句

ddlx_drop 函数用于获取数据库对象的删除语句，该接口有两个重载版本，其中一个版本可以通过对象的名称获取对象的删除语句。

语法

```
FUNCTION kdb_ddl.ddlx_drop (ddlx_name text,
                 ddlx_schema text DEFAULT NULL,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的删除语句。

示例

获取目标函数的删除语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_drop ('func1(int)', 'public', 'FUNCTION');
           ddlx_drop
--------------------------------
 DROP FUNCTION func1(integer);
```

3.40.4.3.2　通过 OID 获取删除语句

ddlx_drop 函数用于获取数据库对象的删除语句，该接口有两个重载版本，其中一个版本可以通过对象的 OID 获取对象的删除语句。

语法

```
FUNCTION kdb_ddl.ddlx_drop (ddlx_id oid,
                 ddlx_type text DEFAULT NULL,
                 ddlx_options text[] DEFAULT '{}'::text[])
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的删除语句。

示例

获取目标函数的删除语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

SELECT kdb_ddl.ddlx_drop ('func1(int)'::regprocedure, 'FUNCTION');
           ddlx_drop
-------------------------------
 DROP FUNCTION func1(integer);
```

3.40.4.4　获取数据库对象授权语句

3.40.4.4.1　通过对象名获取授权语句

ddlx_grants 函数用于获取数据库对象的授权语句，该接口有两个重载版本，其中一个版本可以通过对象的名称获取对象的授权语句。
授权语句只有在授权动作产生后才会显示

语法

```
FUNCTION kdb_ddl.ddlx_grants (ddlx_name text,
                 ddlx_schema text DEFAULT NULL,
                 ddlx_type text DEFAULT NULL
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的授权语句。

示例

获取目标函数的授权语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;
-- 把函数权限授予用户 test_user
GRANT EXECUTE ON FUNCTION func1(int) TO test_user;


SELECT kdb_ddl.ddlx_grants ('func1(int)', 'public', 'FUNCTION');
                                ddlx_grants
---------------------------------------------------------------------------
 REVOKE ALL ON FUNCTION func1(integer) FROM PUBLIC;                       +
 GRANT EXECUTE ON FUNCTION func1(integer) TO PUBLIC;                      +
 GRANT EXECUTE ON FUNCTION func1(integer) TO test WITH GRANT OPTION;      +
 GRANT EXECUTE ON FUNCTION func1(integer) TO test_user;
```

3.40.4.4.2　通过 OID 获取授权语句

ddlx_grants 函数用于获取数据库对象的授权语句，该接口有两个重载版本，其中一个版本可以通过对象的 OID 获取对象的授权语句。授权语句只有在授权动作产生后才会显示。

语法

```
FUNCTION kdb_ddl.ddlx_grants (ddlx_id oid,
                 ddlx_type text DEFAULT NULL)
         RETURN TEXT
```

参数说明

参数与 ddlx_script 函数描述一致，详见 ddlx_script 函数的参数说明 。

返回值说明

对象的授权语句。

示例

获取目标函数的授权语句。

```
\set SQLTERM /
CREATE OR REPLACE FUNCTION func1(int) RETURN INT AS
BEGIN
    RETURN 1;
END;
/
\set SQLTERM ;

-- 把函数权限授予用户 test_user
GRANT EXECUTE ON FUNCTION func1(int) TO test_user;

SELECT kdb_ddl.ddlx_grants ('func1(int)'::regprocedure, 'FUNCTION');
                                ddlx_grants
-----------------------------------------------------------------------------
 REVOKE ALL ON FUNCTION func1(integer) FROM PUBLIC;                        +
 GRANT EXECUTE ON FUNCTION func1(integer) TO PUBLIC;                       +
 GRANT EXECUTE ON FUNCTION func1(integer) TO test WITH GRANT OPTION;       +
 GRANT EXECUTE ON FUNCTION func1(integer) TO test_user;                    +
```

3.40.5　插件 kdb_ddl 卸载方法

删除 kdb_ddl 扩展，示例如下：

```
DROP EXTENSION kdb_ddl;
```

3.40.6　插件 kdb_ddl 升级方法

kdb_ddl 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.41 kdb_exists_expand

3.41.1 插件 kdb_exists_expand 简介

kdb_exists_expand 插件用于优化 EXISTS 子链接，生成执行效率更高的执行计划。

对于在查询条件中存在 EXISTS 子连接的 SQL 语句，如果满足：

1. EXISTS 子链接为 SELECT 类型的查询语句
2. EXISTS 子链接中为 OR 条件

如下：

```
SELECT * from t1 WHERE EXISTS (SELECT 1 from t2 cond1 OR cond2);
```

其中的 cond1 的查询条件和子链接中查询的表 t2 有关，cond2 的查询条件和子链接中查询的表 t2 无关，如：

```
SELECT * from t1 WHERE EXISTS (SELECT 1 from t2 WHERE t2.id =1 OR t1.name = null);
```

- 其中 cond1 为 t2.id =1，与子链接中的表 t2 相关
- 其中 cond2 为 t1.name = null，与子链接中的表 t2 不相关

那么该插件将 EXISTS 子链接的不相关条件做提升优化，可以提高 SQL 查询的执行效率。

3.41.2 插件 kdb_exists_expand 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'kdb_exists_expand'
```

3.41.3 插件 kdb_exists_expand 的参数配置

`kdb_exists_expand.enable`
Exists 子链接优化开关，boolean 类型数据，默认为开启状态。

3.41.4 插件 kdb_exists_expand 的使用方法

KingbaseES 加载 kdb_exists_expand 插件后，碰到满足简介中条件的 SQL 语句时，将 EXISTS 子链接的不相关条件做提升优化。

示例：

启动该功能后，本来应该是一个 Semi join 的执行计划，被改成了两个并列的子计划 InitPlan 1 和 InitPlan 2 去执行：

```
create extension kdb_exists_expand;

load 'kdb_exists_expand';

set kdb_exists_expand.enable to on;

create table t1(id int, name text);

create table t2(id int, name text);

explain (costs off) SELECT * from t1 WHERE EXISTS (SELECT 1 from t2 WHERE t2.id =1 OR t1.name = null);

QUERY PLAN
----------------------------------
Result
  One-Time Filter: ($0 OR $1)
  InitPlan 1 (returns $0)
    ->  Seq Scan on t2
          Filter: (id = 1)
  InitPlan 2 (returns $1)
    ->  Result
          One-Time Filter: false
  ->  Seq Scan on t1
```

3.41.5　插件 kdb_exists_expand 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.41.6　插件 kdb_exists_expand 升级方法

kdb_exists_expand 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.42 kdb_flashback

3.42.1 插件 kdb_flashback 简介

插件 kdb_flashback 是 KingbaseES 的一个扩展插件。主要功能是提供错误数据的快速恢复能力，目前提供的闪回技术包括闪回回收站，闪回查询，闪回版本查询，闪回到任意时间点。

- 插件名为 kdb_flashback
- 插件版本 V1.0

3.42.2 插件 kdb_flashback 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'kdb_flashback'
```

3.42.3 插件 kdb_flashback 的参数配置

`kdb_flashback.enable_flashback_query`
闪回查询的开启参数，默认开启。

`kdb_flashback.db_recyclebin`
闪回回收站开启参数，参数为 bool 类型，参数级别为 PGC_SIGHUP，默认关闭。

3.42.4 插件 kdb_flashback 的使用方法

闪回查询相关的技术使用方法

- 整个闪回技术 (闪回查询和闪回表到指定时间点) 可以通过时间戳和 CSN(commit sequence number) 两种方式进行。
- 两种方式目前都需要依赖提交的时间戳，因此需要开启 track_commit_timestamp 这个参数。另外闪回查询还需要开启 kdb_flashback.enable_flashback_query 的参数。

- 这里解释下时间戳和 CSN 两个快照指定的方式，时间戳即 timestamp 用户可以在闪回查询的快照时间表达式里指定任意一个有效的时间戳。如果允许闪回查询，那么将返回该时刻能够可见的历史快照的数据。CSN 是提交顺序号，第一个有效的 CSN 为 65536000000，CSN 随着提交线性增加。因此如果用户指定一个有效的 CSN，比如 65536000160，那么闪回查询将基于这个 CSN 提交号构建历史快照。

- 闪回查询技术有一定的限制，在数据进行深度清理后和部分 schema change 后将拒绝闪回查询（具体参考闪回查询须知）。

下面具体介绍这几种闪回技术，　在使用任何闪回查询相关的技术前，保证在配置文件中开启 track_commit_timestamp 参数，并重启服务。

3.42.4.1　闪回查询

闪回查询能否返回用户指定历史时刻的快照数据，历史快照时刻指定方式可以通过 timestamp 和 csn 两种方式指定 (具体语法参考 SELECT)。

例 2-56*AS OF 闪回查询（timestamp）* 语句的示例

```
flashback=# create table fb_example(id int, name varchar(100));
CREATE TABLE
flashback=# insert into fb_example values(1, 'name1');
INSERT 0 1
flashback=# insert into fb_example values(2, 'name1');
INSERT 0 1
flashback=# insert into fb_example values(3, 'name1');
INSERT 0 1
flashback=# select * from fb_example;
 id | name
----+-------
  1 | name1
  2 | name1
  3 | name1
(3 rows)

flashback=# \set 'tp1' `date "+%Y-%m-%d %H:%M:%S.%N"`
flashback=# update fb_example set name = null;
UPDATE 3
flashback=# select * from fb_example;
 id | name
----+------
  1 |
  2 |
  3 |
(3 rows)

flashback=# select * from fb_example as of timestamp :'tp1';
 id | name
----+-------
  1 | name1
  2 | name1
```

```
  3 | name1
(3 rows)
```

例 2-56 中的 update 语句误将 name 更改为 null，此时可以通过闪回查询查询到 update 前的数据。

通常 CSN 方式进行闪回查询的时候，需要借助闪回版本查询确定合适的 CSN 号，比如 delete 事务误将 fb_example 表删除了，那么通过版本查询可以确定该 delete 事务的 CSN 为 65536000007，可以通过如下的闪回语句查询删除的数据。

例 2-57*AS OF 闪回查询（CSN）* 语句的示例

```
flashback=# select versions_startscn, versions_endcsn, * from fb_example versions between csn minvalue and
maxvalue;
 versions_startscn | versions_endcsn | id | name
-------------------+-----------------+----+-------
       65536000003 |     65536000006 |  1 | name1
       65536000004 |     65536000006 |  2 | name1
       65536000005 |     65536000006 |  3 | name1
       65536000006 |     65536000007 |  1 |
       65536000006 |     65536000007 |  2 |
       65536000006 |     65536000007 |  3 |
(6 rows)

flashback=# select * from fb_example as of csn 65536000006;
 id | name
----+------
  1 |
  2 |
  3 |
(3 rows)
```

3.42.4.2　闪回版本查询

闪回版本查询的作用主要有以下几个方面：

- 进行历史数据的分析。
- 查看数据的变更历史。
- 确定闪回表的精确时间点。

闪回版本查询拥有和 Oracle 相同的参考伪列，伪列和功能参考下表。

表 3.42.1: 闪回版本查询的伪列

伪列名	描述
VERSIONS_STARTTIME	元组被创建的时间戳
VERSIONS_ENDTIIME	元组被删除的时间戳
VERSIONS_STARTCSN	元组被创建的 commit seqno
VERSIONS_ENDCSN	元组被删除的 commit seqno
VERSIONS_OPERATION	对应的操作 D 表示该元组被删除，I 表示该元组被插入
VEERSIONS_XID	元组被创建的事务 ID

闪回版本查询的 start 和 end 的快照表达式既可以指定一个有效的快照表达式，也可以通过 minvalue 和 maxvalue 两个关键字进行查询。当指定时，将返回该表能够闪回查询的所有的版本信息。

例 2-58* 通过 minvalue 和 maxvalue 进行闪回版本查询（CSN）* 语句的示例

```
flashback=# select versions_startcsn, versions_endcsn, versions_operation, * from fb_example versions
between csn minvalue and maxvalue;
 versions_startcsn | versions_endcsn | versions_operation | id | name
-------------------+-----------------+--------------------+----+-------
       65536000003 |     65536000006 | D                  |  1 | name1
       65536000004 |     65536000006 | D                  |  2 | name1
       65536000005 |     65536000006 | D                  |  3 | name1
       65536000006 |     65536000007 | D                  |  1 |
       65536000006 |     65536000007 | D                  |  2 |
       65536000006 |     65536000007 | D                  |  3 |
       65536000008 |                 | I                  |  5 | name3
       65536000009 |                 | I                  |  6 | name3
(8 rows)
```

通过上面的例子，我们可以追溯 fb_example 的版本变更的历史，65536000006 这个 CSN 的事务做了一次 update 的操作，65536000007 这个 CSN 的事务做了一次 delete 的操作。因此如果我们想查询被删除前的数据可以尝试 65536000006 这个 CSN 快照。

3.42.4.3　闪回表到指定时间点

闪回表到指定的时间点，实际上为用户提供了还原数据的一种手段，如果说闪回查询能够帮助用户查询历史的数据，那么闪回表实际上是帮助用户彻底的找回数据, 具体语法参考 FLASHBACK。

闪回表的通常步骤：

- 通过闪回版本查询确定待闪回的版本范围，查询 startxxx, endxxx, 和 operation 确定边界。
- 通过 AS OF 闪回查询结合确定的边界锁定闪回查询的精确时间点，并观察数据时候符合预期。
- 通过 FLASHBACK TABLE TO [Timestamp | CSN] asof_item，完成表的闪回。

还是参考 2-58 的例子，比如我们推理出错误更新 name 的事务是 65536000006 这个 CSN，那么我们可以指定 65536000005 进行闪回查询。再通过 flashback 闪回到误更前的时刻。

```
flashback=# select * from fb_example as of csn 65536000005;
 id | name
----+-------
  1 | name1
  2 | name1
  3 | name1
(3 rows)

flashback=# flashback table fb_example to csn 65536000005;
FLASHBACK TABLE
flashback=# select * from fb_example;
 id | name
----+-------
  1 | name1
  2 | name1
  3 | name1
(3 rows)
```

3.42.4.4　闪回查询和闪回表技术的使用须知

- 闪回查询和闪回表技术依赖于历史的数据，如果历史数据因为 vacuum、truncate、rewrite 等操作被回收掉，那么会导致无法闪回到这些操作之前的时刻。因此推荐用户在期望使用闪回查询的时候对 vacuum 相关参数做一定的调整（关闭表级的 autovacuum, 推荐调大 vacuum_defer_cleanup_age 的值以降低历史数据被回收的机会）。
- 目前闪回查询和闪回表技术在 vacuum、truncate、和部分 ddl 之后将不允许进行闪回到这些操作之前。
- 闪回查询应用于视图或者物化视图里面应该尽量避免对于常量时间戳和 CSN 的使用，可能会引发 dump 和 restore 的失败。

3.42.4.5　闪回回收站

闪回回收站功能为用户提供一种误删表后还原表的一种手段，闪回回收站分为将回收站中的表闪回到删除之前的状态和清空回收站。

例 2-59*flashback table to before drop* 语句的示例

```
test=# flashback table range_partition_tb to before drop;
FLASHBACK TABLE
test=# \d
                          List of relations
Schema   |            Name            |       Type        | Owner
```

----------±----------±----------±------- flashback | range_partition_tb | partitioned table | system flashback | range_partition_tb_part_01 | table | system flashback | range_partition_tb_part_02 | table | system

(3 rows)

回收站需用户定期维护，避免回收站膨胀，用户可以通过 PURGE 操作对回收站进行维护，PURGE 操作分为删除回收站中一个指定表和清空回收站。清理回收站时，回收站视图 recyclebin 和系统表 sys_recyclebin 中相关对象将被清除。

例 2-60*purge table* 语句的示例

```
test=# select * from recyclebin;
  oid  |       original_name        |           droptime            | type
-------+----------------------------+-------------------------------+-------
 24968 | range_partition_tb_part_02 | 2022-05-20 15:14:53.581052+08 | TABLE
 24965 | range_partition_tb_part_01 | 2022-05-20 15:14:53.581494+08 | TABLE
 24962 | range_partition_tb         | 2022-05-20 15:14:53.582074+08 | TABLE
 24971 | purge_test                 | 2022-05-20 15:15:45.027217+08 | TABLE
 24974 | purge_test1                | 2022-05-20 15:16:49.466221+08 | TABLE
(5 rows)

test=# purge table purge_test;
PURGE
test=# select * from recyclebin;
  oid  |       original_name        |           droptime            | type
-------+----------------------------+-------------------------------+-------
 24968 | range_partition_tb_part_02 | 2022-05-20 15:14:53.581052+08 | TABLE
 24965 | range_partition_tb_part_01 | 2022-05-20 15:14:53.581494+08 | TABLE
 24962 | range_partition_tb         | 2022-05-20 15:14:53.582074+08 | TABLE
 24974 | purge_test1                | 2022-05-20 15:16:49.466221+08 | TABLE
(4 rows)

test=# purge recyclebin;
PURGE
test=# select * from recyclebin;
 oid | original_name | droptime | type
-----+---------------+----------+------
(0 rows)
```

3.42.5　插件 kdb_flashback 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例

```
shared_preload_libraries = ''
```

3.42.6　插件 kdb_flashback 升级方法

kdb_flashback 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.43　kdb_license

3.43.1　插件 kdb_license 简介

插件 kdb_license 是 KingbaseES 的一个扩展插件。主要功能是管理服务器 license 相关的信息。默认加载，属于内置功能。

- 插件名为 kdb_license
- 插件版本 V1.0

3.43.2　插件 kdb_license 加载方式

KingbaseES 数据库默认在初始化数据库时，自动加载 kdb_license 扩展插件。无须人为加载。

3.43.3　插件 kdb_license 的参数配置

无需配置任何参数

3.43.4　插件 kdb_license 的使用方法

KingbaseES 数据库初始化时自动加载 kdb_license 扩展插件。通常用户不直接使用该扩展插件。

3.43.5　插件 kdb_license 卸载方法

kdb_license 无法人为卸载。

3.43.6 插件 kdb_license 升级方法

kdb_license 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.44 kdb_mysql_functions

3.44.1 插件 kdb_mysql_functions 简介

kdb_mysql_functions 是 KingbaseES 中的一个扩展插件，能够提供一些兼容 MySQL 的函数。

3.44.2 插件 kdb_mysql_functions 加载方式

插件 kdb_mysql_functions 是系统初始化插件，系统自动加载。

3.44.3 插件 kdb_mysql_functions 配置参数

无需配置任何参数

3.44.4 插件 kdb_mysql_functions 使用方法

kdb_mysql_functions 提供的函数具体语法和使用内容可参考《KingbaseES 应用开发指南 》的函数 章节。

下表列出了 kdb_mysql_functions 扩展所提供的函数。

表 3.44.1: kdb_mysql_functions 函数列表

函数名称
ADDDATE
ADDTIME
CONVERT_TZ
DATEDIFF
DAY
DAYNAME
DAYOFMONTH

-续表-

函数名称
DAYOFWEEK
DAYOFYEAR
DATE_SUB
EXTRACT（MySQL 模式）
FORMAT_PICO_TIME
FROM_DAYS
FROM_UNIXTIME
GET_FORMAT
HOUR
MAKEDATE
MAKETIME
MICROSECOND
MINUTE
MONTHNAME
CURRENT_TIME（MySQL 模式）
CURRENT_TIMESTAMP（MySQL 模式）
LOCALTIME（MySQL 模式）
LOCALTIMESTAMP（MySQL 模式）
PERIOD_ADD
PERIOD_DIFF
QUARTER
SECOND
SEC_TO_TIME
SUBTIME
TIME（MySQL 模式）

-续表-

函数名称
TIMEDIFF
TIMESTAMP（MySQL 模式）
TIMESTAMPADD
TIMESTAMPDIFF
TO_DAYS
TO_SECONDS
UTC_DATE
UTC_TIME
UTC_TIMESTAMP
WEEK
WEEKDAY
WEEKOFYEAR
YEARWEEK

3.44.5　插件 kdb_mysql_functions 卸载方法

插件 kdb_mysql_functions 是系统初始化插件，用户无法卸载。

3.44.6　插件 kdb_mysql_functions 升级方法

kdb_mysql_functions 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.45 kdb_mysql_rum

3.45.1 插件 kdb_mysql_rum 简介

kdb_mysql_rum 插件功能与rum 插件相同，kdb_mysql_rum 用于使用 MySQL 模式下的 RUM 索引类型，具体参考：rum 索引类型章节。

3.45.2 插件 kdb_mysql_rum 加载方式

```
CREATE EXTENSION kdb_mysql_rum;
```

3.45.3 插件 kdb_mysql_rum 的参数配置

无需配置任何参数

3.45.4 插件 kdb_mysql_rum 的使用方法

与 rum 插件相同，具体参考：rum 索引类型章节。需要注意的是，MySQL 模式下的距离操作符为 <==> 。

3.45.5 插件 kdb_mysql_rum 卸载方法

```
DROP EXTENSION kdb_mysql_rum;
```

3.45.6 插件 kdb_mysql_rum 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION kdb_mysql_rum UPDATE TO '1.1';
```

3.46 kdb_oracle_datatype

3.46.1 插件 kdb_oracle_datatype 简介

插件 kdb_oracle_datatype 初始化内置，兼容 Oracle 数据类型。

- 插件名为 kdb_oracle_datatype
- 插件版本 V1.3

3.46.2 插件 kdb_oracle_datatype 加载方式

默认已经创建，如果没有创建需要在命令行执行 create extension kdb_oracle_datatype;

示例：

```
create extension kdb_oracle_datatype;
```

3.46.3 插件 kdb_oracle_datatype 的参数配置

无需配置任何参数。

3.46.4 插件 kdb_oracle_datatype 的使用方法

加载插件 kdb_oracle_datatype 后，按照如下方式使用相应功能。

3.46.4.1 支持 blob、clob、nclob 类型导入导出和置空

实现数值类型转换为字符类型

语法

调用如下函数使用：

```
blob_import(text)、blob_export(bytea, text)、empty_blob()、
clob_import(text, text)、clob_export(text, text, text)、empty_clob()、
nclob_import(text, text)、nclob_export(text, text, text)、empty_nclob()
```

示例

```
select blob_export('a'::bytea,'test'::text);
blob_export
-------------
```

```
   1
(1 行记录)

select blob_import('test'::text);
blob_import
-------------
a
(1 行记录)

select empty_blob();
empty_blob
------------

(1 行记录)
```

3.46.4.2　bpcharbyte 类型

支持 bpcharbyte 类型

示例

```
select '12'::bpcharbyte;
bpcharbyte
------------
12
(1 行记录)
```

3.46.4.3　bpcharbyte 类型操作符

bpcharbyte 类型之间支持 =、<>、>、>=、<、<= 和 ~<~、~>~、~<=~、~、!~、~*、!~*、、!、*、!* 操作符。

示例

```
select '12'::bpcharbyte ~>=~ '12'::bpcharbyte;
?column?
----------
t
(1 行记录)
```

3.46.4.4　bpcharbyte 类型转换

支持 bpcharbyte 类型和 text、bpchar、varchar 类型转换。

示例：

```
select '12'::bpcharbyte::varchar;
varchar
---------
12
(1 行记录)

select '12'::bpcharbyte::text;
text
------
12
(1 行记录)

select '12'::bpcharbyte::bpchar;
bpchar
--------
12
(1 行记录)
```

3.46.5　插件 kdb_oracle_datatype 卸载方法

系统启动自带插件，无法卸载。

3.46.6　插件 kdb_oracle_datatype 升级方法

kdb_oracle_datatype 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.47　kdb_oracle_datatype_nls

3.47.1　插件 kdb_oracle_datatype_nls 简介

插件 kdb_oracle_datatype_nls 兼容 oracle 日期 date 类型。此插件已经合入到插件 kdb_oracle_datatype 中。

- 插件名为 kdb_oracle_datatype_nls
- 插件版本 V1.0

3.47.2　插件 kdb_oracle_datatype_nls 加载方式

此插件已经合入到插件 kdb_oracle_datatype 中，跟随插件 kdb_oracle_datatype 加载自己加载。

3.47.3　插件 kdb_oracle_datatype_nls 的参数配置

无需配置任何参数。

3.47.4　插件 kdb_oracle_datatype_nls 的使用方法

3.47.4.1　操作符的支持

支持操作符 +、-

示例：

```
select '2019-01-01'::date + '1'::interval as A;
          a
---------------------
2019-01-01 00:00:01
(1 行记录)


select '2019-01-03'::date - '2019-01-01'::date as A;
a
---
2
(1 行记录)
```

3.47.4.2　索引的支持

支持 hash 和 brin 索引。

示例：

```
create table t1(c1 date);
create index hash_index on t1 using hash(c1);

create table t1(c1 date);
create index brin_index on t1 using brin(c1);
```

3.47.5　插件 kdb_oracle_datatype_nls 卸载方法

此插件已经合入到插件 kdb_oracle_datatype 中，插件 kdb_oracle_datatype 为系统启动自带插件，无法卸载。

3.47.6　插件 kdb_oracle_datatype_nls 升级方法

kdb_oracle_datatype_nls 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.48　kdb_orafce

3.48.1　插件 kdb_orafce 简介

kdb_orafce 插件用来兼容 oracle 的 packages 和 functions。

- 插件名为 kdb_orafce
- 插件版本 v3.9

3.48.2　插件 kdb_orafce 加载方式

使用时需要在命令行执行 create extension kdb_orafce;

示例：

```
create extension kdb_orafce;
```

3.48.3　插件 kdb_orafce 的参数配置

无需配置任何参数。

3.48.4　插件 kdb_orafce 的使用方法

加载插件 kdb_orafce 后，按照如下方式使用相应功能。

3.48.4.1　sys_catalog.trunc

围绕特定的参数截断日期。

语法

```
sys_catalog.trunc(value timestamp without time zone, fmt text) return timestamp without time zone;
```

参数

value：指定被截断的值

fmt：指定被截断的格式

返回值

返回被截断的结果值。

注解

fmt 可以为’year’,‘month’,’day’等。

示例

```
select sys_catalog.trunc(now,'year');
     trunc
------------------------
2022-01-01 00:00:00+00
(1 行记录)
```

sys_catalog.round ~~~~~~~~~~~~~~~~~

围绕特定的参数生成日期的近似数。

语法

```
sys_catalog.round(value timestamp without time zone, fmt text) return timestamp without time zone;
```

参数

value：指定被截断的值

fmt：指定被截断的格式

返回值

返回被截断的结果值的近似值。

注解

fmt 可以为’year’,‘month’,’day’等。

示例

```
select sys_catalog.round(now(),'year');
     round
------------------------
2022-01-01 00:00:00+00
(1 行记录)
```

3.48.4.2　sys_catalog.next_day

返回大于指定值的第一个符合目标值的日期。

语法

```
sys_catalog.next_day(value sys_catalog.date, weekday text) return sys_catalog.date;
sys_catalog.next_day(value sys_catalog.date, weekday integer) return sys_catalog.date;
```

参数

value：指定被操作的日期值。

weekday：指定的目标值。

返回值

返回大于指定值的第一个符合目标值的日期。

注解

weekday 可以为’mon’,‘mon’,’monday’等。也可以为 1-7(其中 1 代表周日，2 代表周一…)。

示例

```
select sys_catalog.next_day('2021-12-29'::timestamp without time zone, 'mon');
  next_day
------------
 2022-01-03
(1 行记录)

select sys_catalog.next_day('2021-12-29'::timestamp without time zone, 7);
  next_day
------------
 2022-01-01
(1 行记录)
```

3.48.4.3　sys_catalog.last_day

获取指定日期的所在月份的最后一天。

语法

```
sys_catalog.last_day(value sys_catalog.date) return sys_catalog.date;
```

参数

value 指定被操作的日期值。

返回值

返回指定日期的所在月份的最后一天。

示例

```
select sys_catalog.last_day('2021-11-29'::timestamp without time zone);
  last_day
------------
 2021-11-30
(1 行记录)
```

3.48.4.4　sys_catalog.months_between

获取两个日期之间间隔的月份。

语法

```
sys_catalog.months_between(date1 sys_catalog.date, date2 sys_catalog.date) return numeric;
```

参数

date1：日期 1。

date2：日期 2。

返回值

返回两个日期之间间隔的月份（并不是整数）。

示例

```
select sys_catalog.months_between('2021-12-29','2021-11-15');
  months_between
------------------
 1.45161290322581
(1 行记录)
```

3.48.4.5　sys_catalog.add_months

获取指定的日期增加特定个月份后的日期值。

语法

```
sys_catalog.add_months(day sys_catalog.date, value int) return sys_catalog.date;
```

参数

day：指定被加的日期。

value：指定增加的月份的值。

返回值

返回指定的日期增加特定个月份后的日期值。

示例

```
select sys_catalog.add_months(now(),12);
 add_months
------------
 2022-12-29
(1 行记录)
```

3.48.4.6　sys_catalog.set_nls_sort

使指定的字符集与默认的字符集进行比较和排序。

语法

```
sys_catalog.set_nls_sort(text);
```

参数

text 指定参与比较的字符集。

返回值

无。

示例

```
select sys_catalog.set_nls_sort('en_us.utf8');
 set_nls_sort
--------------

(1 行记录)
```

sys_catalog.instr: ^^^^^^^^^^^^^^^^^^

获取匹配的子串的位置值。

语法

```
sys_catalog.instr(str text, patt text, _start int, nth int) return int;
sys_catalog.instr(str text, patt text, _start int) return int;
sys_catalog.instr(str text, patt text) return int;
```

参数

str 被搜索的字符串。

返回值

匹配的子串的位置值，从 0 开始，没有匹配到返回 0。

示例

```
select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy!','handsome');
 instr
-------
  10
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy!','handsome',15);
 instr
-------
  53
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'handsome',15);
 instr
-------
  53
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'handsome',15,1);
 instr
-------
  53
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'boy',15);
 instr
-------
  19
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'boy',15,1);
 instr
-------
  19
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'boy',15,2);
```

```
 instr
-------
  62
(1 行记录)

select sys_catalog.instr('zqz is a handsome boy.is this correct.yeah , a very handsome boy! a good boy!',
'boy',15,3);
 instr
-------
  74
(1 行记录)
```

3.48.4.7　sys_catalog.to_char

将 ** 参数 ** 转化为 text 类型。

语法

```
sys_catalog.to_char(num smallint) return text;
sys_catalog.to_char(num int) return text;
sys_catalog.to_char(num bigint) return text;
sys_catalog.to_char(num real) return text;
sys_catalog.to_char(num double precision) return text;
sys_catalog.to_char(num numeric) return text;
```

参数

num 被转换的目标值。

返回值

返回 text 类型的参数值。

示例

```
select sys_catalog.to_char(100);
 to_char
---------
 100
(1 行记录)
```

3.48.4.8　sys_catalog.to_number

将参数转化为 numeric。

语法

```
sys_catalog.to_number(str text) return numeric;
```

参数

str 被转换的目标字符串。

返回值

返回 numeric 类型的 ** 参数 ** 值。

示例

```
select sys_catalog.to_number('100');
 to_number
-----------
     100
(1 行记录)
```

3.48.4.9　sys_catalog.to_date

将字符串转化为 timestamp。

语法

```
sys_catalog.to_date(str text) return timestamp;
```

参数

str 被转换的目标字符串。

返回值

返回 timestamp 类型的 ** 参数 ** 值。

示例

```
select sys_catalog.to_date('2021-12-29');
     to_date
---------------------
 2021-12-29 00:00:00
(1 行记录)
```

3.48.4.10　sys_catalog.to_multi_byte

将单字节字符串转化为双字节字符串。

语法

```
sys_catalog.to_multi_byte(str text) return text;
```

参数

str 被转换的单字节字符串。

返回值

返回被转化为双字节的 text 值。

注解

使用时要注意数据库所在的字符集。

示例

```
select to_multi_byte('!@#$%^&*');
  to_multi_byte
------------------
 ！@＃＄％＾＆＊
(1 行记录)
```

3.48.4.11　sys_catalog.to_single_byte

将双字节字符串转化为单字节字符串。

语法

```
sys_catalog.to_single_byte(str text) return text;
```

参数

str 被转换的双字节字符串。

返回值

返回被转化为单字节的 text 值。

注解

使用时要注意数据库所在的字符集。

示例

```
select to_single_byte(to_multi_byte('!@#$%^&*'));
 to_single_byte
----------------
 !@#$%^&*
(1 行记录)
```

3.48.4.12　sinh

计算双曲（线）正弦值

语法

```
sinh(float8) return float8
```

参数

float8 角度值。

返回值

返回双曲（线）正弦值。

示例

```
select sinh(0);
 sinh
------
  0
(1 行记录)
```

3.48.4.13　cosh

计算双曲余弦值

语法

```
cosh(float8) return float8;
```

参数

float8 角度值。

返回值

返回双曲（线）余弦值。

示例

```
select cosh(0);
 cosh
------
  1
(1 行记录)
```

3.48.4.14　tanh

计算双曲正切反双曲正弦值。

语法

```
tanh(float8) return float8;
```

参数

float8 角度值。

返回值

返回 numeric 类型的 ** 参数 ** 值。

示例

```
select tanh(0);
 tanh
------
  0
(1 行记录)
```

3.48.4.15　dump

分析输入的参数。

语法

```
dump("any") return varchar;
dump("any", integer) return varchar;
```

参数

“any”：可以为任何类型的参数。

integer：可以为 2,8,10,16。

返回值

返回解析的参数的类型的 oid，类型长度以及类型对应的值。

integer 指定要返回的类型的值的进制。

示例

```
select dump('123'::int,8);
      dump
-------------------------
 typ=23 len=4: 173,0,0,0
```

```
(1 行记录)

select dump('123',8);
       dump
---------------------------
 typ=705 len=4: 61,62,63,0
(1 行记录)
```

3.48.4.16　plvstr.rvrs

倒转字符串或者部分子串。

语法

```
plvstr.rvrs(str text, _start int, _end int) return text;
plvstr.rvrs(str text, _start int) return text;
plvstr.rvrs(str text) return text;
```

参数

str：目标串。

_start：子串开始的位置值。

_end：子串结束的位置值。

返回值

返回倒转后的字符串。

示例

```
select plvstr.rvrs('abcdefg');
  rvrs
---------
 gfedcba
(1 行记录)

select plvstr.rvrs('abcdefg',0);
  rvrs
---------
 gfedcba
(1 行记录)

select plvstr.rvrs('abcdefg',1);
  rvrs
---------
```

```
 gfedcba
(1 行记录)

select plvstr.rvrs('abcdefg',2);
  rvrs
--------
 gfedcb
(1 行记录)

select plvstr.rvrs('abcdefg',2,3);
 rvrs
------
 cb
(1 行记录)
```

3.48.4.17　oracle.substr

截断字符串。

语法

```
oracle.substr(str text, _start numeric) return text;
oracle.substr(str text, _start numeric, len numeric);
```

参数

str：目标串。

_start：截断的位置。

len：截断的长度。

返回值

返回截断后的字符串。

示例

```
select oracle.substr('zqz is a handsome boy',2);
    substr
----------------------
 qz is a handsome boy
(1 行记录)

select oracle.substr('zqz is a handsome boy',2,10);
   substr
------------
```

```
 qz is a ha
(1 行记录)
```

3.48.4.18　oracle.add_days_to_timestamp

增加日期并转换为 timestamp;

语法

```
oracle.add_days_to_timestamp(oracle.date,integer) return timestamp;
oracle.add_days_to_timestamp(oracle.date,bigint) return timestamp;
oracle.add_days_to_timestamp(oracle.date,smallint) return timestamp;
oracle.add_days_to_timestamp(oracle.date,numeric) return timestamp;
```

参数

oracle.date 目标日期。

返回值

返回增加指定天数后的 timestamp 值。

第二个 ** 参数 ** 可以带有小数。

示例

```
select oracle.add_days_to_timestamp(now(),1);
 add_days_to_timestamp
------------------------
 2021-12-31 10:37:46
(1 行记录)

select oracle.add_days_to_timestamp(now(),2);
 add_days_to_timestamp
------------------------
 2022-01-01 10:37:48
(1 行记录)

select oracle.add_days_to_timestamp(now(),2.5);
 add_days_to_timestamp
------------------------
 2022-01-01 22:37:51
(1 行记录)
```

3.48.4.19　oracle.subtract

减去日期并转换为 timestamp;

语法

```
oracle.subtract (oracle.date, integer) return timestamp;
oracle.subtract (oracle.date, bigint) return timestamp;
oracle.subtract (oracle.date, smallint) return timestamp;
oracle.subtract (oracle.date, numeric) return timestamp;
oracle.subtract(oracle.date,oracle.date) return double precision;
```

参数

oracle.date 目标日期。

返回值

返回减去指定天数或者日期后的 timestamp 值。

第二个 ** 参数 ** 可以带有小数，也可以是指定的日期。

示例

```
select oracle.subtract(now(),2.5);
    subtract
---------------------
 2021-12-27 22:42:32
(1 行记录)

select oracle.subtract(now(),2);
    subtract
---------------------
 2021-12-28 10:42:36
(1 行记录)

select oracle.subtract(now(),now()-1);
 subtract
----------
    1
(1 行记录)
```

3.48.4.20　oracle.add_months

给指定的日期增加月份。

语法

```
oracle.add_months(timestamp with time zone,integer) return timestamp;
```

参数

timestamp：目标 timestamp 值。

integer：增加指定的月份的值。

返回值

返回增加指定月份后的 timestamp 值。

第二个 ** 参数 ** 可以带有小数，也可以是负数。

示例

```
select oracle.add_months(now(),-1);
   add_months
---------------------
 2021-11-30 10:45:47
(1 行记录)

select oracle.add_months(now(),+1);
   add_months
---------------------
 2022-01-30 10:45:53
(1 行记录)

select oracle.add_months(now(),20);
   add_months
---------------------
 2023-08-30 10:45:57
(1 行记录)

select oracle.add_months(now(),1.5);
   add_months
---------------------
 2022-02-28 10:46:00
(1 行记录)
```

3.48.4.21　oracle.last_day

获取指定日期所在的月份的最后一天。

语法

```
oracle.last_day(timestamptz) return timestamp;
```

参数

timestamp 目标 timestamp 值。

返回值

返回指定日期所在的月份的最后一天的 timestamp 值。

示例

```
select oracle.last_day(now());
    last_day
---------------------
 2021-12-31 10:47:12
(1 行记录)

select oracle.last_day(now()-30);
    last_day
---------------------
 2021-11-30 10:47:16
(1 行记录)

select oracle.last_day(now()+3);
    last_day
---------------------
 2022-01-31 10:47:23
(1 行记录)
```

3.48.4.22　oracle.months_between

获取两个日期值之间所间隔的月份

语法

```
oracle.months_between(timestamp with time zone,timestamp with time zone) return numberic;
```

参数

timestamp 目标 timestamp 值。

返回值

返回两个日期值之间所间隔的月份值。两个参数有顺序之分。

示例

```
select oracle.months_between(now(),'2021-01-01 14:31:00');
      months_between
------------------
 16.1612903225806
(1 行记录)

.. code::
select oracle.months_between(now(),'2021-11-30');
 months_between
----------------
        1
(1 行记录)

select oracle.months_between(now(),'2021-10-30');
 months_between
----------------
        2
(1 行记录)

select oracle.months_between(now(),'2022-1-30');
 months_between
----------------
       -1
(1 行记录)
```

3.48.4.23　oracle.next_day

获取指定的接下来的日期。

语法

```
oracle.next_day(timestamp with time zone,integer) return timestamp;
oracle.next_day(timestamp with time zone,text) return timestamp;
```

参数

timestamp：目标 timestamp 值。

integer：需要增加的日期的天数。

text：指定所获取的特定的日期。

返回值

返回指定的接下来的日期。text 可以是’sun’,’sunday’等。

示例

```
select oracle.next_day(now(),2);
    next_day
---------------------
 2022-01-03 13:58:07
(1 行记录)

select oracle.next_day(now(),'sun');
    next_day
---------------------
 2022-01-02 14:03:47
(1 行记录)
```

3.48.4.24　oracle.to_date

将 text 串转换为相对应的日期。

语法

```
oracle.to_date(text) return oracle.date;
oracle.to_date(t1 text, t2 text) return oracle.date;
```

参数

text 目标字符串。

返回值

返回对应的日期。t2 为对应的日期的格式。

示例

```
select oracle.to_date(now());
     to_date
---------------------
 2021-12-30 14:14:26
(1 行记录)

select oracle.to_date(now(), 'yyyy-mm-dd');
     to_date
---------------------
 2021-12-30 00:00:00
(1 行记录)

select oracle.to_date(now(), 'yyyy-mm');
     to_date
---------------------
```

```
 2021-12-01 00:00:00
(1 行记录)
```

3.48.4.25　oracle.to_char

将 timestamp 转换为相应的字符串。

语法

```
oracle.to_char(timestamp) return text;
```

参数

timestamp 目标值。

返回值

返回日期对应的字符串。

示例

```
select oracle.to_char(now());
      to_char
----------------------------
 2021-12-30 14:35:30.462850
(1 行记录)
```

3.48.4.26　oracle.sysdate

获取 session 所在时区的当前的时间。将 timestamp 转换为相应的字符串。

语法

```
oracle.sysdate() retrun text;
```

返回值

返回 session 所在时区的当前的时间。

示例

```
select oracle.sysdate();
     sysdate
---------------------
 2021-12-30 06:37:44
(1 行记录)
```

3.48.4.27　oracle.sessiontimezone

获取 session 的 timezone。

语法

```
oracle.sessiontimezone() return text;
```

参数

无

返回值

返回 session 所在的时区。

示例

```
select oracle.sessiontimezone();
 sessiontimezone
-----------------
 prc
(1 行记录)
```

3.48.4.28　oracle.dbtimezone

获取数据库的 timezone。

语法

```
oracle.dbtimezone() return text;
```

参数

无

返回值

返回数据库所在的时区。

示例

```
select oracle.dbtimezone();
 dbtimezone
------------
 gmt
(1 行记录)
```

3.48.5　插件 kdb_orafce 卸载方法

卸载时 drop extension kdb_orafce 即可。

示例：

```
drop extension kdb_orafce;
```

3.48.6　插件 kdb_orafce 升级方法

kdb_orafce 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.49　kdb_partman

3.49.1　插件 kdb_partman 简介

3.49.1.1　功能和发展

partman 是可帮助使用者更轻松地管理基于 time 或 serialid 的分区表。partman 只需要几个选项，就可以设置分区。

子表的创建全部由 partman 自身管理。对于非本地分区，插件还会维护触发器函数。具有现有数据的表，在非本地分区中，可以将其数据分区为易于管理的小批次；在本地分区中，必须首先创建新的父分区，并在创建过程完成后迁移数据。

3.49.1.2　插入数据机制解释

对于非本地分区，如果尝试将数据插入到超过分区集范围的分区集中，则该数据将放入该集的父表中。与自动创建新分区以匹配该数据相比，这是首选，因为这种策略能够规避插入非分区数据的错误可能会导致产生大量不需要的子表以及由于 DDL 事务而产生的竞争。

check_default() 函数提供对插入到父默认表中的数据的监视，如果数据是有效数据，则 partition_data_* 函数集可以轻松地为该数据分区。这比必须清理可能数百或数千个不需要的分区要容易得多，而且也比抛出错误并丢失数据更好！

3.49.1.3　插入数据的策略

请注意，将来的子表创建基于分区集中的当前数据。这意味着，如果放入”未来”数据，则新创建的表将不基于该值。如果不存在子表，这可能会导致在这期间的数据转到父/默认值，如上所述。

建议你将预制值设置得足够高，以包含要插入的预期数据范围。对于非本地分区，请设置 optimize_trigger 值以有效地处理最常见的数据范围。

3.49.1.4　partman 的设计约束和特性

3.49.1.4.1　时区

保证 partman 运行的操作系统的时区一致性非常重要，尤其是使用基于时间建立的分区。对 kdb_partman 函数的调用将使用客户端在调用函数时设置的时区。这与 KES 客户端的一般工作方式一致。

强烈建议在 UTC 时间内运行数据库系统，以解决由于夏令时更改而无法解决的问题。然后，还要确保将创建分区集和运行维护调用的客户端也设置为 UTC。

3.49.1.4.2　子分区

支持具有多个级别的子分区，但它在 KES 中的用途非常有限，并且在单个分区集（100TB，PB）中的超大数据之外，它几乎没有性能优势。如果你正在寻找提高性能的方法，请考虑在使用子分区之前调整分区间隔，它是数据组织和保留管理的主要方法。

对可以执行的子分区级别没有设置限制，但要明智，并牢记在单个继承集中管理多个表时的性能注意事项。此外，如果单个分区集中的表数变得非常大，则可能必须将 max_locks_per_transaction.conf 设置调整到默认值 64 以上。否则，你可能会遇到共享内存问题，甚至是群集崩溃问题。

如果在调用 run_maintenance() 对所有分区集进行常规维护时遇到竞争问题，可以将 part_config 表中的 automatic_maintenance 列设置为 false。但是，你必须调用 run_maintenance（parent_table），并且必须足够频繁地调用将来的分区。

如果你使用的是 KES V9，则可以使用新的 run_maintenance_proc（）过程来减少竞争问题，因为它在每个分区集的维护后会自动提交。本地子分区不支持逻辑复制的发布者/预订者机制。

3.49.1.4.3　Retention

如果不需要将数据保留在较旧的分区中，则可以使用保留机制来自动删除不需要的子分区。默认情况下，它们只是未继承/分离的，实际上不会被删除，但如果需要，可以对其进行配置。

还有一种方法可以在表不再需要位于数据库中但仍需要保留，要设置保留策略，请在 part_config 表的保留列中输入间隔或整数值。对于基于时间的分区，间隔值将设置仅包含早于该时间的数据的任何分区将被删除；

对于基于 id 的分区，整数值将设置 id 值小于当前最大 id 值减去保留值的任何分区。例如，如果当前最大 id 为 100，保留值为 30，则 id 值小于 70 的任何分区都将被删除。drop 函数运行始终参考的当前最大 id 值。

请记住，对于子分区集，当父表删除了子表时，如果该子表又被分区，则该删除是级联，在这个继承树下面的子表都将被删除。另请注意，由 kdb_partman 管理的分区集必须始终至少有一个子级，因此保留机制永远不会删除集中的最后一个子表。

3.49.1.4.4　约束排除

约束排除是 partman 的一大特性。但是，大多数分区设置的问题在于，这将仅用于分区控制列。如果对分区集中的任何其他列使用 WHERE 条件，则将对所有子表进行扫描，除非这些列上也存在约束。

预测列的值将是什么来预先创建约束可能非常困难或不可能。kdb_partman 具有对分区集中的旧表应用约束的功能，该分区集可能不再对它们进行任何编辑（”old” 被定义为早于 optimize_constraint 配置值）。

它检查给定列中的当前最小值/最大值，然后对该子表应用约束。这可以允许约束排除功能在 WHERE 条件下使用其他列时，潜在地消除扫描较旧的子表。这限制了编辑这些列的能力，但对于适用这些列的情况，它可能会对非常大的分区集的查询性能产生巨大影响。

因此，如果你只插入新数据，这可能非常有用，但如果在整个分区集中定期插入/更新数据，则作用有限。如果数据最终必须在那些较旧的分区中进行编辑，partman 还提供了轻松创建约束的功能。

3.49.1.4.5　自定义时间间隔注意事项

create_parent 函数中提供的时间间隔列表经过优化，可在非本地、基于触发器的分区中非常快地起作用。除了这些值之外，还可以使用其他间隔，但非常影响性能。

对于本地分区，与 kdb_partman 基于触发器的方法不同，对于任何给定的时间间隔，没有不同的分区方法。使用本地方法的所有可能的时间间隔都具有相同的性能特征，并且比任何基于触发器的方法都更好。

如果需要的分区间隔不同于 kdb_partman 提供的优化间隔，强烈建议升级到 KES V9。支持的最小时间间隔为 1 秒，上限由 KES 支持的最小和最大时间戳值限制，此时支持的最小整数间隔为 10。

首次运行 create_parent 函数创建分区集时，在确定要创建的第一个分区时，小于一天的会向小时舍入。例如小于 24 小时但大于 1 分钟的间隔舍入的最近小时。小于 1 分钟的间隔使用舍入的最接近的分钟。

当运行 create_parent 时，可能创建的分区多于预期，并且可能不会创建所有的分区。第一次运行 run_maintenance 将修复缺少的分区，发生这种情况是由于 partman 支持自定义时间间隔的特性造成的。

对于等于或大于 100 年的间隔，partman 将使用千年的实际开始来确定分区名称和约束规则。例如，21 世纪和第 3 个千禧年开始于 2001 年 1 月 1 日（不是 2000 年）。

3.49.1.4.6　命名长度限制

KES 的对象命名长度限制为 63 个字符。因此，如果 partman 尝试创建具有较长名称的对象，则在命名限制之下，将会自动截断对象名称中超出 63 个字符的部分。这可能会导致依赖于具有专门命名的后缀的分区名称出现明显问题。

partman 会自动处理所有子表、触发器函数和触发器的此问题。它将截断现有的父表名称以适合所需的后缀。

请注意，如果表的名称很长，名称相似，则当它们是单独分区集的一部分时，可能会遇到命名冲突。使用基于串行的分区时，请注意，随着时间的推移，表名将被越来越多地截断，以适应更长的分区后缀。因此，建议保留将分区的表名尽可能短。

3.49.1.4.7　唯一约束和更新

KES 中的表继承不允许将父表上的主键或唯一索引/约束应用于所有子表。该约束应用于每个单独的表，但不应用于整个分区集。

例如，这意味着粗心大意的应用程序可能会导致分区集中的主键值重复。同时，check_unique_constraint.py 脚本包含在 kdb_partman 中，可以提供监视。

重要说明：自 4.6.0 版和 KES 11+ 起，kdb_partman 不再支持本地分区的更新插入。请使用 KES 自带的 INSERT…ONCONFLICT 功能。

对于非本地分区和 KES V9 本地分区，INSERT …ONCONFLICT（upsert）在分区触发器和本地分区中都受支持，但都是受限制的。主要限制是，上面提到的，触发 ON CONFLICT 子句的约束冲突仅发生在实际包含数据的单个子表上。

比数据重复更大的问题是 ONCONFLICT DOUPDATE 从句，它可能不会触发并导致数据非常不一致。对于仅插入新数据的情况，upsert 可以提供显著的性能改进。

但是，如果你依赖较旧分区中的数据来导致 upsert 通常会处理的约束冲突，则它可能不起作用。此外，如果生成的 UPDATE 最终违反了该子表的分区约束，它将失败。

kdb_partman 和 KES V9 本地分区目前都不支持需要将行从一个子表移动到另一个子表的 UPDATES。这仅在 KES V8 以上的版本中支持。

kdb_partman 的更新插入功能是可选的，默认情况下关闭，并且仅包含在没有 KES 支持的版本上。目前，如果你尚未实现此功能，强烈建议你升级到 KES V9。

3.49.1.4.8　日志记录/监控

KES 的 Jobmon 插件是可选的，允许审核和监视分区维护。如果 jobmon 安装并配置正确，partman 将自动使用它，无需额外设置。还可以通过使用 part_config 表中的 jobmon 列或在初始设置期间选择 create_parent（）来为每个分区集单独打开或关闭 Jobmon。

请注意，如果你尝试对使用 pg_jobmon 的表进行分区，则必须将 create_parent（）中的 jobmon 选项设置为 false，否则它将被放入永久死锁，因为 pg_jobmon 将尝试写入时，就会分区。默认情况下，任何连续 3 次无法成功运行的函数都会导致 jobmon 引发警报。

这就是为什么默认的预做值设置为 4，以便及时引发警报以进行干预，而无需对 jobmon 进行额外配置。当然，如果需要，你可以将 jobmon 配置为在 3 次故障之前（或之后）发出警报。

如果你在生产环境中运行 partman，强烈建议你安装 jobmon 并配置某种第三方监控（Nagios，Circonus 等），以便在分区失败时发出警报。

3.49.1.4.9　后台进程

KingbaseES 支持创建自定义后台工作线程并在运行时动态加载它们的功能。kdb_partman 的 BGW 基本上只是一个调度程序，它为你运行 run_maintenance（）函数，因此你不必使用外部调度程序（cron 等）。

现在，它与直接调用 run_maintenance（）没有任何不同，但将来可能会改变。如果需要直接在任何特定分区集上调用 run_maintenance（），则仍需要使用外部调度程序手动执行此操作。这仅将 part_config 中的 automatic_maintenance 设置为 true 的分区集。

日志消息被输出到正常的 KES 日志文件，以记录 BGW 何时运行。如果 log_min_messages 设置为"DEBUG1"，则其他级别的日志记录消息被记录。

3.49.2　插件 kdb_partman 加载方式

KingbaseES 数据库默认不加载 kdb_partman, 如果用户需要使用 kdb_partman 的相关功能，请人为手动使用以下命令加载 kdb_partman 插件。

示例：

```
create extension kdb_partman;
```

3.49.3　插件 kdb_partman 的参数配置

无需配置任何参数

3.49.4　插件 kdb_partman 的使用方法

从 4.4.0 开始，SECURITY DEFINER 已从 kdb_partman 中的所有函数中删除。现在，要求超级用户使用 kdb_partman 对于本地分区是可选的。

若要以非超级用户身份运行，运行 kdb_partman 功能和维护的角色必须拥有它们管理的所有分区集的所有权，并有权在将包含它所管理的分区集的任何架构中创建对象。为了便于使用和权限管理，建议创建专用于分区管理的角色。

3.49.4.1　函数

3.49.4.1.1　create 函数

用于创建具有一个父表和继承子表的分区集

```
create_parent(p_parent_table text, p_control text, p_type text, p_interval text, p_constraint_cols text[]
DEFAULT NULL, p_premake int DEFAULT 4, p_automatic_maintenance text DEFAULT 'on', p_start_partition text
DEFAULT NULL, p_inherit_fk boolean DEFAULT true, p_epoch text DEFAULT 'none', p_upsert text DEFAULT '', p_
publications text[] DEFAULT NULL, p_trigger_return_null boolean DEFAULT true, p_template_table text DEFAULT
NULL, p_jobmon boolean DEFAULT true, p_date_trunc_interval text DEFAULT NULL)
RETURNS boolean
```

创建已存在的分区集的子分区集。

```
create_sub_parent(p_top_parent text, p_control text, p_type text, p_interval text, p_native_check text
DEFAULT NULL, p_constraint_cols text[] DEFAULT NULL, p_premake int DEFAULT 4, p_start_partition text DEFAULT
NULL, p_inherit_fk boolean DEFAULT true, p_epoch text DEFAULT 'none', p_upsert text DEFAULT '', p_trigger_
return_null boolean DEFAULT true, p_jobmon boolean DEFAULT true, p_date_trunc_interval text DEFAULT NULL)
RETURNS boolean
```

对在将父表设置为基于时间的分区集之前可能存在的数据进行分区。

```
partition_data_time(p_parent_table text, p_batch_count int DEFAULT 1, p_batch_interval interval DEFAULT
NULL, p_lock_wait numeric DEFAULT 0, p_order text DEFAULT 'ASC', p_analyze boolean DEFAULT true, p_source_
table text DEFAULT NULL, p_ignored_columns text[] DEFAULT NULL) RETURNS bigint
```

在将父表设置为 serial id 分区集之前可能存在的分区数据。

```
partition_data_id(p_parent_table text, p_batch_count int DEFAULT 1, p_batch_interval bigint DEFAULT NULL, p_
lock_wait numeric DEFAULT 0, p_order text DEFAULT 'ASC', p_analyze boolean DEFAULT true, p_source_table text
DEFAULT NULL, p_ignored_columns text[] DEFAULT NULL) RETURNS bigint
```

可以在不同的提交批处理中对数据进行分区，以避免长时间运行的事务和数据竞争问题。

```
partition_data_proc(p_parent_table text, p_interval text DEFAULT NULL, p_batch int DEFAULT NULL, p_wait int
DEFAULT 1, p_source_table text DEFAULT NULL, p_order text DEFAULT 'ASC', p_lock_wait int DEFAULT 0, p_lock_
wait_tries int DEFAULT 10, p_quiet boolean DEFAULT false, p_ignored_columns text[] DEFAULT NULL)
```

为给定的基于时间父表创建子分区。

```
create_partition_time(p_parent_table text, p_partition_times timestamptz[], p_analyze boolean DEFAULT true,
p_start_partition text DEFAULT NULL) RETURNS boolean
```

为给定的基于 serial id 父表创建子分区。

```
create_partition_id(p_parent_table text, p_partition_ids bigint[], p_analyze boolean DEFAULT true, p_start_
partition text DEFAULT NULL) RETURNS boolean
```

为非本地基于时间的分区创建触发器函数

```
create_function_time(p_parent_table text, p_job_id bigint DEFAULT NULL) RETURNS void
```

为非本地基于 serial id 的分区创建触发器函数

```
create_function_id(p_parent_table text, p_job_id bigint DEFAULT NULL) RETURNS void
```

3.49.4.1.2　维护函数

将此函数作为计划作业（cron 等）运行，以便为配置为使用它的分区集自动创建子表

```
run_maintenance(p_parent_table text DEFAULT NULL, p_analyze boolean DEFAULT NULL, p_jobmon boolean DEFAULT
true) RETURNS void
```

对于 KES V9 以上，这是运行分区维护与直接调用 run_maintenance（）函数的首选方法

```
run_maintenance_proc(p_wait int DEFAULT 0, p_analyze boolean DEFAULT NULL, p_jobmon boolean DEFAULT true)
```

运行此函数以监视 kdb_partman 管理的分区集的父表（非本地表）或默认表（本地 KES V9）是否未向其插入行。

```
check_default(p_exact_count boolean DEFAULT true)
```

列出由 kdb_partman 管理的给定分区集的所有子表。

```
show_partitions (p_parent_table text, p_order text DEFAULT 'ASC', p_include_default boolean DEFAULT false)
RETURNS TABLE (partition_schemaname text, partition_tablename text)
```

给定由 kdb_partman 管理的父表和适当的值，返回该值将存在于其中的子分区的名称。

```
show_partition_name (p_parent_table text, p_value text, OUT partition_table text, OUT suffix_timestamp
timestamp, OUT suffix_id bigint, OUT table_exists boolean)
```

给定一个 schema 限定的子表名，返回该子表的相关边界值以及追加到子表名的后缀。

```
@extschema@.show_partition_info (p_child_table text, p_partition_interval text DEFAULT NULL, p_parent_table
text DEFAULT NULL, OUT child_start_time timestamptz, OUT child_end_time timestamptz, OUT child_start_id
bigint, OUT child_end_id bigint, OUT suffix text) RETURNS record
```

用于返回必要命令以在 kdb_partman 中为给定的父表重新创建分区集的函数

```
@extschema@.dump_partitioned_table_definition(p_parent_table text, p_ignore_template_table boolean default
false) RETURNS text
```

用于填充给定父表的子表系列中可能存在的间隔的函数

```
partition_gap_fill(p_parent_table text) RETURNS integer
```

将约束应用于为已配置的列的给定分区集中的子表

```
apply_constraints(p_parent_table text, p_child_table text DEFAULT NULL, p_analyze boolean DEFAULT FALSE, p_
job_id bigint DEFAULT NULL) RETURNS void
```

删除由 kdb_partman 为在 part_config 中配置的列创建的约束。

```
drop_constraints(p_parent_table text, p_child_table text, p_debug boolean DEFAULT false)
```

KES V9 以上，kdb_partman 管理的额外约束

```
reapply_constraints_proc(p_parent_table text, p_drop_constraints boolean DEFAULT false, p_apply_constraints boolean DEFAULT false, p_wait int DEFAULT 0, p_dryrun boolean DEFAULT false)
```

根据父表设置的内容，对所有子表重新应用所有权和 grants。

```
reapply_privileges(p_parent_table text)
```

将分区集中的父表上存在的任何外键应用于所有子表。KES V8 之前使用

```
apply_foreign_keys(p_parent_table text, p_child_table text DEFAULT NULL, p_job_id bigint DEFAULT NULL, p_debug boolean DEFAULT false)
```

从 part_config_sub 表中删除 parent_table 条目

```
stop_sub_partition(p_parent_table text, p_jobmon boolean DEFAULT true) RETURNS boolean
```

3.49.4.1.3　销毁函数

销毁由 partman 创建的函数

```
undo_partition(p_parent_table text, p_batch_count int DEFAULT 1, p_batch_interval text DEFAULT NULL, p_keep_table boolean DEFAULT true, p_lock_wait numeric DEFAULT 0, p_target_table text DEFAULT NULL, p_ignored_columns text[] DEFAULT NULL, p_drop_cascade boolean DEFAULT false, OUT partitions_undone int, OUT rows_undone bigint) RETURNS record
```

在不同的提交批处理中取消分区数据，以避免长时间运行的事务和数据竞争问题

```
undo_partition_proc(p_parent_table text, p_interval text DEFAULT NULL, p_batch int DEFAULT NULL, p_wait int DEFAULT 1, p_target_table text DEFAULT NULL, p_keep_table boolean DEFAULT true, p_lock_wait int DEFAULT 0, p_lock_wait_tries int DEFAULT 10, p_quiet boolean DEFAULT false, p_ignored_columns text[] DEFAULT NULL, p_drop_cascade boolean DEFAULT false)
```

从基于时间的分区集中删除子表

```
drop_partition_time(p_parent_table text, p_retention interval DEFAULT NULL, p_keep_table boolean DEFAULT NULL, p_keep_index boolean DEFAULT NULL, p_retention_schema text DEFAULT NULL) RETURNS int
```

从基于 serial id 的分区集中删除子表

```
drop_partition_id(p_parent_table text, p_retention bigint DEFAULT NULL, p_keep_table boolean DEFAULT NULL, p_keep_index boolean DEFAULT NULL, p_retention_schema text DEFAULT NULL) RETURNS int
```

根据添加列的时间（在设置分区之前或之后），将其放在父级上可能会或可能不会从所有子级中删除它。

```
drop_partition_column(p_parent_table text, p_column text) RETURNS void
```

3.49.4.1.4　脚本

创建分区使用小批量的脚本。

```
partition_data_proc()
```

撤掉本地分区中使用小批量的脚本。

```
undo_partition_proc()
```

使用已有 schema 提取表中数据。

```
dump_partition.py
```

清理一个已有分区中的死数据。

```
vacuum_maintenance.py
```

在分区集中的子表上更改索引后，重新应用这些索引。

```
reapply_indexes.py
```

为表中配置的列重做给定分区集中子表 part_config 约束。

```
reapply_constraints_proc()
```

检查分区集中的所有行对于给定列是否唯一。

```
check_unique_constraints.py
```

3.49.4.1.5　编译

在 KingbaseES 编译之前需要确保 contrib 目录的 makefile 已经添加了 kdb_partman，可以随 KingbaseES 编译。

3.49.4.1.6　示例

3.49.4.1.6.1　创建本地分区

一天一分区

用模版添加主键

```
CREATE SCHEMA IF NOT EXISTS partman;
CREATE EXTENSION kdb_partman SCHEMA partman;

CREATE TABLE partman.time_taptest_table
    (col1 int,
    col2 text default 'stuff',
    col3 timestamptz NOT NULL DEFAULT now())
PARTITION BY RANGE (col3);

CREATE INDEX ON partman.time_taptest_table (col3);
```

```
\d+ partman.time_taptest_table
                                Partitioned table "partman.time_taptest_table"
 Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
 col1   | integer                  |           |          |               | plain    |              |
 col2   | text                     |           |          | 'stuff'::text | extended |              |
 col3   | timestamp with time zone |           | not null | now()         | plain    |              |
Partition key: RANGE (col3)
Indexes:
    "time_taptest_table_col3_idx" btree (col3)
Number of partitions: 0
```

```
CREATE TABLE partman.time_taptest_table_template (LIKE partman.time_taptest_table);
ALTER TABLE partman.time_taptest_table_template ADD PRIMARY KEY (col1);
```

```
\d partman.time_taptest_table_template
         Table "partman.time_taptest_table_template"
Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
col1   | integer                  |           | not null |
col2   | text                     |           |          |
col3   | timestamp with time zone |           | not null |
Indexes:
   "time_taptest_table_template_pkey" PRIMARY KEY, btree (col1)
```

```
SELECT create_parent('partman.time_taptest_table', 'col3', 'native', 'daily', p_template_table := 'partman.
time_taptest_table_template');
 create_parent
---------------
 t
(1 row)
```

```
\d+ partman.time_taptest_table
                                Partitioned table "partman.time_taptest_table"
 Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
 col1   | integer                  |           |          |               | plain    |              |
 col2   | text                     |           |          | 'stuff'::text | extended |              |
 col3   | timestamp with time zone |           | not null | now()         | plain    |              |
Partition key: RANGE (col3)
Indexes:
    "time_taptest_table_col3_idx" btree (col3)
Partitions: partman.time_taptest_table_p2020_10_26 FOR VALUES FROM ('2020-10-26 00:00:00-04') TO ('2020-10-
27 00:00:00-04'),
            partman.time_taptest_table_p2020_10_27 FOR VALUES FROM ('2020-10-27 00:00:00-04') TO ('2020-10-
28 00:00:00-04'),
            partman.time_taptest_table_p2020_10_28 FOR VALUES FROM ('2020-10-28 00:00:00-04') TO ('2020-10-
29 00:00:00-04'),
            partman.time_taptest_table_p2020_10_29 FOR VALUES FROM ('2020-10-29 00:00:00-04') TO ('2020-10-
30 00:00:00-04'),
            partman.time_taptest_table_p2020_10_30 FOR VALUES FROM ('2020-10-30 00:00:00-04') TO ('2020-10-
31 00:00:00-04'),
            partman.time_taptest_table_p2020_10_31 FOR VALUES FROM ('2020-10-31 00:00:00-04') TO ('2020-11-
01 00:00:00-04'),
            partman.time_taptest_table_p2020_11_01 FOR VALUES FROM ('2020-11-01 00:00:00-04') TO ('2020-11-
02 00:00:00-05'),
            partman.time_taptest_table_p2020_11_02 FOR VALUES FROM ('2020-11-02 00:00:00-05') TO ('2020-11-
03 00:00:00-05'),
            partman.time_taptest_table_p2020_11_03 FOR VALUES FROM ('2020-11-03 00:00:00-05') TO ('2020-11-
04 00:00:00-05'),
            partman.time_taptest_table_default DEFAULT
```

```
\d+ partman.time_taptest_table_p2020_10_26
                                Table "partman.time_taptest_table_p2020_10_26"
 Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
 col1   | integer                  |           | not null |               | plain    |              |
 col2   | text                     |           |          | 'stuff'::text | extended |              |
 col3   | timestamp with time zone |           | not null | now()         | plain    |              |
Partition of: partman.time_taptest_table FOR VALUES FROM ('2020-10-26 00:00:00-04') TO ('2020-10-27 00:00:
00-04')
Partition constraint: ((col3 IS NOT NULL) AND (col3 >= '2020-10-26 00:00:00-04'::timestamp with time zone)
AND (col3 < '2020-10-27 00:00:00-04'::timestamp with time zone))
```

```
Indexes:
    "time_taptest_table_p2020_10_26_pkey" PRIMARY KEY, btree (col1)
    "time_taptest_table_p2020_10_26_col3_idx" btree (col3)
Access method: heap
```

3.49.4.1.6.2　10 个 id 值一个分区

10 个 ID 值一个分区

父表创建子表之后添加主键，子表不能自动添加主键，需要手动添加。

```
CREATE TABLE partman.id_taptest_table (
    col1 bigint
    , col2 text not null
    , col3 timestamptz DEFAULT now()
    , col4 text) PARTITION BY RANGE (col1);

CREATE INDEX ON partman.id_taptest_table (col1);
```

```
\d+ partman.id_taptest_table
                              Partitioned table "partman.id_taptest_table"
 Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
 col1   | bigint                   |           |          |         | plain    |              |
 col2   | text                     |           | not null |         | extended |              |
 col3   | timestamp with time zone |           |          | now()   | plain    |              |
 col4   | text                     |           |          |         | extended |              |
Partition key: RANGE (col1)
Indexes:
    "id_taptest_table_col1_idx" btree (col1)
Number of partitions: 0
```

```
SELECT partman.create_parent('partman.id_taptest_table', 'col1', 'native', '10');
 create_parent
---------------
 t
(1 row)
```

```
\d+ partman.id_taptest_table
                              Partitioned table "partman.id_taptest_table"
 Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
 col1   | bigint                   |           |          |         | plain    |              |
 col2   | text                     |           | not null |         | extended |              |
```

```
 col3   | timestamp with time zone |           |          | now()   | plain    |              |
 col4   | text                     |           |          |         | extended |              |
Partition key: RANGE (col1)
Indexes:
    "id_taptest_table_col1_idx" btree (col1)
Partitions: partman.id_taptest_table_p0 FOR VALUES FROM ('0') TO ('10'),
            partman.id_taptest_table_p10 FOR VALUES FROM ('10') TO ('20'),
            partman.id_taptest_table_p20 FOR VALUES FROM ('20') TO ('30'),
            partman.id_taptest_table_p30 FOR VALUES FROM ('30') TO ('40'),
            partman.id_taptest_table_p40 FOR VALUES FROM ('40') TO ('50'),
            partman.id_taptest_table_default DEFAULT
```

```
select template_table from partman.part_config where parent_table = 'partman.id_taptest_table';
                 template_table
-----------------------------------------------
      template_partman_id_taptest_table
```

```
ALTER TABLE template_partman_id_taptest_table ADD PRIMARY KEY (col2);
```

```
INSERT INTO id_taptest_table (col1, col2) VALUES (generate_series(1,20), generate_series(1,20)::text||'stuff
'::text);
```

```
CALL run_maintenance();

\d+ partman.id_taptest_table
                            Partitioned table "partman.id_taptest_table"
 Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
 col1   | bigint                   |           |          |         | plain    |              |
 col2   | text                     |           | not null |         | extended |              |
 col3   | timestamp with time zone |           |          | now()   | plain    |              |
 col4   | text                     |           |          |         | extended |              |
Partition key: RANGE (col1)
Indexes:
    "id_taptest_table_col1_idx" btree (col1)
Partitions: partman.id_taptest_table_p0 FOR VALUES FROM ('0') TO ('10'),
            partman.id_taptest_table_p10 FOR VALUES FROM ('10') TO ('20'),
            partman.id_taptest_table_p20 FOR VALUES FROM ('20') TO ('30'),
            partman.id_taptest_table_p30 FOR VALUES FROM ('30') TO ('40'),
            partman.id_taptest_table_p40 FOR VALUES FROM ('40') TO ('50'),
            partman.id_taptest_table_p50 FOR VALUES FROM ('50') TO ('60'),
            partman.id_taptest_table_p60 FOR VALUES FROM ('60') TO ('70'),
            partman.id_taptest_table_default DEFAULT
```

```
\d partman.id_taptest_table_p40
             Table "partman.id_taptest_table_p40"
 Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
 col1   | bigint                   |           |          |
 col2   | text                     |           | not null |
 col3   | timestamp with time zone |           |          | now()
 col4   | text                     |           |          |
Partition of: partman.id_taptest_table FOR VALUES FROM ('40') TO ('50')
Indexes:
    "id_taptest_table_p40_col1_idx" btree (col1)

\d partman.id_taptest_table_p50
             Table "partman.id_taptest_table_p50"
 Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
 col1   | bigint                   |           |          |
 col2   | text                     |           | not null |
 col3   | timestamp with time zone |           |          | now()
 col4   | text                     |           |          |
Partition of: partman.id_taptest_table FOR VALUES FROM ('50') TO ('60')
Indexes:
    "id_taptest_table_p50_pkey" PRIMARY KEY, btree (col2)
    "id_taptest_table_p50_col1_idx" btree (col1)

\d partman.id_taptest_table_p60
             Table "partman.id_taptest_table_p60"
 Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
 col1   | bigint                   |           |          |
 col2   | text                     |           | not null |
 col3   | timestamp with time zone |           |          | now()
 col4   | text                     |           |          |
Partition of: partman.id_taptest_table FOR VALUES FROM ('60') TO ('70')
Indexes:
    "id_taptest_table_p60_pkey" PRIMARY KEY, btree (col2)
    "id_taptest_table_p60_col1_idx" btree (col1)
```

```
ALTER TABLE partman.id_taptest_table_p0 ADD PRIMARY KEY (col2);
ALTER TABLE partman.id_taptest_table_p10 ADD PRIMARY KEY (col2);
ALTER TABLE partman.id_taptest_table_p20 ADD PRIMARY KEY (col2);
ALTER TABLE partman.id_taptest_table_p30 ADD PRIMARY KEY (col2);
ALTER TABLE partman.id_taptest_table_p40 ADD PRIMARY KEY (col2);
```

3.49.4.1.6.3　分区一个已经存在的表

3.49.4.1.6.4　离线分区

数据无法在迁移前后同时访问，优势在于可以分为比分区间隔更小的批次传输，同时只需要较少的重命名次数。

```
CREATE TABLE public.original_table (
    col1 bigint not null
    , col2 text not null
    , col3 timestamptz DEFAULT now()
    , col4 text);

CREATE INDEX ON public.original_table (col1);

INSERT INTO public.original_table (col1, col2, col3, col4) VALUES (generate_series(1,100000), 'stuff'||
generate_series(1,100000), now(), 'stuff');
```

```
ALTER TABLE public.original_table RENAME to old_nonpartitioned_table;
```

```
CREATE TABLE public.original_table (
    col1 bigint not null
    , col2 text not null
    , col3 timestamptz DEFAULT now()
    , col4 text) PARTITION BY RANGE (col1);

CREATE INDEX ON public.original_table (col1);

SELECT partman.create_parent('public.original_table', 'col1', 'native', '10000');
```

```
\d+ original_table;
                                Partitioned table "public.original_table"
 Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
 col1   | bigint                   |           | not null |         | plain    |              |
 col2   | text                     |           | not null |         | extended |              |
 col3   | timestamp with time zone |           |          | now()   | plain    |              |
 col4   | text                     |           |          |         | extended |              |
Partition key: RANGE (col1)
Indexes:
    "original_table_col1_idx1" btree (col1)
Partitions: original_table_p0 FOR VALUES FROM ('0') TO ('10000'),
            original_table_p10000 FOR VALUES FROM ('10000') TO ('20000'),
            original_table_p20000 FOR VALUES FROM ('20000') TO ('30000'),
            original_table_p30000 FOR VALUES FROM ('30000') TO ('40000'),
            original_table_p40000 FOR VALUES FROM ('40000') TO ('50000'),
```

```
            original_table_default DEFAULT
```

```
CALL partition_data_id('public.original_table', p_batch_interval := '1000', p_batch_count := 200, p_source_
table := 'public.old_nonpartitioned_table');
 partition_data_id
--------------------
 100000
(1 row)

VACUUM ANALYZE public.original_table;
VACUUM
```

```
SELECT count(*) FROM old_nonpartitioned_table ;
 count
-------
     0
(1 row)
```

```
SELECT count(*) FROM original_table;
 count
--------
 100000
(1 row)



\d+ public.original_table
                                  Partitioned table "public.original_table"
 Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
 col1   | bigint                   |           | not null |         | plain    |              |
 col2   | text                     |           | not null |         | extended |              |
 col3   | timestamp with time zone |           |          | now()   | plain    |              |
 col4   | text                     |           |          |         | extended |              |
Partition key: RANGE (col1)
Indexes:
    "original_table_col1_idx1" btree (col1)
Partitions: original_table_p0 FOR VALUES FROM ('0') TO ('10000'),
            original_table_p10000 FOR VALUES FROM ('10000') TO ('20000'),
            original_table_p100000 FOR VALUES FROM ('100000') TO ('110000'),
            original_table_p20000 FOR VALUES FROM ('20000') TO ('30000'),
            original_table_p30000 FOR VALUES FROM ('30000') TO ('40000'),
            original_table_p40000 FOR VALUES FROM ('40000') TO ('50000'),
            original_table_p50000 FOR VALUES FROM ('50000') TO ('60000'),
            original_table_p60000 FOR VALUES FROM ('60000') TO ('70000'),
```

```
            original_table_p70000 FOR VALUES FROM ('70000') TO ('80000'),
            original_table_p80000 FOR VALUES FROM ('80000') TO ('90000'),
            original_table_p90000 FOR VALUES FROM ('90000') TO ('100000'),
            original_table_default DEFAULT

SELECT count(*) FROM original_table_p10000;
 count
-------
 10000
(1 row)
```

3.49.4.1.6.5　在线分区

```
CREATE TABLE public.original_table (
    col1 bigint not null PRIMARY KEY GENERATED ALWAYS AS IDENTITY
    , col2 text not null
    , col3 timestamptz DEFAULT now() not null
    , col4 text);

CREATE INDEX CONCURRENTLY ON public.original_table (col3);


INSERT INTO public.original_table (col2, col3, col4) VALUES ('stuff', generate_series(now() - '1 week'::
interval, now(), '5 minutes'::interval), 'stuff');
```

```
CREATE TABLE public.new_partitioned_table (
    col1 bigint not null GENERATED BY DEFAULT AS IDENTITY
    , col2 text not null
    , col3 timestamptz DEFAULT now() not null
    , col4 text) PARTITION BY RANGE (col3);

CREATE INDEX ON public.new_partitioned_table (col3);
```

```
CREATE TABLE public.original_table_template (LIKE public.original_table);

ALTER TABLE public.original_table_template ADD PRIMARY KEY (col1);
```

```
select min(col3), max(col3) from original_table;
              min              |              max
-------------------------------+-------------------------------
 2020-12-02 19:04:08.559646-05 | 2020-12-09 19:04:08.559646-05
(1 row)
```

```
SELECT partman.create_parent('public.new_partitioned_table', 'col3', 'native', 'daily', p_template_table:=
'public.original_table_template', p_premake := 1, p_start_partition := (CURRENT_TIMESTAMP+'2 days'::
interval)::text);
```

default 分区是 partman 自动创建的。

```
DROP TABLE public.new_partitioned_table_default;
```

```
SELECT CURRENT_TIMESTAMP;
       current_timestamp
-------------------------------
 2020-12-09 19:05:15.358796-05

\d+ new_partitioned_table;
                                          Partitioned table "public.new_partitioned_table"
 Column |           Type           | Collation | Nullable |             Default              | Storage  |
Stats target | Description
--------+--------------------------+-----------+----------+----------------------------------+----------+---
-----------+-------------
 col1   | bigint                   |           | not null | generated by default as identity | plain    |
          |
 col2   | text                     |           | not null |                                  | extended |
          |
 col3   | timestamp with time zone |           | not null | now()                            | plain    |
          |
 col4   | text                     |           |          |                                  | extended |
          |
Partition key: RANGE (col3)
Indexes:
    "new_partitioned_table_col3_idx" btree (col3)
Partitions: new_partitioned_table_p2020_12_11 FOR VALUES FROM ('2020-12-11 00:00:00-05') TO ('2020-12-12 00:
00:00-05')
```

设置 part_config 名字。

```
UPDATE partman.part_config SET parent_table = 'public.original_table', premake = 4 WHERE parent_table =
'public.new_partitioned_table';
UPDATE 1
```

迁移自增长将源表设置为 default

```
BEGIN;

LOCK TABLE public.original_table IN ACCESS EXCLUSIVE MODE;
LOCK TABLE public.new_partitioned_table IN ACCESS EXCLUSIVE MODE;
```

```
SELECT max(col1) FROM public.original_table;

ALTER TABLE public.original_table RENAME TO original_table_default;

-- IF using an IDENTITY column
ALTER TABLE public.original_table_default ALTER col1 DROP IDENTITY;

ALTER TABLE public.new_partitioned_table_p2022_08_04 RENAME TO original_table_p2022_08_04;
ALTER TABLE public.new_partitioned_table RENAME TO original_table;

-- IF using an IDENTITY column
ALTER SEQUENCE public.new_partitioned_table_col1_seq RENAME TO original_table_col1_seq;

-- IF using an IDENTITY column
ALTER TABLE public.original_table ALTER col1 RESTART WITH 100;

ALTER TABLE public.original_table ATTACH PARTITION public.original_table_default DEFAULT;
```

```
COMMIT; or ROLLBACK;
```

迁移

```
CALL partman.partition_data_time('public.original_table', p_batch_count := 200);

NOTICE:  Batch: 1, Rows moved: 60
NOTICE:  Batch: 2, Rows moved: 288
NOTICE:  Batch: 3, Rows moved: 288
NOTICE:  Batch: 4, Rows moved: 288
NOTICE:  Batch: 5, Rows moved: 288
NOTICE:  Batch: 6, Rows moved: 288
NOTICE:  Batch: 7, Rows moved: 288
NOTICE:  Batch: 8, Rows moved: 229
NOTICE:  Total rows moved: 2017
NOTICE:  Ensure to VACUUM ANALYZE the parent (and source table if used) after partitioning data
CALL
Time: 8432.725 ms (00:08.433)

VACUUM ANALYZE original_table;
VACUUM
Time: 60.690 ms
```

设置自增长

```
ALTER TABLE public.original_table ALTER col1 SET GENERATED ALWAYS;
```

```
\d+ original_table;
                                                Partitioned table "public.original_table"
 Column |           Type           | Collation | Nullable |             Default              | Storage  |
Stats target | Description
--------+--------------------------+-----------+----------+----------------------------------+----------+---
-----------+-------------
 col1   | bigint                   |           | not null | generated by default as identity | plain    |
          |
 col2   | text                     |           | not null |                                  | extended |
          |
 col3   | timestamp with time zone |           | not null | now()                            | plain    |
          |
 col4   | text                     |           |          |                                  | extended |
          |
Partition key: RANGE (col3)
Indexes:
    "new_partitioned_table_col3_idx" btree (col3)
Partitions: original_table_p2020_12_02 FOR VALUES FROM ('2020-12-02 00:00:00-05') TO ('2020-12-03 00:00:00-
05'),
            original_table_p2020_12_03 FOR VALUES FROM ('2020-12-03 00:00:00-05') TO ('2020-12-04 00:00:00-
05'),
            original_table_p2020_12_04 FOR VALUES FROM ('2020-12-04 00:00:00-05') TO ('2020-12-05 00:00:00-
05'),
            original_table_p2020_12_05 FOR VALUES FROM ('2020-12-05 00:00:00-05') TO ('2020-12-06 00:00:00-
05'),
            original_table_p2020_12_06 FOR VALUES FROM ('2020-12-06 00:00:00-05') TO ('2020-12-07 00:00:00-
05'),
            original_table_p2020_12_07 FOR VALUES FROM ('2020-12-07 00:00:00-05') TO ('2020-12-08 00:00:00-
05'),
            original_table_p2020_12_08 FOR VALUES FROM ('2020-12-08 00:00:00-05') TO ('2020-12-09 00:00:00-
05'),
            original_table_p2020_12_09 FOR VALUES FROM ('2020-12-09 00:00:00-05') TO ('2020-12-10 00:00:00-
05'),
            original_table_p2020_12_11 FOR VALUES FROM ('2020-12-11 00:00:00-05') TO ('2020-12-12 00:00:00-
05'),
```

检查

```
SELECT partman.run_maintenance('public.original_table');

\d+ original_table;
                                                Partitioned table "public.original_table"
 Column |           Type           | Collation | Nullable |             Default              | Storage  |
```

```
Stats target | Description
--------+-------------------------+-----------+----------+---------------------------------+----------+---
-----------+-------------
 col1   | bigint                  |           | not null | generated by default as identity | plain    |
          |
 col2   | text                    |           | not null |                                 | extended |
          |
 col3   | timestamp with time zone |           | not null | now()                           | plain    |
          |
 col4   | text                    |           |          |                                 | extended |
          |
Partition key: RANGE (col3)
Indexes:
    "new_partitioned_table_col3_idx" btree (col3)
Partitions: original_table_p2020_12_02 FOR VALUES FROM ('2020-12-02 00:00:00-05') TO ('2020-12-03 00:00:00-
05'),
            original_table_p2020_12_03 FOR VALUES FROM ('2020-12-03 00:00:00-05') TO ('2020-12-04 00:00:00-
05'),
            original_table_p2020_12_04 FOR VALUES FROM ('2020-12-04 00:00:00-05') TO ('2020-12-05 00:00:00-
05'),
            original_table_p2020_12_05 FOR VALUES FROM ('2020-12-05 00:00:00-05') TO ('2020-12-06 00:00:00-
05'),
            original_table_p2020_12_06 FOR VALUES FROM ('2020-12-06 00:00:00-05') TO ('2020-12-07 00:00:00-
05'),
            original_table_p2020_12_07 FOR VALUES FROM ('2020-12-07 00:00:00-05') TO ('2020-12-08 00:00:00-
05'),
            original_table_p2020_12_08 FOR VALUES FROM ('2020-12-08 00:00:00-05') TO ('2020-12-09 00:00:00-
05'),
            original_table_p2020_12_09 FOR VALUES FROM ('2020-12-09 00:00:00-05') TO ('2020-12-10 00:00:00-
05'),
            original_table_p2020_12_11 FOR VALUES FROM ('2020-12-11 00:00:00-05') TO ('2020-12-12 00:00:00-
05'),
            original_table_p2020_12_12 FOR VALUES FROM ('2020-12-12 00:00:00-05') TO ('2020-12-13 00:00:00-
05'),
            original_table_p2020_12_13 FOR VALUES FROM ('2020-12-13 00:00:00-05') TO ('2020-12-14 00:00:00-
05'),
            original_table_default DEFAULT
```

检查 default 中是否存在数据

```
SELECT * FROM partman.check_default(p_exact_count := true);
```

填补分区表之间的间隔

```
SELECT * FROM partman.partition_gap_fill('public.original_table');
 partition_gap_fill
--------------------
                  1
(1 row)

\d+ original_table;
                                                Partitioned table "public.original_table"
 Column |           Type           | Collation | Nullable |             Default              | Storage  |
Stats target | Description
--------+--------------------------+-----------+----------+----------------------------------+----------+---
-----------+-------------
 col1   | bigint                   |           | not null | generated by default as identity | plain    |
          |
 col2   | text                     |           | not null |                                  | extended |
          |
 col3   | timestamp with time zone |           | not null | now()                            | plain    |
          |
 col4   | text                     |           |          |                                  | extended |
          |
Partition key: RANGE (col3)
Indexes:
    "new_partitioned_table_col3_idx" btree (col3)
Partitions: original_table_p2020_12_02 FOR VALUES FROM ('2020-12-02 00:00:00-05') TO ('2020-12-03 00:00:00-
05'),
            original_table_p2020_12_03 FOR VALUES FROM ('2020-12-03 00:00:00-05') TO ('2020-12-04 00:00:00-
05'),
            original_table_p2020_12_04 FOR VALUES FROM ('2020-12-04 00:00:00-05') TO ('2020-12-05 00:00:00-
05'),
            original_table_p2020_12_05 FOR VALUES FROM ('2020-12-05 00:00:00-05') TO ('2020-12-06 00:00:00-
05'),
            original_table_p2020_12_06 FOR VALUES FROM ('2020-12-06 00:00:00-05') TO ('2020-12-07 00:00:00-
05'),
            original_table_p2020_12_07 FOR VALUES FROM ('2020-12-07 00:00:00-05') TO ('2020-12-08 00:00:00-
05'),
            original_table_p2020_12_08 FOR VALUES FROM ('2020-12-08 00:00:00-05') TO ('2020-12-09 00:00:00-
05'),
            original_table_p2020_12_09 FOR VALUES FROM ('2020-12-09 00:00:00-05') TO ('2020-12-10 00:00:00-
05'),
            original_table_p2020_12_10 FOR VALUES FROM ('2020-12-10 00:00:00-05') TO ('2020-12-11 00:00:00-
05'),
            original_table_p2020_12_11 FOR VALUES FROM ('2020-12-11 00:00:00-05') TO ('2020-12-12 00:00:00-
05'),
            original_table_p2020_12_12 FOR VALUES FROM ('2020-12-12 00:00:00-05') TO ('2020-12-13 00:00:00-
05'),
```

```
            original_table_p2020_12_13 FOR VALUES FROM ('2020-12-13 00:00:00-05') TO ('2020-12-14 00:00:00-
05'),
            original_table_default DEFAULT
```

正常使用

```
INSERT INTO original_table (col2, col3, col4) VALUES ('newstuff', now(), 'newstuff');
INSERT INTO original_table (col2, col3, col4) VALUES ('newstuff', now(), 'newstuff');
SELECT * FROM original_table ORDER BY col1 DESC limit 5;
```

3.49.4.1.6.6　撤销本地分区

```
CREATE TABLE public.new_regular_table (
    col1 bigint not null GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
    , col2 text not null
    , col3 timestamptz DEFAULT now() not null
    , col4 text);

CREATE INDEX ON public.new_regular_table (col3);
```

迁移数据

```
CALL partman.undo_partition('public.original_table', p_batch_interval := '1 hour'::text, p_batch_count :=
500, p_target_table := 'public.new_regular_table', p_keep_table := false);

NOTICE:  Moved 13 row(s) to the target table. Removed 1 partitions.
NOTICE:  Batch: 1, Partitions undone this batch: 1, Rows undone this batch: 13
NOTICE:  Moved 13 row(s) to the target table. Removed 0 partitions.
NOTICE:  Batch: 2, Partitions undone this batch: 0, Rows undone this batch: 13
NOTICE:  Moved 13 row(s) to the target table. Removed 0 partitions.
NOTICE:  Batch: 3, Partitions undone this batch: 0, Rows undone this batch: 13
[...]
NOTICE:  Batch: 160, Partitions undone this batch: 0, Rows undone this batch: 13
NOTICE:  Moved 5 row(s) to the target table. Removed 1 partitions.
NOTICE:  Batch: 161, Partitions undone this batch: 1, Rows undone this batch: 5
NOTICE:  Moved 0 row(s) to the target table. Removed 4 partitions.
NOTICE:  Total partitions undone: 13, Total rows moved: 2017
NOTICE:  Ensure to VACUUM ANALYZE the old parent & target table after undo has finished
CALL
Time: 163465.195 ms (02:43.465)

VACUUM ANALYZE original_table;
VACUUM
Time: 20.706 ms
```

```
VACUUM ANALYZE new_regular_table;
VACUUM
Time: 20.375 ms
```

设置新表自增长

```
SELECT max(col1) FROM public.original_table;

ALTER TABLE original_table RENAME TO old_partitioned_table;
ALTER SEQUENCE original_table_col1_seq RENAME TO old_partitioned_table_col1_seq;

ALTER TABLE new_regular_table RENAME TO original_table;
ALTER SEQUENCE new_regular_table_col1_seq RENAME TO original_table_col1_seq;
ALTER TABLE public.original_table ALTER col1 RESTART WITH 100;
ALTER TABLE public.original_table ALTER col1 SET GENERATED ALWAYS;
```

可以正常使用

```
INSERT INTO original_table (col2, col3, col4) VALUES ('newstuff', now(), 'newstuff');
INSERT INTO original_table (col2, col3, col4) VALUES ('newstuff', now(), 'newstuff');
SELECT * FROM original_table ORDER BY col1 DESC limit 2;
```

3.49.4.1.6.7　设置触发分区

3.49.4.1.6.8　每天一个分区

```
\d partman.time_taptest_table
             Table "partman.time_taptest_table"
Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
col1   | integer                  |           | not null |
col2   | text                     |           |          |
col3   | timestamp with time zone |           | not null | now()
Indexes:
   "time_taptest_table_pkey" PRIMARY KEY, btree (col1)

SELECT partman.create_parent('partman.time_taptest_table', 'col3', p_type := 'native', p_interval := 'daily
');
create_parent
---------------
t
(1 row)

\d+ partman.time_taptest_table
                                 Table "partman.time_taptest_table"
```

```
Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
col1   | integer                  |           | not null |         | plain    |              |
col2   | text                     |           |          |         | extended |              |
col3   | timestamp with time zone |           | not null | now()   | plain    |              |
Indexes:
   "time_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
   time_taptest_table_part_trig BEFORE INSERT ON partman.time_taptest_table FOR EACH ROW EXECUTE PROCEDURE
partman.time_taptest_table_part_trig_func()
Child tables: partman.time_taptest_table_p2017_03_23,
             partman.time_taptest_table_p2017_03_24,
             partman.time_taptest_table_p2017_03_25,
             partman.time_taptest_table_p2017_03_26,
             partman.time_taptest_table_p2017_03_27,
             partman.time_taptest_table_p2017_03_28,
             partman.time_taptest_table_p2017_03_29,
             partman.time_taptest_table_p2017_03_30,
             partman.time_taptest_table_p2017_03_31
```

```
\sf partman.time_taptest_table_part_trig_func

在 contrib/kdb_partman/sql 目录下，找到 sql 文件，添加自定义函数。
CREATE TABLE partman.time_taptest_table(
  col1 int not null
  , col2 text
  , col3 timestamptz NOT NULL DEFAULT now())
  PARTITION BY RANGE (col3);
CREATE INDEX ON partman.time_taptest_table (col3);
SELECT partman.create_parent('partman.time_taptest_table', 'col3', p_type := 'native', p_interval := 'daily
');
CREATE OR REPLACE FUNCTION partman.time_taptest_table_part_trig_func()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
           DECLARE
           v_count                 int;
           v_partition_name        text;
           v_partition_timestamp   timestamptz;
       BEGIN
       IF TG_OP = 'INSERT' THEN
           v_partition_timestamp := date_trunc('day', NEW.col3);
       IF NEW.col3 >= '2022-07-30 00:00:00-04' AND NEW.col3 < '2022-07-31 00:00:00-04' THEN
                         INSERT INTO partman.time_taptest_table_p2022_07_30 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-07-31 00:00:00-04' AND NEW.col3 < '2022-08-01 00:00:00-04' THEN
```

```
          INSERT INTO partman.time_taptest_table_p2022_07_31 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-01 00:00:00-04' AND NEW.col3 < '2022-08-02 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_01 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-02 00:00:00-04' AND NEW.col3 < '2022-08-03 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_02 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-03 00:00:00-04' AND NEW.col3 < '2022-08-04 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_03 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-04 00:00:00-04' AND NEW.col3 < '2022-08-05 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_04 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-05 00:00:00-04' AND NEW.col3 < '2022-08-06 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_05 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-06 00:00:00-04' AND NEW.col3 < '2022-08-07 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_06 VALUES (NEW.*);
       ELSIF NEW.col3 >= '2022-08-07 00:00:00-04' AND NEW.col3 < '2022-08-08 00:00:00-04' THEN
          INSERT INTO partman.time_taptest_table_p2022_08_07 VALUES (NEW.*);
           ELSE
               v_partition_name := partman.check_name_length('time_taptest_table', to_char(v_partition_
timestamp, 'YYYY_MM_DD'), TRUE);
               SELECT count(*) INTO v_count FROM sys_catalog.sys_tables WHERE schemaname = 'partman'::name
AND tablename = v_partition_name::name;
               IF v_count > 0 THEN
                   EXECUTE format('INSERT INTO %I.%I VALUES($1.*) ', 'partman', v_partition_name) USING NEW;
               ELSE
                   RETURN NEW;
               END IF;
           END IF;
       END IF;
       RETURN NULL;
       END $function$
```

3.49.4.1.6.9　10 个数一个分区

```
\d partman.id_taptest_table
              Table "partman.id_taptest_table"
Column |           Type           |           Modifiers
--------+--------------------------+-------------------------------
col1   | integer                  | not null
col2   | text                     | not null default 'stuff'::text
col3   | timestamp with time zone | default now()
Indexes:
   "id_taptest_table_pkey" PRIMARY KEY, btree (col1)


SELECT partman.create_parent('partman.id_taptest_table', 'col1', 'native', '10');
```

```
create_parent
---------------
t
(1 row)



\d+ partman.id_taptest_table
                                  Table "partman.id_taptest_table"
Column |           Type           |           Modifiers            | Storage  | Stats target | Description
--------+--------------------------+--------------------------------+----------+--------------+-------------
col1   | integer                  | not null                       | plain    |              |
col2   | text                     | not null default 'stuff'::text | extended |              |
col3   | timestamp with time zone | default now()                  | plain    |              |
Indexes:
   "id_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
   id_taptest_table_part_trig BEFORE INSERT ON partman.id_taptest_table FOR EACH ROW EXECUTE PROCEDURE
partman.id_taptest_table_part_trig_func()
Child tables: partman.id_taptest_table_p0,
             partman.id_taptest_table_p10,
             partman.id_taptest_table_p20,
             partman.id_taptest_table_p30,
             partman.id_taptest_table_p40
```

```
\sf partman.id_taptest_table_part_trig_func
CREATE OR REPLACE FUNCTION partman.id_taptest_table_part_trig_func()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
   DECLARE
       v_count                     int;
       v_current_partition_id      bigint;
       v_current_partition_name    text;
       v_id_position               int;
       v_last_partition            text := 'id_taptest_table_p40';
       v_next_partition_id         bigint;
       v_next_partition_name       text;
       v_partition_created         boolean;
   BEGIN
   IF TG_OP = 'INSERT' THEN
       IF NEW.col1 >= 0 AND NEW.col1 < 10 THEN
           INSERT INTO partman.id_taptest_table_p0 VALUES (NEW.*) ;
       ELSIF NEW.col1 >= 10 AND NEW.col1 < 20 THEN
           INSERT INTO partman.id_taptest_table_p10 VALUES (NEW.*) ;
       ELSIF NEW.col1 >= 20 AND NEW.col1 < 30 THEN
```

```
            INSERT INTO partman.id_taptest_table_p20 VALUES (NEW.*) ;
        ELSIF NEW.col1 >= 30 AND NEW.col1 < 40 THEN
            INSERT INTO partman.id_taptest_table_p30 VALUES (NEW.*) ;
        ELSIF NEW.col1 >= 40 AND NEW.col1 < 50 THEN
            INSERT INTO partman.id_taptest_table_p40 VALUES (NEW.*) ;
        ELSE
            v_current_partition_id := NEW.col1 - (NEW.col1 % 10);
            v_current_partition_name := partman.check_name_length('id_taptest_table', v_current_partition_id:
:text, TRUE);
            SELECT count(*) INTO v_count FROM sys_catalog.sys_tables WHERE schemaname = 'partman'::name AND
tablename = v_current_partition_name::n
ame;
            IF v_count > 0 THEN
                EXECUTE format('INSERT INTO %I.%I VALUES($1.*) ', 'partman', v_current_partition_name) USING
NEW;
            ELSE
                RETURN NEW;
            END IF;
        END IF;
    END IF;
    RETURN NULL;
    END $function$
```

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(1,'insert1');
INSERT 0 0
Time: 4.876 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   1 | insert1 | 2017-03-27 14:23:02.769999-04
(1 row)
```

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(2,'insert2');
INSERT 0 0
Time: 1.583 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   2 | insert2 | 2017-03-27 14:23:02.769999-04
(1 row)
```

3.49.4.1.6.10　10 个数一个分区创建的空表使用 upsert 丢弃行冲突

```
Uses same example table as above

 SELECT partman.create_parent('partman.id_taptest_table', 'col1', 'native', '10', p_upsert := 'ON CONFLICT
(col1) DO NOTHING');
 create_parent
---------------
 t
(1 row)

 \d+ partman.id_taptest_table
                                        Table "partman.id_taptest_table"
 Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
 col1   | integer                  |           | not null |               | plain    |              |
 col2   | text                     |           | not null | 'stuff'::text | extended |              |
 col3   | timestamp with time zone |           |          | now()         | plain    |              |
Indexes:
    "id_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
    id_taptest_table_part_trig BEFORE INSERT ON partman.id_taptest_table FOR EACH ROW EXECUTE PROCEDURE
partman.id_taptest_table_part_trig_func()
Child tables: partman.id_taptest_table_p0,
              partman.id_taptest_table_p10,
              partman.id_taptest_table_p20,
              partman.id_taptest_table_p30,
              partman.id_taptest_table_p40
```

创建插入函数，如果有冲突不会插入冲突行

```
\sf partman.id_taptest_table_part_trig_func
CREATE OR REPLACE FUNCTION partman.id_taptest_table_part_trig_func()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
   DECLARE
       v_count                     int;
       v_current_partition_id      bigint;
       v_current_partition_name    text;
       v_id_position               int;
       v_last_partition            text := 'id_taptest_table_p40';
       v_next_partition_id         bigint;
       v_next_partition_name       text;
```

```
        v_partition_created         boolean;
    BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.col1 >= 0 AND NEW.col1 < 10 THEN
            INSERT INTO partman.id_taptest_table_p0 VALUES (NEW.*) ON CONFLICT (col1) DO NOTHING;
        ELSIF NEW.col1 >= 10 AND NEW.col1 < 20 THEN
            INSERT INTO partman.id_taptest_table_p10 VALUES (NEW.*) ON CONFLICT (col1) DO NOTHING;
        ELSIF NEW.col1 >= 20 AND NEW.col1 < 30 THEN
            INSERT INTO partman.id_taptest_table_p20 VALUES (NEW.*) ON CONFLICT (col1) DO NOTHING;
        ELSIF NEW.col1 >= 30 AND NEW.col1 < 40 THEN
            INSERT INTO partman.id_taptest_table_p30 VALUES (NEW.*) ON CONFLICT (col1) DO NOTHING;
        ELSIF NEW.col1 >= 40 AND NEW.col1 < 50 THEN
            INSERT INTO partman.id_taptest_table_p40 VALUES (NEW.*) ON CONFLICT (col1) DO NOTHING;
        ELSE
            v_current_partition_id := NEW.col1 - (NEW.col1 % 10);
            v_current_partition_name := partman.check_name_length('id_taptest_table', v_current_partition_id:
:text, TRUE);
            SELECT count(*) INTO v_count FROM sys_catalog.sys_tables WHERE schemaname = 'partman'::name AND
tablename = v_current_partition_name::n
ame;
            IF v_count > 0 THEN
                EXECUTE format('INSERT INTO %I.%I VALUES($1.*) ON CONFLICT (col1) DO NOTHING', 'partman', v_
current_partition_name) USING NEW;
            ELSE
                RETURN NEW;
            END IF;
        END IF;
    END IF;
    RETURN NULL;
    END $function$
```

插入一个行

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(1,'insert1');
INSERT 0 0
Time: 4.876 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   1 | insert1 | 2017-03-27 14:23:02.769999-04
(1 row)
```

插入冲突行，结果为保留原有行

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(1,'insert2');
INSERT 0 0
Time: 1.583 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   1 | insert1 | 2017-03-27 14:23:02.769999-04
(1 row)
```

3.49.4.1.6.11　用 upsert 更新行冲突

```
Uses same example table as above

 SELECT partman.create_parent('partman.id_taptest_table', 'col1', 'partman', '10', p_upsert := 'ON CONFLICT
(col1) DO UPDATE SET col2=EXCLUDED.col2, col3=EXCLUDED.col3');
 create_parent
---------------
 t
(1 row)

 \d+ partman.id_taptest_table
                                        Table "partman.id_taptest_table"
 Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
 col1   | integer                  |           | not null |               | plain    |              |
 col2   | text                     |           | not null | 'stuff'::text | extended |              |
 col3   | timestamp with time zone |           |          | now()         | plain    |              |
Indexes:
    "id_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
    id_taptest_table_part_trig BEFORE INSERT ON partman.id_taptest_table FOR EACH ROW EXECUTE PROCEDURE
partman.id_taptest_table_part_trig_func()
Child tables: partman.id_taptest_table_p0,
              partman.id_taptest_table_p10,
              partman.id_taptest_table_p20,
              partman.id_taptest_table_p30,
              partman.id_taptest_table_p40
```

创建插入函数，如果有冲突行则更新原有行

```
\sf partman.id_taptest_table_part_trig_func
CREATE OR REPLACE FUNCTION partman.id_taptest_table_part_trig_func()
```

```
RETURNS trigger
LANGUAGE plpgsql
AS $function$
   DECLARE
       v_count                     int;
       v_current_partition_id      bigint;
       v_current_partition_name    text;
       v_id_position               int;
       v_last_partition            text := 'id_taptest_table_p40';
       v_next_partition_id         bigint;
       v_next_partition_name       text;
       v_partition_created         boolean;
   BEGIN
   IF TG_OP = 'INSERT' THEN
       IF NEW.col1 >= 0 AND NEW.col1 < 10 THEN
           INSERT INTO partman.id_taptest_table_p0 VALUES (NEW.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3;
       ELSIF NEW.col1 >= 10 AND NEW.col1 < 20 THEN
           INSERT INTO partman.id_taptest_table_p10 VALUES (NEW.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3;
       ELSIF NEW.col1 >= 20 AND NEW.col1 < 30 THEN
           INSERT INTO partman.id_taptest_table_p20 VALUES (NEW.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3;
       ELSIF NEW.col1 >= 30 AND NEW.col1 < 40 THEN
           INSERT INTO partman.id_taptest_table_p30 VALUES (NEW.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3;
       ELSIF NEW.col1 >= 40 AND NEW.col1 < 50 THEN
           INSERT INTO partman.id_taptest_table_p40 VALUES (NEW.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3;
       ELSE
           v_current_partition_id := NEW.col1 - (NEW.col1 % 10);
           v_current_partition_name := partman.check_name_length('id_taptest_table', v_current_partition_id:
:text, TRUE);
           SELECT count(*) INTO v_count FROM sys_catalog.sys_tables WHERE schemaname = 'partman'::name AND
tablename = v_current_partition_name::n
ame;
           IF v_count > 0 THEN
               EXECUTE format('INSERT INTO %I.%I VALUES($1.*) ON CONFLICT (col1) DO UPDATE SET
col2=EXCLUDED.col2, col3=EXCLUDED.col3', 'partman
', v_current_partition_name) USING NEW;
           ELSE
               RETURN NEW;
           END IF;
       END IF;
   END IF;
```

```
   RETURN NULL;
   END $function$
```

插入行

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(1,'insert1');
INSERT 0 0
Time: 6.012 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   1 | insert1 | 2017-03-27 14:32:26.59552-04
(1 row)
```

插入冲突行，结果为更新原有行

```
INSERT INTO partman.id_taptest_table(col1,col2) VALUES(1,'insert2');
INSERT 0 0
Time: 4.235 ms
SELECT * FROM partman.id_taptest_table;
col1 |  col2   |             col3
------+---------+-------------------------------
   1 | insert2 | 2017-03-27 14:33:00.949928-04
(1 row)
```

3.49.4.1.6.12　创建多级子分区

3.49.4.1.6.13　时间多级子分区

kdb_partman 仅在某些版本支持对本机分区集进行自分区，目前最新的版本 kdb_partman4.4.0 不支持此功能。

以下过程是先创建表，后插入数据

```
\d partman.time_taptest_table
             Table "partman.time_taptest_table"
Column |           Type           | Collation | Nullable | Default
--------+--------------------------+-----------+----------+---------
col1   | integer                  |           | not null |
col2   | text                     |           |          |
col3   | timestamp with time zone |           | not null | now()
Indexes:
   "time_taptest_table_pkey" PRIMARY KEY, btree (col1)
```

```
SELECT partman.create_parent('partman.time_taptest_table', 'col3', 'partman', 'yearly', p_premake := 2);
create_parent
```

```
---------------
t
(1 row)
```

```
SELECT partman.create_sub_parent('partman.time_taptest_table', 'col3', 'partman', 'monthly', p_premake :=
2);
create_sub_parent
-------------------
t
(1 row)

SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
            tablename
-----------------------------------
time_taptest_table
time_taptest_table_p2015
time_taptest_table_p2015_p2015_01
time_taptest_table_p2016
time_taptest_table_p2016_p2016_01
time_taptest_table_p2017
time_taptest_table_p2017_p2017_01
time_taptest_table_p2017_p2017_02
time_taptest_table_p2017_p2017_03
time_taptest_table_p2017_p2017_04
time_taptest_table_p2017_p2017_05
time_taptest_table_p2018
time_taptest_table_p2018_p2018_01
time_taptest_table_p2019
time_taptest_table_p2019_p2019_01
(15 rows)
```

```
SELECT partman.create_sub_parent('partman.time_taptest_table_p2015', 'col3', 'partman', 'daily', p_premake :
= 2);
SELECT partman.create_sub_parent('partman.time_taptest_table_p2016', 'col3', 'partman', 'daily', p_premake :
= 2);
SELECT partman.create_sub_parent('partman.time_taptest_table_p2017', 'col3', 'partman', 'daily', p_premake :
= 2);
SELECT partman.create_sub_parent('partman.time_taptest_table_p2018', 'col3', 'partman', 'daily', p_premake :
= 2);
SELECT partman.create_sub_parent('partman.time_taptest_table_p2019', 'col3', 'partman', 'daily', p_premake :
= 2);

 SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
                    tablename
```

```
----------------------------------------------
 time_taptest_table
 time_taptest_table_p2015
 time_taptest_table_p2015_p2015_01
 time_taptest_table_p2015_p2015_01_p2015_01_01
 time_taptest_table_p2016
 time_taptest_table_p2016_p2016_01
 time_taptest_table_p2016_p2016_01_p2016_01_01
 time_taptest_table_p2017
 time_taptest_table_p2017_p2017_01
 time_taptest_table_p2017_p2017_01_p2017_01_01
 time_taptest_table_p2017_p2017_02
 time_taptest_table_p2017_p2017_02_p2017_02_01
 time_taptest_table_p2017_p2017_03
 time_taptest_table_p2017_p2017_03_p2017_03_25
 time_taptest_table_p2017_p2017_03_p2017_03_26
 time_taptest_table_p2017_p2017_03_p2017_03_27
 time_taptest_table_p2017_p2017_03_p2017_03_28
 time_taptest_table_p2017_p2017_03_p2017_03_29
 time_taptest_table_p2017_p2017_04
 time_taptest_table_p2017_p2017_04_p2017_04_01
 time_taptest_table_p2017_p2017_05
 time_taptest_table_p2017_p2017_05_p2017_05_01
 time_taptest_table_p2018
 time_taptest_table_p2018_p2018_01
 time_taptest_table_p2018_p2018_01_p2018_01_01
 time_taptest_table_p2019
 time_taptest_table_p2019_p2019_01
 time_taptest_table_p2019_p2019_01_p2019_01_01
(28 rows)
```

数值间隔的多级子分区

以下过程是表中已经存在数据

```
\d partman.id_taptest_table
                 Table "partman.id_taptest_table"
Column |           Type           | Collation | Nullable |    Default
--------+--------------------------+-----------+----------+---------------
col1   | integer                  |           | not null |
col2   | text                     |           | not null | 'stuff'::text
col3   | timestamp with time zone |           |          | now()
Indexes:
   "id_taptest_table_pkey" PRIMARY KEY, btree (col1)
```

```
SELECT count(*) FROM partman.id_taptest_table ;
count
--------
100000
(1 row)

SELECT min(col1), max(col1) FROM partman.id_taptest_table ;
min |  max
-----+--------
  1 | 100000
(1 row)
```

```
SELECT partman.create_parent('partman.id_taptest_table', 'col1', 'partman', '10000', p_jobmon := false, p_
premake := 2);
---------------
t
(1 row)

SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
       tablename
--------------------------
id_taptest_table
id_taptest_table_p100000
id_taptest_table_p110000
id_taptest_table_p120000
id_taptest_table_p80000
id_taptest_table_p90000
(6 rows)
```

```
$ python partition_data.py -c host=localhost -p partman.id_taptest_table -t id -i 100
Attempting to turn off autovacuum for partition set...
... Success!
Rows moved: 100
Rows moved: 100
...
Rows moved: 99
...
Rows moved: 100
Rows moved: 1
Total rows moved: 100000
Running vacuum analyze on parent table...
Attempting to reset autovacuum for old parent table and all child tables...
    ... Success!
```

```
SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
       tablename
--------------------------
id_taptest_table
id_taptest_table_p0
id_taptest_table_p10000
id_taptest_table_p100000
id_taptest_table_p110000
id_taptest_table_p120000
id_taptest_table_p20000
id_taptest_table_p30000
id_taptest_table_p40000
id_taptest_table_p50000
id_taptest_table_p60000
id_taptest_table_p70000
id_taptest_table_p80000
id_taptest_table_p90000
(14 rows)
```

```
SELECT partman.create_sub_parent('partman.id_taptest_table', 'col1', 'partman', '1000', p_jobmon := false,
p_premake := 2);
create_sub_parent
-------------------
t
(1 row)
```

```
SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
           tablename
----------------------------------
id_taptest_table
id_taptest_table_p0
id_taptest_table_p0_p0
id_taptest_table_p10000
id_taptest_table_p100000
id_taptest_table_p100000_p100000
id_taptest_table_p100000_p101000
id_taptest_table_p100000_p102000
id_taptest_table_p10000_p10000
id_taptest_table_p110000
id_taptest_table_p110000_p110000
id_taptest_table_p120000
id_taptest_table_p120000_p120000
id_taptest_table_p20000
id_taptest_table_p20000_p20000
```

```
id_taptest_table_p30000
id_taptest_table_p30000_p30000
id_taptest_table_p40000
id_taptest_table_p40000_p40000
id_taptest_table_p50000
id_taptest_table_p50000_p50000
id_taptest_table_p60000
id_taptest_table_p60000_p60000
id_taptest_table_p70000
id_taptest_table_p70000_p70000
id_taptest_table_p80000
id_taptest_table_p80000_p80000
id_taptest_table_p90000
id_taptest_table_p90000_p98000
id_taptest_table_p90000_p99000
(30 rows)
```

此时，数据还在父表中

```
SELECT * FROM partman.check_parent() ORDER BY 1;
            parent_table               | count
---------------------------------------+-------
partman.id_taptest_table_p0      |  9999
partman.id_taptest_table_p10000  | 10000
partman.id_taptest_table_p100000 |     1
partman.id_taptest_table_p20000  | 10000
partman.id_taptest_table_p30000  | 10000
partman.id_taptest_table_p40000  | 10000
partman.id_taptest_table_p50000  | 10000
partman.id_taptest_table_p60000  | 10000
partman.id_taptest_table_p70000  | 10000
partman.id_taptest_table_p80000  | 10000
partman.id_taptest_table_p90000  | 10000
(11 rows)
```

将数据移到子表

```
python partition_data.py -c host=localhost -p partman.id_taptest_table_p0 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p10000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p20000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p30000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p40000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p50000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p60000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p70000 -t id -i 100
```

```
python partition_data.py -c host=localhost -p partman.id_taptest_table_p80000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p90000 -t id -i 100
python partition_data.py -c host=localhost -p partman.id_taptest_table_p100000 -t id -i 100
```

```
SELECT * FROM partman.check_parent() ORDER BY 1;
parent_table | count
--------------+-------
(0 rows)
```

```
SELECT tablename FROM sys_tables WHERE schemaname = 'partman' ORDER BY tablename;
           tablename
----------------------------------
id_taptest_table
id_taptest_table_p0
id_taptest_table_p0_p0
id_taptest_table_p0_p1000
id_taptest_table_p0_p2000
id_taptest_table_p0_p3000
id_taptest_table_p0_p4000
id_taptest_table_p0_p5000
id_taptest_table_p0_p6000
id_taptest_table_p0_p7000
id_taptest_table_p0_p8000
id_taptest_table_p0_p9000
id_taptest_table_p10000
id_taptest_table_p100000
id_taptest_table_p100000_p100000
id_taptest_table_p100000_p101000
id_taptest_table_p100000_p102000
id_taptest_table_p10000_p10000
id_taptest_table_p10000_p11000
id_taptest_table_p10000_p12000
id_taptest_table_p10000_p13000
id_taptest_table_p10000_p14000
id_taptest_table_p10000_p15000
id_taptest_table_p10000_p16000
id_taptest_table_p10000_p17000
id_taptest_table_p10000_p18000
id_taptest_table_p10000_p19000
id_taptest_table_p110000
id_taptest_table_p110000_p110000
id_taptest_table_p120000
id_taptest_table_p120000_p120000
id_taptest_table_p20000
```

```
id_taptest_table_p20000_p20000
id_taptest_table_p20000_p21000
id_taptest_table_p20000_p22000
id_taptest_table_p20000_p23000
id_taptest_table_p20000_p24000
id_taptest_table_p20000_p25000
id_taptest_table_p20000_p26000
id_taptest_table_p20000_p27000
id_taptest_table_p20000_p28000
id_taptest_table_p20000_p29000
id_taptest_table_p30000
id_taptest_table_p30000_p30000
id_taptest_table_p30000_p31000
id_taptest_table_p30000_p32000
id_taptest_table_p30000_p33000
id_taptest_table_p30000_p34000
id_taptest_table_p30000_p35000
id_taptest_table_p30000_p36000
id_taptest_table_p30000_p37000
id_taptest_table_p30000_p38000
id_taptest_table_p30000_p39000
id_taptest_table_p40000
id_taptest_table_p40000_p40000
id_taptest_table_p40000_p41000
id_taptest_table_p40000_p42000
id_taptest_table_p40000_p43000
id_taptest_table_p40000_p44000
id_taptest_table_p40000_p45000
id_taptest_table_p40000_p46000
id_taptest_table_p40000_p47000
id_taptest_table_p40000_p48000
id_taptest_table_p40000_p49000
id_taptest_table_p50000
id_taptest_table_p50000_p50000
id_taptest_table_p50000_p51000
id_taptest_table_p50000_p52000
id_taptest_table_p50000_p53000
id_taptest_table_p50000_p54000
id_taptest_table_p50000_p55000
id_taptest_table_p50000_p56000
id_taptest_table_p50000_p57000
id_taptest_table_p50000_p58000
id_taptest_table_p50000_p59000
id_taptest_table_p60000
id_taptest_table_p60000_p60000
```

```
id_taptest_table_p60000_p61000
id_taptest_table_p60000_p62000
id_taptest_table_p60000_p63000
id_taptest_table_p60000_p64000
id_taptest_table_p60000_p65000
id_taptest_table_p60000_p66000
id_taptest_table_p60000_p67000
id_taptest_table_p60000_p68000
id_taptest_table_p60000_p69000
id_taptest_table_p70000
id_taptest_table_p70000_p70000
id_taptest_table_p70000_p71000
id_taptest_table_p70000_p72000
id_taptest_table_p70000_p73000
id_taptest_table_p70000_p74000
id_taptest_table_p70000_p75000
id_taptest_table_p70000_p76000
id_taptest_table_p70000_p77000
id_taptest_table_p70000_p78000
id_taptest_table_p70000_p79000
id_taptest_table_p80000
id_taptest_table_p80000_p80000
id_taptest_table_p80000_p81000
id_taptest_table_p80000_p82000
id_taptest_table_p80000_p83000
id_taptest_table_p80000_p84000
id_taptest_table_p80000_p85000
id_taptest_table_p80000_p86000
id_taptest_table_p80000_p87000
id_taptest_table_p80000_p88000
id_taptest_table_p80000_p89000
id_taptest_table_p90000
id_taptest_table_p90000_p90000
id_taptest_table_p90000_p91000
id_taptest_table_p90000_p92000
id_taptest_table_p90000_p93000
id_taptest_table_p90000_p94000
id_taptest_table_p90000_p95000
id_taptest_table_p90000_p96000
id_taptest_table_p90000_p97000
id_taptest_table_p90000_p98000
id_taptest_table_p90000_p99000
(119 rows)
```

继续缩小粒度

```
SELECT partman.create_sub_parent('partman.id_taptest_table_p0', 'col1', 'partman', '100', p_jobmon := false,
p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p10000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p20000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p30000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p40000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p50000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p60000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p70000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p80000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p90000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
SELECT partman.create_sub_parent('partman.id_taptest_table_p100000', 'col1', 'partman', '100', p_jobmon :=
false, p_premake := 2);
```

```
SELECT tablename FROM sys_tables WHERE schemaname = 'partman' order by tablename;
               tablename
-----------------------------------------
id_taptest_table
id_taptest_table_p0
id_taptest_table_p0_p0
id_taptest_table_p0_p0_p0
id_taptest_table_p0_p1000
id_taptest_table_p0_p1000_p1000
id_taptest_table_p0_p2000
id_taptest_table_p0_p2000_p2000
...
id_taptest_table_p10000
id_taptest_table_p100000
id_taptest_table_p100000_p100000
id_taptest_table_p100000_p100000_p100000
id_taptest_table_p100000_p100000_p100100
id_taptest_table_p100000_p100000_p100200
id_taptest_table_p100000_p101000
id_taptest_table_p100000_p101000_p101000
id_taptest_table_p100000_p102000
id_taptest_table_p100000_p102000_p102000
```

```
id_taptest_table_p10000_p10000
id_taptest_table_p10000_p10000_p10000
id_taptest_table_p10000_p11000
id_taptest_table_p10000_p11000_p11000
...
id_taptest_table_p90000_p98000
id_taptest_table_p90000_p98000_p98000
id_taptest_table_p90000_p99000
id_taptest_table_p90000_p99000_p99800
id_taptest_table_p90000_p99000_p99900
(225 rows)
```

```
SELECT 'python partition_data.py -c host=localhost -p '||parent_table||' -t id -i 100' FROM partman.part_
config ORDER BY parent_table;

                                                ?column?
-----------------------------------------------------------------------------------------------------------
 python partition_data.py -c host=localhost -p partman.id_taptest_table -t id -i 100
 python partition_data.py -c host=localhost -p partman.id_taptest_table_p0 -t id -i 100
 python partition_data.py -c host=localhost -p partman.id_taptest_table_p0_p0 -t id -i 100
 python partition_data.py -c host=localhost -p partman.id_taptest_table_p0_p1000 -t id -i 100
...
```

```
SELECT tablename FROM sys_tables WHERE schemaname = 'partman' order by tablename;
               tablename
-----------------------------------------
id_taptest_table
id_taptest_table_p0
id_taptest_table_p0_p0
id_taptest_table_p0_p0_p0
id_taptest_table_p0_p0_p100
id_taptest_table_p0_p0_p200
id_taptest_table_p0_p0_p300
id_taptest_table_p0_p0_p400
id_taptest_table_p0_p0_p500
id_taptest_table_p0_p0_p600
id_taptest_table_p0_p0_p700
id_taptest_table_p0_p0_p800
id_taptest_table_p0_p0_p900
id_taptest_table_p0_p1000
id_taptest_table_p0_p1000_p1000
id_taptest_table_p0_p1000_p1100
id_taptest_table_p0_p1000_p1200
id_taptest_table_p0_p1000_p1300
```

```
id_taptest_table_p0_p1000_p1400
id_taptest_table_p0_p1000_p1500
id_taptest_table_p0_p1000_p1600
id_taptest_table_p0_p1000_p1700
id_taptest_table_p0_p1000_p1800
id_taptest_table_p0_p1000_p1900
id_taptest_table_p0_p2000
id_taptest_table_p0_p2000_p2000
id_taptest_table_p0_p2000_p2100
...
id_taptest_table_p90000_p98000_p98800
id_taptest_table_p90000_p98000_p98900
id_taptest_table_p90000_p99000
id_taptest_table_p90000_p99000_p99000
id_taptest_table_p90000_p99000_p99100
id_taptest_table_p90000_p99000_p99200
id_taptest_table_p90000_p99000_p99300
id_taptest_table_p90000_p99000_p99400
id_taptest_table_p90000_p99000_p99500
id_taptest_table_p90000_p99000_p99600
id_taptest_table_p90000_p99000_p99700
id_taptest_table_p90000_p99000_p99800
id_taptest_table_p90000_p99000_p99900
(1124 rows)
```

3.49.4.1.6.14　从触发分区迁移到本地

`test/test-time-daily.sql` 创建触发区分

```
\d+ partman.time_taptest_table
                                  Table "partman.time_taptest_table"
Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
col1   | integer                  |           | not null |         | plain    |              |
col2   | text                     |           |          |         | extended |              |
col3   | timestamp with time zone |           | not null | now()   | plain    |              |
Indexes:
   "time_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
   time_taptest_table_part_trig BEFORE INSERT ON partman.time_taptest_table FOR EACH ROW EXECUTE FUNCTION
partman.time_taptest_table_part_trig_func()
Child tables: partman.time_taptest_table_p2020_01_27,
              partman.time_taptest_table_p2020_01_28,
              partman.time_taptest_table_p2020_01_29,
              partman.time_taptest_table_p2020_01_30,
```

```
            partman.time_taptest_table_p2020_01_31,
            partman.time_taptest_table_p2020_02_01,
            partman.time_taptest_table_p2020_02_02,
            partman.time_taptest_table_p2020_02_03,
            partman.time_taptest_table_p2020_02_04,
            partman.time_taptest_table_p2020_02_05,
            partman.time_taptest_table_p2020_02_06,
            partman.time_taptest_table_p2020_02_07,
            partman.time_taptest_table_p2020_02_08,
            partman.time_taptest_table_p2020_02_09,
            partman.time_taptest_table_p2020_02_10,
            partman.time_taptest_table_p2020_02_11,
            partman.time_taptest_table_p2020_02_12
Access method: heap
```

关闭触发行为

```
UPDATE partman.part_config SET automatic_maintenance = 'off' WHERE parent_table = 'partman.time_taptest_
table';
```

```
UPDATE partman.part_config_sub SET sub_automatic_maintenance = 'off' WHERE sub_parent = 'partman.time_
taptest_table';
UPDATE partman.part_config_sub SET sub_automatic_maintenance = 'off' WHERE sub_parent = 'partman.time_
taptest_table_p2019_12_08';
[...]
```

创建本地分区

```
CREATE TABLE partman.time_taptest_table_native
    (col1 int, col2 text default 'stuff', col3 timestamptz NOT NULL DEFAULT now())
    PARTITION BY RANGE (col3);

CREATE INDEX ON partman.time_taptest_table_native (col3);
```

```
\dt partman.time_taptest_table
                     List of relations
    Schema     |        Name        | Type  |     Owner
--------------+--------------------+-------+---------------
 partman | time_taptest_table | table | partman_owner
(1 row)

\dp+ partman.time_taptest_table
                                                Access privileges
    Schema     |        Name        | Type  |          Access privileges          | Column privileges |
Policies
```

```
--------------+--------------------+-------+-----------------------------------+-------------------+------
----
 partman | time_taptest_table | table | partman_owner=arwdDxt/partman_owner+|                   |
              |                    |       | partman_basic=arwd/partman_owner   +|                   |
              |                    |       | testing=r/partman_owner             |                   |
(1 row)
```

```
ALTER TABLE partman.time_taptest_table_native OWNER TO partman_owner;
GRANT SELECT, INSERT, UPDATE, DELETE ON partman.time_taptest_table_native TO partman_basic;
GRANT SELECT ON partman.time_taptest_table_native TO testing;
```

数据迁移

```
BEGIN;
LOCK TABLE partman.time_taptest_table IN ACCESS EXCLUSIVE MODE NOWAIT;
```

```
SELECT 'ALTER TABLE '||inhrelid::regclass||' NO INHERIT '||inhparent::regclass||';' FROM sys_inherits WHERE
inhparent::regclass = 'partman.time_taptest_table'::regclass;

                                          ?column?
-----------------------------------------------------------------------------------------------------
 ALTER TABLE partman.time_taptest_table_p2019_12_08 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_09 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_10 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_11 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_12 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_13 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_14 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_15 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_16 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_17 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_18 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_19 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_20 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_21 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_22 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_23 NO INHERIT partman.time_taptest_table;
 ALTER TABLE partman.time_taptest_table_p2019_12_24 NO INHERIT partman.time_taptest_table;
```

```
SELECT (
    SELECT 'ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION '||x.child_tablename||' FOR
VALUES FROM ('||quote_literal(y.child_start_time)||') TO ('||quote_literal(y.child_end_time)||');'
    FROM partman.show_partition_info(x.child_tablename, p_parent_table := 'partman.time_taptest_table') y )
FROM (SELECT inhrelid::regclass::text AS child_tablename FROM sys_inherits WHERE inhparent::regclass =
```

```
'partman.time_taptest_table'::regclass) x;


 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_01_27 FOR
VALUES FROM ('2020-01-27 00:00:00-05') TO ('2020-01-28 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_01_28 FOR
VALUES FROM ('2020-01-28 00:00:00-05') TO ('2020-01-29 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_01_29 FOR
VALUES FROM ('2020-01-29 00:00:00-05') TO ('2020-01-30 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_01_30 FOR
VALUES FROM ('2020-01-30 00:00:00-05') TO ('2020-01-31 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_01_31 FOR
VALUES FROM ('2020-01-31 00:00:00-05') TO ('2020-02-01 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_01 FOR
VALUES FROM ('2020-02-01 00:00:00-05') TO ('2020-02-02 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_02 FOR
VALUES FROM ('2020-02-02 00:00:00-05') TO ('2020-02-03 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_03 FOR
VALUES FROM ('2020-02-03 00:00:00-05') TO ('2020-02-04 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_04 FOR
VALUES FROM ('2020-02-04 00:00:00-05') TO ('2020-02-05 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_05 FOR
VALUES FROM ('2020-02-05 00:00:00-05') TO ('2020-02-06 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_06 FOR
VALUES FROM ('2020-02-06 00:00:00-05') TO ('2020-02-07 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_07 FOR
VALUES FROM ('2020-02-07 00:00:00-05') TO ('2020-02-08 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_08 FOR
VALUES FROM ('2020-02-08 00:00:00-05') TO ('2020-02-09 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_09 FOR
VALUES FROM ('2020-02-09 00:00:00-05') TO ('2020-02-10 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_10 FOR
VALUES FROM ('2020-02-10 00:00:00-05') TO ('2020-02-11 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_11 FOR
VALUES FROM ('2020-02-11 00:00:00-05') TO ('2020-02-12 00:00:00-05');
 ALTER TABLE partman.time_taptest_table_native ATTACH PARTITION partman.time_taptest_table_p2020_02_12 FOR
VALUES FROM ('2020-02-12 00:00:00-05') TO ('2020-02-13 00:00:00-05');
```

```
\d+ partman.time_taptest_table
                                 Table "partman.time_taptest_table"
Column |           Type           | Collation | Nullable | Default | Storage  | Stats target | Description
--------+--------------------------+-----------+----------+---------+----------+--------------+-------------
col1   | integer                  |           | not null |         | plain    |              |
col2   | text                     |           |          |         | extended |              |
col3   | timestamp with time zone |           | not null | now()   | plain    |              |
```

```
Indexes:
   "time_taptest_table_pkey" PRIMARY KEY, btree (col1)
Triggers:
   time_taptest_table_part_trig BEFORE INSERT ON partman.time_taptest_table FOR EACH ROW EXECUTE FUNCTION
partman.time_taptest_table_part_trig_func()
Access method: heap
```

```
\d+ partman.time_taptest_table_native
                            Partitioned table "partman.time_taptest_table_native"
Column |           Type           | Collation | Nullable |    Default    | Storage  | Stats target |
Description
--------+--------------------------+-----------+----------+---------------+----------+--------------+-------
------
col1   | integer                  |           |          |               | plain    |              |
col2   | text                     |           |          | 'stuff'::text | extended |              |
col3   | timestamp with time zone |           | not null | now()         | plain    |              |
Partition key: RANGE (col3)
Indexes:
   "time_taptest_table_native_col3_idx" btree (col3)
Partitions: partman.time_taptest_table_p2020_01_27 FOR VALUES FROM ('2020-01-27 00:00:00-05') TO ('2020-01-
28 00:00:00-05'),
           partman.time_taptest_table_p2020_01_28 FOR VALUES FROM ('2020-01-28 00:00:00-05') TO ('2020-01-29
00:00:00-05'),
           partman.time_taptest_table_p2020_01_29 FOR VALUES FROM ('2020-01-29 00:00:00-05') TO ('2020-01-30
00:00:00-05'),
           partman.time_taptest_table_p2020_01_30 FOR VALUES FROM ('2020-01-30 00:00:00-05') TO ('2020-01-31
00:00:00-05'),
           partman.time_taptest_table_p2020_01_31 FOR VALUES FROM ('2020-01-31 00:00:00-05') TO ('2020-02-01
00:00:00-05'),
           partman.time_taptest_table_p2020_02_01 FOR VALUES FROM ('2020-02-01 00:00:00-05') TO ('2020-02-02
00:00:00-05'),
           partman.time_taptest_table_p2020_02_02 FOR VALUES FROM ('2020-02-02 00:00:00-05') TO ('2020-02-03
00:00:00-05'),
           partman.time_taptest_table_p2020_02_03 FOR VALUES FROM ('2020-02-03 00:00:00-05') TO ('2020-02-04
00:00:00-05'),
           partman.time_taptest_table_p2020_02_04 FOR VALUES FROM ('2020-02-04 00:00:00-05') TO ('2020-02-05
00:00:00-05'),
           partman.time_taptest_table_p2020_02_05 FOR VALUES FROM ('2020-02-05 00:00:00-05') TO ('2020-02-06
00:00:00-05'),
           partman.time_taptest_table_p2020_02_06 FOR VALUES FROM ('2020-02-06 00:00:00-05') TO ('2020-02-07
00:00:00-05'),
           partman.time_taptest_table_p2020_02_07 FOR VALUES FROM ('2020-02-07 00:00:00-05') TO ('2020-02-08
00:00:00-05'),
           partman.time_taptest_table_p2020_02_08 FOR VALUES FROM ('2020-02-08 00:00:00-05') TO ('2020-02-09
00:00:00-05'),
```

```
          partman.time_taptest_table_p2020_02_09 FOR VALUES FROM ('2020-02-09 00:00:00-05') TO ('2020-02-10
00:00:00-05'),
          partman.time_taptest_table_p2020_02_10 FOR VALUES FROM ('2020-02-10 00:00:00-05') TO ('2020-02-11
00:00:00-05'),
          partman.time_taptest_table_p2020_02_11 FOR VALUES FROM ('2020-02-11 00:00:00-05') TO ('2020-02-12
00:00:00-05'),
          partman.time_taptest_table_p2020_02_12 FOR VALUES FROM ('2020-02-12 00:00:00-05') TO ('2020-02-13
00:00:00-05')
```

```
ALTER TABLE partman.time_taptest_table RENAME TO time_taptest_table_old;
ALTER TABLE partman.time_taptest_table_native RENAME TO time_taptest_table;
```

```
CREATE TABLE partman.time_taptest_table_default (LIKE partman.time_taptest_table INCLUDING ALL);
ALTER TABLE partman.time_taptest_table ATTACH PARTITION partman.time_taptest_table_default DEFAULT;
```

```
CREATE TABLE partman.partman_time_taptest_table (LIKE partman.time_taptest_table);
ALTER TABLE partman.partman_time_taptest_table OWNER TO partman_owner;
```

```
ALTER TABLE partman.partman_time_taptest_table ADD PRIMARY KEY (col1);
```

```
UPDATE partman.part_config SET partition_type = 'native', template_table = 'partman.partman_time_taptest_
table' WHERE parent_table = 'partman.time_taptest_table';
```

```
COMMIT;
```

3.49.4.1.6.15　迁移已有数据表到 partman

1. 禁用 run_maintenance()
2. 将被迁移表设置为不可写
3. 将表改为和时间相关的名字，如果分区集的命名方式与所包含的相关数据不同，或者根本不相关，则必须改为根据控制列中的最小值进行重命名。

```
-- 将表改为和时间相关的名字
select 'ALTER TABLE '||n.nspname||'.'||c.relname||' RENAME TO '||substring(c.relname from 1 for 4)||'_p'||
to_char(to_timestamp(substring(c.relname from 5), 'YYYYMMDD'), 'IYYY')||'w'||to_char(to_
timestamp(substring(c.relname from 5), 'YYYYMMDD'), 'IW')||';'
 from sys_inherits h
 join sys_class c on h.inhrelid = c.oid
 join sys_namespace n on c.relnamespace = n.oid
 where h.inhparent::regclass = 'tracking.hits'::regclass
 order by c.relname;
```

```
-- 命名方式与所包含的相关数据不同
insert into tracking.hits_aa values (1, generate_series('2016-01-03 01:00:00'::timestamptz, '2016-01-09 23:
00:00'::timestamptz, '1 hour'::interval));
insert into tracking.hits_bb values (2, generate_series('2016-01-10 01:00:00'::timestamptz, '2016-01-16 23:
00:00'::timestamptz, '1 hour'::interval));
insert into tracking.hits_cc values (3, generate_series('2016-01-17 01:00:00'::timestamptz, '2016-01-23 23:
00:00'::timestamptz, '1 hour'::interval));
```

```
DO $rename$
DECLARE
v_min_val           bigint;
v_row               record;
v_sql               text;
BEGIN

-- Adjust your parent table name in the for loop query
FOR v_row IN
SELECT n.nspname AS child_schema, c.relname AS child_table
 FROM sys_inherits h
 JOIN sys_class c ON h.inhrelid = c.oid
 JOIN sys_namespace n ON c.relnamespace = n.oid
 WHERE h.inhparent::regclass = 'tracking.hits'::regclass
 ORDER BY c.relname
LOOP
-- Substitute your control column's name here in the min() function
v_sql := format('SELECT min(id) FROM %I.%I', v_row.child_schema, v_row.child_table);
EXECUTE v_sql INTO v_min_val;

-- Adjust the numerical value after the % to account for whatever your partitioning interval is.
v_min_val := v_min_val - (v_min_val % 1000);

-- Build the sql statement to rename the child table
v_sql := format('ALTER TABLE %I.%I RENAME TO %I'
     , v_row.child_schema
     , v_row.child_table
     , substring(v_row.child_table from 1 for 4)||'_p'||v_min_val::text);

-- I just have it outputing the ALTER statement for review. If you'd like this code to actually run it,
uncomment the EXECUTE below.
RAISE NOTICE '%', v_sql;
-- EXECUTE v_sql;
END LOOP;

END
$rename$;
```

4. 迁移

```
BEGIN;
DROP TRIGGER myoldtrigger ON tracking.hits;
SELECT partman.create_parent('tracking.hits', 'start', 'partman', 'weekly');
COMMIT;



\d+ tracking.hits
Table "tracking.hits"
Column |            Type             | Modifiers | Storage | Stats target | Description
--------+-----------------------------+-----------+---------+--------------+-------------
id     | integer                     | not null  | plain   |              |
start  | timestamp without time zone | not null  | plain   |              |
Triggers:
hits_part_trig BEFORE INSERT ON tracking.hits FOR EACH ROW EXECUTE PROCEDURE tracking.hits_part_trig_func()
Child tables: tracking.hits_p2015w53,
tracking.hits_p2016w01,
tracking.hits_p2016w02,
tracking.hits_p2016w03,
tracking.hits_p2016w04,
tracking.hits_p2016w05,
tracking.hits_p2016w06,
tracking.hits_p2016w07,
tracking.hits_p2016w08,
tracking.hits_p2016w09
```

3.49.5　插件 kdb_partman 卸载方法

用户在不需要使用 kdb_partman 插件的时候，可以使用 DROP EXTENSION 命令将 kdb_partman 插件卸载。

示例：

```
drop extension kdb_partman;
```

3.49.6　插件 kdb_partman 升级方法

kdb_partman 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.50 kdb_protect

3.50.1 插件 kdb_protect 简介

kdb_protect 插件是 KingbaseES 的一个扩展插件。主要功能是提供保护 database、schema、table 等逻辑对象不被 DROP 的能力。

- 插件名为 kdb_protect
- 插件版本 v1.0

3.50.2 插件 kdb_protect 加载方式

使用时需要在命令行执行 create extension kdb_protect;

示例：

```
create extension kdb_protect;
```

3.50.3 插件 kdb_protect 的参数配置

无需配置任何参数。

3.50.4 插件 kdb_protect 的使用方法

加载插件 kdb_protect 后，按照如下方式使用相应功能。

3.50.4.1 kdb_protect.protect_add

添加被保护对象到保护表中。

语法

```
kdb_protect.protect_add(name text, type char);
kdb_protect.protect_add(name text, type char, iscascade bool);
```

参数

name：被保护对象的名称

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

iscascade：是否级联。只对 db、schema 有效。

返回值

成功返回’t’，失败返回’f’或报错

示例

```
select kdb_protect.protect_add('table1', 't');
select kdb_protect.protect_add('table1', 't', true);
```

3.50.4.2　kdb_protect.protect_add_oid

添加被保护对象到保护表中。

语法

```
kdb_protect.protect_add_oid(objid oid, type char);
kdb_protect.protect_add_oid(objid oid, type char, iscascade bool);
```

参数

objid：被保护对象的 oid

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

iscascade：是否级联。只对 db、schema 有效。

返回值

成功返回’t’，失败返回’f’或报错

示例

```
select kdb_protect.protect_add_oid(16394, 't');
select kdb_protect.protect_add_oid(16356, 's', true);
```

3.50.4.3　kdb_protect.protect_remove

从保护表中删除被保护对象。

语法

```
kdb_protect.protect_remove(name text, type char);
kdb_protect.protect_remove(name text, type char, iscascade bool);
```

参数

name：被保护对象的名称

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

iscascade：是否级联。只对 db、schema 有效。

返回值

成功返回’t’，失败返回’f’或报错

示例

```
select kdb_protect.protect_remove('table1.mysch', 't');
select kdb_protect.protect_remove('test', 's', true);
```

3.50.4.4　kdb_protect.protect_remove_oid

从保护表中删除被保护对象。

语法

```
kdb_protect.protect_remove_oid(objid oid, type char);
```

参数

objid：被保护对象的 oid

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

返回值

成功返回’t’，失败返回’f’或报错

示例

```
select kdb_protect.protect_remove_oid(16385, 't');
```

3.50.4.5　kdb_protect.protect_remove_all

删除保护表中所有被保护对象。

语法

```
kdb_protect.protect_remove_all();
```

参数

无

返回值

成功返回’t’，失败返回’f’或报错

示例

```
select kdb_protect.protect_remove_all();
```

3.50.4.6　kdb_protect.protect_show()

显示所有被保护对象。

语法

```
kdb_protect.protect_show();
```

参数

无

返回值

成功返回保护的对象信息，失败报错

示例

```
select * from kdb_protect.protect_show();
```

3.50.4.7　kdb_protect.protect_show_detail()

显示所有被保护对象（更详细信息）。

语法

```
kdb_protect.protect_show_detail();
```

参数

无

返回值

成功返回保护的对象信息，失败报错

示例

```
select * from kdb_protect.protect_show_detail();
```

3.50.4.8　kdb_protect.protect_show_one()

显示一个被保护对象。

语法

```
kdb_protect.protect_show_one(name text, type char);
```

参数

name：被保护对象的名称

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

返回值

成功返回保护的对象信息，失败报错

示例

```
select * from kdb_protect.protect_show_one('table1', 't');
```

3.50.4.9　kdb_protect.protect_show_one_detail(name text, type char)

显示一个被保护对象（更详细信息）。

语法

```
kdb_protect.protect_show_one_detail(name text, type char);
```

参数

name：被保护对象的名称

type：被保护对象的类型，如下：

‘d’--database(数据库)

‘s’--schema(模式)

‘t’--table(表/分区表)

‘v’--view/materialized view(视图/物化视图)

‘f’--function/procedure(函数/存储过程)

返回值

成功返回保护的对象信息，失败报错

示例

```
select * from kdb_protect.protect_show_one_detail('table1', 't');
```

3.50.5　插件 kdb_protect 卸载方法

卸载时 drop extension kdb_protect 即可。

示例：

```
drop extension kdb_protect;
```

3.50.6 插件 kdb_protect 升级方法

kdb_protect 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.51 kdb_raw

3.51.1 插件 kdb_raw 简介

kdb_raw 定义了一套对于 raw 数据类型的操作。支持 utl_raw、utl_enode 和 utl_i18n 内置包。参见《PL/SQL 系统包和类型参考手册》中的 UTL_RAW 包，UTL_ENCODE 包，UTL_I18N 包。

- 插件名为 kdb_raw
- 插件版本 V1.3

3.51.2 插件 kdb_raw 加载方式

KingbaseES 默认不加载 kdb_raw 扩展插件，请使用以下命令加载插件和创建扩展：

```
CREATE EXTENSION kdb_raw;
```

3.51.3 插件 kdb_raw 的参数配置

无需配置任何参数

3.51.4 插件 kdb_raw 卸载方法

如被 dbms_lob 插件依赖，需要先卸载 dbms_lob 插件，或使用 cascade 关键字级联卸载，执行如下命令：

```
drop extension kdb_raw [cascade];
```

3.51.5 插件 kdb_raw 升级方法

如果使用旧版本插件，可以升级到最新版本。

备注:

不支持跨版本升级。建议由低到高依次升级版本。

```
alter extension kdb_raw update to '1.3';
```

3.52 kdb_resource_group

3.52.1 插件 kdb_resource_group 简介

kdb_resource_group 是 KingbaseES 的一个扩展插件，通过 kdb_resource_group 可以实现资源组功能。资源组限额需求主要是让数据库对操作系统资源限制、控制与分离一个进程组的资源（如 CPU、内存、磁盘输入输出等），基于 cgroup 对资源有更大的控制从而避免因操作系统效率低下而产生的过度的资源开销、低效的调度，资源分配不当等问题。

在 KingbaseES 中支持构建和管理资源组实现对资源控制的隔离。资源组功能依赖于操作系统的 cgroup，所以仅支持类 linux 操作系统。

- 插件名为 kdb_resource_group
- 插件版本 V1.0

3.52.2 插件 kdb_resource_group 加载方式

使用并开启资源组需要以下操作：

- 资源组功能依赖于操作系统的 cgroup 功能，所以必须安装 cgroup 并正确挂载和启动，对应的挂载的子系统目录给予操作用户写入和读取权限。
- 启动数据库前需要将动态库添加到 kingbase.conf 文件的 shared_preload_libraries 中，若已启动需重启数据库。

```
shared_preload_libraries = 'kdb_resource_group'
```

3.52.3 插件 kdb_resource_group 的参数配置

- 在 kingbase.conf 文件中设置参数，开启资源组功能，默认是 off 为关闭状态

```
resource_group_activated = on
```

- 在 kingbase.conf 文件中扩展内的非必选参数，设置 cgroup 的挂载主目录，默认为’/sys/fs/cgroup’，修改后需要重启数据库生效

```
kdb_resource_group.resource_group_path = '/sys/fs/cgroup' #(change requires restart)
```

- 在 kingbase.conf 文件中扩展内的非必选参数，设置 kingbase 可使用操作系统 CPU 的总量，默认为 100，取值范围 1~100，修改后需要删除 cgroup 下的 kingbase 节点，命令如下：cgdelete -r cpu:kingbase。并重启数据库生效。

```
kdb_resource_group.resource_group_total_cpu = 100 #(change requires restart)
```

备注:

开启资源组后连接数据库报错或警告时，确认 GUC 参数设置，以及 cgroup 目录的权限和拥有者，可以使用以下命令清除资源组的 kingbase 节点，然后重新连接数据库会再次创建 cgroup 的 kingbase 节点。

```
cgdelete -r cpu:kingbase
```

3.52.4 插件 kdb_resource_group 的使用方法

资源组用来将主机提供的资源 (CPU、内存、、IO) 池化，分配给指定的数据库、用户。所有关联到资源组的用户、数据库共享资源组的资源，并受资源组的可用的资源限制。资源组是多库共享且名称唯一，信息持久化存储到数据库系统中。

创建一个资源组并设置参数值, 只有 superuser 可以创建，最多支持 62 个资源组。例如，rgroup1 为创建的资源组名称, 一个数据库内唯一，该对象不属于某个模式下。

资源组参数列表如下：

表 3.52.1: 资源组参数列表

参数名	意义	其他说明
cpu_list	使用的 CPU 核编号列表	如不指定则使用所有核，默认可用所有 cpu 核，参数与绑核功能有冲突，请尽量避免同时使用
mem_percent	软性限制可使用的最大内存占上层资源组的百分比，在内存资源竞争时生效	默认-1 不限制，限制时取值 1-100。所有第二层资源组百分比之和不能超过 100
mem_limit	硬性限制资源组可使用的内存最大数值	默认-1 不限制，限制时取值不小于 1MB
io_limit	IOPS 限制，每秒 io 次数上限	正常取值范围 [0,2^32-2] ，0 表示不限制。读、写分别独立限制
cpu_cfs_quota_us	cpu 绝对使用比例	取值范围 [1,100]，默认值 -1 默认不限制
cpu_shares	cpu 相对使用比例	取值范围 [10,10000] 默认值采用操作系统设定

备注:

mem_percent 类型限制参数为软性限制，只有在资源竞争时生效。mem_limit 和 mem_percent 同时指定时，两个限额都会起作用。

3.52.4.1　创建资源组

语法：

```
CREATE RESOURCE GROUP group_name [ WITH ({cpu_list = "list" | mem_percent = pct | mem_limit = "space"| io_
limit = desc | cpu_cfs_quota_us= quota | cpu_shares=shares }[, ··· ])]
```

功能说明：

1. 资源组名称不可重复。
2. 资源参数如不指定使用默认值，默认对该资源不设限制，各参数对应默认值请参照资源组参数列表。
3. 创建新的资源组时指定使用的资源参数不可超过 TopResourceGroup 的空余资源。
4. 仅管理员可创建资源组。
5. 语句中的参数大小写不敏感。

参数说明:

group_name：预创建的资源组名称, 字符串，参照数据库表的命名规范。

pct：百分比的数值，正常取值范围 1-100，-1 表示不限制，不可带‘%’如：90 。

list：cpu 核编号字符串，编号不可超过最大 cpu 核数量，以逗号分隔多个编号，cpu 编号为整数。‘-’符号支持范围表示。如：“1,2,5,9,64 “ “1-20”。

desc：字符串，格式为: [主设备号：次设备号]num。主设备号和次设备号对应的是要限定的磁盘的主次设备号。如省略，则默认限定数据目录所在磁盘 (暂未支持默认 8:0)。num 限制对应设备的每秒 IO 次数。正常取值范围 [0 ，2^31-1] ，0 表示不限制。如 16384。

space：表示内存空间的大小，取值不小于 1MB, 单位 B、MB、GB，负数和 0 表示不限制。如：1000000B 。

quota: 兼用原版本参数，取值范围 [1,100]，默认值 -1，不需要附带单位，如：100。

shares: 兼用原版本参数，取值范围 [10,10000] 默认值采用操作系统设定，不需要附带单位，如：1000 。

示例：

```
CREATE RESOURCE GROUP my_group WITH (cpu_list="1-32",MEM_PERCENT = 50, mem_limit = "200MB", io_limit =
123321);
```

```
CREATE RESOURCE GROUP rgroup1 WITH (cpu_cfs_quota_us = 100, cpu_shares = 10000);
```

3.52.4.2　修改资源组

语法：

```
ALTER RESOURCE GROUP group_name [ WITH ({cpu_list = "list" | mem_percent=pct | mem_limit = "space"| io_limit
= number | cpu_cfs_quota_us = quota | cpu_shares = shares }[, … ])]
```

功能说明：

1. 参数及返回值与创建资源组 相同。
2. 修改资源组时指定使用的资源参数不可超过 TopResourceGroup 的空余资源。
3. 仅管理员可修改资源组。
4. 语句中的参数大小写不敏感。

参数说明：

请参照创建资源组 的参数。

示例：

例 1：修改一个资源组的限制值

```
ALTER RESOURCE GROUP my_group WITH (cpu_list="1-32",MEM_PERCENT=50,mem_limit="200MB" ,io_limit=123321);
```

例 2：设置一个资源组的相对 CPU 使用率

基于 cgroup 的 cpu 子系统，可以调用 CPU 的获取量，这里采用了完全公平调度程序（CFS），一个比例分配调度程序，可根据任务优先级/权重或资源组分得的份额，在任务群组（cgroups）间按比例分配 CPU 时间（CPU 带宽）。

在 CFS 中，如果系统有足够的空闲 CPU 周期，资源组可获得比其自有份额更多的 CPU 可用量，因为该调度程序有连续工作的特性。此情况通常会在资源组根据相关共享消耗 CPU 时间时出现。

```
ALTER RESOURCE GROUP rgroup1 SET (cpu_shares = 5000);
```

例 3：设置一个资源组的 CPU 绝对使用率限制

可以设定在某一阶段资源组中所有任务可运行的时间总量，一旦资源组中任务用完按配额分得的时间，它们就会被在此阶段的时间提醒限制流量，并在进入下阶段前禁止运行。以此特性我们通过设置 CFS_QUOTA_US 限制 CPU 使用的绝对上限。将参数的值设定为 100，这表示资源组不需要遵循任何 CPU 绝对限制。这也是每个资源组的默认值。表示 kingbase 占的总 cpu 使用量（基于 GUC 参数 resource_group_total_cpu 计算，默认为 100），多核机器合并计算限制使用率，取值范围不变。

```
ALTER RESOURCE GROUP rgroup1 SET (cpu_cfs_quota_us = 10);
```

3.52.4.3　删除资源组

语法：

```
DROP RESOURCE GROUP group_name
```

功能说明:

1. TopResrouceGroup、DefaultResourceGroup 不可被删除。
2. 仅管理员可删除资源组。
3. 语句中的参数大小写不敏感。

参数说明：

group_name：指定需删除的资源组名称。

示例：

```
drop resource group rgroup1;
```

3.52.4.4　查看资源组

资源组的信息包括：资源组名、资源组参数列表中所有参数，加入资源组的用户和库信息。相关视图如下所示：

`sys_resource_groups` 视图

表 3.52.2: sys_resource_groups 视图

sys_resource_groups 列	含义
resgroup_id	资源组 id
resgroup_name	资源组名称
resgroup_createtime	资源组创建时间
resgroup_users	加入资源组的用户 oid
resgroup_databases	加入资源组的数据库 oid
resgroup_options	资源组的资源限额设置，子项有：cpu_list、mem_percent、mem_limit、io_limit、cpu_cfs_quota_us、cpu_shares

`sys_resgroup` 视图

表 3.52.3: sys_resgroup 视图

sys_resgroup 列	含义
oid	资源组 id
rsgname	资源组名称
parent	父资源组的 id, 固定为 TopResrouceGroup 的 oid(原版本未启用)
rstype	原版本未启用

`sys_resgroupcapability` 视图

表 3.52.4: sys_resgroupcapability 视图

sys_resgroupcapability 列	含义
resgroupid	资源组 id
reslimittype	资源限制类型，仅支持 0，2，3，4
value	资源限制类型对应的限制值

`sys_resauthid` 视图

表 3.52.5: sys_resauthid 视图

sys_resauthid 列	含义
roloid	加入资源组的用户 id
rolname	加入资源组的用户名
resgoid	资源组 id

3.52.4.5　用户和数据库关联资源组

支持 CREATE USER/DATABASE 和 ALTER USER/DATABASE 时提供指定或者修改用户/库使用的主机资源和 DBMS 资源的限制方式。当用户和其访问的库都加入自定义资源组时，以用户的限制资源为准，库的限制会失效。

3.52.4.5.1　创建用户指定资源组

语法:

```
CREATE USER user_name [ [WITH] resource_group group_name];
```

功能说明:

1. 资源限制属性仅对当前用户生效，不支持角色继承。
2. 如不指定用户使用的资源组则使用 DefaultResourceGroup。
3. 不支持为 ROLE 指定资源组。
4. 大小写不敏感。

参数说明:

group_name：取值是用户加入的资源组名称，需事先已创建。如不存在则创建用户失败，并报错。

示例：

```
CREATE USER new_user WITH resource_group 'rsp1';
```

3.52.4.5.2　创建库指定资源组

语法：

```
CREATE DATABASE db_name [ [WITH] resource_group  [=] value [···] ]
```

功能说明：

1. 资源限制属性仅对当前库生效。不从模板库继承资源限制属性。
2. 如不指定库使用的资源组则使用 DefaultResourceGroup。
3. 大小写不敏感。
4. 未指定的资源参数表示不限制。

参数说明:

资源限制参数的格式为: [parameter [=] value] 形式，空格分开多个参数。

示例：

```
CREATE DATABASE new_db WITH  resource_group="usercreategroup";
```

3.52.4.5.3　修改用户资源组

语法：

```
ALTRER USER user_name [WITH]  resource_group 'rsgrp_name';
```

功能说明:

修改用户所在的资源组。

参数说明:

rsgrp_name：指定要加入的资源组名字，需已创建，否则修改失败并留在原资源组。

示例：

```
ALTER USER new_user WITH resource_group 'newgroup';
```

3.52.4.5.4　修改数据库资源组

语法：

```
ALTRER DATABASE db_name [WITH] options;
```

功能说明:

1. 将数据库的资源限制参数修改为 options 指定值。
2. 未指定的参数将保留修改前的限制值。

参数说明:

options 参数请参照 create database 使用的资源参数说明。

示例：

```
ALTER DATABASE new_db WITH resource_group 'new_rsgroup';
```

3.52.4.6　资源组关联用户

通过 SQL 语句关联数据库用户，将此用户加入资源组，限制其使用资源的情况。由于资源组是在数据库下设置的，限制的是用户在某个数据库连接应用下的资源限额情况。此 SQL 语句将 resgroup_user1 用户加入到 rgroup1 资源组中，resgroup_user1 用户将受到 rgroup1 中资源组的资源使用控制。

```
create user resgroup_user1 superuser login;
alter resource group rgroup1 set (user='resgroup_user1');
```

3.52.4.6.1　查看资源组与用户关联关系

查看资源组与用户关联关系，通过系统表 sys_resauthid 可查询数据库用户和资源组的关联。

```
select * from sys_resauthid;
```

3.52.5　插件 kdb_resource_group 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数并设置 `resource_group_activated` 参数后重启数据库。

```
shared_preload_libraries = ''
resource_group_activated = off
```

3.52.6　插件 kdb_resource_group 升级方法

kdb_resource_group 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.53　kdb_schedule

3.53.1　概述

kdb_schedule 是 KingbaseES 的一个内置扩展插件，用于用户自定义定时任务。

在使用该插件前，需了解一些前置知识：

3.53.1.1　权限

kdb_schedule 插件要求使用该插件的用户拥有对应的系统权限和对象权限。

系统权限

使用 kdb_schedule 插件要求用户拥有对应的系统权限，关于系统权限的说明如下：

- 创建外部 job（运行在操作系统上的 job），必须拥有 CREATE JOB 权限、CREATE ANY JOB 权限、CREATE EXTERNAL JOB 权限之一。

- 创建 job、program、schedule 对象，必须拥有 CREATE JOB 权限或 CREATE ANY JOB 权限。
- 删除、修改、使用任意 job、program、schedule 对象，必须拥有 CREATE ANY JOB 权限。
- 使用任意 program 对象，必须拥有 CREATE ANY JOB 权限或 EXECUTE ANY RPOGRAM 权限之一。

以上权限均可通过 GRANT 或 REVOKE 语句来进行授予或回收：

```
GRANT system_privilege TO user_name;
REVOKE system_privilege FROM user_name;
```

备注:

在使用 kdb_schedule 插件前需先加载 sysprivilege 插件，以确保用户可以成功获取 JOB 相关的系统权限。

对象权限

使用 kdb_schedule 插件要求用户拥有对应的对象权限，关于不同对象的权限说明如下：

对象类别	支持权限
job	ALTER
program	ALTER、EXECUTE
schedule	ALTER

以上权限均可通过 GRANT 或 REVOKE 语句来进行授予或回收：

```
GRANT object_privilege ON object_type object_name TO user_name;
REVOKE object_privilege ON object_type object_name FROM user_name;
```

3.53.1.2　日历表示法

日历表示法是用来定义重复间隔的实现方法，DBMS_JOB 和 DBMS_SCHEDULER 中的时间定义需要使用该语法。

语法格式

```
repeat_interval = regular_schedule
regular_schedule = frequency_clause
[";" interval_clause] [";" bymonth_clause] [";" byweekno_clause]
[";" byyearday_clause] [";" bydate_clause] [";" bymonthday_clause]
[";" byday_clause] [";" byhour_clause] [";" byminute_clause] [";" bysecond_clause]
```

```
frequency_clause = "FREQ" "=" ( predefined_frequency predefined_frequency =
"YEARLY" | "MONTHLY" | "WEEKLY" | "DAILY" | "HOURLY" | "MINUTELY" | "SECONDLY" )
interval_clause = "INTERVAL" "=" intervalnum
   intervalnum = 1 through 99
bymonth_clause = "BYMONTH" "=" monthlist
   monthlist = month ( "," month)*
   month = numeric_month | char_month
   numeric_month = 1 | 2 | 3 ...  12
   char_month = "JAN" | "FEB" | "MAR" | "APR" | "MAY" | "JUN" | "JUL" | "AUG" | "SEP" | "OCT" | "NOV" | "DEC
"
bymonthday_clause = "BYMONTHDAY" "=" monthday_list
   monthday_list = monthday ( "," monthday)*
   monthday = [minus] monthdaynum
   monthdaynum = 1 through 31
byhour_clause = "BYHOUR" "=" hour_list
   hour_list = hour ( "," hour)*
   hour = 0 through 23
byminute_clause = "BYMINUTE" "=" minute_list
   minute_list = minute ( "," minute)*
   minute = 0 through 59
byscond_clause = "BYSECOND" "=" second_list
   second_list = second ( "," second)*
   second = 0 through 59
```

参数说明

表 3.53.2: 参数含义

参数	描述
FREQ	重复时间的类型。 支持的输入有： yearly monthly weekly daily hourly minutely secondly
INTERVAL	重复的频率，默认值是 1。
BYMONTH	指定月份。
BYMONTHDAY	指定一月中的某一天。

-续表-

参数	描述
BYHOUR	指定一天中的某一个小时。
BYMINUTE	指定一个小时中的某一分。
BYSECOND	指定一分钟内的某一秒。

示例

每天早上 10 点

```
FREQ=DAILY;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

每个隔 7 天早上 10 点

```
FREQ=DAILY;INTERVAL=7;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

每年 10 月 1 号早上 10 点

```
FREQ=YEARLY;BYMONTH=10;BYMONTHDAY=1;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

3.53.2　kdb_schedule 的使用

3.53.2.1　配置及加载

使用 kdb_schedule 插件之前，需要修改配置文件 kingbase.conf 并重启数据库，这样才能保证该插件的正常使用。

需修改的配置项如下：

配置项	配置项说明	备注
shared_preload _libraries	动态库加载名称	必须配置，否则插件无法正常启用
job_queue_processes	允许用户启动的最大并发数，默认为 0 0 - 不启动自动作业功能	必须配置，否则插件无法正常启用

-续表-

配置项	配置项说明	备注
sys_job.log_level	参数定义该插件的日志等级，可选项如下 (默认值为 DEBUG5)： DEBUG5 DEBUG4 DEBUG3 DEBUG2 DEBUG1 LOG	可选配置项
sys_job.poll_time	设置轮询系统表间隔时间, 取值范围为 1 至 20 秒 单位秒，默认值为 10 秒	可选配置项
max_worker _processes	数据库最大工作后台进程数目，默认为 30 值应满足条件“max_worker_processes >= job_queue _processes + 10”	可选配置项

备注:

1. kdb_schedule 目前不支持 windows 环境下使用。
2. 在特殊环境下（申威等)，初始化数据库时，提示用户 kdb_schedule.so 缺失库文件时，需用户手动 export LD_LIBRARY_PATH 为数据库 lib 目录。

3.53.2.2 使用

插件加载完毕后直接调用插件中的存储过程实现业务需求即可，插件组成请见kdb_schedule 的组成 。

备注:

1. 作业执行过程中，如果发生断电等情况导致的异常宕机，可能导致作业状态错误（一直保持 RUNNING 状态）。此时需要用户删除该作业，然后重建作业。
2. 超级用户拥有相关系统表的 DML 权限，但不建议用户使用。

3.53.2.3 升级

kdb_schedule 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.53.2.4　卸载

删除 kingbase.conf 文件中 shared_preload_libraries 参数值中的“kdb_schedule”后重启数据库。

3.53.3　kdb_schedule 的组成

kdb_schedule 插件组成内容如下（此处的模式指的是数据库对象 schema）：

- DBMS_JOB 模式 及该模式下的存储过程
- DBMS_SCHEDULER 模式 及该模式下的存储过程
- KDB_SCHEDULE 模式 及该模式下的视图

不同的数据库兼容模式下该插件的支持程度不同，下述为 kdb_schedule 插件在不同数据库模式下的兼容情况：

兼容模式	schmea	支持情况
Oracle 兼容模式	DBMS_JOB	支持该模式下所有存储过程
	DBMS_SCHDULER	支持该模式下所有存储过程
	KDB_SCHEDULE	支持该模式下的视图
MySQL 兼容模式	DBMS_JOB	支持该模式下所有存储过程
	DBMS_SCHDULER	支持该模式下所有存储过程
	KDB_SCHEDULE	支持该模式下的视图
SQLServer 兼容模式	DBMS_JOB	不支持
	DBMS_SCHDULER	支持该模式下所有存储过程
	KDB_SCHEDULE	支持该模式下的视图

备注:

SQLServer 兼容模式不支持系统权限、对象权限，用户接口仅限超级用户使用。

3.53.3.1　DBMS_JOB 模式

DBMS_JOB 模式提供调度和管理作业任务的功能。该模式下的存储过程功能都可以由 DBMS_SCHDULE 替代，共包含 9 个存储过程，分别对应 JOB 的 9 个操作：

表 3.53.5: DBMS_JOB 存储过程

存储过程名称	说明
BROKEN 存储过程	中止一个处于可执行状态的 job
CHANGE 存储过程	更改 job 的运行参数
INSTANCE 存储过程	增加 job 执行的连接串信息。
INTERVAL 存储过程	更改 job 运行的时间
NEXT_DATE 存储过程	更改 job 下一次的运行时间
REMOVE 存储过程	删除一个 job
RUN 存储过程	立即运行指定的 job
SUBMIT 存储过程	创建一个 job
WHAT 存储过程	更改 job 执行的动作

需要注意的是，DBMS_JOB 使用时存在如下限制：

1. 需要创建 kdb_schedule 扩展。
2. 仅支持 PL/SQL 类型的任务。
3. 仅支持本地任务。
4. 间隔时间采用日历表示法。

3.53.3.1.1　BROKEN 存储过程

此存储过程将 job 设置为中止，被设置为中止的 job 不会再运行。

语法格式

```
DBMS_JOB.BROKEN(
        job INTEGER,
        broken BOOLEAN,
        next_date TIMESTAMP DEFAULT now());
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- broken 参数，将 job 设置为 enable 或者 disable。true 为 disable，false 为 enable。
- next_date 参数，job 运行的下一个日期。

示例

```
call dbms_job.submit(1, 'CREATE TABLE T1(A int)', now(), 'Freq=daily;BYHOUR=10;BYMINUTE=10;BYSECOND=10',
false, 0, false);
call dbms_job.instance(1, 'user=system dbname=test port=54321 password=123456');
call dbms_job.broken(1,TRUE,now());
```

3.53.3.1.2　CHANGE 存储过程

此存储过程更改 job 中的属性信息。

语法格式

```
DBMS_JOB.CHANGE(
       job INTEGER,
       what TEXT,
       next_date TIMESTAMP,
       interva TEXT,
       instance INTEGER DEFAULT 0,
       force BOOLEAN DEFAULT FALSE);
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- what 参数，job 要运行的 PL/SQL。
- next_date 参数，job 运行的下一个日期。
- interva 参数，job 运行的间隔时间。
- instance 参数，暂不支持，默认为 0。
- force 参数，暂不支持，默认为 FALSE。

示例

```
call dbms_job.change(1, 'CREATE TABLE T1(a INT);', now(), 'Freq=daily;BYHOUR=10;BYMINUTE=10;BYSECOND=10', 0,
false);
```

3.53.3.1.3　INTERVAL 存储过程

此过程更改 job 中的间隔时间属性。

语法格式

```
DBMS_JOB.INTERVAL(
        job INTEGER,
        interva TEXT);
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- interva 参数，job 运行的间隔时间。

示例

```
call dbms_job.interval(1, 'Freq=daily;BYHOUR=10;BYMINUTE=10;BYSECOND=10');
```

3.53.3.1.4　NEXT_DATE 存储过程

此过程更改 job 中的下次运行时间信息。

语法格式

```
DBMS_JOB.NEXT_DATE(
        job INTEGER,
        next_date TIMESTAMP);
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- next_date 参数，job 运行的下一个日期。

示例

```
call dbms_job.next_date(1, now());
```

3.53.3.1.5　REMOVE 存储过程

此过程删除一个 job。

语法格式

```
DBMS_JOB.REMOVE(
        job INTEGER);
```

参数说明

- job 参数，由系统分配的 job 的 ID。

示例

```
call dbms_job.remove(1);
```

3.53.3.1.6　RUN 存储过程

此过程运行一个 job。

语法格式

```
DBMS_JOB.RUN(
        job INTEGER);
```

参数说明

- job 参数，由系统分配的 job 的 ID。

示例

```
call dbms_job.submit(1, 'CREATE TABLE T1(A int)', now(), 'Freq=daily;BYHOUR=10;BYMINUTE=10;BYSECOND=10',
false, 0, false);
call dbms_job.instance(1, 'user=system dbname=test port=54321 password=123456');
call dbms_job.run(1);
```

3.53.3.1.7　SUBMIT 存储过程

此过程创建一个 job。

语法格式

```
DBMS_JOB.SUBMIT(
        OUT job INTEGER,
        what text,
        next_date TIMESTAMP DEFAULT now(),
        interva text DEFAULT NULL,
        no_parse BOOLEAN DEFAULT FALSE,
        instance INTEGER DEFAULT 0,
        force BOOLEAN DEFAULT FALSE);
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- what 参数，job 要运行的 PL/SQL。
- next_date 参数，job 运行的下一个日期。
- interva 参数，job 运行的间隔时间。
- no_parse 参数，暂不支持，默认为 FALSE。

- instance 参数，暂不支持，默认为 0。
- force 参数，暂不支持，默认为 FALSE。

示例

```
\set SQLTERM /
DECLARE
  JOBID INT;
BEGIN
  DBMS_JOB.SUBMIT(jobid, 'CREATE TABLE T1(A int)', now(),
  'Freq=daily;BYHOUR=10;BYMINUTE=10;BYSECOND=10', false, 0, false);
END;
/
\set SQLTERM ;
```

3.53.3.1.8　WHAT 存储过程

此过程更改 job 执行的行为信息。

语法格式

```
DBMS_JOB.WHAT(
        job INTEGER,
        what TEXT);
```

参数说明

- job 参数，由系统分配的 job 的 ID。
- what 参数，job 要运行的 PL/SQL。

示例

```
call dbms_job.what(1, 'CREATE TABLE T1(a INT);');
```

3.53.3.1.9　INSTANCE 存储过程

此过程增加 job 执行的连接串信息。

语法格式

```
DBMS_JOB.INSTANCE(
        job INTEGER,
        instance TEXT);
```

参数说明

- job 参数，由系统分配的 job 的 ID。

- instance 参数，job 要运行数据库的连接串。

示例

```
call dbms_job.instance(1, 'user=system dbname=test port=54321 password=123456');
```

3.53.3.2　DBMS_SCHEDULER 模式

DBMS_SCHEDULER 模式提供调度和管理作业任务的功能，该模式下包含 18 个兼容视图和 10 个存储过程，它们包含了 DBMS_JOB 模式下存储过程的功能，并有所增强。

存储过程名称	描述
CREATE_PROGRAM 存储过程	创建一个程序
DROP_PROGRAM 存储过程	删除一个程序
CREATE_SCHEDULE 存储过程	创建一个调度程序
DROP_SCHEDULE 存储过程	删除一个调度程序
CREATE_JOB 存储过程	创建一个 job 任务
DROP_JOB 存储过程	删除一个 job 任务
RUN_JOB 存储过程	运行一个 job 任务
DISABLE 存储过程	将一个 job 任务状态设置为 DISABLE
ENABLE 存储过程	将一个 job 任务状态设置为 ENABLE

DBMS_SCHEDULER 具有如下限制：

1. 需要创建 kdb_schedule 扩展。
2. 间隔时间采用日历表示法。

3.53.3.2.1　CREATE_PROGRAM 存储过程

创建一个 job 任务的程序。

语法格式

```
DBMS_SCHEDULER.CREATE_PROGRAM(
        program_name TEXT,
        program_type TEXT,
```

```
        program_action TEXT,
        acconnstr TEXT,
        acdbname TEXT,
        number_of_arguments INTEGER DEFAULT 0,
        enabled BOOLEAN DEFAULT FALSE,
        comments TEXT DEFAULT NULL);
```

参数说明

- program_name 参数，程序的名字。
- program_type 参数，程序的类型。有下列类型：PLSQL_BLOCK、STORED_ 存储过程、SQL_SCRIPT、EXECUTABLE、EXTERNAL_SCRIPT、BACKUP_SCRIPT。
- program_action 参数，程序的动作。
- acconnstr 参数，数据库连接串，如果为 NULL 则生成本库任务。
- acdbname 参数，无实际作用，仅作为兼容参数保留。
- number_of_arguments 参数，程序动作的参数，暂不支持, 0 为默认值。
- enabled 参数，程序的状态，暂不支持，false 为默认值。
- comments 参数，程序的注释信息。

示例

1、创建一个在 test 数据库执行的 program：

```
call DBMS_SCHEDULER.CREATE_PROGRAM('program1', 'PLSQL_BLOCK', 'create table t2(A int)', 'user=system
dbname=test port=54321 password=123456', 'test', 0, true, 'this is test program');
```

2、创建一个在本数据库执行的 program：

```
call DBMS_SCHEDULER.CREATE_PROGRAM('program1', 'PLSQL_BLOCK', 'create table t2(A int)', NULL, NULL, 0, true,
'this is test program');
```

3.53.3.2.2　DROP_PROGRAM 存储过程

删除一个 job 任务的程序。

语法格式

```
DBMS_SCHEDULER.DROP_PROGRAM(
        program_name TEXT,
        force BOOLEAN DEFAULT FALSE);
```

参数说明

- program_name 参数，程序的名字。

- force 参数，暂不支持，默认值为 false。

示例

```
call dbms_scheduler.drop_program('program1', true);
```

3.53.3.2.3　CREATE_SCHEDULE 存储过程

创建一个 job 任务的调度程序。

语法格式

```
DBMS_SCHEDULER.CREATE_SCHEDULE(
        schedule_name TEXT,
        start_date TIMESTAMP WITH TIME ZONE DEFAULT NULL,
        repeat_interval TEXT DEFAULT NULL,
        end_date TIMESTAMP WITH TIME ZONE DEFAULT NULL,
        comments TEXT DEFAULT NULL);
```

参数说明

- schedule_name 参数，调度程序的名字。
- start_date 参数，调度程序的开始时间。
- repeat_interval 参数，调度程序的间隔时间。
- end_date 参数，调度程序的结束时间。
- comment 参数，调度程序的注释信息。

示例

```
call DBMS_SCHEDULER.CREATE_SCHEDULE('schedule1', now(), 'Freq=daily;BYHOUR=10; BYMINUTE=10;BYSECOND=10',
NULL, 'this is test schedule');
```

3.53.3.2.4　DROP_SCHEDULE 存储过程

删除一个 job 任务的调度程序。

语法格式

```
DBMS_SCHEDULER.DROP_SCHEDULE(
        schedule_name TEXT,
        force BOOLEAN DEFAULT FALSE);
```

参数说明

- schedule_name 参数，调度程序的名字。

- force 参数，暂不支持，默认值为 false。

示例

```
call DBMS_SCHEDULER.DROP_SCHEDULE('schedule1', true);
```

3.53.3.2.5　CREATE_JOB 存储过程

创建一个 job 任务。

语法格式

```
DBMS_SCHEDULER.CREATE_JOB(
        job_name TEXT,
        program_name TEXT,
        schedule_name TEXT,
        job_class TEXT DEFAULT 'Routine Maintenance',
        enabled BOOLEAN DEFAULT FALSE,
        auto_drop BOOLEAN DEFAULT TRUE,
        comments TEXT DEFAULT NULL,
        credentail_name TEXT DEFAULT NULL,
        destination_name TEXT DEFAULT NULL);
```

参数说明

- job_name 参数，job 的名字。
- program_name 参数，程序的名字。
- schedule_name 参数，调度程序的名字。
- job_class 参数，job 的类型，默认 Routine Maintenance。
- enabled 参数，job 的状态。true 启用，false 禁用。
- auto_drop 参数，job 完成后自动删除，暂不支持，默认 true。
- comment 参数，job 的注释信息。
- credentail_name 参数，暂不支持，默认为 NULL。
- destination_name 参数，暂不支持，默认为 NULL。

示例

```
call DBMS_SCHEDULER.CREATE_PROGRAM('program1', 'PLSQL_BLOCK', 'create table t2(A int)', 'user=system
dbname=test port=54321 password=123456', 'kingbase', 0, true, 'this is test program');
call DBMS_SCHEDULER.CREATE_SCHEDULE('schedule1', now(), 'Freq=daily;BYHOUR=10; BYMINUTE=10;BYSECOND=10',
NULL, 'this is test schedule');
call DBMS_SCHEDULER.CREATE_JOB('job', 'program1', 'schedule1', 'Routine Maintenance', false, true, 'this is
test job');
```

3.53.3.2.6　DROP_JOB 存储过程

删除一个 job 任务。

语法格式

```
DBMS_SCHEDULER.DROP_JOB(
        job_name TEXT,
        force BOOLEAN DEFAULT FALSE,
        defer BOOLEAN DEFAULT FALSE,
        commit_semantics TEXT DEFAULT 'STOP_ON_FIRST_ERROR');
```

参数说明

- job_name 参数，job 的名字。
- force 参数，暂不支持，默认值为 false。
- defer 参数，暂不支持，默认值为 false。
- commit_semantice 参数，暂不支持，默认值为 STOP_ON_FIRST_ERROR。

示例

```
call DBMS_SCHEDULER.DROP_JOB('job', false, false, 'STOP_ON_FIRST_ERROR');
```

3.53.3.2.7　RUN_JOB 存储过程

运行一个 job 任务。

语法格式

```
DBMS_SCHEDULER.RUN_JOB(
        job_name TEXT,
        use_current_session BOOLEAN DEFAULT TRUE);
```

参数说明

- job_name 参数，job 的名字。
- use_current_session 参数，暂不支持，默认值为 true。

示例

```
call DBMS_SCHEDULER.CREATE_PROGRAM('program2', 'PLSQL_BLOCK', 'create table t2(A int)', 'user=system
dbname=test port=54321 password=123456', 'kingbase', 0, true, 'this is test program');
call DBMS_SCHEDULER.CREATE_SCHEDULE('schedule2', now(), 'Freq=daily;BYHOUR=10; BYMINUTE=10;BYSECOND=10',
NULL, 'this is test schedule');
call DBMS_SCHEDULER.CREATE_JOB('job1', 'program2', 'schedule2', 'Routine Maintenance', false, true, 'this is
test job');
call DBMS_SCHEDULER.RUN_JOB('job1', true);
```

3.53.3.2.8　DISABLE 存储过程

禁用一个 job 任务。

语法格式

```
DBMS_SCHEDULER.DISABLE(
        name TEXT,
        force BOOLEAN DEFAULT FALSE,
        commit_semantics TEXT DEFAULT 'STOP_ON_FIRST_ERROR');
```

参数说明

- name 参数，job 的名字。
- force 参数，暂不支持，默认值为 false。
- commit_semantics 参数，暂不支持，默认值为 STOP_ON_FIRST_ERROR。

示例

```
call DBMS_SCHEDULER.DISABLE('job1', false, 'STOP_ON_FIRST_ERROR');
```

3.53.3.2.9　ENABLE 存储过程

启用一个 job 任务。

语法格式

```
DBMS_SCHEDULER.ENABLE(
        name TEXT,
        commit_semantics TEXT DEFAULT 'STOP_ON_FIRST_ERROR');
```

参数说明

- name 参数，job 的名字。
- commit_semantics 参数，暂不支持，默认值为 STOP_ON_FIRST_ERROR。

示例

```
call DBMS_SCHEDULER.ENABLE('job1', 'STOP_ON_FIRST_ERROR');
```

3.53.3.3　KDB_SCHEDULE 模式

DBMS_SCHEDULER 模式提供调度和管理作业任务的功能，该模式下包含 18 个兼容视图，视图用途如下：

视图名称	描述
*_SCHEDULER_EXCEPTION	记录处于异常状态的 schedule 作业信息
*_SCHEDULER_JOB_LOG	记录 schedule 中作业的日志信息
*_SCHEDULER_JOB_RUN_DETAILS	记录 schedule 中作业的详细日志信息
*_SCHEDULER_JOBS	记录 schedule 的作业的信息
*_SCHEDULER_PROGRAMS	记录 scheduler 存储过程的信息
*_SCHEDULER_SCHEDULES	记录 scheduler 计划的信息

KDB_SCHEDULE 模式下的视图请见：

- all_scheduler_exception
- all_scheduler_job_log
- all_scheduler_jobs
- all_scheduler_programs
- all_scheduler_schedules
- dba_scheduler_exception
- dba_scheduler_job_log
- dba_scheduler_job_run_details
- dba_scheduler_jobs
- dba_scheduler_programs
- dba_scheduler_schedules
- user_scheduler_exception
- user_scheduler_job_log
- user_scheduler_job_run_details
- user_scheduler_jobs
- user_scheduler_programs
- user_scheduler_schedules

3.54 kdb_tinyint

3.54.1 插件 kdb_tinyint 简介

插件 kdb_tinyint 是初始化内置，基本类型，支持 tinyint 类型。

- 插件名为 kdb_tinyint
- 插件版本 V1.0

3.54.2 插件 kdb_tinyint 加载方式

默认已经创建，如果没有创建，需要在命令行执行 create extension kdb_tinyint;

示例：

```
create extension kdb_tinyint;
```

3.54.3 插件 kdb_tinyint 的参数配置

无需配置任何参数。

3.54.4 插件 kdb_tinyint 的使用方法

加载插件 kdb_tinyint 后，按照如下方式使用相应功能。

3.54.4.1 conversion functions support

支持 tinyint 类型和数值及字符类型之间转换。

语法

```
create cast (int4 as tinyint) with function tinyint(int4) as assignment;
create cast (tinyint as int4) with function int4(tinyint) as implicit;
create cast (bool as tinyint) with function tinyint(bool) as assignment;
create cast (tinyint as bool) with function bool(tinyint) as assignment;
create cast (int2 as tinyint) with function tinyint(int2) as assignment;
create cast (tinyint as int2) with function int2(tinyint) as implicit;
create cast (int8 as tinyint) with function tinyint(int8) as assignment;
create cast (tinyint as int8) with function int8(tinyint) as implicit;
create cast (float4 as tinyint) with function tinyint(float4) as assignment;
create cast (tinyint as float4) with function float4(tinyint) as implicit;
```

```
create cast (float8 as tinyint) with function tinyint(float8) as assignment;
create cast (tinyint as float8) with function float8(tinyint) as implicit;
create cast (numeric as tinyint) with function tinyint(numeric) as assignment;
create cast (tinyint as numeric) with function tinyinttonumeric(tinyint) as implicit;
create cast (bpchar as tinyint) with function tinyint(bpchar) as implicit;
create cast (tinyint as bpchar) with function text(tinyint) as assignment;
create cast (varchar as tinyint) with function tinyint(varchar) as implicit;
create cast (tinyint as varchar) with function text(tinyint) as assignment;
create cast (bpcharbyte as tinyint) with function tinyint(bpcharbyte) as implicit;
create cast (tinyint as bpcharbyte) with function text(tinyint) as assignment;
create cast (varcharbyte as tinyint) with function tinyint(varcharbyte) as implicit;
create cast (tinyint as varcharbyte) with function text(tinyint) as assignment;
create cast (text as tinyint) with function tinyint(text) as implicit;
create cast (tinyint as text) with function text(tinyint) as implicit;
```

示例

```
select '127'::tinyint::int2 as result;
result
--------
127

select '127'::tinyint::int4 as result;
result
--------
127

select '127'::tinyint::int8 as result;
result
--------
127
```

3.54.4.2　operator functions support

tinyint 类型和 tinyint、int2、int4、int8 之间支持算术操作符 ±/和比较操作符 =、<>、<、<=、>、>= 操作。支持 tinyint 类型和 money 之间操作。

示例

```
select tinyint '8' + int2 '5' as result;
result
--------
13
```

```
select tinyint '8' - tinyint '5' as result;
result
--------
3

select tinyint '8' * tinyint '5' as result;
result
--------
40

select tinyint '8' / tinyint '5' as result;
result
--------
1.6
```

3.54.4.3　index suppor

tinyint 类型支持 btree、bitmap、hash、gin、brin 类型索引。

示例

```
create table tinyint_ind(a tinyint);
CREATE TABLE

insert into tinyint_ind values(generate_series(-128, 127));
INSERT 0 256

create index btree_index on tinyint_ind using btree(a);
CREATE INDEX
```

3.54.4.4　mathematics functions support

tinyint 类型支持支持 mod、abs、and、or、xor、shl、shr、not、um、up 函数。

示例

```
select mod(20::tinyint,3);
mod
-----
2
```

3.54.5　插件 kdb_tinyint 卸载方法

系统启动自带插件，无法卸载。

3.54.6　插件 kdb_tinyint 升级方法

kdb_tinyint 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.55　kdb_utils_function

3.55.1　插件 kdb_utils_function 简介

kdb_utils_function 是 KingbaseES 中的一个扩展插件，能够实现工具函数的功能，如：KingbaseES 版本信息函数，若干字符串函数，若干 xml 函数以及行列转换函数等。

- 插件名为 kdb_utils_function
- 插件版本 V1.3

3.55.2　插件 kdb_utils_function 加载方式

KingbaseES 默认不加载 kdb_utils_function 扩展插件，用户可以使用如下命令手工加载。

示例：

```
create extension kdb_utils_function;
```

3.55.3　插件 kdb_utils_function 配置参数

无需配置任何参数

3.55.4　插件 kdb_utils_function 使用方法

3.55.4.1　函数列表

表 `plug-FUNCTION-TABLE` 列出了 kdb_utils_function 扩展所提供的函数。

表 3.55.1: 扩展函数列表

插件名称	用途
appendchildxml(xml,text,xml)	参考 appendchildxml
appendchildxml(xml,text,xml,text[])	参考 appendchildxml
asciistr(text)	参考 ASCIISTR
deletexml(xml,text)	参考 deletexml
deletexml(xml,text,text[])	参考 deletexml
existsnode(xml,text)	参考 existsnode
existsnode(xml,text,text[])	参考 existsnode
extractvalue(xml,text)	参考 extractvalue
extractvalue(xml,text,text[])	参考 extractvalue
get_kingbasees_version()	获取 KingbaseES 发生版本信息
insertchildxmlafter(xml,text,text,text)	参考 insertchildxmlafter
insertchildxmlafter(xml,text,text,text,text[])	参考 insertchildxmlafter
insertchildxmlafter(xml,text,text,xml)	参考 insertchildxmlafter
insertchildxmlafter(xml,text,text,xml,text[])	参考 insertchildxmlafter
insertchildxmlbefore(xml,text,text,text)	参考 insertchildxmlbefore
insertchildxmlbefore(xml,text,text,text,text[])	参考 insertchildxmlbefore
insertchildxmlbefore(xml,text,text,xml)	参考 insertchildxmlbefore
insertchildxmlbefore(xml,text,text,xml,text[])	参考 insertchildxmlbefore
insertchildxml(xml,text,text,text)	参考 insertchildxml
insertchildxml(xml,text,text,text,text[])	参考 insertchildxml
insertchildxml(xml,text,text,xml)	参考 insertchildxml
insertchildxml(xml,text,text,xml,text[])	参考 insertchildxml
insertxmlafter(xml,text,text,text)	参考 insertxmlafter
insertxmlafter(xml,text,text,text,text[])	参考 insertxmlafter

-续表-

插件名称	用途
insertxmlafter(xml,text,text,xml)	参考 insertxmlafter
insertxmlafter(xml,text,text,xml,text[])	参考 insertxmlafter
insertxmlbefore(xml,text,text,text)	参考 insertxmlbefore
insertxmlbefore(xml,text,text,text,text[])	参考 insertxmlbefore
insertxmlbefore(xml,text,text,xml)	参考 insertxmlbefore
insertxmlbefore(xml,text,text,xml,text[])	参考 insertxmlbefore
updatexml(xml,text,text)	参考 updatexml
updatexml(xml,text,text,text[])	参考 updatexml
updatexml(xml,text,xml)	参考 updatexml
lnnvl(boolean)	参考 LNNVL
group_concat(anyarray,text)	参考 GROUP_CONCAT
nlssort(text,text)	参考 nlssort
unistr(text)	参考 UNISTR
wm_concat(text)	参考聚集函数

示例

```
-- 安装 kdb_utils_function

CREATE EXTENSION kdb_utils_function;

-- 使用 get_kingbasees_version()

select get_kingbasees_version();
                  get_kingbasees_version
-----------------------------------------------------------------
KingBaseES V9.0. (c) Kingbase Corporation. All rights reserved.
(1 row)
```

3.55.5 插件 kdb_utils_function 卸载方法

```
drop extension kdb_utils_function;
```

3.55.6 插件 kdb_utils_function 升级方法

如果使用的是旧版本插件，可以升级到最新版本。

备注:

不支持跨版本升级。建议由低到高依次升级版本。

```
alter extension kdb_utils_function update to '1.5';
```

3.56 kdb_xmltype

3.56.1 插件 kdb_xmltype 简介

kdb_xmltype 是 KingbaseES 的一个扩展插件，主要用于定义 Oracle 兼容 XMLTYPE 类型和相关的操作函数。

- 插件名为 kdb_xmltype
- 插件版本 V1.1.0

3.56.2 插件 kdb_xmltype 加载方式

在使用 kdb_xmltype 之前，需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'kdb_xmltype' # (change requires restart)
```

KingbaseES 默认不加载 kdb_xmltype 扩展插件，请使用以下命令加载插件和创建扩展：

```
CREATE EXTENSION kdb_xmltype;
```

备注:

创建扩展前，需要关闭禁用多态函数参数，执行’set ora_forbid_func_polymorphism to off;’。

3.56.3　插件 kdb_xmltype 的参数配置

无需配置任何参数

3.56.4　插件 kdb_xmltype 的使用方法

3.56.4.1　XMLTYPE 类型

kdb_xmltype 插件定义了 Oracle 兼容的 XMLTYPE 类型，该类型是一个对象类型。

XMLTYPE 类型可以用于定义表和视图的列类型，也可用于定义函数或者过程的参数和返回值类型。XMLTYPE 类型与已有 xml 类型兼容，支持所有 xml 类型处理函数。XMLTYPE 类型支持 XML 格式检查，对输入的非良好格式将报错。

```
CREATE OR REPLACE TYPE xmltype AS OBJECT (
xml_data xml,
constructor function xmltype(self in out xmltype) return self as result,
constructor function xmltype(xml_string in text) return self as result
);
```

XMLTYPE 函数的典型用法举例如下：

```
CREATE TABLE tab_xml (a int, b xmltype);
INSERT INTO tab_xml VALUES (1, '<a><b>1</b><b>2</b></a>');
INSERT INTO tab_xml VALUES (2, xmltype('<a><b>3</b><c>4</c></a>'));
SELECT * FROM tab_xml;
DROP TABLE tab_xml;
```

3.56.4.1.1　输入输出

XMLTYPE 类型的构造函数，接受一个 text 类型的 XML 字符串，返回一个 XMLTYPE 类型实例。

对于不合格的 XML 字符串，xml 解析器的行为受 GUC 参数 `xmloption` 控制。当 `xmloption=content` 时（默认值），xml 解析器将不合格的输入字符串当作 XML 元素内容；当 `xmloption=document` 时，XML 解析器对不合格的输入字符串报错处理。对于空字符串或空值 NULL，均会被 XML 解析器作为 NULL 处理，生成的实例为 NULL。

3.56.4.1.2　操作符

XMLTYPE 类型不支持任何操作符，这一点与内置 xml 类型相同。

3.56.4.1.3　类型转换

XMLTYPE 类型支持与内置 xml 类型之间的双向隐式转换，支持与 text/bpchar/varchar 类型之间的双向隐式赋值转换。

3.56.4.1.4　索引

不支持在 XMLTYPE 类型列上直接创建列索引，但可以使用 xml 函数 extractvalue 创建基于函数的索引。

3.56.4.1.5　xml 函数

基于 XMLTYPE 类型与内置 xml 类型之间的双向隐式转换规则，所有接受内置 xml 类型为参数的函数，也均可接受 XMLTYPE 类型的参数。

3.56.5　XMLTYPE 成员函数

3.56.5.1　createXML 函数

创建并返回一个 XMLType 实例，用于传入数据的字符串和 CLOB 参数必须包含格式正确且有效的 XML 文档。

语法

1、通过字符串创建一个 XMLType 实例。

```
STATIC FUNCTION createXML(
   xmlData IN varchar2)
RETURN XMLType deterministic;
```

2、通过 clob 创建一个 XMLType 实例。

```
STATIC FUNCTION createXML(
   xmlData IN clob)
RETURN XMLType deterministic;
```

参数说明

- xmlData：传入的 xml 数据。目前只支持传入 clob 和 varchar2 两种。

示例

```
create table tab_xml(a int, b xmltype);

insert into tab_xml values (1, xmlType.createXML('<a><b>1</b></a>'::varchar2));
insert into tab_xml values (2, xmlType.createXML('<a><b>2</b></a>'::clob));
select * from tab_xml ;
drop table tab_xml ;
```

3.56.5.2　existsnode 函数

检查 XML 中是否存在结点。如果 XPath 为 NULL 或文档为空，则返回 0，否则返回 1。

语法

```
MEMBER FUNCTION existsNode(
   xpath IN varchar2)
RETURN number deterministic;
```

参数说明

- xpath：Xpath 表达式

示例

```
select xmltype('<a><b>1</b><c>2</c></a>').existsnode('/a/b/text()') from dual;

\set SQLTERM /
declare
  v1 xmltype;
begin
  v1 := xmltype('<a><b>1</b><c>2</c></a>');
  select v1.existsnode('/a/b/text()') from dual;
end;
/
\set SQLTERM ;
```

3.56.5.3　extract 函数

用于提取 XMLType 片段，并返回包含结果节点的 XMLType 实例。如果 XPath 未产生任何节点，则返回 NULL。

语法

```
MEMBER FUNCTION extract(xpath IN varchar2)
RETURN XMLType deterministic;
```

参数说明

- xpath：XPath 表达式

示例

```
select xmltype('<a><b>1</b><c>2</c></a>').extract('/a/b') from dual;

\set SQLTERM /
declare
    v1 xmltype;
begin
    v1 := xmltype('<a><b>1</b><c>2</c></a>');
```

```
    select v1.extract('/a/b/text()') from dual;
end;
/
\set SQLTERM ;
```

3.56.5.4　getclobval 函数

以 CLOB 的形式获取 XMLType 实例的值。

语法

```
MEMBER FUNCTION getClobVal()
RETURN clob deterministic;
```

参数说明

无

示例

```
select xmltype('<a><b>1</b><c>2</c></a>').extract('/a/b/text()').getclobval() from dual;

\set SQLTERM /
declare
    v1 xmltype;
begin
    v1 := xmltype('<a><b>1</b><c>2</c></a>');
    select v1.getclobval() from dual;
    select v1.extract('/a/b/text()').getclobval() from dual;
end;
/
\set SQLTERM ;
```

3.56.5.5　getnumberval 函数

返回一个数值，该值的格式与 XMLType 实例所指向的文本值一致。XMLType 必须是包含数值的有效文本节点。

语法

```
MEMBER FUNCTION getNumberVal()
RETURN number deterministic;
```

参数说明

无

示例

```
select xmltype('<a><b>1</b><c>2</c></a>').extract('/a/b/text()').getnumberval() from dual;

\set SQLTERM /
declare
    v1 xmltype;
begin
    v1 := xmltype('<a><b>1</b><c>2</c></a>');
    select v1.extract('/a/b/text()').getnumberval() from dual;
end;
/
\set SQLTERM ;
```

3.56.5.6　getstringval 函数

以字符串形式获取 XMLType 实例的内容。如果 XML 文档是一个文本结点，则返回文本本身。如果 XML 文档超过 VARCHAR2 最大值 (10485760)，那么运行时就会报错。

语法

```
MEMBER FUNCTION getStringVal()
RETURN varchar2 deterministic;
```

参数说明

无

示例

```
[HAVING condition]

select xmltype('<a><b>1</b><c>2</c></a>').extract('/a/b/text()').getstringval() from dual;

\set SQLTERM /
declare
    v1 xmltype;
begin
    v1 := xmltype('<a><b>1</b><c>2</c></a>');
    select v1.getstringval() from dual;
    select v1.extract('/a/b/text()').getstringval() from dual;
end;
/
\set SQLTERM ;
```

3.56.5.7　XMLSEQUENCE 函数

kdb_xmltype 插件定义了 Oracle 兼容的 XMLSEQUENCE 函数，该函数接受一个 xml 类型的 XML 实例，返回一个包含所有顶层 XML 节点的可变数组。

语法

```
XMLSEQUENCE(xml_instance xml) returns xmlsequencetype
```

参数说明

XMLSEQUENCE 函数、函数返回值的类型都位于 sys 模式下。返回值 `xmlsequencetype` 类型是 XMLTYPE 类型的可变数组。当输入数据为 NULL 时，函数返回 0 行，而不是空值。

示例

XMLSEQUENCE 函数的典型用法是和 TABLE 函数联合使用，用法举例：

以下例子展示了 XMLSEQUENCE 函数怎样将具有多个元素的 xml 文档分割为 VARRAY 单元素文档，之后再用 TABEL 函数构造了一个表对象。

```
CREATE TABLE tab_xml (a int, b xml);
INSERT INTO tab_xml VALUES (1, '<a><b>1</b><b>2</b></a>');
INSERT INTO tab_xml VALUES (2, xml ('<a><b>3</b><c>4</c></a>'));
SELECT t.* FROM tab_xml, table(xmlsequence(extract(b, '/a/*'))) AS t;
DROP TABLE tab_xml;
```

3.56.5.8　VALUE 函数

kdb_xmltype 插件定义了 Oracle 兼容的 VALUE 函数，该函数接受一个范围表别名作为参数，返回 record 类型的对象实例。对象实例的类型与表的 record 类型相同。

语法

```
CREATE OR REPLACE FUNCTION value(x record) RETURNS record
CREATE OR REPLACE FUNCTION value(x xmltype) RETURNS xmltype
```

参数说明

VALUE 函数有两个重载版本，当输入数据为空值 NULL 时，这两个版本函数均返回空值 NULL。

示例

VALUE 函数的典型用法举例：

以下例子创建了 `persons` 表和 `subjects` 表, 分别使用 VALUE 函数返回 `person_t` 对象和 `xmltype` 对象实例。

```
CREATE TYPE person_t AS OBJECT (name varchar2(100), ssn number);
CREATE TABLE persons OF person_t;
```

```
INSERT INTO persons VALUES (person_t('Bob', 1234));
INSERT INTO persons VALUES (person_t('Joe', 32456));
INSERT INTO persons VALUES (person_t('Tim', 5678));
SELECT value(t) FROM persons t;
DROP TABLE persons;
DROP TYPE person_t;

CREATE TABLE subjects OF xmltype;
INSERT INTO subjects VALUES ('<doc><lesson>math</lesson></doc>');
INSERT INTO subjects VALUES ('<doc><lesson>english</lesson></doc>');
INSERT INTO subjects VALUES ('<doc><lesson>chinese</lesson></doc>');
SELECT value(t) FROM subjects t;
DROP TABLE subjects;
```

3.56.6　插件 kdb_xmltype 卸载方法

请使用以下命令卸载 kdb_xmltype 插件。

示例：

```
DROP EXTENSION kdb_xmltype;
```

3.56.7　插件 kdb_xmltype 升级方法

kdb_xmltype 扩展插件随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.57　kingbase_fdw

3.57.1　插件 kingbase_fdw 简介

kingbase_fdw 模块提供了外部数据包装器 kingbase_fdw，它可以被用来访问存储在外部 KingbaseES 服务器中的数据。

这个模块提供的功能大体上覆盖了较老的 dblink 模块的功能。但是 kingbase_fdw 提供了更透明且更兼容标准的语法来访问远程表，并且可以在很多情况下给出更好的性能。

要使用 kingbase_fdw 来为远程访问做准备：

- 使用 CREATE EXTENSION 来安装 kingbase_fdw 扩展。
- 使用 CREATE SERVER 创建一个外部服务器对象，它用来表示你想连接的每一个远程数据库。指定除了 user 和 password 之外的连接信息作为该服务器对象的选项。

- 使用 CREATE USER MAPPING 创建一个用户映射，每一个用户映射都代表你想允许一个数据库用户访问一个外部服务器。指定远程用户名和口令作为用户映射的 user 和 password 选项。
- 为每一个你想访问的远程表使用 CREATE FOREIGN TABLE 或者 IMPORT FOREIGN SCHEMA 创建一个外部表。外部表的列必须匹配被引用的远程表。但是，如果你在外部表对象的选项中指定了正确的远程名称，你可以使用不同于远程表的表名和/或列名。

现在只需要从一个外部表 SELECT 来访问存储在它的底层的远程表中的数据。也可以使用 INSERT、UPDATE 或 DELETE 修改远程表（当然，在用户映射中已经指定的远程用户必须具有做这些事情的权限）。

注意当前 kingbase_fdw 缺少对于带 ON CONFLICT DO UPDATE 子句的 INSERT 语句的支持。不过，它支持 ON CONFLICT DO NOTHING 子句，已提供的唯一索引推断说明会被省略。另请注意 kingbase_fdw 支持 UPDATE 语句在分区表上执行的行移动调用，但是它目前不处理这样的情况：选择将移动行插入到的远程分区也是稍后将更新的 UPDATE 目标分区。

通常推荐一个外部表的列被声明为与被引用的远程表列完全相同的数据类型和排序规则（如果可用）。尽管 kingbase_fdw 目前已经能够容忍在需要时执行数据类型转换，但是当类型或排序规则不匹配时可能会发生奇怪的语义异常，因为远程服务器解释 WHERE 子句时可能会与本地服务器有所不同。

注意一个外部表可以被声明比底层的远程表较少的列，或者使用一种不同的列序。与远程表的列匹配是通过名字而不是位置进行的。

- 插件名为 kingbase_fdw
- 插件版本 V1.0.0

3.57.2　插件 kingbase_fdw 的加载方式

在 ksql 中运行：

```
CREATE EXTENSION kingbase_fdw;
```

3.57.3　插件 kingbase_fdw 的参数配置

3.57.3.1　连接选项

一个使用 kingbase_fdw 外部数据包装器的外部服务器可以使用和 libpq 在连接字符串中能接受的选项，如对象名称选项所述，不过不允许这些选项：

- user 和 password（应该在用户映射中指定这些）
- client_encoding（这是自动从本地服务器编码设置）
- fallback_application_name（总是设置为 kingbase_fdw）

只有超级用户可以在不经过口令认证的情况下连接到外部服务器，因此应总是为属于非超级用户的用户映射指定 password 选项。

3.57.3.2　对象名称选项

这些选项可以被用来控制使用在被发送到远程 KingbaseES 服务器的 SQL 语句中使用的名称。

当一个外部表被使用不同于底层远程表的名称创建时，就需要这些选项。

- schema_name

  这个选项给出用在远程服务器之上的外部表的模式名称，它可以为一个外部表指定。如果这个选项被忽略，该外部表的模式名称将被使用。

- table_name

  这个选项给出用在远程服务器上的外部表给出表名，它可以为一个外部表指定。如果这个选项被忽略，该外部表的名字将被使用。

- column_name

  这个选项给出用在远程服务器上列的列名，它可以为一个外部表的一个列指定。如果这个选项被忽略，该列的名字将被使用。

3.57.3.3　代价估计选项

kingbase_fdw 通过在远程服务器上执行查询来检索远程数据，因此理想的扫描一个外部表的估计代价应该是在远程服务器上完成它的花销，外加一些通信开销。得到这样一个估计的最可靠的方法是询问远程服务器并加上一些通信开销—但是对于简单查询，不值得为获得一个代价估计而额外使用一次远程查询。因此 kingbase_fdw 提供了下列选项来控制如何完成代价估计：

- use_remote_estimate

  这个选项控制 kingbase_fdw 是否发出 EXPLAIN 命令来获得代价估计，它可以为一个外部表或一个外部服务器指定。一个外部表的设置会覆盖它的服务器的任何设置，但是只用于这个表。默认值是 false。

- fdw_startup_cost

  这个选项是一个要被加到那个服务器上所有外部表扫描的估计启动代价的数字值。这表示建立一个连接、在远端解析和规查询的额外负荷等。默认值是 100。

- fdw_tuple_cost

  这个选项是一个数字值，它被用作表示某个服务器上外部表扫描时，每元组额外代价，它可以为某个外部服务器单独指定。表示在服务器之间数据传输的额外负荷。用户可以增加或减少这个数来反映到远程服务器更高或更低的网络延迟。默认值是 0.01。

当 use_remote_estimate 为真时，kingbase_fdw 从远程服务器获得行计数和代价估计，然后在代价估计上加上 fdw_startup_cost 和 fdw_tuple_cost。当 use_remote_estimate 为假时，kingbase_fdw 执行本地行计数和代价估计，并且接着在代价估计上加上 fdw_startup_cost 和 fdw_tuple_cost。这种本地估计不会很准确，除非有远程表统计数据的本地拷贝可用。在外部表上运行 ANALYZE 是更新本地统计数据的方法，这将执行远程表的一次扫描并接

着计算和存储统计数据，就好像表在本地一样。保留本地统计数据可能是一种有用的方法来减少一个远程表的预查询计划负荷—但是如果远程表被频繁更新，本地统计数据将很快就被废弃。

3.57.3.4　远程执行选项

默认情况下，只有使用了内建操作符和函数的 WHERE 子句才会被考虑在远程服务器上执行。

涉及非内建函数的子句将会在取完行后在本地进行检查。如果这类函数在远程服务器上可用并且可以用来产生和本地执行时一样的结果，则可以通过将这种 WHERE 子句发送到远程执行来提高性能。可以用下面的选项控制这种行为：

- extensions

  这个选项是一个用逗号分隔的已安装的 KingbaseES 扩展名称列表，这些扩展在本地和远程服务器上具有兼容的版本。属于一个该列表中扩展的 immutable 函数和操作符将被考虑转移到远程服务器上执行。这个选项只能为外部服务器指定，无法逐个表指定。

  在使用 extensions 选项时，用户应该负责确保列出的扩展在本地和远程服务器上都存在且保持一致。否则，远程查询可能失败或者行为异常。

- fetch_size

  这个选项指定在每次获取行的操作中 kingbase_fdw 应该得到的行数。可以为一个外部表或者外部服务器指定这个选项。在表上指定的选项将会覆盖在服务器级别上指定的选项。默认值为 100。

3.57.3.5　可更新性选项

默认情况下，所有使用 kingbase_fdw 的外部表都被假定是可更新的。这可以使用下列选项覆盖：

- updatable

  这个选项控制 kingbase_fdw 是否允许外部表被使用 INSERT、UPDATE 和 DELETE 命令更新。它可以为一个外部表或一个外部服务器指定。一个表级选项会覆盖一个服务器级选项。默认值是 true。

  当然，如果远程表实际上并非可更新的，将产生一个错误。这个选项的使用主要是允许在不查询远程服务器的情况下在本地抛出错误。但是要注意 information_schema 视图会根据这个选项的设置报告一个 kingbase_fdw 外部表是可更新的（或者不可更新），而不需要远程服务器的任何检查。

3.57.3.6　导入选项

kingbase_fdw 能使用 IMPORT FOREIGN SCHEMA 导入外部表定义。这个命令会在本地服务器上创建外部表定义，这个定义能匹配存在于远程服务器上的表或者视图。如果要被导入的远程表有用户自定义数据类型的列，本地服务器上也必须具有相同名称的兼容类型。

导入行为可以用下列选项自定义（在 IMPORT FOREIGN SCHEMA 命令中给出）：

- import_collate

  这个选项控制是否在从外部服务器导入的外部表定义中包括列的 COLLATE 选项。默认是 true。如果远程服务器具有和本地服务器不同的排序规则名集合，可能需要关闭这个选项，在不同的操作系统上运行时很可能就是这样。

- import_default

  这个选项控制是否在从外部服务器导入的外部表定义中包括列的 DEFAULT 表达式。默认是 false。如果启用这个选项，要当心在远程服务器和本地服务器上计算表达式的方式不同，nextval() 常会导致这类问题。如果导入的默认值表达式使用了一个本地不存在的函数或者操作符，IMPORT 将整个失败。

- import_not_null

  这个选项控制是否在从外部服务器导入的外部表定义中包括列的 NOT NULL 约束。默认是 true。

注意除 NOT NULL 之外的约束将不会从远程表中导入。虽然 KingbaseES 确实支持外部表上的 CHECK 约束，但不会自动导入它们，因为存在本地和远程服务器计算约束表达式方式不同的风险。CHECK 约束中的任何这类不一致都可能导致查询优化中很难检测的错误。因此，如果你希望导入 CHECK 约束，你必须手工来做，并且你应该仔细地验证每一个这种约束的语义。有关处理外部表上 CHECK 约束的更多细节，请见 CREATE FOREIGN TABLE。

自动排除作为其他表的分区的表或外部表。分区表被导入，除非它们是其他表的分区。由于所有数据都可以通过作为分区层次根的分区表来访问，所以这种方法应该允许访问所有数据而不创建额外的对象。

3.57.4　插件 kingbase_fdw 的使用

这里是一个用 kingbase_fdw 创建外部表的例子。首先安装该扩展：

```
CREATE EXTENSION kingbase_fdw;
```

然后使用 CREATE SERVER 创建一个外部服务器。在这个例子中我们希望连接到一个位于主机 192.83.123.89 上并且监听 5432 端口的 KingbaseES 服务器。在该远程服务器上要连接的数据库名为 foreign_db：

```
CREATE SERVER foreign_server
       FOREIGN DATA WRAPPER kingbase_fdw
       OPTIONS (host '192.83.123.89', port '5432', dbname 'foreign_db');
```

需要用 CREATE USER MAPPING 定义一个用户映射来标识在远程服务器上使用哪个角色：

```
CREATE USER MAPPING FOR local_user
       SERVER foreign_server
       OPTIONS (user 'foreign_user', password 'password');
```

现在就可以使用 CREATE FOREIGN TABLE 创建外部表了。在这个例子中我们希望访问远程服务器上名为 some_schema.some_table 的表。它的本地名称是 foreign_table：

```
CREATE FOREIGN TABLE foreign_table (
       id integer NOT NULL,
```

```
        data text
)
        SERVER foreign_server
        OPTIONS (schema_name 'some_schema', table_name 'some_table');
```

CREATE FOREIGN TABLE 中声明的列数据类型和其他性质必须要匹配实际的远程表。列名也必须匹配，不过也可以为个别列附上 column_name 选项以表示它们在远程服务器上对应哪个列。在很多情况中，要手工构造外部表定义，使用 IMPORT FOREIGN SCHEMA 会更好。

3.57.5 插件 kingbase_fdw 的卸载

在 ksql 中运行：

```
DROP EXTENSION kingbase_fdw;
```

3.57.6 插件 kingbase_fdw 的升级

kingbase_fdw 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.58 ktrack

3.58.1 插件 ktrack 简介

ktrack 是 KingbaseES 的一个扩展插件，作为块级别增量备份引擎为物理备份工具提供基础支持。

启用插件后，插件功能自动运行，并提供 SQL 函数供查询使用。

- 插件名为 ktrack
- 插件版本 V2.2

3.58.2 插件 ktrack 加载方式

确认 kingbase.conf 的 shared_preload_libraries 配置项中已包含 ktrack ，同时确认配置项 ktrack.enable 为 1（开启），ktrack.map_size（单位为 MB）为非 0 期望数值（建议为数据库大小的千分之一），重启数据库后执行 `create extension ktrack;` 语句。

```
shared_preload_libraries = 'ktrack'
ktrack.enable = 1
ktrack.map_size = 1024
```

3.58.3　插件 ktrack 的参数配置

ktrack 使用 ktrack.enable 控制功能是否开启，可配置为 0（关闭）或 1（开启），随后运行 `sys_ctl reload` 使其生效。

```
sys_ctl -D data reload
```

ktrack 使用 ktrack.map_size 设置的值设定存储块变化信息的文件的大小（单位为 MB），最大为 1024；改变该值需要重启数据库，之前存储的块变化信息会丢失（该值不变时重启数据库信息不会丢失）。

上述两个参数修改之后，如果设置了物理备份，需要执行一次全量备份。

3.58.4　插件 ktrack 的使用方法

启用 ktrack 插件后，可使用如下四个 SQL 函数做查询。

- ktrack_version()：返回 ktrack 的版本信息
- ktrack_init_lsn()：返回 ktrack 开始记录块变化信息的 LSN
- ktrack_get_pagemapset()：返回自指定 LSN 以来发生改变的块信息
- ktrack_get_change_stat()：返回自指定 LSN 以来发生改变的块信息的统计信息

```
select ktrack_version();
ktrack_version
----------------
2.2
(1 row)

select ktrack_init_lsn();
ktrack_init_lsn
-----------------
0/3000028
(1 row)

select path, pagetotal, pagecount, pagemap from ktrack_get_pagemapset('0/2D000148');
     path          | pagetotal | pagecount |                      pagemap
-------------------+-----------+-----------+-----------------------------------------------------
base/14673/2658    |        39 |         1 | \x00000000400000000000000000000000
(1 row)

select * from ktrack_get_change_stat('0/2417EBA8');
files | pages |        size, MB
-------+-------+------------------------
   1 |     1 | 0.03125000000000000000
(1 row)
```

3.58.5　插件 ktrack 卸载方法

连接到数据库执行 DROP 插件操作，删除 kingbase.conf 文件中 `shared_preload_libraries` 内的 ktrack 参数，删除 ktrack.enable 及 ktrack.map_size 设置，重启数据库。

```
drop extension ktrack;

shared_preload_libraries = ''
```

3.58.6　插件 ktrack 升级方法

ktrack 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

备注:

执行 sys_rewind 会直接修改数据文件，导致 ktrack 记录的块变化信息失去意义，因而 ktrack 在 sys_rewind 后会重新记录块变化信息；该场景下，执行块增量备份时，会自动转化为文件级增量备份。

3.59　lo

3.59.1　插件 lo 简介

在使用大对象（BLOB）数据时，JDBC/ODBC 等标准接口假设 BLOB 数据直接存储在对应表中，所以在数据被修改或删除时旧的 BLOB 数据也会被删除。而在 KingbaseES 中，使用 lo 方式存储的大对象数据是独立的对象，有独立的存储。表中的数据行通过 OID 引用 lo 数据，并且支持多行数据引用同一个 lo 数据，所以在行数据删除时系统并不会自动删除 lo 数据。所以在使用标准 JDBC/ODBC 接口时 lo 数据并不会被自动删除，导致遗留了没有引用的“孤儿”lo 数据。

lo 插件提供了触发器在引用了 lo 数据的行被修改或删除时自动调用 lo_unlink，同时提供了包装 oid domain 的 lo 数据类型。注意 lo 触发器不一定要在用 lo 数据类型的情况下才可以使用，lo 数据类型只是用来方便跟踪系统中使用 lo 触发器管理的 lo 数据。

- 插件名为 lo
- 插件版本 V1.1

3.59.2　插件 lo 加载方式

```
CREATE EXTENSION lo;
```

3.59.3　插件 lo 的参数配置

无需配置任何参数。

3.59.4　插件 lo 的使用方法

```
CREATE TABLE image (title text, raster lo);

CREATE TRIGGER t_raster BEFORE UPDATE OR DELETE on image
    FOR EACH ROW EXECUTE FUNCTION lo_manage(raster);
```

lo 触发器在 DROP TABLE 或 TRUNCATE TABLE 时不会生效；有些应用会自动创建和管理表，这种情况下触发器也不会被创建。在以上这些情况下还是可能产生“孤儿”lo。定期运行 vacuumlo 可以帮助处理这些 lo 对象。

3.59.5　插件 lo 卸载方法

```
DROP EXTENSION lo;
```

3.59.6　插件 lo 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION lo UPDATE TO '1.1';
```

3.60　ltree

3.60.1　插件 ltree 简介:

插件 ltree 是包含 ltree 和 lquery 类型的实现，为树状结构组织的数据提供索引和查询。

- 插件名为 ltree
- 插件版本 v1.1

3.60.2　插件 ltree 加载方式

使用时需要在命令行执行 create extension ltree;

示例：

```
create extension ltree;
```

3.60.3　插件 ltree 的参数配置

无需配置任何参数。

3.60.4　插件 ltree 的使用方法

加载插件 ltree 后，按照如下方式使用相应功能

3.60.4.1　ltree 类型

支持 ltree 类型

示例：

```
select ''::ltree;
ltree
-------

(1 row)

select '1'::ltree;
ltree
-------
1
(1 row)

select '1.2'::ltree;
ltree
-------
1.2
(1 row)

select '1.2._3'::ltree;
ltree
--------
```

```
1.2._3
(1 row)
```

3.60.4.2　lquery 类型

支持 lquery 类型，用于匹配 ltree 值的类似正则表达式的模式。

示例：

```
select '1'::lquery;
lquery
--------
1
(1 row)

select '4|3|2'::lquery;
lquery
--------
4|3|2
(1 row)

select '1.2'::lquery;
lquery
--------
1.2
(1 row)

select '1.4|3|2'::lquery;
lquery
---------
1.4|3|2
(1 row)

select '1.0'::lquery;
lquery
--------
1.0
(1 row)

select '4|3|2.0'::lquery;
lquery
---------
4|3|2.0
(1 row)
```

```
select '1.2.0'::lquery;
lquery
--------
1.2.0
(1 row)

select '1.4|3|2.0'::lquery;
lquery
-----------
1.4|3|2.0
(1 row)

select '1.*'::lquery;
lquery
--------
1.*
(1 row)

select '4|3|2.*'::lquery;
lquery
---------
4|3|2.*
(1 row)

select '1.2.*'::lquery;
lquery
--------
1.2.*
(1 row)

select '1.4|3|2.*'::lquery;
lquery
-----------
1.4|3|2.*
(1 row)

select '*.1.*'::lquery;
lquery
--------
*.1.*
(1 row)

select '*.4|3|2.*'::lquery;
lquery
```

```
-----------
*.4|3|2.*
(1 row)

select '*.1.2.*'::lquery;
lquery
---------
*.1.2.*
(1 row)

select '*.1.4|3|2.*'::lquery;
lquery
-------------
*.1.4|3|2.*
(1 row)

select '1.*.4|3|2'::lquery;
lquery
-----------
1.*.4|3|2
(1 row)

select '1.*.4|3|2.0'::lquery;
lquery
-------------
1.*.4|3|2.0
(1 row)

select '1.*.4|3|2.*{1,4}'::lquery;
lquery
------------------
1.*.4|3|2.*{1,4}
(1 row)

select '1.*.4|3|2.*{,4}'::lquery;
lquery
-----------------
1.*.4|3|2.*{,4}
(1 row)

select '1.*.4|3|2.*{1,}'::lquery;
lquery
-----------------
1.*.4|3|2.*{1,}
(1 row)
```

```
select '1.*.4|3|2.*{1}'::lquery;
lquery
----------------
1.*.4|3|2.*{1}
(1 row)

select 'qwerty%@*.tu'::lquery;
lquery
--------------
qwerty%@*.tu
(1 row)
```

3.60.4.3　ltree2text(ltree)

ltree 类型转换为 text 类型。

示例：

```
select ltree2text('1.2.3.34.sdf');
ltree2text
--------------
1.2.3.34.sdf
(1 row)
```

3.60.4.4　text2ltree(text)

text 类型转换为 ltree 类型。

示例：

```
select text2ltree('1.2.3.34.sdf');
text2ltree
--------------
1.2.3.34.sdf
(1 row)
```

subltree(ltree,int start,int end) ~~~

```
----------
child1
(1 row)
```

subpath(ltree,int offset,int len) ^^^

3.60.4.5　subpath(ltree,int offset)

ltree 根据偏移量 offset 寻找子路径，延伸到路径末尾。如果 offset 为负数则子路径从路径末端开始。

示例：

```
select subpath('top.child1.child2',0);
subpath
-------------------
top.child1.child2
(1 row)

select subpath('top.child1.child2',1);
subpath
---------------
child1.child2
(1 row)

select subpath('top.child1.child2',-2);
    subpath
---------------
child1.child2
(1 行记录)
```

3.60.4.6　index(ltree a,ltree b)

计算第一次在 a 中出现 b 的位置，没找到返回-1。

示例：

```
select index('1.2.3.4.5.6','1.2');
index
-------
0
(1 row)

select index('a.1.2.3.4.5.6','1.2');
index
-------
1
(1 row)

select index('a.1.2.3.4.5.6','1.2.3');
index
-------
1
```

```
(1 row)

select index('a.1.2.3.4.5.6','1.2.3.j');
index
-------
-1
(1 row)

select index('a.1.2.3.4.5.6','1.2.3.j.4.5.5.5.5.5.5');
index
-------
-1
(1 row)

select index('a.1.2.3.4.5.6','1.2.3');
index
-------
1
(1 row)

select index('a.1.2.3.4.5.6','6');
index
-------
6
(1 row)

select index('a.1.2.3.4.5.6','6.1');
index
-------
-1
(1 row)

select index('a.1.2.3.4.5.6','5.6');
index
-------
5
(1 row)

select index('0.1.2.3.5.4.5.6','5.6');
index
-------
6
(1 row)
```

index(ltree a,ltree b,int offset) ~~~

计算第一次在 a 中出现 b 的位置, 从偏移处开始搜索，负偏移量从路径末尾开始搜索，没找到返回-1。

示例：

```
select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',6);
index
-------
6
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',7);
index
-------
9
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',-7);
index
-------
6
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',-4);
index
-------
9
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',-3);
index
-------
9
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',-2);
index
-------
-1
(1 row)

select index('0.1.2.3.5.4.5.6.8.5.6.8','5.6',-20000);
index
-------
6
(1 row)
```

3.60.4.7　lca(ltree,ltree,…)

路径最长的公共前缀部分（最多支持 8 个 ltree 参数）。

示例：

```
select lca('{la.2.3,1.2.3.4.5.6,""}') is null;
?column?
----------
t
(1 row)

select lca('{la.2.3,1.2.3.4.5.6}') is null;
?column?
----------
f
(1 row)

select lca('{1.la.2.3,1.2.3.4.5.6}');
lca
-----
1
(1 row)

select lca('{1.2.3,1.2.3.4.5.6}');
lca
-----
1.2
(1 row)

select lca('{1.2.3}');
lca
-----
1.2
(1 row)

select lca('{1}'), lca('{1}') is null;
     lca | ?column?
    -----+----------
         | f
(1 row)

select lca('{}') is null;
?column?
----------
t
```

```
(1 row)

select lca('1.la.2.3','1.2.3.4.5.6');
lca
-----
1
(1 row)

select lca('1.2.3','1.2.3.4.5.6');
lca
-----
1.2
(1 row)

select lca('1.2.2.3','1.2.3.4.5.6');
lca
-----
1.2
(1 row)

select lca('1.2.2.3','1.2.3.4.5.6','');
lca
-----
(1 row)

select lca('1.2.2.3','1.2.3.4.5.6','2');
lca
-----
(1 row)

select lca('1.2.2.3','1.2.3.4.5.6','1');
lca
-----
(1 row)
```

3.60.4.8　nlevel(ltree)

计算路径中标签数量。

示例：

```
select nlevel('1.2.3.4');
nlevel
--------
```

```
4
(1 row)
```

3.60.5　插件 ltree 卸载方法

卸载时 drop extension ltree 即可。

示例：

```
drop extension ltree;
```

3.60.6　插件 ltree 升级方法

ltree 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.61　memstat

3.61.1　插件 memstat 简介

插件 memstat 是 KingbaseES 的一个扩展插件。memstat 插件主要用于在本地和所有后端中添加有关内存上下文的统计报告。

- 插件名为 memstat
- 插件版本 V1.0

3.61.2　插件 memstat 加载方式

使用该插件需要将’memstat’添加到 kingbase.conf 文件的 `shared_preload_libraries` 列表最后，且需要重启数据库。最后创建插件。

示例：

```
shared_preload_libraries = 'plsql'
-- 添加 memstat
shared_preload_libraries = 'plsql, memstat'
-- 创建插件
create extension memstat;
```

3.61.3　插件 memstat 的参数配置

该模块可以在每个查询开始执行时收集内存统计信息。如果对所有的查询都收集统计信息，在高负载的情况下，开销比较大，因此，使用 memstat.period 变量来控制收集内存统计信息的时间间隔，取值范围为 [0-60*60*24*31]。如果取值为 0，表示每个查询执行时都收集该信息。默认值为 10 秒。

备注:

memstat.period = 10 # seconds

3.61.4　插件 memstat 使用方法

3.61.4.1　local_memory_stats 函数

功能描述

打印当前后端的内存上下文统计信息

语法格式

```
local_memory_stats(
  OUT name text, OUT levels int4,
  OUT nblocks int8, OUT freechunks int8, OUT totalspace int8, OUT freespace int8)
RETURNS SETOF record
```

参数说明

参数名称	描述
name	输出指定文本
levels	输出指定等级
nblocks	输出块的个数
freechunks	输出空余块数
totalspace	输出总共空间大小
freespace	输出剩余空间大小

返回值说明：

输出当前后端的内存上下文统计信息

3.61.4.2　instance_memory_stats 函数

功能描述

打印所有后端的内存上下文统计信息，如果库是通过 `shared_preload_libraries` 预加载的，则可以正常工作。

语法格式

```
instance_memory_stats(
  OUT pid int4, OUT name text, OUT levels int4,
  OUT nblocks int8, OUT freechunks int8, OUT totalspace int8, OUT freespace int8)
RETURNS SETOF record
```

参数说明

参数名称	描述
pid	输出指定 oid
name	输出指定文本
levels	输出指定等级
nblocks	输出块的个数
freechunks	输出空余块数
totalspace	输出总共空间大小
freespace	输出剩余空间大小

返回值说明

输出所有后端的内存上下文统计信息

备注:

要使用 `instance_memory_stats` 函数，需要将 memstat 库添加到 `shared_preload_libraries`。它应该在该列表的最后！

3.61.4.3　memory_stats 视图

功能描述

打印每个后端的内存上下文统计信息

视图说明：

列名	描述
pid	输出指定 oid
totalspace	输出总共空间大小
freespace	输出剩余空间大小

3.61.5　插件 memstat 卸载方法

将 kingbase.conf 文件中 `shared_preload_libraries` 参数去掉 memstat 变量，然后重启数据库，最后删除插件接口。

示例：

```
-- 删除插件接口
drop extension memstat;
```

3.61.6　插件 memstat 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1：

```
ALTER EXTENSION memstat UPDATE TO '1.1';
```

3.62　mysql_fdw

3.62.1　插件 mysql_fdw 简介

mysql_fdw 是 KingbaseES 的一个扩展插件，它提供了一个外部数据包装器，可以方便高效的访问 MySQL 数据库。

要使用 mysql_fdw 来为远程访问做准备：

- 使用 CREATE EXTENSION 来安装 mysql_fdw 扩展。

- 使用 CREATE SERVER 创建一个外部服务器对象，它用来表示你想连接的每一个远程数据库。指定除了 user 和 password 之外的连接信息作为该服务器对象的选项。
- 使用 CREATE USER MAPPING 创建一个用户映射，每一个用户映射都代表你想允许一个数据库用户访问一个外部服务器。指定远程用户名和口令作为用户映射的 user 和 password 选项。
- 为每一个你想访问的远程表使用 CREATE FOREIGN TABLE 或者 IMPORT FOREIGN SCHEMA 创建一个外部表。外部表的列必须匹配被引用的远程表。但是, 如果你在创建外部表对象的 OPTIONS 中指定了正确的 SERVER 名称，你可以使用不同于远程表的表名或列名。

现在只需要从一个外部表使用 SELECT 语句来访问存储在它的 mysql 中的数据。也可以使用 INSERT、UPDATE 或 DELETE 修改远程表（当然，在用户映射中已经指定的远程用户必须具有做这些事情的权限）。

通常推荐一个外部表的列被声明为与被引用的远程表列完全相同的数据类型和排序规则（如果可用）。尽管 mysql_fdw 目前已经能够容忍在需要时执行数据类型转换，但是当类型或排序规则不匹配时可能会发生奇怪的语义异常，因为远程服务器解释 WHERE 子句时可能会与本地服务器有所不同。

- 插件名为 mysql_fdw
- 插件版本 V1.1

3.62.2　插件 mysql_fdw 加载方式

使用 CREATE EXTENSION mysql_fdw; 加载插件。

```
CREATE EXTENSION mysql_fdw;
```

备注:

mysql_fdw 插件不支持 loongarch、mips 和申威处理器架构平台下运行。

3.62.3　插件 mysql_fdw 的参数配置

无需配置任何参数。

3.62.4　插件 mysql_fdw 的使用方法

3.62.4.1　连接

mysql_fdw 会缓存与 MySQL 的连接，因为每个查询创建 MySQL 的成本很高。当 KingbaseES 会话结束时，所有连接都将自动关闭。

3.62.4.2　列

定义外部表时，MySQL 表的列将按定义顺序映射到 KingbaseES 外部表的列上。

mysql_fdw 将只在发送到 MySQL 查询语句中包含在 KingbaseES 查询实际中需要的那些列。

3.62.4.3　数据类型

必须使用 mysql_fdw 可以转换的数据类型定义 KingbaseES 列（参考下面的类型转换表）。如果 MySQL 值超过 KingbaseES 列的大小（如 varchar 列的长度或者最大值），将会报错。

下面这些类型之间的转换将由 mysql_fdw 自动处理：

表 3.62.1: mysql_fdw 支持的类型转换

Mysql type	Possible Kingbase types
TINYINT	int2
SMALLINT	int4
INT	int8
FLOAT	float4
DOUBLE	float8, numeric
STRING	varchar, bpchar, text, json, name,anyenum
DATE	date
TIMESTAMP	time, timestamp, timestamptz
BLOB	bytea
LONG	bit

如果一个数字被转换成布尔值，0 意味着 false，其他的都是 true。

3.62.4.4　CREATE SERVER

使用 CREATE SERVER 创建一个外部服务器对象，它用来表示你想连接的每一个远程数据库。指定除了 user 和 password 之外的连接信息作为该服务器对象的选项。

- host: MySQL 服务器的 IP 地址，默认是 127.0.0.1。
- port: MySQL 服务器的端口，默认是 3306。
- secure_auth: 启用或者禁用安全连接，默认是启用。
- init_command: 连接 MySQL 服务器所执行的 SQL 语句。
- use_remote_estimate: 控制 mysql_fdw 是否向远程发出 explain 命令来获取代价估算，默认不开启。
- reconnect: 如果发现已有连接丢失，启用或禁用 MySQL 的自动连接。默认是禁用自动连接。
- ssl_key: 客户端私钥证书的路径。
- ssl_cert: 客户端公钥证书的路径。
- ssl_ca: 所信任的 CA 证书的路径。
- ssl_capath: 所信任的 SSL CA 证书所在目录的路径名。
- ssl_cipher: SSL 加密所允许的加密列表。
- fetch_size: 该选项指每次获取操作中应该获取的行数，它可以由外表或者外部服务器来指定，表上的设置该值能够覆盖服务器上的值，默认是 100。

3.62.4.5 CREATE USER MAPPING

使用 CREATE USER MAPPING 创建一个用户到外部 mysql 服务器的一个用户映射。

OPTIONS 如下：

- username: 连接 MySQL 数据库的用户名。
- password: 对应连接用户的密码。

3.62.4.6 CREATE FOREIGN TABLE

使用 CREATE FOREIGN TABLE 创建一个用户到外部 mysql 服务器的一个用户表映射。

OPTIONS 如下：

- dbname: MySQL 数据库的数据库名。
- table_name: MySQL 数据库中的表名，默认与外表名相同。
- max_blob_size: 最大的 blob size，超过 64KB 后将被截断，取值范围为 0-64KB；
- fetch_size: 同 server 的参数，默认值 100。

3.62.4.7　WHERE 条件和 ORDER BY 子句

KingbaseES 将使用 WHERE 子句的所有适用部分作为扫描的过滤器。mysql_fdw 构造的 MySQL 查询将包含与这些筛选条件相对应的 WHERE 子句，只要这样的条件可以安全地转换为 MySQL 的 SQL 语句。这个特性，也称为 WHERE 子句的下推，可以极大地减少从 MySQL 检索到的行数，并使 MySQL 的优化器能够选择一个好的计划来访问所需的表。

3.62.4.8　外部表之间的 JOIN

mysql_fdw 可以向下推连接到 MySQL 服务器，也就是说，来自同一个 MySQL 服务器的两张外部表之间的 JOIN 被下推到同一个外部服务器，而不是返回两张表的所有行并在本地服务器执行 JOIN。

发生这种情况时有一些限制：

- 两个表必须在同一外部服务器上定义。
- 目前只包括关系操作符和算术操作符的 JOIN 连接被下推。
- JOIN 必须在 SELECT 语句中。
- mysql_fdw 必须能够下推所有 JOIN 条件和 WHERE 子句。
- 只支持内，左/右外连接，暂不支持全外，半和反连接。
- 当连接子句中含有外部表与本地表混合 JOIN 时，优先外表与外表做连接，本地表与本地表做连接。

使用 ANALYZE 收集两个外部表的表统计信息以确定最佳 JOIN 策略非常重要。

3.62.4.9　聚合函数的下推

mysql_fdw 将聚合函数下推至远端 MySQL 服务器，而不是获取所有行在本地执行聚合函数。

发生这种情况时有一些限制：

-目前下推的聚合函数只包括：max，min，sum，avg 和 count 聚合函数，从而避免了下推了 MySQL 中不支持的聚合函数。

-聚合函数中的 filters 和 order by 不会被下推。

-当聚集函数中含有其他聚集函数如：random() 不会下推

3.62.4.10　修改外部数据

mysql_fdw 支持在外部表上插入、更新和删除。

要使 INSERT/UPDATE 和 DELETE 工作，必须设置 MySQL 表第一列必须唯一。这些列用于标识外部表行，因此请确保列上都设置了该选项。

如果在 INSERT 过程中省略外表列，则该列将设置为 KingbaseES 外表的 DEFAULT 子句中定义的值（如果没有 DEFAULT 子句，则设置为 NULL）。不使用相应 MySQL 列上的默认子句。如果 KingbaseES 外部表不包含 MySQL 表的所有列，则 MySQL DEFAULT 子句将用于外部表定义中未包含的列。

当修改外部数据工作时，性能不是特别好，特别是当许多行受到影响时，因为（由于外部数据包装器的工作方式）必须单独处理每一行。

3.62.4.11　EXPLAIN

KingbaseES 的 EXPLAIN 将显示实际发出给 MySQL 的查询。EXPLAIN VERBOSE 将显示 MySQL 的执行计划。

3.62.4.12　支持 IMPORT FOREIGN SCHEMA

以下是导入外部 SCHEMA 支持的选项：

- schema_case：控制导入模式名的大小写转换

可能的值为：

- keep：保留 MySQL 的名字，通常用大写,（这是默认值）。
- lower：将所有表名和列名转换为小写。
- smart：只翻译在 Mysql 中都是大写的名称。
- import_default：用于判断是否导入默认值。

默认值是 false，即不导入 MySQL 的默认值。

- import_not_null：用于判断是否导入 NOT NULL

默认值是 false，即不导入 NOT NULL 约束。

MySQL SCHEMA 名称必须与 MySQL 中的名称完全相同。由于 KingbaseES 在处理之前将名称转换为小写，因此必须用双引号保护 SCHEMA 名称（例如“SCOTT”）。

LIMIT TO 或 EXCEPT 子句中的表名必须按照上述大小写折叠后在 KingbaseES 中出现的方式编写。

3.62.5　插件 mysql_fdw 的卸载方法

使用 DROP EXTENSION mysql_fdw; 卸载插件.

```
DROP EXTENSION mysql_fdw;
```

3.62.6　插件 mysql_fdw 升级方法

mysql_fdw 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.63　mysql_json

3.63.1　插件 mysql_json 简介

mysql_json 是 KingbaseES 的一个扩展插件，主要定义了兼容 mysql 的 json 系列操作函数。

- 插件名为 mysql_json
- 插件版本 V1.0.0

3.63.2　插件 mysql_json 加载方式

KingbaseES 默认不加载 mysql_json 扩展插件，请使用以下命令加载插件：

```
CREATE EXTENSION mysql_json;
```

注意：

创建扩展前，需要关闭禁用多态函数参数，执行’set ora_forbid_func_polymorphism to off;’。

3.63.3　插件 mysql_json 的参数配置

无需配置任何参数。

3.63.4　插件 mysql_json 的使用方法

3.63.4.1　JSON 操作符

3.63.4.1.1　->

功能描述

‘->’操作符其实是函数 JSON_EXTRACT 只有两个参数时的别名，其功能是返回 json 文档中指定 path 对应的值。JSON_EXTRACT(json_doc,path[,path]…) 函数可以有多个 path，而‘->’操作符只能指定一个 path。

语法格式

```
json-> path
```

参数

- json：json 数据。
- path：jsonpath 类型以前导 $ 字符表示；数组中的成员由 [N] 表示，和正常访问数组元素格式相同；jsonpath 中可使用 */** 通配符。

返回类型

返回值数据类型与参数 json 的数据类型一致。

示例

1. 查询 json 对象指定名称的值：

```
SELECT '{"a":"b"}'::json -> '$.a';
?column?
----------
 "b"
(1 row)
```

2. 查询 json 数组元素：

```
SELECT '{"a":"b"}'::json -> '$[0]';
?column?
------------
{"a": "b"}
(1 row)
```

3. 显式指定右类型：

```
SELECT '{"a":"b"}'::json -> '$[0]'::jsonpath;
?column?
------------
{"a": "b"}
(1 row)
```

3.63.4.1.2　-»

功能描述

‘-»’操作符是‘->’操作符的扩展，以字符串形式返回指定 json 文档中指定 path 对应的值，并取消引用。等同于将‘->’操作符所得结果以字符串形式返回。也可用 JSON_UNQUOTE(JSON_EXTRACT(json, path)) 函数表示。

语法格式

```
json ->> path
```

参数

- json：json 数据。
- path：jsonpath 类型以前导 $ 字符表示；数组中的成员由 [N] 表示，和正常访问数组元素格式相同；jsonpath 中可使用 */** 通配符。

返回类型

字符串

示例

1. 查询 json 对象指定名称的值：

```
SELECT '{"a":"b"}'::json ->> '$.a';
?column?
----------
 b
(1 row)
```

2. 查询 json 数组元素：

```
SELECT '{"a":"b"}'::json ->> '$[0]';
?column?
------------
{"a": "b"}
(1 row)
```

3. 显式指定右类型：

```
SELECT '{"a":"b"}'::json ->> '$[0]'::jsonpath;
?column?
------------
{"a": "b"}
(1 row)
```

3.63.4.2　JSON 操作函数

创建插件后提供一组 json 操作函数。

下表列出了 mysql_json 插件支持的函数并简要的表述了它们。

表 3.63.1: mysql_json 支持的类型转换

函数	简介
json_array_append	末尾添加数组元素
json_array_insert	插入数组元素
json_contains	判断是否包含某个 json 值
json_contains_path	判断某个路径下是否包含 json 值
json_depth	返回 json 文档的最大深度
json_extract	提取 json 值
json_insert	插入新值，但不替换存在的旧值
json_keys	提取 json 中的键值为 json 数组
json_length	返回 json 文档的长度
json_object	以 JSON 对象形式返回输入的键值对（可为空）
json_pretty	以更优美的格式把 json 数据打印出来
json_merge	合并两个或多个 JSON 文档并返回合并的结果。
json_merge_preserve	多个 json 进行合并，相同键名，则键值组成新的对象
json_quote	将 json 转成 json 字符串类型
json_remove	删除 json 数据
json_replace	替换值，只替换已存在的旧值
json_search	按给定字符串关键字搜索 json，返回匹配的路径
json_set	设置值（替换旧值，并插入不存在的新值）
json_type	返回 json 值的类型
json_unquote	去除 json 字符串的引号，将值转成 string 类型
json_valid	判断是否为合法 json 文档

3.63.4.2.1　json_array_append 函数

功能描述

将值附加到 JSON 文档中指定数组的末尾并返回结果。

语法格式

```
JSON_ARRAY_APPEND(json_doc, path, val[,path, val]...)
```

参数说明

参数名称	描述
json_doc	待测试的字符串，可以为列名，字符串符合 json 结构
path	指定路径，可以指定多个路径，需要符合 jsonpath 格式。注：只有指定 path 为 $/$[0] 时为有效格式。
val	待插入的值，可以为列名，给定值可以为任意类型

返回值说明

- NULL：json_doc 或 path 参数中任何一个为 NULL，则返回 NULL；
- JSON 类型 INTEGER：json_doc 为 JSON 类型 INTEGER，指定 path 不存在。
- JSON 类型 DOUBLE：json_doc 为 JSON 类型 DOUBLE，指定 path 不存在。
- JSON 类型 UNSIGNED INTEGER：json_doc 为 JSON 类型 UNSIGNED INTEGER，指定 path 不存在。
- JSON 类型 DECIMAL：json_doc 为 JSON 类型 DECIMAL，指定 path 不存在。
- JSON 类型 STRING：json_doc 为 JSON 类型 STRING，指定 path 不存在。
- JSON 类型 BOOLEAN：json_doc 为 JSON 类型 BOOLEAN，指定 path 不存在。
- JSON 类型 ARRAY：json_doc 为有效 JSON 格式，指定 path 存在。
- JSON 对象：json_doc 为 JSON 对象，指定 path 不存在。

注：如果 JSON 文档不是有效的 JSON 格式或者 path 不符合路径表达式，或者 path 中包含 * 或 ** 通配符，则返回错误。

示例

```
select json_array_append('1', '$[0]', '{"a": 1}');
```

3.63.4.2.2　json_array_insert 函数

功能描述

更新 JSON 文档，将值插入到 JSON 文档中指定数组中，并返回修改后的文档。指定多个 path 的时候，按照从左到右顺序，每插入一个值，重新计算文档结构，后续 path 对应新文档中 path。

语法格式

```
JSON_ARRAY_INSERT(json_doc, path,val[, path,val]...)
```

参数说明

参数名称	描述
json_doc	更新字符串，可以为列名, 字符串符合 json 结构
path	指定路径，可以指定多个路径；只能使用 json 数组类型 path 结构，其余方式报错。
val	待插入的值，可以为列名，给定值可以为任意类型

返回值说明

- NULL：json_doc 或 path 参数中任何一个为 NULL，则返回 NULL。
- JSON 类型 INTEGER：json_doc 为 JSON 类型 INTEGER。
- JSON 类型 DOUBLE：json_doc 为 JSON 类型 DOUBLE。
- JSON 类型 UNSIGNED INTEGER：json_doc 为 JSON 类型 UNSIGNED INTEGER。
- JSON 类型 DECIMAL：json_doc 为 JSON 类型 DECIMAL。
- JSON 类型 STRING：json_doc 为 JSON 类型 STRING。
- JSON 类型 BOOLEAN：json_doc 为 JSON 类型 BOOLEAN。
- JSON 类型 ARRAY：json_doc 为有效 JSON 格式，指定 path 存在。

注：如果 JSON 文档不是有效的 JSON 格式或者 path 不符合路径表达式，或者 path 中包含 * 或 ** 通配符，则返回错误。

示例

```
select json_array_insert('[]', '$[0]', '{"a": 1}');
```

3.63.4.2.3　json_contains 函数

功能描述

不指定路径时查找在 target 文档最外层是否包含 candidate；指定路径时查找在 target 文档的 path 路径下，是否包含 candidate。

语法格式

```
JSON_CONTAINS(target, candidate[,path])
```

参数说明

参数名称	描述
target	待检查目标字符串，可以为列名，字符串需要符合 json 结构
candidate	指定查找字符串，可以为列名，字符串需要符合 json 结构
path	target 中路径或 NULL。

注：按照入参顺序检查参数格式，当参数为 NULL 时，不检查后续格式，直接返回 NULL；但 target 和 candidate 不符合 json 格式时，返回错误。

返回值说明

- NULL：target 或 candidate 为 NULL，或 target 中指定的 path 路径不存在
- boolean：target 中包含 candidate 返回 true，否则返回 false

注：如果 target 或 candidate 不是有效 JSON 格式，则返回错误。

示例

```
select json_contains('"abc"', '"abc"', '$[0]');
```

3.63.4.2.4　json_contains_path 函数

功能描述

返回 JSON 文档中是否包含指定 path。

语法格式

```
JSON_CONTAINS_PATH(json_doc, one_or_all, path[, path] ...)
```

参数说明

参数名称	描述
json_doc	待测试的字符串，可以为列名，字符串符合 json 结构。
one_or_all	One: 如果文档中至少存在一个路径返回 true，否则返回 false；all: 如果所有路径都存在于文档中返回 true，否则返回 false。
path	至少指定一个 path，可以指定多个 path。path 中可以使用 */**通配符，* 代表对象节点**，代表数组。

返回值说明

- NULL：json_doc 或 path 为 NULL;
- boolean：
  - one：如果文档中至少存在一个路径返回 true，否则返回 false
  - all：如果所有路径都存在于文档中返回 true，否则返回 false。

注：如果参数不是有效 JSON 格式，则返回错误。

示例

```
select json_contains_path('[1,2,[3,4,5,6]]', 'all', NULL);
```

3.63.4.2.5　json_depth 函数

功能描述

返回 JSON 文档的最大深度。

- 非空数组/非空 json 对象：非最底层，每加一层嵌套，深度加一。
- 当最底层为空数组/空对象深度累加 1；
- 非空数组/非空对象深度累加 2；
- 当 json 文档有多个分支时，最大深度分支作为 json 文档深度；

语法格式

```
JSON_DEPTH(json_doc)
```

参数说明

参数名称	描述
json_doc	待测试的字符串，可以为列名。字符串需符合 json 结构，json_doc 字符串最大长度 1G

返回值说明

- NULL：json_doc 为 NULL;
- Integer：
  - JSON_DOC 为有效的 json 格式, 返回 json_doc 最大深度；
  - 空数组/空对象/标量均返回 1；

注：如果参数不是有效 JSON 格式，则返回错误。

示例

```
select json_depth('{"a": 1, "b": 2}');
```

3.63.4.2.6　json_extract 函数

功能描述

返回 JSON 指定 path 对应的值。

语法格式

```
JSON_EXTRACT(json_doc,  path[, path] ...)
```

参数说明

参数名称	描述
json_doc	待测试的字符串，可以为列名，字符串符合 json 结构。json_doc 的长度最长不能超过 1G。
path	指定的路径，可以指定一个，也可以指定多个。

返回值说明

- NULL：json_doc 或任何指定 path 为 NULL；或者指定的 path 全部不存在
- JSON 类型 INTEGER：指定 path 对应值为 integer 类型
- JSON 类型 DOUBLE：指定 path 对应值为 double 类型
- JSON 类型 UNSIGNED INTEGER：指定 path 对应值为 unsigned integer 类型
- JSON 类型 DECIMAL：指定 path 对应值为 decimal 类型
- JSON 类型 STRING：指定 path 对应值为 string 类型
- JSON 类型 BOOLEAN：指定 path 对应值为 boolean 类型
- JSON 类型 ARRAY：指定 path 对应值为 array 类型；或者指定多个 path 返回多个结果，使用 array 返回。
- JSON：指定 path 对应值为 json 类型

示例

```
select json_extract('"1.0"', '$**[0]');
```

3.63.4.2.7　json_insert 函数

功能描述

更新 JSON 文档，将其插入文档中的数组中并返回修改后的文档。

语法格式

```
JSON_INSERT(json_doc, path,val[,path, val]...)
```

参数说明

参数名称	描述
json_doc	待变更的 Json 文档；
path	更新 JSON 文档路径;
val	更新路径对应的值;

返回值说明

- NULL：json_doc 参数或任何指定 path 为 NULL，返回 NULL；当指定 path 为根目录 ($), 并且指定值为 NULL，返回 NULL；
- json 基本类型/标量：更新结果为 json 基本类型/标量，则返回 json 基本类型/标量；

- json 类型数组：更新结果为 json 数组，则返回 json 基本 json 数组；
- json 对象；更新结果为 json 对象，则返回 json 基本 json 对象；

注：如果 json_doc 参数不是有效的 JSON 文档，或者任何 path 参数不是有效的 path 表达式，或者包含 * 或 ** 通配符，则会发生错误。

示例

```
select json_insert('[]', '$[last]', 1);
```

3.63.4.2.8　json_keys 函数

功能描述

更新 JSON 文档，将其插入文档中的数组中并返回修改后的文档。

语法格式

```
JSON_KEYS(json_doc, path,val[,path, val]...)
```

参数说明

参数名称	描述
json_doc	待变更的 Json 文档;
path	更新 JSON 文档路径;
val	更新路径对应的值;

返回值说明

- NULL：json_doc 参数或任何指定 path 为 NULL，返回 NULL；当指定 path 为根目录 ($), 并且指定值为 NULL，返回 NULL；
- json 基本类型/标量：更新结果为 json 基本类型/标量，则返回 json 基本类型/标量；
- json 类型数组：更新结果为 json 数组，则返回 json 基本 json 数组；
- json 对象；更新结果为 json 对象，则返回 json 基本 json 对象；

注：如果 json_doc 参数不是有效的 JSON 文档，或者任何 path 参数不是有效的 path 表达式，或者包含 * 或 ** 通配符，则会发生错误。

示例

```
select json_keys('[]', '$[last]', 1);
```

3.63.4.2.9　json_length 函数

功能描述

返回 json 文档的长度，如果不给定 path，返回最上层 Key 个数；如果给定 path，返回指定 path 层 Key 个数。

- 标量：长度为 1；
- 数组：长度为数组元素的数量；
- JSON 对象：长度为 JSON 的成员数量。

语法格式

```
JSON_LENGTH(json_doc[,path])
```

参数说明

参数名称	描述
json_doc	待测试的字符串，可以为列名；字符串符合 json 结构。json_doc 字符串最大长度 1G
path	指定的路径

返回值说明

- NULL：JSON_DOC 为空，或 path 指定路径不存在
- Integer：如果不给定 path，返回 json_doc 文档长度。

注：如果参数不是有效 JSON 格式或给定 path 格式错误，则返回错误。

示例

```
select json_length('["1", {"a": 1}, true, null, false]');
```

3.63.4.2.10　json_object 函数

功能描述

以 JSON 对象形式返回输入的键值对（可为空）。当任意键名为 NULL 或者参数个数为单数时报错。

语法格式

```
JSON_OBJECT([key1,value1[,key2,value2,…])
```

参数说明

参数名称	描述
key1	对象中的键，必输。
value1	对象中的键的值，必输。
[,key2,value2,…]	多个键值对，可选。

返回值说明

返回类型为包含所有键值对的 JSON 对象。由于 JSON 对象中的所有键均为字符串，因此 JSON_OBJECT() 会将非字符串类型的 key 转为字符串类型。

示例

```
SELECT JSON_OBJECT('id', 87, 'name', 'carrot');
+----------------------------------------+
| JSON_OBJECT('id', 87, 'name', 'carrot') |
+----------------------------------------+
| {"id": 87, "name": "carrot"}            |
+----------------------------------------+
```

3.63.4.2.11　json_pretty 函数

功能描述

以更优美的格式把 json 数据打印出来。

语法格式

```
JSON_PRETTY(json_val)
```

参数说明

json 类型或以更优美的格式把 json 数据打印出来。者可以转换为 json 的字符串。

返回值说明

字符串以更优美的格式把 json 数据打印出来。

示例

```
SELECT json_pretty('[1,2,3, [1]]');
 json_pretty
-------------
 [          +
     1,     +
     2,     +
     3,     +
     [      +
         1  +
     ]      +
 ]
(1 row)
```

3.63.4.2.12　json_merge 函数

功能描述

合并两个或多个 JSON 文档并返回合并的结果。

- 若任何参数为 NULL，则返回 NULL。
- 若任何参数不是有效的 JSON 文档，则会发生错误。
- 若任何参数为 NULL，或者指定的 path 不存在，则返回 NULL；
- 若参数不是有效 JSON 格式，则返回错误。

语法格式

```
JSON_MERGE(json_doc,json_doc[,json_doc]...)
```

参数说明

参数名称	描述
json_doc	一个 JSON 文档，必输。

返回值说明

- NULL: 如果任何参数为 NULL，则返回 NULL；
- JSON 类型数组：如果多个 json_doc 都不是 json 对象，返回 json 类型数组
- JSON 对象：所有 json_doc 都是 json 对象，返回 json 对象。

示例

```
SELECT json_merge('1', '{"a":1}', '{"a":2}');
       json_merge
-------------------------
 [1, {"a": 1}, {"a": 2}]
(1 row)
```

3.63.4.2.13　json_merge_preserve 函数

功能描述

合并两个或多个 JSON 文档并返回合并的结果。

语法格式

```
JSON_MERGE_PRESERVE(json_doc, json_doc[,json_doc]...)
```

参数说明

参数名称	描述
json_doc	合并字符串，可以为列名；字符串符合 json 结构

返回值说明

- NULL：如果任何参数为 NULL，或者指定的 path 不存在，则返回 NULL；
- JSON 类型数组：如果多个 json_doc 都不是 json 对象，返回 json 类型数组
- JSON 对象：所有 json_doc 都是 json 对象，返回 json 对象。

注：如果任何参数不是有效 JSON 格式，则返回错误。

示例

```
select json_merge_preserve('1', '[1,2,3]');
```

3.63.4.2.14　json_quote 函数

功能描述

将字符串用双引号引起来并转义内部引号和其他字符来将字符串转换为 JSON 值，然后将结果作为 JSON 格式字符串返回。

语法格式

```
JSON_QUOTE(string)
```

参数说明

参数名称	描述
strng	待转换字符串，可为字符串，NULL

返回值说明

- NULL：如果 string 为 NULL，则返回 NULL；
- JSON 类型 string：如果输入为字符串，则返回 json 格式字符串。

示例

```
select json_quote('true');
```

3.63.4.2.15　json_remove 函数

功能描述

从 JSON 文档中删除数据并返回结果。

语法格式

```
JSON_REMOVE(json_doc,path[, path]...)
```

参数说明

参数名称	描述
json_doc	要查询的 Json 文档；
path	指定删除路径，可以指定多个路径。

返回值说明

- NULL：json_doc，path 参数中任何一个为 NULL，则返回 NULL;
- json 基本类型/标量：更新结果为 json 基本类型/标量，则返回 json 基本类型/标量；
- json 类型数组：更新结果为 json 数组，则返回 json 基本 json 数组；

- json 对象；更新结果为 json 对象，则返回 json 基本 json 对象；

注：如果 json_doc 参数不是有效的 JSON 文档，或者任何 path 参数不是有效的 path 表达式，或者是 $ 或包含 * 或 ** 通配符，则会发生错误。

示例

```
select json_remove('12345', '$[0]', '$[1]');
```

3.63.4.2.16　json_replace 函数

功能描述

替换 JSON 文档中的现有值并返回结果。

语法格式

```
JSON_REPLACE(json_doc, path,val[,path, val]...)
```

参数说明

参数名称	描述
json_doc	要更新的 Json 文档；
path	更新 JSON 文档路径；
val	更新路径对应的值，必须符合 json 格式；

返回值说明

- NULL：json_doc 参数或任何指定 path 为 NULL，返回 NULL；当指定 path 为根目录 ($)，并且指定值为 NULL，返回 NULL；
- json 基本类型/标量：更新结果为 json 基本类型/标量，则返回 json 基本类型/标量；
- json 类型数组：更新结果为 json 数组，则返回 json 基本 json 数组；
- json 对象；更新结果为 json 对象，则返回 json 基本 json 对象；

注：如果 json_doc 参数不是有效的 JSON 文档或任何 path 参数不是有效的 path 表达式或包含 * 或 ** 通配符，则会发生错误。

示例

```
select json_replace('123', '$.a', 1);
```

3.63.4.2.17　json_search 函数

功能描述

返回符合查询条件的 key 对应的 JSON 路径所组成的数组，若不存在，则返回 NULL。

语法格式

```
JSON_SEARCH(json_doc, one_or_all, search_str[,escape_char[, path]...])
```

参数说明

参数名称	描述
json_doc	要查询的 Json 文档；
one_or_all	查询的终止条件 one：搜索在第一个匹配项后终止，并返回一个路径字符串。all：搜索返回所有匹配的路径字符串，因此不包括重复的路径。如果有多个字符串，它们将自动包装为一个数组。数组元素的顺序是不确定的.
search_str	搜索参数，在 search_str 中，通配符% 和 _ 可以如同在 LIKE 上一样运行，其中% 用于匹配多个字符（包括 0 个），_ 则仅可匹配一个字符；
escape_char	指定转义符，escape_char 参数指定时要求必须是常量（为空或者一个字符），当 escape_char 参数为 NULL 或者不指定的情况下，系统默认使用作为转义字符；
path	指定检索路径，可以指定多个路径

返回值说明

- NULL：json_doc，search_str，或 path 参数中任何一个为 NULL，则返回 NULL; 文档中不存在 path, 返回 NULL;search_str 未找到, 返回 NULL;
- json 类型 ARRAY：返回多个 key 列表；
- json 类型 String：返回单个 key；

示例

```
select json_search('"abc"', 'all', 'abc');
```

3.63.4.2.18　json_set 函数

功能描述

在 JSON 文档中插入或更新数据并返回结果。

语法格式

```
JSON_SET(json_doc, path,val[,path, val]...)
```

参数说明

参数名称	描述
json_doc	要查询的 Json 文档；
path	更新 JSON 文档路径
val	更新路径对应的值

返回值说明

- NULL：json_doc 参数或任何指定 path 为 NULL，返回 NULL；当指定 path 为根目录 ($), 并且指定值为 NULL，返回 NULL；
- json 基本类型/标量：更新结果为 json 基本类型/标量，则返回 json 基本类型/标量；
- json 类型数组：更新结果为 json 数组，则返回 json 基本 json 数组；
- json 对象；更新结果为 json 对象，则返回 json 基本 json 对象；

注：如果 json_doc 参数不是有效的 JSON 文档，或者任何 path 参数不是有效的 path 表达式或包含 * 或 ** 通配符，则会发生错误。

示例

```
select json_set('123', '$[1]',1);
select json_set('[]', '$[0]', json_object('a', 1));
select json_set('[1,2,3,4]', '$[last]', 1);
select json_set('{}', '$[last]', 1);

\set SQLTERM /
DECLARE
js json := '{"a":{"b":1}, "c":2}';
BEGIN
  select json_set(js, '$.d', json_array(1,2,3), '$.a.c', json_array(1,2,3));
END;
/
```

3.63.4.2.19　json_type 函数

功能描述

返回 JSON 值类型

语法格式

```
JSON_TYPE(json_val)
```

参数说明

参数名称	描述
json_val	待测试的字符串，可以为列名；字符串符合 json 结构；

返回值说明

- JSON 类型：OBJECT，ARRAY，BOOLEAN，NULL
- 数字类型：INTEGER，DOUBLE，DECIMAL
- 时间类型：DATETIME，DATE，TIME
- 字符串类型：STRING
- 二进制类型：BLOB，BIT
- 其他类型：OPAQUE

注：如果参数不符合 json 结构，则返回错误。

示例

```
select json_type('1234567890123456789012345678 90');
select json_type('"a"');
select json_type('[]');
select json_type('[1,"a", true, null]');
```

3.63.4.2.20　json_unquote 函数

功能描述

取消引用 JSON 值，并以字符串的形式返回。

语法格式

```
JSON_unquote(json_value)
```

参数说明

参数名称	描述
json_value	待测试的字符串，可以为列名；字符串符合 json 结构。

返回值说明

- NULL：如果 json_value 为 NULL，则返回 NULL
- string 类型字符串：如果 json_doc 为有效的 json 格式，则返回转换之后的字符串。

注：如果该值以双引号开头和结尾，但不是有效的 JSON 字符串，则返回错误。

示例

```
select json_unquote('"a"');
select json_unquote("a");
select json_unquote('["a", "b", "c"]');
select json_unquote('[{"a": 1}]');
select json_unquote('{"a": 1, "b": {}}');
```

3.63.4.2.21　json_valid 函数

功能描述

判断 val 是否有良好的 json 结构。

语法格式

```
JSON_VALID(val)
```

参数说明

参数名称	描述
val	待测试的字符串，可以为列名，数据类型可以为字符串、数字或空值。

返回值说明

- boolean 类型：合法 json 格式返回 true，非法 json 格式返回 false，
- NULL：参数为 NULL 则返回 NULL

示例

```
select json_valid('1');
select json_valid('"aa"');
```

3.63.5　插件 mysql_json 的卸载方法

通过 drop extension 来卸载插件。

```
DROP EXTENSION mysql_json;
```

3.63.6　插件 mysql_json 升级方法

mysql_json 扩展插件随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.64　oracle_fdw

3.64.1　插件 oracle_fdw 简介

oracle_fdw 是 KingbaseES 的一个扩展插件，它提供了一个外部数据包装器，可以方便高效的访问 oracle 数据库，包括 WHERE 条件和所需列的下推以及全面的 EXPLAIN 支持。

3.64.2　插件 oracle_fdw 的加载方式

在 ksql 中运行：

```
create extension oracle_fdw;
```

备注:

1. oracle_fdw 需要依赖 GLIBC。2.9 及以上的版本才能运行。
2. 如果用户是在 widnows 下运行的 KingbaseES，需要安装 oracle 的客户端库，并且把安装后的目录添加到 PATH 环境变量下。
3. oracle_fdw 插件不支持 loongarch、mips 和申威处理器架构平台下运行。

3.64.3 插件 oracle_fdw 的参数配置

3.64.3.1 Foreign data wrapper 参数

- nls_lang (可选)

将 Oracle 的 NLS_LANG 环境变量设置为此值。NLS_LANG 的格式为"language_territory.charset"(例如 AMERICAN_AMERICA.AL32UTF8)。这必须与你的数据库编码匹配。如果未设置此值，oracle_fdw 将自动做正确的事情，如果他不能，则发出警告。仅当你知道自己在做什么时才设置此选项。

3.64.3.2 Foreign server 参数

- dbserver (必填)

远程数据库的 Oracle 数据库连接字符串. 这个可以是 Oracle 支持的任何形式，只要你的 Oracle 客户端进行了相应的配置。将此设置为本地 (“BEQUEATH”) 连接的空字符串

- isolation_level (可选，默认值是 serializable)

在 Oracle 数据库中使用的事务隔离级别。该值可以是 serializable, read_committed, read_only

请注意，单个 KingbaseES 语句期间可以多次查询 Oracle 表 (例如，在一个 nested loop join 期间)。为了确保不会出现由并发事务的竞争条件引起的不一致，事务隔离级别必须保证事务读取稳定性。只有 Oracle 的 SERIALIZABLE 或者 READ ONLY 隔离级别才能保证这一点

但 oracle 的 SERIALIZABLE 实现的相当糟糕，并在意外情况下导致序列化错误 (ORA-08177)，例如插入到表中。使用 READ COMMITTED 事务可以解决此问题，但存在不一致的风险。如果你想使用它，请检查你的执行计划是否可以多次执行外部扫描。

- nchar (boolean 类型，可选，默认值是 off)

将此选项设置为 on 会在 Oracle 端选择跟昂贵的字符转换。如果你使用单字节 Oracle 数据库字符集，但 NCHAR 或 NVARCHAR2 列包含数据库字符集中无法表示的字符，则这是必填的

将 nchar 设置为 on 会对性能产生显著的影响，并且会在 Update 语句中导致 ORA-01461 错误，当该语句设置的字符串超过 2000 字节时 (如果你又 MAX_STRING_SIZE=EXTENDED，则为 16383)。此错误似乎是 Oracle 的错误。

3.64.3.3 User mapping 参数

- user (必填)

会话的 Oracle 用户名

如果你不想在 KingbaseES 数据库中存储 Oracle 凭据 (一种简单的方法是使用外部密码存储)，请将其设置为外部身份验证的空字符串

- password (必填)

Oracle 用户的密码

3.64.3.4 Foreign table 参数

- table (必填)

Oracle 表名。此名称必须与 Oracle 系统目录中出现的完全相同，因此通常只包含大写字母

要基于任意 Oracle 查询定义外部表，请将此选项设置为括号中的查询，例如

```
OPTIONS (table '(SELECT col FROM tab WHERE val = ''string'')')
```

在这汇总情况下不要设置 scheam 选项 INSERT、UPDATE 和 DELETE 将作用于简单查询中定义的外部表；如果你想避免这种情况 (或对更复杂的查询造成混淆 Oracle 错误消息)，请使用 table 选项 readonly

- dblink (可选)

Oracle database link 的名字。这个名称必须与 Oracle 系统目录中出现的完全相同，因此通常只包含大写字母

- schema (可选)

表的 scheam。对应访问不属于连接 Oracle 用户的表很有用。此名称必须要 Oracle 系统目录中出现的完全相同，因此通常只包含大写字母

- max_long (可选，默认值是 32767)

Oracle 表中 LONG, LONG RAW, XMLTYPE 类型的列的最大长度。可能的值是 1 到 1073741823 之间的整数 (kingbase 中一个字节的最大大小)。此内存量至少分配两次，因此大量的值将消耗大量内存

- readonly (可选，默认值是 false)

INSERT, UPDATE, DELETE 只允许操作这个选项没有被设置成 yes/on/true 的表

- sample_percent (可选，默认值是”100”)

此选项仅影响 ANALYZE 处理，可用于在合理的时间内 ANALYZE 非常大的表

该值必须介于 0.000001 和 100 之间，并定义将随机选择以计算 KingbaseES 表统计信息的 Oracle 表块的百分比。这是使用 Oracle 中 SAMPLE BLOCK(x) 子句完成的

对于使用 Oracle 查询定义的表，ANALYZE 将失败并出现 ORA-00933，而对于使用复杂 Oracle 视图定义的表，ANALYZE 可能会失败并出现 ORA-01446

- prefetch (可选，默认值是 200)

设置在外部表扫描期间通过 kingbase 与 Oracle 之间单次往返获取的行数。这个是使用 Oracle 行预取实现的。该值必须介于 0 和 10240 之间，其中零值禁用预取

较高的值可以提高性能，但是会在 kingbase 服务器上使用更多的内存

请注意，由于 Oracle 的限制，如果 Oracle 查询涉及 BLOB, CLOB 或 BFILE 列，行预取将不起作用。因此，如果你获取许多行，对外部表上此类列的查询将执行的很差

3.64.3.5　列参数

- key (可选，默认值是 false)

如果被设置成了 yes/on/true，Oracle 表中对应的列被认为是主键列。为了让 UPDATE, DELETE 工作，必须将所有的主键列都设置上这个选项

- strip_zero (可选，默认值是 false)

如果设置为 yes/on/true，ASCII 0 字符将在传输过程中从字符串中删除。此类字符在 Oracle 中有效，但是在 KingbaseES 中无效，因此再被 oracle_fdw 读取时会导致错误。此选项仅针对 character，character varying 和 text 类型的列

3.64.4　插件 oracle_fdw 的使用方法

3.64.4.1　Oracle 权限

Oracle 用户需要 CREATE SESSION 权限和对指定表或视图的 SELECT 权限。

对于 EXPLAIN VERBOSE，用户还需要对 V*SQLV*SQL_PLAN 的 SELECT 权限。

3.64.4.2　连接

oracle_fdw 会缓存与 oracle 的连接，因为每个查询创建 oracle 的成本很高。当 KingbaseES 会话结束时，所有连接都将自动关闭。

函数 oraclr_colse_connections() 可用于关闭所有缓存的 oracle 连接。这对于长时间运行的不经常访问外部表的会话非常有用。可以避免阻塞并开放 oracle 所需的资源。对于正在修改 oracle 数据的事务中不能使用此函数。

3.64.4.3　列

定义外部表时，oracle 表的列将按定义顺序映射到 kingbase 列。

oracle_fdw 将只在 oracle 查询中包含 kingbase 查询实际需要的那些列。

kingbase 表可以比 oracle 表有更多或更少的列。如果有更多的列，并且使用了这些列，将会收到警告并返回空值。

如果要更新或删除，请确保在属于表主键的所有列上都设置了 key 选项，否则将导致错误。

3.64.4.4　数据类型

必须使用 oracle_fdw 可以转换的数据类型定义 kingbase 列（参考下面的类型转换表）。此限制仅在实际使用列时才强制执行，因此只要不访问不可转换的数据类型，就可以为它们定义伪列（此方法仅适用于 SELECT，而不适用于修改外部数据）。如果 oracle 值超过 kingbase 列的大小（如 varchar 列的长度或者最大值），将会报错。

下面这些类型之间的转换将由 oracle_fdw 自动处理：

表 3.64.1: oracle_fdw 支持的类型转换

Oracle type	Possible Kingbase types
CHAR	char, varchar, text
NCHAR	char, varchar, text
VARCHAR	char, varchar, text
VARCHAR2	char, varchar, text, json
NVARCHAR2	char, varchar, text
CLOB	char, varchar, text, json
LONG	char, varchar, text
RAW	uuid, bytea
BLOB	bytea
BFILE	bytea (read-only)
LONG RAW	bytea
NUMBER	numeric, float4, float8, char, varchar, text
NUMBER(n,m) with m<=0	numeric, float4, float8, int2, int4, int8, boolean, char, varchar,text
FLOAT	numeric, float4, float8, char, varchar, text
BINARY_FLOAT	numeric, float4, float8, char, varchar, text
BINARY_DOUBLE	numeric, float4, float8, char, varchar, text
DATE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH TIME ZONE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH LOCAL TIME ZONE	date, timestamp, timestamptz, char, varchar, text

-续表-

Oracle type	Possible Kingbase types
INTERVAL YEAR TO MONTH	interval, char, varchar, text
INTERVAL DAY TO SECOND	interval, char, varchar, text
XMLTYPE	xml, char, varchar, text
MDSYS.SDO_GEOMETRY	geometry

如果一个数字被转换成布尔值，0 意味着 false，其他的都是 true。

插入或更新 XMLTYPE 只适用于不超过 VARCHAR2 数据类型的最大长度（4000 或 32767，取决于 MAX_STRING_SIZE 参数）的值。

oracle_fdw 目前不支持 NCLOB，因为 Oracle 无法自动将其转换为客户端编码。

如果需要的转换的类型不在上述列表中，请在 Oracle 或 KingbaseES 中定义适当的视图。

3.64.4.5　WHERE 条件和 ORDER BY 子句

kingbase 将使用 WHERE 子句的所有适用部分作为扫描的过滤器。Oracle_fdw 构造的 Oracle 查询将包含与这些筛选条件相对应的 WHERE 子句，只要这样的条件可以安全地转换为 Oracle SQL。这个特性，也称为 WHERE 子句的下推，可以极大地减少从 Oracle 检索到的行数，并使 Oracle 的优化器能够选择一个好的计划来访问所需的表。

同样，ORDERBY 子句将尽可能下推到 Oracle。请注意，按字符串排序的 ORDER-BY 条件不会被下推，因为 KingbaseES 和 Oracle 中的排序顺序不能保证相同。

要利用它，请尝试对外部表使用简单的条件。选择与 Oracle 类型相对应的 kingbase 列数据类型，否则条件将无法转换。

表达式 now()、transaction_timestamp()、current_timestamp、current_date 和 localtimestamp 将被正确翻译。

EXPLAIN 的输出将显示所使用的 Oracle 查询，因此可以看到哪些条件被转换为 Oracle 以及如何转换。

3.64.4.6　外部表之间的 JOIN

oracle_fdw 可以向下推连接到 oracle 服务器，也就是说，两个外部表之间的连接将导致在 oracle 端执行连接的单个 oracle 查询。

发生这种情况时有一些限制：

- 两个表必须在同一外部服务器上定义。

- 三个或更多表之间的 JOIN 不会被向下推。
- JOIN 接必须在 SELECT 语句中。
- oracle_fdw 必须能够下推所有 JOIN 条件和 WHERE 子句。
- 没有 JOIN 条件的交叉 JOIN 不会向下推。
- 如果一个 JOIN 被下推，ORDER BY 子句将不会被下推。

使用 ANALYZE 收集两个外部表的表统计信息以确定最佳 JOIN 策略非常重要。

3.64.4.7　修改外部数据

oracle_fdw 支持在外部表上插入、更新和删除。这在默认情况下是允许的，并且可以通过设置 readonly table 选项来禁用。

要使 UPDATE 和 DELETE 工作，必须设置与 Oracle 表的主键列对应的列的 key column 选项。这些列用于标识外部表行，因此请确保在属于主键的所有列上都设置了该选项。

如果在 INSERT 过程中省略外表列，则该列将设置为 kingbase 外表的 DEFAULT 子句中定义的值（如果没有 DEFAULT 子句，则设置为 NULL）。不使用相应 Oracle 列上的默认子句。如果 kingbase 外部表不包含 Oracle 表的所有列，则 Oracle DEFAULT 子句将用于外部表定义中未包含的列。

INSERT、UPDATE 和 DELETE 上的 RETURNING 子句受支持，但 Oracle 数据类型为 LONG 和 LONG RAW 的列除外（RETURNING 子句中 Oracle 不支持这些数据类型）。

用 AFTER 和 FOR 为每行定义的触发器要求外表中没有 Oracle 数据类型为 LONG 或 LONG RAW 的列。这是因为这样的触发器使用了上面提到的 RETURNING 子句。

当修改外部数据工作时，性能不是特别好，特别是当许多行受到影响时，因为（由于外部数据包装器的工作方式）必须单独处理每一行。

事务被转发到 Oracle，因此 BEGIN、COMMIT、ROLLBACK 和 SAVEPOINT 按预期工作。不支持涉及 Oracle 的准备语句。有关详细信息，请参见“内部构件”一节。

由于 oracle_fdw 默认情况下使用序列化事务，因此数据修改语句可能导致序列化失败：

ORA-08177: can’t serialize access for this transaction

如果并发事务修改了表，并且在长时间运行的事务中更可能发生这种情况。这些错误可以通过它们的 SQLSTATE（40001）来识别。使用 oracle_fdw 的应用程序应该重试由于此错误而失败的事务。

可以使用不同的事务隔离级别，有关讨论，请参阅外部服务器选项。

3.64.4.8　EXPLAIN

kingbase 的 EXPLAIN 将显示实际发出给 Oracle 的查询。EXPLAIN VERBOSE 将显示 Oracle 的执行计划（该计划不适用于 Oracle server 9i 或更早版本）。

3.64.4.9　ANALYZE

可以使用 ANALYZE 收集外部表的统计信息。这得到了 oracle_fdw 的支持。

如果没有统计信息，kingbase 就无法估计外部表上查询的行数，这可能会导致选择错误的执行计划。

kingbase 不会像对普通表那样使用 autovacuum 守护进程自动收集外部表的统计信息，因此在创建外部表之后以及远程表发生重大更改时，对外部表运行 ANALYZE 尤为重要。

分析 Oracle 外部表将导致完整的顺序表扫描。可以使用 table 选项 sample_percent，通过只使用 Oracle 表的一个示例来加快速度。

3.64.4.10　PostGIS 支持

数据类型 geometry 仅在安装 PostGIS 时可用。

仅支持二维和三维的 POINT, LINE, POLYGON, MULTIPOINT, MULTILINE, MULTIPOLYGONT 等几何体类型。不支持空的 PostGIS 几何图形，因为它们在 Oracle Spatial 中没有等效的几何图形。

Oracle SRID 的空值将转换为 0，反之亦然。对于 Oracle SRID 和 PostGIS SRID 之间的其他转换，请创建一个 srid.map 文件在 kingbase 的 share 目录中。该文件的每一行都应包含一个 Oracle SRID 和相应的 PostGIS SRID，用空格隔开。保持文件小以获得良好的性能。

3.64.4.11　支持 IMPORT FOREIGN SCHEMA

除了导入外部 SCHEMA 的文档外，还应考虑以下内容：

导入外部 SCHEMA 将为 ALL_TAB_COLUMNS 中的所有对象创建外部表。包括表、视图和物化视图，但不包括同义词。

以下是导入外部 SCHEMA 支持的选项：

- case：控制导入期间表名和列名的大小写折叠

  可能的值为：

  - keep：保留甲骨文中的名字，通常用大写。
  - lower：将所有表名和列名转换为小写。
  - smart：只翻译在 Oracle 中都是大写的名称（这是默认值）。

- collation：用于 case 选项中的 lower 和 smart 值

  默认值是 default，这是数据库的默认排序规则。只支持 sys_catalog SCHEMA 中的排序规则。有关可能值的列表，请参 sys_collation 目录中的 collname 值。

- dblink：访问 SCHEMA 的 Oracle 数据库链接

  此名称必须完全按照 Oracle 系统目录中的名称书写，因此通常仅由大写字母组成。

- readonly：在所有导入的表上设置 readonly 选项

- max_long：在所有导入的表上设置 max_long 选项
- sample_percent：在所有导入的表上设置 sample_percent 选项
- prefetch：在所有导入的表上设置 prefetch 选项

Oracle SCHEMA 名称必须与 Oracle 中的名称完全相同，因此通常使用大写。由于 kingbase 在处理之前将名称转换为小写，因此必须用双引号保护 SCHEMA 名称（例如“SCOTT”）。

LIMIT TO 或 EXCEPT 子句中的表名必须按照上述大小写折叠后在 kingbase 中出现的方式编写。

请注意，导入外部 SCHEMA 不适用于 oracle server 8i。

备注:

oracle_fdw 目前不支持批量导入大量数据，如需导入数据，请使用 oracle 数据导入工具进行操作。在使用 oracle_fdw 时，其内部函数会将 WHERE 子句中的条件进行倒序处理，因此会导致 WHERE 条件的执行顺序与本地执行的 WHERE 条件顺序相反。如果 WHERE 子句中引用了序列，那么会因条件倒序引发结果与预期不符的情况，使用需要注意应避开此类情况。

3.64.4.12　示例

```
CREATE EXTENSION oracle_fdw;
-- TWO_TASK or ORACLE_HOME and ORACLE_SID must be set in the server's environment for this to work
CREATE SERVER oracle FOREIGN DATA WRAPPER oracle_fdw OPTIONS (dbserver '//serverip:port/orcl', isolation_
level 'read_committed');
CREATE USER MAPPING FOR PUBLIC SERVER oracle OPTIONS (user 'C##WQ', password '123');
-- drop the Oracle tables if they exist
DO
$$BEGIN
 SELECT oracle_execute('oracle', 'DROP TABLE C##WQ.typetest1 PURGE');
EXCEPTION
 WHEN OTHERS THEN
    NULL;
END;$$;
DO
$$BEGIN
 SELECT oracle_execute('oracle', 'DROP TABLE C##WQ.gis PURGE');
EXCEPTION
 WHEN OTHERS THEN
    NULL;
END;$$;
SELECT oracle_execute(
        'oracle',
        E'CREATE TABLE C##WQ.typetest1 (\n'
```

```
        '   id  NUMBER(5)\n'
        '      CONSTRAINT typetest1_pkey PRIMARY KEY,\n'
        '   c   CHAR(10 CHAR),\n'
        '   nc  NCHAR(10),\n'
        '   vc  VARCHAR2(10 CHAR),\n'
        '   nvc NVARCHAR2(10),\n'
        '   lc  CLOB,\n'
        '   r   RAW(10),\n'
        '   u   RAW(16),\n'
        '   lb  BLOB,\n'
        '   lr  LONG RAW,\n'
        '   b   NUMBER(1),\n'
        '   num NUMBER(7,5),\n'
        '   fl  BINARY_FLOAT,\n'
        '   db  BINARY_DOUBLE,\n'
        '   d   DATE,\n'
        '   ts  TIMESTAMP WITH TIME ZONE,\n'
        '   ids INTERVAL DAY TO SECOND,\n'
        '   iym INTERVAL YEAR TO MONTH\n'
        ') SEGMENT CREATION IMMEDIATE'
     );
oracle_execute
----------------

(1 row)

SELECT oracle_execute(
        'oracle',
        E'CREATE TABLE C##WQ.gis (\n'
        '   id  NUMBER(5) PRIMARY KEY,\n'
        '   g   MDSYS.SDO_GEOMETRY\n'
        ') SEGMENT CREATION IMMEDIATE'
     );
oracle_execute
----------------

(1 row)

-- gather statistics
SELECT oracle_execute(
        'oracle',
        E'BEGIN\n'
        '   DBMS_STATS.GATHER_TABLE_STATS (''C##WQ'', ''TYPETEST1'', NULL, 100);\n'
        'END;'
     );
```

```
oracle_execute
----------------

(1 row)

SELECT oracle_execute(
        'oracle',
        E'BEGIN\n'
        '   DBMS_STATS.GATHER_TABLE_STATS (''C##WQ'', ''GIS'', NULL, 100);\n'
        'END;'
     );
oracle_execute
----------------

(1 row)

-- create the foreign tables
CREATE FOREIGN TABLE typetest1 (
 id  integer OPTIONS (key 'yes') NOT NULL,
 q   double precision,
 c   character(10),
 nc  character(10),
 vc  character varying(10),
 nvc character varying(10),
 lc  text,
 r   bytea,
 u   uuid,
 lb  bytea,
 lr  bytea,
 b   boolean,
 num numeric(7,5),
 fl  float,
 db  double precision,
 d   date,
 ts  timestamp with time zone,
 ids interval,
 iym interval
) SERVER oracle OPTIONS (table 'TYPETEST1');
ALTER FOREIGN TABLE typetest1 DROP q;
-- a table that is missing some fields
CREATE FOREIGN TABLE shorty (
 id  integer OPTIONS (key 'yes') NOT NULL,
 c   character(10)
) SERVER oracle OPTIONS (table 'TYPETEST1');
-- a table that has some extra fields
```

```
CREATE FOREIGN TABLE longy (
 id  integer OPTIONS (key 'yes') NOT NULL,
 c   character(10),
 nc  character(10),
 vc  character varying(10),
 nvc character varying(10),
 lc  text,
 r   bytea,
 u   uuid,
 lb  bytea,
 lr  bytea,
 b   boolean,
 num numeric(7,5),
 fl  float,
 db  double precision,
 d   date,
 ts  timestamp with time zone,
 ids interval,
 iym interval,
 x   integer
) SERVER oracle OPTIONS (table 'TYPETEST1');

INSERT INTO typetest1 (id, c, nc, vc, nvc, lc, r, u, lb, lr, b, num, fl, db, d, ts, ids, iym) VALUES (
 1,
 'fixed char',
 'nat''l char',
 'varlena',
 'nat''l var',
 'character large object',
 bytea('\xDEADBEEF'),
 uuid('055e26fa-f1d8-771f-e053-1645990add93'),
 bytea('\xDEADBEEF'),
 bytea('\xDEADBEEF'),
 TRUE,
 3.14159,
 3.14159,
 3.14159,
 '1968-10-20',
 '2009-01-26 15:02:54.893532 PST',
 '1 day 2 hours 30 seconds 1 microsecond',
 '-6 months'
);
INSERT INTO shorty (id, c) VALUES (2, NULL);
INSERT INTO typetest1 (id, c, nc, vc, nvc, lc, r, u, lb, lr, b, num, fl, db, d, ts, ids, iym) VALUES (
 3,
```

```
 E'a\u001B\u0007\u000D\u007Fb',
 E'a\u001B\u0007\u000D\u007Fb',
 E'a\u001B\u0007\u000D\u007Fb',
 E'a\u001B\u0007\u000D\u007Fb',
 E'a\u001B\u0007\u000D\u007Fb ABC' || repeat('X', 9000),
 bytea('\xDEADF00D'),
 uuid('055f3b32-a02c-4532-e053-1645990a6db2'),
 bytea('\xDEADF00DDEADF00DDEADF00D'),
 bytea('\xDEADF00DDEADF00DDEADF00D'),
 FALSE,
 -2.71828,
 -2.71828,
 -2.71828,
 '0044-03-15 BC',
 '0044-03-15 12:00:00 BC',
 '-2 days -12 hours -30 minutes',
 '-2 years -6 months'
);

INSERT INTO typetest1 (id, c, nc, vc, nvc, lc, r, u, lb, lr, b, num, fl, db, d, ts, ids, iym) VALUES (
 4,
 'short',
 'short',
 'short',
 'short',
 'short',
 bytea('\xDEADF00D'),
 uuid('0560ee34-2ef9-1137-e053-1645990ac874'),
 bytea('\xDEADF00D'),
 bytea('\xDEADF00D'),
 NULL,
 0,
 0,
 0,
 NULL,
 NULL,
 '23:59:59.999999',
 '3 years'
);
```

3.64.5　插件 oracle_fdw 的卸载方法

在 ksql 中运行：

```
drop extension oracle_fdw;
```

3.64.6 插件 oracle_fdw 的升级方法

oracle_fdw 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.65 owa_util

3.65.1 插件 owa_util 简介

插件 owa_util 是 KingbaseES 的一个扩展插件。插件 owa_util 功能是提供 owa_util 系统包。

owa_util 系统包提供了 WHO_CALLED_ME 过程。

- 插件名为 owa_util
- 插件版本 V1.0

3.65.2 插件 owa_util 加载方式

该插件为初始化数据库实例时默认创建。

3.65.3 插件 owa_util 的参数配置

owa_util 扩展插件无需配置任何参数。

3.65.4 插件 owa_util 使用方法

3.65.4.1 返回过程调用者的信息

语法

```
OWA_UTIL.WHO_CALLED_ME(
   owner          OUT      VARCHAR2,
   name           OUT      VARCHAR2,
   lineno         OUT      NUMBER,
   caller_t       OUT      VARCHAR2);
```

功能描述

这个过程返回有关调用它的 PL/SQL 代码单元的信息（以输出参数的形式）。

参数说明

参数	描述
owner	该程序单元的拥有者
name	程序单元的名称
lineno	程序单元中发出调用的行号
caller_t	进行调用的程序单元的类型。可能有：匿名块、触发器、包体、类型体、过程和函数。过程和函数仅适用于独立的过程和函数

备注:

- 如果调用程序单元是在包中子程序，那么 name 就是包名。
- 如果调用程序单元是独立的过程或函数，那么 name 就是过程或函数的名称。
- 如果调用程序单元是匿名块的一部分，那么 name 则为 NULL。

3.65.4.2　示例

将 who_called_me 子程序作为 proc1 过程的一部分，只要有 PL/SQL 程序单元调用 proc1，那么可以得到 proc1 的 owner、name、lineno 和 caller_t。若是匿名块调用 proc1, 那么 owner 和 name 则为 NULL。

示例如下:

```
\set SQLTERM /
create or replace procedure proc1(id number) as
owner_name varchar2(100);
caller_name varchar2(100);
line_number number;
caller_type varchar2(100);
begin
    owa_util.who_called_me(owner_name,caller_name,line_number,caller_type);
    dbms_output.put_line('[id]:' || id || '   '
                         || '[caller_type]:' || caller_type || '    '
                         || '[owner_name]:' || owner_name || '    '
                         || '[caller_name]:' || caller_name || '    '
                         || '[line_number]:' || line_number || '    '
```

```
                        );
end;
/
set serverout on
begin
    proc1(1);
end;
/
\set SQLTERM ;
[id]:1   [caller_type]:ANONYMOUS BLOCK    [owner_name]:    [caller_name]:    [line_number]:2
```

3.65.5 插件 owa_util 卸载方法

无法卸载。

3.65.6 插件 owa_util 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1：

```
ALTER EXTENSION owa_util UPDATE TO '1.1';
```

3.66 pageinspect

3.66.1 插件 pageinspect 简介

pageinspect 提供函数可以查看数据库页面的内容，对于问题排查、故障处置都很有用。所有函数默认只能被超级用户使用。

- 插件名为 pageinspect
- 插件版本 V1.8

3.66.2 插件 pageinspect 加载方式

```
CREATE EXTENSION pageinspect;
```

3.66.3　插件 pageinspect 的参数配置

无需配置任何参数。

3.66.4　插件 pageinspect 的使用方法

插件提供以下函数：

3.66.4.1　通用函数

`get_raw_page(relname text, fork text, blkno int) returns bytea`

get_raw_page 读取指定关系、指定 fork 的指定块并以 bytea 二进制的形式返回页面数据。函数可以获得该块在某一时刻一致的副本。fork 字符对应关系：'main' 对应数据；'fsm' 对应 FSM 空闲空间页面；'vm' 对应 VM 可见性页面；'init' 对应初始页面。

`get_raw_page(relname text, blkno int) returns bytea`

用于读取数据 fork 的简化版 get_raw_page。等效于 get_raw_page(relname, 'main', blkno)。

`page_header(page bytea) returns record`

page_header 显示使用标准页面结构的堆表、索引页面的页头信息。传入 get_raw_page 获得的页面数据作为参数 page。

示例

```
CREATE TABLE t001(id int,msg varchar(10));
INSERT INTO t001 VALUES(1,'test');

SELECT * FROM page_header(get_raw_page('t001', 0));
    lsn     | checksum | flags | lower | upper  | special | pagesize | version | prune_xid
------------+----------+-------+-------+--------+---------+----------+---------+-----------
 0/4A2D1BB0 |        0 |     5 |  1168 | -32768 |  -32768 |   -32768 |       5 |         0
(1 row)
```

返回的列对应页头结构 PageHeaderData 中的各个成员。

checksum 存放页面的校验和，如果页面被损坏它可能是不正确的。如果当前实例没有开启页面校验和，输出的值没有实际意义。

`page_checksum(page bytea, blkno int4) returns smallint`

page_checksum 计算页面的校验和。传入 get_raw_page 获得的页面数据作为参数 page。

示例

```
SELECT page_checksum(get_raw_page('t001', 0), 0);
 page_checksum
---------------
         -4188
(1 row)
```

注意校验和的计算和块号相关，因此应该传入和获取的页面的块号（除非在做调试）。

用这个函数计算的校验和可以用来和函数 page_header 的结果中的 checksum 进行比较。如果示例开启了页面校验和则两个值应该相等。

`fsm_page_contents(page bytea) returns text`

fsm_page_contents 显示 FSM 页面的内容和结构，页面中二叉树结构每个节点会输出一行，只有不为 0 的节点输出。

示例

```
vacuum t001;
SELECT fsm_page_contents(get_raw_page('t001', 'fsm', 0));
 fsm_page_contents
-------------------
 0: 254           +
 1: 254           +
 3: 254           +
 7: 254           +
 15: 254          +
 31: 254          +
 63: 254          +
 127: 254         +
 255: 254         +
 511: 254         +
 1023: 254        +
 2047: 254        +
 4095: 254        +
 8191: 254        +
 16383: 254       +
 fp_next_slot: 0  +

(1 row)
```

3.66.4.2　堆表函数

`heap_page_items(page bytea) returns setof record`

heap_page_items 显示一个堆表页面上所有的行指针，对于使用中的行指针，元组头部和元组原始数据也

会被显示。不管元组对于拷贝原始页面时的 MVCC 快照是否可见，它们都会被显示。

示例，二进制输出部分使用…省略

```
DROP TABLE t001;
CREATE TABLE t001(id int,msg1 varchar(10),msg2 varchar(10),msg3 varchar(10),msg4 varchar(10),msg5
varchar(10),msg6 varchar(10),msg7 varchar(10),msg8 varchar(10));
INSERT INTO t001 VALUES(1,'msgr1cc1','msgr1c2','msgr1c3','msgr1c4','msgr1c5','msgr1c6','msgr1c7','msgr1c8');
INSERT INTO t001 VALUES(2,'msgr2cc1','msgr2c2','msgr2c3','msgr2c4','msgr2c5','msgr2c6','msgr2c7','msgr2c8');
INSERT INTO t001 VALUES(3,'msgr3cc1','msgr3c2','msgr3c3','msgr3c4','msgr3c5','msgr3c6','msgr3c7','msgr3c8');
INSERT INTO t001 VALUES(4,'msgr4cc1','msgr4c2','msgr4c3','msgr4c4','msgr4c5','msgr4c6','msgr4c7','msgr4c8');
SELECT * from heap_page_items(get_raw_page('t001', 'main', 0));
 lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_field3 | t_ctid | t_infomask2 | t_infomask | t_ho
ff | t_bits | t_oid |                                                                          t_data

----+--------+----------+--------+--------+--------+----------+--------+-------------+------------+-----
---+--------+-------+-----------------------------------------------------------------------------------------
-----------------------------------------------------------
  1 |  32672 |        1 |     93 |   1396 |      0 |        0 | (0,1)  |           9 |       2050 |
24 |        |       | \x01000000136d73677231636331116d736772316332116d736772316333116d736772316334116d73
6772316335116d736772316336116d736772316337116d736772316338
  2 |  32576 |        1 |     93 |   1397 |      0 |        0 | (0,2)  |           9 |       2050 |
24 |        |       | \x02000000136d73677232636331116d736772326332116d736772326333116d736772326334116d73
6772326335116d736772326336116d736772326337116d736772326338
  3 |  32480 |        1 |     93 |   1398 |      0 |        0 | (0,3)  |           9 |       2050 |
24 |        |       | \x03000000136d73677233636331116d736772336332116d736772336333116d736772336334116d73
6772336335116d736772336336116d736772336337116d736772336338
  4 |  32384 |        1 |     93 |   1399 |      0 |        0 | (0,4)  |           9 |       2050 |
24 |        |       | \x04000000136d73677234636331116d736772346332116d736772346333116d736772346334116d73
6772346335116d736772346336116d736772346337116d736772346338
(4 rows)
```

`tuple_data_split(rel_oid oid, t_data bytea, t_infomask integer, t_infomask2 integer, t_bits text [, do_detoast bool]) returns bytea[]`

tuple_data_split 以和 backend 内部相同的方式将元组数据分解成属性。用 heap_page_items 的返回值中的列作为参数传入。

示例

```
SELECT tuple_data_split('t001'::regclass, t_data, t_infomask, t_infomask2, t_bits) from heap_page_items(get_
raw_page('t001', 'main', 0));
                                                                                          tuple_data_spli
t
-----------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------
 {"\\x01000000","\\x136d73677231636331","\\x116d736772316332","\\x116d736772316333","\\x116d736772316334
```

```
","\\x116d736772316335","\\x116d736772316336","\\x116d736772316337","\\x116d736772316338"}
 {"\\x02000000","\\x136d73677232636331","\\x116d736772326332","\\x116d736772326333","\\x116d736772326334
","\\x116d736772326335","\\x116d736772326336","\\x116d736772326337","\\x116d736772326338"}
 {"\\x03000000","\\x136d73677233636331","\\x116d736772336332","\\x116d736772336333","\\x116d736772336334
","\\x116d736772336335","\\x116d736772336336","\\x116d736772336337","\\x116d736772336338"}
 {"\\x04000000","\\x136d73677234636331","\\x116d736772346332","\\x116d736772346333","\\x116d736772346334
","\\x116d736772346335","\\x116d736772346336","\\x116d736772346337","\\x116d736772346338"}
(4 rows)
```

`heap_page_item_attrs(page bytea, rel_oid regclass [, do_detoast bool]) returns setof record`

heap_page_item_attrs 和 heap_page_items 效果相同，除了它会把元组的原始数据返回为属性数组，如果设置了 do_detoast 为 true（默认为 false），属性会被反 TOAST。

示例

```
SELECT * FROM heap_page_item_attrs(get_raw_page('t001', 0), 't001'::regclass);
 lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_field3 | t_ctid | t_infomask2 | t_infomask | t_ho
ff | t_bits | t_oid |
           t_attrs

----+--------+----------+--------+--------+--------+----------+--------+-------------+------------+-----
---+--------+-------+-----------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
--------
  1 |  32672 |        1 |     93 |   1396 |      0 |        0 | (0,1)  |           9 |       2050 |
24 |        |       | {"\\x01000000","\\x136d73677231636331","\\x116d736772316332","\\x116d736772316333"
,"\\x116d736772316334","\\x116d736772316335","\\x116d736772316336","\\x116d736772316337","\\x116d7367723
16338"}
  2 |  32576 |        1 |     93 |   1397 |      0 |        0 | (0,2)  |           9 |       2050 |
24 |        |       | {"\\x02000000","\\x136d73677232636331","\\x116d736772326332","\\x116d736772326333"
,"\\x116d736772326334","\\x116d736772326335","\\x116d736772326336","\\x116d736772326337","\\x116d7367723
26338"}
  3 |  32480 |        1 |     93 |   1398 |      0 |        0 | (0,3)  |           9 |       2050 |
24 |        |       | {"\\x03000000","\\x136d73677233636331","\\x116d736772336332","\\x116d736772336333"
,"\\x116d736772336334","\\x116d736772336335","\\x116d736772336336","\\x116d736772336337","\\x116d7367723
36338"}
  4 |  32384 |        1 |     93 |   1399 |      0 |        0 | (0,4)  |           9 |       2050 |
24 |        |       | {"\\x04000000","\\x136d73677234636331","\\x116d736772346332","\\x116d736772346333"
,"\\x116d736772346334","\\x116d736772346335","\\x116d736772346336","\\x116d736772346337","\\x116d7367723
46338"}
(4 rows)
```

3.66.4.3　B 树索引函数

`bt_metap(relname text)` 返回 `record`

bt_metap 返回 B 树索引 meta 页面的信息。

示例

```
DROP TABLE t001;
CREATE TABLE t001(id int,msg varchar(10));
INSERT INTO t001 VALUES(1,'test');
INSERT INTO t001 VALUES(2,'test');
INSERT INTO t001 VALUES(3,'test');
INSERT INTO t001 VALUES(4,'test');
update t001 set msg='test_new';
CREATE INDEX t001_pkey ON t001(id);
vacuum t001;
SELECT * FROM bt_metap('t001_pkey');
 magic  | version | root | level | fastroot | fastlevel | last_cleanup_num_delpages | last_cleanup_num_
tuples
--------+---------+------+-------+----------+-----------+---------------------------+-----------------------
--
 340322 |       4 |    1 |     0 |        1 |         0 |                         0 |
4
(1 row)
```

`bt_page_stats(relname text, blkno int)` `returns record`

bt_page_stats 返回指定 B 树索引指定块的页面统计信息。

示例

```
SELECT * FROM bt_page_stats('t001_pkey', 1);
 blkno | type | live_items | dead_items | avg_item_size | page_size | free_size | btpo_prev | btpo_next |
btpo_level | btpo_flags
-------+------+------------+------------+---------------+-----------+-----------+-----------+-----------+---
---------+------------
     1 | l    |          4 |          0 |            16 |     32768 |     32632 |         0 |         0 |
      0 |          3
(1 row)
```

`bt_page_items(relname text, blkno int)` `returns setof record`

bt_page_items 返回指定 B 树索引指定块的页面中的所有索引项。

示例

```
SELECT * FROM bt_page_items('t001_pkey', 1);
 itemoffset | ctid  | itemlen | nulls | vars |          data
```

```
------------+-------+---------+-------+------+-------------------------
          1 | (0,1) |      16 | f     | f    | 01 00 00 00 00 00 00 00
          2 | (0,3) |      16 | f     | f    | 02 00 00 00 00 00 00 00
          3 | (0,4) |      16 | f     | f    | 03 00 00 00 00 00 00 00
          4 | (0,2) |      16 | f     | f    | 04 00 00 00 00 00 00 00
(4 rows)
```

`bt_page_items(page bytea) returns setof record`

bt_page_items 接收 get_raw_page 的输出作为参数，返回内容和上面的函数相同。

3.66.4.4 位图索引函数

`bm_metap(relname text)`

bm_metap 返回指定的 bitmap 索引 meta 页面的信息。

示例

```
DROP TABLE t001;
CREATE TABLE t001(id int,msg varchar(10));
INSERT INTO t001 VALUES(1,'test');
INSERT INTO t001 VALUES(2,'test');
INSERT INTO t001 VALUES(3,'test');
INSERT INTO t001 VALUES(4,'test');
CREATE INDEX idx_t ON t001 USING bitmap(id);

SELECT * FROM bm_metap('idx_t');
   magic    | version | auxrelid | auxindexrelid | lovlastblknum
------------+---------+----------+---------------+---------------
 1112101965 |       2 |    16421 |         16424 |             1
(1 rows)
```

`bm_lov_page_items(relname text, blkno int4)`

bm_lov_page_items 返回指定的 bitmap 索引的 lov 页保存的信息。

示例

```
SELECT * FROM bm_lov_page_items('idx_t',1);
 itemoffset | lov_head_blkno | lov_tail_blkno |   last_complete_word    |        last_word        | last
_tid | last_setbit_tid | is_last_complete_word_fill | is_last_word_fill
------------+----------------+----------------+-------------------------+-------------------------+-----
-----+-----------------+----------------------------+-------------------
          1 |     4294967295 |     4294967295 | ff ff ff ff ff ff ff ff | 00 00 00 00 00 00 00 00 |
   0 |               0 | f                          | f
          2 |     4294967295 |     4294967295 | ff ff ff ff ff ff ff ff | 00 00 00 00 00 00 00 01 |
```

```
   0 |              1 | f                          | f
         3 |     4294967295 |     4294967295 | ff ff ff ff ff ff ff ff | 00 00 00 00 00 00 00 02 |
   0 |              2 | f                          | f
         4 |     4294967295 |     4294967295 | ff ff ff ff ff ff ff ff | 00 00 00 00 00 00 00 04 |
   0 |              3 | f                          | f
         5 |     4294967295 |     4294967295 | ff ff ff ff ff ff ff ff | 00 00 00 00 00 00 00 08 |
   0 |              4 | f                          | f
(5 rows)
```

bm_bitmap_page_header(relname text, blkno int4)

bm_bitmap_page_header 返回指定的 bitmap 索引的位图页的统计信息。

示例

```
insert into t001 select n,'test' from generate_series(1,130) as n;
insert into t001 select n,'test' from generate_series(1,130) as n;
insert into t001 select n,'test' from generate_series(1,130) as n;
insert into t001 select n,'test' from generate_series(1,130) as n;
insert into t001 select n,'test' from generate_series(1,130) as n;
insert into t001 select n,'test' from generate_series(1,130) as n;
SELECT * FROM bm_bitmap_page_header('idx_t',2);
 num_words | next_blkno | last_tid
-----------+------------+----------
        10 | 4294967295 |      704
(1 row)
```

bm_bitmap_page_items(relname text, blkno int4)

bm_bitmap_page_items 返回指定的 bitmap 索引的位图页中保存的位图信息。

示例

```
SELECT * FROM bm_bitmap_page_items('idx_t',2);
 word_num | compressed |      content_word
----------+------------+-------------------------
        0 | t          | 00 00 00 00 00 00 00 01
        1 | f          | 80 00 00 00 00 00 00 00
        2 | t          | 00 00 00 00 00 00 00 02
        3 | f          | 00 00 00 00 00 00 00 02
        4 | t          | 00 00 00 00 00 00 00 01
        5 | f          | 00 00 00 00 00 00 00 08
        6 | t          | 00 00 00 00 00 00 00 01
        7 | f          | 00 00 00 00 00 00 00 20
        8 | t          | 00 00 00 00 00 00 00 01
        9 | f          | 00 00 00 00 00 00 00 80
(10 rows)
```

3.66.5 插件 pageinspect 卸载方法

```
DROP EXTENSION pageinspect;
```

3.66.6 插件 pageinspect 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION pageinspect UPDATE TO '1.1';
```

3.67 passwordcheck

3.67.1 插件 passwordcheck 简介

passwordcheck 插件主要用于管理口令复杂度。

口令的复杂度检查是由数据库安全员对口令的最小长度，所包含的数字、英文字母、特殊符号的数目进行设置后，在数据库管理员创建和修改用户时，自动对口令进行相关方面的检查。如果口令不满足指定的条件，那么创建用户将不成功。

KingbaseES 通过插件的方式来进行口令的复杂度管理。这种方式更为灵活，当数据库的实用场景需要进行口令的复杂度管理时，加载插件即可。而不需要该功能时，卸载插件即可。

KingbaseES 中通过 4 个全局级参数配合插件来实现用户口令复杂度管理。

- 插件名为 passwordcheck
- 插件版本 V1.0

3.67.2 插件 passwordcheck 加载方式

在使用 passwordcheck 之前，需要将其添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库使配置生效。

```
shared_preload_libraries = 'passwordcheck'
```

重启数据库后，执行命令 create extension passwordcheck ，插件加载成功。

3.67.3　插件 passwordcheck 的参数配置

- passwordcheck.enable

  口令复杂度开关，默认为关闭状态。

- passwordcheck.password_length

  口令的最小长度，取值范围为 [8，63]，缺省为 8。

- passwordcheck.password_condition_letter

  口令至少包含几个字母，取值范围为 [2，61]，缺省为 2。

- passwordcheck.password_condition_digit

  口令至少包含几个数字，取值范围为 [2，61]，缺省为 2。

- passwordcheck.password_condition_punct

  口令至少包含几个特殊字符，取值范围为 [0，59]，缺省为 0。其中特殊符号为除空白符、英文字母、单引号和数字外的所有可见字符。

3.67.4　插件 passwordcheck 的使用方法

加载 passwordcheck 插件后，在数据库管理员创建和修改用户时，自动对口令进行相关方面的检查。如果口令不满足指定的条件，那么创建用户将不成功。当不需要该功能时，卸载插件即可。

示例：

```
-- 打开口令复杂度开关

\c test system
create extension passwordcheck;
CREATE EXTENSION

\c test sso
show passwordcheck.enable;
 passwordcheck.enable
----------------------
off
(1 行记录)

alter system set passwordcheck.enable=on;
ALTER SYSTEM

select sys_reload_conf();
 sys_reload_conf
-----------------
```

```
t
(1 行记录)

show passwordcheck.enable;
 passwordcheck.enable
----------------------
on
(1 行记录)

-- 修改口令最小长度为 10

\c test sso
SHOW passwordcheck.password_length;
 passwordcheck.password_length
-------------------------------
 8
(1 row)

alter system set passwordcheck.password_length = 10;
ALTER SYSTEM

select sys_reload_conf();
 sys_reload_conf
-----------------
t
(1 行记录)

SHOW passwordcheck.password_length;
 passwordcheck.password_length
-------------------------------
 10
(1 row)

\c test system
CREATE USER u_pwd PASSWORD '123ab';
ERROR:  password length 5 is too short, should be longer than min
password length 10.

CREATE USER u_pwd PASSWORD '1234567890ab';
CREATE ROLE

-- 修改口令最少包含 3 个字母

\c test sso
SHOW passwordcheck.password_condition_letter;
```

```
 passwordcheck.password_condition_letter
----------------------------------------
 2
(1 row)

alter system set passwordcheck.password_condition_letter = 3;
ALTER SYSTEM

select sys_reload_conf();
 sys_reload_conf
-----------------
t
(1 行记录)

SHOW passwordcheck.password_condition_letter;
 passwordcheck.password_condition_letter
----------------------------------------
 3
(1 row)

\c test system
ALTER USER u_pwd PASSWORD '1234567890';
ERROR:  Password should contain at least 3 letter and the current
number is 0

ALTER USER u_pwd PASSWORD '1234567890ab';
ERROR:  Password should contain at least 3 letter and the current
number is 2

ALTER USER u_pwd PASSWORD '1234567890abC';
ALTER ROLE

-- 修改口令最少包含 3 个数字

\c test sso
SHOW passwordcheck.password_condition_digit;
 passwordcheck.password_condition_digit
----------------------------------------
 2
(1 row)

alter system set passwordcheck.password_condition_digit = 3;
ALTER SYSTEM

select sys_reload_conf();
```

```
 sys_reload_conf
-----------------
t
(1 行记录)

SHOW passwordcheck.password_condition_digit;
 passwordcheck.password_condition_digit
----------------------------------------
 3
(1 row)

\c test system
ALTER USER u_pwd PASSWORD 'abcdefghij';
ERROR:  Password should contain at least 3 digit and the current
number is 0

ALTER USER u_pwd PASSWORD 'abcdefghij123';
ALTER ROLE

-- 修改口令最少包含 3 个特殊字符

\c test sso
SHOW passwordcheck.password_condition_punct;
 passwordcheck.password_condition_punct
----------------------------------------
 0
(1 row)

alter system set passwordcheck.password_condition_punct = 2;
ALTER SYSTEM

select sys_reload_conf();
 sys_reload_conf
-----------------
t
(1 行记录)

SHOW passwordcheck.password_condition_punct;
 passwordcheck.password_condition_punct
----------------------------------------
 2
(1 row)

\c test system
ALTER USER u_pwd PASSWORD '1234567890abc.';
```

```
ERROR:  Password should contain at least 2 punct and the current
number is 1

ALTER USER u_pwd PASSWORD '1234567890abc./';
ALTER ROLE
```

3.67.5　插件 passwordcheck 卸载方法

通过 drop extension 来卸载插件：

```
drop extension passwordcheck;
```

如果设置了 shared_preload_libraries，则需要删除‘passwordcheck’：

```
shared_preload_libraries = ''
```

3.67.6　插件 passwordcheck 升级方法

passwordcheck 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.68　passwordhistory

3.68.1　插件 passwordhistory 简介

passwordhistory 插件主要用于管理口令历史。

口令的历史检查是由数据库管理员对初次设定的口令或更改过的口令使用天数进行设置后，在修改用户口令时，自动对口令已使用天数进行相关方面的检查。如果口令不满足指定的条件，那么更改口令将不成功。

KingbaseES 通过插件的方式来进行口令历史管理。这种方式更为灵活，当数据库的实用场景需要进行口令历史管理时，加载插件即可。而不需要该功能时，卸载插件即可。

KingbaseES 中通过 2 个全局级参数配合插件来实现用户口令历史管理。

- 插件名为 passwordhistory
- 插件版本 V1.0

3.68.2　插件 passwordhistory 加载方式

在使用 passwordhistory 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

示例：

```
shared_preload_libraries = 'passwordhistory'
```

3.68.3　插件 passwordhistory 的参数配置

`passwordhistory.enable`

密码历史开关，默认为关闭状态。

`passwordhistory.password_time`

设定口令历史天数，缺省为 0。

示例：

```
-- 创建插件并打开口令历史开关

create extension passwordhistory;
CREATE EXTENSION

show passwordhistory.enable;
 passwordhistory.enable
------------------------
off
(1 行记录)

alter system set  passwordhistory.enable=on;
ALTER SYSTEM

select sys_reload_conf();
 sys_reload_conf
-----------------
t
(1 行记录)

show passwordhistory.enable;
 passwordhistory.enable
------------------------
on
(1 行记录)
```

```
-- 设定口令历史天数为 2

show passwordhistory.password_time;
 passwordhistory.password_time
------------------------------
 0
(1 row)

SET passwordhistory.password_time = 2;

call sys_reload_conf();
 sys_reload_conf
-----------------
 t
(1 row)

SHOW passwordhistory.password_time;
 passwordhistory.password_time
------------------------------
 2
(1 row)

create user test with password '123';
CREATE ROLE
alter user test with password '123';
ERROR:  The password has been used recently.
```

3.68.4　插件 passwordhistory 的使用方法

加载 passwordhistory 插件后，在修改用户口令时，程序会自动对口令已使用天数进行相关方面的检查。如果口令不满足指定的条件，那么更改口令将不成功。

3.68.5　插件 passwordhistory 卸载方法

通过 drop extension 来卸载插件：

```
drop extension passwordhistory;
```

如果设置了 shared_preload_libraries，则需要删除‘passwordhistory’：

```
shared_preload_libraries = ''
```

3.68.6 插件 passwordhistory 升级方法

passwordhistory 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.69 pg_pathman

3.69.1 插件 pg_pathman 简介

3.69.1.1 背景信息

pg_pathman 插件是一款分区管理插件，提供了分区优化机制以此来提高分区表的性能。它将分区配置存储在 pathman——config 表中，表的信息会缓存在内存中，同时使用 HOOK 来实现 RELATION 的替换，效率非常高。目前该工具支持 range 和 hash 两种分区方式，range 使用 binary search 查找对应的分区，hash 使用 hash search 查找对应的分区。

3.69.1.2 pg_pathman 特性

1）目前支持 HASH 分区、RANGE 分区。

2）支持自动分区管理（通过函数接口创建分区，自动将主表数据迁移到分区表），或手工分区管理（通过函数实现，将已有的表绑定到分区表，或者从分区表剥离）。

3）支持的分区字段类型包括 int、float、date 以及其他常用类型，包括自定义的 domain。

4）有效的分区表查询计划（JOINs、subselects 等）。

5）使用 RuntimeAppend & RuntimeMergeAppend 自定义计划节点实现了动态分区选择。

6）PartitionFilter：一种有效的插入触发器替换方法。

7）支持自动新增分区（目前仅支持 RANGE 分区表）。

8）支持 copy from/to 直接读取或写入分区表，提高效率。

9）支持分区字段的更新，需要添加触发器，如果不需要更新分区字段，则不建议添加这个触发器，会产生一定的性能影响。

10）允许用户自定义回调函数，在创建分区时会自动触发。

11）非堵塞式创建分区表，以及后台自动将主表数据非堵塞式迁移到分区表。

12）支持 FDW，通过配置参数 pg_pathman.insert_into_fdw=(disabled | postgres | any_fdw) 支持 postgres_fdw 或任意 FDW。

3.69.2　pg_pathman 插件的使用方法

3.69.2.1　创建 pg_pathman 插件扩展

```
create extension pg_pathman;
CREATE EXTENSION
```

3.69.2.2　查看已安装的扩展

使用 dx 命令可以查看已安装的扩展，还可以查看到 pg_pathman 的具体版本。

```
test=# \dx
                                                    List of installed extensions
        Name         | Version |    Schema    |                                                    Description
---------------------+---------+--------------+-------------------------------------------------------------------------------------------------------------
 kdb_license         | 1.0     | pg_catalog   | kdb_license extension
 kingbase_version    | 1.0     | pg_catalog   | This is a utility that provides function related to version number,  it is used to get the Kingbase version number.
 pg_pathman          | 1.5     | public       | Partitioning tool for PostgreSQL
 plpgsql             | 1.0     | pg_catalog   | PL/pgSQL procedural language
 src_restrict        | 1.0     | src_restrict | src restrict plugin
 sys_anon            | 1.0     | anon         | provides data masking functionality
 sys_hm              | 1.0     | pg_catalog   | Kingbase Healthy Check
 sys_stat_statements | 1.10    | public       | track parsing, planning and execution statistics of all SQL statements executed
 sysaudit            | 1.0     | sysaudit     | provides auditing functionality
 sysmac              | 1.0     | sysmac       | Mac for Kingbase
 xlog_record_read    | 1.0     | pg_catalog   | xlog_record_read functions
(11 rows)
```

3.69.2.3　插件升级

升级插件版本时，需要：

1、升级对应集群到最新版本。

2、执行 SQL 版本更新命令，如下所示：

```
ALTER EXTENSION pg_pathman UPDATE;
SET pg_pathman.enable = t;
```

3.69.2.4　相关视图和表

pg_pathman 使用函数来维护分区表，并且创建了一些视图，可以查看分区表的状态，具体如下：

1、pathman_config

```
CREATE TABLE IF NOT EXISTS pathman_config (
    partrel         REGCLASS NOT NULL PRIMARY KEY,  -- 主表 oid
    attname         TEXT NOT NULL,  -- 分区列名
    parttype        INTEGER NOT NULL,  -- 分区类型 (hash or range)
    range_interval  TEXT,  -- range 分区的 interval
CHECK (parttype IN (1, 2)) /* check for allowed part types */ );
```

2、pathman_config_params

```
CREATE TABLE IF NOT EXISTS pathman_config_params (
    partrel        REGCLASS NOT NULL PRIMARY KEY,  -- 主表 oid
    enable_parent  BOOLEAN NOT NULL DEFAULT TRUE,  -- 是否在优化器中过滤主表
    auto           BOOLEAN NOT NULL DEFAULT TRUE,  -- insert 时是否自动扩展不存在的分区
    init_callback  REGPROCEDURE NOT NULL DEFAULT 0);  -- create partition 时的回调函数 oid
```

3、pathman_concurrent_part_tasks

```
CREATE OR REPLACE FUNCTION show_concurrent_part_tasks()
RETURNS TABLE (
    userid     REGROLE,
    pid        INT,
    dbid       OID,
    relid      REGCLASS,
    processed  INT,
    status     TEXT)
AS 'pg_pathman', 'show_concurrent_part_tasks_internal'
LANGUAGE C STRICT;
CREATE OR REPLACE VIEW pathman_concurrent_part_tasks
AS SELECT * FROM show_concurrent_part_tasks();
```

4、pathman_partition_list

```
CREATE OR REPLACE FUNCTION show_partition_list()
RETURNS TABLE (
    parent     REGCLASS,
    partition  REGCLASS,
    parttype   INT4,
    partattr   TEXT,
    range_min  TEXT,
    range_max  TEXT)
AS 'pg_pathman', 'show_partition_list_internal'
LANGUAGE C STRICT;

CREATE OR REPLACE VIEW pathman_partition_list
AS SELECT * FROM show_partition_list();
```

3.69.2.5　分区管理

1、RANGE 分区有四个管理函数用来创建范围分区。其中两个可以指定起始值、间隔、分区个数，其函数定义如下：

```
create_range_partitions(relation       REGCLASS,  -- 主表 OID
                        attribute      TEXT,      -- 分区列名
```

```
                        start_value    ANYELEMENT,  -- 开始值
                        p_interval     ANYELEMENT,  -- 间隔；任意类型，适合任意类型的分区表
                        p_count        INTEGER DEFAULT NULL,   --  分多少个区
                        partition_data BOOLEAN DEFAULT TRUE);   --  是否立即将数据从主表迁移到分区，不建议这么
使用，建议使用非堵塞式的迁移（ 调用 partition_table_concurrently() ）

create_range_partitions(relation       REGCLASS,  -- 主表 OID
                        attribute      TEXT,      -- 分区列名
                        start_value    ANYELEMENT,  -- 开始值
                        p_interval     INTERVAL,    -- 间隔；interval 类型，用于时间分区表
                        p_count        INTEGER DEFAULT NULL,   --  分多少个区
                        partition_data BOOLEAN DEFAULT TRUE)   --  是否立即将数据从主表迁移到分区，不建议这么
使用，建议使用非堵塞式的迁移（ 调用 partition_table_concurrently() ）
```

另外两个可以指定起始值、终值、间隔，其定义如下：

```
create_partitions_from_range(relation       REGCLASS,  -- 主表 OID
                             attribute      TEXT,      -- 分区列名
                             start_value    ANYELEMENT,  -- 开始值
                             end_value      ANYELEMENT,  -- 结束值
                             p_interval     ANYELEMENT,  -- 间隔；任意类型，适合任意类型的分区表
                             partition_data BOOLEAN DEFAULT TRUE)   --  是否立即将数据从主表迁移到分区，不建议
这么使用，建议使用非堵塞式的迁移（ 调用 partition_table_concurrently() ）

create_partitions_from_range(relation       REGCLASS,  -- 主表 OID
                             attribute      TEXT,      -- 分区列名
                             start_value    ANYELEMENT,  -- 开始值
                             end_value      ANYELEMENT,  -- 结束值
                             p_interval     INTERVAL,    -- 间隔；interval 类型，用于时间分区表
                             partition_data BOOLEAN DEFAULT TRUE)   --  是否立即将数据从主表迁移到分区，不建议
这么使用，建议使用非堵塞式的迁移（ 调用 partition_table_concurrently() ）
```

备注:

在 RANGE 分区表使用过程中，建议您: a、分区列必须有 not null 约束 b、分区个数必须能覆盖已有的所有记录 c、使用非堵塞式迁移接口。d、数据迁移完成后，禁用主表。

示例：

```
\set VERBOSITY terse
SET search_path = 'public';
CREATE SCHEMA pathman;
CREATE EXTENSION pg_pathman SCHEMA pathman;
CREATE SCHEMA test;
```

```
CREATE TABLE test.range_rel (
        id      SERIAL PRIMARY KEY,
        dt      TIMESTAMP,
        txt     TEXT);
CREATE INDEX ON test.range_rel (dt);
INSERT INTO test.range_rel (dt, txt)
SELECT g, md5(g::TEXT) FROM generate_series('2015-01-01', '2015-04-30', '1 day'::interval) as g;
\set VERBOSITY default
SELECT pathman.create_range_partitions('test.range_rel', 'dt', '2015-01-01'::DATE, '1 month'::INTERVAL);
```

```
ERROR:  failed to analyze partitioning expression "dt"
DETAIL:  column "dt" should be marked NOT NULL
CONTEXT:  SQL statement "SELECT pathman.validate_expression(parent_relid, expression)"
PL/pgSQL function pathman.prepare_for_partitioning(regclass,text,boolean) line 9 at PERFORM
SQL statement "SELECT pathman.prepare_for_partitioning(parent_relid,
                                                       expression,
                                                       partition_data)"
PL/pgSQL function pathman.create_range_partitions(regclass,text,anyelement,interval,integer,boolean) line 11 at PERFORM
\set VERBOSITY terse
ALTER TABLE test.range_rel ALTER COLUMN dt SET NOT NULL;
```

```
\set VERBOSITY terse
ALTER TABLE test.range_rel ALTER COLUMN dt SET NOT NULL;
SELECT pathman.create_range_partitions('test.range_rel', 'dt', '2015-01-01'::DATE, '1 month'::INTERVAL, 2);
```

```
ERROR:  not enough partitions to fit all values of "dt"
```

```
SELECT pathman.create_range_partitions('test.range_rel', 'DT', '2015-01-01'::DATE, '1 month'::INTERVAL);
```

```
 create_range_partitions
-------------------------
                       4
(1 row)
```

2、HASH 分区有一个管理函数用来创建范围分区，可以指定起始值、间隔、分区个数，具体如下：

```
create_hash_partitions(relation         REGCLASS,  -- 主表 OID
                       attribute        TEXT,      -- 分区列名
                       partitions_count INTEGER,   -- 打算创建多少个分区
                       partition_data   BOOLEAN DEFAULT TRUE)   --  是否立即将数据从主表迁移到分区，不建议这么使用，建议使用非堵塞式的迁移（ 调用 partition_table_concurrently() ）
```

使用非堵塞式的迁移接口

```
partition_table_concurrently(relation   REGCLASS,              -- 主表 OID
                             batch_size INTEGER DEFAULT 1000,  -- 一个事务批量迁移多少记录
                             sleep_time FLOAT8 DEFAULT 1.0)    -- 获得行锁失败时，休眠多久再次获取，重试 60 次退出任务。
```

备注:

在 HASH 分区表使用过程中，建议您：a、分区列必须有 not null 约束。b、使用非堵塞式迁移接口。c、数据迁移完成后，禁用主表。d、pg_pathman 不会受制于表达式的写法，所以 select * from part_table where c 分区列 = xxx; 这样的写法也能用于 HASH 分区的。e、HASH 分区列不局限于 int 类型的列，会使用 HASH 函数自动转换。

示例：

```
CREATE TABLE test.hash_rel (
        id              SERIAL PRIMARY KEY,
        value   INTEGER);
INSERT INTO test.hash_rel VALUES (1, 1);
INSERT INTO test.hash_rel VALUES (2, 2);
INSERT INTO test.hash_rel VALUES (3, 3);
\set VERBOSITY default
SELECT pathman.create_hash_partitions('test.hash_rel', 'value', 3);
```

```
ERROR:  failed to analyze partitioning expression "value"
DETAIL:  column "value" should be marked NOT NULL
CONTEXT:  SQL statement "SELECT pathman.validate_expression(parent_relid, expression)"
PL/pgSQL function pathman.prepare_for_partitioning(regclass,text,boolean) line 9 at PERFORM
SQL statement "SELECT pathman.prepare_for_partitioning(parent_relid,
                                                       expression,
                                                       partition_data)"
PL/pgSQL function pathman.create_hash_partitions(regclass,text,integer,boolean,text[],text[]) line 3 at PERFORM
```

```
\set VERBOSITY terse
ALTER TABLE test.hash_rel ALTER COLUMN value SET NOT NULL;
SELECT pathman.create_hash_partitions('test.hash_rel', 'value', 3, partition_data:=false);
```

```
 create_hash_partitions
------------------------
                      3
(1 row)
```

3、数据迁移到分区如果创建分区表时，未将主表数据迁移到分区，那么可以使用非堵塞式的迁移接口，将数据迁移到分区。用法如下：

```
with tmp as (delete from 主表 limit xx nowait returning *) insert into 分区 select * from tmp
```

或者使用

```
select array_agg(ctid) from 主表 limit xx for update nowati
```

进行标示，然后执行 delete 和 insert。

函数接口如下：

```
partition_table_concurrently(relation   REGCLASS,              -- 主表 OID
                             batch_size INTEGER DEFAULT 1000,  -- 一个事务批量迁移多少记录
                             sleep_time FLOAT8 DEFAULT 1.0)    -- 获得行锁失败时，休眠多久再次获取，重试 60 次
退出任务。
```

4、分裂范围分区如果某个分区太大，想分裂为两个分区，可以使用如下方法（目前仅支持 RANGE 分区表）：

```
split_range_partition(partition      REGCLASS,            -- 分区 oid
                  split_value    ANYELEMENT,          -- 分裂值
                  partition_name TEXT DEFAULT NULL)   -- 分裂后新增的分区表名
                              Table "public.part_test"
```

5、合并范围分区目前仅支持 RANGE 分区，调用如下接口：

指定两个需要合并分区，必须为相邻分区

```
merge_range_partitions(partition1 REGCLASS, partition2 REGCLASS)
```

6、向后添加范围分区

如果已经对主表进行了分区，将来需要增加分区的话，有几种方法，一种是向后新增分区（即在末尾追加分区）。

新增分区时，会使用初次创建该分区表时的 interval 作为间隔。可以在 pathman_config 中查询每个分区表初次创建时的 interval。

添加分区接口（目前不支持指定表空间）：

```
append_range_partition(parent         REGCLASS,            -- 主表 OID
                   partition_name TEXT DEFAULT NULL,   -- 新增的分区表名，默认不需要输入
                   tablespace     TEXT DEFAULT NULL)   -- 新增的分区表放到哪个表空间，默认不需要输入
```

7、向前添加范围分区在头部追加分区：

```
prepend_range_partition(parent         REGCLASS,
                    partition_name TEXT DEFAULT NULL,
                    tablespace     TEXT DEFAULT NULL)
```

8、添加分区指定分区起始值的方式添加分区，只要创建的分区和已有分区不会存在数据交叉就可以创建成功。也就是说使用这种方法，不要求强制创建连续的分区，例如已有分区覆盖了 2010-2015 的范围，您可以直接创建一个 2020 年的分区表，不需要覆盖 2015 到 2020 的范围：

```
add_range_partition(relation       REGCLASS,    -- 主表 OID
                start_value    ANYELEMENT,  -- 起始值
                end_value      ANYELEMENT,  -- 结束值
                partition_name TEXT DEFAULT NULL,  -- 分区名
                tablespace     TEXT DEFAULT NULL)  -- 分区创建在哪个表空间下
```

9、删除分区删除单个范围分区：

```
drop_range_partition(partition TEXT,   -- 分区名称
                   delete_data BOOLEAN DEFAULT TRUE)  -- 是否删除分区数据，如果 false，表示分区数据迁移到主
表。
```

```
Drop RANGE partition and all of its data if delete_data is true.
```

删除所有分区，并且指定是否要将数据迁移到主表。接口如下：

```
drop_partitions(parent      REGCLASS,
             delete_data BOOLEAN DEFAULT FALSE)

Drop partitions of the parent table (both foreign and local relations).
If delete_data is false, the data is copied to the parent table first.
Default is false.
```

10、绑定分区（已有的表加入分区表）将已有的表，绑定到已有的某个分区主表。已有的表与主表要保持一致的结构，包括 dropped columns （查看 pg_attribute 的一致性）。绑定分区时，自动创建继承关系，自动创建约束。接口如下：

```
attach_range_partition(relation    REGCLASS,    -- 主表 OID
                    partition   REGCLASS,    -- 分区表 OID
                    start_value ANYELEMENT,  -- 起始值
                    end_value   ANYELEMENT)  -- 结束值
```

11、解绑分区（将分区变成普通表）将分区从主表的继承关系中删除，不删数据，删除继承关系，删除约束。接口如下：

```
detach_range_partition(partition REGCLASS)  -- 指定分区名，转换为普通表
```

12、永久禁止分区表 pg_pathman 插件您可以针对单个分区主表禁用 pg_pathman。接口函数如下：

```
disable_pathman_for(relation TEXT)

Permanently disable pg_pathman partitioning mechanism for the specified parent table and remove the insert
trigger if it exists.
All partitions and data remain unchanged.
```

禁用 pg_pathman 插件后，继承关系和约束不会变化，只是 pg_pathman 插件不介入 custom scan 执行计划

备注:

disable_pathman_for 没有可逆操作，请慎使用。

3.69.2.6　高级分区管理

1、禁用主表当主表的数据全部迁移到分区后，可以禁用主表。接口函数如下：

```
set_enable_parent(relation REGCLASS, value BOOLEAN)


Include/exclude parent table into/from query plan.

In original PostgreSQL planner parent table is always included into query plan even if it's empty which can
lead to additional overhead.

You can use disable_parent() if you are never going to use parent table as a storage.

Default value depends on the partition_data parameter that was specified during initial partitioning in
create_range_partitions() or create_partitions_from_range() functions.

If the partition_data parameter was true then all data have already been migrated to partitions and parent
table disabled.

Otherwise it is enabled.
```

示例如下所示：

```
select set_enable_parent('part_test', false);
```

2、自动扩展分区范围分区表，允许自动扩展分区。如果新插入的数据不在已有的分区范围内，会自动创建分区。

```
set_auto(relation REGCLASS, value BOOLEAN)

Enable/disable auto partition propagation (only for RANGE partitioning).

It is enabled by default.
```

备注:

不建议开启自动扩展范围分区，不合理的自动扩展可能会消耗大量的时间

3、回调函数（创建每个分区时都会触发）回调函数是在每创建一个分区时会自动触发调用的函数。例如，可以用在 DDL 逻辑复制中，将 DDL 语句记录下来，存放到表中。回调函数如下：

```
set_init_callback(relation REGCLASS, callback REGPROC DEFAULT 0)

Set partition creation callback to be invoked for each attached or created partition (both HASH and RANGE).

The callback must have the following signature:
```

```
part_init_callback(args JSONB) RETURNS VOID.

Parameter arg consists of several fields whose presence depends on partitioning type:

/* RANGE-partitioned table abc (child abc_4) */
{
    "parent":    "abc",
    "parttype":  "2",
    "partition": "abc_4",
    "range_max": "401",
    "range_min": "301"
}

/* HASH-partitioned table abc (child abc_0) */
{
    "parent":    "abc",
    "parttype":  "1",
    "partition": "abc_0"
}
```

3.70　pldbgapi

3.70.1　插件 pldbgapi 简介

插件 pldbgapi 是 KingbaseES 的一个扩展插件。主要用于 PL/pgSQL 语言的存储过程/函数的代码调试。

- 插件名为 pldbgapi
- 插件版本 V1.2

3.70.2　插件 pldbgapi 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。因此只需要在客户端工具执行 create extension 命令即可。

```
-- 创建插件
create extension pldbgapi;
```

备注:

在当前模式和 plpgsql_debug 模式下均创建了一套调试接口，因此在任意模式下调试接口可通过模式限定调用。

3.70.3　插件 pldbgapi 的参数配置

pldbgapi 扩展插件无需配置任何参数。

3.70.4　插件 pldbgapi 使用方法

3.70.4.1　pldebugger 调试流程

以下详细介绍其局部断点调试和全局断点调试流程的命令行调试流程。只是在目标后端与调试端建立连接的流程有稍微差别，实际的调试的流程总体一致。

pldebugger 的调试操作是建立在目标后端和调试端正常通信的前提下，其详细的调试流程如下：

1. 创建监听，局部断点调试根据调试对象的 OID 去建立监听，全局断点调试则不需要带调试对象的 OID 而是创建全局监听。
2. 创建连接，全局调试与局部调试的连接方式稍有不同，对于局部调试，首先是调用待调试的对象，让目标后端进程以通信服务器端的身份发起连接，然后在调试端调用端口连接函数 pldbg_attah_to_port 让调试端以通信客户端的身份与目标后端建立连接；对于全局断点调试，先设置全局断点，然后调用 pldbg_wait_for_target 函数发起连接并等待接受目标后端的连接。此时调用待调试的对象就可以与调试端建立通信连接。
3. 调试设置，可以根据需要设置和删除断点信息，包括局部断点和全局断点。
4. 执行调试操作，pldebugger 提供了三种调试执行的方式，分别为单步进入（step into）、单步跳过（step over）以及继续执行（continue）。
5. 信息查询，在调试执行的过程中，可以查询源代码信息，查询和修改变量值，查询调用堆栈以及当前系统版本等。

3.70.4.2　接口函数

pldebugger 调试工具支持对 PL/pgSQL 语言的函数或存储过程进行调试操作，包括操作断点、执行代码、调试数据、查看调用堆栈、查看源码信息、查看版本信息等功能。并提供局部断点调试与全局断点调试两种调试模式。

调试接口函数列表如下：

表 3.70.1: 接口函数列表

接口函数	简介
plpgsql_oid_debug 函数	创建局部监听

-续表-

接口函数	简介
pldbg_attach_to_port 函数	连接目标后端
pldbg_create_listener 函数	创建全局监听
pldbg_set_global_breakpoint 函数	设置全局断点
pldbg_wait_for_target 函数	等待连接
pldbg_set_breakpoint 函数	设置断点
pldbg_get_breakpoints 函数	查询断点
pldbg_drop_breakpoint 函数	删除断点
pldbg_wait_for_breakpoint 函数	返回建立连接时断点所在行的信息
pldbg_step_into 函数	单步进入（step into）
pldbg_step_over 函数	单步跳过（step over）
pldbg_continue 函数	继续执行（continue）
pldbg_abort_target 函数	中止执行
pldbg_get_variables 函数	查看变量
pldbg_deposit_value 函数	修改变量
pldbg_get_stack 函数	查看堆栈信息
pldbg_select_frame 函数	追溯堆栈信息
pldbg_get_source 函数	查询源代码信息
plsql_extend_get_proc_ddl 函数	获取函数、存储过程源码
pldbg_get_proxy_info 函数	查询当前系统版本信息

3.70.4.2.1　plpgsql_oid_debug 函数

功能描述

在同一个 session 下负责监听该函数或存储过程，创建监听状态。该函数在建立监听的过程中会设置一个系统默认断点，行号为‘-1’。

语法格式

```
FUNCTION plpgsql_oid_debug( FUNCTIONOID OID ) RETURNS INTEGER
```

参数说明

参数名称	描述
FUNCTION	待调试的自定义函数或存储过程 OID

返回值说明

调用成功返回 0，调用失败无返回值，存在报错信息如下：

1. 当 OID 输入为 0 时报错信息：no target specified。
2. 当 OID 无效时报错信息：cache lookup failed for FUNCTION。
3. 当不是 owner 或 superuser 时报错信息：must be owner or superuser to create a breakpoint。

3.70.4.2.2　pldbg_attach_to_port 函数

功能描述

此函数用于响应 PLDBGBREAK 端口号。该函数连接到监听给定端口的 Target Backend 端，从而形成 Debugger Client，启动调试过程，等待后续调试工作。一个 Debugger Client 能够响应多个不同 PLDBGBREAK 端口，即可以支持多个并发会话。

语法格式

```
FUNCTION pldbg_attach_to_port(portNumber INTEGER) RETURNS INTEGER
```

参数说明

参数名称	描述
portNumber	调用自定义函数或存储过程时显示的进程 ID 号

返回值说明

返回标识此特定调试的会话句柄。

3.70.4.2.3　pldbg_create_listener 函数

功能描述

创建一个 TCP 端口，为全局断点创建一个监听器，方便后续函数连接到该目标进程。

语法格式

```
FUNCTION pldbg_create_listener() RETURNS INTEGER
```

返回值说明

调用成功返回 debug_session 号。

3.70.4.2.4　pldbg_set_global_breakpoint 函数

功能描述

创建一个全局断点。

语法格式

```
FUNCTION pldbg_set_global_breakpoint( session  INT, FUNCTION OID, lineNumber INT, targetPID INT ) RETURNS
boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
FUNCTION	自定义函数或存储过程 OID
lineNumber	行号（可为 NULL）
targetPID	目标端 PID（可为 NULL）

返回值说明

调用成功返回 t，调用失败会报错。

3.70.4.2.5　pldbg_wait_for_target 函数

功能描述

发起 socket 连接，等待目标后端进程调用自定义函数或者存储过程来连接到调试端，以触发全局断点。

语法格式

```
FUNCTION pldbg_wait_for_target( session INTEGER ) RETURNS INTEGER
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

调用成功返回 target backend 的 PID。

3.70.4.2.6　pldbg_set_breakpoint 函数

功能描述

在目标进程中设置一个“局部”断点。

此函数将 PLDBG_SET_BREAKPOINT 指令发送到 Targert Backend 端，在自定义函数或者存储过程的指定位置设置断点，然后等待下一个可执行语句到达该目标断点。

语法格式

```
FUNCTION pldbg_set_breakpoint( session INTEGER, FUNCTION OID, lineNumber INTEGER ) RETURNS boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄需要设置的断点行号
FUNCTION	自定义函数或存储过程 OID
lineNumber	若行号传入为 NULL，不会报错也不会设置断点

返回值说明

断点设置成功返回为 t，断点设置失败返回为 f。

3.70.4.2.7　pldbg_get_breakpoints 函数

功能描述

显示设置的所有断点。此函数将 PLDBG_GET_BREAKPOINTS 指令发送到 Targert Backend 端，返回查询到的断点元组信息。如果没有断点信息，则返回为空。

语法格式

```
FUNCTION pldbg_get_breakpoints( session INTEGER ) RETURNS SETOF breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一组“breakpoint”元组信息，该元组信息包括自定义函数或者存储过程的 OID，断点设置的行号 lineNumber，Target 端的名字 targetName。

3.70.4.2.8　pldbg_drop_breakpoint 函数

功能描述

在目标进程中删除一个指定行号的断点。此函数将 PLDBG_CLEAR_BREAKPOINT 指令发送到 Targert Backend 端，删除指定断点信息，并返回删除成功与否的状态。

语法格式

```
FUNCTION pldbg_drop_breakpoint( session INTEGER, FUNCTION OID, lineNumber INTEGER ) RETURNS boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
FUNCTION	自定义函数或存储过程的 OID
lineNumber	所要删除断点的行号

返回值说明

断点删除成功返回 t，断点删除失败返回 f。

3.70.4.2.9　pldbg_wait_for_breakpoint 函数

功能描述

调用该函数，可以返回刚建立连接时断点所在行的信息。

语法格式

```
FUNCTION pldbg_wait_for_breakpoint( session INTEGER ) RETURNS  breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为一个“breakpoint”的元组，该元组包含自定义函数或者存储过程的 OID 以及刚建立连接时停止的行号。

3.70.4.2.10　pldbg_step_into 函数

功能描述

单步执行，遇到子函数或子存储过程会进入到相应的部分。此函数将 PLDBG_STEP_INTO 命令发送到 Targert Backend，等待目标后端进程执行语句后，等待下一个可执行语句到达目标。

语法格式

```
FUNCTION pldbg_step_into( session INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组（但不一定是真正的断点信息，因为单步执行有可能遇到断点，有可能没有遇见断点）。该元组包含自定义函数或者存储过程的 OID 以及目标当前停止的行号，以及自定义函数或者存储过程名。

3.70.4.2.11　pldbg_step_over 函数

功能描述

单步执行，遇见子函数或子存储过程会跳过内部执行过程。此函数将 PLDBG_STEP_OVER 命令发送到 Targert Backend，待目标后端进程执行语句后，等待下一个可执行语句到达目标，并将执行后需要返回的结果返回。

语法格式

```
FUNCTION pldbg_step_over( session INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组（但不一定是真正的断点信息，因为单步执行有可能遇到断点，有可能没有遇见断点）。该元组包含自定义函数或者存储过程的 OID 以及目标当前停止的行号，以及自定义函数或者存储过程名。

3.70.4.2.12　pldbg_continue 函数

功能描述

从当前位置继续执行，直到遇见下一个断点，如果没有断点，将直接执行到自定义函数或存储过程的最后。此函数将“PLDBG_CONTINUE”指令发送到 Targert Backend 端，目标后端进程执行，然后等待目标到达下一个断点，并返回相应的结果信息。

语法格式

```
FUNCTION pldbg_continue( session INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组，该元组包含自定义函数或者存储过程的 OID 以及目标当前停止的行号，以及自定义函数或者存储过程名。

3.70.4.2.13　pldbg_abort_target 函数

功能描述

终止当前的执行。此函数将 PLDBG_ABORT 命令发送到 Targert Backend，然后终止当前的执行，并返回终止成功与否的状态。

语法格式

```
FUNCTION pldbg_abort_target( session INTEGER ) RETURNS SETOF boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

终止成功返回 t。

3.70.4.2.14　pldbg_get_variables 函数

功能描述

获取一个变量或者参数的信息列表。此函数将 PLDBG_GET_VARIABLES 指令发送到 Targert Backend 端，服务器查询到所有变量的信息后返回变量或参数的信息列表。

语法格式

```
FUNCTION pldbg_get_variables( session INTEGER ) RETURNS SETOF var
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一个 var 元组集合，包含信息：VarName；Class；lineNumber；Unique；isConst；notNull；dataTypeOID；value(rest of string)。其中每个变量的含义如下：

1. varName: 变量或参数的名称；

2. Class：变量或参数的类别。如果自定义函数或存储过程有输入参数时，该参数类型将标记为“A”，语句块里的参数将标记为“L”；
3. lineNumber：声明变量或参数的行号；
4. Unique：标记该名称在自定义函数或者存储过程范围内是否唯一，如果名称不是唯一，调试器客户端将使用行号来区分同名的变量或参数；
5. isConst：标记变量或参数是否为常量；
6. notNull：标记变量或参数是否不为空；
7. dataTypeOID：该变量 sys_type 中对应的 OID；
8. value(rest of string)：变量的值。

3.70.4.2.15　pldbg_deposit_value 函数

功能描述

在调试过程中，将一个新值赋给指定的变量。此函数将 PLDBG_DEPOSIT 指令发送到 Targert Backend 端，服务器查询该变量然后修改变量的值，最后将修改成功与否的状态返回。

语法格式

```
FUNCTION pldbg_deposit_value( session INTEGER, varName TEXT, lineNumber INTEGER, value TEXT ) RETURNS
boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
varName	变量名
lineNumber	变量定义的行号
value	变量新值

返回值说明

新值设置成功返回 t，新值设置失败返回 f。

3.70.4.2.16　pldbg_get_stack 函数

功能描述

从 Targert Backend 端获取调用堆栈信息。此函数将 PLDBG_GET_STACK 指令发送到 Targert Backend 端，返回相应调用堆栈信息。

语法格式

```
FUNCTION pldbg_get_stack( session INTEGER ) RETURNS SETOF frame
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一组堆栈框架信息元组，包含：CallCount；targetName；funcOID；lineNumber；arguments(rest of string)。

1. CallCount: 堆栈深度。
2. targetName: 调用函数或存储过程的名称。
3. funcOID: 自定义函数或者存储过程 OID。
4. lineNumber: 当前停留的行号。
5. arguments(rest of string): 参数的信息。

3.70.4.2.17　pldbg_select_frame 函数

功能描述

将调试器焦点（当前在堆栈中的执行位置）更改为指定的帧。此函数将 PLDBG_SELECT_FRAME 命令发送到 Targert Backend，然后循环查询指定的帧，如果传入的帧不存在，将返回当前帧的信息，如果传入的帧信息存在，将返回相应的查询信息。

语法格式

```
FUNCTION pldbg_select_frame( session INTEGER, frame INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
frame	指定帧

返回值说明

此函数返回类型为“breakpoint”的元组，其中包含自定义函数或者存储过程 OID，以及当前目标在选定帧中停止的行号。

3.70.4.2.18　pldbg_get_source 函数

功能描述

从 Target Backend 获取自定义函数或存储过程正在执行的源代码。此函数将包含 PLDBG_GET_SOURCE，FUNCTIONOID 的字符串发送到 Targert Backend，查询源代码信息并返回。

语法格式

```
FUNCTION pldbg_get_source( session INTEGER, func OID ) RETURNS TEXT
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
func	自定义函数或存储过程的 OID

返回值说明

返回给定函数或存储过程的源代码信息。

3.70.4.2.19　plsql_extend_get_proc_ddl 函数

功能描述

用以支撑 pldebugger 获取函数、存储过程源码。

语法格式

```
FUNCTION plpgsql_debug.plsql_extend_get_proc_ddl(func OID) returns text
```

参数说明

参数名称	描述
func	函数、存储过程的 oid。

返回值说明

函数、存储过程的源码。

参数异常

参数名称	用例	结果
func	NULL	返回 NULL
	0	报错 invalid object oid
	不存在的 oid	报错 cache lookup failed for function

3.70.4.2.20　pldbg_get_proxy_info 函数

功能描述

获取服务器版本，Proxy Backend API 版本等信息。

语法格式

```
FUNCTION pldbg_get_proxy_info() RETURNS proxyInfo
```

返回值说明

返回服务器和代理 API 的版本信息。

3.70.4.2.21　局部断点调试示例

创建调试环境和用例：

```
--创建 pldebugger 环境
create extension pldbgapi;

--创建用来调试的存储过程
CREATE OR REPLACE PROCEDURE public.pro1()
language 'plpgsql'
AS $$
BEGIN
   raise notice '123';
   raise notice 'success';
END;
$$;
```

```
CREATE OR REPLACE PROCEDURE public.pro4()
LANGUAGE 'plpgsql'
AS
$$
declare
   i int := 2;
begin
   i = 3;
   raise notice 'i= %',i;
   i = 4;
   raise notice 'i= %',i;
   raise notice '0:ready.....';
   begin
       raise notice '1:ready.....';
       call pro1();
       begin
           raise notice 'success';
       end;
   end;
end;
$$;

--查询出需要调试的存储过程的 oid
select oid from sys_proc where proname = 'pro4';
 oid
-------
16918
(1 row)
```

application client：

```
--应用端开启局部监听
select plpgsql_oid_debug(16918);
plsql_oid_debug
-------------------
                0
(1 row)

--开始调试存储过程 pro4
call pro4();
NOTICE:  PLDBGBREAK:3     --该进程控制器转交给另一进程，直至调试结束，3 为连接的端口号
```

debugger client：

```
--调试端连接
select pldbg_attach_to_port(3);
pldbg_attach_to_port
----------------------
                    1
(1 row)

--查看源码
select pldbg_get_source(1,16918);
 pldbg_get_source
--------------------------------------
                                     +
declare                              +
    i int := 2;                      +
begin                                +
    i = 3;                           +
    raise notice 'i= %',i;           +
    i = 4;                           +
    raise notice 'i= %',i;           +
    raise notice '0:ready.....';     +
    begin                            +
        raise notice '1:ready.....';+
        call pro1();                 +
        begin                        +
            raise notice 'success'; +
        end;                         +
    end;                             +
end
(1 row)

--设置断点
select pldbg_set_breakpoint(1,16918,7);
pldbg_set_breakpoint
----------------------------
t
(1 row)

select pldbg_set_breakpoint(1,16918,12);
pldbg_set_breakpoint
----------------------
t
(1 row)

select pldbg_set_breakpoint(1,16918,15);
pldbg_set_breakpoint
```

```
----------------------
t
(1 row)

--获取当前设置的断点信息
select pldbg_get_breakpoints(1);
pldbg_get_breakpoints
-----------------------
(16918,-1,"")
(16918,12,"")
(16918,15,"")
(16918,7,"")
(4 rows)

--单步执行，遇到子程序进入
select pldbg_step_into(1);
pldbg_step_into
-----------------------
(16918,7,"pro4()")
(1 row)

select pldbg_step_into(1);
pldbg_step_into
-----------------------
(16918,8,"pro4()")
(1 row)

select pldbg_step_into(1);
pldbg_step_into
-----------------------
(16918,9,"pro4()")
(1 row)

--单步执行，遇到子程序跳过
select pldbg_step_over(1);
pldbg_step_over
-----------------------
(16918,10,"pro4()")
(1 row)

--继续执行程序，直到断点处或者程序结束
select pldbg_continue(1);
pldbg_continue
----------------------
(16918,13,"pro4()")
```

```
(1 row)

--当 application client 执行完毕后，debug client 进程会挂起，直到 application client 再次执行测试函数或存储过程
select pldbg_continue(1);
```

application client：

```
--应用端存储过程执行完成
call pro4();
NOTICE:  PLDBGBREAK:3
NOTICE:  i= 3
NOTICE:  i= 4
NOTICE:  0:ready.....
NOTICE:  1:ready.....
NOTICE:  123
NOTICE:  success
NOTICE:  success
CALL

--再次调起
call pro4();
```

debugger client：

```
--调试端进入调试状态，调用 pldbg_abort_target，终止调试
select pldbg_abort_target(1);
pldbg_abort_target
--------------------
t
(1 row)
```

application client：

```
--当 debug client 调用 select pldbg_abort_target(1) 后，执行被终止
call pro4();
ERROR:  canceling statement due to user request
CONTEXT:  PL/pgSQL        FUNCTION pro4() line 5 at assignment
```

3.70.4.2.22　全局断点调试示例

debugger client：

```
--调试端创建全局监听
select pldbg_create_listener();
```

```
pldbg_create_listener
-----------------------
                     1
(1 row)

--设置全局断点，未指定行号默认为-1，即 BEGIN 块的第一行
select pldbg_set_global_breakpoint(1,16918,null,null);
pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--等待测试函数被调用，无测试函数调用该进程会挂起
select pldbg_wait_for_target(1);
```

application client：

```
--应用端执行测试函数
call pro4();
```

debugger client：

```
--测试函数调用后，返回测试函数 session 的进程号，开始调试
select pldbg_wait_for_target(1);
pldbg_wait_for_target
-----------------------------
                      235966
(1 row)

--设置局部断点
select pldbg_set_breakpoint(1,16918,5);
pldbg_set_breakpoint
----------------------
t
(1 row)

--设置全局断点
select pldbg_set_global_breakpoint(1,16918,12,null);
pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--获取变量信息
select pldbg_get_variables(1);
```

```
pldbg_get_variables
---------------------------
(i,L,3,t,f,f,23,2)
(1 row)

--单步进入
select pldbg_step_into(1);
pldbg_step_into
-----------------------
(16918,7,"pro4()")
(1 row)


select pldbg_step_into(1);
pldbg_step_into
-----------------------
(16918,8,"pro4()")
(1 row)

--获取修改后的变量信息
select pldbg_get_variables(1);
pldbg_get_variables
---------------------
(i,L,3,t,f,f,23,3)
(1 row)

--继续执行到断点处
select pldbg_continue(1);
  pldbg_continue
---------------------
(16918,13,"pro4()")
(1 row)

--单步进入子程序 pro1
select pldbg_step_into(1);
 pldbg_step_into
--------------------
(16917,4,"pro1()")
(1 row)

--获取堆栈信息
select pldbg_get_stack(1);
   pldbg_get_stack
--------------------------
(0,"pro1()",16917,3,"")
```

```
(1,"pro4()",16918,12,"")
(2 rows)


--追溯堆栈信息
select pldbg_select_frame(1,1);
pldbg_select_frame
---------------------
(16918,13,"pro4()")
(1 row)


--继续执行到程序结束后挂起，等待下一次程序调用
select pldbg_continue(1);
  pldbg_continue
---------------------
(16918,15,"pro4()")
(1 row)

select pldbg_continue(1);
```

application client：

```
--应用端程序执行完成
call pro4();
NOTICE:  i= 3
NOTICE:  i= 4
NOTICE:  0:ready
NOTICE:  1:ready
NOTICE:  123
NOTICE:  success
NOTICE:  success
CALL
```

3.70.4.2.23　修改调试时数据及删除断点调试示例

创建测试环境和用例：

```
--创建 pldebugger 环境
create extension pldbgapi;

--创建用来调试的存储过程
create or replace procedure pro5(i int)
LANGUAGE 'plpgsql'
as $$
```

```
declare
   j int;
begin
   raise notice 'i = %',i;
   j = i;
   raise notice 'j = %',j;
   j = 1;
   raise notice 'j = %',j;
   i = 2;
   raise notice 'i = %',i;
end;
$$;

--查询出需要调试的存储过程的 oid
select oid from sys_proc where proname = 'pro5';
 oid
-------
16919
(1 row)
```

debugger client：

```
--创建全局监听
select pldbg_create_listener();
pldbg_create_listener
-----------------------
                     1
(1 row)

--设置全局断点
select pldbg_set_global_breakpoint(1,16919,null,null);
pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--等待测试存储过程调用
select pldbg_wait_for_target(1);
```

application client：

```
call pro5(3);
```

debugger client：

```
--子程序已调用，开始调试
select pldbg_wait_for_target(1);
pldbg_wait_for_target
-----------------------------
                   313888
(1 row)

--返回建立连接时断点所在行的信息（全局断点行号为 null，默认当前在 BEGIN 块的第一行）
select pldbg_wait_for_breakpoint(1);
pldbg_wait_for_breakpoint
---------------------------
(16919,6,"pro5(integer)")
(1 row)

--单步执行
select pldbg_step_into(1);
  pldbg_step_into
---------------------------
(16919,7,"pro5(integer)")
(1 row)

--获取存储过程源码
select pldbg_get_source(1,16919);
  pldbg_get_source
------------------------------
                             +
declare                      +
    j int;                   +
begin                        +
    raise notice 'i = %',i;+
    j = i;                   +
    raise notice 'j = %',j;+
    j = 1;                   +
    raise notice 'j = %',j;+
    i = 2;                   +
    raise notice 'i = %',i;+
end;
(1 row)

--查看变量的值
select pldbg_get_variables(1);
 pldbg_get_variables
-----------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,NULL)
```

```
(2 rows)

--修改变量 j 的值
select pldbg_deposit_value(1, 'j', 3, '10');
pldbg_deposit_value
---------------------
f
(1 row)

--再次查看变量 j 的值，已被改变
select pldbg_get_variables(1);
pldbg_get_variables
---------------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,10)
(2 rows)

--单步执行，变量 j 在程序中被赋值
select pldbg_step_into(1);
  pldbg_step_into
----------------------------
(16919,8,"pro5(integer)")
(1 row)

--查看变量 j 的值，已改变
select pldbg_get_variables(1);
pldbg_get_variables
---------------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,3)
(2 rows)

--设置局部断点
select pldbg_set_breakpoint(1,16919,5);
pldbg_set_breakpoint
----------------------
t
(1 row)

select pldbg_set_breakpoint(1,16919,7);
pldbg_set_breakpoint
----------------------
t
(1 row)
```

```
select pldbg_set_breakpoint(1,16919,9);
pldbg_set_breakpoint
----------------------
t
(1 row)

--查看断点信息
select pldbg_get_breakpoints(1);
pldbg_get_breakpoints
-----------------------
(16919,-1,"")
(16919,7,"")
(16919,9,"")
(16919,-1,"")
(16919,5,"")
(5 rows)

--删除断点
select pldbg_drop_breakpoint(1,16919,5);
pldbg_drop_breakpoint
-----------------------
t
(1 row)

select pldbg_drop_breakpoint(1,16919,9);
pldbg_drop_breakpoint
-----------------------
t
(1 row)

--查看断点信息
select pldbg_get_breakpoints(1);
pldbg_get_breakpoints
-----------------------------
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
(3 rows)

--设置全局断点
select pldbg_set_global_breakpoint(1,16919,11,null);
pldbg_set_global_breakpoint
-----------------------------
t
(1 row)
```

```
--查看断点信息
select pldbg_get_breakpoints(1);
pldbg_get_breakpoints
-----------------------------
(16919,11,"")
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
(4 rows)

--设置断点行号为 0 时无效
select pldbg_drop_breakpoint(1,16919,0);
pldbg_drop_breakpoint
-----------------------
f
(1 row)

--设置断点行号为 NULL 时无效
select pldbg_drop_breakpoint(1,16919,NULL);
pldbg_drop_breakpoint
-----------------------

(1 row)

--查看断点信息，0 和 NULL 断点都未设置成功
select pldbg_get_breakpoints(1);
pldbg_get_breakpoints
-----------------------
(16919,11,"")
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
(4 rows)

--查看版本信息
select * from pldbg_get_proxy_info();
                                          serverversionstr                                                |
serverversionnum | proxyapiver | serverprocessid
-----------------------------------------------------------------------------------------------------------+--
----------------+-------------+-----------------
PostgreSQL 12.1 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-36), 64-bit |
       120001 |           3 |          311197
(1 row)
```

3.70.5　插件 pldbgapi 卸载方法

卸载 pldbgapi 插件接口，只需要执行以下命令：

```
-- 删除插件
drop extension pldbgapi;
```

3.70.6　插件 pldbgapi 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.1 升级到 1.2：

```
ALTER EXTENSION pldbgapi UPDATE TO '1.2';
```

3.71　plsql_pldbgapi

3.71.1　插件 plsql_pldbgapi 简介

插件 plsql_pldbgapi 是 KingbaseES 的一个扩展插件。主要用于 PL/SQL 语言的存储过程、函数、包、触发器函数、匿名块、object type 方法的代码调试。

- 插件名为 plsql_pldbgapi
- 插件版本 V1.2

3.71.2　插件 plsql_pldbgapi 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。因此只需要在客户端工具执行 create extension 命令即可。

```
-- 创建插件
create extension plsql_pldbgapi;
```

备注:

在当前模式和 plsql_debug 模式下均创建了一套调试接口，因此在任意模式下调试接口可通过模式限定调用。

3.71.3　插件 plsql_pldbgapi 的参数配置

plsql_pldbgapi 扩展插件无需配置任何参数。

3.71.4　插件 plsql_pldbgapi 使用方法

3.71.4.1　plsql_debugger 调试流程

以下详细介绍其局部断点调试和全局断点调试流程的命令行调试流程。只是在目标后端与调试端建立连接的流程有稍微差别，实际的调试的流程总体一致。

plsql_debugger 的调试操作是建立在目标后端和调试端正常通信的前提下，其详细的调试流程如下：

1. 创建监听，局部断点调试根据调试对象的 OID 去建立监听，全局断点调试则不需要带调试对象的 OID 而是创建全局监听。
2. 创建连接，全局调试与局部调试的连接方式稍有不同，对于局部调试，首先是调用待调试的对象，让目标后端进程以通信服务器端的身份发起连接，然后在调试端调用端口连接函数 plsql_pldbg_attah_to_port 让调试端以通信客户端的身份与目标后端建立连接；对于全局断点调试，先设置全局断点，然后调用 plsql_pldbg_wait_for_target 函数发起连接并等待接受目标后端的连接。此时调用待调试的对象就可以与调试端建立通信连接。
3. 调试设置，可以根据需要设置和删除断点信息，包括局部断点和全局断点。
4. 执行调试操作，plsql_debugger 提供了三种调试执行的方式，分别为单步进入（step into）、单步跳过（step over）以及继续执行（continue）。
5. 信息查询，在调试执行的过程中，可以查询源代码信息，查询和修改变量值，查询调用堆栈以及当前系统版本等。

3.71.4.2　接口函数

plsql_debugger 调试工具支持对 PL/SQL 语言的存储过程、函数、包、触发器函数、匿名块、object type 方法进行代码调试操作，包括操作断点、执行代码、调试数据、查看调用堆栈、查看源码信息、查看版本信息等功能。并提供局部断点调试与全局断点调试两种调试模式。

调试接口函数列表如下：

表 3.71.1: 接口函数列表

接口函数	简介
plsql_oid_debug 函数	创建局部监听
plsql_oid_debug_number 函数	对指定行设置断点

-续表-

接口函数	简介
plsql_pldbg_attach_to_port 函数	连接目标后端
plsql_pldbg_create_listener 函数	创建全局监听
plsql_pldbg_set_global_breakpoint 函数	设置全局断点
plsql_pldbg_wait_for_target 函数	等待连接
plsql_pldbg_set_breakpoint 函数	设置断点
plsql_pldbg_get_breakpoints 函数	查询断点
plsql_pldbg_drop_breakpoint 函数	删除断点
plsql_pldbg_wait_for_breakpoint 函数	返回建立连接时断点所在行的信息
plsql_pldbg_step_into 函数	单步进入（step into）
plsql_pldbg_step_over 函数	单步跳过（step over）
plsql_pldbg_continue 函数	继续执行（continue）
plsql_pldbg_step_out 函数	单步跳出（step out）
plsql_pldbg_step_end 函数	步至方法结束
plsql_pldbg_run_to_next_exception 函数	进入下一个异常
plsql_pldbg_run_to_cursor_line 函数	运行至光标处
plsql_pldbg_alter_breakpoint_hit_number 函数	修改断点击中次数
plsql_pldbg_alter_breakpoint_msg 函数	修改断点的描述信息
plsql_pldbg_alter_breakpoint_status 函数	修改指定断点的状态
plsql_pldbg_alter_all_breakpoint_status 函数	修改全部断点的状态
plsql_pldbg_set_breakpoint_src 函数	插入或更新源断点的状态和信息
plsql_pldbg_get_breakpoints_src 函数	获取当前会话的本地源断点信息
plsql_pldbg_set_exception_breakpoint_src 函数	插入或更新异常断点的状态和信息
plsql_pldbg_get_exception_breakpoints_src 函数	获取所有异常断点的状态和信息
plsql_pldbg_drop_exception_breakpoint 函数	删除异常断点

-续表-

接口函数	简介
plsql_pldbg_drop_all_breakpoint 函数	删除所有本地源断点和所有异常断点
plsql_pldbg_abort_target 函数	中止执行
plsql_pldbg_get_variables 函数	查看变量
plsql_pldbg_deposit_value 函数	修改变量
plsql_pldbg_get_stack 函数	查看堆栈信息
plsql_pldbg_select_frame 函数	追溯堆栈信息
plsql_pldbg_get_source 函数	查询源代码信息
plsql_extend_get_proc_ddl 函数	获取函数、存储过程源码
plsql_pldbg_get_proxy_info 函数	查询当前系统版本信息
plsql_pldbg_set_connection_timeout 函数	目标会话等待连接的超时设置
plsql_pldbg_set_operation_timeout 函数	目标会话等待调试会话操作命令的超时设置

3.71.4.2.1　plsql_oid_debug 函数

功能描述

在同一个 session 下负责监听该调试对象，创建监听状态。该函数在建立监听的过程中会设置一个系统默认断点，行号为‘-1’。

语法格式

```
FUNCTION plsql_oid_debug (functionOID OID) RETURNS INTEGER
```

参数说明

参数名称	描述
functionOID	存储过程、函数、包、触发器函数、objecttype 方法、匿名块的 oid，其中匿名块为 9999

返回值说明

调用成功返回 0，调用失败无返回值，存在报错信息如下：

1. 当 OID 输入为 0 时报错信息：no target specified。
2. 当 OID 无效时报错信息：cache lookup failed for function。
3. 当不是 owner 或 superuser 时报错信息：must be owner or superuser to create a breakpoint。

3.71.4.2.2　plsql_oid_debug_number 函数

功能描述

对指定调试对象的指定行下监听断点。

语法格式

```
FUNCTION plsql_oid_debug_number(func OID, linenumber INTEGER default -1) RETURNS INTEGER
```

参数说明

参数名称	描述
func	调试对象的 OID
linenumber	调试对象的指定行，默认值为-1

返回值说明

断点设置成功则返回 0；断点设置失败则会进行报错。

参数异常说明

参数名称	用例	结果
fucn	0	报错 no target specified
	不存在 PLSQL 对象 OID	报错 cache lookup failed xxx
	不是当前 oid 的 owner 且不是超级用户	must be owner or superuser to create a breakpoint
	NULL	返回 NULL
linenumber	NULL	返回 NULL

3.71.4.2.3　plsql_pldbg_attach_to_port 函数

功能描述

此函数用于响应 PLDBGBREAK 端口号。该函数连接到监听给定端口的 Target Backend 端，从而形成 Debugger Client，启动调试过程，等待后续调试工作。一个 Debugger Client 能够响应多个不同 PLDBGBREAK 端口，即可以支持多个并发会话。

语法格式

```
FUNCTION plsql_pldbg_attach_to_port(portNumber INTEGER) RETURNS INTEGER
```

参数说明

参数名称	描述
portNumber	调用调试对象显示的进程 ID 号

返回值说明

返回标识此特定调试的会话句柄。

3.71.4.2.4　plsql_pldbg_create_listener 函数

功能描述

创建一个 TCP 端口，为全局断点创建一个监听器，方便后续函数连接到该目标进程。

语法格式

```
FUNCTION plsql_pldbg_create_listener() RETURNS INTEGER
```

返回值说明

调用成功返回 debug_session 号。

3.71.4.2.5　plsql_pldbg_set_global_breakpoint 函数

功能描述

创建一个全局断点。

语法格式

```
FUNCTION plsql_pldbg_set_global_breakpoint(session INT, FUNCTION OID, lineNumber INT, targetPID INT) RETURNS
boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
FUNCTION	调试对象的 OID
lineNumber	行号（可为 NULL）
targetPID	目标端 PID（可为 NULL）

返回值说明

调用成功返回 t，调用失败会报错。

3.71.4.2.6　plsql_pldbg_wait_for_target 函数

功能描述

发起 socket 连接，等待目标后端进程调用调试对象来连接到调试端，以触发全局断点。

语法格式

```
FUNCTION plsql_pldbg_wait_for_target(session INTEGER) RETURNS INTEGER
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

调用成功返回 target backend 的 PID。

3.71.4.2.7　plsql_pldbg_set_breakpoint 函数

功能描述

在目标进程中设置一个“局部”断点。

此函数将 PLDBG_SET_BREAKPOINT 指令发送到 Targert Backend 端，在调试对象的指定位置设置断点，然后等待下一个可执行语句到达该目标断点。

语法格式

```
FUNCTION plsql_pldbg_set_breakpoint(session INTEGER, FUNCTION OID, lineNumber INTEGER) RETURNS boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄需要设置的断点行号
FUNCTION	调试对象的 OID
lineNumber	若行号传入为 NULL，不会报错也不会设置断点

返回值说明

断点设置成功返回为 t，断点设置失败返回为 f。

3.71.4.2.8　plsql_pldbg_get_breakpoints 函数

功能描述

显示设置的所有断点。此函数将 PLDBG_GET_BREAKPOINTS 指令发送到 Targert Backend 端，返回查询到的断点元组信息。如果没有断点信息，则返回为空。

语法格式

```
FUNCTION plsql_pldbg_get_breakpoints( session   INTEGER ) RETURNS SETOF breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一组“breakpoint”元组信息，该元组信息包括调试对象的 OID，断点设置的行号 lineNumber，Target 端的名字 targetName。

3.71.4.2.9　plsql_pldbg_drop_breakpoint 函数

功能描述

在目标进程中删除一个指定行号的断点。此函数将 PLDBG_CLEAR_BREAKPOINT 指令发送到 Targert Backend 端，删除指定断点信息，并返回删除成功与否的状态。

语法格式

```
FUNCTION plsql_pldbg_drop_breakpoint( session   INTEGER,    FUNCTION OID, lineNumber INTEGER ) RETURNS
boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
FUNCTION	调试对象的 OID
lineNumber	所要删除断点的行号

返回值说明

断点删除成功返回 t，断点删除失败返回 f。

3.71.4.2.10　plsql_pldbg_wait_for_breakpoint 函数

功能描述

调用该函数，可以返回刚建立连接时断点所在行的信息。

语法格式

```
FUNCTION plsql_pldbg_wait_for_breakpoint( session   INTEGER ) RETURNS  breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为一个“breakpoint”的元组，该元组包含调试对象的 OID 以及刚建立连接时停止的行号。

3.71.4.2.11　plsql_pldbg_step_into 函数

功能描述

单步执行，遇到子函数或子存储过程会进入到相应的部分。此函数将 PLDBG_STEP_INTO 命令发送到 Targert Backend，等待目标后端进程执行语句后，等待下一个可执行语句到达目标。

语法格式

```
FUNCTION plsql_pldbg_step_into( session   INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组（但不一定是真正的断点信息，因为单步执行有可能遇到断点，有可能没有遇见断点）。该元组包含调试对象的 OID 以及目标当前停止的行号，以及调试对象名。

3.71.4.2.12　plsql_pldbg_step_over 函数

功能描述

单步执行，遇见子函数或子存储过程会跳过内部执行过程。此函数将 PLDBG_STEP_OVER 命令发送到 Targert Backend，待目标后端进程执行语句后，等待下一个可执行语句到达目标，并将执行后需要返回的结果返回。

语法格式

```
FUNCTION plsql_pldbg_step_over( session   INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组（但不一定是真正的断点信息，因为单步执行有可能遇到断点，有可能没有遇见断点）。该元组包含调试对象的 OID 以及目标当前停止的行号，以及调试对象名。

3.71.4.2.13　plsql_pldbg_continue 函数

功能描述

从当前位置继续执行，直到遇见下一个断点，如果没有断点，将直接执行到调试对象的最后。此函数将“PLDBG_CONTINUE”指令发送到 Targert Backend 端，目标后端进程执行，然后等待目标到达下一个断点，并返回相应的结果信息。

语法格式

```
FUNCTION plsql_pldbg_continue( session   INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

此函数返回类型为“breakpoint”的元组，该元组包含调试对象的 OID 以及目标当前停止的行号，以及调试对象名。

3.71.4.2.14　plsql_pldbg_step_out 函数

功能描述

与断点无关；跳回上层堆栈的下一条语句。

语法格式

```
FUNCTION plsql_pldbg_step_out( session INTEGER ) RETURNS plsql_breakpoint
```

参数说明

参数名称	描述
session	会话句柄

返回值说明

返回一个 plsql_breakpoint 结构，用来表示位置信息。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

3.71.4.2.15　plsql_pldbg_step_end 函数

功能描述

断点优先；步至当前堆栈末尾语句。最后一条语句作用类似于 step over。

语法格式

```
FUNCTION plsql_pldbg_step_end( session INTEGER ) RETURNS plsql_breakpoint
```

参数说明

参数名称	描述
session	会话句柄

返回值说明

返回一个 plsql_breakpoint 结构，用来表示位置信息。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

3.71.4.2.16　plsql_pldbg_run_to_next_exception 函数

功能描述

断点优先；停在触发异常语句的地方（包括异常处理模块的语句），能够停在未生成调试信息的区域。

语法格式

```
FUNCTION plsql_pldbg_run_to_next_exception( session INTEGER ) RETURNS plsql_breakpoint
```

参数说明

参数名称	描述
session	会话句柄

返回值说明

返回一个 plsql_breakpoint 结构，用来表示位置信息。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

3.71.4.2.17　plsql_pldbg_run_to_cursor_line 函数

功能描述

无视断点，直接调试目标行代码。调试途中的断点，状态保持不变。必须要生成调试信息才可以进行调试。

语法格式

```
FUNCTION plsql_pldbg_run_to_cursor_line( session INTEGER, func OID, linenumber INTEGER ) RETURNS plsql_
breakpoint
```

参数说明

参数名称	描述
session	会话句柄
func	调试对象的 OID
linenumber	调试对象的指定行

返回值说明

返回一个 plsql_breakpoint 结构，用来表示位置信息。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
func	NULL	返回 NULL
	嵌套函数 OID	报错 Debugging nested functions requires the use of the outeroid
linenumber	NULL	返回 NULL

3.71.4.2.18　plsql_pldbg_alter_breakpoint_hit_number 函数

功能描述

修改断点击中次数，当断点击中的总次数与设定断点次数相同则进行中断。击中断点后，后期则每次击中断点会进行中断。

语法格式

```
FUNCTION plsql_pldbg_alter_breakpoint_hit_number( session INTEGER, func OID, linenumber INTEGER, hitnumber
INTEGER) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄
func	调试对象的 OID
linenumber	调试对象的指定行
hitnumber	设置的断点击中次数，小于 1 则设置为 1

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
func	NULL	返回 NULL
linenumber	NULL	返回 NULL
hitnumber	<1	hitnumber 设置为 1

使用说明

1. 未找到断点信息返回 false。
2. 无论执行什么操作，只要该行上有断点就需要计入内部执行次数。
3. 再次调用 PLSQL 时，当前断点内部击中次数会重新计数。
4. 更新断点击中次数后，则会更新断点内部击中次数。
5. 该方式也被称为断点传送计数。

3.71.4.2.19　plsql_pldbg_alter_breakpoint_msg 函数

功能描述

修改断点的描述信息。

语法格式

```
FUNCTION plsql_pldbg_alter_breakpoint_msg( session INTEGER, func OID, linenumber INTEGER, breakpointdesc
text) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄
func	调试对象的 OID
linenumber	调试对象的指定行
breakpointdesc	断点的描述信息，可设置为 NULL

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
func	NULL	返回 NULL
linenumber	NULL	返回 NULL

3.71.4.2.20　plsql_pldbg_alter_breakpoint_status 函数

功能描述

修改指定断点的状态。

语法格式

```
FUNCTION plsql_pldbg_alter_breakpoint_status( session INTEGER, func OID, linenumber INTEGER,
breakpointstatus boolean) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄
func	调试对象的 OID
linenumber	调试对象的指定行
breakpointstatus	断点状态

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
func	NULL	返回 NULL
linenumber	NULL	返回 NULL
breakpointstatus	NULL	返回 NULL

使用说明

断点状态分为启用与禁用，当断点状态为禁用时，则断点击中后不会进行中断。若指定断点不存在则返回 false。

3.71.4.2.21　plsql_pldbg_alter_all_breakpoint_status 函数

功能描述

修改全部断点的状态。

语法格式

```
FUNCTION plsql_pldbg_alter_all_breakpoint_status( session INTEGER, breakpointstatus boolean) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄
breakpointstatus	设置全部断点的状态

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

-续表-

参数名称	用例	结果
breakpointstatus	NULL	返回 NULL

3.71.4.2.22　plsql_pldbg_set_breakpoint_src 函数

功能描述

插入或更新源断点的状态、击中次数、描述信息、条件信息。

语法格式

```
FUNCTION plsql_pldbg_set_breakpoint_src(session INTEGER, func OID, linenumber INTEGER, breakpointstatus
boolean, hitnumber INTEGER, breakpointdesc text, breakpointcond text default NULL) returns boolean
```

参数说明

参数名称	描述
session	会话句柄
func	对象的 OID
linenumber	行号
breakpointstatus	断点状态
hitnumber	断点击中次数
breakpointdesc	断点描述信息
breakpointcond	断点条件，默认值为 NULL

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL

-续表-

参数名称	用例	结果
	会话句柄无效	报错 invalid session handle
func	NULL	返回 NULL
	嵌套函数 OID	报错 Debugging nested functions requires the use of the outeroid
linenumber	NULL	返回 NULL
breakpointstatus	NULL	返回 NULL
hitnumber	NULL	返回 NULL
	<1	hitnumber 设置为 1
breakpointdesc	NULL	将断点描述信息设置为空串

使用说明

不支持条件断点，因此 breakpointcond 不生效。更新源断点后，内部当前击中次数会进行重置。

3.71.4.2.23　plsql_pldbg_get_breakpoints_src 函数

功能描述

获取当前会话的本地源断点信息。

语法格式

```
FUNCTION plsql_pldbg_get_breakpoints_src(in session INTEGER, out func OID, out linenumber INTEGER, out
breakpointstatus boolean, out hitnumber INTEGER, out breakpointdesc text, out breakpointcond text) returns
setof record
AS  '$libdir/plsql_plugin_debugger' LANGUAGE C STRICT
```

参数说明

参数名称	描述
session	会话句柄
func	对象的 OID
linenumber	行号
breakpointstatus	断点状态

-续表-

参数名称	描述
hitnumber	断点击中次数
breakpointdesc	断点描述信息
breakpointcond	断点条件

返回值说明

返回一个多行多列的断点信息数据。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

3.71.4.2.24　plsql_pldbg_set_exception_breakpoint_src 函数

功能描述

插入或更新异常断点的状态、击中次数、描述信息、条件信息。

语法格式

```
FUNCTION plsql_pldbg_set_exception_breakpoint_src(session INTEGER, exceptionsqlcode INTEGER, linenumber
INTEGER, breakpointstatus boolean, hitnumber INTEGER, breakpointdesc text, breakpointcond text default NULL)
returns boolean
```

参数说明

参数名称	描述
session	会话句柄
exceptionsqlcode	异常的 sqlcode，当为 0 时可以匹配所有异常
linenumber	行号，必须填写-1
breakpointstatus	异常断点状态

-续表-

参数名称	描述
hitnumber	异常断点击中次数
breakpointdesc	异常断点描述信息
breakpointcond	异常断点条件，默认值为 NULL

返回值说明

设置成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
exceptionsqlcode	NULL	返回 NULL
linenumber	NULL	返回 NULL
	不等于-1	报错 The line number of an exception breakpoint can only be -1.
breakpointstatus	NULL	返回 NULL
hitnumber	NULL	返回 NULL
	<1	hitnumber 设置为 1
breakpointdesc	NULL	将断点描述信息设置为空串

使用说明

1. 不支持条件断点，因此 breakpointcond 不生效。
2. 通过“select * from SYS_GET_PREDEFINED_EXCEPTION_DETAIL()”查询异常 sqlcode。
3. 当 exceptionsqlcode 为 0 时，能够匹配所有的异常。
4. plsql_pldbg_alter_all_breakpoint_status、plsql_pldbg_drop_all_breakpoint 会影响异常断点。
5. 异常断点更新后，内部击中次数会进行重置。

6. 再次执行 PLSQL 时会重新计数断点击中次数。

3.71.4.2.25　plsql_pldbg_get_exception_breakpoints_src 函数

功能描述

获取所有异常断点的状态、击中次数、描述信息、条件信息。

语法格式

```
FUNCTION plsql_pldbg_get_exception_breakpoints_src(in session INTEGER, out exceptionsqlcode INTEGER, out
linenumber INTEGER, out breakpointstatus boolean, out hitnumber INTEGER, out breakpointdesc text, out
breakpointcond text) returns setof record
```

参数说明

参数名称	描述
session	会话句柄
exceptionsqlcode	异常的 sqlcode
linenumber	行号
breakpointstatus	异常断点状态
hitnumber	异常断点击中次数
breakpointdesc	异常断点描述信息
breakpointcond	异常断点条件

返回值说明

返回一个多行多列的断点信息数据。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

使用说明

不支持条件断点，因此 breakpointcond 不生效。

3.71.4.2.26　plsql_pldbg_drop_exception_breakpoint 函数

功能描述

删除异常断点。

语法格式

```
FUNCTION plsql_pldbg_drop_exception_breakpoint( session INTEGER, exceptionsqlcode INTEGER, linenumber
INTEGER ) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄
exceptionsqlcode	异常的 sqlcode
linenumber	异常断点的行号必须为

返回值说明

删除成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle
exceptionsqlcode	NULL	返回 NULL
	不等于-1	报错 The line number of an exception breakpoint can only be -1
linenumber	NULL	返回 NULL

使用说明

不支持条件断点，因此 breakpointcond 不生效。

3.71.4.2.27　plsql_pldbg_drop_all_breakpoint 函数

功能描述

删除所有本地源断点和所有异常断点。

语法格式

```
FUNCTION plsql_pldbg_drop_all_breakpoint(session INTEGER) RETURNS boolean
```

参数说明

参数名称	描述
session	会话句柄

返回值说明

删除成功返回 true，否则返回 false。

参数异常说明

参数名称	用例	结果
session	NULL	返回 NULL
	会话句柄无效	报错 invalid session handle

使用说明

不支持条件断点，因此 breakpointcond 不生效。

3.71.4.2.28　plsql_pldbg_abort_target 函数

功能描述

终止当前的执行。此函数将 PLDBG_ABORT 命令发送到 Targert Backend，然后终止当前的执行，并返回终止成功与否的状态。

语法格式

```
FUNCTION plsql_pldbg_abort_target( session   INTEGER ) RETURNS SETOF boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

终止成功返回 t。

3.71.4.2.29　plsql_pldbg_get_variables 函数

功能描述

获取一个变量或者参数的信息列表。此函数将 PLDBG_GET_VARIABLES 指令发送到 Targert Backend 端，服务器查询到所有变量的信息后返回变量或参数的信息列表。

语法格式

```
FUNCTION plsql_pldbg_get_variables( session   INTEGER ) RETURNS SETOF var
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一个 var 元组集合，包含信息：VarName；Class；lineNumber；Unique；isConst；notNull；dataTypeOID；value(rest of string)。其中每个变量的含义如下：

1. varName: 变量或参数的名称；
2. Class：变量或参数的类别。如果调试对象有输入参数时，该参数类型将标记为“A”，语句块里的参数将标记为“L”；
3. lineNumber：声明变量或参数的行号；
4. Unique：标记该名称在调试对象范围内是否唯一，如果名称不是唯一，调试器客户端将使用行号来区分同名的变量或参数；
5. isConst：标记变量或参数是否为常量；
6. notNull：标记变量或参数是否不为空；
7. dataTypeOID：该变量 sys_type 中对应的 OID；

8. value(rest of string)：变量的值。

3.71.4.2.30　plsql_pldbg_deposit_value 函数

功能描述

在调试过程中，将一个新值赋给指定的变量。此函数将 PLDBG_DEPOSIT 指令发送到 Targert Backend 端，服务器查询该变量然后修改变量的值，最后将修改成功与否的状态返回。

语法格式

```
FUNCTION plsql_pldbg_deposit_value( session   INTEGER, varName TEXT, lineNumber INTEGER, value TEXT )
RETURNS boolean
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
varName	变量名
lineNumber	变量定义的行号
value	变量新值

返回值说明

新值设置成功返回 t，新值设置失败返回 f。

3.71.4.2.31　plsql_pldbg_get_stack 函数

功能描述

从 Targert Backend 端获取调用堆栈信息。此函数将 PLDBG_GET_STACK 指令发送到 Targert Backend 端，返回相应调用堆栈信息。

语法格式

```
FUNCTION plsql_pldbg_get_stack( session   INTEGER ) RETURNS SETOF frame
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄

返回值说明

返回一组堆栈框架信息元组，包含：CallCount；targetName；funcOID；lineNumber；arguments(rest of string)。

1. CallCount: 堆栈深度。
2. targetName: 调用调试对象的名称。
3. funcOID: 调试对象 OID。
4. lineNumber: 当前停留的行号。
5. arguments(rest of string): 参数的信息。

3.71.4.2.32　plsql_pldbg_select_frame 函数

功能描述

将调试器焦点（当前在堆栈中的执行位置）更改为指定的帧。此函数将 PLDBG_SELECT_FRAME 命令发送到 Targert Backend，然后循环查询指定的帧，如果传入的帧不存在，将返回当前帧的信息，如果传入的帧信息存在，将返回相应的查询信息。

语法格式

```
FUNCTION plsql_pldbg_select_frame( session   INTEGER, frame INTEGER ) RETURNS breakpoint
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
frame	指定帧

返回值说明

此函数返回类型为“breakpoint”的元组，其中包含调试对象 OID，以及当前目标在选定帧中停止的行号。

3.71.4.2.33　plsql_pldbg_get_source 函数

功能描述

从 Target Backend 获取存储过程、函数、包、触发器函数、object type 方法、匿名块正在执行的源代码。此函数将包含 PLDBG_GET_SOURCE，functionOID 的字符串发送到 Targert Backend，查询源代码信息并返回。

语法格式

```
FUNCTION plsql_pldbg_get_source( session INTEGER, func OID ) RETURNS TEXT
```

参数说明

参数名称	描述
session	标识特定调试的会话句柄
func	调试对象的 OID

返回值说明

返回给定调试对象的源代码信息。

3.71.4.2.34　plsql_extend_get_proc_ddl 函数

功能描述

用以支撑 plsql_pldebugger 获取函数、存储过程源码。

语法格式

```
FUNCTION plsql_debug.plsql_extend_get_proc_ddl(func OID) returns text
```

参数说明

参数名称	描述
func	函数、存储过程的 oid。

返回值说明

函数、存储过程的源码。

参数异常说明

参数名称	用例	结果
func	NULL	返回 NULL
	0	报错 invalid object oid
	不存在的 oid	报错 cache lookup failed for function

3.71.4.2.35　plsql_pldbg_get_proxy_info 函数

功能描述

获取服务器版本，Proxy Backend API 版本等信息。

语法格式

```
FUNCTION plsql_pldbg_get_proxy_info() RETURNS proxyInfo
```

返回值说明

返回服务器和代理 PAI 的版本信息。

3.71.4.2.36　plsql_pldbg_set_connection_timeout 函数

功能描述

目标会话等待连接的超时设置。若目标会话等待超时，则进行报错处理。仅在目标端执行。

语法格式

```
FUNCTION plsql_pldbg_set_connection_timeout(timeout INTEGER) RETURNS int
```

参数说明

参数名称	描述
timeout	目标会话等待连接的超时设置。单位为秒。

返回值说明

返回设置的时间。

参数异常说明

参数名称	用例	结果
timetout	<0	返回 NULL
	NULL	返回 NULL

使用说明

内部默认连接超时为 1 小时。timeout 设置为 0 为不限制时间。超时报错 select() timeout in plsql_pldbgapi。

3.71.4.2.37　plsql_pldbg_set_operation_timeout 函数

功能描述

调试连接后，目标会话等待调试会话的操作超时设置。仅在目标端执行。

语法格式

```
FUNCTION plsql_pldbg_set_operation_timeout(timeout INTEGER) RETURNS int
```

参数说明

参数名称	描述
timeout	目标会话等待调试会话操作命令的超时设置。单位为秒。

返回值说明

返回设置的时间。

参数异常说明

参数名称	用例	结果
timeout	<0	返回 NULL
	NULL	返回 NULL

使用说明

内部默认不限制时间。timeout 设置为 0 为不限制时间。超时报错 errors communicating with the proxy。

3.71.4.2.38　局部断点调试示例

创建调试环境和用例：

```
--创建 plsql_debugger 环境
create extension plsql_pldbgapi;

--创建用来调试的存储过程
\set SQLTERM /
CREATE OR REPLACE PROCEDURE public.pro1() AS
```

```
BEGIN
   raise notice '123';
   raise notice 'success';
END;
/

CREATE OR REPLACE PROCEDURE public.pro4() AS
declare
   i int := 2;
begin
   i = 3;
   raise notice 'i= %',i;
   i = 4;
   raise notice 'i= %',i;
   raise notice '0:ready.....';
   begin
       raise notice '1:ready.....';
       call pro1();
       begin
           raise notice 'success';
       end;
   end;
end;
/
\set SQLTERM ;

--查询出需要调试的存储过程的 oid
select oid from sys_proc where proname = 'pro4';
 oid
-------
16918
(1 row)
```

application client：

```
--应用端开启局部监听
select plsql_oid_debug(16918);
plsql_oid_debug
-------------------
                0
(1 row)

--开始调试存储过程 pro4
call pro4();
NOTICE:  PLDBGBREAK:3     --该进程控制器转交给另一进程，直至调试结束，3 为连接的端口号
```

debugger client：

```
--调试端连接
select plsql_pldbg_attach_to_port(3);
plsql_pldbg_attach_to_port
----------------------
                   1
(1 row)

--查看源码
select plsql_pldbg_get_source(1,16918);
 plsql_pldbg_get_source
-------------------------------------
                                    +
declare                             +
    i int := 2;                     +
begin                               +
    i = 3;                          +
    raise notice 'i= %',i;          +
    i = 4;                          +
    raise notice 'i= %',i;          +
    raise notice '0:ready.....';    +
    begin                           +
        raise notice '1:ready.....';+
        call pro1();                +
        begin                       +
            raise notice 'success'; +
        end;                        +
    end;                            +
end
(1 row)

--设置断点
select plsql_pldbg_set_breakpoint(1,16918,7);
plsql_pldbg_set_breakpoint
---------------------------
t
(1 row)

select plsql_pldbg_set_breakpoint(1,16918,12);
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

select plsql_pldbg_set_breakpoint(1,16918,15);
```

```
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

--获取当前设置的断点信息
select plsql_pldbg_get_breakpoints(1);
plsql_pldbg_get_breakpoints
----------------------------
(16918,15,"")
(16918,7,"")
(16918,12,"")
(16918,-1,"")
(4 rows)

--单步执行，遇到子程序进入
select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16918,6,"pro4()")
(1 row)

select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16918,7,"pro4()")
(1 row)

select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16918,8,"pro4()")
(1 row)

--单步执行，遇到子程序跳过
select plsql_pldbg_step_over(1);
plsql_pldbg_step_over
-----------------------
(16918,9,"pro4()")
(1 row)

--继续执行程序，直到断点处或者程序结束
select plsql_pldbg_continue(1);
plsql_pldbg_continue
----------------------
```

```
(16918,12,"pro4()")
(1 row)

--当 application client 执行完毕后，debug client 进程会挂起，直到 application client 再次执行测试调试对象
select plsql_pldbg_continue(1);
```

application client：

```
--应用端存储过程执行完成
call pro4();
NOTICE:  PLDBGBREAK:3
NOTICE:  i= 3
NOTICE:  i= 4
NOTICE:  0:ready.....
NOTICE:  1:ready.....
NOTICE:  123
NOTICE:  success
NOTICE:  success
CALL

--再次调起
call pro4();
```

debugger client：

```
--调试端进入调试状态，调用 plsql_pldbg_abort_target，终止调试
select plsql_pldbg_abort_target(1);
plsql_pldbg_abort_target
--------------------
t
(1 row)
```

application client：

```
-- 当 debug client 调用 select plsql_pldbg_abort_target(1) 后，执行被终止
call pro4();
ERROR:  canceling statement due to user request
CONTEXT:  PL/SQL    FUNCTION pro4() line 5 at assignment
```

3.71.4.2.39　全局断点调试示例

debugger client：

```
--调试端创建全局监听
select plsql_pldbg_create_listener();
```

```
plsql_pldbg_create_listener
-----------------------
                     1
(1 row)

--设置全局断点，未指定行号默认为-1，即 BEGIN 块的第一行
select plsql_pldbg_set_global_breakpoint(1,16918,null,null);
plsql_pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--等待测试函数被调用，无测试函数调用该进程会挂起
select plsql_pldbg_wait_for_target(1);
```

application client：

```
--应用端执行测试函数
call pro4();
```

debugger client：

```
--测试函数调用后，返回测试函数 session 的进程号，开始调试
select plsql_pldbg_wait_for_target(1);
plsql_pldbg_wait_for_target
-----------------------------
                      427689
(1 row)

--设置局部断点
select plsql_pldbg_set_breakpoint(1,16918,5);
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

--设置全局断点
select plsql_pldbg_set_global_breakpoint(1,16918,12,null);
plsql_pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--获取变量信息
select plsql_pldbg_get_variables(1);
```

```
plsql_pldbg_get_variables
---------------------------
(i,L,3,t,f,f,23,2)
(1 row)

--单步进入
select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16918,6,"pro4()")
(1 row)


select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16918,7,"pro4()")
(1 row)

--获取修改后的变量信息
select plsql_pldbg_get_variables(1);
plsql_pldbg_get_variables
---------------------
(i,L,3,t,f,f,23,3)
(1 row)

--继续执行到断点处
select plsql_pldbg_continue(1);
plsql_pldbg_continue
----------------------
(16918,12,"pro4()")
(1 row)

--单步进入子程序 pro1
select plsql_pldbg_step_into(1);
plsql_pldbg_step_into
-----------------------
(16917,3,"pro1()")
(1 row)

--获取堆栈信息
select plsql_pldbg_get_stack(1);
 plsql_pldbg_get_stack
---------------------------
(0,"pro1()",16917,3,"")
```

```
(1,"pro4()",16918,12,"")
(2 rows)

--追溯堆栈信息
select plsql_pldbg_select_frame(1,1);
plsql_pldbg_select_frame
--------------------------
(16918,12,"pro4()")
(1 row)

--继续执行到程序结束后挂起，等待下一次程序调用
select plsql_pldbg_continue(1);
```

application client：

```
--应用端程序执行完成
call pro4();
NOTICE:  i= 3
NOTICE:  i= 4
NOTICE:  0:ready
NOTICE:  1:ready
NOTICE:  123
NOTICE:  success
NOTICE:  success
CALL
```

3.71.4.2.40　修改调试时数据及删除断点调试示例

创建测试环境和用例：

```
--创建 plsql_debugger 环境
create extension plsql_pldbgapi;

--创建用来调试的存储过程
\set SQLTERM /
create or replace procedure pro5(i int) as
declare
   j int;
begin
   raise notice 'i = %',i;
   j = i;
   raise notice 'j = %',j;
   j = 1;
   raise notice 'j = %',j;
   i = 2;
```

```
   raise notice 'i = %',i;
end;
/
\set SQLTERM ;

--查询出需要调试的存储过程的 oid
select oid from sys_proc where proname = 'pro5';
 oid
-------
16919
(1 row)
```

debugger client：

```
--创建全局监听
select plsql_pldbg_create_listener();
plsql_pldbg_create_listener
-----------------------
                     1
(1 row)

--设置全局断点
select plsql_pldbg_set_global_breakpoint(1,16919,null,null);
plsql_pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--等待测试存储过程调用
select plsql_pldbg_wait_for_target(1);
```

application client：

```
call pro5(3);
```

debugger client：

```
--子程序已调用，开始调试
select plsql_pldbg_wait_for_target(1);
plsql_pldbg_wait_for_target
-----------------------------
                    120947
(1 row)

--返回建立连接时断点所在行的信息（全局断点行号为 null，默认当前在 BEGIN 块的第一行）
select plsql_pldbg_wait_for_breakpoint(1);
```

```
plsql_pldbg_wait_for_breakpoint
--------------------------------
(16919,5,"pro5(integer)")
(1 row)

--单步执行
select plsql_pldbg_step_into(1);
  plsql_pldbg_step_into
---------------------------
(16919,6,"pro5(integer)")
(1 row)

--获取存储过程源码
select plsql_pldbg_get_source(1,16919);
  plsql_pldbg_get_source
-----------------------------
                            +
declare                     +
    j int;                  +
begin                       +
    raise notice 'i = %',i;+
    j = i;                  +
    raise notice 'j = %',j;+
    j = 1;                  +
    raise notice 'j = %',j;+
    i = 2;                  +
    raise notice 'i = %',i;+
end
(1 row)

--查看变量的值
select plsql_pldbg_get_variables(1);
plsql_pldbg_get_variables
---------------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,NULL)
(2 rows)

--修改变量 j 的值
select plsql_pldbg_deposit_value(1, 'j', 3, '10');
plsql_pldbg_deposit_value
---------------------
t
(1 row)
```

```
--再次查看变量 j 的值，已被改变
select plsql_pldbg_get_variables(1);
plsql_pldbg_get_variables
---------------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,10)
(2 rows)

--单步执行，变量 j 在程序中被赋值
select plsql_pldbg_step_into(1);
  plsql_pldbg_step_into
---------------------------
(16919,7,"pro5(integer)")
(1 row)

--查看变量 j 的值，已改变
select plsql_pldbg_get_variables(1);
plsql_pldbg_get_variables
---------------------------
(i,A,0,t,f,f,23,3)
(j,L,3,t,f,f,23,3)
(2 rows)

--设置局部断点
select plsql_pldbg_set_breakpoint(1,16919,5);
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

select plsql_pldbg_set_breakpoint(1,16919,7);
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

select plsql_pldbg_set_breakpoint(1,16919,9);
plsql_pldbg_set_breakpoint
----------------------
t
(1 row)

--查看断点信息
select plsql_pldbg_get_breakpoints(1);
plsql_pldbg_get_breakpoints
```

```
-----------------------------
(16919,-1,"")
(16919,5,"")
(16919,9,"")
(16919,7,"")
(16919,-1,"")
(5 rows)

--删除断点
select plsql_pldbg_drop_breakpoint(1,16919,5);
plsql_pldbg_drop_breakpoint
-----------------------
t
(1 row)

select plsql_pldbg_drop_breakpoint(1,16919,9);
plsql_pldbg_drop_breakpoint
-----------------------
t
(1 row)

--查看断点信息
select plsql_pldbg_get_breakpoints(1);
plsql_pldbg_get_breakpoints
-----------------------------
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
(3 rows)

--设置全局断点
select plsql_pldbg_set_global_breakpoint(1,16919,11,null);
plsql_pldbg_set_global_breakpoint
-----------------------------
t
(1 row)

--查看断点信息
select plsql_pldbg_get_breakpoints(1);
plsql_pldbg_get_breakpoints
-----------------------------
(16919,11,"")
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
```

```
(4 rows)

--设置断点行号为 0 时无效
select plsql_pldbg_drop_breakpoint(1,16919,0);
plsql_pldbg_drop_breakpoint
-----------------------
f
(1 row)

--设置断点行号为 NULL 时无效
select plsql_pldbg_drop_breakpoint(1,16919,NULL);
plsql_pldbg_drop_breakpoint
-----------------------

(1 row)

--查看断点信息，0 和 NULL 断点都未设置成功
select plsql_pldbg_get_breakpoints(1);
plsql_pldbg_get_breakpoints
-----------------------------
(16919,11,"")
(16919,-1,"")
(16919,7,"")
(16919,-1,"")
(4 rows)

--查看版本信息
select * from plsql_pldbg_get_proxy_info();
                                serverversionstr
| serverversionnum | proxyapiver | serverprocessid
-----------------------------------------------------------------------------------------------------------------
-+------------------+-------------+-----------------
KingbaseES V009R001 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-36), 64-bit
|                8 |           3 |          437829
(1 row)
```

3.71.5　插件 plsql_pldbgapi 卸载方法

卸载 plsql_pldbgapi 插件接口，只需要执行以下命令：

```
-- 删除插件
drop extension plsql_pldbgapi;
```

3.71.6 插件 plsql_pldbgapi 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 1.1 升级到 1.2：

```
ALTER EXTENSION plsql_pldbgapi UPDATE TO '1.2';
```

3.72 plsql_plprofiler

3.72.1 插件 plsql_plprofiler 简介

插件 plsql_plprofiler 是 KingbaseES 的一个扩展插件。plsql_plprofiler 插件支持收集分析器（性能）数据以提高性能或确定 PL/SQL 应用程序的代码覆盖率。应用程序可以使用代码覆盖率数据来集中他们的增量测试工作。

此插件可以为所有 PL/SQL 单元生成分析信息，此信息包括每行的执行次数、执行该行所花费的总时间以及该行在多次执行中消耗的最长时间。

plsql_plprofiler 中的监听方式分为本地监控、全局监控、以及指定会话监控三种。其中，本地监控模式是指只对当前 session 运行的对象进行监控；全局监控模式是指对所有 session 运行的对象进行监控；指定会话模式是指对某个指定 session 上运行的对象进行监控。

三种模式同时启动的情况下优先级是全局监控模式 > 指定会话监听模式 > 本地监听模式。

plsql_plprofiler 中的数据共享方式分为 local 和 global 两种。其中，local 模式数据不共享，仅当前 session 可见；global 模式数据共享，所有 session 可见。

- 插件名为 plsql_plprofiler
- 插件版本 V4.2

3.72.2 插件 plsql_plprofiler 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。因此只需要在客户端工具执行 create extension 命令即可。

```
-- 创建插件
create extension plsql_plprofiler;
```

备注:

在当前模式和 plsql_profiler 模式下均创建了一套分析接口，因此在任意模式下分析接口可通过模式限定调用。

3.72.3　插件 plsql_plprofiler 的参数配置

plsql_plprofiler 定义了三个系统参数：

- plsql_plprofiler.max_callgraphs

  设置内存中存储堆栈关系的行数，默认值为 20000。

- plsql_plprofiler.max_functions

  设置内存中可存储的对象个数，默认值为 2000。

- plsql_plprofiler.max_lines

  设置内存中分析数据记录行数，默认值为 200000。

3.72.4　插件 plsql_plprofiler 使用方法

3.72.4.1　plsql_plprofiler 使用说明

支持 profiler 的对象：

- 支持函数分析
- 支持存储过程分析
- 支持包分析

  包中支持分析的对象：包中初始化块、包中函数、包中存储过程。包中对象的分析数据记录到包对应的 oid。

- 支持触发器分析
- 支持嵌套函数分析
- 支持匿名块分析

  匿名块的分析数据记录到 oid 为 9999 的行。

- 支持 object type 分析

  object type 中支持分析的对象：member 方法、construct 方法、static 方法。object type 中分析数据记录到各个方法对应的 oid。

备注:

如果是重复执行同一个对象（以 oid 为标识），需要清理之前记录的数据。

支持 profiler 的是数据信息分为三类：

PLSQL 对象执行信息

PLSQL 对象堆栈信息

PLSQL 对象源码信息

plsql_plprofiler **获取到的执行信息如下：**

- 对象 oid

  plsql_plprofiler 对象的 oid，包中的所有对象共用包的 oid、匿名块的 oid 为 9999、object type 中的对象使用方法本身的 oid。嵌套函数的 oid 使用祖先函数的 oid。

- 语句所在行

  0 行表示该对象的整体数据。

  如果 PL/SQL 源码在一行上，则该语句的任何执行相关数据都属于该行。

  如果 PL/SQL 源码在多行上，则该语句的任何执行相关数据都属于该语句的第一行。

  如果 PL/SQL 源码的一行出现多个语句时，则多条语句的任何执行相关数据都属于该行。

- 执行次数

  记录某一行代码的执行次数。一行代码不一定是一条语句。

- 总共耗时

  记录某一行代码的总耗时。如果是控制语句则会包括整个控制语句中包含的语句的时间。时间单位 us。

- 最长耗时

  记录某一行代码耗时最长的时间。时间单位 us。

备注:

1. 当执行对象抛出异常未被捕获时，只会记录整个对象的执行时间，其他数据均为 0；当执行对象抛出异常被捕获时，异常语句的数据为 0，其他语句数据正常记录。
2. 如果执行对象中有事务操作（commit/rollback），该对象记录的总耗时是事务操作前的总时间。
3. 当开启 set serverout on 时，每条 SQL 语句都会执行 dbms_output 包中函数。因此执行语句时，plsql_plprofiler 会记录调用 dbms_output 包的数据，如：调用数据 clean 函数后，首先清除执行数据，然后会调用 dbms_ouput 包中函数。

---

plsql_plprofiler **获取到的堆栈信息如下：**

- 堆栈

  plsql_plprofiler 对象的堆栈

- 调用次数

  该对象被调用的总次数。

- 总耗时

  该对象总花费时间。

- 调用对象耗时

  该对象调用对象的耗时。

- 自身耗时

  该对象除去调用其他对象的耗时。

plsql_plprofiler **获取到的源码信息如下：**

- oid
- 行号
- 源码

  由于匿名块在数据库中没有做存储，所以无法获取匿名块的源码信息。

3.72.4.2　接口函数

接口函数	简介
pl_profiler_set_enabled_local 函数	启停当前 session 分析器
pl_profiler_set_enabled_pid 函数	启停指定 session 分析器
pl_profiler_get_enabled_local 函数	查看当前 session 分析器状态
pl_profiler_get_enabled_pid 函数	查看指定 session 分析器状态
pl_profiler_set_enabled_global 函数	启停数据库级分析器
pl_profiler_get_enabled_global 函数	查看全局监控是否启用的状态
pl_profiler_collect_data 函数	单次数据 flush
pl_profiler_set_collect_interval 函数	指定时间间隔数据 flush
pl_profiler_get_collect_interval 函数	查看定时迁移的状态
pl_profiler_reset_local 函数	本地数据 clean
pl_profiler_reset_shared 函数	全局数据 clean
pl_profiler_linestats_local 函数	查看本地的对象的执行信息
pl_profiler_linestats_shared 函数	查看全局的对象的执行信息
pl_profiler_func_oids_local 函数	查看本地已执行 plsql 对象
pl_profiler_func_oids_shared 函数	查看全局已执行 plsql 对象

-续表-

接口函数	简介
pl_profiler_callgraph_local 函数	查看本地的堆栈关系
pl_profiler_callgraph_shared 函数	查看全局的堆栈关系
pl_profiler_get_stack 函数	OID 转对象名
pl_profiler_funcs_source 函数	查看指定对象源码信息
pl_profiler_version 函数	查看整型版本信息
pl_profiler_versionstr 函数	查看字符串版本信息
pl_profiler_lines_overflow 函数	查看行数是否过界

3.72.4.2.1　pl_profiler_set_enabled_local 函数

功能描述

开启或关闭当前会话分析器。开启分析器则会对 session 中执行的所有 plsql 对象进行分析，并将信息进行暂存。

语法格式

```
FUNCTION pl_profiler_set_enabled_local(enabled bool) RETURNS bool
```

参数说明：

输入为 true 则为启用分析器，输入为 false 则为关闭分析器。

输出说明

返回值有三种：true、false、null（空），如果输入和输出一致，表示程序执行成功；反之，程序执行失败。

3.72.4.2.2　pl_profiler_set_enabled_pid 函数

功能描述

开启或关闭指定会话分析器。开启分析器则会对 session 中执行的所有 plsql 对象进行分析，并将信息进行暂存。

语法格式

```
FUNCTION pl_profiler_set_enabled_pid(pid int4) RETURNS bool
```

参数说明：

需要监控进程的 PID，每次只有一个 session 可以通过本方式启停分析器。输入 PID 为有效 seesion 时，启动分析器。PID 为无效 session 时，则关闭已启动的监听器。

输出说明

输入进程号非 0 则返回 true，输入进程号为 0 则返回为 false。

3.72.4.2.3　pl_profiler_get_enabled_local 函数

功能描述

查看当前会话分析器状态。

语法格式

```
FUNCTION pl_profiler_get_enabled_local() RETURNS bool
```

输出说明

返回值为 true 则监控状态为启用，返回值为 false 则监控为未启用。

3.72.4.2.4　pl_profiler_get_enabled_pid 函数

功能描述

查看指定会话级分析器状态。

语法格式

```
FUNCTION pl_profiler_get_enabled_pid() RETURNS bool
```

输出说明

返回值为 true 则监控状态为启用，返回值为 false 则监控为未启用。

3.72.4.2.5　pl_profiler_set_enabled_global 函数

功能描述

开启或关闭数据库级分析器。开启全局监控则会对所有会话执行的所有 plsql 对象进行分析，并将信息进行暂存。

语法格式

```
FUNCTION pl_profiler_set_enabled_global(enabled bool) RETURNS bool
```

参数说明：

输入为 true 则为启用全局监控，输入为 false 则为关闭全局监控。

输出说明

返回值有三种：true、false、null（空）。如果输入和输出一致，表示程序执行成功；反之，程序执行失败。

3.72.4.2.6　pl_profiler_get_enabled_global 函数

功能描述

查看全局监控是否启用的状态。

语法格式

```
FUNCTION pl_profiler_get_enabled_global() RETURNS bool
```

输出说明

返回值为 true 则监控状态为启用，返回值为 false 则监控状态为未启用。

3.72.4.2.7　pl_profiler_collect_data 函数

功能描述

将本地数据迁移到全局数据中。数据迁移后，本地的分析数据会被清除（执行次数，最长时间，总共花费时间）。

语法格式

```
FUNCTION pl_profiler_collect_data() RETURNS int4
```

输出说明

返回值：返回为-1 则表示没有将 plprofiler 插件加载到 shared_preload_libraries 中，返回为 0 则表示迁移成功或没有数据需要迁移。

备注:

如果全局数据区内容已满，此时本地数据不会完全迁移到全局数据区中，同时系统报错内存已超出。

3.72.4.2.8　pl_profiler_set_collect_interval 函数

功能描述

设置时间（秒级单位）自动将本地数据迁移到全局数据中。数据迁移后，本地的分析数据会被清除（执行次数，最长时间，总共花费时间）。

语法格式

```
FUNCTION pl_profiler_set_collect_interval(seconds int4) RETURNS bool
```

输入说明：

输入为秒级单位。

输出说明

当输入 seconds 为负数，返回值为 t, 无实际意义；

当输入 seconds 为 0 时，返回值为 f, 无实际意义；

当输入 seconds 为大于 0 时，返回值为 t，设置迁移时间有效。

备注:

如当前事务是：XACT_EVENT_COMMIT,、XACT_EVENT_PARALLEL_COMMIT、XACT_EVENT_ABORT、XACT_EVENT_PARALLEL_ABORT 且已设置有效的迁移时间，则立即将本地数据迁移到全局数据中。

3.72.4.2.9　pl_profiler_get_collect_interval 函数

功能描述

查看是否定时迁移的状态。

语法格式

```
FUNCTION pl_profiler_get_collect_interval() RETURNS bool
```

输出说明

返回值：根据是否定时的设置返回对应状态。如果之前用户设置的定时时间非 0 则该函数返回 t，如果之前设置定时时间为 0 则返回 f。

3.72.4.2.10　pl_profiler_reset_local 函数

功能描述

清除本地存储的所有数据信息。

语法格式

```
FUNCTION pl_profiler_reset_local() RETURNS void
```

3.72.4.2.11　pl_profiler_reset_shared 函数

功能描述

清除全局存储的所有数据信息。

语法格式

```
FUNCTION pl_profiler_reset_shared() RETURNS void
```

3.72.4.2.12　pl_profiler_linestats_local 函数

功能描述

查看本地的对象的执行信息

语法格式

```
FUNCTION pl_profiler_linestats_local(
  OUT func_oid oid,
  OUT line_number int8,
  OUT exec_count int8,
  OUT total_time int8,
  OUT longest_time int8
)
RETURNS SETOF record
AS 'MODULE_PATHNAME'
LANGUAGE C ROWS 1000000;
```

输出说明

返回一个具有以下结构的数据集合：

func_oid：类型为 oid，表示已经监控的对象 oid；

line_number：类型为 int8，表示当前语句行号；

exec_count：类型为 int8，表示执行次数；

total_time：类型为 int8，表示当前语句执行总时长；

longest_time：类型为 int8，表示当前语句执行的最长一次时长。

备注:

line_number 会根据源码（系统表中存储源码）进行计算，line_number 为 0 的行表示了当前对象的执行次数，总共执行时长，最长一次执行时间。

3.72.4.2.13　pl_profiler_linestats_shared 函数

功能描述

查看全局的对象的执行信息

语法格式

```
FUNCTION pl_profiler_linestats_shared(
  OUT func_oid oid,
  OUT line_number int8,
  OUT exec_count int8,
  OUT total_time int8,
  OUT longest_time int8
)
RETURNS SETOF record
AS 'MODULE_PATHNAME'
LANGUAGE C ROWS 1000000;
```

输出说明

返回一个具有以下结构的多行：

func_oid：类型为 oid，表示已经监控的对象 oid；

line_number：类型为 int8，表示当前语句行号；

exec_count：类型为 int8，表示执行次数；

total_time：类型为 int8，表示当前语句执行总时长；

longest_time：类型为 int8，表示当前语句执行的最长一次时长。

备注:

line_number 会根据源码（系统表中存储源码）进行计算，line_number 为 0 的行表示了当前对象的执行次数，总共执行时长，最长一次执行时间。

3.72.4.2.14　pl_profiler_func_oids_local 函数

功能描述

查看本地已执行 plsql 对象。

语法格式

```
FUNCTION pl_profiler_func_oids_local() RETURNS oid[]
```

输出说明

返回值：返回 plsql 对象的 oid 数组。

3.72.4.2.15　pl_profiler_func_oids_shared 函数

功能描述

查看全局已执行 plsql 对象

语法格式

```
FUNCTION pl_profiler_func_oids_shared() RETURNS oid[]
```

输出说明

返回值：返回 plsql 对象的 oid 数组。

3.72.4.2.16　pl_profiler_callgraph_local 函数

功能描述

查看本地已监控的 plsql 对象及子程序调用信息。

语法格式

```
FUNCTION pl_profiler_callgraph_local(
  OUT stack oid[],
  OUT call_count int8,
  OUT us_total int8,
  OUT us_children int8,
  OUT us_self int8
)
RETURNS SETOF record
AS 'MODULE_PATHNAME'
LANGUAGE C ROWS 1000000;
```

输出说明

返回一个具有以下结构的多行：

stack：类型为 oid[]，表示已经监控的对象 oid 数组，能够表明监控对象的堆栈关系；

call_count：类型为 int8，表示当前对象调用次数；

us_total：类型为 int8，表示执行该对象总共花费时间；

us_children：类型为 int8，表示执行该对象中的子程序总共花费时间；

us_self：类型为 int8，表示执行该对象除子程序外，对象本身语句花费时间。

备注:

如果调用的 plsql 对象中没有调用子程序，则调用 pl_profiler_callgraph_local 函数只有一行信息；如果调用的 plsql 对象中存在调用子程序，则调用 pl_profiler_callgraph_local 函数除了显示调用的 plsql 对象，还有显示子程序的性能数据信息，并且子程序性能数据信息中的 stack 会显示堆栈关系，如 {16513,16515,16514}，其中 16513 为调用的 plsql 对象，16515 为 16513 的子程序，16514 为 61515 的子程序。

3.72.4.2.17　pl_profiler_callgraph_shared 函数

功能描述

查看全局的 plsql 对象及子程序调用信息。

语法格式

```
FUNCTION pl_profiler_callgraph_shared(
  OUT stack oid[],
  OUT call_count int8,
  OUT us_total int8,
  OUT us_children int8,
  OUT us_self int8
)
RETURNS SETOF record
AS 'MODULE_PATHNAME'
LANGUAGE C ROWS 1000000;
```

输出说明

返回一个具有以下结构的多行：

stack：类型为 oid[]，表示已经监控的对象 oid 数组，能够表明监控对象的堆栈关系；

call_count：类型为 int8，表示当前对象调用次数；

us_total：类型为 int8，表示执行该对象总共花费时间；

us_children：类型为 int8，表示执行该对象中的子程序总共花费时间；

us_self：类型为 int8，表示执行该对象除子程序外，对象本身语句花费时间。

备注:

如果调用的 plsql 对象中没有调用子程序，则调用 pl_profiler_callgraph_shared 函数只有一行信息；如果调用的 plsql 对象中存在调用子程序，则调用 pl_profiler_callgraph_shared 函数除了显示调用的 plsql 对象，还有显示子程序的性能数据信息，并且子程序性能数据信息中的 stack 会显示堆栈关系，如 {16513,16515,16514}，其中 16513 为调用的 plsql 对象，16515 为 16513 的子程序，16514 为 61515 的子程序。

3.72.4.2.18　pl_profiler_get_stack 函数

功能描述

将 OID 转成对象名称及当前模式名信息。

语法格式

```
FUNCTION pl_profiler_get_stack(stack oid[])
RETURNS text[]
AS 'MODULE_PATHNAME'
LANGUAGE C STRICT;
```

输入说明：

stack 为一个 oid 的数组，通常为 pl_profiler_func_oids_shared 接口或 pl_profiler_func_oids_local 接口的返回值。

输出说明

将传入的 oid 转化为模式 + 函数名称的 text 数组。

备注:

如果传入的 oid 对应的模式名或对象名称不存在，则 text 数组中会返回”<unknown>.<unknown>()” 的信息。若传入 oid 数组存在堆栈关系，则可以转化为带有堆栈关系的 text 数组。

3.72.4.2.19　pl_profiler_funcs_source 函数

功能描述

查看指定对象源码信息。

语法格式

```
FUNCTION pl_profiler_funcs_source(
  IN  func_oids oid[],
  OUT func_oid oid,
  OUT line_number int8,
  OUT source text
)
RETURNS SETOF record
AS 'MODULE_PATHNAME'
LANGUAGE C ROWS 1000000 STRICT;
```

输入说明：

func_oids：传入的一个对象 oid 数组。

输出说明

返回值：通过传入对象 oid 数组，输出以下结构的多行：

func_oid：对象 oid；

line_number：当前行号；

source：当前行号对应源码。

备注:

系统会自动增加一行 line_number 为 0 的行，并且 suorce 为“-- Line 0”。如果不能查询到输入 oid 的源码，则系统报错。不能查询匿名块的源码。

3.72.4.2.20　pl_profiler_version 函数

功能描述

查看整型版本信息。

语法格式

```
FUNCTION pl_profiler_version() RETURNS integer
```

输出说明

返回值：输出当前的版本值。

3.72.4.2.21　pl_profiler_versionstr 函数

功能描述

查看字符串版本信息。

语法格式

```
FUNCTION pl_profiler_versionstr() RETURNS text
```

输出说明

返回值：输出当前版本的字符串信息。

3.72.4.2.22　pl_profiler_lines_overflow 函数

功能描述

查看存储对象具体信息结构的行数是否过界。

语法格式

```
FUNCTION pl_profiler_lines_overflow() RETURNS bool
```

输出说明

返回值：如果 profiler_linestats 结构内存过界，则返回 t, 否则返回 f。

3.72.4.3　典型用例

Profiler 每次运行的的过程应该包括以下步骤：

1. 启动分析器
2. 运行应用程序
3. 关闭分析器
4. 数据 flush
5. 分析数据

用户在使用 profiler 前，需要创建 profiler 扩展。

profiler 收集数据，通过调用 pl_profiler_set_enabled_local 函数来控制分析器的启动或者停止。分析器收集到的数据存储在内存中，如果用户断开连接，分析器将不会自动存储数据，需要显示调用 pl_profiler_collect_data 函数将数据刷到共享内存。需要注意的是，目前 profiler 数据不会落盘存储，所以一旦重启数据库，分析数据将会丢失。

```
--创建扩展
create extension plsql_plprofiler;
--创建对象
create table t1(i int);
\set SQLTERM /
create or replace procedure p1() as
begin
    raise notice 'this is procedure p1';
    for i in 1..1000 loop
        insert
        into
        t1
        values(99999);
    end loop;
    if 1000 < 100 then null; else null; end if;
end;
/
create or replace procedure p2() as
begin
    raise notice 'this is procedure p2';
    p1();
end;
/
create or replace procedure p3() as
begin
    raise notice 'this is procedure p3';
    p2();
    if 1 < 100 then
```

```
        raise notice '1 < 100';
    end if;
end;
/
\set SQLTERM ;
--清理本地数据
select pl_profiler_reset_local();
--清理全局数据
select pl_profiler_reset_shared();
--启动分析器
select pl_profiler_set_enabled_global(true);
--运行应用程序
call p3();
--关闭分析器
select pl_profiler_set_enabled_global(false);
--数据 flush
select pl_profiler_collect_data();
--分析数据
----查看已记录的对象（可选）
select * from pl_profiler_func_oids_shared();
----oid 转对象名（可选）
select * from pl_profiler_get_stack(pl_profiler_func_oids_shared());
```

```
\set SQLTERM ;
   --清理本地数据
   select pl_profiler_reset_local();
pl_profiler_reset_local
-------------------------

(1 row)

   --清理全局数据
   select pl_profiler_reset_shared();
pl_profiler_reset_shared
--------------------------

(1 row)

   --启动分析器
   select pl_profiler_set_enabled_global(true);
pl_profiler_set_enabled_global
--------------------------------
t
(1 row)
```

```
   --运行应用程序
   call p3();
NOTICE:  this is procedure p3
NOTICE:  this is procedure p2
NOTICE:  this is procedure p1
NOTICE:  1 < 100
CALL
   --关闭分析器
   select pl_profiler_set_enabled_global(false);
pl_profiler_set_enabled_global
--------------------------------
f
(1 row)

   --数据 flush
   select pl_profiler_collect_data();
pl_profiler_collect_data
--------------------------
          0
(1 row)

   --分析数据
   ----查看已记录的对象（可选）
   select * from pl_profiler_func_oids_shared();
pl_profiler_func_oids_shared
------------------------------
{16793,16792,16794}
(1 row)

   ----oid 转对象名（可选）
   select * from pl_profiler_get_stack(pl_profiler_func_oids_shared());
             pl_profiler_get_stack
-------------------------------------------------------------------------
{"public.p2() oid=16793","public.p1() oid=16792","public.p3() oid=16794"}
(1 row)
```

```
----查看数据（可选）
select  func_oid, func_oid::regproc as funcname,line_number, source from pl_profiler_funcs_source(pl_
profiler_func_oids_shared());
```

```
----查看数据（可选）
select  func_oid, func_oid::regproc as funcname,line_number, source from pl_profiler_funcs_source(pl_
profiler_func_oids_shared());
func_oid | funcname | line_number |                        source
```

```
----------+----------+-------------+-----------------------------------------------------
 16793 | p2       |           0 | -- Line 0
 16793 | p2       |           1 |
 16793 | p2       |           2 |     begin
 16793 | p2       |           3 |         raise notice 'this is procedure p2';
 16793 | p2       |           4 |         p1();
 16793 | p2       |           5 |     end
 16792 | p1       |           0 | -- Line 0
 16792 | p1       |           1 |
 16792 | p1       |           2 |     begin
 16792 | p1       |           3 |         raise notice 'this is procedure p1';
 16792 | p1       |           4 |         for i in 1..1000 loop
 16792 | p1       |           5 |             insert
 16792 | p1       |           6 |             into
 16792 | p1       |           7 |             t1
 16792 | p1       |           8 |             values(99999);
 16792 | p1       |           9 |         end loop;
 16792 | p1       |          10 |         if 1000 < 100 then null; else null; end if;
 16792 | p1       |          11 |     end
 16794 | p3       |           0 | -- Line 0
 16794 | p3       |           1 |
 16794 | p3       |           2 |     begin
 16794 | p3       |           3 |         raise notice 'this is procedure p3';
 16794 | p3       |           4 |         p2();
 16794 | p3       |           5 |         if 1 < 100 then
 16794 | p3       |           6 |             raise notice '1 < 100';
 16794 | p3       |           7 |         end if;
 16794 | p3       |           8 |     end
(27 rows)
```

```
----查看对象调用关系
select * from pl_profiler_callgraph_shared();

  stack               | call_count | us_total | us_children | us_self
---------------------+------------+----------+-------------+---------
{16794,16793}         | 1          | 8175     | 7990        | 185
{16794,16793,16792}   | 1          | 7990     | 0           | 7990
{16794}               | 1          | 8473     | 8175        | 298
(3 rows)
```

```
-----源码和分析数据对应关系
SELECT L.func_oid::regproc as funcname,
       L.func_oid as func_oid,
       L.line_number,
```

```
       sum(L.exec_count)::bigint AS exec_count,
       sum(L.total_time)::bigint AS total_time,
       max(L.longest_time)::bigint AS longest_time,
       S.source
FROM pl_profiler_linestats_shared() L
JOIN pl_profiler_funcs_source(pl_profiler_func_oids_shared) S
     ON S.func_oid = L.func_oid AND S.line_number = L.line_number
     GROUP BY L.func_oid, L.line_number, S.source
    ORDER BY L.func_oid, L.line_number;

 funcname | func_oid | line_number | exec_count | total_time | longest_time |                       source
----------+----------+-------------+------------+------------+--------------+------------------------------
----------------------
 p1       |    16792 |           0 |          1 |       7990 |         7990 | -- Line 0
 p1       |    16792 |           1 |          0 |          0 |            0 |
 p1       |    16792 |           2 |          1 |       7989 |         7989 |     begin
 p1       |    16792 |           3 |          1 |         14 |           14 |         raise notice 'this is
procedure p1';
 p1       |    16792 |           4 |          1 |       7830 |         7830 |         for i in 1..1000 loop
 p1       |    16792 |           5 |       1000 |       7181 |          314 |             insert
 p1       |    16792 |           6 |          0 |          0 |            0 |             into
 p1       |    16792 |           7 |          0 |          0 |            0 |             t1
 p1       |    16792 |           8 |          0 |          0 |            0 |             values(99999);
 p1       |    16792 |           9 |          0 |          0 |            0 |         end loop;
 p1       |    16792 |          10 |          2 |          0 |            0 |         if 1000 < 100 then
null; else null; end if;
 p1       |    16792 |          11 |          0 |          0 |            0 |     end
 p2       |    16793 |           0 |          1 |       8175 |         8175 | -- Line 0
 p2       |    16793 |           1 |          0 |          0 |            0 |
 p2       |    16793 |           2 |          1 |       8174 |         8174 |     begin
 p2       |    16793 |           3 |          1 |         22 |           22 |         raise notice 'this is
procedure p2';
 p2       |    16793 |           4 |          1 |       8151 |         8151 |         p1();
 p2       |    16793 |           5 |          0 |          0 |            0 |     end
 p3       |    16794 |           0 |          1 |       8473 |         8473 | -- Line 0
 p3       |    16794 |           1 |          0 |          0 |            0 |
 p3       |    16794 |           2 |          1 |       8471 |         8471 |     begin
 p3       |    16794 |           3 |          1 |         50 |           50 |         raise notice 'this is
procedure p3';
 p3       |    16794 |           4 |          1 |       8365 |         8365 |         p2();
 p3       |    16794 |           5 |          1 |         52 |           52 |         if 1 < 100 then
 p3       |    16794 |           6 |          1 |         13 |           13 |             raise notice '1 <
100';
 p3       |    16794 |           7 |          0 |          0 |            0 |         end if;
 p3       |    16794 |           8 |          0 |          0 |            0 |     end
```

```
(27 rows)
```

3.72.5　插件 plsql_plprofiler 卸载方法

卸载 plsql_plprofiler 插件接口，只需要执行以下命令：

```
-- 删除插件
drop extension plsql_plprofiler;
```

3.72.6　插件 plsql_plprofiler 升级方法

若该插件有升级版本则通过 alter extension 来升级插件。

示例，由 4.1 升级到 4.2：

```
ALTER EXTENSION plsql_plprofiler UPDATE TO '4.2';
```

3.73　restricted_dba

3.73.1　插件 restricted_dba 简介

restricted_dba 是一个 KingbaseES 的扩展，可以实现对超级用户权限进行限制。

受限 DBA 是对数据库超级管理员用户权限进行限制的插件，开启此插件后，管理员的权限应当与普通用户一致，即限制了管理员用户的操作权限，包括系统级权限级及对象级权限。

- 插件名为 restricted_dba
- 插件版本 V1.0

3.73.2　插件 restricted_dba 加载方法

在使用 restricted_dba 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'restricted_dba,  sepapower' # (change requires restart)
```

备注:

它依赖于《KingbaseES 数据库安全指南》分立插件

3.73.3　插件 restricted_dba 的参数配置

`restricted_dba.restricted_enable`

此参数标识是否开启受限 DBA 功能，默认是 false 即不开启受限 DBA。此参数应由安全员开启。

3.73.3.1　示例

```
create database res_test_db;
create role res_test_super with superuser createrole login;
\c res_test_db res_test_super
alter system set restricted_dba.restricted_enable = true;
ERROR:  permission denied
\c res_test_db sso
alter system set restricted_dba.restricted_enable = true;
call sys_reload_conf();
 sys_reload_conf
----------------
 t
(1 row)

\c res_test_db res_test_super
show restricted_dba.restricted_enable;
 restricted_dba.restricted_enable
----------------------------------
 on
(1 row)

create role res_test with login;
\c res_test_db res_test
create table res_test_table (id int);
create view res_test_view (id) as select * from res_test_table;
\c res_test_db res_test_super
alter role res_test;
ERROR:  Restricted DBA can not alter user
select * from res_test_table;
ERROR:  permission denied for table res_test_table
select * from res_test_view;
ERROR:  permission denied for view res_test_view
```

3.73.4　插件 restricted_dba 使用方法

具体参考《KingbaseES 数据库安全指南》受限 DBA 章节。

3.73.5　插件 restricted_dba 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.73.6　插件 restricted_dba 升级方法

restricted_dba 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.74　roaringbitmap

插件 roaringbitmap 简介 ----------

roaringbitmap 是 KingbaseES 的扩展插件，可高效压缩位图，解决由于位图的稀疏存储而导致的空间浪费问题。支持 roaring bitmap 的存取、集合操作，以及聚合运算等。

```
该插件仅在 Oracle 模式下支持。
```

3.74.1　插件 roaringbitmap 加载方式

```
CREATE EXTENSION roaringbitmap;
```

3.74.2　插件 roaringbitmap 的参数配置

无需配置任何参数。

3.74.3　插件 roaringbitmap 外部接口

3.74.3.1　数据类型

roaringbitmap

本插件实现的基本类型，roaringbitmap 类型。

3.74.3.2　函数

名称	描述
roaringbitmap_in	将字符串转换为 roaringbitmap 类型
roaringbitmap_out	将 roaringbitmap 类型转换为字符串
roaringbitmap_recv	将二进制转换为 roaringbitmap 类型
roaringbitmap_send	将 roaringbitmap 类型转换为二进制类型
roaringbitmap	将二进制形式转换为 roaringbitmap 类型
rb_and	将两个 roaringbitmap 类型进行与操作
rb_or	将两个 roaringbitmap 类型进行或操作
rb_xor	将两个 roaringbitmap 类型进行亦或操作
rb_andnot	将两个 roaringbitmap 类型进行 not 操作
rb_add	将 roaringbitmap 类型集合中添加 int 类型值
rb_remove	将 roaringbitmap 类型集合中移除 int 类型值
rb_shiftright	将 roaringbitmap 类型集合中的元素加上 int 类型值
rb_shiftleft	将 roaringbitmap 类型集合中的元素减去 int 类型值
rb_contains	判断两个 roaringbitmap 变量的包含关系
rb_containedby	判断两个 roaringbitmap 变量的包含关系
rb_intersect	返回两个 roaringbitmap 变量的交集
rb_equals	判断两个 roaringbitmap 变量是否相等
rb_not_equals	判断两个 roaringbitmap 变量是否不相等
rb_build	通过 int 数组构建 roaringbitmap 变量
rb_index	返回 int 类型值在 roaringbitmap 变量集合中的索引
rb_cardinality	返回 roaringbitmap 中的元素数量
rb_and_cardinality	返回两个集合交集的元素数量

-续表-

名称	描述
rb_or_cardinality	返回两个集合并集的元素数量
rb_xor_cardinality	返回两个集合亦或操作后的元素数量
rb_andnot_cardinality	返回两个集合差集操作后的元素数量
rb_is_empty	判断 roaringbitmap 集合是否为空
rb_fill	对 roaringbitmap 进行填充
rb_remove	对 roaringbitmap 指定序列删除
rb_range	返回 roaringbitmap 中在值域范围内的集合
rb_range_cardinality	返回 roaringbitmap 中在值域范围内的元素数量
rb_min	返回 roaringbitmap 中的最小值
rb_max	返回 roaringbitmap 中的最大值
rb_rank	返回 roaringbitmap 中小于指定值的个数
rb_select	从 roaringbitmap 中返回指定数量的集合
rb_to_array	将 roaringbitmap 转换成 int 数组
rb_iterate	将 roaringbitmap 以表的形式输出
rb_or_agg	聚集函数，对 roaringbitmap 进行与操作
rb_or_cardinality_agg	聚集函数，对 roaringbitmap 进行与操作之后的元素个数
rb_and_agg	聚集函数，对 roaringbitmap 进行或操作
rb_and_cardinality_agg	聚集函数，对 roaringbitmap 进行或操作之后的元素个数
rb_xor_agg	聚集函数，对 roaringbitmap 进行亦或操作
rb_xor_cardinality_agg	聚集函数，对 roaringbitmap 进行亦或操作之后的元素个数
rb_gt	对两个 roaringbitmap 类型进行大于比较
rb_ge	对两个 roaringbitmap 类型进行大于等于比较
rb_lt	对两个 roaringbitmap 类型进行小于比较
rb_le	对两个 roaringbitmap 类型进行大于等于比较

-续表-

名称	描述
rb_cmp	对两个 roaringbitmap 类型进行比较

3.74.3.3　类型转换

支持如下类型转换：

源类型	目标类型
roaringbitmap	bytea
bytea	roaringbitmap

3.74.3.4　操作符

左类型	操作符	右类型
roaringbitmap	&	roaringbitmap
roaringbitmap		roaringbitmap
roaringbitmap	&	int4
roaringbitmap	#	roaringbitmap
roaringbitmap	<<	bigint
roaringbitmap	>>	bigint
roaringbitmap		roaringbitmap
roaringbitmap		int4
roaringbitmap	@>	roaringbitmap
roaringbitmap	@>	Int4
roaringbitmap	@<	roaringbitmap

-续表-

左类型	操作符	右类型
Int4	@<	roaringbitmap
roaringbitmap	&&	roaringbitmap
roaringbitmap	=	roaringbitmap
roaringbitmap	<>	roaringbitmap
roaringbitmap	>	roaringbitmap
roaringbitmap	>=	roaringbitmap
roaringbitmap	<	roaringbitmap
roaringbitmap	<=	roaringbitmap

3.74.3.5　索引

支持 Btree 索引。

3.74.4　插件 roaringbitmap 卸载方法

```
DROP EXTENSION roaringbitmap;
```

3.75　roledisable

3.75.1　插件 roledisable 简介

roledisable 是 KingbaseES 的扩展插件。主要用在管理数据库的角色是否生效。可以在不删除角色的前提下，使本地库内使角色被禁用或者使禁用的的角色启用。

- 插件名为 roledisable
- 插件版本 V1.0

3.75.2　插件 roledisable 加载方式

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数，重启数据库时自动加载。

```
shared_preload_libraries = 'roledisable'
\c - system
create extension roledisable;
```

3.75.2.1　查看角色状态

SYSTEM 可以通过 roledisable.sys_role_status 视图来查看系统所有角色在当前库内的启用/禁用状态。

```
SELECT * FROM roledisable.sys_role_status;
```

3.75.2.2　字段说明

`oid`
角色在数据库内的 ID。

`rolename`
角色在数据库内的名称。

`status`
角色的状态 Enable 表示角色被启用，Disable 表示角色被禁用。

3.75.2.3　启用角色

将角色设置为启用状态。

```
ALTER ROLE rolename ENABLE;
```

执行权限: 需要以系统管理员权限执行，以非系统管理员执行时报告错误。

角色要求：被启用的角色需事先存在。

新建角色：新创建的角色默认为启用状态。

3.75.2.4　禁用角色

将角色设置为禁用状态。

```
ALTER ROLE rolename DISABLE;
```

执行权限：需要以系统管理员权限执行，以非系统管理员执行时报告错误。

角色要求：被禁用的角色需事先存在。

系统内置角色：系统内置角色无法被禁用。如初始化用户、sao、sso 等无法被禁用。

3.75.2.5　被禁用角色的表现

- 当角色被禁用后，断开权限继承关系，不能从该角色直接和间接继承权限，间接使用被禁用角色的权限时应该报角色被禁用或权限错误。
- 以 SET ROLE 切换到被禁用的角色时报告错误。
- 当会话使用的系统管理员角色被禁用时，因系统管理员绕过权限检查因此不会对其造成权限的影响。
- 当前会话使用的非系统管理员角色被禁用时，被禁用角色对权限继承关系产生影响因缓存机制可能不会立即生效，当切换角色权限检查或重新登录后生效。
- 角色在使用期间被禁用时，以被禁用角色（非系统管理员）执行需检查权限的操作时，报当前角色被禁用错误。
- 角色被禁用状态下，可以修改角色权限和角色关系。如使用 GRANT 和 REVOKE 对其操作，使用 ALTER ROLE 对其操作。

3.75.3　插件 roledisable 的参数配置

无需配置任何参数。

3.75.4　插件 roledisable 的使用方法

加载 roledisable 插件后，程序可以管理数据库的角色是否生效。可以在不删除角色的前提下，使本地库内使角色被禁用或者使禁用的的角色启用。

示例：

```
-- 创建插件
\c test system
create extension roledisable;
CREATE EXTENSION
```

3.75.5　插件 roledisable 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.75.6 插件 roledisable 升级方法

roledisable 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.76 rum

3.76.1 插件 rum 简介

具体参考：RUM 索引类型章节。

- 插件名为 rum
- 插件版本 V1.3

3.76.2 插件 rum 加载方式

```
CREATE EXTENSION rum;
```

3.76.3 插件 rum 的参数配置

无需配置任何参数

3.76.4 插件 rum 的使用方法

具体参考：RUM 索引类型章节。

3.76.5 插件 rum 卸载方法

```
DROP EXTENSION rum;
```

3.76.6 插件 rum 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION rum UPDATE TO '1.1';
```

3.77 security_utils

3.77.1 插件 security_utils 简介

security_utils 是 KingbaseES 的一个扩展插件，主要用于支持一些安全方面的功能，目前支持的功能有日志密码脱敏，弱口令扫描。

- 插件名为 security_utils
- 插件版本 V1.0

3.77.2 插件 security_utils 加载方式

在使用 security_utils 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

示例：

```
shared_preload_libraries = 'security_utils'
```

3.77.3 插件 security_utils 的参数配置

`security_utils.password_masking_enable`
启动日志密码脱敏功能，取值范围为 on 或者 off，缺省值为 off。

```
show security_utils.password_masking_enable;
 security_utils.password_masking_enable
----------------------------------------
 off
(1 row)

\c - system
alter system set security_utils.password_masking_enable = on;
select sys_reload_conf();
```

```
show security_utils.password_masking_enable;
 security_utils.password_masking_enable
----------------------------------------
 on
(1 row)
```

security_utils.weak_pwd_check_enable
弱口令检查功能，取值范围为 on 或者 off，默认为关闭状态。

3.77.4　插件 security_utils 使用方法

3.77.4.1　口令脱敏

KingbaseES 加载 security_utils 插件，并修改了参数后，当开启 DDL 日志时，日志中涉及密码的 SQL 语句可以进行脱敏处理。

示例

```
---开启密码脱敏
\c - system
create extension kdb_schedule;
create extension security_utils;
alter system set security_utils.password_masking_enable=on;
select sys_reload_conf();

---设置日志级别
alter system set log_statement='ddl';
select sys_reload_conf();

---执行 user 的包含密码的 ddl 语句
drop user testpwdtmu1;
create user testpwdtmu1 with password '12345678ab';
alter user testpwdtmu1 with password '12345678abc';
drop user testpwdtmu1;

---查看 sys_log 下的日志文件，上述语句 password 脱敏，以 * 显示
ERROR:  role "testpwdtmu1" does not exist
STATEMENT:  drop user testpwdtmu1;
LOG:  statement: create user testpwdtmu1 with password '**********';
LOG:  statement: alter user testpwdtmu1 with password '***********';
LOG:  statement: drop user testpwdtmu1;

---清空环境
alter system set security_utils.password_masking_enable=off;
```

```
select sys_reload_conf();
drop extension security_utils;
```

3.77.4.2　弱口令扫描

弱口令字典配置语法

```
--设置弱口令字典
set weak password [if not exists] 'wpwd1'[, 'wpwd2'...];
--删除弱口令字典
remove weak password [if exists] 'wpwd1'[, 'wpwd2'...];
```

说明

- 可以同时创建或删除多个弱口令，使用方法见上配置语法。
- 弱口令字符串类型为 text，不限制长度，且配置语法不限制弱口令个数。
- 当存在 if not exists 或 if exists，将忽略已存在或不存在的弱口令，避免报错，否则会报错提示。
- 不允许输入的弱口令为 null，例如 set weak password null。
- 输入的弱口令为空字符串时，例如 set weak password ‘’ ，处理方法与第三点相同。
- 弱口令扫描支持三种口令加密方式，包括 sha256、md5、sm3 。

提示:

1. 当弱口令扫描与口令复杂度功能同时开启时，且检查的口令既属于弱口令，又不符合口令复杂度要求，报错提示的内容与两个功能的加载顺序相关，报错提示会显示写在 shared_preload_libraries 中靠前的插件的提示内容。2. 当数据库中已创建了弱口令扫描的扩展 security_utils ，请使用数据库管理员进行备份恢复以避免报错导致备份恢复失败。

示例

开启弱口令扫描功能：

```
shared_preload_libraries = 'security_utils'

\c - system
create extension security_utils;

show security_utils.weak_pwd_check_enable;
security_utils.weak_pwd_check_enable
------------------------------------
off
```

```
(1 行记录)

alter system set security_utils.weak_pwd_check_enable = on;

select sys_reload_conf();
sys_reload_conf
-----------------
t
(1 行记录)

show security_utils.weak_pwd_check_enable;
security_utils.weak_pwd_check_enable
------------------------------------
on
(1 行记录)
```

在功能开启后，数据库管理员 system 登录数据库后，可以进行弱口令字典的配置：

```
\c - system
set weak password '123qwe';--成功
set weak password '123qwe','1234asdf';--失败，重复弱口令报错
错误:  weak passwd dic "123qwe" already exists
set weak password if not exists '123qwe','1234asdf';--成功，忽略已存在的弱口令

--查看弱口令配置
select * from security_utils.weak_pwd_dic;
 weak_passwd |         created_time
-------------+------------------------------
 123qwe      | 2022-08-01 15:11:21.102412+08
 1234asdf    | 2022-08-01 15:12:52.640116+08
(2 rows)
```

创建新用户 u1，使用弱口令作为口令：

```
\c - system
create user u1 with password '123qwe';--失败，口令不符合弱口令检查
错误:  password is too weak!
create user u1 with password '123qwer';--成功
alter user u1 with password '123qwe';
错误:  password is too weak!--失败，口令不符合弱口令检查
```

数据库管理员进行弱口令扫描：

```
\c - system
--将 u1 用户的口令设置为弱口令字典
set weak password '123qwer';
```

```
--进行弱口令扫描
select * from security_utils.weak_passwd_scan_result;
 username | weak_passwd
----------+-------------
 u1       | 123qwer
(1 row)
```

数据库管理员删除弱口令配置：

```
\c - system
--查看现有弱口令配置
select * from security_utils.weak_pwd_dic;
 weak_passwd |         created_time
-------------+------------------------------
 123qwe      | 2022-08-01 15:11:21.102412+08
 1234asdf    | 2022-08-01 15:12:52.640116+08
 123qwer     | 2022-08-01 15:22:12.404342+08
(3 rows)

--删除弱口令配置
remove weak password '123qwe';--成功
remove weak password '123qwe','1234asdf','123qwer';--失败，不存在的弱口令报错
错误:  weak passwd dic "123qwe" does not exists
remove weak password if exists '123qwe','1234asdf','123qwer';--成功，忽略不存在的弱口令

--查看弱口令字典
select * from security_utils.weak_pwd_dic;
 weak_passwd | created_time
-------------+--------------
(0 rows)
```

3.77.5　相关函数

3.77.5.1　1.check_guc_in_cluster()

功能

在集群环境中检查主备机之间安全相关 guc 参数是否相等。

返回值

返回 bool 型。

返回‘t’：表示 guc 一致，或者没有备机服务器；

返回‘f’：表示主备机 guc 不一致、不是在 esrep 数据库中、不是主机服务器，或者无 repmgr.nodes 表。

3.77.5.2　2.set_guc_in_cluster()

功能

在集群环境中将主机安全相关 guc 参数，设置到各个备机服务器中。

返回值

返回 bool 型。

返回‘t’：表示 guc 设置成功；

返回‘f’：表示不是在 esrep 数据库中、不是主机服务器，或者无 repmgr.nodes 表。

注意

1）如果无法检查主备机之间的 guc 参数，则会在日志中记录 command failed:xxxx。不影响程序查看比较后面的 guc 参数。

2）建议主备机配置文件 shared_preload_libraries 相同。

3.77.5.2.1　定时任务

集群在部署阶段设置 sync_security_guc 参数：

如果设置为 yes，则会在部署集群的时候创建 kdb_schedule、security_utils 这两个插件，并配置相关 guc 参数。集群环境通过 kdb_schedule 定时调用 security_utils.set_guc_in_cluster 函数，可实现集群数据库服务器自动同步主备机安全 guc 参数。

如果设置为 no，则不执行任何操作。默认值为 no。

3.77.6　插件 security_utils 卸载方法

通过 drop extension 来卸载插件：

```
drop extension security_utils;
```

如果设置了 shared_preload_libraries，则需要删除‘security_utils’：

```
shared_preload_libraries = ''
```

3.77.7　插件 security_utils 升级方法

security_utils 扩展插件随着 KingbaseES 安装包一并升级。但请注意，V008R006C006B0021 版本之前的版本，当使用 sys_upgrade 升级后无法正常使用弱口令功能时，请删除扩展并重新创建。

3.78 seg

3.78.1 插件 seg 简介

seg 是 KingbaseES 的一个扩展插件，这个插件可以为程序提供 seg 数据类型的使用，是为了表示线段或者浮点区间而实现的一种数据类型。

seg 可以表示区间端点中的不确定性，这使得它对表达实验室测量特别有用。

- 插件名为 seg
- 插件版本 V1.3

3.78.2 插件 seg 加载方式

KingbaseES 数据库默认将它添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

```
shared_preload_libraries = 'seg'
```

3.78.3 插件 seg 的参数配置

无需配置任何参数

3.78.4 插件 seg 的使用方法

加载 seg 插件后，程序内可以使用 seg 数据类型。

示例：

```
-- 创建插件

\c test system
create extension seg;
CREATE EXTENSION

-- 使用示例

SELECT '1.0e+7'::seg AS seg;
   seg
---------
 1.0e+07
(1 行记录)
```

```
SELECT '0.000000120'::seg AS seg;
   seg
----------
 1.20e-07
(1 行记录)

SELECT '0 .. 1'::seg AS seg;
  seg
--------
 0 .. 1
(1 行记录)

SELECT '24 .. 33.20'::seg != '24 .. 33.21'::seg AS bool;
 bool
------
 t
(1 行记录)
```

3.78.5　插件 seg 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.78.6　插件 seg 升级方法

seg 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.79　send_mails

3.79.1　插件 send_mails 简介

send_mails 是 KingbaseES 的一个扩展插件，主要用于通过 SMTP 协议，向指定收件人发送邮件，邮件内容自定义。

- 插件名为 send_mails
- 插件版本 V1.0

3.79.2　插件 send_mails 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例如下：

```
shared_preload_libraries = 'send_mails'
```

3.79.3　插件 send_mails 的参数配置

`send_mails.mail_server`

收件人邮件服务器的 IP 地址。参数为字符串类型，默认为空，不允许配置为空。

`send_mails.mail_port`

收件人邮件服务器的端口号。参数为 int 类型，参数范围是 [0,65535]，默认为 0，不允许配置为 0。

`send_mails.mail_login`

发件人登录的邮箱地址。参数为字符串类型，默认为空，不允许配置为空。

`send_mails.mail_password`

发件人的邮箱密码。参数为字符串类型，默认为空，不允许配置为空。

`send_mails.mail_from`

发件人的邮箱。参数为字符串类型，默认为空，不允许配置为空。

`send_mails.mail_to`

收件人的邮箱。参数为字符串类型，默认为空，不允许配置为空。

3.79.4　插件 send_mails 使用方法

KingbaseES 加载 send_mails 插件，打开审计入侵检测邮件告警开关 sysaudit.ids_mail_enable，配置上述邮件发送 send_mails 扩展的参数后，实现邮件告警功能。

示例

```
---step1: 开启审计、审计入侵、发送邮件开关
\c - sao
alter system set sysaudit.enable=on;
alter system set sysaudit.ids=on;
alter system set sysaudit.ids_mail_enable=on;
select sys_reload_conf();
```

```
---step2: 设置邮件配置
\c - system
alter system set send_mails.mail_server = '192.168.4.9';
alter system set send_mails.mail_port = '25';
alter system set send_mails.mail_login = 'test@kingbase.com';
alter system set send_mails.mail_password = '12345678ab***';
alter system set send_mails.mail_from = 'test@kingbase.com';
alter system set send_mails.mail_to = 'test@kingbase.com';
select sys_reload_conf();

---step3: 设置审计规则
\c - system
drop table a;
create table a(id int primary key);
\c - sao
select sysaudit.set_audit_stmt('insert table','system','','');

---step4: 设置审计入侵规则
SELECT sysaudit.create_ids_rule('rule1','insert table','','public','a','SUCCESSFUL','','',10,3);

---step5: 进行触发入侵规则操作
\c - system
insert into a values(1);
insert into a values(1);
insert into a values(2);
insert into a values(3);

---step6: 查看邮件
-- Instrusion Detection Report
-- User name : system
-- Rule name : rule1
-- Illegal Action : insert table
-- SQL text : 'insert into a values(3);'
-- Instrusion time : 2023-08-14 19:56:04
-- Please note
\c - sao
select sysaudit.drop_ids_rule('rule1');
```

3.79.5　插件 send_mails 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例如下：

```
shared_preload_libraries = ''
```

3.79.6 插件 send_mails 升级方法

send_mails 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.80 sepapower

3.80.1 插件 sepapower 简介

sepapower 是 KingbaseES 的一个扩展插件，主要用于支持三权分立功能。三权分立插件默认加载。

具体的三权分立功能介绍，参考《KingbaseES 数据库安全指南》三权分立章节。

- 插件名为 sepapower
- 插件版本 V1.0

3.80.2 插件 sepapower 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'sepapower'
```

3.80.3 插件 sepapower 的参数配置

`sepapower.separate_power_grant`
是否由 sso 用户控制使用 DCL 语句，取值范围为 true 或者 false ，缺省值为 false。

```
\c - system
show sepapower.separate_power_grant;
sepapower.separate_power_grant
--------------------------------
 off
(1 row)

alter system set sepapower.separate_power_grant = on;
```

```
--重启数据库实例
./sys_ctl -D data restart

show sepapower.separate_power_grant;
sepapower.separate_power_grant
--------------------------------
 on
(1 row)
```

sepapower.check_noprivileges_grant
是否对无授权权限操作进行报错处理，取值范围为 true 或者 false ，缺省值为 false。

```
show sepapower.check_noprivileges_grant;
sepapower.check_noprivileges_grant
------------------------------------
 off
(1 row)

alter system set sepapower.check_noprivileges_grant = on;
select sys_reload_conf();

show sepapower.check_noprivileges_grant;
sepapower.check_noprivileges_grant
------------------------------------
 on
(1 row)
```

3.80.4　插件 sepapower 使用方法

具体参考《KingbaseES 数据库安全指南》三权分立章节。

3.80.5　插件 sepapower 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.80.6 插件 sepapower 升级方法

sepapower 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.81 src_restrict

3.81.1 插件 src_restrict 简介

src_restrict 是 KingbaseES 的一个扩展插件，主要用于支持来源限制功能，该功能通过 hba 的黑白名单来实现。插件 src_restrict 默认加载。

- 插件名为 src_restrict
- 插件版本 V1.0

3.81.2 插件 src_restrict 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'src_restrict'
```

3.81.3 插件 src_restrict 的参数配置

src_restrict.enable
是否允许使用黑白名单功能，取值范围为 true 或者 false ，缺省值为 true。

```
\c - sso
show src_restrict.enable;
 src_restrict.enable
---------------------
 on
(1 row)

alter system set src_restrict.enable = off;
select sys_reload_conf();
show src_restrict.enable;
 src_restrict.enable
---------------------
```

```
 off
(1 row)
```

3.81.4　插件 src_restrict 使用方法

3.81.4.1　系统函数 src_restrict.add_rules

```
src_restrict.add_rules(
filter_type int,
rule_user text,
rule_ip text,
outrule_ip text,
limited_info text)
```

功能说明

添加来源限制的规则，可通过参数选择配置白名单或黑名单。

参数说明

`filter_type`

限制规则的类型，0 为白名单，1 为黑名单。

`rule_user`

限制的用户名，多个用户时需使用逗号分隔，null 表示限制所有用户。当用户名中存在特殊字符时，请将该用户名使用双引号括起来，避免用户名识别失败。

用户名参数最大长度 255。

`rule_ip`

限制的 IP，多个 IP 之间需使用逗号分隔，null 表示限制所有 IP。

允许使用 * 号进行 IP 网段匹配，但网段后不允许出现具体 IP，例如 192.168.*.123。

`outrule_ip`

不受限制的 IP，与 rule_ip 相同。

不受限制仅表示该参数与 rule_ip 的类型相反，例如 filter_type 为 0，则 rule_ip 为白名单，outrule_ip 为黑名单，不影响黑白名单优先级。

`limited_info`

限制规则，内容中包含 IP 段，mac 地址，时间段，分别使用关键字 ip_range、mac、time_range 进行标识。

关键字使用“=”号连接具体的规则内容，同一关键字内的内容，如果想输入多条，用“/”号连接，不同关键字之间用“;”号连接。填写格式为“ip_range=XXXXXX~XXXXXX/XXXXXX~XXXXXX;mac=XXXXX/XXXXX;time_range=XXXXXX~XXXXXX/XXXXXX~XXXXXX”。其中三个字段可以选择填写，也可以置空。

返回值说明

无返回值

3.81.4.2　系统函数 src_restrict.remove_rules

```
src_restrict.remove_rules(
filter_type int,
rule_user text,
rule_ip text
limited_info text)
```

功能说明

删除来源限制的规则。

参数说明

`filter_type`

规则的类型，0 为白名单，1 为黑名单。

`rule_user`

规则的用户名，多个用户时需使用逗号分隔，null 等于 all。当用户名中存在特殊字符时，请将该用户名使用双引号括起来，避免用户名识别失败。

用户名参数最大长度 255。

`rule_ip`

规则的 IP，只允许删除一个 IP，不支持删除多个规则，null 等于 0.0.0.0/0。

允许使用 * 号进行 IP 网段匹配，但网段后不允许出现具体 IP，例如 192.168.*.123。

`limited_info`

限制规则，内容中包含 IP 段，mac 地址，时间段，分别使用关键字 ip_range、mac、time_range 进行标识。

关键字使用“=”号连接具体的规则内容，同一关键字内的内容，如果想输入多条，用“/”号连接，不同关键字之间用“;”号连接。填写格式为“ip_range=XXXXXX~XXXXXX/XXXXXX~XXXXXX;mac=XXXXX/XXXXX;time_range=XXXXXX~XXXXXX/XXXXXX~XXXXXX”。其中三个字段可以选择填写，也可以置空。

返回值说明

无返回值

提示:

删除函数中，要删除的规则中类型，用户名（包括用户个数），规则 IP 都要与规则一一对应，否则将无法删除。

3.81.4.3　系统视图 src_restrict.show_rules

所有用户都可以通过该视图查看当前的限制规则。

```
select * from src_restrict.show_rules;
```

参数说明

`filter_type`

规则的类型，allow 为白名单，reject 为黑名单。

`rule_user`

规则的用户名。

`rule_ip`

规则的 IP。

`netmask`

规则 IP 对应的子网掩码。

`login_limited_info`

登录限制信息（包括 IP 段，mac 地址，日期时间段）。

3.81.4.4　示例

```
\c - sso

--添加对用 u1 户添加黑名单规则
select src_restrict.add_rules(1,'u1','10.12.1.*','','ip_range=10.12.1.10~10.12.1.29/10.12.1.30~10.12.1.40;
mac=d8:bb:c1:c3:77:23/d8:bb:c1:c3:77:25;time_range=20200101-12:01:07~20210910-03:04:05/20220101-12:01:07~
20260910-03:04:05');


--查看当前黑白名单
select * from src_restrict.show_rules;
```

```
--删除刚添加的规则
select src_restrict.remove_rules(1,'u1','10.12.1.*','ip_range=10.12.1.10~10.12.1.29/10.12.1.30~10.12.1.40;
mac=d8:bb:c1:c3:77:23/d8:bb:c1:c3:77:25;time_range=20200101-12:01:07~20210910-03:04:05/20220101-12:01:07~
20260910-03:04:05');
```

3.81.5　插件 src_restrict 卸载方法

插件 src_restrict 是系统初始化插件，用户无法卸载。

3.81.6　插件 src_restrict 升级方法

src_restrict 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.82　sslinfo

3.82.1　插件 sslinfo 简介

sslinfo 是 KingbaseES 的一个扩展插件，sslinfo 插件提供当前客户端提供的 SSL 证书的有关信息。如果当前连接不使用 SSL，这个模块就没有用处（大部分函数将返回 NULL）。

本模块提供的一些信息，也可以使用内置系统视图 sys_stat_ssl 来获得。

- 插件名为 sslinfo
- 插件版本 V1.2

3.82.2　插件 sslinfo 加载方式

KingbaseES 数据库可将它添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

```
shared_preload_libraries = 'sslinfo'
```

3.82.3　插件 sslinfo 的参数配置

无需配置任何参数

3.82.4　插件 sslinfo 的使用方法

加载 sslinfo 插件后，程序可以提供当前客户端提供的 SSL 证书的有关信息。

示例：

```
-- 创建插件
\c test system
create extension sslinfo;
CREATE EXTENSION
```

3.82.5　插件 sslinfo 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.82.6　插件 sslinfo 升级方法

sslinfo 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.83　sso_update_user

3.83.1　插件 sso_update_user 简介

sso_update_user 是 KingbaseES 的一个扩展插件，可以满足某些特殊场景的三权分立需求，对管理员和安全员的权限做出进一步的划分。

- 插件名为 sso_update_user
- 插件版本 V1.0

3.83.2　插件 sso_update_user 加载方式

KingbaseES 数据库默认将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，重启数据库时自动加载。

示例：

```
shared_preload_libraries = 'sso_update_user'
```

3.83.3　插件 sso_update_user 的参数配置

`sso_update_user.sso_update_user_enable` 默认为关闭状态。
切换用户为安全管理员 sso，打开插件开关。

```
show sso_update_user.sso_update_user_enable;

sso_update_user.sso_update_user_enable
--------------------------------------
off
(1 行记录)

\c - sso
alter system set sso_update_user.sso_update_user_enable = true;
ALTER SYSTEM

select sys_reload_conf();
sys_reload_conf
-----------------
t
(1 行记录)

show sso_update_user.sso_update_user_enable;
sso_update_user.sso_update_user_enable
--------------------------------------
on
(1 行记录)
```

3.83.4　插件 sso_update_user 使用方法

具体参考《KingbaseES 数据库安全指南》插件增强支持章节。

3.83.5　插件 sso_update_user 卸载方式

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.83.6　插件 sso_update_user 升级方法

sso_update_user 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.84 sys_anon

3.84.1 插件 sys_anon 简介

sys_anon 是 KingbaseES 的一个扩展插件，主要用于支持数据脱敏功能和应用级敏感数据保护。

数据脱敏，指对某些敏感信息通过脱敏规则进行数据的变形，实现敏感隐私数据的可靠保护。具体参考《KingbaseES 数据库安全指南》数据脱敏。

- 插件名为 sys_anon
- 插件版本 V1.0

3.84.2 插件 sys_anon 加载方式

在使用 sys_anon 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

示例：

```
shared_preload_libraries = 'sys_anon'
```

3.84.3 插件 sys_anon 的参数配置

`anon.enable`
启动数据脱敏功能的开关，取值范围为 true 或者 false ，缺省值为 false。

```
show anon.enable;
 anon.enable
-------------
 off
(1 row)

\c - sso
alter system set anon.enable = on;
select sys_reload_conf();
show anon.enable;
 anon.enable
-------------
 on
(1 row)
```

3.84.4　插件 sys_anon 使用方法

具体参考《KingbaseES 数据库安全指南》数据脱敏。

3.84.5　插件 sys_anon 卸载方法

通过 drop extension 来卸载插件：

```
drop extension sys_anon;
```

如果设置了 shared_preload_libraries，则需要删除‘sys_anon’：

```
shared_preload_libraries = ''
```

3.84.6　插件 sys_anon 升级方法

具体参考《KingbaseES 数据库安全指南》数据脱敏。

3.85　sys_audlog

3.85.1　插件 sys_audlog 简介

sys_audlog 是一个 KingbaseES 的扩展，可以实现帐户异常登录锁定以及账户登录信息显示。这种方式更为灵活，当数据库的实用场景需要进行帐户异常登录锁定以及账户登录信息显示时，加载插件即可。而不需要该功能时，卸载插件即可。

sys_audlog 的相关参数由数据库安全员负责配置。

- 插件名为 sys_audlog
- 插件版本 V1.0

3.85.2　插件 sys_audlog 加载方式

在使用 sys_audlog 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'sys_audlog'
```

3.85.3　插件 sys_audlog 的参数配置

- sys_audlog.error_user_connect_times

  允许用户连续登录失败的最大次数，用户登录失败的次数大于超过该值，用户自动锁定，取值范围为 [0,INT_MAX]，缺省为 0。

- sys_audlog.max_error_user_connect_times

  用户登录失败次数的最大值界限，error_user_connect_times 的最大取值，取值范围为 [0,INT_MAX]，缺省为 2147483647。

- sys_audlog.error_user_connect_interval

  用户被锁定时间，若用户被锁定的时间超过了该参数，则该用户可自动解锁。单位是分钟，取值范围为 [0，INT_MAX]，0 时关闭自动解锁功能，需手动解锁，缺省为 0。

3.85.4　插件 sys_audlog 的使用方法

具体参考《KingbaseES 安全指南》“账户异常登录锁定”章节和“账户登录信息显示”章节。

3.85.5　插件 sys_audlog 卸载方法

修改 kingbase.conf 文件中 shared_preload_libraries 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.85.6　插件 sys_audlog 升级方法

sys_audlog 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.86　sys_buffercache

3.86.1　插件 sys_buffercache 简介

插件 sys_buffercache 是 KingbaseES 的一个扩展插件。提供了一种方法用来实时检查共享缓冲区。

- 插件名为 sys_buffercache
- 插件版本 V1.3

3.86.2　插件 sys_buffercache 加载方式

通过 create extension 来加载插件。

示例：

```
create extension sys_buffercache;
```

3.86.3　插件 sys_buffercache 的参数配置

无需配置任何参数。

3.86.4　插件 sys_buffercache 的使用方法

KingbaseES 加载 sys_buffercache 插件后，通过视图 sys_buffercache 查看。

sys_buffercache 视图字段说明：

名称	类型	描述
bufferid	integer	ID，在范围 1..shared_buffers 中
relfilenode	oid	关系的文件节点编号
reltablespace	oid	关系的表空间 OID
reldatabase	oid	关系的数据库 OID
relforknumber	smallint	关系内的分支编号
relblocknumber	bigint	关系的页码
isdirty	boolean	页面是否为脏页
usagecount	smallint	Clock-sweep 访问计数
pinning_backends	integer	固定此缓冲区的后端数

共享缓存中的每一个缓冲区都有一行。未使用的缓冲区的行中显示为除 bufferid 之外的所有字段都为空。共享的系统目录被显示为属于数据库零。

因为缓冲是所有数据库共享的，通常会有不属于当前数据库的关系的页面。这意味着对于一些行在 sys_class 中可能不会有匹配的连接行，或者甚至有错误的连接。如果你试图与 sys_class 连接，将连接限制于 reldatabase 等于当前数据库 OID 或零的行是一个好主意。

当访问 sys_buffercache 视图时，内部缓冲区管理器会被锁住足够长时间来拷贝视图将显示的所有缓冲区状态数据。这确保了该视图会产生一个一致的结果集合，而不会不必要地长时间阻塞普通的缓冲区活动。尽管如此，如果经常读取这个视图还是会对数据库性能产生一些影响。

3.86.5　插件 sys_buffercache 卸载方法

通过 drop extension 来卸载插件。

示例：

```
drop extension sys_buffercache;
```

3.86.6　插件 sys_buffercache 升级方法

通过 alter extension 来升级插件。

示例，由 1.2 升级到 1.3：

```
ALTER EXTENSION sys_buffercache UPDATE TO '1.3';
```

3.87　sys_bulkload

3.87.1　插件 sys_bulkload 简介

sys_bulkload 是一种用于 KingbaseES 快速加载数据的工具。

- 插件名为 sys_bulkload
- 插件版本 V2.0

3.87.2　插件 sys_bulkload 加载方式

启动 KingbaseES 数据库后执行 SQL 命令创建插件。

示例：

```
create extension sys_bulkload;
```

3.87.3 插件 sys_bulkload 的参数配置

无需配置任何参数。

3.87.4 插件 sys_bulkload 的使用方法

KingbaseES 加载 sys_bulkload 插件后，无法单独使用，需要与客户端 sys_bulkload 工具配合使用，详细使用说明请参考客户端 sys_bulkload 工具 。

3.87.5 插件 sys_bulkload 卸载方法

在数据库中执行 SQL 命令卸载插件。

示例：

```
drop extension sys_bulkload;
```

3.87.6 插件 sys_bulkload 升级方法

sys_bulkload 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级插件。

3.88 sys_freespacemap

3.88.1 插件 sys_freespacemap 简介

sys_freespacemap 提供了查看 FSM 页面的方法。插件提供函数来显示 FSM 页面中记录的指定关系指定页面的值或是指定关系全部页面的值。默认仅限超级用户、sys_stat_scan_tables 角色的成员使用。

- 插件名为 sys_freespacemap
- 插件版本 V1.2

3.88.2 插件 sys_freespacemap 加载方式

插件为内置插件，自动加载。

3.88.3 插件 sys_freespacemap 的参数配置

无需配置任何参数

3.88.4　插件 sys_freespacemap 的使用方法

sys_freespacemap 提供以下函数：

sys_freespace(rel regclass IN, blkno bigint IN) returns int2
返回指定关系指定块在 FSM 页面中记录的空闲空间值。

sys_freespace(rel regclass IN, blkno OUT bigint, avail OUT int2)
返回指定关系中每个块在 FSM 页面中记录的空闲空间值。返回 (blkno bigint, avail int2) 类型的集合，每一行对应关系中的一个块。

FSM 页面中存储的空闲空间值并不是准确的。值会按块大小的 1/256 四舍五入，而且值在表页面被修改时不会实时更新。对于索引来说，FSM 页面记录的是空闲页面而不是页面中的空闲空间，返回值只代表页面是否为整个空闲。

3.88.5　插件 sys_freespacemap 卸载方法

插件为内置插件，不允许卸载。

3.88.6　插件 sys_freespacemap 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION sys_freespacemap UPDATE TO '1.1';
```

3.89　sys_jieba

3.89.1　插件 sys_jieba 简介

sys_jieba 是一个 KingbaseES 的扩展，主要用于全文搜索的中文分词。sys_jieba 中文分词插件支持 utf8 字符集。

- 插件名为 sys_jieba
- 插件版本 V1.0

3.89.2　插件 sys_jieba 加载方式

```
CREATE EXTENSION sys_jieba;
```

3.89.3　插件 sys_jieba 的参数配置

无需配置任何参数

3.89.4　插件 sys_jieba 的使用方法

```
CREATE EXTENSION sys_jieba;

SELECT * FROM ts_parse('jieba', 'KingbaseES 中文分词插件测试');
 tokid |   token
-------+------------
     2 | KingbaseES
     2 | 中文
     2 | 分词
     2 | 插件
     2 | 测试
(5 rows)

SELECT to_tsvector('jiebacfg','KingbaseES 中文分词插件测试');
                      to_tsvector
-----------------------------------------------------------
 ' ':2 kingbasees':1 '中文':2 '分词':3 '插件':4 '测试':5
(1 row)

SELECT to_tsquery('jiebacfg', '分词');
 to_tsquery
------------
 '分词'
(1 row)
```

3.89.5　插件 sys_jieba 的卸载方法

```
drop extension sys_jieba;
```

3.89.6 插件 sys_jieba 升级方法

sys_jieba 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.90 sys_kwr

3.90.1 插件 sys_kwr 简介

sys_kwr 是 Kingbase Auto Workload Repertories 的简称，该插件是 KingbaseES 的一个扩展插件。主要功能是通过周期性自动记录性能统计相关的快照，分析 KingbaseES 的操作系统运行环境、数据库时间组成、等待事件和 TOP SQL 等性能指标，为数据库性能调优提供指导。

sys_kwr 基于性能快照提供的性能报告包括：KWR、KWR DIFF、KSH、KDDM，详细介绍可以参考《性能调优指南》。

- 插件名为 sys_kwr。
- 当前最新可用版本为 1.9

信息:

自 1.6 版本起，sys_ksh 插件合入 sys_kwr 插件，sys_ksh 插件不再作为单独插件使用。

注意:

在主备切换后，sys_kwr 生成的第一个快照的快照数据可能不准确，使用时请予以留意。

3.90.2 插件 sys_kwr 加载方式

KingbaseES 数据库默认将 sys_kwr 添加到 kingbase.conf 文件的 shared_preload_libraries 中，初始化数据库时自动加载。

示例如下：

```
shared_preload_libraries = 'liboracle_parser, sys_kwr, sys_stat_statements'
```

3.90.3 插件 sys_kwr 的参数配置

KWR、KWR DIFF 相关参数：

参数名	默认值	介绍
sys_kwr.enable	off	控制是否开启 KWR 自动快照
sys_kwr.topn	20	显示 KWR 报告中排名前 n 条的信息，最少为 10，最多为 100
sys_kwr.history_days	8	快照保留时间，最少 1 天，最多 1000 天
sys_kwr.interval	60	自动快照创建间隔，最短 10 分钟，最长 144000 分钟（100 天）
sys_kwr.language	‘chinese’	KWR 报告、KWR DIFF 报告使用的语言，默认为中文（chinese 或 chn），可设置为英文（english 或 eng）
sys_kwr.database	‘kingbase’	sys_kwr 1.8 版本新增参数，用于指定创建自动快照的数据库。配置该参数后，创建自动快照的进程会连接至目标数据库并周期性地创建 KWR 快照，同时采集包括数据库对象统计在内的全部性能统计指标 - 参数为 TEXT 类型，区分大小写，且仅可配置一个数据库 - 参数默认值为’kingbase’，如果不设置该参数，则系统使用默认的 kingbase 库创建自动快照
sys_kwr.track_objects	off	控制是否统计数据库对象的使用情况，在 KWR 1.7 版本之前默认值为 on，自 KWR 1.7 版本之后默认值为 off
sys_kwr.track_os	on	控制是否统计操作系统运行数据
sys_kwr.track_windows_os_info	on	控制在 Windows 平台下是否统计操作系统运行数据。若该参数为 off，则每次连接 KWR 插件所在的数据库时，会消耗大约 300 毫秒用于初始化 Windows 下的 WMI 库
sys_kwr.snapshot_max_rows	2147483647	KWR 快照一次最多采集的条目数

KSH 相关参数：

参数名	默认值	介绍
sys_kwr.collect_ksh	off	控制是否开启 KSH 数据采集
sys_kwr.ringbuf_size	100000	KSH 内存环形缓冲区的大小，该参数决定了 ksh 在内存中可以存放多少条数据
sys_kwr.history_days	8	KSH 历史数据在磁盘上的保留时间

见续表

表 3.90.2 – 续表

参数名	默认值	介绍
sys_kwr.language	‘chinese’	KSH 报告使用的语言
sys_kwr.database	‘kingbase’	sys_kwr 1.8 版本新增参数，用于设置在指定的数据库中收集 KSH 统计信息并生成报告。该参数为 TEXT 类型，区分大小写，且仅可配置一个数据库

信息:

在 sys_kwr 1.5 版本之前（包括 1.5 版本），sys_ksh 插件作为独立插件存在，其典型用法如下所示：

```
# kingbase.conf
shared_preload_libraries = 'liboracle_parser, sys_stat_statements, sys_ksh'
sys_stat_statements.track = 'top' # default = 'none'
track_activities = on             # default = on
sys_ksh.enable = on               # default = off
sys_ksh.language = 'chinese'
```

KDDM 相关参数：

参数名	默认值	介绍
sys_kwr.language	‘chinese’	指定 KDDM 报告使用的语言，TEXT 类型

信息:

在 sys_kwr 1.5 版本之前（包括 1.5 版本），存在参数 sys_kddm.language 独立指定 KDDM 报告使用的语言，之后版本中该参数与 KWR 报告合并。

3.90.4　插件 sys_kwr 的使用方法

3.90.4.1　KWR 功能

3.90.4.1.1　KWR 功能概述

KWR 的基本原理：数据库实例运行过程中不断产生一些统计数据，比如对某个表的访问次数，数据页的内存命中次数，某个等待事件发生的次数和总时间，SQL 语句的解析时间等，这些统计数据 KingbaseES 服务器的后台性能

监控进程 kwr collector 周期性地（默认每小时）自动采集，存储至 KWR 快照库中，这些快照默认保存 8 天，到期后旧的快照被自动删除。

当 KingbaseES 出现性能问题的时，可以使用快照列表中对应时间段的快照序列生成 KWR 报告，辅助数据库管理员定位性能问题的根本原因。KWR 快照中主要包含如下几类数据：

1. SQL 语句执行过程中产生的等待事件，I/O 和执行时间
2. TOP SQL
3. 数据库对象统计信息
4. 数据库实例统计信息

KDDM 基于 KWR 快照提供性能诊断和建议的报告。

3.90.4.1.2　KWR 推荐参数配置

为了正常使用 KWR 的报告功能，建议在 kingbase.conf 中配置以下参数；其中，shared_preload_libraries 根据实际需要配置即可：

```
shared_preload_libraries = 'liboracle_parser, sys_kwr, sys_stat_statements'
sys_kwr.enable = on
track_counts = on
track_io_timing = on
track_functions = 'all'
sys_stat_statements.track = 'top'
track_sql = on
track_instance = on
track_sql_item = 'sqltime, sqlwait, sqlio'
track_instance_item = 'instevent, instio, instlock'
```

上述参数的意义如下，统计相关参数见配置管理手册的运行时统计数据章节，sys_stat_statements 相关参数见sys_stat_statements 插件手册，其它参数一般情况下使用默认值即可，具体可见插件 sys_kwr 的参数配置。

信息:

- 开启自动快照功能需要将sys_kwr.enable设置为 on。
- 默认在 kingbase 库上创建 sys_kwr 插件，可以使用 GUC 参数sys_kwr.database进行设置。
- shared_preload_libraries 中应至少包含：sys_kwr, sys_stat_statements。

3.90.4.1.3　KWR 快照

3.90.4.1.3.1　自动快照

KingbaseES 启动服务后，kwr collector 后台进程将每个小时创建一次性能快照，并默认保存至 kingbase 库中。

一般情况下，用户无需要手动删除已有快照，KWR 会在每次创建新快照时，自动删除存在时间超过sys_kwr.history_days天数的快照，避免因快照数据过多导致性能问题。

查询快照列表：

```
SELECT snap_id, sess_count, snap_version FROM perf.kwr_snapshots;
 snap_id | sess_count | snap_version
---------+------------+--------------
       1 |        103 | 1.9
       2 |        103 | 1.9
       3 |        102 | 1.9
       4 |        102 | 1.9
(4 行记录)
```

- 仅需将sys_kwr.enable设置为 on（需要重启服务器）便会开启自动快照功能。
- 后台进程 kwr collector 每分钟会检查最新快照的创建时间与当前时间的时间间隔是否大于sys_kwr.interval，若大于，则创建新快照，并删除超过保留时间的快照。
- 若本次自动快照没有创建成功，则在下一个检查周期（一分钟后）继续尝试创建快照。

3.90.4.1.3.2　手动快照

手动快照通过用户手动执行 SQL 语句创建快照。

```
SELECT * FROM perf.create_snapshot();
create_snapshot
-----------------
                5
(1 行记录)
```

相关 GUC 参数配置完毕后，运行 perf.create_snapshot() 函数即可手动创建快照，并返回新的快照号，同时在 perf.kwr_snapshots 表中新增一条快照记录。

信息:

任何情况下都可以执行 `SELECT * FROM perf.create_snapshot();` 手动创建快照。

3.90.4.1.3.3　快照管理

sys_kwr 插件提供两个管理接口用于删除快照数据：

1. 删除指定快照数据：`perf.drop_snapshots({start_id}, {end_id})`
2. 删除全部快照数据：`perf.reset_snapshots()`

参数说明：

- start_id：被删除快照的起始快照 ID。
- end_id：被删除快照的结束快照 ID。

示例：

```
SELECT * FROM perf.drop_snapshots(1, 3);
SELECT * FROM perf.reset_snapshots();
```

3.90.4.1.4　KWR 报告

3.90.4.1.4.1　生成报告

存在 KWR 快照数据后，便可调用以下 2 个函数生成 KWR 报告：

1. 生成 KWR 报告并将报告保存至 KingbaseES 的 data/sys_log 目录下：

```
SELECT * FROM perf.kwr_report(start_id integer, end_id integer, format text = 'text', database text = NULL);
```

2. 生成 KWR 报告并将报告保存至指定的 file_path 路径下：

```
SELECT * FROM perf.kwr_report_to_file(start_id integer, end_id integer, format text, file_path text,
database text = NULL);
```

函数参数说明：

- start_id：起始快照 ID。
- end_id：结束快照 ID。
- format：报告格式，目前仅支持’text’和’html’格式，建议使用’html’格式。
- file_path：在服务器上保存报告文件的路径，用户在该路径下必须有写文件的权限。
- database：用于设置在报告中按数据库展示性能指标，仅 sys_kwr 1.8 及以上版本支持。
  - 如果不指定该参数，报告将展示全实例的性能指标。
  - 如果指定了该参数，报告将展示指定数据库的性能指标。
  - 如果指定的数据库不存在，系统将给出错误提示。

生成 KWR 报告示例：

1. 生成全实例 TEXT 格式的 KWR 报告：

```
SELECT * FROM perf.kwr_report(1,9,'text');
```

2. 生成指定数据库（以 kingbase 为例）TEXT 格式的 KWR 报告：

```
SELECT * FROM perf.kwr_report(1,9,'text','kingbase');
```

3. 生成全实例 HTML 格式的 KWR 报告：

```
SELECT * FROM perf.kwr_report(1,9,'html');
```

4. 生成指定数据库（以 kingbase 为例）HTML 格式的 KWR 报告：

```
SELECT * FROM perf.kwr_report(1,9,'html','kingbase');
```

信息:

- 若不指定保存路径，生成的 KWR 报告将自动保存至 data 目录的 sys_log 子目录下，可以通过浏览器查看 HTML 格式的 KWR 报告。
- KWR 报告的分析，请参考《性能调优指南》的 KWR 报告部分。

---

5. 生成全实例 HTML 格式的 KWR 报告并将报告保存至指定路径下：

```
SELECT * FROM perf.kwr_report_to_file(1,9,'html','/home/kingbase/kwr_1_9.html');
```

3.90.4.1.4.2　常见问题

1. 目前无法在高可用备节点上创建 KWR 快照：

```
kingbase=# SELECT * FROM perf.create_snapshot();
ERROR:  kwr can not run on standby
背景:  在 RAISE 的第 5 行的 PL/pgSQL 函数 kwr_check_env()
SQL 语句 "SELECT kwr_check_env()"
在 PERFORM 的第 31 行的 PL/pgSQL 函数 create_snapshot()
```

也无法在备节点上生成 KWR 报告：

```
kingbase=# SELECT * FROM perf.kwr_report(1,9,'html');
ERROR:  kwr can not run on standby
背景:  在 RAISE 的第 5 行的 PL/pgSQL 函数 kwr_check_env()
SQL 语句 "SELECT kwr_check_env()"
在 PERFORM 的第 13 行的 PL/pgSQL 函数 kwr_generate_report(integer,integer,text,text)
在赋值的第 6 行的 PL/pgSQL 函数 kwr_report(integer,integer,text,text)
```

2. 如果别的会话正在当前库上创建 KWR 快照（尚未完成），为防止快照管理数据混乱，此时不允许创建快照，必须等待正在创建的快照创建完成：

```
kingbase=# SELECT * FROM perf.create_snapshot();
NOTICE:  Wait another KWR snapshot finish
 create_snapshot
-----------------
```

```
             -1
(1 行记录)
```

3. KWR 完整报告依赖五个重要统计开关：track_sql = on、track_instance = on、sys_stat_statements.track = ‘top’、track_counts = on、sys_kwr.track_os = on，否则将提示以下警告，且生成的 KWR 报告部分子报告内容为空：

```
kingbase=# SELECT * FROM perf.create_snapshot();
NOTICE:  sys_stat_statements.track is not top
NOTICE:  track_sql is not on
NOTICE:  track_instance is not on
NOTICE:  track_counts is not on
NOTICE:  sys_kwr.track_os is not on
 create_snapshot
-----------------
               2
(1 行记录)
```

4. 如果指定的快照不存在，生成报告时会报错：

```
kingbase=# SELECT * FROM perf.kwr_report(1,9,'html');
ERROR:  The end snapshot 9 does not exist
背景：  在 RAISE 的第 33 行的 PL/pgSQL 函数 check_snapid(integer,integer)
SQL 语句 "SELECT check_snapid(start_id, end_id)"
在 PERFORM 的第 25 行的 PL/pgSQL 函数 kwr_generate_report(integer,integer,text,text)
在赋值的第 6 行的 PL/pgSQL 函数 kwr_report(integer,integer,text,text)
```

5. 目前，存在部分统计信息 sys_kwr 插件无法在 Windows 平台采集，导致 KWR 报告中的部分内容为空。

6. 删除快照时，最新的快照将作为基线而不能被删除：

```
kingbase=# SELECT * FROM perf.drop_snapshots(3,5);
NOTICE:  The last snapshot 5 is always kept for baseline
 drop_snapshots
----------------

(1 行记录)
```

3.90.4.2　KWR DIFF 功能

3.90.4.2.1　KWR DIFF 功能概述

KWR 报告是 KWR DIFF 报告的基础，在数据库运行过程中，通常在业务的高峰期和低谷期，或者在数据库 GUC 参数调整之后，数据库的性能指标会发生变化，分析这种变化的根本原因对于数据库管理员分析和优化数据库

性能是非常必要的。KWR DIFF 报告用于分析两个 KWR 报告之间的差异，找出性能变化的原因或分析性能优化的效果。

KWR DIFF 报告仅支持 HTML 格式输出。

3.90.4.2.2　KWR DIFF 推荐参数配置

KWR DIFF 报告属于 KWR 的一个子模块，继承 KWR 的现有 GUC 参数，具体请参考KWR 相关 GUC 配置参数的说明。

3.90.4.2.3　KWR DIFF 报告

生成 KWR DIFF 报告的函数接口包含如下两个：

- `perf.kwr_diff_report_to_file({snap_1}, {snap_2}, {snap_3}, {snap_4}, {file_path}, {database})`：该函数可在用户指定的目录下生成 KWR DIFF 报告。
- `perf.kwr_diff_report({snap_1}, {snap_2}, {snap_3}, {snap_4}, {database})`：该函数在 data 目录的 sys_log 子目录中生成 KWR DIFF 报告，同时在 KSQL 中生成在线报告。若 sys_log 目录不存在，则仅生成在线报告。报告的默认命名格式为：kwr_diff_1_2_3_4_20220531_140633.html，即：kwr_diff_4 个快照 ID_年月日 _ 时分秒.html。

通过 KWR 自动快照或手动快照创建一系列 KWR 快照后，可通过上述函数创建 KWR DIFF 报告。

参数说明：

- snap_1：生成第一份 KWR 报告的起始快照 ID。
- snap_2：生成第一份 KWR 报告的结束快照 ID。
- snap_3：生成第二份 KWR 报告的起始快照 ID。
- snap_4：生成第二份 KWR 报告的结束快照 ID。
- file_path：报告的保存路径，例如：‘/home/kingbase/kwr_diff_report.html’。
- database：指定生成报告的数据库，例如：‘kingbase’。若不指定该参数，则生成全实例的报告，该参数仅 sys_kwr 1.8 及以上版本支持。

示例：

```
CREATE EXTENSION sys_kwr;
SELECT * FROM perf.create_snapshot();
CREATE TABLE t1(a int);
INSERT INTO t1 SELECT generate_series(1, 1000000);
SELECT * FROM perf.create_snapshot();
INSERT INTO t1 SELECT generate_series(1, 1000000);
SELECT * FROM perf.create_snapshot();
INSERT INTO t1 SELECT generate_series(1, 1000000);
SELECT * FROM perf.create_snapshot();
SELECT * FROM perf.kwr_diff_report_to_file(1, 2, 3, 4, '/home/kingbase/kwr_diff_rpt_00.html');
```

3.90.4.3　KSH 功能

3.90.4.3.1　KSH 功能概述

KSH 以每秒采样的方式进行活跃会话的数据收集，并将采集数据存入内存环形缓冲区中，采集的数据主要包括：会话、应用、等待事件、命令类型、QueryId 等。KSH 的主要使用场景为：当前或历史的某个时间点，发生了什么样的异常，系统在运行什么任务。

3.90.4.3.2　KSH 推荐参数配置

KSH 功能相关的 GUC 参数在 kingbase.conf 中的推荐设置：

```
shared_preload_libraries = 'liboracle_parser, sys_stat_statements, sys_kwr'
track_activities = on              # default = on
sys_stat_statements.max = 10000    # default = 5000
sys_stat_statements.track = 'top'  # default = none
sys_kwr.collect_ksh = on           # default = off
sys_kwr.ringbuf_size = 200000      # default = 100000
sys_kwr.history_days = 3           # default = 8
sys_kwr.language = 'chinese'       # default = 'chinese'
```

其中，sys_kwr 相关参数已在插件 sys_kwr 的参数配置介绍，统计相关参数见配置管理手册的运行时统计数据章节，sys_stat_statements 相关参数见sys_stat_statements 插件手册：

- track_activities：跟踪活跃会话的等待事件、执行的 SQL 语句、当前状态等，默认为 on。

注意:

KSH 功能需要同时配置 track_activities 和sys_kwr.collect_ksh，开启该功能后会有一定的性能损耗。

在 sys_kwr 1.5 版本之前（包括 1.5 版本），sys_ksh 作为独立插件存在，其相关参数如下所示：

```
sys_ksh.enable：控制是否开启 KSH 数据采集，默认为 off
sys_ksh.ringbuf_size：KSH 内存环形缓冲区的大小，默认为 100000 条
sys_ksh.history_days：KSH 历史数据在磁盘上的保留时间，默认为 8 天
sys_ksh.language：KSH 报告使用的语言，默认为中文（chinses 或 chn）
```

3.90.4.3.3　KSH 采集数据说明

KSH 采集的数据主要包括以下信息：

字段	类型	描述
ts	timestamptz	采样时间
pid	integer	进程 ID
client_addr	text	客户端 IP 地址
app_name	text	应用名称
db_id	oid	数据库 OID
db_name	text	数据库名称
user_id	oid	用户 OID
user_name	text	用户名
phase	text	执行阶段，如：PARSE、PLAN、EXECUTION、UNKNOWN
event_type	text	等待事件类型
event	text	等待事件
query_id	bigint	QueryId
cmd_type	text	指令类型，如：INSERT、SELECT、UPDATE、DELETE、UTILITY 等
backend_type	text	后端类型，如：autovacuum、autovacuum worker 等
state	text	运行状态，如：running、idle 等
is_simple	boolean	简单查询语句标志
table_id	oid	IO 操作变量的表 OID
func	text	运行 PLSQL 的函数名
learder_pid	integer	父进程 PID（针对并行）
nmspc_id	oid	表空间 OID

3.90.4.3.4　KSH 内部视图和关系表

- `perf.ksh_history_data`：KSH 持久化存储已采集数据的统计表
- `perf.ksh_history`：基于 perf.ksh_history_data，将 perf.ksh_history_data 表与快照表联动的视图

- perf.session_history：查询内存环形缓冲区中 KSH 数据的视图
- perf.ksh_statements：存储 SQL 语句字符串的统计表

3.90.4.3.5　KSH 报告

存在以下 4 个函数用于生成 KSH 报告：

1. 指定时间区间生成 KSH 报告

```
SELECT * FROM perf.ksh_report({start_ts}, {duration}, {slot_width}, {database});
```

2. 指定时间区间生成 KSH 报告并保存至指定路径

```
SELECT * FROM perf.ksh_report_to_file({start_ts}, {duration}, {slot_width}, {file_path}, {format},
{database});
```

3. 指定快照号生成两个快照期间的 KSH 报告

```
SELECT * FROM perf.ksh_report_by_snapshots({start_snapid}, {end_snapid}, {slot_width}, {format}, {database}
);
```

4. 指定快照号生成 KSH 报告并保存至指定路径

```
SELECT * FROM perf.ksh_report_to_file_by_snapshots({start_snapid}, {end_snapid}, {file_path}, {format},
{slot_width}, {database});
```

上述参数说明：

- start_ts：报告生成的起始时间，单位为分钟。如果不指定该参数，则默认为系统当前时间减去 15 分钟。
- duration：报告时长，默认为 15 分钟，最大不超过 60 分钟。
- slot_width：报告的最小时间区间，该参数为 0 时系统自动计算最优区间，默认值为 0。
- start_snapid：起始快照号。
- end_snapid：结束快照号。
- file_path：报告生成路径，例如：‘/home/kingbase/ksh_report.html’。
- format：报告生成格式，可选择’HTML’和’TEXT’两种格式，默认为 TEXT 格式。
- database：指定生成报告的数据库，例如：‘kingbase’。若不指定该参数，则生成全实例的报告。

示例：

```
-- 使用默认值生成报告
SELECT * FROM perf.ksh_report();
SELECT * FROM perf.ksh_report_to_file(file_path => '/data/ksh.txt');
SELECT * FROM perf.ksh_report_by_snapshots(3, 4);
```

```
SELECT * FROM perf.ksh_report_to_file_by_snapshots(3, 4, file_path => '/data/ksh.txt');

-- 使用实际值生成报告
SELECT * FROM perf.ksh_report('2024-02-13 14:12:46',15,0,'text','kingbase');
SELECT * FROM perf.ksh_report_to_file('2024-02-13 14:12:46',15,0,'/home/kingbase/ksh_report.html','html',
'kingbase');
SELECT * FROM perf.ksh_report_by_snapshots(1,2,0,'text','kingbase');
SELECT * FROM perf.ksh_report_to_file_by_snapshots(1,2,'/home/kingbase/ksh_report.html','html', 0,'kingbase
');
```

备注:

- ksh_report_to_file、ksh_report_by_snapshots、ksh_report_to_file_by_snapshots 函数仅 sys_kwr 1.6 及以上版本支持。
- 按库展示报告功能（database 参数）仅 sys_kwr 1.8 及以上版本支持。

3.90.4.4　KDDM 功能

3.90.4.4.1　KDDM 功能概述

KDDM 是 KingbaseES 性能自动诊断和建议的报告，该模块是 sys_kwr 插件的一部分。

目前 KDDM 报告仅支持 TEXT 格式输出，不支持 HTML 格式输出。

3.90.4.4.2　KDDM 推荐参数配置

信息:

KDDM 报告依赖于 KWR 快照，KWR 快照涉及的 GUC 参数，具体请参考KWR 相关 GUC 配置参数的说明。

3.90.4.4.3　KDDM 报告

存在以下 3 个函数用于生成 KDDM 报告：

1. 指定快照号和数据库生成 KDDM 报告：

```
SELECT * FROM perf.kddm_report({snap_1}, {snap_2}, {database});
```

信息:

KDDM 报告会在数据库服务器 data 目录下的 sys_log 子目录下自动保存一份。

2. 指定快照号和数据库生成 KDDM 报告并保存至指定目录：

```
SELECT * FROM perf.kddm_report_to_file({snap_1}, {snap_2}, {file_path}, {database});
```

3. 指定快照号和 queryid 生成 SQL 报告：

```
SELECT * FROM perf.kddm_sql_report({snap_1}, {snap_2}, {query_id});
```

根据快照范围和指定的 queryid，给出 SQL 详细报告，可用于分析耗时较多的 SQL 语句的 CPU 和 I/O 资源使用情况。

通过 KWR 自动快照或者手动快照功能创建一系列 KWR 快照后，可以通过上述函数创建 KDDM 报告。

参数说明：

- snap_1：生成报告的起始快照 ID。
- snap_2：生成报告的结束快照 ID。
- file_path：报告生成路径，例如：‘/home/kingbase/kddm_1_2.txt’。
- database：指定生成报告的数据库，例如：‘kingbase’。若不指定该参数，则生成全实例的报告。该参数仅 sys_kwr 1.8 及以上版本支持。
- query_id：SQL 语句的唯一标识符。

备注:

在 KDDM 报告中，如果 SQL 语句显示为“Unknown”，则表示该语句的等待事件统计值被高频采集到了，但其 SQL 语句字符串等数据未被采集到。这是因为 sys_stat_statements 的容量过小和性能影响导致无法捕获这些低频统计数据。若此类情况持续出现，可通过适当增大 `sys_stat_statements.max` 的值来解决。但需注意，这可能会影响系统性能，建议在调整前进行充分的权衡和测试。

示例：

```
CREATE EXTENSION sys_kwr;
SELECT * FROM perf.create_snapshot();
INSERT INTO t1 SELECT generate_series(1, 1000000);
SELECT * FROM perf.create_snapshot();
SELECT * FROM perf.kddm_report(1,2);
```

3.90.4.4.4　GUC 参数建议

KDDM 提供 GUC 参数建议功能，根据数据库服务器的硬件情况和用户指定的业务类型，返回建议结果。具体而言，KDDM 通过指定最大连接数、CPU 核心数、内存大小、业务类型给出配置参数建议，函数声明如下：

```
TEXT perf.kddm_guc_advisor(
   IN conn BIGINT = 0,
   IN service_type TEXT = 'oltp',
   IN cpu BIGINT = 0,
   IN memory BIGINT = 0);
```

参数说明：

- conn：最大连接数，默认自动获取，取值范围：1~1000。
- cpu：CPU 核心数，默认自动获取，取值范围：1~1000。
- memory：内存大小，单位为 MB，默认自动获取，取值范围：128MB~1T。
- service_type：业务类型，默认为’oltp’，可选类型：
  - oltp：事务型数据库服务器。
  - olap：分析性数据库服务器。
  - web：Web 服务器。
  - desktop：桌面应用。
  - mixed：混合应用。

示例：

```
SELECT * FROM perf.kddm_guc_advisor();
```

3.90.5　插件 sys_kwr 卸载方法

通过 DROP EXTENSION 命令卸载插件。

示例：

```
DROP EXTENSION sys_kwr;
```

3.90.6　插件 sys_kwr 升级方法

通过 ALTER EXTENSION 命令升级插件。

示例，由 sys_kwr 1.7 版本升级至 sys_kwr 1.8 版本:

```
ALTER EXTENSION sys_kwr UPDATE TO '1.8';
```

备注:

- sys_kwr 插件不支持跨版本升级，例如：无法从 1.6 版本直接升级至 1.8 版本。
- sys_kwr 插件当前的 1.9 版本不支持升级，使用 1.9 版本插件需卸载插件后重新安装。

3.91 sys_prewarm

3.91.1 插件 sys_prewarm 简介

插件 sys_prewarm 是 KingbaseES 的一个扩展插件。提供了一种方便的方法把关系数据载入到操作系统缓冲区或者 KingbaseES 缓冲区。可以使用 sys_prewarm 函数手工执行预热，或者通过对配置文件的修改实现在系统重启时的自动预热。当在 shared_preload_libraries 中包括 sys_prewarm 时系统将自动执行预热，系统将运行一个后台工作者 autoprewarm master，它会周期性地把共享内存中的内容记录在一个名为 autoprewarm.blocks 的文件中，并且在重新启动后用后台工作者重新载入块。

- 插件名为 sys_prewarm
- 插件版本 V1.2

3.91.2 插件 sys_prewarm 加载方式

通过 create extension 来加载插件。

示例：

```
create extension sys_prewarm;
```

如果使用自动预热，则还需要设置在 kingbase.conf 文件中修改 shared_preload_libraries：

```
shared_preload_libraries = 'sys_prewarm'
```

3.91.3 插件 sys_prewarm 的参数配置

手工预热无需配置任何参数。

自动预热包括 2 个参数：

sys_prewarm.autoprewarm (boolean)

控制服务器是否应该运行 autoprewarm 工作者。默认这个参数为 on。

sys_prewarm.autoprewarm_interval (int)

这是更新 autoprewarm.blocks 的间隔。默认是 300 秒。如果被设置为 0，该文件将不会以常规的间隔方式转储，而是只在服务器关闭时转储。

请在 kingbase.conf 文件中配置参数

3.91.4　插件 sys_prewarm 的使用方法

KingbaseES 加载 sys_prewarm 插件后，通过函数来对关系表进行手工预热或自动预热。

手工预热使用 sys_prewarm() 函数：

```
sys_prewarm(regclass, mode text default 'buffer', fork text default 'main',
            first_block int8 default null, last_block int8 default null)
RETURNS int8
```

该函数第一个参数是要预热的关系表。第二个参数是要使用的预热方法（prefetch、read 或者 buffer）。第三个参数是要被预热的关系分叉，通常是 main。第四个参数是要预热的第一个块号（NULL 等同于零）。第五个参数是要预热的最后一个块号（NULL 表示一直预热到关系的最后一个块）。返回值是被预热的块数。

手工预热示例：

```
select sys_prewarm('t1');
```

自动预热使用以下 2 个函数：

```
autoprewarm_start_worker() RETURNS void
```

该函数启动主要的 autoprewarm 工作者。通常情况下系统会自动启动工作者，但是如果没有在服务器启动时配置自动预热并且用户希望在稍晚的时候启动该工作者，这个函数就能发挥作用。

```
autoprewarm_dump_now() RETURNS int8
```

该函数立即更新 autoprewarm.blocks。如果 autoprewarm 工作者没有运行但用户希望它在下一次重启后运行，则这个函数会很有用。返回值是写入到 autoprewarm.blocks 中的记录数。

3.91.5　插件 sys_prewarm 卸载方法

通过 drop extension 来卸载插件。

示例：

```
drop extension sys_prewarm;
```

如果使用自动预热，设置了 shared_preload_libraries，则需要删除‘sys_prewarm’。

```
shared_preload_libraries = ''
```

3.91.6　插件 sys_prewarm 升级方法

通过 alter extension 来升级插件。

示例，由 1.1 升级到 1.2：

```
ALTER EXTENSION sys_prewarm UPDATE TO '1.2';
```

3.92　sys_qualstats

3.92.1　插件 sys_qualstats 简介

sys_qualstats 是一个 KingbaseES 的扩展，用于保存 WHERE 语句和 JOIN 子句中谓词的统计信息。如果希望能够分析数据库中的最常执行的 quals（谓词），这非常的有用，该插件利用这点来提供索引建议。

- 插件名为 sys_qualstats
- 插件版本 V1.0.9

3.92.2　插件 sys_qualstats 加载方式

在使用 sys_qualstats 之前，需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'sys_qualstats,sys_stat_statements'
# (change requires restart)
```

sys_qualstats 插件依赖 sys_stat_statements 插件。

进入 KingbaseES 中执行语句创建插件：

```
create extension sys_qualstats;
create extension sys_stat_statements;
```

系统自动创建 sys_stat_statements 插件。

3.92.3　插件 sys_qualstats 的参数配置

可以在 kingbase.conf 中配置以下信息：

- sys_qualstats.enabled：默认为 off 是否应该启用 sys_qualstats。
- sys_qualstats.max：默认为 1000，要保存的条目数。

- sys_qualstats.resolve_oids：默认为 off。确定在谓词收集期间，对象的实际名称是否应该和 OID 一起存储，开销是不可忽视的，因为每个条目将占用 616 个字节而不是 168 个字节。
- sys_qualstats.track_constants：默认为 on，是否单独跟踪每个常量值。如果为 true，则每个谓词的每个新值都会产生一个新条目。
- sys_qualstats.sample_rate：默认为 0.01，应该被采样的查询比例。1 表示对每个查询进行采样，0 表示基本禁用该功能，-1 表示自动调整。
- sys_stat_statements.track=‘top’：sys_stat_statements 插件配置，设置数据采集级别为’top’。

3.92.4　插件 sys_qualstats 的使用方法

3.92.4.1　sys_qualstats

返回每个限定符的计数

userid：执行查询的用户 oid。

dbid：已经执行查询的数据库的 oid。

lrelid, lattnum：如果存在左侧 var，表示左侧 var 的关系和属性编号的 oid。

rrelid, rattnum：如果存在右侧 var，表示右侧 var 的关系和属性编号的 oid。

opno：表达式中使用运算符的 oid。

qualid：父 AND 表达式的规范化标识符，该标识符的计算不包括常量。

qualnodeid：简单谓词的规范标识符，该标识符的计算包括常量。

uniquequalid：简单谓词的唯一标识符，该标识符计算包括常量。

parenthash：父 AND 表达式的 hash 值。

nbfiltered：此谓词丢弃的元组数。

nodehash：谓词 hash。

count：此谓词出现的总次数。

queryid：如果安装了 sys_stat_statements，表示此查询的 queryid，用来追溯查询的 SQL。

constvalue：右侧常量字符串表示形式，截断为 80 个字符。

execution_count：表示这个语句执行了多少次。

eval_type：评估类型，f 表示扫描后评估的谓词，i 表示索引谓词。

```
\x
select * from sys_qualstats;
-[ RECORD 1 ]-----+---------------------
userid            | 10
```

```
dbid              | 16249
lrelid            | 17227
lattnum           | 1
opno              | 96
rrelid            |
rattnum           |
qualid            | 570210875
uniquequalid      | 420950748
qualnodeid        | 2712005185
uniquequalnodeid  | 2320559360
occurences        | 1
execution_count   | 3
nbfiltered        | 2
constant_position | 28
queryid           | -3480922324520548892
constvalue        | 100::integer
eval_type         | f
```

使用\x 将数据纵向排列，方便阅读。

3.92.4.2　sys_qualstats_reset()

重置内部计数器并清除每个遇到的 qual（谓词）

```
select * from sys_qualstats_reset();
sys_qualstats_reset
--------------------

(1 row)
```

3.92.4.3　sys_qualstats_example_query

返回给定 queryid 的 SQL 语句

```
select * from sys_qualstats_example_query(-3480922324520548892);
        sys_qualstats_example_query
----------------------------------------------------
 select * from t1 where id = 100 and name = 'abc';
(1 row)
```

3.92.4.4 sys_qualstats_names

返回所有存储的 SQL 语句

```
select * from sys_qualstats_names();
-[ RECORD 1 ]-----+---------------------
userid            | 10
dbid              | 16249
lrelid            | 17227
lattnum           | 1
opno              | 96
rrelid            |
rattnum           |
qualid            | 570210875
uniquequalid      | 420950748
qualnodeid        | 2712005185
uniquequalnodeid  | 2320559360
occurences        | 1
execution_count   | 3
nbfiltered        | 2
constant_position | 28
queryid           | -3480922324520548892
constvalue        | 100::integer
eval_type         | f
rolname           |
dbname            |
lrelname          |
lattname          |
opname            |
rrelname          |
rattname          |
```

3.92.4.5 sys_qualstats_example_queries

返回所有存储的查询 sql

```
select query from sys_qualstats_example_queries();
                    query
--------------------------------------------------
select * from t1 where id = 100;
select * from t1 where id = 100 and name = 'abc';
select * from t1 where id = 100 and age = 300;
(3 rows)
```

3.92.4.6　sys_qualstats_pretty

显示 sys_qualstats 视图中每个属性的可读形式

```
select * from sys_qualstats_pretty;
left_schema | left_table | left_column |   operator   | right_schema | right_table | right_column |
occurences | execution_count | nbfiltered
------------+------------+-------------+---------------+--------------+-------------+--------------+--------
----+-----------------+------------
public      | t1         | age         | sys_catalog.= |              |             |              |
 1 |               3 |          3
public      | t1         | id          | sys_catalog.= |              |             |              |
 4 |              12 |          9
public      | t1         | name        | sys_catalog.= |              |             |              |
 1 |               3 |          2
```

3.92.4.7　sys_qualstats_all

对每个属性/运算符的计数求和

```
select * from sys_qualstats_all;
dbid  | relid | userid |       queryid        | attnums | opno |   qualid   | occurences | execution_count |
nbfiltered | qualnodeid
-------+-------+--------+----------------------+---------+------+------------+------------+-----------------
+------------+------------
16249 | 17227 |     10 | -3480922324520548892 | {1}     |   96 |  570210875 |          1 |               3 |
         2 |  570210875
16249 | 17227 |     10 | -3480922324520548892 | {3}     |   98 |  570210875 |          1 |               3 |
         2 |  570210875
16249 | 17227 |     10 |  6307461809864648304 | {1,2}   |   96 | 2519621490 |          2 |               6 |
         6 | 2519621490
16249 | 17227 |     10 |  6828434147671954623 | {1}     |   96 |            |          2 |               6 |
         4 | 2712005185
```

3.92.4.8　sys_qualstats_indexes

查找不存在索引的属性，并给出需要创建索引建议

```
select * from sys_qualstats_indexes;
relid | attnames  |      possible_types      | execution_count
-------+-----------+--------------------------+-----------------
t1    | {age,id}  | {brin,btree,hash}        |               6
t1    | {id}      | {brin,btree,hash}        |               6
t1    | {id,name} | {brin,btree,hash,spgist} |               3
```

3.92.4.9　sys_qualstats_by_query

仅返回形式为 VAR OPERATOR CONSTANT 的谓词

```
select * from sys_qualstats_by_query;
-[ RECORD 1 ]----+-----------------------------------------------------------------------
uniquequalnodeid | 41333584
dbid             | 16249
userid           | 10
qualnodeid       | 1621559404
occurences       | 1
execution_count  | 9
nbfiltered       | 0
queryid          | -6920971673199143593
constvalues      | {2::integer,2::integer}
quals            | {"(16457,1,523,i)","(16457,1,525,i)"}
```

3.92.4.10　使用索引建议

3.92.4.10.1　参数配置

kingbase.conf 中设置

```
shared_preload_libraries = 'sys_qualstats,sys_stat_statements'
# (change requires restart)

sys_stat_statements.track='top'
sys_qualstats.sample_rate=1
```

3.92.4.10.2　创建插件

```
create extension  sys_qualstats;
create extension  sys_hypo;
```

插件 sys_stat_statements 由系统自动创建。

3.92.4.10.3　示例

3.92.4.10.3.1　准备测试数据

```
create table t1(id int, name text);
INSERT INTO t1 SELECT (random() * 1000000)::int, md5(g::text) FROM generate_series(1, 1000000) g;
Select * from t1 where id = 100;

id  |               name
----+----------------------------------
100 | 71735fbd796ae9e347ad82f208f1232b
100 | 4fea58f776ef5d6e22156e08c2191b09
100 | 177848a70c96523b9c7a18dae7f9631d
(3 rows)
```

3.92.4.10.3.2　查看执行计划

```
explain analyze Select * from t1 where id = 100;
                    QUERY PLAN
-----------------------------------------------------------
Gather  (cost=1000.00..15375.79 rows=5292 width=36) (actual time=18.471..183.579 rows=3 loops=1)
    Workers Planned: 2
    Workers Launched: 0
    ->  Parallel Seq Scan on t1  (cost=0.00..13846.59 rows=2205 width=36) (actual time=18.043..183.089
rows=3
loops=1)
        Filter: (id = 100)
        Rows Removed by Filter: 999997
Planning Time: 0.134 ms
Execution Time: 183.658 ms
(8 rows)
```

3.92.4.10.3.3　查看索引建议

new_cost：创建索引以后的 cost 值。

old_cost：无索引情况的 cost 值。

ddl_index：创建 index 的 sql 语句。

benefit：创建索引后的收益值。

```
\x
select * from index_recommendation_by_qual;
-[ RECORD 1 ]---+----------------------------------------
```

```
nspname         | public
relid           | t1
attnames        | {id}
possible_types  | {brin,btree,hash}
execution_count | 1000000
queryid         | -8700169081888605904
query           | Select * from t1 where id = 100;
ddl_index       | CREATE INDEX ON public.t1 USING btree (id)
old_cost        | 15375.79
new_cost        | 7864.65
benefit         | 48.85 %
maintenece_cost | 95.000000
```

使用\x 将数据纵向排列，方便阅读。

3.92.4.10.4　根据索引建议创建索引

```
CREATE INDEX ON public.t1 USING btree (id);
```

3.92.4.10.5　查看创建完索引后的执行计划

```
explain analyze Select * from t1 where id = 100;
            QUERY PLAN
-------------------------------------------------------
Index Scan using t1_id_idx on t1  (cost=0.42..12.46 rows=2 width=37) (actual time=0.053..0.060 rows=2
loops=1)
    Index Cond: (id = 100)
Planning Time: 0.171 ms
Execution Time: 0.165 ms
(4 rows)
```

3.92.5　插件 sys_qualstats 卸载方法

进入 KingbaseES 执行语句删除插件

```
drop extension sys_qualstats ;
drop extension sys_hypo ;
```

3.92.6　插件 sys_qualstats 升级方法

sys_qualstats 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.93 sys_recovery

3.93.1 插件 sys_recovery 简介

插件 sys_recovery 是数据找回工具，通过读取死元组来恢复不可见的表数据，可以找回 update / delete / rollback / drop column 操作后的数据，并以表的形式返回，可以方便的各种查询，帮助寻找需要的数据。

- 插件名为 sys_recovery
- 插件版本为 V1.0

3.93.2 插件 sys_recovery 加载方式

```
CREATE EXTENSION sys_recovery;
```

3.93.3 插件 sys_recovery 的参数说明配置

函数接口 sys_recovery() 按输出格式返回结果集（数组或表格）。

```
select * from sys_recovery(regclass, recoveryrow bool DEFAULT true)
     as (col1 type1, col2 type2, ··· , colN typeN);
```

传入参数

- regclass（关系名）
- recoveryrow（类型为 bool，t 表示仅被恢复的元组，f 表示所有元组；不传入此项参数时，默认值为 true）

输出按照 as 后面指定的列进行输出。输出列包含：

1. 表中的列（type 是对应列的类型）
2. 特殊列

   sys_recovery 也支持指定输出表对象的系统隐含列，供恢复数据的筛选查询，如：

   - xmax、xmin（类型为 xid）
   - ctid（类型为 tid）
   - cmin、cmax（类型为 cid）
   - tableoid（类型为 oid）

   如果列名或类型名错误、不匹配都会报错。

3.93.4　插件 sys_recovery 的使用方法

加载 sys_recovery 插件后，可使用如下函数对指定表查询待恢复的数据。

备注:

恢复数据前，请先设置 vacuum_defer_cleanup_age，防止死元组被清除。

---

1. 仅查询待恢复的数据

例如对表 test 进行查询。假设 test 有两列数据，分别是 int 列 a 和 char 列 b。

```
select * from sys_recovery('test') as (a int, b char);
```

2. 查询待恢复数据时附带系统列

例如对表 test 进行查询。假设 test 有两列数据，分别是 int 列 a 和 char 列 b。需要查询的系统列包括 xmin、xmax 和 ctid。

```
select * from sys_recovery('test') as (ctid tid, xmin xid, xmax xid, a int, b char);
```

3. 查询包括可见数据和待恢复数据在内的所有数据

例如对表 test 进行查询。假设 test 有两列数据，分别是 int 列 a 和 char 列 b。

```
select * from sys_recovery('test', false) as (a int, b char);
```

4. 查询已删除列的数据

例如对表 test 进行查询。假设 test 有两列数据，分别是 int 列 a 和 char 列 b，列 b 已被删除。

此时需要先行获取已删除列的 attnum 值。

```
select attnum from sys_attribute, sys_class where attrelid = sys_class.oid and sys_class.relname='test' and
attname~'dropped';
```

再通过返回的 attnum 值进行查询。假设此处 attnum 返回值为 2。

```
select * from sys_recovery('test') as (a int, dropped_attnum_2 char);
```

3.93.5　插件 sys_recovery 卸载方法

通过 drop extension 来卸载插件。

```
drop extension sys_recovery;
```

3.93.6 插件 sys_recovery 升级方法

通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1

```
ALTER EXTENSION sys_recovery UPDATE TO '1.1';
```

3.93.7 其它说明

sys_recovery 通过读取死元组来恢复不可见的表数据。如果死元组已经被清除掉，那么 sys_recovery 便不能恢复出数据。

- vacuum_defer_cleanup_age

  设置保留恢复最近多少个事务的数据。建议在数据恢复场景首先设置一个较大的 vacuum_defer_cleanup_age 参数来避免待恢复的数据被清除。vacuum_defer_cleanup_age 范围内的死元组无法被清理，可能会导致表膨胀，占用更多的存储空间。

3.94 sys_rman

3.94.1 插件 sys_rman 简介

插件 sys_rman 是 KingbaseES 的一个扩展插件。主要功能是提供查询备份集和归档的接口函数。

- 插件名为 sys_rman
- 插件版本 V1.0

3.94.2 插件 sys_rman 加载方式

执行 `create extension sys_rman;` 语句。

3.94.3 插件 sys_rman 的参数配置

无需配置任何参数

3.94.4 插件 sys_rman 的使用方法

输出备份集和归档基本信息：`select sys_rman.basic_info();`

输出已归档文件的容量、起止时间和归档子目录等汇总信息：`select sys_rman.archive_info();`

输出备份集和归档容量信息、占用磁盘空间百分比和磁盘剩余空间信息：`select sys_rman.disk_info();`

3.94.5　插件 sys_rman 卸载方法

执行 `drop extension sys_rman;` 语句。

3.94.6　插件 sys_rman 升级方法

sys_rman 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

3.95　sys_spacequota

3.95.1　插件 sys_spacequota 简介

sys_spacequota 可以对指定表空间的大小进行限额，一旦大于该限额，对此表空间里的数据进行 insert, update, copy to, select into, create index 操作将会被警告或终止（默认为警告，警告或者终止可以通过 GUC 参数 `spacequota.fullwarningtype = 0/1` 来配置）。

- 插件名为 sys_spacequota
- 插件版本 V1.0

3.95.2　插件 sys_spacequota 加载方式

使用时需要创建扩展。

示例：

```
create extension sys_spacequota;
```

3.95.3　插件 sys_spacequota 的参数配置

`spacequota.warning_email_enable`

表空间限额邮件告警的开关，参数为 bool 类型，默认为 false。

3.95.4 插件 sys_spacequota 的使用方法

3.95.4.1 开关及查询:

开关：创建扩展后，再开启参数开关，限额功能生效，开关不能执行在创建扩展之前。

```
alter system set spacequota.enable = on ;
call sys_reload_conf();
```

查询信息：

```
select * from sys_space_quota;
```

3.95.4.2 remove_space_quota

删除表空间限额, 可调用 remove_space_quota(oid) 方法。

语法

```
remove_space_quota(oid)
```

参数

oid 表空间的 oid 值。

返回值

无返回值。

示例：

```
call remove_space_quota(16384);
```

3.95.4.3 space_quota_check

核对表空间的限额，一旦大于该限额，对此表空间里的数据进行 insert, update, copy to , select into, create index 操作将会被警告或终止（默认为警告，警告或者终止可以通过 GUC 参数 `spacequota.fullwarningtype = 0/1` 来配置）。

语法

```
space_quota_check()
```

参数

无参数。

返回值

无返回值。

3.95.4.4 set_space_quota(oid, quota)

设置或更改表空间限额, 可调用 set_space_quota(oid, quota) 方法，quota 的单位为字节。

语法

```
set_space_quota(oid, quota)。
```

参数

oid：表空间的 oid 值。

quota：设置的表空间限额值。

返回值

无返回值。

示例：

```
CALL set_space_quota(18273,1118880);
CALL

SELECT spcname,oid FROM sys_tablespace;
spcname    |  oid
-------------+-------
 sys_default |  1663
 sys_global  |  1664
 sysaudit    |  1986
 space1      | 18273
(4 rows)
```

备注:

系统表无法设置空间限额。

3.95.5 插件 sys_spacequota 卸载方法

不需要使用时删除插件即可。

示例：

```
drop extension sys_spacequota;
```

3.95.6　插件 sys_spacequota 升级方法

sys_spacequota 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.96　sys_sqltune

3.96.1　插件 sys_sqltune 简介

插件 sys_sqltune 是 KingbaseES 的一个扩展插件。主要功能是通过收集统计信息对 SQL 进行语句级调优。当前版本具备 SQL 监控功能及 SQL 调优建议功能。

SQL 监控提供有关实时 SQL 监控及数据库操作的监视信息。通过动态性能视图或者监控报告，提供实时的 SQL 语句执行过程中的状态和统计信息。

SQL 调优建议包括：SQL 改写建议、SQL 访问路径建议、统计信息收集建议。使用 SQL 调优建议，优化器根据数据库内部状态，给出调优建议，DBA 应用这些调优建议后，能够使被调优的 SQL 语句性能获得提升。

- 插件名为 sys_sqltune
- 插件版本 V1.1

3.96.2　插件 sys_sqltune 加载方式

将他添加到 kingbase.conf 文件的 `shared_preload_libraries` 中，重启数据库时自动加载。

示例如下：

```
shared_preload_libraries = 'sys_sqltune'
```

并在数据库中创建插件：

```
CREATE EXTENSION sys_sqltune;
```

3.96.3　插件 sys_sqltune 的参数配置

使用 SQL 监控的全部功能，建议在 kingbase.conf 里添加以下参数：

```
shared_preload_libraries = 'plsql, sys_stat_statements, sys_sqltune'
sql_monitor.track = 'all'
sys_stat_statements.track = 'all'
enable_hint = on
```

上述参数的意义如下：

```
plsql：支持 plsql，此处主要提供 PACKAGE 特性相关功能。
sys_stat_statements：跟踪统计 SQL 语句插件，具体请参考 sys_stat_statements 插件，此处主要提供 query_id 数据项。

sql_monitor.track：开启对 SQL 语句和数据库操作的监控，推荐用 'all'，默认为'none'。
sys_stat_statements.track：跟踪统计 SQL 语句的访问，推荐用 'all'，默认为'none'。
enable_hint = on：开启数据库内核 hint 功能，默认为 off。
```

全部相关参数如下：

- sql_monitor.track：会话级参数

  SQL 语句监控层级，ENUM 类型，默认值为 none。

  - TOP：监控顶层 SQL 语句
  - all：监控嵌套层数小于 64 层的 SQL 语句
  - none：不监控

- sql_monitor.max：系统级参数

  视图最大数据量，整数，默认 1000，最小 100。当系统保存的数据超出最大数量后，扔掉一些陈旧的统计结果，腾出部分新的空间给新监控的 SQL。

- sql_monitor.track_options ：会话级参数

  采集统计信息控制参数，ENUM 类型，默认为’basic’。

  - basic：采集语句全部统计信息和执行计划节点基本统计信息。
  - full：在’basic’基础上，采集执行计划节点的 buffer、wal 相关统计信息，有一定性能影响。

- sql_monitor.language：会话级参数

  网页版监控报告使用语言，TEXT 类型，默认为中文（chinese 或 chn），可选为英文 (english 或 eng) 。

  - chinese/chn：网页版报告输出中文。
  - english/eng：网页版报告输出英文。

3.96.4　插件 sys_sqltune 的使用方法

3.96.4.1　概述

sys_sqltune 主要功能是通过收集统计信息对 SQL 进行语句级调优。当前版本具备 SQL 监控功能。

3.96.4.2　SQL 监控概述

对于执行时间较长的 SQL 语句，提供 SQL 监控功能，可以监控其执行状态，以及执行过程中各计划节点的统计数据，并提供 SQL 监控报告功能。

3.96.4.3　SQL 监控功能

语句符合条件开始监控后，系统自动将其数据插入共享内存中的动态视图 v*sql*$_m$*onitorv*sql_plan_monitor 中，间隔约 1 秒刷新一次数据。

- **自动监控** SQL 执行时间超过 5 秒的 SQL 语句被列入监控对象。

- **手动监控** SQL 可通过对语句添加 HINT 来控制语句是否会被监控，需通过 enable_hint GUC 参数开启 HINT 功能。

- **并行语句监控**当语句计划并行，组件在执行开始时将其纳入监控范围，其申请的每个并行 worker 都会在 v$sql_monitor 中插入一条数据。

- **多层嵌套语句监控**当语句存在嵌套关系，系统会监控其最顶层的 64 层语句，将其中执行时间超过 5 秒的语句列入监控对象，存在嵌套关系的语句包括：

1. PL/PGSQL-存在多层嵌套调用
2. 带有 SELECT 子句的 DML、DDL 语句-下层为 SELECT 语句
3. EXPLAIN ANALYZE 语句-上层为 EXPLAIN 语句无执行计划，下层为实际执行，有执行计划。

- **监控视图**组件将数据存储在动态视图 v*sql*$_m$*onitorv*sql_plan_monitor 中，视图最大值受 sql_monitor.max 限制。

- **监控报告**组件可生成详细监控报告和列表监控报告，包括 TEXT 和 HTML 两种类型，其中网页版报告支持中英文，TEXT 版报告仅支持英文。

3.96.4.4　SQL 监控函数接口

在 Oracle 模式下，SQL 监控通过 PACKAGE DBMS_SQL_MONITOR 提供函数接口

DBMS_SQL_MONITOR 包信息参考《PLSQL 系统包和类型参考手册》中的 DBMS_SQL_MONITOR 包章节。

在 PG 与 MySQL 模式下，SQL 监控通过 SCHEMA perf 提供函数接口，功能、参数、返回值与 DBMS_SQL_MONITOR 相同

- PERF.SQL_MONITOR_RESET()

  用于清空 v$sql_monitor$ vsql_plan_monitor 视图中的所有数据。

- PERF.REPORT_SQL_MONITOR()

  该函数用于生成监控详细报告，可用参数指定报告类型和生成报告的数据。

- PERF.REPORT_SQL_MONITOR_LIST()

  该函数用于生成监控列表报告，可用参数指定报告类型和生成报告的数据。

- PERF.REPORT_SQL_MONITOR_TO_FILE()

  函数用于生成监控详细报告，报告生成参照 REPORT_SQL_MONITOR 函数，并将报告插入到磁盘指定位置。

- PERF.REPORT_SQL_MONITOR_LIST_TO_FILE()

  函数用于生成监控列表报告，报告生成参照 REPORT_SQL_MONITOR_LIST 函数，并将报告插入到磁盘指定位置。

- PERF.QUICK_TUNE_BY_SQL (sql_text text)

  函数用于输入一条 SQL 语句，生成调优建议报告。sql_text 表示 SQL 文本。

- PERF.QUICK_TUNE_BY_QUERYID (queryid bigint)

  函数用于输入 SQL 语句的查询 ID，生成调优建议报告。queryid 表示 SQL 语句的查询 ID。

- PERF.QUICK_TUNE_BY_SQL_TO_FILE (sql_text text, format text, file_path text)

  函数用于输入一条 SQL 语句，生成调优建议报告到指定文件。sql_text 表示 SQL 文本；format 表示报告格式，仅支持”text”; file_path 表示报告路径和名称。

- PERF.QUICK_TUNE_BY_QUERYID_TO_FILE (queryid bigint, format text, file_path text)

  函数用于输入 SQL 语句的查询 ID，生成调优建议报告到指定文件。queryid 表示 SQL 语句的查询 ID；format 表示报告格式，仅支持”text”; file_path 表示报告路径和名称。

3.96.4.5　生成监控报告

通过下面的语句产生几条被监控 SQL 语句注入数据

```
set sql_monitor.track = 'all';
set enable_hint = on;

SELECT * FROM sys_sleep(5);
CREATE TABLE smon_test(id bigint);
```

```
INSERT INTO smon_test SELECT i FROM generate_series(1, 20000) i;
SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;
```

查看视图

```
SELECT * FROM V$SQL_MONITOR;
SELECT * FROM V$SQL_PLAN_MONITOR;
```

生成 TEXT 版监控报告

```
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR(type=>'TEXT');
```

生成网页版监控报告

```
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR(type=>'HTML');
```

生成英文网页版监控报告

```
set sql_monitor.language = 'english';
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR(type=>'HTML');
```

3.96.4.6　SQL 监控视图

sql 监控采集的数据主要包括以下信息：

表 3.96.1: V$SQL_MONITOR

字段	类型	描述
sql_id	bigint	SQL ID，基于 SQL 文本的 SQL 标识
exec_id	bigint	执行 ID，每次执行加一（实例级共享）
session_id	text	会话 ID(会话启动时间.PID)
db_id	oid	数据库 OID
user_id	oid	用户 OID
query_id	bigint	SQL 语句 query id（来自 sys_stat_statements）
start_ts	timestamp	最初启动时间
last_ts	timestamp	最后更新时间
px_server	bigint	并行执行号，主进程为-1

-续表-

字段	类型	描述
px_is_cross_instance	boolean	语句是否计划并行
px_servers_allocated	bigint	计划最大并行数
px_servers_requested	bigint	被监控并行数
duration	double precision	语句总运行时间
status	text	运行状态
statup_time	double precision	节点实际执行总时间，单位毫秒
total_time	double precision	节点实际执行活动时间，单位毫秒
query	text	SQL 语句原文
lock_wait_time	double precision	后端重量级锁等待时间
lightlock_wait_time	double precision	后端轻量级锁等待时间
buffer_wait_time	double precision	服务器进程访问数据缓冲区时间
client_wait_time	double precision	客户端等待时间
extension_wait_time	double precision	扩展插件等待时间
timeout_wait_time	double precision	进程间通讯等待时间
ipc_wait_time	double precision	超时等待时间
user_io_wait_time	double precision	IO 等待时间
other_wait_time	double precision	其他等待时间
tuples_num	bigint	节点输出元组总数
loops_num	bigint	节点调用次数
shared_blks_hit	bigint	共享内存命中
shared_blks_read	bigint	共享内存块读数量
shared_blks_dirtied	bigint	共享磁盘块脏块数量
shared_blks_written	bigint	共享磁盘块写数量
local_blks_hit	bigint	本地内存命中

-续表-

字段	类型	描述
local_blks_read	bigint	本地内存块读数量
local_blks_dirtied	bigint	本地磁盘块脏块数量
local_blks_written	bigint	本地磁盘块写数量
temp_blks_read	bigint	临时块读
temp_blks_written	bigint	临时块写
blk_read_time	double precision	读花费时间，单位毫秒
blk_write_time	double precision	写花费时间，单位毫秒
wal_records	bigint	产生的 WAL 记录数量
wal_fpi	bigint	产生的 WAL full page images 数量
wal_bytes	bigint	产生的 WAL 记录的大小

- exec_id 加上 session_id 可以唯一的标识语句的单次执行
- 语句并行时，主进程和并行 worker 之间共用相同 exec_id、session_id，主进程 px_server 为-1，并行 worker 从 0 开始累加。
- query_id 来自sys_stat_statements 插件，可能为 0。
- query 字段表示 SQL 语句原文，最多显示 200 个字符。

表 3.96.2: V$SQL_PLAN_MONITOR

字段	类型	描述
sql_id	bigint	SQL ID，基于 SQL 文本的 SQL 标识
exec_id	bigint	执行 ID，每次执行加一（实例级共享）
session_id	text	会话 ID(会话启动时间.PID)
db_id	oid	数据库 OID
user_id	oid	用户 OID
query_id	bigint	SQL 语句 query id（来自 sys_stat_statements）

-续表-

字段	类型	描述
start_ts	timestamp	最初启动时间
last_ts	timestamp	最后更新时间
status	text	计划节点状态
plan_index	bigint	计划节点编号
plan_type	text	计划节点类型
plan_depth	bigint	计划节点在计划树中的深度
plan_object_name	text	计划对象名
plan_object_type	text	计划对象类型
plan_rows	bigint	计划预计行数
plan_statup_cost	bigint	计划预计启动成本
plan_total_cost	bigint	计划预计总成本
statup_time	double precision	节点实际执行总时间，单位毫秒
total_time	double precision	节点实际执行活动时间，单位毫秒
tuples_num	bigint	节点输出元组总数
loops_num	bigint	节点调用次数
shared_blks_hit	bigint	共享内存命中
shared_blks_read	bigint	共享内存块读数量
shared_blks_dirtied	bigint	共享磁盘块脏块数量
shared_blks_written	bigint	共享磁盘块写数量
local_blks_hit	bigint	本地内存命中
local_blks_read	bigint	本地内存块读数量
local_blks_dirtied	bigint	本地磁盘块脏块数量
local_blks_written	bigint	本地磁盘块写数量
temp_blks_read	bigint	临时块读

-续表-

字段	类型	描述
temp_blks_written	bigint	临时块写
blk_read_time	double precision	读花费时间，单位毫秒
blk_write_time	double precision	写花费时间，单位毫秒
wal_records	bigint	产生的 WAL 记录数量
wal_fpi	bigint	产生的 WAL full page images 数量
wal_bytes	bigint	产生的 WAL 记录的大小

- exec_id 、plan_index 加上 session_id 可以唯一的标识语句的单次执行中的单个执行节点。
- 语句并行时，主进程和并行 worker 之间共用相同 exec_id、session_id，主进程 px_server 为-1，并行 worker 从 0 开始累加。
- query_id 来自sys_stat_statements 插件，可能为 0。
- 执行节点的 blk、wal 相关统计数据默认不统计，可通过设置 sql_monitor.track_options 为'full' 开启统计

3.96.5　插件 sys_sqltune 卸载方法

进入 KingbaseES 卸载插件

示例

```
DROP EXTENSION sys_sqltune;
```

3.96.6　插件 sys_sqltune 升级方法

通过 alter extension 来升级插件。

示例，由 1.0 升级到 1.1

```
ALTER EXTENSION sys_sqltune UPDATE TO '1.1';
```

3.97 sys_squeeze

3.97.1 插件 sys_squeeze 简介

sys_squeeze 是 KingbaseES 的一个扩展插件，该组件将提供人工调用命令实现对表 dead tuple 的清理工作。该组件在清理表空间的过程中，不会全程加排他锁，能保证运行期间尽可能不影响对目标表的访问。该插件的实现依赖于逻辑解码，因此使用该插件之前必须保证数据库 wal_level 等级设置为’logical’。

该插件提供了清理表空间的函数，创建插件并设置好 wal_level 后，通过调用 select squeeze.squeeze_table(opt1,opt2,opt3,opt4,opt5) 进行清理操作。各参数的说明请见参数配置一节。

- 插件名为 sys_squeeze
- 插件版本 V1.4

其他说明：

- sys_squeeze 不支持对包含复合类型的表进行清理
- sys_squeeze 依赖逻辑复制，要求目标表有主键或唯一约束作为 IDENTITY INDEX，并占用一个复制槽
- sys_squeeze 清理重建表时，尽量避免该表的 ALTER TABLE、ALTER INDEX、VACUUM FULL、CLUSTER 或 TRUNCATE 并发操作
- sys_squeeze 不支持对分区表主表直接清理重建，需要对子表执行清理重建
- sys_squeeze 建议在非业务高峰期执行，建议尽量避免对大表使用
- sys_squeeze 需要额外的临时存储空间，请确保 data 目录或目标表空间容量充足，存储空间容量要求最多为目标表（包括其索引、toast 表）的容量大小
- sys_queeze 不支持仅存在非空约束的表

3.97.2 插件 sys_squeeze 加载方式

首先确认 sys_squeeze 是否已存在于 kingbase.conf 的 shared_preload_libraries 配置项中，正常情况下 sys_squeeze 默认会加载到 shared_preload_libraries，可直接执行下一步语句；如果没有配置，需要将‘sys_squeeze’追加到该配置中，并重启数据库。

执行 `create extension sys_squeeze;` 语句。

```
shared_preload_libraries = 'sys_squeeze'
```

3.97.3 插件 sys_squeeze 的参数配置

函数接口 squeeze.squeeze_table(opt1,opt2,opt3,opt4,opt5) 有 5 个可配置参数：

opt1: 填写所要清理的目标表的模式名，必须项

opt2: 填写所要清理的目标表名，必须项

opt3: 指定该表中已存在的 index 名，若指定，会在新生成的表空间中按照此 index 顺序物理排列 tuple，可选项，不指定填 null

opt4: 指定将新生成的表置入指定的表空间。可选项，若不指定填 null，表示仍然使用原有表空间。

opt5: 将相应 index 置入指定的表空间。可选项，若不指定填 null，表示仍然使用原有表空间。

3.97.4 插件 sys_squeeze 的使用方法

加载 sys_squeeze 插件后，可使用如下函数对指定表做清理。

1. 普通的清理

   例如要对 public 模式下的 test 表做清理

```
select squeeze.squeeze_table('public','test',null,null,null);
```

2. 指定按照索引顺序对表内元组进行物理排序

   例如要对 public 模式下的 test 表，按照已有的索引 idx 对表内元组重排

```
select squeeze.squeeze_table('public','test','idx',null,null);
```

3. 指定将重排后的表置入指定的表空间

   例如要对 public 模式下的 test 表，放入表空间 tbl_spc1

```
select squeeze.squeeze_table('public','test',null,'tbl_spc1',null);
```

4. 指定将重排后的表及索引置入指定的表空间

   例如要对 public 模式下的 test 表以及索引 idx，放入表空间 tbl_spc1

```
select squeeze.squeeze_table('public','test',null,'tbl_spc1',ARRAY[['idx','tbl_spc1']]);
```

Sqlserver 模式下应使用对应的数组表达形式

```
select squeeze.squeeze_table('public','test',null,'tbl_spc1',ARRAY{{'idx','tbl_spc1'}});
```

3.97.5 插件 sys_squeeze 卸载方法

执行 `drop extension sys_squeeze;` 语句。删除 kingbase.conf 文件中 `shared_preload_libraries` 内的 sys_squeeze 参数，重启数据库。

3.97.6 插件 sys_squeeze 升级方法

sys_squeeze 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

3.98 sys_stat_statements

3.98.1 插件 sys_stat_statements 简介

插件 sys_stat_statements 是 KingbaseES 的一个扩展插件，该插件能够提供服务器端已执行 SQL 语句的统计信息，帮助数据库管理员进行数据库性能分析和优化工作，尤其在分析查询性能开销和识别 TOP SQL 方面有重要作用。

- 插件名为 sys_stat_statements。
- 自 V9 版本起，此插件已内置化管理，在初始化数据库实例时会自动完成加载。
- 此功能模块默认处于关闭状态，需通过设置参数 `sys_stat_statements.track = 'top'`进行启用。
- 当前最新可用版本为 1.11。

3.98.2 插件 sys_stat_statements 加载方式

在 kingbase.conf 文件的 shared_preload_libraries 参数中新增 sys_stat_statements 后重启数据库。

```
shared_preload_libraries = '..., sys_stat_statements'
```

3.98.3 插件 sys_stat_statements 的参数配置

该插件的主要配置参数

- sys_stat_statements.max（**整数**）

模块跟踪的最大语句数（即 sys_stat_statements_all 视图的最大行数），默认值为 5000；若模块跟踪的语句数大于最大语句数，则执行次数较少的语句的统计信息将被丢弃。该参数只能在服务器启动时设置。

- sys_stat_statements.track（**枚举**）

  控制模块跟踪语句的级别，该参数有以下三种设置：

  - top：仅跟踪客户端直接发出的语句（顶层语句）。
  - all：跟踪全部语句，即不仅跟踪客户端直接发出的语句，而且跟踪嵌套的子层语句（例如函数内调用的语句）。
  - none：不跟踪任何语句。

注意:

- 默认值为’none’，仅超级用户可以更改此设置。
- 在 V8R5 版本中，该参数默认值为’top’。

- sys_stat_statements.track_parse（boolean）

  是否跟踪解析时间和次数，默认值为 on，仅超级用户可以更改此设置。

- sys_stat_statements.track_plan（boolean）

  是否跟踪计划时间和次数，默认值为 on，仅超级用户可以更改此设置。

- sys_stat_statements.track_utility（boolean）

  控制模块是否跟踪实用程序语句（实用程序语句是除 SELECT，INSERT，UPDATE 和 DELETE 之外的所有语句），默认值为 on，仅超级用户可以更改此设置。

- sys_stat_statements.save（boolean）

  指定是否在服务器关闭时保存语句的统计信息，默认值为 on。若指定为 off，则统计信息不会在服务器关闭时保存，也不会在服务器启动时重新加载。该参数仅支持服务器启动时在 kingbase.conf 文件中设置，或在服务器命令行中通过 alter system 方式设置。

注意:

该模块根据 sys_stat_statements.max 参数的大小成比例的消耗共享内存。请注意，即使 sys_stat_statements.track 设置为’none’，只要服务器启动时加载该模块，便会消耗共享内存。

3.98.4 插件 sys_stat_statements 的使用方法

3.98.4.1 sys_stat_statements 的推荐参数配置

sys_stat_statements 的典型参数配置是：

```
# kingbase.conf
shared_preload_libraries ='sys_stat_statements'
sys_stat_statements.max = 10000
sys_stat_statements.track = 'top'
```

3.98.4.2 sys_stat_statements 的视图

KingbaseES 加载 sys_stat_statements 插件后，包含以下两个视图：

视图名称	说明
sys_stat_statements	顶层 SQL 语句统计列表
sys_stat_statements_all	顶层和子层 SQL 语句统计列表

sys_stat_statements 视图结构如下：

表 3.98.2: sys_stat_statements 视图结构

名称	类型	描述
userid	oid	执行该语句的用户的 OID
dbid	oid	在其中执行该语句的数据库的 OID
queryid	bigint	内部哈希码，从语句的解析树计算得来
query	text	语句的文本形式
parses	bigint	该语句被解析的次数
total_parse_time	double precision	在该语句中解析花费的总时间，以毫秒计
min_parse_time	double precision	在该语句中解析花费的最小时间，以毫秒计
max_parse_time	double precision	在该语句中解析花费的最大时间，以毫秒计
mean_parse_time	double precision	在该语句中解析花费的平均时间，以毫秒计
stddev_parse_time	double precision	在该语句中解析花费时间的总体标准偏差，以毫秒计
plans	bigint	该语句执行 Plan 的次数
total_plan_time	double precision	在该语句中执行 Plan 花费的总时间，以毫秒计
min_plan_time	double precision	在该语句中执行 Plan 花费的最小时间，以毫秒计
max_plan_time	double precision	在该语句中执行 Plan 花费的最大时间，以毫秒计
mean_plan_time	double precision	在该语句中执行 Plan 花费的平均时间，以毫秒计
stddev_plan_time	double precision	在该语句中执行 Plan 花费时间的总体标准偏差，以毫秒计
calls	bigint	被执行的次数

-续表-

名称	类型	描述
total_exec_time	double precision	在该语句中花费的总时间，以毫秒计（该字段在 V8R5 里面为：total_time）
min_exec_time	double precision	在该语句中花费的最小时间，以毫秒计（该字段在 V8R5 里面为：min_time）
max_exec_time	double precision	在该语句中花费的最大时间，以毫秒计（该字段在 V8R5 里面为：max_time）
mean_exec_time	double precision	在该语句中花费的平均时间，以毫秒计（该字段在 V8R5 里面为：mean_time）
stddev_exec_time	double precision	在该语句中花费时间的总体标准偏差，以毫秒计（该字段在 V8R5 里面为：stddev_time）
rows	bigint	该语句检索或影响的行总数
shared_blks_hit	bigint	该语句造成的共享块缓冲命中总数
shared_blks_read	bigint	该语句读取的共享块的总数
shared_blks_dirtied	bigint	该语句弄脏的共享块的总数
shared_blks_written	bigint	该语句写入的共享块的总数
local_blks_hit	bigint	该语句造成的本地块缓冲命中总数
local_blks_read	bigint	该语句读取的本地块的总数
local_blks_dirtied	bigint	该语句弄脏的本地块的总数
local_blks_written	bigint	该语句写入的本地块的总数
temp_blks_read	bigint	该语句读取的临时块的总数
temp_blks_written	bigint	该语句写入的临时块的总数
blk_read_time	double precision	该语句花在读取块上的总时间，以毫秒计
blk_write_time	double precision	该语句花在写入块上的总时间，以毫秒计

sys_stat_statements_all 视图结构如下：

表 3.98.3: sys_stat_statements_all 视图结构

名称	类型	描述
userid	oid	执行该语句的用户的 OID
dbid	oid	在其中执行该语句的数据库的 OID
queryid	bigint	内部哈希码，从语句的解析树计算得来
parent_queryid	bigint	上一层 SQL 的哈希码
query	text	语句的文本形式
parses	bigint	该语句被解析的次数
total_parse_time	double precision	在该语句中解析花费的总时间，以毫秒计
min_parse_time	double precision	在该语句中解析花费的最小时间，以毫秒计
max_parse_time	double precision	在该语句中解析花费的最大时间，以毫秒计
mean_parse_time	double precision	在该语句中解析花费的平均时间，以毫秒计
stddev_parse_time	double precision	在该语句中解析花费时间的总体标准偏差，以毫秒计
plans	bigint	该语句执行 Plan 的次数
total_plan_time	double precision	在该语句中执行 Plan 花费的总时间，以毫秒计
min_plan_time	double precision	在该语句中执行 Plan 花费的最小时间，以毫秒计
max_plan_time	double precision	在该语句中执行 Plan 花费的最大时间，以毫秒计
mean_plan_time	double precision	在该语句中执行 Plan 花费的平均时间，以毫秒计
stddev_plan_time	double precision	在该语句中执行 Plan 花费时间的总体标准偏差，以毫秒计
calls	bigint	被执行的次数
total_exec_time	double precision	在该语句中花费的总时间，以毫秒计（该字段在 V8R5 里面为：total _time）
min_exec_time	double precision	在该语句中花费的最小时间，以毫秒计（该字段在 V8R5 里面为：min _time）
max_exec_time	double precision	在该语句中花费的最大时间，以毫秒计（该字段在 V8R5 里面为：max _time）

-续表-

名称	类型	描述
mean_exec_time	double precision	在该语句中花费的平均时间，以毫秒计（该字段在 V8R5 里面为：mean_time）
stddev_exec_time	double precision	在该语句中花费时间的总体标准偏差，以毫秒计（该字段在 V8R5 里面为：stddev_time）
rows	bigint	该语句检索或影响的行总数
shared_blks_hit	bigint	该语句造成的共享块缓冲命中总数
shared_blks_read	bigint	该语句读取的共享块的总数
shared_blks_dirtied	bigint	该语句弄脏的共享块的总数
shared_blks_written	bigint	该语句写入的共享块的总数
local_blks_hit	bigint	该语句造成的本地块缓冲命中总数
local_blks_read	bigint	该语句读取的本地块的总数
local_blks_dirtied	bigint	该语句弄脏的本地块的总数
local_blks_written	bigint	该语句写入的本地块的总数
temp_blks_read	bigint	该语句读取的临时块的总数
temp_blks_written	bigint	该语句写入的临时块的总数
blk_read_time	double precision	该语句花在读取块上的总时间，以毫秒计
blk_write_time	double precision	该语句花在写入块上的总时间，以毫秒计

出于安全原因，非超级用户不允许查看其他用户执行的 SQL 文本或 queryid。

sys_stat_statements 视图中的查询文本与 queryid 具有一一对应关系。通常情况下，若两个有执行计划的查询（SELECT、INSERT、UPDATE 和 DELETE 语句）在语义上等效（仅常量值不同），则它们会被视为同一个条目整合至 sys_stat_statements 视图；其中，SQL 文本中的常量值将被替换为’?’。对于实用程序语句，在被整合至 sys_stat_statements 视图时，将严格根据 SQL 文本字符串进行比较。

UTILITY 语句中的 CALL 语句与有执行计划的查询（SELECT、INSERT、UPDATE 和 DELETE 语句）具有类似的 queryid 计算机制和常量替换逻辑。需要注意的是，由于 PLSQL/PLPGSQL 存在结果集缓存机制，在执行

sys_stat_statements_reset() 后，部分 CALL proc(xxx) 语句中的常量将无法被正确替换。

3.98.4.3　sys_stat_statements 的函数

1. `sys_stat_statements(showtext boolean)`

   功能：查询 TOP SQL 列表

   参数：

   - showtext boolean：是否显示 SQL 字符串

   返回值：TOP SQL 列表

2. `sys_stat_statements_all(showtext boolean)`

   功能：查询全部 SQL 列表

   参数：

   - showtext boolean：是否显示 SQL 字符串

   返回值：全部 SQL 列表

3. `sys_stat_statements_limit_len(showtext boolean, limit_query_len int)`

   功能：查询 TOP SQL 列表，限制返回的 SQL 字符串的最大长度，该函数为 1.9 版本新增。

   参数：

   - showtext boolean：是否显示 SQL 字符串
   - limit_query_len：返回 SQL 字符串的最大长度，超过该长度的字符将被截断

   返回值：TOP SQL 列表

4. `sys_stat_statements_reset()`

   功能：重置（清空）TOP SQL 列表

3.98.5　插件 sys_stat_statements 卸载方法

sys_stat_statements 插件是系统初始化插件，用户无法卸载。

3.98.6　插件 sys_stat_statements 升级方法

sys_stat_statements 插件通常伴随 KingbaseES 安装包一并升级，用户无须单独升级该插件。

3.99 sys_trgm

3.99.1 插件 sys_trgm 简介

sys_trgm 模块提供用于决定基于 trigram 匹配的字母数字文本相似度的函数和操作符，以及支持快速搜索相似字符串的索引操作符类。

- 插件名为 sys_trgm
- 插件版本 V1.6

3.99.2 插件 sys_trgm 加载方式

```
create extension sys_trgm;
```

3.99.3 插件 sys_trgm 的参数配置

无需配置任何参数

3.99.4 插件 sys_trgm 的使用方法

3.99.4.1 函数

`sys_trgm` 函数显示 sys_trgm 模块所提供的函数。

表 3.99.1: sys_trgm 函数

函数	返回值	描述
similarity(text, text)	real	返回一个数字指示两个参数有多相似。该结果的范围是 0（指示两个字符串完全不相似）到 1（指示两个字符串完全一样）。
show_trgm(text)	text[]	返回一个给定字符串中所有的 trigram 组成的一个数组（实际上除了调试很少有用）。
word_similarity(text, text)	real	返回一个数字表示第一个字符串中的 trigram 集合与第二个字符串中 trigram 的有序集中任何连续部分的最大相似度。详情请见下文的解释。

-续表-

函数	返回值	描述
strict_word_similarity(text, text)	real	与 word_similarity(text, text) 相同，但是强制连续部分的边界与词边界相匹配。由于我们没有跨词的 trigram，这个函数实际上返回第一个字符串和第二个字符串任意连续部分的相似度。
show_limit()	real	返回% 操作符使用的当前相似度阈值。例如，这设定两个词被认为足够相似时，它们之间应满足的最小相似度（已废弃）。
set_limit(real)	real	设定% 操作符使用的当前相似度阈值。该阈值必须介于 0 和 1 之间（默认为 0.3）。返回传递进来的同一个值（已废弃）。

考虑下面的例子：

```
SELECT word_similarity('word', 'two words');
 word_similarity
-----------------
             0.8
(1 row)
```

在第一个字符串中，trigram 集合是 {” w”,” wo”,“wor”,“ord”,“rd “}。在第二个字符串中，trigram 的有序集是 {” t”,” tw”,“two”,“wo “,” w”,” wo”,“wor”,“ord”,“rds”,“ds “}。在第二个字符串中最相似的 trigram 有序集的部分是 {” w”,” wo”,“wor”,“ord”}，并且相似度是 0.8。

这个函数返回的值可以大概地理解为第一个字符串和第二个字符串任意子串的最大相似度。不过，这个函数不会对该部分的边界加入填充。因此，除了失配的词边界之外，第二个字符串中存在的额外字符的数目没有被考虑。

同时，strict_word_similarity(text, text) 在第二个字符串中选择一个由词构成的部分。在上面的例子中，strict_word_similarity(text, text) 会选择单个词’words’形成的部分，其 trigram 集合为 {” w”,” wo”,“wor”,“ord”,“rds”,”ds ”}。

```
SELECT strict_word_similarity('word', 'two words'), similarity('word', 'words');
 strict_word_similarity | similarity
------------------------+------------
              0.5714286 |  0.5714286
(1 row)
```

因此，strict_word_similarity(text, text) 函数对于计算整个词的相似度有用，而 word_similarity(text, text) 更适合于计算词的部分相似度。

备注:

在从一个字符串中提取 trigram 时，sys_trgm 会忽略非词字符（非字母数字）。在决定字符串中所含的 trigram 集合时，每一个词被认为具有两个空格前缀和一个空格后缀。例如，字符串“cat”中的 trigram 集合是：“c”、“ca”、“cat”以及“at ”。字符串“foo|bar”中的 trigram 集合是：“f”、“fo”、“foo”、“oo ”、“b”、“ba”、“bar”以及“ar ”。

3.99.4.2 操作符

`sys_trgm` 操作符显示 sys_trgm 模块所提供的操作符。

表 3.99.2: sys_trgm 操作符

操作符	返回值	描述
text % text	boolean	如果参数具有超过 sys_trgm.similarity_threshold 设置的当前相似度阈值的相似度，则返回 true。
text <% text	boolean	如果第一个参数中的 trigram 集合与第二个参数中有序 trigram 集合的一个连续部分之间的相似度超过 sys_trgm.word_similarity_threshold 参数设置的当前词相似度阈值，则返回 true。
text %> text	boolean	<% 操作符的交换子。
text <<% text	boolean	如果第二个参数有有序 trigram 集合的一个连续部分匹配词边界，并且其与第一个参数的 trigram 集合的相似度超过 sys_trgm.strict_word_similarity_threshold 参数设置的当前严格词相似度阈值，则返回 true。
text %>> text	boolean	<<% 操作符的交换子。
text <-> text	real	返回参数之间的“距离”，即 1 减去 similarity() 值。
text <<-> text	real	返回参数之间的“距离”，它是 1 减去 word_similarity() 的值。
text <->> text	real	<<-> 操作符的交换子。
text <<<-> text	real	返回参数之间的“距离”，也就是 1 减去 strict_word_similarity() 的值。
text <->>> text	real	<<<-> 操作符的交换子。

3.99.4.3　GUC 参数

sys_trgm.similarity_threshold (real)
设置% 操作符使用的当前相似度阈值。该阈值必须位于 0 和 1 之间（默认是 0.3）。

sys_trgm.word_similarity_threshold (real)
设置 <% 和%> 操作符使用的当前词相似度阈值。该阈值必须位于 0 和 1 之间（默认是 0.6）。

sys_trgm.strict_word_similarity_threshold (real)
设置 «% 和%» 运算符使用的当前严格单词相似性阈值。阈值必须介于 0 和 1 之间（默认值为 0.5）。

3.99.4.4　索引支持

sys_trgm 模块提供了 GiST 和 GIN 索引操作符类，这允许你在一个文本列上创建索引用于快速相似度搜索的目的。这些索引类型支持上述的相似度操作符，并且额外支持基于 trigram 的索引搜索用于 LIKE、ILIKE、~、~* 和 = 查询。sys_trgm 默认版本中，相似性比较不区分大小写。不支持!= 操作符，= 操作符可能不如常规的 B-树索引高效。

例子：

```
CREATE TABLE test_trgm (t text);

INSERT INTO test_trgm VALUES ('words'), ('word'), ('two words'),('ord');

CREATE INDEX trgm_idx ON test_trgm USING GIST (t gist_trgm_ops);
```

或

```
CREATE INDEX trgm_idx ON test_trgm USING GIN (t gin_trgm_ops);
```

GIST 索引 gist_trgm_ops 操作符类近似地将一组三元组表示为位图签名。它的可选整数参数 siglen 决定了签名的长度（以字节为单位）。默认长度是 12 字节。签名长度的有效值在 1 到 2016 字节之间。更长的签名可以更精确地搜索（扫描更小的一部分索引和更少的堆页面），代价是索引更大。例如，创建一个签名长度为 32 字节的索引：

```
CREATE INDEX trgm_idx ON test_trgm USING GIST (t gist_trgm_ops(siglen=32));
```

此时，你将有一个 t 列上的索引，你可以用它进行相似度搜索。一个典型的查询是

```
SELECT t, similarity(t, 'word') AS sml
FROM test_trgm
WHERE t ilike '%word%'
ORDER BY sml DESC, t;
 t         |    sml
```

```
-----------+------------
 word      |          1
 words     |  0.5714286
 two words | 0.36363637
(3 rows)
```

这将返回在文本列中与 word 足够相似的所有值，按最佳匹配到最差匹配的方式排序。索引将被用来让这种搜索变快，即使在一个非常大的数据集上。

上述查询的一种变体是

```
SELECT t, t <-> 'word' AS dist
FROM test_trgm
ORDER BY dist LIMIT 10;
 t         |    dist
-----------+------------
 word      |          0
 words     |  0.4285714
 two words |  0.6363636
 ord       | 0.71428573
(4 rows)
```

这能够用 GiST 索引有效地实现，但是用 GIN 索引无法做到。当只想要少数最接近的匹配时，这通常会比第一种形式更好。

也可以把一个 t 列上的索引用于词相似度或者严格词相似度。典型的查询是：

```
SELECT t, word_similarity('word', t) AS sml
FROM test_trgm
WHERE 'word' <% t
ORDER BY sml DESC, t;
 t         | sml
-----------+-----
 word      |   1
 two words | 0.8
 words     | 0.8
(3 rows)
```

和

```
SELECT t, strict_word_similarity('word', t) AS sml
FROM test_trgm
WHERE 'word' <<% t
ORDER BY sml DESC, t;
 t         |    sml
-----------+-----------
```

```
 word      |         1
 two words | 0.5714286
 words     | 0.5714286
(3 rows)
```

这将返回文本列中符合条件的所有值：这些值在其对应的有序 trigram 集中有一个连续部分与 word 的 trigram 集合足够相似，这些值会按照最好匹配到最差匹配的顺序排列。即便在非常大的数据集上，索引也将使得这一操作的速度够快。

上述查询可能的变体有：

```
SELECT t, 'word' <<-> t AS dist
FROM test_trgm
ORDER BY dist LIMIT 10;
 t         |    dist
-----------+------------
 word      |          0
 words     | 0.19999999
 two words | 0.19999999
 ord       |        0.6
(4 rows)
```

和

```
SELECT t, 'word' <<<-> t AS dist
FROM test_trgm
ORDER BY dist LIMIT 10;
 t         |    dist
-----------+------------
 word      |          0
 words     |  0.4285714
 two words |  0.4285714
 ord       | 0.71428573
(4 rows)
```

这可以用 GiST 索引很高效地实现，但是用 GIN 索引不行。

这些索引类型也支持用于 LIKE 和 ILIKE 的索引搜索，例如

```
t
------
word
(1 row)
```

该索引搜索通过从搜索字符串中抽取 trigram 并且在索引中查找它们来工作。搜索字符串中有更多 trigram，索引搜索的效率更高。不像基于 B-树的搜索，搜索字符串不需要是左锚定的。

这些索引类型也支持用于正则表达式匹配（~ 和 ~* 操作符）的索引搜索，例如

```
SELECT * FROM test_trgm WHERE t ~ '(wo|rd)';
 t
-----------
 words
 word
 two words
 ord
(4 rows)
```

该索引搜索通过从正则表达式中抽取 trigram 并且在索引中查找它们来工作。正则表达式中能抽取出更多 trigram，索引搜索的效率更高。不像基于 B-树的搜索，搜索字符串不需要是左锚定的。

对于 LIKE 和正则表达式搜索，记住没有可抽取 trigram 的模式将退化成一个全索引扫描。

GiST 和 GIN 索引之间的选择依赖于 GiST 和 GIN 的相对性能特性，这在其他地方讨论。

3.99.4.5　文本搜索集成

在与一个全文索引联合使用时，trigram 匹配是一种非常有用的工具。特别是它能有助于识别拼写错误的输入词，这些词直接用全文搜索机制是不会被匹配的。

第一步是生成一个包含文档中所有唯一词的辅助表：

```
CREATE TABLE words AS SELECT word FROM
       ts_stat('SELECT to_tsvector(''simple'', bodytext) FROM documents');
```

其中 documents 是一个具有我们希望搜索的文本域 bodytext 的表。对 to_tsvector 函数使用 simple 配置而不是使用语言相关的配置的原因是，我们想要一个原始（没有去掉词根的）词的列表。

接下来，在词列上创建一个 trigram 索引：

```
CREATE INDEX words_idx ON words USING GIN(word gin_trgm_ops);
```

现在，类似于前面例子的一个 SELECT 查询可以被用来为用户搜索术语中的拼写不当的词建议拼写。要求被选择的词也与拼写不当的词具有相似的长度是一种有用的额外测试。

备注:

由于 words 表已经被生成为一个单独的、静态的表，它将需要被定期地重新生成，这样它能合理地与文档集合保持一致。但是要求它完全与文档集合同步通常是不必要的。

3.99.5　插件 sys_trgm 卸载方法

```
drop extension sys_trgm;
```

3.99.6　插件 sys_trgm 升级方法

sys_trgm 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.100　sys_visibility

3.100.1　插件 sys_visibility 简介

sys_visibility 提供了查看表的 VM 页数据和页级别可见性的函数。它还提供了用来检查 VM 页和表数据的一致性、以及强制删除 VM 重建的函数。默认情况下，sys_truncate_visibility_map(relation regclass) 只能由超级用户执行。其他函数可以由超级用户和 sys_stat_scan_tables 角色的成员执行。

- 插件名为 sys_visibility
- 插件版本 V1.2

3.100.2　插件 sys_visibility 加载方式

```
CREATE EXTENSION sys_visibility;
```

3.100.3　插件 sys_visibility 的参数配置

无需配置任何参数。

3.100.4　插件 sys_visibility 的使用方法

sys_visibility 提供了以下函数：

- sys_visibility_map(relation regclass, blkno bigint, all_visible OUT boolean, all_frozen OUT boolean) returns record

  返回给定关系（relation）的给定块（blkno）在 VM 页面中的“全部可见”（all_visible）和“全部冻结”（all_frozen）位。

- sys_visibility(relation regclass, blkno bigint, all_visible OUT boolean, all_frozen OUT boolean, pd_all_visible OUT boolean) returns record

  返回给定关系（relation）的给定块（blkno）在 VM 页面中的“全部可见”（all_visible）和“全部冻结”（all_frozen）位，外加块上的“全部可见”位（pd_all_visible）。

- sys_visibility_map(relation regclass, blkno OUT bigint, all_visible OUT boolean, all_frozen OUT boolean) returns setof record

  返回给定关系（relation）的每一个块（blkno）在 VM 页面中的“全部可见”（all_visible）和“全部冻结”（all_frozen）位。

- sys_visibility(relation regclass, blkno OUT bigint, all_visible OUT boolean, all_frozen OUT boolean, pd_all_visible OUT boolean) returns setof record

  返回给定关系（relation）的每一个块（blkno）在 VM 页面中的“全部可见”（all_visible）和“全部冻结”（all_frozen）位，外加块上的“全部可见”位（pd_all_visible）。

- sys_visibility_map_summary(relation regclass, all_visible OUT bigint, all_frozen OUT bigint) returns record

  返回给定关系（relation）的 VM 页面中记录的“全部可见”页面（all_visible）和“全部冻结”页面（all_frozen）的数量。

- sys_check_frozen(relation regclass, t_ctid OUT tid) returns setof tid

  返回给定关系（relation）中没有冻结但在 VM 页面中标记“全部冻结”的元组的 TID（t_ctid）。函数返回非空的结果说明 VM 页面已经损坏。

- sys_check_visible(relation regclass, t_ctid OUT tid) returns setof tid

  返回给定关系（relation）中没有全部可见但在 VM 页面中标记“全部可见”的元组的 TID（t_ctid）。函数返回非空的结果说明 VM 页面已经损坏。

- sys_truncate_visibility_map(relation regclass) returns void

  清空给定关系（relation）的 VM 页面。如果该关系的 VM 页面可能已经损坏，函数可以用来强制重建 VM 页面。清空后在该关系上的第一次 VACUUM 会重建 VM 页面，在此之前 VM 页面被认为存储了全 0 数据。

3.100.5　插件 sys_visibility 卸载方法

```
DROP EXTENSION sys_visibility;
```

3.100.6　插件 sys_visibility 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION sys_visibility UPDATE TO '1.1';
```

3.101　sysaudit

3.101.1　插件 sysaudit 简介

sysaudit 是 KingbaseES 的一个扩展插件，主要用于支持数据库审计功能，对数据库系统中发生的动作（或事件），将其对应的操作对象、操作时间等信息记录下来。

KingbaseES 数据库提供了一套完整的审计机制，用来保证对数据库中的各种行为进行监控，进而为数据库的安全、可靠和有效提供有力的保障。

- 插件名为 sysaudit
- 插件版本 V1.0

3.101.2　插件 sysaudit 加载方式

插件 sysaudit 是系统初始化插件，系统自动加载。

3.101.3　插件 sysaudit 的参数配置

具体参考《KingbaseES 数据库安全指南》数据库审计。

3.101.4　插件 sysaudit 使用方法

具体参考《KingbaseES 数据库安全指南》数据库审计。

3.101.5　插件 sysaudit 卸载方法

插件 sysaudit 是系统初始化插件，用户无法卸载。

3.101.6　插件 sysaudit 升级方法

具体参考《KingbaseES 数据库安全指南》数据库审计。

3.102 sysencrypt

3.102.1 插件 sysencrypt 简介

sysencrypt 是一个 KingbaseES 的扩展，可以实现数据加解密。这种方式更为灵活，当数据库的实用场景需要数据加解密时，加载插件即可。而不需要该功能时，卸载插件即可。

- 插件名为 sysencrypt
- 插件版本 V1.0

3.102.2 插件 sysencrypt 加载方式

在使用 sysencrypt 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

```
shared_preload_libraries = 'sysencrypt' # (change requires restart)

-- 创建插件
\c - system
create extension sysencrypt;
```

3.102.3 插件 sysencrypt 参数配置

`sysencrypt.encrypt_user_tablespace`
是否默认创建表空间加密，默认值为 false 即创建表空间默认不加密。

```
show sysencrypt.encrypt_user_tablespace;
 encrypt_user_tablespace
-------------------------
 off
(1 row)

\c - system
alter system set sysencrypt.encrypt_user_tablespace to on;
select sys_reload_conf();
show sysencrypt.encrypt_user_tablespace;
 encrypt_user_tablespace
-------------------------
 on
(1 row)
```

3.102.4　插件 sysencrypt 使用方法

具体参考《KingbaseES 数据库安全指南》加密插件。

3.102.5　插件 sysencrypt 卸载方法

通过 drop extension 来卸载插件：

```
drop extension sysencrypt;
```

如果设置了 shared_preload_libraries，则需要删除‘sysencrypt’：

```
shared_preload_libraries = ''
```

3.102.6　插件 sysencrypt 升级方法

sysencrypt 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.103　sysmac

3.103.1　插件 sysmac 简介

sysmac 是 KingbaseES 的一个扩展插件，主要用于支持标记和强制访问控制，保护用户数据，防止非法窃取。

强制访问控制（MAC）与 DAC 相比，MAC 提供更严格和灵活的控制方式。MAC 首先为所控制的主体和客体指派安全标记，然后依据这些标记进行访问仲裁。并且，只有主体标记能支配客体标记时才允许主体访问。

- 插件名为 sysmac
- 插件版本 V1.0

3.103.2　插件 sysmac 加载方式

在使用 sysmac 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 KingbaseES 数据库。

示例：

```
shared_preload_libraries = 'sysmac'
```

3.103.3　插件 sysmac 的参数配置

`sysmac.enable_mac` 启动强制访问控制功能的开关，取值为 on 或者 off，缺省为 off。

```
show sysmac.enable_mac;
 sysmac.enable_mac
-------------------
 off
(1 row)

\c - sso
alter system set sysmac.enable_mac = on;
select sys_reload_conf();
show sysmac.enable_mac;
 sysmac.enable_mac
-------------------
 on
(1 row)
```

`sysmac.enable_col_mac`
启用列级强制访问控制功能的开关，默认是 false，不开启。

3.103.4　插件 sysmac 使用方法

具体参考《KingbaseES 数据库安全指南》标记和强制访问控制。

3.103.5　插件 sysmac 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.103.6　插件 sysmac 升级方法

插件 sysmac 与插件 sys_anon 的升级方式相同，具体参考《KingbaseES 数据库安全指南》标记和强制访问控制。

3.104 sysprivilege

3.104.1 插件 sysprivilege 简介

sysprivilege 是一个 KingbaseES 的扩展，可以实现对数据库系统 ANY 权限的扩展。这种方式更为灵活，当数据库的实用场景需要系统 ANY 权限时，加载插件即可。而不需要该功能时，卸载插件即可。

- 插件名为 sysprivilege
- 插件版本 V1.0

3.104.2 插件 sysprivilege 加载方式

在使用 sysprivilege 插件之前，我们需要添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启 kingbase 数据库。

示例：

```
shared_preload_libraries = 'sysprivilege'
```

3.104.3 插件 sysprivilege 的参数配置

无

3.104.4 插件 sysprivilege 使用方法

系统 ANY 权限是属于系统级权限，目前支持的对象为四种：表，视图，序列，存储过程。ANY 权限还为每种权限设置了 ADMIN 选项，标志是否为当前权限的 ADMIN 用户，如果是 ADMIN 用户，那么允许当前用户授权 ANY 权限给其他用户。注意:ANY 权限不可以操作数据库系统对象。

如下表所示：

表 3.104.1: ANY 权限介绍

对象类型	权限	说明
TABLE	CREATE ANY TABLE	允许在所有模式下创建表
TABLE	ALTER ANY TABLE	允许修改所有模式下的所有表
TABLE	DROP ANY TABLE	允许删除或截断所有模式下的所有表

-续表-

对象类型	权限	说明
TABLE	SELECT ANY TABLE	允许在所有模式下查询所有表
TABLE	UPDATE ANY TABLE	允许在所有模式下更新所有表
TABLE	INSERT ANY TABLE	允许在所有模式下的所有表中插入数据
TABLE	DELETE ANY TABLE	允许在所有模式下的所有表中删除数据
VIEW	CREATE ANY VIEW	允许在所有模式下创建视图
VIEW	DROP ANY VIEW	允许在所有模式下删除视图
SEQUENCE	CREATE ANY SEQUENCE	允许在所有模式下创建序列
SEQUENCE	ALTER ANY SEQUENCE	允许修改所有模式下的序列
SEQUENCE	DROP ANY SEQUENCE	允许删除所有模式下的序列
SEQUENCE	SELECT ANY SEQUENCE	允许对所有模式下的序列执行 currval，seedval，lastval 和 incrementval 函数
SEQUENCE	UPDATE ANY SEQUENCE	允许对所有模式下的序列执行 setval 和 nextval 函数
PROCEDURE	CREATE ANY PROCEDURE	允许在所有模式下创建存储过程和函数
PROCEDURE	ALTER ANY PROCEDURE	允许修改所有模式下的存储过程和函数
PROCEDURE	DROP ANY PROCEDURE	允许删除所有模式下的存储过程和函数
PROCEDURE	EXECUTE ANY PROCEDURE	允许执行所有模式下或包中存储过程和函数、引用任意包中变量

3.104.4.1　相关表和视图

- sys_sysprivilege

是一个系统表，查询出所有拥有系统 ANY 权限的用户。

sys_sysprivilege 视图的每个字段意义：

grantee	privilege	admin_option
用户 OID	权限 ID	是否是 ADMIN 用户

- user_any_privs

是一个系统视图，用于提供权限 ID 的转换，显示为权限具体的名字。每个用户只可以查看自己的拥有的 ANY 权限。

user_any_privs 视图的每个字段意义：

grantee	privilege	admin_option
用户 OID	权限名字	是否是 ADMIN 用户

3.104.4.2　示例

```
create user u1;
create user u2;
set session authorization u1;
create table testu1(id int);
insert into testu1 values(123);
set session authorization u2;
select * from testu1; -- error
set session authorization system;
grant select any table to u2;
set session authorization u2;
select * from testu1;
```

3.104.5　插件 sysprivilege 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

示例：

```
shared_preload_libraries = ''
```

3.104.6 插件 sysprivilege 升级方法

sysprivilege 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级这些插件。

3.105 sysreuse_residual_data

3.105.1 插件 sysreuse_residual_data 简介

sysreuse_residual_data 是 KingbaseES 扩展插件，通过资源申请和释放的地方清除介质上的残留信息，以达到客体重用的要求。

- 插件名为 sysreuse_residual_data
- 插件版本 V1.0

3.105.2 插件 sysreuse_residual_data 加载方式

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = 'sysreuse_residual_data'
```

3.105.3 插件 sysreuse_residual_data 的参数配置

`sysreuse_residual_data.enable_obj_reuse`

启用客体重用功能，取值为 on 或者 off，缺省为 off。

```
\c test system
SHOW sysreuse_residual_data.enable_obj_reuse;
 sysreuse_residual_data.enable_obj_reuse
-------------------------------
 off
(1 row)

ALTER SYSTEM SET sysreuse_residual_data.enable_obj_reuse = true;
SELECT sys_reload_conf();

SHOW sysreuse_residual_data.enable_obj_reuse;
```

```
 sysreuse_residual_data.enable_obj_reuse
--------------------------------
 on
(1 row)
```

3.105.4　插件 sysreuse_residual_data 的使用方法

加载 sysreuse_residual_data 插件后，程序通过资源申请和释放的地方清除介质上的残留信息，以达到客体重用的要求。

共享缓冲区销毁的方式：全 0 覆盖。

磁盘上文件残留信息销毁：采用置 0 方法。

示例：

```
-- 创建插件
\c test system
create extension sysreuse_residual_data;
CREATE EXTENSION
```

3.105.5　插件 sysreuse_residual_data 卸载方法

修改 kingbase.conf 文件中 `shared_preload_libraries` 参数后重启数据库。

```
shared_preload_libraries = ''
```

3.105.6　插件 sysreuse_residual_data 升级方法

sysreuse_residual_data 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.106　tablefunc

3.106.1　插件 tablefunc 简介

插件 tablefunc 是 KingbaseES 的一个扩展插件。tablefunc 插件提供了多个返回表 (多行) 的函数。tablefunc 插件可作为如何编写返回多行的 C 函数的例子。

- 插件名为 tablefunc

- 插件版本 V1.0

3.106.2　插件 tablefunc 加载方式

KingbaseES 数据库默认不加载 tablefunc 插件，用户可以使用如下命令自行人为加载。

示例：

```
create extension tablefunc;
```

3.106.3　插件 tablefunc 的参数配置

无需配置任何参数

3.106.4　插件 tablefunc 的使用方法

表 tablefunc 函数列出了 tablefunc 插件提供的函数。

函数	返回	描述
normal_rand(int numvals, float8 mean, float8 stddev)	setof float8	产生一个正太分布的随机值集合
croasstab(text sql)	setof record	产生一个包含多行的外加 N 个值的数据透视表，其中 N 由调用查询中指定的行类型决定
croasstabN(text sql)	setof table _croasstab _N	产生一个包含行名称外加 N 个值的“数据透视表”，croasstabl2,croasstabl3 和 croasstabl4 是预定义的，但你可以按照下文所述创建额外的 croasstablN 函数
croasstable(text source_sql, text category_sql)	setof record	产生一个“数据透视表”，其值列由第二个查询指定
crosstab(text sql, int N)	setof record	crosstab(text) 的废弃版本。参数 N 现在被忽略，因为值列的数量总是由调用查询所决定
connectby(text relname, text keyid_fld, text parent_keyid_fld [,textorderby_fld], text start _with, int max_depth [, text branch_delim])	setof record	产生一个层次树结构的表达

3.106.5　插件 tablefunc 卸载方法

用户如想卸载 tablefunc 插件可使用如下命令。

示例：

```
drop extension tablefunc;
```

3.106.6　插件 tablefunc 升级方法

tablefunc 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.107　tcn

3.107.1　插件 tcn 简介

插件 tcn 是 KingbaseES 的一个扩展插件。tcn 模块提供一个触发器函数，它通知监听者有关它所附着的任意表上的改变。它必须被用作一个行级 AFTER 触发器。

在一个 CREATE TRIGGER 语句中只可以为该函数提供一个参数，并且是可选的。如果提供该参数，它将被作为用于通知的频道名。如果忽略它，频道名将使用 tcn。

通知的负载由表名、一个指示所执行操作类型的字母以及用于主键列的列名/值对构成。每一部分都用逗号与下一部分隔开。为了便于解析对正则表达式的使用，表和列名总是被包裹在双引号内，并且数据值总是被包裹在单引号内。嵌入的引号都被双写。

- 插件名为 tcn
- 插件版本 V1.0

3.107.2　插件 tcn 加载方式

KingbaseES 数据库默认不加载 tcn 扩展插件。如果用户想使用此扩展插件功能。请参考如下示例，使用 CREATE EXTENSION 命令进行加载。

示例：

```
CREATE EXTENSION tcn;
```

3.107.3　插件 tcn 的参数配置

无需配置任何参数。

3.107.4　插件 tcn 的使用方法

用户如果想使用 tcn 扩展插件，请参考如下示例。

示例：

```
create table tcndata ( a int not null, b date not null, c text, primary key (a, b));
CREATE TABLE

create trigger tcndata_tcn_trigger after insert or update or delete on tcndata for each row execute function
triggered_change_notification();
CREATE TRIGGER

listen tcn;
LISTEN

insert into tcndata values (1, date '2012-12-22', 'one'),(1, date '2012-12-23', 'another'),(2, date '2012-
12-23', 'two');
INSERT 0 3
Asynchronous notification "tcn" with payload ""tcndata",I,"a"='1',"b"='2012-12-22'" received from server
process with PID 22770.
Asynchronous notification "tcn" with payload ""tcndata",I,"a"='1',"b"='2012-12-23'" received from server
process with PID 22770.
Asynchronous notification "tcn" with payload ""tcndata",I,"a"='2',"b"='2012-12-23'" received from server
process with PID 22770.

update tcndata set c = 'uno' where a = 1;
UPDATE 2
Asynchronous notification "tcn" with payload ""tcndata",U,"a"='1',"b"='2012-12-22'" received from server
process with PID 22770.
Asynchronous notification "tcn" with payload ""tcndata",U,"a"='1',"b"='2012-12-23'" received from server
process with PID 22770.

delete from tcndata where a = 1 and b = date '2012-12-22';
DELETE 1
Asynchronous notification "tcn" with payload ""tcndata",D,"a"='1',"b"='2012-12-22'" received from server
process with PID 22770.
```

3.107.5　插件 tcn 卸载方法

如果用户想卸载 tcn 扩展插件，请使用 DROP EXTENSION 命令进行卸载。

示例：

```
DROP EXTENSION tcn;
```

3.107.6　插件 tcn 升级方法

tcn 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.108　tds_fdw

3.108.1　插件 tds_fdw 简介

tds_fdw 是一个 KingbaseES 外部数据包装器，可以连接到使用表格数据流（TDS）协议的数据库，如 Sybase 数据库和 Microsoft SQL Server。

这个外部数据包装器需要一个实现 DB-Library 接口的库，比如 FreeTDS。本插件中已包含 FreeTDS-1.3。

当前版本暂不支持 JOIN 下推，以及写入操作（DML 操作）。

当 match_column_names 设为 enable 时，支持包含列名的 WHERE 条件下推。

- 插件名为 tds_fdw
- 插件版本 V2.0.2

3.108.2　插件 tds_fdw 加载方式

```
CREATE EXTENSION tds_fdw;
```

3.108.3　插件 tds_fdw 的参数配置

3.108.3.1　外部服务器参数

外部服务器允许设置以下参数选项：

- servername

  必选：是

  默认值：127.0.0.1

  外部服务器的服务器名、地址或主机名。

  可以是一个 DSN，如 freetds.conf 中所述。请参考 FreeTDS 名称查找。

可以将此选项设置为以逗号分隔的服务器名称列表，然后尝试每个服务器，直到连接成功。这种配置方法对于自动故障切换至备用服务器非常有用。

- port

  必选：否

  外部服务器的端口。如果 servername 是 DSN 的话，可以在 freetds.conf 指定端口，不需要在此提供。

- database

  必选：否

  外部服务器将要连接到的数据库名称。

- dbuse

  必选：否

  默认值：0

  当该选项为 0 时，tds_fdw 将直接连接到数据库。否则 tds_fdw 将连接到服务器的默认数据库，然后通过调用 DB-Li brary 中的 dbuse() 函数选择数据库。

  对于 Azure，目前需要把 dbuse 选项设置为 0。

- language

  必选：否

  用于消息的语言设置以及日期格式的区域设置。在大多数系统中，FreeTDS 可能默认为 `us_english` 。也可以在 freetds.conf 中进行修改。

- character_set

  必选：否

  用于连接的客户端字符集。

  对于 TDS 协议版本在 7.0+ 的，连接始终使用 UCS-2，因此该参数在这些情况下不起作用。

- tds_version

  必选：否

  用于该外部服务器的 TDS 协议的版本。参考 FreeTDS 中 Choosing a TDS protocol version 和 History of TDS Versions 章节的说明。

- msg_handler

  必选：否

  默认值：blackhole

  用于 TDS 消息处理程序的函数。可选值为 `notice` 和 `blackhole`。使用 `notice` 选项时，TDS 消息将转换为 KingbaseES 通知。使用 `blackhole` 选项时，TDS 消息将被忽略。

- fdw_startup_cost

  必选：否

  用于表示在查询计划中使用此 FDW 的开销。

- fdw_tuple_cost

  必选：否

  用于表示从查询计划中使用的此外部服务器获取行为的开销。

外部服务器定义过程中可接受的外部表参数：

- use_remote_estimate
- row_estimate_method

3.108.3.2　外部表参数

创建外部表允许设置以下参数选项：

- query

  必选：是（与 table 互斥）

  用于查询外部表的查询语句。

- schema_name

  必选：否

  表所在的模式名。模式名也可以包含在 table_name 中，因此这里不是必须的。

- table_name

  别名：table

  必选：是（与 query 互斥）

  要查询的外部服务器上的表。

- match_column_names

  必选：否

  设置是通过比较本地列和远程列的列名来进行匹配，还是直接使用它们在结果集中的排列顺序。

- use_remote_estimate

  必选：否

  设置通过在远程服务器上执行一些操作来估算表的大小（如 `row_estimate_method` 所定义）；还是仅使用本地估计（如 `local_tuple_estimate` 所定义）。

- local_tuple_estimate

  必选：否

  当 `use_remote_estimate` 关闭时，使用的估计元组数的本地设置。

- row_estimate_method

  必选：否

  默认值：`execute`

  可选值包含以下内容：

  – `execute`：在远程服务器上执行查询，并获取查询中的实际行数。

  – `showplan_all`：使用 MS SQL Server 的 `SET SHOWPLAN_ALL` 来获取估计的行数。

外部表列参数：

- column_name

  必选：否

  远程服务器上列的名称。如果未设置，则假定远程列名与本地列名相同。如果表的 `match_column_names` 设置为 `0`，则将不会使用列名，此选项的设置将被忽略。

3.108.3.3　外部数据包装器参数

创建外部数据包装器允许设置以下参数选项：

- username

  必选：是

  外部服务器上的用户名。

- password

  必选：是

  外部服务器上用户名对应的密码。

3.108.3.4　导入外部模式参数

导入外部模式允许设置以下参数选项：

- import_default

  必选：否

  默认值：`false`

  控制外部表的定义中是否包含列的默认值表达式。

- import_not_null

  必选：否

  默认值：`true`

  控制外部表定义中是否包含列的非空约束。

3.108.3.5　变量

- tds_fdw.show_before_row_memory_stats：在获取每一行之前，将内存上下文统计信息打印到 KingbaseES 日志中。
- tds_fdw.show_after_row_memory_stats：在获取每一行之后，将内存上下文统计信息打印到 KingbaseES 日志中。
- tds_fdw.show_finished_memory_stats：在查询完成之后，将内存上下文统计信息打印到 KingbaseES 日志中。

3.108.4　插件 tds_fdw 使用方法

3.108.4.1　创建外部服务器

示例

```
CREATE SERVER mssql_svr
        FOREIGN DATA WRAPPER tds_fdw
        OPTIONS (servername '127.0.0.1', port '1433', database 'tds_fdw_test', tds_version '7.1');
```

上面例子中 servername，port，database，tds_version 请根据实际环境填写

3.108.4.2　创建外部表

示例

使用 `table_name` 定义：

```
CREATE FOREIGN TABLE mssql_table (
        id integer,
        data varchar)
        SERVER mssql_svr
        OPTIONS (table_name 'dbo.mytable', row_estimate_method 'showplan_all');
```

或者使用 `schema_name` 和 `table_name` 定义：

```
CREATE FOREIGN TABLE mssql_table (
        id integer,
        data varchar)
        SERVER mssql_svr
        OPTIONS (schema_name 'dbo', table_name 'mytable', row_estimate_method 'showplan_all');
```

或者使用 query 定义：

```
CREATE FOREIGN TABLE mssql_table (
        id integer,
        data varchar)
        SERVER mssql_svr
        OPTIONS (query 'SELECT * FROM dbo.mytable', row_estimate_method 'showplan_all');
```

或者设置一个远程表的列：

```
CREATE FOREIGN TABLE mssql_table (
        id integer,
        data varchar OPTIONS (column_name 'data'))
        SERVER mssql_svr
        OPTIONS (schema_name 'dbo', table_name 'mytable', row_estimate_method 'showplan_all');
```

3.108.4.3　创建外部数据包装器

示例

```
CREATE USER MAPPING FOR system
        SERVER mssql_svr
        OPTIONS (username 'sa', password '');
```

3.108.4.4　导入外部模式

示例

```
IMPORT FOREIGN SCHEMA dbo
        EXCEPT (mssql_table)
        FROM SERVER mssql_svr
        INTO public
        OPTIONS (import_default 'true');
```

3.108.4.5　变量

设置变量

要设置变量，请使用 `set` 命令：

```
SET tds_fdw.show_finished_memory_stats=1;
```

3.108.4.6　EXPLAIN

`EXPLAIN (VERBOSE)` 将显示在远程系统上发出的查询语句。

3.108.4.7　关于字符集和编译的说明

1. 如果在使用 Unicode 数据时，MS SQL Server 出现如下错误：

```
NOTICE: DB-Library notic: Msg #: 4004, Msg state: 1, Msg: Unicode data in a Unicode-only collation or ntext
data cannot be sent to clients using DB-Library (such as ISQL) or ODBC version 3.7 or earlier., Server:
PILLIUM\SQLEXPRESS, Process:,
ERROR: DB-Library error: DB #: 4004, DB Msg: General SQL Server error: Check messages from the SQL Server
```

可能需要在 freetds.conf 中手动设置 tds 版本为 7.0 或更高。请参考 freetds.conf 文件和 FreeTDS 的 `Choosing a TDS procotol version` 章节。

尽管许多较新版本的 TDS 协议仅使用 USC-2 与服务器进行通信，FreeTDS 将 UCS-2 转换为你选择的客户端字符集。要设置客户端字符集，可以在 `freetds.conf` 文件中设置 `client chatset`。参考 `freetds.conf` 文件和 `Localization and TDS 7.0` 章节。

3.108.4.8　加密连接 MSSQL

加密连接由 FreeTDS 处理。需要在 `freetds.conf` 中进行配置。可以在 `freetds.conf` 中搜索 `encryption`。

3.108.5　插件 tds_fdw 卸载方法

```
DROP EXTENSION tds_fdw;
```

3.108.6　插件 tds_fdw 升级方法

```
ALTER EXTENSION tds_fdw UPDATE TO new_version;
```

3.109 tsm_system_rows

3.109.1 插件 tsm_system_rows 简介

插件 tsm_system_rows 是 KingbaseES 的一个扩展插件。提供了表采样方法 SYSTEM_ROWS，它可以用在 SELECT 命令的 TABLESAMPLE 子句中。

这种表采样方法接受一个整数参数，它是要读取的最大行数。得到的采样将总是包含正好这么多行，除非该表中没有足够的行，在那种情况下整个表都会被选择出来。

和内建的 SYSTEM 采样方法一样，SYSTEM_ROWS 执行块级别的采样，所以采样不是完全随机的，而是服从于聚簇效果，特别是只要求少量行时。SYSTEM_ROWS 不支持 REPEATABLE 子句。

- 插件名为 tsm_system_rows
- 插件版本 V1.0

3.109.2 插件 tsm_system_rows 加载方式

KingbaseES 数据库默认不加载 tsm_system_rows 扩展插件，如果用户想使用 tsm_system_rows 功能，请使用 CREATE EXTENSION 命令进行加载。

示例：

```
CREATE EXTENSION tsm_system_rows;
```

3.109.3 插件 tsm_system_rows 的参数配置

无需配置任何参数

3.109.4 插件 tsm_system_rows 的使用方法

KingbaseES 加载 tsm_system_rows 插件后，用户使用 tsm_system_rows 可参考如下示例.

示例：

```
SELECT * FROM my_table TABLESAMPLE SYSTEM_ROWS(100);
```

这个命令从表 my_table 中返回一个 100 行的采样 (除非该表没有 100 个可见行，那时将会返回其中所有的行)。

3.109.5 插件 tsm_system_rows 卸载方法

如果用户不想再使用 tsm_system_rows 扩展插件的功能，可以使用 DROP EXTENSION 命令进行卸载。

示例：

```
DROP EXTENSION tsm_system_rows;
```

3.109.6 插件 tsm_system_rows 升级方法

tsm_system_rows 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.110 tsm_system_time

3.110.1 插件 tsm_system_time 简介

插件 tsm_system_time 提供了表采样方法 SYSTEM_TIME，它可以用在 SELECT 命令的 TABLESAMPLE 子句中。这种表采样方法接受一个浮点类型的参数，指定花在读取上的最大时间，单位毫秒。这个方法可以直接控制采样的查询时间，但代价是采样的大小难以预测。和内建的 SYSTEM 采样方法一样，SYSTEM_TIME 执行块级别的采样，所以采样不是完全随机的，而是服从于聚簇效果，特别是只采样少量行时。

SYSTEM_TIME 不支持 REPEATABLE 子句。

- 插件名为 tsm_system_time
- 插件版本 V1.0

3.110.2 插件 tsm_system_time 加载方式

```
CREATE EXTENSION tsm_system_time;
```

3.110.3 插件 tsm_system_time 的参数配置

无需配置任何参数

3.110.4 插件 tsm_system_time 的使用方法

KingbaseES 加载 tsm_system_time 插件后，用户使用 tsm_system_time 可参考如下示例。

示例：

```
SELECT * FROM my_table TABLESAMPLE SYSTEM_TIME(1000);
```

这个命令从表 my_table 中返回 1000 毫秒内可以读取到的采样（如果表在时间内被读完将返回所有行）。

3.110.5　插件 tsm_system_time 卸载方法

```
DROP EXTENSION tsm_system_time;
```

3.110.6　插件 tsm_system_time 升级方法

通过 ALTER EXTENSION 升级插件。

示例，升级到 1.1：

```
ALTER EXTENSION tsm_system_time UPDATE TO '1.1';
```

3.111　unaccent

3.111.1　插件 unaccent 简介

unaccent 是一个文本搜索字典，它能从词位中移除重音（附加符号）。它是一个过滤词典，这表示它的输出总是会被传递给下一个字典（如果有），这和字典的通常行为不同。这允许为全文搜索做与重音无关的处理。

unaccent 的当前实现不能被用作 thesaurus 字典的正规化字典。

3.111.2　插件 unaccent 加载方式

```
create extension unaccent;
```

3.111.3　插件 unaccent 的参数配置

unaccent 字典接受下列选项：

* RULES 是包含翻译规则列表的文件的基本名。这个文件必须被存储在 $SHAREDIR/tsearch_data/$SHAREDIR 表示 KingbaseES 安装的共享数据目录）中。它的名称必须以.rules（不包含在 RULES 参数中）结束。

规则文件具有下面的格式：

* 每一行表示一个由带有重音的字符和不带重音的字符构成的对。第一个字符将被翻译成第二个。例如：

```
À        A
Ằ        A
Ặ        A
```

两个字符必须由空格分隔，并且一行上的任何前导或尾随空白都将被忽略。

* 或者，如果一行只给出一个字符，则删除该字符的实例; 这在用单独的字符表示重音的语言中是有用的。

* 实际上，每个“字符”可以是不包含空格的任何字符串，因此，除了去除变音符之外，unaccent 字典也可以用于其他类型的字符串替换。

* 规则文件必须以 UTF-8 编码方式存储。加载时，数据将自动转换为当前数据库的编码。任何含有不可翻译字符的行都将被忽略，因此规则文件可以包含当前编码中不适用的规则。在 unaccent.rules 中可以找到一个更完整的例子，它可以直接用于大部分欧洲语言，当 unaccent 模块被安装时，它被安装在 $SHAREDIR/tsearch_data/中。

3.111.4　插件 unaccent 的使用方法

安装 unaccent 扩展会创建一个文本搜索模板 unaccent 和一个基于前者的字典 unaccent。unaccent 字典有默认的参数设置 RULES=‘unaccent’，这会让该字典使用标准的 unaccent.rules 文件。如果希望修改该参数，可以

```
alter text search dictionary unaccent (rules='my_rules');
```

其中 my_rules 需要用户自行建立。

或者基于该模板创建新的字典。

要测试该字典，可以尝试如下方法：

```
select ts_lexize('unaccent','Hôtel');
ts_lexize
-----------
{Hotel}
(1 row)
```

这里是一个展示把 unaccent 字典插入到一个文本搜索配置的例子：

```
create text search configuration fr ( copy = french );

alter text search configuration fr
        alter mapping for hword, hword_part, word
        with unaccent, french_stem;

select to_tsvector('fr','Hôtels de la Mer');
    to_tsvector
--------------------
 'hotel':1 'mer':4
(1 row)

select to_tsvector('fr','Hôtel de la Mer') @@ to_tsquery('fr','Hotels');
 ?column?
----------
```

```
 t
(1 row)

select ts_headline('fr','Hộtel de la Mer',to_tsquery('fr','Hotels'));
      ts_headline
------------------------
 <b>Hộtel</b> de la Mer
(1 row)
```

unaccent() 函数从一个给定的字符串中移除重音（附加符号）。基本上，它是 unaccent 字典的一个包装器，但是它能在普通的文本搜索环境之外使用。

```
unaccent([dictionary regdictionary, ] string text) returns text
```

如果省略 dictionary 参数，则使用名为 unaccent 并且与 unaccent() 函数有相同模式的文本搜索词典。

例如：

```
select unaccent('unaccent', 'Hộtel');
 unaccent
-----------
 Hotel
(1 row)


select unaccent('Hộtel');
 unaccent
-----------
 Hotel
(1 row)
```

3.111.5　插件 unaccent 卸载方法

```
drop extension unaccent;
```

3.111.6　插件 unaccent 升级方法

unaccent 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.112　utl_file

3.112.1　插件 utl_file 简介

插件 utl_file 补充了一些对于文件操作的函数。

- 插件名为 utl_file
- 插件版本 V1.0

3.112.2　插件 utl_file 加载方式

在命令行执行 create extension utl_file;

示例

```
create extension utl_file;
```

3.112.3　插件 utl_file 的参数配置

无需配置任何参数

3.112.4　插件 utl_file 的使用方法

加载插件 ltree 后，按照如下方式使用相应功能

3.112.4.1　utl_file.fopen

utl_file.fopen 打开一个文件，返回一个文件句柄

语法

```
fopen(location text, filename text, open_mode text, max_linesize integer, encoding name) 返回 integer
fopen(location text, filename text, open_mode text, max_linesize integer) 返回 integer
```

参数

location：指定文件的位置。

filename：指定要打开的文件名。

open_mode：指定文件要打开的模式。

encoding：指定编码字符集。

返回值

UTL_FILE.FILE_TYPE（int 类型）。open_mode 可以是’R’，‘W’，‘A’。utl_file.fopen 为 procedure。

示例

```
DROP DIRECTORY IF EXISTS MY_DIR;

CREATE DIRECTORY MY_DIR AS '/tmp/'; -- 这个目录需要存在，或者人为手工创建。

    DROP PROCEDURE IF EXISTS TEST_FOPEN;

    \set SQLTERM /

CREATE OR REPLACE PROCEDURE TEST_FOPEN(
    V_OUT_DATA IN VARCHAR2,
    V_SQLPARAM IN VARCHAR2
) AS
DECLARE
    OUT_DATA VARCHAR2(30000);
    FILE_HANDLE UTL_FILE.FILE_TYPE;
    FILD_NAME VARCHAR2(1000) DEFAULT 'output.txt';
BEGIN
    OUT_DATA := V_OUT_DATA;
    FILE_HANDLE := UTL_FILE.FOPEN('MY_DIR', FILD_NAME, 'A', 30000);
    UTL_FILE.PUT_LINE(FILE_HANDLE, OUT_DATA);
    UTL_FILE.PUT_LINE(FILE_HANDLE, V_SQLPARAM);
    UTL_FILE.FCLOSE(FILE_HANDLE);
END TEST_FOPEN;
    /

    \set SQLTERM ;

CALL TEST_FOPEN('111', '222'); --执行成功后可看到输入的文字。

    \! cat /tmp/output.txt
```

3.112.4.2　utl_file.put_line

utl_file.put_line 在文件中写入一行内容

语法

```
put_line(file file_type, buffer text);
put_line(file file_type, buffer text, autoflush bool);
```

参数

file：指定 UTL_FILE.FILE_TYPE 类型值。

buffer：指定目标文件名。

autoflush：指定是否自动刷新。

返回值

无返回值。

注解

utl_file.put_line 为 procedure。

示例

```
DROP DIRECTORY IF EXISTS MY_DIR;

CREATE DIRECTORY MY_DIR AS '/tmp/'; -- 这个目录需要存在，或者人为手工创建。

DROP PROCEDURE IF EXISTS TEST_FOPEN;

\set SQLTERM /

CREATE OR REPLACE PROCEDURE TEST_PUT_LINE(
    V_OUT_DATA IN VARCHAR2,
    V_SQLPARAM IN VARCHAR2
)
AS
DECLARE
    OUT_DATA VARCHAR2(30000);
    FILE_HANDLE UTL_FILE.FILE_TYPE;
    FILD_NAME VARCHAR2(1000) DEFAULT 'output.txt';
BEGIN
    OUT_DATA := V_OUT_DATA;
    FILE_HANDLE := UTL_FILE.FOPEN('MY_DIR', FILD_NAME, 'A', 30000);
    UTL_FILE.PUT_LINE(FILE_HANDLE, OUT_DATA);
    UTL_FILE.PUT_LINE(FILE_HANDLE, V_SQLPARAM);
    UTL_FILE.FCLOSE(FILE_HANDLE);
END TEST_PUT_LINE;
    /

    \set SQLTERM ;

call TEST_PUT_LINE('111', '222'); --执行成功后可看到输入的文字。

    \! cat /tmp/output.txt
```

3.112.4.3　utl_file.put

utl_file.put 将数据强制写入到文件中

语法

```
put(file file_type, buffer text);
```

参数

file：指定 UTL_FILE.FILE_TYPE 类型值。buffer：指定目标文件名。

返回值

无返回值

注解

utl_file.put 为 procedure。

示例

同 utl_file.putline。

3.112.4.4　utl_file.fclose

utl_file.fclose 关闭目标文件

语法

```
fclose(file file_type);
```

参数

file 指定 UTL_FILE.FILE_TYPE 类型值。

返回值

无返回值。

返回值

utl_file.fclose 为 procedure。

示例

```
\set SQLTERM /

CREATE or replace PROCEDURE proc1()
as
DECLARE
dirname          VARCHAR(100) DEFAULT 'my_dir';
v_empfile_src    UTL_FILE.FILE_TYPE;
```

```
v_empfile_tgt    UTL_FILE.FILE_TYPE;
v_src_file       VARCHAR(20) DEFAULT 'output.txt';
v_dest_file      VARCHAR(20) DEFAULT 'test.txt';
v_empline        VARCHAR(200);
begin
v_empfile_src = UTL_FILE.FOPEN(dirname,v_src_file,'r');
v_empfile_tgt = UTL_FILE.FOPEN(dirname,v_dest_file,'w');
LOOP
 BEGIN
 UTL_FILE.GET_LINE(v_empfile_src,v_empline);
 UTL_FILE.PUT(v_empfile_tgt,v_empline);
 UTL_FILE.NEW_LINE(v_empfile_tgt);
 UTL_FILE.FFLUSH(v_empfile_tgt);
 EXCEPTION
  WHEN OTHERS THEN
    EXIT;
 END;
END LOOP;
DBMS_OUTPUT.PUT_LINE('Updated file: ' || v_dest_file);
UTL_FILE.FCLOSE(v_empfile_tgt);
UTL_FILE.FCLOSE(v_empfile_src);
END proc1;
/

\set SQLTERM ;
```

3.112.4.5　utl_file.fclose_all

utl_file.fclose_all 关闭所有文件

语法

```
utl_file.fclose_all();
```

参数

无参数

返回值

无返回值。

注解

utl_file.fclose_all 为 procedure。

示例

```
\set SQLTERM /

CREATE OR REPLACE FUNCTION read_file(dir text) RETURNS void AS $$
DECLARE
f utl_file.file_type;
BEGIN
f := utl_file.fopen(dir, 'regress_orafce.txt', 'r');
FOR i IN 1..11 LOOP
RAISE NOTICE '[%] >>%<<', i, utl_file.get_line(f);
END LOOP;
RAISE NOTICE '>>%<<', utl_file.get_line(f, 4);
RAISE NOTICE '>>%<<', utl_file.get_line(f, 4);
RAISE NOTICE '>>%<<', utl_file.get_line(f);
RAISE NOTICE '>>%<<', utl_file.get_line(f);
EXCEPTION
-- WHEN no_data_found THEN,  8.1 plpgsql doesn't know no_data_found
WHEN others THEN
  RAISE NOTICE 'finish % ', sqlerrm;
  RAISE NOTICE 'is_open = %', utl_file.is_open(f);
  PERFORM utl_file.fclose_all();
  RAISE NOTICE 'is_open = %', utl_file.is_open(f);
END;
$$ LANGUAGE plpgsql;
/

\set SQLTERM ;
```

3.112.4.6　utl_file.get_line

utl_file.get_line 返回文件的一行。

语法

```
get_line(file integer, OUT buffer text);
get_line(file integer, OUT buffer text, len integer);
```

参数

file：指定 UTL_FILE.FILE_TYPE 类型值。

buffer：指定输出的文件。

len：指定想要输出的特定行。

返回值

无返回值。

注解

utl_file.get_line 为 procedure。

示例

```
\set SQLTERM /

CREATE OR REPLACE PROCEDURE TEST_GET_LINE as
vInHandle UTL_FILE.FILE_TYPE;
vNewLine  VARCHAR2(250);
BEGIN
vInHandle = utl_file.fopen('MY_DIR', 'test.txt','R');
utl_file.get_line(vInHandle, vNewLine);
RAISE NOTICE '>>%<<', vNewLine;
utl_file.fclose(vInHandle);
END TEST_GET_LINE;
/

\set SQLTERM ;
```

3.112.4.7　utl_file.new_line

utl_file.new_line 为文本增加行终止符。

语法

```
new_line(file integer);
new_line(file integer, lines int);
```

参数

file：指定 UTL_FILE.FILE_TYPE 类型值。

lines：指定需要操作的行数。

返回值

无返回值。

注解

utl_file.new_line 为 procedure。

示例

```
\set SQLTERM /

CREATE DIRECTORY MY_DIR AS '/tmp/';
declare
```

```
vInHandle utl_file.file_type;
begin
vInHandle := utl_file.fopen('MY_DIR', 'output.txt', 'w');
utl_file.new_line(vInHandle, 2);
utl_file.fclose(vInHandle);
end;
/

\set SQLTERM ;
```

3.112.4.8　utl_file.fflush

utl_file.fflush 清空目标文件。

语法

```
fflush(file integer);
```

参数

file 指定 UTL_FILE.FILE_TYPE 类型值。

返回值

无返回值。

注解

utl_file.fflush 为 procedure。

示例

```
\set SQLTERM /

CREATE or replace PROCEDURE proc1()
as
DECLARE
dirname          VARCHAR(100) DEFAULT 'my_dir';
v_empfile_src    UTL_FILE.FILE_TYPE;
v_empfile_tgt    UTL_FILE.FILE_TYPE;
v_src_file       VARCHAR(20) DEFAULT 'output.txt';
v_dest_file      VARCHAR(20) DEFAULT 'test.txt';
v_empline        VARCHAR(200);
begin
v_empfile_src = UTL_FILE.FOPEN(dirname,v_src_file,'r');
v_empfile_tgt = UTL_FILE.FOPEN(dirname,v_dest_file,'w');
LOOP
```

```
 BEGIN
 UTL_FILE.GET_LINE(v_empfile_src,v_empline);
 UTL_FILE.PUT(v_empfile_tgt,v_empline);
 UTL_FILE.NEW_LINE(v_empfile_tgt);
 UTL_FILE.FFLUSH(v_empfile_tgt);
 EXCEPTION
  WHEN OTHERS THEN
    EXIT;
 END;
END LOOP;
DBMS_OUTPUT.PUT_LINE('Updated file: ' || v_dest_file);
UTL_FILE.FCLOSE(v_empfile_tgt);
UTL_FILE.FCLOSE(v_empfile_src);
END proc1;
/

\set SQLTERM ;
```

3.112.5　插件 utl_file 卸载方法

卸载时 drop extension utl_file 即可。

示例

```
drop extension utl_file;
```

3.112.6　插件 utl_file 升级方法

utl_file 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.113　uuid-ossp

3.113.1　插件 uuid-ossp 简介

插件 uuid-ossp 是 uuid 生成工具。

- 插件名为 uuid-ossp
- 插件版本 V1.0

3.113.2　插件 uuid-ossp 加载方式

使用时需要在命令行执行 create extension “uuid-ossp”；

示例：

```
create extension "uuid-ossp";
```

3.113.3　插件 uuid-ossp 的参数配置

无需配置任何参数。

3.113.4　插件 uuid-ossp 的使用方法

加载插件 uuid-ossp 后，用 uuid_generate_v1() 或 uuid_generate_v4() 函数生成 UUID 值。uuid_generate_v1() 与计算机的 MAC 地址和时间戳有关。uuid_generate_v4() 生成随机的 uuid。

示例：

```
select uuid_generate_v1();
            uuid_generate_v1
--------------------------------------
c12bedf0-d7b4-11ec-8340-20040feac55c
(1 行记录)

select uuid_generate_v4();
            uuid_generate_v4
--------------------------------------
83e19b37-8195-4823-a6b7-82a1ed5cc668
(1 行记录)
```

3.113.5　插件 uuid-ossp 卸载方法

卸载时 drop extension uuid-ossp 即可。

示例：

```
drop extension "uuid-ossp";
```

3.113.6　插件 uuid-ossp 升级方法

uuid-ossp 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

3.114 walminer

3.114.1 插件 walminer 简介

3.114.1.1 什么是 WalMiner

WalMiner 是 KingbaseES 的 WAL(write ahead logs) 日志解析工具，旨在挖掘 WAL 日志所有的有用信息，从而为 KingbaseES 的数据恢复提供支持。

3.114.1.2 WalMiner 的适用场景

- 用户误删除或更新数据。
- 物理备份损坏的情况下可以尝试使用 WalMiner。

3.114.1.3 WalMiner 的限制与约束

- WalMiner 工具的使用必须以数据库超级用户执行。
- 数据库能正常运行。
- full_page_writes 必须为开启状态。
- 拥有完整连续的 WAL 日志。
- 能导出正常使用的数据字典。
- 解析过程中，日志所需数据字典没有发生改变。
- 不支持 DDL 的解析。
- 日志文件或 history 文件损坏的情况下，有可能解析不出数据。
- 不能将数据库升级前后产生的 wal 日志混在一起解析。例如：升级前后的归档目录是同一个，并对归档目录下的日志进行解析。

3.114.2 WalMiner 的运行环境准备

主要针对以上限制与约束，提前做的一些配置和检查工作

3.114.2.1 开启 full_page_writes

KingbaseES 默认开启 `full_page_writes` 。如果未开启，则不能使用 WalMiner 进行日志解析。WalMiner 只解析 `full_page_writes` 开启后生成的日志。

- 开启方式：

  修改 kingbase.conf 文件中 full_page_writes = on ，并重启数据库。

3.114.2.2　WAL 日志连续性检查

查看 sys_wal 目录或指定目录 WAL 日志文件的编号。

3.114.2.3　安装

```
-- 创建 WalMiner 扩展
create extension walminer;
```

3.114.2.4　数据字典检查

数据字典是保存 KingbaseES 元数据的文件，解析 WAL 日志时，可以使用已存在的数据字典文件，如果不使用数据字典文件，则默认使用本库的元数据信息。

- 导出数据字典方式

```
-- 可传入目录或文件全路径
-- 如果传入目录，会在目录下找 dictionary.d 文件，如果文件不存在，则报错
select walminer_load_dictionary('$KBDATA/sys_walminer/wm_datadict');
select walminer_load_dictionary('$KBDATA/sys_walminer/wm_datadict/dictionary.d');
```

3.114.3　WalMiner 接口使用介绍

3.114.3.1　添加日志

```
-- 可添加文件夹或文件
-- 可多次执行，会自动去重
-- 添加文件时，会对文件进行判断，如果不是 WAL 日志，则会报错
select walminer_wal_add('$KBDATA/sys_wal/');
select walminer_wal_add('$KBDATA/sys_wal/000000010000000000000001');
```

可以多次添加多个不同的目录，WalMiner 会自动根据日志文件名称进行排序和去重。比如可以同时添加在线日志目录和归档日志目录。

如果不执行日志导入，直接执行日志分析，则默认会添加 $KBDATA 目录下的 sys_wal 目录，使用在线日志进行解析时会提示警告信息：

```
NOTICE:  Add wal from current sys_wal directory, do not suggest use this way in produce
```

3.114.3.2　展示日志列表

```
-- 展示已添加的 WAL 日志列表
select walminer_wal_list();
```

3.114.3.3　删除日志

```
-- 可删除文件和文件夹
-- 可多次执行
select walminer_wal_remove('$KBDATA/sys_wal/');
select walminer_wal_remove('$KBDATA/sys_wal/000000010000000000000001');
```

3.114.3.4　导入 history 文件

```
-- path:  history 文件绝对路径
-- cover: 是否覆盖已存在的 history 信息
select walminer_load_history(in path CSTRING, in cover bool DEFAULT true);
```

通过 walminer_load_history 函数可以导入 history 文件，指定时间线的信息，协助 WalMiner 解析数据。

会加载 history 文件里所有时间线信息，并且在最后添加 history 文件名中的时间线息。

```
[kingbase@kingbase] cat '$KBDATA/sys_wal/00000002.history'
1       0/50000A0       no recovery target specified
```

```
test=# select walminer_load_history('/home/kingbase/sys_wal/00000002.history', true);
NOTICE:  load history file "/home/kingbase/sys_wal/00000002.history"
 walminer_load_history
-----------------------
 t
(1 行记录)
```

上面的示例会加载时间线 1 和时间线 2 的信息。

如果已加载了 00000003.history，然后再去加载 00000002.history，默认会覆盖已加载的时间线。

如果不想被覆盖，可以将 cover 参数设置为 false。

```
test=# select walminer_load_history('/home/kingbase/sys_wal/00000002.history', false);
NOTICE:  load history file "/home/kingbase/sys_wal/00000002.history"
 walminer_load_history
-----------------------
 t
(1 行记录)
```

执行 walminer_wal_add 函数导入 WAL 日志，如果指定的参数是目录，并且目录下存在 history 文件，则会自动

加载所有的 history 文件。会以最大时间线的 history 文件为准。

```
test=# select walminer_all();
NOTICE:  Add wal from current sys_wal directory, do not suggest use this way in produce
NOTICE:  load history file "/home/kingbase/sys_wal/00000003.history"
NOTICE:  history file "/home/kingbase/sys_wal/00000002.history" skipped.
   walminer_all
-------------------
 sys_miner success
(1 行记录)
```

如果执行 walminer_wal_add 函数导入了时间线 1、2、3 的信息，但是只想解析时间线 1、2 的数据，则可以

手动调用 walminer_load_history 强制覆盖时间线信息。

```
test=# select walminer_all();
NOTICE:  Add wal from current sys_wal directory, do not suggest use this way in produce
NOTICE:  load history file "/home/kingbase/sys_wal/00000003.history"
NOTICE:  history file "/home/kingbase/sys_wal/00000002.history" skipped.
   walminer_all
-------------------
 sys_miner success
(1 行记录)

test=# select walminer_load_history('/home/kingbase/sys_wal/00000002.history', true);
NOTICE:  load history file "/home/kingbase/sys_wal/00000002.history"
 walminer_load_history
-----------------------
 t
(1 行记录)
```

上面的示例只会解析时间线 1、2 的数据。

反之，如果先手动添加了 00000002.history 的时间线信息，如果 sys_wal 目录下存在时间线大于 2 的 history 文件，

则调用 walminer_wal_add(‘directory’) 函数时会覆盖掉 00000002.history 的信息。

3.114.3.5　导出数据字典

```
-- 可传入目录或文件全路径
-- 如果传入目录，数据字典默认名称为 dictionary.d
select walminer_build_dictionary('$KBDATA/sys_walminer/wm_datadict');
select walminer_build_dictionary('$KBDATA/sys_walminer/wm_datadict/dictionary.d');
```

3.114.3.6　导入数据字典

如果不导入数据字典就执行解析的话，会默认使用在线数据字典。

导入数据字典函数可以多次执行，会自动覆盖前一次加载的数据字典。

```
-- 可传入目录或文件全路径
-- 如果传入目录，会在目录下找 dictionary.d 文件，如果文件不存在，则报错
select walminer_load_dictionary('$KBDATA/sys_walminer/wm_datadict');
select walminer_load_dictionary('$KBDATA/sys_walminer/wm_datadict/dictionary.d');
```

3.114.3.7　数据解析

3.114.3.7.1　普通解析

```
-- 解析已加入的所有 WAL 日志
-- 如果没有添加 WAL 日志，则解析 data/sys_wal 目录下所有 WAL 日志
-- 解析过程中遇到 WAL 日志错误、或 WAL 日志不连续，会在错误或不连续的地方停止解析
select walminer_all();

-- 根据时间范围解析
-- 在 add 的 WAL 日志中查找对应时间范围的 WAL 记录
select walminer_by_time(startTime text, endTime text);

-- 根据 LSN 范围解析
-- 在 add 的 WAL 日志中查找对应 LSN 范围的 WAL 记录
select walminer_by_lsn(startLSN sys_lsn, endLSN sys_lsn);
```

解析完成后会自动删除生成的中间文件、已添加的 WAL 日志、已导入的数据字典等。

在时间范围解析中，如果开始时间设置为空，则从已加入的 WAL 日志列表中第一个日志的第一个 LSN 开始解析。

在时间范围解析中，如果结束时间设置为空，则在已加入的 WAL 日志列表中最后一个日志的最后一个 LSN 结束解析。

解析时，会从给定的范围开始解析，会从找到的第一个 checkpoint 点开始缓存 FPW，所以如果在日志的开始部分没有找到 checkpoint，在解析这部分日志时可能会提示警告信息：

```
Can not find for FPW for relnode 16789, blkno 3, ignore redo.
```

本示例中的 16789 表示文件名，3 表示数据块号，具体数值视实际应用场景而变化。

3.114.3.7.2　精准解析

精确解析是指 WalMiner 程序会自动界定需要解析的 WAL 日志列表，并在给定的解析开始点（开始 LSN 或开始时间）往前探索一个 checkpoint 开始点 c1，从 c1 点开始记录 FPW，然后就可以完美解析指定的范围（LSN 范围或时间范围）。如果在给定的 WAL 日志列表内没有找到 c1 点，那么此次解析会报错停止。

```
-- 根据时间范围解析
-- 在 add 的 WAL 日志中查找对应时间范围的 WAL 记录
-- true/false 代表是否精准解析
select walminer_by_time(startTime text, endTime text, true);

-- 根据 LSN 范围解析
-- 在 add 的 WAL 日志中查找对应 LSN 范围的 WAL 记录
-- true/false 代表是否精准解析
select walminer_by_lsn(startLSN sys_lsn, endLSN sys_lsn, true);
```

解析完成后会自动删除生成的中间文件、已添加的 WAL 日志、已导入的数据字典等。

3.114.3.7.3　单表解析

```
-- 根据时间范围解析
-- 在 add 的 WAL 日志中查找对应时间范围的 WAL 记录
-- true/false 代表是否精准解析
-- reloid 表示需要解析的表 OID
select walminer_by_time(startTime text, endTime text, true, reloid oid);
select walminer_by_time(startTime text, endTime text, false, reloid oid);

-- 根据 LSN 范围解析
-- 在 add 的 WAL 日志中查找对应 LSN 范围的 WAL 记录
-- true/false 代表是否精准解析
-- reloid 表示需要解析的表 OID
select walminer_by_lsn(startLSN sys_lsn, endLSN sys_lsn, true, reloid oid);
select walminer_by_lsn(startLSN sys_lsn, endLSN sys_lsn, false, reloid oid);
```

解析完成后会自动删除生成的中间文件、已添加的 WAL 日志、已导入的数据字典等。

3.114.3.8　查看解析结果

```
select * from walminer_contents;
```

walminer_contents 表字段描述信息：

字段名称	字段类型	字段描述
sqlno	integer	SQL 语句在事务里的编号
xid	bigint	当前事务的 xid
topxid	bigint	父事务的 xid
sqlkind	integer	SQL 语句类型（INSERT/DELETE/UPDATE）
minerd	boolean	是否能解析
timestamp	timestamp with time zone	事务提交时间
op_text	text	redo 语句
undo_text	text	undo 语句
complete	boolean	事务是否已完成
schema	text	模式名
relation	text	表名
start_lsn	sys_lsn	事务开始的 LSN 号
commit_lsn	sys_lsn	事务提交时的 LSN 号

3.114.3.9　结束解析

```
-- 会清空解析过程中生成的中间文件
select walminer_stop();
```

3.114.4　WalMiner 使用示例

3.114.4.1　解析本库 WAL 日志

解析本库 WAL 日志无需添加日志和数据字典

3.114.4.1.1　创建扩展

```
-- 创建扩展
kingbase=# create extension walminer ;
CREATE EXTENSION
```

3.114.4.1.2　执行普通解析

```
-- 执行解析
kingbase=# select walminer_all();
NOTICE:  Add wal from current sys_wal directory, do not suggest use this way in produce
NOTICE:  Switch wal to 000000010000000000000001 on time 2023-07-03 17:23:32.474565+08
   walminer_all
-------------------
 sys_miner success
(1 row)
```

3.114.4.1.3　时间范围解析

```
-- 时间范围解析
kingbase=# select walminer_by_time('2023-07-02 11:27:00', '2023-09-08 11:37:00');
NOTICE:  Switch wal to 000000010000000000000001 on time 2023-07-03 17:24:58.686513+08
 walminer_by_time
-------------------
 sys_miner success
(1 row)
```

3.114.4.1.4　LSN 范围解析

```
-- LSN 范围解析
kingbase=# select walminer_by_lsn('0/010000A0', '0/016E6578');
NOTICE:  Add wal from current sys_wal directory, do not suggest use this way in produce
NOTICE:  Switch wal to 000000010000000000000001 on time 2023-07-03 17:25:38.038238+08
  walminer_by_lsn
-------------------
```

```
 sys_miner success
(1 row)
```

3.114.4.1.5　单表解析

```
-- 单表解析
-- 开始时间/LSN  结束时间/LSN 是否精准解析 表 OID
select walminer_by_time('2021-09-08 11:27:00', '2021-09-08 11:37:00', false, 16452);
select walminer_by_lsn('0/010000A0', '0/016E6578', 'false', 16452);
```

单表解析时，必须指定是否为精准解析。

3.114.4.1.6　查看分析结果

```
-- 查看分析结果
kingbase=# select * from walminer_contents ;
-[ RECORD 1 ]-------------------------------------------------
sqlno      | 1
xid        | 1042
topxid     | 0
sqlkind    | 1
minerd     | t
timestamp  | 2023-07-03 17:18:37.360008+08
op_text    | INSERT INTO public.test(id ,val) VALUES(1 ,'111')
undo_text  | DELETE FROM public.test WHERE id=1 AND val='111'
complete   | t
schema     | public
relation   | test
start_lsn  | 0/1DB66B8
commit_lsn | 0/1DB6720
-[ RECORD 2 ]-------------------------------------------------
sqlno      | 1
xid        | 1043
topxid     | 0
sqlkind    | 1
minerd     | t
timestamp  | 2023-07-03 17:18:41.040578+08
op_text    | INSERT INTO public.test(id ,val) VALUES(2 ,'222')
undo_text  | DELETE FROM public.test WHERE id=2 AND val='222'
complete   | t
schema     | public
relation   | test
start_lsn  | 0/1DB6720
```

```
commit_lsn | 0/1DB6788
```

3.114.4.1.7　结束分析

```
-- 结束分析
kingbase=# select walminer_stop();
  walminer_stop
------------------
 walminer stoped!
(1 row)
```

3.114.4.2　解析其他数据库的 WAL 日志

解析其他库的日志暂不支持解析自定义数据类型，如果其他库的 WAL 日志中存在自定义数据类型，而本库没有对应的数据类型则会解析失败。

3.114.4.2.1　从其他数据库导出数据字典

```
-- 导出数据字典
kingbase=# select walminer_build_dictionary('/home/kingbase/dict');
 walminer_build_dictionary
---------------------------
Dictionary build success!
(1 row)
```

导出的数据字典名称为：dict

```
[kingbase@kingbase ~]$ ll /home/kingbase/dict
-rw------- 1 kingbase kingbase 1252568 Jul  3 17:19 /home/kingbase/dict
```

3.114.4.2.2　导入其他数据库的数据字典

```
-- 导入数据字典
kingbase=# select walminer_load_dictionary('/home/kingbase/dict');
 walminer_load_dictionary
--------------------------
 Dictionary load success!
(1 row)
```

3.114.4.2.3　添加日志文件

```
-- 添加日志文件
kingbase=# select walminer_wal_add('/home/kingbase/sys_wal/000000010000000000000001');
  walminer_wal_add
--------------------
 1 file add success
(1 row)
```

3.114.4.2.4　日志解析

```
-- 普通解析
select walminer_all();

-- 时间范围解析
select walminer_by_time('2021-09-08 11:27:00', '2021-09-08 11:37:00');

-- LSN 范围解析
select walminer_by_lsn('0/010000A0', '0/016E6578');

-- 单表解析
select walminer_by_time('2021-09-08 11:27:00', '2021-09-08 11:37:00', false, 16452);
select walminer_by_lsn('0/010000A0', '0/016E6578', 'false', 16452);
```

```
kingbase=# select walminer_by_lsn('0/010000A0', '0/016E6578', false, 16384);
NOTICE:  Switch wal to 000000010000000000000001 on time 2023-07-03 17:32:50.646287+08
  walminer_by_lsn
-------------------
 sys_miner success
(1 row)
```

3.114.4.2.5　查看分析结果

```
-- 查看分析结果
kingbase=# select * from walminer_contents ;
-[ RECORD 1 ]-------------------------------------------------
sqlno      | 1
xid        | 1042
topxid     | 0
sqlkind    | 1
minerd     | t
timestamp  | 2023-07-03 17:18:37.360008+08
op_text    | INSERT INTO public.test(id ,val) VALUES(1 ,'111')
```

```
undo_text  | DELETE FROM public.test WHERE id=1 AND val='111'
complete   | t
schema     | public
relation   | test
start_lsn  | 0/1DB66B8
commit_lsn | 0/1DB6720
-[ RECORD 2 ]-----------------------------------------------
sqlno      | 1
xid        | 1043
topxid     | 0
sqlkind    | 1
minerd     | t
timestamp  | 2023-07-03 17:18:41.040578+08
op_text    | INSERT INTO public.test(id ,val) VALUES(2 ,'222')
undo_text  | DELETE FROM public.test WHERE id=2 AND val='222'
complete   | t
schema     | public
relation   | test
start_lsn  | 0/1DB6720
commit_lsn | 0/1DB6788
```

3.114.4.2.6　结束分析

结束分析会删除分析过程中生成的中间文件、导入的数据字典、WAL 日志文件以及分析结果

```
-- 结束分析
kingbase=# select walminer_stop();
  walminer_stop
------------------
 walminer stoped!
(1 row)
```

3.114.5　WalMiner 常见问题处理

3.114.5.1　执行 walminer 解析报错 oid 不存在

```
-- 执行 walminer_by_lsn 报错
kingbase=# select walminer_by_lsn('0/430008E8', '0/4393D0F0', true, 42762) ;
ERROR:  Can not find reloid 42762 in datadicationary
背景:   SQL function "walminer_by_lsn" statement 2
```

- 原因分析：

walminer 的解析动作依赖于数据字典的完整性。在此之前某次解析操作后残留了数据字典，使得此后解析使用的都是残留的数据字典。此数据字典较老，没有包含 oid 为 42762 的表。

- 场景：
  解析中途退出等原因可能造成数据字典残留。
- 解决：
  方法 1：执行 walminer_stop 后，再次解析。walminer_stop 函数会清除解析过程中生成的中间文件、已添加的 wal 日志文件列表、导入的数据字典等资源。

  方法 2：手动导入与日志匹配的数据字典。确保数据字典的完整性。

3.115　xml2

3.115.1　插件 xml2 简介

xml2 是 KingbaseES 的一个扩展插件，主要提供 XPath 查询和 XSLT 功能。

- 插件名为 xml2
- 插件版本 V1.1

在核心服务器中就已经有基于 SQL/XML 标准的 XML 相关功能，这些功能覆盖了 XML 语法检查和 XPath 查询。xml2 同样支持这些功能，但是提供的 API 并不是完全兼容，保留 xml2 插件的目的是为了保持向后兼容。我们已经有计划将 xml2 从 KingbaseES 一个未来版本中移除，因此我们鼓励您尝试转换您的应用。

3.115.2　插件 xml2 加载方式

```
CREATE EXTENSION xml2;
```

3.115.3　插件 xml2 的参数配置

无需配置任何参数。

3.115.4　插件 xml2 的使用方法

下表展示了 xml2 插件提供的函数。这些函数提供了直接的 XML 解析和 XPath 查询。所有参数都是 text 类型，为了简洁省略了参数类型。

表 3.115.1: xml2 函数

函数	返回	描述
xml_is_well_formed(document)	bool	这个函数解析其参数中的文档文本并且在该文档是一个结构良好的 XML 时返回真。
xpath_string(document, query)	text	在提供的文档上计算 XPath 查询，并且将结果造型为指定的类型。
xpath_number(document, query)	float4	在提供的文档上计算 XPath 查询，并且将结果造型为指定的类型。
xpath_bool(document, query)	bool	在提供的文档上计算 XPath 查询，并且将结果造型为指定的类型。
xpath_nodeset(document, query, toptag, itemtag)	text	这个函数在文档上计算查询并且把结果包装在 XML 标签中。如果 toptag 或者 itemtag 是一个空字符串，相关的标签会被忽略。
xpath_nodeset(document, query)	text	与 xpath_nodeset(document, query, toptag,itemtag) 相似，但是结果忽略两种标签。
xpath_nodeset(document, query, itemtag)	text	与 xpath_nodeset(document, query, toptag,itemtag) 相似，但是结果忽略 toptag。
xpath_list(document, query, separator)	text	这个函数返回多个值，并且用指定的分隔符分隔，例如分隔符是”,”，结果就是 Value1,Value 2,Value 3。
xpath_list(document, query)	text	这是上面函数的一个包装器，它用”,” 作为分隔符。

3.115.4.1　xpath_table

```
xpath_table(text key, text document, text relation, text xpaths, text criteria) returns setof record
```

xpath_table 是一个表函数，它在一组文档中的每一个上计算一组 XPath 查询，并且将结果作为一个表返回。来自原始文档表的主键域被返回为结果的第一列，这样结果集可以被用于连接。

表 3.115.2: xpath_table 参数

参数	描述
key	“key”字段的名称—这只是被用作输出表中第一列的字段，即它标识每个输出行是来自于哪个记录
document	包含 XML 文档的域的名称

-续表-

参数	描述
relation	包含文档的表或视图的名称
xpaths	一个或多个 XPath 表达式，用英文竖线分隔
criteria	WHERE 子句的内容，这不能被忽略，因此如果你想要处理关系中的所有行，可以使用 true 或 1=1

这些参数（除了 XPath 字符串）会被替换到一个纯粹的 SQL SELECT 语句中，因此可以是特定位置上合法的任何内容。（** 注意：这种简单方法要求您验证任何用户提供的值，以避免 SQL 注入攻击 **）

```
SELECT <key>, <document> FROM <relation> WHERE <criteria>;
```

该函数必须被使用在一个 FROM 表达式中，并带有一个 AS 子句来指定输出列，例如：

```
SELECT * FROM
  xpath_table('article_id', 'article_xml', 'articles',
              '/article/author|/article/pages|/article/title',
              'date_entered > ''2003-01-01'' ')
  AS t(article_id integer, author text, page_count integer, title text);
```

AS 子句定义了输出表中列的名称和类型。第一个是“key”域并且剩下的对应于 XPath 查询。如果 XPath 查询比结果列多，额外的查询将被忽略。如果结果列比 XPath 查询多，额外的列将是 NULL。

备注:

这个例子将 page_count 结果列定义为一个整数。该函数在内部处理字符串表示，因此当您想要在输出中使用整数时，它将采用 XPath 结果的字符串表示，并使用 KingbaseES 的输入函数来把它转换成整数（或者 AS 子句要求的任何类型）。如果无法执行此操作（例如：结果为空），则会导致错误。因此如果您认为您的数据有任何问题，您可能希望仅将 text 作为列类型。

调用的 SELECT 语句不必只是 `SELECT *`，它可以按名称引用输出列或者将它们连接到其他表。该函数会产生一个虚拟表，您可以在其上执行您想要的任何操作（例如聚集、连接、排序等）。因此我们也可以有下面的更复杂的例子。当然，您也可以把所有这些包装在一个视图中。

```
SELECT t.title, p.fullname, p.email FROM
  xpath_table('article_id', 'article_xml', 'articles',
              '/article/title|/article/author/@id',
              'xpath_string(article_xml,''/article/@date'') > ''2003-03-20'' ')
  AS t(article_id integer, title text, author_id integer), tblPeopleInfo AS p
WHERE t.author_id = p.person_id;
```

3.115.4.1.1　多值结果

xpath_table 函数假定每一个 XPath 查询的结果可能是多值的，因此该函数返回的行数可能与输入文档的数目不同。被返回的第一行包含来自每一个查询的第一个结果，第二行则是来自每一个查询的第二个结果。如果其中一个查询的值比其他查询少，则会为它返回空值。

在某些情况下，用户将知道给定的 XPath 查询将只返回一个结果（可能是唯一文档标识符）。如果和一个返回多值的 XPath 查询一起使用，单值结果将只出现在结果的第一行中。对于这种情况的解决方案是使用键字段作为针对一个更简单 XPath 查询连接的一部分。一个例子：

```
CREATE TABLE test (
    id int PRIMARY KEY,
    xml text
);

INSERT INTO test VALUES (1, '<doc num="C1">
<line num="L1"><a>1</a><b>2</b><c>3</c></line>
<line num="L2"><a>11</a><b>22</b><c>33</c></line>
</doc>');

INSERT INTO test VALUES (2, '<doc num="C2">
<line num="L1"><a>111</a><b>222</b><c>333</c></line>
<line num="L2"><a>111</a><b>222</b><c>333</c></line>
</doc>');

SELECT * FROM
  xpath_table('id','xml','test',
              '/doc/@num|/doc/line/@num|/doc/line/a|/doc/line/b|/doc/line/c',
              'true')
  AS t(id int, doc_num varchar(10), line_num varchar(10), val1 int, val2 int, val3 int)
WHERE id = 1 ORDER BY doc_num, line_num;

 id | doc_num | line_num | val1 | val2 | val3
----+---------+----------+------+------+------
  1 | C1      | L1       |    1 |    2 |    3
  1 |         | L2       |   11 |   22 |   33
```

要在每一行上得到 doc_num，解决方案是使用两次 xpath_table 调用并连接结果：

```
SELECT t.*,i.doc_num FROM
  xpath_table('id', 'xml', 'test',
              '/doc/line/@num|/doc/line/a|/doc/line/b|/doc/line/c',
              'true')
  AS t(id int, line_num varchar(10), val1 int, val2 int, val3 int),
```

```
  xpath_table('id', 'xml', 'test', '/doc/@num', 'true')
  AS i(id int, doc_num varchar(10))
WHERE i.id=t.id AND i.id=1
ORDER BY doc_num, line_num;

 id | line_num | val1 | val2 | val3 | doc_num
----+----------+------+------+------+---------
  1 | L1       |    1 |    2 |    3 | C1
  1 | L2       |   11 |   22 |   33 | C1
(2 rows)
```

3.115.4.2　XSLT 函数

如果安装了 libxslt，那么可以使用下列函数：

```
xslt_process(text document, text stylesheet, text paramlist) returns text
```

这个函数将 XSL 样式表应用于文档，并且返回转换过的结果。paramlist 是一个被用在转换中的参数赋值列表，以 `a=1,b=2` 的形式指定。

备注:

参数解析是非常简单的，参数值不能包含逗号！

还有一个双参数版本的 xslt_process，它不会向转换传递任何参数。

3.115.5　插件 xml2 卸载方法

```
DROP EXTENSION xml2;
```

3.115.6　插件 xml2 升级方法

xml2 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级该插件。

```
ALTER EXTENSION xml2 UPDATE TO new_version;
```

3.116 zhparser

3.116.1 插件 zhparser 简介

zhparser 是一个 KingbaseES 的扩展，主要用于全文搜索的中文分词。zhparser 中文分词插件支持 utf8 和 gbk 字符集。

- 插件名为 zhparser
- 插件版本 V1.0

3.116.2 插件 zhparser 加载方式

```
CREATE EXTENSION zhparser;
```

3.116.3 插件 zhparser 的参数配置

`zhparser.dict_in_memory`
将词典全部加载到内存中，bool 类型，默认值：false

`zhparser.punctuation_ignore`
忽略所有的标点等特殊符号，bool 类型，默认值：false

`zhparser.seg_with_duality`
闲散文字自动以二元分词法聚合，bool 类型，默认值：false

`zhparser.multi_short`
短词复合，bool 类型，默认值：false

`zhparser.multi_duality`
散字二元复合，bool 类型，默认值：false

`zhparser.multi_zmain`
重要单字复合，bool 类型，默认值：false

`zhparser.multi_zall`
全部单字复合，bool 类型，默认值：false

`zhparser.extra_dicts`
用户自定义词典文件列表，自定义词典文件必须放在 share/tsearch_data 目录中，示例：zhparser.extra_dicts = ‘dict.txt,mydict.xdb’，string 类型，默认值：无

3.116.4　插件 zhparser 的使用方法

```
CREATE EXTENSION zhparser;

CREATE TEXT SEARCH CONFIGURATION testzhcfg (PARSER = zhparser);

ALTER TEXT SEARCH CONFIGURATION testzhcfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;

SELECT * FROM ts_parse('zhparser', 'KingbaseES 中文分词插件测试');
tokid |   token
-------+------------
  101 | KingbaseES
  110 | 中文
  118 | 分词
  110 | 插件
  118 | 测试
 (5 rows)

SELECT to_tsvector('testzhcfg','KingbaseES 中文分词插件测试');
                     to_tsvector
-----------------------------------------------------
 'kingbasees':1 '中文':2 '分词':3 '插件':4 '测试':5
 (1 row)

SELECT to_tsquery('testzhcfg', '分词');
   to_tsquery
------------
  '分词'
 (1 row)
```

中国纪年法分词检索步骤说明：

```
1、创建插件 zhparser
CREATE EXTENSION zhparser;

2、创建配置 lunar，并在配置中建立词的映射，将想要的记号类型如 t,n,m,r,x,d,v,k,a，绑定到词典 simple 上。注意：其中 t 为必选项，其他均为任选项。
CREATE TEXT SEARCH CONFIGURATION lunar (PARSER = zhparser);

ALTER TEXT SEARCH CONFIGURATION lunar ADD MAPPING FOR n,m,r,t,x,d,v,k,a WITH simple;

部分记号类型的说明如下：
tokid      记号类型        描述
97            a           adjective
100           d           adverb
```

```
107          k           tail
109          m           numral
110          n           noun
114          r           pronoun
116          t           time
118          v           verb
120          x           unknown

其中记号类型的 tokid 可通过 ts_parser 函数查询，如:
SELECT * FROM ts_parse('zhparser', '甲');
 tokid | token
-------+-------
   110 | 甲
(1 rows)

3、添加用户自定义词典文件列表，自定义词典文件必须放在$SHAREDIR/tsearch_data 目录中（其中$SHAREDIR 表示
KingbaseES 安装的共享数据目录）。
中国纪年法分词检索所需的词典文件已添加到指定路径，可直接使用该文件，选择的文件与数据库初始化时的编码一致。其中
gbk 编码的文件名为 lunar_calendar.txt，utf-8 编码的文件名为 lunar_calendar.utf8.txt。
注意：添加词典后需重启数据库才生效!

ALTER SYSTEM SET zhparser.extra_dicts='lunar_calendar.utf8.txt';
show zhparser.extra_dicts;
 zhparser.extra_dicts
----------------------
 lunar_calendar.utf8.txt
(1 row)

4、测试分词检索
SELECT to_tsvector('lunar','甲子年乙丑年丙寅年丁卯年戊辰年己巳年庚午年辛未年');
                                        to_tsvector
-------------------------------------------------------------------------------------------
 '丁卯年':4 '丙寅年':3 '乙丑年':2 '己巳年':6 '庚午年':7 '戊辰年':5 '甲子年':1 '辛未年':8
(1 row)

SELECT to_tsquery('lunar','今年是壬寅年');
        to_tsquery
---------------------------
 '今年' & '是' & '壬寅年'
(1 row)
```

3.116.5　插件 zhparser 的卸载方法

```
drop extension zhparser;
```

3.116.6　插件 zhparser 升级方法

zhparser 扩展插件通常随着 KingbaseES 安装包一并升级。通常情况下用户无须单独升级些插件。

版权声明

中电科金仓（北京）科技股份有限公司（简称：电科金仓）版权所有，并保留对本手册及本声明的一切权利。

未得到电科金仓的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。电科金仓保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。

本手册仅作为使用指导，电科金仓在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

技术支持

- 电科金仓官方网站：http://www.kingbase.com.cn/
- 电科金仓文档中心：http://help.kingbase.com.cn/
- 全国服务热线：400-601-1188
- 电科金仓技术支持与反馈信箱：support@kingbase.com.cn

服务周期承诺

由于市场需求在不断变化，技术创新和发展的进程不断加剧，产品的版本更迭不可避免。电科金仓对于产品版本生命周期的有效管理，有助于您提前规划项目，更好地从产品服务终止上过渡。

表 1: KingbaseES 产品生命周期里程碑

关键里程碑点	定义
产品发布日期	产品正式发布版本，即 GA（general availability）版本的发布日期。
停止销售日期	正式停止销售的日期，版本停止接受订单日。该日之后，产品将不再销售。
停止功能升级日期	在该日期之后，不再提供新特性和新硬件支持。但依旧提供错误修复、安全修复、功能维护等服务。
停止功能维护日期	在该日期之后，不再维护功能，修复问题。但依旧提供安全修复等服务
停止安全维护日期	在该日期之后，不再发布补丁版本修复中高风险漏洞，仅提供有限的支持。
产品服务终止日期	停止提供产品服务和支持的日期。包括软件维护版本，缺陷修复，以及针对该产品的所有服务支持（包括服务热线和远程/现场支持）。

服务周期策略

金仓数据库管理系统 KingbaseES 产品确保以下的服务周期：

1）产品自发布之日起至产品停止功能升级（包含新特性、新硬件支持）之日止不少于 5 年。

2）产品停止功能升级之日起至产品停止功能维护（主要包括问题修复）之日止不少于 4 年。

3）产品功能维护停止之日起至产品停止安全维护（包括中高风险漏洞修复）之日止不少于 2 年。

针对金仓数据库管理系统 KingbaseES V9 产品（发布日期：2022 年 7 月 15 日），其具体服务周期安排如下：

- 产品销售日期：以产品销售合同所约定日期为准。
- 停止升级日期：自产品销售日期起计算，满 5 年之日止。
- 服务终止日期：自产品销售日期起计算，满 11 年之日止。

服务终止策略

金仓数据库管理系统 KingbaseES 产品确保在销售后，至少提供 6 年的服务支持。

备注:

电科金仓将会综合各方因素来确定产品服务终止日期。并将在实际产品服务终止日期之前至少 90 天，通过公开方式宣布产品服务终止日期。